

ECMA-262, 10th edition, June 2019

ECMAScript® 2019

Language Specification

Contributing to this Specification

This specification is developed on GitHub with the help of the ECMAScript community. There are a number of ways to contribute to the development of this specification:

GitHub Repository: <https://github.com/tc39/ecma262>

Issues: [All Issues](#), [File a New Issue](#)

Pull Requests: [All Pull Requests](#), [Create a New Pull Request](#)

Test Suite: [Test262](#)

Editors:

- [Brian Terlson \(@bterlson\)](#)
- [Bradley Farias \(@bradleymeck\)](#)
- [Jordan Harband \(@ljharb\)](#)

Community:

- Mailing list: [es-discuss](#)
- IRC: [#tc39](#) on [freenode](#)

Refer to the [colophon](#) for more information on how this document is created.

Introduction

This Ecma Standard defines the ECMAScript 2019 Language. It is the tenth edition of the ECMAScript Language Specification. Since publication of the first edition in 1997, ECMAScript has grown to be one of the world's most widely used general-purpose programming languages. It is best known as the language embedded in web browsers but has also been widely adopted for server and embedded applications.

ECMAScript is based on several originating technologies, the most well-known being JavaScript (Netscape) and JScript (Microsoft). The language was invented by Brendan Eich at Netscape and first appeared in that company's Navigator 2.0 browser. It has appeared in all subsequent browsers from Netscape and in all browsers from Microsoft starting with Internet Explorer 3.0.

The development of the ECMAScript Language Specification started in November 1996. The first edition of this Ecma Standard was adopted by the Ecma General Assembly of June 1997.

That Ecma Standard was submitted to ISO/IEC JTC 1 for adoption under the fast-track procedure, and approved as international standard ISO/IEC 16262, in April 1998. The Ecma General Assembly of June 1998 approved the second edition of ECMA-262 to keep it fully aligned with ISO/IEC 16262. Changes between the first and the second edition are editorial in nature.

The third edition of the Standard introduced powerful regular expressions, better string handling, new control statements, try/catch exception handling, tighter definition of errors, formatting for numeric output and minor changes in anticipation of future language growth. The third edition of the ECMAScript standard was adopted by the Ecma General Assembly of December 1999 and published as ISO/IEC 16262:2002 in June 2002.

After publication of the third edition, ECMAScript achieved massive adoption in conjunction with the World Wide Web where it has become the programming language that is supported by essentially all web browsers. Significant work was done to develop a fourth edition of ECMAScript. However, that work was not completed and not published as the fourth edition of ECMAScript but some of it was incorporated into the development of the sixth edition.

The fifth edition of ECMAScript (published as ECMA-262 5th edition) codified de facto interpretations of the language specification that have become common among browser implementations and added support for new features that had emerged since the publication of the third edition. Such features include accessor properties, reflective creation and inspection of objects, program control of property attributes, additional array manipulation functions, support for the JSON object encoding format, and a strict mode that provides enhanced error checking and program security. The fifth edition was adopted by the Ecma General Assembly of December 2009.

The fifth edition was submitted to ISO/IEC JTC 1 for adoption under the fast-track procedure, and approved as international standard ISO/IEC 16262:2011. Edition 5.1 of the ECMAScript Standard incorporated minor corrections and is the same text as ISO/IEC 16262:2011. The 5.1 Edition was adopted by the Ecma General Assembly of June 2011.

Focused development of the sixth edition started in 2009, as the fifth edition was being prepared for publication. However, this was preceded by significant experimentation and language enhancement design efforts dating to the publication of the third edition in 1999. In a very real sense, the completion of the sixth edition is the culmination of a fifteen year effort. The goals for this addition included providing better support for large applications, library creation, and for use of ECMAScript as a compilation target for other languages. Some of its major enhancements included modules, class declarations, lexical block scoping, iterators and generators, promises for asynchronous programming, destructuring patterns, and proper tail calls. The ECMAScript library of built-ins was expanded to support additional data abstractions including maps, sets, and arrays of binary numeric values as well as additional support for Unicode supplemental characters in strings and regular expressions. The built-ins were also made extensible via subclassing. The sixth edition provides the foundation for regular, incremental language and library enhancements. The sixth edition was adopted by the General Assembly of June 2015.

ECMAScript 2016 was the first ECMAScript edition released under Ecma TC39's new yearly release cadence and open development process. A plain-text source document was built from the ECMAScript 2015 source document to serve as the base for further development entirely on GitHub. Over the year of this standard's development, hundreds of pull requests and issues were filed representing thousands of bug fixes, editorial fixes and other improvements. Additionally, numerous software tools were developed to aid in this effort including Ecmascript, Ecmascriptdown, and Grammarkdown. ES2016 also included support for a new exponentiation operator and adds a new method to Array.prototype called **includes**.

ECMAScript 2017 introduced Async Functions, Shared Memory, and Atomics along with smaller language and library enhancements, bug fixes, and editorial updates. Async functions improve the asynchronous programming experience by

providing syntax for promise-returning functions. Shared Memory and Atomics introduce a new [memory model](#) that allows multi-[agent](#) programs to communicate using atomic operations that ensure a well-defined execution order even on parallel CPUs. This specification also includes new static methods on Object: **Object.values**, **Object.entries**, and **Object.getOwnPropertyDescriptors**.

ECMAScript 2018 introduced support for asynchronous iteration via the AsyncIterator protocol and async generators. It also included four new regular expression features: the dotAll flag, named capture groups, Unicode property escapes, and look-behind assertions. Lastly it included rest parameter and spread operator support for object properties.

This specification, the 10th edition, introduces a few new built-in functions: **flat** and **flatMap** on **Array.prototype** for flattening arrays, **Object.fromEntries** for directly turning the return value of **Object.entries** into a new Object, and **trimStart** and **trimEnd** on **String.prototype** as better-named alternatives to the widely implemented but non-standard **String.prototype.trimLeft** and **trimRight** built-ins. In addition, this specification includes a few minor updates to syntax and semantics. Updated syntax includes optional catch binding parameters and allowing U+2028 (LINE SEPARATOR) and U+2029 (PARAGRAPH SEPARATOR) in string literals to align with JSON. Other updates include requiring that **Array.prototype.sort** be a stable sort, requiring that **JSON.stringify** return well-formed UTF-8 regardless of input, and clarifying **Function.prototype.toString** by requiring that it either return the corresponding original source text or a standard placeholder.

Dozens of individuals representing many organizations have made very significant contributions within Ecma TC39 to the development of this edition and to the prior editions. In addition, a vibrant community has emerged supporting TC39's ECMAScript efforts. This community has reviewed numerous drafts, filed thousands of bug reports, performed implementation experiments, contributed test suites, and educated the world-wide developer community about ECMAScript. Unfortunately, it is impossible to identify and acknowledge every person and organization who has contributed to this effort.

Allen Wirfs-Brock

ECMA-262, Project Editor, 6th Edition

Brian Terlson

ECMA-262, Project Editor, 7th through 10th Editions

1 Scope

This Standard defines the ECMAScript 2019 general-purpose programming language.

2 Conformance

A conforming implementation of ECMAScript must provide and support all the types, values, objects, properties, functions, and program syntax and semantics described in this specification.

A conforming implementation of ECMAScript must interpret source text input in conformance with the latest version of the Unicode Standard and ISO/IEC 10646.

A conforming implementation of ECMAScript that provides an application programming interface (API) that supports

programs that need to adapt to the linguistic and cultural conventions used by different human languages and countries must implement the interface defined by the most recent edition of ECMA-402 that is compatible with this specification.

A conforming implementation of ECMAScript may provide additional types, values, objects, properties, and functions beyond those described in this specification. In particular, a conforming implementation of ECMAScript may provide properties not described in this specification, and values for those properties, for objects that are described in this specification.

A conforming implementation of ECMAScript may support program and regular expression syntax not described in this specification. In particular, a conforming implementation of ECMAScript may support program syntax that makes use of the “future reserved words” listed in subclause 11.6.2.2 of this specification.

A conforming implementation of ECMAScript must not implement any extension that is listed as a Forbidden Extension in subclause 16.2.

3 Normative References

The following referenced documents are indispensable for the application of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

ISO/IEC 10646 *Information Technology – Universal Multiple-Octet Coded Character Set (UCS) plus Amendment 1:2005, Amendment 2:2006, Amendment 3:2008, and Amendment 4:2008*, plus additional amendments and corrigenda, or successor

ECMA-402, *ECMAScript 2015 Internationalization API Specification*.

<https://ecma-international.org/publications/standards/Ecma-402.htm>

ECMA-404, *The JSON Data Interchange Format*.

<https://ecma-international.org/publications/standards/Ecma-404.htm>

4 Overview

This section contains a non-normative overview of the ECMAScript language.

ECMAScript is an object-oriented programming language for performing computations and manipulating computational objects within a host environment. ECMAScript as defined here is not intended to be computationally self-sufficient; indeed, there are no provisions in this specification for input of external data or output of computed results. Instead, it is expected that the computational environment of an ECMAScript program will provide not only the objects and other facilities described in this specification but also certain environment-specific objects, whose description and behaviour are beyond the scope of this specification except to indicate that they may provide certain properties that can be accessed and certain functions that can be called from an ECMAScript program.

ECMAScript was originally designed to be used as a scripting language, but has become widely used as a general-purpose programming language. A *scripting language* is a programming language that is used to manipulate, customize, and automate the facilities of an existing system. In such systems, useful functionality is already available through a user interface, and the scripting language is a mechanism for exposing that functionality to program control. In this way, the existing system is said to provide a host environment of objects and facilities, which completes the capabilities of the

scripting language. A scripting language is intended for use by both professional and non-professional programmers.

ECMAScript was originally designed to be a *Web scripting language*, providing a mechanism to enliven Web pages in browsers and to perform server computation as part of a Web-based client-server architecture. ECMAScript is now used to provide core scripting capabilities for a variety of host environments. Therefore the core language is specified in this document apart from any particular host environment.

ECMAScript usage has moved beyond simple scripting and it is now used for the full spectrum of programming tasks in many different environments and scales. As the usage of ECMAScript has expanded, so has the features and facilities it provides. ECMAScript is now a fully featured general-purpose programming language.

Some of the facilities of ECMAScript are similar to those used in other programming languages; in particular C, Java™, Self, and Scheme as described in:

ISO/IEC 9899:1996, *Programming Languages – C*.

Gosling, James, Bill Joy and Guy Steele. *The Java™ Language Specification*. Addison Wesley Publishing Co., 1996.

Ungar, David, and Smith, Randall B. Self: The Power of Simplicity. *OOPSLA '87 Conference Proceedings*, pp. 227-241, Orlando, FL, October 1987.

IEEE Standard for the Scheme Programming Language. IEEE Std 1178-1990.

4.1 Web Scripting

A web browser provides an ECMAScript host environment for client-side computation including, for instance, objects that represent windows, menus, pop-ups, dialog boxes, text areas, anchors, frames, history, cookies, and input/output. Further, the host environment provides a means to attach scripting code to events such as change of focus, page and image loading, unloading, error and abort, selection, form submission, and mouse actions. Scripting code appears within the HTML and the displayed page is a combination of user interface elements and fixed and computed text and images. The scripting code is reactive to user interaction, and there is no need for a main program.

A web server provides a different host environment for server-side computation including objects representing requests, clients, and files; and mechanisms to lock and share data. By using browser-side and server-side scripting together, it is possible to distribute computation between the client and server while providing a customized user interface for a Web-based application.

Each Web browser and server that supports ECMAScript supplies its own host environment, completing the ECMAScript execution environment.

4.2 ECMAScript Overview

The following is an informal overview of ECMAScript—not all parts of the language are described. This overview is not part of the standard proper.

ECMAScript is object-based: basic language and host facilities are provided by objects, and an ECMAScript program is a cluster of communicating objects. In ECMAScript, an *object* is a collection of zero or more *properties* each with *attributes* that determine how each property can be used—for example, when the Writable attribute for a property is set to **false**, any attempt by executed ECMAScript code to assign a different value to the property fails. Properties are containers that hold other objects, *primitive values*, or *functions*. A primitive value is a member of one of the following

built-in types: **Undefined**, **Null**, **Boolean**, **Number**, **String**, and **Symbol**; an object is a member of the built-in type **Object**; and a function is a callable object. A function that is associated with an object via a property is called a *method*.

ECMAScript defines a collection of *built-in objects* that round out the definition of ECMAScript entities. These built-in objects include the [global object](#); objects that are fundamental to the [runtime semantics](#) of the language including **Object**, **Function**, **Boolean**, **Symbol**, and various **Error** objects; objects that represent and manipulate numeric values including **Math**, **Number**, and **Date**; the text processing objects **String** and **RegExp**; objects that are indexed collections of values including **Array** and nine different kinds of Typed Arrays whose elements all have a specific numeric data representation; keyed collections including **Map** and **Set** objects; objects supporting structured data including the **JSON** object, **ArrayBuffer**, **SharedArrayBuffer**, and **DataView**; objects supporting control abstractions including generator functions and **Promise** objects; and reflection objects including **Proxy** and **Reflect**.

ECMAScript also defines a set of built-in *operators*. ECMAScript operators include various unary operations, multiplicative operators, additive operators, bitwise shift operators, relational operators, equality operators, binary bitwise operators, binary logical operators, assignment operators, and the comma operator.

Large ECMAScript programs are supported by *modules* which allow a program to be divided into multiple sequences of statements and declarations. Each module explicitly identifies declarations it uses that need to be provided by other modules and which of its declarations are available for use by other modules.

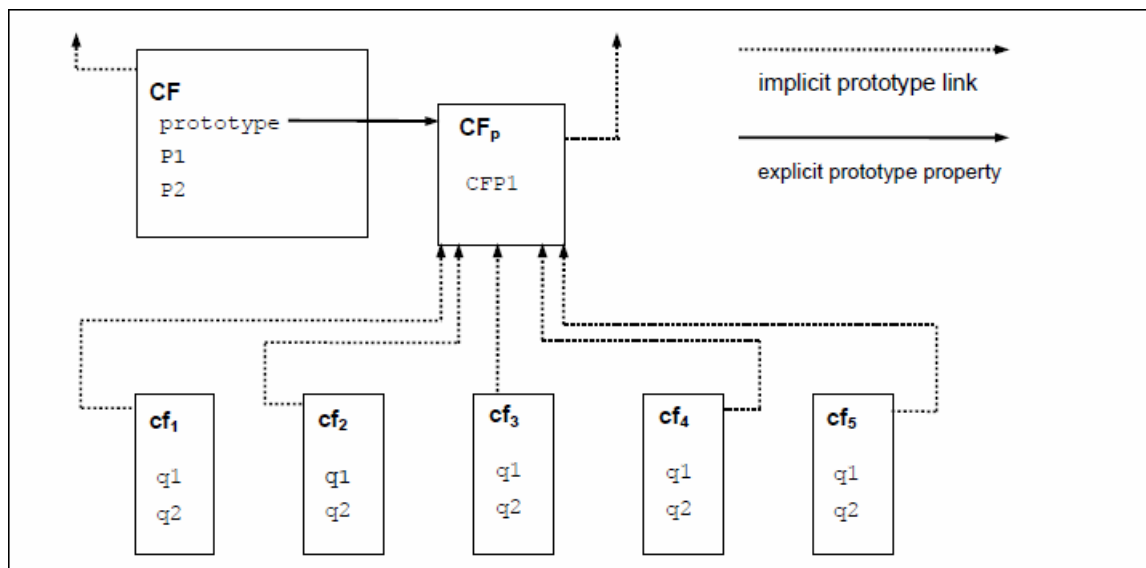
ECMAScript syntax intentionally resembles Java syntax. ECMAScript syntax is relaxed to enable it to serve as an easy-to-use scripting language. For example, a variable is not required to have its type declared nor are types associated with properties, and defined functions are not required to have their declarations appear textually before calls to them.

4.2.1 Objects

Even though ECMAScript includes syntax for class definitions, ECMAScript objects are not fundamentally class-based such as those in C++, Smalltalk, or Java. Instead objects may be created in various ways including via a literal notation or via *constructors* which create objects and then execute code that initializes all or part of them by assigning initial values to their properties. Each [constructor](#) is a function that has a property named "**prototype**" that is used to implement *prototype-based inheritance* and *shared properties*. Objects are created by using constructors in **new** expressions; for example, **new Date(2009, 11)** creates a new Date object. Invoking a [constructor](#) without using **new** has consequences that depend on the [constructor](#). For example, **Date()** produces a string representation of the current date and time rather than an object.

Every object created by a [constructor](#) has an implicit reference (called the object's *prototype*) to the value of its [constructor](#)'s "**prototype**" property. Furthermore, a prototype may have a non-null implicit reference to its prototype, and so on; this is called the *prototype chain*. When a reference is made to a property in an object, that reference is to the property of that name in the first object in the prototype chain that contains a property of that name. In other words, first the object mentioned directly is examined for such a property; if that object contains the named property, that is the property to which the reference refers; if that object does not contain the named property, the prototype for that object is examined next; and so on.

Figure 1: Object/Prototype Relationships



In a class-based object-oriented language, in general, state is carried by instances, methods are carried by classes, and inheritance is only of structure and behaviour. In ECMAScript, the state and methods are carried by objects, while structure, behaviour, and state are all inherited.

All objects that do not directly contain a particular property that their prototype contains share that property and its value. Figure 1 illustrates this:

CF is a [constructor](#) (and also an object). Five objects have been created by using **new** expressions: **cf₁**, **cf₂**, **cf₃**, **cf₄**, and **cf₅**. Each of these objects contains properties named **q1** and **q2**. The dashed lines represent the implicit prototype relationship; so, for example, **cf₃**'s prototype is **CF_p**. The [constructor](#), **CF**, has two properties itself, named **P1** and **P2**, which are not visible to **CF_p**, **cf₁**, **cf₂**, **cf₃**, **cf₄**, or **cf₅**. The property named **CFP1** in **CF_p** is shared by **cf₁**, **cf₂**, **cf₃**, **cf₄**, and **cf₅** (but not by **CF**), as are any properties found in **CF_p**'s implicit prototype chain that are not named **q1**, **q2**, or **CFP1**. Notice that there is no implicit prototype link between **CF** and **CF_p**.

Unlike most class-based object languages, properties can be added to objects dynamically by assigning values to them. That is, constructors are not required to name or assign values to all or any of the constructed object's properties. In the above diagram, one could add a new shared property for **cf₁**, **cf₂**, **cf₃**, **cf₄**, and **cf₅** by assigning a new value to the property in **CF_p**.

Although ECMAScript objects are not inherently class-based, it is often convenient to define class-like abstractions based upon a common pattern of [constructor](#) functions, prototype objects, and methods. The ECMAScript built-in objects themselves follow such a class-like pattern. Beginning with ECMAScript 2015, the ECMAScript language includes syntactic class definitions that permit programmers to concisely define objects that conform to the same class-like abstraction pattern used by the built-in objects.

4.2.2 The Strict Variant of ECMAScript

The ECMAScript Language recognizes the possibility that some users of the language may wish to restrict their usage of some features available in the language. They might do so in the interests of security, to avoid what they consider to be error-prone features, to get enhanced error checking, or for other reasons of their choosing. In support of this possibility, ECMAScript defines a strict variant of the language. The strict variant of the language excludes some specific syntactic and semantic features of the regular ECMAScript language and modifies the detailed semantics of some features. The strict variant also specifies additional error conditions that must be reported by throwing error exceptions in situations that are not specified as errors by the non-strict form of the language.

The strict variant of ECMAScript is commonly referred to as the *strict mode* of the language. Strict mode selection and use of the strict mode syntax and semantics of ECMAScript is explicitly made at the level of individual ECMAScript source text units. Because strict mode is selected at the level of a syntactic source text unit, strict mode only imposes restrictions that have local effect within such a source text unit. Strict mode does not restrict or modify any aspect of the ECMAScript semantics that must operate consistently across multiple source text units. A complete ECMAScript program may be composed of both strict mode and non-strict mode ECMAScript source text units. In this case, strict mode only applies when actually executing code that is defined within a strict mode source text unit.

In order to conform to this specification, an ECMAScript implementation must implement both the full unrestricted ECMAScript language and the strict variant of the ECMAScript language as defined by this specification. In addition, an implementation must support the combination of unrestricted and strict mode source text units into a single composite program.

4.3 Terms and Definitions

For the purposes of this document, the following terms and definitions apply.

4.3.1 type

set of data values as defined in clause 6 of this specification

4.3.2 primitive value

member of one of the types Undefined, Null, Boolean, Number, Symbol, or String as defined in clause 6

NOTE

A primitive value is a datum that is represented directly at the lowest level of the language implementation.

4.3.3 object

member of the type Object

NOTE

An object is a collection of properties and has a single prototype object. The prototype may be the null value.

4.3.4 constructor

[function object](#) that creates and initializes objects

NOTE

The value of a [constructor](#)'s **prototype** property is a prototype object that is used to implement inheritance and shared properties.

4.3.5 prototype

object that provides shared properties for other objects

NOTE

When a [constructor](#) creates an object, that object implicitly references the [constructor](#)'s **prototype** property for the purpose of resolving property references. The [constructor](#)'s **prototype** property can be referenced by the program expression [constructor.prototype](#), and properties added to an object's prototype are shared, through inheritance, by all objects sharing the prototype. Alternatively, a new object may be created with an explicitly specified prototype by using the **Object.create** built-in function.

4.3.6 ordinary object

object that has the default behaviour for the essential internal methods that must be supported by all objects

4.3.7 exotic object

object that does not have the default behaviour for one or more of the essential internal methods

NOTE

Any object that is not an ordinary object is an [exotic object](#).

4.3.8 standard object

object whose semantics are defined by this specification

4.3.9 built-in object

object specified and supplied by an ECMAScript implementation

NOTE

Standard built-in objects are defined in this specification. An ECMAScript implementation may specify and supply additional kinds of built-in objects. A *built-in* [constructor](#) is a built-in object that is also a [constructor](#).

4.3.10 undefined value

primitive value used when a variable has not been assigned a value

4.3.11 Undefined type

type whose sole value is the **undefined** value

4.3.12 null value

primitive value that represents the intentional absence of any object value

4.3.13 Null type

type whose sole value is the **null** value

4.3.14 Boolean value

member of the Boolean type

NOTE

There are only two Boolean values, **true** and **false**.

4.3.15 Boolean type

type consisting of the primitive values **true** and **false**

4.3.16 Boolean object

member of the Object type that is an instance of the standard built-in **Boolean constructor**

NOTE

A Boolean object is created by using the **Boolean constructor** in a **new** expression, supplying a Boolean value as an argument. The resulting object has an internal slot whose value is the Boolean value. A Boolean object can be coerced to a Boolean value.

4.3.17 String value

primitive value that is a finite ordered sequence of zero or more 16-bit unsigned integer values

NOTE

A String value is a member of the String type. Each integer value in the sequence usually represents a single 16-bit unit of UTF-16 text. However, ECMAScript does not place any restrictions or requirements on the values except that they must be 16-bit unsigned integers.

4.3.18 String type

set of all possible String values

4.3.19 String object

member of the Object type that is an instance of the standard built-in **String constructor**

NOTE

A String object is created by using the **String constructor** in a **new** expression, supplying a String value as an argument. The resulting object has an internal slot whose value is the String value. A String object can be coerced to a String value by calling the **String constructor** as a function (21.1.1.1).

4.3.20 Number value

primitive value corresponding to a double-precision 64-bit binary format IEEE 754-2008 value

NOTE

A Number value is a member of the Number type and is a direct representation of a number.

4.3.21 Number type

set of all possible Number values including the special “Not-a-Number” (NaN) value, positive infinity, and negative infinity

4.3.22 Number object

member of the Object type that is an instance of the standard built-in **Number** constructor

NOTE

A Number object is created by using the **Number** constructor in a **new** expression, supplying a number value as an argument. The resulting object has an internal slot whose value is the number value. A Number object can be coerced to a number value by calling the **Number** constructor as a function (20.1.1.1).

4.3.23 Infinity

number value that is the positive infinite number value

4.3.24 NaN

number value that is an IEEE 754-2008 “Not-a-Number” value

4.3.25 Symbol value

primitive value that represents a unique, non-String Object property key

4.3.26 Symbol type

set of all possible Symbol values

4.3.27 Symbol object

member of the Object type that is an instance of the standard built-in **Symbol** constructor

4.3.28 function

member of the Object type that may be invoked as a subroutine

NOTE

In addition to its properties, a function contains executable code and state that determine how it behaves when invoked. A function's code may or may not be written in ECMAScript.

4.3.29 built-in function

built-in object that is a function

NOTE

Examples of built-in functions include **parseInt** and **Math.exp**. An implementation may provide implementation-dependent built-in functions that are not described in this specification.

4.3.30 property

part of an object that associates a key (either a String value or a Symbol value) and a value

NOTE

Depending upon the form of the property the value may be represented either directly as a data value (a primitive value, an object, or a [function object](#)) or indirectly by a pair of accessor functions.

4.3.31 method

function that is the value of a property

NOTE

When a function is called as a method of an object, the object is passed to the function as its **this** value.

4.3.32 built-in method

method that is a built-in function

NOTE

Standard built-in methods are defined in this specification, and an ECMAScript implementation may specify and provide other additional built-in methods.

4.3.33 attribute

internal value that defines some characteristic of a property

4.3.34 own property

property that is directly contained by its object

4.3.35 inherited property

property of an object that is not an own property but is a property (either own or inherited) of the object's prototype

4.4 Organization of This Specification

The remainder of this specification is organized as follows:

Clause 5 defines the notational conventions used throughout the specification.

Clauses 6-9 define the execution environment within which ECMAScript programs operate.

Clauses 10-16 define the actual ECMAScript programming language including its syntactic encoding and the execution semantics of all language features.

Clauses 17-26 define the ECMAScript standard library. They include the definitions of all of the standard objects that are available for use by ECMAScript programs as they execute.

Clause 27 describes the memory consistency model of accesses on SharedArrayBuffer-backed memory and methods of the Atomics object.

5 Notational Conventions

5.1 Syntactic and Lexical Grammars

5.1.1 Context-Free Grammars

A *context-free grammar* consists of a number of *productions*. Each production has an abstract symbol called a *nonterminal* as its *left-hand side*, and a sequence of zero or more nonterminal and *terminal* symbols as its *right-hand side*. For each grammar, the terminal symbols are drawn from a specified alphabet.

A chain production is a production that has exactly one nonterminal symbol on its right-hand side along with zero or more terminal symbols.

Starting from a sentence consisting of a single distinguished nonterminal, called the goal symbol, a given context-free grammar specifies a *language*, namely, the (perhaps infinite) set of possible sequences of terminal symbols that can result from repeatedly replacing any nonterminal in the sequence with a right-hand side of a production for which the nonterminal is the left-hand side.

5.1.2 The Lexical and RegExp Grammars

A *lexical grammar* for ECMAScript is given in clause 11. This grammar has as its terminal symbols Unicode code points that conform to the rules for *SourceCharacter* defined in 10.1. It defines a set of productions, starting from the *goal symbol* *InputElementDiv*, *InputElementTemplateTail*, or *InputElementRegExp*, or *InputElementRegExpOrTemplateTail*, that describe how sequences of such code points are translated into a sequence of input elements.

Input elements other than white space and comments form the terminal symbols for the syntactic grammar for ECMAScript and are called ECMAScript *tokens*. These tokens are the reserved words, identifiers, literals, and punctuators of the ECMAScript language. Moreover, line terminators, although not considered to be tokens, also become

part of the stream of input elements and guide the process of automatic semicolon insertion (11.9). Simple white space and single-line comments are discarded and do not appear in the stream of input elements for the syntactic grammar. A *MultiLineComment* (that is, a comment of the form `/*...*/` regardless of whether it spans more than one line) is likewise simply discarded if it contains no line terminator; but if a *MultiLineComment* contains one or more line terminators, then it is replaced by a single line terminator, which becomes part of the stream of input elements for the syntactic grammar.

A *RegExp grammar* for ECMAScript is given in 21.2.1. This grammar also has as its terminal symbols the code points as defined by *SourceCharacter*. It defines a set of productions, starting from the *goal symbol Pattern*, that describe how sequences of code points are translated into regular expression patterns.

Productions of the lexical and RegExp grammars are distinguished by having two colons “::” as separating punctuation. The lexical and RegExp grammars share some productions.

5.1.3 The Numeric String Grammar

Another grammar is used for translating Strings into numeric values. This grammar is similar to the part of the lexical grammar having to do with numeric literals and has as its terminal symbols *SourceCharacter*. This grammar appears in 7.1.3.1.

Productions of the numeric string grammar are distinguished by having three colons “:::” as punctuation.

5.1.4 The Syntactic Grammar

The *syntactic grammar* for ECMAScript is given in clauses 11, 12, 13, 14, and 15. This grammar has ECMAScript tokens defined by the lexical grammar as its terminal symbols (5.1.2). It defines a set of productions, starting from two alternative goal symbols *Script* and *Module*, that describe how sequences of tokens form syntactically correct independent components of ECMAScript programs.

When a stream of code points is to be parsed as an ECMAScript *Script* or *Module*, it is first converted to a stream of input elements by repeated application of the lexical grammar; this stream of input elements is then parsed by a single application of the syntactic grammar. The input stream is syntactically in error if the tokens in the stream of input elements cannot be parsed as a single instance of the goal nonterminal (*Script* or *Module*), with no tokens left over.

When a parse is successful, it constructs a *parse tree*, a rooted tree structure in which each node is a Parse Node. Each Parse Node is an *instance* of a symbol in the grammar; it represents a span of the source text that can be derived from that symbol. The root node of the parse tree, representing the whole of the source text, is an instance of the parse's *goal symbol*. When a Parse Node is an instance of a nonterminal, it is also an instance of some production that has that nonterminal as its left-hand side. Moreover, it has zero or more *children*, one for each symbol on the production's right-hand side: each child is a Parse Node that is an instance of the corresponding symbol.

New Parse Nodes are instantiated for each invocation of the parser and never reused between parses even of identical source text. Parse Nodes are considered the same Parse Node if and only if they represent the same span of source text, are instances of the same grammar symbol, and resulted from the same parser invocation.

NOTE 1

Parsing the same String multiple times will lead to different Parse Nodes, e.g., as occurs in:

```
eval(str); eval(str);
```

NOTE 2

Parse Nodes are specification artefacts, and implementations are not required to use an analogous data structure.

Productions of the syntactic grammar are distinguished by having just one colon “:” as punctuation.

The syntactic grammar as presented in clauses 12, 13, 14 and 15 is not a complete account of which token sequences are accepted as a correct ECMAScript *Script* or *Module*. Certain additional token sequences are also accepted, namely, those that would be described by the grammar if only semicolons were added to the sequence in certain places (such as before line terminator characters). Furthermore, certain token sequences that are described by the grammar are not considered acceptable if a line terminator character appears in certain “awkward” places.

In certain cases, in order to avoid ambiguities, the syntactic grammar uses generalized productions that permit token sequences that do not form a valid ECMAScript *Script* or *Module*. For example, this technique is used for object literals and object destructuring patterns. In such cases a more restrictive *supplemental grammar* is provided that further restricts the acceptable token sequences. Typically, an **early error** rule will then define an error condition if “*P* is not covering an *N*”, where *P* is a Parse Node (an instance of the generalized production) and *N* is a nonterminal from the supplemental grammar. Here, the sequence of tokens originally matched by *P* is parsed again using *N* as the **goal symbol**. (If *N* takes grammatical parameters, then they are set to the same values used when *P* was originally parsed.) An error occurs if the sequence of tokens cannot be parsed as a single instance of *N*, with no tokens left over. Subsequently, algorithms access the result of the parse using a phrase of the form “the *N* that is covered by *P*”. This will always be a Parse Node (an instance of *N*, unique for a given *P*), since any parsing failure would have been detected by an **early error** rule.

5.1.5 Grammar Notation

Terminal symbols of the lexical, RegExp, and numeric string grammars are shown in **fixed width** font, both in the productions of the grammars and throughout this specification whenever the text directly refers to such a terminal symbol. These are to appear in a script exactly as written. All terminal symbol code points specified in this way are to be understood as the appropriate Unicode code points from the Basic Latin range, as opposed to any similar-looking code points from other Unicode ranges.

Nonterminal symbols are shown in *italic* type. The definition of a nonterminal (also called a “production”) is introduced by the name of the nonterminal being defined followed by one or more colons. (The number of colons indicates to which grammar the production belongs.) One or more alternative right-hand sides for the nonterminal then follow on succeeding lines. For example, the syntactic definition:

```
WhileStatement :  
    while ( Expression ) Statement
```

states that the nonterminal *WhileStatement* represents the token **while**, followed by a left parenthesis token, followed by an *Expression*, followed by a right parenthesis token, followed by a *Statement*. The occurrences of *Expression* and *Statement* are themselves nonterminals. As another example, the syntactic definition:

```
ArgumentList :  
    AssignmentExpression  
    ArgumentList , AssignmentExpression
```

states that an *ArgumentList* may represent either a single *AssignmentExpression* or an *ArgumentList*, followed by a comma, followed by an *AssignmentExpression*. This definition of *ArgumentList* is recursive, that is, it is defined in terms of itself. The result is that an *ArgumentList* may contain any positive number of arguments, separated by commas, where each argument expression is an *AssignmentExpression*. Such recursive definitions of nonterminals are common.

The subscripted suffix “_{opt}”, which may appear after a terminal or nonterminal, indicates an optional symbol. The alternative containing the optional symbol actually specifies two right-hand sides, one that omits the optional element and one that includes it. This means that:

VariableDeclaration :
BindingIdentifier *Initializer*_{opt}

is a convenient abbreviation for:

VariableDeclaration :
BindingIdentifier
BindingIdentifier *Initializer*

and that:

IterationStatement :
for (*LexicalDeclaration* *Expression*_{opt} ; *Expression*_{opt}) *Statement*

is a convenient abbreviation for:

IterationStatement :
for (*LexicalDeclaration* ; *Expression*_{opt}) *Statement*
for (*LexicalDeclaration* *Expression* ; *Expression*_{opt}) *Statement*

which in turn is an abbreviation for:

IterationStatement :
for (*LexicalDeclaration* ;) *Statement*
for (*LexicalDeclaration* ; *Expression*) *Statement*
for (*LexicalDeclaration* *Expression* ;) *Statement*
for (*LexicalDeclaration* *Expression* ; *Expression*) *Statement*

so, in this example, the nonterminal *IterationStatement* actually has four alternative right-hand sides.

A production may be parameterized by a subscripted annotation of the form “[parameters]”, which may appear as a suffix to the nonterminal symbol defined by the production. “parameters” may be either a single name or a comma separated list of names. A parameterized production is shorthand for a set of productions defining all combinations of the parameter names, preceded by an underscore, appended to the parameterized nonterminal symbol. This means that:

*StatementList*_[Return] :
ReturnStatement
ExpressionStatement

is a convenient abbreviation for:

StatementList :
ReturnStatement
ExpressionStatement

StatementList_Return :
ReturnStatement
ExpressionStatement

and that:

```
StatementList[Return, In] :  
    ReturnStatement  
    ExpressionStatement
```

is an abbreviation for:

```
StatementList :  
    ReturnStatement  
    ExpressionStatement
```

```
StatementList_Return :  
    ReturnStatement  
    ExpressionStatement
```

```
StatementList_In :  
    ReturnStatement  
    ExpressionStatement
```

```
StatementList_Return_In :  
    ReturnStatement  
    ExpressionStatement
```

Multiple parameters produce a combinatory number of productions, not all of which are necessarily referenced in a complete grammar.

References to nonterminals on the right-hand side of a production can also be parameterized. For example:

```
StatementList :  
    ReturnStatement  
    ExpressionStatement[+In]
```

is equivalent to saying:

```
StatementList :  
    ReturnStatement  
    ExpressionStatement_In
```

and:

```
StatementList :  
    ReturnStatement  
    ExpressionStatement[~In]
```

is equivalent to:

```
StatementList :  
    ReturnStatement  
    ExpressionStatement
```

A nonterminal reference may have both a parameter list and an “opt” suffix. For example:

VariableDeclaration :

BindingIdentifier *Initializer* [+In] *opt*

is an abbreviation for:

VariableDeclaration :

BindingIdentifier

BindingIdentifier *Initializer* *In*

Prefixing a parameter name with “?” on a right-hand side nonterminal reference makes that parameter value dependent upon the occurrence of the parameter name on the reference to the current production's left-hand side symbol. For example:

VariableDeclaration [In] :

BindingIdentifier *Initializer* [?In]

is an abbreviation for:

VariableDeclaration :

BindingIdentifier *Initializer*

VariableDeclaration *In* :

BindingIdentifier *Initializer* *In*

If a right-hand side alternative is prefixed with “[+parameter]” that alternative is only available if the named parameter was used in referencing the production's nonterminal symbol. If a right-hand side alternative is prefixed with “[~parameter]” that alternative is only available if the named parameter was *not* used in referencing the production's nonterminal symbol. This means that:

StatementList [Return] :

[+Return] *ReturnStatement*

ExpressionStatement

is an abbreviation for:

StatementList :

ExpressionStatement

StatementList *Return* :

ReturnStatement

ExpressionStatement

and that:

StatementList [Return] :

[~Return] *ReturnStatement*

ExpressionStatement

is an abbreviation for:

StatementList :

ReturnStatement

ExpressionStatement

StatementList_Return :
ExpressionStatement

When the words “**one of**” follow the colon(s) in a grammar definition, they signify that each of the terminal symbols on the following line or lines is an alternative definition. For example, the lexical grammar for ECMAScript contains the production:

NonZeroDigit :: one of
1 2 3 4 5 6 7 8 9

which is merely a convenient abbreviation for:

NonZeroDigit ::
1
2
3
4
5
6
7
8
9

If the phrase “[empty]” appears as the right-hand side of a production, it indicates that the production's right-hand side contains no terminals or nonterminals.

If the phrase “[lookahead $\notin$ *set*]” appears in the right-hand side of a production, it indicates that the production may not be used if the immediately following input token sequence is a member of the given *set*. The *set* can be written as a comma separated list of one or two element terminal sequences enclosed in curly brackets. For convenience, the set can also be written as a nonterminal, in which case it represents the set of all terminals to which that nonterminal could expand. If the *set* consists of a single terminal the phrase “[lookahead $\neq$ *terminal*]” may be used.

For example, given the definitions:

DecimalDigit :: one of
0 1 2 3 4 5 6 7 8 9

DecimalDigits ::
DecimalDigit
DecimalDigits DecimalDigit

the definition:

LookaheadExample ::
n [lookahead $\notin$ { **1** , **3** , **5** , **7** , **9** }] *DecimalDigits*
DecimalDigit [lookahead $\notin$ *DecimalDigit*]

matches either the letter **n** followed by one or more decimal digits the first of which is even, or a decimal digit not followed by another decimal digit.

Similarly, if the phrase “[lookahead $\in$ *set*]” appears in the right-hand side of a production, it indicates that the production may only be used if the immediately following input token sequence is a member of the given *set*. If the *set* consists of a

single terminal the phrase “[lookahead = *terminal*]” may be used.

If the phrase “[no *LineTerminator* here]” appears in the right-hand side of a production of the syntactic grammar, it indicates that the production is a *restricted production*: it may not be used if a *LineTerminator* occurs in the input stream at the indicated position. For example, the production:

ThrowStatement :
 throw [no *LineTerminator* here] *Expression* ;

indicates that the production may not be used if a *LineTerminator* occurs in the script between the **throw** token and the *Expression*.

Unless the presence of a *LineTerminator* is forbidden by a restricted production, any number of occurrences of *LineTerminator* may appear between any two consecutive tokens in the stream of input elements without affecting the syntactic acceptability of the script.

When an alternative in a production of the lexical grammar or the numeric string grammar appears to be a multi-code point token, it represents the sequence of code points that would make up such a token.

The right-hand side of a production may specify that certain expansions are not permitted by using the phrase “**but not**” and then indicating the expansions to be excluded. For example, the production:

Identifier ::
 IdentifierName but not *ReservedWord*

means that the nonterminal *Identifier* may be replaced by any sequence of code points that could replace *IdentifierName* provided that the same sequence of code points could not replace *ReservedWord*.

Finally, a few nonterminal symbols are described by a descriptive phrase in sans-serif type in cases where it would be impractical to list all the alternatives:

SourceCharacter ::
 any Unicode code point

5.2 Algorithm Conventions

The specification often uses a numbered list to specify steps in an algorithm. These algorithms are used to precisely specify the required semantics of ECMAScript language constructs. The algorithms are not intended to imply the use of any specific implementation technique. In practice, there may be more efficient algorithms available to implement a given feature.

Algorithms may be explicitly parameterized, in which case the names and usage of the parameters must be provided as part of the algorithm's definition.

Algorithm steps may be subdivided into sequential substeps. Substeps are indented and may themselves be further divided into indented substeps. Outline numbering conventions are used to identify substeps with the first level of substeps labelled with lower case alphabetic characters and the second level of substeps labelled with lower case roman numerals. If more than three levels are required these rules repeat with the fourth level using numeric labels. For example:

1. Top-level step
 - a. Substep.

- b. Substep.
 - i. Subsubstep.
 - 1. Subsubsubstep
 - a. Subsubsubsubstep
 - i. Subsubsubsubsubstep

A step or substep may be written as an “if” predicate that conditions its substeps. In this case, the substeps are only applied if the predicate is true. If a step or substep begins with the word “else”, it is a predicate that is the negation of the preceding “if” predicate step at the same level.

A step may specify the iterative application of its substeps.

A step that begins with “Assert:” asserts an invariant condition of its algorithm. Such assertions are used to make explicit algorithmic invariants that would otherwise be implicit. Such assertions add no additional semantic requirements and hence need not be checked by an implementation. They are used simply to clarify algorithms.

Algorithm steps may declare named aliases for any value using the form “Let *x* be *someValue*”. These aliases are reference-like in that both *x* and *someValue* refer to the same underlying data and modifications to either are visible to both. Algorithm steps that want to avoid this reference-like behaviour should explicitly make a copy of the right-hand side: “Let *x* be a copy of *someValue*” creates a shallow copy of *someValue*.

Once declared, an alias may be referenced in any subsequent steps and must not be referenced from steps prior to the alias's declaration. Aliases may be modified using the form “Set *x* to *someOtherValue*”.

5.2.1 Abstract Operations

In order to facilitate their use in multiple parts of this specification, some algorithms, called abstract operations, are named and written in parameterized functional form so that they may be referenced by name from within other algorithms. Abstract operations are typically referenced using a functional application style such as `OperationName(arg1, arg2)`. Some abstract operations are treated as polymorphically dispatched methods of class-like specification abstractions. Such method-like abstract operations are typically referenced using a method application style such as `someValue.OperationName(arg1, arg2)`.

5.2.2 Syntax-Directed Operations

A syntax-directed operation is a named operation whose definition consists of algorithms, each of which is associated with one or more productions from one of the ECMAScript grammars. A production that has multiple alternative definitions will typically have a distinct algorithm for each alternative. When an algorithm is associated with a grammar production, it may reference the terminal and nonterminal symbols of the production alternative as if they were parameters of the algorithm. When used in this manner, nonterminal symbols refer to the actual alternative definition that is matched when parsing the source text. The source text matched by a grammar production is the portion of the source text that starts at the beginning of the first terminal that participated in the match and ends at the end of the last terminal that participated in the match.

When an algorithm is associated with a production alternative, the alternative is typically shown without any “[]” grammar annotations. Such annotations should only affect the syntactic recognition of the alternative and have no effect on the associated semantics for the alternative.

Syntax-directed operations are invoked with a parse node and, optionally, other parameters by using the conventions on steps 1, 3, and 4 in the following algorithm:

1. Let *status* be the result of performing SyntaxDirectedOperation of *SomeNonTerminal*.
2. Let *someParseNode* be the parse of some source text.
3. Perform SyntaxDirectedOperation of *someParseNode*.
4. Perform SyntaxDirectedOperation of *someParseNode* passing "**value**" as the argument.

Unless explicitly specified otherwise, all chain productions have an implicit definition for every operation that might be applied to that production's left-hand side nonterminal. The implicit definition simply reapplies the same operation with the same parameters, if any, to the *chain production*'s sole right-hand side nonterminal and then returns the result. For example, assume that some algorithm has a step of the form: "Return the result of evaluating *Block*" and that there is a production:

Block :
 { *StatementList* }

but the Evaluation operation does not associate an algorithm with that production. In that case, the Evaluation operation implicitly includes an association of the form:

Runtime Semantics: Evaluation

Block : { *StatementList* }

1. Return the result of evaluating *StatementList*.

5.2.3 Runtime Semantics

Algorithms which specify semantics that must be called at runtime are called runtime semantics. Runtime semantics are defined by *abstract operations* or syntax-directed operations. Such algorithms always return a completion record.

5.2.3.1 Implicit Completion Values

The algorithms of this specification often implicitly return *Completion* Records whose [[Type]] is *normal*. Unless it is otherwise obvious from the context, an algorithm statement that returns a value that is not a *Completion Record*, such as:

1. Return "**Infinity**".

means the same thing as:

1. Return *NormalCompletion*("Infinity").

However, if the value expression of a "return" statement is a *Completion Record* construction literal, the resulting *Completion Record* is returned. If the value expression is a call to an abstract operation, the "return" statement simply returns the *Completion Record* produced by the abstract operation.

The abstract operation *Completion*(*completionRecord*) is used to emphasize that a previously computed *Completion Record* is being returned. The *Completion* abstract operation takes a single argument, *completionRecord*, and performs the following steps:

1. *Assert*: *completionRecord* is a *Completion Record*.
2. Return *completionRecord* as the *Completion Record* of this abstract operation.

A "return" statement without a value in an algorithm step means the same thing as:

1. Return *NormalCompletion*(*undefined*).

Any reference to a [Completion Record](#) value that is in a context that does not explicitly require a complete [Completion Record](#) value is equivalent to an explicit reference to the `[[Value]]` field of the [Completion Record](#) value unless the [Completion Record](#) is an [abrupt completion](#).

5.2.3.2 Throw an Exception

Algorithms steps that say to throw an exception, such as

1. Throw a **TypeError** exception.

mean the same things as:

1. Return [ThrowCompletion](#)(a newly created **TypeError** object).

5.2.3.3 ReturnIfAbrupt

Algorithms steps that say or are otherwise equivalent to:

1. [ReturnIfAbrupt](#)(*argument*).

mean the same thing as:

1. If *argument* is an [abrupt completion](#), return *argument*.
2. Else if *argument* is a [Completion Record](#), set *argument* to *argument*.`[[Value]]`.

Algorithms steps that say or are otherwise equivalent to:

1. [ReturnIfAbrupt](#)([AbstractOperation](#)()).

mean the same thing as:

1. Let *hygienicTemp* be [AbstractOperation](#)().
2. If *hygienicTemp* is an [abrupt completion](#), return *hygienicTemp*.
3. Else if *hygienicTemp* is a [Completion Record](#), set *hygienicTemp* to *hygienicTemp*.`[[Value]]`.

Where *hygienicTemp* is ephemeral and visible only in the steps pertaining to [ReturnIfAbrupt](#).

Algorithms steps that say or are otherwise equivalent to:

1. Let *result* be [AbstractOperation](#)([ReturnIfAbrupt](#)(*argument*)).

mean the same thing as:

1. If *argument* is an [abrupt completion](#), return *argument*.
2. If *argument* is a [Completion Record](#), set *argument* to *argument*.`[[Value]]`.
3. Let *result* be [AbstractOperation](#)(*argument*).

5.2.3.4 ReturnIfAbrupt Shorthands

Invocations of [abstract operations](#) and syntax-directed operations that are prefixed by `?` indicate that [ReturnIfAbrupt](#) should be applied to the resulting [Completion Record](#). For example, the step:

1. `? OperationName()`.

is equivalent to the following step:

1. `ReturnIfAbrupt(OperationName())`.

Similarly, for method application style, the step:

1. `? someValue.OperationName()`.

is equivalent to:

1. `ReturnIfAbrupt(someValue.OperationName())`.

Similarly, prefix `!` is used to indicate that the following invocation of an abstract or syntax-directed operation will never return an `abrupt completion` and that the resulting `Completion Record`'s `[[Value]]` field should be used in place of the return value of the operation. For example, the step:

1. Let `val` be `! OperationName()`.

is equivalent to the following steps:

1. Let `val` be `OperationName()`.
2. `Assert: val` is never an `abrupt completion`.
3. If `val` is a `Completion Record`, set `val` to `val.[[Value]]`.

Syntax-directed operations for `runtime semantics` make use of this shorthand by placing `!` or `?` before the invocation of the operation:

1. Perform `! SyntaxDirectedOperation of NonTerminal`.

5.2.4 Static Semantics

Context-free grammars are not sufficiently powerful to express all the rules that define whether a stream of input elements form a valid ECMAScript *Script* or *Module* that may be evaluated. In some situations additional rules are needed that may be expressed using either ECMAScript algorithm conventions or prose requirements. Such rules are always associated with a production of a grammar and are called the static semantics of the production.

Static Semantic Rules have names and typically are defined using an algorithm. Named Static Semantic Rules are associated with grammar productions and a production that has multiple alternative definitions will typically have for each alternative a distinct algorithm for each applicable named static semantic rule.

Unless otherwise specified every grammar production alternative in this specification implicitly has a definition for a static semantic rule named `Contains` which takes an argument named `symbol` whose value is a terminal or nonterminal of the grammar that includes the associated production. The default definition of `Contains` is:

1. For each child node `child` of this `Parse Node`, do
 - a. If `child` is an instance of `symbol`, return **true**.
 - b. If `child` is an instance of a nonterminal, then
 - i. Let `contained` be the result of `child Contains symbol`.
 - ii. If `contained` is **true**, return **true**.
2. Return **false**.

The above definition is explicitly over-ridden for specific productions.

A special kind of static semantic rule is an Early Error Rule. **Early error** rules define **early error** conditions (see clause 16) that are associated with specific grammar productions. Evaluation of most **early error** rules are not explicitly invoked within the algorithms of this specification. A conforming implementation must, prior to the first evaluation of a *Script* or *Module*, validate all of the **early error** rules of the productions used to parse that *Script* or *Module*. If any of the **early error** rules are violated the *Script* or *Module* is invalid and cannot be evaluated.

5.2.5 Mathematical Operations

Mathematical operations such as addition, subtraction, negation, multiplication, division, and the mathematical functions defined later in this clause should always be understood as computing exact mathematical results on mathematical real numbers, which unless otherwise noted do not include infinities and do not include a negative zero that is distinguished from positive zero. Algorithms in this standard that model floating-point arithmetic include explicit steps, where necessary, to handle infinities and signed zero and to perform rounding. If a mathematical operation or function is applied to a floating-point number, it should be understood as being applied to the exact mathematical value represented by that floating-point number; such a floating-point number must be finite, and if it is **+0** or **-0** then the corresponding mathematical value is simply 0.

The mathematical function `abs(x)` produces the absolute value of *x*, which is **-*x*** if *x* is negative (less than zero) and otherwise is *x* itself.

The mathematical function `min(x1, x2, ..., xN)` produces the mathematically smallest of *x*1 through *x**N*. The mathematical function `max(x1, x2, ..., xN)` produces the mathematically largest of *x*1 through *x**N*. The domain and range of these mathematical functions include $+\infty$ and $-\infty$.

The notation “*x* modulo *y*” (*y* must be finite and nonzero) computes a value *k* of the same sign as *y* (or zero) such that `abs(k) < abs(y)` and $x - k = q \times y$ for some integer *q*.

The mathematical function `floor(x)` produces the largest integer (closest to positive infinity) that is not larger than *x*.

NOTE

`floor(x) = x - (x modulo 1).`

6 ECMAScript Data Types and Values

Algorithms within this specification manipulate values each of which has an associated type. The possible value types are exactly those defined in this clause. Types are further subclassified into ECMAScript language types and specification types.

Within this specification, the notation “Type(*x*)” is used as shorthand for “the type of *x*” where “type” refers to the ECMAScript language and specification types defined in this clause. When the term “empty” is used as if it was naming a value, it is equivalent to saying “no value of any type”.

6.1 ECMAScript Language Types

An ECMAScript language type corresponds to values that are directly manipulated by an ECMAScript programmer using the ECMAScript language. The ECMAScript language types are Undefined, Null, Boolean, String, Symbol, Number, and Object. An ECMAScript language value is a value that is characterized by an ECMAScript language type.

6.1.1 The Undefined Type

The Undefined type has exactly one value, called **undefined**. Any variable that has not been assigned a value has the value **undefined**.

6.1.2 The Null Type

The Null type has exactly one value, called **null**.

6.1.3 The Boolean Type

The Boolean type represents a logical entity having two values, called **true** and **false**.

6.1.4 The String Type

The String type is the set of all ordered sequences of zero or more 16-bit unsigned integer values (“elements”) up to a maximum length of $2^{53} - 1$ elements. The String type is generally used to represent textual data in a running ECMAScript program, in which case each element in the String is treated as a UTF-16 code unit value. Each element is regarded as occupying a position within the sequence. These positions are indexed with nonnegative integers. The first element (if any) is at index 0, the next element (if any) at index 1, and so on. The length of a String is the number of elements (i.e., 16-bit values) within it. The empty String has length zero and therefore contains no elements.

ECMAScript operations that do not interpret String contents apply no further semantics. Operations that do interpret String values treat each element as a single UTF-16 code unit. However, ECMAScript does not restrict the value of or relationships between these code units, so operations that further interpret String contents as sequences of Unicode code points encoded in UTF-16 must account for ill-formed subsequences. Such operations apply special treatment to every code unit with a numeric value in the inclusive range 0xD800 to 0xDBFF (defined by the Unicode Standard as a leading surrogate, or more formally as a high-surrogate code unit) and every code unit with a numeric value in the inclusive range 0xDC00 to 0xDFFF (defined as a trailing surrogate, or more formally as a low-surrogate code unit) using the following rules:

A code unit that is not a **leading surrogate** and not a **trailing surrogate** is interpreted as a code point with the same value.

A sequence of two code units, where the first code unit *c1* is a **leading surrogate** and the second code unit *c2* a **trailing surrogate**, is a surrogate pair and is interpreted as a code point with the value $(c1 - 0xD800) \times 0x400 + (c2 - 0xDC00) + 0x10000$. (See 10.1.2)

A code unit that is a **leading surrogate** or **trailing surrogate**, but is not part of a **surrogate pair**, is interpreted as a code point with the same value.

The function **String.prototype.normalize** (see 21.1.3.12) can be used to explicitly normalize a String value. **String.prototype.localeCompare** (see 21.1.3.10) internally normalizes String values, but no other operations implicitly normalize the strings upon which they operate. Only operations that are explicitly specified to be language or locale sensitive produce language-sensitive results.

NOTE

The rationale behind this design was to keep the implementation of Strings as simple and high-performing as possible. If ECMAScript source text is in Normalized Form C, string literals are guaranteed to also be normalized, as long as they do

not contain any Unicode escape sequences.

In this specification, the phrase "the string-concatenation of *A*, *B*, ..." (where each argument is a String value, a code unit, or a sequence of code units) denotes the String value whose sequence of code units is the concatenation of the code units (in order) of each of the arguments (in order).

6.1.5 The Symbol Type

The Symbol type is the set of all non-String values that may be used as the key of an Object property (6.1.7).

Each possible Symbol value is unique and immutable.

Each Symbol value immutably holds an associated value called `[[Description]]` that is either **undefined** or a String value.

6.1.5.1 Well-Known Symbols

Well-known symbols are built-in Symbol values that are explicitly referenced by algorithms of this specification. They are typically used as the keys of properties whose values serve as extension points of a specification algorithm. Unless otherwise specified, well-known symbols values are shared by all realms (8.2).

Within this specification a well-known symbol is referred to by using a notation of the form `@@name`, where "name" is one of the values listed in Table 1.

Table 1: Well-known Symbols

Specification Name	<code>[[Description]]</code>	Value and Purpose
<code>@@asyncIterator</code>	<code>"Symbol.asyncIterator"</code>	A method that returns the default AsyncIterator for an object. Called by the semantics of the for-await-of statement.
<code>@@hasInstance</code>	<code>"Symbol.hasInstance"</code>	A method that determines if a constructor object recognizes an object as one of the constructor's instances. Called by the semantics of the instanceof operator.
<code>@@isConcatSpreadable</code>	<code>"Symbol.isConcatSpreadable"</code>	A Boolean valued property that if true indicates that an object should be flattened to its array elements by Array.prototype.concat .
<code>@@iterator</code>	<code>"Symbol.iterator"</code>	A method that returns the default Iterator for an object. Called by the semantics of the for-of statement.
<code>@@match</code>	<code>"Symbol.match"</code>	A regular expression method that matches the regular expression against a string. Called by the String.prototype.match method.
<code>@@replace</code>	<code>"Symbol.replace"</code>	A regular expression method that replaces matched substrings of a string. Called by the String.prototype.replace method.

<code>@@search</code>	<code>"Symbol.search"</code>	A regular expression method that returns the index within a string that matches the regular expression. Called by the <code>String.prototype.search</code> method.
<code>@@species</code>	<code>"Symbol.species"</code>	A function valued property that is the <code>constructor</code> function that is used to create derived objects.
<code>@@split</code>	<code>"Symbol.split"</code>	A regular expression method that splits a string at the indices that match the regular expression. Called by the <code>String.prototype.split</code> method.
<code>@@toPrimitive</code>	<code>"Symbol.toPrimitive"</code>	A method that converts an object to a corresponding primitive value. Called by the <code>ToPrimitive</code> abstract operation.
<code>@@toStringTag</code>	<code>"Symbol.toStringTag"</code>	A String valued property that is used in the creation of the default string description of an object. Accessed by the built-in method <code>Object.prototype.toString</code> .
<code>@@unscopables</code>	<code>"Symbol.unscopables"</code>	An object valued property whose own and inherited property names are property names that are excluded from the <code>with</code> environment bindings of the associated object.

6.1.6 The Number Type

The Number type has exactly 18437736874454810627 (that is, $2^{64} - 2^{53} + 3$) values, representing the double-precision 64-bit format IEEE 754-2008 values as specified in the IEEE Standard for Binary Floating-Point Arithmetic, except that the 9007199254740990 (that is, $2^{53} - 2$) distinct “Not-a-Number” values of the IEEE Standard are represented in ECMAScript as a single special **NaN** value. (Note that the **NaN** value is produced by the program expression **NaN**.) In some implementations, external code might be able to detect a difference between various Not-a-Number values, but such behaviour is implementation-dependent; to ECMAScript code, all **NaN** values are indistinguishable from each other.

NOTE

The bit pattern that might be observed in an `ArrayBuffer` (see 24.1) or a `SharedArrayBuffer` (see 24.2) after a Number value has been stored into it is not necessarily the same as the internal representation of that Number value used by the ECMAScript implementation.

There are two other special values, called **positive Infinity** and **negative Infinity**. For brevity, these values are also referred to for expository purposes by the symbols $+\infty$ and $-\infty$, respectively. (Note that these two infinite Number values are produced by the program expressions **+Infinity** (or simply **Infinity**) and **-Infinity**.)

The other 18437736874454810624 (that is, $2^{64} - 2^{53}$) values are called the finite numbers. Half of these are positive numbers and half are negative numbers; for every finite positive Number value there is a corresponding negative value having the same magnitude.

Note that there is both a **positive zero** and a **negative zero**. For brevity, these values are also referred to for expository purposes by the symbols **+0** and **-0**, respectively. (Note that these two different zero Number values are produced by the program expressions **+0** (or simply **0**) and **-0**.)

The 18437736874454810622 (that is, $2^{64} - 2^{53} - 2$) finite nonzero values are of two kinds:

18428729675200069632 (that is, $2^{64} - 2^{54}$) of them are normalized, having the form

$$s \times m \times 2^e$$

where s is +1 or -1, m is a positive integer less than 2^{53} but not less than 2^{52} , and e is an integer ranging from -1074 to 971, inclusive.

The remaining 9007199254740990 (that is, $2^{53} - 2$) values are denormalized, having the form

$$s \times m \times 2^e$$

where s is +1 or -1, m is a positive integer less than 2^{52} , and e is -1074.

Note that all the positive and negative integers whose magnitude is no greater than 2^{53} are representable in the Number type (indeed, the integer 0 has two representations, **+0** and **-0**).

A finite number has an *odd significand* if it is nonzero and the integer m used to express it (in one of the two forms shown above) is odd. Otherwise, it has an *even significand*.

In this specification, the phrase “the Number value for x ” where x represents an exact real mathematical quantity (which might even be an irrational number such as π) means a Number value chosen in the following manner. Consider the set of all finite values of the Number type, with **-0** removed and with two additional values added to it that are not representable in the Number type, namely 2^{1024} (which is $+1 \times 2^{53} \times 2^{971}$) and -2^{1024} (which is $-1 \times 2^{53} \times 2^{971}$). Choose the member of this set that is closest in value to x . If two values of the set are equally close, then the one with an even significand is chosen; for this purpose, the two extra values 2^{1024} and -2^{1024} are considered to have even significands. Finally, if 2^{1024} was chosen, replace it with $+\infty$; if -2^{1024} was chosen, replace it with $-\infty$; if **+0** was chosen, replace it with **-0** if and only if x is less than zero; any other chosen value is used unchanged. The result is the Number value for x . (This procedure corresponds exactly to the behaviour of the IEEE 754-2008 “round to nearest, ties to even” mode.)

Some ECMAScript operators deal only with integers in specific ranges such as -2^{31} through $2^{31} - 1$, inclusive, or in the range 0 through $2^{16} - 1$, inclusive. These operators accept any value of the Number type but first convert each such value to an integer value in the expected range. See the descriptions of the numeric conversion operations in 7.1.

6.1.7 The Object Type

An Object is logically a collection of properties. Each property is either a data property, or an accessor property:

A data property associates a key value with an [ECMAScript language value](#) and a set of Boolean attributes.

An accessor property associates a key value with one or two accessor functions, and a set of Boolean attributes. The accessor functions are used to store or retrieve an [ECMAScript language value](#) that is associated with the property.

Properties are identified using key values. A property key value is either an ECMAScript String value or a Symbol value. All String and Symbol values, including the empty string, are valid as property keys. A property name is a property key

that is a String value.

An integer index is a String-valued property key that is a canonical numeric String (see 7.1.16) and whose numeric value is either +0 or a positive integer $\leq 2^{53} - 1$. An array index is an integer index whose numeric value i is in the range $+0 \leq i < 2^{32} - 1$.

Property keys are used to access properties and their values. There are two kinds of access for properties: *get* and *set*, corresponding to value retrieval and assignment, respectively. The properties accessible via get and set access includes both *own properties* that are a direct part of an object and *inherited properties* which are provided by another associated object via a property inheritance relationship. Inherited properties may be either own or inherited properties of the associated object. Each own property of an object must each have a key value that is distinct from the key values of the other own properties of that object.

All objects are logically collections of properties, but there are multiple forms of objects that differ in their semantics for accessing and manipulating their properties. Ordinary objects are the most common form of objects and have the default object semantics. An exotic object is any form of object whose property semantics differ in any way from the default semantics.

6.1.7.1 Property Attributes

Attributes are used in this specification to define and explain the state of Object properties. A data property associates a key value with the attributes listed in Table 2.

Table 2: Attributes of a Data Property

Attribute Name	Value Domain	Description
[[Value]]	Any ECMAScript language type	The value retrieved by a get access of the property.
[[Writable]]	Boolean	If false , attempts by ECMAScript code to change the property's [[Value]] attribute using [[Set]] will not succeed.
[[Enumerable]]	Boolean	If true , the property will be enumerated by a for-in enumeration (see 13.7.5). Otherwise, the property is said to be non-enumerable.
[[Configurable]]	Boolean	If false , attempts to delete the property, change the property to be an accessor property, or change its attributes (other than [[Value]], or changing [[Writable]] to false) will fail.

An accessor property associates a key value with the attributes listed in Table 3.

Table 3: Attributes of an Accessor Property

Attribute Name	Value Domain	Description
[[Get]]	Object Undefined	If the value is an Object it must be a function object. The function's [[Call]] internal method (Table 6) is called with an empty arguments list to retrieve the property value each time a get access of the property is performed.

[[Set]]	Object Undefined	If the value is an Object it must be a function object . The function's [[Call]] internal method (Table 6) is called with an arguments list containing the assigned value as its sole argument each time a set access of the property is performed. The effect of a property's [[Set]] internal method may, but is not required to, have an effect on the value returned by subsequent calls to the property's [[Get]] internal method.
[[Enumerable]]	Boolean	If true , the property is to be enumerated by a for-in enumeration (see 13.7.5). Otherwise, the property is said to be non-enumerable.
[[Configurable]]	Boolean	If false , attempts to delete the property, change the property to be a data property , or change its attributes will fail.

If the initial values of a property's attributes are not explicitly specified by this specification, the default value defined in [Table 4](#) is used.

Table 4: Default Attribute Values

Attribute Name	Default Value
[[Value]]	undefined
[[Get]]	undefined
[[Set]]	undefined
[[Writable]]	false
[[Enumerable]]	false
[[Configurable]]	false

6.1.7.2 Object Internal Methods and Internal Slots

The actual semantics of objects, in ECMAScript, are specified via algorithms called *internal methods*. Each object in an ECMAScript engine is associated with a set of internal methods that defines its runtime behaviour. These internal methods are not part of the ECMAScript language. They are defined by this specification purely for expository purposes. However, each object within an implementation of ECMAScript must behave as specified by the internal methods associated with it. The exact manner in which this is accomplished is determined by the implementation.

Internal method names are polymorphic. This means that different object values may perform different algorithms when a common internal method name is invoked upon them. That actual object upon which an internal method is invoked is the “target” of the invocation. If, at runtime, the implementation of an algorithm attempts to use an internal method of an object that the object does not support, a **TypeError** exception is thrown.

Internal slots correspond to internal state that is associated with objects and used by various ECMAScript specification algorithms. Internal slots are not object properties and they are not inherited. Depending upon the specific internal slot specification, such state may consist of values of any [ECMAScript language type](#) or of specific ECMAScript specification type values. Unless explicitly specified otherwise, internal slots are allocated as part of the process of

creating an object and may not be dynamically added to an object. Unless specified otherwise, the initial value of an internal slot is the value **undefined**. Various algorithms within this specification create objects that have internal slots. However, the ECMAScript language provides no direct way to associate internal slots with an object.

Internal methods and internal slots are identified within this specification using names enclosed in double square brackets `[[ ]]`.

[Table 5](#) summarizes the *essential internal methods* used by this specification that are applicable to all objects created or manipulated by ECMAScript code. Every object must have algorithms for all of the essential internal methods. However, all objects do not necessarily use the same algorithms for those methods.

The “Signature” column of [Table 5](#) and other similar tables describes the invocation pattern for each internal method. The invocation pattern always includes a parenthesized list of descriptive parameter names. If a parameter name is the same as an ECMAScript type name then the name describes the required type of the parameter value. If an internal method explicitly returns a value, its parameter list is followed by the symbol “→” and the type name of the returned value. The type names used in signatures refer to the types defined in [clause 6](#) augmented by the following additional names. “*any*” means the value may be any [ECMAScript language type](#). An internal method implicitly returns a [Completion Record](#). In addition to its parameters, an internal method always has access to the object that is the target of the method invocation.

Table 5: Essential Internal Methods

Internal Method	Signature	Description
[[GetPrototypeOf]]	() → Object Null	Determine the object that provides inherited properties for this object. A null value indicates that there are no inherited properties.
[[SetPrototypeOf]]	(Object Null) → Boolean	Associate this object with another object that provides inherited properties. Passing null indicates that there are no inherited properties. Returns true indicating that the operation was completed successfully or false indicating that the operation was not successful.
[[IsExtensible]]	() → Boolean	Determine whether it is permitted to add additional properties to this object.
[[PreventExtensions]]	() → Boolean	Control whether new properties may be added to this object. Returns true if the operation was successful or false if the operation was unsuccessful.
[[GetOwnProperty]]	(<i>propertyKey</i>) → Undefined Property Descriptor	Return a Property Descriptor for the own property of this object whose key is <i>propertyKey</i> , or undefined if no such property exists.
[[DefineOwnProperty]]	(<i>propertyKey</i> , <i>PropertyDescriptor</i>) → Boolean	Create or alter the own property, whose key is <i>propertyKey</i> , to have the state described by <i>PropertyDescriptor</i> . Return true if that property was successfully created/updated or false if the property could not be created or updated.
[[HasProperty]]	(<i>propertyKey</i>) → Boolean	Return a Boolean value indicating whether this object already has either an own or inherited property whose key is <i>propertyKey</i> .

[[Get]]	(<i>propertyKey</i> , <i>Receiver</i>) → <i>any</i>	Return the value of the property whose key is <i>propertyKey</i> from this object. If any ECMAScript code must be executed to retrieve the property value, <i>Receiver</i> is used as the this value when evaluating the code.
[[Set]]	(<i>propertyKey</i> , <i>value</i> , <i>Receiver</i>) → Boolean	Set the value of the property whose key is <i>propertyKey</i> to <i>value</i> . If any ECMAScript code must be executed to set the property value, <i>Receiver</i> is used as the this value when evaluating the code. Returns true if the property value was set or false if it could not be set.
[[Delete]]	(<i>propertyKey</i>) → Boolean	Remove the own property whose key is <i>propertyKey</i> from this object. Return false if the property was not deleted and is still present. Return true if the property was deleted or is not present.
[[OwnPropertyKeys]]	() → List of propertyKey	Return a List whose elements are all of the own property keys for the object.

Table 6 summarizes additional essential internal methods that are supported by objects that may be called as functions. A function object is an object that supports the [[Call]] internal method. A constructor is an object that supports the [[Construct]] internal method. Every object that supports [[Construct]] must support [[Call]]; that is, every **constructor** must be a **function object**. Therefore, a **constructor** may also be referred to as a *constructor function* or *constructor function object*.

Table 6: Additional Essential Internal Methods of Function Objects

Internal Method	Signature	Description
[[Call]]	(<i>any</i> , a List of <i>any</i>) → <i>any</i>	Executes code associated with this object. Invoked via a function call expression. The arguments to the internal method are a this value and a list containing the arguments passed to the function by a call expression. Objects that implement this internal method are <i>callable</i> .
[[Construct]]	(a List of <i>any</i> , Object) → Object	Creates an object. Invoked via the new or super operators. The first argument to the internal method is a list containing the arguments of the operator. The second argument is the object to which the new operator was initially applied. Objects that implement this internal method are called <i>constructors</i> . A function object is not necessarily a constructor and such non- constructor function objects do not have a [[Construct]] internal method.

The semantics of the essential internal methods for ordinary objects and standard exotic objects are specified in clause 9. If any specified use of an internal method of an **exotic object** is not supported by an implementation, that usage must throw a **TypeError** exception when attempted.

6.1.7.3 Invariants of the Essential Internal Methods

The Internal Methods of Objects of an ECMAScript engine must conform to the list of invariants specified below. Ordinary ECMAScript Objects as well as all standard exotic objects in this specification maintain these invariants. ECMAScript Proxy objects maintain these invariants by means of runtime checks on the result of traps invoked on the [[ProxyHandler]] object.

Any implementation provided exotic objects must also maintain these invariants for those objects. Violation of these invariants may cause ECMAScript code to have unpredictable behaviour and create security issues. However, violation of these invariants must never compromise the memory safety of an implementation.

An implementation must not allow these invariants to be circumvented in any manner such as by providing alternative interfaces that implement the functionality of the essential internal methods without enforcing their invariants.

Definitions:

The *target* of an internal method is the object upon which the internal method is called.

A target is *non-extensible* if it has been observed to return **false** from its `[[IsExtensible]]` internal method, or **true** from its `[[PreventExtensions]]` internal method.

A *non-existent* property is a property that does not exist as an own property on a non-extensible target.

All references to *SameValue* are according to the definition of the *SameValue* algorithm.

`[[GetPrototypeOf]] ( )`

The Type of the return value must be either Object or Null.

If target is non-extensible, and `[[GetPrototypeOf]]` returns a value *V*, then any future calls to `[[GetPrototypeOf]]` should return the *SameValue* as *V*.

NOTE 1

An object's prototype chain should have finite length (that is, starting from any object, recursively applying the `[[GetPrototypeOf]]` internal method to its result should eventually lead to the value **null**). However, this requirement is not enforceable as an object level invariant if the prototype chain includes any exotic objects that do not use the ordinary object definition of `[[GetPrototypeOf]]`. Such a circular prototype chain may result in infinite loops when accessing object properties.

`[[SetPrototypeOf]] ( V )`

The Type of the return value must be Boolean.

If target is non-extensible, `[[SetPrototypeOf]]` must return **false**, unless *V* is the *SameValue* as the target's observed `[[GetPrototypeOf]]` value.

`[[IsExtensible]] ( )`

The Type of the return value must be Boolean.

If `[[IsExtensible]]` returns **false**, all future calls to `[[IsExtensible]]` on the target must return **false**.

`[[PreventExtensions]] ( )`

The Type of the return value must be Boolean.

If `[[PreventExtensions]]` returns **true**, all future calls to `[[IsExtensible]]` on the target must return **false** and the target is now considered non-extensible.

`[[GetOwnProperty]] ( P )`

The Type of the return value must be either *Property Descriptor* or Undefined.

If the Type of the return value is *Property Descriptor*, the return value must be a *complete property descriptor*.

If *P* is described as a non-configurable, non-writable own *data property*, all future calls to `[[GetOwnProperty]] ( P )` must return *Property Descriptor* whose `[[Value]]` is *SameValue* as *P*'s `[[Value]]` attribute.

If *P*'s attributes other than `[[Writable]]` may change over time or if the property might be deleted, then *P*'s `[[Configurable]]` attribute must be **true**.

If the `[[Writable]]` attribute may change from **false** to **true**, then the `[[Configurable]]` attribute must be **true**.

If the target is non-extensible and *P* is non-existent, then all future calls to `[[GetOwnProperty]]` (*P*) on the target must describe *P* as non-existent (i.e. `[[GetOwnProperty]]` (*P*) must return **undefined**).

NOTE 2

As a consequence of the third invariant, if a property is described as a **data property** and it may return different values over time, then either or both of the `[[Writable]]` and `[[Configurable]]` attributes must be **true** even if no mechanism to change the value is exposed via the other internal methods.

`[[DefineOwnProperty]]` (*P*, *Desc*)

The Type of the return value must be Boolean.

`[[DefineOwnProperty]]` must return **false** if *P* has previously been observed as a non-configurable own property of the target, unless either:

1. *P* is a writable **data property**. A non-configurable writable **data property** can be changed into a non-configurable non-writable **data property**.
2. All attributes of *Desc* are the **SameValue** as *P*'s attributes.

`[[DefineOwnProperty]]` (*P*, *Desc*) must return **false** if target is non-extensible and *P* is a non-existent own property. That is, a non-extensible target object cannot be extended with new properties.

`[[HasProperty]]` (*P*)

The Type of the return value must be Boolean.

If *P* was previously observed as a non-configurable own data or **accessor property** of the target, `[[HasProperty]]` must return **true**.

`[[Get]]` (*P*, *Receiver*)

If *P* was previously observed as a non-configurable, non-writable own **data property** of the target with value *V*, then `[[Get]]` must return the **SameValue** as *V*.

If *P* was previously observed as a non-configurable own **accessor property** of the target whose `[[Get]]` attribute is **undefined**, the `[[Get]]` operation must return **undefined**.

`[[Set]]` (*P*, *V*, *Receiver*)

The Type of the return value must be Boolean.

If *P* was previously observed as a non-configurable, non-writable own **data property** of the target, then `[[Set]]` must return **false** unless *V* is the **SameValue** as *P*'s `[[Value]]` attribute.

If *P* was previously observed as a non-configurable own **accessor property** of the target whose `[[Set]]` attribute is **undefined**, the `[[Set]]` operation must return **false**.

`[[Delete]]` (*P*)

The Type of the return value must be Boolean.

If *P* was previously observed as a non-configurable own data or **accessor property** of the target, `[[Delete]]` must return **false**.

`[[OwnPropertyKeys]]` ()

The return value must be a **List**.

The returned [List](#) must not contain any duplicate entries.

The Type of each element of the returned [List](#) is either String or Symbol.

The returned [List](#) must contain at least the keys of all non-configurable own properties that have previously been observed.

If the object is non-extensible, the returned [List](#) must contain only the keys of all own properties of the object that are observable using `[[GetOwnProperty]]`.

[[Construct]] ()

The Type of the return value must be Object.

6.1.7.4 Well-Known Intrinsic Objects

Well-known intrinsics are built-in objects that are explicitly referenced by the algorithms of this specification and which usually have [realm](#)-specific identities. Unless otherwise specified each intrinsic object actually corresponds to a set of similar objects, one per [realm](#).

Within this specification a reference such as `%name%` means the intrinsic object, associated with the current [realm](#), corresponding to the name. Determination of the current [realm](#) and its intrinsics is described in [8.3](#). The well-known intrinsics are listed in [Table 7](#).

Table 7: Well-Known Intrinsic Objects

Intrinsic Name	Global Name	ECMAScript Language Association
%Array%	Array	The Array constructor (22.1.1)
%ArrayBuffer%	ArrayBuffer	The ArrayBuffer constructor (24.1.2)
%ArrayBufferPrototype%	ArrayBuffer.prototype	The initial value of the prototype data property of %ArrayBuffer% .
%ArrayIteratorPrototype%		The prototype of Array iterator objects (22.1.5)
%ArrayPrototype%	Array.prototype	The initial value of the prototype data property of %Array% (22.1.3)
%ArrayProto_entries%	Array.prototype.entries	The initial value of the entries data property of %ArrayPrototype% (22.1.3.4)
%ArrayProto_forEach%	Array.prototype.forEach	The initial value of the forEach data property of %ArrayPrototype% (22.1.3.12)
%ArrayProto_keys%	Array.prototype.keys	The initial value of the keys data property of %ArrayPrototype% (22.1.3.16)

%ArrayProto_values%	Array.prototype.values	The initial value of the values data property of %ArrayPrototype% (22.1.3.32)
%AsyncFromSyncIteratorPrototype%		The prototype of async-from-sync iterator objects (25.1.4)
%AsyncFunction%		The constructor of async function objects (25.7.1)
%AsyncFunctionPrototype%		The initial value of the prototype data property of %AsyncFunction%
%AsyncGenerator%		The initial value of the prototype property of %AsyncGeneratorFunction%
%AsyncGeneratorFunction%		The constructor of async iterator objects (25.3.1)
%AsyncGeneratorPrototype%		The initial value of the prototype property of %AsyncGenerator%
%AsyncIteratorPrototype%		An object that all standard built-in async iterator objects indirectly inherit from
%Atomics%	Atomics	The Atomics object (24.4)
%Boolean%	Boolean	The Boolean constructor (19.3.1)
%BooleanPrototype%	Boolean.prototype	The initial value of the prototype data property of %Boolean% (19.3.3)
%DataView%	DataView	The DataView constructor (24.3.2)
%DataViewPrototype%	DataView.prototype	The initial value of the prototype data property of %DataView%
%Date%	Date	The Date constructor (20.3.2)
%DatePrototype%	Date.prototype	The initial value of the prototype data property of %Date% .
%decodeURI%	decodeURI	The decodeURI function (18.2.6.2)

%decodeURIComponent%	decodeURIComponent	The decodeURIComponent function (18.2.6.3)
%encodeURI%	encodeURI	The encodeURI function (18.2.6.4)
%encodeURIComponent%	encodeURIComponent	The encodeURIComponent function (18.2.6.5)
%Error%	Error	The Error constructor (19.5.1)
%ErrorPrototype%	Error.prototype	The initial value of the prototype data property of %Error%
%eval%	eval	The eval function (18.2.1)
%EvalError%	EvalError	The EvalError constructor (19.5.5.1)
%EvalErrorPrototype%	EvalError.prototype	The initial value of the prototype data property of %EvalError%
%Float32Array%	Float32Array	The Float32Array constructor (22.2)
%Float32ArrayPrototype%	Float32Array.prototype	The initial value of the prototype data property of %Float32Array%
%Float64Array%	Float64Array	The Float64Array constructor (22.2)
%Float64ArrayPrototype%	Float64Array.prototype	The initial value of the prototype data property of %Float64Array%
%Function%	Function	The Function constructor (19.2.1)
%FunctionPrototype%	Function.prototype	The initial value of the prototype data property of %Function%
%Generator%		The initial value of the prototype data property of %GeneratorFunction%
%GeneratorFunction%		The constructor of generator objects (25.2.1)

%GeneratorPrototype%		The initial value of the prototype data property of %Generator%
%Int8Array%	Int8Array	The Int8Array constructor (22.2)
%Int8ArrayPrototype%	Int8Array.prototype	The initial value of the prototype data property of %Int8Array%
%Int16Array%	Int16Array	The Int16Array constructor (22.2)
%Int16ArrayPrototype%	Int16Array.prototype	The initial value of the prototype data property of %Int16Array%
%Int32Array%	Int32Array	The Int32Array constructor (22.2)
%Int32ArrayPrototype%	Int32Array.prototype	The initial value of the prototype data property of %Int32Array%
%isFinite%	isFinite	The isFinite function (18.2.2)
%isNaN%	isNaN	The isNaN function (18.2.3)
%IteratorPrototype%		An object that all standard built-in iterator objects indirectly inherit from
%JSON%	JSON	The JSON object (24.5)
%JSONParse%	JSON.parse	The initial value of the parse data property of %JSON%
%JSONStringify%	JSON.stringify	The initial value of the stringify data property of %JSON%
%Map%	Map	The Map constructor (23.1.1)
%MapIteratorPrototype%		The prototype of Map iterator objects (23.1.5)
%MapPrototype%	Map.prototype	The initial value of the prototype data property of %Map%
%Math%	Math	The Math object (20.2)
%Number%	Number	The Number constructor (20.1.1)

<code>%NumberPrototype%</code>	<code>Number.prototype</code>	The initial value of the prototype data property of <code>%Number%</code>
<code>%Object%</code>	<code>Object</code>	The Object constructor (19.1.1)
<code>%ObjectPrototype%</code>	<code>Object.prototype</code>	The initial value of the prototype data property of <code>%Object%</code> (19.1.3)
<code>%ObjProto_toString%</code>	<code>Object.prototype.toString</code>	The initial value of the toString data property of <code>%ObjectPrototype%</code> (19.1.3.6)
<code>%ObjProto_valueOf%</code>	<code>Object.prototype.valueOf</code>	The initial value of the valueOf data property of <code>%ObjectPrototype%</code> (19.1.3.7)
<code>%parseFloat%</code>	<code>parseFloat</code>	The parseFloat function (18.2.4)
<code>%parseInt%</code>	<code>parseInt</code>	The parseInt function (18.2.5)
<code>%Promise%</code>	<code>Promise</code>	The Promise constructor (25.6.3)
<code>%PromisePrototype%</code>	<code>Promise.prototype</code>	The initial value of the prototype data property of <code>%Promise%</code>
<code>%PromiseProto_then%</code>	<code>Promise.prototype.then</code>	The initial value of the then data property of <code>%PromisePrototype%</code> (25.6.5.4)
<code>%Promise_all%</code>	<code>Promise.all</code>	The initial value of the all data property of <code>%Promise%</code> (25.6.4.1)
<code>%Promise_reject%</code>	<code>Promise.reject</code>	The initial value of the reject data property of <code>%Promise%</code> (25.6.4.4)
<code>%Promise_resolve%</code>	<code>Promise.resolve</code>	The initial value of the resolve data property of <code>%Promise%</code> (25.6.4.5)
<code>%Proxy%</code>	<code>Proxy</code>	The Proxy constructor (26.2.1)
<code>%RangeError%</code>	<code>RangeError</code>	The RangeError constructor (19.5.5.2)
<code>%RangeErrorPrototype%</code>	<code>RangeError.prototype</code>	The initial value of the prototype data property of <code>%RangeError%</code>

%ReferenceError%	ReferenceError	The ReferenceError constructor (19.5.5.3)
%ReferenceErrorPrototype%	ReferenceError.prototype	The initial value of the prototype data property of %ReferenceError%
%Reflect%	Reflect	The Reflect object (26.1)
%RegExp%	RegExp	The RegExp constructor (21.2.3)
%RegExpPrototype%	RegExp.prototype	The initial value of the prototype data property of %RegExp%
%Set%	Set	The Set constructor (23.2.1)
%SetIteratorPrototype%		The prototype of Set iterator objects (23.2.5)
%SetPrototype%	Set.prototype	The initial value of the prototype data property of %Set%
%SharedArrayBuffer%	SharedArrayBuffer	The SharedArrayBuffer constructor (24.2.2)
%SharedArrayBufferPrototype%	SharedArrayBuffer.prototype	The initial value of the prototype data property of %SharedArrayBuffer%
%String%	String	The String constructor (21.1.1)
%StringIteratorPrototype%		The prototype of String iterator objects (21.1.5)
%StringPrototype%	String.prototype	The initial value of the prototype data property of %String%
%Symbol%	Symbol	The Symbol constructor (19.4.1)
%SymbolPrototype%	Symbol.prototype	The initial value of the prototype data property of %Symbol% (19.4.3)
%SyntaxError%	SyntaxError	The SyntaxError constructor (19.5.5.4)
%SyntaxErrorPrototype%	SyntaxError.prototype	The initial value of the prototype data property of %SyntaxError%

%ThrowTypeError%		A function object that unconditionally throws a new instance of %TypeError%
%TypedArray%		The super class of all typed Array constructors (22.2.1)
%TypedArrayPrototype%		The initial value of the prototype data property of %TypedArray%
%TypeError%	TypeError	The TypeError constructor (19.5.5.5)
%TypeErrorPrototype%	TypeError.prototype	The initial value of the prototype data property of %TypeError%
%Uint8Array%	Uint8Array	The Uint8Array constructor (22.2)
%Uint8ArrayPrototype%	Uint8Array.prototype	The initial value of the prototype data property of %Uint8Array%
%Uint8ClampedArray%	Uint8ClampedArray	The Uint8ClampedArray constructor (22.2)
%Uint8ClampedArrayPrototype%	Uint8ClampedArray.prototype	The initial value of the prototype data property of %Uint8ClampedArray%
%Uint16Array%	Uint16Array	The Uint16Array constructor (22.2)
%Uint16ArrayPrototype%	Uint16Array.prototype	The initial value of the prototype data property of %Uint16Array%
%Uint32Array%	Uint32Array	The Uint32Array constructor (22.2)
%Uint32ArrayPrototype%	Uint32Array.prototype	The initial value of the prototype data property of %Uint32Array%
%URIError%	URIError	The URIError constructor (19.5.5.6)
%URIErrorPrototype%	URIError.prototype	The initial value of the prototype data property of %URIError%

<code>%WeakMap%</code>	<code>WeakMap</code>	The WeakMap constructor (23.3.1)
<code>%WeakMapPrototype%</code>	<code>WeakMap.prototype</code>	The initial value of the prototype data property of <code>%WeakMap%</code>
<code>%WeakSet%</code>	<code>WeakSet</code>	The WeakSet constructor (23.4.1)
<code>%WeakSetPrototype%</code>	<code>WeakSet.prototype</code>	The initial value of the prototype data property of <code>%WeakSet%</code>

6.2 ECMAScript Specification Types

A specification type corresponds to meta-values that are used within algorithms to describe the semantics of ECMAScript language constructs and ECMAScript language types. The specification types include [Reference](#), [List](#), [Completion](#), [Property Descriptor](#), [Lexical Environment](#), [Environment Record](#), and [Data Block](#). Specification type values are specification artefacts that do not necessarily correspond to any specific entity within an ECMAScript implementation. Specification type values may be used to describe intermediate results of ECMAScript expression evaluation but such values cannot be stored as properties of objects or values of ECMAScript language variables.

6.2.1 The List and Record Specification Types

The List type is used to explain the evaluation of argument lists (see [12.3.6](#)) in **new** expressions, in function calls, and in other algorithms where a simple ordered list of values is needed. Values of the List type are simply ordered sequences of list elements containing the individual values. These sequences may be of any length. The elements of a list may be randomly accessed using 0-origin indices. For notational convenience an array-like syntax can be used to access List elements. For example, `arguments[2]` is shorthand for saying the 3rd element of the List `arguments`.

For notational convenience within this specification, a literal syntax can be used to express a new List value. For example, « 1, 2 » defines a List value that has two elements each of which is initialized to a specific value. A new empty List can be expressed as « ».

The Record type is used to describe data aggregations within the algorithms of this specification. A Record type value consists of one or more named fields. The value of each field is either an ECMAScript value or an abstract value represented by a name associated with the Record type. Field names are always enclosed in double brackets, for example `[[Value]]`.

For notational convenience within this specification, an object literal-like syntax can be used to express a Record value. For example, `{ [[Field1]]: 42, [[Field2]]: false, [[Field3]]: empty }` defines a Record value that has three fields, each of which is initialized to a specific value. Field name order is not significant. Any fields that are not explicitly listed are considered to be absent.

In specification text and algorithms, dot notation may be used to refer to a specific field of a Record value. For example, if R is the record shown in the previous paragraph then `R.[[Field2]]` is shorthand for “the field of R named `[[Field2]]`”.

Schema for commonly used Record field combinations may be named, and that name may be used as a prefix to a literal Record value to identify the specific kind of aggregations that is being described. For example: `PropertyDescriptor { [[Value]]: 42, [[Writable]]: false, [[Configurable]]: true }`.

6.2.2 The Set and Relation Specification Types

The Set type is used to explain a collection of unordered elements for use in the [memory model](#). Values of the Set type are simple collections of elements, where no element appears more than once. Elements may be added to and removed from Sets. Sets may be unioned, intersected, or subtracted from each other.

The Relation type is used to explain constraints on Sets. Values of the Relation type are Sets of ordered pairs of values from its value domain. For example, a Relation on events is a set of ordered pairs of events. For a Relation R and two values a and b in the value domain of R , $a R b$ is shorthand for saying the ordered pair (a, b) is a member of R . A Relation is least with respect to some conditions when it is the smallest Relation that satisfies those conditions.

A strict partial order is a Relation value R that satisfies the following.

For all a , b , and c in R 's domain:

It is not the case that $a R a$, and
If $a R b$ and $b R c$, then $a R c$.

NOTE 1

The two properties above are called, in order, irreflexivity and transitivity.

A strict total order is a Relation value R that satisfies the following.

For all a , b , and c in R 's domain:

a is identical to b or $a R b$ or $b R a$, and
It is not the case that $a R a$, and
If $a R b$ and $b R c$, then $a R c$.

NOTE 2

The three properties above are called, in order, totality, irreflexivity, and transitivity.

6.2.3 The Completion Record Specification Type

The Completion type is a [Record](#) used to explain the runtime propagation of values and control flow such as the behaviour of statements (**break**, **continue**, **return** and **throw**) that perform nonlocal transfers of control.

Values of the Completion type are [Record](#) values whose fields are defined as by [Table 8](#). Such values are referred to as Completion Records.

Table 8: Completion Record Fields

Field Name	Value	Meaning
[[Type]]	One of normal, break, continue, return, or throw	The type of completion that occurred.
[[Value]]	any ECMAScript language value or empty	The value that was produced.
[[Target]]	any ECMAScript string or empty	The target label for directed control transfers.

The term “abrupt completion” refers to any completion with a [[Type]] value other than normal.

6.2.3.1 Await

Algorithm steps that say

1. Let *completion* be *Await(value)*.

mean the same thing as:

1. Let *asyncContext* be the *running execution context*.
2. Let *promise* be ? *PromiseResolve(%Promise%, « value »)*.
3. Let *stepsFulfilled* be the algorithm steps defined in *Await Fulfilled Functions*.
4. Let *onFulfilled* be *CreateBuiltinFunction(stepsFulfilled, « [[AsyncContext]] »)*.
5. Set *onFulfilled*.[[AsyncContext]] to *asyncContext*.
6. Let *stepsRejected* be the algorithm steps defined in *Await Rejected Functions*.
7. Let *onRejected* be *CreateBuiltinFunction(stepsRejected, « [[AsyncContext]] »)*.
8. Set *onRejected*.[[AsyncContext]] to *asyncContext*.
9. Perform ! *PerformPromiseThen(promise, onFulfilled, onRejected)*.
10. Remove *asyncContext* from the *execution context stack* and restore the *execution context* that is at the top of the *execution context stack* as the *running execution context*.
11. Set the code evaluation state of *asyncContext* such that when evaluation is resumed with a *Completion completion*, the following steps of the algorithm that invoked *Await* will be performed, with *completion* available.
12. Return.
13. NOTE: This returns to the evaluation of the operation that had most previously resumed evaluation of *asyncContext*.

where all variables in the above steps, with the exception of *completion*, are ephemeral and visible only in the steps pertaining to *Await*.

NOTE

Await can be combined with the ? and ! prefixes, so that for example

1. Let *result* be ? *Await(value)*.

means the same thing as:

1. Let *result* be *Await(value)*.
2. *ReturnIfAbrupt(result)*.

6.2.3.1.1 Await Fulfilled Functions

An *Await* fulfilled function is an anonymous built-in function that is used as part of the *Await* specification device to deliver the promise fulfillment value to the caller as a normal completion. Each *Await* fulfilled function has an [[AsyncContext]] internal slot.

When an *Await* fulfilled function is called with argument *value*, the following steps are taken:

1. Let *F* be the *active function object*.
2. Let *asyncContext* be *F*.[[AsyncContext]].
3. Let *prevContext* be the *running execution context*.
4. *Suspend prevContext*.
5. Push *asyncContext* onto the *execution context stack*; *asyncContext* is now the *running execution context*.

- Resume the suspended evaluation of *asyncContext* using `NormalCompletion(value)` as the result of the operation that suspended it.
- Assert:** When we reach this step, *asyncContext* has already been removed from the *execution context stack* and *prevContext* is the currently *running execution context*.
- Return **undefined**.

The "**length**" property of an *Await* fulfilled function is 1.

6.2.3.1.2 Await Rejected Functions

An *Await* rejected function is an anonymous built-in function that is used as part of the *Await* specification device to deliver the promise rejection reason to the caller as an abrupt throw completion. Each *Await* rejected function has an `[[AsyncContext]]` internal slot.

When an *Await* rejected function is called with argument *reason*, the following steps are taken:

- Let *F* be the *active function object*.
- Let *asyncContext* be *F*.`[[AsyncContext]]`.
- Let *prevContext* be the *running execution context*.
- Suspend prevContext*.
- Push *asyncContext* onto the *execution context stack*; *asyncContext* is now the *running execution context*.
- Resume the suspended evaluation of *asyncContext* using `ThrowCompletion(reason)` as the result of the operation that suspended it.
- Assert:** When we reach this step, *asyncContext* has already been removed from the *execution context stack* and *prevContext* is the currently *running execution context*.
- Return **undefined**.

The "**length**" property of an *Await* rejected function is 1.

6.2.3.2 NormalCompletion

The abstract operation `NormalCompletion` with a single *argument*, such as:

- Return `NormalCompletion(argument)`.

Is a shorthand that is defined as follows:

- Return `Completion` { `[[Type]]`: normal, `[[Value]]`: *argument*, `[[Target]]`: empty }.

6.2.3.3 ThrowCompletion

The abstract operation `ThrowCompletion` with a single *argument*, such as:

- Return `ThrowCompletion(argument)`.

Is a shorthand that is defined as follows:

- Return `Completion` { `[[Type]]`: throw, `[[Value]]`: *argument*, `[[Target]]`: empty }.

6.2.3.4 UpdateEmpty (*completionRecord*, *value*)

The abstract operation `UpdateEmpty` with arguments *completionRecord* and *value* performs the following steps:

1. **Assert:** If *completionRecord*.[[Type]] is either **return** or **throw**, then *completionRecord*.[[Value]] is not **empty**.
2. If *completionRecord*.[[Value]] is not **empty**, return **Completion**(*completionRecord*).
3. Return **Completion** { [[Type]]: *completionRecord*.[[Type]], [[Value]]: *value*, [[Target]]: *completionRecord*.[[Target]] }.

6.2.4 The Reference Specification Type

NOTE

The Reference type is used to explain the behaviour of such operators as **delete**, **typeof**, the assignment operators, the **super** keyword and other language features. For example, the left-hand operand of an assignment is expected to produce a reference.

A Reference is a resolved name or property binding. A Reference consists of three components, the base value component, the referenced name component, and the Boolean-valued strict reference flag. The base value component is either **undefined**, an Object, a Boolean, a String, a Symbol, a Number, or an **Environment Record**. A base value component of **undefined** indicates that the Reference could not be resolved to a binding. The referenced name component is a String or Symbol value.

A Super Reference is a Reference that is used to represent a name binding that was expressed using the **super** keyword. A **Super Reference** has an additional *thisValue* component, and its base value component will never be an **Environment Record**.

The following **abstract operations** are used in this specification to operate on references:

6.2.4.1 GetBase (*V*)

1. **Assert:** **Type**(*V*) is **Reference**.
2. Return the base value component of *V*.

6.2.4.2 GetReferencedName (*V*)

1. **Assert:** **Type**(*V*) is **Reference**.
2. Return the referenced name component of *V*.

6.2.4.3 IsStrictReference (*V*)

1. **Assert:** **Type**(*V*) is **Reference**.
2. Return the strict reference flag of *V*.

6.2.4.4 HasPrimitiveBase (*V*)

1. **Assert:** **Type**(*V*) is **Reference**.
2. If **Type**(*V*'s base value component) is Boolean, String, Symbol, or Number, return **true**; otherwise return **false**.

6.2.4.5 IsPropertyReference (*V*)

1. **Assert:** **Type**(*V*) is **Reference**.
2. If either the base value component of *V* is an Object or **HasPrimitiveBase**(*V*) is **true**, return **true**; otherwise return **false**.

6.2.4.6 IsUnresolvableReference (V)

1. **Assert**: `Type(V)` is `Reference`.
2. If the base value component of `V` is **undefined**, return **true**; otherwise return **false**.

6.2.4.7 IsSuperReference (V)

1. **Assert**: `Type(V)` is `Reference`.
2. If `V` has a `thisValue` component, return **true**; otherwise return **false**.

6.2.4.8 GetValue (V)

1. **ReturnIfAbrupt**(`V`).
2. If `Type(V)` is not `Reference`, return `V`.
3. Let `base` be `GetBase(V)`.
4. If `IsUnresolvableReference(V)` is **true**, throw a **ReferenceError** exception.
5. If `IsPropertyReference(V)` is **true**, then
 - a. If `HasPrimitiveBase(V)` is **true**, then
 - i. **Assert**: In this case, `base` will never be **undefined** or **null**.
 - ii. Set `base` to ! `ToObject(base)`.
 - b. Return ? `base.[[Get]](GetReferencedName(V), GetThisValue(V))`.
6. Else `base` must be an `Environment Record`,
 - a. Return ? `base.GetBindingValue(GetReferencedName(V), IsStrictReference(V))` (see 8.1.1).

NOTE

The object that may be created in step 5.a.ii is not accessible outside of the above abstract operation and the ordinary object `[[Get]]` internal method. An implementation might choose to avoid the actual creation of the object.

6.2.4.9 PutValue (V, W)

1. **ReturnIfAbrupt**(`V`).
2. **ReturnIfAbrupt**(`W`).
3. If `Type(V)` is not `Reference`, throw a **ReferenceError** exception.
4. Let `base` be `GetBase(V)`.
5. If `IsUnresolvableReference(V)` is **true**, then
 - a. If `IsStrictReference(V)` is **true**, then
 - i. Throw a **ReferenceError** exception.
 - b. Let `globalObj` be `GetGlobalObject()`.
 - c. Return ? `Set(globalObj, GetReferencedName(V), W, false)`.
6. Else if `IsPropertyReference(V)` is **true**, then
 - a. If `HasPrimitiveBase(V)` is **true**, then
 - i. **Assert**: In this case, `base` will never be **undefined** or **null**.
 - ii. Set `base` to ! `ToObject(base)`.
 - b. Let `succeeded` be ? `base.[[Set]](GetReferencedName(V), W, GetThisValue(V))`.
 - c. If `succeeded` is **false** and `IsStrictReference(V)` is **true**, throw a **TypeError** exception.
 - d. Return.
7. Else `base` must be an `Environment Record`,
 - a. Return ? `base.SetMutableBinding(GetReferencedName(V), W, IsStrictReference(V))` (see 8.1.1).

NOTE

The object that may be created in step 6.a.ii is not accessible outside of the above algorithm and the ordinary object `[[Set]]` internal method. An implementation might choose to avoid the actual creation of that object.

6.2.4.10 **GetThisValue (*V*)**

1. **Assert:** `IsPropertyReference(V)` is **true**.
2. If `IsSuperReference(V)` is **true**, then
 - a. Return the value of the `thisValue` component of the reference *V*.
3. Return `GetBase(V)`.

6.2.4.11 **InitializeReferencedBinding (*V*, *W*)**

1. `ReturnIfAbrupt(V)`.
2. `ReturnIfAbrupt(W)`.
3. **Assert:** `Type(V)` is `Reference`.
4. **Assert:** `IsUnresolvableReference(V)` is **false**.
5. Let *base* be `GetBase(V)`.
6. **Assert:** *base* is an `Environment Record`.
7. Return *base*.`InitializeBinding(GetReferencedName(V), W)`.

6.2.5 The Property Descriptor Specification Type

The Property Descriptor type is used to explain the manipulation and reification of Object property attributes. Values of the Property Descriptor type are Records. Each field's name is an attribute name and its value is a corresponding attribute value as specified in 6.1.7.1. In addition, any field may be present or absent. The schema name used within this specification to tag literal descriptions of Property Descriptor records is “PropertyDescriptor”.

Property Descriptor values may be further classified as data Property Descriptors and accessor Property Descriptors based upon the existence or use of certain fields. A data Property Descriptor is one that includes any fields named either `[[Value]]` or `[[Writable]]`. An accessor Property Descriptor is one that includes any fields named either `[[Get]]` or `[[Set]]`. Any Property Descriptor may have fields named `[[Enumerable]]` and `[[Configurable]]`. A Property Descriptor value may not be both a data Property Descriptor and an accessor Property Descriptor; however, it may be neither. A generic Property Descriptor is a Property Descriptor value that is neither a data Property Descriptor nor an accessor Property Descriptor. A fully populated Property Descriptor is one that is either an accessor Property Descriptor or a data Property Descriptor and that has all of the fields that correspond to the property attributes defined in either Table 2 or Table 3.

The following [abstract operations](#) are used in this specification to operate upon Property Descriptor values:

6.2.5.1 **IsAccessorDescriptor (*Desc*)**

When the abstract operation `IsAccessorDescriptor` is called with [Property Descriptor](#) *Desc*, the following steps are taken:

1. If *Desc* is **undefined**, return **false**.
2. If both *Desc*.`[[Get]]` and *Desc*.`[[Set]]` are absent, return **false**.
3. Return **true**.

6.2.5.2 **IsDataDescriptor (*Desc*)**

When the abstract operation `IsDataDescriptor` is called with *Property Descriptor Desc*, the following steps are taken:

1. If *Desc* is **undefined**, return **false**.
2. If both *Desc*.[[Value]] and *Desc*.[[Writable]] are absent, return **false**.
3. Return **true**.

6.2.5.3 `IsGenericDescriptor` (*Desc*)

When the abstract operation `IsGenericDescriptor` is called with *Property Descriptor Desc*, the following steps are taken:

1. If *Desc* is **undefined**, return **false**.
2. If `IsAccessorDescriptor(Desc)` and `IsDataDescriptor(Desc)` are both **false**, return **true**.
3. Return **false**.

6.2.5.4 `FromPropertyDescriptor` (*Desc*)

When the abstract operation `FromPropertyDescriptor` is called with *Property Descriptor Desc*, the following steps are taken:

1. If *Desc* is **undefined**, return **undefined**.
2. Let *obj* be `ObjectCreate(%ObjectPrototype%)`.
3. **Assert**: *obj* is an extensible ordinary object with no own properties.
4. If *Desc* has a [[Value]] field, then
 - a. Perform `CreateDataProperty(obj, "value", Desc.[[Value]])`.
5. If *Desc* has a [[Writable]] field, then
 - a. Perform `CreateDataProperty(obj, "writable", Desc.[[Writable]])`.
6. If *Desc* has a [[Get]] field, then
 - a. Perform `CreateDataProperty(obj, "get", Desc.[[Get]])`.
7. If *Desc* has a [[Set]] field, then
 - a. Perform `CreateDataProperty(obj, "set", Desc.[[Set]])`.
8. If *Desc* has an [[Enumerable]] field, then
 - a. Perform `CreateDataProperty(obj, "enumerable", Desc.[[Enumerable]])`.
9. If *Desc* has a [[Configurable]] field, then
 - a. Perform `CreateDataProperty(obj, "configurable", Desc.[[Configurable]])`.
10. **Assert**: All of the above `CreateDataProperty` operations return **true**.
11. Return *obj*.

6.2.5.5 `ToPropertyDescriptor` (*Obj*)

When the abstract operation `ToPropertyDescriptor` is called with object *Obj*, the following steps are taken:

1. If `Type(Obj)` is not **Object**, throw a **TypeError** exception.
2. Let *desc* be a new *Property Descriptor* that initially has no fields.
3. Let *hasEnumerable* be `? HasProperty(Obj, "enumerable")`.
4. If *hasEnumerable* is **true**, then
 - a. Let *enumerable* be `ToBoolean(? Get(Obj, "enumerable"))`.
 - b. Set *desc*.[[Enumerable]] to *enumerable*.
5. Let *hasConfigurable* be `? HasProperty(Obj, "configurable")`.
6. If *hasConfigurable* is **true**, then
 - a. Let *configurable* be `ToBoolean(? Get(Obj, "configurable"))`.

- b. Set *desc*.[[Configurable]] to *configurable*.
7. Let *hasValue* be ? *HasProperty*(*Obj*, "value").
8. If *hasValue* is **true**, then
 - a. Let *value* be ? *Get*(*Obj*, "value").
 - b. Set *desc*.[[Value]] to *value*.
9. Let *hasWritable* be ? *HasProperty*(*Obj*, "writable").
10. If *hasWritable* is **true**, then
 - a. Let *writable* be *ToBoolean*(? *Get*(*Obj*, "writable")).
 - b. Set *desc*.[[Writable]] to *writable*.
11. Let *hasGet* be ? *HasProperty*(*Obj*, "get").
12. If *hasGet* is **true**, then
 - a. Let *getter* be ? *Get*(*Obj*, "get").
 - b. If *IsCallable*(*getter*) is **false** and *getter* is not **undefined**, throw a **TypeError** exception.
 - c. Set *desc*.[[Get]] to *getter*.
13. Let *hasSet* be ? *HasProperty*(*Obj*, "set").
14. If *hasSet* is **true**, then
 - a. Let *setter* be ? *Get*(*Obj*, "set").
 - b. If *IsCallable*(*setter*) is **false** and *setter* is not **undefined**, throw a **TypeError** exception.
 - c. Set *desc*.[[Set]] to *setter*.
15. If *desc*.[[Get]] is present or *desc*.[[Set]] is present, then
 - a. If *desc*.[[Value]] is present or *desc*.[[Writable]] is present, throw a **TypeError** exception.
16. Return *desc*.

6.2.5.6 CompletePropertyDescriptor (*Desc*)

When the abstract operation CompletePropertyDescriptor is called with *Property Descriptor Desc*, the following steps are taken:

1. **Assert**: *Desc* is a *Property Descriptor*.
2. Let *like* be *Record* { [[Value]]: **undefined**, [[Writable]]: **false**, [[Get]]: **undefined**, [[Set]]: **undefined**, [[Enumerable]]: **false**, [[Configurable]]: **false** }.
3. If *IsGenericDescriptor*(*Desc*) is **true** or *IsDataDescriptor*(*Desc*) is **true**, then
 - a. If *Desc* does not have a [[Value]] field, set *Desc*.[[Value]] to *like*.[[Value]].
 - b. If *Desc* does not have a [[Writable]] field, set *Desc*.[[Writable]] to *like*.[[Writable]].
4. Else,
 - a. If *Desc* does not have a [[Get]] field, set *Desc*.[[Get]] to *like*.[[Get]].
 - b. If *Desc* does not have a [[Set]] field, set *Desc*.[[Set]] to *like*.[[Set]].
5. If *Desc* does not have an [[Enumerable]] field, set *Desc*.[[Enumerable]] to *like*.[[Enumerable]].
6. If *Desc* does not have a [[Configurable]] field, set *Desc*.[[Configurable]] to *like*.[[Configurable]].
7. Return *Desc*.

6.2.6 The Lexical Environment and Environment Record Specification Types

The *Lexical Environment* and *Environment Record* types are used to explain the behaviour of name resolution in nested functions and blocks. These types and the operations upon them are defined in 8.1.

6.2.7 Data Blocks

The Data Block specification type is used to describe a distinct and mutable sequence of byte-sized (8 bit) numeric values. A Data Block value is created with a fixed number of bytes that each have the initial value 0.

For notational convenience within this specification, an array-like syntax can be used to access the individual bytes of a Data Block value. This notation presents a Data Block value as a 0-origin integer-indexed sequence of bytes. For example, if *db* is a 5 byte Data Block value then *db*[2] can be used to access its 3rd byte.

A data block that resides in memory that can be referenced from multiple agents concurrently is designated a Shared Data Block. A Shared Data Block has an identity (for the purposes of equality testing Shared Data Block values) that is *address-free*: it is tied not to the virtual addresses the block is mapped to in any process, but to the set of locations in memory that the block represents. Two data blocks are equal only if the sets of the locations they contain are equal; otherwise, they are not equal and the intersection of the sets of locations they contain is empty. Finally, Shared Data Blocks can be distinguished from Data Blocks.

The semantics of Shared Data Blocks is defined using Shared Data Block events by the [memory model](#). [Abstract operations](#) below introduce Shared Data Block events and act as the interface between evaluation semantics and the event semantics of the [memory model](#). The events form a [candidate execution](#), on which the [memory model](#) acts as a filter. Please consult the [memory model](#) for full semantics.

Shared Data Block events are modeled by Records, defined in the [memory model](#).

The following [abstract operations](#) are used in this specification to operate upon Data Block values:

6.2.7.1 CreateByteDataBlock (*size*)

When the abstract operation CreateByteDataBlock is called with integer argument *size*, the following steps are taken:

1. [Assert](#): *size* ≥ 0.
2. Let *db* be a new [Data Block](#) value consisting of *size* bytes. If it is impossible to create such a [Data Block](#), throw a **RangeError** exception.
3. Set all of the bytes of *db* to 0.
4. Return *db*.

6.2.7.2 CreateSharedByteDataBlock (*size*)

When the abstract operation CreateSharedByteDataBlock is called with integer argument *size*, the following steps are taken:

1. [Assert](#): *size* ≥ 0.
2. Let *db* be a new [Shared Data Block](#) value consisting of *size* bytes. If it is impossible to create such a [Shared Data Block](#), throw a **RangeError** exception.
3. Let *execution* be the [[CandidateExecution]] field of the [surrounding agent's Agent Record](#).
4. Let *eventList* be the [[EventList]] field of the element in *execution*.[[EventsRecords]] whose [[AgentSignifier]] is [AgentSignifier](#)() .
5. Let *zero* be « 0 ».
6. For each index *i* of *db*, do
 - a. Append [WriteSharedMemory](#) { [[Order]]: "Init", [[NoTear]]: true, [[Block]]: *db*, [[ByteIndex]]: *i*, [[ElementSize]]: 1, [[Payload]]: *zero* } to *eventList*.
7. Return *db*.

6.2.7.3 CopyDataBlockBytes (*toBlock*, *toIndex*, *fromBlock*, *fromIndex*, *count*)

When the abstract operation CopyDataBlockBytes is called, the following steps are taken:

1. **Assert:** *fromBlock* and *toBlock* are distinct **Data Block** or **Shared Data Block** values.
2. **Assert:** *fromIndex*, *toIndex*, and *count* are integer values ≥ 0 .
3. Let *fromSize* be the number of bytes in *fromBlock*.
4. **Assert:** $\text{fromIndex} + \text{count} \leq \text{fromSize}$.
5. Let *toSize* be the number of bytes in *toBlock*.
6. **Assert:** $\text{toIndex} + \text{count} \leq \text{toSize}$.
7. Repeat, while *count* > 0
 - a. If *fromBlock* is a **Shared Data Block**, then
 - i. Let *execution* be the `[[CandidateExecution]]` field of the **surrounding agent's Agent Record**.
 - ii. Let *eventList* be the `[[EventList]]` field of the element in *execution*.`[[EventsRecords]]` whose `[[AgentSignifier]]` is **AgentSignifier()**.
 - iii. Let *bytes* be a **List** of length 1 that contains a nondeterministically chosen byte value.
 - iv. NOTE: In implementations, *bytes* is the result of a non-atomic read instruction on the underlying hardware. The nondeterminism is a semantic prescription of the **memory model** to describe observable behaviour of hardware with weak consistency.
 - v. Let *readEvent* be **ReadSharedMemory** { `[[Order]]`: **"Unordered"**, `[[NoTear]]`: **true**, `[[Block]]`: *fromBlock*, `[[ByteIndex]]`: *fromIndex*, `[[ElementSize]]`: 1 }.
 - vi. Append *readEvent* to *eventList*.
 - vii. Append **Chosen Value Record** { `[[Event]]`: *readEvent*, `[[ChosenValue]]`: *bytes* } to *execution*.`[[ChosenValues]]`.
 - viii. If *toBlock* is a **Shared Data Block**, then
 1. Append **WriteSharedMemory** { `[[Order]]`: **"Unordered"**, `[[NoTear]]`: **true**, `[[Block]]`: *toBlock*, `[[ByteIndex]]`: *toIndex*, `[[ElementSize]]`: 1, `[[Payload]]`: *bytes* } to *eventList*.
 - ix. Else,
 1. Set *toBlock*[*toIndex*] to *bytes*[0].
 - b. Else,
 - i. **Assert:** *toBlock* is not a **Shared Data Block**.
 - ii. Set *toBlock*[*toIndex*] to *fromBlock*[*fromIndex*].
 - c. Increment *toIndex* and *fromIndex* each by 1.
 - d. Decrement *count* by 1.
8. Return **NormalCompletion**(empty).

7 Abstract Operations

These operations are not a part of the ECMAScript language; they are defined here to solely to aid the specification of the semantics of the ECMAScript language. Other, more specialized **abstract operations** are defined throughout this specification.

7.1 Type Conversion

The ECMAScript language implicitly performs automatic type conversion as needed. To clarify the semantics of certain constructs it is useful to define a set of conversion **abstract operations**. The conversion **abstract operations** are polymorphic; they can accept a value of any **ECMAScript language type**. But no other specification types are used with

these operations.

7.1.1 ToPrimitive (*input* [, *PreferredType*])

The abstract operation ToPrimitive takes an *input* argument and an optional argument *PreferredType*. The abstract operation ToPrimitive converts its *input* argument to a non-Object type. If an object is capable of converting to more than one primitive type, it may use the optional hint *PreferredType* to favour that type. Conversion occurs according to the following algorithm:

1. **Assert:** *input* is an ECMAScript language value.
2. If *Type*(*input*) is Object, then
 - a. If *PreferredType* is not present, let *hint* be "default".
 - b. Else if *PreferredType* is hint String, let *hint* be "string".
 - c. Else *PreferredType* is hint Number, let *hint* be "number".
 - d. Let *exoticToPrim* be ? *GetMethod*(*input*, @@toPrimitive).
 - e. If *exoticToPrim* is not **undefined**, then
 - i. Let *result* be ? *Call*(*exoticToPrim*, *input*, « *hint* »).
 - ii. If *Type*(*result*) is not Object, return *result*.
 - iii. Throw a **TypeError** exception.
 - f. If *hint* is "default", set *hint* to "number".
 - g. Return ? *OrdinaryToPrimitive*(*input*, *hint*).
3. Return *input*.

NOTE

When ToPrimitive is called with no hint, then it generally behaves as if the hint were Number. However, objects may over-ride this behaviour by defining a @@toPrimitive method. Of the objects defined in this specification only Date objects (see 20.3.4.45) and Symbol objects (see 19.4.3.5) over-ride the default ToPrimitive behaviour. Date objects treat no hint as if the hint were String.

7.1.1.1 OrdinaryToPrimitive (*O*, *hint*)

When the abstract operation OrdinaryToPrimitive is called with arguments *O* and *hint*, the following steps are taken:

1. **Assert:** *Type*(*O*) is Object.
2. **Assert:** *Type*(*hint*) is String and its value is either "string" or "number".
3. If *hint* is "string", then
 - a. Let *methodNames* be « "toString", "valueOf" ».
4. Else,
 - a. Let *methodNames* be « "valueOf", "toString" ».
5. For each *name* in *methodNames* in List order, do
 - a. Let *method* be ? *Get*(*O*, *name*).
 - b. If *IsCallable*(*method*) is **true**, then
 - i. Let *result* be ? *Call*(*method*, *O*).
 - ii. If *Type*(*result*) is not Object, return *result*.
6. Throw a **TypeError** exception.

7.1.2 ToBoolean (*argument*)

The abstract operation ToBoolean converts *argument* to a value of type Boolean according to Table 9:

Table 9: ToBoolean Conversions

Argument Type	Result
Undefined	Return false .
Null	Return false .
Boolean	Return <i>argument</i> .
Number	If <i>argument</i> is +0 , -0 , or NaN , return false ; otherwise return true .
String	If <i>argument</i> is the empty String (its length is zero), return false ; otherwise return true .
Symbol	Return true .
Object	Return true .

7.1.3 ToNumber (*argument*)

The abstract operation ToNumber converts *argument* to a value of type Number according to Table 10:

Table 10: ToNumber Conversions

Argument Type	Result
Undefined	Return NaN .
Null	Return +0 .
Boolean	If <i>argument</i> is true , return 1. If <i>argument</i> is false , return +0 .
Number	Return <i>argument</i> (no conversion).
String	See grammar and conversion algorithm below.
Symbol	Throw a TypeError exception.
Object	Apply the following steps: 1. Let <i>primValue</i> be ? ToPrimitive(<i>argument</i>, hint Number). 2. Return ? ToNumber(<i>primValue</i>).

7.1.3.1 ToNumber Applied to the String Type

ToNumber applied to Strings applies the following grammar to the input String interpreted as a sequence of UTF-16 encoded code points (6.1.4). If the grammar cannot interpret the String as an expansion of *StringNumericLiteral*, then the result of ToNumber is **NaN**.

NOTE 1

The terminal symbols of this grammar are all composed of characters in the Unicode Basic Multilingual Plane (BMP). Therefore, the result of ToNumber will be **NaN** if the string contains any *leading surrogate* or *trailing surrogate* code

units, whether paired or unpaired.

Syntax

```
StringNumericLiteral :::  
    StrWhiteSpaceopt  
    StrWhiteSpaceopt StrNumericLiteral StrWhiteSpaceopt  
  
StrWhiteSpace :::  
    StrWhiteSpaceChar StrWhiteSpaceopt  
  
StrWhiteSpaceChar :::  
    WhiteSpace  
    LineTerminator  
  
StrNumericLiteral :::  
    StrDecimalLiteral  
    BinaryIntegerLiteral  
    OctalIntegerLiteral  
    HexIntegerLiteral  
  
StrDecimalLiteral :::  
    StrUnsignedDecimalLiteral  
    + StrUnsignedDecimalLiteral  
    - StrUnsignedDecimalLiteral  
  
StrUnsignedDecimalLiteral :::  
    Infinity  
    DecimalDigits . DecimalDigitsopt ExponentPartopt  
    . DecimalDigits ExponentPartopt  
    DecimalDigits ExponentPartopt
```

All grammar symbols not explicitly defined above have the definitions used in the Lexical Grammar for numeric literals (11.8.3)

NOTE 2

Some differences should be noted between the syntax of a *StringNumericLiteral* and a *NumericLiteral*:

- A *StringNumericLiteral* may include leading and/or trailing white space and/or line terminators.
- A *StringNumericLiteral* that is decimal may have any number of leading 0 digits.
- A *StringNumericLiteral* that is decimal may include a + or - to indicate its sign.
- A *StringNumericLiteral* that is empty or contains only white space is converted to +0.
- Infinity** and **-Infinity** are recognized as a *StringNumericLiteral* but not as a *NumericLiteral*.

7.1.3.1.1 Runtime Semantics: MV

The conversion of a String to a Number value is similar overall to the determination of the Number value for a numeric literal (see 11.8.3), but some of the details are different, so the process for converting a String numeric literal to a value

of Number type is given here. This value is determined in two steps: first, a mathematical value (MV) is derived from the String numeric literal; second, this mathematical value is rounded as described below. The MV on any grammar symbol, not provided below, is the MV for that symbol defined in 11.8.3.1.

The MV of *StringNumericLiteral* ::: [empty] is 0.

The MV of *StringNumericLiteral* ::: *StrWhiteSpace* is 0.

The MV of *StringNumericLiteral* ::: *StrWhiteSpace StrNumericLiteral StrWhiteSpace* is the MV of *StrNumericLiteral*, no matter whether white space is present or not.

The MV of *StrNumericLiteral* ::: *StrDecimalLiteral* is the MV of *StrDecimalLiteral*.

The MV of *StrNumericLiteral* ::: *BinaryIntegerLiteral* is the MV of *BinaryIntegerLiteral*.

The MV of *StrNumericLiteral* ::: *OctalIntegerLiteral* is the MV of *OctalIntegerLiteral*.

The MV of *StrNumericLiteral* ::: *HexIntegerLiteral* is the MV of *HexIntegerLiteral*.

The MV of *StrDecimalLiteral* ::: *StrUnsignedDecimalLiteral* is the MV of *StrUnsignedDecimalLiteral*.

The MV of *StrDecimalLiteral* ::: + *StrUnsignedDecimalLiteral* is the MV of *StrUnsignedDecimalLiteral*.

The MV of *StrDecimalLiteral* ::: - *StrUnsignedDecimalLiteral* is the negative of the MV of *StrUnsignedDecimalLiteral*. (Note that if the MV of *StrUnsignedDecimalLiteral* is 0, the negative of this MV is also 0. The rounding rule described below handles the conversion of this signless mathematical zero to a floating-point +0 or -0 as appropriate.)

The MV of *StrUnsignedDecimalLiteral* ::: **Infinity** is 10^{10000} (a value so large that it will round to $+\infty$).

The MV of *StrUnsignedDecimalLiteral* ::: *DecimalDigits* . is the MV of *DecimalDigits*.

The MV of *StrUnsignedDecimalLiteral* ::: *DecimalDigits* . *DecimalDigits* is the MV of the first *DecimalDigits* plus (the MV of the second *DecimalDigits* times 10^{-n}), where n is the number of code points in the second *DecimalDigits*.

The MV of *StrUnsignedDecimalLiteral* ::: *DecimalDigits* . *ExponentPart* is the MV of *DecimalDigits* times 10^e , where e is the MV of *ExponentPart*.

The MV of *StrUnsignedDecimalLiteral* ::: *DecimalDigits* . *DecimalDigits* *ExponentPart* is (the MV of the first *DecimalDigits* plus (the MV of the second *DecimalDigits* times 10^{-n})) times 10^e , where n is the number of code points in the second *DecimalDigits* and e is the MV of *ExponentPart*.

The MV of *StrUnsignedDecimalLiteral* ::: . *DecimalDigits* is the MV of *DecimalDigits* times 10^{-n} , where n is the number of code points in *DecimalDigits*.

The MV of *StrUnsignedDecimalLiteral* ::: . *DecimalDigits* *ExponentPart* is the MV of *DecimalDigits* times 10^{e-n} , where n is the number of code points in *DecimalDigits* and e is the MV of *ExponentPart*.

The MV of *StrUnsignedDecimalLiteral* ::: *DecimalDigits* is the MV of *DecimalDigits*.

The MV of *StrUnsignedDecimalLiteral* ::: *DecimalDigits* *ExponentPart* is the MV of *DecimalDigits* times 10^e , where e is the MV of *ExponentPart*.

Once the exact MV for a String numeric literal has been determined, it is then rounded to a value of the Number type. If the MV is 0, then the rounded value is +0 unless the first non white space code point in the String numeric literal is "-", in which case the rounded value is -0. Otherwise, the rounded value must be the Number value for the MV (in the sense defined in 6.1.6), unless the literal includes a *StrUnsignedDecimalLiteral* and the literal has more than 20 significant digits, in which case the Number value may be either the Number value for the MV of a literal produced by replacing each significant digit after the 20th with a 0 digit or the Number value for the MV of a literal produced by replacing each significant digit after the 20th with a 0 digit and then incrementing the literal at the 20th digit position. A digit is significant if it is not part of an *ExponentPart* and

it is not 0; or

there is a nonzero digit to its left and there is a nonzero digit, not in the *ExponentPart*, to its right.

7.1.4 ToInteger (*argument*)

The abstract operation ToInteger converts *argument* to an integral numeric value. This abstract operation functions as follows:

1. Let *number* be ? ToNumber(*argument*).
2. If *number* is NaN, return +0.
3. If *number* is +0, -0, +∞, or -∞, return *number*.
4. Return the number value that is the same sign as *number* and whose magnitude is floor(abs(*number*)).

7.1.5 ToInt32 (*argument*)

The abstract operation ToInt32 converts *argument* to one of 2^{32} integer values in the range -2^{31} through $2^{31} - 1$, inclusive. This abstract operation functions as follows:

1. Let *number* be ? ToNumber(*argument*).
2. If *number* is NaN, +0, -0, +∞, or -∞, return +0.
3. Let *int* be the mathematical value that is the same sign as *number* and whose magnitude is floor(abs(*number*)).
4. Let *int32bit* be *int* modulo 2^{32} .
5. If *int32bit* $\geq 2^{31}$, return *int32bit* - 2^{32} ; otherwise return *int32bit*.

NOTE

Given the above definition of ToInt32:

The ToInt32 abstract operation is idempotent: if applied to a result that it produced, the second application leaves that value unchanged.

ToInt32(ToUint32(*x*)) is equal to ToInt32(*x*) for all values of *x*. (It is to preserve this latter property that +∞ and -∞ are mapped to +0.)

ToInt32 maps -0 to +0.

7.1.6 ToUint32 (*argument*)

The abstract operation ToUint32 converts *argument* to one of 2^{32} integer values in the range 0 through $2^{32} - 1$, inclusive. This abstract operation functions as follows:

1. Let *number* be ? ToNumber(*argument*).
2. If *number* is NaN, +0, -0, +∞, or -∞, return +0.
3. Let *int* be the mathematical value that is the same sign as *number* and whose magnitude is floor(abs(*number*)).
4. Let *int32bit* be *int* modulo 2^{32} .
5. Return *int32bit*.

NOTE

Given the above definition of ToUint32:

Step 5 is the only difference between ToUint32 and ToInt32.

The ToUint32 abstract operation is idempotent: if applied to a result that it produced, the second application leaves that value unchanged.

ToUint32(ToInt32(*x*)) is equal to ToUint32(*x*) for all values of *x*. (It is to preserve this latter property that +∞ and -

∞ are mapped to **+0**.)
ToUint32 maps **-0** to **+0**.

7.1.7 ToInt16 (*argument*)

The abstract operation ToInt16 converts *argument* to one of 2^{16} integer values in the range -32768 through 32767, inclusive. This abstract operation functions as follows:

1. Let *number* be ? ToNumber(*argument*).
2. If *number* is NaN, **+0**, **-0**, $+\infty$, or $-\infty$, return **+0**.
3. Let *int* be the mathematical value that is the same sign as *number* and whose magnitude is $\text{floor}(\text{abs}(\text{number}))$.
4. Let *int16bit* be *int* modulo 2^{16} .
5. If $\text{int16bit} \geq 2^{15}$, return $\text{int16bit} - 2^{16}$; otherwise return *int16bit*.

7.1.8 ToUint16 (*argument*)

The abstract operation ToUint16 converts *argument* to one of 2^{16} integer values in the range 0 through $2^{16} - 1$, inclusive. This abstract operation functions as follows:

1. Let *number* be ? ToNumber(*argument*).
2. If *number* is NaN, **+0**, **-0**, $+\infty$, or $-\infty$, return **+0**.
3. Let *int* be the mathematical value that is the same sign as *number* and whose magnitude is $\text{floor}(\text{abs}(\text{number}))$.
4. Let *int16bit* be *int* modulo 2^{16} .
5. Return *int16bit*.

NOTE

Given the above definition of ToUint16:

The substitution of 2^{16} for 2^{32} in step 4 is the only difference between ToUint32 and ToUint16.
ToUint16 maps **-0** to **+0**.

7.1.9 ToInt8 (*argument*)

The abstract operation ToInt8 converts *argument* to one of 2^8 integer values in the range -128 through 127, inclusive. This abstract operation functions as follows:

1. Let *number* be ? ToNumber(*argument*).
2. If *number* is NaN, **+0**, **-0**, $+\infty$, or $-\infty$, return **+0**.
3. Let *int* be the mathematical value that is the same sign as *number* and whose magnitude is $\text{floor}(\text{abs}(\text{number}))$.
4. Let *int8bit* be *int* modulo 2^8 .
5. If $\text{int8bit} \geq 2^7$, return $\text{int8bit} - 2^8$; otherwise return *int8bit*.

7.1.10 ToUint8 (*argument*)

The abstract operation `ToUint8` converts *argument* to one of 2^8 integer values in the range 0 through 255, inclusive. This abstract operation functions as follows:

1. Let *number* be ? `ToNumber(argument)`.
2. If *number* is `NaN`, `+0`, `-0`, `+` ∞ , or `-` ∞ , return `+0`.
3. Let *int* be the mathematical value that is the same sign as *number* and whose magnitude is `floor(abs(number))`.
4. Let *int8bit* be *int* modulo 2^8 .
5. Return *int8bit*.

7.1.11 ToUint8Clamp (*argument*)

The abstract operation `ToUint8Clamp` converts *argument* to one of 2^8 integer values in the range 0 through 255, inclusive. This abstract operation functions as follows:

1. Let *number* be ? `ToNumber(argument)`.
2. If *number* is `NaN`, return `+0`.
3. If *number* ≤ 0 , return `+0`.
4. If *number* ≥ 255 , return 255.
5. Let *f* be `floor(number)`.
6. If *f* + 0.5 < *number*, return *f* + 1.
7. If *number* < *f* + 0.5, return *f*.
8. If *f* is odd, return *f* + 1.
9. Return *f*.

NOTE

Unlike the other ECMAScript integer conversion abstract operation, `ToUint8Clamp` rounds rather than truncates non-integer values and does not convert `+` ∞ to 0. `ToUint8Clamp` does “round half to even” tie-breaking. This differs from `Math.round` which does “round half up” tie-breaking.

7.1.12 ToString (*argument*)

The abstract operation `ToString` converts *argument* to a value of type `String` according to Table 11:

Table 11: `ToString` Conversions

Argument Type	Result
Undefined	Return <code>"undefined"</code> .
Null	Return <code>"null"</code> .
Boolean	If <i>argument</i> is <code>true</code> , return <code>"true"</code> .
	If <i>argument</i> is <code>false</code> , return <code>"false"</code> .
Number	Return <code>NumberToString(argument)</code> .
String	Return <i>argument</i> .

Symbol	Throw a TypeError exception.
Object	Apply the following steps: 1. Let <i>primValue</i> be ? ToPrimitive(<i>argument</i>, hint String). 2. Return ? ToString(<i>primValue</i>).

7.1.12.1 NumberToString (*m*)

The abstract operation NumberToString converts a Number *m* to String format as follows:

1. If *m* is **NaN**, return the String "**NaN**".
2. If *m* is **+0** or **-0**, return the String "**0**".
3. If *m* is less than zero, return the string-concatenation of "-" and ! **NumberToString**(-*m*).
4. If *m* is $+\infty$, return the String "**Infinity**".
5. Otherwise, let *n*, *k*, and *s* be integers such that $k \geq 1$, $10^{k-1} \leq s < 10^k$, the Number value for $s \times 10^{n-k}$ is *m*, and *k* is as small as possible. Note that *k* is the number of digits in the decimal representation of *s*, that *s* is not divisible by 10, and that the least significant digit of *s* is not necessarily uniquely determined by these criteria.
6. If $k \leq n \leq 21$, return the string-concatenation of:
 - the code units of the *k* digits of the decimal representation of *s* (in order, with no leading zeroes)
 - n* - *k* occurrences of the code unit 0x0030 (DIGIT ZERO)
7. If $0 < n \leq 21$, return the string-concatenation of:
 - the code units of the most significant *n* digits of the decimal representation of *s*
 - the code unit 0x002E (FULL STOP)
 - the code units of the remaining *k* - *n* digits of the decimal representation of *s*
8. If $-6 < n \leq 0$, return the string-concatenation of:
 - the code unit 0x0030 (DIGIT ZERO)
 - the code unit 0x002E (FULL STOP)
 - n* occurrences of the code unit 0x0030 (DIGIT ZERO)
 - the code units of the *k* digits of the decimal representation of *s*
9. Otherwise, if *k* = 1, return the string-concatenation of:
 - the code unit of the single digit of *s*
 - the code unit 0x0065 (LATIN SMALL LETTER E)
 - the code unit 0x002B (PLUS SIGN) or the code unit 0x002D (HYPHEN-MINUS) according to whether *n* - 1 is positive or negative
 - the code units of the decimal representation of the integer **abs**(*n* - 1) (with no leading zeroes)
10. Return the string-concatenation of:
 - the code units of the most significant digit of the decimal representation of *s*
 - the code unit 0x002E (FULL STOP)
 - the code units of the remaining *k* - 1 digits of the decimal representation of *s*
 - the code unit 0x0065 (LATIN SMALL LETTER E)
 - the code unit 0x002B (PLUS SIGN) or the code unit 0x002D (HYPHEN-MINUS) according to whether *n* - 1 is positive or negative
 - the code units of the decimal representation of the integer **abs**(*n* - 1) (with no leading zeroes)

NOTE 1

The following observations may be useful as guidelines for implementations, but are not part of the normative

requirements of this Standard:

If *x* is any Number value other than **-0**, then **ToNumber(ToString(*x*))** is exactly the same Number value as *x*.
The least significant digit of *s* is not always uniquely determined by the requirements listed in step 5.

NOTE 2

For implementations that provide more accurate conversions than required by the rules above, it is recommended that the following alternative version of step 5 be used as a guideline:

5. Otherwise, let *n*, *k*, and *s* be integers such that $k \geq 1$, $10^{k-1} \leq s < 10^k$, the Number value for $s \times 10^{n-k}$ is *m*, and *k* is as small as possible. If there are multiple possibilities for *s*, choose the value of *s* for which $s \times 10^{n-k}$ is closest in value to *m*. If there are two such possible values of *s*, choose the one that is even. Note that *k* is the number of digits in the decimal representation of *s* and that *s* is not divisible by 10.

NOTE 3

Implementers of ECMAScript may find useful the paper and code written by David M. Gay for binary-to-decimal conversion of floating-point numbers:

Gay, David M. Correctly Rounded Binary-Decimal and Decimal-Binary Conversions. Numerical Analysis, Manuscript 90-10. AT&T Bell Laboratories (Murray Hill, New Jersey). November 30, 1990. Available as <http://ampl.com/REFS/abstracts.html#rounding>. Associated code available as <http://netlib.sandia.gov/fp/dtoa.c> and as http://netlib.sandia.gov/fp/g_fmt.c and may also be found at the various **netlib** mirror sites.

7.1.13 ToObject (*argument*)

The abstract operation ToObject converts *argument* to a value of type Object according to Table 12:

Table 12: ToObject Conversions

Argument Type	Result
Undefined	Throw a TypeError exception.
Null	Throw a TypeError exception.
Boolean	Return a new Boolean object whose [[BooleanData]] internal slot is set to <i>argument</i> . See 19.3 for a description of Boolean objects.
Number	Return a new Number object whose [[NumberData]] internal slot is set to <i>argument</i> . See 20.1 for a description of Number objects.
String	Return a new String object whose [[StringData]] internal slot is set to <i>argument</i> . See 21.1 for a description of String objects.
Symbol	Return a new Symbol object whose [[SymbolData]] internal slot is set to <i>argument</i> . See 19.4 for a description of Symbol objects.
Object	Return <i>argument</i> .

7.1.14 ToPropertyKey (*argument*)

The abstract operation ToPropertyKey converts *argument* to a value that can be used as a property key by performing the following steps:

1. Let *key* be ? ToPrimitive(*argument*, hint String).
2. If Type(*key*) is Symbol, then
 - a. Return *key*.
3. Return ! ToString(*key*).

7.1.15 ToLength (*argument*)

The abstract operation ToLength converts *argument* to an integer suitable for use as the length of an array-like object. It performs the following steps:

1. Let *len* be ? ToInteger(*argument*).
2. If *len* ≤ +0, return +0.
3. Return min(*len*, 2⁵³ - 1).

7.1.16 CanonicalNumericIndexString (*argument*)

The abstract operation CanonicalNumericIndexString returns *argument* converted to a numeric value if it is a String representation of a Number that would be produced by ToString, or the string "-0". Otherwise, it returns **undefined**. This abstract operation functions as follows:

1. Assert: Type(*argument*) is String.
2. If *argument* is "-0", return -0.
3. Let *n* be ! ToNumber(*argument*).
4. If SameValue(! ToString(*n*), *argument*) is **false**, return **undefined**.
5. Return *n*.

A *canonical numeric string* is any String value for which the CanonicalNumericIndexString abstract operation does not return **undefined**.

7.1.17 ToIndex (*value*)

The abstract operation ToIndex returns *value* argument converted to a numeric value if it is a valid integer index value. This abstract operation functions as follows:

1. If *value* is **undefined**, then
 - a. Let *index* be 0.
2. Else,
 - a. Let *integerIndex* be ? ToInteger(*value*).
 - b. If *integerIndex* < 0, throw a **RangeError** exception.
 - c. Let *index* be ! ToLength(*integerIndex*).
 - d. If SameValueZero(*integerIndex*, *index*) is **false**, throw a **RangeError** exception.
3. Return *index*.

7.2 Testing and Comparison Operations

7.2.1 RequireObjectCoercible (*argument*)

The abstract operation RequireObjectCoercible throws an error if *argument* is a value that cannot be converted to an Object using [ToObject](#). It is defined by [Table 13](#):

Table 13: **RequireObjectCoercible** Results

Argument Type	Result
Undefined	Throw a TypeError exception.
Null	Throw a TypeError exception.
Boolean	Return <i>argument</i> .
Number	Return <i>argument</i> .
String	Return <i>argument</i> .
Symbol	Return <i>argument</i> .
Object	Return <i>argument</i> .

7.2.2 IsArray (*argument*)

The abstract operation IsArray takes one argument *argument*, and performs the following steps:

1. If [Type\(argument\)](#) is not Object, return **false**.
2. If *argument* is an Array [exotic object](#), return **true**.
3. If *argument* is a Proxy [exotic object](#), then
 - a. If *argument*.[[ProxyHandler]] is **null**, throw a **TypeError** exception.
 - b. Let *target* be *argument*.[[ProxyTarget]].
 - c. Return ? [IsArray\(target\)](#).
4. Return **false**.

7.2.3 IsCallable (*argument*)

The abstract operation IsCallable determines if *argument*, which must be an [ECMAScript language value](#), is a callable function with a [\[\[Call\]\]](#) internal method.

1. If [Type\(argument\)](#) is not Object, return **false**.
2. If *argument* has a [\[\[Call\]\]](#) internal method, return **true**.
3. Return **false**.

7.2.4 IsConstructor (*argument*)

The abstract operation IsConstructor determines if *argument*, which must be an [ECMAScript language value](#), is a [function object](#) with a [\[\[Construct\]\]](#) internal method.

1. If [Type\(argument\)](#) is not Object, return **false**.

2. If *argument* has a `[[Construct]]` internal method, return **true**.
3. Return **false**.

7.2.5 IsExtensible (*O*)

The abstract operation IsExtensible is used to determine whether additional properties can be added to the object that is *O*. A Boolean value is returned. This abstract operation performs the following steps:

1. **Assert:** `Type(O)` is Object.
2. Return ? *O*.`[[IsExtensible]]()`.

7.2.6 IsInteger (*argument*)

The abstract operation IsInteger determines if *argument* is a finite integer numeric value.

1. If `Type(argument)` is not Number, return **false**.
2. If *argument* is NaN, +∞, or -∞, return **false**.
3. If `floor(abs(argument)) ≠ abs(argument)`, return **false**.
4. Return **true**.

7.2.7 IsPropertyKey (*argument*)

The abstract operation IsPropertyKey determines if *argument*, which must be an [ECMAScript language value](#), is a value that may be used as a property key.

1. If `Type(argument)` is String, return **true**.
2. If `Type(argument)` is Symbol, return **true**.
3. Return **false**.

7.2.8 IsRegExp (*argument*)

The abstract operation IsRegExp with argument *argument* performs the following steps:

1. If `Type(argument)` is not Object, return **false**.
2. Let *matcher* be ? `Get(argument, @@match)`.
3. If *matcher* is not **undefined**, return `ToBoolean(matcher)`.
4. If *argument* has a `[[RegExpMatcher]]` internal slot, return **true**.
5. Return **false**.

7.2.9 IsStringPrefix (*p*, *q*)

The abstract operation IsStringPrefix determines if String *p* is a prefix of String *q*.

1. **Assert:** `Type(p)` is String.
2. **Assert:** `Type(q)` is String.
3. If *q* can be the [string-concatenation](#) of *p* and some other String *r*, return **true**. Otherwise, return **false**.
4. NOTE: Any String is a prefix of itself, because *r* may be the empty String.

7.2.10 SameValue (*x*, *y*)

The internal comparison abstract operation SameValue(*x*, *y*), where *x* and *y* are ECMAScript language values, produces **true** or **false**. Such a comparison is performed as follows:

1. If Type(*x*) is different from Type(*y*), return **false**.
2. If Type(*x*) is Number, then
 - a. If *x* is NaN and *y* is NaN, return **true**.
 - b. If *x* is +0 and *y* is -0, return **false**.
 - c. If *x* is -0 and *y* is +0, return **false**.
 - d. If *x* is the same Number value as *y*, return **true**.
 - e. Return **false**.
3. Return SameValueNonNumber(*x*, *y*).

NOTE

This algorithm differs from the Strict Equality Comparison Algorithm in its treatment of signed zeroes and NaNs.

7.2.11 SameValueZero (*x*, *y*)

The internal comparison abstract operation SameValueZero(*x*, *y*), where *x* and *y* are ECMAScript language values, produces **true** or **false**. Such a comparison is performed as follows:

1. If Type(*x*) is different from Type(*y*), return **false**.
2. If Type(*x*) is Number, then
 - a. If *x* is NaN and *y* is NaN, return **true**.
 - b. If *x* is +0 and *y* is -0, return **true**.
 - c. If *x* is -0 and *y* is +0, return **true**.
 - d. If *x* is the same Number value as *y*, return **true**.
 - e. Return **false**.
3. Return SameValueNonNumber(*x*, *y*).

NOTE

SameValueZero differs from SameValue only in its treatment of +0 and -0.

7.2.12 SameValueNonNumber (*x*, *y*)

The internal comparison abstract operation SameValueNonNumber(*x*, *y*), where neither *x* nor *y* are Number values, produces **true** or **false**. Such a comparison is performed as follows:

1. Assert: Type(*x*) is not Number.
2. Assert: Type(*x*) is the same as Type(*y*).
3. If Type(*x*) is Undefined, return **true**.
4. If Type(*x*) is Null, return **true**.
5. If Type(*x*) is String, then
 - a. If *x* and *y* are exactly the same sequence of code units (same length and same code units at corresponding indices), return **true**; otherwise, return **false**.
6. If Type(*x*) is Boolean, then
 - a. If *x* and *y* are both **true** or both **false**, return **true**; otherwise, return **false**.

7. If **Type**(*x*) is Symbol, then
 - a. If *x* and *y* are both the same Symbol value, return **true**; otherwise, return **false**.
8. If *x* and *y* are the same Object value, return **true**. Otherwise, return **false**.

7.2.13 Abstract Relational Comparison

The comparison $x < y$, where *x* and *y* are values, produces **true**, **false**, or **undefined** (which indicates that at least one operand is NaN). In addition to *x* and *y* the algorithm takes a Boolean flag named *LeftFirst* as a parameter. The flag is used to control the order in which operations with potentially visible side-effects are performed upon *x* and *y*. It is necessary because ECMAScript specifies left to right evaluation of expressions. The default value of *LeftFirst* is **true** and indicates that the *x* parameter corresponds to an expression that occurs to the left of the *y* parameter's corresponding expression. If *LeftFirst* is **false**, the reverse is the case and operations must be performed upon *y* before *x*. Such a comparison is performed as follows:

1. If the *LeftFirst* flag is **true**, then
 - a. Let *px* be ? **ToPrimitive**(*x*, hint Number).
 - b. Let *py* be ? **ToPrimitive**(*y*, hint Number).
2. Else the order of evaluation needs to be reversed to preserve left to right evaluation,
 - a. Let *py* be ? **ToPrimitive**(*y*, hint Number).
 - b. Let *px* be ? **ToPrimitive**(*x*, hint Number).
3. If **Type**(*px*) is String and **Type**(*py*) is String, then
 - a. If **IsStringPrefix**(*py*, *px*) is **true**, return **false**.
 - b. If **IsStringPrefix**(*px*, *py*) is **true**, return **true**.
 - c. Let *k* be the smallest nonnegative integer such that the code unit at index *k* within *px* is different from the code unit at index *k* within *py*. (There must be such a *k*, for neither String is a prefix of the other.)
 - d. Let *m* be the integer that is the numeric value of the code unit at index *k* within *px*.
 - e. Let *n* be the integer that is the numeric value of the code unit at index *k* within *py*.
 - f. If $m < n$, return **true**. Otherwise, return **false**.
4. Else,
 - a. NOTE: Because *px* and *py* are primitive values evaluation order is not important.
 - b. Let *nx* be ? **ToNumber**(*px*).
 - c. Let *ny* be ? **ToNumber**(*py*).
 - d. If *nx* is NaN, return **undefined**.
 - e. If *ny* is NaN, return **undefined**.
 - f. If *nx* and *ny* are the same Number value, return **false**.
 - g. If *nx* is +0 and *ny* is -0, return **false**.
 - h. If *nx* is -0 and *ny* is +0, return **false**.
 - i. If *nx* is +∞, return **false**.
 - j. If *ny* is +∞, return **true**.
 - k. If *ny* is -∞, return **false**.
 - l. If *nx* is -∞, return **true**.
 - m. If the mathematical value of *nx* is less than the mathematical value of *ny*—note that these mathematical values are both finite and not both zero—return **true**. Otherwise, return **false**.

NOTE 1

Step 3 differs from step 7 in the algorithm for the addition operator + (12.8.3) by using the logical-and operation instead of the logical-or operation.

NOTE 2

The comparison of Strings uses a simple lexicographic ordering on sequences of code unit values. There is no attempt to

use the more complex, semantically oriented definitions of character or string equality and collating order defined in the Unicode specification. Therefore String values that are canonically equal according to the Unicode standard could test as unequal. In effect this algorithm assumes that both Strings are already in normalized form. Also, note that for strings containing supplementary characters, lexicographic ordering on sequences of UTF-16 code unit values differs from that on sequences of code point values.

7.2.14 Abstract Equality Comparison

The comparison $x == y$, where x and y are values, produces **true** or **false**. Such a comparison is performed as follows:

1. If $\text{Type}(x)$ is the same as $\text{Type}(y)$, then
 - a. Return the result of performing **Strict Equality Comparison** $x === y$.
2. If x is **null** and y is **undefined**, return **true**.
3. If x is **undefined** and y is **null**, return **true**.
4. If $\text{Type}(x)$ is Number and $\text{Type}(y)$ is String, return the result of the comparison $x == !\text{ToNumber}(y)$.
5. If $\text{Type}(x)$ is String and $\text{Type}(y)$ is Number, return the result of the comparison $!\text{ToNumber}(x) == y$.
6. If $\text{Type}(x)$ is Boolean, return the result of the comparison $!\text{ToNumber}(x) == y$.
7. If $\text{Type}(y)$ is Boolean, return the result of the comparison $x == !\text{ToNumber}(y)$.
8. If $\text{Type}(x)$ is either String, Number, or Symbol and $\text{Type}(y)$ is Object, return the result of the comparison $x == \text{ToPrimitive}(y)$.
9. If $\text{Type}(x)$ is Object and $\text{Type}(y)$ is either String, Number, or Symbol, return the result of the comparison $\text{ToPrimitive}(x) == y$.
10. Return **false**.

7.2.15 Strict Equality Comparison

The comparison $x === y$, where x and y are values, produces **true** or **false**. Such a comparison is performed as follows:

1. If $\text{Type}(x)$ is different from $\text{Type}(y)$, return **false**.
2. If $\text{Type}(x)$ is Number, then
 - a. If x is **NaN**, return **false**.
 - b. If y is **NaN**, return **false**.
 - c. If x is the same Number value as y , return **true**.
 - d. If x is **+0** and y is **-0**, return **true**.
 - e. If x is **-0** and y is **+0**, return **true**.
 - f. Return **false**.
3. Return $\text{SameValueNonNumber}(x, y)$.

NOTE

This algorithm differs from the **SameValue** Algorithm in its treatment of signed zeroes and NaNs.

7.3 Operations on Objects

7.3.1 Get (O , P)

The abstract operation **Get** is used to retrieve the value of a specific property of an object. The operation is called with arguments *O* and *P* where *O* is the object and *P* is the property key. This abstract operation performs the following steps:

1. **Assert**: **Type**(*O*) is **Object**.
2. **Assert**: **IsPropertyKey**(*P*) is **true**.
3. Return ? *O*.[[Get]](*P*, *O*).

7.3.2 GetV (*V*, *P*)

The abstract operation **GetV** is used to retrieve the value of a specific property of an **ECMAScript language value**. If the value is not an object, the property lookup is performed using a wrapper object appropriate for the type of the value. The operation is called with arguments *V* and *P* where *V* is the value and *P* is the property key. This abstract operation performs the following steps:

1. **Assert**: **IsPropertyKey**(*P*) is **true**.
2. Let *O* be ? **ToObject**(*V*).
3. Return ? *O*.[[Get]](*P*, *V*).

7.3.3 Set (*O*, *P*, *V*, *Throw*)

The abstract operation **Set** is used to set the value of a specific property of an object. The operation is called with arguments *O*, *P*, *V*, and *Throw* where *O* is the object, *P* is the property key, *V* is the new value for the property and *Throw* is a Boolean flag. This abstract operation performs the following steps:

1. **Assert**: **Type**(*O*) is **Object**.
2. **Assert**: **IsPropertyKey**(*P*) is **true**.
3. **Assert**: **Type**(*Throw*) is **Boolean**.
4. Let *success* be ? *O*.[[Set]](*P*, *V*, *O*).
5. If *success* is **false** and *Throw* is **true**, throw a **TypeError** exception.
6. Return *success*.

7.3.4 CreateDataProperty (*O*, *P*, *V*)

The abstract operation **CreateDataProperty** is used to create a new own property of an object. The operation is called with arguments *O*, *P*, and *V* where *O* is the object, *P* is the property key, and *V* is the value for the property. This abstract operation performs the following steps:

1. **Assert**: **Type**(*O*) is **Object**.
2. **Assert**: **IsPropertyKey**(*P*) is **true**.
3. Let *newDesc* be the PropertyDescriptor { [[Value]]: *V*, [[Writable]]: **true**, [[Enumerable]]: **true**, [[Configurable]]: **true** }.
4. Return ? *O*.[[DefineOwnProperty]](*P*, *newDesc*).

NOTE

This abstract operation creates a property whose attributes are set to the same defaults used for properties created by the ECMAScript language assignment operator. Normally, the property will not already exist. If it does exist and is not configurable or if *O* is not extensible, [[DefineOwnProperty]] will return **false**.

7.3.5 CreateMethodProperty (*O*, *P*, *V*)

The abstract operation CreateMethodProperty is used to create a new own property of an object. The operation is called with arguments *O*, *P*, and *V* where *O* is the object, *P* is the property key, and *V* is the value for the property. This abstract operation performs the following steps:

1. **Assert**: `Type(O)` is Object.
2. **Assert**: `IsPropertyKey(P)` is **true**.
3. Let *newDesc* be the PropertyDescriptor { `[[Value]]`: *V*, `[[Writable]]`: **true**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **true** }.
4. Return ? *O*.`[[DefineOwnProperty]]`(*P*, *newDesc*).

NOTE

This abstract operation creates a property whose attributes are set to the same defaults used for built-in methods and methods defined using class declaration syntax. Normally, the property will not already exist. If it does exist and is not configurable or if *O* is not extensible, `[[DefineOwnProperty]]` will return **false**.

7.3.6 CreateDataPropertyOrThrow (*O*, *P*, *V*)

The abstract operation CreateDataPropertyOrThrow is used to create a new own property of an object. It throws a **TypeError** exception if the requested property update cannot be performed. The operation is called with arguments *O*, *P*, and *V* where *O* is the object, *P* is the property key, and *V* is the value for the property. This abstract operation performs the following steps:

1. **Assert**: `Type(O)` is Object.
2. **Assert**: `IsPropertyKey(P)` is **true**.
3. Let *success* be ? `CreateDataProperty(O, P, V)`.
4. If *success* is **false**, throw a **TypeError** exception.
5. Return *success*.

NOTE

This abstract operation creates a property whose attributes are set to the same defaults used for properties created by the ECMAScript language assignment operator. Normally, the property will not already exist. If it does exist and is not configurable or if *O* is not extensible, `[[DefineOwnProperty]]` will return **false** causing this operation to throw a **TypeError** exception.

7.3.7 DefinePropertyOrThrow (*O*, *P*, *desc*)

The abstract operation DefinePropertyOrThrow is used to call the `[[DefineOwnProperty]]` internal method of an object in a manner that will throw a **TypeError** exception if the requested property update cannot be performed. The operation is called with arguments *O*, *P*, and *desc* where *O* is the object, *P* is the property key, and *desc* is the [Property Descriptor](#) for the property. This abstract operation performs the following steps:

1. **Assert**: `Type(O)` is Object.
2. **Assert**: `IsPropertyKey(P)` is **true**.
3. Let *success* be ? *O*.`[[DefineOwnProperty]]`(*P*, *desc*).
4. If *success* is **false**, throw a **TypeError** exception.
5. Return *success*.

7.3.8 DeletePropertyOrThrow (*O*, *P*)

The abstract operation DeletePropertyOrThrow is used to remove a specific own property of an object. It throws an exception if the property is not configurable. The operation is called with arguments *O* and *P* where *O* is the object and *P* is the property key. This abstract operation performs the following steps:

1. **Assert:** Type(*O*) is Object.
2. **Assert:** IsPropertyKey(*P*) is **true**.
3. Let *success* be ? *O*.[[Delete]](*P*).
4. If *success* is **false**, throw a **TypeError** exception.
5. Return *success*.

7.3.9 GetMethod (*V*, *P*)

The abstract operation GetMethod is used to get the value of a specific property of an [ECMAScript language value](#) when the value of the property is expected to be a function. The operation is called with arguments *V* and *P* where *V* is the [ECMAScript language value](#), *P* is the property key. This abstract operation performs the following steps:

1. **Assert:** IsPropertyKey(*P*) is **true**.
2. Let *func* be ? GetV(*V*, *P*).
3. If *func* is either **undefined** or **null**, return **undefined**.
4. If IsCallable(*func*) is **false**, throw a **TypeError** exception.
5. Return *func*.

7.3.10 HasProperty (*O*, *P*)

The abstract operation HasProperty is used to determine whether an object has a property with the specified property key. The property may be either an own or inherited. A Boolean value is returned. The operation is called with arguments *O* and *P* where *O* is the object and *P* is the property key. This abstract operation performs the following steps:

1. **Assert:** Type(*O*) is Object.
2. **Assert:** IsPropertyKey(*P*) is **true**.
3. Return ? *O*.[[HasProperty]](*P*).

7.3.11 HasOwnProperty (*O*, *P*)

The abstract operation HasOwnProperty is used to determine whether an object has an own property with the specified property key. A Boolean value is returned. The operation is called with arguments *O* and *P* where *O* is the object and *P* is the property key. This abstract operation performs the following steps:

1. **Assert:** Type(*O*) is Object.
2. **Assert:** IsPropertyKey(*P*) is **true**.
3. Let *desc* be ? *O*.[[GetOwnProperty]](*P*).
4. If *desc* is **undefined**, return **false**.
5. Return **true**.

7.3.12 Call (*F*, *V* [, *argumentsList*])

The abstract operation Call is used to call the [[Call]] internal method of a [function object](#). The operation is called with

arguments *F*, *V*, and optionally *argumentsList* where *F* is the [function object](#), *V* is an [ECMAScript language value](#) that is the **this** value of the `[[Call]]`, and *argumentsList* is the value passed to the corresponding argument of the internal method. If *argumentsList* is not present, a new empty [List](#) is used as its value. This abstract operation performs the following steps:

1. If *argumentsList* is not present, set *argumentsList* to a new empty [List](#).
2. If `IsCallable(F)` is **false**, throw a **TypeError** exception.
3. Return ? *F*.`[[Call]]`(*V*, *argumentsList*).

7.3.13 Construct (*F* [, *argumentsList* [, *newTarget*]])

The abstract operation Construct is used to call the `[[Construct]]` internal method of a [function object](#). The operation is called with arguments *F*, and optionally *argumentsList*, and *newTarget* where *F* is the [function object](#). *argumentsList* and *newTarget* are the values to be passed as the corresponding arguments of the internal method. If *argumentsList* is not present, a new empty [List](#) is used as its value. If *newTarget* is not present, *F* is used as its value. This abstract operation performs the following steps:

1. If *newTarget* is not present, set *newTarget* to *F*.
2. If *argumentsList* is not present, set *argumentsList* to a new empty [List](#).
3. **Assert**: `IsConstructor(F)` is **true**.
4. **Assert**: `IsConstructor(newTarget)` is **true**.
5. Return ? *F*.`[[Construct]]`(*argumentsList*, *newTarget*).

NOTE

If *newTarget* is not present, this operation is equivalent to: `new F(...argumentsList)`

7.3.14 SetIntegrityLevel (*O*, *level*)

The abstract operation SetIntegrityLevel is used to fix the set of own properties of an object. This abstract operation performs the following steps:

1. **Assert**: `Type(O)` is Object.
2. **Assert**: *level* is either "**sealed**" or "**frozen**".
3. Let *status* be ? *O*.`[[PreventExtensions]]`() .
4. If *status* is **false**, return **false**.
5. Let *keys* be ? *O*.`[[OwnPropertyKeys]]`() .
6. If *level* is "**sealed**", then
 - a. For each element *k* of *keys*, do
 - i. Perform ? `DefinePropertyOrThrow`(*O*, *k*, `PropertyDescriptor` { `[[Configurable]]`: **false** }).
7. Else *level* is "**frozen**",
 - a. For each element *k* of *keys*, do
 - i. Let *currentDesc* be ? *O*.`[[GetOwnProperty]]`(*k*) .
 - ii. If *currentDesc* is not **undefined**, then
 1. If `IsAccessorDescriptor`(*currentDesc*) is **true**, then
 - a. Let *desc* be the `PropertyDescriptor` { `[[Configurable]]`: **false** } .
 2. Else,
 - a. Let *desc* be the `PropertyDescriptor` { `[[Configurable]]`: **false**, `[[Writable]]`: **false** } .
 3. Perform ? `DefinePropertyOrThrow`(*O*, *k*, *desc*) .

8. Return **true**.

7.3.15 TestIntegrityLevel (*O*, *level*)

The abstract operation TestIntegrityLevel is used to determine if the set of own properties of an object are fixed. This abstract operation performs the following steps:

1. **Assert:** *Type*(*O*) is Object.
2. **Assert:** *level* is either "**sealed**" or "**frozen**".
3. Let *status* be ? *IsExtensible*(*O*).
4. If *status* is **true**, return **false**.
5. NOTE: If the object is extensible, none of its properties are examined.
6. Let *keys* be ? *O*.[[OwnPropertyKeys]]().
7. For each element *k* of *keys*, do
 - a. Let *currentDesc* be ? *O*.[[GetOwnProperty]](*k*).
 - b. If *currentDesc* is not **undefined**, then
 - i. If *currentDesc*.[[Configurable]] is **true**, return **false**.
 - ii. If *level* is "**frozen**" and *IsDataDescriptor*(*currentDesc*) is **true**, then
 1. If *currentDesc*.[[Writable]] is **true**, return **false**.
8. Return **true**.

7.3.16 CreateArrayFromList (*elements*)

The abstract operation CreateArrayFromList is used to create an Array object whose elements are provided by a *List*. This abstract operation performs the following steps:

1. **Assert:** *elements* is a *List* whose elements are all ECMAScript language values.
2. Let *array* be ! *ArrayCreate*(0).
3. Let *n* be 0.
4. For each element *e* of *elements*, do
 - a. Let *status* be *CreateDataProperty*(*array*, ! *ToString*(*n*), *e*).
 - b. **Assert:** *status* is **true**.
 - c. Increment *n* by 1.
5. Return *array*.

7.3.17 CreateListFromArrayLike (*obj* [, *elementTypes*])

The abstract operation CreateListFromArrayLike is used to create a *List* value whose elements are provided by the indexed properties of an array-like object, *obj*. The optional argument *elementTypes* is a *List* containing the names of ECMAScript Language Types that are allowed for element values of the *List* that is created. This abstract operation performs the following steps:

1. If *elementTypes* is not present, set *elementTypes* to « Undefined, Null, Boolean, String, Symbol, Number, Object ».
2. If *Type*(*obj*) is not Object, throw a **TypeError** exception.
3. Let *len* be ? *ToLength*(? *Get*(*obj*, "**length**").
4. Let *list* be a new empty *List*.
5. Let *index* be 0.
6. Repeat, while *index* < *len*
 - a. Let *indexName* be ! *ToString*(*index*).

- b. Let *next* be ? *Get*(*obj*, *indexName*).
 - c. If *Type*(*next*) is not an element of *elementTypes*, throw a **TypeError** exception.
 - d. Append *next* as the last element of *list*.
 - e. Increase *index* by 1.
7. Return *list*.

7.3.18 Invoke (*V*, *P* [, *argumentsList*])

The abstract operation Invoke is used to call a method property of an **ECMAScript language value**. The operation is called with arguments *V*, *P*, and optionally *argumentsList* where *V* serves as both the lookup point for the property and the **this** value of the call, *P* is the property key, and *argumentsList* is the list of arguments values passed to the method. If *argumentsList* is not present, a new empty **List** is used as its value. This abstract operation performs the following steps:

1. **Assert**: *IsPropertyKey*(*P*) is **true**.
2. If *argumentsList* is not present, set *argumentsList* to a new empty **List**.
3. Let *func* be ? *GetV*(*V*, *P*).
4. Return ? *Call*(*func*, *V*, *argumentsList*).

7.3.19 OrdinaryHasInstance (*C*, *O*)

The abstract operation OrdinaryHasInstance implements the default algorithm for determining if an object *O* inherits from the instance object inheritance path provided by **constructor** *C*. This abstract operation performs the following steps:

1. If *IsCallable*(*C*) is **false**, return **false**.
2. If *C* has a *[[BoundTargetFunction]]* internal slot, then
 - a. Let *BC* be *C*.*[[BoundTargetFunction]]*.
 - b. Return ? *InstanceofOperator*(*O*, *BC*).
3. If *Type*(*O*) is not **Object**, return **false**.
4. Let *P* be ? *Get*(*C*, "**prototype**").
5. If *Type*(*P*) is not **Object**, throw a **TypeError** exception.
6. Repeat,
 - a. Set *O* to ? *O*.*[[GetPrototypeOf]]*() .
 - b. If *O* is **null**, return **false**.
 - c. If *SameValue*(*P*, *O*) is **true**, return **true**.

7.3.20 SpeciesConstructor (*O*, *defaultConstructor*)

The abstract operation SpeciesConstructor is used to retrieve the **constructor** that should be used to create new objects that are derived from the argument object *O*. The *defaultConstructor* argument is the **constructor** to use if a **constructor** @@species property cannot be found starting from *O*. This abstract operation performs the following steps:

1. **Assert**: *Type*(*O*) is **Object**.
2. Let *C* be ? *Get*(*O*, "**constructor**").
3. If *C* is **undefined**, return *defaultConstructor*.
4. If *Type*(*C*) is not **Object**, throw a **TypeError** exception.
5. Let *S* be ? *Get*(*C*, @@species).
6. If *S* is either **undefined** or **null**, return *defaultConstructor*.
7. If *IsConstructor*(*S*) is **true**, return *S*.

8. Throw a **TypeError** exception.

7.3.21 EnumerableOwnPropertyNames (*O*, *kind*)

When the abstract operation EnumerableOwnPropertyNames is called with Object *O* and String *kind* the following steps are taken:

1. **Assert**: *Type*(*O*) is Object.
2. Let *ownKeys* be ? *O*.[[OwnPropertyKeys]]().
3. Let *properties* be a new empty List.
4. For each element *key* of *ownKeys* in List order, do
 - a. If *Type*(*key*) is String, then
 - i. Let *desc* be ? *O*.[[GetOwnProperty]](*key*).
 - ii. If *desc* is not **undefined** and *desc*.[[Enumerable]] is **true**, then
 1. If *kind* is "**key**", append *key* to *properties*.
 2. Else,
 - a. Let *value* be ? *Get*(*O*, *key*).
 - b. If *kind* is "**value**", append *value* to *properties*.
 - c. Else,
 - i. **Assert**: *kind* is "**key+value**".
 - ii. Let *entry* be *CreateArrayFromList*(« *key*, *value* »).
 - iii. Append *entry* to *properties*.
 5. Order the elements of *properties* so they are in the same relative order as would be produced by the Iterator that would be returned if the *EnumerateObjectProperties* internal method were invoked with *O*.
 6. Return *properties*.

7.3.22 GetFunctionRealm (*obj*)

The abstract operation GetFunctionRealm with argument *obj* performs the following steps:

1. **Assert**: *obj* is a callable object.
2. If *obj* has a [[Realm]] internal slot, then
 - a. Return *obj*.[[Realm]].
3. If *obj* is a Bound Function *exotic object*, then
 - a. Let *target* be *obj*.[[BoundTargetFunction]].
 - b. Return ? *GetFunctionRealm*(*target*).
4. If *obj* is a Proxy *exotic object*, then
 - a. If *obj*.[[ProxyHandler]] is **null**, throw a **TypeError** exception.
 - b. Let *proxyTarget* be *obj*.[[ProxyTarget]].
 - c. Return ? *GetFunctionRealm*(*proxyTarget*).
5. Return the current Realm Record.

NOTE

Step 5 will only be reached if *obj* is a non-standard function *exotic object* that does not have a [[Realm]] internal slot.

7.3.23 CopyDataProperties (*target*, *source*, *excludedItems*)

When the abstract operation CopyDataProperties is called with arguments *target*, *source*, and *excludedItems*, the

following steps are taken:

1. **Assert**: *Type(target)* is Object.
2. **Assert**: *excludedItems* is a **List** of property keys.
3. If *source* is **undefined** or **null**, return *target*.
4. Let *from* be ! *ToObject(source)*.
5. Let *keys* be ? *from*.[[OwnPropertyKeys]]().
6. For each element *nextKey* of *keys* in **List** order, do
 - a. Let *excluded* be **false**.
 - b. For each element *e* of *excludedItems* in **List** order, do
 - i. If *SameValue(e, nextKey)* is **true**, then
 1. Set *excluded* to **true**.
 - c. If *excluded* is **false**, then
 - i. Let *desc* be ? *from*.[[GetOwnProperty]](*nextKey*).
 - ii. If *desc* is not **undefined** and *desc*.[[Enumerable]] is **true**, then
 1. Let *propValue* be ? *Get(from, nextKey)*.
 2. Perform ! *CreateDataProperty(target, nextKey, propValue)*.
7. Return *target*.

NOTE

The target passed in here is always a newly created object which is not directly accessible in case of an error being thrown.

7.4 Operations on Iterator Objects

See Common Iteration Interfaces (25.1).

7.4.1 GetIterator (*obj* [, *hint* [, *method*]])

The abstract operation GetIterator with argument *obj* and optional arguments *hint* and *method* performs the following steps:

1. If *hint* is not present, set *hint* to **sync**.
2. **Assert**: *hint* is either **sync** or **async**.
3. If *method* is not present, then
 - a. If *hint* is **async**, then
 - i. Set *method* to ? *GetMethod(obj, @@asyncIterator)*.
 - ii. If *method* is **undefined**, then
 1. Let *syncMethod* be ? *GetMethod(obj, @@iterator)*.
 2. Let *syncIteratorRecord* be ? *GetIterator(obj, sync, syncMethod)*.
 3. Return ? *CreateAsyncFromSyncIterator(syncIteratorRecord)*.
 - b. Otherwise, set *method* to ? *GetMethod(obj, @@iterator)*.
4. Let *iterator* be ? *Call(method, obj)*.
5. If *Type(iterator)* is not Object, throw a **TypeError** exception.
6. Let *nextMethod* be ? *GetV(iterator, "next")*.
7. Let *iteratorRecord* be **Record** { [[Iterator]]: *iterator*, [[NextMethod]]: *nextMethod*, [[Done]]: **false** }.
8. Return *iteratorRecord*.

7.4.2 IteratorNext (*iteratorRecord* [, *value*])

The abstract operation IteratorNext with argument *iteratorRecord* and optional argument *value* performs the following steps:

1. If *value* is not present, then
 - a. Let *result* be ? Call(*iteratorRecord*.[[NextMethod]], *iteratorRecord*.[[Iterator]], « »).
2. Else,
 - a. Let *result* be ? Call(*iteratorRecord*.[[NextMethod]], *iteratorRecord*.[[Iterator]], « *value* »).
3. If Type(*result*) is not Object, throw a **TypeError** exception.
4. Return *result*.

7.4.3 IteratorComplete (*iterResult*)

The abstract operation IteratorComplete with argument *iterResult* performs the following steps:

1. Assert: Type(*iterResult*) is Object.
2. Return ToBoolean(? Get(*iterResult*, "done")).

7.4.4 IteratorValue (*iterResult*)

The abstract operation IteratorValue with argument *iterResult* performs the following steps:

1. Assert: Type(*iterResult*) is Object.
2. Return ? Get(*iterResult*, "value").

7.4.5 IteratorStep (*iteratorRecord*)

The abstract operation IteratorStep with argument *iteratorRecord* requests the next value from *iteratorRecord*.[[Iterator]] by calling *iteratorRecord*.[[NextMethod]] and returns either **false** indicating that the iterator has reached its end or the IteratorResult object if a next value is available. IteratorStep performs the following steps:

1. Let *result* be ? IteratorNext(*iteratorRecord*).
2. Let *done* be ? IteratorComplete(*result*).
3. If *done* is **true**, return **false**.
4. Return *result*.

7.4.6 IteratorClose (*iteratorRecord*, *completion*)

The abstract operation IteratorClose with arguments *iteratorRecord* and *completion* is used to notify an iterator that it should perform any actions it would normally perform when it has reached its completed state:

1. Assert: Type(*iteratorRecord*.[[Iterator]]) is Object.
2. Assert: *completion* is a Completion Record.
3. Let *iterator* be *iteratorRecord*.[[Iterator]].
4. Let *return* be ? GetMethod(*iterator*, "return").
5. If *return* is **undefined**, return Completion(*completion*).
6. Let *innerResult* be Call(*return*, *iterator*, « »).
7. If *completion*.[[Type]] is throw, return Completion(*completion*).
8. If *innerResult*.[[Type]] is throw, return Completion(*innerResult*).

9. If `Type(innerResult.[[Value]])` is not Object, throw a **TypeError** exception.
10. Return `Completion(completion)`.

7.4.7 AsyncIteratorClose (*iteratorRecord*, *completion*)

The abstract operation AsyncIteratorClose with arguments *iteratorRecord* and *completion* is used to notify an async iterator that it should perform any actions it would normally perform when it has reached its completed state:

1. **Assert**: `Type(iteratorRecord.[[Iterator]])` is Object.
2. **Assert**: *completion* is a Completion Record.
3. Let *iterator* be `iteratorRecord.[[Iterator]]`.
4. Let *return* be ? `GetMethod(iterator, "return")`.
5. If *return* is **undefined**, return `Completion(completion)`.
6. Let *innerResult* be `Call(return, iterator, « »)`.
7. If *innerResult*.`[[Type]]` is normal, set *innerResult* to `Await(innerResult.[[Value]])`.
8. If *completion*.`[[Type]]` is throw, return `Completion(completion)`.
9. If *innerResult*.`[[Type]]` is throw, return `Completion(innerResult)`.
10. If `Type(innerResult.[[Value]])` is not Object, throw a **TypeError** exception.
11. Return `Completion(completion)`.

7.4.8 CreateIterResultObject (*value*, *done*)

The abstract operation CreateIterResultObject with arguments *value* and *done* creates an object that supports the IteratorResult interface by performing the following steps:

1. **Assert**: `Type(done)` is Boolean.
2. Let *obj* be `ObjectCreate(%ObjectPrototype%)`.
3. Perform `CreateDataProperty(obj, "value", value)`.
4. Perform `CreateDataProperty(obj, "done", done)`.
5. Return *obj*.

7.4.9 CreateListIteratorRecord (*list*)

The abstract operation CreateListIteratorRecord with argument *list* creates an Iterator (25.1.1.2) object record whose next method returns the successive elements of *list*. It performs the following steps:

1. Let *iterator* be `ObjectCreate(%IteratorPrototype%, « [[IteratedList]], [[ListIteratorNextIndex]] »)`.
2. Set *iterator*.`[[IteratedList]]` to *list*.
3. Set *iterator*.`[[ListIteratorNextIndex]]` to 0.
4. Let *steps* be the algorithm steps defined in ListIterator **next** (7.4.9.1).
5. Let *next* be `CreateBuiltinFunction(steps, « »)`.
6. Return `Record { [[Iterator]]: iterator, [[NextMethod]]: next, [[Done]]: false }`.

NOTE

The list iterator object is never directly accessible to ECMAScript code.

7.4.9.1 ListIterator next ()

The ListIterator **next** method is a standard built-in [function object](#) (clause 17) that performs the following steps:

1. Let *O* be the **this** value.
2. **Assert**: [Type](#)(*O*) is Object.
3. **Assert**: *O* has an [\[\[IteratedList\]\]](#) internal slot.
4. Let *list* be *O*.[\[\[IteratedList\]\]](#).
5. Let *index* be *O*.[\[\[ListIteratorNextIndex\]\]](#).
6. Let *len* be the number of elements of *list*.
7. If *index* $\geq$ *len*, then
 - a. Return [CreateIterResultObject](#)(**undefined**, **true**).
8. Set *O*.[\[\[ListIteratorNextIndex\]\]](#) to *index* + 1.
9. Return [CreateIterResultObject](#)(*list*[*index*], **false**).

8 Executable Code and Execution Contexts

8.1 Lexical Environments

A Lexical Environment is a specification type used to define the association of *Identifiers* to specific variables and functions based upon the lexical nesting structure of ECMAScript code. A Lexical Environment consists of an [Environment Record](#) and a possibly null reference to an *outer* Lexical Environment. Usually a Lexical Environment is associated with some specific syntactic structure of ECMAScript code such as a *FunctionDeclaration*, a *BlockStatement*, or a *Catch* clause of a *TryStatement* and a new Lexical Environment is created each time such code is evaluated.

An [Environment Record](#) records the identifier bindings that are created within the scope of its associated Lexical Environment. It is referred to as the Lexical Environment's EnvironmentRecord.

The outer environment reference is used to model the logical nesting of Lexical Environment values. The outer reference of a (inner) Lexical Environment is a reference to the Lexical Environment that logically surrounds the inner Lexical Environment. An outer Lexical Environment may, of course, have its own outer Lexical Environment. A Lexical Environment may serve as the outer environment for multiple inner Lexical Environments. For example, if a *FunctionDeclaration* contains two nested *FunctionDeclarations* then the Lexical Environments of each of the nested functions will have as their outer Lexical Environment the Lexical Environment of the current evaluation of the surrounding function.

A global environment is a Lexical Environment which does not have an outer environment. The [global environment](#)'s outer environment reference is **null**. A [global environment](#)'s EnvironmentRecord may be prepopulated with identifier bindings and includes an associated [global object](#) whose properties provide some of the [global environment](#)'s identifier bindings. As ECMAScript code is executed, additional properties may be added to the [global object](#) and the initial properties may be modified.

A module environment is a Lexical Environment that contains the bindings for the top level declarations of a *Module*. It also contains the bindings that are explicitly imported by the *Module*. The outer environment of a [module environment](#) is a [global environment](#).

A function environment is a Lexical Environment that corresponds to the invocation of an ECMAScript [function object](#). A [function environment](#) may establish a new **this** binding. A [function environment](#) also captures the state necessary to support **super** method invocations.

Lexical Environments and [Environment Record](#) values are purely specification mechanisms and need not correspond to any specific artefact of an ECMAScript implementation. It is impossible for an ECMAScript program to directly access or manipulate such values.

8.1.1 Environment Records

There are two primary kinds of Environment Record values used in this specification: *declarative Environment Records* and *object Environment Records*. Declarative Environment Records are used to define the effect of ECMAScript language syntactic elements such as *FunctionDeclarations*, *VariableDeclarations*, and *Catch* clauses that directly associate identifier bindings with ECMAScript language values. Object Environment Records are used to define the effect of ECMAScript elements such as *WithStatement* that associate identifier bindings with the properties of some object. Global Environment Records and function Environment Records are specializations that are used for specifically for *Script* global declarations and for top-level declarations within functions.

For specification purposes Environment Record values are values of the [Record](#) specification type and can be thought of as existing in a simple object-oriented hierarchy where Environment Record is an abstract class with three concrete subclasses, declarative Environment Record, object Environment Record, and global Environment Record. Function Environment Records and module Environment Records are subclasses of declarative Environment Record. The abstract class includes the abstract specification methods defined in [Table 14](#). These abstract methods have distinct concrete algorithms for each of the concrete subclasses.

Table 14: Abstract Methods of Environment Records

Method	Purpose
HasBinding(<i>N</i>)	Determine if an Environment Record has a binding for the String value <i>N</i> . Return true if it does and false if it does not.
CreateMutableBinding(<i>N</i> , <i>D</i>)	Create a new but uninitialized mutable binding in an Environment Record . The String value <i>N</i> is the text of the bound name. If the Boolean argument <i>D</i> is true the binding may be subsequently deleted.
CreateImmutableBinding(<i>N</i> , <i>S</i>)	Create a new but uninitialized immutable binding in an Environment Record . The String value <i>N</i> is the text of the bound name. If <i>S</i> is true then attempts to set it after it has been initialized will always throw an exception, regardless of the strict mode setting of operations that reference that binding.
InitializeBinding(<i>N</i> , <i>V</i>)	Set the value of an already existing but uninitialized binding in an Environment Record . The String value <i>N</i> is the text of the bound name. <i>V</i> is the value for the binding and is a value of any ECMAScript language type .
SetMutableBinding(<i>N</i> , <i>V</i> , <i>S</i>)	Set the value of an already existing mutable binding in an Environment Record . The String value <i>N</i> is the text of the bound name. <i>V</i> is the value for the binding and may be a value of any ECMAScript language type . <i>S</i> is a Boolean flag. If <i>S</i> is true and the binding cannot be set throw a TypeError exception.
GetBindingValue(<i>N</i> , <i>S</i>)	Returns the value of an already existing binding from an Environment Record . The String value <i>N</i> is the text of the bound name. <i>S</i> is used to identify references originating in strict mode code or that otherwise require strict mode reference semantics. If <i>S</i> is true and the binding does not exist throw a ReferenceError exception. If the binding exists but is uninitialized a ReferenceError is thrown, regardless of the value of <i>S</i> .

DeleteBinding(<i>N</i>)	Delete a binding from an Environment Record . The String value <i>N</i> is the text of the bound name. If a binding for <i>N</i> exists, remove the binding and return true . If the binding exists but cannot be removed return false . If the binding does not exist return true .
HasThisBinding()	Determine if an Environment Record establishes a this binding. Return true if it does and false if it does not.
HasSuperBinding()	Determine if an Environment Record establishes a super method binding. Return true if it does and false if it does not.
WithBaseObject()	If this Environment Record is associated with a with statement, return the with object. Otherwise, return undefined .

8.1.1.1 Declarative Environment Records

Each declarative [Environment Record](#) is associated with an ECMAScript program scope containing variable, constant, let, class, module, import, and/or function declarations. A declarative [Environment Record](#) binds the set of identifiers defined by the declarations contained within its scope.

The behaviour of the concrete specification methods for declarative Environment Records is defined by the following algorithms.

8.1.1.1.1 HasBinding (*N*)

The concrete [Environment Record](#) method HasBinding for declarative Environment Records simply determines if the argument identifier is one of the identifiers bound by the record:

1. Let *envRec* be the declarative [Environment Record](#) for which the method was invoked.
2. If *envRec* has a binding for the name that is the value of *N*, return **true**.
3. Return **false**.

8.1.1.1.2 CreateMutableBinding (*N*, *D*)

The concrete [Environment Record](#) method CreateMutableBinding for declarative Environment Records creates a new mutable binding for the name *N* that is uninitialized. A binding must not already exist in this [Environment Record](#) for *N*. If Boolean argument *D* has the value **true** the new binding is marked as being subject to deletion.

1. Let *envRec* be the declarative [Environment Record](#) for which the method was invoked.
2. **Assert**: *envRec* does not already have a binding for *N*.
3. Create a mutable binding in *envRec* for *N* and record that it is uninitialized. If *D* is **true**, record that the newly created binding may be deleted by a subsequent DeleteBinding call.
4. Return [NormalCompletion](#)(empty).

8.1.1.1.3 CreateImmutableBinding (*N*, *S*)

The concrete [Environment Record](#) method CreateImmutableBinding for declarative Environment Records creates a new immutable binding for the name *N* that is uninitialized. A binding must not already exist in this [Environment Record](#) for *N*. If the Boolean argument *S* has the value **true** the new binding is marked as a strict binding.

1. Let *envRec* be the declarative [Environment Record](#) for which the method was invoked.

2. **Assert:** *envRec* does not already have a binding for *N*.
3. Create an immutable binding in *envRec* for *N* and record that it is uninitialized. If *S* is **true**, record that the newly created binding is a strict binding.
4. Return **NormalCompletion**(empty).

8.1.1.1.4 InitializeBinding (*N*, *V*)

The concrete **Environment Record** method **InitializeBinding** for declarative Environment Records is used to set the bound value of the current binding of the identifier whose name is the value of the argument *N* to the value of argument *V*. An uninitialized binding for *N* must already exist.

1. Let *envRec* be the declarative **Environment Record** for which the method was invoked.
2. **Assert:** *envRec* must have an uninitialized binding for *N*.
3. Set the bound value for *N* in *envRec* to *V*.
4. **Record** that the binding for *N* in *envRec* has been initialized.
5. Return **NormalCompletion**(empty).

8.1.1.1.5 SetMutableBinding (*N*, *V*, *S*)

The concrete **Environment Record** method **SetMutableBinding** for declarative Environment Records attempts to change the bound value of the current binding of the identifier whose name is the value of the argument *N* to the value of argument *V*. A binding for *N* normally already exists, but in rare cases it may not. If the binding is an immutable binding, a **TypeError** is thrown if *S* is **true**.

1. Let *envRec* be the declarative **Environment Record** for which the method was invoked.
2. If *envRec* does not have a binding for *N*, then
 - a. If *S* is **true**, throw a **ReferenceError** exception.
 - b. Perform *envRec*.CreateMutableBinding(*N*, **true**).
 - c. Perform *envRec*.InitializeBinding(*N*, *V*).
 - d. Return **NormalCompletion**(empty).
3. If the binding for *N* in *envRec* is a strict binding, set *S* to **true**.
4. If the binding for *N* in *envRec* has not yet been initialized, throw a **ReferenceError** exception.
5. Else if the binding for *N* in *envRec* is a mutable binding, change its bound value to *V*.
6. Else,
 - a. **Assert:** This is an attempt to change the value of an immutable binding.
 - b. If *S* is **true**, throw a **TypeError** exception.
7. Return **NormalCompletion**(empty).

NOTE

An example of ECMAScript code that results in a missing binding at step 2 is:

```
function f(){eval("var x; x = (delete x, 0);")}
```

8.1.1.1.6 GetBindingValue (*N*, *S*)

The concrete **Environment Record** method **GetBindingValue** for declarative Environment Records simply returns the value of its bound identifier whose name is the value of the argument *N*. If the binding exists but is uninitialized a **ReferenceError** is thrown, regardless of the value of *S*.

1. Let *envRec* be the declarative **Environment Record** for which the method was invoked.
2. **Assert**: *envRec* has a binding for *N*.
3. If the binding for *N* in *envRec* is an uninitialized binding, throw a **ReferenceError** exception.
4. Return the value currently bound to *N* in *envRec*.

8.1.1.1.7 DeleteBinding (*N*)

The concrete **Environment Record** method DeleteBinding for declarative Environment Records can only delete bindings that have been explicitly designated as being subject to deletion.

1. Let *envRec* be the declarative **Environment Record** for which the method was invoked.
2. **Assert**: *envRec* has a binding for the name that is the value of *N*.
3. If the binding for *N* in *envRec* cannot be deleted, return **false**.
4. Remove the binding for *N* from *envRec*.
5. Return **true**.

8.1.1.1.8 HasThisBinding ()

Regular declarative Environment Records do not provide a **this** binding.

1. Return **false**.

8.1.1.1.9 HasSuperBinding ()

Regular declarative Environment Records do not provide a **super** binding.

1. Return **false**.

8.1.1.1.10 WithBaseObject ()

Declarative Environment Records always return **undefined** as their WithBaseObject.

1. Return **undefined**.

8.1.1.2 Object Environment Records

Each object **Environment Record** is associated with an object called its *binding object*. An object **Environment Record** binds the set of string identifier names that directly correspond to the property names of its binding object. Property keys that are not strings in the form of an *IdentifierName* are not included in the set of bound identifiers. Both own and inherited properties are included in the set regardless of the setting of their `[[Enumerable]]` attribute. Because properties can be dynamically added and deleted from objects, the set of identifiers bound by an object **Environment Record** may potentially change as a side-effect of any operation that adds or deletes properties. Any bindings that are created as a result of such a side-effect are considered to be a mutable binding even if the Writable attribute of the corresponding property has the value **false**. Immutable bindings do not exist for object Environment Records.

Object Environment Records created for **with** statements (13.11) can provide their binding object as an implicit **this** value for use in function calls. The capability is controlled by a *withEnvironment* Boolean value that is associated with each object **Environment Record**. By default, the value of *withEnvironment* is **false** for any object **Environment Record**.

The behaviour of the concrete specification methods for object Environment Records is defined by the following algorithms.

8.1.1.2.1 HasBinding (*N*)

The concrete [Environment Record](#) method HasBinding for object Environment Records determines if its associated binding object has a property whose name is the value of the argument *N*:

1. Let *envRec* be the object [Environment Record](#) for which the method was invoked.
2. Let *bindings* be the binding object for *envRec*.
3. Let *foundBinding* be ? [HasProperty](#)(*bindings*, *N*).
4. If *foundBinding* is **false**, return **false**.
5. If the *withEnvironment* flag of *envRec* is **false**, return **true**.
6. Let *unscopables* be ? [Get](#)(*bindings*, @@unscopables).
7. If [Type](#)(*unscopables*) is Object, then
 - a. Let *blocked* be [ToBoolean](#)(? [Get](#)(*unscopables*, *N*)).
 - b. If *blocked* is **true**, return **false**.
8. Return **true**.

8.1.1.2.2 CreateMutableBinding (*N*, *D*)

The concrete [Environment Record](#) method CreateMutableBinding for object Environment Records creates in an [Environment Record](#)'s associated binding object a property whose name is the String value and initializes it to the value **undefined**. If Boolean argument *D* has the value **true** the new property's [\[\[Configurable\]\]](#) attribute is set to **true**; otherwise it is set to **false**.

1. Let *envRec* be the object [Environment Record](#) for which the method was invoked.
2. Let *bindings* be the binding object for *envRec*.
3. Return ? [DefinePropertyOrThrow](#)(*bindings*, *N*, PropertyDescriptor { [\[\[Value\]\]](#): **undefined**, [\[\[Writable\]\]](#): **true**, [\[\[Enumerable\]\]](#): **true**, [\[\[Configurable\]\]](#): *D* }).

NOTE

Normally *envRec* will not have a binding for *N* but if it does, the semantics of [DefinePropertyOrThrow](#) may result in an existing binding being replaced or shadowed or cause an [abrupt completion](#) to be returned.

8.1.1.2.3 CreateImmutableBinding (*N*, *S*)

The concrete [Environment Record](#) method CreateImmutableBinding is never used within this specification in association with object Environment Records.

8.1.1.2.4 InitializeBinding (*N*, *V*)

The concrete [Environment Record](#) method InitializeBinding for object Environment Records is used to set the bound value of the current binding of the identifier whose name is the value of the argument *N* to the value of argument *V*. An uninitialized binding for *N* must already exist.

1. Let *envRec* be the object [Environment Record](#) for which the method was invoked.
2. **Assert**: *envRec* must have an uninitialized binding for *N*.
3. **Record** that the binding for *N* in *envRec* has been initialized.
4. Return ? *envRec*.SetMutableBinding(*N*, *V*, **false**).

NOTE

In this specification, all uses of CreateMutableBinding for object Environment Records are immediately followed by a

call to `InitializeBinding` for the same name. Hence, implementations do not need to explicitly track the initialization state of individual object `Environment Record` bindings.

8.1.1.2.5 `SetMutableBinding ( N, V, S )`

The concrete `Environment Record` method `SetMutableBinding` for object Environment Records attempts to set the value of the `Environment Record`'s associated binding object's property whose name is the value of the argument `N` to the value of argument `V`. A property named `N` normally already exists but if it does not or is not currently writable, error handling is determined by the value of the Boolean argument `S`.

1. Let `envRec` be the object `Environment Record` for which the method was invoked.
2. Let `bindings` be the binding object for `envRec`.
3. Return ? `Set(bindings, N, V, S)`.

8.1.1.2.6 `GetBindingValue ( N, S )`

The concrete `Environment Record` method `GetBindingValue` for object Environment Records returns the value of its associated binding object's property whose name is the String value of the argument identifier `N`. The property should already exist but if it does not the result depends upon the value of the `S` argument:

1. Let `envRec` be the object `Environment Record` for which the method was invoked.
2. Let `bindings` be the binding object for `envRec`.
3. Let `value` be ? `HasProperty(bindings, N)`.
4. If `value` is **false**, then
 - a. If `S` is **false**, return the value **undefined**; otherwise throw a **ReferenceError** exception.
5. Return ? `Get(bindings, N)`.

8.1.1.2.7 `DeleteBinding ( N )`

The concrete `Environment Record` method `DeleteBinding` for object Environment Records can only delete bindings that correspond to properties of the environment object whose `[[Configurable]]` attribute have the value **true**.

1. Let `envRec` be the object `Environment Record` for which the method was invoked.
2. Let `bindings` be the binding object for `envRec`.
3. Return ? `bindings.[[Delete]](N)`.

8.1.1.2.8 `HasThisBinding ( )`

Regular object Environment Records do not provide a **this** binding.

1. Return **false**.

8.1.1.2.9 `HasSuperBinding ( )`

Regular object Environment Records do not provide a **super** binding.

1. Return **false**.

8.1.1.2.10 `WithBaseObject ( )`

Object Environment Records return **undefined** as their `WithBaseObject` unless their `withEnvironment` flag is **true**.

1. Let *envRec* be the object [Environment Record](#) for which the method was invoked.
2. If the *withEnvironment* flag of *envRec* is **true**, return the binding object for *envRec*.
3. Otherwise, return **undefined**.

8.1.1.3 Function Environment Records

A function Environment Record is a declarative [Environment Record](#) that is used to represent the top-level scope of a function and, if the function is not an *ArrowFunction*, provides a **this** binding. If a function is not an *ArrowFunction* function and references **super**, its function Environment Record also contains the state that is used to perform **super** method invocations from within the function.

Function Environment Records have the additional state fields listed in [Table 15](#).

Table 15: Additional Fields of Function Environment Records

Field Name	Value	Meaning
[[ThisValue]]	Any	This is the this value used for this invocation of the function.
[[ThisBindingStatus]]	"lexical" "initialized" "uninitialized"	If the value is "lexical", this is an <i>ArrowFunction</i> and does not have a local this value.
[[FunctionObject]]	Object	The function object whose invocation caused this Environment Record to be created.
[[HomeObject]]	Object undefined	If the associated function has super property accesses and is not an <i>ArrowFunction</i> , [[HomeObject]] is the object that the function is bound to as a method. The default value for [[HomeObject]] is undefined .
[[NewTarget]]	Object undefined	If this Environment Record was created by the [[Construct]] internal method, [[NewTarget]] is the value of the [[Construct]] <i>newTarget</i> parameter. Otherwise, its value is undefined .

Function Environment Records support all of the declarative [Environment Record](#) methods listed in [Table 14](#) and share the same specifications for all of those methods except for HasThisBinding and HasSuperBinding. In addition, function Environment Records support the methods listed in [Table 16](#):

Table 16: Additional Methods of Function Environment Records

Method	Purpose
BindThisValue(V)	Set the [[ThisValue]] and record that it has been initialized.
GetThisBinding()	Return the value of this Environment Record 's this binding. Throws a ReferenceError if the this binding has not been initialized.
GetSuperBase()	Return the object that is the base for super property accesses bound in this Environment Record . The object is derived from this Environment Record 's [[HomeObject]] field. The value undefined indicates that super property accesses will produce runtime errors.

The behaviour of the additional concrete specification methods for function Environment Records is defined by the following algorithms:

8.1.1.3.1 BindThisValue (*V*)

1. Let *envRec* be the [function Environment Record](#) for which the method was invoked.
2. **Assert:** *envRec*.[[ThisBindingStatus]] is not **"lexical"**.
3. If *envRec*.[[ThisBindingStatus]] is **"initialized"**, throw a **ReferenceError** exception.
4. Set *envRec*.[[ThisValue]] to *V*.
5. Set *envRec*.[[ThisBindingStatus]] to **"initialized"**.
6. Return *V*.

8.1.1.3.2 HasThisBinding ()

1. Let *envRec* be the [function Environment Record](#) for which the method was invoked.
2. If *envRec*.[[ThisBindingStatus]] is **"lexical"**, return **false**; otherwise, return **true**.

8.1.1.3.3 HasSuperBinding ()

1. Let *envRec* be the [function Environment Record](#) for which the method was invoked.
2. If *envRec*.[[ThisBindingStatus]] is **"lexical"**, return **false**.
3. If *envRec*.[[HomeObject]] has the value **undefined**, return **false**; otherwise, return **true**.

8.1.1.3.4 GetThisBinding ()

1. Let *envRec* be the [function Environment Record](#) for which the method was invoked.
2. **Assert:** *envRec*.[[ThisBindingStatus]] is not **"lexical"**.
3. If *envRec*.[[ThisBindingStatus]] is **"uninitialized"**, throw a **ReferenceError** exception.
4. Return *envRec*.[[ThisValue]].

8.1.1.3.5 GetSuperBase ()

1. Let *envRec* be the [function Environment Record](#) for which the method was invoked.
2. Let *home* be *envRec*.[[HomeObject]].
3. If *home* has the value **undefined**, return **undefined**.
4. **Assert:** [Type](#)(*home*) is Object.
5. Return ? *home*.[[GetPrototypeOf]]().

8.1.1.4 Global Environment Records

A global [Environment Record](#) is used to represent the outer most scope that is shared by all of the ECMAScript *Script* elements that are processed in a common [realm](#). A global [Environment Record](#) provides the bindings for built-in globals (clause 18), properties of the [global object](#), and for all top-level declarations (13.2.8, 13.2.10) that occur within a *Script*.

A global [Environment Record](#) is logically a single record but it is specified as a composite encapsulating an object [Environment Record](#) and a declarative [Environment Record](#). The object [Environment Record](#) has as its base object the [global object](#) of the associated [Realm Record](#). This [global object](#) is the value returned by the global [Environment Record](#)'s [GetThisBinding](#) concrete method. The object [Environment Record](#) component of a global [Environment Record](#) contains the bindings for all built-in globals (clause 18) and all bindings introduced by a *FunctionDeclaration*, *GeneratorDeclaration*, *AsyncFunctionDeclaration*, *AsyncGeneratorDeclaration*, or *VariableStatement* contained in

global code. The bindings for all other ECMAScript declarations in global code are contained in the declarative [Environment Record](#) component of the global [Environment Record](#).

Properties may be created directly on a [global object](#). Hence, the object [Environment Record](#) component of a global [Environment Record](#) may contain both bindings created explicitly by *FunctionDeclaration*, *GeneratorDeclaration*, *AsyncFunctionDeclaration*, *AsyncGeneratorDeclaration*, or *VariableDeclaration* declarations and bindings created implicitly as properties of the [global object](#). In order to identify which bindings were explicitly created using declarations, a global [Environment Record](#) maintains a list of the names bound using its `CreateGlobalVarBinding` and `CreateGlobalFunctionBinding` concrete methods.

Global Environment Records have the additional fields listed in [Table 17](#) and the additional methods listed in [Table 18](#).

Table 17: Additional Fields of Global Environment Records

Field Name	Value	Meaning
[[ObjectRecord]]	Object	Binding object is the global object . It contains global built-in bindings as well as Environment Record <i>FunctionDeclaration</i> , <i>GeneratorDeclaration</i> , <i>AsyncFunctionDeclaration</i> , <i>AsyncGeneratorDeclaration</i> , and <i>VariableDeclaration</i> bindings in global code for the associated realm .
[[GlobalThisValue]]	Object	The value returned by this in global scope. Hosts may provide any ECMAScript Object value.
[[DeclarativeRecord]]	Declarative Environment Record	Contains bindings for all declarations in global code for the associated realm code except for <i>FunctionDeclaration</i> , <i>GeneratorDeclaration</i> , <i>AsyncFunctionDeclaration</i> , <i>AsyncGeneratorDeclaration</i> , and <i>VariableDeclaration</i> <i>bindings</i> .
[[VarNames]]	List of String	The string names bound by <i>FunctionDeclaration</i> , <i>GeneratorDeclaration</i> , <i>AsyncFunctionDeclaration</i> , <i>AsyncGeneratorDeclaration</i> , and <i>VariableDeclaration</i> declarations in global code for the associated realm .

Table 18: Additional Methods of Global Environment Records

Method	Purpose
<code>GetThisBinding()</code>	Return the value of this Environment Record 's this binding.
<code>HasVarDeclaration (N)</code>	Determines if the argument identifier has a binding in this Environment Record that was created using a <i>VariableDeclaration</i> , <i>FunctionDeclaration</i> , <i>GeneratorDeclaration</i> , <i>AsyncFunctionDeclaration</i> , or <i>AsyncGeneratorDeclaration</i> .
<code>HasLexicalDeclaration (N)</code>	Determines if the argument identifier has a binding in this Environment Record that was created using a lexical declaration such as a <i>LexicalDeclaration</i> or a <i>ClassDeclaration</i> .
<code>HasRestrictedGlobalProperty (N)</code>	Determines if the argument is the name of a global object property that may not be shadowed by a global lexical binding.
<code>CanDeclareGlobalVar (N)</code>	Determines if a corresponding <code>CreateGlobalVarBinding</code> call would succeed if called for the same argument <i>N</i> .

CanDeclareGlobalFunction (N)	Determines if a corresponding CreateGlobalFunctionBinding call would succeed if called for the same argument <i>N</i> .
CreateGlobalVarBinding(N, D)	Used to create and initialize to undefined a global var binding in the [[ObjectRecord]] component of a global Environment Record . The binding will be a mutable binding. The corresponding global object property will have attribute values appropriate for a var . The String value <i>N</i> is the bound name. If <i>D</i> is true the binding may be deleted. Logically equivalent to CreateMutableBinding followed by a SetMutableBinding but it allows var declarations to receive special treatment.
CreateGlobalFunctionBinding(N, V, D)	Create and initialize a global function binding in the [[ObjectRecord]] component of a global Environment Record . The binding will be a mutable binding. The corresponding global object property will have attribute values appropriate for a function . The String value <i>N</i> is the bound name. <i>V</i> is the initialization value. If the Boolean argument <i>D</i> is true the binding may be deleted. Logically equivalent to CreateMutableBinding followed by a SetMutableBinding but it allows function declarations to receive special treatment.

The behaviour of the concrete specification methods for global Environment Records is defined by the following algorithms.

8.1.1.4.1 HasBinding (*N*)

The concrete **Environment Record** method HasBinding for global Environment Records simply determines if the argument identifier is one of the identifiers bound by the record:

1. Let *envRec* be the global **Environment Record** for which the method was invoked.
2. Let *DclRec* be *envRec*.[[DeclarativeRecord]].
3. If *DclRec*.HasBinding(*N*) is **true**, return **true**.
4. Let *ObjRec* be *envRec*.[[ObjectRecord]].
5. Return ? *ObjRec*.HasBinding(*N*).

8.1.1.4.2 CreateMutableBinding (*N*, *D*)

The concrete **Environment Record** method CreateMutableBinding for global Environment Records creates a new mutable binding for the name *N* that is uninitialized. The binding is created in the associated DeclarativeRecord. A binding for *N* must not already exist in the DeclarativeRecord. If Boolean argument *D* has the value **true** the new binding is marked as being subject to deletion.

1. Let *envRec* be the global **Environment Record** for which the method was invoked.
2. Let *DclRec* be *envRec*.[[DeclarativeRecord]].
3. If *DclRec*.HasBinding(*N*) is **true**, throw a **TypeError** exception.
4. Return *DclRec*.CreateMutableBinding(*N*, *D*).

8.1.1.4.3 CreateImmutableBinding (*N*, *S*)

The concrete **Environment Record** method CreateImmutableBinding for global Environment Records creates a new immutable binding for the name *N* that is uninitialized. A binding must not already exist in this **Environment Record** for

N. If the Boolean argument *S* has the value **true** the new binding is marked as a strict binding.

1. Let *envRec* be the global **Environment Record** for which the method was invoked.
2. Let *DclRec* be *envRec*.[[DeclarativeRecord]].
3. If *DclRec*.HasBinding(*N*) is **true**, throw a **TypeError** exception.
4. Return *DclRec*.CreateImmutableBinding(*N*, *S*).

8.1.1.4.4 InitializeBinding (*N*, *V*)

The concrete **Environment Record** method InitializeBinding for global Environment Records is used to set the bound value of the current binding of the identifier whose name is the value of the argument *N* to the value of argument *V*. An uninitialized binding for *N* must already exist.

1. Let *envRec* be the global **Environment Record** for which the method was invoked.
2. Let *DclRec* be *envRec*.[[DeclarativeRecord]].
3. If *DclRec*.HasBinding(*N*) is **true**, then
 - a. Return *DclRec*.InitializeBinding(*N*, *V*).
4. **Assert**: If the binding exists, it must be in the object **Environment Record**.
5. Let *ObjRec* be *envRec*.[[ObjectRecord]].
6. Return ? *ObjRec*.InitializeBinding(*N*, *V*).

8.1.1.4.5 SetMutableBinding (*N*, *V*, *S*)

The concrete **Environment Record** method SetMutableBinding for global Environment Records attempts to change the bound value of the current binding of the identifier whose name is the value of the argument *N* to the value of argument *V*. If the binding is an immutable binding, a **TypeError** is thrown if *S* is **true**. A property named *N* normally already exists but if it does not or is not currently writable, error handling is determined by the value of the Boolean argument *S*.

1. Let *envRec* be the global **Environment Record** for which the method was invoked.
2. Let *DclRec* be *envRec*.[[DeclarativeRecord]].
3. If *DclRec*.HasBinding(*N*) is **true**, then
 - a. Return *DclRec*.SetMutableBinding(*N*, *V*, *S*).
4. Let *ObjRec* be *envRec*.[[ObjectRecord]].
5. Return ? *ObjRec*.SetMutableBinding(*N*, *V*, *S*).

8.1.1.4.6 GetBindingValue (*N*, *S*)

The concrete **Environment Record** method GetBindingValue for global Environment Records returns the value of its bound identifier whose name is the value of the argument *N*. If the binding is an uninitialized binding throw a **ReferenceError** exception. A property named *N* normally already exists but if it does not or is not currently writable, error handling is determined by the value of the Boolean argument *S*.

1. Let *envRec* be the global **Environment Record** for which the method was invoked.
2. Let *DclRec* be *envRec*.[[DeclarativeRecord]].
3. If *DclRec*.HasBinding(*N*) is **true**, then
 - a. Return *DclRec*.GetBindingValue(*N*, *S*).
4. Let *ObjRec* be *envRec*.[[ObjectRecord]].
5. Return ? *ObjRec*.GetBindingValue(*N*, *S*).

8.1.1.4.7 DeleteBinding (*N*)

The concrete [Environment Record](#) method `DeleteBinding` for global Environment Records can only delete bindings that have been explicitly designated as being subject to deletion.

1. Let *envRec* be the global [Environment Record](#) for which the method was invoked.
2. Let *DclRec* be *envRec*.`[[DeclarativeRecord]]`.
3. If *DclRec*.`HasBinding(N)` is **true**, then
 - a. Return *DclRec*.`DeleteBinding(N)`.
4. Let *ObjRec* be *envRec*.`[[ObjectRecord]]`.
5. Let *globalObject* be the binding object for *ObjRec*.
6. Let *existingProp* be ? `HasOwnProperty(globalObject, N)`.
7. If *existingProp* is **true**, then
 - a. Let *status* be ? *ObjRec*.`DeleteBinding(N)`.
 - b. If *status* is **true**, then
 - i. Let *varNames* be *envRec*.`[[VarNames]]`.
 - ii. If *N* is an element of *varNames*, remove that element from the *varNames*.
 - c. Return *status*.
8. Return **true**.

8.1.1.4.8 `HasThisBinding ( )`

1. Return **true**.

8.1.1.4.9 `HasSuperBinding ( )`

1. Return **false**.

8.1.1.4.10 `WithBaseObject ( )`

Global Environment Records always return **undefined** as their `WithBaseObject`.

1. Return **undefined**.

8.1.1.4.11 `GetThisBinding ( )`

1. Let *envRec* be the global [Environment Record](#) for which the method was invoked.
2. Return *envRec*.`[[GlobalThisValue]]`.

8.1.1.4.12 `HasVarDeclaration ( N )`

The concrete [Environment Record](#) method `HasVarDeclaration` for global Environment Records determines if the argument identifier has a binding in this record that was created using a *VariableStatement* or a *FunctionDeclaration*:

1. Let *envRec* be the global [Environment Record](#) for which the method was invoked.
2. Let *varDeclaredNames* be *envRec*.`[[VarNames]]`.
3. If *varDeclaredNames* contains *N*, return **true**.
4. Return **false**.

8.1.1.4.13 `HasLexicalDeclaration ( N )`

The concrete [Environment Record](#) method `HasLexicalDeclaration` for global Environment Records determines if the argument identifier has a binding in this record that was created using a lexical declaration such as a *LexicalDeclaration*

or a *ClassDeclaration*:

1. Let *envRec* be the global **Environment Record** for which the method was invoked.
2. Let *DclRec* be *envRec*.[[DeclarativeRecord]].
3. Return *DclRec*.HasBinding(*N*).

8.1.1.4.14 HasRestrictedGlobalProperty (*N*)

The concrete **Environment Record** method HasRestrictedGlobalProperty for global Environment Records determines if the argument identifier is the name of a property of the **global object** that must not be shadowed by a global lexical binding:

1. Let *envRec* be the global **Environment Record** for which the method was invoked.
2. Let *ObjRec* be *envRec*.[[ObjectRecord]].
3. Let *globalObject* be the binding object for *ObjRec*.
4. Let *existingProp* be ? *globalObject*.[[GetOwnProperty]](*N*).
5. If *existingProp* is **undefined**, return **false**.
6. If *existingProp*.[[Configurable]] is **true**, return **false**.
7. Return **true**.

NOTE

Properties may exist upon a **global object** that were directly created rather than being declared using a var or function declaration. A global lexical binding may not be created that has the same name as a non-configurable property of the **global object**. The global property **undefined** is an example of such a property.

8.1.1.4.15 CanDeclareGlobalVar (*N*)

The concrete **Environment Record** method CanDeclareGlobalVar for global Environment Records determines if a corresponding CreateGlobalVarBinding call would succeed if called for the same argument *N*. Redundant var declarations and var declarations for pre-existing **global object** properties are allowed.

1. Let *envRec* be the global **Environment Record** for which the method was invoked.
2. Let *ObjRec* be *envRec*.[[ObjectRecord]].
3. Let *globalObject* be the binding object for *ObjRec*.
4. Let *hasProperty* be ? **HasOwnProperty**(*globalObject*, *N*).
5. If *hasProperty* is **true**, return **true**.
6. Return ? **IsExtensible**(*globalObject*).

8.1.1.4.16 CanDeclareGlobalFunction (*N*)

The concrete **Environment Record** method CanDeclareGlobalFunction for global Environment Records determines if a corresponding CreateGlobalFunctionBinding call would succeed if called for the same argument *N*.

1. Let *envRec* be the global **Environment Record** for which the method was invoked.
2. Let *ObjRec* be *envRec*.[[ObjectRecord]].
3. Let *globalObject* be the binding object for *ObjRec*.
4. Let *existingProp* be ? *globalObject*.[[GetOwnProperty]](*N*).
5. If *existingProp* is **undefined**, return ? **IsExtensible**(*globalObject*).
6. If *existingProp*.[[Configurable]] is **true**, return **true**.
7. If **IsDataDescriptor**(*existingProp*) is **true** and *existingProp* has attribute values { [[Writable]]: **true**, [[Enumerable]]:

true }, return **true**.

8. Return **false**.

8.1.1.4.17 CreateGlobalVarBinding (*N*, *D*)

The concrete [Environment Record](#) method `CreateGlobalVarBinding` for global Environment Records creates and initializes a mutable binding in the associated object [Environment Record](#) and records the bound name in the associated `[[VarNames]]` [List](#). If a binding already exists, it is reused and assumed to be initialized.

1. Let *envRec* be the global [Environment Record](#) for which the method was invoked.
2. Let *ObjRec* be *envRec*.`[[ObjectRecord]]`.
3. Let *globalObject* be the binding object for *ObjRec*.
4. Let *hasProperty* be ? `HasOwnProperty(globalObject, N)`.
5. Let *extensible* be ? `IsExtensible(globalObject)`.
6. If *hasProperty* is **false** and *extensible* is **true**, then
 - a. Perform ? *ObjRec*.`CreateMutableBinding(N, D)`.
 - b. Perform ? *ObjRec*.`InitializeBinding(N, undefined)`.
7. Let *varDeclaredNames* be *envRec*.`[[VarNames]]`.
8. If *varDeclaredNames* does not contain *N*, then
 - a. Append *N* to *varDeclaredNames*.
9. Return `NormalCompletion(empty)`.

8.1.1.4.18 CreateGlobalFunctionBinding (*N*, *V*, *D*)

The concrete [Environment Record](#) method `CreateGlobalFunctionBinding` for global Environment Records creates and initializes a mutable binding in the associated object [Environment Record](#) and records the bound name in the associated `[[VarNames]]` [List](#). If a binding already exists, it is replaced.

1. Let *envRec* be the global [Environment Record](#) for which the method was invoked.
2. Let *ObjRec* be *envRec*.`[[ObjectRecord]]`.
3. Let *globalObject* be the binding object for *ObjRec*.
4. Let *existingProp* be ? *globalObject*.`[[GetOwnProperty]](N)`.
5. If *existingProp* is **undefined** or *existingProp*.`[[Configurable]]` is **true**, then
 - a. Let *desc* be the PropertyDescriptor { `[[Value]]`: *V*, `[[Writable]]`: **true**, `[[Enumerable]]`: **true**, `[[Configurable]]`: *D* }.
6. Else,
 - a. Let *desc* be the PropertyDescriptor { `[[Value]]`: *V* }.
7. Perform ? `DefinePropertyOrThrow(globalObject, N, desc)`.
8. [Record](#) that the binding for *N* in *ObjRec* has been initialized.
9. Perform ? `Set(globalObject, N, V, false)`.
10. Let *varDeclaredNames* be *envRec*.`[[VarNames]]`.
11. If *varDeclaredNames* does not contain *N*, then
 - a. Append *N* to *varDeclaredNames*.
12. Return `NormalCompletion(empty)`.

NOTE

Global function declarations are always represented as own properties of the [global object](#). If possible, an existing own property is reconfigured to have a standard set of attribute values. Steps 8-9 are equivalent to what calling the `InitializeBinding` concrete method would do and if *globalObject* is a Proxy will produce the same sequence of Proxy trap calls.

8.1.1.5 Module Environment Records

A module [Environment Record](#) is a declarative [Environment Record](#) that is used to represent the outer scope of an ECMAScript *Module*. In addition to normal mutable and immutable bindings, module Environment Records also provide immutable import bindings which are bindings that provide indirect access to a target binding that exists in another [Environment Record](#).

Module Environment Records support all of the declarative [Environment Record](#) methods listed in [Table 14](#) and share the same specifications for all of those methods except for `GetBindingValue`, `DeleteBinding`, `HasThisBinding` and `GetThisBinding`. In addition, module Environment Records support the methods listed in [Table 19](#):

Table 19: Additional Methods of Module Environment Records

Method	Purpose
<code>CreateImportBinding(N, M, N2)</code>	Create an immutable indirect binding in a module Environment Record . The String value <i>N</i> is the text of the bound name. <i>M</i> is a Module Record , and <i>N2</i> is a binding that exists in <i>M</i> 's module Environment Record .
<code>GetThisBinding()</code>	Return the value of this Environment Record 's this binding.

The behaviour of the additional concrete specification methods for module Environment Records are defined by the following algorithms:

8.1.1.5.1 `GetBindingValue ( N, S )`

The concrete [Environment Record](#) method `GetBindingValue` for module Environment Records returns the value of its bound identifier whose name is the value of the argument *N*. However, if the binding is an indirect binding the value of the target binding is returned. If the binding exists but is uninitialized a **ReferenceError** is thrown.

1. **Assert:** *S* is **true**.
2. Let *envRec* be the module [Environment Record](#) for which the method was invoked.
3. **Assert:** *envRec* has a binding for *N*.
4. If the binding for *N* is an indirect binding, then
 - a. Let *M* and *N2* be the indirection values provided when this binding for *N* was created.
 - b. Let *targetEnv* be *M*.[[Environment]].
 - c. If *targetEnv* is **undefined**, throw a **ReferenceError** exception.
 - d. Let *targetER* be *targetEnv*'s [EnvironmentRecord](#).
 - e. Return ? *targetER*.`GetBindingValue(N2, true)`.
5. If the binding for *N* in *envRec* is an uninitialized binding, throw a **ReferenceError** exception.
6. Return the value currently bound to *N* in *envRec*.

NOTE

S will always be **true** because a *Module* is always [strict mode code](#).

8.1.1.5.2 `DeleteBinding ( N )`

The concrete [Environment Record](#) method `DeleteBinding` for module Environment Records refuses to delete bindings.

1. **Assert**: This method is never invoked. See 12.5.3.1.

NOTE

Module Environment Records are only used within strict code and an **early error** rule prevents the delete operator, in strict code, from being applied to a **Reference** that would resolve to a module **Environment Record** binding. See 12.5.3.1.

8.1.1.5.3 HasThisBinding ()

Module Environment Records provide a **this** binding.

1. Return **true**.

8.1.1.5.4 GetThisBinding ()

1. Return **undefined**.

8.1.1.5.5 CreateImportBinding (*N*, *M*, *N2*)

The concrete **Environment Record** method CreateImportBinding for module Environment Records creates a new initialized immutable indirect binding for the name *N*. A binding must not already exist in this **Environment Record** for *N*. *M* is a **Module Record**, and *N2* is the name of a binding that exists in *M*'s module **Environment Record**. Accesses to the value of the new binding will indirectly access the bound value of the target binding.

1. Let *envRec* be the module **Environment Record** for which the method was invoked.
2. **Assert**: *envRec* does not already have a binding for *N*.
3. **Assert**: *M* is a **Module Record**.
4. **Assert**: When *M*.[[Environment]] is instantiated it will have a direct binding for *N2*.
5. Create an immutable indirect binding in *envRec* for *N* that references *M* and *N2* as its target binding and record that the binding is initialized.
6. Return **NormalCompletion**(empty).

8.1.2 Lexical Environment Operations

The following **abstract operations** are used in this specification to operate upon lexical environments:

8.1.2.1 GetIdentifierReference (*lex*, *name*, *strict*)

The abstract operation GetIdentifierReference is called with a **Lexical Environment** *lex*, a String *name*, and a Boolean flag *strict*. The value of *lex* may be **null**. When called, the following steps are performed:

1. If *lex* is the value **null**, then
 - a. Return a value of type **Reference** whose base value component is **undefined**, whose referenced name component is *name*, and whose strict reference flag is *strict*.
2. Let *envRec* be *lex*'s **EnvironmentRecord**.
3. Let *exists* be ? *envRec*.HasBinding(*name*).
4. If *exists* is **true**, then
 - a. Return a value of type **Reference** whose base value component is *envRec*, whose referenced name component is *name*, and whose strict reference flag is *strict*.
5. Else,

- a. Let *outer* be the value of *lex*'s outer environment reference.
- b. Return ? *GetIdentifierReference*(*outer*, *name*, *strict*).

8.1.2.2 NewDeclarativeEnvironment (*E*)

When the abstract operation NewDeclarativeEnvironment is called with a *Lexical Environment* as argument *E* the following steps are performed:

1. Let *env* be a new *Lexical Environment*.
2. Let *envRec* be a new declarative *Environment Record* containing no bindings.
3. Set *env*'s *EnvironmentRecord* to *envRec*.
4. Set the outer lexical environment reference of *env* to *E*.
5. Return *env*.

8.1.2.3 NewObjectEnvironment (*O*, *E*)

When the abstract operation NewObjectEnvironment is called with an Object *O* and a *Lexical Environment* *E* as arguments, the following steps are performed:

1. Let *env* be a new *Lexical Environment*.
2. Let *envRec* be a new object *Environment Record* containing *O* as the binding object.
3. Set *env*'s *EnvironmentRecord* to *envRec*.
4. Set the outer lexical environment reference of *env* to *E*.
5. Return *env*.

8.1.2.4 NewFunctionEnvironment (*F*, *newTarget*)

When the abstract operation NewFunctionEnvironment is called with arguments *F* and *newTarget* the following steps are performed:

1. *Assert*: *F* is an ECMAScript function.
2. *Assert*: *Type*(*newTarget*) is Undefined or Object.
3. Let *env* be a new *Lexical Environment*.
4. Let *envRec* be a new *function Environment Record* containing no bindings.
5. Set *envRec*.[[FunctionObject]] to *F*.
6. If *F*.[[ThisMode]] is lexical, set *envRec*.[[ThisBindingStatus]] to "lexical".
7. Else, set *envRec*.[[ThisBindingStatus]] to "uninitialized".
8. Let *home* be *F*.[[HomeObject]].
9. Set *envRec*.[[HomeObject]] to *home*.
10. Set *envRec*.[[NewTarget]] to *newTarget*.
11. Set *env*'s *EnvironmentRecord* to *envRec*.
12. Set the outer lexical environment reference of *env* to *F*.[[Environment]].
13. Return *env*.

8.1.2.5 NewGlobalEnvironment (*G*, *thisValue*)

When the abstract operation NewGlobalEnvironment is called with arguments *G* and *thisValue*, the following steps are performed:

1. Let *env* be a new *Lexical Environment*.

- Let *objRec* be a new object **Environment Record** containing *G* as the binding object.
- Let *dclRec* be a new declarative **Environment Record** containing no bindings.
- Let *globalRec* be a new global **Environment Record**.
- Set *globalRec*.[[ObjectRecord]] to *objRec*.
- Set *globalRec*.[[GlobalThisValue]] to *thisValue*.
- Set *globalRec*.[[DeclarativeRecord]] to *dclRec*.
- Set *globalRec*.[[VarNames]] to a new empty **List**.
- Set *env*'s **EnvironmentRecord** to *globalRec*.
- Set the outer lexical environment reference of *env* to **null**.
- Return *env*.

8.1.2.6 NewModuleEnvironment (*E*)

When the abstract operation NewModuleEnvironment is called with a **Lexical Environment** argument *E* the following steps are performed:

- Let *env* be a new **Lexical Environment**.
- Let *envRec* be a new module **Environment Record** containing no bindings.
- Set *env*'s **EnvironmentRecord** to *envRec*.
- Set the outer lexical environment reference of *env* to *E*.
- Return *env*.

8.2 Realms

Before it is evaluated, all ECMAScript code must be associated with a realm. Conceptually, a **realm** consists of a set of intrinsic objects, an ECMAScript **global environment**, all of the ECMAScript code that is loaded within the scope of that **global environment**, and other associated state and resources.

A **realm** is represented in this specification as a Realm Record with the fields specified in Table 20:

Table 20: **Realm Record** Fields

Field Name	Value	Meaning
[[Intrinsics]]	Record whose field names are intrinsic keys and whose values are objects	The intrinsic values used by code associated with this realm
[[GlobalObject]]	Object	The global object for this realm
[[GlobalEnv]]	Lexical Environment	The global environment for this realm

[[TemplateMap]]	A List of Record { [[Site]]: Parse Node, [[Array]]: Object }.	Template objects are canonicalized separately for each realm using its Realm Record's [[TemplateMap]]. Each [[Site]] value is a Parse Node that is a TemplateLiteral. The associated [[Array]] value is the corresponding template object that is passed to a tag function.
-----------------	--	---

NOTE

Once a Parse Node becomes unreachable, the corresponding [[Array]] is also unreachable, and it would be unobservable if an implementation removed the pair from the [[TemplateMap]] list.

[[HostDefined]]	Any, default value is undefined .	Field reserved for use by host environments that need to associate additional information with a Realm Record.
-----------------	--	--

8.2.1 CreateRealm ()

The abstract operation CreateRealm with no arguments performs the following steps:

1. Let *realmRec* be a new Realm Record.
2. Perform CreateIntrinsics(*realmRec*).
3. Set *realmRec*.[[GlobalObject]] to **undefined**.
4. Set *realmRec*.[[GlobalEnv]] to **undefined**.
5. Set *realmRec*.[[TemplateMap]] to a new empty List.
6. Return *realmRec*.

8.2.2 CreateIntrinsics (*realmRec*)

The abstract operation CreateIntrinsics with argument *realmRec* performs the following steps:

1. Let *intrinsics* be a new Record.
2. Set *realmRec*.[[Intrinsics]] to *intrinsics*.
3. Let *objProto* be ObjectCreate(**null**).
4. Set *intrinsics*.[[%ObjectPrototype%]] to *objProto*.
5. Let *throwerSteps* be the algorithm steps specified in 9.2.9.1 for the %ThrowTypeError% function.
6. Let *thrower* be CreateBuiltinFunction(*throwerSteps*, « », *realmRec*, **null**).
7. Set *intrinsics*.[[%ThrowTypeError%]] to *thrower*.
8. Let *noSteps* be an empty sequence of algorithm steps.
9. Let *funcProto* be CreateBuiltinFunction(*noSteps*, « », *realmRec*, *objProto*).
10. Set *intrinsics*.[[%FunctionPrototype%]] to *funcProto*.
11. Call *thrower*.[[SetPrototypeOf]](*funcProto*).
12. Perform AddRestrictedFunctionProperties(*funcProto*, *realmRec*).
13. Set fields of *intrinsics* with the values listed in Table 7 that have not already been handled above. The field names are the names listed in column one of the table. The value of each field is a new object value fully and recursively populated with property values as defined by the specification of each object in clauses 18-26. All object property values are newly created object values. All values that are built-in function objects are created by performing CreateBuiltinFunction(<steps>, <slots>, *realmRec*, <prototype>) where <steps> is the definition of that function provided by this specification, <slots> is a list of the names, if any, of the function's specified internal slots, and <prototype> is the specified value of the function's [[Prototype]] internal slot. The creation of the intrinsics and

their properties must be ordered to avoid any dependencies upon objects that have not yet been created.

14. Return *intrinsics*.

8.2.3 SetRealmGlobalObject (*realmRec*, *globalObj*, *thisValue*)

The abstract operation SetRealmGlobalObject with arguments *realmRec*, *globalObj*, and *thisValue* performs the following steps:

1. If *globalObj* is **undefined**, then
 - a. Let *intrinsics* be *realmRec*.[[Intrinsics]].
 - b. Set *globalObj* to *ObjectCreate*(*intrinsics*.[[%ObjectPrototype%]]).
2. Assert: *Type*(*globalObj*) is Object.
3. If *thisValue* is **undefined**, set *thisValue* to *globalObj*.
4. Set *realmRec*.[[GlobalObject]] to *globalObj*.
5. Let *newGlobalEnv* be *NewGlobalEnvironment*(*globalObj*, *thisValue*).
6. Set *realmRec*.[[GlobalEnv]] to *newGlobalEnv*.
7. Return *realmRec*.

8.2.4 SetDefaultGlobalBindings (*realmRec*)

The abstract operation SetDefaultGlobalBindings with argument *realmRec* performs the following steps:

1. Let *global* be *realmRec*.[[GlobalObject]].
2. For each property of the Global Object specified in clause 18, do
 - a. Let *name* be the String value of the *property name*.
 - b. Let *desc* be the fully populated *data property* descriptor for the property containing the specified attributes for the property. For properties listed in 18.2, 18.3, or 18.4 the value of the [[Value]] attribute is the corresponding intrinsic object from *realmRec*.
 - c. Perform ? *DefinePropertyOrThrow*(*global*, *name*, *desc*).
3. Return *global*.

8.3 Execution Contexts

An execution context is a specification device that is used to track the runtime evaluation of code by an ECMAScript implementation. At any point in time, there is at most one execution context per *agent* that is actually executing code. This is known as the *agent's* running execution context. All references to the *running execution context* in this specification denote the *running execution context* of the *surrounding agent*.

The execution context stack is used to track execution contexts. The *running execution context* is always the top element of this stack. A new execution context is created whenever control is transferred from the executable code associated with the currently *running execution context* to executable code that is not associated with that execution context. The newly created execution context is pushed onto the stack and becomes the *running execution context*.

An execution context contains whatever implementation specific state is necessary to track the execution progress of its associated code. Each execution context has at least the state components listed in Table 21.

Table 21: State Components for All Execution Contexts

Component	Purpose
-----------	---------

code evaluation state	Any state needed to perform, suspend, and resume evaluation of the code associated with this execution context .
Function	If this execution context is evaluating the code of a function object , then the value of this component is that function object . If the context is evaluating the code of a <i>Script</i> or <i>Module</i> , the value is null .
Realm	The Realm Record from which associated code accesses ECMAScript resources.
ScriptOrModule	The Module Record or Script Record from which associated code originates. If there is no originating script or module, as is the case for the original execution context created in InitializeHostDefinedRealm , the value is null .

Evaluation of code by the [running execution context](#) may be suspended at various points defined within this specification. Once the [running execution context](#) has been suspended a different execution context may become the [running execution context](#) and commence evaluating its code. At some later time a suspended execution context may again become the [running execution context](#) and continue evaluating its code at the point where it had previously been suspended. Transition of the [running execution context](#) status among execution contexts usually occurs in stack-like last-in/first-out manner. However, some ECMAScript features require non-LIFO transitions of the [running execution context](#).

The value of the [Realm](#) component of the [running execution context](#) is also called the current Realm Record. The value of the Function component of the [running execution context](#) is also called the active function object.

Execution contexts for ECMAScript code have the additional state components listed in [Table 22](#).

Table 22: Additional State Components for ECMAScript Code Execution Contexts

Component	Purpose
LexicalEnvironment	Identifies the Lexical Environment used to resolve identifier references made by code within this execution context .
VariableEnvironment	Identifies the Lexical Environment whose EnvironmentRecord holds bindings created by <i>VariableStatements</i> within this execution context .

The LexicalEnvironment and VariableEnvironment components of an execution context are always Lexical Environments.

Execution contexts representing the evaluation of generator objects have the additional state components listed in [Table 23](#).

Table 23: Additional State Components for Generator Execution Contexts

Component	Purpose
Generator	The GeneratorObject that this execution context is evaluating.

In most situations only the [running execution context](#) (the top of the [execution context stack](#)) is directly manipulated by algorithms within this specification. Hence when the terms “LexicalEnvironment”, and “VariableEnvironment” are used without qualification they are in reference to those components of the [running execution context](#).

An execution context is purely a specification mechanism and need not correspond to any particular artefact of an ECMAScript implementation. It is impossible for ECMAScript code to directly access or observe an execution context.

8.3.1 GetActiveScriptOrModule ()

The GetActiveScriptOrModule abstract operation is used to determine the running script or module, based on the [running execution context](#). GetActiveScriptOrModule performs the following steps:

1. If the [execution context stack](#) is empty, return **null**.
2. Let *ec* be the topmost [execution context](#) on the [execution context stack](#) whose ScriptOrModule component is not **null**.
3. If no such [execution context](#) exists, return **null**. Otherwise, return *ec*'s ScriptOrModule component.

8.3.2 ResolveBinding (*name* [, *env*])

The ResolveBinding abstract operation is used to determine the binding of *name* passed as a String value. The optional argument *env* can be used to explicitly provide the [Lexical Environment](#) that is to be searched for the binding. During execution of ECMAScript code, ResolveBinding is performed using the following algorithm:

1. If *env* is not present or if *env* is **undefined**, then
 - a. Set *env* to the [running execution context](#)'s LexicalEnvironment.
2. **Assert:** *env* is a [Lexical Environment](#).
3. If the code matching the syntactic production that is being evaluated is contained in [strict mode code](#), let *strict* be **true**, else let *strict* be **false**.
4. Return ? [GetIdentifierReference](#)(*env*, *name*, *strict*).

NOTE

The result of ResolveBinding is always a [Reference](#) value with its referenced name component equal to the *name* argument.

8.3.3 GetThisEnvironment ()

The abstract operation GetThisEnvironment finds the [Environment Record](#) that currently supplies the binding of the keyword **this**. GetThisEnvironment performs the following steps:

1. Let *lex* be the [running execution context](#)'s LexicalEnvironment.
2. Repeat,
 - a. Let *envRec* be *lex*'s [EnvironmentRecord](#).
 - b. Let *exists* be *envRec*.HasThisBinding().
 - c. If *exists* is **true**, return *envRec*.
 - d. Let *outer* be the value of *lex*'s outer environment reference.
 - e. **Assert:** *outer* is not **null**.
 - f. Set *lex* to *outer*.

NOTE

The loop in step 2 will always terminate because the list of environments always ends with the [global environment](#) which has a **this** binding.

8.3.4 ResolveThisBinding ()

The abstract operation ResolveThisBinding determines the binding of the keyword **this** using the LexicalEnvironment of the [running execution context](#). ResolveThisBinding performs the following steps:

1. Let *envRec* be [GetThisEnvironment\(\)](#).
2. Return ? *envRec*.GetThisBinding().

8.3.5 GetNewTarget ()

The abstract operation GetNewTarget determines the NewTarget value using the LexicalEnvironment of the [running execution context](#). GetNewTarget performs the following steps:

1. Let *envRec* be [GetThisEnvironment\(\)](#).
2. **Assert:** *envRec* has a `[[NewTarget]]` field.
3. Return *envRec*.`[[NewTarget]]`.

8.3.6 GetGlobalObject ()

The abstract operation GetGlobalObject returns the [global object](#) used by the currently [running execution context](#). GetGlobalObject performs the following steps:

1. Let *ctx* be the [running execution context](#).
2. Let *currentRealm* be *ctx*'s [Realm](#).
3. Return *currentRealm*.`[[GlobalObject]]`.

8.4 Jobs and Job Queues

A Job is an abstract operation that initiates an ECMAScript computation when no other ECMAScript computation is currently in progress. A Job abstract operation may be defined to accept an arbitrary set of job parameters.

Execution of a Job can be initiated only when there is no [running execution context](#) and the [execution context stack](#) is empty. A PendingJob is a request for the future execution of a Job. A PendingJob is an internal [Record](#) whose fields are specified in [Table 24](#). Once execution of a Job is initiated, the Job always executes to completion. No other Job may be initiated until the currently running Job completes. However, the currently running Job or external events may cause the enqueueing of additional PendingJobs that may be initiated sometime after completion of the currently running Job.

Table 24: PendingJob [Record](#) Fields

Field Name	Value	Meaning
<code>[[Job]]</code>	The name of a Job abstract operation	This is the abstract operation that is performed when execution of this PendingJob is initiated.
<code>[[Arguments]]</code>	A List	The List of argument values that are to be passed to <code>[[Job]]</code> when it is activated.
<code>[[Realm]]</code>	A Realm Record	The Realm Record for the initial execution context when this PendingJob is initiated.

[[ScriptOrModule]]	A Script Record or Module Record	The script or module for the initial execution context when this PendingJob is initiated.
[[HostDefined]]	Any, default value is undefined .	Field reserved for use by host environments that need to associate additional information with a pending Job.

A Job Queue is a FIFO queue of PendingJob records. Each Job Queue has a name and the full set of available Job Queues are defined by an ECMAScript implementation. Every ECMAScript implementation has at least the Job Queues defined in [Table 25](#).

Each [agent](#) has its own set of named Job Queues. All references to a named job queue in this specification denote the named job queue of the [surrounding agent](#).

Table 25: Required Job Queues

Name	Purpose
ScriptJobs	Jobs that validate and evaluate ECMAScript <i>Script</i> and <i>Module</i> source text. See clauses 10 and 15.
PromiseJobs	Jobs that are responses to the settlement of a Promise (see 25.6).

A request for the future execution of a Job is made by enqueueing, on a Job Queue, a PendingJob record that includes a Job abstract operation name and any necessary argument values. When there is no [running execution context](#) and the [execution context stack](#) is empty, the ECMAScript implementation removes the first PendingJob from a Job Queue and uses the information contained in it to create an [execution context](#) and starts execution of the associated Job abstract operation.

The PendingJob records from a single Job Queue are always initiated in FIFO order. This specification does not define the order in which multiple Job Queues are serviced. An ECMAScript implementation may interweave the FIFO evaluation of the PendingJob records of a Job Queue with the evaluation of the PendingJob records of one or more other Job Queues. An implementation must define what occurs when there are no [running execution context](#) and all Job Queues are empty.

NOTE

Typically an ECMAScript implementation will have its Job Queues pre-initialized with at least one PendingJob and one of those Jobs will be the first to be executed. An implementation might choose to free all resources and terminate if the current Job completes and all Job Queues are empty. Alternatively, it might choose to wait for a some implementation specific [agent](#) or mechanism to enqueue new PendingJob requests.

The following [abstract operations](#) are used to create and manage Jobs and Job Queues:

8.4.1 EnqueueJob (*queueName*, *job*, *arguments*)

The EnqueueJob abstract operation requires three arguments: *queueName*, *job*, and *arguments*. It performs the following steps:

1. **Assert:** [Type](#)(*queueName*) is String and its value is the name of a Job Queue recognized by this implementation.
2. **Assert:** *job* is the name of a Job.

3. **Assert**: *arguments* is a **List** that has the same number of elements as the number of parameters required by *job*.
4. Let *callerContext* be the **running execution context**.
5. Let *callerRealm* be *callerContext*'s **Realm**.
6. Let *callerScriptOrModule* be *callerContext*'s **ScriptOrModule**.
7. Let *pending* be PendingJob { **[[Job]]**: *job*, **[[Arguments]]**: *arguments*, **[[Realm]]**: *callerRealm*, **[[ScriptOrModule]]**: *callerScriptOrModule*, **[[HostDefined]]**: **undefined** }.
8. Perform any implementation or host environment defined processing of *pending*. This may include modifying the **[[HostDefined]]** field or any other field of *pending*.
9. Add *pending* at the back of the Job Queue named by *queueName*.
10. Return **NormalCompletion**(empty).

8.5 InitializeHostDefinedRealm ()

The abstract operation InitializeHostDefinedRealm performs the following steps:

1. Let *realm* be **CreateRealm**().
2. Let *newContext* be a new **execution context**.
3. Set the Function of *newContext* to **null**.
4. Set the **Realm** of *newContext* to *realm*.
5. Set the **ScriptOrModule** of *newContext* to **null**.
6. Push *newContext* onto the **execution context stack**; *newContext* is now the **running execution context**.
7. If the host requires use of an **exotic object** to serve as *realm*'s **global object**, let *global* be such an object created in an implementation-defined manner. Otherwise, let *global* be **undefined**, indicating that an ordinary object should be created as the **global object**.
8. If the host requires that the **this** binding in *realm*'s global scope return an object other than the **global object**, let *thisValue* be such an object created in an implementation-defined manner. Otherwise, let *thisValue* be **undefined**, indicating that *realm*'s global **this** binding should be the **global object**.
9. Perform **SetRealmGlobalObject**(*realm*, *global*, *thisValue*).
10. Let *globalObj* be ? **SetDefaultGlobalBindings**(*realm*).
11. Create any implementation-defined **global object** properties on *globalObj*.
12. Return **NormalCompletion**(empty).

8.6 RunJobs ()

The abstract operation RunJobs performs the following steps:

1. Perform ? **InitializeHostDefinedRealm**().
2. In an implementation-dependent manner, obtain the ECMAScript source texts (see clause 10) and any associated host-defined values for zero or more ECMAScript scripts and/or ECMAScript modules. For each such *sourceText* and *hostDefined*, do
 - a. If *sourceText* is the source code of a script, then
 - i. Perform **EnqueueJob**("ScriptJobs", **ScriptEvaluationJob**, « *sourceText*, *hostDefined* »).
 - b. Else *sourceText* is the source code of a module,
 - i. Perform **EnqueueJob**("ScriptJobs", **TopLevelModuleEvaluationJob**, « *sourceText*, *hostDefined* »).
3. Repeat,
 - a. **Suspend** the **running execution context** and remove it from the **execution context stack**.
 - b. **Assert**: The **execution context stack** is now empty.

- c. Let *nextQueue* be a non-empty Job Queue chosen in an implementation-defined manner. If all Job Queues are empty, the result is implementation-defined.
- d. Let *nextPending* be the PendingJob record at the front of *nextQueue*. Remove that record from *nextQueue*.
- e. Let *newContext* be a new *execution context*.
- f. Set *newContext*'s Function to **null**.
- g. Set *newContext*'s Realm to *nextPending*.[[Realm]].
- h. Set *newContext*'s ScriptOrModule to *nextPending*.[[ScriptOrModule]].
- i. Push *newContext* onto the *execution context stack*; *newContext* is now the *running execution context*.
- j. Perform any implementation or host environment defined job initialization using *nextPending*.
- k. Let *result* be the result of performing the abstract operation named by *nextPending*.[[Job]] using the elements of *nextPending*.[[Arguments]] as its arguments.
- l. If *result* is an *abrupt completion*, perform *HostReportErrors*(« *result*.[[Value]] »).

8.7 Agents

An agent comprises a set of ECMAScript execution contexts, an *execution context stack*, a *running execution context*, a set of named job queues, an Agent Record, and an executing thread. Except for the *executing thread*, the constituents of an *agent* belong exclusively to that *agent*.

An *agent*'s *executing thread* executes the jobs in the *agent*'s job queues on the *agent*'s execution contexts independently of other agents, except that an *executing thread* may be used as the *executing thread* by multiple agents, provided none of the agents sharing the thread have an *Agent Record* whose [[CanBlock]] property is **true**.

NOTE 1

Some web browsers share a single *executing thread* across multiple unrelated tabs of a browser window, for example.

While an *agent*'s *executing thread* executes the jobs in the *agent*'s job queues, the *agent* is the surrounding agent for the code in those jobs. The code uses the *surrounding agent* to access the specification level execution objects held within the *agent*: the *running execution context*, the *execution context stack*, the named job queues, and the *Agent Record*'s fields.

Table 26: *Agent Record* Fields

Field Name	Value	Meaning
[[LittleEndian]]	Boolean	The default value computed for the <i>isLittleEndian</i> parameter when it is needed by the algorithms <i>GetValueFromBuffer</i> and <i>SetValueInBuffer</i> . The choice is implementation-dependent and should be the alternative that is most efficient for the implementation. Once the value has been observed it cannot change.
[[CanBlock]]	Boolean	Determines whether the <i>agent</i> can block or not.
[[Signifier]]	Any globally-unique value	Uniquely identifies the <i>agent</i> within its <i>agent cluster</i> .
[[IsLockFree1]]	Boolean	true if atomic operations on one-byte values are lock-free, false otherwise.
[[IsLockFree2]]	Boolean	true if atomic operations on two-byte values are lock-free, false otherwise.

[[CandidateExecution]] A See the [memory model](#).
candidate
execution
Record

Once the values of [[Signifier]], [[IsLockFree1]], and [[IsLockFree2]] have been observed by any [agent](#) in the [agent cluster](#) they cannot change.

NOTE 2

The values of [[IsLockFree1]] and [[IsLockFree2]] are not necessarily determined by the hardware, but may also reflect implementation choices that can vary over time and between ECMAScript implementations.

There is no [[IsLockFree4]] property: 4-byte atomic operations are always lock-free.

In practice, if an atomic operation is implemented with any type of lock the operation is not lock-free. Lock-free does not imply wait-free: there is no upper bound on how many machine steps may be required to complete a lock-free atomic operation.

That an atomic access of size n is lock-free does not imply anything about the (perceived) atomicity of non-atomic accesses of size n , specifically, non-atomic accesses may still be performed as a sequence of several separate memory accesses. See [ReadSharedMemory](#) and [WriteSharedMemory](#) for details.

NOTE 3

An [agent](#) is a specification mechanism and need not correspond to any particular artefact of an ECMAScript implementation.

8.7.1 AgentSignifier ()

The abstract operation AgentSignifier takes no arguments. It performs the following steps:

1. Let [AR](#) be the [Agent Record](#) of the [surrounding agent](#).
2. Return [AR](#).[[Signifier]].

8.7.2 AgentCanSuspend ()

The abstract operation AgentCanSuspend takes no arguments. It performs the following steps:

1. Let [AR](#) be the [Agent Record](#) of the [surrounding agent](#).
2. Return [AR](#).[[CanBlock]].

NOTE

In some environments it may not be reasonable for a given [agent](#) to suspend. For example, in a web browser environment, it may be reasonable to disallow suspending a document's main event handling thread, while still allowing workers' event handling threads to suspend.

8.8 Agent Clusters

An agent cluster is a maximal set of agents that can communicate by operating on shared memory.

NOTE 1

Programs within different agents may share memory by unspecified means. At a minimum, the backing memory for `SharedArrayBuffer` objects can be shared among the agents in the cluster.

There may be agents that can communicate by message passing that cannot share memory; they are never in the same agent cluster.

Every [agent](#) belongs to exactly one agent cluster.

NOTE 2

The agents in a cluster need not all be alive at some particular point in time. If [agent A](#) creates another [agent B](#), after which **A** terminates and **B** creates [agent C](#), the three agents are in the same cluster if **A** could share some memory with **B** and **B** could share some memory with **C**.

All agents within a cluster must have the same value for the `[[LittleEndian]]` property in their respective [Agent Records](#).

NOTE 3

If different agents within an agent cluster have different values of `[[LittleEndian]]` it becomes hard to use shared memory for multi-byte data.

All agents within a cluster must have the same values for the `[[IsLockFree1]]` property in their respective [Agent Records](#); similarly for the `[[IsLockFree2]]` property.

All agents within a cluster must have different values for the `[[Signifier]]` property in their respective [Agent Records](#).

An embedding may deactivate (stop forward progress) or activate (resume forward progress) an [agent](#) without the [agent's](#) knowledge or cooperation. If the embedding does so, it must not leave some agents in the cluster active while other agents in the cluster are deactivated indefinitely.

NOTE 4

The purpose of the preceding restriction is to avoid a situation where an [agent](#) deadlocks or starves because another [agent](#) has been deactivated. For example, if an HTML shared worker that has a lifetime independent of documents in any windows were allowed to share memory with the dedicated worker of such an independent document, and the document and its dedicated worker were to be deactivated while the dedicated worker holds a lock (say, the document is pushed into its window's history), and the shared worker then tries to acquire the lock, then the shared worker will be blocked until the dedicated worker is activated again, if ever. Meanwhile other workers trying to access the shared worker from other windows will starve.

The implication of the restriction is that it will not be possible to share memory between agents that don't belong to the same suspend/wake collective within the embedding.

An embedding may terminate an [agent](#) without any of the [agent's](#) cluster's other agents' prior knowledge or cooperation. If an [agent](#) is terminated not by programmatic action of its own or of another [agent](#) in the cluster but by forces external to the cluster, then the embedding must choose one of two strategies: Either terminate all the agents in the cluster, or provide reliable APIs that allow the agents in the cluster to coordinate so that at least one remaining member of the cluster will be able to detect the termination, with the termination data containing enough information to identify the

[agent](#) that was terminated.

NOTE 5

Examples of that type of termination are: operating systems or users terminating agents that are running in separate processes; the embedding itself terminating an [agent](#) that is running in-process with the other agents when per-[agent](#) resource accounting indicates that the [agent](#) is runaway.

Prior to any evaluation of any ECMAScript code by any [agent](#) in a cluster, the `[[CandidateExecution]]` field of the [Agent Record](#) for all agents in the cluster is set to the initial [candidate execution](#). The initial [candidate execution](#) is an [empty candidate execution](#) whose `[[EventsRecords]]` field is a [List](#) containing, for each [agent](#), an [Agent Events Record](#) whose `[[AgentSignifier]]` field is that [agent](#)'s signifier, and whose `[[EventList]]` and `[[AgentSynchronizesWith]]` fields are empty Lists.

NOTE 6

All agents in an agent cluster share the same [candidate execution](#) in its [Agent Record](#)'s `[[CandidateExecution]]` field. The [candidate execution](#) is a specification mechanism used by the [memory model](#).

NOTE 7

An agent cluster is a specification mechanism and need not correspond to any particular artefact of an ECMAScript implementation.

8.9 Forward Progress

For an [agent](#) to *make forward progress* is for it to perform an evaluation step according to this specification.

An [agent](#) becomes *blocked* when its [running execution context](#) waits synchronously and indefinitely for an external event. Only agents whose [Agent Record](#)'s `[[CanBlock]]` property is **true** can become blocked in this sense. An *unblocked agent* is one that is not blocked.

Implementations must ensure that:

- every unblocked [agent](#) with a dedicated [executing thread](#) eventually makes forward progress
- in a set of agents that share an [executing thread](#), one [agent](#) eventually makes forward progress
- an [agent](#) does not cause another [agent](#) to become blocked except via explicit APIs that provide blocking.

NOTE

This, along with the liveness guarantee in the [memory model](#), ensures that all "**SeqCst**" writes eventually become observable to all agents.

9 Ordinary and Exotic Objects Behaviours

9.1 Ordinary Object Internal Methods and Internal Slots

All ordinary objects have an internal slot called `[[Prototype]]`. The value of this internal slot is either **null** or an object and is used for implementing inheritance. Data properties of the `[[Prototype]]` object are inherited (and visible as

properties of the child object) for the purposes of get access, but not for set access. Accessor properties are inherited for both get access and set access.

Every ordinary object has a Boolean-valued `[[Extensible]]` internal slot which is used to fulfill the extensibility-related internal method invariants specified in 6.1.7.3. Namely, once the value of an object's `[[Extensible]]` internal slot has been set to **false**, it is no longer possible to add properties to the object, to modify the value of the object's `[[Prototype]]` internal slot, or to subsequently change the value of `[[Extensible]]` to **true**.

In the following algorithm descriptions, assume *O* is an ordinary object, *P* is a property key value, *V* is any ECMAScript language value, and *Desc* is a Property Descriptor record.

Each ordinary object internal method delegates to a similarly-named abstract operation. If such an abstract operation depends on another internal method, then the internal method is invoked on *O* rather than calling the similarly-named abstract operation directly. These semantics ensure that exotic objects have their overridden internal methods invoked when ordinary object internal methods are applied to them.

9.1.1 `[[GetPrototypeOf]]` ()

When the `[[GetPrototypeOf]]` internal method of *O* is called, the following steps are taken:

1. Return ! `OrdinaryGetPrototypeOf(O)`.

9.1.1.1 `OrdinaryGetPrototypeOf` (*O*)

When the abstract operation `OrdinaryGetPrototypeOf` is called with Object *O*, the following steps are taken:

1. Return *O*.`[[Prototype]]`.

9.1.2 `[[SetPrototypeOf]]` (*V*)

When the `[[SetPrototypeOf]]` internal method of *O* is called with argument *V*, the following steps are taken:

1. Return ! `OrdinarySetPrototypeOf(O, V)`.

9.1.2.1 `OrdinarySetPrototypeOf` (*O*, *V*)

When the abstract operation `OrdinarySetPrototypeOf` is called with Object *O* and value *V*, the following steps are taken:

1. **Assert**: Either `Type(V)` is Object or `Type(V)` is Null.
2. Let *extensible* be *O*.`[[Extensible]]`.
3. Let *current* be *O*.`[[Prototype]]`.
4. If `SameValue(V, current)` is **true**, return **true**.
5. If *extensible* is **false**, return **false**.
6. Let *p* be *V*.
7. Let *done* be **false**.
8. Repeat, while *done* is **false**,
 - a. If *p* is **null**, set *done* to **true**.
 - b. Else if `SameValue(p, O)` is **true**, return **false**.
 - c. Else,
 - i. If *p*.`[[GetPrototypeOf]]` is not the ordinary object internal method defined in 9.1.1, set *done* to **true**.
 - ii. Else, set *p* to *p*.`[[Prototype]]`.

9. Set $O.[[Prototype]]$ to V .
10. Return **true**.

NOTE

The loop in step 8 guarantees that there will be no circularities in any prototype chain that only includes objects that use the ordinary object definitions for $[[GetPrototypeOf]]$ and $[[SetPrototypeOf]]$.

9.1.3 $[[IsExtensible]]$ ()

When the $[[IsExtensible]]$ internal method of O is called, the following steps are taken:

1. Return ! $\text{OrdinaryIsExtensible}(O)$.

9.1.3.1 $\text{OrdinaryIsExtensible}$ (O)

When the abstract operation $\text{OrdinaryIsExtensible}$ is called with Object O , the following steps are taken:

1. Return $O.[[Extensible]]$.

9.1.4 $[[PreventExtensions]]$ ()

When the $[[PreventExtensions]]$ internal method of O is called, the following steps are taken:

1. Return ! $\text{OrdinaryPreventExtensions}(O)$.

9.1.4.1 $\text{OrdinaryPreventExtensions}$ (O)

When the abstract operation $\text{OrdinaryPreventExtensions}$ is called with Object O , the following steps are taken:

1. Set $O.[[Extensible]]$ to **false**.
2. Return **true**.

9.1.5 $[[GetOwnProperty]]$ (P)

When the $[[GetOwnProperty]]$ internal method of O is called with property key P , the following steps are taken:

1. Return ! $\text{OrdinaryGetOwnProperty}(O, P)$.

9.1.5.1 $\text{OrdinaryGetOwnProperty}$ (O, P)

When the abstract operation $\text{OrdinaryGetOwnProperty}$ is called with Object O and with property key P , the following steps are taken:

1. **Assert:** $\text{IsPropertyKey}(P)$ is **true**.
2. If O does not have an own property with key P , return **undefined**.
3. Let D be a newly created **Property Descriptor** with no fields.
4. Let X be O 's own property whose key is P .
5. If X is a **data property**, then
 - a. Set $D.[[Value]]$ to the value of X 's $[[Value]]$ attribute.
 - b. Set $D.[[Writable]]$ to the value of X 's $[[Writable]]$ attribute.

6. Else *X* is an **accessor property**,
 - a. Set *D*.[[Get]] to the value of *X*'s [[Get]] attribute.
 - b. Set *D*.[[Set]] to the value of *X*'s [[Set]] attribute.
7. Set *D*.[[Enumerable]] to the value of *X*'s [[Enumerable]] attribute.
8. Set *D*.[[Configurable]] to the value of *X*'s [[Configurable]] attribute.
9. Return *D*.

9.1.6 [[DefineOwnProperty]] (*P*, *Desc*)

When the [[DefineOwnProperty]] internal method of *O* is called with property key *P* and **Property Descriptor** *Desc*, the following steps are taken:

1. Return ? **OrdinaryDefineOwnProperty**(*O*, *P*, *Desc*).

9.1.6.1 OrdinaryDefineOwnProperty (*O*, *P*, *Desc*)

When the abstract operation OrdinaryDefineOwnProperty is called with Object *O*, property key *P*, and **Property Descriptor** *Desc*, the following steps are taken:

1. Let *current* be ? *O*.[[GetOwnProperty]](*P*).
2. Let *extensible* be ? **IsExtensible**(*O*).
3. Return **ValidateAndApplyPropertyDescriptor**(*O*, *P*, *extensible*, *Desc*, *current*).

9.1.6.2 IsCompatiblePropertyDescriptor (*Extensible*, *Desc*, *Current*)

When the abstract operation IsCompatiblePropertyDescriptor is called with Boolean value *Extensible*, and Property Descriptors *Desc*, and *Current*, the following steps are taken:

1. Return **ValidateAndApplyPropertyDescriptor**(**undefined**, **undefined**, *Extensible*, *Desc*, *Current*).

9.1.6.3 ValidateAndApplyPropertyDescriptor (*O*, *P*, *extensible*, *Desc*, *current*)

When the abstract operation ValidateAndApplyPropertyDescriptor is called with Object *O*, property key *P*, Boolean value *extensible*, and Property Descriptors *Desc*, and *current*, the following steps are taken:

NOTE

If **undefined** is passed as *O*, only validation is performed and no object updates are performed.

1. **Assert**: If *O* is not **undefined**, then **IsPropertyKey**(*P*) is **true**.
2. If *current* is **undefined**, then
 - a. If *extensible* is **false**, return **false**.
 - b. **Assert**: *extensible* is **true**.
 - c. If **IsGenericDescriptor**(*Desc*) is **true** or **IsDataDescriptor**(*Desc*) is **true**, then
 - i. If *O* is not **undefined**, create an own **data property** named *P* of object *O* whose [[Value]], [[Writable]], [[Enumerable]] and [[Configurable]] attribute values are described by *Desc*. If the value of an attribute field of *Desc* is absent, the attribute of the newly created property is set to its default value.
 - d. Else *Desc* must be an accessor **Property Descriptor**,
 - i. If *O* is not **undefined**, create an own **accessor property** named *P* of object *O* whose [[Get]], [[Set]], [[Enumerable]] and [[Configurable]] attribute values are described by *Desc*. If the value of an attribute

field of *Desc* is absent, the attribute of the newly created property is set to its default value.

- e. Return **true**.
3. If every field in *Desc* is absent, return **true**.
4. If *current*.[[Configurable]] is **false**, then
 - a. If *Desc*.[[Configurable]] is present and its value is **true**, return **false**.
 - b. If *Desc*.[[Enumerable]] is present and the [[Enumerable]] fields of *current* and *Desc* are the Boolean negation of each other, return **false**.
5. If *IsGenericDescriptor*(*Desc*) is **true**, no further validation is required.
6. Else if *IsDataDescriptor*(*current*) and *IsDataDescriptor*(*Desc*) have different results, then
 - a. If *current*.[[Configurable]] is **false**, return **false**.
 - b. If *IsDataDescriptor*(*current*) is **true**, then
 - i. If *O* is not **undefined**, convert the property named *P* of object *O* from a **data property** to an **accessor property**. Preserve the existing values of the converted property's [[Configurable]] and [[Enumerable]] attributes and set the rest of the property's attributes to their default values.
 - c. Else,
 - i. If *O* is not **undefined**, convert the property named *P* of object *O* from an **accessor property** to a **data property**. Preserve the existing values of the converted property's [[Configurable]] and [[Enumerable]] attributes and set the rest of the property's attributes to their default values.
7. Else if *IsDataDescriptor*(*current*) and *IsDataDescriptor*(*Desc*) are both **true**, then
 - a. If *current*.[[Configurable]] is **false** and *current*.[[Writable]] is **false**, then
 - i. If *Desc*.[[Writable]] is present and *Desc*.[[Writable]] is **true**, return **false**.
 - ii. If *Desc*.[[Value]] is present and *SameValue*(*Desc*.[[Value]], *current*.[[Value]]) is **false**, return **false**.
 - iii. Return **true**.
8. Else *IsAccessorDescriptor*(*current*) and *IsAccessorDescriptor*(*Desc*) are both **true**,
 - a. If *current*.[[Configurable]] is **false**, then
 - i. If *Desc*.[[Set]] is present and *SameValue*(*Desc*.[[Set]], *current*.[[Set]]) is **false**, return **false**.
 - ii. If *Desc*.[[Get]] is present and *SameValue*(*Desc*.[[Get]], *current*.[[Get]]) is **false**, return **false**.
 - iii. Return **true**.
9. If *O* is not **undefined**, then
 - a. For each field of *Desc* that is present, set the corresponding attribute of the property named *P* of object *O* to the value of the field.
10. Return **true**.

9.1.7 [[HasProperty]] (*P*)

When the [[HasProperty]] internal method of *O* is called with property key *P*, the following steps are taken:

1. Return ? *OrdinaryHasProperty*(*O*, *P*).

9.1.7.1 OrdinaryHasProperty (*O*, *P*)

When the abstract operation *OrdinaryHasProperty* is called with Object *O* and with property key *P*, the following steps are taken:

1. **Assert**: *IsPropertyKey*(*P*) is **true**.
2. Let *hasOwn* be ? *O*.[[GetOwnProperty]](*P*).
3. If *hasOwn* is not **undefined**, return **true**.
4. Let *parent* be ? *O*.[[GetPrototypeOf]]().
5. If *parent* is not **null**, then

- a. Return ? *parent*.[[HasProperty]](*P*).
6. Return **false**.

9.1.8 [[Get]] (*P*, *Receiver*)

When the [[Get]] internal method of *O* is called with property key *P* and ECMAScript language value *Receiver*, the following steps are taken:

1. Return ? OrdinaryGet(*O*, *P*, *Receiver*).

9.1.8.1 OrdinaryGet (*O*, *P*, *Receiver*)

When the abstract operation OrdinaryGet is called with Object *O*, property key *P*, and ECMAScript language value *Receiver*, the following steps are taken:

1. Assert: IsPropertyKey(*P*) is **true**.
2. Let *desc* be ? *O*.[[GetOwnProperty]](*P*).
3. If *desc* is **undefined**, then
 - a. Let *parent* be ? *O*.[[GetPrototypeOf]]().
 - b. If *parent* is **null**, return **undefined**.
 - c. Return ? *parent*.[[Get]](*P*, *Receiver*).
4. If IsDataDescriptor(*desc*) is **true**, return *desc*.[[Value]].
5. Assert: IsAccessorDescriptor(*desc*) is **true**.
6. Let *getter* be *desc*.[[Get]].
7. If *getter* is **undefined**, return **undefined**.
8. Return ? Call(*getter*, *Receiver*).

9.1.9 [[Set]] (*P*, *V*, *Receiver*)

When the [[Set]] internal method of *O* is called with property key *P*, value *V*, and ECMAScript language value *Receiver*, the following steps are taken:

1. Return ? OrdinarySet(*O*, *P*, *V*, *Receiver*).

9.1.9.1 OrdinarySet (*O*, *P*, *V*, *Receiver*)

When the abstract operation OrdinarySet is called with Object *O*, property key *P*, value *V*, and ECMAScript language value *Receiver*, the following steps are taken:

1. Assert: IsPropertyKey(*P*) is **true**.
2. Let *ownDesc* be ? *O*.[[GetOwnProperty]](*P*).
3. Return OrdinarySetWithOwnDescriptor(*O*, *P*, *V*, *Receiver*, *ownDesc*).

9.1.9.2 OrdinarySetWithOwnDescriptor (*O*, *P*, *V*, *Receiver*, *ownDesc*)

When the abstract operation OrdinarySetWithOwnDescriptor is called with Object *O*, property key *P*, value *V*, ECMAScript language value *Receiver*, and Property Descriptor (or **undefined**) *ownDesc*, the following steps are taken:

1. Assert: IsPropertyKey(*P*) is **true**.
2. If *ownDesc* is **undefined**, then

- a. Let *parent* be ? *O*.[[GetPrototypeOf]]().
- b. If *parent* is not **null**, then
 - i. Return ? *parent*.[[Set]](*P*, *V*, *Receiver*).
- c. Else,
 - i. Set *ownDesc* to the PropertyDescriptor { [[Value]]: **undefined**, [[Writable]]: **true**, [[Enumerable]]: **true**, [[Configurable]]: **true** }.
3. If *IsDataDescriptor*(*ownDesc*) is **true**, then
 - a. If *ownDesc*.[[Writable]] is **false**, return **false**.
 - b. If *Type*(*Receiver*) is not Object, return **false**.
 - c. Let *existingDescriptor* be ? *Receiver*.[[GetOwnProperty]](*P*).
 - d. If *existingDescriptor* is not **undefined**, then
 - i. If *IsAccessorDescriptor*(*existingDescriptor*) is **true**, return **false**.
 - ii. If *existingDescriptor*.[[Writable]] is **false**, return **false**.
 - iii. Let *valueDesc* be the PropertyDescriptor { [[Value]]: *V* }.
 - iv. Return ? *Receiver*.[[DefineOwnProperty]](*P*, *valueDesc*).
 - e. Else *Receiver* does not currently have a property *P*,
 - i. Return ? *CreateDataProperty*(*Receiver*, *P*, *V*).
4. **Assert**: *IsAccessorDescriptor*(*ownDesc*) is **true**.
5. Let *setter* be *ownDesc*.[[Set]].
6. If *setter* is **undefined**, return **false**.
7. Perform ? *Call*(*setter*, *Receiver*, « *V* »).
8. Return **true**.

9.1.10 [[Delete]] (*P*)

When the [[Delete]] internal method of *O* is called with property key *P*, the following steps are taken:

1. Return ? *OrdinaryDelete*(*O*, *P*).

9.1.10.1 OrdinaryDelete (*O*, *P*)

When the abstract operation OrdinaryDelete is called with Object *O* and property key *P*, the following steps are taken:

1. **Assert**: *IsPropertyKey*(*P*) is **true**.
2. Let *desc* be ? *O*.[[GetOwnProperty]](*P*).
3. If *desc* is **undefined**, return **true**.
4. If *desc*.[[Configurable]] is **true**, then
 - a. Remove the own property with name *P* from *O*.
 - b. Return **true**.
5. Return **false**.

9.1.11 [[OwnPropertyKeys]] ()

When the [[OwnPropertyKeys]] internal method of *O* is called, the following steps are taken:

1. Return ! *OrdinaryOwnPropertyKeys*(*O*).

9.1.11.1 OrdinaryOwnPropertyKeys (*O*)

When the abstract operation OrdinaryOwnPropertyKeys is called with Object *O*, the following steps are taken:

1. Let *keys* be a new empty List.
2. For each own property key *P* of *O* that is an array index, in ascending numeric index order, do
 - a. Add *P* as the last element of *keys*.
3. For each own property key *P* of *O* that is a String but is not an array index, in ascending chronological order of property creation, do
 - a. Add *P* as the last element of *keys*.
4. For each own property key *P* of *O* that is a Symbol, in ascending chronological order of property creation, do
 - a. Add *P* as the last element of *keys*.
5. Return *keys*.

9.1.12 ObjectCreate (*proto* [, *internalSlotsList*])

The abstract operation ObjectCreate with argument *proto* (an object or null) is used to specify the runtime creation of new ordinary objects. The optional argument *internalSlotsList* is a List of the names of additional internal slots that must be defined as part of the object. If the list is not provided, a new empty List is used. This abstract operation performs the following steps:

1. If *internalSlotsList* is not present, set *internalSlotsList* to a new empty List.
2. Let *obj* be a newly created object with an internal slot for each name in *internalSlotsList*.
3. Set *obj*'s essential internal methods to the default ordinary object definitions specified in 9.1.
4. Set *obj*.[[Prototype]] to *proto*.
5. Set *obj*.[[Extensible]] to **true**.
6. Return *obj*.

9.1.13 OrdinaryCreateFromConstructor (*constructor*, *intrinsicDefaultProto* [, *internalSlotsList*])

The abstract operation OrdinaryCreateFromConstructor creates an ordinary object whose [[Prototype]] value is retrieved from a *constructor*'s **prototype** property, if it exists. Otherwise the intrinsic named by *intrinsicDefaultProto* is used for [[Prototype]]. The optional *internalSlotsList* is a List of the names of additional internal slots that must be defined as part of the object. If the list is not provided, a new empty List is used. This abstract operation performs the following steps:

1. **Assert:** *intrinsicDefaultProto* is a String value that is this specification's name of an intrinsic object. The corresponding object must be an intrinsic that is intended to be used as the [[Prototype]] value of an object.
2. Let *proto* be ? GetPrototypeFromConstructor(*constructor*, *intrinsicDefaultProto*).
3. Return ObjectCreate(*proto*, *internalSlotsList*).

9.1.14 GetPrototypeFromConstructor (*constructor*, *intrinsicDefaultProto*)

The abstract operation GetPrototypeFromConstructor determines the [[Prototype]] value that should be used to create an object corresponding to a specific *constructor*. The value is retrieved from the *constructor*'s **prototype** property, if it exists. Otherwise the intrinsic named by *intrinsicDefaultProto* is used for [[Prototype]]. This abstract operation performs the following steps:

1. **Assert:** *intrinsicDefaultProto* is a String value that is this specification's name of an intrinsic object. The corresponding object must be an intrinsic that is intended to be used as the [[Prototype]] value of an object.

2. **Assert:** `IsCallable(constructor)` is **true**.
3. Let *proto* be `? Get(constructor, "prototype")`.
4. If `Type(proto)` is not `Object`, then
 - a. Let *realm* be `? GetFunctionRealm(constructor)`.
 - b. Set *proto* to *realm*'s intrinsic object named `intrinsicDefaultProto`.
5. Return *proto*.

NOTE

If *constructor* does not supply a `[[Prototype]]` value, the default value that is used is obtained from the *realm* of the *constructor* function rather than from the *running execution context*.

9.2 ECMAScript Function Objects

ECMAScript function objects encapsulate parameterized ECMAScript code closed over a lexical environment and support the dynamic evaluation of that code. An ECMAScript *function object* is an ordinary object and has the same internal slots and the same internal methods as other ordinary objects. The code of an ECMAScript *function object* may be either *strict mode code* (10.2.1) or *non-strict code*. An ECMAScript *function object* whose code is *strict mode code* is called a strict function. One whose code is not *strict mode code* is called a non-strict function.

ECMAScript function objects have the additional internal slots listed in Table 27.

Table 27: Internal Slots of ECMAScript Function Objects

Internal Slot	Type	Description
<code>[[Environment]]</code>	<i>Lexical Environment</i>	The <i>Lexical Environment</i> that the function was closed over. Used as the outer environment when evaluating the code of the function.
<code>[[FormalParameters]]</code>	<i>Parse Node</i>	The root parse node of the source text that defines the function's formal parameter list.
<code>[[FunctionKind]]</code>	String	Either "normal" , "classConstructor" , "generator" , "async" , or "async generator" .
<code>[[ECMAScriptCode]]</code>	<i>Parse Node</i>	The root parse node of the source text that defines the function's body.
<code>[[ConstructorKind]]</code>	String	Either "base" or "derived" .
<code>[[Realm]]</code>	<i>Realm Record</i>	The <i>realm</i> in which the function was created and which provides any intrinsic objects that are accessed when evaluating the function.
<code>[[ScriptOrModule]]</code>	<i>Script Record</i> or <i>Module Record</i>	The script or module in which the function was created.
<code>[[ThisMode]]</code>	(lexical, strict, global)	Defines how this references are interpreted within the formal parameters and code body of the function. lexical means that this refers to the this value of a lexically enclosing function. strict means that the this value is used exactly as provided by an invocation of the function. global means that a this value of undefined is interpreted as a reference to the <i>global object</i> .

[[Strict]]	Boolean	true if this is a strict function , false if this is a non-strict function .
[[HomeObject]]	Object	If the function uses super , this is the object whose [[GetPrototypeOf]] provides the object where super property lookups begin.
[[SourceText]]	String	The source text that defines the function.

All ECMAScript function objects have the [[Call]] internal method defined here. ECMAScript functions that are also constructors in addition have the [[Construct]] internal method.

9.2.1 [[Call]] (*thisArgument*, *argumentsList*)

The [[Call]] internal method for an ECMAScript [function object](#) *F* is called with parameters *thisArgument* and *argumentsList*, a [List](#) of ECMAScript language values. The following steps are taken:

1. **Assert**: *F* is an ECMAScript [function object](#).
2. If *F*.[[FunctionKind]] is "**classConstructor**", throw a **TypeError** exception.
3. Let *callerContext* be the [running execution context](#).
4. Let *calleeContext* be [PrepareForOrdinaryCall](#)(*F*, **undefined**).
5. **Assert**: *calleeContext* is now the [running execution context](#).
6. Perform [OrdinaryCallBindThis](#)(*F*, *calleeContext*, *thisArgument*).
7. Let *result* be [OrdinaryCallEvaluateBody](#)(*F*, *argumentsList*).
8. Remove *calleeContext* from the [execution context stack](#) and restore *callerContext* as the [running execution context](#).
9. If *result*.[[Type]] is **return**, return [NormalCompletion](#)(*result*.[[Value]]).
10. [ReturnIfAbrupt](#)(*result*).
11. Return [NormalCompletion](#)(**undefined**).

NOTE

When *calleeContext* is removed from the [execution context stack](#) in step 8 it must not be destroyed if it is suspended and retained for later resumption by an accessible generator object.

9.2.1.1 PrepareForOrdinaryCall (*F*, *newTarget*)

When the abstract operation [PrepareForOrdinaryCall](#) is called with [function object](#) *F* and ECMAScript language value *newTarget*, the following steps are taken:

1. **Assert**: [Type](#)(*newTarget*) is **Undefined** or **Object**.
2. Let *callerContext* be the [running execution context](#).
3. Let *calleeContext* be a new ECMAScript code [execution context](#).
4. Set the [Function](#) of *calleeContext* to *F*.
5. Let *calleeRealm* be *F*.[[Realm]].
6. Set the [Realm](#) of *calleeContext* to *calleeRealm*.
7. Set the [ScriptOrModule](#) of *calleeContext* to *F*.[[ScriptOrModule]].
8. Let *localEnv* be [NewFunctionEnvironment](#)(*F*, *newTarget*).
9. Set the [LexicalEnvironment](#) of *calleeContext* to *localEnv*.
10. Set the [VariableEnvironment](#) of *calleeContext* to *localEnv*.
11. If *callerContext* is not already suspended, suspend *callerContext*.

12. Push *calleeContext* onto the *execution context stack*; *calleeContext* is now the *running execution context*.
13. NOTE: Any exception objects produced after this point are associated with *calleeRealm*.
14. Return *calleeContext*.

9.2.1.2 OrdinaryCallBindThis (*F*, *calleeContext*, *thisArgument*)

When the abstract operation OrdinaryCallBindThis is called with *function object F*, *execution context calleeContext*, and ECMAScript value *thisArgument*, the following steps are taken:

1. Let *thisMode* be *F*.[[ThisMode]].
2. If *thisMode* is *lexical*, return *NormalCompletion(undefined)*.
3. Let *calleeRealm* be *F*.[[Realm]].
4. Let *localEnv* be the *LexicalEnvironment* of *calleeContext*.
5. If *thisMode* is *strict*, let *thisValue* be *thisArgument*.
6. Else,
 - a. If *thisArgument* is *undefined* or *null*, then
 - i. Let *globalEnv* be *calleeRealm*.[[GlobalEnv]].
 - ii. Let *globalEnvRec* be *globalEnv*'s *EnvironmentRecord*.
 - iii. *Assert*: *globalEnvRec* is a global *Environment Record*.
 - iv. Let *thisValue* be *globalEnvRec*.[[GlobalThisValue]].
 - b. Else,
 - i. Let *thisValue* be ! *ToObject(thisArgument)*.
 - ii. NOTE: *ToObject* produces wrapper objects using *calleeRealm*.
7. Let *envRec* be *localEnv*'s *EnvironmentRecord*.
8. *Assert*: *envRec* is a *function Environment Record*.
9. *Assert*: The next step never returns an *abrupt completion* because *envRec*.[[ThisBindingStatus]] is not *"initialized"*.
10. Return *envRec*.BindThisValue(*thisValue*).

9.2.1.3 OrdinaryCallEvaluateBody (*F*, *argumentsList*)

When the abstract operation OrdinaryCallEvaluateBody is called with *function object F* and *List argumentsList*, the following steps are taken:

1. Return the result of EvaluateBody of the parsed code that is *F*.[[ECMAScriptCode]] passing *F* and *argumentsList* as the arguments.

9.2.2 [[Construct]] (*argumentsList*, *newTarget*)

The [[Construct]] internal method for an ECMAScript *function object F* is called with parameters *argumentsList* and *newTarget*. *argumentsList* is a possibly empty *List* of ECMAScript language values. The following steps are taken:

1. *Assert*: *F* is an ECMAScript *function object*.
2. *Assert*: *Type(newTarget)* is *Object*.
3. Let *callerContext* be the *running execution context*.
4. Let *kind* be *F*.[[ConstructorKind]].
5. If *kind* is *"base"*, then
 - a. Let *thisArgument* be ? *OrdinaryCreateFromConstructor(newTarget, "%ObjectPrototype%")*.
6. Let *calleeContext* be *PrepareForOrdinaryCall(F, newTarget)*.

7. **Assert:** *calleeContext* is now the **running execution context**.
8. If *kind* is **"base"**, perform **OrdinaryCallBindThis**(*F*, *calleeContext*, *thisArgument*).
9. Let *constructorEnv* be the **LexicalEnvironment** of *calleeContext*.
10. Let *envRec* be *constructorEnv*'s **EnvironmentRecord**.
11. Let *result* be **OrdinaryCallEvaluateBody**(*F*, *argumentsList*).
12. Remove *calleeContext* from the **execution context stack** and restore *callerContext* as the **running execution context**.
13. If *result*.[[Type]] is **return**, then
 - a. If **Type**(*result*.[[Value]]) is **Object**, return **NormalCompletion**(*result*.[[Value]]).
 - b. If *kind* is **"base"**, return **NormalCompletion**(*thisArgument*).
 - c. If *result*.[[Value]] is not **undefined**, throw a **TypeError** exception.
14. Else, **ReturnIfAbrupt**(*result*).
15. Return ? *envRec*.GetThisBinding().

9.2.3 FunctionAllocate (*functionPrototype*, *strict*, *functionKind*)

The abstract operation **FunctionAllocate** requires the three arguments *functionPrototype*, *strict* and *functionKind*. **FunctionAllocate** performs the following steps:

1. **Assert:** **Type**(*functionPrototype*) is **Object**.
2. **Assert:** *functionKind* is either **"normal"**, **"non-constructor"**, **"generator"**, **"async"**, or **"async generator"**.
3. If *functionKind* is **"normal"**, let *needsConstruct* be **true**.
4. Else, let *needsConstruct* be **false**.
5. If *functionKind* is **"non-constructor"**, set *functionKind* to **"normal"**.
6. Let *F* be a newly created **ECMAScript function object** with the internal slots listed in [Table 27](#). All of those internal slots are initialized to **undefined**.
7. Set *F*'s essential internal methods to the default ordinary object definitions specified in [9.1](#).
8. Set *F*.[[Call]] to the definition specified in [9.2.1](#).
9. If *needsConstruct* is **true**, then
 - a. Set *F*.[[Construct]] to the definition specified in [9.2.2](#).
 - b. Set *F*.[[ConstructorKind]] to **"base"**.
10. Set *F*.[[Strict]] to *strict*.
11. Set *F*.[[FunctionKind]] to *functionKind*.
12. Set *F*.[[Prototype]] to *functionPrototype*.
13. Set *F*.[[Extensible]] to **true**.
14. Set *F*.[[Realm]] to the **current Realm Record**.
15. Return *F*.

9.2.4 FunctionInitialize (*F*, *kind*, *ParameterList*, *Body*, *Scope*)

The abstract operation **FunctionInitialize** requires the arguments: a **function object** *F*, *kind* which is one of (Normal, Method, Arrow), a parameter list **Parse Node** specified by *ParameterList*, a body **Parse Node** specified by *Body*, a **Lexical Environment** specified by *Scope*. **FunctionInitialize** performs the following steps:

1. Let *len* be the **ExpectedArgumentCount** of *ParameterList*.
2. Perform ! **SetFunctionLength**(*F*, *len*).
3. Let *Strict* be *F*.[[Strict]].
4. Set *F*.[[Environment]] to *Scope*.

5. Set $F.[[FormalParameters]]$ to *ParameterList*.
6. Set $F.[[ECMAScriptCode]]$ to *Body*.
7. Set $F.[[ScriptOrModule]]$ to *GetActiveScriptOrModule()*.
8. If *kind* is *Arrow*, set $F.[[ThisMode]]$ to *lexical*.
9. Else if *Strict* is **true**, set $F.[[ThisMode]]$ to *strict*.
10. Else, set $F.[[ThisMode]]$ to *global*.
11. Return F .

9.2.5 FunctionCreate (*kind*, *ParameterList*, *Body*, *Scope*, *Strict* [, *prototype*])

The abstract operation FunctionCreate requires the arguments: *kind* which is one of (Normal, Method, Arrow), a parameter list *Parse Node* specified by *ParameterList*, a body *Parse Node* specified by *Body*, a *Lexical Environment* specified by *Scope*, a Boolean flag *Strict*, and optionally, an object *prototype*. FunctionCreate performs the following steps:

1. If *prototype* is not present, then
 - a. Set *prototype* to the intrinsic object *%FunctionPrototype%*.
2. If *kind* is not *Normal*, let *allocKind* be **"non-constructor"**.
3. Else, let *allocKind* be **"normal"**.
4. Let F be *FunctionAllocate*(*prototype*, *Strict*, *allocKind*).
5. Return *FunctionInitialize*(F , *kind*, *ParameterList*, *Body*, *Scope*).

9.2.6 GeneratorFunctionCreate (*kind*, *ParameterList*, *Body*, *Scope*, *Strict*)

The abstract operation GeneratorFunctionCreate requires the arguments: *kind* which is one of (Normal, Method), a parameter list *Parse Node* specified by *ParameterList*, a body *Parse Node* specified by *Body*, a *Lexical Environment* specified by *Scope*, and a Boolean flag *Strict*. GeneratorFunctionCreate performs the following steps:

1. Let *functionPrototype* be the intrinsic object *%Generator%*.
2. Let F be *FunctionAllocate*(*functionPrototype*, *Strict*, **"generator"**).
3. Return *FunctionInitialize*(F , *kind*, *ParameterList*, *Body*, *Scope*).

9.2.7 AsyncGeneratorFunctionCreate (*kind*, *ParameterList*, *Body*, *Scope*, *Strict*)

The abstract operation AsyncGeneratorFunctionCreate requires the arguments: *kind* which is one of (Normal, Method), a parameter list *Parse Node* specified by *ParameterList*, a body *Parse Node* specified by *Body*, a *Lexical Environment* specified by *Scope*, and a Boolean flag *Strict*. AsyncGeneratorFunctionCreate performs the following steps:

1. Let *functionPrototype* be the intrinsic object *%AsyncGenerator%*.
2. Let F be ! *FunctionAllocate*(*functionPrototype*, *Strict*, **"generator"**).
3. Return ! *FunctionInitialize*(F , *kind*, *ParameterList*, *Body*, *Scope*).

9.2.8 AsyncFunctionCreate (*kind*, *parameters*, *body*, *Scope*, *Strict*)

The abstract operation AsyncFunctionCreate requires the arguments: *kind* which is one of (Normal, Method, Arrow), a parameter list *Parse Node* specified by *parameters*, a body *Parse Node* specified by *body*, a *Lexical Environment* specified by *Scope*, and a Boolean flag *Strict*. AsyncFunctionCreate performs the following steps:

1. Let *functionPrototype* be the intrinsic object *%AsyncFunctionPrototype%*.

2. Let *F* be ! **FunctionAllocate**(*functionPrototype*, *Strict*, "async").
3. Return ! **FunctionInitialize**(*F*, *kind*, *parameters*, *body*, *Scope*).

9.2.9 AddRestrictedFunctionProperties (*F*, *realm*)

The abstract operation **AddRestrictedFunctionProperties** is called with a **function object** *F* and **Realm Record** *realm* as its argument. It performs the following steps:

1. **Assert**: *realm*.[[Intrinsics]].[[%ThrowTypeError%]] exists and has been initialized.
2. Let *thrower* be *realm*.[[Intrinsics]].[[%ThrowTypeError%]].
3. Perform ! **DefinePropertyOrThrow**(*F*, "caller", PropertyDescriptor { [[Get]]: *thrower*, [[Set]]: *thrower*, [[Enumerable]]: **false**, [[Configurable]]: **true** }).
4. Return ! **DefinePropertyOrThrow**(*F*, "arguments", PropertyDescriptor { [[Get]]: *thrower*, [[Set]]: *thrower*, [[Enumerable]]: **false**, [[Configurable]]: **true** }).

9.2.9.1 %ThrowTypeError% ()

The %ThrowTypeError% intrinsic is an anonymous built-in **function object** that is defined once for each **realm**. When %ThrowTypeError% is called it performs the following steps:

1. Throw a **TypeError** exception.

The value of the [[Extensible]] internal slot of a %ThrowTypeError% function is **false**.

The "length" property of a %ThrowTypeError% function has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **false** }.

9.2.10 MakeConstructor (*F* [, *writablePrototype* [, *prototype*]])

The abstract operation **MakeConstructor** requires a **Function** argument *F* and optionally, a Boolean *writablePrototype* and an object *prototype*. If *prototype* is provided it is assumed to already contain, if needed, a "constructor" property whose value is *F*. This operation converts *F* into a **constructor** by performing the following steps:

1. **Assert**: *F* is an ECMAScript **function object**.
2. **Assert**: **IsConstructor**(*F*) is **true**.
3. **Assert**: *F* is an extensible object that does not have a **prototype** own property.
4. If *writablePrototype* is not present, set *writablePrototype* to **true**.
5. If *prototype* is not present, then
 - a. Set *prototype* to **ObjectCreate**(%ObjectPrototype%).
 - b. Perform ! **DefinePropertyOrThrow**(*prototype*, "constructor", PropertyDescriptor { [[Value]]: *F*, [[Writable]]: *writablePrototype*, [[Enumerable]]: **false**, [[Configurable]]: **true** }).
6. Perform ! **DefinePropertyOrThrow**(*F*, "prototype", PropertyDescriptor { [[Value]]: *prototype*, [[Writable]]: *writablePrototype*, [[Enumerable]]: **false**, [[Configurable]]: **false** }).
7. Return **NormalCompletion**(**undefined**).

9.2.11 MakeClassConstructor (*F*)

The abstract operation **MakeClassConstructor** with argument *F* performs the following steps:

1. **Assert**: *F* is an ECMAScript **function object**.

1. **Assert:** F .[[FunctionKind]] is **"normal"**.
3. Set F .[[FunctionKind]] to **"classConstructor"**.
4. Return **NormalCompletion(undefined)**.

9.2.12 MakeMethod (F , $homeObject$)

The abstract operation MakeMethod with arguments F and $homeObject$ configures F as a method by performing the following steps:

1. **Assert:** F is an ECMAScript **function object**.
2. **Assert:** **Type**($homeObject$) is **Object**.
3. Set F .[[HomeObject]] to $homeObject$.
4. Return **NormalCompletion(undefined)**.

9.2.13 SetFunctionName (F , $name$ [, $prefix$])

The abstract operation SetFunctionName requires a Function argument F , a String or Symbol argument $name$ and optionally a String argument $prefix$. This operation adds a **name** property to F by performing the following steps:

1. **Assert:** F is an extensible object that does not have a **name** own property.
2. **Assert:** **Type**($name$) is either **Symbol** or **String**.
3. **Assert:** If $prefix$ is present, then **Type**($prefix$) is **String**.
4. If **Type**($name$) is **Symbol**, then
 - a. Let $description$ be $name$'s [[Description]] value.
 - b. If $description$ is **undefined**, set $name$ to the empty String.
 - c. Else, set $name$ to the **string-concatenation** of "[", $description$, and "]"
5. If $prefix$ is present, then
 - a. Set $name$ to the **string-concatenation** of $prefix$, the code unit 0x0020 (SPACE), and $name$.
6. Return ! **DefinePropertyOrThrow**(F , **"name"**, PropertyDescriptor { [[Value]]: $name$, [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **true** }).

9.2.14 SetFunctionLength (F , $length$)

The abstract operation SetFunctionLength requires a Function argument F and a Number argument $length$. This operation adds a **"length"** property to F by performing the following steps:

1. **Assert:** F is an extensible object that does not have a **"length"** own property.
2. **Assert:** **Type**($length$) is **Number**.
3. **Assert:** $length \geq 0$ and ! **ToInteger**($length$) is equal to $length$.
4. Return ! **DefinePropertyOrThrow**(F , **"length"**, PropertyDescriptor { [[Value]]: $length$, [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **true** }).

9.2.15 FunctionDeclarationInstantiation ($func$, $argumentsList$)

NOTE 1

When an **execution context** is established for evaluating an ECMAScript function a new **function Environment Record** is created and bindings for each formal parameter are instantiated in that **Environment Record**. Each declaration in the function body is also instantiated. If the function's formal parameters do not include any default value initializers then the body declarations are instantiated in the same **Environment Record** as the parameters. If default value parameter

initializers exist, a second [Environment Record](#) is created for the body declarations. Formal parameters and functions are initialized as part of `FunctionDeclarationInstantiation`. All other bindings are initialized during evaluation of the function body.

`FunctionDeclarationInstantiation` is performed as follows using arguments *func* and *argumentsList*. *func* is the [function object](#) for which the [execution context](#) is being established.

1. Let *calleeContext* be the [running execution context](#).
2. Let *env* be the `LexicalEnvironment` of *calleeContext*.
3. Let *envRec* be *env*'s `EnvironmentRecord`.
4. Let *code* be *func*.[[ECMAScriptCode]].
5. Let *strict* be *func*.[[Strict]].
6. Let *formals* be *func*.[[FormalParameters]].
7. Let *parameterNames* be the `BoundNames` of *formals*.
8. If *parameterNames* has any duplicate entries, let *hasDuplicates* be **true**. Otherwise, let *hasDuplicates* be **false**.
9. Let *simpleParameterList* be `IsSimpleParameterList` of *formals*.
10. Let *hasParameterExpressions* be `ContainsExpression` of *formals*.
11. Let *varNames* be the `VarDeclaredNames` of *code*.
12. Let *varDeclarations* be the `VarScopedDeclarations` of *code*.
13. Let *lexicalNames* be the `LexicallyDeclaredNames` of *code*.
14. Let *functionNames* be a new empty [List](#).
15. Let *functionsToInitialize* be a new empty [List](#).
16. For each *d* in *varDeclarations*, in reverse list order, do
 - a. If *d* is neither a `VariableDeclaration` nor a `ForBinding` nor a `BindingIdentifier`, then
 - i. **Assert:** *d* is either a `FunctionDeclaration`, a `GeneratorDeclaration`, an `AsyncFunctionDeclaration`, or an `AsyncGeneratorDeclaration`.
 - ii. Let *fn* be the sole element of the `BoundNames` of *d*.
 - iii. If *fn* is not an element of *functionNames*, then
 1. Insert *fn* as the first element of *functionNames*.
 2. NOTE: If there are multiple function declarations for the same name, the last declaration is used.
 3. Insert *d* as the first element of *functionsToInitialize*.
17. Let *argumentsObjectNeeded* be **true**.
18. If *func*.[[ThisMode]] is `lexical`, then
 - a. NOTE: Arrow functions never have an arguments objects.
 - b. Set *argumentsObjectNeeded* to **false**.
19. Else if **"arguments"** is an element of *parameterNames*, then
 - a. Set *argumentsObjectNeeded* to **false**.
20. Else if *hasParameterExpressions* is **false**, then
 - a. If **"arguments"** is an element of *functionNames* or if **"arguments"** is an element of *lexicalNames*, then
 - i. Set *argumentsObjectNeeded* to **false**.
21. For each String *paramName* in *parameterNames*, do
 - a. Let *alreadyDeclared* be *envRec*.`HasBinding(paramName)`.
 - b. NOTE: Early errors ensure that duplicate parameter names can only occur in non-strict functions that do not have parameter default values or rest parameters.
 - c. If *alreadyDeclared* is **false**, then
 - i. Perform ! *envRec*.`CreateMutableBinding(paramName, false)`.
 - ii. If *hasDuplicates* is **true**, then
 1. Perform ! *envRec*.`InitializeBinding(paramName, undefined)`.

22. If *argumentsObjectNeeded* is **true**, then
 - a. If *strict* is **true** or if *simpleParameterList* is **false**, then
 - i. Let *ao* be *CreateUnmappedArgumentsObject*(*argumentsList*).
 - b. Else,
 - i. NOTE: mapped argument object is only provided for non-strict functions that don't have a rest parameter, any parameter default value initializers, or any destructured parameters.
 - ii. Let *ao* be *CreateMappedArgumentsObject*(*func*, *formals*, *argumentsList*, *envRec*).
 - c. If *strict* is **true**, then
 - i. Perform ! *envRec*.CreateImmutableBinding(**"arguments"**, **false**).
 - d. Else,
 - i. Perform ! *envRec*.CreateMutableBinding(**"arguments"**, **false**).
 - e. Call *envRec*.InitializeBinding(**"arguments"**, *ao*).
 - f. Let *parameterBindings* be a new *List* of *parameterNames* with **"arguments"** appended.
23. Else,
 - a. Let *parameterBindings* be *parameterNames*.
24. Let *iteratorRecord* be *CreateListIteratorRecord*(*argumentsList*).
25. If *hasDuplicates* is **true**, then
 - a. Perform ? IteratorBindingInitialization for *formals* with *iteratorRecord* and **undefined** as arguments.
26. Else,
 - a. Perform ? IteratorBindingInitialization for *formals* with *iteratorRecord* and *env* as arguments.
27. If *hasParameterExpressions* is **false**, then
 - a. NOTE: Only a single lexical environment is needed for the parameters and top-level vars.
 - b. Let *instantiatedVarNames* be a copy of the *List* *parameterBindings*.
 - c. For each *n* in *varNames*, do
 - i. If *n* is not an element of *instantiatedVarNames*, then
 1. Append *n* to *instantiatedVarNames*.
 2. Perform ! *envRec*.CreateMutableBinding(*n*, **false**).
 3. Call *envRec*.InitializeBinding(*n*, **undefined**).
 - d. Let *varEnv* be *env*.
 - e. Let *varEnvRec* be *envRec*.
28. Else,
 - a. NOTE: A separate *Environment Record* is needed to ensure that closures created by expressions in the formal parameter list do not have visibility of declarations in the function body.
 - b. Let *varEnv* be *NewDeclarativeEnvironment*(*env*).
 - c. Let *varEnvRec* be *varEnv*'s *EnvironmentRecord*.
 - d. Set the VariableEnvironment of *calleeContext* to *varEnv*.
 - e. Let *instantiatedVarNames* be a new empty *List*.
 - f. For each *n* in *varNames*, do
 - i. If *n* is not an element of *instantiatedVarNames*, then
 1. Append *n* to *instantiatedVarNames*.
 2. Perform ! *varEnvRec*.CreateMutableBinding(*n*, **false**).
 3. If *n* is not an element of *parameterBindings* or if *n* is an element of *functionNames*, let *initialValue* be **undefined**.
 4. Else,
 - a. Let *initialValue* be ! *envRec*.GetBindingValue(*n*, **false**).
 5. Call *varEnvRec*.InitializeBinding(*n*, *initialValue*).
 6. NOTE: vars whose names are the same as a formal parameter, initially have the same value as the corresponding initialized parameter.

29. NOTE: Annex B.3.3.1 adds additional steps at this point.
30. If *strict* is **false**, then
 - a. Let *lexEnv* be *NewDeclarativeEnvironment*(*varEnv*).
 - b. NOTE: Non-strict functions use a separate lexical *Environment Record* for top-level lexical declarations so that a *direct eval* can determine whether any var scoped declarations introduced by the eval code conflict with pre-existing top-level lexically scoped declarations. This is not needed for strict functions because a strict *direct eval* always places all declarations into a new *Environment Record*.
31. Else, let *lexEnv* be *varEnv*.
32. Let *lexEnvRec* be *lexEnv*'s *EnvironmentRecord*.
33. Set the *LexicalEnvironment* of *calleeContext* to *lexEnv*.
34. Let *lexDeclarations* be the *LexicallyScopedDeclarations* of *code*.
35. For each element *d* in *lexDeclarations*, do
 - a. NOTE: A lexically declared name cannot be the same as a function/generator declaration, formal parameter, or a var name. Lexically declared names are only instantiated here but not initialized.
 - b. For each element *dn* of the *BoundNames* of *d*, do
 - i. If *IsConstantDeclaration* of *d* is **true**, then
 1. Perform ! *lexEnvRec*.*CreateImmutableBinding*(*dn*, **true**).
 - ii. Else,
 1. Perform ! *lexEnvRec*.*CreateMutableBinding*(*dn*, **false**).
36. For each *Parse Node f* in *functionsToInitialize*, do
 - a. Let *fn* be the sole element of the *BoundNames* of *f*.
 - b. Let *fo* be the result of performing *InstantiateFunctionObject* for *f* with argument *lexEnv*.
 - c. Perform ! *varEnvRec*.*SetMutableBinding*(*fn*, *fo*, **false**).
37. Return *NormalCompletion*(empty).

NOTE 2

B.3.3 provides an extension to the above algorithm that is necessary for backwards compatibility with web browser implementations of ECMAScript that predate ECMAScript 2015.

NOTE 3

Parameter *Initializers* may contain *direct eval* expressions. Any top level declarations of such evals are only visible to the eval code (10.2). The creation of the environment for such declarations is described in 14.1.19.

9.3 Built-in Function Objects

The built-in function objects defined in this specification may be implemented as either ECMAScript function objects (9.2) whose behaviour is provided using ECMAScript code or as implementation provided function exotic objects whose behaviour is provided in some other manner. In either case, the effect of calling such functions must conform to their specifications. An implementation may also provide additional built-in function objects that are not defined in this specification.

If a built-in *function object* is implemented as an *exotic object* it must have the ordinary object behaviour specified in 9.1. All such function exotic objects also have *[[Prototype]]*, *[[Extensible]]*, *[[Realm]]*, and *[[ScriptOrModule]]* internal slots.

Unless otherwise specified every built-in *function object* has the *%FunctionPrototype%* object as the initial value of its *[[Prototype]]* internal slot.

The behaviour specified for each built-in function via algorithm steps or other means is the specification of the function

body behaviour for both `[[Call]]` and `[[Construct]]` invocations of the function. However, `[[Construct]]` invocation is not supported by all built-in functions. For each built-in function, when invoked with `[[Call]]`, the `[[Call]]` *thisArgument* provides the **this** value, the `[[Call]]` *argumentsList* provides the named parameters, and the `NewTarget` value is **undefined**. When invoked with `[[Construct]]`, the **this** value is uninitialized, the `[[Construct]]` *argumentsList* provides the named parameters, and the `[[Construct]]` *newTarget* parameter provides the `NewTarget` value. If the built-in function is implemented as an ECMAScript *function object* then this specified behaviour must be implemented by the ECMAScript code that is the body of the function. Built-in functions that are ECMAScript function objects must be strict functions. If a built-in *constructor* has any `[[Call]]` behaviour other than throwing a **TypeError** exception, an ECMAScript implementation of the function must be done in a manner that does not cause the function's `[[FunctionKind]]` internal slot to have the value **"classConstructor"**.

Built-in function objects that are not identified as constructors do not implement the `[[Construct]]` internal method unless otherwise specified in the description of a particular function. When a built-in *constructor* is called as part of a **new** expression the *argumentsList* parameter of the invoked `[[Construct]]` internal method provides the values for the built-in *constructor*'s named parameters.

Built-in functions that are not constructors do not have a **prototype** property unless otherwise specified in the description of a particular function.

If a built-in *function object* is not implemented as an ECMAScript function it must provide `[[Call]]` and `[[Construct]]` internal methods that conform to the following definitions:

9.3.1 `[[Call]]` (*thisArgument*, *argumentsList*)

The `[[Call]]` internal method for a built-in *function object* *F* is called with parameters *thisArgument* and *argumentsList*, a *List* of ECMAScript language values. The following steps are taken:

1. Let *callerContext* be the *running execution context*.
2. If *callerContext* is not already suspended, suspend *callerContext*.
3. Let *calleeContext* be a new ECMAScript code *execution context*.
4. Set the *Function* of *calleeContext* to *F*.
5. Let *calleeRealm* be *F*.`[[Realm]]`.
6. Set the *Realm* of *calleeContext* to *calleeRealm*.
7. Set the *ScriptOrModule* of *calleeContext* to *F*.`[[ScriptOrModule]]`.
8. Perform any necessary implementation-defined initialization of *calleeContext*.
9. Push *calleeContext* onto the *execution context stack*; *calleeContext* is now the *running execution context*.
10. Let *result* be the *Completion Record* that is the result of evaluating *F* in an implementation-defined manner that conforms to the specification of *F*. *thisArgument* is the **this** value, *argumentsList* provides the named parameters, and the `NewTarget` value is **undefined**.
11. Remove *calleeContext* from the *execution context stack* and restore *callerContext* as the *running execution context*.
12. Return *result*.

NOTE

When *calleeContext* is removed from the *execution context stack* it must not be destroyed if it has been suspended and retained by an accessible generator object for later resumption.

9.3.2 `[[Construct]]` (*argumentsList*, *newTarget*)

The `[[Construct]]` internal method for built-in [function object](#) *F* is called with parameters *argumentsList* and *newTarget*. The steps performed are the same as `[[Call]]` (see 9.3.1) except that step 10 is replaced by:

10. Let *result* be the [Completion Record](#) that is the result of evaluating *F* in an implementation-defined manner that conforms to the specification of *F*. The **this** value is uninitialized, *argumentsList* provides the named parameters, and *newTarget* provides the `NewTarget` value.

9.3.3 CreateBuiltinFunction (*steps*, *internalSlotsList* [, *realm* [, *prototype*]])

The abstract operation `CreateBuiltinFunction` takes arguments *steps*, *internalSlotsList*, *realm*, and *prototype*. The argument *internalSlotsList* is a [List](#) of the names of additional internal slots that must be defined as part of the object. `CreateBuiltinFunction` returns a built-in [function object](#) created by the following steps:

1. **Assert:** *steps* is either a set of algorithm steps or other definition of a function's behaviour provided in this specification.
2. If *realm* is not present, set *realm* to the current [Realm Record](#).
3. **Assert:** *realm* is a [Realm Record](#).
4. If *prototype* is not present, set *prototype* to *realm*.`[[Intrinsics]].[%FunctionPrototype%]`.
5. Let *func* be a new built-in [function object](#) that when called performs the action described by *steps*. The new [function object](#) has internal slots whose names are the elements of *internalSlotsList*. The initial value of each of those internal slots is **undefined**.
6. Set *func*.`[[Realm]]` to *realm*.
7. Set *func*.`[[Prototype]]` to *prototype*.
8. Set *func*.`[[Extensible]]` to **true**.
9. Set *func*.`[[ScriptOrModule]]` to **null**.
10. Return *func*.

Each built-in function defined in this specification is created by calling the `CreateBuiltinFunction` abstract operation.

9.4 Built-in Exotic Object Internal Methods and Slots

This specification defines several kinds of built-in exotic objects. These objects generally behave similar to ordinary objects except for a few specific situations. The following exotic objects use the ordinary object internal methods except where it is explicitly specified otherwise below:

9.4.1 Bound Function Exotic Objects

A bound function is an [exotic object](#) that wraps another [function object](#). A bound function is callable (it has a `[[Call]]` internal method and may have a `[[Construct]]` internal method). Calling a bound function generally results in a call of its wrapped function.

Bound function objects do not have the internal slots of ECMAScript function objects defined in [Table 27](#). Instead they have the internal slots defined in [Table 28](#).

Table 28: Internal Slots of Bound Function Exotic Objects

Internal Slot	Type	Description
---------------	------	-------------

[[BoundTargetFunction]]	Callable Object	The wrapped function object .
[[BoundThis]]	Any	The value that is always passed as the this value when calling the wrapped function.
[[BoundArguments]]	List of Any	A list of values whose elements are used as the first arguments to any call to the wrapped function.

Bound function objects provide all of the essential internal methods as specified in [9.1](#). However, they use the following definitions for the essential internal methods of function objects.

9.4.1.1 [[Call]] (*thisArgument*, *argumentsList*)

When the [[Call]] internal method of a [bound function exotic object](#), *F*, which was created using the bind function is called with parameters *thisArgument* and *argumentsList*, a [List](#) of ECMAScript language values, the following steps are taken:

1. Let *target* be *F*.[[BoundTargetFunction]].
2. Let *boundThis* be *F*.[[BoundThis]].
3. Let *boundArgs* be *F*.[[BoundArguments]].
4. Let *args* be a new list containing the same values as the list *boundArgs* in the same order followed by the same values as the list *argumentsList* in the same order.
5. Return ? [Call](#)(*target*, *boundThis*, *args*).

9.4.1.2 [[Construct]] (*argumentsList*, *newTarget*)

When the [[Construct]] internal method of a [bound function exotic object](#), *F* that was created using the bind function is called with a list of arguments *argumentsList* and *newTarget*, the following steps are taken:

1. Let *target* be *F*.[[BoundTargetFunction]].
2. [Assert](#): [IsConstructor](#)(*target*) is **true**.
3. Let *boundArgs* be *F*.[[BoundArguments]].
4. Let *args* be a new list containing the same values as the list *boundArgs* in the same order followed by the same values as the list *argumentsList* in the same order.
5. If [SameValue](#)(*F*, *newTarget*) is **true**, set *newTarget* to *target*.
6. Return ? [Construct](#)(*target*, *args*, *newTarget*).

9.4.1.3 BoundFunctionCreate (*targetFunction*, *boundThis*, *boundArgs*)

The abstract operation BoundFunctionCreate with arguments *targetFunction*, *boundThis* and *boundArgs* is used to specify the creation of new Bound Function exotic objects. It performs the following steps:

1. [Assert](#): [Type](#)(*targetFunction*) is Object.
2. Let *proto* be ? *targetFunction*.[[GetPrototypeOf]]().
3. Let *obj* be a newly created object.
4. Set *obj*'s essential internal methods to the default ordinary object definitions specified in [9.1](#).
5. Set *obj*.[[Call]] as described in [9.4.1.1](#).
6. If [IsConstructor](#)(*targetFunction*) is **true**, then

- a. Set *obj*.[[Construct]] as described in 9.4.1.2.
7. Set *obj*.[[Prototype]] to *proto*.
8. Set *obj*.[[Extensible]] to **true**.
9. Set *obj*.[[BoundTargetFunction]] to *targetFunction*.
10. Set *obj*.[[BoundThis]] to *boundThis*.
11. Set *obj*.[[BoundArguments]] to *boundArgs*.
12. Return *obj*.

9.4.2 Array Exotic Objects

An *Array object* is an *exotic object* that gives special treatment to *array index* property keys (see 6.1.7). A property whose *property name* is an *array index* is also called an *element*. Every Array object has a non-configurable "**length**" property whose value is always a nonnegative integer less than 2^{32} . The value of the "**length**" property is numerically greater than the name of every own property whose name is an *array index*; whenever an own property of an Array object is created or changed, other properties are adjusted as necessary to maintain this invariant. Specifically, whenever an own property is added whose name is an *array index*, the value of the "**length**" property is changed, if necessary, to be one more than the numeric value of that *array index*; and whenever the value of the "**length**" property is changed, every own property whose name is an *array index* whose value is not smaller than the new length is deleted. This constraint applies only to own properties of an Array object and is unaffected by "**length**" or *array index* properties that may be inherited from its prototypes.

NOTE

A String *property name* *P* is an *array index* if and only if `ToString(ToUint32(P))` is equal to *P* and `ToUint32(P)` is not equal to $2^{32} - 1$.

Array exotic objects provide an alternative definition for the `[[DefineOwnProperty]]` internal method. Except for that internal method, Array exotic objects provide all of the other essential internal methods as specified in 9.1.

9.4.2.1 `[[DefineOwnProperty]]` (*P*, *Desc*)

When the `[[DefineOwnProperty]]` internal method of an Array *exotic object* *A* is called with property key *P*, and *Property Descriptor* *Desc*, the following steps are taken:

1. **Assert:** `IsPropertyKey(P)` is **true**.
2. If *P* is "**length**", then
 - a. Return ? `ArraySetLength(A, Desc)`.
3. Else if *P* is an *array index*, then
 - a. Let *oldLenDesc* be `OrdinaryGetOwnProperty(A, "length")`.
 - b. **Assert:** *oldLenDesc* will never be **undefined** or an accessor descriptor because Array objects are created with a *length data property* that cannot be deleted or reconfigured.
 - c. Let *oldLen* be *oldLenDesc*.[[Value]].
 - d. Let *index* be ! `ToUint32(P)`.
 - e. If *index* ≥ *oldLen* and *oldLenDesc*.[[Writable]] is **false**, return **false**.
 - f. Let *succeeded* be ! `OrdinaryDefineOwnProperty(A, P, Desc)`.
 - g. If *succeeded* is **false**, return **false**.
 - h. If *index* ≥ *oldLen*, then
 - i. Set *oldLenDesc*.[[Value]] to *index* + 1.
 - ii. Let *succeeded* be `OrdinaryDefineOwnProperty(A, "length", oldLenDesc)`.

- iii. **Assert:** *succeeded* is **true**.
- i. Return **true**.
- 4. Return `OrdinaryDefineOwnProperty(A, P, Desc)`.

9.4.2.2 ArrayCreate (*length* [, *proto*])

The abstract operation ArrayCreate with argument *length* (either 0 or a positive integer) and optional argument *proto* is used to specify the creation of new Array exotic objects. It performs the following steps:

1. **Assert:** *length* is an integer Number ≥ 0 .
2. If *length* is **-0**, set *length* to **+0**.
3. If *length* $> 2^{32} - 1$, throw a **RangeError** exception.
4. If *proto* is not present, set *proto* to the intrinsic object `%ArrayPrototype%`.
5. Let *A* be a newly created Array exotic object.
6. Set *A*'s essential internal methods except for `[[DefineOwnProperty]]` to the default ordinary object definitions specified in 9.1.
7. Set *A*.`[[DefineOwnProperty]]` as specified in 9.4.2.1.
8. Set *A*.`[[Prototype]]` to *proto*.
9. Set *A*.`[[Extensible]]` to **true**.
10. Perform `! OrdinaryDefineOwnProperty(A, "length", PropertyDescriptor { [[Value]]: length, [[Writable]]: true, [[Enumerable]]: false, [[Configurable]]: false })`.
11. Return *A*.

9.4.2.3 ArraySpeciesCreate (*originalArray*, *length*)

The abstract operation ArraySpeciesCreate with arguments *originalArray* and *length* is used to specify the creation of a new Array object using a *constructor* function that is derived from *originalArray*. It performs the following steps:

1. **Assert:** *length* is an integer Number ≥ 0 .
2. If *length* is **-0**, set *length* to **+0**.
3. Let *isArray* be `? IsArray(originalArray)`.
4. If *isArray* is **false**, return `? ArrayCreate(length)`.
5. Let *C* be `? Get(originalArray, "constructor")`.
6. If `IsConstructor(C)` is **true**, then
 - a. Let *thisRealm* be the current Realm Record.
 - b. Let *realmC* be `? GetFunctionRealm(C)`.
 - c. If *thisRealm* and *realmC* are not the same Realm Record, then
 - i. If `SameValue(C, realmC.[[Intrinsics]].[%Array%])` is **true**, set *C* to **undefined**.
7. If `Type(C)` is Object, then
 - a. Set *C* to `? Get(C, @@species)`.
 - b. If *C* is **null**, set *C* to **undefined**.
8. If *C* is **undefined**, return `? ArrayCreate(length)`.
9. If `IsConstructor(C)` is **false**, throw a **TypeError** exception.
10. Return `? Construct(C, « length »)`.

NOTE

If *originalArray* was created using the standard built-in Array *constructor* for a *realm* that is not the *realm* of the *running execution context*, then a new Array is created using the *realm* of the *running execution context*. This maintains compatibility with Web browsers that have historically had that behaviour for the `Array.prototype` methods that now are

defined using `ArraySpeciesCreate`.

9.4.2.4 `ArraySetLength` (*A*, *Desc*)

When the abstract operation `ArraySetLength` is called with an Array *exotic object* *A*, and *Property Descriptor* *Desc*, the following steps are taken:

1. If *Desc*.[[Value]] is absent, then
 - a. Return `OrdinaryDefineOwnProperty`(*A*, "length", *Desc*).
2. Let *newLenDesc* be a copy of *Desc*.
3. Let *newLen* be ? `ToUint32`(*Desc*.[[Value]]).
4. Let *numberLen* be ? `ToNumber`(*Desc*.[[Value]]).
5. If *newLen* ≠ *numberLen*, throw a **RangeError** exception.
6. Set *newLenDesc*.[[Value]] to *newLen*.
7. Let *oldLenDesc* be `OrdinaryGetOwnProperty`(*A*, "length").
8. **Assert**: *oldLenDesc* will never be **undefined** or an accessor descriptor because Array objects are created with a length *data property* that cannot be deleted or reconfigured.
9. Let *oldLen* be *oldLenDesc*.[[Value]].
10. If *newLen* ≥ *oldLen*, then
 - a. Return `OrdinaryDefineOwnProperty`(*A*, "length", *newLenDesc*).
11. If *oldLenDesc*.[[Writable]] is **false**, return **false**.
12. If *newLenDesc*.[[Writable]] is absent or has the value **true**, let *newWritable* be **true**.
13. Else,
 - a. Need to defer setting the [[Writable]] attribute to **false** in case any elements cannot be deleted.
 - b. Let *newWritable* be **false**.
 - c. Set *newLenDesc*.[[Writable]] to **true**.
14. Let *succeeded* be ! `OrdinaryDefineOwnProperty`(*A*, "length", *newLenDesc*).
15. If *succeeded* is **false**, return **false**.
16. Repeat, while *newLen* < *oldLen*,
 - a. Decrease *oldLen* by 1.
 - b. Let *deleteSucceeded* be ! *A*.[[Delete]](! `ToString`(*oldLen*)).
 - c. If *deleteSucceeded* is **false**, then
 - i. Set *newLenDesc*.[[Value]] to *oldLen* + 1.
 - ii. If *newWritable* is **false**, set *newLenDesc*.[[Writable]] to **false**.
 - iii. Perform ! `OrdinaryDefineOwnProperty`(*A*, "length", *newLenDesc*).
 - iv. Return **false**.
17. If *newWritable* is **false**, then
 - a. Return `OrdinaryDefineOwnProperty`(*A*, "length", `PropertyDescriptor` { [[Writable]]: **false** }). This call will always return **true**.
18. Return **true**.

NOTE

In steps 3 and 4, if *Desc*.[[Value]] is an object then its **valueOf** method is called twice. This is legacy behaviour that was specified with this effect starting with the 2nd Edition of this specification.

9.4.3 String Exotic Objects

A *String object* is an [exotic object](#) that encapsulates a String value and exposes virtual integer-indexed data properties corresponding to the individual code unit elements of the String value. String exotic objects always have a [data property](#) named **"length"** whose value is the number of code unit elements in the encapsulated String value. Both the code unit data properties and the **"length"** property are non-writable and non-configurable.

String exotic objects have the same internal slots as ordinary objects. They also have a `[[StringData]]` internal slot.

String exotic objects provide alternative definitions for the following internal methods. All of the other String [exotic object](#) essential internal methods that are not defined below are as specified in [9.1](#).

9.4.3.1 `[[GetOwnProperty]] ( P )`

When the `[[GetOwnProperty]]` internal method of a String [exotic object](#) *S* is called with property key *P*, the following steps are taken:

1. **Assert:** `IsPropertyKey(P)` is **true**.
2. Let *desc* be `OrdinaryGetOwnProperty(S, P)`.
3. If *desc* is not **undefined**, return *desc*.
4. Return ! `StringGetOwnProperty(S, P)`.

9.4.3.2 `[[DefineOwnProperty]] ( P, Desc )`

When the `[[DefineOwnProperty]]` internal method of a String [exotic object](#) *S* is called with property key *P*, and [Property Descriptor](#) *Desc*, the following steps are taken:

1. **Assert:** `IsPropertyKey(P)` is **true**.
2. Let *stringDesc* be ! `StringGetOwnProperty(S, P)`.
3. If *stringDesc* is not **undefined**, then
 - a. Let *extensible* be *S*.`[[Extensible]]`.
 - b. Return ! `IsCompatiblePropertyDescriptor(extensible, Desc, stringDesc)`.
4. Return ! `OrdinaryDefineOwnProperty(S, P, Desc)`.

9.4.3.3 `[[OwnPropertyKeys]] ( )`

When the `[[OwnPropertyKeys]]` internal method of a String [exotic object](#) *O* is called, the following steps are taken:

1. Let *keys* be a new empty [List](#).
2. Let *str* be *O*.`[[StringData]]`.
3. **Assert:** `Type(str)` is String.
4. Let *len* be the length of *str*.
5. For each integer *i* starting with 0 such that *i* < *len*, in ascending order, do
 - a. Add ! `ToString(i)` as the last element of *keys*.
6. For each own property key *P* of *O* such that *P* is an [array index](#) and `ToInteger(P)` ≥ *len*, in ascending numeric index order, do
 - a. Add *P* as the last element of *keys*.
7. For each own property key *P* of *O* such that `Type(P)` is String and *P* is not an [array index](#), in ascending chronological order of property creation, do
 - a. Add *P* as the last element of *keys*.
8. For each own property key *P* of *O* such that `Type(P)` is Symbol, in ascending chronological order of property creation, do

- a. Add *P* as the last element of *keys*.
9. Return *keys*.

9.4.3.4 StringCreate (*value*, *prototype*)

The abstract operation StringCreate with arguments *value* and *prototype* is used to specify the creation of new String exotic objects. It performs the following steps:

1. **Assert:** Type(*value*) is String.
2. Let *S* be a newly created String exotic object.
3. Set *S*.[[StringData]] to *value*.
4. Set *S*'s essential internal methods to the default ordinary object definitions specified in 9.1.
5. Set *S*.[[GetOwnProperty]] as specified in 9.4.3.1.
6. Set *S*.[[DefineOwnProperty]] as specified in 9.4.3.2.
7. Set *S*.[[OwnPropertyKeys]] as specified in 9.4.3.3.
8. Set *S*.[[Prototype]] to *prototype*.
9. Set *S*.[[Extensible]] to **true**.
10. Let *length* be the number of code unit elements in *value*.
11. Perform ! DefinePropertyOrThrow(*S*, "**length**", PropertyDescriptor { [[Value]]: *length*, [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **false** }).
12. Return *S*.

9.4.3.5 StringGetOwnProperty (*S*, *P*)

The abstract operation StringGetOwnProperty called with arguments *S* and *P* performs the following steps:

1. **Assert:** *S* is an Object that has a [[StringData]] internal slot.
2. **Assert:** IsPropertyKey(*P*) is **true**.
3. If Type(*P*) is not String, return **undefined**.
4. Let *index* be ! CanonicalNumericIndexString(*P*).
5. If *index* is **undefined**, return **undefined**.
6. If IsInteger(*index*) is **false**, return **undefined**.
7. If *index* = **-0**, return **undefined**.
8. Let *str* be *S*.[[StringData]].
9. **Assert:** Type(*str*) is String.
10. Let *len* be the length of *str*.
11. If *index* < 0 or *len* ≤ *index*, return **undefined**.
12. Let *resultStr* be the String value of length 1, containing one code unit from *str*, specifically the code unit at index *index*.
13. Return a PropertyDescriptor { [[Value]]: *resultStr*, [[Writable]]: **false**, [[Enumerable]]: **true**, [[Configurable]]: **false** }.

9.4.4 Arguments Exotic Objects

Most ECMAScript functions make an arguments object available to their code. Depending upon the characteristics of the function definition, its arguments object is either an ordinary object or an *arguments exotic object*. An arguments exotic object is an exotic object whose array index properties map to the formal parameters bindings of an invocation of its associated ECMAScript function.

Arguments exotic objects have the same internal slots as ordinary objects. They also have a `[[ParameterMap]]` internal slot. Ordinary arguments objects also have a `[[ParameterMap]]` internal slot whose value is always undefined. For ordinary argument objects the `[[ParameterMap]]` internal slot is only used by `Object.prototype.toString` (19.1.3.6) to identify them as such.

Arguments exotic objects provide alternative definitions for the following internal methods. All of the other arguments exotic object essential internal methods that are not defined below are as specified in 9.1

NOTE 1

The integer-indexed data properties of an arguments exotic object whose numeric name values are less than the number of formal parameters of the corresponding function object initially share their values with the corresponding argument bindings in the function's execution context. This means that changing the property changes the corresponding value of the argument binding and vice-versa. This correspondence is broken if such a property is deleted and then redefined or if the property is changed into an accessor property. If the arguments object is an ordinary object, the values of its properties are simply a copy of the arguments passed to the function and there is no dynamic linkage between the property values and the formal parameter values.

NOTE 2

The ParameterMap object and its property values are used as a device for specifying the arguments object correspondence to argument bindings. The ParameterMap object and the objects that are the values of its properties are not directly observable from ECMAScript code. An ECMAScript implementation does not need to actually create or use such objects to implement the specified semantics.

NOTE 3

Ordinary arguments objects define a non-configurable accessor property named `"callee"` which throws a `TypeError` exception on access. The `"callee"` property has a more specific meaning for arguments exotic objects, which are created only for some class of non-strict functions. The definition of this property in the ordinary variant exists to ensure that it is not defined in any other manner by conforming ECMAScript implementations.

NOTE 4

ECMAScript implementations of arguments exotic objects have historically contained an accessor property named `"caller"`. Prior to ECMAScript 2017, this specification included the definition of a throwing `"caller"` property on ordinary arguments objects. Since implementations do not contain this extension any longer, ECMAScript 2017 dropped the requirement for a throwing `"caller"` accessor.

9.4.4.1 `[[GetOwnProperty]]` (*P*)

The `[[GetOwnProperty]]` internal method of an arguments exotic object when called with a property key *P* performs the following steps:

1. Let *args* be the arguments object.
2. Let *desc* be `OrdinaryGetOwnProperty(args, P)`.
3. If *desc* is `undefined`, return *desc*.
4. Let *map* be *args*.`[[ParameterMap]]`.
5. Let *isMapped* be `! HasOwnProperty(map, P)`.
6. If *isMapped* is `true`, then
 - a. Set *desc*.`[[Value]]` to `Get(map, P)`.
7. Return *desc*.

9.4.4.2 `[[DefineOwnProperty]]` (*P*, *Desc*)

The `[[DefineOwnProperty]]` internal method of an arguments *exotic object* when called with a property key *P* and Property Descriptor *Desc* performs the following steps:

1. Let *args* be the arguments object.
2. Let *map* be *args*.[[ParameterMap]].
3. Let *isMapped* be *HasOwnProperty*(*map*, *P*).
4. Let *newArgDesc* be *Desc*.
5. If *isMapped* is **true** and *IsDataDescriptor*(*Desc*) is **true**, then
 - a. If *Desc*.[[Value]] is not present and *Desc*.[[Writable]] is present and its value is **false**, then
 - i. Set *newArgDesc* to a copy of *Desc*.
 - ii. Set *newArgDesc*.[[Value]] to *Get*(*map*, *P*).
6. Let *allowed* be ? *OrdinaryDefineOwnProperty*(*args*, *P*, *newArgDesc*).
7. If *allowed* is **false**, return **false**.
8. If *isMapped* is **true**, then
 - a. If *IsAccessorDescriptor*(*Desc*) is **true**, then
 - i. Call *map*.[[Delete]](*P*).
 - b. Else,
 - i. If *Desc*.[[Value]] is present, then
 1. Let *setStatus* be *Set*(*map*, *P*, *Desc*.[[Value]], **false**).
 2. *Assert*: *setStatus* is **true** because formal parameters mapped by argument objects are always writable.
 - ii. If *Desc*.[[Writable]] is present and its value is **false**, then
 1. Call *map*.[[Delete]](*P*).
9. Return **true**.

9.4.4.3 `[[Get]]` (*P*, *Receiver*)

The `[[Get]]` internal method of an arguments *exotic object* when called with a property key *P* and ECMAScript language value *Receiver* performs the following steps:

1. Let *args* be the arguments object.
2. Let *map* be *args*.[[ParameterMap]].
3. Let *isMapped* be ! *HasOwnProperty*(*map*, *P*).
4. If *isMapped* is **false**, then
 - a. Return ? *OrdinaryGet*(*args*, *P*, *Receiver*).
5. Else *map* contains a formal parameter mapping for *P*,
 - a. Return *Get*(*map*, *P*).

9.4.4.4 `[[Set]]` (*P*, *V*, *Receiver*)

The `[[Set]]` internal method of an arguments *exotic object* when called with property key *P*, value *V*, and ECMAScript language value *Receiver* performs the following steps:

1. Let *args* be the arguments object.
2. If *SameValue*(*args*, *Receiver*) is **false**, then
 - a. Let *isMapped* be **false**.
3. Else,
 - a. Let *map* be *args*.[[ParameterMap]].
 - b. Let *isMapped* be ! *HasOwnProperty*(*map*, *P*).

4. If *isMapped* is **true**, then
 - a. Let *setStatus* be *Set*(*map*, *P*, *V*, **false**).
 - b. **Assert**: *setStatus* is **true** because formal parameters mapped by argument objects are always writable.
5. Return ? *OrdinarySet*(*args*, *P*, *V*, *Receiver*).

9.4.4.5 *[[Delete]]* (*P*)

The *[[Delete]]* internal method of an arguments *exotic object* when called with a property key *P* performs the following steps:

1. Let *args* be the arguments object.
2. Let *map* be *args*.*[[ParameterMap]]*.
3. Let *isMapped* be ! *HasOwnProperty*(*map*, *P*).
4. Let *result* be ? *OrdinaryDelete*(*args*, *P*).
5. If *result* is **true** and *isMapped* is **true**, then
 - a. Call *map*.*[[Delete]]*(*P*).
6. Return *result*.

9.4.4.6 *CreateUnmappedArgumentsObject* (*argumentsList*)

The abstract operation *CreateUnmappedArgumentsObject* called with an argument *argumentsList* performs the following steps:

1. Let *len* be the number of elements in *argumentsList*.
2. Let *obj* be *ObjectCreate*(%*ObjectPrototype*%, « *[[ParameterMap]]* »).
3. Set *obj*.*[[ParameterMap]]* to **undefined**.
4. Perform *DefinePropertyOrThrow*(*obj*, "**length**", PropertyDescriptor { *[[Value]]*: *len*, *[[Writable]]*: **true**, *[[Enumerable]]*: **false**, *[[Configurable]]*: **true** }).
5. Let *index* be 0.
6. Repeat, while *index* < *len*,
 - a. Let *val* be *argumentsList*[*index*].
 - b. Perform *CreateDataProperty*(*obj*, ! *ToString*(*index*), *val*).
 - c. Increase *index* by 1.
7. Perform ! *DefinePropertyOrThrow*(*obj*, @@*iterator*, PropertyDescriptor { *[[Value]]*: %*ArrayProto_values*%, *[[Writable]]*: **true**, *[[Enumerable]]*: **false**, *[[Configurable]]*: **true** }).
8. Perform ! *DefinePropertyOrThrow*(*obj*, "**callee**", PropertyDescriptor { *[[Get]]*: %*ThrowTypeError*%, *[[Set]]*: %*ThrowTypeError*%, *[[Enumerable]]*: **false**, *[[Configurable]]*: **false** }).
9. Return *obj*.

9.4.4.7 *CreateMappedArgumentsObject* (*func*, *formals*, *argumentsList*, *env*)

The abstract operation *CreateMappedArgumentsObject* is called with object *func*, *Parse Node formals*, *List argumentsList*, and *Environment Record env*. The following steps are performed:

1. **Assert**: *formals* does not contain a rest parameter, any binding patterns, or any initializers. It may contain duplicate identifiers.
2. Let *len* be the number of elements in *argumentsList*.
3. Let *obj* be a newly created arguments *exotic object* with a *[[ParameterMap]]* internal slot.
4. Set *obj*.*[[GetOwnProperty]]* as specified in 9.4.4.1.
5. Set *obj*.*[[DefineOwnProperty]]* as specified in 9.4.4.2.

6. Set *obj*.[[Get]] as specified in 9.4.4.3.
7. Set *obj*.[[Set]] as specified in 9.4.4.4.
8. Set *obj*.[[Delete]] as specified in 9.4.4.5.
9. Set the remainder of *obj*'s essential internal methods to the default ordinary object definitions specified in 9.1.
10. Set *obj*.[[Prototype]] to %ObjectPrototype%.
11. Set *obj*.[[Extensible]] to **true**.
12. Let *map* be ObjectCreate(**null**).
13. Set *obj*.[[ParameterMap]] to *map*.
14. Let *parameterNames* be the BoundNames of *formals*.
15. Let *numberOfParameters* be the number of elements in *parameterNames*.
16. Let *index* be 0.
17. Repeat, while *index* < *len*,
 - a. Let *val* be *argumentsList*[*index*].
 - b. Perform CreateDataProperty(*obj*, ! ToString(*index*), *val*).
 - c. Increase *index* by 1.
18. Perform DefinePropertyOrThrow(*obj*, "length", PropertyDescriptor { [[Value]]: *len*, [[Writable]]: **true**, [[Enumerable]]: **false**, [[Configurable]]: **true** }).
19. Let *mappedNames* be a new empty List.
20. Let *index* be *numberOfParameters* - 1.
21. Repeat, while *index* ≥ 0,
 - a. Let *name* be *parameterNames*[*index*].
 - b. If *name* is not an element of *mappedNames*, then
 - i. Add *name* as an element of the list *mappedNames*.
 - ii. If *index* < *len*, then
 1. Let *g* be MakeArgGetter(*name*, *env*).
 2. Let *p* be MakeArgSetter(*name*, *env*).
 3. Perform *map*.[[DefineOwnProperty]](! ToString(*index*), PropertyDescriptor { [[Set]]: *p*, [[Get]]: *g*, [[Enumerable]]: **false**, [[Configurable]]: **true** }).
 - c. Decrease *index* by 1.
22. Perform ! DefinePropertyOrThrow(*obj*, @@iterator, PropertyDescriptor { [[Value]]: %ArrayProto_values%, [[Writable]]: **true**, [[Enumerable]]: **false**, [[Configurable]]: **true** }).
23. Perform ! DefinePropertyOrThrow(*obj*, "callee", PropertyDescriptor { [[Value]]: *func*, [[Writable]]: **true**, [[Enumerable]]: **false**, [[Configurable]]: **true** }).
24. Return *obj*.

9.4.4.7.1 MakeArgGetter (*name*, *env*)

The abstract operation MakeArgGetter called with String *name* and Environment Record *env* creates a built-in function object that when executed returns the value bound for *name* in *env*. It performs the following steps:

1. Let *steps* be the steps of an ArgGetter function as specified below.
2. Let *getter* be CreateBuiltinFunction(*steps*, « [[Name]], [[Env]] »).
3. Set *getter*.[[Name]] to *name*.
4. Set *getter*.[[Env]] to *env*.
5. Return *getter*.

An ArgGetter function is an anonymous built-in function with [[Name]] and [[Env]] internal slots. When an ArgGetter function that expects no arguments is called it performs the following steps:

1. Let *f* be the [active function object](#).
2. Let *name* be *f*.[[Name]].
3. Let *env* be *f*.[[Env]].
4. Return *env*.GetBindingValue(*name*, **false**).

NOTE

ArgGetter functions are never directly accessible to ECMAScript code.

9.4.4.7.2 MakeArgSetter (*name*, *env*)

The abstract operation MakeArgSetter called with String *name* and [Environment Record](#) *env* creates a built-in [function object](#) that when executed sets the value bound for *name* in *env*. It performs the following steps:

1. Let *steps* be the steps of an ArgSetter function as specified below.
2. Let *setter* be [CreateBuiltinFunction](#)(*steps*, « [[Name]], [[Env]] »).
3. Set *setter*.[[Name]] to *name*.
4. Set *setter*.[[Env]] to *env*.
5. Return *setter*.

An ArgSetter function is an anonymous built-in function with [[Name]] and [[Env]] internal slots. When an ArgSetter function is called with argument *value* it performs the following steps:

1. Let *f* be the [active function object](#).
2. Let *name* be *f*.[[Name]].
3. Let *env* be *f*.[[Env]].
4. Return *env*.SetMutableBinding(*name*, *value*, **false**).

NOTE

ArgSetter functions are never directly accessible to ECMAScript code.

9.4.5 Integer-Indexed Exotic Objects

An Integer-Indexed exotic object is an [exotic object](#) that performs special handling of [integer index](#) property keys.

[Integer-Indexed exotic objects](#) have the same internal slots as ordinary objects and additionally [[ViewedArrayBuffer]], [[ArrayLength]], [[ByteOffset]], and [[TypedArrayName]] internal slots.

[Integer-Indexed exotic objects](#) provide alternative definitions for the following internal methods. All of the other [Integer-Indexed exotic object](#) essential internal methods that are not defined below are as specified in [9.1](#).

9.4.5.1 [[GetOwnProperty]] (*P*)

When the [[GetOwnProperty]] internal method of an [Integer-Indexed exotic object](#) *O* is called with property key *P*, the following steps are taken:

1. **Assert:** IsPropertyKey(*P*) is **true**.
2. **Assert:** *O* is an Object that has a [[ViewedArrayBuffer]] internal slot.
3. If Type(*P*) is String, then
 - a. Let *numericIndex* be ! [CanonicalNumericIndexString](#)(*P*).

- b. If *numericIndex* is not **undefined**, then
 - i. Let *value* be ? *IntegerIndexedElementGet*(*O*, *numericIndex*).
 - ii. If *value* is **undefined**, return **undefined**.
 - iii. Return a PropertyDescriptor { [[Value]]: *value*, [[Writable]]: **true**, [[Enumerable]]: **true**, [[Configurable]]: **false** }.
4. Return *OrdinaryGetOwnProperty*(*O*, *P*).

9.4.5.2 [[HasProperty]] (*P*)

When the [[HasProperty]] internal method of an *Integer-Indexed exotic object* *O* is called with property key *P*, the following steps are taken:

1. **Assert**: *IsPropertyKey*(*P*) is **true**.
2. **Assert**: *O* is an Object that has a [[ViewedArrayBuffer]] internal slot.
3. If *Type*(*P*) is String, then
 - a. Let *numericIndex* be ! *CanonicalNumericIndexString*(*P*).
 - b. If *numericIndex* is not **undefined**, then
 - i. Let *buffer* be *O*.[[ViewedArrayBuffer]].
 - ii. If *IsDetachedBuffer*(*buffer*) is **true**, throw a **TypeError** exception.
 - iii. If *IsInteger*(*numericIndex*) is **false**, return **false**.
 - iv. If *numericIndex* = **-0**, return **false**.
 - v. If *numericIndex* < 0, return **false**.
 - vi. If *numericIndex* ≥ *O*.[[ArrayLength]], return **false**.
 - vii. Return **true**.
4. Return ? *OrdinaryHasProperty*(*O*, *P*).

9.4.5.3 [[DefineOwnProperty]] (*P*, *Desc*)

When the [[DefineOwnProperty]] internal method of an *Integer-Indexed exotic object* *O* is called with property key *P*, and *Property Descriptor* *Desc*, the following steps are taken:

1. **Assert**: *IsPropertyKey*(*P*) is **true**.
2. **Assert**: *O* is an Object that has a [[ViewedArrayBuffer]] internal slot.
3. If *Type*(*P*) is String, then
 - a. Let *numericIndex* be ! *CanonicalNumericIndexString*(*P*).
 - b. If *numericIndex* is not **undefined**, then
 - i. If *IsInteger*(*numericIndex*) is **false**, return **false**.
 - ii. If *numericIndex* = **-0**, return **false**.
 - iii. If *numericIndex* < 0, return **false**.
 - iv. Let *length* be *O*.[[ArrayLength]].
 - v. If *numericIndex* ≥ *length*, return **false**.
 - vi. If *IsAccessorDescriptor*(*Desc*) is **true**, return **false**.
 - vii. If *Desc* has a [[Configurable]] field and if *Desc*.[[Configurable]] is **true**, return **false**.
 - viii. If *Desc* has an [[Enumerable]] field and if *Desc*.[[Enumerable]] is **false**, return **false**.
 - ix. If *Desc* has a [[Writable]] field and if *Desc*.[[Writable]] is **false**, return **false**.
 - x. If *Desc* has a [[Value]] field, then
 1. Let *value* be *Desc*.[[Value]].
 2. Return ? *IntegerIndexedElementSet*(*O*, *numericIndex*, *value*).
 - xi. Return **true**.

4. Return ! **OrdinaryDefineOwnProperty**(*O*, *P*, *Desc*).

9.4.5.4 **[[Get]]** (*P*, *Receiver*)

When the **[[Get]]** internal method of an **Integer-Indexed exotic object** *O* is called with property key *P* and **ECMAScript language value** *Receiver*, the following steps are taken:

1. **Assert**: **IsPropertyKey**(*P*) is **true**.
2. If **Type**(*P*) is String, then
 - a. Let *numericIndex* be ! **CanonicalNumericIndexString**(*P*).
 - b. If *numericIndex* is not **undefined**, then
 - i. Return ? **IntegerIndexedElementGet**(*O*, *numericIndex*).
3. Return ? **OrdinaryGet**(*O*, *P*, *Receiver*).

9.4.5.5 **[[Set]]** (*P*, *V*, *Receiver*)

When the **[[Set]]** internal method of an **Integer-Indexed exotic object** *O* is called with property key *P*, value *V*, and **ECMAScript language value** *Receiver*, the following steps are taken:

1. **Assert**: **IsPropertyKey**(*P*) is **true**.
2. If **Type**(*P*) is String, then
 - a. Let *numericIndex* be ! **CanonicalNumericIndexString**(*P*).
 - b. If *numericIndex* is not **undefined**, then
 - i. Return ? **IntegerIndexedElementSet**(*O*, *numericIndex*, *V*).
3. Return ? **OrdinarySet**(*O*, *P*, *V*, *Receiver*).

9.4.5.6 **[[OwnPropertyKeys]]** ()

When the **[[OwnPropertyKeys]]** internal method of an **Integer-Indexed exotic object** *O* is called, the following steps are taken:

1. Let *keys* be a new empty **List**.
2. **Assert**: *O* is an Object that has **[[ViewedArrayBuffer]]**, **[[ArrayLength]]**, **[[ByteOffset]]**, and **[[TypedArrayName]]** internal slots.
3. Let *len* be *O*.**[[ArrayLength]]**.
4. For each integer *i* starting with 0 such that *i* < *len*, in ascending order, do
 - a. Add ! **ToString**(*i*) as the last element of *keys*.
5. For each own property key *P* of *O* such that **Type**(*P*) is String and *P* is not an **integer index**, in ascending chronological order of property creation, do
 - a. Add *P* as the last element of *keys*.
6. For each own property key *P* of *O* such that **Type**(*P*) is Symbol, in ascending chronological order of property creation, do
 - a. Add *P* as the last element of *keys*.
7. Return *keys*.

9.4.5.7 **IntegerIndexedObjectCreate** (*prototype*, *internalSlotsList*)

The abstract operation **IntegerIndexedObjectCreate** with arguments *prototype* and *internalSlotsList* is used to specify the creation of new **Integer-Indexed exotic objects**. The argument *internalSlotsList* is a **List** of the names of additional internal slots that must be defined as part of the object. **IntegerIndexedObjectCreate** performs the following steps:

1. **Assert:** *internalSlotsList* contains the names `[[ViewedArrayBuffer]]`, `[[ArrayLength]]`, `[[ByteOffset]]`, and `[[TypedArrayName]]`.
2. Let *A* be a newly created object with an internal slot for each name in *internalSlotsList*.
3. Set *A*'s essential internal methods to the default ordinary object definitions specified in 9.1.
4. Set *A*.`[[GetOwnProperty]]` as specified in 9.4.5.1.
5. Set *A*.`[[HasProperty]]` as specified in 9.4.5.2.
6. Set *A*.`[[DefineOwnProperty]]` as specified in 9.4.5.3.
7. Set *A*.`[[Get]]` as specified in 9.4.5.4.
8. Set *A*.`[[Set]]` as specified in 9.4.5.5.
9. Set *A*.`[[OwnPropertyKeys]]` as specified in 9.4.5.6.
10. Set *A*.`[[Prototype]]` to *prototype*.
11. Set *A*.`[[Extensible]]` to **true**.
12. Return *A*.

9.4.5.8 IntegerIndexedElementGet (*O*, *index*)

The abstract operation IntegerIndexedElementGet with arguments *O* and *index* performs the following steps:

1. **Assert:** *Type*(*index*) is Number.
2. **Assert:** *O* is an Object that has `[[ViewedArrayBuffer]]`, `[[ArrayLength]]`, `[[ByteOffset]]`, and `[[TypedArrayName]]` internal slots.
3. Let *buffer* be *O*.`[[ViewedArrayBuffer]]`.
4. If *IsDetachedBuffer*(*buffer*) is **true**, throw a **TypeError** exception.
5. If *IsInteger*(*index*) is **false**, return **undefined**.
6. If *index* = **-0**, return **undefined**.
7. Let *length* be *O*.`[[ArrayLength]]`.
8. If *index* < 0 or *index* ≥ *length*, return **undefined**.
9. Let *offset* be *O*.`[[ByteOffset]]`.
10. Let *arrayTypeName* be the String value of *O*.`[[TypedArrayName]]`.
11. Let *elementSize* be the Number value of the Element Size value specified in Table 59 for *arrayTypeName*.
12. Let *indexedPosition* be (*index* × *elementSize*) + *offset*.
13. Let *elementType* be the String value of the Element Type value in Table 59 for *arrayTypeName*.
14. Return *GetValueFromBuffer*(*buffer*, *indexedPosition*, *elementType*, **true**, "Unordered").

9.4.5.9 IntegerIndexedElementSet (*O*, *index*, *value*)

The abstract operation IntegerIndexedElementSet with arguments *O*, *index*, and *value* performs the following steps:

1. **Assert:** *Type*(*index*) is Number.
2. **Assert:** *O* is an Object that has `[[ViewedArrayBuffer]]`, `[[ArrayLength]]`, `[[ByteOffset]]`, and `[[TypedArrayName]]` internal slots.
3. Let *numValue* be ? *ToNumber*(*value*).
4. Let *buffer* be *O*.`[[ViewedArrayBuffer]]`.
5. If *IsDetachedBuffer*(*buffer*) is **true**, throw a **TypeError** exception.
6. If *IsInteger*(*index*) is **false**, return **false**.
7. If *index* = **-0**, return **false**.
8. Let *length* be *O*.`[[ArrayLength]]`.
9. If *index* < 0 or *index* ≥ *length*, return **false**.
10. Let *offset* be *O*.`[[ByteOffset]]`.

11. Let *arrayTypeName* be the String value of *O*.[[TypedArrayName]].
12. Let *elementSize* be the Number value of the Element Size value specified in Table 59 for *arrayTypeName*.
13. Let *indexedPosition* be $(\text{index} \times \text{elementSize}) + \text{offset}$.
14. Let *elementType* be the String value of the Element Type value in Table 59 for *arrayTypeName*.
15. Perform *SetValueInBuffer*(*buffer*, *indexedPosition*, *elementType*, *numValue*, **true**, "Unordered").
16. Return **true**.

9.4.6 Module Namespace Exotic Objects

A *module namespace object* is an **exotic object** that exposes the bindings exported from an ECMAScript *Module* (See 15.2.3). There is a one-to-one correspondence between the String-keyed own properties of a module namespace **exotic object** and the binding names exported by the *Module*. The exported bindings include any bindings that are indirectly exported using **export *** export items. Each String-valued own property key is the StringValue of the corresponding exported binding name. These are the only String-keyed properties of a module namespace **exotic object**. Each such property has the attributes { [[Writable]]: **true**, [[Enumerable]]: **true**, [[Configurable]]: **false** }. Module namespace objects are not extensible.

Module namespace objects have the internal slots defined in Table 29.

Table 29: Internal Slots of Module Namespace Exotic Objects

Internal Slot	Type	Description
[[Module]]	Module Record	The Module Record whose exports this namespace exposes.
[[Exports]]	List of String	A List containing the String values of the exported names exposed as own properties of this object. The list is ordered as if an Array of those String values had been sorted using Array.prototype.sort using undefined as <i>comparefn</i> .
[[Prototype]]	Null	This slot always contains the value null (see 9.4.6.1).

Module namespace exotic objects provide alternative definitions for all of the internal methods except *[[GetPrototypeOf]]*, which behaves as defined in 9.1.1.

9.4.6.1 [[SetPrototypeOf]] (*V*)

When the *[[SetPrototypeOf]]* internal method of a module namespace **exotic object** *O* is called with argument *V*, the following steps are taken:

1. Return ? *SetImmutablePrototype*(*O*, *V*).

9.4.6.2 [[IsExtensible]] ()

When the *[[IsExtensible]]* internal method of a module namespace **exotic object** *O* is called, the following steps are taken:

1. Return **false**.

9.4.6.3 **[[PreventExtensions]]** ()

When the **[[PreventExtensions]]** internal method of a module namespace **exotic object** *O* is called, the following steps are taken:

1. Return **true**.

9.4.6.4 **[[GetOwnProperty]]** (*P*)

When the **[[GetOwnProperty]]** internal method of a module namespace **exotic object** *O* is called with property key *P*, the following steps are taken:

1. If **Type**(*P*) is Symbol, return **OrdinaryGetOwnProperty**(*O*, *P*).
2. Let *exports* be *O*.**[[Exports]]**.
3. If *P* is not an element of *exports*, return **undefined**.
4. Let *value* be ? *O*.**[[Get]]**(*P*, *O*).
5. Return PropertyDescriptor { **[[Value]]**: *value*, **[[Writable]]**: **true**, **[[Enumerable]]**: **true**, **[[Configurable]]**: **false** }.

9.4.6.5 **[[DefineOwnProperty]]** (*P*, *Desc*)

When the **[[DefineOwnProperty]]** internal method of a module namespace **exotic object** *O* is called with property key *P* and **Property Descriptor** *Desc*, the following steps are taken:

1. If **Type**(*P*) is Symbol, return **OrdinaryDefineOwnProperty**(*O*, *P*, *Desc*).
2. Let *current* be ? *O*.**[[GetOwnProperty]]**(*P*).
3. If *current* is **undefined**, return **false**.
4. If **IsAccessorDescriptor**(*Desc*) is **true**, return **false**.
5. If *Desc*.**[[Writable]]** is present and has value **false**, return **false**.
6. If *Desc*.**[[Enumerable]]** is present and has value **false**, return **false**.
7. If *Desc*.**[[Configurable]]** is present and has value **true**, return **false**.
8. If *Desc*.**[[Value]]** is present, return **SameValue**(*Desc*.**[[Value]]**, *current*.**[[Value]]**).
9. Return **true**.

9.4.6.6 **[[HasProperty]]** (*P*)

When the **[[HasProperty]]** internal method of a module namespace **exotic object** *O* is called with property key *P*, the following steps are taken:

1. If **Type**(*P*) is Symbol, return **OrdinaryHasProperty**(*O*, *P*).
2. Let *exports* be *O*.**[[Exports]]**.
3. If *P* is an element of *exports*, return **true**.
4. Return **false**.

9.4.6.7 **[[Get]]** (*P*, *Receiver*)

When the **[[Get]]** internal method of a module namespace **exotic object** *O* is called with property key *P* and **ECMAScript language value** *Receiver*, the following steps are taken:

1. **Assert**: **IsPropertyKey**(*P*) is **true**.
2. If **Type**(*P*) is Symbol, then
 - a. Return ? **OrdinaryGet**(*O*, *P*, *Receiver*).

3. Let *exports* be *O*.[[Exports]].
4. If *P* is not an element of *exports*, return **undefined**.
5. Let *m* be *O*.[[Module]].
6. Let *binding* be ! *m*.ResolveExport(*P*, « »).
7. **Assert**: *binding* is a ResolvedBinding Record.
8. Let *targetModule* be *binding*.[[Module]].
9. **Assert**: *targetModule* is not **undefined**.
10. Let *targetEnv* be *targetModule*.[[Environment]].
11. If *targetEnv* is **undefined**, throw a **ReferenceError** exception.
12. Let *targetEnvRec* be *targetEnv*'s EnvironmentRecord.
13. Return ? *targetEnvRec*.GetBindingValue(*binding*.[[BindingName]], **true**).

NOTE

ResolveExport is idempotent and side-effect free. An implementation might choose to pre-compute or cache the ResolveExport results for the [[Exports]] of each module namespace exotic object.

9.4.6.8 [[Set]] (*P*, *V*, *Receiver*)

When the [[Set]] internal method of a module namespace exotic object *O* is called with property key *P*, value *V*, and ECMAScript language value *Receiver*, the following steps are taken:

1. Return **false**.

9.4.6.9 [[Delete]] (*P*)

When the [[Delete]] internal method of a module namespace exotic object *O* is called with property key *P*, the following steps are taken:

1. **Assert**: IsPropertyKey(*P*) is **true**.
2. If Type(*P*) is Symbol, then
 - a. Return ? OrdinaryDelete(*O*, *P*).
3. Let *exports* be *O*.[[Exports]].
4. If *P* is an element of *exports*, return **false**.
5. Return **true**.

9.4.6.10 [[OwnPropertyKeys]] ()

When the [[OwnPropertyKeys]] internal method of a module namespace exotic object *O* is called, the following steps are taken:

1. Let *exports* be a copy of *O*.[[Exports]].
2. Let *symbolKeys* be ! OrdinaryOwnPropertyKeys(*O*).
3. Append all the entries of *symbolKeys* to the end of *exports*.
4. Return *exports*.

9.4.6.11 ModuleNamespaceCreate (*module*, *exports*)

The abstract operation ModuleNamespaceCreate with arguments *module*, and *exports* is used to specify the creation of new module namespace exotic objects. It performs the following steps:

1. **Assert:** *module* is a **Module Record**.
2. **Assert:** *module*.[[Namespace]] is **undefined**.
3. **Assert:** *exports* is a **List** of String values.
4. Let *M* be a newly created object.
5. Set *M*'s essential internal methods to the definitions specified in 9.4.6.
6. Set *M*.[[Module]] to *module*.
7. Let *sortedExports* be a new **List** containing the same values as the list *exports* where the values are ordered as if an Array of the same values had been sorted using **Array.prototype.sort** using **undefined** as *comparefn*.
8. Set *M*.[[Exports]] to *sortedExports*.
9. Create own properties of *M* corresponding to the definitions in 26.3.
10. Set *module*.[[Namespace]] to *M*.
11. Return *M*.

9.4.7 Immutable Prototype Exotic Objects

An immutable prototype exotic object is an **exotic object** that has a [[Prototype]] internal slot that will not change once it is initialized.

Immutable prototype exotic objects have the same internal slots as ordinary objects. They are exotic only in the following internal methods. All other internal methods of immutable prototype exotic objects that are not explicitly defined below are instead defined as in **ordinary objects**.

9.4.7.1 [[SetPrototypeOf]] (*V*)

When the [[SetPrototypeOf]] internal method of an **immutable prototype exotic object** *O* is called with argument *V*, the following steps are taken:

1. Return ? **SetImmutablePrototype**(*O*, *V*).

9.4.7.2 SetImmutablePrototype (*O*, *V*)

When the SetImmutablePrototype abstract operation is called with arguments *O* and *V*, the following steps are taken:

1. **Assert:** Either **Type**(*V*) is Object or **Type**(*V*) is Null.
2. Let *current* be ? *O*.[[GetPrototypeOf]]().
3. If **SameValue**(*V*, *current*) is **true**, return **true**.
4. Return **false**.

9.5 Proxy Object Internal Methods and Internal Slots

A proxy object is an **exotic object** whose essential internal methods are partially implemented using ECMAScript code. Every proxy object has an internal slot called [[ProxyHandler]]. The value of [[ProxyHandler]] is an object, called the proxy's *handler object*, or **null**. Methods (see Table 30) of a handler object may be used to augment the implementation for one or more of the proxy object's internal methods. Every proxy object also has an internal slot called [[ProxyTarget]] whose value is either an object or the **null** value. This object is called the proxy's *target object*.

Table 30: Proxy Handler Methods

Internal Method	Handler Method
-----------------	----------------

[[GetPrototypeOf]]	getPrototypeOf
[[SetPrototypeOf]]	setPrototypeOf
[[IsExtensible]]	isExtensible
[[PreventExtensions]]	preventExtensions
[[GetOwnProperty]]	getOwnPropertyDescriptor
[[DefineOwnProperty]]	defineProperty
[[HasProperty]]	has
[[Get]]	get
[[Set]]	set
[[Delete]]	deleteProperty
[[OwnPropertyKeys]]	ownKeys
[[Call]]	apply
[[Construct]]	construct

When a handler method is called to provide the implementation of a proxy object internal method, the handler method is passed the proxy's target object as a parameter. A proxy's handler object does not necessarily have a method corresponding to every essential internal method. Invoking an internal method on the proxy results in the invocation of the corresponding internal method on the proxy's target object if the handler object does not have a method corresponding to the internal trap.

The [[ProxyHandler]] and [[ProxyTarget]] internal slots of a proxy object are always initialized when the object is created and typically may not be modified. Some proxy objects are created in a manner that permits them to be subsequently *revoked*. When a proxy is revoked, its [[ProxyHandler]] and [[ProxyTarget]] internal slots are set to **null** causing subsequent invocations of internal methods on that proxy object to throw a **TypeError** exception.

Because proxy objects permit the implementation of internal methods to be provided by arbitrary ECMAScript code, it is possible to define a proxy object whose handler methods violates the invariants defined in 6.1.7.3. Some of the internal method invariants defined in 6.1.7.3 are essential integrity invariants. These invariants are explicitly enforced by the proxy object internal methods specified in this section. An ECMAScript implementation must be robust in the presence of all possible invariant violations.

In the following algorithm descriptions, assume *O* is an ECMAScript proxy object, *P* is a property key value, *V* is any ECMAScript language value and *Desc* is a **Property Descriptor** record.

9.5.1 [[GetPrototypeOf]] ()

When the [[GetPrototypeOf]] internal method of a Proxy **exotic object** *O* is called, the following steps are taken:

1. Let *handler* be *O*.[[ProxyHandler]].
2. If *handler* is **null**, throw a **TypeError** exception.
3. **Assert:** **Type**(*handler*) is Object.

4. Let *target* be *O*.[[ProxyTarget]].
5. Let *trap* be ? *GetMethod*(*handler*, "getPrototypeOf").
6. If *trap* is **undefined**, then
 - a. Return ? *target*.[[GetPrototypeOf]]().
7. Let *handlerProto* be ? *Call*(*trap*, *handler*, « *target* »).
8. If *Type*(*handlerProto*) is neither Object nor Null, throw a **TypeError** exception.
9. Let *extensibleTarget* be ? *IsExtensible*(*target*).
10. If *extensibleTarget* is **true**, return *handlerProto*.
11. Let *targetProto* be ? *target*.[[GetPrototypeOf]]().
12. If *SameValue*(*handlerProto*, *targetProto*) is **false**, throw a **TypeError** exception.
13. Return *handlerProto*.

NOTE

[[GetPrototypeOf]] for proxy objects enforces the following invariants:

The result of [[GetPrototypeOf]] must be either an Object or **null**.

If the target object is not extensible, [[GetPrototypeOf]] applied to the proxy object must return the same value as [[GetPrototypeOf]] applied to the proxy object's target object.

9.5.2 [[SetPrototypeOf]] (*V*)

When the [[SetPrototypeOf]] internal method of a Proxy *exotic object* *O* is called with argument *V*, the following steps are taken:

1. **Assert**: Either *Type*(*V*) is Object or *Type*(*V*) is Null.
2. Let *handler* be *O*.[[ProxyHandler]].
3. If *handler* is **null**, throw a **TypeError** exception.
4. **Assert**: *Type*(*handler*) is Object.
5. Let *target* be *O*.[[ProxyTarget]].
6. Let *trap* be ? *GetMethod*(*handler*, "setPrototypeOf").
7. If *trap* is **undefined**, then
 - a. Return ? *target*.[[SetPrototypeOf]](*V*).
8. Let *booleanTrapResult* be *ToBoolean*(? *Call*(*trap*, *handler*, « *target*, *V* »)).
9. If *booleanTrapResult* is **false**, return **false**.
10. Let *extensibleTarget* be ? *IsExtensible*(*target*).
11. If *extensibleTarget* is **true**, return **true**.
12. Let *targetProto* be ? *target*.[[GetPrototypeOf]]().
13. If *SameValue*(*V*, *targetProto*) is **false**, throw a **TypeError** exception.
14. Return **true**.

NOTE

[[SetPrototypeOf]] for proxy objects enforces the following invariants:

The result of [[SetPrototypeOf]] is a Boolean value.

If the target object is not extensible, the argument value must be the same as the result of [[GetPrototypeOf]] applied to target object.

9.5.3 `[[IsExtensible]]` ()

When the `[[IsExtensible]]` internal method of a Proxy *exotic object* *O* is called, the following steps are taken:

1. Let *handler* be *O*.[[ProxyHandler]].
2. If *handler* is **null**, throw a **TypeError** exception.
3. **Assert**: `Type(handler)` is Object.
4. Let *target* be *O*.[[ProxyTarget]].
5. Let *trap* be ? `GetMethod(handler, "isExtensible")`.
6. If *trap* is **undefined**, then
 - a. Return ? *target*.[[IsExtensible]]().
7. Let *booleanTrapResult* be `ToBoolean(? Call(trap, handler, « target »))`.
8. Let *targetResult* be ? *target*.[[IsExtensible]]().
9. If `SameValue(booleanTrapResult, targetResult)` is **false**, throw a **TypeError** exception.
10. Return *booleanTrapResult*.

NOTE

`[[IsExtensible]]` for proxy objects enforces the following invariants:

The result of `[[IsExtensible]]` is a Boolean value.

`[[IsExtensible]]` applied to the proxy object must return the same value as `[[IsExtensible]]` applied to the proxy object's target object with the same argument.

9.5.4 `[[PreventExtensions]]` ()

When the `[[PreventExtensions]]` internal method of a Proxy *exotic object* *O* is called, the following steps are taken:

1. Let *handler* be *O*.[[ProxyHandler]].
2. If *handler* is **null**, throw a **TypeError** exception.
3. **Assert**: `Type(handler)` is Object.
4. Let *target* be *O*.[[ProxyTarget]].
5. Let *trap* be ? `GetMethod(handler, "preventExtensions")`.
6. If *trap* is **undefined**, then
 - a. Return ? *target*.[[PreventExtensions]]().
7. Let *booleanTrapResult* be `ToBoolean(? Call(trap, handler, « target »))`.
8. If *booleanTrapResult* is **true**, then
 - a. Let *targetIsExtensible* be ? *target*.[[IsExtensible]]().
 - b. If *targetIsExtensible* is **true**, throw a **TypeError** exception.
9. Return *booleanTrapResult*.

NOTE

`[[PreventExtensions]]` for proxy objects enforces the following invariants:

The result of `[[PreventExtensions]]` is a Boolean value.

`[[PreventExtensions]]` applied to the proxy object only returns **true** if `[[IsExtensible]]` applied to the proxy object's target object is **false**.

9.5.5 [[GetOwnProperty]] (*P*)

When the [[GetOwnProperty]] internal method of a Proxy *exotic object* *O* is called with property key *P*, the following steps are taken:

1. **Assert:** IsPropertyKey(*P*) is **true**.
2. Let *handler* be *O*.[[ProxyHandler]].
3. If *handler* is **null**, throw a **TypeError** exception.
4. **Assert:** Type(*handler*) is Object.
5. Let *target* be *O*.[[ProxyTarget]].
6. Let *trap* be ? GetMethod(*handler*, "getOwnPropertyDescriptor").
7. If *trap* is **undefined**, then
 - a. Return ? *target*.[[GetOwnProperty]](*P*).
8. Let *trapResultObj* be ? Call(*trap*, *handler*, « *target*, *P* »).
9. If Type(*trapResultObj*) is neither Object nor Undefined, throw a **TypeError** exception.
10. Let *targetDesc* be ? *target*.[[GetOwnProperty]](*P*).
11. If *trapResultObj* is **undefined**, then
 - a. If *targetDesc* is **undefined**, return **undefined**.
 - b. If *targetDesc*.[[Configurable]] is **false**, throw a **TypeError** exception.
 - c. Let *extensibleTarget* be ? IsExtensible(*target*).
 - d. If *extensibleTarget* is **false**, throw a **TypeError** exception.
 - e. Return **undefined**.
12. Let *extensibleTarget* be ? IsExtensible(*target*).
13. Let *resultDesc* be ? ToPropertyDescriptor(*trapResultObj*).
14. Call CompletePropertyDescriptor(*resultDesc*).
15. Let *valid* be IsCompatiblePropertyDescriptor(*extensibleTarget*, *resultDesc*, *targetDesc*).
16. If *valid* is **false**, throw a **TypeError** exception.
17. If *resultDesc*.[[Configurable]] is **false**, then
 - a. If *targetDesc* is **undefined** or *targetDesc*.[[Configurable]] is **true**, then
 - i. Throw a **TypeError** exception.
18. Return *resultDesc*.

NOTE

[[GetOwnProperty]] for proxy objects enforces the following invariants:

The result of [[GetOwnProperty]] must be either an Object or **undefined**.

A property cannot be reported as non-existent, if it exists as a non-configurable own property of the target object.

A property cannot be reported as non-existent, if it exists as an own property of the target object and the target object is not extensible.

A property cannot be reported as existent, if it does not exist as an own property of the target object and the target object is not extensible.

A property cannot be reported as non-configurable, if it does not exist as an own property of the target object or if it exists as a configurable own property of the target object.

9.5.6 [[DefineOwnProperty]] (*P*, *Desc*)

When the [[DefineOwnProperty]] internal method of a Proxy *exotic object* *O* is called with property key *P* and *Property Descriptor* *Desc*, the following steps are taken:

1. **Assert:** `IsPropertyKey(P)` is **true**.
2. Let *handler* be `O.[[ProxyHandler]]`.
3. If *handler* is **null**, throw a **TypeError** exception.
4. **Assert:** `Type(handler)` is Object.
5. Let *target* be `O.[[ProxyTarget]]`.
6. Let *trap* be `? GetMethod(handler, "defineProperty")`.
7. If *trap* is **undefined**, then
 - a. Return `? target.[[DefineOwnProperty]](P, Desc)`.
8. Let *descObj* be `FromPropertyDescriptor(Desc)`.
9. Let *booleanTrapResult* be `ToBoolean(? Call(trap, handler, « target, P, descObj »))`.
10. If *booleanTrapResult* is **false**, return **false**.
11. Let *targetDesc* be `? target.[[GetOwnProperty]](P)`.
12. Let *extensibleTarget* be `? IsExtensible(target)`.
13. If *Desc* has a `[[Configurable]]` field and if *Desc*.`[[Configurable]]` is **false**, then
 - a. Let *settingConfigFalse* be **true**.
14. Else, let *settingConfigFalse* be **false**.
15. If *targetDesc* is **undefined**, then
 - a. If *extensibleTarget* is **false**, throw a **TypeError** exception.
 - b. If *settingConfigFalse* is **true**, throw a **TypeError** exception.
16. Else *targetDesc* is not **undefined**,
 - a. If `IsCompatiblePropertyDescriptor(extensibleTarget, Desc, targetDesc)` is **false**, throw a **TypeError** exception.
 - b. If *settingConfigFalse* is **true** and *targetDesc*.`[[Configurable]]` is **true**, throw a **TypeError** exception.
17. Return **true**.

NOTE

`[[DefineOwnProperty]]` for proxy objects enforces the following invariants:

The result of `[[DefineOwnProperty]]` is a Boolean value.

A property cannot be added, if the target object is not extensible.

A property cannot be non-configurable, unless there exists a corresponding non-configurable own property of the target object.

If a property has a corresponding target object property then applying the [Property Descriptor](#) of the property to the target object using `[[DefineOwnProperty]]` will not throw an exception.

9.5.7 `[[HasProperty]]` (*P*)

When the `[[HasProperty]]` internal method of a Proxy [exotic object](#) *O* is called with property key *P*, the following steps are taken:

1. **Assert:** `IsPropertyKey(P)` is **true**.
2. Let *handler* be `O.[[ProxyHandler]]`.
3. If *handler* is **null**, throw a **TypeError** exception.
4. **Assert:** `Type(handler)` is Object.
5. Let *target* be `O.[[ProxyTarget]]`.
6. Let *trap* be `? GetMethod(handler, "has")`.
7. If *trap* is **undefined**, then

- a. Return ? *target*.[[HasProperty]](*P*).
8. Let *booleanTrapResult* be ToBoolean(? Call(*trap*, *handler*, « *target*, *P* »)).
9. If *booleanTrapResult* is **false**, then
 - a. Let *targetDesc* be ? *target*.[[GetOwnProperty]](*P*).
 - b. If *targetDesc* is not **undefined**, then
 - i. If *targetDesc*.[[Configurable]] is **false**, throw a **TypeError** exception.
 - ii. Let *extensibleTarget* be ? IsExtensible(*target*).
 - iii. If *extensibleTarget* is **false**, throw a **TypeError** exception.
10. Return *booleanTrapResult*.

NOTE

[[HasProperty]] for proxy objects enforces the following invariants:

The result of [[HasProperty]] is a Boolean value.

A property cannot be reported as non-existent, if it exists as a non-configurable own property of the target object.

A property cannot be reported as non-existent, if it exists as an own property of the target object and the target object is not extensible.

9.5.8 [[Get]] (*P*, *Receiver*)

When the [[Get]] internal method of a Proxy *exotic object* *O* is called with property key *P* and ECMAScript language value *Receiver*, the following steps are taken:

1. Assert: IsPropertyKey(*P*) is **true**.
2. Let *handler* be *O*.[[ProxyHandler]].
3. If *handler* is **null**, throw a **TypeError** exception.
4. Assert: Type(*handler*) is Object.
5. Let *target* be *O*.[[ProxyTarget]].
6. Let *trap* be ? GetMethod(*handler*, "get").
7. If *trap* is **undefined**, then
 - a. Return ? *target*.[[Get]](*P*, *Receiver*).
8. Let *trapResult* be ? Call(*trap*, *handler*, « *target*, *P*, *Receiver* »).
9. Let *targetDesc* be ? *target*.[[GetOwnProperty]](*P*).
10. If *targetDesc* is not **undefined** and *targetDesc*.[[Configurable]] is **false**, then
 - a. If IsDataDescriptor(*targetDesc*) is **true** and *targetDesc*.[[Writable]] is **false**, then
 - i. If SameValue(*trapResult*, *targetDesc*.[[Value]]) is **false**, throw a **TypeError** exception.
 - b. If IsAccessorDescriptor(*targetDesc*) is **true** and *targetDesc*.[[Get]] is **undefined**, then
 - i. If *trapResult* is not **undefined**, throw a **TypeError** exception.
11. Return *trapResult*.

NOTE

[[Get]] for proxy objects enforces the following invariants:

The value reported for a property must be the same as the value of the corresponding target object property if the target object property is a non-writable, non-configurable own *data property*.

The value reported for a property must be **undefined** if the corresponding target object property is a non-configurable own *accessor property* that has **undefined** as its [[Get]] attribute.

9.5.9 `[[Set]]` (*P*, *V*, *Receiver*)

When the `[[Set]]` internal method of a Proxy *exotic object* *O* is called with property key *P*, value *V*, and ECMAScript language value *Receiver*, the following steps are taken:

1. **Assert**: `IsPropertyKey(P)` is **true**.
2. Let *handler* be *O*.`[[ProxyHandler]]`.
3. If *handler* is **null**, throw a **TypeError** exception.
4. **Assert**: `Type(handler)` is Object.
5. Let *target* be *O*.`[[ProxyTarget]]`.
6. Let *trap* be ? `GetMethod(handler, "set")`.
7. If *trap* is **undefined**, then
 - a. Return ? *target*.`[[Set]]`(*P*, *V*, *Receiver*).
8. Let *booleanTrapResult* be `ToBoolean(? Call(trap, handler, « target, P, V, Receiver »))`.
9. If *booleanTrapResult* is **false**, return **false**.
10. Let *targetDesc* be ? *target*.`[[GetOwnProperty]]`(*P*).
11. If *targetDesc* is not **undefined** and *targetDesc*.`[[Configurable]]` is **false**, then
 - a. If `IsDataDescriptor(targetDesc)` is **true** and *targetDesc*.`[[Writable]]` is **false**, then
 - i. If `SameValue(V, targetDesc. [[Value]])` is **false**, throw a **TypeError** exception.
 - b. If `IsAccessorDescriptor(targetDesc)` is **true**, then
 - i. If *targetDesc*.`[[Set]]` is **undefined**, throw a **TypeError** exception.
12. Return **true**.

NOTE

`[[Set]]` for proxy objects enforces the following invariants:

The result of `[[Set]]` is a Boolean value.

Cannot change the value of a property to be different from the value of the corresponding target object property if the corresponding target object property is a non-writable, non-configurable own *data property*.

Cannot set the value of a property if the corresponding target object property is a non-configurable own *accessor property* that has **undefined** as its `[[Set]]` attribute.

9.5.10 `[[Delete]]` (*P*)

When the `[[Delete]]` internal method of a Proxy *exotic object* *O* is called with property key *P*, the following steps are taken:

1. **Assert**: `IsPropertyKey(P)` is **true**.
2. Let *handler* be *O*.`[[ProxyHandler]]`.
3. If *handler* is **null**, throw a **TypeError** exception.
4. **Assert**: `Type(handler)` is Object.
5. Let *target* be *O*.`[[ProxyTarget]]`.
6. Let *trap* be ? `GetMethod(handler, "deleteProperty")`.
7. If *trap* is **undefined**, then
 - a. Return ? *target*.`[[Delete]]`(*P*).

8. Let *booleanTrapResult* be **ToBoolean**(? **Call**(*trap*, *handler*, « *target*, *P* »)).
9. If *booleanTrapResult* is **false**, return **false**.
10. Let *targetDesc* be ? *target*.[[**GetOwnProperty**]](*P*).
11. If *targetDesc* is **undefined**, return **true**.
12. If *targetDesc*.[[**Configurable**]] is **false**, throw a **TypeError** exception.
13. Return **true**.

NOTE

[[**Delete**]] for proxy objects enforces the following invariants:

The result of [[**Delete**]] is a Boolean value.

A property cannot be reported as deleted, if it exists as a non-configurable own property of the target object.

9.5.11 [[**OwnPropertyKeys**]] ()

When the [[**OwnPropertyKeys**]] internal method of a Proxy *exotic object* *O* is called, the following steps are taken:

1. Let *handler* be *O*.[[**ProxyHandler**]].
2. If *handler* is **null**, throw a **TypeError** exception.
3. **Assert**: **Type**(*handler*) is Object.
4. Let *target* be *O*.[[**ProxyTarget**]].
5. Let *trap* be ? **GetMethod**(*handler*, "**ownKeys**").
6. If *trap* is **undefined**, then
 - a. Return ? *target*.[[**OwnPropertyKeys**]]().
7. Let *trapResultArray* be ? **Call**(*trap*, *handler*, « *target* »).
8. Let *trapResult* be ? **CreateListFromArrayLike**(*trapResultArray*, « String, Symbol »).
9. If *trapResult* contains any duplicate entries, throw a **TypeError** exception.
10. Let *extensibleTarget* be ? **IsExtensible**(*target*).
11. Let *targetKeys* be ? *target*.[[**OwnPropertyKeys**]]().
12. **Assert**: *targetKeys* is a **List** containing only String and Symbol values.
13. **Assert**: *targetKeys* contains no duplicate entries.
14. Let *targetConfigurableKeys* be a new empty **List**.
15. Let *targetNonconfigurableKeys* be a new empty **List**.
16. For each element *key* of *targetKeys*, do
 - a. Let *desc* be ? *target*.[[**GetOwnProperty**]](*key*).
 - b. If *desc* is not **undefined** and *desc*.[[**Configurable**]] is **false**, then
 - i. Append *key* as an element of *targetNonconfigurableKeys*.
 - c. Else,
 - i. Append *key* as an element of *targetConfigurableKeys*.
17. If *extensibleTarget* is **true** and *targetNonconfigurableKeys* is empty, then
 - a. Return *trapResult*.
18. Let *uncheckedResultKeys* be a new **List** which is a copy of *trapResult*.
19. For each *key* that is an element of *targetNonconfigurableKeys*, do
 - a. If *key* is not an element of *uncheckedResultKeys*, throw a **TypeError** exception.
 - b. Remove *key* from *uncheckedResultKeys*.
20. If *extensibleTarget* is **true**, return *trapResult*.
21. For each *key* that is an element of *targetConfigurableKeys*, do

- a. If *key* is not an element of *uncheckedResultKeys*, throw a **TypeError** exception.
 - b. Remove *key* from *uncheckedResultKeys*.
22. If *uncheckedResultKeys* is not empty, throw a **TypeError** exception.
 23. Return *trapResult*.

NOTE

[[OwnPropertyKeys]] for proxy objects enforces the following invariants:

The result of [[OwnPropertyKeys]] is a *List*.

The returned *List* contains no duplicate entries.

The Type of each result *List* element is either String or Symbol.

The result *List* must contain the keys of all non-configurable own properties of the target object.

If the target object is not extensible, then the result *List* must contain all the keys of the own properties of the target object and no other values.

9.5.12 [[Call]] (*thisArgument*, *argumentsList*)

The [[Call]] internal method of a Proxy *exotic object* *O* is called with parameters *thisArgument* and *argumentsList*, a *List* of ECMAScript language values. The following steps are taken:

1. Let *handler* be *O*.[[ProxyHandler]].
2. If *handler* is **null**, throw a **TypeError** exception.
3. **Assert**: *Type(handler)* is Object.
4. Let *target* be *O*.[[ProxyTarget]].
5. Let *trap* be ? *GetMethod(handler, "apply")*.
6. If *trap* is **undefined**, then
 - a. Return ? *Call(target, thisArgument, argumentsList)*.
7. Let *argArray* be *CreateArrayFromList(argumentsList)*.
8. Return ? *Call(trap, handler, « target, thisArgument, argArray »)*.

NOTE

A Proxy *exotic object* only has a [[Call]] internal method if the initial value of its [[ProxyTarget]] internal slot is an object that has a [[Call]] internal method.

9.5.13 [[Construct]] (*argumentsList*, *newTarget*)

The [[Construct]] internal method of a Proxy *exotic object* *O* is called with parameters *argumentsList* which is a possibly empty *List* of ECMAScript language values and *newTarget*. The following steps are taken:

1. Let *handler* be *O*.[[ProxyHandler]].
2. If *handler* is **null**, throw a **TypeError** exception.
3. **Assert**: *Type(handler)* is Object.
4. Let *target* be *O*.[[ProxyTarget]].
5. **Assert**: *IsConstructor(target)* is **true**.
6. Let *trap* be ? *GetMethod(handler, "construct")*.
7. If *trap* is **undefined**, then
 - a. Return ? *Construct(target, argumentsList, newTarget)*.

8. Let *argArray* be `CreateArrayFromList(argumentsList)`.
9. Let *newObj* be `? Call(trap, handler, « target, argArray, newTarget »)`.
10. If `Type(newObj)` is not `Object`, throw a **TypeError** exception.
11. Return *newObj*.

NOTE 1

A Proxy *exotic object* only has a `[[Construct]]` internal method if the initial value of its `[[ProxyTarget]]` internal slot is an object that has a `[[Construct]]` internal method.

NOTE 2

`[[Construct]]` for proxy objects enforces the following invariants:

The result of `[[Construct]]` must be an `Object`.

9.5.14 ProxyCreate (*target*, *handler*)

The abstract operation ProxyCreate with arguments *target* and *handler* is used to specify the creation of new Proxy exotic objects. It performs the following steps:

1. If `Type(target)` is not `Object`, throw a **TypeError** exception.
2. If *target* is a Proxy *exotic object* and *target*.`[[ProxyHandler]]` is **null**, throw a **TypeError** exception.
3. If `Type(handler)` is not `Object`, throw a **TypeError** exception.
4. If *handler* is a Proxy *exotic object* and *handler*.`[[ProxyHandler]]` is **null**, throw a **TypeError** exception.
5. Let *P* be a newly created object.
6. Set *P*'s essential internal methods (except for `[[Call]]` and `[[Construct]]`) to the definitions specified in 9.5.
7. If `IsCallable(target)` is **true**, then
 - a. Set *P*.`[[Call]]` as specified in 9.5.12.
 - b. If `IsConstructor(target)` is **true**, then
 - i. Set *P*.`[[Construct]]` as specified in 9.5.13.
8. Set *P*.`[[ProxyTarget]]` to *target*.
9. Set *P*.`[[ProxyHandler]]` to *handler*.
10. Return *P*.

10 ECMAScript Language: Source Code

10.1 Source Text

Syntax

SourceCharacter ::

any Unicode code point

ECMAScript code is expressed using Unicode. ECMAScript source text is a sequence of code points. All Unicode code point values from U+0000 to U+10FFFF, including surrogate code points, may occur in source text where permitted by the ECMAScript grammars. The actual encodings used to store and interchange ECMAScript source text is not relevant to this specification. Regardless of the external source text encoding, a conforming ECMAScript implementation

processes the source text as if it was an equivalent sequence of *SourceCharacter* values, each *SourceCharacter* being a Unicode code point. Conforming ECMAScript implementations are not required to perform any normalization of source text, or behave as though they were performing normalization of source text.

The components of a combining character sequence are treated as individual Unicode code points even though a user might think of the whole sequence as a single character.

NOTE

In string literals, regular expression literals, template literals and identifiers, any Unicode code point may also be expressed using Unicode escape sequences that explicitly express a code point's numeric value. Within a comment, such an escape sequence is effectively ignored as part of the comment.

ECMAScript differs from the Java programming language in the behaviour of Unicode escape sequences. In a Java program, if the Unicode escape sequence `\u000A`, for example, occurs within a single-line comment, it is interpreted as a line terminator (Unicode code point U+000A is LINE FEED (LF)) and therefore the next code point is not part of the comment. Similarly, if the Unicode escape sequence `\u000A` occurs within a string literal in a Java program, it is likewise interpreted as a line terminator, which is not allowed within a string literal—one must write `\n` instead of `\u000A` to cause a LINE FEED (LF) to be part of the String value of a string literal. In an ECMAScript program, a Unicode escape sequence occurring within a comment is never interpreted and therefore cannot contribute to termination of the comment. Similarly, a Unicode escape sequence occurring within a string literal in an ECMAScript program always contributes to the literal and is never interpreted as a line terminator or as a code point that might terminate the string literal.

10.1.1 Static Semantics: UTF16Encoding (*cp*)

The UTF16Encoding of a numeric code point value, *cp*, is determined as follows:

1. **Assert:** $0 \leq cp \leq 0x10FFFF$.
2. If $cp \leq 0xFFFF$, return *cp*.
3. Let *cu1* be $\text{floor}((cp - 0x10000) / 0x400) + 0xD800$.
4. Let *cu2* be $((cp - 0x10000) \bmod 0x400) + 0xDC00$.
5. Return the code unit sequence consisting of *cu1* followed by *cu2*.

10.1.2 Static Semantics: UTF16Decode (*lead*, *trail*)

Two code units, *lead* and *trail*, that form a UTF-16 surrogate pair are converted to a code point by performing the following steps:

1. **Assert:** *lead* is a leading surrogate and *trail* is a trailing surrogate.
2. Let *cp* be $(\text{lead} - 0xD800) \times 0x400 + (\text{trail} - 0xDC00) + 0x10000$.
3. Return the code point *cp*.

10.2 Types of Source Code

There are four types of ECMAScript code:

Global code is source text that is treated as an ECMAScript *Script*. The global code of a particular *Script* does not include any source text that is parsed as part of a *FunctionDeclaration*, *FunctionExpression*, *GeneratorDeclaration*,

GeneratorExpression, *AsyncFunctionDeclaration*, *AsyncFunctionExpression*, *AsyncGeneratorDeclaration*, *AsyncGeneratorExpression*, *MethodDefinition*, *ArrowFunction*, *AsyncArrowFunction*, *ClassDeclaration*, or *ClassExpression*.

Eval code is the source text supplied to the built-in **eval** function. More precisely, if the parameter to the built-in **eval** function is a String, it is treated as an ECMAScript *Script*. The eval code for a particular invocation of **eval** is the global code portion of that *Script*.

Function code is source text that is parsed to supply the value of the `[[ECMAScriptCode]]` and `[[FormalParameters]]` internal slots (see 9.2) of an ECMAScript *function object*. The function code of a particular ECMAScript function does not include any source text that is parsed as the function code of a nested *FunctionDeclaration*, *FunctionExpression*, *GeneratorDeclaration*, *GeneratorExpression*, *AsyncFunctionDeclaration*, *AsyncFunctionExpression*, *AsyncGeneratorDeclaration*, *AsyncGeneratorExpression*, *MethodDefinition*, *ArrowFunction*, *AsyncArrowFunction*, *ClassDeclaration*, or *ClassExpression*.

Module code is source text that is code that is provided as a *ModuleBody*. It is the code that is directly evaluated when a module is initialized. The module code of a particular module does not include any source text that is parsed as part of a nested *FunctionDeclaration*, *FunctionExpression*, *GeneratorDeclaration*, *GeneratorExpression*, *AsyncFunctionDeclaration*, *AsyncFunctionExpression*, *AsyncGeneratorDeclaration*, *AsyncGeneratorExpression*, *MethodDefinition*, *ArrowFunction*, *AsyncArrowFunction*, *ClassDeclaration*, or *ClassExpression*.

NOTE

Function code is generally provided as the bodies of Function Definitions (14.1), Arrow Function Definitions (14.2), Method Definitions (14.3), Generator Function Definitions (14.4), Async Function Definitions (14.7), Async Generator Function Definitions (14.5), and Async Arrow Functions (14.8). Function code is also derived from the arguments to the **Function constructor** (19.2.1.1), the **GeneratorFunction constructor** (25.2.1.1), and the **AsyncFunction constructor** (25.7.1.1).

10.2.1 Strict Mode Code

An ECMAScript *Script* syntactic unit may be processed using either unrestricted or strict mode syntax and semantics. Code is interpreted as strict mode code in the following situations:

Global code is strict mode code if it begins with a *Directive Prologue* that contains a *Use Strict Directive*.

Module code is always strict mode code.

All parts of a *ClassDeclaration* or a *ClassExpression* are strict mode code.

Eval code is strict mode code if it begins with a *Directive Prologue* that contains a *Use Strict Directive* or if the call to **eval** is a *direct eval* that is contained in strict mode code.

Function code is strict mode code if the associated *FunctionDeclaration*, *FunctionExpression*, *GeneratorDeclaration*, *GeneratorExpression*, *AsyncFunctionDeclaration*, *AsyncFunctionExpression*, *AsyncGeneratorDeclaration*, *AsyncGeneratorExpression*, *MethodDefinition*, *ArrowFunction*, or *AsyncArrowFunction* is contained in strict mode code or if the code that produces the value of the function's `[[ECMAScriptCode]]` internal slot begins with a *Directive Prologue* that contains a *Use Strict Directive*.

Function code that is supplied as the arguments to the built-in **Function**, **Generator**, **AsyncFunction**, and **AsyncGenerator** constructors is strict mode code if the last argument is a String that when processed is a *FunctionBody* that begins with a *Directive Prologue* that contains a *Use Strict Directive*.

ECMAScript code that is not strict mode code is called non-strict code.

10.2.2 Non-ECMAScript Functions

An ECMAScript implementation may support the evaluation of function exotic objects whose evaluative behaviour is expressed in some implementation-defined form of executable code other than via ECMAScript code. Whether a [function object](#) is an ECMAScript code function or a non-ECMAScript function is not semantically observable from the perspective of an ECMAScript code function that calls or is called by such a non-ECMAScript function.

11 ECMAScript Language: Lexical Grammar

The source text of an ECMAScript *Script* or *Module* is first converted into a sequence of input elements, which are tokens, line terminators, comments, or white space. The source text is scanned from left to right, repeatedly taking the longest possible sequence of code points as the next input element.

There are several situations where the identification of lexical input elements is sensitive to the syntactic grammar context that is consuming the input elements. This requires multiple goal symbols for the lexical grammar. The *InputElementRegExpOrTemplateTail* goal is used in syntactic grammar contexts where a *RegularExpressionLiteral*, a *TemplateMiddle*, or a *TemplateTail* is permitted. The *InputElementRegExp* [goal symbol](#) is used in all syntactic grammar contexts where a *RegularExpressionLiteral* is permitted but neither a *TemplateMiddle*, nor a *TemplateTail* is permitted. The *InputElementTemplateTail* goal is used in all syntactic grammar contexts where a *TemplateMiddle* or a *TemplateTail* is permitted but a *RegularExpressionLiteral* is not permitted. In all other contexts, *InputElementDiv* is used as the lexical [goal symbol](#).

NOTE

The use of multiple lexical goals ensures that there are no lexical ambiguities that would affect automatic semicolon insertion. For example, there are no syntactic grammar contexts where both a leading division or division-assignment, and a leading *RegularExpressionLiteral* are permitted. This is not affected by semicolon insertion (see [11.9](#)); in examples such as the following:

```
a = b
/hi/g.exec(c).map(d);
```

where the first non-whitespace, non-comment code point after a *LineTerminator* is U+002F (SOLIDUS) and the syntactic context allows division or division-assignment, no semicolon is inserted at the *LineTerminator*. That is, the above example is interpreted in the same way as:

```
a = b / hi / g.exec(c).map(d);
```

Syntax

```
InputElementDiv ::
    WhiteSpace
    LineTerminator
    Comment
    CommonToken
    DivPunctuator
    RightBracePunctuator
```

```
InputElementRegExp ::
```

WhiteSpace
LineTerminator
Comment
CommonToken
RightBracePunctuator
RegularExpressionLiteral

InputElementRegExpOrTemplateTail ::

WhiteSpace
LineTerminator
Comment
CommonToken
RegularExpressionLiteral
TemplateSubstitutionTail

InputElementTemplateTail ::

WhiteSpace
LineTerminator
Comment
CommonToken
DivPunctuator
TemplateSubstitutionTail

11.1 Unicode Format-Control Characters

The Unicode format-control characters (i.e., the characters in category “Cf” in the Unicode Character Database such as LEFT-TO-RIGHT MARK or RIGHT-TO-LEFT MARK) are control codes used to control the formatting of a range of text in the absence of higher-level protocols for this (such as mark-up languages).

It is useful to allow format-control characters in source text to facilitate editing and display. All format control characters may be used within comments, and within string literals, template literals, and regular expression literals.

U+200C (ZERO WIDTH NON-JOINER) and U+200D (ZERO WIDTH JOINER) are format-control characters that are used to make necessary distinctions when forming words or phrases in certain languages. In ECMAScript source text these code points may also be used in an *IdentifierName* after the first character.

U+FEFF (ZERO WIDTH NO-BREAK SPACE) is a format-control character used primarily at the start of a text to mark it as Unicode and to allow detection of the text's encoding and byte order. <ZWNBSP> characters intended for this purpose can sometimes also appear after the start of a text, for example as a result of concatenating files. In ECMAScript source text <ZWNBSP> code points are treated as white space characters (see 11.2).

The special treatment of certain format-control characters outside of comments, string literals, and regular expression literals is summarized in Table 31.

Table 31: Format-Control Code Point Usage

Code Point	Name	Abbreviation	Usage
U+200C	ZERO WIDTH NON-JOINER	<ZWNJ>	<i>IdentifierPart</i>
U+200D	ZERO WIDTH JOINER	<ZWJ>	<i>IdentifierPart</i>

11.2 White Space

White space code points are used to improve source text readability and to separate tokens (indivisible lexical units) from each other, but are otherwise insignificant. White space code points may occur between any two tokens and at the start or end of input. White space code points may occur within a *StringLiteral*, a *RegularExpressionLiteral*, a *Template*, or a *TemplateSubstitutionTail* where they are considered significant code points forming part of a literal value. They may also occur within a *Comment*, but cannot appear within any other kind of token.

The ECMAScript white space code points are listed in [Table 32](#).

Table 32: White Space Code Points		
Code Point	Name	Abbreviation
U+0009	CHARACTER TABULATION	<TAB>
U+000B	LINE TABULATION	<VT>
U+000C	FORM FEED (FF)	<FF>
U+0020	SPACE	<SP>
U+00A0	NO-BREAK SPACE	<NBSP>
U+FEFF	ZERO WIDTH NO-BREAK SPACE	<ZWNBSP>
Other category “Zs” Any other Unicode “Space_Separator” code point		<USP>

ECMAScript implementations must recognize as *WhiteSpace* code points listed in the “Space_Separator” (“Zs”) category.

NOTE
Other than for the code points listed in [Table 32](#), ECMAScript *WhiteSpace* intentionally excludes all code points that have the Unicode “White_Space” property but which are not classified in category “Space_Separator” (“Zs”).

Syntax

WhiteSpace ::

<TAB>

<VT>

<FF>

<SP>

<NBSP>

<ZWNBSP>

<USP>

11.3 Line Terminators

Like white space code points, line terminator code points are used to improve source text readability and to separate tokens (indivisible lexical units) from each other. However, unlike white space code points, line terminators have some influence over the behaviour of the syntactic grammar. In general, line terminators may occur between any two tokens, but there are a few places where they are forbidden by the syntactic grammar. Line terminators also affect the process of automatic semicolon insertion (11.9). A line terminator cannot occur within any token except a *StringLiteral*, *Template*, or *TemplateSubstitutionTail*. <LF> and <CR> line terminators cannot occur within a *StringLiteral* token except as part of a *LineContinuation*.

A line terminator can occur within a *MultiLineComment* but cannot occur within a *SingleLineComment*.

Line terminators are included in the set of white space code points that are matched by the `\s` class in regular expressions.

The ECMAScript line terminator code points are listed in Table 33.

Table 33: Line Terminator Code Points

Code Point	Unicode Name	Abbreviation
U+000A	LINE FEED (LF)	<LF>
U+000D	CARRIAGE RETURN (CR)	<CR>
U+2028	LINE SEPARATOR	<LS>
U+2029	PARAGRAPH SEPARATOR	<PS>

Only the Unicode code points in Table 33 are treated as line terminators. Other new line or line breaking Unicode code points are not treated as line terminators but are treated as white space if they meet the requirements listed in Table 32. The sequence <CR><LF> is commonly used as a line terminator. It should be considered a single *SourceCharacter* for the purpose of reporting line numbers.

Syntax

LineTerminator ::

- <LF>
- <CR>
- <LS>
- <PS>

LineTerminatorSequence ::

- <LF>
- <CR>[lookahead ≠ <LF>]
- <LS>
- <PS>
- <CR><LF>

11.4 Comments

Comments can be either single or multi-line. Multi-line comments cannot nest.

Because a single-line comment can contain any Unicode code point except a *LineTerminator* code point, and because of the general rule that a token is always as long as possible, a single-line comment always consists of all code points from the *//* marker to the end of the line. However, the *LineTerminator* at the end of the line is not considered to be part of the single-line comment; it is recognized separately by the lexical grammar and becomes part of the stream of input elements for the syntactic grammar. This point is very important, because it implies that the presence or absence of single-line comments does not affect the process of automatic semicolon insertion (see 11.9).

Comments behave like white space and are discarded except that, if a *MultiLineComment* contains a line terminator code point, then the entire comment is considered to be a *LineTerminator* for purposes of parsing by the syntactic grammar.

Syntax

Comment ::

MultiLineComment

SingleLineComment

MultiLineComment ::

/ MultiLineCommentChars_{opt} */*

MultiLineCommentChars ::

MultiLineNotAsteriskChar MultiLineCommentChars_{opt}

** PostAsteriskCommentChars_{opt}*

PostAsteriskCommentChars ::

MultiLineNotForwardSlashOrAsteriskChar MultiLineCommentChars_{opt}

** PostAsteriskCommentChars_{opt}*

MultiLineNotAsteriskChar ::

SourceCharacter but not ***

MultiLineNotForwardSlashOrAsteriskChar ::

SourceCharacter but not one of */* or ***

SingleLineComment ::

// SingleLineCommentChars_{opt}

SingleLineCommentChars ::

SingleLineCommentChar SingleLineCommentChars_{opt}

SingleLineCommentChar ::

SourceCharacter but not *LineTerminator*

11.5 Tokens

Syntax

CommonToken ::

IdentifierName

Punctuator
NumericLiteral
StringLiteral
Template

NOTE

The *DivPunctuator*, *RegularExpressionLiteral*, *RightBracePunctuator*, and *TemplateSubstitutionTail* productions derive additional tokens that are not included in the *CommonToken* production.

11.6 Names and Keywords

IdentifierName and *ReservedWord* are tokens that are interpreted according to the Default Identifier Syntax given in Unicode Standard Annex #31, Identifier and Pattern Syntax, with some small modifications. *ReservedWord* is an enumerated subset of *IdentifierName*. The syntactic grammar defines *Identifier* as an *IdentifierName* that is not a *ReservedWord*. The Unicode identifier grammar is based on character properties specified by the Unicode Standard. The Unicode code points in the specified categories in the latest version of the Unicode standard must be treated as in those categories by all conforming ECMAScript implementations. ECMAScript implementations may recognize identifier code points defined in later editions of the Unicode Standard.

NOTE 1

This standard specifies specific code point additions: U+0024 (DOLLAR SIGN) and U+005F (LOW LINE) are permitted anywhere in an *IdentifierName*, and the code points U+200C (ZERO WIDTH NON-JOINER) and U+200D (ZERO WIDTH JOINER) are permitted anywhere after the first code point of an *IdentifierName*.

Unicode escape sequences are permitted in an *IdentifierName*, where they contribute a single Unicode code point to the *IdentifierName*. The code point is expressed by the *CodePoint* of the *UnicodeEscapeSequence* (see 11.8.4). The `\` preceding the *UnicodeEscapeSequence* and the `u` and `{ }` code units, if they appear, do not contribute code points to the *IdentifierName*. A *UnicodeEscapeSequence* cannot be used to put a code point into an *IdentifierName* that would otherwise be illegal. In other words, if a `\ UnicodeEscapeSequence` sequence were replaced by the *SourceCharacter* it contributes, the result must still be a valid *IdentifierName* that has the exact same sequence of *SourceCharacter* elements as the original *IdentifierName*. All interpretations of *IdentifierName* within this specification are based upon their actual code points regardless of whether or not an escape sequence was used to contribute any particular code point.

Two *IdentifierNames* that are canonically equivalent according to the Unicode standard are *not* equal unless, after replacement of each *UnicodeEscapeSequence*, they are represented by the exact same sequence of code points.

Syntax

IdentifierName ::
 IdentifierStart
 IdentifierName *IdentifierPart*

IdentifierStart ::
 UnicodeIDStart
 \$
 —
 \ *UnicodeEscapeSequence*

IdentifierPart ::

UnicodeIDContinue

\$

\ *UnicodeEscapeSequence*

<ZWJ>

<ZWJ>

UnicodeIDStart ::

any Unicode code point with the Unicode property “ID_Start”

UnicodeIDContinue ::

any Unicode code point with the Unicode property “ID_Continue”

The definitions of the nonterminal *UnicodeEscapeSequence* is given in [11.8.4](#).

NOTE 2

The nonterminal *IdentifierPart* derives `_` via *UnicodeIDContinue*.

NOTE 3

The sets of code points with Unicode properties “ID_Start” and “ID_Continue” include, respectively, the code points with Unicode properties “Other_ID_Start” and “Other_ID_Continue”.

11.6.1 Identifier Names

11.6.1.1 Static Semantics: Early Errors

IdentifierStart :: \ *UnicodeEscapeSequence*

It is a Syntax Error if $SV(UnicodeEscapeSequence)$ is none of "\$", or "_", or the [UTF16Encoding](#) of a code point matched by the *UnicodeIDStart* lexical grammar production.

IdentifierPart :: \ *UnicodeEscapeSequence*

It is a Syntax Error if $SV(UnicodeEscapeSequence)$ is none of "\$", or "_", or the [UTF16Encoding](#) of either <ZWJ> or <ZWJ>, or the [UTF16Encoding](#) of a Unicode code point that would be matched by the *UnicodeIDContinue* lexical grammar production.

11.6.1.2 Static Semantics: StringValue

IdentifierName ::

IdentifierStart

IdentifierName *IdentifierPart*

1. Return the String value consisting of the sequence of code units corresponding to *IdentifierName*. In determining the sequence any occurrences of \ *UnicodeEscapeSequence* are first replaced with the code point represented by the *UnicodeEscapeSequence* and then the code points of the entire *IdentifierName* are converted to code units by [UTF16Encoding](#) each code point.

11.6.2 Reserved Words

A reserved word is an *IdentifierName* that cannot be used as an *Identifier*.

Syntax

ReservedWord ::
 Keyword
 FutureReservedWord
 NullLiteral
 BooleanLiteral

NOTE

The *ReservedWord* definitions are specified as literal sequences of specific *SourceCharacter* elements. A code point in a *ReservedWord* cannot be expressed by a `\ UnicodeEscapeSequence`.

11.6.2.1 Keywords

The following tokens are ECMAScript keywords and may not be used as *Identifiers* in ECMAScript programs.

Syntax

Keyword :: **one of**
 await break case catch class const continue debugger default delete do
 else export extends finally for function if import in instanceof new
 return super switch this throw try typeof var void while with yield

NOTE

In some contexts **yield** and **await** are given the semantics of an *Identifier*. See 12.1.1. In [strict mode code](#), **let** and **static** are treated as reserved words through static semantic restrictions (see 12.1.1, 13.3.1.1, 13.7.5.1, and 14.6.1) rather than the lexical grammar.

11.6.2.2 Future Reserved Words

The following tokens are reserved for use as keywords in future language extensions.

Syntax

FutureReservedWord ::
 enum

NOTE

Use of the following tokens within [strict mode code](#) is also reserved. That usage is restricted using static semantic restrictions (see 12.1.1) rather than the lexical grammar:

implements package protected
interface private public

11.7 Punctuators

Syntax

Punctuator :: **one of**

{ () [] . . . ; , < > <= >= == != === !== + - * % ** ++ -- << >> >>> & | ^ ! ~
&& || ? : = += -= *= %= **= <=> >>= &= |= ^= =>

DivPunctuator ::

/
/=

RightBracePunctuator ::

}

11.8 Literals

11.8.1 Null Literals

Syntax

NullLiteral ::

null

11.8.2 Boolean Literals

Syntax

BooleanLiteral ::

true
false

11.8.3 Numeric Literals

Syntax

NumericLiteral ::

DecimalLiteral
BinaryIntegerLiteral
OctalIntegerLiteral
HexIntegerLiteral

DecimalLiteral ::

DecimalIntegerLiteral . *DecimalDigits*_{opt} *ExponentPart*_{opt}
. *DecimalDigits* *ExponentPart*_{opt}
DecimalIntegerLiteral *ExponentPart*_{opt}

DecimalIntegerLiteral ::

0
NonZeroDigit *DecimalDigits*_{opt}

DecimalDigits ::
 DecimalDigit
 DecimalDigits *DecimalDigit*

DecimalDigit :: **one of**
 0 1 2 3 4 5 6 7 8 9

NonZeroDigit :: **one of**
 1 2 3 4 5 6 7 8 9

ExponentPart ::
 ExponentIndicator *SignedInteger*

ExponentIndicator :: **one of**
 e E

SignedInteger ::
 DecimalDigits
 + *DecimalDigits*
 - *DecimalDigits*

BinaryIntegerLiteral ::
 0b *BinaryDigits*
 0B *BinaryDigits*

BinaryDigits ::
 BinaryDigit
 BinaryDigits *BinaryDigit*

BinaryDigit :: **one of**
 0 1

OctalIntegerLiteral ::
 0o *OctalDigits*
 0O *OctalDigits*

OctalDigits ::
 OctalDigit
 OctalDigits *OctalDigit*

OctalDigit :: **one of**
 0 1 2 3 4 5 6 7

HexIntegerLiteral ::
 0x *HexDigits*
 0X *HexDigits*

HexDigits ::
 HexDigit
 HexDigits *HexDigit*

HexDigit :: **one of**

The *SourceCharacter* immediately following a *NumericLiteral* must not be an *IdentifierStart* or *DecimalDigit*.

NOTE

For example: **3in** is an error and not the two input elements **3** and **in**.

A conforming implementation, when processing *strict mode code*, must not extend, as described in B.1.1, the syntax of *NumericLiteral* to include *LegacyOctalIntegerLiteral*, nor extend the syntax of *DecimalIntegerLiteral* to include *NonOctalDecimalIntegerLiteral*.

11.8.3.1 Static Semantics: MV

A numeric literal stands for a value of the Number type. This value is determined in two steps: first, a mathematical value (MV) is derived from the literal; second, this mathematical value is rounded as described below.

The MV of *NumericLiteral* :: *DecimalLiteral* is the MV of *DecimalLiteral*.

The MV of *NumericLiteral* :: *BinaryIntegerLiteral* is the MV of *BinaryIntegerLiteral*.

The MV of *NumericLiteral* :: *OctalIntegerLiteral* is the MV of *OctalIntegerLiteral*.

The MV of *NumericLiteral* :: *HexIntegerLiteral* is the MV of *HexIntegerLiteral*.

The MV of *DecimalLiteral* :: *DecimalIntegerLiteral* . is the MV of *DecimalIntegerLiteral*.

The MV of *DecimalLiteral* :: *DecimalIntegerLiteral* . *DecimalDigits* is the MV of *DecimalIntegerLiteral* plus (the MV of *DecimalDigits* $\times 10^{-n}$), where *n* is the number of code points in *DecimalDigits*.

The MV of *DecimalLiteral* :: *DecimalIntegerLiteral* . *ExponentPart* is the MV of *DecimalIntegerLiteral* $\times 10^e$, where *e* is the MV of *ExponentPart*.

The MV of *DecimalLiteral* :: *DecimalIntegerLiteral* . *DecimalDigits* *ExponentPart* is (the MV of *DecimalIntegerLiteral* plus (the MV of *DecimalDigits* $\times 10^{-n}$)) $\times 10^e$, where *n* is the number of code points in *DecimalDigits* and *e* is the MV of *ExponentPart*.

The MV of *DecimalLiteral* :: . *DecimalDigits* is the MV of *DecimalDigits* $\times 10^{-n}$, where *n* is the number of code points in *DecimalDigits*.

The MV of *DecimalLiteral* :: . *DecimalDigits* *ExponentPart* is the MV of *DecimalDigits* $\times 10^{e-n}$, where *n* is the number of code points in *DecimalDigits* and *e* is the MV of *ExponentPart*.

The MV of *DecimalLiteral* :: *DecimalIntegerLiteral* is the MV of *DecimalIntegerLiteral*.

The MV of *DecimalLiteral* :: *DecimalIntegerLiteral* *ExponentPart* is the MV of *DecimalIntegerLiteral* $\times 10^e$, where *e* is the MV of *ExponentPart*.

The MV of *DecimalIntegerLiteral* :: 0 is 0.

The MV of *DecimalIntegerLiteral* :: *NonZeroDigit* is the MV of *NonZeroDigit*.

The MV of *DecimalIntegerLiteral* :: *NonZeroDigit* *DecimalDigits* is (the MV of *NonZeroDigit* $\times 10^n$) plus the MV of *DecimalDigits*, where *n* is the number of code points in *DecimalDigits*.

The MV of *DecimalDigits* :: *DecimalDigit* is the MV of *DecimalDigit*.

The MV of *DecimalDigits* :: *DecimalDigits* *DecimalDigit* is (the MV of *DecimalDigits* $\times 10$) plus the MV of *DecimalDigit*.

The MV of *ExponentPart* :: *ExponentIndicator* *SignedInteger* is the MV of *SignedInteger*.

The MV of *SignedInteger* :: *DecimalDigits* is the MV of *DecimalDigits*.

The MV of *SignedInteger* :: + *DecimalDigits* is the MV of *DecimalDigits*.

The MV of *SignedInteger* :: - *DecimalDigits* is the negative of the MV of *DecimalDigits*.

The MV of *DecimalDigit* :: 0 or of *HexDigit* :: 0 or of *OctalDigit* :: 0 or of *BinaryDigit* :: 0 is 0.

The MV of *DecimalDigit* :: **1** or of *NonZeroDigit* :: **1** or of *HexDigit* :: **1** or of *OctalDigit* :: **1** or of *BinaryDigit* :: **1** is 1.

The MV of *DecimalDigit* :: **2** or of *NonZeroDigit* :: **2** or of *HexDigit* :: **2** or of *OctalDigit* :: **2** is 2.

The MV of *DecimalDigit* :: **3** or of *NonZeroDigit* :: **3** or of *HexDigit* :: **3** or of *OctalDigit* :: **3** is 3.

The MV of *DecimalDigit* :: **4** or of *NonZeroDigit* :: **4** or of *HexDigit* :: **4** or of *OctalDigit* :: **4** is 4.

The MV of *DecimalDigit* :: **5** or of *NonZeroDigit* :: **5** or of *HexDigit* :: **5** or of *OctalDigit* :: **5** is 5.

The MV of *DecimalDigit* :: **6** or of *NonZeroDigit* :: **6** or of *HexDigit* :: **6** or of *OctalDigit* :: **6** is 6.

The MV of *DecimalDigit* :: **7** or of *NonZeroDigit* :: **7** or of *HexDigit* :: **7** or of *OctalDigit* :: **7** is 7.

The MV of *DecimalDigit* :: **8** or of *NonZeroDigit* :: **8** or of *HexDigit* :: **8** is 8.

The MV of *DecimalDigit* :: **9** or of *NonZeroDigit* :: **9** or of *HexDigit* :: **9** is 9.

The MV of *HexDigit* :: **a** or of *HexDigit* :: **A** is 10.

The MV of *HexDigit* :: **b** or of *HexDigit* :: **B** is 11.

The MV of *HexDigit* :: **c** or of *HexDigit* :: **C** is 12.

The MV of *HexDigit* :: **d** or of *HexDigit* :: **D** is 13.

The MV of *HexDigit* :: **e** or of *HexDigit* :: **E** is 14.

The MV of *HexDigit* :: **f** or of *HexDigit* :: **F** is 15.

The MV of *BinaryIntegerLiteral* :: **0b** *BinaryDigits* is the MV of *BinaryDigits*.

The MV of *BinaryIntegerLiteral* :: **0B** *BinaryDigits* is the MV of *BinaryDigits*.

The MV of *BinaryDigits* :: *BinaryDigit* is the MV of *BinaryDigit*.

The MV of *BinaryDigits* :: *BinaryDigits* *BinaryDigit* is (the MV of *BinaryDigits* × 2) plus the MV of *BinaryDigit*.

The MV of *OctalIntegerLiteral* :: **0o** *OctalDigits* is the MV of *OctalDigits*.

The MV of *OctalIntegerLiteral* :: **0O** *OctalDigits* is the MV of *OctalDigits*.

The MV of *OctalDigits* :: *OctalDigit* is the MV of *OctalDigit*.

The MV of *OctalDigits* :: *OctalDigits* *OctalDigit* is (the MV of *OctalDigits* × 8) plus the MV of *OctalDigit*.

The MV of *HexIntegerLiteral* :: **0x** *HexDigits* is the MV of *HexDigits*.

The MV of *HexIntegerLiteral* :: **0X** *HexDigits* is the MV of *HexDigits*.

The MV of *HexDigits* :: *HexDigit* is the MV of *HexDigit*.

The MV of *HexDigits* :: *HexDigits* *HexDigit* is (the MV of *HexDigits* × 16) plus the MV of *HexDigit*.

Once the exact MV for a numeric literal has been determined, it is then rounded to a value of the *Number* type. If the MV is 0, then the rounded value is **+0**; otherwise, the rounded value must be the *Number* value for the MV (as specified in 6.1.6), unless the literal is a *DecimalLiteral* and the literal has more than 20 significant digits, in which case the *Number* value may be either the *Number* value for the MV of a literal produced by replacing each significant digit after the 20th with a **0** digit or the *Number* value for the MV of a literal produced by replacing each significant digit after the 20th with a **0** digit and then incrementing the literal at the 20th significant digit position. A digit is *significant* if it is not part of an *ExponentPart* and

it is not **0**; or

there is a nonzero digit to its left and there is a nonzero digit, not in the *ExponentPart*, to its right.

11.8.4 String Literals

NOTE 1

A string literal is zero or more Unicode code points enclosed in single or double quotes. Unicode code points may also be represented by an escape sequence. All code points may appear literally in a string literal except for the closing quote code points, U+005C (REVERSE SOLIDUS), U+000D (CARRIAGE RETURN), and U+000A (LINE FEED). Any code points may appear in the form of an escape sequence. String literals evaluate to ECMAScript String values. When

generating these String values Unicode code points are UTF-16 encoded as defined in 10.1.1. Code points belonging to the Basic Multilingual Plane are encoded as a single code unit element of the string. All other code points are encoded as two code unit elements of the string.

Syntax

StringLiteral ::

" *DoubleStringCharacters*_{opt} "
' *SingleStringCharacters*_{opt} '

DoubleStringCharacters ::

DoubleStringCharacter *DoubleStringCharacters*_{opt}

SingleStringCharacters ::

SingleStringCharacter *SingleStringCharacters*_{opt}

DoubleStringCharacter ::

SourceCharacter but not one of " or \ or *LineTerminator*
<LS>
<PS>
\ *EscapeSequence*
LineContinuation

SingleStringCharacter ::

SourceCharacter but not one of ' or \ or *LineTerminator*
<LS>
<PS>
\ *EscapeSequence*
LineContinuation

LineContinuation ::

\ *LineTerminatorSequence*

EscapeSequence ::

CharacterEscapeSequence
0 [lookahead $\notin$ *DecimalDigit*]
HexEscapeSequence
UnicodeEscapeSequence

A conforming implementation, when processing *strict mode code*, must not extend the syntax of *EscapeSequence* to include *LegacyOctalEscapeSequence* as described in B.1.2.

CharacterEscapeSequence ::

SingleEscapeCharacter
NonEscapeCharacter

SingleEscapeCharacter :: **one of**

' " \ **b f n r t v**

NonEscapeCharacter ::

SourceCharacter but not one of *EscapeCharacter* or *LineTerminator*

EscapeCharacter ::

SingleEscapeCharacter

DecimalDigit

x

u

HexEscapeSequence ::

x *HexDigit* *HexDigit*

UnicodeEscapeSequence ::

u *Hex4Digits*

u { *CodePoint* }

Hex4Digits ::

HexDigit *HexDigit* *HexDigit* *HexDigit*

The definition of the nonterminal *HexDigit* is given in 11.8.3. *SourceCharacter* is defined in 10.1.

NOTE 2

<LF> and <CR> cannot appear in a string literal, except as part of a *LineContinuation* to produce the empty code points sequence. The proper way to include either in the String value of a string literal is to use an escape sequence such as `\n` or `\u000A`.

11.8.4.1 Static Semantics: StringValue

StringLiteral ::

" *DoubleStringCharacters*_{opt} "

' *SingleStringCharacters*_{opt} '

1. Return the String value whose code units are the SV of this *StringLiteral*.

11.8.4.2 Static Semantics: SV

A string literal stands for a value of the String type. The String value (SV) of the literal is described in terms of code unit values contributed by the various parts of the string literal. As part of this process, some Unicode code points within the string literal are interpreted as having a mathematical value (MV), as described below or in 11.8.3.

The SV of *StringLiteral* :: " " is the empty code unit sequence.

The SV of *StringLiteral* :: ' ' is the empty code unit sequence.

The SV of *StringLiteral* :: " *DoubleStringCharacters* " is the SV of *DoubleStringCharacters*.

The SV of *StringLiteral* :: ' *SingleStringCharacters* ' is the SV of *SingleStringCharacters*.

The SV of *DoubleStringCharacters* :: *DoubleStringCharacter* is a sequence of up to two code units that is the SV of *DoubleStringCharacter*.

The SV of *DoubleStringCharacters* :: *DoubleStringCharacter* *DoubleStringCharacters* is a sequence of up to two code units that is the SV of *DoubleStringCharacter* followed by the code units of the SV of *DoubleStringCharacters* in order.

The SV of *SingleStringCharacters* :: *SingleStringCharacter* is a sequence of up to two code units that is the SV

of *SingleStringCharacter*.

The SV of *SingleStringCharacters* :: *SingleStringCharacter SingleStringCharacters* is a sequence of up to two code units that is the SV of *SingleStringCharacter* followed by the code units of the SV of *SingleStringCharacters* in order.

The SV of *DoubleStringCharacter* :: *SourceCharacter* but not one of " or \ or *LineTerminator* is the [UTF16Encoding](#) of the code point value of *SourceCharacter*.

The SV of *DoubleStringCharacter* :: <LS> is the code unit 0x2028 (LINE SEPARATOR).

The SV of *DoubleStringCharacter* :: <PS> is the code unit 0x2029 (PARAGRAPH SEPARATOR).

The SV of *DoubleStringCharacter* :: \ *EscapeSequence* is the SV of the *EscapeSequence*.

The SV of *DoubleStringCharacter* :: *LineContinuation* is the empty code unit sequence.

The SV of *SingleStringCharacter* :: *SourceCharacter* but not one of ' or \ or *LineTerminator* is the [UTF16Encoding](#) of the code point value of *SourceCharacter*.

The SV of *SingleStringCharacter* :: <LS> is the code unit 0x2028 (LINE SEPARATOR).

The SV of *SingleStringCharacter* :: <PS> is the code unit 0x2029 (PARAGRAPH SEPARATOR).

The SV of *SingleStringCharacter* :: \ *EscapeSequence* is the SV of the *EscapeSequence*.

The SV of *SingleStringCharacter* :: *LineContinuation* is the empty code unit sequence.

The SV of *EscapeSequence* :: *CharacterEscapeSequence* is the SV of the *CharacterEscapeSequence*.

The SV of *EscapeSequence* :: 0 is the code unit 0x0000 (NULL).

The SV of *EscapeSequence* :: *HexEscapeSequence* is the SV of the *HexEscapeSequence*.

The SV of *EscapeSequence* :: *UnicodeEscapeSequence* is the SV of the *UnicodeEscapeSequence*.

The SV of *CharacterEscapeSequence* :: *SingleEscapeCharacter* is the code unit whose value is determined by the *SingleEscapeCharacter* according to [Table 34](#).

Table 34: String Single Character Escape Sequences

Escape Sequence	Code Unit Value	Unicode Character Name	Symbol
\b	0x0008	BACKSPACE	<BS>
\t	0x0009	CHARACTER TABULATION	<HT>
\n	0x000A	LINE FEED (LF)	<LF>
\v	0x000B	LINE TABULATION	<VT>
\f	0x000C	FORM FEED (FF)	<FF>
\r	0x000D	CARRIAGE RETURN (CR)	<CR>
\"	0x0022	QUOTATION MARK	"
\'	0x0027	APOSTROPHE	'
\\	0x005C	REVERSE SOLIDUS	\

The SV of *CharacterEscapeSequence* :: *NonEscapeCharacter* is the SV of the *NonEscapeCharacter*.

The SV of *NonEscapeCharacter* :: *SourceCharacter* but not one of *EscapeCharacter* or *LineTerminator* is the [UTF16Encoding](#) of the code point value of *SourceCharacter*.

The SV of *HexEscapeSequence* :: x *HexDigit HexDigit* is the code unit whose value is (16 times the MV of the first *HexDigit*) plus the MV of the second *HexDigit*.

The SV of *UnicodeEscapeSequence* :: u *Hex4Digits* is the SV of *Hex4Digits*.

The SV of *Hex4Digits* :: *HexDigit HexDigit HexDigit HexDigit* is the code unit whose value is (0x1000 times the MV of the first *HexDigit*) plus (0x100 times the MV of the second *HexDigit*) plus (0x10 times the MV of the third *HexDigit*) plus the MV of the fourth *HexDigit*.

The SV of *UnicodeEscapeSequence* :: **u**{ *CodePoint* } is the [UTF16Encoding](#) of the MV of *CodePoint*.

11.8.5 Regular Expression Literals

NOTE 1

A regular expression literal is an input element that is converted to a **RegExp** object (see [21.2](#)) each time the literal is evaluated. Two regular expression literals in a program evaluate to regular expression objects that never compare as **===** to each other even if the two literals' contents are identical. A **RegExp** object may also be created at runtime by **new RegExp** or calling the **RegExp constructor** as a function (see [21.2.3](#)).

The productions below describe the syntax for a regular expression literal and are used by the input element scanner to find the end of the regular expression literal. The source text comprising the *RegularExpressionBody* and the *RegularExpressionFlags* are subsequently parsed again using the more stringent ECMAScript Regular Expression grammar ([21.2.1](#)).

An implementation may extend the ECMAScript Regular Expression grammar defined in [21.2.1](#), but it must not extend the *RegularExpressionBody* and *RegularExpressionFlags* productions defined below or the productions used by these productions.

Syntax

RegularExpressionLiteral ::
 / *RegularExpressionBody* / *RegularExpressionFlags*

RegularExpressionBody ::
 RegularExpressionFirstChar *RegularExpressionChars*

RegularExpressionChars ::
 [empty]
 RegularExpressionChars *RegularExpressionChar*

RegularExpressionFirstChar ::
 RegularExpressionNonTerminator but not one of * or \ or / or [
 RegularExpressionBackslashSequence
 RegularExpressionClass

RegularExpressionChar ::
 RegularExpressionNonTerminator but not one of \ or / or [
 RegularExpressionBackslashSequence
 RegularExpressionClass

RegularExpressionBackslashSequence ::
 \ *RegularExpressionNonTerminator*

RegularExpressionNonTerminator ::
 SourceCharacter but not *LineTerminator*

RegularExpressionClass ::

[*RegularExpressionClassChars*]

RegularExpressionClassChars ::

[empty]

RegularExpressionClassChars *RegularExpressionClassChar*

RegularExpressionClassChar ::

RegularExpressionNonTerminator but not one of] or \

RegularExpressionBackslashSequence

RegularExpressionFlags ::

[empty]

RegularExpressionFlags *IdentifierPart*

NOTE 2

Regular expression literals may not be empty; instead of representing an empty regular expression literal, the code unit sequence `//` starts a single-line comment. To specify an empty regular expression, use: `/(?:)/`.

11.8.5.1 Static Semantics: Early Errors

RegularExpressionFlags :: *RegularExpressionFlags* *IdentifierPart*

It is a Syntax Error if *IdentifierPart* contains a Unicode escape sequence.

11.8.5.2 Static Semantics: BodyText

RegularExpressionLiteral :: / *RegularExpressionBody* / *RegularExpressionFlags*

1. Return the source text that was recognized as *RegularExpressionBody*.

11.8.5.3 Static Semantics: FlagText

RegularExpressionLiteral :: / *RegularExpressionBody* / *RegularExpressionFlags*

1. Return the source text that was recognized as *RegularExpressionFlags*.

11.8.6 Template Literal Lexical Components

Syntax

Template ::

NoSubstitutionTemplate

TemplateHead

NoSubstitutionTemplate ::

⋈ *TemplateCharacters*_{opt} ⋈

TemplateHead ::

⋈ *TemplateCharacters*_{opt} \${

TemplateSubstitutionTail ::

TemplateMiddle

TemplateTail

TemplateMiddle ::

 } *TemplateCharacters*_{opt} \${

TemplateTail ::

 } *TemplateCharacters*_{opt} `

TemplateCharacters ::

TemplateCharacter *TemplateCharacters*_{opt}

TemplateCharacter ::

 \$ [lookahead ≠ {}]

 \ *EscapeSequence*

 \ *NotEscapeSequence*

LineContinuation

LineTerminatorSequence

SourceCharacter but not one of ` or \ or \$ or *LineTerminator*

NotEscapeSequence ::

 0 *DecimalDigit*

DecimalDigit but not 0

 x [lookahead ≠ *HexDigit*]

 x *HexDigit* [lookahead ≠ *HexDigit*]

 u [lookahead ≠ *HexDigit*] [lookahead ≠ {}]

 u *HexDigit* [lookahead ≠ *HexDigit*]

 u *HexDigit* *HexDigit* [lookahead ≠ *HexDigit*]

 u *HexDigit* *HexDigit* *HexDigit* [lookahead ≠ *HexDigit*]

 u { [lookahead ≠ *HexDigit*]

 u { *NotCodePoint* [lookahead ≠ *HexDigit*]

 u { *CodePoint* [lookahead ≠ *HexDigit*] [lookahead ≠ {}]

NotCodePoint ::

HexDigits but only if MV of *HexDigits* > 0x10FFFF

CodePoint ::

HexDigits but only if MV of *HexDigits* ≤ 0x10FFFF

A conforming implementation must not use the extended definition of *EscapeSequence* described in [B.1.2](#) when parsing a *TemplateCharacter*.

NOTE

TemplateSubstitutionTail is used by the *InputElementTemplateTail* alternative lexical goal.

11.8.6.1 Static Semantics: TV and TRV

A template literal component is interpreted as a sequence of Unicode code points. The Template Value (TV) of a literal component is described in terms of code unit values (SV, [11.8.4](#)) contributed by the various parts of the template literal component. As part of this process, some Unicode code points within the template component are interpreted as having a mathematical value (MV, [11.8.3](#)). In determining a TV, escape sequences are replaced by the UTF-16 code unit(s) of the Unicode code point represented by the escape sequence. The Template Raw Value (TRV) is similar to a Template Value

with the difference that in TRVs escape sequences are interpreted literally.

The TV and TRV of *NoSubstitutionTemplate* :: `` is the empty code unit sequence.

The TV and TRV of *TemplateHead* :: ` \${ is the empty code unit sequence.

The TV and TRV of *TemplateMiddle* :: } \${ is the empty code unit sequence.

The TV and TRV of *TemplateTail* :: } ` is the empty code unit sequence.

The TV of *NoSubstitutionTemplate* :: ` *TemplateCharacters* ` is the TV of *TemplateCharacters*.

The TV of *TemplateHead* :: ` *TemplateCharacters* \${ is the TV of *TemplateCharacters*.

The TV of *TemplateMiddle* :: } *TemplateCharacters* \${ is the TV of *TemplateCharacters*.

The TV of *TemplateTail* :: } *TemplateCharacters* ` is the TV of *TemplateCharacters*.

The TV of *TemplateCharacters* :: *TemplateCharacter* is the TV of *TemplateCharacter*.

The TV of *TemplateCharacters* :: *TemplateCharacter* *TemplateCharacters* is **undefined** if either the TV of *TemplateCharacter* is **undefined** or the TV of *TemplateCharacters* is **undefined**. Otherwise, it is a sequence consisting of the code units of the TV of *TemplateCharacter* followed by the code units of the TV of *TemplateCharacters*.

The TV of *TemplateCharacter* :: *SourceCharacter* but not one of ` or \ or \$ or *LineTerminator* is the [UTF16Encoding](#) of the code point value of *SourceCharacter*.

The TV of *TemplateCharacter* :: \$ is the code unit 0x0024 (DOLLAR SIGN).

The TV of *TemplateCharacter* :: \ *EscapeSequence* is the SV of *EscapeSequence*.

The TV of *TemplateCharacter* :: \ *NotEscapeSequence* is **undefined**.

The TV of *TemplateCharacter* :: *LineContinuation* is the TV of *LineContinuation*.

The TV of *TemplateCharacter* :: *LineTerminatorSequence* is the TRV of *LineTerminatorSequence*.

The TV of *LineContinuation* :: \ *LineTerminatorSequence* is the empty code unit sequence.

The TRV of *NoSubstitutionTemplate* :: ` *TemplateCharacters* ` is the TRV of *TemplateCharacters*.

The TRV of *TemplateHead* :: ` *TemplateCharacters* \${ is the TRV of *TemplateCharacters*.

The TRV of *TemplateMiddle* :: } *TemplateCharacters* \${ is the TRV of *TemplateCharacters*.

The TRV of *TemplateTail* :: } *TemplateCharacters* ` is the TRV of *TemplateCharacters*.

The TRV of *TemplateCharacters* :: *TemplateCharacter* is the TRV of *TemplateCharacter*.

The TRV of *TemplateCharacters* :: *TemplateCharacter* *TemplateCharacters* is a sequence consisting of the code units of the TRV of *TemplateCharacter* followed by the code units of the TRV of *TemplateCharacters*.

The TRV of *TemplateCharacter* :: *SourceCharacter* but not one of ` or \ or \$ or *LineTerminator* is the [UTF16Encoding](#) of the code point value of *SourceCharacter*.

The TRV of *TemplateCharacter* :: \$ is the code unit 0x0024 (DOLLAR SIGN).

The TRV of *TemplateCharacter* :: \ *EscapeSequence* is the sequence consisting of the code unit 0x005C (REVERSE SOLIDUS) followed by the code units of TRV of *EscapeSequence*.

The TRV of *TemplateCharacter* :: \ *NotEscapeSequence* is the sequence consisting of the code unit 0x005C (REVERSE SOLIDUS) followed by the code units of TRV of *NotEscapeSequence*.

The TRV of *TemplateCharacter* :: *LineContinuation* is the TRV of *LineContinuation*.

The TRV of *TemplateCharacter* :: *LineTerminatorSequence* is the TRV of *LineTerminatorSequence*.

The TRV of *EscapeSequence* :: *CharacterEscapeSequence* is the TRV of the *CharacterEscapeSequence*.

The TRV of *EscapeSequence* :: 0 is the code unit 0x0030 (DIGIT ZERO).

The TRV of *EscapeSequence* :: *HexEscapeSequence* is the TRV of the *HexEscapeSequence*.

The TRV of *EscapeSequence* :: *UnicodeEscapeSequence* is the TRV of the *UnicodeEscapeSequence*.

The TRV of *NotEscapeSequence* :: 0 *DecimalDigit* is the sequence consisting of the code unit 0x0030 (DIGIT ZERO) followed by the code units of the TRV of *DecimalDigit*.

The TRV of *NotEscapeSequence* :: x [lookahead $\notin$ *HexDigit*] is the code unit 0x0078 (LATIN SMALL LETTER X).

The TRV of *NotEscapeSequence* :: **x** *HexDigit* [lookahead $\notin$ *HexDigit*] is the sequence consisting of the code unit 0x0078 (LATIN SMALL LETTER X) followed by the code units of the TRV of *HexDigit*.

The TRV of *NotEscapeSequence* :: **u** [lookahead $\notin$ *HexDigit*] [lookahead $\neq$ {] is the code unit 0x0075 (LATIN SMALL LETTER U).

The TRV of *NotEscapeSequence* :: **u** *HexDigit* [lookahead $\notin$ *HexDigit*] is the sequence consisting of the code unit 0x0075 (LATIN SMALL LETTER U) followed by the code units of the TRV of *HexDigit*.

The TRV of *NotEscapeSequence* :: **u** *HexDigit* *HexDigit* [lookahead $\notin$ *HexDigit*] is the sequence consisting of the code unit 0x0075 (LATIN SMALL LETTER U) followed by the code units of the TRV of the first *HexDigit* followed by the code units of the TRV of the second *HexDigit*.

The TRV of *NotEscapeSequence* :: **u** *HexDigit* *HexDigit* *HexDigit* [lookahead $\notin$ *HexDigit*] is the sequence consisting of the code unit 0x0075 (LATIN SMALL LETTER U) followed by the code units of the TRV of the first *HexDigit* followed by the code units of the TRV of the second *HexDigit* followed by the code units of the TRV of the third *HexDigit*.

The TRV of *NotEscapeSequence* :: **u** { [lookahead $\notin$ *HexDigit*] is the sequence consisting of the code unit 0x0075 (LATIN SMALL LETTER U) followed by the code unit 0x007B (LEFT CURLY BRACKET).

The TRV of *NotEscapeSequence* :: **u** { *NotCodePoint* [lookahead $\notin$ *HexDigit*] is the sequence consisting of the code unit 0x0075 (LATIN SMALL LETTER U) followed by the code unit 0x007B (LEFT CURLY BRACKET) followed by the code units of the TRV of *NotCodePoint*.

The TRV of *NotEscapeSequence* :: **u** { *CodePoint* [lookahead $\notin$ *HexDigit*] [lookahead $\neq$]} is the sequence consisting of the code unit 0x0075 (LATIN SMALL LETTER U) followed by the code unit 0x007B (LEFT CURLY BRACKET) followed by the code units of the TRV of *CodePoint*.

The TRV of *DecimalDigit* :: **one of 0 1 2 3 4 5 6 7 8 9** is the SV of the *SourceCharacter* that is that single code point.

The TRV of *CharacterEscapeSequence* :: *SingleEscapeCharacter* is the TRV of the *SingleEscapeCharacter*.

The TRV of *CharacterEscapeSequence* :: *NonEscapeCharacter* is the SV of the *NonEscapeCharacter*.

The TRV of *SingleEscapeCharacter* :: **one of ' " \ b f n r t v** is the SV of the *SourceCharacter* that is that single code point.

The TRV of *HexEscapeSequence* :: **x** *HexDigit* *HexDigit* is the sequence consisting of the code unit 0x0078 (LATIN SMALL LETTER X) followed by TRV of the first *HexDigit* followed by the TRV of the second *HexDigit*.

The TRV of *UnicodeEscapeSequence* :: **u** *Hex4Digits* is the sequence consisting of the code unit 0x0075 (LATIN SMALL LETTER U) followed by TRV of *Hex4Digits*.

The TRV of *UnicodeEscapeSequence* :: **u**{ *CodePoint* } is the sequence consisting of the code unit 0x0075 (LATIN SMALL LETTER U) followed by the code unit 0x007B (LEFT CURLY BRACKET) followed by TRV of *CodePoint* followed by the code unit 0x007D (RIGHT CURLY BRACKET).

The TRV of *Hex4Digits* :: *HexDigit* *HexDigit* *HexDigit* *HexDigit* is the sequence consisting of the TRV of the first *HexDigit* followed by the TRV of the second *HexDigit* followed by the TRV of the third *HexDigit* followed by the TRV of the fourth *HexDigit*.

The TRV of *HexDigits* :: *HexDigit* is the TRV of *HexDigit*.

The TRV of *HexDigits* :: *HexDigits* *HexDigit* is the sequence consisting of TRV of *HexDigits* followed by TRV of *HexDigit*.

The TRV of a *HexDigit* is the SV of the *SourceCharacter* that is that *HexDigit*.

The TRV of *LineContinuation* :: **** *LineTerminatorSequence* is the sequence consisting of the code unit 0x005C (REVERSE SOLIDUS) followed by the code units of TRV of *LineTerminatorSequence*.

The TRV of *LineTerminatorSequence* :: **<LF>** is the code unit 0x000A (LINE FEED).

The TRV of *LineTerminatorSequence* :: **<CR>** is the code unit 0x000A (LINE FEED).

The TRV of *LineTerminatorSequence* :: **<LS>** is the code unit 0x2028 (LINE SEPARATOR).

The TRV of *LineTerminatorSequence* :: **<PS>** is the code unit 0x2029 (PARAGRAPH SEPARATOR).

The TRV of *LineTerminatorSequence* :: <CR><LF> is the sequence consisting of the code unit 0x000A (LINE FEED).

NOTE

TV excludes the code units of *LineContinuation* while TRV includes them. <CR><LF> and <CR> *LineTerminatorSequences* are normalized to <LF> for both TV and TRV. An explicit *EscapeSequence* is needed to include a <CR> or <CR><LF> sequence.

11.9 Automatic Semicolon Insertion

Most ECMAScript statements and declarations must be terminated with a semicolon. Such semicolons may always appear explicitly in the source text. For convenience, however, such semicolons may be omitted from the source text in certain situations. These situations are described by saying that semicolons are automatically inserted into the source code token stream in those situations.

11.9.1 Rules of Automatic Semicolon Insertion

In the following rules, “token” means the actual recognized lexical token determined using the current lexical [goal symbol](#) as described in clause 11.

There are three basic rules of semicolon insertion:

1. When, as the source text is parsed from left to right, a token (called the *offending token*) is encountered that is not allowed by any production of the grammar, then a semicolon is automatically inserted before the offending token if one or more of the following conditions is true:

The offending token is separated from the previous token by at least one *LineTerminator*.

The offending token is `}`.

The previous token is `)` and the inserted semicolon would then be parsed as the terminating semicolon of a do-while statement (13.7.2).

2. When, as the source text is parsed from left to right, the end of the input stream of tokens is encountered and the parser is unable to parse the input token stream as a single instance of the goal nonterminal, then a semicolon is automatically inserted at the end of the input stream.
3. When, as the source text is parsed from left to right, a token is encountered that is allowed by some production of the grammar, but the production is a *restricted production* and the token would be the first token for a terminal or nonterminal immediately following the annotation “[no *LineTerminator* here]” within the restricted production (and therefore such a token is called a restricted token), and the restricted token is separated from the previous token by at least one *LineTerminator*, then a semicolon is automatically inserted before the restricted token.

However, there is an additional overriding condition on the preceding rules: a semicolon is never inserted automatically if the semicolon would then be parsed as an empty statement or if that semicolon would become one of the two semicolons in the header of a **for** statement (see 13.7.4).

NOTE

The following are the only restricted productions in the grammar:

```
UpdateExpression[Yield, Await] :  
    LeftHandSideExpression[?Yield, ?Await] [no LineTerminator here] ++
```

```

    LeftHandSideExpression[?Yield, ?Await] [no LineTerminator here] --

ContinueStatement[Yield, Await] :
    continue ;
    continue [no LineTerminator here] LabelIdentifier[?Yield, ?Await] ;

BreakStatement[Yield, Await] :
    break ;
    break [no LineTerminator here] LabelIdentifier[?Yield, ?Await] ;

ReturnStatement[Yield, Await] :
    return ;
    return [no LineTerminator here] Expression[+In, ?Yield, ?Await] ;

ThrowStatement[Yield, Await] :
    throw [no LineTerminator here] Expression[+In, ?Yield, ?Await] ;

ArrowFunction[In, Yield, Await] :
    ArrowParameters[?Yield, ?Await] [no LineTerminator here] => ConciseBody[?In]

YieldExpression[In, Await] :
    yield [no LineTerminator here] AssignmentExpression[?In, +Yield, ?Await]
    yield [no LineTerminator here] * AssignmentExpression[?In, +Yield, ?Await]

```

The practical effect of these restricted productions is as follows:

When a **++** or **--** token is encountered where the parser would treat it as a postfix operator, and at least one *LineTerminator* occurred between the preceding token and the **++** or **--** token, then a semicolon is automatically inserted before the **++** or **--** token.

When a **continue**, **break**, **return**, **throw**, or **yield** token is encountered and a *LineTerminator* is encountered before the next token, a semicolon is automatically inserted after the **continue**, **break**, **return**, **throw**, or **yield** token.

The resulting practical advice to ECMAScript programmers is:

A postfix **++** or **--** operator should appear on the same line as its operand.

An *Expression* in a **return** or **throw** statement or an *AssignmentExpression* in a **yield** expression should start on the same line as the **return**, **throw**, or **yield** token.

A *LabelIdentifier* in a **break** or **continue** statement should be on the same line as the **break** or **continue** token.

11.9.2 Examples of Automatic Semicolon Insertion

The source

```
{ 1 2 } 3
```

is not a valid sentence in the ECMAScript grammar, even with the automatic semicolon insertion rules. In contrast, the

source

```
{ 1  
2 } 3
```

is also not a valid ECMAScript sentence, but is transformed by automatic semicolon insertion into the following:

```
{ 1  
;2 ;} 3;
```

which is a valid ECMAScript sentence.

The source

```
for (a; b  
)
```

is not a valid ECMAScript sentence and is not altered by automatic semicolon insertion because the semicolon is needed for the header of a **for** statement. Automatic semicolon insertion never inserts one of the two semicolons in the header of a **for** statement.

The source

```
return  
a + b
```

is transformed by automatic semicolon insertion into the following:

```
return;  
a + b;
```

NOTE 1

The expression **a + b** is not treated as a value to be returned by the **return** statement, because a *LineTerminator* separates it from the token **return**.

The source

```
a = b  
++c
```

is transformed by automatic semicolon insertion into the following:

```
a = b;  
++c;
```

NOTE 2

The token **++** is not treated as a postfix operator applying to the variable **b**, because a *LineTerminator* occurs between **b** and **++**.

The source

```
if (a > b)
```



```
else c = d
```

is not a valid ECMAScript sentence and is not altered by automatic semicolon insertion before the **else** token, even though no production of the grammar applies at that point, because an automatically inserted semicolon would then be parsed as an empty statement.

The source

```
a = b + c
(d + e).print()
```

is *not* transformed by automatic semicolon insertion, because the parenthesized expression that begins the second line can be interpreted as an argument list for a function call:

```
a = b + c(d + e).print()
```

In the circumstance that an assignment statement must begin with a left parenthesis, it is a good idea for the programmer to provide an explicit semicolon at the end of the preceding statement rather than to rely on automatic semicolon insertion.

12 ECMAScript Language: Expressions

12.1 Identifiers

Syntax

*IdentifierReference*_[Yield, Await] :

Identifier

_[~Yield] **yield**

_[~Await] **await**

*BindingIdentifier*_[Yield, Await] :

Identifier

yield

await

*LabelIdentifier*_[Yield, Await] :

Identifier

_[~Yield] **yield**

_[~Await] **await**

Identifier :

IdentifierName but not *ReservedWord*

NOTE

yield and **await** are permitted as *BindingIdentifier* in the grammar, and prohibited with [static semantics](#) below, to prohibit automatic semicolon insertion in cases such as

```
let
```

`await 0;`

12.1.1 Static Semantics: Early Errors

BindingIdentifier : *Identifier*

It is a Syntax Error if the code matched by this production is contained in [strict mode code](#) and the StringValue of *Identifier* is **"arguments"** or **"eval"**.

IdentifierReference : **yield**

BindingIdentifier : **yield**

LabelIdentifier : **yield**

It is a Syntax Error if the code matched by this production is contained in [strict mode code](#).

IdentifierReference : **await**

BindingIdentifier : **await**

LabelIdentifier : **await**

It is a Syntax Error if the [goal symbol](#) of the syntactic grammar is *Module*.

BindingIdentifier : **yield**

It is a Syntax Error if this production has a [Yield] parameter.

BindingIdentifier : **await**

It is a Syntax Error if this production has an [Await] parameter.

IdentifierReference : *Identifier*

BindingIdentifier : *Identifier*

LabelIdentifier : *Identifier*

It is a Syntax Error if this production has a [Yield] parameter and StringValue of *Identifier* is **"yield"**.

It is a Syntax Error if this production has an [Await] parameter and StringValue of *Identifier* is **"await"**.

Identifier : *IdentifierName* but not *ReservedWord*

It is a Syntax Error if this phrase is contained in [strict mode code](#) and the StringValue of *IdentifierName* is: **"implements"**, **"interface"**, **"let"**, **"package"**, **"private"**, **"protected"**, **"public"**, **"static"**, or **"yield"**.

It is a Syntax Error if the [goal symbol](#) of the syntactic grammar is *Module* and the StringValue of *IdentifierName* is **"await"**.

It is a Syntax Error if StringValue of *IdentifierName* is the same String value as the StringValue of any *ReservedWord* except for **yield** or **await**.

NOTE

StringValue of *IdentifierName* normalizes any Unicode escape sequences in *IdentifierName* hence such escapes cannot be used to write an *Identifier* whose code point sequence is the same as a *ReservedWord*.

12.1.2 Static Semantics: BoundNames

BindingIdentifier : *Identifier*

1. Return a new [List](#) containing the StringValue of *Identifier*.

BindingIdentifier : **yield**

1. Return a new [List](#) containing "yield".

BindingIdentifier : **await**

1. Return a new [List](#) containing "await".

12.1.3 Static Semantics: AssignmentTargetType

IdentifierReference : *Identifier*

1. If this *IdentifierReference* is contained in [strict mode code](#) and StringValue of *Identifier* is "eval" or "arguments", return strict.
2. Return simple.

IdentifierReference : **yield**

1. Return simple.

IdentifierReference : **await**

1. Return simple.

12.1.4 Static Semantics: StringValue

IdentifierReference : **yield**

BindingIdentifier : **yield**

LabelIdentifier : **yield**

1. Return "yield".

IdentifierReference : **await**

BindingIdentifier : **await**

LabelIdentifier : **await**

1. Return "await".

Identifier : *IdentifierName* but not *ReservedWord*

1. Return the StringValue of *IdentifierName*.

12.1.5 Runtime Semantics: BindingInitialization

With parameters *value* and *environment*.

NOTE

undefined is passed for *environment* to indicate that a [PutValue](#) operation should be used to assign the initialization value. This is the case for **var** statements and formal parameter lists of some non-strict functions (See [9.2.15](#)). In those

cases a lexical binding is hoisted and preinitialized prior to evaluation of its initializer.

BindingIdentifier : *Identifier*

1. Let *name* be StringValue of *Identifier*.
2. Return ? [InitializeBoundName](#)(*name*, *value*, *environment*).

BindingIdentifier : **yield**

1. Return ? [InitializeBoundName](#)("yield", *value*, *environment*).

BindingIdentifier : **await**

1. Return ? [InitializeBoundName](#)("await", *value*, *environment*).

12.1.5.1 Runtime Semantics: InitializeBoundName (*name*, *value*, *environment*)

1. [Assert](#): [Type](#)(*name*) is String.
2. If *environment* is not **undefined**, then
 - a. Let *env* be the [EnvironmentRecord](#) component of *environment*.
 - b. Perform *env*.[InitializeBinding](#)(*name*, *value*).
 - c. Return [NormalCompletion](#)(**undefined**).
3. Else,
 - a. Let *lhs* be [ResolveBinding](#)(*name*).
 - b. Return ? [PutValue](#)(*lhs*, *value*).

12.1.6 Runtime Semantics: Evaluation

IdentifierReference : *Identifier*

1. Return ? [ResolveBinding](#)(StringValue of *Identifier*).

IdentifierReference : **yield**

1. Return ? [ResolveBinding](#)("yield").

IdentifierReference : **await**

1. Return ? [ResolveBinding](#)("await").

NOTE 1

The result of evaluating an *IdentifierReference* is always a value of type [Reference](#).

NOTE 2

In [non-strict code](#), the keyword **yield** may be used as an identifier. Evaluating the *IdentifierReference* resolves the binding of **yield** as if it was an *Identifier*. Early Error restriction ensures that such an evaluation only can occur for [non-strict code](#).

12.2 Primary Expression

Syntax

```

PrimaryExpression[Yield, Await] :
    this
    IdentifierReference[?Yield, ?Await]
    Literal
    ArrayLiteral[?Yield, ?Await]
    ObjectLiteral[?Yield, ?Await]
    FunctionExpression
    ClassExpression[?Yield, ?Await]
    GeneratorExpression
    AsyncFunctionExpression
    AsyncGeneratorExpression
    RegularExpressionLiteral
    TemplateLiteral[?Yield, ?Await, ~Tagged]
    CoverParenthesizedExpressionAndArrowParameterList[?Yield, ?Await]

```

```

CoverParenthesizedExpressionAndArrowParameterList[Yield, Await] :
    ( Expression[+In, ?Yield, ?Await] )
    ( Expression[+In, ?Yield, ?Await] , )
    ( )
    ( ... BindingIdentifier[?Yield, ?Await] )
    ( ... BindingPattern[?Yield, ?Await] )
    ( Expression[+In, ?Yield, ?Await] , ... BindingIdentifier[?Yield, ?Await] )
    ( Expression[+In, ?Yield, ?Await] , ... BindingPattern[?Yield, ?Await] )

```

Supplemental Syntax

When processing an instance of the production

PrimaryExpression : *CoverParenthesizedExpressionAndArrowParameterList*

the interpretation of *CoverParenthesizedExpressionAndArrowParameterList* is refined using the following grammar:

```

ParenthesizedExpression[Yield, Await] :
    ( Expression[+In, ?Yield, ?Await] )

```

12.2.1 Semantics

12.2.1.1 Static Semantics: CoveredParenthesizedExpression

CoverParenthesizedExpressionAndArrowParameterList : (*Expression*)

1. Return the *ParenthesizedExpression* that is **covered** by *CoverParenthesizedExpressionAndArrowParameterList*.

12.2.1.2 Static Semantics: HasName

PrimaryExpression : *CoverParenthesizedExpressionAndArrowParameterList*

1. Let *expr* be CoveredParenthesizedExpression of *CoverParenthesizedExpressionAndArrowParameterList*.
2. If IsFunctionDefinition of *expr* is **false**, return **false**.
3. Return HasName of *expr*.

12.2.1.3 Static Semantics: IsFunctionDefinition

PrimaryExpression :

this

IdentifierReference

Literal

ArrayLiteral

ObjectLiteral

RegularExpressionLiteral

TemplateLiteral

1. Return **false**.

PrimaryExpression : *CoverParenthesizedExpressionAndArrowParameterList*

1. Let *expr* be CoveredParenthesizedExpression of *CoverParenthesizedExpressionAndArrowParameterList*.
2. Return IsFunctionDefinition of *expr*.

12.2.1.4 Static Semantics: IsIdentifierRef

PrimaryExpression : *IdentifierReference*

1. Return **true**.

PrimaryExpression :

this

Literal

ArrayLiteral

ObjectLiteral

FunctionExpression

ClassExpression

GeneratorExpression

AsyncFunctionExpression

AsyncGeneratorExpression

RegularExpressionLiteral

TemplateLiteral

CoverParenthesizedExpressionAndArrowParameterList

1. Return **false**.

12.2.1.5 Static Semantics: AssignmentTargetType

PrimaryExpression :

this

Literal

ArrayLiteral

ObjectLiteral

FunctionExpression

ClassExpression

GeneratorExpression

AsyncFunctionExpression

AsyncGeneratorExpression

RegularExpressionLiteral

TemplateLiteral

1. Return **invalid**.

PrimaryExpression : *CoverParenthesizedExpressionAndArrowParameterList*

1. Let *expr* be CoveredParenthesizedExpression of *CoverParenthesizedExpressionAndArrowParameterList*.
2. Return AssignmentTargetType of *expr*.

12.2.2 The **this** Keyword

12.2.2.1 Runtime Semantics: Evaluation

PrimaryExpression : **this**

1. Return ? [ResolveThisBinding\(\)](#).

12.2.3 Identifier Reference

See [12.1](#) for *IdentifierReference*.

12.2.4 Literals

Syntax

Literal :

NullLiteral

BooleanLiteral

NumericLiteral

StringLiteral

12.2.4.1 Runtime Semantics: Evaluation

Literal : *NullLiteral*

1. Return **null**.

Literal : *BooleanLiteral*

1. If *BooleanLiteral* is the token **false**, return **false**.
2. If *BooleanLiteral* is the token **true**, return **true**.

Literal : *NumericLiteral*

1. Return the number whose value is MV of *NumericLiteral* as defined in [11.8.3](#).

Literal : *StringLiteral*

1. Return the StringValue of *StringLiteral* as defined in [11.8.4.1](#).

12.2.5 Array Initializer

NOTE

An *ArrayLiteral* is an expression describing the initialization of an Array object, using a list, of zero or more expressions each of which represents an array element, enclosed in square brackets. The elements need not be literals; they are evaluated each time the array initializer is evaluated.

Array elements may be elided at the beginning, middle or end of the element list. Whenever a comma in the element list is not preceded by an *AssignmentExpression* (i.e., a comma at the beginning or after another comma), the missing array element contributes to the length of the Array and increases the index of subsequent elements. Elided array elements are not defined. If an element is elided at the end of an array, that element does not contribute to the length of the Array.

Syntax

```
ArrayLiteral[Yield, Await] :  
    [ Elisionopt ]  
    [ ElementList[?Yield, ?Await] ]  
    [ ElementList[?Yield, ?Await] , Elisionopt ]  
  
ElementList[Yield, Await] :  
    Elisionopt AssignmentExpression[+In, ?Yield, ?Await]  
    Elisionopt SpreadElement[?Yield, ?Await]  
    ElementList[?Yield, ?Await] , Elisionopt AssignmentExpression[+In, ?Yield, ?Await]  
    ElementList[?Yield, ?Await] , Elisionopt SpreadElement[?Yield, ?Await]  
  
Elision :  
    ,  
    Elision ,  
  
SpreadElement[Yield, Await] :  
    ... AssignmentExpression[+In, ?Yield, ?Await]
```

12.2.5.1 Static Semantics: ElisionWidth

Elision : ,

1. Return the numeric value 1.

Elision : *Elision* ,

1. Let *preceding* be the ElisionWidth of *Elision*.
2. Return *preceding* + 1.

12.2.5.2 Runtime Semantics: ArrayAccumulation

With parameters *array* and *nextIndex*.

ElementList : *Elision* *AssignmentExpression*

1. Let *padding* be the ElisionWidth of *Elision*; if *Elision* is not present, use the numeric value zero.
2. Let *initResult* be the result of evaluating *AssignmentExpression*.

3. Let *initValue* be ? *GetValue*(*initResult*).
4. Let *created* be *CreateDataProperty*(*array*, *ToString*(*ToUint32*(*nextIndex* + *padding*)), *initValue*).
5. **Assert:** *created* is **true**.
6. Return *nextIndex* + *padding* + 1.

ElementList : *Elision SpreadElement*

1. Let *padding* be the *ElisionWidth* of *Elision*; if *Elision* is not present, use the numeric value zero.
2. Return the result of performing *ArrayAccumulation* for *SpreadElement* with arguments *array* and *nextIndex* + *padding*.

ElementList : *ElementList* , *Elision AssignmentExpression*

1. Let *postIndex* be the result of performing *ArrayAccumulation* for *ElementList* with arguments *array* and *nextIndex*.
2. *ReturnIfAbrupt*(*postIndex*).
3. Let *padding* be the *ElisionWidth* of *Elision*; if *Elision* is not present, use the numeric value zero.
4. Let *initResult* be the result of evaluating *AssignmentExpression*.
5. Let *initValue* be ? *GetValue*(*initResult*).
6. Let *created* be *CreateDataProperty*(*array*, *ToString*(*ToUint32*(*postIndex* + *padding*)), *initValue*).
7. **Assert:** *created* is **true**.
8. Return *postIndex* + *padding* + 1.

ElementList : *ElementList* , *Elision SpreadElement*

1. Let *postIndex* be the result of performing *ArrayAccumulation* for *ElementList* with arguments *array* and *nextIndex*.
2. *ReturnIfAbrupt*(*postIndex*).
3. Let *padding* be the *ElisionWidth* of *Elision*; if *Elision* is not present, use the numeric value zero.
4. Return the result of performing *ArrayAccumulation* for *SpreadElement* with arguments *array* and *postIndex* + *padding*.

SpreadElement : . . . *AssignmentExpression*

1. Let *spreadRef* be the result of evaluating *AssignmentExpression*.
2. Let *spreadObj* be ? *GetValue*(*spreadRef*).
3. Let *iteratorRecord* be ? *GetIterator*(*spreadObj*).
4. Repeat,
 - a. Let *next* be ? *IteratorStep*(*iteratorRecord*).
 - b. If *next* is **false**, return *nextIndex*.
 - c. Let *nextValue* be ? *IteratorValue*(*next*).
 - d. Let *status* be *CreateDataProperty*(*array*, *ToString*(*ToUint32*(*nextIndex*)), *nextValue*).
 - e. **Assert:** *status* is **true**.
 - f. Increase *nextIndex* by 1.

NOTE

CreateDataProperty is used to ensure that own properties are defined for the array even if the standard built-in Array prototype object has been modified in a manner that would preclude the creation of new own properties using *[[Set]]*.

12.2.5.3 Runtime Semantics: Evaluation

ArrayLiteral : [*Elision*]

1. Let *array* be ! *ArrayCreate*(0).
2. Let *pad* be the *ElisionWidth* of *Elision*; if *Elision* is not present, use the numeric value zero.
3. Perform *Set*(*array*, "length", *ToUint32*(*pad*), false).
4. NOTE: The above *Set* cannot fail because of the nature of the object returned by *ArrayCreate*.
5. Return *array*.

ArrayLiteral : [*ElementList*]

1. Let *array* be ! *ArrayCreate*(0).
2. Let *len* be the result of performing *ArrayAccumulation* for *ElementList* with arguments *array* and 0.
3. *ReturnIfAbrupt*(*len*).
4. Perform *Set*(*array*, "length", *ToUint32*(*len*), false).
5. NOTE: The above *Set* cannot fail because of the nature of the object returned by *ArrayCreate*.
6. Return *array*.

ArrayLiteral : [*ElementList* , *Elision*]

1. Let *array* be ! *ArrayCreate*(0).
2. Let *len* be the result of performing *ArrayAccumulation* for *ElementList* with arguments *array* and 0.
3. *ReturnIfAbrupt*(*len*).
4. Let *padding* be the *ElisionWidth* of *Elision*; if *Elision* is not present, use the numeric value zero.
5. Perform *Set*(*array*, "length", *ToUint32*(*padding* + *len*), false).
6. NOTE: The above *Set* cannot fail because of the nature of the object returned by *ArrayCreate*.
7. Return *array*.

12.2.6 Object Initializer

NOTE 1

An object initializer is an expression describing the initialization of an Object, written in a form resembling a literal. It is a list of zero or more pairs of property keys and associated values, enclosed in curly brackets. The values need not be literals; they are evaluated each time the object initializer is evaluated.

Syntax

```
ObjectLiteral[Yield, Await] :
    { }
    { PropertyDefinitionList[?Yield, ?Await] }
    { PropertyDefinitionList[?Yield, ?Await] , }
```

```
PropertyDefinitionList[Yield, Await] :
    PropertyDefinition[?Yield, ?Await]
    PropertyDefinitionList[?Yield, ?Await] , PropertyDefinition[?Yield, ?Await]
```

```
PropertyDefinition[Yield, Await] :
    IdentifierReference[?Yield, ?Await]
    CoverInitializedName[?Yield, ?Await]
    PropertyName[?Yield, ?Await] : AssignmentExpression[+In, ?Yield, ?Await]
    MethodDefinition[?Yield, ?Await]
    ... AssignmentExpression[+In, ?Yield, ?Await]
```

```

PropertyName[Yield, Await] :
    LiteralPropertyName
    ComputedPropertyName[?Yield, ?Await]

LiteralPropertyName :
    IdentifierName
    StringLiteral
    NumericLiteral

ComputedPropertyName[Yield, Await] :
    [ AssignmentExpression[+In, ?Yield, ?Await] ]

CoverInitializedName[Yield, Await] :
    IdentifierReference[?Yield, ?Await] Initializer[+In, ?Yield, ?Await]

Initializer[In, Yield, Await] :
    = AssignmentExpression[?In, ?Yield, ?Await]

```

NOTE 2

MethodDefinition is defined in 14.3.

NOTE 3

In certain contexts, *ObjectLiteral* is used as a cover grammar for a more restricted secondary grammar. The *CoverInitializedName* production is necessary to fully cover these secondary grammars. However, use of this production results in an early Syntax Error in normal contexts where an actual *ObjectLiteral* is expected.

12.2.6.1 Static Semantics: Early Errors

PropertyDefinition : *MethodDefinition*

It is a Syntax Error if *HasDirectSuper* of *MethodDefinition* is **true**.

In addition to describing an actual object initializer the *ObjectLiteral* productions are also used as a cover grammar for *ObjectAssignmentPattern* and may be recognized as part of a *CoverParenthesizedExpressionAndArrowParameterList*. When *ObjectLiteral* appears in a context where *ObjectAssignmentPattern* is required the following Early Error rules are **not** applied. In addition, they are not applied when initially parsing a *CoverParenthesizedExpressionAndArrowParameterList* or *CoverCallExpressionAndAsyncArrowHead*.

PropertyDefinition : *CoverInitializedName*

Always throw a Syntax Error if code matches this production.

NOTE

This production exists so that *ObjectLiteral* can serve as a cover grammar for *ObjectAssignmentPattern*. It cannot occur in an actual object initializer.

12.2.6.2 Static Semantics: ComputedPropertyContains

With parameter *symbol*.

PropertyName : *LiteralPropertyName*

1. Return **false**.

PropertyName : *ComputedPropertyName*

1. Return the result of *ComputedPropertyName* Contains *symbol*.

12.2.6.3 Static Semantics: Contains

With parameter *symbol*.

PropertyDefinition : *MethodDefinition*

1. If *symbol* is *MethodDefinition*, return **true**.
2. Return the result of *ComputedPropertyContains* for *MethodDefinition* with argument *symbol*.

NOTE

Static semantic rules that depend upon substructure generally do not look into function definitions.

LiteralPropertyName : *IdentifierName*

1. If *symbol* is a *ReservedWord*, return **false**.
2. If *symbol* is an *Identifier* and *StringValue* of *symbol* is the same value as the *StringValue* of *IdentifierName*, return **true**.
3. Return **false**.

12.2.6.4 Static Semantics: IsComputedPropertyKey

PropertyName : *LiteralPropertyName*

1. Return **false**.

PropertyName : *ComputedPropertyName*

1. Return **true**.

12.2.6.5 Static Semantics: PropName

PropertyDefinition : *IdentifierReference*

1. Return *StringValue* of *IdentifierReference*.

PropertyDefinition : . . . *AssignmentExpression*

1. Return **empty**.

PropertyDefinition : *PropertyName* : *AssignmentExpression*

1. Return *PropName* of *PropertyName*.

LiteralPropertyName : *IdentifierName*

1. Return *StringValue* of *IdentifierName*.

LiteralPropertyName : *StringLiteral*

1. Return the *String* value whose code units are the SV of the *StringLiteral*.

LiteralPropertyName : *NumericLiteral*

1. Let *nbr* be the result of forming the value of the *NumericLiteral*.
2. Return ! *ToString*(*nbr*).

ComputedPropertyName : [*AssignmentExpression*]

1. Return empty.

12.2.6.6 Static Semantics: *PropertyNameList*

PropertyDefinitionList : *PropertyDefinition*

1. If *PropName* of *PropertyDefinition* is empty, return a new empty *List*.
2. Return a new *List* containing *PropName* of *PropertyDefinition*.

PropertyDefinitionList : *PropertyDefinitionList* , *PropertyDefinition*

1. Let *list* be *PropertyNameList* of *PropertyDefinitionList*.
2. If *PropName* of *PropertyDefinition* is empty, return *list*.
3. Append *PropName* of *PropertyDefinition* to the end of *list*.
4. Return *list*.

12.2.6.7 Runtime Semantics: Evaluation

ObjectLiteral : { }

1. Return *ObjectCreate*(%*ObjectPrototype*%).

ObjectLiteral :

{ *PropertyDefinitionList* }

{ *PropertyDefinitionList* , }

1. Let *obj* be *ObjectCreate*(%*ObjectPrototype*%).
2. Perform ? *PropertyDefinitionEvaluation* of *PropertyDefinitionList* with arguments *obj* and **true**.
3. Return *obj*.

LiteralPropertyName : *IdentifierName*

1. Return *StringValue* of *IdentifierName*.

LiteralPropertyName : *StringLiteral*

1. Return the *String* value whose code units are the SV of the *StringLiteral*.

LiteralPropertyName : *NumericLiteral*

1. Let *nbr* be the result of forming the value of the *NumericLiteral*.
2. Return ! *ToString*(*nbr*).

ComputedPropertyName : [*AssignmentExpression*]

1. Let *exprValue* be the result of evaluating *AssignmentExpression*.
2. Let *propName* be ? *GetValue*(*exprValue*).
3. Return ? *ToPropertyKey*(*propName*).

12.2.6.8 Runtime Semantics: PropertyDefinitionEvaluation

With parameters *object* and *enumerable*.

PropertyDefinitionList : *PropertyDefinitionList* , *PropertyDefinition*

1. Perform ? *PropertyDefinitionEvaluation* of *PropertyDefinitionList* with arguments *object* and *enumerable*.
2. Return the result of performing *PropertyDefinitionEvaluation* of *PropertyDefinition* with arguments *object* and *enumerable*.

PropertyDefinition : . . . *AssignmentExpression*

1. Let *exprValue* be the result of evaluating *AssignmentExpression*.
2. Let *fromValue* be ? *GetValue*(*exprValue*).
3. Let *excludedNames* be a new empty *List*.
4. Return ? *CopyDataProperties*(*object*, *fromValue*, *excludedNames*).

PropertyDefinition : *IdentifierReference*

1. Let *propName* be *StringValue* of *IdentifierReference*.
2. Let *exprValue* be the result of evaluating *IdentifierReference*.
3. Let *propValue* be ? *GetValue*(*exprValue*).
4. **Assert**: *enumerable* is **true**.
5. **Assert**: *object* is an ordinary, extensible object with no non-configurable properties.
6. Return ! *CreateDataPropertyOrThrow*(*object*, *propName*, *propValue*).

PropertyDefinition : *PropertyName* : *AssignmentExpression*

1. Let *propKey* be the result of evaluating *PropertyName*.
2. *ReturnIfAbrupt*(*propKey*).
3. If *IsAnonymousFunctionDefinition*(*AssignmentExpression*) is **true**, then
 - a. Let *propValue* be the result of performing *NamedEvaluation* for *AssignmentExpression* with argument *propKey*.
4. Else,
 - a. Let *exprValueRef* be the result of evaluating *AssignmentExpression*.
 - b. Let *propValue* be ? *GetValue*(*exprValueRef*).
5. **Assert**: *enumerable* is **true**.
6. **Assert**: *object* is an ordinary, extensible object with no non-configurable properties.
7. Return ! *CreateDataPropertyOrThrow*(*object*, *propKey*, *propValue*).

NOTE

An alternative semantics for this production is given in B.3.1.

12.2.7 Function Defining Expressions

See 14.1 for *PrimaryExpression* : *FunctionExpression* .

See 14.4 for *PrimaryExpression* : *GeneratorExpression* .

See 14.6 for *PrimaryExpression* : *ClassExpression* .

See 14.7 for *PrimaryExpression* : *AsyncFunctionExpression* .

See 14.5 for *PrimaryExpression* : *AsyncGeneratorExpression* .

12.2.8 Regular Expression Literals

Syntax

See 11.8.5.

12.2.8.1 Static Semantics: Early Errors

PrimaryExpression : *RegularExpressionLiteral*

It is a Syntax Error if *BodyText* of *RegularExpressionLiteral* cannot be recognized using the [goal symbol Pattern](#) of the ECMAScript RegExp grammar specified in 21.2.1.

It is a Syntax Error if *FlagText* of *RegularExpressionLiteral* contains any code points other than "g", "i", "m", "s", "u", or "y", or if it contains the same code point more than once.

12.2.8.2 Runtime Semantics: Evaluation

PrimaryExpression : *RegularExpressionLiteral*

1. Let *pattern* be the String value consisting of the [UTF16Encoding](#) of each code point of *BodyText* of *RegularExpressionLiteral*.
2. Let *flags* be the String value consisting of the [UTF16Encoding](#) of each code point of *FlagText* of *RegularExpressionLiteral*.
3. Return [RegExpCreate](#)(*pattern*, *flags*).

12.2.9 Template Literals

Syntax

*TemplateLiteral*_[Yield, Await, Tagged] :
 NoSubstitutionTemplate
 *SubstitutionTemplate*_[?Yield, ?Await, ?Tagged]

*SubstitutionTemplate*_[Yield, Await, Tagged] :
 TemplateHead *Expression*_[+In, ?Yield, ?Await] *TemplateSpans*_[?Yield, ?Await, ?Tagged]

*TemplateSpans*_[Yield, Await, Tagged] :
 TemplateTail
 *TemplateMiddleList*_[?Yield, ?Await, ?Tagged] *TemplateTail*

*TemplateMiddleList*_[Yield, Await, Tagged] :
 TemplateMiddle *Expression*_[+In, ?Yield, ?Await]
 *TemplateMiddleList*_[?Yield, ?Await, ?Tagged] *TemplateMiddle* *Expression*_[+In, ?Yield, ?Await]

12.2.9.1 Static Semantics: Early Errors

TemplateLiteral : *NoSubstitutionTemplate*

It is a Syntax Error if the number of elements in the result of TemplateStrings of *TemplateLiteral* with argument **false** is greater than $2^{32} - 1$.

It is a Syntax Error if the [Tagged] parameter was not set and *NoSubstitutionTemplate* Contains *NotEscapeSequence*.

SubstitutionTemplate : *TemplateHead* *Expression* *TemplateSpans*

It is a Syntax Error if the [Tagged] parameter was not set and *TemplateHead* Contains *NotEscapeSequence*.

TemplateSpans : *TemplateTail*

It is a Syntax Error if the [Tagged] parameter was not set and *TemplateTail* Contains *NotEscapeSequence*.

*TemplateMiddleList*_[Yield, Await, Tagged] :

TemplateMiddle *Expression*_[+In, ?Yield, ?Await]

*TemplateMiddleList*_[?Yield, ?Await, ?Tagged] *TemplateMiddle* *Expression*_[+In, ?Yield, ?Await]

It is a Syntax Error if the [Tagged] parameter was not set and *TemplateMiddle* Contains *NotEscapeSequence*.

12.2.9.2 Static Semantics: TemplateStrings

With parameter *raw*.

TemplateLiteral : *NoSubstitutionTemplate*

1. If *raw* is **false**, then
 - a. Let *string* be the TV of *NoSubstitutionTemplate*.
2. Else,
 - a. Let *string* be the TRV of *NoSubstitutionTemplate*.
3. Return a **List** containing the single element, *string*.

SubstitutionTemplate : *TemplateHead* *Expression* *TemplateSpans*

1. If *raw* is **false**, then
 - a. Let *head* be the TV of *TemplateHead*.
2. Else,
 - a. Let *head* be the TRV of *TemplateHead*.
3. Let *tail* be TemplateStrings of *TemplateSpans* with argument *raw*.
4. Return a **List** containing *head* followed by the elements, in order, of *tail*.

TemplateSpans : *TemplateTail*

1. If *raw* is **false**, then
 - a. Let *tail* be the TV of *TemplateTail*.
2. Else,
 - a. Let *tail* be the TRV of *TemplateTail*.
3. Return a **List** containing the single element, *tail*.

TemplateSpans : *TemplateMiddleList* *TemplateTail*

1. Let *middle* be TemplateStrings of *TemplateMiddleList* with argument *raw*.
2. If *raw* is **false**, then
 - a. Let *tail* be the TV of *TemplateTail*.

3. Else,
 - a. Let *tail* be the TRV of *TemplateTail*.
4. Return a *List* containing the elements, in order, of *middle* followed by *tail*.

TemplateMiddleList : *TemplateMiddle Expression*

1. If *raw* is **false**, then
 - a. Let *string* be the TV of *TemplateMiddle*.
2. Else,
 - a. Let *string* be the TRV of *TemplateMiddle*.
3. Return a *List* containing the single element, *string*.

TemplateMiddleList : *TemplateMiddleList TemplateMiddle Expression*

1. Let *front* be *TemplateStrings* of *TemplateMiddleList* with argument *raw*.
2. If *raw* is **false**, then
 - a. Let *last* be the TV of *TemplateMiddle*.
3. Else,
 - a. Let *last* be the TRV of *TemplateMiddle*.
4. Append *last* as the last element of the *List front*.
5. Return *front*.

12.2.9.3 Runtime Semantics: ArgumentListEvaluation

TemplateLiteral : *NoSubstitutionTemplate*

1. Let *templateLiteral* be this *TemplateLiteral*.
2. Let *siteObj* be *GetTemplateObject(templateLiteral)*.
3. Return a *List* containing the one element which is *siteObj*.

SubstitutionTemplate : *TemplateHead Expression TemplateSpans*

1. Let *templateLiteral* be this *TemplateLiteral*.
2. Let *siteObj* be *GetTemplateObject(templateLiteral)*.
3. Let *firstSubRef* be the result of evaluating *Expression*.
4. Let *firstSub* be ? *GetValue(firstSubRef)*.
5. Let *restSub* be *SubstitutionEvaluation* of *TemplateSpans*.
6. *ReturnIfAbrupt(restSub)*.
7. *Assert*: *restSub* is a *List*.
8. Return a *List* whose first element is *siteObj*, whose second elements is *firstSub*, and whose subsequent elements are the elements of *restSub*, in order. *restSub* may contain no elements.

12.2.9.4 Runtime Semantics: GetTemplateObject (*templateLiteral*)

The abstract operation *GetTemplateObject* is called with a *Parse Node*, *templateLiteral*, as an argument. It performs the following steps:

1. Let *rawStrings* be *TemplateStrings* of *templateLiteral* with argument **true**.
2. Let *realm* be the current *Realm Record*.
3. Let *templateRegistry* be *realm*.[[*TemplateMap*]].
4. For each element *e* of *templateRegistry*, do
 - a. If *e*.[[*Site*]] is the same *Parse Node* as *templateLiteral*, then

- i. Return *e*.[[Array]].
5. Let *cookedStrings* be TemplateStrings of *templateLiteral* with argument **false**.
6. Let *count* be the number of elements in the List *cookedStrings*.
7. **Assert**: *count* ≤ 2³² - 1.
8. Let *template* be ! *ArrayCreate*(*count*).
9. Let *rawObj* be ! *ArrayCreate*(*count*).
10. Let *index* be 0.
11. Repeat, while *index* < *count*
 - a. Let *prop* be ! *ToString*(*index*).
 - b. Let *cookedValue* be the String value *cookedStrings*[*index*].
 - c. Call *template*.[[DefineOwnProperty]](*prop*, PropertyDescriptor { [[Value]]: *cookedValue*, [[Writable]]: **false**, [[Enumerable]]: **true**, [[Configurable]]: **false** }).
 - d. Let *rawValue* be the String value *rawStrings*[*index*].
 - e. Call *rawObj*.[[DefineOwnProperty]](*prop*, PropertyDescriptor { [[Value]]: *rawValue*, [[Writable]]: **false**, [[Enumerable]]: **true**, [[Configurable]]: **false** }).
 - f. Increase *index* by 1.
12. Perform *SetIntegrityLevel*(*rawObj*, "frozen").
13. Call *template*.[[DefineOwnProperty]]("raw", PropertyDescriptor { [[Value]]: *rawObj*, [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **false** }).
14. Perform *SetIntegrityLevel*(*template*, "frozen").
15. Append the Record { [[Site]]: *templateLiteral*, [[Array]]: *template* } to *templateRegistry*.
16. Return *template*.

NOTE 1

The creation of a template object cannot result in an abrupt completion.

NOTE 2

Each *TemplateLiteral* in the program code of a realm is associated with a unique template object that is used in the evaluation of tagged Templates (12.2.9.6). The template objects are frozen and the same template object is used each time a specific tagged Template is evaluated. Whether template objects are created lazily upon first evaluation of the *TemplateLiteral* or eagerly prior to first evaluation is an implementation choice that is not observable to ECMAScript code.

NOTE 3

Future editions of this specification may define additional non-enumerable properties of template objects.

12.2.9.5 Runtime Semantics: SubstitutionEvaluation

TemplateSpans : *TemplateTail*

1. Return a new empty List.

TemplateSpans : *TemplateMiddleList* *TemplateTail*

1. Return the result of SubstitutionEvaluation of *TemplateMiddleList*.

TemplateMiddleList : *TemplateMiddle* *Expression*

1. Let *subRef* be the result of evaluating *Expression*.
2. Let *sub* be ? *GetValue*(*subRef*).
3. Return a List containing only *sub*.

TemplateMiddleList : *TemplateMiddleList* *TemplateMiddle* *Expression*

1. Let *preceding* be the result of SubstitutionEvaluation of *TemplateMiddleList*.
2. **ReturnIfAbrupt**(*preceding*).
3. Let *nextRef* be the result of evaluating *Expression*.
4. Let *next* be ? **GetValue**(*nextRef*).
5. Append *next* as the last element of the *List preceding*.
6. Return *preceding*.

12.2.9.6 Runtime Semantics: Evaluation

TemplateLiteral : *NoSubstitutionTemplate*

1. Return the String value whose code units are the elements of the TV of *NoSubstitutionTemplate* as defined in 11.8.6.

SubstitutionTemplate : *TemplateHead* *Expression* *TemplateSpans*

1. Let *head* be the TV of *TemplateHead* as defined in 11.8.6.
2. Let *subRef* be the result of evaluating *Expression*.
3. Let *sub* be ? **GetValue**(*subRef*).
4. Let *middle* be ? **ToString**(*sub*).
5. Let *tail* be the result of evaluating *TemplateSpans*.
6. **ReturnIfAbrupt**(*tail*).
7. Return the *string-concatenation* of *head*, *middle*, and *tail*.

NOTE 1

The string conversion semantics applied to the *Expression* value are like **String.prototype.concat** rather than the **+** operator.

TemplateSpans : *TemplateTail*

1. Let *tail* be the TV of *TemplateTail* as defined in 11.8.6.
2. Return the String value consisting of the code units of *tail*.

TemplateSpans : *TemplateMiddleList* *TemplateTail*

1. Let *head* be the result of evaluating *TemplateMiddleList*.
2. **ReturnIfAbrupt**(*head*).
3. Let *tail* be the TV of *TemplateTail* as defined in 11.8.6.
4. Return the *string-concatenation* of *head* and *tail*.

TemplateMiddleList : *TemplateMiddle* *Expression*

1. Let *head* be the TV of *TemplateMiddle* as defined in 11.8.6.
2. Let *subRef* be the result of evaluating *Expression*.
3. Let *sub* be ? **GetValue**(*subRef*).
4. Let *middle* be ? **ToString**(*sub*).
5. Return the sequence of code units consisting of the code units of *head* followed by the elements of *middle*.

NOTE 2

The string conversion semantics applied to the *Expression* value are like **String.prototype.concat** rather than

the **+** operator.

TemplateMiddleList : *TemplateMiddleList* *TemplateMiddle* *Expression*

1. Let *rest* be the result of evaluating *TemplateMiddleList*.
2. **ReturnIfAbrupt**(*rest*).
3. Let *middle* be the TV of *TemplateMiddle* as defined in 11.8.6.
4. Let *subRef* be the result of evaluating *Expression*.
5. Let *sub* be ? **GetValue**(*subRef*).
6. Let *last* be ? **ToString**(*sub*).
7. Return the sequence of code units consisting of the elements of *rest* followed by the code units of *middle* followed by the elements of *last*.

NOTE 3

The string conversion semantics applied to the *Expression* value are like **String.prototype.concat** rather than the **+** operator.

12.2.10 The Grouping Operator

12.2.10.1 Static Semantics: Early Errors

PrimaryExpression : *CoverParenthesizedExpressionAndArrowParameterList*

It is a Syntax Error if *CoverParenthesizedExpressionAndArrowParameterList* is not **covering** a *ParenthesizedExpression*.

All Early Error rules for *ParenthesizedExpression* and its derived productions also apply to *CoveredParenthesizedExpression* of *CoverParenthesizedExpressionAndArrowParameterList*.

12.2.10.2 Static Semantics: IsFunctionDefinition

ParenthesizedExpression : (*Expression*)

1. Return **IsFunctionDefinition** of *Expression*.

12.2.10.3 Static Semantics: AssignmentTargetType

ParenthesizedExpression : (*Expression*)

1. Return **AssignmentTargetType** of *Expression*.

12.2.10.4 Runtime Semantics: NamedEvaluation

With parameter *name*.

PrimaryExpression : *CoverParenthesizedExpressionAndArrowParameterList*

1. Let *expr* be *CoveredParenthesizedExpression* of *CoverParenthesizedExpressionAndArrowParameterList*.
2. Return the result of performing **NamedEvaluation** for *expr* with argument *name*.

ParenthesizedExpression : (*Expression*)

1. **Assert**: **IsAnonymousFunctionDefinition**(*Expression*) is **true**.

2. Return the result of performing NamedEvaluation for *Expression* with argument *name*.

12.2.10.5 Runtime Semantics: Evaluation

PrimaryExpression : *CoverParenthesizedExpressionAndArrowParameterList*

1. Let *expr* be CoveredParenthesizedExpression of *CoverParenthesizedExpressionAndArrowParameterList*.
2. Return the result of evaluating *expr*.

ParenthesizedExpression : (*Expression*)

1. Return the result of evaluating *Expression*. This may be of type *Reference*.

NOTE

This algorithm does not apply *GetValue* to the result of evaluating *Expression*. The principal motivation for this is so that operators such as **delete** and **typeof** may be applied to parenthesized expressions.

12.3 Left-Hand-Side Expressions

Syntax

```
MemberExpression[Yield, Await] :  
    PrimaryExpression[?Yield, ?Await]  
    MemberExpression[?Yield, ?Await] [ Expression[+In, ?Yield, ?Await] ]  
    MemberExpression[?Yield, ?Await] . IdentifierName  
    MemberExpression[?Yield, ?Await] TemplateLiteral[?Yield, ?Await, +Tagged]  
    SuperProperty[?Yield, ?Await]  
    MetaProperty  
    new MemberExpression[?Yield, ?Await] Arguments[?Yield, ?Await]
```

```
SuperProperty[Yield, Await] :  
    super [ Expression[+In, ?Yield, ?Await] ]  
    super . IdentifierName
```

```
MetaProperty :  
    NewTarget
```

```
NewTarget :  
    new . target
```

```
NewExpression[Yield, Await] :  
    MemberExpression[?Yield, ?Await]  
    new NewExpression[?Yield, ?Await]
```

```
CallExpression[Yield, Await] :  
    CoverCallExpressionAndAsyncArrowHead[?Yield, ?Await]  
    SuperCall[?Yield, ?Await]  
    CallExpression[?Yield, ?Await] Arguments[?Yield, ?Await]
```

```

CallExpression[?Yield, ?Await] [ Expression[+In, ?Yield, ?Await] ]
CallExpression[?Yield, ?Await] . IdentifierName
CallExpression[?Yield, ?Await] TemplateLiteral[?Yield, ?Await, +Tagged]

```

```

SuperCall[Yield, Await] :
    super Arguments[?Yield, ?Await]

```

```

Arguments[Yield, Await] :
    ( )
    ( ArgumentList[?Yield, ?Await] )
    ( ArgumentList[?Yield, ?Await] , )

```

```

ArgumentList[Yield, Await] :
    AssignmentExpression[+In, ?Yield, ?Await]
    ... AssignmentExpression[+In, ?Yield, ?Await]
    ArgumentList[?Yield, ?Await] , AssignmentExpression[+In, ?Yield, ?Await]
    ArgumentList[?Yield, ?Await] , ... AssignmentExpression[+In, ?Yield, ?Await]

```

```

LeftHandSideExpression[Yield, Await] :
    NewExpression[?Yield, ?Await]
    CallExpression[?Yield, ?Await]

```

Supplemental Syntax

When processing an instance of the production *CallExpression* : *CoverCallExpressionAndAsyncArrowHead* the interpretation of *CoverCallExpressionAndAsyncArrowHead* is refined using the following grammar:

```

CallMemberExpression[Yield, Await] :
    MemberExpression[?Yield, ?Await] Arguments[?Yield, ?Await]

```

12.3.1 Static Semantics

12.3.1.1 Static Semantics: CoveredCallExpression

CallExpression : *CoverCallExpressionAndAsyncArrowHead*

1. Return the *CallMemberExpression* that is **covered** by *CoverCallExpressionAndAsyncArrowHead*.

12.3.1.2 Static Semantics: Contains

With parameter *symbol*.

MemberExpression : *MemberExpression* . *IdentifierName*

1. If *MemberExpression* Contains *symbol* is **true**, return **true**.
2. If *symbol* is a *ReservedWord*, return **false**.
3. If *symbol* is an *Identifier* and StringValue of *symbol* is the same value as the StringValue of *IdentifierName*, return **true**.
4. Return **false**.

SuperProperty : **super** . *IdentifierName*

1. If *symbol* is the *ReservedWord* **super**, return **true**.
2. If *symbol* is a *ReservedWord*, return **false**.
3. If *symbol* is an *Identifier* and *StringValue* of *symbol* is the same value as the *StringValue* of *IdentifierName*, return **true**.
4. Return **false**.

CallExpression : *CallExpression* . *IdentifierName*

1. If *CallExpression* Contains *symbol* is **true**, return **true**.
2. If *symbol* is a *ReservedWord*, return **false**.
3. If *symbol* is an *Identifier* and *StringValue* of *symbol* is the same value as the *StringValue* of *IdentifierName*, return **true**.
4. Return **false**.

12.3.1.3 Static Semantics: IsFunctionDefinition

MemberExpression :

MemberExpression [*Expression*]
MemberExpression . *IdentifierName*
MemberExpression *TemplateLiteral*
SuperProperty
MetaProperty
new *MemberExpression* *Arguments*

NewExpression :

new *NewExpression*

LeftHandSideExpression :

CallExpression

1. Return **false**.

12.3.1.4 Static Semantics: IsDestructuring

MemberExpression : *PrimaryExpression*

1. If *PrimaryExpression* is either an *ObjectLiteral* or an *ArrayLiteral*, return **true**.
2. Return **false**.

MemberExpression :

MemberExpression [*Expression*]
MemberExpression . *IdentifierName*
MemberExpression *TemplateLiteral*
SuperProperty
MetaProperty
new *MemberExpression* *Arguments*

NewExpression :

new *NewExpression*

LeftHandSideExpression :

CallExpression

1. Return **false**.

12.3.1.5 Static Semantics: **IsIdentifierRef**

MemberExpression :

MemberExpression [*Expression*]

MemberExpression . *IdentifierName*

MemberExpression *TemplateLiteral*

SuperProperty

MetaProperty

new *MemberExpression* *Arguments*

NewExpression :

new *NewExpression*

LeftHandSideExpression :

CallExpression

1. Return **false**.

12.3.1.6 Static Semantics: **AssignmentTargetType**

CallExpression :

CallExpression [*Expression*]

CallExpression . *IdentifierName*

MemberExpression :

MemberExpression [*Expression*]

MemberExpression . *IdentifierName*

SuperProperty

1. Return **simple**.

CallExpression :

CoverCallExpressionAndAsyncArrowHead

SuperCall

CallExpression *Arguments*

CallExpression *TemplateLiteral*

NewExpression :

new *NewExpression*

MemberExpression :

MemberExpression *TemplateLiteral*

new *MemberExpression* *Arguments*

NewTarget :

new . **target**

1. Return invalid.

12.3.2 Property Accessors

NOTE

Properties are accessed by name, using either the dot notation:

MemberExpression . *IdentifierName*

CallExpression . *IdentifierName*

or the bracket notation:

MemberExpression [*Expression*]

CallExpression [*Expression*]

The dot notation is explained by the following syntactic conversion:

MemberExpression . *IdentifierName*

is identical in its behaviour to

MemberExpression [<identifier-name-string>]

and similarly

CallExpression . *IdentifierName*

is identical in its behaviour to

CallExpression [<identifier-name-string>]

where <identifier-name-string> is the result of evaluating StringValue of *IdentifierName*.

12.3.2.1 Runtime Semantics: Evaluation

MemberExpression : *MemberExpression* [*Expression*]

1. Let *baseReference* be the result of evaluating *MemberExpression*.
2. Let *baseValue* be ? *GetValue*(*baseReference*).
3. Let *propertyNameReference* be the result of evaluating *Expression*.
4. Let *propertyNameValue* be ? *GetValue*(*propertyNameReference*).
5. Let *bv* be ? *RequireObjectCoercible*(*baseValue*).
6. Let *propertyKey* be ? *ToPropertyKey*(*propertyNameValue*).
7. If the code matched by this *MemberExpression* is *strict mode code*, let *strict* be **true**, else let *strict* be **false**.
8. Return a value of type *Reference* whose base value component is *bv*, whose referenced name component is *propertyKey*, and whose strict reference flag is *strict*.

MemberExpression : *MemberExpression* . *IdentifierName*

1. Let *baseReference* be the result of evaluating *MemberExpression*.
2. Let *baseValue* be ? *GetValue*(*baseReference*).
3. Let *bv* be ? *RequireObjectCoercible*(*baseValue*).
4. Let *propertyNameString* be StringValue of *IdentifierName*.

5. If the code matched by this *MemberExpression* is *strict mode code*, let *strict* be **true**, else let *strict* be **false**.
6. Return a value of type *Reference* whose base value component is *bv*, whose referenced name component is *propertyNameString*, and whose strict reference flag is *strict*.

CallExpression : *CallExpression* [*Expression*]

Is evaluated in exactly the same manner as *MemberExpression* : *MemberExpression* [*Expression*] except that the contained *CallExpression* is evaluated in step 1.

CallExpression : *CallExpression* . *IdentifierName*

Is evaluated in exactly the same manner as *MemberExpression* : *MemberExpression* . *IdentifierName* except that the contained *CallExpression* is evaluated in step 1.

12.3.3 The new Operator

12.3.3.1 Runtime Semantics: Evaluation

NewExpression : **new** *NewExpression*

1. Return ? *EvaluateNew*(*NewExpression*, empty).

MemberExpression : **new** *MemberExpression* *Arguments*

1. Return ? *EvaluateNew*(*MemberExpression*, *Arguments*).

12.3.3.1.1 Runtime Semantics: EvaluateNew (*constructExpr*, *arguments*)

The abstract operation *EvaluateNew* with arguments *constructExpr*, and *arguments* performs the following steps:

1. *Assert*: *constructExpr* is either a *NewExpression* or a *MemberExpression*.
2. *Assert*: *arguments* is either empty or an *Arguments*.
3. Let *ref* be the result of evaluating *constructExpr*.
4. Let *constructor* be ? *GetValue*(*ref*).
5. If *arguments* is empty, let *argList* be a new empty *List*.
6. Else,
 - a. Let *argList* be *ArgumentListEvaluation* of *arguments*.
 - b. *ReturnIfAbrupt*(*argList*).
7. If *IsConstructor*(*constructor*) is **false**, throw a **TypeError** exception.
8. Return ? *Construct*(*constructor*, *argList*).

12.3.4 Function Calls

12.3.4.1 Runtime Semantics: Evaluation

CallExpression : *CoverCallExpressionAndAsyncArrowHead*

1. Let *expr* be *CoveredCallExpression* of *CoverCallExpressionAndAsyncArrowHead*.
2. Let *memberExpr* be the *MemberExpression* of *expr*.
3. Let *arguments* be the *Arguments* of *expr*.
4. Let *ref* be the result of evaluating *memberExpr*.
5. Let *func* be ? *GetValue*(*ref*).

6. If **Type**(*ref*) is **Reference** and **IsPropertyReference**(*ref*) is **false** and **GetReferencedName**(*ref*) is "**eval**", then
 - a. If **SameValue**(*func*, %eval%) is **true**, then
 - i. Let *argList* be ? **ArgumentListEvaluation** of *arguments*.
 - ii. If *argList* has no elements, return **undefined**.
 - iii. Let *evalText* be the first element of *argList*.
 - iv. If the source code matching this *CallExpression* is **strict mode code**, let *strictCaller* be **true**. Otherwise let *strictCaller* be **false**.
 - v. Let *evalRealm* be the **current Realm Record**.
 - vi. Perform ? **HostEnsureCanCompileStrings**(*evalRealm*, *evalRealm*).
 - vii. Return ? **PerformEval**(*evalText*, *evalRealm*, *strictCaller*, **true**).
7. Let *thisCall* be this *CallExpression*.
8. Let *tailCall* be **IsInTailPosition**(*thisCall*).
9. Return ? **EvaluateCall**(*func*, *ref*, *arguments*, *tailCall*).

A *CallExpression* evaluation that executes step 6.a.vii is a direct eval.

CallExpression : *CallExpression* Arguments

1. Let *ref* be the result of evaluating *CallExpression*.
2. Let *func* be ? **GetValue**(*ref*).
3. Let *thisCall* be this *CallExpression*.
4. Let *tailCall* be **IsInTailPosition**(*thisCall*).
5. Return ? **EvaluateCall**(*func*, *ref*, *Arguments*, *tailCall*).

12.3.4.2 Runtime Semantics: EvaluateCall (*func*, *ref*, *arguments*, *tailPosition*)

The abstract operation EvaluateCall takes as arguments a value *func*, a value *ref*, a **Parse Node** *arguments*, and a Boolean argument *tailPosition*. It performs the following steps:

1. If **Type**(*ref*) is **Reference**, then
 - a. If **IsPropertyReference**(*ref*) is **true**, then
 - i. Let *thisValue* be **GetThisValue**(*ref*).
 - b. Else the base of *ref* is an **Environment Record**,
 - i. Let *refEnv* be **GetBase**(*ref*).
 - ii. Let *thisValue* be *refEnv*.WithBaseObject().
2. Else **Type**(*ref*) is not **Reference**,
 - a. Let *thisValue* be **undefined**.
3. Let *argList* be **ArgumentListEvaluation** of *arguments*.
4. **ReturnIfAbrupt**(*argList*).
5. If **Type**(*func*) is not **Object**, throw a **TypeError** exception.
6. If **IsCallable**(*func*) is **false**, throw a **TypeError** exception.
7. If *tailPosition* is **true**, perform **PrepareForTailCall**().
8. Let *result* be **Call**(*func*, *thisValue*, *argList*).
9. **Assert**: If *tailPosition* is **true**, the above call will not return here, but instead evaluation will continue as if the following return has already occurred.
10. **Assert**: If *result* is not an **abrupt completion**, then **Type**(*result*) is an **ECMAScript language type**.
11. Return *result*.

12.3.5 The super Keyword

12.3.5.1 Runtime Semantics: Evaluation

SuperProperty : **super** [*Expression*]

1. Let *env* be *GetThisEnvironment*().
2. Let *actualThis* be ? *env*.*GetThisBinding*().
3. Let *propertyNameReference* be the result of evaluating *Expression*.
4. Let *propertyNameValue* be ? *GetValue*(*propertyNameReference*).
5. Let *propertyKey* be ? *ToPropertyKey*(*propertyNameValue*).
6. If the code matched by this *SuperProperty* is *strict mode code*, let *strict* be **true**, else let *strict* be **false**.
7. Return ? *MakeSuperPropertyReference*(*actualThis*, *propertyKey*, *strict*).

SuperProperty : **super** . *IdentifierName*

1. Let *env* be *GetThisEnvironment*().
2. Let *actualThis* be ? *env*.*GetThisBinding*().
3. Let *propertyKey* be *StringValue* of *IdentifierName*.
4. If the code matched by this *SuperProperty* is *strict mode code*, let *strict* be **true**, else let *strict* be **false**.
5. Return ? *MakeSuperPropertyReference*(*actualThis*, *propertyKey*, *strict*).

SuperCall : **super** *Arguments*

1. Let *newTarget* be *GetNewTarget*().
2. **Assert**: *Type*(*newTarget*) is **Object**.
3. Let *func* be ? *GetSuperConstructor*().
4. Let *argList* be *ArgumentListEvaluation* of *Arguments*.
5. **ReturnIfAbrupt**(*argList*).
6. Let *result* be ? *Construct*(*func*, *argList*, *newTarget*).
7. Let *thisER* be *GetThisEnvironment*().
8. Return ? *thisER*.*BindThisValue*(*result*).

12.3.5.2 Runtime Semantics: GetSuperConstructor ()

The abstract operation *GetSuperConstructor* performs the following steps:

1. Let *envRec* be *GetThisEnvironment*().
2. **Assert**: *envRec* is a *function Environment Record*.
3. Let *activeFunction* be *envRec*.[[*FunctionObject*]].
4. **Assert**: *activeFunction* is an ECMAScript *function object*.
5. Let *superConstructor* be ! *activeFunction*.[[*GetPrototypeOf*]]().
6. If *IsConstructor*(*superConstructor*) is **false**, throw a **TypeError** exception.
7. Return *superConstructor*.

12.3.5.3 Runtime Semantics: MakeSuperPropertyReference (*actualThis*, *propertyKey*, *strict*)

The abstract operation *MakeSuperPropertyReference* with arguments *actualThis*, *propertyKey*, and *strict* performs the following steps:

1. Let *env* be *GetThisEnvironment*().
2. **Assert**: *env*.*HasSuperBinding*() is **true**.
3. Let *baseValue* be ? *env*.*GetSuperBase*().
4. Let *bv* be ? *RequireObjectCoercible*(*baseValue*).

- Return a value of type **Reference** that is a **Super Reference** whose base value component is *bv*, whose referenced name component is *propertyKey*, whose thisValue component is *actualThis*, and whose strict reference flag is *strict*.

12.3.6 Argument Lists

NOTE

The evaluation of an argument list produces a **List** of values.

12.3.6.1 Runtime Semantics: ArgumentListEvaluation

Arguments : ()

- Return a new empty **List**.

ArgumentList : *AssignmentExpression*

- Let *ref* be the result of evaluating *AssignmentExpression*.
- Let *arg* be ? **GetValue**(*ref*).
- Return a **List** whose sole item is *arg*.

ArgumentList : . . . *AssignmentExpression*

- Let *list* be a new empty **List**.
- Let *spreadRef* be the result of evaluating *AssignmentExpression*.
- Let *spreadObj* be ? **GetValue**(*spreadRef*).
- Let *iteratorRecord* be ? **GetIterator**(*spreadObj*).
- Repeat,
 - Let *next* be ? **IteratorStep**(*iteratorRecord*).
 - If *next* is **false**, return *list*.
 - Let *nextArg* be ? **IteratorValue**(*next*).
 - Append *nextArg* as the last element of *list*.

ArgumentList : *ArgumentList* , *AssignmentExpression*

- Let *precedingArgs* be **ArgumentListEvaluation** of *ArgumentList*.
- ReturnIfAbrupt**(*precedingArgs*).
- Let *ref* be the result of evaluating *AssignmentExpression*.
- Let *arg* be ? **GetValue**(*ref*).
- Append *arg* to the end of *precedingArgs*.
- Return *precedingArgs*.

ArgumentList : *ArgumentList* , . . . *AssignmentExpression*

- Let *precedingArgs* be **ArgumentListEvaluation** of *ArgumentList*.
- ReturnIfAbrupt**(*precedingArgs*).
- Let *spreadRef* be the result of evaluating *AssignmentExpression*.
- Let *iteratorRecord* be ? **GetIterator**(? **GetValue**(*spreadRef*)).
- Repeat,
 - Let *next* be ? **IteratorStep**(*iteratorRecord*).
 - If *next* is **false**, return *precedingArgs*.
 - Let *nextArg* be ? **IteratorValue**(*next*).

- d. Append *nextArg* as the last element of *precedingArgs*.

12.3.7 Tagged Templates

NOTE

A tagged template is a function call where the arguments of the call are derived from a *TemplateLiteral* (12.2.9). The actual arguments include a template object (12.2.9.4) and the values produced by evaluating the expressions embedded within the *TemplateLiteral*.

12.3.7.1 Runtime Semantics: Evaluation

MemberExpression : *MemberExpression* *TemplateLiteral*

1. Let *tagRef* be the result of evaluating *MemberExpression*.
2. Let *tagFunc* be ? *GetValue*(*tagRef*).
3. Let *thisCall* be this *MemberExpression*.
4. Let *tailCall* be *IsInTailPosition*(*thisCall*).
5. Return ? *EvaluateCall*(*tagFunc*, *tagRef*, *TemplateLiteral*, *tailCall*).

CallExpression : *CallExpression* *TemplateLiteral*

1. Let *tagRef* be the result of evaluating *CallExpression*.
2. Let *tagFunc* be ? *GetValue*(*tagRef*).
3. Let *thisCall* be this *CallExpression*.
4. Let *tailCall* be *IsInTailPosition*(*thisCall*).
5. Return ? *EvaluateCall*(*tagFunc*, *tagRef*, *TemplateLiteral*, *tailCall*).

12.3.8 Meta Properties

12.3.8.1 Runtime Semantics: Evaluation

NewTarget : **new** . **target**

1. Return *GetNewTarget*().

12.4 Update Expressions

Syntax

*UpdateExpression*_[Yield, Await] :

*LeftHandSideExpression*_[?Yield, ?Await]

*LeftHandSideExpression*_[?Yield, ?Await] [no *LineTerminator* here] ++

*LeftHandSideExpression*_[?Yield, ?Await] [no *LineTerminator* here] --

++ *UnaryExpression*_[?Yield, ?Await]

-- *UnaryExpression*_[?Yield, ?Await]

12.4.1 Static Semantics: Early Errors

UpdateExpression :

LeftHandSideExpression ++

LeftHandSideExpression --

It is an early [Reference](#) Error if AssignmentTargetType of *LeftHandSideExpression* is invalid.

It is an early Syntax Error if AssignmentTargetType of *LeftHandSideExpression* is strict.

UpdateExpression :

++ *UnaryExpression*

-- *UnaryExpression*

It is an early [Reference](#) Error if AssignmentTargetType of *UnaryExpression* is invalid.

It is an early Syntax Error if AssignmentTargetType of *UnaryExpression* is strict.

12.4.2 Static Semantics: IsFunctionDefinition

UpdateExpression :

LeftHandSideExpression ++

LeftHandSideExpression --

++ *UnaryExpression*

-- *UnaryExpression*

1. Return **false**.

12.4.3 Static Semantics: AssignmentTargetType

UpdateExpression :

LeftHandSideExpression ++

LeftHandSideExpression --

++ *UnaryExpression*

-- *UnaryExpression*

1. Return invalid.

12.4.4 Postfix Increment Operator

12.4.4.1 Runtime Semantics: Evaluation

UpdateExpression : *LeftHandSideExpression* ++

1. Let *lhs* be the result of evaluating *LeftHandSideExpression*.
2. Let *oldValue* be ? [ToNumber](#)(? [GetValue](#)(*lhs*)).
3. Let *newValue* be the result of adding the value 1 to *oldValue*, using the same rules as for the + operator (see [12.8.5](#)).
4. Perform ? [PutValue](#)(*lhs*, *newValue*).
5. Return *oldValue*.

12.4.5 Postfix Decrement Operator

12.4.5.1 Runtime Semantics: Evaluation

UpdateExpression : *LeftHandSideExpression* --

1. Let *lhs* be the result of evaluating *LeftHandSideExpression*.
2. Let *oldValue* be ? *ToNumber*(? *GetValue*(*lhs*)).
3. Let *newValue* be the result of subtracting the value 1 from *oldValue*, using the same rules as for the *-* operator (see 12.8.5).
4. Perform ? *PutValue*(*lhs*, *newValue*).
5. Return *oldValue*.

12.4.6 Prefix Increment Operator

12.4.6.1 Runtime Semantics: Evaluation

UpdateExpression : ++ *UnaryExpression*

1. Let *expr* be the result of evaluating *UnaryExpression*.
2. Let *oldValue* be ? *ToNumber*(? *GetValue*(*expr*)).
3. Let *newValue* be the result of adding the value 1 to *oldValue*, using the same rules as for the *+* operator (see 12.8.5).
4. Perform ? *PutValue*(*expr*, *newValue*).
5. Return *newValue*.

12.4.7 Prefix Decrement Operator

12.4.7.1 Runtime Semantics: Evaluation

UpdateExpression : -- *UnaryExpression*

1. Let *expr* be the result of evaluating *UnaryExpression*.
2. Let *oldValue* be ? *ToNumber*(? *GetValue*(*expr*)).
3. Let *newValue* be the result of subtracting the value 1 from *oldValue*, using the same rules as for the *-* operator (see 12.8.5).
4. Perform ? *PutValue*(*expr*, *newValue*).
5. Return *newValue*.

12.5 Unary Operators

Syntax

```
UnaryExpression[Yield, Await] :  
  UpdateExpression[?Yield, ?Await]  
  delete UnaryExpression[?Yield, ?Await]  
  void UnaryExpression[?Yield, ?Await]  
  typeof UnaryExpression[?Yield, ?Await]  
  + UnaryExpression[?Yield, ?Await]  
  - UnaryExpression[?Yield, ?Await]  
  ~ UnaryExpression[?Yield, ?Await]
```


! UnaryExpression_[?Yield, ?Await]
 [+Await] AwaitExpression_[?Yield]

12.5.1 Static Semantics: IsFunctionDefinition

UnaryExpression :

- delete** UnaryExpression
- void** UnaryExpression
- typeof** UnaryExpression
- +** UnaryExpression
- UnaryExpression
- ~** UnaryExpression
- !** UnaryExpression
- AwaitExpression

1. Return **false**.

12.5.2 Static Semantics: AssignmentTargetType

UnaryExpression :

- delete** UnaryExpression
- void** UnaryExpression
- typeof** UnaryExpression
- +** UnaryExpression
- UnaryExpression
- ~** UnaryExpression
- !** UnaryExpression
- AwaitExpression

1. Return invalid.

12.5.3 The delete Operator

12.5.3.1 Static Semantics: Early Errors

UnaryExpression : **delete** UnaryExpression

It is a Syntax Error if the *UnaryExpression* is contained in **strict mode code** and the derived *UnaryExpression* is *PrimaryExpression* : *IdentifierReference* .

It is a Syntax Error if the derived *UnaryExpression* is

PrimaryExpression : *CoverParenthesizedExpressionAndArrowParameterList*

and *CoverParenthesizedExpressionAndArrowParameterList* ultimately derives a phrase that, if used in place of *UnaryExpression*, would produce a Syntax Error according to these rules. This rule is recursively applied.

NOTE

The last rule means that expressions such as **delete** ((**(foo)**)) produce early errors because of recursive application of the first rule.

12.5.3.2 Runtime Semantics: Evaluation

UnaryExpression : **delete** *UnaryExpression*

1. Let *ref* be the result of evaluating *UnaryExpression*.
2. **ReturnIfAbrupt**(*ref*).
3. If **Type**(*ref*) is not **Reference**, return **true**.
4. If **IsUnresolvableReference**(*ref*) is **true**, then
 - a. **Assert**: **IsStrictReference**(*ref*) is **false**.
 - b. Return **true**.
5. If **IsPropertyReference**(*ref*) is **true**, then
 - a. If **IsSuperReference**(*ref*) is **true**, throw a **ReferenceError** exception.
 - b. Let *baseObj* be ! **ToObject**(**GetBase**(*ref*)).
 - c. Let *deleteStatus* be ? *baseObj*.[[Delete]](**GetReferencedName**(*ref*)).
 - d. If *deleteStatus* is **false** and **IsStrictReference**(*ref*) is **true**, throw a **TypeError** exception.
 - e. Return *deleteStatus*.
6. Else *ref* is a **Reference** to an **Environment Record** binding,
 - a. Let *bindings* be **GetBase**(*ref*).
 - b. Return ? *bindings*.DeleteBinding(**GetReferencedName**(*ref*)).

NOTE

When a **delete** operator occurs within **strict mode code**, a **SyntaxError** exception is thrown if its *UnaryExpression* is a direct reference to a variable, function argument, or function name. In addition, if a **delete** operator occurs within **strict mode code** and the property to be deleted has the attribute { [[Configurable]]: **false** }, a **TypeError** exception is thrown.

12.5.4 The void Operator

12.5.4.1 Runtime Semantics: Evaluation

UnaryExpression : **void** *UnaryExpression*

1. Let *expr* be the result of evaluating *UnaryExpression*.
2. Perform ? **GetValue**(*expr*).
3. Return **undefined**.

NOTE

GetValue must be called even though its value is not used because it may have observable side-effects.

12.5.5 The typeof Operator

12.5.5.1 Runtime Semantics: Evaluation

UnaryExpression : **typeof** *UnaryExpression*

1. Let *val* be the result of evaluating *UnaryExpression*.
2. If **Type**(*val*) is **Reference**, then
 - a. If **IsUnresolvableReference**(*val*) is **true**, return **"undefined"**.
3. Set *val* to ? **GetValue**(*val*).

4. Return a String according to [Table 35](#).

Table 35: typeof Operator Results

Type of <i>val</i>	Result
Undefined	"undefined"
Null	"object"
Boolean	"boolean"
Number	"number"
String	"string"
Symbol	"symbol"
Object (ordinary and does not implement <code>[[Call]]</code>)	"object"
Object (standard exotic and does not implement <code>[[Call]]</code>)	"object"
Object (implements <code>[[Call]]</code>)	"function"
Object (non-standard exotic and does not implement <code>[[Call]]</code>)	Implementation-defined. Must not be "undefined", "boolean", "function", "number", "symbol", or "string".

NOTE

Implementations are discouraged from defining new **typeof** result values for non-standard exotic objects. If possible "object" should be used for such objects.

12.5.6 Unary + Operator

NOTE

The unary + operator converts its operand to Number type.

12.5.6.1 Runtime Semantics: Evaluation

UnaryExpression : + *UnaryExpression*

- 1. Let *expr* be the result of evaluating *UnaryExpression*.
- 2. Return ? **ToNumber**(? **GetValue**(*expr*)).

12.5.7 Unary - Operator

NOTE

The unary - operator converts its operand to Number type and then negates it. Negating +0 produces -0, and negating -0 produces +0.

12.5.7.1 Runtime Semantics: Evaluation

UnaryExpression : - *UnaryExpression*

1. Let *expr* be the result of evaluating *UnaryExpression*.
2. Let *oldValue* be ? *ToNumber*(? *GetValue*(*expr*)).
3. If *oldValue* is NaN, return NaN.
4. Return the result of negating *oldValue*; that is, compute a Number with the same magnitude but opposite sign.

12.5.8 Bitwise NOT Operator (~)

12.5.8.1 Runtime Semantics: Evaluation

UnaryExpression : ~ *UnaryExpression*

1. Let *expr* be the result of evaluating *UnaryExpression*.
2. Let *oldValue* be ? *ToInt32*(? *GetValue*(*expr*)).
3. Return the result of applying bitwise complement to *oldValue*. The result is a signed 32-bit integer.

12.5.9 Logical NOT Operator (!)

12.5.9.1 Runtime Semantics: Evaluation

UnaryExpression : ! *UnaryExpression*

1. Let *expr* be the result of evaluating *UnaryExpression*.
2. Let *oldValue* be ? *ToBoolean*(? *GetValue*(*expr*)).
3. If *oldValue* is true, return false.
4. Return true.

12.6 Exponentiation Operator

Syntax

```
ExponentiationExpression[Yield, Await] :  
    UnaryExpression[?Yield, ?Await]  
    UpdateExpression[?Yield, ?Await] ** ExponentiationExpression[?Yield, ?Await]
```

12.6.1 Static Semantics: IsFunctionDefinition

```
ExponentiationExpression :  
    UpdateExpression ** ExponentiationExpression
```

1. Return false.

12.6.2 Static Semantics: AssignmentTargetType

```
ExponentiationExpression :  
    UpdateExpression ** ExponentiationExpression
```

1. Return invalid.

12.6.3 Runtime Semantics: Evaluation

ExponentiationExpression : *UpdateExpression* ** *ExponentiationExpression*

1. Let *left* be the result of evaluating *UpdateExpression*.
2. Let *leftValue* be ? *GetValue*(*left*).
3. Let *right* be the result of evaluating *ExponentiationExpression*.
4. Let *rightValue* be ? *GetValue*(*right*).
5. Let *base* be ? *ToNumber*(*leftValue*).
6. Let *exponent* be ? *ToNumber*(*rightValue*).
7. Return the result of Applying the ** operator with *base* and *exponent* as specified in 12.6.4.

12.6.4 Applying the ** Operator

Returns an implementation-dependent approximation of the result of raising *base* to the power *exponent*.

- If *exponent* is NaN, the result is NaN.
- If *exponent* is +0, the result is 1, even if *base* is NaN.
- If *exponent* is -0, the result is 1, even if *base* is NaN.
- If *base* is NaN and *exponent* is nonzero, the result is NaN.
- If *abs*(*base*) > 1 and *exponent* is +∞, the result is +∞.
- If *abs*(*base*) > 1 and *exponent* is -∞, the result is +0.
- If *abs*(*base*) is 1 and *exponent* is +∞, the result is NaN.
- If *abs*(*base*) is 1 and *exponent* is -∞, the result is NaN.
- If *abs*(*base*) < 1 and *exponent* is +∞, the result is +0.
- If *abs*(*base*) < 1 and *exponent* is -∞, the result is +∞.
- If *base* is +∞ and *exponent* > 0, the result is +∞.
- If *base* is +∞ and *exponent* < 0, the result is +0.
- If *base* is -∞ and *exponent* > 0 and *exponent* is an odd integer, the result is -∞.
- If *base* is -∞ and *exponent* > 0 and *exponent* is not an odd integer, the result is +∞.
- If *base* is -∞ and *exponent* < 0 and *exponent* is an odd integer, the result is -0.
- If *base* is -∞ and *exponent* < 0 and *exponent* is not an odd integer, the result is +0.
- If *base* is +0 and *exponent* > 0, the result is +0.
- If *base* is +0 and *exponent* < 0, the result is +∞.
- If *base* is -0 and *exponent* > 0 and *exponent* is an odd integer, the result is -0.
- If *base* is -0 and *exponent* > 0 and *exponent* is not an odd integer, the result is +0.
- If *base* is -0 and *exponent* < 0 and *exponent* is an odd integer, the result is -∞.
- If *base* is -0 and *exponent* < 0 and *exponent* is not an odd integer, the result is +∞.
- If *base* < 0 and *base* is finite and *exponent* is finite and *exponent* is not an integer, the result is NaN.

NOTE

The result of *base* ** *exponent* when *base* is 1 or -1 and *exponent* is +Infinity or -Infinity differs from IEEE 754-2008. The first edition of ECMAScript specified a result of NaN for this operation, whereas later versions of IEEE 754-2008 specified 1. The historical ECMAScript behaviour is preserved for compatibility reasons.

12.7 Multiplicative Operators

12.7 Multiplicative Operators

Syntax

```
MultiplicativeExpression[Yield, Await] :  
  ExponentiationExpression[?Yield, ?Await]  
  MultiplicativeExpression[?Yield, ?Await] MultiplicativeOperator  
    ExponentiationExpression[?Yield, ?Await]  
  
MultiplicativeOperator : one of  
  * / %
```

12.7.1 Static Semantics: IsFunctionDefinition

MultiplicativeExpression : *MultiplicativeExpression* *MultiplicativeOperator* *ExponentiationExpression*

1. Return **false**.

12.7.2 Static Semantics: AssignmentTargetType

MultiplicativeExpression : *MultiplicativeExpression* *MultiplicativeOperator* *ExponentiationExpression*

1. Return invalid.

12.7.3 Runtime Semantics: Evaluation

MultiplicativeExpression : *MultiplicativeExpression* *MultiplicativeOperator* *ExponentiationExpression*

1. Let *left* be the result of evaluating *MultiplicativeExpression*.
2. Let *leftValue* be ? *GetValue*(*left*).
3. Let *right* be the result of evaluating *ExponentiationExpression*.
4. Let *rightValue* be ? *GetValue*(*right*).
5. Let *lnum* be ? *ToNumber*(*leftValue*).
6. Let *rnum* be ? *ToNumber*(*rightValue*).
7. Return the result of applying the *MultiplicativeOperator* (*, /, or %) to *lnum* and *rnum* as specified in 12.7.3.1, 12.7.3.2, or 12.7.3.3.

12.7.3.1 Applying the * Operator

The * *MultiplicativeOperator* performs multiplication, producing the product of its operands. Multiplication is commutative. Multiplication is not always associative in ECMAScript, because of finite precision.

The result of a floating-point multiplication is governed by the rules of IEEE 754-2008 binary double-precision arithmetic:

If either operand is **NaN**, the result is **NaN**.

The sign of the result is positive if both operands have the same sign, negative if the operands have different signs.

Multiplication of an infinity by a zero results in **NaN**.

Multiplication of an infinity by an infinity results in an infinity. The sign is determined by the rule already stated above.

Multiplication of an infinity by a finite nonzero value results in a signed infinity. The sign is determined by the rule already stated above.

In the remaining cases, where neither an infinity nor **NaN** is involved, the product is computed and rounded to the nearest representable value using IEEE 754-2008 round to nearest, ties to even mode. If the magnitude is too large to represent, the result is then an infinity of appropriate sign. If the magnitude is too small to represent, the result is then a zero of appropriate sign. The ECMAScript language requires support of gradual underflow as defined by IEEE 754-2008.

12.7.3.2 Applying the / Operator

The */ MultiplicativeOperator* performs division, producing the quotient of its operands. The left operand is the dividend and the right operand is the divisor. ECMAScript does not perform integer division. The operands and result of all division operations are double-precision floating-point numbers. The result of division is determined by the specification of IEEE 754-2008 arithmetic:

If either operand is **NaN**, the result is **NaN**.

The sign of the result is positive if both operands have the same sign, negative if the operands have different signs.

Division of an infinity by an infinity results in **NaN**.

Division of an infinity by a zero results in an infinity. The sign is determined by the rule already stated above.

Division of an infinity by a nonzero finite value results in a signed infinity. The sign is determined by the rule already stated above.

Division of a finite value by an infinity results in zero. The sign is determined by the rule already stated above.

Division of a zero by a zero results in **NaN**; division of zero by any other finite value results in zero, with the sign determined by the rule already stated above.

Division of a nonzero finite value by a zero results in a signed infinity. The sign is determined by the rule already stated above.

In the remaining cases, where neither an infinity, nor a zero, nor **NaN** is involved, the quotient is computed and rounded to the nearest representable value using IEEE 754-2008 round to nearest, ties to even mode. If the magnitude is too large to represent, the operation overflows; the result is then an infinity of appropriate sign. If the magnitude is too small to represent, the operation underflows and the result is a zero of the appropriate sign. The ECMAScript language requires support of gradual underflow as defined by IEEE 754-2008.

12.7.3.3 Applying the % Operator

The *% MultiplicativeOperator* yields the remainder of its operands from an implied division; the left operand is the dividend and the right operand is the divisor.

NOTE

In C and C++, the remainder operator accepts only integral operands; in ECMAScript, it also accepts floating-point operands.

The result of a floating-point remainder operation as computed by the *%* operator is not the same as the “remainder” operation defined by IEEE 754-2008. The IEEE 754-2008 “remainder” operation computes the remainder from a rounding division, not a truncating division, and so its behaviour is not analogous to that of the usual integer remainder operator. Instead the ECMAScript language defines *%* on floating-point operations to behave in a manner analogous to that of the Java integer remainder operator; this may be compared with the C library function `fmod`.

The result of an ECMAScript floating-point remainder operation is determined by the rules of IEEE arithmetic:

If either operand is **NaN**, the result is **NaN**.

The sign of the result equals the sign of the dividend.

If the dividend is an infinity, or the divisor is a zero, or both, the result is **NaN**.

If the dividend is finite and the divisor is an infinity, the result equals the dividend.

If the dividend is a zero and the divisor is nonzero and finite, the result is the same as the dividend.

In the remaining cases, where neither an infinity, nor a zero, nor **NaN** is involved, the floating-point remainder r from a dividend n and a divisor d is defined by the mathematical relation $r = n - (d \times q)$ where q is an integer that is negative only if n/d is negative and positive only if n/d is positive, and whose magnitude is as large as possible without exceeding the magnitude of the true mathematical quotient of n and d . r is computed and rounded to the nearest representable value using IEEE 754-2008 round to nearest, ties to even mode.

12.8 Additive Operators

Syntax

```
AdditiveExpression[Yield, Await] :  
    MultiplicativeExpression[?Yield, ?Await]  
    AdditiveExpression[?Yield, ?Await] + MultiplicativeExpression[?Yield, ?Await]  
    AdditiveExpression[?Yield, ?Await] - MultiplicativeExpression[?Yield, ?Await]
```

12.8.1 Static Semantics: IsFunctionDefinition

```
AdditiveExpression :  
    AdditiveExpression + MultiplicativeExpression  
    AdditiveExpression - MultiplicativeExpression
```

1. Return **false**.

12.8.2 Static Semantics: AssignmentTargetType

```
AdditiveExpression :  
    AdditiveExpression + MultiplicativeExpression  
    AdditiveExpression - MultiplicativeExpression
```

1. Return invalid.

12.8.3 The Addition Operator (+)

NOTE

The addition operator either performs string concatenation or numeric addition.

12.8.3.1 Runtime Semantics: Evaluation

AdditiveExpression : *AdditiveExpression* + *MultiplicativeExpression*

1. Let *lref* be the result of evaluating *AdditiveExpression*.
2. Let *lval* be ? *GetValue*(*lref*).
3. Let *rref* be the result of evaluating *MultiplicativeExpression*.
4. Let *rval* be ? *GetValue*(*rref*).
5. Let *lprim* be ? *ToPrimitive*(*lval*).

6. Let *rprim* be ? [ToPrimitive\(rval\)](#).
7. If [Type\(lprim\)](#) is String or [Type\(rprim\)](#) is String, then
 - a. Let *lstr* be ? [ToString\(lprim\)](#).
 - b. Let *rstr* be ? [ToString\(rprim\)](#).
 - c. Return the [string-concatenation](#) of *lstr* and *rstr*.
8. Let *lnum* be ? [ToNumber\(lprim\)](#).
9. Let *rnum* be ? [ToNumber\(rprim\)](#).
10. Return the result of applying the addition operation to *lnum* and *rnum*. See the Note below [12.8.5](#).

NOTE 1

No hint is provided in the calls to [ToPrimitive](#) in steps 5 and 6. All standard objects except Date objects handle the absence of a hint as if the hint Number were given; Date objects handle the absence of a hint as if the hint String were given. Exotic objects may handle the absence of a hint in some other manner.

NOTE 2

Step 7 differs from step 3 of the [Abstract Relational Comparison](#) algorithm, by using the logical-or operation instead of the logical-and operation.

12.8.4 The Subtraction Operator (-)

12.8.4.1 Runtime Semantics: Evaluation

AdditiveExpression : *AdditiveExpression* - *MultiplicativeExpression*

1. Let *lref* be the result of evaluating *AdditiveExpression*.
2. Let *lval* be ? [GetValue\(lref\)](#).
3. Let *rref* be the result of evaluating *MultiplicativeExpression*.
4. Let *rval* be ? [GetValue\(rref\)](#).
5. Let *lnum* be ? [ToNumber\(lval\)](#).
6. Let *rnum* be ? [ToNumber\(rval\)](#).
7. Return the result of applying the subtraction operation to *lnum* and *rnum*. See the note below [12.8.5](#).

12.8.5 Applying the Additive Operators to Numbers

The + operator performs addition when applied to two operands of numeric type, producing the sum of the operands. The - operator performs subtraction, producing the difference of two numeric operands.

Addition is a commutative operation, but not always associative.

The result of an addition is determined using the rules of IEEE 754-2008 binary double-precision arithmetic:

If either operand is **NaN**, the result is **NaN**.

The sum of two infinities of opposite sign is **NaN**.

The sum of two infinities of the same sign is the infinity of that sign.

The sum of an infinity and a finite value is equal to the infinite operand.

The sum of two negative zeroes is **-0**. The sum of two positive zeroes, or of two zeroes of opposite sign, is **+0**.

The sum of a zero and a nonzero finite value is equal to the nonzero operand.

The sum of two nonzero finite values of the same magnitude and opposite sign is **+0**.

In the remaining cases, where neither an infinity, nor a zero, nor **NaN** is involved, and the operands have the same sign or have different magnitudes, the sum is computed and rounded to the nearest representable value using IEEE

754-2008 round to nearest, ties to even mode. If the magnitude is too large to represent, the operation overflows and the result is then an infinity of appropriate sign. The ECMAScript language requires support of gradual underflow as defined by IEEE 754-2008.

NOTE

The `-` operator performs subtraction when applied to two operands of numeric type, producing the difference of its operands; the left operand is the minuend and the right operand is the subtrahend. Given numeric operands **a** and **b**, it is always the case that **a - b** produces the same result as **a + (-b)**.

12.9 Bitwise Shift Operators

Syntax

```
ShiftExpression[Yield, Await] :  
    AdditiveExpression[?Yield, ?Await]  
    ShiftExpression[?Yield, ?Await] << AdditiveExpression[?Yield, ?Await]  
    ShiftExpression[?Yield, ?Await] >> AdditiveExpression[?Yield, ?Await]  
    ShiftExpression[?Yield, ?Await] >>> AdditiveExpression[?Yield, ?Await]
```

12.9.1 Static Semantics: IsFunctionDefinition

```
ShiftExpression :  
    ShiftExpression << AdditiveExpression  
    ShiftExpression >> AdditiveExpression  
    ShiftExpression >>> AdditiveExpression
```

1. Return **false**.

12.9.2 Static Semantics: AssignmentTargetType

```
ShiftExpression :  
    ShiftExpression << AdditiveExpression  
    ShiftExpression >> AdditiveExpression  
    ShiftExpression >>> AdditiveExpression
```

1. Return **invalid**.

12.9.3 The Left Shift Operator (<<)

NOTE

Performs a bitwise left shift operation on the left operand by the amount specified by the right operand.

12.9.3.1 Runtime Semantics: Evaluation

ShiftExpression : *ShiftExpression* << *AdditiveExpression*

1. Let *lref* be the result of evaluating *ShiftExpression*.

2. Let *lval* be ? *GetValue(lref)*.
3. Let *rref* be the result of evaluating *AdditiveExpression*.
4. Let *rval* be ? *GetValue(rref)*.
5. Let *lnum* be ? *ToInt32(lval)*.
6. Let *rnum* be ? *ToUInt32(rval)*.
7. Let *shiftCount* be the result of masking out all but the least significant 5 bits of *rnum*, that is, compute *rnum* & 0x1F.
8. Return the result of left shifting *lnum* by *shiftCount* bits. The result is a signed 32-bit integer.

12.9.4 The Signed Right Shift Operator (>>)

NOTE

Performs a sign-filling bitwise right shift operation on the left operand by the amount specified by the right operand.

12.9.4.1 Runtime Semantics: Evaluation

ShiftExpression : *ShiftExpression* >> *AdditiveExpression*

1. Let *lref* be the result of evaluating *ShiftExpression*.
2. Let *lval* be ? *GetValue(lref)*.
3. Let *rref* be the result of evaluating *AdditiveExpression*.
4. Let *rval* be ? *GetValue(rref)*.
5. Let *lnum* be ? *ToInt32(lval)*.
6. Let *rnum* be ? *ToUInt32(rval)*.
7. Let *shiftCount* be the result of masking out all but the least significant 5 bits of *rnum*, that is, compute *rnum* & 0x1F.
8. Return the result of performing a sign-extending right shift of *lnum* by *shiftCount* bits. The most significant bit is propagated. The result is a signed 32-bit integer.

12.9.5 The Unsigned Right Shift Operator (>>>)

NOTE

Performs a zero-filling bitwise right shift operation on the left operand by the amount specified by the right operand.

12.9.5.1 Runtime Semantics: Evaluation

ShiftExpression : *ShiftExpression* >>> *AdditiveExpression*

1. Let *lref* be the result of evaluating *ShiftExpression*.
2. Let *lval* be ? *GetValue(lref)*.
3. Let *rref* be the result of evaluating *AdditiveExpression*.
4. Let *rval* be ? *GetValue(rref)*.
5. Let *lnum* be ? *ToUInt32(lval)*.
6. Let *rnum* be ? *ToUInt32(rval)*.
7. Let *shiftCount* be the result of masking out all but the least significant 5 bits of *rnum*, that is, compute *rnum* & 0x1F.
8. Return the result of performing a zero-filling right shift of *lnum* by *shiftCount* bits. Vacated bits are filled with zero. The result is an unsigned 32-bit integer.

12.10 Relational Operators

NOTE 1

The result of evaluating a relational operator is always of type Boolean, reflecting whether the relationship named by the operator holds between its two operands.

Syntax

```
RelationalExpression[In, Yield, Await] :  
    ShiftExpression[?Yield, ?Await]  
    RelationalExpression[?In, ?Yield, ?Await] < ShiftExpression[?Yield, ?Await]  
    RelationalExpression[?In, ?Yield, ?Await] > ShiftExpression[?Yield, ?Await]  
    RelationalExpression[?In, ?Yield, ?Await] <= ShiftExpression[?Yield, ?Await]  
    RelationalExpression[?In, ?Yield, ?Await] >= ShiftExpression[?Yield, ?Await]  
    RelationalExpression[?In, ?Yield, ?Await] instanceof ShiftExpression[?Yield, ?Await]  
    [+In] RelationalExpression[+In, ?Yield, ?Await] in ShiftExpression[?Yield, ?Await]
```

NOTE 2

The _[In] grammar parameter is needed to avoid confusing the **in** operator in a relational expression with the **in** operator in a **for** statement.

12.10.1 Static Semantics: IsFunctionDefinition

```
RelationalExpression :  
    RelationalExpression < ShiftExpression  
    RelationalExpression > ShiftExpression  
    RelationalExpression <= ShiftExpression  
    RelationalExpression >= ShiftExpression  
    RelationalExpression instanceof ShiftExpression  
    RelationalExpression in ShiftExpression
```

1. Return **false**.

12.10.2 Static Semantics: AssignmentTargetType

```
RelationalExpression :  
    RelationalExpression < ShiftExpression  
    RelationalExpression > ShiftExpression  
    RelationalExpression <= ShiftExpression  
    RelationalExpression >= ShiftExpression  
    RelationalExpression instanceof ShiftExpression  
    RelationalExpression in ShiftExpression
```

1. Return invalid.

12.10.3 Runtime Semantics: Evaluation

RelationalExpression : *RelationalExpression* < *ShiftExpression*

1. Let *lref* be the result of evaluating *RelationalExpression*.
2. Let *lval* be ? *GetValue(lref)*.
3. Let *rref* be the result of evaluating *ShiftExpression*.
4. Let *rval* be ? *GetValue(rref)*.
5. Let *r* be the result of performing *Abstract Relational Comparison lval < rval*.
6. *ReturnIfAbrupt(r)*.
7. If *r* is **undefined**, return **false**. Otherwise, return *r*.

RelationalExpression : *RelationalExpression* > *ShiftExpression*

1. Let *lref* be the result of evaluating *RelationalExpression*.
2. Let *lval* be ? *GetValue(lref)*.
3. Let *rref* be the result of evaluating *ShiftExpression*.
4. Let *rval* be ? *GetValue(rref)*.
5. Let *r* be the result of performing *Abstract Relational Comparison rval < lval* with *LeftFirst* equal to **false**.
6. *ReturnIfAbrupt(r)*.
7. If *r* is **undefined**, return **false**. Otherwise, return *r*.

RelationalExpression : *RelationalExpression* <= *ShiftExpression*

1. Let *lref* be the result of evaluating *RelationalExpression*.
2. Let *lval* be ? *GetValue(lref)*.
3. Let *rref* be the result of evaluating *ShiftExpression*.
4. Let *rval* be ? *GetValue(rref)*.
5. Let *r* be the result of performing *Abstract Relational Comparison rval < lval* with *LeftFirst* equal to **false**.
6. *ReturnIfAbrupt(r)*.
7. If *r* is **true** or **undefined**, return **false**. Otherwise, return **true**.

RelationalExpression : *RelationalExpression* >= *ShiftExpression*

1. Let *lref* be the result of evaluating *RelationalExpression*.
2. Let *lval* be ? *GetValue(lref)*.
3. Let *rref* be the result of evaluating *ShiftExpression*.
4. Let *rval* be ? *GetValue(rref)*.
5. Let *r* be the result of performing *Abstract Relational Comparison lval < rval*.
6. *ReturnIfAbrupt(r)*.
7. If *r* is **true** or **undefined**, return **false**. Otherwise, return **true**.

RelationalExpression : *RelationalExpression* **instanceof** *ShiftExpression*

1. Let *lref* be the result of evaluating *RelationalExpression*.
2. Let *lval* be ? *GetValue(lref)*.
3. Let *rref* be the result of evaluating *ShiftExpression*.
4. Let *rval* be ? *GetValue(rref)*.
5. Return ? *InstanceofOperator(lval, rval)*.

RelationalExpression : *RelationalExpression* **in** *ShiftExpression*

1. Let *lref* be the result of evaluating *RelationalExpression*.
2. Let *lval* be ? *GetValue(lref)*.

3. Let *rref* be the result of evaluating *ShiftExpression*.
4. Let *rval* be ? *GetValue*(*rref*).
5. If *Type*(*rval*) is not Object, throw a **TypeError** exception.
6. Return ? *HasProperty*(*rval*, *ToPropertyKey*(*lval*)).

12.10.4 Runtime Semantics: InstanceofOperator (*V*, *target*)

The abstract operation *InstanceofOperator*(*V*, *target*) implements the generic algorithm for determining if ECMAScript value *V* is an instance of object *target* either by consulting *target*'s @@hasInstance method or, if absent, determining whether the value of *target*'s **prototype** property is present in *V*'s prototype chain. This abstract operation performs the following steps:

1. If *Type*(*target*) is not Object, throw a **TypeError** exception.
2. Let *instOfHandler* be ? *GetMethod*(*target*, @@hasInstance).
3. If *instOfHandler* is not **undefined**, then
 - a. Return *ToBoolean*(? *Call*(*instOfHandler*, *target*, « *V* »)).
4. If *IsCallable*(*target*) is **false**, throw a **TypeError** exception.
5. Return ? *OrdinaryHasInstance*(*target*, *V*).

NOTE

Steps 4 and 5 provide compatibility with previous editions of ECMAScript that did not use a @@hasInstance method to define the **instanceof** operator semantics. If an object does not define or inherit @@hasInstance it uses the default **instanceof** semantics.

12.11 Equality Operators

NOTE

The result of evaluating an equality operator is always of type Boolean, reflecting whether the relationship named by the operator holds between its two operands.

Syntax

```

EqualityExpression[In, Yield, Await] :
  RelationalExpression[?In, ?Yield, ?Await]
  EqualityExpression[?In, ?Yield, ?Await] == RelationalExpression[?In, ?Yield, ?Await]
  EqualityExpression[?In, ?Yield, ?Await] != RelationalExpression[?In, ?Yield, ?Await]
  EqualityExpression[?In, ?Yield, ?Await] === RelationalExpression[?In, ?Yield, ?Await]
  EqualityExpression[?In, ?Yield, ?Await] !== RelationalExpression[?In, ?Yield, ?Await]

```

12.11.1 Static Semantics: IsFunctionDefinition

```

EqualityExpression :
  EqualityExpression == RelationalExpression
  EqualityExpression != RelationalExpression
  EqualityExpression === RelationalExpression
  EqualityExpression !== RelationalExpression

```

1. Return **false**.

12.11.2 Static Semantics: AssignmentTargetType

EqualityExpression :

EqualityExpression == *RelationalExpression*
EqualityExpression != *RelationalExpression*
EqualityExpression === *RelationalExpression*
EqualityExpression !== *RelationalExpression*

1. Return invalid.

12.11.3 Runtime Semantics: Evaluation

EqualityExpression : *EqualityExpression* == *RelationalExpression*

1. Let *lref* be the result of evaluating *EqualityExpression*.
2. Let *lval* be ? *GetValue*(*lref*).
3. Let *rref* be the result of evaluating *RelationalExpression*.
4. Let *rval* be ? *GetValue*(*rref*).
5. Return the result of performing *Abstract Equality Comparison* *rval* == *lval*.

EqualityExpression : *EqualityExpression* != *RelationalExpression*

1. Let *lref* be the result of evaluating *EqualityExpression*.
2. Let *lval* be ? *GetValue*(*lref*).
3. Let *rref* be the result of evaluating *RelationalExpression*.
4. Let *rval* be ? *GetValue*(*rref*).
5. Let *r* be the result of performing *Abstract Equality Comparison* *rval* == *lval*.
6. If *r* is **true**, return **false**. Otherwise, return **true**.

EqualityExpression : *EqualityExpression* === *RelationalExpression*

1. Let *lref* be the result of evaluating *EqualityExpression*.
2. Let *lval* be ? *GetValue*(*lref*).
3. Let *rref* be the result of evaluating *RelationalExpression*.
4. Let *rval* be ? *GetValue*(*rref*).
5. Return the result of performing *Strict Equality Comparison* *rval* === *lval*.

EqualityExpression : *EqualityExpression* !== *RelationalExpression*

1. Let *lref* be the result of evaluating *EqualityExpression*.
2. Let *lval* be ? *GetValue*(*lref*).
3. Let *rref* be the result of evaluating *RelationalExpression*.
4. Let *rval* be ? *GetValue*(*rref*).
5. Let *r* be the result of performing *Strict Equality Comparison* *rval* === *lval*.
6. If *r* is **true**, return **false**. Otherwise, return **true**.

NOTE 1

Given the above definition of equality:

String comparison can be forced by: `" " + a == " " + b`.

Numeric comparison can be forced by: `+a == +b`.

Boolean comparison can be forced by: `!a == !b`.

NOTE 2

The equality operators maintain the following invariants:

A **!=** **B** is equivalent to **!(A == B)**.

A **==** **B** is equivalent to **B == A**, except in the order of evaluation of **A** and **B**.

NOTE 3

The equality operator is not always transitive. For example, there might be two distinct String objects, each representing the same String value; each String object would be considered equal to the String value by the `==` operator, but the two String objects would not be equal to each other. For example:

`new String("a") == "a"` and `"a" == new String("a")` are both **true**.

`new String("a") == new String("a")` is **false**.

NOTE 4

Comparison of Strings uses a simple equality test on sequences of code unit values. There is no attempt to use the more complex, semantically oriented definitions of character or string equality and collating order defined in the Unicode specification. Therefore Strings values that are canonically equal according to the Unicode standard could test as unequal. In effect this algorithm assumes that both Strings are already in normalized form.

12.12 Binary Bitwise Operators

Syntax

*BitwiseANDExpression*_[In, Yield, Await] :

*EqualityExpression*_[?In, ?Yield, ?Await]

*BitwiseANDExpression*_[?In, ?Yield, ?Await] **&** *EqualityExpression*_[?In, ?Yield, ?Await]

*BitwiseXORExpression*_[In, Yield, Await] :

*BitwiseANDExpression*_[?In, ?Yield, ?Await]

*BitwiseXORExpression*_[?In, ?Yield, ?Await] **^** *BitwiseANDExpression*_[?In, ?Yield, ?Await]

*BitwiseORExpression*_[In, Yield, Await] :

*BitwiseXORExpression*_[?In, ?Yield, ?Await]

*BitwiseORExpression*_[?In, ?Yield, ?Await] **|** *BitwiseXORExpression*_[?In, ?Yield, ?Await]

12.12.1 Static Semantics: IsFunctionDefinition

BitwiseANDExpression : *BitwiseANDExpression* **&** *EqualityExpression*

BitwiseXORExpression : *BitwiseXORExpression* **^** *BitwiseANDExpression*

BitwiseORExpression : *BitwiseORExpression* **|** *BitwiseXORExpression*

1. Return **false**.

12.12.2 Static Semantics: AssignmentTargetType

BitwiseANDExpression : *BitwiseANDExpression* & *EqualityExpression*

BitwiseXORExpression : *BitwiseXORExpression* ^ *BitwiseANDExpression*

BitwiseORExpression : *BitwiseORExpression* | *BitwiseXORExpression*

1. Return invalid.

12.12.3 Runtime Semantics: Evaluation

The production $A : A @ B$, where @ is one of the bitwise operators in the productions above, is evaluated as follows:

1. Let *lref* be the result of evaluating *A*.
2. Let *lval* be ? *GetValue*(*lref*).
3. Let *rref* be the result of evaluating *B*.
4. Let *rval* be ? *GetValue*(*rref*).
5. Let *lnum* be ? *ToInt32*(*lval*).
6. Let *rnum* be ? *ToInt32*(*rval*).
7. Return the result of applying the bitwise operator @ to *lnum* and *rnum*. The result is a signed 32-bit integer.

12.13 Binary Logical Operators

Syntax

*LogicalANDExpression*_[In, Yield, Await] :

*BitwiseORExpression*_[?In, ?Yield, ?Await]

*LogicalANDExpression*_[?In, ?Yield, ?Await] && *BitwiseORExpression*_[?In, ?Yield, ?Await]

*LogicalORExpression*_[In, Yield, Await] :

*LogicalANDExpression*_[?In, ?Yield, ?Await]

*LogicalORExpression*_[?In, ?Yield, ?Await] || *LogicalANDExpression*_[?In, ?Yield, ?Await]

NOTE

The value produced by a && or || operator is not necessarily of type Boolean. The value produced will always be the value of one of the two operand expressions.

12.13.1 Static Semantics: IsFunctionDefinition

LogicalANDExpression : *LogicalANDExpression* && *BitwiseORExpression*

LogicalORExpression : *LogicalORExpression* || *LogicalANDExpression*

1. Return **false**.

12.13.2 Static Semantics: AssignmentTargetType

LogicalANDExpression : *LogicalANDExpression* && *BitwiseORExpression*

LogicalORExpression : *LogicalORExpression* || *LogicalANDExpression*

1. Return invalid.

12.13.3 Runtime Semantics: Evaluation

LogicalANDExpression : *LogicalANDExpression* && *BitwiseORExpression*

1. Let *lref* be the result of evaluating *LogicalANDExpression*.
2. Let *lval* be ? *GetValue*(*lref*).
3. Let *lbool* be *ToBoolean*(*lval*).
4. If *lbool* is **false**, return *lval*.
5. Let *rref* be the result of evaluating *BitwiseORExpression*.
6. Return ? *GetValue*(*rref*).

LogicalORExpression : *LogicalORExpression* || *LogicalANDExpression*

1. Let *lref* be the result of evaluating *LogicalORExpression*.
2. Let *lval* be ? *GetValue*(*lref*).
3. Let *lbool* be *ToBoolean*(*lval*).
4. If *lbool* is **true**, return *lval*.
5. Let *rref* be the result of evaluating *LogicalANDExpression*.
6. Return ? *GetValue*(*rref*).

12.14 Conditional Operator (? :)

Syntax

*ConditionalExpression*_[In, Yield, Await] :

*LogicalORExpression*_[?In, ?Yield, ?Await]

*LogicalORExpression*_[?In, ?Yield, ?Await] ? *AssignmentExpression*_[+In, ?Yield, ?Await] :

*AssignmentExpression*_[?In, ?Yield, ?Await]

NOTE

The grammar for a *ConditionalExpression* in ECMAScript is slightly different from that in C and Java, which each allow the second subexpression to be an *Expression* but restrict the third expression to be a *ConditionalExpression*. The motivation for this difference in ECMAScript is to allow an assignment expression to be governed by either arm of a conditional and to eliminate the confusing and fairly useless case of a comma expression as the centre expression.

12.14.1 Static Semantics: IsFunctionDefinition

ConditionalExpression : *LogicalORExpression* ? *AssignmentExpression* : *AssignmentExpression*

1. Return **false**.

12.14.2 Static Semantics: AssignmentTargetType

ConditionalExpression : *LogicalORExpression* ? *AssignmentExpression* : *AssignmentExpression*

1. Return invalid.

12.14.3 Runtime Semantics: Evaluation

ConditionalExpression : *LogicalORExpression* ? *AssignmentExpression* : *AssignmentExpression*

1. Let *lref* be the result of evaluating *LogicalORExpression*.
2. Let *lval* be *ToBoolean*(? *GetValue*(*lref*)).
3. If *lval* is **true**, then
 - a. Let *trueRef* be the result of evaluating the first *AssignmentExpression*.
 - b. Return ? *GetValue*(*trueRef*).
4. Else,
 - a. Let *falseRef* be the result of evaluating the second *AssignmentExpression*.
 - b. Return ? *GetValue*(*falseRef*).

12.15 Assignment Operators

Syntax

*AssignmentExpression*_[In, Yield, Await] :

*ConditionalExpression*_[?In, ?Yield, ?Await]

[+Yield] *YieldExpression*_[?In, ?Await]

*ArrowFunction*_[?In, ?Yield, ?Await]

*AsyncArrowFunction*_[?In, ?Yield, ?Await]

*LeftHandSideExpression*_[?Yield, ?Await] = *AssignmentExpression*_[?In, ?Yield, ?Await]

*LeftHandSideExpression*_[?Yield, ?Await] *AssignmentOperator*

*AssignmentExpression*_[?In, ?Yield, ?Await]

AssignmentOperator : **one of**

***= /= %= += -= <=> >>= &= ^= |= **=**

12.15.1 Static Semantics: Early Errors

AssignmentExpression : *LeftHandSideExpression* = *AssignmentExpression*

It is a Syntax Error if *LeftHandSideExpression* is either an *ObjectLiteral* or an *ArrayLiteral* and *LeftHandSideExpression* is not **covering** an *AssignmentPattern*.

It is an early **Reference** Error if *LeftHandSideExpression* is neither an *ObjectLiteral* nor an *ArrayLiteral* and *AssignmentTargetType* of *LeftHandSideExpression* is **invalid**.

It is an early Syntax Error if *LeftHandSideExpression* is neither an *ObjectLiteral* nor an *ArrayLiteral* and *AssignmentTargetType* of *LeftHandSideExpression* is **strict**.

AssignmentExpression : *LeftHandSideExpression* *AssignmentOperator* *AssignmentExpression*

It is an early **Reference** Error if *AssignmentTargetType* of *LeftHandSideExpression* is **invalid**.

It is an early Syntax Error if *AssignmentTargetType* of *LeftHandSideExpression* is **strict**.

12.15.2 Static Semantics: IsFunctionDefinition

AssignmentExpression :

ArrowFunction

AsyncArrowFunction

1. Return **true**.

AssignmentExpression :

YieldExpression

LeftHandSideExpression = *AssignmentExpression*

LeftHandSideExpression *AssignmentOperator* *AssignmentExpression*

1. Return **false**.

12.15.3 Static Semantics: AssignmentTargetType

AssignmentExpression :

YieldExpression

ArrowFunction

AsyncArrowFunction

LeftHandSideExpression = *AssignmentExpression*

LeftHandSideExpression *AssignmentOperator* *AssignmentExpression*

1. Return invalid.

12.15.4 Runtime Semantics: Evaluation

AssignmentExpression : *LeftHandSideExpression* = *AssignmentExpression*

1. If *LeftHandSideExpression* is neither an *ObjectLiteral* nor an *ArrayLiteral*, then
 - a. Let *lref* be the result of evaluating *LeftHandSideExpression*.
 - b. **ReturnIfAbrupt**(*lref*).
 - c. If **IsAnonymousFunctionDefinition**(*AssignmentExpression*) and **IsIdentifierRef** of *LeftHandSideExpression* are both **true**, then
 - i. Let *rval* be the result of performing **NamedEvaluation** for *AssignmentExpression* with argument **GetReferencedName**(*lref*).
 - d. Else,
 - i. Let *rref* be the result of evaluating *AssignmentExpression*.
 - ii. Let *rval* be ? **GetValue**(*rref*).
 - e. Perform ? **PutValue**(*lref*, *rval*).
 - f. Return *rval*.
2. Let *assignmentPattern* be the *AssignmentPattern* that is **covered** by *LeftHandSideExpression*.
3. Let *rref* be the result of evaluating *AssignmentExpression*.
4. Let *rval* be ? **GetValue**(*rref*).
5. Perform ? **DestructuringAssignmentEvaluation** of *assignmentPattern* using *rval* as the argument.
6. Return *rval*.

AssignmentExpression : *LeftHandSideExpression* *AssignmentOperator* *AssignmentExpression*

1. Let *lref* be the result of evaluating *LeftHandSideExpression*.
2. Let *lval* be ? **GetValue**(*lref*).
3. Let *rref* be the result of evaluating *AssignmentExpression*.
4. Let *rval* be ? **GetValue**(*rref*).
5. Let *op* be the @ where *AssignmentOperator* is @=.
6. Let *r* be the result of applying *op* to *lval* and *rval* as if evaluating the expression *lval op rval*.
7. Perform ? **PutValue**(*lref*, *r*).
8. Return *r*.

NOTE

When an assignment occurs within [strict mode code](#), it is a runtime error if [lref](#) in step 1.f of the first algorithm or step 7 of the second algorithm it is an unresolvable reference. If it is, a **ReferenceError** exception is thrown. The *LeftHandSideExpression* also may not be a reference to a [data property](#) with the attribute value { [\[\[Writable\]\]](#): **false** }, to an [accessor property](#) with the attribute value { [\[\[Set\]\]](#): **undefined** }, nor to a non-existent property of an object for which the [IsExtensible](#) predicate returns the value **false**. In these cases a **TypeError** exception is thrown.

12.15.5 Destructuring Assignment

Supplemental Syntax

In certain circumstances when processing an instance of the production *AssignmentExpression* :

LeftHandSideExpression = *AssignmentExpression* the following grammar is used to refine the interpretation of *LeftHandSideExpression*.

*AssignmentPattern*_[Yield, Await] :

*ObjectAssignmentPattern*_[?Yield, ?Await]

*ArrayAssignmentPattern*_[?Yield, ?Await]

*ObjectAssignmentPattern*_[Yield, Await] :

{ }

{ *AssignmentRestProperty*_[?Yield, ?Await] }

{ *AssignmentPropertyList*_[?Yield, ?Await] }

{ *AssignmentPropertyList*_[?Yield, ?Await] , *AssignmentRestProperty*_[?Yield, ?Await] *opt* }

*ArrayAssignmentPattern*_[Yield, Await] :

[*Elision*_{opt} *AssignmentRestElement*_[?Yield, ?Await] *opt*]

[*AssignmentElementList*_[?Yield, ?Await]]

[*AssignmentElementList*_[?Yield, ?Await] , *Elision*_{opt}

*AssignmentRestElement*_[?Yield, ?Await] *opt*]

*AssignmentRestProperty*_[Yield, Await] :

... *DestructuringAssignmentTarget*_[?Yield, ?Await]

*AssignmentPropertyList*_[Yield, Await] :

*AssignmentProperty*_[?Yield, ?Await]

*AssignmentPropertyList*_[?Yield, ?Await] , *AssignmentProperty*_[?Yield, ?Await]

*AssignmentElementList*_[Yield, Await] :

*AssignmentElisionElement*_[?Yield, ?Await]

*AssignmentElementList*_[?Yield, ?Await] , *AssignmentElisionElement*_[?Yield, ?Await]

*AssignmentElisionElement*_[Yield, Await] :

*Elision*_{opt} *AssignmentElement*_[?Yield, ?Await]

*AssignmentProperty*_[Yield, Await] :

*IdentifierReference*_[?Yield, ?Await] *Initializer*_[+In, ?Yield, ?Await] *opt*

*PropertyName*_[?Yield, ?Await] : *AssignmentElement*_[?Yield, ?Await]

*AssignmentElement*_[Yield, Await] :

*DestructuringAssignmentTarget*_[?Yield, ?Await] *Initializer*_[+In, ?Yield, ?Await] *opt*

*AssignmentRestElement*_[Yield, Await] :

... *DestructuringAssignmentTarget*_[?Yield, ?Await]

*DestructuringAssignmentTarget*_[Yield, Await] :

*LeftHandSideExpression*_[?Yield, ?Await]

12.15.5.1 Static Semantics: Early Errors

AssignmentProperty : *IdentifierReference* *Initializer*

It is a Syntax Error if *AssignmentTargetType* of *IdentifierReference* is not **simple**.

AssignmentRestProperty : ... *DestructuringAssignmentTarget*

It is a Syntax Error if *DestructuringAssignmentTarget* is an *ArrayLiteral* or an *ObjectLiteral*.

DestructuringAssignmentTarget : *LeftHandSideExpression*

It is a Syntax Error if *LeftHandSideExpression* is either an *ObjectLiteral* or an *ArrayLiteral* and if *LeftHandSideExpression* is not **covering** an *AssignmentPattern*.

It is a Syntax Error if *LeftHandSideExpression* is neither an *ObjectLiteral* nor an *ArrayLiteral* and *AssignmentTargetType*(*LeftHandSideExpression*) is not **simple**.

12.15.5.2 Runtime Semantics: DestructuringAssignmentEvaluation

With parameter *value*.

ObjectAssignmentPattern : { }

1. Perform ? **RequireObjectCoercible**(*value*).
2. Return **NormalCompletion**(empty).

ObjectAssignmentPattern :

{ *AssignmentPropertyList* }
{ *AssignmentPropertyList* , }

1. Perform ? **RequireObjectCoercible**(*value*).
2. Perform ? **PropertyDestructuringAssignmentEvaluation** for *AssignmentPropertyList* using *value* as the argument.
3. Return **NormalCompletion**(empty).

ArrayAssignmentPattern : []

1. Let *iteratorRecord* be ? **GetIterator**(*value*).
2. Return ? **IteratorClose**(*iteratorRecord*, **NormalCompletion**(empty)).

ArrayAssignmentPattern : [*Elision*]

1. Let *iteratorRecord* be ? **GetIterator**(*value*).

2. Let *result* be the result of performing IteratorDestructuringAssignmentEvaluation of *Elision* with *iteratorRecord* as the argument.
3. If *iteratorRecord*.[[Done]] is **false**, return ? *IteratorClose*(*iteratorRecord*, *result*).
4. Return *result*.

ArrayAssignmentPattern : [*Elision AssignmentRestElement*]

1. Let *iteratorRecord* be ? *GetIterator*(*value*).
2. If *Elision* is present, then
 - a. Let *status* be the result of performing IteratorDestructuringAssignmentEvaluation of *Elision* with *iteratorRecord* as the argument.
 - b. If *status* is an abrupt completion, then
 - i. **Assert**: *iteratorRecord*.[[Done]] is **true**.
 - ii. Return *Completion*(*status*).
3. Let *result* be the result of performing IteratorDestructuringAssignmentEvaluation of *AssignmentRestElement* with *iteratorRecord* as the argument.
4. If *iteratorRecord*.[[Done]] is **false**, return ? *IteratorClose*(*iteratorRecord*, *result*).
5. Return *result*.

ArrayAssignmentPattern : [*AssignmentElementList*]

1. Let *iteratorRecord* be ? *GetIterator*(*value*).
2. Let *result* be the result of performing IteratorDestructuringAssignmentEvaluation of *AssignmentElementList* using *iteratorRecord* as the argument.
3. If *iteratorRecord*.[[Done]] is **false**, return ? *IteratorClose*(*iteratorRecord*, *result*).
4. Return *result*.

ArrayAssignmentPattern : [*AssignmentElementList* , *Elision AssignmentRestElement*]

1. Let *iteratorRecord* be ? *GetIterator*(*value*).
2. Let *status* be the result of performing IteratorDestructuringAssignmentEvaluation of *AssignmentElementList* using *iteratorRecord* as the argument.
3. If *status* is an abrupt completion, then
 - a. If *iteratorRecord*.[[Done]] is **false**, return ? *IteratorClose*(*iteratorRecord*, *status*).
 - b. Return *Completion*(*status*).
4. If *Elision* is present, then
 - a. Set *status* to the result of performing IteratorDestructuringAssignmentEvaluation of *Elision* with *iteratorRecord* as the argument.
 - b. If *status* is an abrupt completion, then
 - i. **Assert**: *iteratorRecord*.[[Done]] is **true**.
 - ii. Return *Completion*(*status*).
5. If *AssignmentRestElement* is present, then
 - a. Set *status* to the result of performing IteratorDestructuringAssignmentEvaluation of *AssignmentRestElement* with *iteratorRecord* as the argument.
6. If *iteratorRecord*.[[Done]] is **false**, return ? *IteratorClose*(*iteratorRecord*, *status*).
7. Return *Completion*(*status*).

ObjectAssignmentPattern : { *AssignmentRestProperty* }

1. Perform ? *RequireObjectCoercible*(*value*).
2. Let *excludedNames* be a new empty *List*.

3. Return the result of performing `RestDestructuringAssignmentEvaluation` of `AssignmentRestProperty` with *value* and *excludedNames* as the arguments.

ObjectAssignmentPattern : { *AssignmentPropertyList* , *AssignmentRestProperty* }

1. Perform ? `RequireObjectCoercible`(*value*).
2. Let *excludedNames* be the result of performing ? `PropertyDestructuringAssignmentEvaluation` for *AssignmentPropertyList* using *value* as the argument.
3. Return the result of performing `RestDestructuringAssignmentEvaluation` of *AssignmentRestProperty* with *value* and *excludedNames* as the arguments.

12.15.5.3 Runtime Semantics: `PropertyDestructuringAssignmentEvaluation`

With parameter *value*.

NOTE

The following operations collect a list of all destructured property names.

AssignmentPropertyList : *AssignmentPropertyList* , *AssignmentProperty*

1. Let *propertyNames* be the result of performing ? `PropertyDestructuringAssignmentEvaluation` for *AssignmentPropertyList* using *value* as the argument.
2. Let *nextNames* be the result of performing ? `PropertyDestructuringAssignmentEvaluation` for *AssignmentProperty* using *value* as the argument.
3. Append each item in *nextNames* to the end of *propertyNames*.
4. Return *propertyNames*.

AssignmentProperty : *IdentifierReference* *Initializer*

1. Let *P* be `StringValue` of *IdentifierReference*.
2. Let *lref* be ? `ResolveBinding`(*P*).
3. Let *v* be ? `GetV`(*value*, *P*).
4. If *Initializer*_{opt} is present and *v* is **undefined**, then
 - a. If `IsAnonymousFunctionDefinition`(*Initializer*) is **true**, then
 - i. Set *v* to the result of performing `NamedEvaluation` for *Initializer* with argument *P*.
 - b. Else,
 - i. Let *defaultValue* be the result of evaluating *Initializer*.
 - ii. Set *v* to ? `GetValue`(*defaultValue*).
5. Perform ? `PutValue`(*lref*, *v*).
6. Return a new `List` containing *P*.

AssignmentProperty : *PropertyName* : *AssignmentElement*

1. Let *name* be the result of evaluating *PropertyName*.
2. `ReturnIfAbrupt`(*name*).
3. Perform ? `KeyedDestructuringAssignmentEvaluation` of *AssignmentElement* with *value* and *name* as the arguments.
4. Return a new `List` containing *name*.

12.15.5.4 Runtime Semantics: `RestDestructuringAssignmentEvaluation`

With parameters *value* and *excludedNames*.

AssignmentRestProperty : . . . *DestructuringAssignmentTarget*

1. Let *lref* be the result of evaluating *DestructuringAssignmentTarget*.
2. **ReturnIfAbrupt**(*lref*).
3. Let *restObj* be **ObjectCreate**(%ObjectPrototype%).
4. Perform ? **CopyDataProperties**(*restObj*, *value*, *excludedNames*).
5. Return **PutValue**(*lref*, *restObj*).

12.15.5.5 Runtime Semantics: **IteratorDestructuringAssignmentEvaluation**

With parameter *iteratorRecord*.

AssignmentElementList : *AssignmentElisionElement*

1. Return the result of performing **IteratorDestructuringAssignmentEvaluation** of *AssignmentElisionElement* using *iteratorRecord* as the argument.

AssignmentElementList : *AssignmentElementList* , *AssignmentElisionElement*

1. Perform ? **IteratorDestructuringAssignmentEvaluation** of *AssignmentElementList* using *iteratorRecord* as the argument.
2. Return the result of performing **IteratorDestructuringAssignmentEvaluation** of *AssignmentElisionElement* using *iteratorRecord* as the argument.

AssignmentElisionElement : *AssignmentElement*

1. Return the result of performing **IteratorDestructuringAssignmentEvaluation** of *AssignmentElement* with *iteratorRecord* as the argument.

AssignmentElisionElement : *Elision* *AssignmentElement*

1. Perform ? **IteratorDestructuringAssignmentEvaluation** of *Elision* with *iteratorRecord* as the argument.
2. Return the result of performing **IteratorDestructuringAssignmentEvaluation** of *AssignmentElement* with *iteratorRecord* as the argument.

Elision : ,

1. If *iteratorRecord*.[[Done]] is **false**, then
 - a. Let *next* be **IteratorStep**(*iteratorRecord*).
 - b. If *next* is an **abrupt completion**, set *iteratorRecord*.[[Done]] to **true**.
 - c. **ReturnIfAbrupt**(*next*).
 - d. If *next* is **false**, set *iteratorRecord*.[[Done]] to **true**.
2. Return **NormalCompletion**(empty).

Elision : *Elision* ,

1. Perform ? **IteratorDestructuringAssignmentEvaluation** of *Elision* with *iteratorRecord* as the argument.
2. If *iteratorRecord*.[[Done]] is **false**, then
 - a. Let *next* be **IteratorStep**(*iteratorRecord*).
 - b. If *next* is an **abrupt completion**, set *iteratorRecord*.[[Done]] to **true**.
 - c. **ReturnIfAbrupt**(*next*).
 - d. If *next* is **false**, set *iteratorRecord*.[[Done]] to **true**.

3. Return **NormalCompletion**(empty).

AssignmentElement : *DestructuringAssignmentTarget* *Initializer*

1. If *DestructuringAssignmentTarget* is neither an *ObjectLiteral* nor an *ArrayLiteral*, then
 - a. Let *lref* be the result of evaluating *DestructuringAssignmentTarget*.
 - b. **ReturnIfAbrupt**(*lref*).
2. If *iteratorRecord*.[[Done]] is **false**, then
 - a. Let *next* be **IteratorStep**(*iteratorRecord*).
 - b. If *next* is an **abrupt completion**, set *iteratorRecord*.[[Done]] to **true**.
 - c. **ReturnIfAbrupt**(*next*).
 - d. If *next* is **false**, set *iteratorRecord*.[[Done]] to **true**.
 - e. Else,
 - i. Let *value* be **IteratorValue**(*next*).
 - ii. If *value* is an **abrupt completion**, set *iteratorRecord*.[[Done]] to **true**.
 - iii. **ReturnIfAbrupt**(*value*).
3. If *iteratorRecord*.[[Done]] is **true**, let *value* be **undefined**.
4. If *Initializer* is present and *value* is **undefined**, then
 - a. If **IsAnonymousFunctionDefinition**(*Initializer*) and **IsIdentifierRef** of *DestructuringAssignmentTarget* are both **true**, then
 - i. Let *v* be the result of performing **NamedEvaluation** for *Initializer* with argument **GetReferencedName**(*lref*).
 - b. Else,
 - i. Let *defaultValue* be the result of evaluating *Initializer*.
 - ii. Let *v* be ? **GetValue**(*defaultValue*).
5. Else, let *v* be *value*.
6. If *DestructuringAssignmentTarget* is an *ObjectLiteral* or an *ArrayLiteral*, then
 - a. Let *nestedAssignmentPattern* be the *AssignmentPattern* that is **covered** by *DestructuringAssignmentTarget*.
 - b. Return the result of performing **DestructuringAssignmentEvaluation** of *nestedAssignmentPattern* with *v* as the argument.
7. Return ? **PutValue**(*lref*, *v*).

NOTE

Left to right evaluation order is maintained by evaluating a *DestructuringAssignmentTarget* that is not a destructuring pattern prior to accessing the iterator or evaluating the *Initializer*.

AssignmentRestElement : . . . *DestructuringAssignmentTarget*

1. If *DestructuringAssignmentTarget* is neither an *ObjectLiteral* nor an *ArrayLiteral*, then
 - a. Let *lref* be the result of evaluating *DestructuringAssignmentTarget*.
 - b. **ReturnIfAbrupt**(*lref*).
2. Let *A* be ! **ArrayCreate**(0).
3. Let *n* be 0.
4. Repeat, while *iteratorRecord*.[[Done]] is **false**,
 - a. Let *next* be **IteratorStep**(*iteratorRecord*).
 - b. If *next* is an **abrupt completion**, set *iteratorRecord*.[[Done]] to **true**.
 - c. **ReturnIfAbrupt**(*next*).
 - d. If *next* is **false**, set *iteratorRecord*.[[Done]] to **true**.
 - e. Else,

- i. Let *nextValue* be *IteratorValue(next)*.
 - ii. If *nextValue* is an abrupt completion, set *iteratorRecord*.[[Done]] to **true**.
 - iii. *ReturnIfAbrupt(nextValue)*.
 - iv. Let *status* be *CreateDataProperty(A, ! ToString(n), nextValue)*.
 - v. *Assert: status* is **true**.
 - vi. Increment *n* by 1.
5. If *DestructuringAssignmentTarget* is neither an *ObjectLiteral* nor an *ArrayLiteral*, then
 - a. Return ? *PutValue(lref, A)*.
 6. Let *nestedAssignmentPattern* be the *AssignmentPattern* that is *covered* by *DestructuringAssignmentTarget*.
 7. Return the result of performing *DestructuringAssignmentEvaluation* of *nestedAssignmentPattern* with *A* as the argument.

12.15.5.6 Runtime Semantics: KeyedDestructuringAssignmentEvaluation

With parameters *value* and *propertyName*.

AssignmentElement : *DestructuringAssignmentTarget* *Initializer*

1. If *DestructuringAssignmentTarget* is neither an *ObjectLiteral* nor an *ArrayLiteral*, then
 - a. Let *lref* be the result of evaluating *DestructuringAssignmentTarget*.
 - b. *ReturnIfAbrupt(lref)*.
2. Let *v* be ? *GetV(value, propertyName)*.
3. If *Initializer* is present and *v* is **undefined**, then
 - a. If *IsAnonymousFunctionDefinition(Initializer)* and *IsIdentifierRef* of *DestructuringAssignmentTarget* are both **true**, then
 - i. Let *rhsValue* be the result of performing *NamedEvaluation* for *Initializer* with argument *GetReferencedName(lref)*.
 - b. Else,
 - i. Let *defaultValue* be the result of evaluating *Initializer*.
 - ii. Let *rhsValue* be ? *GetValue(defaultValue)*.
4. Else, let *rhsValue* be *v*.
5. If *DestructuringAssignmentTarget* is an *ObjectLiteral* or an *ArrayLiteral*, then
 - a. Let *assignmentPattern* be the *AssignmentPattern* that is *covered* by *DestructuringAssignmentTarget*.
 - b. Return the result of performing *DestructuringAssignmentEvaluation* of *assignmentPattern* with *rhsValue* as the argument.
6. Return ? *PutValue(lref, rhsValue)*.

12.16 Comma Operator (,)

Syntax

*Expression*_[In, Yield, Await] :

*AssignmentExpression*_[?In, ?Yield, ?Await]

*Expression*_[?In, ?Yield, ?Await] , *AssignmentExpression*_[?In, ?Yield, ?Await]

12.16.1 Static Semantics: IsFunctionDefinition

Expression : *Expression* , *AssignmentExpression*

1. Return **false**.

12.16.2 Static Semantics: AssignmentTargetType

Expression : *Expression* , *AssignmentExpression*

1. Return invalid.

12.16.3 Runtime Semantics: Evaluation

Expression : *Expression* , *AssignmentExpression*

1. Let *lref* be the result of evaluating *Expression*.
2. Perform ? **GetValue**(*lref*).
3. Let *rref* be the result of evaluating *AssignmentExpression*.
4. Return ? **GetValue**(*rref*).

NOTE

GetValue must be called even though its value is not used because it may have observable side-effects.

13 ECMAScript Language: Statements and Declarations

Syntax

```
Statement[Yield, Await, Return] :  
    BlockStatement[?Yield, ?Await, ?Return]  
    VariableStatement[?Yield, ?Await]  
    EmptyStatement  
    ExpressionStatement[?Yield, ?Await]  
    IfStatement[?Yield, ?Await, ?Return]  
    BreakableStatement[?Yield, ?Await, ?Return]  
    ContinueStatement[?Yield, ?Await]  
    BreakStatement[?Yield, ?Await]  
    [+Return] ReturnStatement[?Yield, ?Await]  
    WithStatement[?Yield, ?Await, ?Return]  
    LabelledStatement[?Yield, ?Await, ?Return]  
    ThrowStatement[?Yield, ?Await]  
    TryStatement[?Yield, ?Await, ?Return]  
    DebuggerStatement
```

```
Declaration[Yield, Await] :  
    HoistableDeclaration[?Yield, ?Await, ~Default]  
    ClassDeclaration[?Yield, ?Await, ~Default]  
    LexicalDeclaration[+In, ?Yield, ?Await]
```

*HoistableDeclaration*_[Yield, Await, Default] :
 *FunctionDeclaration*_[?Yield, ?Await, ?Default]
 *GeneratorDeclaration*_[?Yield, ?Await, ?Default]
 *AsyncFunctionDeclaration*_[?Yield, ?Await, ?Default]
 *AsyncGeneratorDeclaration*_[?Yield, ?Await, ?Default]

*BreakableStatement*_[Yield, Await, Return] :
 *IterationStatement*_[?Yield, ?Await, ?Return]
 *SwitchStatement*_[?Yield, ?Await, ?Return]

13.1 Statement Semantics

13.1.1 Static Semantics: ContainsDuplicateLabels

With parameter *labelSet*.

Statement :
 VariableStatement
 EmptyStatement
 ExpressionStatement
 ContinueStatement
 BreakStatement
 ReturnStatement
 ThrowStatement
 DebuggerStatement

1. Return **false**.

13.1.2 Static Semantics: ContainsUndefinedBreakTarget

With parameter *labelSet*.

Statement :
 VariableStatement
 EmptyStatement
 ExpressionStatement
 ContinueStatement
 ReturnStatement
 ThrowStatement
 DebuggerStatement

1. Return **false**.

13.1.3 Static Semantics: ContainsUndefinedContinueTarget

With parameters *iterationSet* and *labelSet*.

Statement :

VariableStatement
EmptyStatement
ExpressionStatement
BreakStatement
ReturnStatement
ThrowStatement
DebuggerStatement

1. Return **false**.

BreakableStatement : *IterationStatement*

1. Let *newIterationSet* be a copy of *iterationSet* with all the elements of *labelSet* appended.
2. Return *ContainsUndefinedContinueTarget* of *IterationStatement* with arguments *newIterationSet* and « ».

13.1.4 Static Semantics: DeclarationPart

HoistableDeclaration : *FunctionDeclaration*

1. Return *FunctionDeclaration*.

HoistableDeclaration : *GeneratorDeclaration*

1. Return *GeneratorDeclaration*.

HoistableDeclaration : *AsyncFunctionDeclaration*

1. Return *AsyncFunctionDeclaration*.

HoistableDeclaration : *AsyncGeneratorDeclaration*

1. Return *AsyncGeneratorDeclaration*.

Declaration : *ClassDeclaration*

1. Return *ClassDeclaration*.

Declaration : *LexicalDeclaration*

1. Return *LexicalDeclaration*.

13.1.5 Static Semantics: VarDeclaredNames

Statement :

EmptyStatement
ExpressionStatement
ContinueStatement
BreakStatement
ReturnStatement
ThrowStatement
DebuggerStatement

1. Return a new empty [List](#).

13.1.6 Static Semantics: VarScopedDeclarations

Statement :

EmptyStatement

ExpressionStatement

ContinueStatement

BreakStatement

ReturnStatement

ThrowStatement

DebuggerStatement

1. Return a new empty [List](#).

13.1.7 Runtime Semantics: LabelledEvaluation

With parameter *labelSet*.

BreakableStatement : *IterationStatement*

1. Let *stmtResult* be the result of performing LabelledEvaluation of *IterationStatement* with argument *labelSet*.
2. If *stmtResult*.[[Type]] is **break**, then
 - a. If *stmtResult*.[[Target]] is **empty**, then
 - i. If *stmtResult*.[[Value]] is **empty**, set *stmtResult* to [NormalCompletion\(undefined\)](#).
 - ii. Else, set *stmtResult* to [NormalCompletion\(stmtResult.\[\[Value\]\]\)](#).
3. Return [Completion\(stmtResult\)](#).

BreakableStatement : *SwitchStatement*

1. Let *stmtResult* be the result of evaluating *SwitchStatement*.
2. If *stmtResult*.[[Type]] is **break**, then
 - a. If *stmtResult*.[[Target]] is **empty**, then
 - i. If *stmtResult*.[[Value]] is **empty**, set *stmtResult* to [NormalCompletion\(undefined\)](#).
 - ii. Else, set *stmtResult* to [NormalCompletion\(stmtResult.\[\[Value\]\]\)](#).
3. Return [Completion\(stmtResult\)](#).

NOTE

A *BreakableStatement* is one that can be exited via an unlabelled *BreakStatement*.

13.1.8 Runtime Semantics: Evaluation

HoistableDeclaration :

GeneratorDeclaration

AsyncFunctionDeclaration

AsyncGeneratorDeclaration

1. Return [NormalCompletion\(empty\)](#).

HoistableDeclaration : *FunctionDeclaration*

1. Return the result of evaluating *FunctionDeclaration*.

BreakableStatement :
IterationStatement
SwitchStatement

1. Let *newLabelSet* be a new empty *List*.
2. Return the result of performing LabelledEvaluation of this *BreakableStatement* with argument *newLabelSet*.

13.2 Block

Syntax

*BlockStatement*_[Yield, Await, Return] :
*Block*_[?Yield, ?Await, ?Return]

*Block*_[Yield, Await, Return] :
 { *StatementList*_[?Yield, ?Await, ?Return] *opt* }

*StatementList*_[Yield, Await, Return] :
*StatementListItem*_[?Yield, ?Await, ?Return]
*StatementList*_[?Yield, ?Await, ?Return] *StatementListItem*_[?Yield, ?Await, ?Return]

*StatementListItem*_[Yield, Await, Return] :
*Statement*_[?Yield, ?Await, ?Return]
*Declaration*_[?Yield, ?Await]

13.2.1 Static Semantics: Early Errors

Block : { *StatementList* }

It is a Syntax Error if the LexicallyDeclaredNames of *StatementList* contains any duplicate entries.
 It is a Syntax Error if any element of the LexicallyDeclaredNames of *StatementList* also occurs in the VarDeclaredNames of *StatementList*.

13.2.2 Static Semantics: ContainsDuplicateLabels

With parameter *labelSet*.

Block : { }

1. Return **false**.

StatementList : *StatementList* *StatementListItem*

1. Let *hasDuplicates* be ContainsDuplicateLabels of *StatementList* with argument *labelSet*.
2. If *hasDuplicates* is **true**, return **true**.
3. Return ContainsDuplicateLabels of *StatementListItem* with argument *labelSet*.

StatementListItem : *Declaration*

1. Return **false**.

13.2.3 Static Semantics: ContainsUndefinedBreakTarget

With parameter *labelSet*.

Block : { }

1. Return **false**.

StatementList : *StatementList* *StatementListItem*

1. Let *hasUndefinedLabels* be ContainsUndefinedBreakTarget of *StatementList* with argument *labelSet*.
2. If *hasUndefinedLabels* is **true**, return **true**.
3. Return ContainsUndefinedBreakTarget of *StatementListItem* with argument *labelSet*.

StatementListItem : *Declaration*

1. Return **false**.

13.2.4 Static Semantics: ContainsUndefinedContinueTarget

With parameters *iterationSet* and *labelSet*.

Block : { }

1. Return **false**.

StatementList : *StatementList* *StatementListItem*

1. Let *hasUndefinedLabels* be ContainsUndefinedContinueTarget of *StatementList* with arguments *iterationSet* and « ».
2. If *hasUndefinedLabels* is **true**, return **true**.
3. Return ContainsUndefinedContinueTarget of *StatementListItem* with arguments *iterationSet* and « ».

StatementListItem : *Declaration*

1. Return **false**.

13.2.5 Static Semantics: LexicallyDeclaredNames

Block : { }

1. Return a new empty *List*.

StatementList : *StatementList* *StatementListItem*

1. Let *names* be LexicallyDeclaredNames of *StatementList*.
2. Append to *names* the elements of the LexicallyDeclaredNames of *StatementListItem*.
3. Return *names*.

StatementListItem : *Statement*

1. If *Statement* is *Statement* : *LabelledStatement* , return LexicallyDeclaredNames of *LabelledStatement*.
2. Return a new empty *List*.

StatementListItem : *Declaration*

1. Return the BoundNames of *Declaration*.

13.2.6 Static Semantics: LexicallyScopedDeclarations

StatementList : *StatementList* *StatementListItem*

1. Let *declarations* be LexicallyScopedDeclarations of *StatementList*.
2. Append to *declarations* the elements of the LexicallyScopedDeclarations of *StatementListItem*.
3. Return *declarations*.

StatementListItem : *Statement*

1. If *Statement* is *Statement* : *LabelledStatement* , return LexicallyScopedDeclarations of *LabelledStatement*.
2. Return a new empty *List*.

StatementListItem : *Declaration*

1. Return a new *List* containing DeclarationPart of *Declaration*.

13.2.7 Static Semantics: TopLevelLexicallyDeclaredNames

StatementList : *StatementList* *StatementListItem*

1. Let *names* be TopLevelLexicallyDeclaredNames of *StatementList*.
2. Append to *names* the elements of the TopLevelLexicallyDeclaredNames of *StatementListItem*.
3. Return *names*.

StatementListItem : *Statement*

1. Return a new empty *List*.

StatementListItem : *Declaration*

1. If *Declaration* is *Declaration* : *HoistableDeclaration* , then
 - a. Return « ».
2. Return the BoundNames of *Declaration*.

NOTE

At the top level of a function, or script, function declarations are treated like var declarations rather than like lexical declarations.

13.2.8 Static Semantics: TopLevelLexicallyScopedDeclarations

Block : { }

1. Return a new empty *List*.

StatementList : *StatementList* *StatementListItem*

1. Let *declarations* be TopLevelLexicallyScopedDeclarations of *StatementList*.
2. Append to *declarations* the elements of the TopLevelLexicallyScopedDeclarations of *StatementListItem*.
3. Return *declarations*.

StatementListItem : *Statement*

1. Return a new empty [List](#).

StatementListItem : *Declaration*

1. If *Declaration* is *Declaration* : *HoistableDeclaration* , then
 - a. Return « ».
2. Return a new [List](#) containing *Declaration*.

13.2.9 Static Semantics: TopLevelVarDeclaredNames

Block : { }

1. Return a new empty [List](#).

StatementList : *StatementList* *StatementListItem*

1. Let *names* be TopLevelVarDeclaredNames of *StatementList*.
2. Append to *names* the elements of the TopLevelVarDeclaredNames of *StatementListItem*.
3. Return *names*.

StatementListItem : *Declaration*

1. If *Declaration* is *Declaration* : *HoistableDeclaration* , then
 - a. Return the BoundNames of *HoistableDeclaration*.
2. Return a new empty [List](#).

StatementListItem : *Statement*

1. If *Statement* is *Statement* : *LabelledStatement* , return TopLevelVarDeclaredNames of *Statement*.
2. Return VarDeclaredNames of *Statement*.

NOTE

At the top level of a function or script, inner function declarations are treated like var declarations.

13.2.10 Static Semantics: TopLevelVarScopedDeclarations

Block : { }

1. Return a new empty [List](#).

StatementList : *StatementList* *StatementListItem*

1. Let *declarations* be TopLevelVarScopedDeclarations of *StatementList*.
2. Append to *declarations* the elements of the TopLevelVarScopedDeclarations of *StatementListItem*.
3. Return *declarations*.

StatementListItem : *Statement*

1. If *Statement* is *Statement* : *LabelledStatement* , return TopLevelVarScopedDeclarations of *Statement*.
2. Return VarScopedDeclarations of *Statement*.

StatementListItem : *Declaration*

1. If *Declaration* is *Declaration* : *HoistableDeclaration* , then
 - a. Let *declaration* be *DeclarationPart* of *HoistableDeclaration*.
 - b. Return « *declaration* ».
2. Return a new empty *List*.

13.2.11 Static Semantics: VarDeclaredNames

Block : { }

1. Return a new empty *List*.

StatementList : *StatementList* *StatementListItem*

1. Let *names* be *VarDeclaredNames* of *StatementList*.
2. Append to *names* the elements of the *VarDeclaredNames* of *StatementListItem*.
3. Return *names*.

StatementListItem : *Declaration*

1. Return a new empty *List*.

13.2.12 Static Semantics: VarScopedDeclarations

Block : { }

1. Return a new empty *List*.

StatementList : *StatementList* *StatementListItem*

1. Let *declarations* be *VarScopedDeclarations* of *StatementList*.
2. Append to *declarations* the elements of the *VarScopedDeclarations* of *StatementListItem*.
3. Return *declarations*.

StatementListItem : *Declaration*

1. Return a new empty *List*.

13.2.13 Runtime Semantics: Evaluation

Block : { }

1. Return *NormalCompletion*(empty).

Block : { *StatementList* }

1. Let *oldEnv* be the *running execution context*'s *LexicalEnvironment*.
2. Let *blockEnv* be *NewDeclarativeEnvironment*(*oldEnv*).
3. Perform *BlockDeclarationInstantiation*(*StatementList*, *blockEnv*).
4. Set the *running execution context*'s *LexicalEnvironment* to *blockEnv*.
5. Let *blockValue* be the result of evaluating *StatementList*.
6. Set the *running execution context*'s *LexicalEnvironment* to *oldEnv*.
7. Return *blockValue*.

NOTE 1

No matter how control leaves the *Block* the *LexicalEnvironment* is always restored to its former state.

StatementList : *StatementList* *StatementListItem*

1. Let *sl* be the result of evaluating *StatementList*.
2. **ReturnIfAbrupt**(*sl*).
3. Let *s* be the result of evaluating *StatementListItem*.
4. Return **Completion**(**UpdateEmpty**(*s*, *sl*)).

NOTE 2

The value of a *StatementList* is the value of the last value-producing item in the *StatementList*. For example, the following calls to the **eval** function all return the value 1:

```
eval("1;;;")
eval("1;{}")
eval("1;var a;")
```

13.2.14 Runtime Semantics: BlockDeclarationInstantiation (*code*, *env*)

NOTE

When a *Block* or *CaseBlock* is evaluated a new declarative **Environment Record** is created and bindings for each block scoped variable, constant, function, or class declared in the block are instantiated in the **Environment Record**.

BlockDeclarationInstantiation is performed as follows using arguments *code* and *env*. *code* is the **Parse Node** corresponding to the body of the block. *env* is the **Lexical Environment** in which bindings are to be created.

1. Let *envRec* be *env*'s **EnvironmentRecord**.
2. **Assert**: *envRec* is a declarative **Environment Record**.
3. Let *declarations* be the **LexicallyScopedDeclarations** of *code*.
4. For each element *d* in *declarations*, do
 - a. For each element *dn* of the **BoundNames** of *d*, do
 - i. If **IsConstantDeclaration** of *d* is **true**, then
 1. Perform ! *envRec*.**CreateImmutableBinding**(*dn*, **true**).
 - ii. Else,
 1. Perform ! *envRec*.**CreateMutableBinding**(*dn*, **false**).
 - b. If *d* is a *FunctionDeclaration*, a *GeneratorDeclaration*, an *AsyncFunctionDeclaration*, or an *AsyncGeneratorDeclaration*, then
 - i. Let *fn* be the sole element of the **BoundNames** of *d*.
 - ii. Let *fo* be the result of performing **InstantiateFunctionObject** for *d* with argument *env*.
 - iii. Perform *envRec*.**InitializeBinding**(*fn*, *fo*).

13.3 Declarations and the Variable Statement

13.3.1 Let and Const Declarations

NOTE

let and **const** declarations define variables that are scoped to the **running execution context**'s **LexicalEnvironment**.

The variables are created when their containing [Lexical Environment](#) is instantiated but may not be accessed in any way until the variable's *LexicalBinding* is evaluated. A variable defined by a *LexicalBinding* with an *Initializer* is assigned the value of its *Initializer*'s *AssignmentExpression* when the *LexicalBinding* is evaluated, not when the variable is created. If a *LexicalBinding* in a **let** declaration does not have an *Initializer* the variable is assigned the value **undefined** when the *LexicalBinding* is evaluated.

Syntax

```

LexicalDeclaration[In, Yield, Await] :
    LetOrConst BindingList[?In, ?Yield, ?Await] ;

LetOrConst :
    let
    const

BindingList[In, Yield, Await] :
    LexicalBinding[?In, ?Yield, ?Await]
    BindingList[?In, ?Yield, ?Await] , LexicalBinding[?In, ?Yield, ?Await]

LexicalBinding[In, Yield, Await] :
    BindingIdentifier[?Yield, ?Await] Initializer[?In, ?Yield, ?Await] opt
    BindingPattern[?Yield, ?Await] Initializer[?In, ?Yield, ?Await]

```

13.3.1.1 Static Semantics: Early Errors

LexicalDeclaration : *LetOrConst BindingList* ;

It is a Syntax Error if the BoundNames of *BindingList* contains "**let**".

It is a Syntax Error if the BoundNames of *BindingList* contains any duplicate entries.

LexicalBinding : *BindingIdentifier Initializer*

It is a Syntax Error if *Initializer* is not present and IsConstantDeclaration of the *LexicalDeclaration* containing this *LexicalBinding* is **true**.

13.3.1.2 Static Semantics: BoundNames

LexicalDeclaration : *LetOrConst BindingList* ;

1. Return the BoundNames of *BindingList*.

BindingList : *BindingList* , *LexicalBinding*

1. Let *names* be the BoundNames of *BindingList*.
2. Append to *names* the elements of the BoundNames of *LexicalBinding*.
3. Return *names*.

LexicalBinding : *BindingIdentifier Initializer*

1. Return the BoundNames of *BindingIdentifier*.

LexicalBinding : *BindingPattern Initializer*

1. Return the BoundNames of *BindingPattern*.

13.3.1.3 Static Semantics: IsConstantDeclaration

LexicalDeclaration : *LetOrConst BindingList* ;

1. Return IsConstantDeclaration of *LetOrConst*.

LetOrConst : **let**

1. Return **false**.

LetOrConst : **const**

1. Return **true**.

13.3.1.4 Runtime Semantics: Evaluation

LexicalDeclaration : *LetOrConst BindingList* ;

1. Let *next* be the result of evaluating *BindingList*.
2. **ReturnIfAbrupt**(*next*).
3. Return **NormalCompletion**(empty).

BindingList : *BindingList* , *LexicalBinding*

1. Let *next* be the result of evaluating *BindingList*.
2. **ReturnIfAbrupt**(*next*).
3. Return the result of evaluating *LexicalBinding*.

LexicalBinding : *BindingIdentifier*

1. Let *lhs* be **ResolveBinding**(StringValue of *BindingIdentifier*).
2. Return **InitializeReferencedBinding**(*lhs*, **undefined**).

NOTE

A **static semantics** rule ensures that this form of *LexicalBinding* never occurs in a **const** declaration.

LexicalBinding : *BindingIdentifier Initializer*

1. Let *bindingId* be StringValue of *BindingIdentifier*.
2. Let *lhs* be **ResolveBinding**(*bindingId*).
3. If **IsAnonymousFunctionDefinition**(*Initializer*) is **true**, then
 - a. Let *value* be the result of performing **NamedEvaluation** for *Initializer* with argument *bindingId*.
4. Else,
 - a. Let *rhs* be the result of evaluating *Initializer*.
 - b. Let *value* be ? **GetValue**(*rhs*).
5. Return **InitializeReferencedBinding**(*lhs*, *value*).

LexicalBinding : *BindingPattern Initializer*

1. Let *rhs* be the result of evaluating *Initializer*.
2. Let *value* be ? **GetValue**(*rhs*).
3. Let *env* be the **running execution context**'s **LexicalEnvironment**.

4. Return the result of performing BindingInitialization for *BindingPattern* using *value* and *env* as the arguments.

13.3.2 Variable Statement

NOTE

A **var** statement declares variables that are scoped to the [running execution context](#)'s VariableEnvironment. Var variables are created when their containing [Lexical Environment](#) is instantiated and are initialized to **undefined** when created. Within the scope of any VariableEnvironment a common *BindingIdentifier* may appear in more than one *VariableDeclaration* but those declarations collectively define only one variable. A variable defined by a *VariableDeclaration* with an *Initializer* is assigned the value of its *Initializer*'s *AssignmentExpression* when the *VariableDeclaration* is executed, not when the variable is created.

Syntax

```
VariableStatement[Yield, Await] :  
    var VariableDeclarationList[+In, ?Yield, ?Await] ;  
  
VariableDeclarationList[In, Yield, Await] :  
    VariableDeclaration[?In, ?Yield, ?Await]  
    VariableDeclarationList[?In, ?Yield, ?Await] , VariableDeclaration[?In, ?Yield, ?Await]  
  
VariableDeclaration[In, Yield, Await] :  
    BindingIdentifier[?Yield, ?Await] Initializer[?In, ?Yield, ?Await] opt  
    BindingPattern[?Yield, ?Await] Initializer[?In, ?Yield, ?Await]
```

13.3.2.1 Static Semantics: BoundNames

VariableDeclarationList : *VariableDeclarationList* , *VariableDeclaration*

1. Let *names* be BoundNames of *VariableDeclarationList*.
2. Append to *names* the elements of BoundNames of *VariableDeclaration*.
3. Return *names*.

VariableDeclaration : *BindingIdentifier* *Initializer*

1. Return the BoundNames of *BindingIdentifier*.

VariableDeclaration : *BindingPattern* *Initializer*

1. Return the BoundNames of *BindingPattern*.

13.3.2.2 Static Semantics: VarDeclaredNames

VariableStatement : **var** *VariableDeclarationList* ;

1. Return BoundNames of *VariableDeclarationList*.

13.3.2.3 Static Semantics: VarScopedDeclarations

VariableDeclarationList : *VariableDeclaration*

1. Return a new [List](#) containing *VariableDeclaration*.

VariableDeclarationList : *VariableDeclarationList* , *VariableDeclaration*

1. Let *declarations* be VarScopedDeclarations of *VariableDeclarationList*.
2. Append *VariableDeclaration* to *declarations*.
3. Return *declarations*.

13.3.2.4 Runtime Semantics: Evaluation

VariableStatement : **var** *VariableDeclarationList* ;

1. Let *next* be the result of evaluating *VariableDeclarationList*.
2. **ReturnIfAbrupt**(*next*).
3. Return **NormalCompletion**(empty).

VariableDeclarationList : *VariableDeclarationList* , *VariableDeclaration*

1. Let *next* be the result of evaluating *VariableDeclarationList*.
2. **ReturnIfAbrupt**(*next*).
3. Return the result of evaluating *VariableDeclaration*.

VariableDeclaration : *BindingIdentifier*

1. Return **NormalCompletion**(empty).

VariableDeclaration : *BindingIdentifier* *Initializer*

1. Let *bindingId* be StringValue of *BindingIdentifier*.
2. Let *lhs* be ? **ResolveBinding**(*bindingId*).
3. If **IsAnonymousFunctionDefinition**(*Initializer*) is **true**, then
 - a. Let *value* be the result of performing NamedEvaluation for *Initializer* with argument *bindingId*.
4. Else,
 - a. Let *rhs* be the result of evaluating *Initializer*.
 - b. Let *value* be ? **GetValue**(*rhs*).
5. Return ? **PutValue**(*lhs*, *value*).

NOTE

If a *VariableDeclaration* is nested within a with statement and the *BindingIdentifier* in the *VariableDeclaration* is the same as a **property name** of the binding object of the with statement's object **Environment Record**, then step 6 will assign *value* to the property instead of assigning to the VariableEnvironment binding of the *Identifier*.

VariableDeclaration : *BindingPattern* *Initializer*

1. Let *rhs* be the result of evaluating *Initializer*.
2. Let *rval* be ? **GetValue**(*rhs*).
3. Return the result of performing BindingInitialization for *BindingPattern* passing *rval* and **undefined** as arguments.

13.3.3 Destructuring Binding Patterns

Syntax

*BindingPattern*_[Yield, Await] :
 *ObjectBindingPattern*_[?Yield, ?Await]

*ArrayBindingPattern*_[?Yield, ?Await]

*ObjectBindingPattern*_[Yield, Await] :

```
{ }  
{ BindingRestProperty[?Yield, ?Await] }  
{ BindingPropertyList[?Yield, ?Await] }  
{ BindingPropertyList[?Yield, ?Await] , BindingRestProperty[?Yield, ?Await] opt }
```

*ArrayBindingPattern*_[Yield, Await] :

```
[ Elisionopt BindingRestElement[?Yield, ?Await] opt ]  
[ BindingElementList[?Yield, ?Await] ]  
[ BindingElementList[?Yield, ?Await] , Elisionopt BindingRestElement[?Yield, ?Await] opt ]
```

*BindingRestProperty*_[Yield, Await] :

... *BindingIdentifier*_[?Yield, ?Await]

*BindingPropertyList*_[Yield, Await] :

```
BindingProperty[?Yield, ?Await]  
BindingPropertyList[?Yield, ?Await] , BindingProperty[?Yield, ?Await]
```

*BindingElementList*_[Yield, Await] :

```
BindingElisionElement[?Yield, ?Await]  
BindingElementList[?Yield, ?Await] , BindingElisionElement[?Yield, ?Await]
```

*BindingElisionElement*_[Yield, Await] :

*Elision*_{*opt*} *BindingElement*_[?Yield, ?Await]

*BindingProperty*_[Yield, Await] :

```
SingleNameBinding[?Yield, ?Await]  
PropertyName[?Yield, ?Await] : BindingElement[?Yield, ?Await]
```

*BindingElement*_[Yield, Await] :

```
SingleNameBinding[?Yield, ?Await]  
BindingPattern[?Yield, ?Await] Initializer[+In, ?Yield, ?Await] opt
```

*SingleNameBinding*_[Yield, Await] :

*BindingIdentifier*_[?Yield, ?Await] *Initializer*_[+In, ?Yield, ?Await] *opt*

*BindingRestElement*_[Yield, Await] :

```
... BindingIdentifier[?Yield, ?Await]  
... BindingPattern[?Yield, ?Await]
```

13.3.3.1 Static Semantics: BoundNames

ObjectBindingPattern : { }

1. Return a new empty [List](#).

ArrayBindingPattern : [*Elision*]

1. Return a new empty [List](#).

ArrayBindingPattern : [*Elision BindingRestElement*]

1. Return the BoundNames of *BindingRestElement*.

ArrayBindingPattern : [*BindingElementList* , *Elision*]

1. Return the BoundNames of *BindingElementList*.

ArrayBindingPattern : [*BindingElementList* , *Elision BindingRestElement*]

1. Let *names* be BoundNames of *BindingElementList*.
2. Append to *names* the elements of BoundNames of *BindingRestElement*.
3. Return *names*.

BindingPropertyList : *BindingPropertyList* , *BindingProperty*

1. Let *names* be BoundNames of *BindingPropertyList*.
2. Append to *names* the elements of BoundNames of *BindingProperty*.
3. Return *names*.

BindingElementList : *BindingElementList* , *BindingElisionElement*

1. Let *names* be BoundNames of *BindingElementList*.
2. Append to *names* the elements of BoundNames of *BindingElisionElement*.
3. Return *names*.

BindingElisionElement : *Elision BindingElement*

1. Return BoundNames of *BindingElement*.

BindingProperty : *PropertyName* : *BindingElement*

1. Return the BoundNames of *BindingElement*.

SingleNameBinding : *BindingIdentifier* *Initializer*

1. Return the BoundNames of *BindingIdentifier*.

BindingElement : *BindingPattern* *Initializer*

1. Return the BoundNames of *BindingPattern*.

13.3.3.2 Static Semantics: ContainsExpression

ObjectBindingPattern : { }

1. Return **false**.

ArrayBindingPattern : [*Elision*]

1. Return **false**.

ArrayBindingPattern : [*Elision BindingRestElement*]

1. Return ContainsExpression of *BindingRestElement*.

ArrayBindingPattern : [*BindingElementList* , *Elision*]

1. Return ContainsExpression of *BindingElementList*.

ArrayBindingPattern : [*BindingElementList* , *Elision* *BindingRestElement*]

1. Let *has* be ContainsExpression of *BindingElementList*.
2. If *has* is **true**, return **true**.
3. Return ContainsExpression of *BindingRestElement*.

BindingPropertyList : *BindingPropertyList* , *BindingProperty*

1. Let *has* be ContainsExpression of *BindingPropertyList*.
2. If *has* is **true**, return **true**.
3. Return ContainsExpression of *BindingProperty*.

BindingElementList : *BindingElementList* , *BindingElisionElement*

1. Let *has* be ContainsExpression of *BindingElementList*.
2. If *has* is **true**, return **true**.
3. Return ContainsExpression of *BindingElisionElement*.

BindingElisionElement : *Elision* *BindingElement*

1. Return ContainsExpression of *BindingElement*.

BindingProperty : *PropertyName* : *BindingElement*

1. Let *has* be IsComputedPropertyKey of *PropertyName*.
2. If *has* is **true**, return **true**.
3. Return ContainsExpression of *BindingElement*.

BindingElement : *BindingPattern* *Initializer*

1. Return **true**.

SingleNameBinding : *BindingIdentifier*

1. Return **false**.

SingleNameBinding : *BindingIdentifier* *Initializer*

1. Return **true**.

BindingRestElement : . . . *BindingIdentifier*

1. Return **false**.

BindingRestElement : . . . *BindingPattern*

1. Return ContainsExpression of *BindingPattern*.

13.3.3.3 Static Semantics: HasInitializer

BindingElement : *BindingPattern*

1. Return **false**.

BindingElement : *BindingPattern* Initializer

1. Return **true**.

SingleNameBinding : *BindingIdentifier*

1. Return **false**.

SingleNameBinding : *BindingIdentifier* Initializer

1. Return **true**.

13.3.3.4 Static Semantics: **IsSimpleParameterList**

BindingElement : *BindingPattern*

1. Return **false**.

BindingElement : *BindingPattern* Initializer

1. Return **false**.

SingleNameBinding : *BindingIdentifier*

1. Return **true**.

SingleNameBinding : *BindingIdentifier* Initializer

1. Return **false**.

13.3.3.5 Runtime Semantics: **BindingInitialization**

With parameters *value* and *environment*.

NOTE

When **undefined** is passed for *environment* it indicates that a **PutValue** operation should be used to assign the initialization value. This is the case for formal parameter lists of non-strict functions. In that case the formal parameter bindings are preinitialized in order to deal with the possibility of multiple parameters with the same name.

BindingPattern : *ObjectBindingPattern*

1. Perform ? **RequireObjectCoercible**(*value*).
2. Return the result of performing **BindingInitialization** for *ObjectBindingPattern* using *value* and *environment* as arguments.

BindingPattern : *ArrayBindingPattern*

1. Let *iteratorRecord* be ? **GetIterator**(*value*).
2. Let *result* be **IteratorBindingInitialization** for *ArrayBindingPattern* using *iteratorRecord* and *environment* as arguments.
3. If *iteratorRecord*.[[Done]] is **false**, return ? **IteratorClose**(*iteratorRecord*, *result*).

4. Return *result*.

ObjectBindingPattern : { }

1. Return *NormalCompletion*(empty).

ObjectBindingPattern :

{ *BindingPropertyList* }

{ *BindingPropertyList* , }

1. Perform ? *PropertyBindingInitialization* for *BindingPropertyList* using *value* and *environment* as the arguments.
2. Return *NormalCompletion*(empty).

ObjectBindingPattern : { *BindingRestProperty* }

1. Let *excludedNames* be a new empty *List*.
2. Return the result of performing *RestBindingInitialization* of *BindingRestProperty* with *value*, *environment*, and *excludedNames* as the arguments.

ObjectBindingPattern : { *BindingPropertyList* , *BindingRestProperty* }

1. Let *excludedNames* be the result of performing ? *PropertyBindingInitialization* of *BindingPropertyList* using *value* and *environment* as arguments.
2. Return the result of performing *RestBindingInitialization* of *BindingRestProperty* with *value*, *environment*, and *excludedNames* as the arguments.

13.3.3.6 Runtime Semantics: *PropertyBindingInitialization*

With parameters *value* and *environment*.

NOTE

These collect a list of all bound property names rather than just empty completion.

BindingPropertyList : *BindingPropertyList* , *BindingProperty*

1. Let *boundNames* be the result of performing ? *PropertyBindingInitialization* for *BindingPropertyList* using *value* and *environment* as arguments.
2. Let *nextNames* be the result of performing ? *PropertyBindingInitialization* for *BindingProperty* using *value* and *environment* as arguments.
3. Append each item in *nextNames* to the end of *boundNames*.
4. Return *boundNames*.

BindingProperty : *SingleNameBinding*

1. Let *name* be the string that is the only element of *BoundNames* of *SingleNameBinding*.
2. Perform ? *KeyedBindingInitialization* for *SingleNameBinding* using *value*, *environment*, and *name* as the arguments.
3. Return a new *List* containing *name*.

BindingProperty : *PropertyName* : *BindingElement*

1. Let *P* be the result of evaluating *PropertyName*.
2. *ReturnIfAbrupt*(*P*).

3. Perform ? KeyedBindingInitialization of *BindingElement* with *value*, *environment*, and *P* as the arguments.
4. Return a new *List* containing *P*.

13.3.3.7 Runtime Semantics: RestBindingInitialization

With parameters *value*, *environment*, and *excludedNames*.

BindingRestProperty : . . . *BindingIdentifier*

1. Let *lhs* be ? *ResolveBinding*(StringValue of *BindingIdentifier*, *environment*).
2. Let *restObj* be *ObjectCreate*(%ObjectPrototype%).
3. Perform ? *CopyDataProperties*(*restObj*, *value*, *excludedNames*).
4. If *environment* is **undefined**, return *PutValue*(*lhs*, *restObj*).
5. Return *InitializeReferencedBinding*(*lhs*, *restObj*).

13.3.3.8 Runtime Semantics: IteratorBindingInitialization

With parameters *iteratorRecord* and *environment*.

NOTE

When **undefined** is passed for *environment* it indicates that a *PutValue* operation should be used to assign the initialization value. This is the case for formal parameter lists of non-strict functions. In that case the formal parameter bindings are preinitialized in order to deal with the possibility of multiple parameters with the same name.

ArrayBindingPattern : []

1. Return *NormalCompletion*(empty).

ArrayBindingPattern : [*Elision*]

1. Return the result of performing *IteratorDestructuringAssignmentEvaluation* of *Elision* with *iteratorRecord* as the argument.

ArrayBindingPattern : [*Elision* *BindingRestElement*]

1. If *Elision* is present, then
 - a. Perform ? *IteratorDestructuringAssignmentEvaluation* of *Elision* with *iteratorRecord* as the argument.
2. Return the result of performing *IteratorBindingInitialization* for *BindingRestElement* with *iteratorRecord* and *environment* as arguments.

ArrayBindingPattern : [*BindingElementList*]

1. Return the result of performing *IteratorBindingInitialization* for *BindingElementList* with *iteratorRecord* and *environment* as arguments.

ArrayBindingPattern : [*BindingElementList* ,]

1. Return the result of performing *IteratorBindingInitialization* for *BindingElementList* with *iteratorRecord* and *environment* as arguments.

ArrayBindingPattern : [*BindingElementList* , *Elision*]

1. Perform ? *IteratorBindingInitialization* for *BindingElementList* with *iteratorRecord* and *environment* as arguments.

2. Return the result of performing `IteratorDestructuringAssignmentEvaluation` of *Elision* with *iteratorRecord* as the argument.

ArrayBindingPattern : [*BindingElementList* , *Elision BindingRestElement*]

1. Perform ? `IteratorBindingInitialization` for *BindingElementList* with *iteratorRecord* and *environment* as arguments.
2. If *Elision* is present, then
 - a. Perform ? `IteratorDestructuringAssignmentEvaluation` of *Elision* with *iteratorRecord* as the argument.
3. Return the result of performing `IteratorBindingInitialization` for *BindingRestElement* with *iteratorRecord* and *environment* as arguments.

BindingElementList : *BindingElisionElement*

1. Return the result of performing `IteratorBindingInitialization` for *BindingElisionElement* with *iteratorRecord* and *environment* as arguments.

BindingElementList : *BindingElementList* , *BindingElisionElement*

1. Perform ? `IteratorBindingInitialization` for *BindingElementList* with *iteratorRecord* and *environment* as arguments.
2. Return the result of performing `IteratorBindingInitialization` for *BindingElisionElement* using *iteratorRecord* and *environment* as arguments.

BindingElisionElement : *BindingElement*

1. Return the result of performing `IteratorBindingInitialization` of *BindingElement* with *iteratorRecord* and *environment* as the arguments.

BindingElisionElement : *Elision BindingElement*

1. Perform ? `IteratorDestructuringAssignmentEvaluation` of *Elision* with *iteratorRecord* as the argument.
2. Return the result of performing `IteratorBindingInitialization` of *BindingElement* with *iteratorRecord* and *environment* as the arguments.

BindingElement : *SingleNameBinding*

1. Return the result of performing `IteratorBindingInitialization` for *SingleNameBinding* with *iteratorRecord* and *environment* as the arguments.

SingleNameBinding : *BindingIdentifier Initializer*

1. Let *bindingId* be `StringValue` of *BindingIdentifier*.
2. Let *lhs* be ? `ResolveBinding`(*bindingId*, *environment*).
3. If *iteratorRecord*.[[Done]] is **false**, then
 - a. Let *next* be `IteratorStep`(*iteratorRecord*).
 - b. If *next* is an **abrupt completion**, set *iteratorRecord*.[[Done]] to **true**.
 - c. `ReturnIfAbrupt`(*next*).
 - d. If *next* is **false**, set *iteratorRecord*.[[Done]] to **true**.
 - e. Else,
 - i. Let *v* be `IteratorValue`(*next*).
 - ii. If *v* is an **abrupt completion**, set *iteratorRecord*.[[Done]] to **true**.
 - iii. `ReturnIfAbrupt`(*v*).
4. If *iteratorRecord*.[[Done]] is **true**, let *v* be **undefined**.
5. If *Initializer* is present and *v* is **undefined**, then

- a. If **IsAnonymousFunctionDefinition**(*Initializer*) is **true**, then
 - i. Set *v* to the result of performing **NamedEvaluation** for *Initializer* with argument *bindingId*.
 - b. Else,
 - i. Let *defaultValue* be the result of evaluating *Initializer*.
 - ii. Set *v* to ? **GetValue**(*defaultValue*).
6. If *environment* is **undefined**, return ? **PutValue**(*lhs*, *v*).
 7. Return **InitializeReferencedBinding**(*lhs*, *v*).

BindingElement : *BindingPattern* *Initializer*

1. If *iteratorRecord*.[[Done]] is **false**, then
 - a. Let *next* be **IteratorStep**(*iteratorRecord*).
 - b. If *next* is an **abrupt completion**, set *iteratorRecord*.[[Done]] to **true**.
 - c. **ReturnIfAbrupt**(*next*).
 - d. If *next* is **false**, set *iteratorRecord*.[[Done]] to **true**.
 - e. Else,
 - i. Let *v* be **IteratorValue**(*next*).
 - ii. If *v* is an **abrupt completion**, set *iteratorRecord*.[[Done]] to **true**.
 - iii. **ReturnIfAbrupt**(*v*).
2. If *iteratorRecord*.[[Done]] is **true**, let *v* be **undefined**.
3. If *Initializer* is present and *v* is **undefined**, then
 - a. Let *defaultValue* be the result of evaluating *Initializer*.
 - b. Set *v* to ? **GetValue**(*defaultValue*).
4. Return the result of performing **BindingInitialization** of *BindingPattern* with *v* and *environment* as the arguments.

BindingRestElement : . . . *BindingIdentifier*

1. Let *lhs* be ? **ResolveBinding**(StringValue of *BindingIdentifier*, *environment*).
2. Let *A* be ! **ArrayCreate**(0).
3. Let *n* be 0.
4. Repeat,
 - a. If *iteratorRecord*.[[Done]] is **false**, then
 - i. Let *next* be **IteratorStep**(*iteratorRecord*).
 - ii. If *next* is an **abrupt completion**, set *iteratorRecord*.[[Done]] to **true**.
 - iii. **ReturnIfAbrupt**(*next*).
 - iv. If *next* is **false**, set *iteratorRecord*.[[Done]] to **true**.
 - b. If *iteratorRecord*.[[Done]] is **true**, then
 - i. If *environment* is **undefined**, return ? **PutValue**(*lhs*, *A*).
 - ii. Return **InitializeReferencedBinding**(*lhs*, *A*).
 - c. Let *nextValue* be **IteratorValue**(*next*).
 - d. If *nextValue* is an **abrupt completion**, set *iteratorRecord*.[[Done]] to **true**.
 - e. **ReturnIfAbrupt**(*nextValue*).
 - f. Let *status* be **CreateDataProperty**(*A*, ! **ToString**(*n*), *nextValue*).
 - g. **Assert**: *status* is **true**.
 - h. Increment *n* by 1.

BindingRestElement : . . . *BindingPattern*

1. Let *A* be ! **ArrayCreate**(0).
2. Let *n* be 0.

3. Repeat,
 - a. If *iteratorRecord*.[[Done]] is **false**, then
 - i. Let *next* be *IteratorStep*(*iteratorRecord*).
 - ii. If *next* is an abrupt completion, set *iteratorRecord*.[[Done]] to **true**.
 - iii. *ReturnIfAbrupt*(*next*).
 - iv. If *next* is **false**, set *iteratorRecord*.[[Done]] to **true**.
 - b. If *iteratorRecord*.[[Done]] is **true**, then
 - i. Return the result of performing *BindingInitialization* of *BindingPattern* with *A* and *environment* as the arguments.
 - c. Let *nextValue* be *IteratorValue*(*next*).
 - d. If *nextValue* is an abrupt completion, set *iteratorRecord*.[[Done]] to **true**.
 - e. *ReturnIfAbrupt*(*nextValue*).
 - f. Let *status* be *CreateDataProperty*(*A*, ! *ToString*(*n*), *nextValue*).
 - g. *Assert*: *status* is **true**.
 - h. Increment *n* by 1.

13.3.3.9 Runtime Semantics: KeyedBindingInitialization

With parameters *value*, *environment*, and *propertyName*.

NOTE

When **undefined** is passed for *environment* it indicates that a *PutValue* operation should be used to assign the initialization value. This is the case for formal parameter lists of non-strict functions. In that case the formal parameter bindings are preinitialized in order to deal with the possibility of multiple parameters with the same name.

BindingElement : *BindingPattern* *Initializer*

1. Let *v* be ? *GetV*(*value*, *propertyName*).
2. If *Initializer* is present and *v* is **undefined**, then
 - a. Let *defaultValue* be the result of evaluating *Initializer*.
 - b. Set *v* to ? *GetValue*(*defaultValue*).
3. Return the result of performing *BindingInitialization* for *BindingPattern* passing *v* and *environment* as arguments.

SingleNameBinding : *BindingIdentifier* *Initializer*

1. Let *bindingId* be *StringValue* of *BindingIdentifier*.
2. Let *lhs* be ? *ResolveBinding*(*bindingId*, *environment*).
3. Let *v* be ? *GetV*(*value*, *propertyName*).
4. If *Initializer* is present and *v* is **undefined**, then
 - a. If *IsAnonymousFunctionDefinition*(*Initializer*) is **true**, then
 - i. Set *v* to the result of performing *NamedEvaluation* for *Initializer* with argument *bindingId*.
 - b. Else,
 - i. Let *defaultValue* be the result of evaluating *Initializer*.
 - ii. Set *v* to ? *GetValue*(*defaultValue*).
5. If *environment* is **undefined**, return ? *PutValue*(*lhs*, *v*).
6. Return *InitializeReferencedBinding*(*lhs*, *v*).

13.4 Empty Statement

Syntax

EmptyStatement :
;

13.4.1 Runtime Semantics: Evaluation

EmptyStatement : ;

1. Return [NormalCompletion](#)(empty).

13.5 Expression Statement

Syntax

*ExpressionStatement*_[Yield, Await] :
[lookahead $\notin$ { { , **function** , **async** [no *LineTerminator* here] **function** , **class** , **let** [] }]
*Expression*_[+In, ?Yield, ?Await] ;

NOTE

An *ExpressionStatement* cannot start with a U+007B (LEFT CURLY BRACKET) because that might make it ambiguous with a *Block*. An *ExpressionStatement* cannot start with the **function** or **class** keywords because that would make it ambiguous with a *FunctionDeclaration*, a *GeneratorDeclaration*, or a *ClassDeclaration*. An *ExpressionStatement* cannot start with **async function** because that would make it ambiguous with an *AsyncFunctionDeclaration* or a *AsyncGeneratorDeclaration*. An *ExpressionStatement* cannot start with the two token sequence **let** [because that would make it ambiguous with a **let** *LexicalDeclaration* whose first *LexicalBinding* was an *ArrayBindingPattern*.

13.5.1 Runtime Semantics: Evaluation

ExpressionStatement : *Expression* ;

1. Let *exprRef* be the result of evaluating *Expression*.
2. Return ? [GetValue](#)(*exprRef*).

13.6 The if Statement

Syntax

*IfStatement*_[Yield, Await, Return] :
if (*Expression*_[+In, ?Yield, ?Await]) *Statement*_[?Yield, ?Await, ?Return] **else**
*Statement*_[?Yield, ?Await, ?Return]
if (*Expression*_[+In, ?Yield, ?Await]) *Statement*_[?Yield, ?Await, ?Return]

Each **else** for which the choice of associated **if** is ambiguous shall be associated with the nearest possible **if** that would otherwise have no corresponding **else**.

13.6.1 Static Semantics: Early Errors

IfStatement :

```
if ( Expression ) Statement else Statement  
if ( Expression ) Statement
```

It is a Syntax Error if `IsLabelledFunction(Statement)` is **true**.

NOTE

It is only necessary to apply this rule if the extension specified in B.3.2 is implemented.

13.6.2 Static Semantics: ContainsDuplicateLabels

With parameter *labelSet*.

IfStatement : **if** (*Expression*) *Statement* **else** *Statement*

1. Let *hasDuplicate* be `ContainsDuplicateLabels` of the first *Statement* with argument *labelSet*.
2. If *hasDuplicate* is **true**, return **true**.
3. Return `ContainsDuplicateLabels` of the second *Statement* with argument *labelSet*.

IfStatement : **if** (*Expression*) *Statement*

1. Return `ContainsDuplicateLabels` of *Statement* with argument *labelSet*.

13.6.3 Static Semantics: ContainsUndefinedBreakTarget

With parameter *labelSet*.

IfStatement : **if** (*Expression*) *Statement* **else** *Statement*

1. Let *hasUndefinedLabels* be `ContainsUndefinedBreakTarget` of the first *Statement* with argument *labelSet*.
2. If *hasUndefinedLabels* is **true**, return **true**.
3. Return `ContainsUndefinedBreakTarget` of the second *Statement* with argument *labelSet*.

IfStatement : **if** (*Expression*) *Statement*

1. Return `ContainsUndefinedBreakTarget` of *Statement* with argument *labelSet*.

13.6.4 Static Semantics: ContainsUndefinedContinueTarget

With parameters *iterationSet* and *labelSet*.

IfStatement : **if** (*Expression*) *Statement* **else** *Statement*

1. Let *hasUndefinedLabels* be `ContainsUndefinedContinueTarget` of the first *Statement* with arguments *iterationSet* and « ».
2. If *hasUndefinedLabels* is **true**, return **true**.
3. Return `ContainsUndefinedContinueTarget` of the second *Statement* with arguments *iterationSet* and « ».

IfStatement : **if** (*Expression*) *Statement*

1. Return `ContainsUndefinedContinueTarget` of *Statement* with arguments *iterationSet* and « ».

13.6.5 Static Semantics: VarDeclaredNames

IfStatement : **if** (*Expression*) *Statement* **else** *Statement*

1. Let *names* be `VarDeclaredNames` of the first *Statement*.
2. Append to *names* the elements of the `VarDeclaredNames` of the second *Statement*.
3. Return *names*.

IfStatement : **if** (*Expression*) *Statement*

1. Return the `VarDeclaredNames` of *Statement*.

13.6.6 Static Semantics: VarScopedDeclarations

IfStatement : **if** (*Expression*) *Statement* **else** *Statement*

1. Let *declarations* be `VarScopedDeclarations` of the first *Statement*.
2. Append to *declarations* the elements of the `VarScopedDeclarations` of the second *Statement*.
3. Return *declarations*.

IfStatement : **if** (*Expression*) *Statement*

1. Return the `VarScopedDeclarations` of *Statement*.

13.6.7 Runtime Semantics: Evaluation

IfStatement : **if** (*Expression*) *Statement* **else** *Statement*

1. Let *exprRef* be the result of evaluating *Expression*.
2. Let *exprValue* be `ToBoolean(? GetValue(exprRef))`.
3. If *exprValue* is **true**, then
 - a. Let *stmtCompletion* be the result of evaluating the first *Statement*.
4. Else,
 - a. Let *stmtCompletion* be the result of evaluating the second *Statement*.
5. Return `Completion(UpdateEmpty(stmtCompletion, undefined))`.

IfStatement : **if** (*Expression*) *Statement*

1. Let *exprRef* be the result of evaluating *Expression*.
2. Let *exprValue* be `ToBoolean(? GetValue(exprRef))`.
3. If *exprValue* is **false**, then
 - a. Return `NormalCompletion(undefined)`.
4. Else,
 - a. Let *stmtCompletion* be the result of evaluating *Statement*.
 - b. Return `Completion(UpdateEmpty(stmtCompletion, undefined))`.

13.7 Iteration Statements

Syntax

*IterationStatement*_[Yield, Await, Return] :

```
do Statement[?Yield, ?Await, ?Return] while ( Expression[+In, ?Yield, ?Await] ) ;
while ( Expression[+In, ?Yield, ?Await] ) Statement[?Yield, ?Await, ?Return]
for ( [lookahead ∉ { let [ ]}] Expression[~In, ?Yield, ?Await] opt ;
    Expression[+In, ?Yield, ?Await] opt ; Expression[+In, ?Yield, ?Await] opt )
    Statement[?Yield, ?Await, ?Return]
for ( var VariableDeclarationList[~In, ?Yield, ?Await] ;
    Expression[+In, ?Yield, ?Await] opt ; Expression[+In, ?Yield, ?Await] opt )
    Statement[?Yield, ?Await, ?Return]
for ( LexicalDeclaration[~In, ?Yield, ?Await] Expression[+In, ?Yield, ?Await] opt ;
    Expression[+In, ?Yield, ?Await] opt ) Statement[?Yield, ?Await, ?Return]
for ( [lookahead ∉ { let [ ]}] LeftHandSideExpression[?Yield, ?Await] in
    Expression[+In, ?Yield, ?Await] ) Statement[?Yield, ?Await, ?Return]
for ( var ForBinding[?Yield, ?Await] in Expression[+In, ?Yield, ?Await] )
    Statement[?Yield, ?Await, ?Return]
for ( ForDeclaration[?Yield, ?Await] in Expression[+In, ?Yield, ?Await] )
    Statement[?Yield, ?Await, ?Return]
for ( [lookahead ≠ let] LeftHandSideExpression[?Yield, ?Await] of
    AssignmentExpression[+In, ?Yield, ?Await] ) Statement[?Yield, ?Await, ?Return]
for ( var ForBinding[?Yield, ?Await] of AssignmentExpression[+In, ?Yield, ?Await] )
    Statement[?Yield, ?Await, ?Return]
for ( ForDeclaration[?Yield, ?Await] of AssignmentExpression[+In, ?Yield, ?Await] )
    Statement[?Yield, ?Await, ?Return]
[+Await] for await ( [lookahead ≠ let] LeftHandSideExpression[?Yield, ?Await] of
    AssignmentExpression[+In, ?Yield, ?Await] ) Statement[?Yield, ?Await, ?Return]
[+Await] for await ( var ForBinding[?Yield, ?Await] of
    AssignmentExpression[+In, ?Yield, ?Await] ) Statement[?Yield, ?Await, ?Return]
[+Await] for await ( ForDeclaration[?Yield, ?Await] of
    AssignmentExpression[+In, ?Yield, ?Await] ) Statement[?Yield, ?Await, ?Return]
```

*ForDeclaration*_[Yield, Await] :

LetOrConst ForBinding_[?Yield, ?Await]

*ForBinding*_[Yield, Await] :

BindingIdentifier_[?Yield, ?Await]

BindingPattern_[?Yield, ?Await]

NOTE

This section is extended by Annex B.3.6.

13.7.1 Semantics

13.7.1.1 Static Semantics: Early Errors

IterationStatement :

```
do Statement while ( Expression ) ;  
while ( Expression ) Statement  
for ( Expressionopt ; Expressionopt ; Expressionopt ) Statement  
for ( var VariableDeclarationList ; Expressionopt ; Expressionopt ) Statement  
for ( LexicalDeclaration Expressionopt ; Expressionopt ) Statement  
for ( LeftHandSideExpression in Expression ) Statement  
for ( var ForBinding in Expression ) Statement  
for ( ForDeclaration in Expression ) Statement  
for ( LeftHandSideExpression of AssignmentExpression ) Statement  
for ( var ForBinding of AssignmentExpression ) Statement  
for ( ForDeclaration of AssignmentExpression ) Statement  
for await ( LeftHandSideExpression of AssignmentExpression ) Statement  
for await ( var ForBinding of AssignmentExpression ) Statement  
for await ( ForDeclaration of AssignmentExpression ) Statement
```

It is a Syntax Error if `IsLabelledFunction(Statement)` is **true**.

NOTE

It is only necessary to apply this rule if the extension specified in B.3.2 is implemented.

13.7.1.2 Runtime Semantics: LoopContinues (*completion*, *labelSet*)

The abstract operation LoopContinues with arguments *completion* and *labelSet* is defined by the following steps:

1. If *completion*.[[Type]] is normal, return **true**.
2. If *completion*.[[Type]] is not continue, return **false**.
3. If *completion*.[[Target]] is empty, return **true**.
4. If *completion*.[[Target]] is an element of *labelSet*, return **true**.
5. Return **false**.

NOTE

Within the *Statement* part of an *IterationStatement* a *ContinueStatement* may be used to begin a new iteration.

13.7.2 The do-while Statement

13.7.2.1 Static Semantics: ContainsDuplicateLabels

With parameter *labelSet*.

IterationStatement : **do** Statement **while** (Expression) ;

1. Return ContainsDuplicateLabels of *Statement* with argument *labelSet*.

13.7.2.2 Static Semantics: ContainsUndefinedBreakTarget

With parameter *labelSet*.

IterationStatement : **do** *Statement* **while** (*Expression*) ;

1. Return *ContainsUndefinedBreakTarget* of *Statement* with argument *labelSet*.

13.7.2.3 Static Semantics: *ContainsUndefinedContinueTarget*

With parameters *iterationSet* and *labelSet*.

IterationStatement : **do** *Statement* **while** (*Expression*) ;

1. Return *ContainsUndefinedContinueTarget* of *Statement* with arguments *iterationSet* and « ».

13.7.2.4 Static Semantics: *VarDeclaredNames*

IterationStatement : **do** *Statement* **while** (*Expression*) ;

1. Return the *VarDeclaredNames* of *Statement*.

13.7.2.5 Static Semantics: *VarScopedDeclarations*

IterationStatement : **do** *Statement* **while** (*Expression*) ;

1. Return the *VarScopedDeclarations* of *Statement*.

13.7.2.6 Runtime Semantics: *LabelledEvaluation*

With parameter *labelSet*.

IterationStatement : **do** *Statement* **while** (*Expression*) ;

1. Let *V* be **undefined**.
2. Repeat,
 - a. Let *stmtResult* be the result of evaluating *Statement*.
 - b. If *LoopContinues*(*stmtResult*, *labelSet*) is **false**, return *Completion*(*UpdateEmpty*(*stmtResult*, *V*)).
 - c. If *stmtResult*.[[Value]] is not empty, set *V* to *stmtResult*.[[Value]].
 - d. Let *exprRef* be the result of evaluating *Expression*.
 - e. Let *exprValue* be ? *GetValue*(*exprRef*).
 - f. If *ToBoolean*(*exprValue*) is **false**, return *NormalCompletion*(*V*).

13.7.3 The **while** Statement

13.7.3.1 Static Semantics: *ContainsDuplicateLabels*

With parameter *labelSet*.

IterationStatement : **while** (*Expression*) *Statement*

1. Return *ContainsDuplicateLabels* of *Statement* with argument *labelSet*.

13.7.3.2 Static Semantics: *ContainsUndefinedBreakTarget*

With parameter *labelSet*.

IterationStatement : **while** (*Expression*) *Statement*

1. Return *ContainsUndefinedBreakTarget* of *Statement* with argument *labelSet*.

13.7.3.3 Static Semantics: *ContainsUndefinedContinueTarget*

With parameters *iterationSet* and *labelSet*.

IterationStatement : **while** (*Expression*) *Statement*

1. Return *ContainsUndefinedContinueTarget* of *Statement* with arguments *iterationSet* and « ».

13.7.3.4 Static Semantics: *VarDeclaredNames*

IterationStatement : **while** (*Expression*) *Statement*

1. Return the *VarDeclaredNames* of *Statement*.

13.7.3.5 Static Semantics: *VarScopedDeclarations*

IterationStatement : **while** (*Expression*) *Statement*

1. Return the *VarScopedDeclarations* of *Statement*.

13.7.3.6 Runtime Semantics: *LabelledEvaluation*

With parameter *labelSet*.

IterationStatement : **while** (*Expression*) *Statement*

1. Let *V* be **undefined**.
2. Repeat,
 - a. Let *exprRef* be the result of evaluating *Expression*.
 - b. Let *exprValue* be ? *GetValue*(*exprRef*).
 - c. If *ToBoolean*(*exprValue*) is **false**, return *NormalCompletion*(*V*).
 - d. Let *stmtResult* be the result of evaluating *Statement*.
 - e. If *LoopContinues*(*stmtResult*, *labelSet*) is **false**, return *Completion*(*UpdateEmpty*(*stmtResult*, *V*)).
 - f. If *stmtResult*.[[Value]] is not empty, set *V* to *stmtResult*.[[Value]].

13.7.4 The **for** Statement

13.7.4.1 Static Semantics: *Early Errors*

IterationStatement : **for** (*LexicalDeclaration* *Expression* ; *Expression*) *Statement*

It is a Syntax Error if any element of the *BoundNames* of *LexicalDeclaration* also occurs in the *VarDeclaredNames* of *Statement*.

13.7.4.2 Static Semantics: *ContainsDuplicateLabels*

With parameter *labelSet*.

IterationStatement :

```

for ( Expressionopt ; Expressionopt ; Expressionopt ) Statement
for ( var VariableDeclarationList ; Expressionopt ; Expressionopt ) Statement
for ( LexicalDeclaration Expressionopt ; Expressionopt ) Statement

```

1. Return ContainsDuplicateLabels of *Statement* with argument *labelSet*.

13.7.4.3 Static Semantics: ContainsUndefinedBreakTarget

With parameter *labelSet*.

IterationStatement :

```

for ( Expressionopt ; Expressionopt ; Expressionopt ) Statement
for ( var VariableDeclarationList ; Expressionopt ; Expressionopt ) Statement
for ( LexicalDeclaration Expressionopt ; Expressionopt ) Statement

```

1. Return ContainsUndefinedBreakTarget of *Statement* with argument *labelSet*.

13.7.4.4 Static Semantics: ContainsUndefinedContinueTarget

With parameters *iterationSet* and *labelSet*.

IterationStatement :

```

for ( Expressionopt ; Expressionopt ; Expressionopt ) Statement
for ( var VariableDeclarationList ; Expressionopt ; Expressionopt ) Statement
for ( LexicalDeclaration Expressionopt ; Expressionopt ) Statement

```

1. Return ContainsUndefinedContinueTarget of *Statement* with arguments *iterationSet* and « ».

13.7.4.5 Static Semantics: VarDeclaredNames

IterationStatement : **for** (Expression ; Expression ; Expression) Statement

1. Return the VarDeclaredNames of *Statement*.

IterationStatement : **for** (**var** VariableDeclarationList ; Expression ; Expression) Statement

1. Let *names* be BoundNames of *VariableDeclarationList*.
2. Append to *names* the elements of the VarDeclaredNames of *Statement*.
3. Return *names*.

IterationStatement : **for** (LexicalDeclaration Expression ; Expression) Statement

1. Return the VarDeclaredNames of *Statement*.

13.7.4.6 Static Semantics: VarScopedDeclarations

IterationStatement : **for** (Expression ; Expression ; Expression) Statement

1. Return the VarScopedDeclarations of *Statement*.

IterationStatement : **for** (**var** VariableDeclarationList ; Expression ; Expression) Statement

1. Let *declarations* be VarScopedDeclarations of *VariableDeclarationList*.

2. Append to *declarations* the elements of the VarScopedDeclarations of *Statement*.
3. Return *declarations*.

IterationStatement : **for** (*LexicalDeclaration Expression* ; *Expression*) *Statement*

1. Return the VarScopedDeclarations of *Statement*.

13.7.4.7 Runtime Semantics: LabelledEvaluation

With parameter *labelSet*.

IterationStatement : **for** (*Expression* ; *Expression* ; *Expression*) *Statement*

1. If the first *Expression* is present, then
 - a. Let *exprRef* be the result of evaluating the first *Expression*.
 - b. Perform ? *GetValue*(*exprRef*).
2. Return ? *ForBodyEvaluation*(the second *Expression*, the third *Expression*, *Statement*, « », *labelSet*).

IterationStatement : **for** (**var** *VariableDeclarationList* ; *Expression* ; *Expression*) *Statement*

1. Let *varDcl* be the result of evaluating *VariableDeclarationList*.
2. *ReturnIfAbrupt*(*varDcl*).
3. Return ? *ForBodyEvaluation*(the first *Expression*, the second *Expression*, *Statement*, « », *labelSet*).

IterationStatement : **for** (*LexicalDeclaration Expression* ; *Expression*) *Statement*

1. Let *oldEnv* be the *running execution context*'s *LexicalEnvironment*.
2. Let *loopEnv* be *NewDeclarativeEnvironment*(*oldEnv*).
3. Let *loopEnvRec* be *loopEnv*'s *EnvironmentRecord*.
4. Let *isConst* be the result of performing *IsConstantDeclaration* of *LexicalDeclaration*.
5. Let *boundNames* be the *BoundNames* of *LexicalDeclaration*.
6. For each element *dn* of *boundNames*, do
 - a. If *isConst* is **true**, then
 - i. Perform ! *loopEnvRec*.*CreateImmutableBinding*(*dn*, **true**).
 - b. Else,
 - i. Perform ! *loopEnvRec*.*CreateMutableBinding*(*dn*, **false**).
7. Set the *running execution context*'s *LexicalEnvironment* to *loopEnv*.
8. Let *forDcl* be the result of evaluating *LexicalDeclaration*.
9. If *forDcl* is an *abrupt completion*, then
 - a. Set the *running execution context*'s *LexicalEnvironment* to *oldEnv*.
 - b. Return *Completion*(*forDcl*).
10. If *isConst* is **false**, let *perIterationLets* be *boundNames*; otherwise let *perIterationLets* be « ».
11. Let *bodyResult* be *ForBodyEvaluation*(the first *Expression*, the second *Expression*, *Statement*, *perIterationLets*, *labelSet*).
12. Set the *running execution context*'s *LexicalEnvironment* to *oldEnv*.
13. Return *Completion*(*bodyResult*).

13.7.4.8 Runtime Semantics: ForBodyEvaluation (*test*, *increment*, *stmt*, *perIterationBindings*, *labelSet*)

The abstract operation *ForBodyEvaluation* with arguments *test*, *increment*, *stmt*, *perIterationBindings*, and *labelSet* is performed as follows:

1. Let *V* be **undefined**.
2. Perform ? *CreatePerIterationEnvironment(perIterationBindings)*.
3. Repeat,
 - a. If *test* is not [empty], then
 - i. Let *testRef* be the result of evaluating *test*.
 - ii. Let *testValue* be ? *GetValue(testRef)*.
 - iii. If *ToBoolean(testValue)* is **false**, return *NormalCompletion(V)*.
 - b. Let *result* be the result of evaluating *stmt*.
 - c. If *LoopContinues(result, labelSet)* is **false**, return *Completion(UpdateEmpty(result, V))*.
 - d. If *result.[[Value]]* is not **empty**, set *V* to *result.[[Value]]*.
 - e. Perform ? *CreatePerIterationEnvironment(perIterationBindings)*.
 - f. If *increment* is not [empty], then
 - i. Let *incRef* be the result of evaluating *increment*.
 - ii. Perform ? *GetValue(incRef)*.

13.7.4.9 Runtime Semantics: CreatePerIterationEnvironment (*perIterationBindings*)

The abstract operation *CreatePerIterationEnvironment* with argument *perIterationBindings* is performed as follows:

1. If *perIterationBindings* has any elements, then
 - a. Let *lastIterationEnv* be the *running execution context*'s *LexicalEnvironment*.
 - b. Let *lastIterationEnvRec* be *lastIterationEnv*'s *EnvironmentRecord*.
 - c. Let *outer* be *lastIterationEnv*'s outer environment reference.
 - d. **Assert**: *outer* is not **null**.
 - e. Let *thisIterationEnv* be *NewDeclarativeEnvironment(outer)*.
 - f. Let *thisIterationEnvRec* be *thisIterationEnv*'s *EnvironmentRecord*.
 - g. For each element *bn* of *perIterationBindings*, do
 - i. Perform ! *thisIterationEnvRec.CreateMutableBinding(bn, false)*.
 - ii. Let *lastValue* be ? *lastIterationEnvRec.GetBindingValue(bn, true)*.
 - iii. Perform *thisIterationEnvRec.InitializeBinding(bn, lastValue)*.
 - h. Set the *running execution context*'s *LexicalEnvironment* to *thisIterationEnv*.
2. Return **undefined**.

13.7.5 The for-in, for-of, and for-await-of Statements

13.7.5.1 Static Semantics: Early Errors

IterationStatement :

```

for ( LeftHandSideExpression in Expression ) Statement
for ( LeftHandSideExpression of AssignmentExpression ) Statement
for await ( LeftHandSideExpression of AssignmentExpression ) Statement

```

It is a Syntax Error if *LeftHandSideExpression* is either an *ObjectLiteral* or an *ArrayLiteral* and if *LeftHandSideExpression* is not *covering* an *AssignmentPattern*.

If *LeftHandSideExpression* is either an *ObjectLiteral* or an *ArrayLiteral* and if *LeftHandSideExpression* is *covering* an *AssignmentPattern* then the following rules are not applied. Instead, the Early Error rules for *AssignmentPattern* are used.

It is a Syntax Error if AssignmentTargetType of *LeftHandSideExpression* is not **simple**.

It is a Syntax Error if the *LeftHandSideExpression* is *CoverParenthesizedExpressionAndArrowParameterList* : (*Expression*) and *Expression* derives a phrase that would produce a Syntax Error according to these rules if that phrase were substituted for *LeftHandSideExpression*. This rule is recursively applied.

NOTE

The last rule means that the other rules are applied even if parentheses surround *Expression*.

IterationStatement :

```
for ( ForDeclaration in Expression ) Statement
for ( ForDeclaration of AssignmentExpression ) Statement
for await ( ForDeclaration of AssignmentExpression ) Statement
```

It is a Syntax Error if the BoundNames of *ForDeclaration* contains "**let**".

It is a Syntax Error if any element of the BoundNames of *ForDeclaration* also occurs in the VarDeclaredNames of *Statement*.

It is a Syntax Error if the BoundNames of *ForDeclaration* contains any duplicate entries.

13.7.5.2 Static Semantics: BoundNames

ForDeclaration : *LetOrConst ForBinding*

1. Return the BoundNames of *ForBinding*.

13.7.5.3 Static Semantics: ContainsDuplicateLabels

With parameter *labelSet*.

IterationStatement :

```
for ( LeftHandSideExpression in Expression ) Statement
for ( var ForBinding in Expression ) Statement
for ( ForDeclaration in Expression ) Statement
for ( LeftHandSideExpression of AssignmentExpression ) Statement
for ( var ForBinding of AssignmentExpression ) Statement
for ( ForDeclaration of AssignmentExpression ) Statement
for await ( LeftHandSideExpression of AssignmentExpression ) Statement
for await ( var ForBinding of AssignmentExpression ) Statement
for await ( ForDeclaration of AssignmentExpression ) Statement
```

1. Return ContainsDuplicateLabels of *Statement* with argument *labelSet*.

NOTE

This section is extended by Annex [B.3.6](#).

13.7.5.4 Static Semantics: ContainsUndefinedBreakTarget

With parameter *labelSet*.

IterationStatement :

```

for ( LeftHandSideExpression in Expression ) Statement
for ( var ForBinding in Expression ) Statement
for ( ForDeclaration in Expression ) Statement
for ( LeftHandSideExpression of AssignmentExpression ) Statement
for ( var ForBinding of AssignmentExpression ) Statement
for ( ForDeclaration of AssignmentExpression ) Statement
for await ( LeftHandSideExpression of AssignmentExpression ) Statement
for await ( var ForBinding of AssignmentExpression ) Statement
for await ( ForDeclaration of AssignmentExpression ) Statement

```

1. Return `ContainsUndefinedBreakTarget` of *Statement* with argument *labelSet*.

NOTE

This section is extended by Annex B.3.6.

13.7.5.5 Static Semantics: `ContainsUndefinedContinueTarget`

With parameters *iterationSet* and *labelSet*.

IterationStatement :

```

for ( LeftHandSideExpression in Expression ) Statement
for ( var ForBinding in Expression ) Statement
for ( ForDeclaration in Expression ) Statement
for ( LeftHandSideExpression of AssignmentExpression ) Statement
for ( var ForBinding of AssignmentExpression ) Statement
for ( ForDeclaration of AssignmentExpression ) Statement
for await ( LeftHandSideExpression of AssignmentExpression ) Statement
for await ( var ForBinding of AssignmentExpression ) Statement
for await ( ForDeclaration of AssignmentExpression ) Statement

```

1. Return `ContainsUndefinedContinueTarget` of *Statement* with arguments *iterationSet* and « ».

NOTE

This section is extended by Annex B.3.6.

13.7.5.6 Static Semantics: `IsDestructuring`

ForDeclaration : *LetOrConst* *ForBinding*

1. Return `IsDestructuring` of *ForBinding*.

ForBinding : *BindingIdentifier*

1. Return **false**.

ForBinding : *BindingPattern*

1. Return **true**.

NOTE

This section is extended by Annex [B.3.6](#).

13.7.5.7 Static Semantics: VarDeclaredNames

IterationStatement : **for** (*LeftHandSideExpression* **in** *Expression*) *Statement*

1. Return the VarDeclaredNames of *Statement*.

IterationStatement : **for** (**var** *ForBinding* **in** *Expression*) *Statement*

1. Let *names* be the BoundNames of *ForBinding*.
2. Append to *names* the elements of the VarDeclaredNames of *Statement*.
3. Return *names*.

IterationStatement : **for** (*ForDeclaration* **in** *Expression*) *Statement*

1. Return the VarDeclaredNames of *Statement*.

IterationStatement :

for (*LeftHandSideExpression* **of** *AssignmentExpression*) *Statement*
for await (*LeftHandSideExpression* **of** *AssignmentExpression*) *Statement*

1. Return the VarDeclaredNames of *Statement*.

IterationStatement :

for (**var** *ForBinding* **of** *AssignmentExpression*) *Statement*
for await (**var** *ForBinding* **of** *AssignmentExpression*) *Statement*

1. Let *names* be the BoundNames of *ForBinding*.
2. Append to *names* the elements of the VarDeclaredNames of *Statement*.
3. Return *names*.

IterationStatement :

for (*ForDeclaration* **of** *AssignmentExpression*) *Statement*
for await (*ForDeclaration* **of** *AssignmentExpression*) *Statement*

1. Return the VarDeclaredNames of *Statement*.

NOTE

This section is extended by Annex [B.3.6](#).

13.7.5.8 Static Semantics: VarScopedDeclarations

IterationStatement : **for** (*LeftHandSideExpression* **in** *Expression*) *Statement*

1. Return the VarScopedDeclarations of *Statement*.

IterationStatement : **for** (**var** *ForBinding* **in** *Expression*) *Statement*

1. Let *declarations* be a [List](#) containing *ForBinding*.
2. Append to *declarations* the elements of the VarScopedDeclarations of *Statement*.
3. Return *declarations*.

IterationStatement :

```
for ( ForDeclaration in Expression ) Statement  
for await ( LeftHandSideExpression of AssignmentExpression ) Statement
```

1. Return the VarScopedDeclarations of *Statement*.

IterationStatement :

```
for ( LeftHandSideExpression of AssignmentExpression ) Statement  
for await ( var ForBinding of AssignmentExpression ) Statement
```

1. Return the VarScopedDeclarations of *Statement*.

IterationStatement :

```
for ( var ForBinding of AssignmentExpression ) Statement  
for await ( var ForBinding of AssignmentExpression ) Statement
```

1. Let *declarations* be a List containing *ForBinding*.
2. Append to *declarations* the elements of the VarScopedDeclarations of *Statement*.
3. Return *declarations*.

IterationStatement :

```
for ( ForDeclaration of AssignmentExpression ) Statement  
for await ( ForDeclaration of AssignmentExpression ) Statement
```

1. Return the VarScopedDeclarations of *Statement*.

NOTE

This section is extended by Annex B.3.6.

13.7.5.9 Runtime Semantics: BindingInitialization

With parameters *value* and *environment*.

NOTE

undefined is passed for *environment* to indicate that a **PutValue** operation should be used to assign the initialization value. This is the case for **var** statements and the formal parameter lists of some non-strict functions (see 9.2.15). In those cases a lexical binding is hoisted and preinitialized prior to evaluation of its initializer.

ForDeclaration : *LetOrConst ForBinding*

1. Return the result of performing BindingInitialization for *ForBinding* passing *value* and *environment* as the arguments.

13.7.5.10 Runtime Semantics: BindingInstantiation

With parameter *environment*.

ForDeclaration : *LetOrConst ForBinding*

1. Let *envRec* be *environment*'s EnvironmentRecord.
2. **Assert**: *envRec* is a declarative Environment Record.

3. For each element *name* of the BoundNames of *ForBinding*, do
 - a. If IsConstantDeclaration of *LetOrConst* is **true**, then
 - i. Perform ! *envRec*.CreateImmutableBinding(*name*, **true**).
 - b. Else,
 - i. Perform ! *envRec*.CreateMutableBinding(*name*, **false**).

13.7.5.11 Runtime Semantics: LabelledEvaluation

With parameter *labelSet*.

IterationStatement : **for** (*LeftHandSideExpression* **in** *Expression*) *Statement*

1. Let *keyResult* be ? *ForIn/OfHeadEvaluation*(« », *Expression*, enumerate).
2. Return ? *ForIn/OfBodyEvaluation*(*LeftHandSideExpression*, *Statement*, *keyResult*, enumerate, assignment, *labelSet*).

IterationStatement : **for** (**var** *ForBinding* **in** *Expression*) *Statement*

1. Let *keyResult* be ? *ForIn/OfHeadEvaluation*(« », *Expression*, enumerate).
2. Return ? *ForIn/OfBodyEvaluation*(*ForBinding*, *Statement*, *keyResult*, enumerate, varBinding, *labelSet*).

IterationStatement : **for** (*ForDeclaration* **in** *Expression*) *Statement*

1. Let *keyResult* be ? *ForIn/OfHeadEvaluation*(BoundNames of *ForDeclaration*, *Expression*, enumerate).
2. Return ? *ForIn/OfBodyEvaluation*(*ForDeclaration*, *Statement*, *keyResult*, enumerate, lexicalBinding, *labelSet*).

IterationStatement : **for** (*LeftHandSideExpression* **of** *AssignmentExpression*) *Statement*

1. Let *keyResult* be ? *ForIn/OfHeadEvaluation*(« », *AssignmentExpression*, iterate).
2. Return ? *ForIn/OfBodyEvaluation*(*LeftHandSideExpression*, *Statement*, *keyResult*, iterate, assignment, *labelSet*).

IterationStatement : **for** (**var** *ForBinding* **of** *AssignmentExpression*) *Statement*

1. Let *keyResult* be ? *ForIn/OfHeadEvaluation*(« », *AssignmentExpression*, iterate).
2. Return ? *ForIn/OfBodyEvaluation*(*ForBinding*, *Statement*, *keyResult*, iterate, varBinding, *labelSet*).

IterationStatement : **for** (*ForDeclaration* **of** *AssignmentExpression*) *Statement*

1. Let *keyResult* be ? *ForIn/OfHeadEvaluation*(BoundNames of *ForDeclaration*, *AssignmentExpression*, iterate).
2. Return ? *ForIn/OfBodyEvaluation*(*ForDeclaration*, *Statement*, *keyResult*, iterate, lexicalBinding, *labelSet*).

IterationStatement : **for await** (*LeftHandSideExpression* **of** *AssignmentExpression*) *Statement*

1. Let *keyResult* be ? *ForIn/OfHeadEvaluation*(« », *AssignmentExpression*, async-iterate).
2. Return ? *ForIn/OfBodyEvaluation*(*LeftHandSideExpression*, *Statement*, *keyResult*, iterate, assignment, *labelSet*, async).

IterationStatement : **for await** (**var** *ForBinding* **of** *AssignmentExpression*) *Statement*

1. Let *keyResult* be ? *ForIn/OfHeadEvaluation*(« », *AssignmentExpression*, async-iterate).
2. Return ? *ForIn/OfBodyEvaluation*(*ForBinding*, *Statement*, *keyResult*, iterate, varBinding, *labelSet*, async).

IterationStatement : **for await** (*ForDeclaration* **of** *AssignmentExpression*) *Statement*

1. Let *keyResult* be ? **ForIn/OfHeadEvaluation**(BoundNames of *ForDeclaration*, *AssignmentExpression*, **async-iterate**).
2. Return ? **ForIn/OfBodyEvaluation**(*ForDeclaration*, *Statement*, *keyResult*, **iterate**, **lexicalBinding**, *labelSet*, **async**).

NOTE

This section is extended by Annex B.3.6.

13.7.5.12 Runtime Semantics: ForIn/OfHeadEvaluation (*TDZnames*, *expr*, *iterationKind*)

The abstract operation ForIn/OfHeadEvaluation is called with arguments *TDZnames*, *expr*, and *iterationKind*. The value of *iterationKind* is either **enumerate**, **iterate**, or **async-iterate**.

1. Let *oldEnv* be the **running execution context**'s **LexicalEnvironment**.
2. If *TDZnames* is not an empty **List**, then
 - a. **Assert**: *TDZnames* has no duplicate entries.
 - b. Let *TDZ* be **NewDeclarativeEnvironment**(*oldEnv*).
 - c. Let *TDZEnvRec* be *TDZ*'s **EnvironmentRecord**.
 - d. For each string *name* in *TDZnames*, do
 - i. Perform ! *TDZEnvRec*.**CreateMutableBinding**(*name*, **false**).
 - e. Set the **running execution context**'s **LexicalEnvironment** to *TDZ*.
3. Let *exprRef* be the result of evaluating *expr*.
4. Set the **running execution context**'s **LexicalEnvironment** to *oldEnv*.
5. Let *exprValue* be ? **GetValue**(*exprRef*).
6. If *iterationKind* is **enumerate**, then
 - a. If *exprValue* is **undefined** or **null**, then
 - i. Return **Completion** { [[Type]]: **break**, [[Value]]: **empty**, [[Target]]: **empty** }.
 - b. Let *obj* be ! **ToObject**(*exprValue*).
 - c. Return ? **EnumerateObjectProperties**(*obj*).
7. Else,
 - a. **Assert**: *iterationKind* is **iterate**.
 - b. If *iterationKind* is **async-iterate**, let *iteratorHint* be **async**.
 - c. Else, let *iteratorHint* be **sync**.
 - d. Return ? **GetIterator**(*exprValue*, *iteratorHint*).

13.7.5.13 Runtime Semantics: ForIn/OfBodyEvaluation (*lhs*, *stmt*, *iteratorRecord*, *iterationKind*, *lhsKind*, *labelSet* [, *iteratorKind*])

The abstract operation ForIn/OfBodyEvaluation is called with arguments *lhs*, *stmt*, *iteratorRecord*, *iterationKind*, *lhsKind*, *labelSet*, and optional argument *iteratorKind*. The value of *lhsKind* is either **assignment**, **varBinding** or **lexicalBinding**. The value of *iteratorKind* is either **sync** or **async**.

1. If *iteratorKind* is not present, set *iteratorKind* to **sync**.
2. Let *oldEnv* be the **running execution context**'s **LexicalEnvironment**.
3. Let *V* be **undefined**.
4. Let *destructuring* be **IsDestructuring** of *lhs*.
5. If *destructuring* is **true** and if *lhsKind* is **assignment**, then
 - a. **Assert**: *lhs* is a *LeftHandSideExpression*.
 - b. Let *assignmentPattern* be the *AssignmentPattern* that is **covered** by *lhs*.
6. Repeat,

- a. Let *nextResult* be ? *Call*(*iteratorRecord*.[[NextMethod]], *iteratorRecord*.[[Iterator]], « »).
- b. If *iteratorKind* is **async**, then set *nextResult* to ? *Await*(*nextResult*).
- c. If *Type*(*nextResult*) is not **Object**, throw a **TypeError** exception.
- d. Let *done* be ? *IteratorComplete*(*nextResult*).
- e. If *done* is **true**, return *NormalCompletion*(*V*).
- f. Let *nextValue* be ? *IteratorValue*(*nextResult*).
- g. If *lhsKind* is either **assignment** or **varBinding**, then
 - i. If *destructuring* is **false**, then
 1. Let *lhsRef* be the result of evaluating *lhs*. (It may be evaluated repeatedly.)
- h. Else,
 - i. **Assert**: *lhsKind* is **lexicalBinding**.
 - ii. **Assert**: *lhs* is a *ForDeclaration*.
 - iii. Let *iterationEnv* be *NewDeclarativeEnvironment*(*oldEnv*).
 - iv. Perform *BindingInstantiation* for *lhs* passing *iterationEnv* as the argument.
 - v. Set the **running execution context**'s *LexicalEnvironment* to *iterationEnv*.
 - vi. If *destructuring* is **false**, then
 1. **Assert**: *lhs* binds a single name.
 2. Let *lhsName* be the sole element of *BoundNames* of *lhs*.
 3. Let *lhsRef* be ! *ResolveBinding*(*lhsName*).
- i. If *destructuring* is **false**, then
 - i. If *lhsRef* is an **abrupt completion**, then
 1. Let *status* be *lhsRef*.
 - ii. Else if *lhsKind* is **lexicalBinding**, then
 1. Let *status* be *InitializeReferencedBinding*(*lhsRef*, *nextValue*).
 - iii. Else,
 1. Let *status* be *PutValue*(*lhsRef*, *nextValue*).
- j. Else,
 - i. If *lhsKind* is **assignment**, then
 1. Let *status* be the result of performing *DestructuringAssignmentEvaluation* of *assignmentPattern* using *nextValue* as the argument.
 - ii. Else if *lhsKind* is **varBinding**, then
 1. **Assert**: *lhs* is a *ForBinding*.
 2. Let *status* be the result of performing *BindingInitialization* for *lhs* passing *nextValue* and **undefined** as the arguments.
 - iii. Else,
 1. **Assert**: *lhsKind* is **lexicalBinding**.
 2. **Assert**: *lhs* is a *ForDeclaration*.
 3. Let *status* be the result of performing *BindingInitialization* for *lhs* passing *nextValue* and *iterationEnv* as arguments.
- k. If *status* is an **abrupt completion**, then
 - i. Set the **running execution context**'s *LexicalEnvironment* to *oldEnv*.
 - ii. If *iteratorKind* is **async**, return ? *AsyncIteratorClose*(*iteratorRecord*, *status*).
 - iii. If *iterationKind* is **enumerate**, then
 1. Return *status*.
 - iv. Else,
 1. **Assert**: *iterationKind* is **iterate**.
 2. Return ? *IteratorClose*(*iteratorRecord*, *status*).
- l. Let *result* be the result of evaluating *stmt*.

- m. Set the [running execution context](#)'s LexicalEnvironment to *oldEnv*.
- n. If [LoopContinues](#)(*result*, *labelSet*) is **false**, then
 - i. If *iterationKind* is **enumerate**, then
 - 1. Return [Completion](#)([UpdateEmpty](#)(*result*, *V*)).
 - ii. Else,
 - 1. [Assert](#): *iterationKind* is **iterate**.
 - 2. Set *status* to [UpdateEmpty](#)(*result*, *V*).
 - 3. If *iteratorKind* is **async**, return ? [AsyncIteratorClose](#)(*iteratorRecord*, *status*).
 - 4. Return ? [IteratorClose](#)(*iteratorRecord*, *status*).
- o. If *result*.[[Value]] is not **empty**, set *V* to *result*.[[Value]].

13.7.5.14 Runtime Semantics: Evaluation

ForBinding : *BindingIdentifier*

1. Let *bindingId* be StringValue of *BindingIdentifier*.
2. Return ? [ResolveBinding](#)(*bindingId*).

13.7.5.15 EnumerateObjectProperties (*O*)

When the abstract operation EnumerateObjectProperties is called with argument *O*, the following steps are taken:

1. [Assert](#): [Type](#)(*O*) is Object.
2. Return an Iterator object ([25.1.1.2](#)) whose **next** method iterates over all the String-valued keys of enumerable properties of *O*. The iterator object is never directly accessible to ECMAScript code. The mechanics and order of enumerating the properties is not specified but must conform to the rules specified below.

The iterator's **throw** and **return** methods are **null** and are never invoked. The iterator's **next** method processes object properties to determine whether the property key should be returned as an iterator value. Returned property keys do not include keys that are Symbols. Properties of the target object may be deleted during enumeration. A property that is deleted before it is processed by the iterator's **next** method is ignored. If new properties are added to the target object during enumeration, the newly added properties are not guaranteed to be processed in the active enumeration. A [property name](#) will be returned by the iterator's **next** method at most once in any enumeration.

Enumerating the properties of the target object includes enumerating properties of its prototype, and the prototype of the prototype, and so on, recursively; but a property of a prototype is not processed if it has the same name as a property that has already been processed by the iterator's **next** method. The values of [[Enumerable]] attributes are not considered when determining if a property of a prototype object has already been processed. The enumerable property names of prototype objects must be obtained by invoking EnumerateObjectProperties passing the prototype object as the argument. EnumerateObjectProperties must obtain the own property keys of the target object by calling its [[OwnPropertyKeys]] internal method. Property attributes of the target object must be obtained by calling its [[GetOwnProperty]] internal method.

NOTE

The following is an informative definition of an ECMAScript generator function that conforms to these rules:

```
function* EnumerateObjectProperties(obj) {
  const visited = new Set();
  for (const key of Reflect.ownKeys(obj)) {
    if (typeof key === "symbol") continue;
    const desc = Reflect.getOwnPropertyDescriptor(obj, key);
```

```

    if (desc) {
      visited.add(key);
      if (desc.enumerable) yield key;
    }
  }
  const proto = Reflect.getPrototypeOf(obj);
  if (proto === null) return;
  for (const protoKey of EnumerateObjectProperties(proto)) {
    if (!visited.has(protoKey)) yield protoKey;
  }
}

```

13.8 The continue Statement

Syntax

```

ContinueStatement[Yield, Await] :
    continue ;
    continue [no LineTerminator here] LabelIdentifier[?Yield, ?Await] ;

```

13.8.1 Static Semantics: Early Errors

ContinueStatement : **continue** ;

ContinueStatement : **continue** LabelIdentifier ;

It is a Syntax Error if this *ContinueStatement* is not nested, directly or indirectly (but not crossing function boundaries), within an *IterationStatement*.

13.8.2 Static Semantics: ContainsUndefinedContinueTarget

With parameters *iterationSet* and *labelSet*.

ContinueStatement : **continue** ;

1. Return **false**.

ContinueStatement : **continue** LabelIdentifier ;

1. If the StringValue of *LabelIdentifier* is not an element of *iterationSet*, return **true**.
2. Return **false**.

13.8.3 Runtime Semantics: Evaluation

ContinueStatement : **continue** ;

1. Return **Completion** { [[Type]]: **continue**, [[Value]]: empty, [[Target]]: empty }.

ContinueStatement : **continue** LabelIdentifier ;

1. Let *label* be the StringValue of *LabelIdentifier*.
2. Return Completion { [[Type]]: continue, [[Value]]: empty, [[Target]]: *label* }.

13.9 The break Statement

Syntax

```
BreakStatement[Yield, Await] :
    break ;
    break [no LineTerminator here] LabelIdentifier[?Yield, ?Await] ;
```

13.9.1 Static Semantics: Early Errors

BreakStatement : **break** ;

It is a Syntax Error if this *BreakStatement* is not nested, directly or indirectly (but not crossing function boundaries), within an *IterationStatement* or a *SwitchStatement*.

13.9.2 Static Semantics: ContainsUndefinedBreakTarget

With parameter *labelSet*.

BreakStatement : **break** ;

1. Return **false**.

BreakStatement : **break** *LabelIdentifier* ;

1. If the StringValue of *LabelIdentifier* is not an element of *labelSet*, return **true**.
2. Return **false**.

13.9.3 Runtime Semantics: Evaluation

BreakStatement : **break** ;

1. Return Completion { [[Type]]: break, [[Value]]: empty, [[Target]]: empty }.

BreakStatement : **break** *LabelIdentifier* ;

1. Let *label* be the StringValue of *LabelIdentifier*.
2. Return Completion { [[Type]]: break, [[Value]]: empty, [[Target]]: *label* }.

13.10 The return Statement

Syntax

```
ReturnStatement[Yield, Await] :
    return ;
    return [no LineTerminator here] Expression[+In, ?Yield, ?Await] ;
```

NOTE

A **return** statement causes a function to cease execution and, in most cases, returns a value to the caller. If *Expression* is omitted, the return value is **undefined**. Otherwise, the return value is the value of *Expression*. A **return** statement may not actually return a value to the caller depending on surrounding context. For example, in a **try** block, a **return** statement's completion record may be replaced with another completion record during evaluation of the **finally** block.

13.10.1 Runtime Semantics: Evaluation

ReturnStatement : **return** ;

1. Return [Completion](#) { [[Type]]: return, [[Value]]: **undefined**, [[Target]]: empty }.

ReturnStatement : **return** *Expression* ;

1. Let *exprRef* be the result of evaluating *Expression*.
2. Let *exprValue* be ? [GetValue](#)(*exprRef*).
3. If ! [GetGeneratorKind](#)() is **async**, set *exprValue* to ? [Await](#)(*exprValue*).
4. Return [Completion](#) { [[Type]]: return, [[Value]]: *exprValue*, [[Target]]: empty }.

13.11 The with Statement

Syntax

*WithStatement*_[Yield, Await, Return] :
 with (*Expression*_[+In, ?Yield, ?Await]) *Statement*_[?Yield, ?Await, ?Return]

NOTE

The **with** statement adds an object [Environment Record](#) for a computed object to the lexical environment of the [running execution context](#). It then executes a statement using this augmented lexical environment. Finally, it restores the original lexical environment.

13.11.1 Static Semantics: Early Errors

WithStatement : **with** (*Expression*) *Statement*

It is a Syntax Error if the code that matches this production is contained in [strict mode code](#).

It is a Syntax Error if [IsLabelledFunction](#)(*Statement*) is **true**.

NOTE

It is only necessary to apply the second rule if the extension specified in [B.3.2](#) is implemented.

13.11.2 Static Semantics: ContainsDuplicateLabels

With parameter *labelSet*.

WithStatement : **with** (*Expression*) *Statement*

1. Return ContainsDuplicateLabels of *Statement* with argument *labelSet*.

13.11.3 Static Semantics: ContainsUndefinedBreakTarget

With parameter *labelSet*.

WithStatement : **with** (*Expression*) *Statement*

1. Return ContainsUndefinedBreakTarget of *Statement* with argument *labelSet*.

13.11.4 Static Semantics: ContainsUndefinedContinueTarget

With parameters *iterationSet* and *labelSet*.

WithStatement : **with** (*Expression*) *Statement*

1. Return ContainsUndefinedContinueTarget of *Statement* with arguments *iterationSet* and « ».

13.11.5 Static Semantics: VarDeclaredNames

WithStatement : **with** (*Expression*) *Statement*

1. Return the VarDeclaredNames of *Statement*.

13.11.6 Static Semantics: VarScopedDeclarations

WithStatement : **with** (*Expression*) *Statement*

1. Return the VarScopedDeclarations of *Statement*.

13.11.7 Runtime Semantics: Evaluation

WithStatement : **with** (*Expression*) *Statement*

1. Let *val* be the result of evaluating *Expression*.
2. Let *obj* be ? ToObject(? GetValue(*val*)).
3. Let *oldEnv* be the running execution context's LexicalEnvironment.
4. Let *newEnv* be NewObjectEnvironment(*obj*, *oldEnv*).
5. Set the *withEnvironment* flag of *newEnv*'s EnvironmentRecord to **true**.
6. Set the running execution context's LexicalEnvironment to *newEnv*.
7. Let *C* be the result of evaluating *Statement*.
8. Set the running execution context's LexicalEnvironment to *oldEnv*.
9. Return Completion(UpdateEmpty(*C*, **undefined**)).

NOTE

No matter how control leaves the embedded *Statement*, whether normally or by some form of abrupt completion or exception, the LexicalEnvironment is always restored to its former state.

13.12 The **switch** Statement

Syntax

```
SwitchStatement[Yield, Await, Return] :  
    switch ( Expression[+In, ?Yield, ?Await] ) CaseBlock[?Yield, ?Await, ?Return]  
  
CaseBlock[Yield, Await, Return] :  
    { CaseClauses[?Yield, ?Await, ?Return] opt }  
    { CaseClauses[?Yield, ?Await, ?Return] opt DefaultClause[?Yield, ?Await, ?Return]  
      CaseClauses[?Yield, ?Await, ?Return] opt }  
  
CaseClauses[Yield, Await, Return] :  
    CaseClause[?Yield, ?Await, ?Return]  
    CaseClauses[?Yield, ?Await, ?Return] CaseClause[?Yield, ?Await, ?Return]  
  
CaseClause[Yield, Await, Return] :  
    case Expression[+In, ?Yield, ?Await] : StatementList[?Yield, ?Await, ?Return] opt  
  
DefaultClause[Yield, Await, Return] :  
    default : StatementList[?Yield, ?Await, ?Return] opt
```

13.12.1 Static Semantics: Early Errors

SwitchStatement : **switch** (*Expression*) *CaseBlock*

It is a Syntax Error if the LexicallyDeclaredNames of *CaseBlock* contains any duplicate entries.

It is a Syntax Error if any element of the LexicallyDeclaredNames of *CaseBlock* also occurs in the VarDeclaredNames of *CaseBlock*.

13.12.2 Static Semantics: ContainsDuplicateLabels

With parameter *labelSet*.

SwitchStatement : **switch** (*Expression*) *CaseBlock*

1. Return ContainsDuplicateLabels of *CaseBlock* with argument *labelSet*.

CaseBlock : { }

1. Return **false**.

CaseBlock : { *CaseClauses* *DefaultClause* *CaseClauses* }

1. If the first *CaseClauses* is present, then
 - a. Let *hasDuplicates* be ContainsDuplicateLabels of the first *CaseClauses* with argument *labelSet*.
 - b. If *hasDuplicates* is **true**, return **true**.
2. Let *hasDuplicates* be ContainsDuplicateLabels of *DefaultClause* with argument *labelSet*.
3. If *hasDuplicates* is **true**, return **true**.
4. If the second *CaseClauses* is not present, return **false**.
5. Return ContainsDuplicateLabels of the second *CaseClauses* with argument *labelSet*.

CaseClauses : *CaseClauses* *CaseClause*

1. Let *hasDuplicates* be ContainsDuplicateLabels of *CaseClauses* with argument *labelSet*.
2. If *hasDuplicates* is **true**, return **true**.
3. Return ContainsDuplicateLabels of *CaseClause* with argument *labelSet*.

CaseClause : **case** *Expression* : *StatementList*

1. If the *StatementList* is present, return ContainsDuplicateLabels of *StatementList* with argument *labelSet*.
2. Return **false**.

DefaultClause : **default** : *StatementList*

1. If the *StatementList* is present, return ContainsDuplicateLabels of *StatementList* with argument *labelSet*.
2. Return **false**.

13.12.3 Static Semantics: ContainsUndefinedBreakTarget

With parameter *labelSet*.

SwitchStatement : **switch** (*Expression*) *CaseBlock*

1. Return ContainsUndefinedBreakTarget of *CaseBlock* with argument *labelSet*.

CaseBlock : { }

1. Return **false**.

CaseBlock : { *CaseClauses* *DefaultClause* *CaseClauses* }

1. If the first *CaseClauses* is present, then
 - a. Let *hasUndefinedLabels* be ContainsUndefinedBreakTarget of the first *CaseClauses* with argument *labelSet*.
 - b. If *hasUndefinedLabels* is **true**, return **true**.
2. Let *hasUndefinedLabels* be ContainsUndefinedBreakTarget of *DefaultClause* with argument *labelSet*.
3. If *hasUndefinedLabels* is **true**, return **true**.
4. If the second *CaseClauses* is not present, return **false**.
5. Return ContainsUndefinedBreakTarget of the second *CaseClauses* with argument *labelSet*.

CaseClauses : *CaseClauses* *CaseClause*

1. Let *hasUndefinedLabels* be ContainsUndefinedBreakTarget of *CaseClauses* with argument *labelSet*.
2. If *hasUndefinedLabels* is **true**, return **true**.
3. Return ContainsUndefinedBreakTarget of *CaseClause* with argument *labelSet*.

CaseClause : **case** *Expression* : *StatementList*

1. If the *StatementList* is present, return ContainsUndefinedBreakTarget of *StatementList* with argument *labelSet*.
2. Return **false**.

DefaultClause : **default** : *StatementList*

1. If the *StatementList* is present, return ContainsUndefinedBreakTarget of *StatementList* with argument *labelSet*.
2. Return **false**.

13.12.4 Static Semantics: ContainsUndefinedContinueTarget

With parameters *iterationSet* and *labelSet*.

SwitchStatement : **switch** (*Expression*) *CaseBlock*

1. Return ContainsUndefinedContinueTarget of *CaseBlock* with arguments *iterationSet* and « ».

CaseBlock : { }

1. Return **false**.

CaseBlock : { *CaseClauses* *DefaultClause* *CaseClauses* }

1. If the first *CaseClauses* is present, then
 - a. Let *hasUndefinedLabels* be ContainsUndefinedContinueTarget of the first *CaseClauses* with arguments *iterationSet* and « ».
 - b. If *hasUndefinedLabels* is **true**, return **true**.
2. Let *hasUndefinedLabels* be ContainsUndefinedContinueTarget of *DefaultClause* with arguments *iterationSet* and « ».
3. If *hasUndefinedLabels* is **true**, return **true**.
4. If the second *CaseClauses* is not present, return **false**.
5. Return ContainsUndefinedContinueTarget of the second *CaseClauses* with arguments *iterationSet* and « ».

CaseClauses : *CaseClauses* *CaseClause*

1. Let *hasUndefinedLabels* be ContainsUndefinedContinueTarget of *CaseClauses* with arguments *iterationSet* and « ».
2. If *hasUndefinedLabels* is **true**, return **true**.
3. Return ContainsUndefinedContinueTarget of *CaseClause* with arguments *iterationSet* and « ».

CaseClause : **case** *Expression* : *StatementList*

1. If the *StatementList* is present, return ContainsUndefinedContinueTarget of *StatementList* with arguments *iterationSet* and « ».
2. Return **false**.

DefaultClause : **default** : *StatementList*

1. If the *StatementList* is present, return ContainsUndefinedContinueTarget of *StatementList* with arguments *iterationSet* and « ».
2. Return **false**.

13.12.5 Static Semantics: LexicallyDeclaredNames

CaseBlock : { }

1. Return a new empty *List*.

CaseBlock : { *CaseClauses* *DefaultClause* *CaseClauses* }

1. If the first *CaseClauses* is present, let *names* be the LexicallyDeclaredNames of the first *CaseClauses*.
2. Else, let *names* be a new empty *List*.
3. Append to *names* the elements of the LexicallyDeclaredNames of the *DefaultClause*.
4. If the second *CaseClauses* is not present, return *names*.

5. Return the result of appending to *names* the elements of the LexicallyDeclaredNames of the second *CaseClauses*.

CaseClauses : *CaseClauses CaseClause*

1. Let *names* be LexicallyDeclaredNames of *CaseClauses*.
2. Append to *names* the elements of the LexicallyDeclaredNames of *CaseClause*.
3. Return *names*.

CaseClause : **case** *Expression* : *StatementList*

1. If the *StatementList* is present, return the LexicallyDeclaredNames of *StatementList*.
2. Return a new empty *List*.

DefaultClause : **default** : *StatementList*

1. If the *StatementList* is present, return the LexicallyDeclaredNames of *StatementList*.
2. Return a new empty *List*.

13.12.6 Static Semantics: LexicallyScopedDeclarations

CaseBlock : { }

1. Return a new empty *List*.

CaseBlock : { *CaseClauses DefaultClause CaseClauses* }

1. If the first *CaseClauses* is present, let *declarations* be the LexicallyScopedDeclarations of the first *CaseClauses*.
2. Else, let *declarations* be a new empty *List*.
3. Append to *declarations* the elements of the LexicallyScopedDeclarations of the *DefaultClause*.
4. If the second *CaseClauses* is not present, return *declarations*.
5. Return the result of appending to *declarations* the elements of the LexicallyScopedDeclarations of the second *CaseClauses*.

CaseClauses : *CaseClauses CaseClause*

1. Let *declarations* be LexicallyScopedDeclarations of *CaseClauses*.
2. Append to *declarations* the elements of the LexicallyScopedDeclarations of *CaseClause*.
3. Return *declarations*.

CaseClause : **case** *Expression* : *StatementList*

1. If the *StatementList* is present, return the LexicallyScopedDeclarations of *StatementList*.
2. Return a new empty *List*.

DefaultClause : **default** : *StatementList*

1. If the *StatementList* is present, return the LexicallyScopedDeclarations of *StatementList*.
2. Return a new empty *List*.

13.12.7 Static Semantics: VarDeclaredNames

SwitchStatement : **switch** (*Expression*) *CaseBlock*

1. Return the VarDeclaredNames of *CaseBlock*.

CaseBlock : { }

1. Return a new empty [List](#).

CaseBlock : { *CaseClauses* *DefaultClause* *CaseClauses* }

1. If the first *CaseClauses* is present, let *names* be the VarDeclaredNames of the first *CaseClauses*.
2. Else, let *names* be a new empty [List](#).
3. Append to *names* the elements of the VarDeclaredNames of the *DefaultClause*.
4. If the second *CaseClauses* is not present, return *names*.
5. Return the result of appending to *names* the elements of the VarDeclaredNames of the second *CaseClauses*.

CaseClauses : *CaseClauses* *CaseClause*

1. Let *names* be VarDeclaredNames of *CaseClauses*.
2. Append to *names* the elements of the VarDeclaredNames of *CaseClause*.
3. Return *names*.

CaseClause : **case** *Expression* : *StatementList*

1. If the *StatementList* is present, return the VarDeclaredNames of *StatementList*.
2. Return a new empty [List](#).

DefaultClause : **default** : *StatementList*

1. If the *StatementList* is present, return the VarDeclaredNames of *StatementList*.
2. Return a new empty [List](#).

13.12.8 Static Semantics: VarScopedDeclarations

SwitchStatement : **switch** (*Expression*) *CaseBlock*

1. Return the VarScopedDeclarations of *CaseBlock*.

CaseBlock : { }

1. Return a new empty [List](#).

CaseBlock : { *CaseClauses* *DefaultClause* *CaseClauses* }

1. If the first *CaseClauses* is present, let *declarations* be the VarScopedDeclarations of the first *CaseClauses*.
2. Else, let *declarations* be a new empty [List](#).
3. Append to *declarations* the elements of the VarScopedDeclarations of the *DefaultClause*.
4. If the second *CaseClauses* is not present, return *declarations*.
5. Return the result of appending to *declarations* the elements of the VarScopedDeclarations of the second *CaseClauses*.

CaseClauses : *CaseClauses* *CaseClause*

1. Let *declarations* be VarScopedDeclarations of *CaseClauses*.
2. Append to *declarations* the elements of the VarScopedDeclarations of *CaseClause*.
3. Return *declarations*.

CaseClause : **case** *Expression* : *StatementList*

1. If the *StatementList* is present, return the VarScopedDeclarations of *StatementList*.
2. Return a new empty [List](#).

DefaultClause : **default** : *StatementList*

1. If the *StatementList* is present, return the VarScopedDeclarations of *StatementList*.
2. Return a new empty [List](#).

13.12.9 Runtime Semantics: CaseBlockEvaluation

With parameter *input*.

CaseBlock : { }

1. Return [NormalCompletion](#)(**undefined**).

CaseBlock : { *CaseClauses* }

1. Let *V* be **undefined**.
2. Let *A* be the [List](#) of *CaseClause* items in *CaseClauses*, in source text order.
3. Let *found* be **false**.
4. For each *CaseClause* *C* in *A*, do
 - a. If *found* is **false**, then
 - i. Set *found* to ? [CaseClauseIsSelected](#)(*C*, *input*).
 - b. If *found* is **true**, then
 - i. Let *R* be the result of evaluating *C*.
 - ii. If *R*.[[Value]] is not **empty**, set *V* to *R*.[[Value]].
 - iii. If *R* is an [abrupt completion](#), return [Completion](#)([UpdateEmpty](#)(*R*, *V*)).
5. Return [NormalCompletion](#)(*V*).

CaseBlock : { *CaseClauses* *DefaultClause* *CaseClauses* }

1. Let *V* be **undefined**.
2. If the first *CaseClauses* is present, then
 - a. Let *A* be the [List](#) of *CaseClause* items in the first *CaseClauses*, in source text order.
3. Else,
 - a. Let *A* be « ».
4. Let *found* be **false**.
5. For each *CaseClause* *C* in *A*, do
 - a. If *found* is **false**, then
 - i. Set *found* to ? [CaseClauseIsSelected](#)(*C*, *input*).
 - b. If *found* is **true**, then
 - i. Let *R* be the result of evaluating *C*.
 - ii. If *R*.[[Value]] is not **empty**, set *V* to *R*.[[Value]].
 - iii. If *R* is an [abrupt completion](#), return [Completion](#)([UpdateEmpty](#)(*R*, *V*)).
6. Let *foundInB* be **false**.
7. If the second *CaseClauses* is present, then
 - a. Let *B* be the [List](#) of *CaseClause* items in the second *CaseClauses*, in source text order.
8. Else,
 - a. Let *B* be « ».

9. If *found* is **false**, then
 - a. For each *CaseClause* *C* in *B*, do
 - i. If *foundInB* is **false**, then
 1. Set *foundInB* to ? *CaseClauseIsSelected*(*C*, *input*).
 - ii. If *foundInB* is **true**, then
 1. Let *R* be the result of evaluating *CaseClause* *C*.
 2. If *R*.[[Value]] is not **empty**, set *V* to *R*.[[Value]].
 3. If *R* is an **abrupt completion**, return *Completion*(*UpdateEmpty*(*R*, *V*)).
10. If *foundInB* is **true**, return *NormalCompletion*(*V*).
11. Let *R* be the result of evaluating *DefaultClause*.
12. If *R*.[[Value]] is not **empty**, set *V* to *R*.[[Value]].
13. If *R* is an **abrupt completion**, return *Completion*(*UpdateEmpty*(*R*, *V*)).
14. For each *CaseClause* *C* in *B* (NOTE: this is another complete iteration of the second *CaseClauses*), do
 - a. Let *R* be the result of evaluating *CaseClause* *C*.
 - b. If *R*.[[Value]] is not **empty**, set *V* to *R*.[[Value]].
 - c. If *R* is an **abrupt completion**, return *Completion*(*UpdateEmpty*(*R*, *V*)).
15. Return *NormalCompletion*(*V*).

13.12.10 Runtime Semantics: *CaseClauseIsSelected* (*C*, *input*)

The abstract operation *CaseClauseIsSelected*, given *CaseClause* *C* and value *input*, determines whether *C* matches *input*.

1. **Assert**: *C* is an instance of the production *CaseClause* : **case** *Expression* : *StatementList* .
2. Let *exprRef* be the result of evaluating the *Expression* of *C*.
3. Let *clauseSelector* be ? *GetValue*(*exprRef*).
4. Return the result of performing *Strict Equality Comparison* *input* === *clauseSelector*.

NOTE

This operation does not execute *C*'s *StatementList* (if any). The *CaseBlock* algorithm uses its return value to determine which *StatementList* to start executing.

13.12.11 Runtime Semantics: Evaluation

SwitchStatement : **switch** (*Expression*) *CaseBlock*

1. Let *exprRef* be the result of evaluating *Expression*.
2. Let *switchValue* be ? *GetValue*(*exprRef*).
3. Let *oldEnv* be the **running execution context**'s *LexicalEnvironment*.
4. Let *blockEnv* be *NewDeclarativeEnvironment*(*oldEnv*).
5. Perform *BlockDeclarationInstantiation*(*CaseBlock*, *blockEnv*).
6. Set the **running execution context**'s *LexicalEnvironment* to *blockEnv*.
7. Let *R* be the result of performing *CaseBlockEvaluation* of *CaseBlock* with argument *switchValue*.
8. Set the **running execution context**'s *LexicalEnvironment* to *oldEnv*.
9. Return *R*.

NOTE

No matter how control leaves the *SwitchStatement* the *LexicalEnvironment* is always restored to its former state.

CaseClause : **case** *Expression* :

1. Return [NormalCompletion\(empty\)](#).

CaseClause : **case** *Expression* : *StatementList*

1. Return the result of evaluating *StatementList*.

DefaultClause : **default** :

1. Return [NormalCompletion\(empty\)](#).

DefaultClause : **default** : *StatementList*

1. Return the result of evaluating *StatementList*.

13.13 Labelled Statements

Syntax

```

LabelledStatement[Yield, Await, Return] :
    LabelIdentifier[?Yield, ?Await] : LabelledItem[?Yield, ?Await, ?Return]

LabelledItem[Yield, Await, Return] :
    Statement[?Yield, ?Await, ?Return]
    FunctionDeclaration[?Yield, ?Await, ~Default]

```

NOTE

A *Statement* may be prefixed by a label. Labelled statements are only used in conjunction with labelled **break** and **continue** statements. ECMAScript has no **goto** statement. A *Statement* can be part of a *LabelledStatement*, which itself can be part of a *LabelledStatement*, and so on. The labels introduced this way are collectively referred to as the “current label set” when describing the semantics of individual statements.

13.13.1 Static Semantics: Early Errors

LabelledItem : *FunctionDeclaration*

It is a Syntax Error if any source text matches this rule.

NOTE

An alternative definition for this rule is provided in [B.3.2](#).

13.13.2 Static Semantics: ContainsDuplicateLabels

With parameter *labelSet*.

LabelledStatement : *LabelIdentifier* : *LabelledItem*

1. Let *label* be the StringValue of *LabelIdentifier*.
2. If *label* is an element of *labelSet*, return **true**.
3. Let *newLabelSet* be a copy of *labelSet* with *label* appended.

4. Return ContainsDuplicateLabels of *LabelledItem* with argument *newLabelSet*.

LabelledItem : *FunctionDeclaration*

1. Return **false**.

13.13.3 Static Semantics: ContainsUndefinedBreakTarget

With parameter *labelSet*.

LabelledStatement : *LabelIdentifier* : *LabelledItem*

1. Let *label* be the StringValue of *LabelIdentifier*.
2. Let *newLabelSet* be a copy of *labelSet* with *label* appended.
3. Return ContainsUndefinedBreakTarget of *LabelledItem* with argument *newLabelSet*.

LabelledItem : *FunctionDeclaration*

1. Return **false**.

13.13.4 Static Semantics: ContainsUndefinedContinueTarget

With parameters *iterationSet* and *labelSet*.

LabelledStatement : *LabelIdentifier* : *LabelledItem*

1. Let *label* be the StringValue of *LabelIdentifier*.
2. Let *newLabelSet* be a copy of *labelSet* with *label* appended.
3. Return ContainsUndefinedContinueTarget of *LabelledItem* with arguments *iterationSet* and *newLabelSet*.

LabelledItem : *FunctionDeclaration*

1. Return **false**.

13.13.5 Static Semantics: IsLabelledFunction (*stmt*)

The abstract operation IsLabelledFunction with argument *stmt* performs the following steps:

1. If *stmt* is not a *LabelledStatement*, return **false**.
2. Let *item* be the *LabelledItem* of *stmt*.
3. If *item* is *LabelledItem* : *FunctionDeclaration* , return **true**.
4. Let *subStmt* be the *Statement* of *item*.
5. Return IsLabelledFunction(*subStmt*).

13.13.6 Static Semantics: LexicallyDeclaredNames

LabelledStatement : *LabelIdentifier* : *LabelledItem*

1. Return the LexicallyDeclaredNames of *LabelledItem*.

LabelledItem : *Statement*

1. Return a new empty *List*.

LabelledItem : *FunctionDeclaration*

1. Return BoundNames of *FunctionDeclaration*.

13.13.7 Static Semantics: LexicallyScopedDeclarations

LabelledStatement : *LabelIdentifier* : *LabelledItem*

1. Return the LexicallyScopedDeclarations of *LabelledItem*.

LabelledItem : *Statement*

1. Return a new empty [List](#).

LabelledItem : *FunctionDeclaration*

1. Return a new [List](#) containing *FunctionDeclaration*.

13.13.8 Static Semantics: TopLevelLexicallyDeclaredNames

LabelledStatement : *LabelIdentifier* : *LabelledItem*

1. Return a new empty [List](#).

13.13.9 Static Semantics: TopLevelLexicallyScopedDeclarations

LabelledStatement : *LabelIdentifier* : *LabelledItem*

1. Return a new empty [List](#).

13.13.10 Static Semantics: TopLevelVarDeclaredNames

LabelledStatement : *LabelIdentifier* : *LabelledItem*

1. Return the TopLevelVarDeclaredNames of *LabelledItem*.

LabelledItem : *Statement*

1. If *Statement* is *Statement* : *LabelledStatement* , return TopLevelVarDeclaredNames of *Statement*.
2. Return VarDeclaredNames of *Statement*.

LabelledItem : *FunctionDeclaration*

1. Return BoundNames of *FunctionDeclaration*.

13.13.11 Static Semantics: TopLevelVarScopedDeclarations

LabelledStatement : *LabelIdentifier* : *LabelledItem*

1. Return the TopLevelVarScopedDeclarations of *LabelledItem*.

LabelledItem : *Statement*

1. If *Statement* is *Statement* : *LabelledStatement* , return TopLevelVarScopedDeclarations of *Statement*.
2. Return VarScopedDeclarations of *Statement*.

LabelledItem : *FunctionDeclaration*

1. Return a new [List](#) containing *FunctionDeclaration*.

13.13.12 Static Semantics: VarDeclaredNames

LabelledStatement : *LabelIdentifier* : *LabelledItem*

1. Return the VarDeclaredNames of *LabelledItem*.

LabelledItem : *FunctionDeclaration*

1. Return a new empty [List](#).

13.13.13 Static Semantics: VarScopedDeclarations

LabelledStatement : *LabelIdentifier* : *LabelledItem*

1. Return the VarScopedDeclarations of *LabelledItem*.

LabelledItem : *FunctionDeclaration*

1. Return a new empty [List](#).

13.13.14 Runtime Semantics: LabelledEvaluation

With parameter [labelSet](#).

LabelledStatement : *LabelIdentifier* : *LabelledItem*

1. Let [label](#) be the StringValue of *LabelIdentifier*.
2. Append [label](#) as an element of [labelSet](#).
3. Let [stmtResult](#) be LabelledEvaluation of *LabelledItem* with argument [labelSet](#).
4. If [stmtResult](#).[[Type]] is **break** and [SameValue](#)([stmtResult](#).[[Target]], [label](#)) is **true**, then
 - a. Set [stmtResult](#) to [NormalCompletion](#)([stmtResult](#).[[Value]]).
5. Return [Completion](#)([stmtResult](#)).

LabelledItem : *Statement*

1. If *Statement* is either a *LabelledStatement* or a *BreakableStatement*, then
 - a. Return LabelledEvaluation of *Statement* with argument [labelSet](#).
2. Else,
 - a. Return the result of evaluating *Statement*.

LabelledItem : *FunctionDeclaration*

1. Return the result of evaluating *FunctionDeclaration*.

13.13.15 Runtime Semantics: Evaluation

LabelledStatement : *LabelIdentifier* : *LabelledItem*

1. Let [newLabelSet](#) be a new empty [List](#).
2. Return LabelledEvaluation of this *LabelledStatement* with argument [newLabelSet](#).

13.14 The **throw** Statement

Syntax

```
ThrowStatement[Yield, Await] :  
    throw [no LineTerminator here] Expression[+In, ?Yield, ?Await] ;
```

13.14.1 Runtime Semantics: Evaluation

ThrowStatement : **throw** *Expression* ;

1. Let *exprRef* be the result of evaluating *Expression*.
2. Let *exprValue* be ? *GetValue*(*exprRef*).
3. Return *ThrowCompletion*(*exprValue*).

13.15 The **try** Statement

Syntax

```
TryStatement[Yield, Await, Return] :  
    try Block[?Yield, ?Await, ?Return] Catch[?Yield, ?Await, ?Return]  
    try Block[?Yield, ?Await, ?Return] Finally[?Yield, ?Await, ?Return]  
    try Block[?Yield, ?Await, ?Return] Catch[?Yield, ?Await, ?Return]  
        Finally[?Yield, ?Await, ?Return]  
  
Catch[Yield, Await, Return] :  
    catch ( CatchParameter[?Yield, ?Await] ) Block[?Yield, ?Await, ?Return]  
    catch Block[?Yield, ?Await, ?Return]  
  
Finally[Yield, Await, Return] :  
    finally Block[?Yield, ?Await, ?Return]  
  
CatchParameter[Yield, Await] :  
    BindingIdentifier[?Yield, ?Await]  
    BindingPattern[?Yield, ?Await]
```

NOTE

The **try** statement encloses a block of code in which an exceptional condition can occur, such as a runtime error or a **throw** statement. The **catch** clause provides the exception-handling code. When a catch clause catches an exception, its *CatchParameter* is bound to that exception.

13.15.1 Static Semantics: Early Errors

Catch : **catch** (*CatchParameter*) *Block*

It is a Syntax Error if BoundNames of *CatchParameter* contains any duplicate elements.

It is a Syntax Error if any element of the BoundNames of *CatchParameter* also occurs in the LexicallyDeclaredNames of *Block*.

It is a Syntax Error if any element of the BoundNames of *CatchParameter* also occurs in the VarDeclaredNames of *Block*.

NOTE

An alternative [static semantics](#) for this production is given in [B.3.5](#).

13.15.2 Static Semantics: ContainsDuplicateLabels

With parameter *labelSet*.

TryStatement : **try** *Block* *Catch*

1. Let *hasDuplicates* be ContainsDuplicateLabels of *Block* with argument *labelSet*.
2. If *hasDuplicates* is **true**, return **true**.
3. Return ContainsDuplicateLabels of *Catch* with argument *labelSet*.

TryStatement : **try** *Block* *Finally*

1. Let *hasDuplicates* be ContainsDuplicateLabels of *Block* with argument *labelSet*.
2. If *hasDuplicates* is **true**, return **true**.
3. Return ContainsDuplicateLabels of *Finally* with argument *labelSet*.

TryStatement : **try** *Block* *Catch* *Finally*

1. Let *hasDuplicates* be ContainsDuplicateLabels of *Block* with argument *labelSet*.
2. If *hasDuplicates* is **true**, return **true**.
3. Let *hasDuplicates* be ContainsDuplicateLabels of *Catch* with argument *labelSet*.
4. If *hasDuplicates* is **true**, return **true**.
5. Return ContainsDuplicateLabels of *Finally* with argument *labelSet*.

Catch : **catch** (*CatchParameter*) *Block*

1. Return ContainsDuplicateLabels of *Block* with argument *labelSet*.

13.15.3 Static Semantics: ContainsUndefinedBreakTarget

With parameter *labelSet*.

TryStatement : **try** *Block* *Catch*

1. Let *hasUndefinedLabels* be ContainsUndefinedBreakTarget of *Block* with argument *labelSet*.
2. If *hasUndefinedLabels* is **true**, return **true**.
3. Return ContainsUndefinedBreakTarget of *Catch* with argument *labelSet*.

TryStatement : **try** *Block* *Finally*

1. Let *hasUndefinedLabels* be ContainsUndefinedBreakTarget of *Block* with argument *labelSet*.
2. If *hasUndefinedLabels* is **true**, return **true**.
3. Return ContainsUndefinedBreakTarget of *Finally* with argument *labelSet*.

TryStatement : **try** *Block* *Catch* *Finally*

1. Let *hasUndefinedLabels* be ContainsUndefinedBreakTarget of *Block* with argument *labelSet*.
2. If *hasUndefinedLabels* is **true**, return **true**.
3. Let *hasUndefinedLabels* be ContainsUndefinedBreakTarget of *Catch* with argument *labelSet*.
4. If *hasUndefinedLabels* is **true**, return **true**.
5. Return ContainsUndefinedBreakTarget of *Finally* with argument *labelSet*.

Catch : **catch** (*CatchParameter*) *Block*

1. Return ContainsUndefinedBreakTarget of *Block* with argument *labelSet*.

13.15.4 Static Semantics: ContainsUndefinedContinueTarget

With parameters *iterationSet* and *labelSet*.

TryStatement : **try** *Block* *Catch*

1. Let *hasUndefinedLabels* be ContainsUndefinedContinueTarget of *Block* with arguments *iterationSet* and « ».
2. If *hasUndefinedLabels* is **true**, return **true**.
3. Return ContainsUndefinedContinueTarget of *Catch* with arguments *iterationSet* and « ».

TryStatement : **try** *Block* *Finally*

1. Let *hasUndefinedLabels* be ContainsUndefinedContinueTarget of *Block* with arguments *iterationSet* and « ».
2. If *hasUndefinedLabels* is **true**, return **true**.
3. Return ContainsUndefinedContinueTarget of *Finally* with arguments *iterationSet* and « ».

TryStatement : **try** *Block* *Catch* *Finally*

1. Let *hasUndefinedLabels* be ContainsUndefinedContinueTarget of *Block* with arguments *iterationSet* and « ».
2. If *hasUndefinedLabels* is **true**, return **true**.
3. Let *hasUndefinedLabels* be ContainsUndefinedContinueTarget of *Catch* with arguments *iterationSet* and « ».
4. If *hasUndefinedLabels* is **true**, return **true**.
5. Return ContainsUndefinedContinueTarget of *Finally* with arguments *iterationSet* and « ».

Catch : **catch** (*CatchParameter*) *Block*

1. Return ContainsUndefinedContinueTarget of *Block* with arguments *iterationSet* and « ».

13.15.5 Static Semantics: VarDeclaredNames

TryStatement : **try** *Block* *Catch*

1. Let *names* be VarDeclaredNames of *Block*.
2. Append to *names* the elements of the VarDeclaredNames of *Catch*.
3. Return *names*.

TryStatement : **try** *Block* *Finally*

1. Let *names* be VarDeclaredNames of *Block*.
2. Append to *names* the elements of the VarDeclaredNames of *Finally*.
3. Return *names*.

TryStatement : **try** *Block* *Catch* *Finally*

1. Let *names* be VarDeclaredNames of *Block*.
2. Append to *names* the elements of the VarDeclaredNames of *Catch*.
3. Append to *names* the elements of the VarDeclaredNames of *Finally*.
4. Return *names*.

Catch : **catch** (*CatchParameter*) *Block*

1. Return the VarDeclaredNames of *Block*.

13.15.6 Static Semantics: VarScopedDeclarations

TryStatement : **try** *Block Catch*

1. Let *declarations* be VarScopedDeclarations of *Block*.
2. Append to *declarations* the elements of the VarScopedDeclarations of *Catch*.
3. Return *declarations*.

TryStatement : **try** *Block Finally*

1. Let *declarations* be VarScopedDeclarations of *Block*.
2. Append to *declarations* the elements of the VarScopedDeclarations of *Finally*.
3. Return *declarations*.

TryStatement : **try** *Block Catch Finally*

1. Let *declarations* be VarScopedDeclarations of *Block*.
2. Append to *declarations* the elements of the VarScopedDeclarations of *Catch*.
3. Append to *declarations* the elements of the VarScopedDeclarations of *Finally*.
4. Return *declarations*.

Catch : **catch** (*CatchParameter*) *Block*

1. Return the VarScopedDeclarations of *Block*.

13.15.7 Runtime Semantics: CatchClauseEvaluation

With parameter *thrownValue*.

Catch : **catch** (*CatchParameter*) *Block*

1. Let *oldEnv* be the running execution context's LexicalEnvironment.
2. Let *catchEnv* be NewDeclarativeEnvironment(*oldEnv*).
3. Let *catchEnvRec* be *catchEnv*'s EnvironmentRecord.
4. For each element *argName* of the BoundNames of *CatchParameter*, do
 - a. Perform ! *catchEnvRec*.CreateMutableBinding(*argName*, **false**).
5. Set the running execution context's LexicalEnvironment to *catchEnv*.
6. Let *status* be the result of performing BindingInitialization for *CatchParameter* passing *thrownValue* and *catchEnv* as arguments.
7. If *status* is an abrupt completion, then
 - a. Set the running execution context's LexicalEnvironment to *oldEnv*.
 - b. Return Completion(*status*).
8. Let *B* be the result of evaluating *Block*.

9. Set the [running execution context](#)'s LexicalEnvironment to *oldEnv*.
10. Return [Completion](#)(*B*).

Catch : **catch** *Block*

1. Return the result of evaluating *Block*.

NOTE

No matter how control leaves the *Block* the LexicalEnvironment is always restored to its former state.

13.15.8 Runtime Semantics: Evaluation

TryStatement : **try** *Block* *Catch*

1. Let *B* be the result of evaluating *Block*.
2. If *B*.[[Type]] is throw, let *C* be CatchClauseEvaluation of *Catch* with argument *B*.[[Value]].
3. Else, let *C* be *B*.
4. Return [Completion](#)([UpdateEmpty](#)(*C*, **undefined**)).

TryStatement : **try** *Block* *Finally*

1. Let *B* be the result of evaluating *Block*.
2. Let *F* be the result of evaluating *Finally*.
3. If *F*.[[Type]] is normal, set *F* to *B*.
4. Return [Completion](#)([UpdateEmpty](#)(*F*, **undefined**)).

TryStatement : **try** *Block* *Catch* *Finally*

1. Let *B* be the result of evaluating *Block*.
2. If *B*.[[Type]] is throw, let *C* be CatchClauseEvaluation of *Catch* with argument *B*.[[Value]].
3. Else, let *C* be *B*.
4. Let *F* be the result of evaluating *Finally*.
5. If *F*.[[Type]] is normal, set *F* to *C*.
6. Return [Completion](#)([UpdateEmpty](#)(*F*, **undefined**)).

13.16 The debugger Statement

Syntax

DebuggerStatement :
debugger ;

13.16.1 Runtime Semantics: Evaluation

NOTE

Evaluating a *DebuggerStatement* may allow an implementation to cause a breakpoint when run under a debugger. If a debugger is not present or active this statement has no observable effect.

DebuggerStatement : **debugger** ;

1. If an implementation-defined debugging facility is available and enabled, then
 - a. Perform an implementation-defined debugging action.
 - b. Let *result* be an implementation-defined **Completion** value.
2. Else,
 - a. Let *result* be **NormalCompletion**(empty).
3. Return *result*.

14 ECMAScript Language: Functions and Classes

NOTE

Various ECMAScript language elements cause the creation of ECMAScript function objects (9.2). Evaluation of such functions starts with the execution of their **[[Call]]** internal method (9.2.1).

14.1 Function Definitions

Syntax

```

FunctionDeclaration[Yield, Await, Default] :
    function BindingIdentifier[?Yield, ?Await] ( FormalParameters[~Yield, ~Await] ) {
        FunctionBody[~Yield, ~Await] }
    [+Default] function ( FormalParameters[~Yield, ~Await] ) { FunctionBody[~Yield, ~Await] }

FunctionExpression :
    function BindingIdentifier[~Yield, ~Await] opt ( FormalParameters[~Yield, ~Await] ) {
        FunctionBody[~Yield, ~Await] }

UniqueFormalParameters[Yield, Await] :
    FormalParameters[?Yield, ?Await]

FormalParameters[Yield, Await] :
    [empty]
    FunctionRestParameter[?Yield, ?Await]
    FormalParameterList[?Yield, ?Await]
    FormalParameterList[?Yield, ?Await] ,
    FormalParameterList[?Yield, ?Await] , FunctionRestParameter[?Yield, ?Await]

FormalParameterList[Yield, Await] :
    FormalParameter[?Yield, ?Await]
    FormalParameterList[?Yield, ?Await] , FormalParameter[?Yield, ?Await]

FunctionRestParameter[Yield, Await] :
    BindingRestElement[?Yield, ?Await]

FormalParameter[Yield, Await] :
    BindingElement[?Yield, ?Await]

```

```

FunctionBody[Yield, Await] :
    FunctionStatementList[?Yield, ?Await]

FunctionStatementList[Yield, Await] :
    StatementList[?Yield, ?Await, +Return] opt

```

14.1.1 Directive Prologues and the Use Strict Directive

A Directive Prologue is the longest sequence of *ExpressionStatements* occurring as the initial *StatementListItems* or *ModuleItems* of a *FunctionBody*, a *ScriptBody*, or a *ModuleBody* and where each *ExpressionStatement* in the sequence consists entirely of a *StringLiteral* token followed by a semicolon. The semicolon may appear explicitly or may be inserted by automatic semicolon insertion. A [Directive Prologue](#) may be an empty sequence.

A Use Strict Directive is an *ExpressionStatement* in a [Directive Prologue](#) whose *StringLiteral* is either the exact code unit sequences **"use strict"** or **'use strict'**. A [Use Strict Directive](#) may not contain an *EscapeSequence* or *LineContinuation*.

A [Directive Prologue](#) may contain more than one [Use Strict Directive](#). However, an implementation may issue a warning if this occurs.

NOTE

The *ExpressionStatements* of a [Directive Prologue](#) are evaluated normally during evaluation of the containing production. Implementations may define implementation specific meanings for *ExpressionStatements* which are not a [Use Strict Directive](#) and which occur in a [Directive Prologue](#). If an appropriate notification mechanism exists, an implementation should issue a warning if it encounters in a [Directive Prologue](#) an *ExpressionStatement* that is not a [Use Strict Directive](#) and which does not have a meaning defined by the implementation.

14.1.2 Static Semantics: Early Errors

```

FunctionDeclaration : function BindingIdentifier ( FormalParameters ) { FunctionBody }
FunctionDeclaration : function ( FormalParameters ) { FunctionBody }
FunctionExpression : function BindingIdentifier ( FormalParameters ) { FunctionBody }

```

If the source code matching this production is [strict mode code](#), the Early Error rules for *UniqueFormalParameters* : *FormalParameters* are applied.

If the source code matching this production is [strict mode code](#), it is a Syntax Error if *BindingIdentifier* is present and the *StringValue* of *BindingIdentifier* is **"eval"** or **"arguments"**.

It is a Syntax Error if *ContainsUseStrict* of *FunctionBody* is **true** and *IsSimpleParameterList* of *FormalParameters* is **false**.

It is a Syntax Error if any element of the *BoundNames* of *FormalParameters* also occurs in the *LexicallyDeclaredNames* of *FunctionBody*.

It is a Syntax Error if *FormalParameters* *Contains SuperProperty* is **true**.

It is a Syntax Error if *FunctionBody* *Contains SuperProperty* is **true**.

It is a Syntax Error if *FormalParameters* *Contains SuperCall* is **true**.

It is a Syntax Error if *FunctionBody* *Contains SuperCall* is **true**.

NOTE 1

The *LexicallyDeclaredNames* of a *FunctionBody* does not include identifiers bound using *var* or function declarations.

UniqueFormalParameters : *FormalParameters*

It is a Syntax Error if *BoundNames* of *FormalParameters* contains any duplicate elements.

FormalParameters : *FormalParameterList*

It is a Syntax Error if *IsSimpleParameterList* of *FormalParameterList* is **false** and *BoundNames* of *FormalParameterList* contains any duplicate elements.

NOTE 2

Multiple occurrences of the same *BindingIdentifier* in a *FormalParameterList* is only allowed for functions which have simple parameter lists and which are not defined in [strict mode code](#).

FunctionBody : *FunctionStatementList*

It is a Syntax Error if the *LexicallyDeclaredNames* of *FunctionStatementList* contains any duplicate entries.

It is a Syntax Error if any element of the *LexicallyDeclaredNames* of *FunctionStatementList* also occurs in the *VarDeclaredNames* of *FunctionStatementList*.

It is a Syntax Error if *ContainsDuplicateLabels* of *FunctionStatementList* with argument « » is **true**.

It is a Syntax Error if *ContainsUndefinedBreakTarget* of *FunctionStatementList* with argument « » is **true**.

It is a Syntax Error if *ContainsUndefinedContinueTarget* of *FunctionStatementList* with arguments « » and « » is **true**.

14.1.3 Static Semantics: BoundNames

FunctionDeclaration : **function** *BindingIdentifier* (*FormalParameters*) { *FunctionBody* }

1. Return the *BoundNames* of *BindingIdentifier*.

FunctionDeclaration : **function** (*FormalParameters*) { *FunctionBody* }

1. Return « **"*default*"** ».

NOTE

"*default*" is used within this specification as a synthetic name for hoistable anonymous functions that are defined using export declarations.

FormalParameters : [empty]

1. Return a new empty [List](#).

FormalParameters : *FormalParameterList* , *FunctionRestParameter*

1. Let *names* be *BoundNames* of *FormalParameterList*.
2. Append to *names* the *BoundNames* of *FunctionRestParameter*.
3. Return *names*.

FormalParameterList : *FormalParameterList* , *FormalParameter*

1. Let *names* be *BoundNames* of *FormalParameterList*.
2. Append to *names* the *BoundNames* of *FormalParameter*.
3. Return *names*.

14.1.4 Static Semantics: Contains

With parameter *symbol*.

FunctionDeclaration : **function** *BindingIdentifier* (*FormalParameters*) { *FunctionBody* }

FunctionDeclaration : **function** (*FormalParameters*) { *FunctionBody* }

FunctionExpression : **function** *BindingIdentifier* (*FormalParameters*) { *FunctionBody* }

1. Return **false**.

NOTE

Static semantic rules that depend upon substructure generally do not look into function definitions.

14.1.5 Static Semantics: ContainsExpression

FormalParameters : [empty]

1. Return **false**.

FormalParameters : *FormalParameterList* , *FunctionRestParameter*

1. If ContainsExpression of *FormalParameterList* is **true**, return **true**.
2. Return ContainsExpression of *FunctionRestParameter*.

FormalParameterList : *FormalParameterList* , *FormalParameter*

1. If ContainsExpression of *FormalParameterList* is **true**, return **true**.
2. Return ContainsExpression of *FormalParameter*.

14.1.6 Static Semantics: ContainsUseStrict

FunctionBody : *FunctionStatementList*

1. If the [Directive Prologue](#) of *FunctionStatementList* contains a [Use Strict Directive](#), return **true**; otherwise, return **false**.

14.1.7 Static Semantics: ExpectedArgumentCount

FormalParameters : [empty]

1. Return 0.

FormalParameters : *FormalParameterList* , *FunctionRestParameter*

1. Return ExpectedArgumentCount of *FormalParameterList*.

NOTE

The ExpectedArgumentCount of a *FormalParameterList* is the number of *FormalParameters* to the left of either the rest parameter or the first *FormalParameter* with an Initializer. A *FormalParameter* without an initializer is allowed after the first parameter with an initializer but such parameters are considered to be optional with **undefined** as their default value.

FormalParameterList : *FormalParameterList* , *FormalParameter*

1. Let *count* be ExpectedArgumentCount of *FormalParameterList*.
2. If HasInitializer of *FormalParameterList* is **true** or HasInitializer of *FormalParameter* is **true**, return *count*.
3. Return *count* + 1.

14.1.8 Static Semantics: HasInitializer

FormalParameterList : *FormalParameterList* , *FormalParameter*

1. If HasInitializer of *FormalParameterList* is **true**, return **true**.
2. Return HasInitializer of *FormalParameter*.

14.1.9 Static Semantics: HasName

FunctionExpression : **function** (*FormalParameters*) { *FunctionBody* }

1. Return **false**.

FunctionExpression : **function** *BindingIdentifier* (*FormalParameters*) { *FunctionBody* }

1. Return **true**.

14.1.10 Static Semantics: IsAnonymousFunctionDefinition (*expr*)

The abstract operation IsAnonymousFunctionDefinition determines if its argument is a function definition that does not bind a name. The argument *expr* is the result of parsing an *AssignmentExpression* or *Initializer*. The following steps are taken:

1. If IsFunctionDefinition of *expr* is **false**, return **false**.
2. Let *hasName* be the result of HasName of *expr*.
3. If *hasName* is **true**, return **false**.
4. Return **true**.

14.1.11 Static Semantics: IsConstantDeclaration

FunctionDeclaration : **function** *BindingIdentifier* (*FormalParameters*) { *FunctionBody* }

FunctionDeclaration : **function** (*FormalParameters*) { *FunctionBody* }

1. Return **false**.

14.1.12 Static Semantics: IsFunctionDefinition

FunctionExpression : **function** *BindingIdentifier* (*FormalParameters*) { *FunctionBody* }

1. Return **true**.

14.1.13 Static Semantics: IsSimpleParameterList

FormalParameters : [empty]

1. Return **true**.

FormalParameters : *FormalParameterList* , *FunctionRestParameter*

1. Return **false**.

FormalParameterList : *FormalParameterList* , *FormalParameter*

1. If *IsSimpleParameterList* of *FormalParameterList* is **false**, return **false**.
2. Return *IsSimpleParameterList* of *FormalParameter*.

FormalParameter : *BindingElement*

1. Return *IsSimpleParameterList* of *BindingElement*.

14.1.14 Static Semantics: LexicallyDeclaredNames

FunctionStatementList : [empty]

1. Return a new empty [List](#).

FunctionStatementList : *StatementList*

1. Return *TopLevelLexicallyDeclaredNames* of *StatementList*.

14.1.15 Static Semantics: LexicallyScopedDeclarations

FunctionStatementList : [empty]

1. Return a new empty [List](#).

FunctionStatementList : *StatementList*

1. Return the *TopLevelLexicallyScopedDeclarations* of *StatementList*.

14.1.16 Static Semantics: VarDeclaredNames

FunctionStatementList : [empty]

1. Return a new empty [List](#).

FunctionStatementList : *StatementList*

1. Return *TopLevelVarDeclaredNames* of *StatementList*.

14.1.17 Static Semantics: VarScopedDeclarations

FunctionStatementList : [empty]

1. Return a new empty [List](#).

FunctionStatementList : *StatementList*

1. Return the *TopLevelVarScopedDeclarations* of *StatementList*.

14.1.18 Runtime Semantics: EvaluateBody

With parameters [functionObject](#) and [List argumentsList](#).

FunctionBody : *FunctionStatementList*

1. Perform ? [FunctionDeclarationInstantiation](#)(*functionObject*, *argumentsList*).
2. Return the result of evaluating *FunctionStatementList*.

14.1.19 Runtime Semantics: [IteratorBindingInitialization](#)

With parameters *iteratorRecord* and *environment*.

NOTE 1

When **undefined** is passed for *environment* it indicates that a [PutValue](#) operation should be used to assign the initialization value. This is the case for formal parameter lists of non-strict functions. In that case the formal parameter bindings are preinitialized in order to deal with the possibility of multiple parameters with the same name.

FormalParameters : [empty]

1. Return [NormalCompletion](#)(empty).

FormalParameters : *FormalParameterList* , *FunctionRestParameter*

1. Perform ? [IteratorBindingInitialization](#) for *FormalParameterList* using *iteratorRecord* and *environment* as the arguments.
2. Return the result of performing [IteratorBindingInitialization](#) for *FunctionRestParameter* using *iteratorRecord* and *environment* as the arguments.

FormalParameterList : *FormalParameterList* , *FormalParameter*

1. Perform ? [IteratorBindingInitialization](#) for *FormalParameterList* using *iteratorRecord* and *environment* as the arguments.
2. Return the result of performing [IteratorBindingInitialization](#) for *FormalParameter* using *iteratorRecord* and *environment* as the arguments.

FormalParameter : *BindingElement*

1. If *ContainsExpression* of *BindingElement* is **false**, return the result of performing [IteratorBindingInitialization](#) for *BindingElement* using *iteratorRecord* and *environment* as the arguments.
2. Let *currentContext* be the [running execution context](#).
3. Let *originalEnv* be the [VariableEnvironment](#) of *currentContext*.
4. **Assert**: The [VariableEnvironment](#) and [LexicalEnvironment](#) of *currentContext* are the same.
5. **Assert**: *environment* and *originalEnv* are the same.
6. Let *paramVarEnv* be [NewDeclarativeEnvironment](#)(*originalEnv*).
7. Set the [VariableEnvironment](#) of *currentContext* to *paramVarEnv*.
8. Set the [LexicalEnvironment](#) of *currentContext* to *paramVarEnv*.
9. Let *result* be the result of performing [IteratorBindingInitialization](#) for *BindingElement* using *iteratorRecord* and *environment* as the arguments.
10. Set the [VariableEnvironment](#) of *currentContext* to *originalEnv*.
11. Set the [LexicalEnvironment](#) of *currentContext* to *originalEnv*.
12. Return *result*.

NOTE 2

The new [Environment Record](#) created in step 6 is only used if the *BindingElement* contains a [direct eval](#).

FunctionRestParameter : *BindingRestElement*

1. If ContainsExpression of *BindingRestElement* is **false**, return the result of performing IteratorBindingInitialization for *BindingRestElement* using *iteratorRecord* and *environment* as the arguments.
2. Let *currentContext* be the running execution context.
3. Let *originalEnv* be the VariableEnvironment of *currentContext*.
4. **Assert**: The VariableEnvironment and LexicalEnvironment of *currentContext* are the same.
5. **Assert**: *environment* and *originalEnv* are the same.
6. Let *paramVarEnv* be NewDeclarativeEnvironment(*originalEnv*).
7. Set the VariableEnvironment of *currentContext* to *paramVarEnv*.
8. Set the LexicalEnvironment of *currentContext* to *paramVarEnv*.
9. Let *result* be the result of performing IteratorBindingInitialization for *BindingRestElement* using *iteratorRecord* and *environment* as the arguments.
10. Set the VariableEnvironment of *currentContext* to *originalEnv*.
11. Set the LexicalEnvironment of *currentContext* to *originalEnv*.
12. Return *result*.

NOTE 3

The new **Environment Record** created in step 6 is only used if the *BindingRestElement* contains a **direct eval**.

14.1.20 Runtime Semantics: InstantiateFunctionObject

With parameter *scope*.

FunctionDeclaration : **function** *BindingIdentifier* (*FormalParameters*) { *FunctionBody* }

1. If the function code for *FunctionDeclaration* is **strict mode code**, let *strict* be **true**. Otherwise let *strict* be **false**.
2. Let *name* be StringValue of *BindingIdentifier*.
3. Let *F* be FunctionCreate(Normal, *FormalParameters*, *FunctionBody*, *scope*, *strict*).
4. Perform MakeConstructor(*F*).
5. Perform SetFunctionName(*F*, *name*).
6. Set *F*.[[SourceText]] to the source text matched by *FunctionDeclaration*.
7. Return *F*.

FunctionDeclaration : **function** (*FormalParameters*) { *FunctionBody* }

1. Let *F* be FunctionCreate(Normal, *FormalParameters*, *FunctionBody*, *scope*, **true**).
2. Perform MakeConstructor(*F*).
3. Perform SetFunctionName(*F*, "default").
4. Set *F*.[[SourceText]] to the source text matched by *FunctionDeclaration*.
5. Return *F*.

NOTE

An anonymous *FunctionDeclaration* can only occur as part of an **export default** declaration, and its function code is therefore always **strict mode code**.

14.1.21 Runtime Semantics: NamedEvaluation

With parameter *name*.

FunctionExpression : **function** (*FormalParameters*) { *FunctionBody* }

1. Let *closure* be the result of evaluating this *FunctionExpression*.
2. Perform *SetFunctionName*(*closure*, *name*).
3. Return *closure*.

14.1.22 Runtime Semantics: Evaluation

FunctionDeclaration : **function** *BindingIdentifier* (*FormalParameters*) { *FunctionBody* }

1. Return *NormalCompletion*(empty).

NOTE 1

An alternative semantics is provided in B.3.3.

FunctionDeclaration : **function** (*FormalParameters*) { *FunctionBody* }

1. Return *NormalCompletion*(empty).

FunctionExpression : **function** (*FormalParameters*) { *FunctionBody* }

1. If the function code for *FunctionExpression* is *strict mode code*, let *strict* be **true**. Otherwise let *strict* be **false**.
2. Let *scope* be the *LexicalEnvironment* of the *running execution context*.
3. Let *closure* be *FunctionCreate*(*Normal*, *FormalParameters*, *FunctionBody*, *scope*, *strict*).
4. Perform *MakeConstructor*(*closure*).
5. Set *closure*.[[SourceText]] to the source text matched by *FunctionExpression*.
6. Return *closure*.

FunctionExpression : **function** *BindingIdentifier* (*FormalParameters*) { *FunctionBody* }

1. If the function code for *FunctionExpression* is *strict mode code*, let *strict* be **true**. Otherwise let *strict* be **false**.
2. Let *scope* be the *running execution context*'s *LexicalEnvironment*.
3. Let *funcEnv* be *NewDeclarativeEnvironment*(*scope*).
4. Let *envRec* be *funcEnv*'s *EnvironmentRecord*.
5. Let *name* be *StringValue* of *BindingIdentifier*.
6. Perform *envRec*.*CreateImmutableBinding*(*name*, **false**).
7. Let *closure* be *FunctionCreate*(*Normal*, *FormalParameters*, *FunctionBody*, *funcEnv*, *strict*).
8. Perform *MakeConstructor*(*closure*).
9. Perform *SetFunctionName*(*closure*, *name*).
10. Set *closure*.[[SourceText]] to the source text matched by *FunctionExpression*.
11. Perform *envRec*.*InitializeBinding*(*name*, *closure*).
12. Return *closure*.

NOTE 2

The *BindingIdentifier* in a *FunctionExpression* can be referenced from inside the *FunctionExpression*'s *FunctionBody* to allow the function to call itself recursively. However, unlike in a *FunctionDeclaration*, the *BindingIdentifier* in a *FunctionExpression* cannot be referenced from and does not affect the scope enclosing the *FunctionExpression*.

NOTE 3

A **prototype** property is automatically created for every function defined using a *FunctionDeclaration* or *FunctionExpression*, to allow for the possibility that the function will be used as a *constructor*.

FunctionStatementList : [empty]

1. Return **NormalCompletion**(**undefined**).

14.2 Arrow Function Definitions

Syntax

*ArrowFunction*_[In, Yield, Await] :
 *ArrowParameters*_[?Yield, ?Await] [no *LineTerminator* here] => *ConciseBody*_[?In]

*ArrowParameters*_[Yield, Await] :
 *BindingIdentifier*_[?Yield, ?Await]
 *CoverParenthesizedExpressionAndArrowParameterList*_[?Yield, ?Await]

*ConciseBody*_[In] :
 [lookahead ≠ {] *AssignmentExpression*_[?In, ~Yield, ~Await]
 { *FunctionBody*_[~Yield, ~Await] }

Supplemental Syntax

When the production

ArrowParameters : *CoverParenthesizedExpressionAndArrowParameterList*

is recognized the following grammar is used to refine the interpretation of

CoverParenthesizedExpressionAndArrowParameterList:

*ArrowFormalParameters*_[Yield, Await] :
 (*UniqueFormalParameters*_[?Yield, ?Await])

14.2.1 Static Semantics: Early Errors

ArrowFunction : *ArrowParameters* => *ConciseBody*

It is a Syntax Error if *ArrowParameters* Contains *YieldExpression* is **true**.

It is a Syntax Error if *ArrowParameters* Contains *AwaitExpression* is **true**.

It is a Syntax Error if ContainsUseStrict of *ConciseBody* is **true** and IsSimpleParameterList of *ArrowParameters* is **false**.

It is a Syntax Error if any element of the BoundNames of *ArrowParameters* also occurs in the LexicallyDeclaredNames of *ConciseBody*.

ArrowParameters : *CoverParenthesizedExpressionAndArrowParameterList*

It is a Syntax Error if *CoverParenthesizedExpressionAndArrowParameterList* is not **covering** an *ArrowFormalParameters*.

All **early error** rules for *ArrowFormalParameters* and its derived productions also apply to CoveredFormalsList of *CoverParenthesizedExpressionAndArrowParameterList*.

14.2.2 Static Semantics: BoundNames

ArrowParameters : *CoverParenthesizedExpressionAndArrowParameterList*

1. Let *formals* be CoveredFormalsList of *CoverParenthesizedExpressionAndArrowParameterList*.
2. Return the BoundNames of *formals*.

14.2.3 Static Semantics: Contains

With parameter *symbol*.

ArrowFunction : *ArrowParameters* => *ConciseBody*

1. If *symbol* is not one of *NewTarget*, *SuperProperty*, *SuperCall*, **super** or **this**, return **false**.
2. If *ArrowParameters* Contains *symbol* is **true**, return **true**.
3. Return *ConciseBody* Contains *symbol*.

NOTE

Normally, Contains does not look inside most function forms. However, Contains is used to detect **new.target**, **this**, and **super** usage within an *ArrowFunction*.

ArrowParameters : *CoverParenthesizedExpressionAndArrowParameterList*

1. Let *formals* be CoveredFormalsList of *CoverParenthesizedExpressionAndArrowParameterList*.
2. Return *formals* Contains *symbol*.

14.2.4 Static Semantics: ContainsExpression

ArrowParameters : *BindingIdentifier*

1. Return **false**.

14.2.5 Static Semantics: ContainsUseStrict

ConciseBody : *AssignmentExpression*

1. Return **false**.

14.2.6 Static Semantics: ExpectedArgumentCount

ArrowParameters : *BindingIdentifier*

1. Return 1.

14.2.7 Static Semantics: HasName

ArrowFunction : *ArrowParameters* => *ConciseBody*

1. Return **false**.

14.2.8 Static Semantics: IsSimpleParameterList

ArrowParameters : *BindingIdentifier*

1. Return **true**.

ArrowParameters : *CoverParenthesizedExpressionAndArrowParameterList*

1. Let *formals* be CoveredFormalsList of *CoverParenthesizedExpressionAndArrowParameterList*.
2. Return IsSimpleParameterList of *formals*.

14.2.9 Static Semantics: CoveredFormalsList

ArrowParameters : *BindingIdentifier*

1. Return this *ArrowParameters*.

CoverParenthesizedExpressionAndArrowParameterList :

```
( Expression )
( )
( ... BindingIdentifier )
( ... BindingPattern )
( Expression , ... BindingIdentifier )
( Expression , ... BindingPattern )
```

1. Return the *ArrowFormalParameters* that is *covered* by *CoverParenthesizedExpressionAndArrowParameterList*.

14.2.10 Static Semantics: LexicallyDeclaredNames

ConciseBody : *AssignmentExpression*

1. Return a new empty *List*.

14.2.11 Static Semantics: LexicallyScopedDeclarations

ConciseBody : *AssignmentExpression*

1. Return a new empty *List*.

14.2.12 Static Semantics: VarDeclaredNames

ConciseBody : *AssignmentExpression*

1. Return a new empty *List*.

14.2.13 Static Semantics: VarScopedDeclarations

ConciseBody : *AssignmentExpression*

1. Return a new empty *List*.

14.2.14 Runtime Semantics: IteratorBindingInitialization

With parameters *iteratorRecord* and *environment*.

NOTE

When **undefined** is passed for *environment* it indicates that a *PutValue* operation should be used to assign the initialization value. This is the case for formal parameter lists of non-strict functions. In that case the formal parameter bindings are preinitialized in order to deal with the possibility of multiple parameters with the same name.

ArrowParameters : *BindingIdentifier*

1. **Assert:** *iteratorRecord*.[[Done]] is **false**.
2. Let *next* be *IteratorStep*(*iteratorRecord*).
3. If *next* is an abrupt completion, set *iteratorRecord*.[[Done]] to **true**.
4. **ReturnIfAbrupt**(*next*).
5. If *next* is **false**, set *iteratorRecord*.[[Done]] to **true**.
6. Else,
 - a. Let *v* be *IteratorValue*(*next*).
 - b. If *v* is an abrupt completion, set *iteratorRecord*.[[Done]] to **true**.
 - c. **ReturnIfAbrupt**(*v*).
7. If *iteratorRecord*.[[Done]] is **true**, let *v* be **undefined**.
8. Return the result of performing BindingInitialization for *BindingIdentifier* using *v* and *environment* as the arguments.

14.2.15 Runtime Semantics: EvaluateBody

With parameters *functionObject* and *List argumentsList*.

ConciseBody : *AssignmentExpression*

1. Perform ? **FunctionDeclarationInstantiation**(*functionObject*, *argumentsList*).
2. Let *exprRef* be the result of evaluating *AssignmentExpression*.
3. Let *exprValue* be ? **GetValue**(*exprRef*).
4. Return **Completion** { [[Type]]: return, [[Value]]: *exprValue*, [[Target]]: empty }.

14.2.16 Runtime Semantics: NamedEvaluation

With parameter *name*.

ArrowFunction : *ArrowParameters* => *ConciseBody*

1. Let *closure* be the result of evaluating this *ArrowFunction*.
2. Perform **SetFunctionName**(*closure*, *name*).
3. Return *closure*.

14.2.17 Runtime Semantics: Evaluation

ArrowFunction : *ArrowParameters* => *ConciseBody*

1. If the function code for this *ArrowFunction* is **strict mode code**, let *strict* be **true**. Otherwise let *strict* be **false**.
2. Let *scope* be the *LexicalEnvironment* of the **running execution context**.
3. Let *parameters* be *CoveredFormalsList* of *ArrowParameters*.
4. Let *closure* be **FunctionCreate**(*Arrow*, *parameters*, *ConciseBody*, *scope*, *strict*).
5. Set *closure*.[[SourceText]] to the source text matched by *ArrowFunction*.
6. Return *closure*.

NOTE

An *ArrowFunction* does not define local bindings for **arguments**, **super**, **this**, or **new.target**. Any reference to **arguments**, **super**, **this**, or **new.target** within an *ArrowFunction* must resolve to a binding in a lexically enclosing environment. Typically this will be the *Function Environment* of an immediately enclosing function. Even though an *ArrowFunction* may contain references to **super**, the **function object** created in step 4 is not made into a

method by performing [MakeMethod](#). An *ArrowFunction* that references **super** is always contained within a non-*ArrowFunction* and the necessary state to implement **super** is accessible via the [scope](#) that is captured by the [function object](#) of the *ArrowFunction*.

14.3 Method Definitions

Syntax

```
MethodDefinition[Yield, Await] :  
    PropertyName[?Yield, ?Await] ( UniqueFormalParameters[~Yield, ~Await] ) {  
        FunctionBody[~Yield, ~Await] }  
    GeneratorMethod[?Yield, ?Await]  
    AsyncMethod[?Yield, ?Await]  
    AsyncGeneratorMethod[?Yield, ?Await]  
get PropertyName[?Yield, ?Await] ( ) { FunctionBody[~Yield, ~Await] }  
set PropertyName[?Yield, ?Await] ( PropertySetParameterList ) {  
    FunctionBody[~Yield, ~Await] }
```

```
PropertySetParameterList :  
    FormalParameter[~Yield, ~Await]
```

14.3.1 Static Semantics: Early Errors

MethodDefinition : *PropertyName* (*UniqueFormalParameters*) { *FunctionBody* }

It is a Syntax Error if ContainsUseStrict of *FunctionBody* is **true** and IsSimpleParameterList of *UniqueFormalParameters* is **false**.

It is a Syntax Error if any element of the BoundNames of *UniqueFormalParameters* also occurs in the LexicallyDeclaredNames of *FunctionBody*.

MethodDefinition : **set** *PropertyName* (*PropertySetParameterList*) { *FunctionBody* }

It is a Syntax Error if BoundNames of *PropertySetParameterList* contains any duplicate elements.

It is a Syntax Error if ContainsUseStrict of *FunctionBody* is **true** and IsSimpleParameterList of *PropertySetParameterList* is **false**.

It is a Syntax Error if any element of the BoundNames of *PropertySetParameterList* also occurs in the LexicallyDeclaredNames of *FunctionBody*.

14.3.2 Static Semantics: ComputedPropertyContains

With parameter [symbol](#).

```
MethodDefinition :  
    PropertyName ( UniqueFormalParameters ) { FunctionBody }  
get PropertyName ( ) { FunctionBody }  
set PropertyName ( PropertySetParameterList ) { FunctionBody }
```

1. Return the result of `ComputedPropertyContains` for *PropertyName* with argument *symbol*.

14.3.3 Static Semantics: ExpectedArgumentCount

PropertySetParameterList : *FormalParameter*

1. If `HasInitializer` of *FormalParameter* is **true**, return 0.
2. Return 1.

14.3.4 Static Semantics: HasDirectSuper

MethodDefinition : *PropertyName* (*UniqueFormalParameters*) { *FunctionBody* }

1. If *UniqueFormalParameters* Contains *SuperCall* is **true**, return **true**.
2. Return *FunctionBody* Contains *SuperCall*.

MethodDefinition : **get** *PropertyName* () { *FunctionBody* }

1. Return *FunctionBody* Contains *SuperCall*.

MethodDefinition : **set** *PropertyName* (*PropertySetParameterList*) { *FunctionBody* }

1. If *PropertySetParameterList* Contains *SuperCall* is **true**, return **true**.
2. Return *FunctionBody* Contains *SuperCall*.

14.3.5 Static Semantics: PropName

MethodDefinition :

PropertyName (*UniqueFormalParameters*) { *FunctionBody* }

get *PropertyName* () { *FunctionBody* }

set *PropertyName* (*PropertySetParameterList*) { *FunctionBody* }

1. Return `PropName` of *PropertyName*.

14.3.6 Static Semantics: SpecialMethod

MethodDefinition : *PropertyName* (*UniqueFormalParameters*) { *FunctionBody* }

1. Return **false**.

MethodDefinition :

GeneratorMethod

AsyncMethod

AsyncGeneratorMethod

get *PropertyName* () { *FunctionBody* }

set *PropertyName* (*PropertySetParameterList*) { *FunctionBody* }

1. Return **true**.

14.3.7 Runtime Semantics: DefineMethod

With parameters *object* and optional parameter *functionPrototype*.

MethodDefinition : *PropertyName* (*UniqueFormalParameters*) { *FunctionBody* }

1. Let *propKey* be the result of evaluating *PropertyName*.
2. **ReturnIfAbrupt**(*propKey*).
3. If the function code for this *MethodDefinition* is **strict mode code**, let *strict* be **true**. Otherwise let *strict* be **false**.
4. Let *scope* be the **running execution context**'s **LexicalEnvironment**.
5. If *functionPrototype* is present as a parameter, then
 - a. Let *kind* be **Normal**.
 - b. Let *prototype* be *functionPrototype*.
6. Else,
 - a. Let *kind* be **Method**.
 - b. Let *prototype* be the intrinsic object **%FunctionPrototype%**.
7. Let *closure* be **FunctionCreate**(*kind*, *UniqueFormalParameters*, *FunctionBody*, *scope*, *strict*, *prototype*).
8. Perform **MakeMethod**(*closure*, *object*).
9. Set *closure*.[[SourceText]] to the source text matched by *MethodDefinition*.
10. Return the **Record** { [[Key]]: *propKey*, [[Closure]]: *closure* }.

14.3.8 Runtime Semantics: PropertyDefinitionEvaluation

With parameters *object* and *enumerable*.

MethodDefinition : *PropertyName* (*UniqueFormalParameters*) { *FunctionBody* }

1. Let *methodDef* be **DefineMethod** of *MethodDefinition* with argument *object*.
2. **ReturnIfAbrupt**(*methodDef*).
3. Perform **SetFunctionName**(*methodDef*.[[Closure]], *methodDef*.[[Key]]).
4. Let *desc* be the **PropertyDescriptor** { [[Value]]: *methodDef*.[[Closure]], [[Writable]]: **true**, [[Enumerable]]: *enumerable*, [[Configurable]]: **true** }.
5. Return ? **DefinePropertyOrThrow**(*object*, *methodDef*.[[Key]], *desc*).

MethodDefinition : **get** *PropertyName* () { *FunctionBody* }

1. Let *propKey* be the result of evaluating *PropertyName*.
2. **ReturnIfAbrupt**(*propKey*).
3. If the function code for this *MethodDefinition* is **strict mode code**, let *strict* be **true**. Otherwise let *strict* be **false**.
4. Let *scope* be the **running execution context**'s **LexicalEnvironment**.
5. Let *formalParameterList* be an instance of the production *FormalParameters* : [empty] .
6. Let *closure* be **FunctionCreate**(**Method**, *formalParameterList*, *FunctionBody*, *scope*, *strict*).
7. Perform **MakeMethod**(*closure*, *object*).
8. Perform **SetFunctionName**(*closure*, *propKey*, **"get"**).
9. Set *closure*.[[SourceText]] to the source text matched by *MethodDefinition*.
10. Let *desc* be the **PropertyDescriptor** { [[Get]]: *closure*, [[Enumerable]]: *enumerable*, [[Configurable]]: **true** }.
11. Return ? **DefinePropertyOrThrow**(*object*, *propKey*, *desc*).

MethodDefinition : **set** *PropertyName* (*PropertySetParameterList*) { *FunctionBody* }

1. Let *propKey* be the result of evaluating *PropertyName*.
2. **ReturnIfAbrupt**(*propKey*).
3. If the function code for this *MethodDefinition* is **strict mode code**, let *strict* be **true**. Otherwise let *strict* be **false**.
4. Let *scope* be the **running execution context**'s **LexicalEnvironment**.

5. Let *closure* be **FunctionCreate**(*Method*, *PropertySetParameterList*, *FunctionBody*, *scope*, *strict*).
6. Perform **MakeMethod**(*closure*, *object*).
7. Perform **SetFunctionName**(*closure*, *propKey*, "set").
8. Set *closure*.[[SourceText]] to the source text matched by *MethodDefinition*.
9. Let *desc* be the PropertyDescriptor { [[Set]]: *closure*, [[Enumerable]]: *enumerable*, [[Configurable]]: **true** }.
10. Return ? **DefinePropertyOrThrow**(*object*, *propKey*, *desc*).

14.4 Generator Function Definitions

Syntax

*GeneratorMethod*_[Yield, Await] :

```
* PropertyName[?Yield, ?Await] ( UniqueFormalParameters[+Yield, ~Await] ) {
  GeneratorBody }
```

*GeneratorDeclaration*_[Yield, Await, Default] :

```
function * BindingIdentifier[?Yield, ?Await] ( FormalParameters[+Yield, ~Await] ) {
  GeneratorBody }
[+Default] function * ( FormalParameters[+Yield, ~Await] ) { GeneratorBody }
```

GeneratorExpression :

```
function * BindingIdentifier[+Yield, ~Await] opt ( FormalParameters[+Yield, ~Await] ) {
  GeneratorBody }
```

GeneratorBody :

```
FunctionBody[+Yield, ~Await]
```

*YieldExpression*_[In, Await] :

```
yield
yield [no LineTerminator here] AssignmentExpression[?In, +Yield, ?Await]
yield [no LineTerminator here] * AssignmentExpression[?In, +Yield, ?Await]
```

NOTE 1

The syntactic context immediately following **yield** requires use of the *InputElementRegExpOrTemplateTail* lexical goal.

NOTE 2

YieldExpression cannot be used within the *FormalParameters* of a generator function because any expressions that are part of *FormalParameters* are evaluated before the resulting generator object is in a resumable state.

NOTE 3

[Abstract operations](#) relating to generator objects are defined in [25.4.3](#).

14.4.1 Static Semantics: Early Errors

GeneratorMethod : * *PropertyName* (*UniqueFormalParameters*) { *GeneratorBody* }

It is a Syntax Error if *HasDirectSuper* of *GeneratorMethod* is **true**.

It is a Syntax Error if *UniqueFormalParameters* Contains *YieldExpression* is **true**.

It is a Syntax Error if ContainsUseStrict of *GeneratorBody* is **true** and IsSimpleParameterList of *UniqueFormalParameters* is **false**.

It is a Syntax Error if any element of the BoundNames of *UniqueFormalParameters* also occurs in the LexicallyDeclaredNames of *GeneratorBody*.

GeneratorDeclaration : **function** * *BindingIdentifier* (*FormalParameters*) { *GeneratorBody* }

GeneratorDeclaration : **function** * (*FormalParameters*) { *GeneratorBody* }

GeneratorExpression : **function** * *BindingIdentifier* (*FormalParameters*) { *GeneratorBody* }

If the source code matching this production is [strict mode code](#), the Early Error rules for *UniqueFormalParameters* : *FormalParameters* are applied.

If the source code matching this production is [strict mode code](#), it is a Syntax Error if *BindingIdentifier* is present and the StringValue of *BindingIdentifier* is **"eval"** or **"arguments"**.

It is a Syntax Error if ContainsUseStrict of *GeneratorBody* is **true** and IsSimpleParameterList of *FormalParameters* is **false**.

It is a Syntax Error if any element of the BoundNames of *FormalParameters* also occurs in the LexicallyDeclaredNames of *GeneratorBody*.

It is a Syntax Error if *FormalParameters* Contains *YieldExpression* is **true**.

It is a Syntax Error if *FormalParameters* Contains *SuperProperty* is **true**.

It is a Syntax Error if *GeneratorBody* Contains *SuperProperty* is **true**.

It is a Syntax Error if *FormalParameters* Contains *SuperCall* is **true**.

It is a Syntax Error if *GeneratorBody* Contains *SuperCall* is **true**.

14.4.2 Static Semantics: BoundNames

GeneratorDeclaration : **function** * *BindingIdentifier* (*FormalParameters*) { *GeneratorBody* }

1. Return the BoundNames of *BindingIdentifier*.

GeneratorDeclaration : **function** * (*FormalParameters*) { *GeneratorBody* }

1. Return « **"*default*"** ».

NOTE

"*default*" is used within this specification as a synthetic name for hoistable anonymous functions that are defined using export declarations.

14.4.3 Static Semantics: ComputedPropertyContains

With parameter [symbol](#).

GeneratorMethod : * *PropertyName* (*UniqueFormalParameters*) { *GeneratorBody* }

1. Return the result of ComputedPropertyContains for *PropertyName* with argument [symbol](#).

14.4.4 Static Semantics: Contains

With parameter [symbol](#).

GeneratorDeclaration : **function** * *BindingIdentifier* (*FormalParameters*) { *GeneratorBody* }

GeneratorDeclaration : **function** * (*FormalParameters*) { *GeneratorBody* }

GeneratorExpression : **function** * *BindingIdentifier* (*FormalParameters*) { *GeneratorBody* }

1. Return **false**.

NOTE

Static semantic rules that depend upon substructure generally do not look into function definitions.

14.4.5 Static Semantics: HasDirectSuper

GeneratorMethod : * *PropertyName* (*UniqueFormalParameters*) { *GeneratorBody* }

1. If *UniqueFormalParameters* Contains *SuperCall* is **true**, return **true**.
2. Return *GeneratorBody* Contains *SuperCall*.

14.4.6 Static Semantics: HasName

GeneratorExpression : **function** * (*FormalParameters*) { *GeneratorBody* }

1. Return **false**.

GeneratorExpression : **function** * *BindingIdentifier* (*FormalParameters*) { *GeneratorBody* }

1. Return **true**.

14.4.7 Static Semantics: IsConstantDeclaration

GeneratorDeclaration : **function** * *BindingIdentifier* (*FormalParameters*) { *GeneratorBody* }

GeneratorDeclaration : **function** * (*FormalParameters*) { *GeneratorBody* }

1. Return **false**.

14.4.8 Static Semantics: IsFunctionDefinition

GeneratorExpression : **function** * *BindingIdentifier* (*FormalParameters*) { *GeneratorBody* }

1. Return **true**.

14.4.9 Static Semantics: PropName

GeneratorMethod : * *PropertyName* (*UniqueFormalParameters*) { *GeneratorBody* }

1. Return PropName of *PropertyName*.

14.4.10 Runtime Semantics: EvaluateBody

With parameters *functionObject* and List *argumentsList*.

GeneratorBody : *FunctionBody*

1. Perform ? *FunctionDeclarationInstantiation*(*functionObject*, *argumentsList*).
2. Let *G* be ? *OrdinaryCreateFromConstructor*(*functionObject*, "%GeneratorPrototype%", « [[GeneratorState]], [[GeneratorContext]] »).

3. Perform **GeneratorStart**(*G*, *FunctionBody*).
4. Return **Completion** { [[Type]]: return, [[Value]]: *G*, [[Target]]: empty }.

14.4.11 Runtime Semantics: InstantiateFunctionObject

With parameter *scope*.

GeneratorDeclaration : **function** * *BindingIdentifier* (*FormalParameters*) { *GeneratorBody* }

1. If the function code for *GeneratorDeclaration* is **strict mode code**, let *strict* be **true**. Otherwise let *strict* be **false**.
2. Let *name* be StringValue of *BindingIdentifier*.
3. Let *F* be **GeneratorFunctionCreate**(Normal, *FormalParameters*, *GeneratorBody*, *scope*, *strict*).
4. Let *prototype* be **ObjectCreate**(%GeneratorPrototype%).
5. Perform **DefinePropertyOrThrow**(*F*, "**prototype**", PropertyDescriptor { [[Value]]: *prototype*, [[Writable]]: **true**, [[Enumerable]]: **false**, [[Configurable]]: **false** }).
6. Perform **SetFunctionName**(*F*, *name*).
7. Set *F*.[[SourceText]] to the source text matched by *GeneratorDeclaration*.
8. Return *F*.

GeneratorDeclaration : **function** * (*FormalParameters*) { *GeneratorBody* }

1. Let *F* be **GeneratorFunctionCreate**(Normal, *FormalParameters*, *GeneratorBody*, *scope*, **true**).
2. Let *prototype* be **ObjectCreate**(%GeneratorPrototype%).
3. Perform **DefinePropertyOrThrow**(*F*, "**prototype**", PropertyDescriptor { [[Value]]: *prototype*, [[Writable]]: **true**, [[Enumerable]]: **false**, [[Configurable]]: **false** }).
4. Perform **SetFunctionName**(*F*, "**default**").
5. Set *F*.[[SourceText]] to the source text matched by *GeneratorDeclaration*.
6. Return *F*.

NOTE

An anonymous *GeneratorDeclaration* can only occur as part of an **export default** declaration, and its function code is therefore always **strict mode code**.

14.4.12 Runtime Semantics: PropertyDefinitionEvaluation

With parameters *object* and *enumerable*.

GeneratorMethod : * *PropertyName* (*UniqueFormalParameters*) { *GeneratorBody* }

1. Let *propKey* be the result of evaluating *PropertyName*.
2. **ReturnIfAbrupt**(*propKey*).
3. If the function code for this *GeneratorMethod* is **strict mode code**, let *strict* be **true**. Otherwise let *strict* be **false**.
4. Let *scope* be the **running execution context**'s **LexicalEnvironment**.
5. Let *closure* be **GeneratorFunctionCreate**(Method, *UniqueFormalParameters*, *GeneratorBody*, *scope*, *strict*).
6. Perform **MakeMethod**(*closure*, *object*).
7. Let *prototype* be **ObjectCreate**(%GeneratorPrototype%).
8. Perform **DefinePropertyOrThrow**(*closure*, "**prototype**", PropertyDescriptor { [[Value]]: *prototype*, [[Writable]]: **true**, [[Enumerable]]: **false**, [[Configurable]]: **false** }).
9. Perform **SetFunctionName**(*closure*, *propKey*).

10. Set *closure*.[[SourceText]] to the source text matched by *GeneratorMethod*.
11. Let *desc* be the PropertyDescriptor { [[Value]]: *closure*, [[Writable]]: **true**, [[Enumerable]]: *enumerable*, [[Configurable]]: **true** }.
12. Return ? *DefinePropertyOrThrow*(*object*, *propKey*, *desc*).

14.4.13 Runtime Semantics: NamedEvaluation

With parameter *name*.

GeneratorExpression : **function** * (*FormalParameters*) { *GeneratorBody* }

1. Let *closure* be the result of evaluating this *GeneratorExpression*.
2. Perform *SetFunctionName*(*closure*, *name*).
3. Return *closure*.

14.4.14 Runtime Semantics: Evaluation

GeneratorExpression : **function** * (*FormalParameters*) { *GeneratorBody* }

1. If the function code for this *GeneratorExpression* is *strict mode code*, let *strict* be **true**. Otherwise let *strict* be **false**.
2. Let *scope* be the LexicalEnvironment of the *running execution context*.
3. Let *closure* be *GeneratorFunctionCreate*(Normal, *FormalParameters*, *GeneratorBody*, *scope*, *strict*).
4. Let *prototype* be *ObjectCreate*(%GeneratorPrototype%).
5. Perform *DefinePropertyOrThrow*(*closure*, "**prototype**", PropertyDescriptor { [[Value]]: *prototype*, [[Writable]]: **true**, [[Enumerable]]: **false**, [[Configurable]]: **false** }).
6. Set *closure*.[[SourceText]] to the source text matched by *GeneratorExpression*.
7. Return *closure*.

GeneratorExpression : **function** * *BindingIdentifier* (*FormalParameters*) { *GeneratorBody* }

1. If the function code for this *GeneratorExpression* is *strict mode code*, let *strict* be **true**. Otherwise let *strict* be **false**.
2. Let *scope* be the *running execution context*'s LexicalEnvironment.
3. Let *funcEnv* be *NewDeclarativeEnvironment*(*scope*).
4. Let *envRec* be *funcEnv*'s *EnvironmentRecord*.
5. Let *name* be StringValue of *BindingIdentifier*.
6. Perform *envRec*.CreateImmutableBinding(*name*, **false**).
7. Let *closure* be *GeneratorFunctionCreate*(Normal, *FormalParameters*, *GeneratorBody*, *funcEnv*, *strict*).
8. Let *prototype* be *ObjectCreate*(%GeneratorPrototype%).
9. Perform *DefinePropertyOrThrow*(*closure*, "**prototype**", PropertyDescriptor { [[Value]]: *prototype*, [[Writable]]: **true**, [[Enumerable]]: **false**, [[Configurable]]: **false** }).
10. Perform *SetFunctionName*(*closure*, *name*).
11. Perform *envRec*.InitializeBinding(*name*, *closure*).
12. Set *closure*.[[SourceText]] to the source text matched by *GeneratorExpression*.
13. Return *closure*.

NOTE

The *BindingIdentifier* in a *GeneratorExpression* can be referenced from inside the *GeneratorExpression*'s *FunctionBody* to allow the generator code to call itself recursively. However, unlike in a *GeneratorDeclaration*, the *BindingIdentifier* in

a *GeneratorExpression* cannot be referenced from and does not affect the scope enclosing the *GeneratorExpression*.

YieldExpression : **yield**

1. Let *generatorKind* be ! *GetGeneratorKind*().
2. If *generatorKind* is **async**, then return ? *AsyncGeneratorYield*(**undefined**).
3. Otherwise, return ? *GeneratorYield*(*CreateIterResultObject*(**undefined**, **false**)).

YieldExpression : **yield** *AssignmentExpression*

1. Let *generatorKind* be ! *GetGeneratorKind*().
2. Let *exprRef* be the result of evaluating *AssignmentExpression*.
3. Let *value* be ? *GetValue*(*exprRef*).
4. If *generatorKind* is **async**, then return ? *AsyncGeneratorYield*(*value*).
5. Otherwise, return ? *GeneratorYield*(*CreateIterResultObject*(*value*, **false**)).

YieldExpression : **yield** * *AssignmentExpression*

1. Let *generatorKind* be ! *GetGeneratorKind*().
2. Let *exprRef* be the result of evaluating *AssignmentExpression*.
3. Let *value* be ? *GetValue*(*exprRef*).
4. Let *iteratorRecord* be ? *GetIterator*(*value*, *generatorKind*).
5. Let *iterator* be *iteratorRecord*.[[Iterator]].
6. Let *received* be *NormalCompletion*(**undefined**).
7. Repeat,
 - a. If *received*.[[Type]] is **normal**, then
 - i. Let *innerResult* be ? *Call*(*iteratorRecord*.[[NextMethod]], *iteratorRecord*.[[Iterator]], « *received*.[[Value]] »).
 - ii. If *generatorKind* is **async**, then set *innerResult* to ? *Await*(*innerResult*).
 - iii. If *Type*(*innerResult*) is not **Object**, throw a **TypeError** exception.
 - iv. Let *done* be ? *IteratorComplete*(*innerResult*).
 - v. If *done* is **true**, then
 1. Return ? *IteratorValue*(*innerResult*).
 - vi. If *generatorKind* is **async**, then set *received* to *AsyncGeneratorYield*(? *IteratorValue*(*innerResult*)).
 - vii. Else, set *received* to *GeneratorYield*(*innerResult*).
 - b. Else if *received*.[[Type]] is **throw**, then
 - i. Let *throw* be ? *GetMethod*(*iterator*, "**throw**").
 - ii. If *throw* is not **undefined**, then
 1. Let *innerResult* be ? *Call*(*throw*, *iterator*, « *received*.[[Value]] »).
 2. If *generatorKind* is **async**, then set *innerResult* to ? *Await*(*innerResult*).
 3. NOTE: Exceptions from the inner iterator **throw** method are propagated. Normal completions from an inner **throw** method are processed similarly to an inner **next**.
 4. If *Type*(*innerResult*) is not **Object**, throw a **TypeError** exception.
 5. Let *done* be ? *IteratorComplete*(*innerResult*).
 6. If *done* is **true**, then
 - a. Return ? *IteratorValue*(*innerResult*).
 7. If *generatorKind* is **async**, then set *received* to *AsyncGeneratorYield*(? *IteratorValue*(*innerResult*)).
 8. Else, set *received* to *GeneratorYield*(*innerResult*).
 - iii. Else,

1. NOTE: If *iterator* does not have a **throw** method, this throw is going to terminate the **yield*** loop. But first we need to give *iterator* a chance to clean up.
 2. Let *closeCompletion* be *Completion* { [[Type]]: normal, [[Value]]: empty, [[Target]]: empty }.
 3. If *generatorKind* is **async**, perform ? *AsyncIteratorClose*(*iteratorRecord*, *closeCompletion*).
 4. Else, perform ? *IteratorClose*(*iteratorRecord*, *closeCompletion*).
 5. NOTE: The next step throws a **TypeError** to indicate that there was a **yield*** protocol violation: *iterator* does not have a **throw** method.
 6. Throw a **TypeError** exception.
- c. Else,
- i. **Assert**: *received*.[[Type]] is **return**.
 - ii. Let *return* be ? *GetMethod*(*iterator*, "**return**").
 - iii. If *return* is **undefined**, then
 1. If *generatorKind* is **async**, then set *received*.[[Value]] to ? *Await*(*received*.[[Value]]).
 2. Return *Completion*(*received*).
 - iv. Let *innerReturnResult* be ? *Call*(*return*, *iterator*, « *received*.[[Value]] »).
 - v. If *generatorKind* is **async**, then set *innerReturnResult* to ? *Await*(*innerReturnResult*).
 - vi. If *Type*(*innerReturnResult*) is not **Object**, throw a **TypeError** exception.
 - vii. Let *done* be ? *IteratorComplete*(*innerReturnResult*).
 - viii. If *done* is **true**, then
 1. Let *value* be ? *IteratorValue*(*innerReturnResult*).
 2. Return *Completion* { [[Type]]: **return**, [[Value]]: *value*, [[Target]]: empty }.
 - ix. If *generatorKind* is **async**, then set *received* to *AsyncGeneratorYield*(? *IteratorValue*(*innerReturnResult*)).
 - x. Else, set *received* to *GeneratorYield*(*innerReturnResult*).

14.5 Async Generator Function Definitions

Syntax

*AsyncGeneratorMethod*_[Yield, Await] :

```
async [no LineTerminator here] * PropertyName[?Yield, ?Await] (
  UniqueFormalParameters[+Yield, +Await] ) { AsyncGeneratorBody }
```

*AsyncGeneratorDeclaration*_[Yield, Await, Default] :

```
async [no LineTerminator here] function * BindingIdentifier[?Yield, ?Await] (
  FormalParameters[+Yield, +Await] ) { AsyncGeneratorBody }
[+Default] async [no LineTerminator here] function * ( FormalParameters[+Yield, +Await] ) {
  AsyncGeneratorBody }
```

AsyncGeneratorExpression :

```
async [no LineTerminator here] function * BindingIdentifier[+Yield, +Await] opt (
  FormalParameters[+Yield, +Await] ) { AsyncGeneratorBody }
```

AsyncGeneratorBody :

```
FunctionBody[+Yield, +Await]
```

NOTE 1

YieldExpression and *AwaitExpression* cannot be used within the *FormalParameters* of an async generator function because any expressions that are part of *FormalParameters* are evaluated before the resulting async generator object is in a resumable state.

NOTE 2

[Abstract operations](#) relating to async generator objects are defined in [25.5.3](#).

14.5.1 Static Semantics: Early Errors

AsyncGeneratorMethod : **async** * *PropertyName* (*UniqueFormalParameters*) { *AsyncGeneratorBody* }

It is a Syntax Error if *HasDirectSuper* of *AsyncGeneratorMethod* is **true**.

It is a Syntax Error if *UniqueFormalParameters* Contains *YieldExpression* is **true**.

It is a Syntax Error if *UniqueFormalParameters* Contains *AwaitExpression* is **true**.

It is a Syntax Error if *ContainsUseStrict* of *AsyncGeneratorBody* is **true** and *IsSimpleParameterList* of *UniqueFormalParameters* is **false**.

It is a Syntax Error if any element of the *BoundNames* of *UniqueFormalParameters* also occurs in the *LexicallyDeclaredNames* of *AsyncGeneratorBody*.

AsyncGeneratorDeclaration : **async function** * *BindingIdentifier* (*FormalParameters*) {
AsyncGeneratorBody }

AsyncGeneratorDeclaration : **async function** * (*FormalParameters*) { *AsyncGeneratorBody* }

AsyncGeneratorExpression : **async function** * *BindingIdentifier* (*FormalParameters*) {
AsyncGeneratorBody }

If the source code matching this production is [strict mode code](#), the Early Error rules for *UniqueFormalParameters* : *FormalParameters* are applied.

If the source code matching this production is [strict mode code](#), it is a Syntax Error if *BindingIdentifier* is the *IdentifierName* **eval** or the *IdentifierName* **arguments**.

It is a Syntax Error if *ContainsUseStrict* of *AsyncGeneratorBody* is **true** and *IsSimpleParameterList* of *FormalParameters* is **false**.

It is a Syntax Error if any element of the *BoundNames* of *FormalParameters* also occurs in the *LexicallyDeclaredNames* of *AsyncGeneratorBody*.

It is a Syntax Error if *FormalParameters* Contains *YieldExpression* is **true**.

It is a Syntax Error if *FormalParameters* Contains *AwaitExpression* is **true**.

It is a Syntax Error if *FormalParameters* Contains *SuperProperty* is **true**.

It is a Syntax Error if *AsyncGeneratorBody* Contains *SuperProperty* is **true**.

It is a Syntax Error if *FormalParameters* Contains *SuperCall* is **true**.

It is a Syntax Error if *AsyncGeneratorBody* Contains *SuperCall* is **true**.

14.5.2 Static Semantics: BoundNames

AsyncGeneratorDeclaration : **async function** * *BindingIdentifier* (*FormalParameters*) {
AsyncGeneratorBody }

1. Return the *BoundNames* of *BindingIdentifier*.

AsyncGeneratorDeclaration : **async function** * (*FormalParameters*) { *AsyncGeneratorBody* }

1. Return « **"*default*"** ».

NOTE

"***default***" is used within this specification as a synthetic name for hoistable anonymous functions that are defined using export declarations.

14.5.3 Static Semantics: ComputedPropertyContains

With parameter *symbol*.

AsyncGeneratorMethod : **async** * *PropertyName* (*UniqueFormalParameters*) { *AsyncGeneratorBody* }

1. Return the result of *ComputedPropertyContains* for *PropertyName* with argument *symbol*.

14.5.4 Static Semantics: Contains

With parameter *symbol*.

AsyncGeneratorDeclaration : **async function** * *BindingIdentifier* (*FormalParameters*) {
AsyncGeneratorBody }

AsyncGeneratorDeclaration : **async function** * (*FormalParameters*) { *AsyncGeneratorBody* }

AsyncGeneratorExpression : **async function** * *BindingIdentifier* (*FormalParameters*) {
AsyncGeneratorBody }

1. Return **false**.

NOTE

Static semantic rules that depend upon substructure generally do not look into function definitions.

14.5.5 Static Semantics: HasDirectSuper

AsyncGeneratorMethod : **async** * *PropertyName* (*UniqueFormalParameters*) { *AsyncGeneratorBody* }

1. If *UniqueFormalParameters* Contains *SuperCall* is **true**, return **true**.
2. Return *AsyncGeneratorBody* Contains *SuperCall*.

14.5.6 Static Semantics: HasName

AsyncGeneratorExpression : **async function** * (*FormalParameters*) { *AsyncGeneratorBody* }

1. Return **false**.

AsyncGeneratorExpression : **async function** * *BindingIdentifier* (*FormalParameters*) {
AsyncGeneratorBody }

1. Return **true**.

14.5.7 Static Semantics: IsConstantDeclaration

AsyncGeneratorDeclaration : **async function** * *BindingIdentifier* (*FormalParameters*) {
AsyncGeneratorBody }

AsyncGeneratorDeclaration : **async function** * (*FormalParameters*) { *AsyncGeneratorBody* }

1. Return **false**.

14.5.8 Static Semantics: IsFunctionDefinition

AsyncGeneratorExpression : **async function** * *BindingIdentifier* (*FormalParameters*) {
AsyncGeneratorBody }

1. Return **true**.

14.5.9 Static Semantics: PropName

AsyncGeneratorMethod : **async** * *PropertyName* (*UniqueFormalParameters*) { *AsyncGeneratorBody* }

1. Return PropName of *PropertyName*.

14.5.10 Runtime Semantics: EvaluateBody

With parameters *functionObject* and List *argumentsList*.

AsyncGeneratorBody : *FunctionBody*

1. Perform ? *FunctionDeclarationInstantiation*(*functionObject*, *argumentsList*).
2. Let *generator* be ? *OrdinaryCreateFromConstructor*(*functionObject*, "%AsyncGeneratorPrototype%", «
[[AsyncGeneratorState]], [[AsyncGeneratorContext]], [[AsyncGeneratorQueue]] »).
3. Perform ! *AsyncGeneratorStart*(*generator*, *FunctionBody*).
4. Return *Completion* { [[Type]]: return, [[Value]]: *generator*, [[Target]]: empty }.

14.5.11 Runtime Semantics: InstantiateFunctionObject

With parameter *scope*.

AsyncGeneratorDeclaration : **async function** * *BindingIdentifier* (*FormalParameters*) {
AsyncGeneratorBody }

1. If the function code for *AsyncGeneratorDeclaration* is *strict mode code*, let *strict* be **true**. Otherwise let *strict* be **false**.
2. Let *name* be StringValue of *BindingIdentifier*.
3. Let *F* be ! *AsyncGeneratorFunctionCreate*(Normal, *FormalParameters*, *AsyncGeneratorBody*, *scope*, *strict*).
4. Let *prototype* be ! *ObjectCreate*(%AsyncGeneratorPrototype%).
5. Perform ! *DefinePropertyOrThrow*(*F*, "**prototype**", PropertyDescriptor { [[Value]]: *prototype*, [[Writable]]: **true**, [[Enumerable]]: **false**, [[Configurable]]: **false** }).
6. Perform ! *SetFunctionName*(*F*, *name*).
7. Set *F*.[[SourceText]] to the source text matched by *AsyncGeneratorDeclaration*.
8. Return *F*.

AsyncGeneratorDeclaration : **async function** * (*FormalParameters*) { *AsyncGeneratorBody* }

1. If the function code for *AsyncGeneratorDeclaration* is *strict mode code*, let *strict* be **true**. Otherwise let *strict* be **false**.
2. Let *F* be *AsyncGeneratorFunctionCreate*(Normal, *FormalParameters*, *AsyncGeneratorBody*, *scope*, *strict*).
3. Let *prototype* be *ObjectCreate*(%AsyncGeneratorPrototype%).

4. Perform `DefinePropertyOrThrow(F, "prototype", PropertyDescriptor { [[Value]]: prototype, [[Writable]]: true, [[Enumerable]]: false, [[Configurable]]: false })`.
5. Perform `SetFunctionName(F, "default")`.
6. Set `F.[[SourceText]]` to the source text matched by *AsyncGeneratorDeclaration*.
7. Return *F*.

NOTE

An anonymous *AsyncGeneratorDeclaration* can only occur as part of an **export default** declaration.

14.5.12 Runtime Semantics: PropertyDefinitionEvaluation

With parameter *object* and *enumerable*.

AsyncGeneratorMethod : **async** * *PropertyName* (*UniqueFormalParameters*) { *AsyncGeneratorBody* }

1. Let *propKey* be the result of evaluating *PropertyName*.
2. `ReturnIfAbrupt(propKey)`.
3. If the function code for this *AsyncGeneratorMethod* is **strict mode code**, let *strict* be **true**. Otherwise let *strict* be **false**.
4. Let *scope* be the **running execution context**'s **LexicalEnvironment**.
5. Let *closure* be ! `AsyncGeneratorFunctionCreate(Method, UniqueFormalParameters, AsyncGeneratorBody, scope, strict)`.
6. Perform ! `MakeMethod(closure, object)`.
7. Let *prototype* be ! `ObjectCreate(%AsyncGeneratorPrototype%)`.
8. Perform ! `DefinePropertyOrThrow(closure, "prototype", PropertyDescriptor { [[Value]]: prototype, [[Writable]]: true, [[Enumerable]]: false, [[Configurable]]: false })`.
9. Perform ! `SetFunctionName(closure, propKey)`.
10. Set *closure*.`[[SourceText]]` to the source text matched by *AsyncGeneratorMethod*.
11. Let *desc* be `PropertyDescriptor { [[Value]]: closure, [[Writable]]: true, [[Enumerable]]: enumerable, [[Configurable]]: true }`.
12. Return ? `DefinePropertyOrThrow(object, propKey, desc)`.

14.5.13 Runtime Semantics: NamedEvaluation

With parameter *name*.

AsyncGeneratorExpression : **async function** * (*FormalParameters*) { *AsyncGeneratorBody* }

1. Let *closure* be the result of evaluating this *AsyncGeneratorExpression*.
2. Perform `SetFunctionName(closure, name)`.
3. Return *closure*.

14.5.14 Runtime Semantics: Evaluation

AsyncGeneratorExpression : **async function** * (*FormalParameters*) { *AsyncGeneratorBody* }

1. If the function code for this *AsyncGeneratorExpression* is **strict mode code**, let *strict* be **true**. Otherwise let *strict* be **false**.
2. Let *scope* be the **LexicalEnvironment** of the **running execution context**.

3. Let *closure* be ! *AsyncGeneratorFunctionCreate*(*Normal*, *FormalParameters*, *AsyncGeneratorBody*, *scope*, *strict*).
4. Let *prototype* be ! *ObjectCreate*(%*AsyncGeneratorPrototype*%).
5. Perform ! *DefinePropertyOrThrow*(*closure*, "**prototype**", PropertyDescriptor { [[Value]]: *prototype*, [[Writable]]: **true**, [[Enumerable]]: **false**, [[Configurable]]: **false** }).
6. Set *closure*.[[SourceText]] to the source text matched by *AsyncGeneratorExpression*.
7. Return *closure*.

AsyncGeneratorExpression : **async function** * *BindingIdentifier* (*FormalParameters*) {
AsyncGeneratorBody }

1. If the function code for this *AsyncGeneratorExpression* is *strict mode code*, let *strict* be **true**. Otherwise let *strict* be **false**.
2. Let *scope* be the *running execution context*'s *LexicalEnvironment*.
3. Let *funcEnv* be ! *NewDeclarativeEnvironment*(*scope*).
4. Let *envRec* be *funcEnv*'s *EnvironmentRecord*.
5. Let *name* be StringValue of *BindingIdentifier*.
6. Perform ! *envRec*.CreateImmutableBinding(*name*).
7. Let *closure* be ! *AsyncGeneratorFunctionCreate*(*Normal*, *FormalParameters*, *AsyncGeneratorBody*, *funcEnv*, *strict*).
8. Let *prototype* be ! *ObjectCreate*(%*AsyncGeneratorPrototype*%).
9. Perform ! *DefinePropertyOrThrow*(*closure*, "**prototype**", PropertyDescriptor { [[Value]]: *prototype*, [[Writable]]: **true**, [[Enumerable]]: **false**, [[Configurable]]: **false** }).
10. Perform ! *SetFunctionName*(*closure*, *name*).
11. Perform ! *envRec*.InitializeBinding(*name*, *closure*).
12. Set *closure*.[[SourceText]] to the source text matched by *AsyncGeneratorExpression*.
13. Return *closure*.

NOTE

The *BindingIdentifier* in an *AsyncGeneratorExpression* can be referenced from inside the *AsyncGeneratorExpression*'s *AsyncGeneratorBody* to allow the generator code to call itself recursively. However, unlike in an *AsyncGeneratorDeclaration*, the *BindingIdentifier* in an *AsyncGeneratorExpression* cannot be referenced from and does not affect the scope enclosing the *AsyncGeneratorExpression*.

14.6 Class Definitions

Syntax

```

ClassDeclaration[Yield, Await, Default] :
    class BindingIdentifier[?Yield, ?Await] ClassTail[?Yield, ?Await]
    [+Default] class ClassTail[?Yield, ?Await]

ClassExpression[Yield, Await] :
    class BindingIdentifier[?Yield, ?Await] opt ClassTail[?Yield, ?Await]

ClassTail[Yield, Await] :
    ClassHeritage[?Yield, ?Await] opt { ClassBody[?Yield, ?Await] opt }
```

```

ClassHeritage[Yield, Await] :
    extends LeftHandSideExpression[?Yield, ?Await]

ClassBody[Yield, Await] :
    ClassElementList[?Yield, ?Await]

ClassElementList[Yield, Await] :
    ClassElement[?Yield, ?Await]
    ClassElementList[?Yield, ?Await] ClassElement[?Yield, ?Await]

ClassElement[Yield, Await] :
    MethodDefinition[?Yield, ?Await]
    static MethodDefinition[?Yield, ?Await]
    ;

```

NOTE

A class definition is always [strict mode code](#).

14.6.1 Static Semantics: Early Errors

ClassTail : *ClassHeritage* { *ClassBody* }

It is a Syntax Error if *ClassHeritage* is not present and the following algorithm evaluates to **true**:

1. Let *constructor* be ConstructorMethod of *ClassBody*.
2. If *constructor* is empty, return **false**.
3. Return HasDirectSuper of *constructor*.

ClassBody : *ClassElementList*

It is a Syntax Error if PrototypePropertyNameList of *ClassElementList* contains more than one occurrence of **"constructor"**.

ClassElement : *MethodDefinition*

It is a Syntax Error if PropName of *MethodDefinition* is not **"constructor"** and HasDirectSuper of *MethodDefinition* is **true**.

It is a Syntax Error if PropName of *MethodDefinition* is **"constructor"** and SpecialMethod of *MethodDefinition* is **true**.

ClassElement : **static** *MethodDefinition*

It is a Syntax Error if HasDirectSuper of *MethodDefinition* is **true**.

It is a Syntax Error if PropName of *MethodDefinition* is **"prototype"**.

14.6.2 Static Semantics: BoundNames

ClassDeclaration : **class** *BindingIdentifier* *ClassTail*

1. Return the BoundNames of *BindingIdentifier*.

ClassDeclaration : **class** *ClassTail*

1. Return « **"*default*"** ».

14.6.3 Static Semantics: ConstructorMethod

ClassElementList : *ClassElement*

1. If *ClassElement* is *ClassElement* : ; , return empty.
2. If *IsStatic* of *ClassElement* is **true**, return empty.
3. If *PropName* of *ClassElement* is not **"constructor"**, return empty.
4. Return *ClassElement*.

ClassElementList : *ClassElementList* *ClassElement*

1. Let *head* be *ConstructorMethod* of *ClassElementList*.
2. If *head* is not empty, return *head*.
3. If *ClassElement* is *ClassElement* : ; , return empty.
4. If *IsStatic* of *ClassElement* is **true**, return empty.
5. If *PropName* of *ClassElement* is not **"constructor"**, return empty.
6. Return *ClassElement*.

NOTE

Early Error rules ensure that there is only one method definition named **"constructor"** and that it is not an [accessor property](#) or generator definition.

14.6.4 Static Semantics: Contains

With parameter *symbol*.

ClassTail : *ClassHeritage* { *ClassBody* }

1. If *symbol* is *ClassBody*, return **true**.
2. If *symbol* is *ClassHeritage*, then
 - a. If *ClassHeritage* is present, return **true**; otherwise return **false**.
3. Let *inHeritage* be *ClassHeritage* Contains *symbol*.
4. If *inHeritage* is **true**, return **true**.
5. Return the result of *ComputedPropertyContains* for *ClassBody* with argument *symbol*.

NOTE

Static semantic rules that depend upon substructure generally do not look into class bodies except for *PropertyNames*.

14.6.5 Static Semantics: ComputedPropertyContains

With parameter *symbol*.

ClassElementList : *ClassElementList* *ClassElement*

1. Let *inList* be the result of *ComputedPropertyContains* for *ClassElementList* with argument *symbol*.

2. If *inList* is **true**, return **true**.
3. Return the result of *ComputedPropertyContains* for *ClassElement* with argument *symbol*.

ClassElement : *MethodDefinition*

1. Return the result of *ComputedPropertyContains* for *MethodDefinition* with argument *symbol*.

ClassElement : **static** *MethodDefinition*

1. Return the result of *ComputedPropertyContains* for *MethodDefinition* with argument *symbol*.

ClassElement : ;

1. Return **false**.

14.6.6 Static Semantics: HasName

ClassExpression : **class** *ClassTail*

1. Return **false**.

ClassExpression : **class** *BindingIdentifier* *ClassTail*

1. Return **true**.

14.6.7 Static Semantics: IsConstantDeclaration

ClassDeclaration : **class** *BindingIdentifier* *ClassTail*

ClassDeclaration : **class** *ClassTail*

1. Return **false**.

14.6.8 Static Semantics: IsFunctionDefinition

ClassExpression : **class** *BindingIdentifier* *ClassTail*

1. Return **true**.

14.6.9 Static Semantics: IsStatic

ClassElement : *MethodDefinition*

1. Return **false**.

ClassElement : **static** *MethodDefinition*

1. Return **true**.

ClassElement : ;

1. Return **false**.

14.6.10 Static Semantics: NonConstructorMethodDefinitions

ClassElementList : *ClassElement*

1. If *ClassElement* is *ClassElement* : ; , return a new empty [List](#).
2. If *IsStatic* of *ClassElement* is **false** and *PropName* of *ClassElement* is "**constructor**", return a new empty [List](#).
3. Return a [List](#) containing *ClassElement*.

ClassElementList : *ClassElementList* *ClassElement*

1. Let *list* be *NonConstructorMethodDefinitions* of *ClassElementList*.
2. If *ClassElement* is *ClassElement* : ; , return *list*.
3. If *IsStatic* of *ClassElement* is **false** and *PropName* of *ClassElement* is "**constructor**", return *list*.
4. Append *ClassElement* to the end of *list*.
5. Return *list*.

14.6.11 Static Semantics: PrototypePropertyNameList

ClassElementList : *ClassElement*

1. If *PropName* of *ClassElement* is **empty**, return a new empty [List](#).
2. If *IsStatic* of *ClassElement* is **true**, return a new empty [List](#).
3. Return a [List](#) containing *PropName* of *ClassElement*.

ClassElementList : *ClassElementList* *ClassElement*

1. Let *list* be *PrototypePropertyNameList* of *ClassElementList*.
2. If *PropName* of *ClassElement* is **empty**, return *list*.
3. If *IsStatic* of *ClassElement* is **true**, return *list*.
4. Append *PropName* of *ClassElement* to the end of *list*.
5. Return *list*.

14.6.12 Static Semantics: PropName

ClassElement : ;

1. Return **empty**.

14.6.13 Runtime Semantics: ClassDefinitionEvaluation

With parameters *classBinding* and *className*.

ClassTail : *ClassHeritage* { *ClassBody* }

1. Let *lex* be the *LexicalEnvironment* of the [running execution context](#).
2. Let *classScope* be [NewDeclarativeEnvironment](#)(*lex*).
3. Let *classScopeEnvRec* be *classScope*'s *EnvironmentRecord*.
4. If *classBinding* is not **undefined**, then
 - a. Perform *classScopeEnvRec*.*CreateImmutableBinding*(*classBinding*, **true**).
5. If *ClassHeritage*_{opt} is not present, then
 - a. Let *protoParent* be the intrinsic object [%ObjectPrototype%](#).
 - b. Let *constructorParent* be the intrinsic object [%FunctionPrototype%](#).
6. Else,
 - a. Set the [running execution context](#)'s *LexicalEnvironment* to *classScope*.
 - b. Let *superclassRef* be the result of evaluating *ClassHeritage*.

- c. Set the **running execution context**'s LexicalEnvironment to *lex*.
- d. Let *superclass* be ? GetValue(*superclassRef*).
- e. If *superclass* is **null**, then
 - i. Let *protoParent* be **null**.
 - ii. Let *constructorParent* be the intrinsic object %FunctionPrototype%.
- f. Else if IsConstructor(*superclass*) is **false**, throw a **TypeError** exception.
- g. Else,
 - i. Let *protoParent* be ? Get(*superclass*, "prototype").
 - ii. If Type(*protoParent*) is neither Object nor Null, throw a **TypeError** exception.
 - iii. Let *constructorParent* be *superclass*.
7. Let *proto* be ObjectCreate(*protoParent*).
8. If *ClassBody*_{opt} is not present, let *constructor* be empty.
9. Else, let *constructor* be ConstructorMethod of *ClassBody*.
10. If *constructor* is empty, then
 - a. If *ClassHeritage*_{opt} is present, then
 - i. Set *constructor* to the result of parsing the source text


```
constructor(... args){ super (...args);}
```

 using the syntactic grammar with the **goal symbol** *MethodDefinition*_[~Yield, ~Await] .
 - b. Else,
 - i. Set *constructor* to the result of parsing the source text


```
constructor() { }
```

 using the syntactic grammar with the **goal symbol** *MethodDefinition*_[~Yield, ~Await] .
11. Set the **running execution context**'s LexicalEnvironment to *classScope*.
12. Let *constructorInfo* be the result of performing DefineMethod for *constructor* with arguments *proto* and *constructorParent* as the optional *functionPrototype* argument.
13. **Assert**: *constructorInfo* is not an abrupt completion.
14. Let *F* be *constructorInfo*.[[Closure]].
15. If *ClassHeritage*_{opt} is present, set *F*.[[ConstructorKind]] to "**derived**".
16. Perform MakeConstructor(*F*, **false**, *proto*).
17. Perform MakeClassConstructor(*F*).
18. If *className* is not **undefined**, then
 - a. Perform SetFunctionName(*F*, *className*).
19. Perform CreateMethodProperty(*proto*, "**constructor**", *F*).
20. If *ClassBody*_{opt} is not present, let *methods* be a new empty List.
21. Else, let *methods* be NonConstructorMethodDefinitions of *ClassBody*.
22. For each *ClassElement* *m* in order from *methods*, do
 - a. If IsStatic of *m* is **false**, then
 - i. Let *status* be the result of performing PropertyDefinitionEvaluation for *m* with arguments *proto* and **false**.
 - b. Else,
 - i. Let *status* be the result of performing PropertyDefinitionEvaluation for *m* with arguments *F* and **false**.
 - c. If *status* is an abrupt completion, then
 - i. Set the **running execution context**'s LexicalEnvironment to *lex*.
 - ii. Return Completion(*status*).
23. Set the **running execution context**'s LexicalEnvironment to *lex*.

24. If *classBinding* is not **undefined**, then
 - a. Perform *classScopeEnvRec*.InitializeBinding(*classBinding*, *F*).
25. Return *F*.

14.6.14 Runtime Semantics: BindingClassDeclarationEvaluation

ClassDeclaration : **class** *BindingIdentifier* *ClassTail*

1. Let *className* be StringValue of *BindingIdentifier*.
2. Let *value* be the result of ClassDefinitionEvaluation of *ClassTail* with arguments *className* and *className*.
3. ReturnIfAbrupt(*value*).
4. Set *value*.[[SourceText]] to the source text matched by *ClassDeclaration*.
5. Let *env* be the running execution context's LexicalEnvironment.
6. Perform ? InitializeBoundName(*className*, *value*, *env*).
7. Return *value*.

ClassDeclaration : **class** *ClassTail*

1. Let *value* be the result of ClassDefinitionEvaluation of *ClassTail* with arguments **undefined** and **"default"**.
2. ReturnIfAbrupt(*value*).
3. Set *value*.[[SourceText]] to the source text matched by *ClassDeclaration*.
4. Return *value*.

NOTE

ClassDeclaration : **class** *ClassTail* only occurs as part of an *ExportDeclaration* and establishing its binding is handled as part of the evaluation action for that production. See 15.2.3.11.

14.6.15 Runtime Semantics: NamedEvaluation

With parameter *name*.

ClassExpression : **class** *ClassTail*

1. Return the result of ClassDefinitionEvaluation of *ClassTail* with arguments **undefined** and *name*.

14.6.16 Runtime Semantics: Evaluation

ClassDeclaration : **class** *BindingIdentifier* *ClassTail*

1. Perform ? BindingClassDeclarationEvaluation of this *ClassDeclaration*.
2. Return NormalCompletion(empty).

NOTE

ClassDeclaration : **class** *ClassTail* only occurs as part of an *ExportDeclaration* and is never directly evaluated.

ClassExpression : **class** *BindingIdentifier* *ClassTail*

1. If *BindingIdentifier*_{opt} is not present, let *className* be **undefined**.
2. Else, let *className* be StringValue of *BindingIdentifier*.
3. Let *value* be the result of ClassDefinitionEvaluation of *ClassTail* with arguments *className* and *className*.
4. ReturnIfAbrupt(*value*).

5. Set *value*.[[SourceText]] to the source text matched by *ClassExpression*.
6. Return *value*.

14.7 Async Function Definitions

Syntax

```

AsyncFunctionDeclaration[Yield, Await, Default] :
  async [no LineTerminator here] function BindingIdentifier[?Yield, ?Await] (
    FormalParameters[~Yield, +Await] ) { AsyncFunctionBody }
  [+Default] async [no LineTerminator here] function ( FormalParameters[~Yield, +Await] ) {
    AsyncFunctionBody }

```

```

AsyncFunctionExpression :
  async [no LineTerminator here] function ( FormalParameters[~Yield, +Await] ) {
    AsyncFunctionBody }
  async [no LineTerminator here] function BindingIdentifier[~Yield, +Await] (
    FormalParameters[~Yield, +Await] ) { AsyncFunctionBody }

```

```

AsyncMethod[Yield, Await] :
  async [no LineTerminator here] PropertyName[?Yield, ?Await] (
    UniqueFormalParameters[~Yield, +Await] ) { AsyncFunctionBody }

```

```

AsyncFunctionBody :
  FunctionBody[~Yield, +Await]

```

```

AwaitExpression[Yield] :
  await UnaryExpression[?Yield, +Await]

```

NOTE 1

await is parsed as an *AwaitExpression* when the _[Await] parameter is present. The _[Await] parameter is present in the following contexts:

In an *AsyncFunctionBody*.

In the *FormalParameters* of an *AsyncFunctionDeclaration*, *AsyncFunctionExpression*, *AsyncGeneratorDeclaration*, or *AsyncGeneratorExpression*. *AwaitExpression* in this position is a Syntax error via [static semantics](#).

When *Module* is the syntactic [goal symbol](#) and the _[Await] parameter is absent, **await** is parsed as a keyword and will be a Syntax error. When *Script* is the syntactic [goal symbol](#), **await** may be parsed as an identifier when the _[Await] parameter is absent. This includes the following contexts:

Anywhere outside of an *AsyncFunctionBody* or *FormalParameters* of an *AsyncFunctionDeclaration*, *AsyncFunctionExpression*, *AsyncGeneratorDeclaration*, or *AsyncGeneratorExpression*.

In the *BindingIdentifier* of a *FunctionExpression*, *GeneratorExpression*, or *AsyncGeneratorExpression*.

NOTE 2

Unlike *YieldExpression*, it is a Syntax Error to omit the operand of an *AwaitExpression*. You must await something.

14.7.1 Static Semantics: Early Errors

AsyncMethod : **async** *PropertyName* (*UniqueFormalParameters*) { *AsyncFunctionBody* }

It is a Syntax Error if *ContainsUseStrict* of *AsyncFunctionBody* is **true** and *IsSimpleParameterList* of *UniqueFormalParameters* is **false**.

It is a Syntax Error if *HasDirectSuper* of *AsyncMethod* is **true**.

It is a Syntax Error if *UniqueFormalParameters* *ContainsAwaitExpression* is **true**.

It is a Syntax Error if any element of the *BoundNames* of *UniqueFormalParameters* also occurs in the *LexicallyDeclaredNames* of *AsyncFunctionBody*.

AsyncFunctionDeclaration : **async function** *BindingIdentifier* (*FormalParameters*) { *AsyncFunctionBody* }

AsyncFunctionDeclaration : **async function** (*FormalParameters*) { *AsyncFunctionBody* }

AsyncFunctionExpression : **async function** (*FormalParameters*) { *AsyncFunctionBody* }

AsyncFunctionExpression : **async function** *BindingIdentifier* (*FormalParameters*) { *AsyncFunctionBody* }

It is a Syntax Error if *ContainsUseStrict* of *AsyncFunctionBody* is **true** and *IsSimpleParameterList* of *FormalParameters* is **false**.

It is a Syntax Error if *FormalParameters* *ContainsAwaitExpression* is **true**.

If the source code matching this production is strict code, the Early Error rules for *UniqueFormalParameters* : *FormalParameters* are applied.

If the source code matching this production is strict code, it is a Syntax Error if *BindingIdentifier* is present and the *StringValue* of *BindingIdentifier* is **"eval"** or **"arguments"**.

It is a Syntax Error if any element of the *BoundNames* of *FormalParameters* also occurs in the *LexicallyDeclaredNames* of *AsyncFunctionBody*.

It is a Syntax Error if *FormalParameters* *ContainsSuperProperty* is **true**.

It is a Syntax Error if *AsyncFunctionBody* *ContainsSuperProperty* is **true**.

It is a Syntax Error if *FormalParameters* *ContainsSuperCall* is **true**.

It is a Syntax Error if *AsyncFunctionBody* *ContainsSuperCall* is **true**.

14.7.2 Static Semantics: BoundNames

AsyncFunctionDeclaration : **async function** *BindingIdentifier* (*FormalParameters*) { *AsyncFunctionBody* }

1. Return the *BoundNames* of *BindingIdentifier*.

AsyncFunctionDeclaration : **async function** (*FormalParameters*) { *AsyncFunctionBody* }

1. Return « **"*default*"** ».

NOTE

"*default*" is used within this specification as a synthetic name for hoistable anonymous functions that are defined using export declarations.

14.7.3 Static Semantics: ComputedPropertyContains

With parameter *symbol*.

AsyncMethod : **async** *PropertyName* (*UniqueFormalParameters*) { *AsyncFunctionBody* }

1. Return the result of *ComputedPropertyContains* for *PropertyName* with argument *symbol*.

14.7.4 Static Semantics: Contains

With parameter *symbol*.

AsyncFunctionDeclaration : **async function** *BindingIdentifier* (*FormalParameters*) { *AsyncFunctionBody* }

AsyncFunctionDeclaration : **async function** (*FormalParameters*) { *AsyncFunctionBody* }

AsyncFunctionExpression : **async function** (*FormalParameters*) { *AsyncFunctionBody* }

AsyncFunctionExpression : **async function** *BindingIdentifier* (*FormalParameters*) { *AsyncFunctionBody* }

1. Return **false**.

14.7.5 Static Semantics: HasDirectSuper

AsyncMethod : **async** *PropertyName* (*UniqueFormalParameters*) { *AsyncFunctionBody* }

1. If *UniqueFormalParameters* Contains *SuperCall* is **true**, return **true**.
2. Return *AsyncFunctionBody* Contains *SuperCall*.

14.7.6 Static Semantics: HasName

AsyncFunctionExpression : **async function** (*FormalParameters*) { *AsyncFunctionBody* }

1. Return **false**.

AsyncFunctionExpression : **async function** *BindingIdentifier* (*FormalParameters*) { *AsyncFunctionBody* }

1. Return **true**.

14.7.7 Static Semantics: IsConstantDeclaration

AsyncFunctionDeclaration : **async function** *BindingIdentifier* (*FormalParameters*) { *AsyncFunctionBody* }

AsyncFunctionDeclaration : **async function** (*FormalParameters*) { *AsyncFunctionBody* }

1. Return **false**.

14.7.8 Static Semantics: IsFunctionDefinition

AsyncFunctionExpression : **async function** (*FormalParameters*) { *AsyncFunctionBody* }

AsyncFunctionExpression : **async function** *BindingIdentifier* (*FormalParameters*) { *AsyncFunctionBody* }

1. Return **true**.

14.7.9 Static Semantics: PropName

AsyncMethod : **async** *PropertyName* (*UniqueFormalParameters*) { *AsyncFunctionBody* }

1. Return *PropName* of *PropertyName*.

14.7.10 Runtime Semantics: InstantiateFunctionObject

With parameter *scope*.

AsyncFunctionDeclaration : **async function** *BindingIdentifier* (*FormalParameters*) { *AsyncFunctionBody* }

1. If the function code for *AsyncFunctionDeclaration* is **strict mode code**, let *strict* be **true**. Otherwise, let *strict* be **false**.
2. Let *name* be StringValue of *BindingIdentifier*.
3. Let *F* be ! **AsyncFunctionCreate**(Normal, *FormalParameters*, *AsyncFunctionBody*, *scope*, *strict*).
4. Perform ! **SetFunctionName**(*F*, *name*).
5. Set *F*.[[SourceText]] to the source text matched by *AsyncFunctionDeclaration*.
6. Return *F*.

AsyncFunctionDeclaration : **async function** (*FormalParameters*) { *AsyncFunctionBody* }

1. If the function code for *AsyncFunctionDeclaration* is **strict mode code**, let *strict* be **true**. Otherwise, let *strict* be **false**.
2. Let *F* be ! **AsyncFunctionCreate**(Normal, *FormalParameters*, *AsyncFunctionBody*, *scope*, *strict*).
3. Perform ! **SetFunctionName**(*F*, "default").
4. Set *F*.[[SourceText]] to the source text matched by *AsyncFunctionDeclaration*.
5. Return *F*.

14.7.11 Runtime Semantics: EvaluateBody

With parameters *functionObject* and List *argumentsList*.

AsyncFunctionBody : *FunctionBody*

1. Let *promiseCapability* be ! **NewPromiseCapability**(%Promise%).
2. Let *declResult* be **FunctionDeclarationInstantiation**(*functionObject*, *argumentsList*).
3. If *declResult* is not an abrupt completion, then
 - a. Perform ! **AsyncFunctionStart**(*promiseCapability*, *FunctionBody*).
4. Else *declResult* is an abrupt completion,
 - a. Perform ! **Call**(*promiseCapability*.[[Reject]], **undefined**, « *declResult*.[[Value]] »).
5. Return **Completion** { [[Type]]: return, [[Value]]: *promiseCapability*.[[Promise]], [[Target]]: empty }.

14.7.12 Runtime Semantics: PropertyDefinitionEvaluation

With parameters *object* and *enumerable*.

AsyncMethod : **async** *PropertyName* (*UniqueFormalParameters*) { *AsyncFunctionBody* }

1. Let *propKey* be the result of evaluating *PropertyName*.
2. **ReturnIfAbrupt**(*propKey*).
3. If the function code for this *AsyncMethod* is **strict mode code**, let *strict* be **true**. Otherwise let *strict* be **false**.
4. Let *scope* be the LexicalEnvironment of the **running execution context**.
5. Let *closure* be ! **AsyncFunctionCreate**(Method, *UniqueFormalParameters*, *AsyncFunctionBody*, *scope*, *strict*).
6. Perform ! **MakeMethod**(*closure*, *object*).
7. Perform ! **SetFunctionName**(*closure*, *propKey*).
8. Set *closure*.[[SourceText]] to the source text matched by *AsyncMethod*.
9. Let *desc* be the PropertyDescriptor { [[Value]]: *closure*, [[Writable]]: **true**, [[Enumerable]]: *enumerable*, [[Configurable]]: **true** }.
10. Return ? **DefinePropertyOrThrow**(*object*, *propKey*, *desc*).

14.7.13 Runtime Semantics: NamedEvaluation

With parameter *name*.

AsyncFunctionExpression : **async function** (*FormalParameters*) { *AsyncFunctionBody* }

1. Let *closure* be the result of evaluating this *AsyncFunctionExpression*.
2. Perform **SetFunctionName**(*closure*, *name*).
3. Return *closure*.

14.7.14 Runtime Semantics: Evaluation

AsyncFunctionDeclaration : **async function** *BindingIdentifier* (*FormalParameters*) { *AsyncFunctionBody* }

1. Return **NormalCompletion**(empty).

AsyncFunctionDeclaration : **async function** (*FormalParameters*) { *AsyncFunctionBody* }

1. Return **NormalCompletion**(empty).

AsyncFunctionExpression : **async function** (*FormalParameters*) { *AsyncFunctionBody* }

1. If the function code for *AsyncFunctionExpression* is **strict mode code**, let *strict* be **true**. Otherwise let *strict* be **false**.
2. Let *scope* be the **LexicalEnvironment** of the **running execution context**.
3. Let *closure* be ! **AsyncFunctionCreate**(**Normal**, *FormalParameters*, *AsyncFunctionBody*, *scope*, *strict*).
4. Set *closure*.[[SourceText]] to the source text matched by *AsyncFunctionExpression*.
5. Return *closure*.

AsyncFunctionExpression : **async function** *BindingIdentifier* (*FormalParameters*) { *AsyncFunctionBody* }

1. If the function code for *AsyncFunctionExpression* is **strict mode code**, let *strict* be **true**. Otherwise let *strict* be **false**.
2. Let *scope* be the **LexicalEnvironment** of the **running execution context**.
3. Let *funcEnv* be ! **NewDeclarativeEnvironment**(*scope*).
4. Let *envRec* be *funcEnv*'s **EnvironmentRecord**.
5. Let *name* be **StringValue** of *BindingIdentifier*.
6. Perform ! *envRec*.**CreateImmutableBinding**(*name*).
7. Let *closure* be ! **AsyncFunctionCreate**(**Normal**, *FormalParameters*, *AsyncFunctionBody*, *funcEnv*, *strict*).
8. Perform ! **SetFunctionName**(*closure*, *name*).
9. Perform ! *envRec*.**InitializeBinding**(*name*, *closure*).
10. Set *closure*.[[SourceText]] to the source text matched by *AsyncFunctionExpression*.
11. Return *closure*.

AwaitExpression : **await** *UnaryExpression*

1. Let *exprRef* be the result of evaluating *UnaryExpression*.
2. Let *value* be ? **GetValue**(*exprRef*).
3. Return ? **Await**(*value*).

14.8 Async Arrow Function Definitions

Syntax

*AsyncArrowFunction*_[In, Yield, Await] :

async [no *LineTerminator* here] *AsyncArrowBindingIdentifier*_[?Yield] [no *LineTerminator* here] =>
 *AsyncConciseBody*_[?In]

*CoverCallExpressionAndAsyncArrowHead*_[?Yield, ?Await] [no *LineTerminator* here] =>
 *AsyncConciseBody*_[?In]

*AsyncConciseBody*_[In] :

[lookahead ≠ { }] *AssignmentExpression*_[?In, ~Yield, +Await]
{ *AsyncFunctionBody* }

*AsyncArrowBindingIdentifier*_[Yield] :

*BindingIdentifier*_[?Yield, +Await]

*CoverCallExpressionAndAsyncArrowHead*_[Yield, Await] :

*MemberExpression*_[?Yield, ?Await] *Arguments*_[?Yield, ?Await]

Supplemental Syntax

When processing an instance of the production *AsyncArrowFunction* : *CoverCallExpressionAndAsyncArrowHead* => *AsyncConciseBody* the interpretation of *CoverCallExpressionAndAsyncArrowHead* is refined using the following grammar:

AsyncArrowHead :

async [no *LineTerminator* here] *ArrowFormalParameters*_[~Yield, +Await]

14.8.1 Static Semantics: Early Errors

AsyncArrowFunction : **async** *AsyncArrowBindingIdentifier* => *AsyncConciseBody*

It is a Syntax Error if any element of the BoundNames of *AsyncArrowBindingIdentifier* also occurs in the LexicallyDeclaredNames of *AsyncConciseBody*.

AsyncArrowFunction : *CoverCallExpressionAndAsyncArrowHead* => *AsyncConciseBody*

It is a Syntax Error if *CoverCallExpressionAndAsyncArrowHead* Contains *YieldExpression* is **true**.

It is a Syntax Error if *CoverCallExpressionAndAsyncArrowHead* Contains *AwaitExpression* is **true**.

It is a Syntax Error if *CoverCallExpressionAndAsyncArrowHead* is not **covering** an *AsyncArrowHead*.

It is a Syntax Error if any element of the BoundNames of *CoverCallExpressionAndAsyncArrowHead* also occurs in the LexicallyDeclaredNames of *AsyncConciseBody*.

It is a Syntax Error if ContainsUseStrict of *AsyncConciseBody* is **true** and IsSimpleParameterList of *CoverCallExpressionAndAsyncArrowHead* is **false**.

All Early Error rules for *AsyncArrowHead* and its derived productions apply to CoveredAsyncArrowHead of *CoverCallExpressionAndAsyncArrowHead*.

14.8.2 Static Semantics: CoveredAsyncArrowHead

CoverCallExpressionAndAsyncArrowHead : *MemberExpression* *Arguments*

1. Return the *AsyncArrowHead* that is **covered** by *CoverCallExpressionAndAsyncArrowHead*.

14.8.3 Static Semantics: BoundNames

CoverCallExpressionAndAsyncArrowHead : MemberExpression Arguments

1. Let *head* be CoveredAsyncArrowHead of *CoverCallExpressionAndAsyncArrowHead*.
2. Return the BoundNames of *head*.

14.8.4 Static Semantics: Contains

With parameter *symbol*.

AsyncArrowFunction : async AsyncArrowBindingIdentifier => AsyncConciseBody

1. If *symbol* is not one of *NewTarget*, *SuperProperty*, *SuperCall*, **super**, or **this**, return **false**.
2. Return *AsyncConciseBody* Contains *symbol*.

AsyncArrowFunction : CoverCallExpressionAndAsyncArrowHead => AsyncConciseBody

1. If *symbol* is not one of *NewTarget*, *SuperProperty*, *SuperCall*, **super**, or **this**, return **false**.
2. Let *head* be CoveredAsyncArrowHead of *CoverCallExpressionAndAsyncArrowHead*.
3. If *head* Contains *symbol* is **true**, return **true**.
4. Return *AsyncConciseBody* Contains *symbol*.

NOTE

Normally, Contains does not look inside most function forms. However, Contains is used to detect **new.target**, **this**, and **super** usage within an AsyncArrowFunction.

14.8.5 Static Semantics: ContainsExpression

AsyncArrowBindingIdentifier : BindingIdentifier

1. Return **false**.

14.8.6 Static Semantics: ExpectedArgumentCount

AsyncArrowBindingIdentifier : BindingIdentifier

1. Return 1.

14.8.7 Static Semantics: HasName

AsyncArrowFunction : async AsyncArrowBindingIdentifier => AsyncConciseBody

AsyncArrowFunction : CoverCallExpressionAndAsyncArrowHead => AsyncConciseBody

1. Return **false**.

14.8.8 Static Semantics: IsSimpleParameterList

AsyncArrowBindingIdentifier : BindingIdentifier

1. Return **true**.

CoverCallExpressionAndAsyncArrowHead : MemberExpression Arguments

1. Let *head* be CoveredAsyncArrowHead of *CoverCallExpressionAndAsyncArrowHead*.
2. Return *IsSimpleParameterList* of *head*.

14.8.9 Static Semantics: LexicallyDeclaredNames

AsyncConciseBody : *AssignmentExpression*

1. Return a new empty *List*.

14.8.10 Static Semantics: LexicallyScopedDeclarations

AsyncConciseBody : *AssignmentExpression*

1. Return a new empty *List*.

14.8.11 Static Semantics: VarDeclaredNames

AsyncConciseBody : *AssignmentExpression*

1. Return a new empty *List*.

14.8.12 Static Semantics: VarScopedDeclarations

AsyncConciseBody : *AssignmentExpression*

1. Return a new empty *List*.

14.8.13 Runtime Semantics: IteratorBindingInitialization

With parameters *iteratorRecord* and *environment*.

AsyncArrowBindingIdentifier : *BindingIdentifier*

1. Assert: *iteratorRecord*.[[Done]] is **false**.
2. Let *next* be *IteratorStep*(*iteratorRecord*).
3. If *next* is an abrupt completion, set *iteratorRecord*.[[Done]] to **true**.
4. *ReturnIfAbrupt*(*next*).
5. If *next* is **false**, set *iteratorRecord*.[[Done]] to **true**.
6. Else,
 - a. Let *v* be *IteratorValue*(*next*).
 - b. If *v* is an abrupt completion, set *iteratorRecord*.[[Done]] to **true**.
 - c. *ReturnIfAbrupt*(*v*).
7. If *iteratorRecord*.[[Done]] is **true**, let *v* be **undefined**.
8. Return the result of performing *BindingInitialization* for *BindingIdentifier* using *v* and *environment* as the arguments.

14.8.14 Runtime Semantics: EvaluateBody

With parameters *functionObject* and *List argumentsList*.

AsyncConciseBody : *AssignmentExpression*

1. Let *promiseCapability* be ! *NewPromiseCapability*(%Promise%).

2. Let *declResult* be *FunctionDeclarationInstantiation*(*functionObject*, *argumentsList*).
3. If *declResult* is not an *abrupt completion*, then
 - a. Perform ! *AsyncFunctionStart*(*promiseCapability*, *AssignmentExpression*).
4. Else *declResult* is an *abrupt completion*,
 - a. Perform ! *Call*(*promiseCapability*.[[Reject]], **undefined**, « *declResult*.[[Value]] »).
5. Return *Completion* { [[Type]]: *return*, [[Value]]: *promiseCapability*.[[Promise]], [[Target]]: *empty* }.

AsyncConciseBody : { *AsyncFunctionBody* }

1. Return the result of *EvaluateBody* of *AsyncFunctionBody* passing *functionObject* and *argumentsList* as the arguments.

14.8.15 Runtime Semantics: NamedEvaluation

With parameter *name*.

AsyncArrowFunction : **async** *AsyncArrowBindingIdentifier* => *AsyncConciseBody*

AsyncArrowFunction : *CoverCallExpressionAndAsyncArrowHead* => *AsyncConciseBody*

1. Let *closure* be the result of evaluating this *AsyncArrowFunction*.
2. Perform *SetFunctionName*(*closure*, *name*).
3. Return *closure*.

14.8.16 Runtime Semantics: Evaluation

AsyncArrowFunction : **async** *AsyncArrowBindingIdentifier* => *AsyncConciseBody*

1. If the function code for this *AsyncArrowFunction* is *strict mode code*, let *strict* be **true**. Otherwise, let *strict* be **false**.
2. Let *scope* be the *LexicalEnvironment* of the *running execution context*.
3. Let *parameters* be *AsyncArrowBindingIdentifier*.
4. Let *closure* be ! *AsyncFunctionCreate*(*Arrow*, *parameters*, *AsyncConciseBody*, *scope*, *strict*).
5. Return *closure*.

AsyncArrowFunction : *CoverCallExpressionAndAsyncArrowHead* => *AsyncConciseBody*

1. If the function code for this *AsyncArrowFunction* is *strict mode code*, let *strict* be **true**. Otherwise, let *strict* be **false**.
2. Let *scope* be the *LexicalEnvironment* of the *running execution context*.
3. Let *head* be *CoveredAsyncArrowHead* of *CoverCallExpressionAndAsyncArrowHead*.
4. Let *parameters* be the *ArrowFormalParameters* of *head*.
5. Let *closure* be ! *AsyncFunctionCreate*(*Arrow*, *parameters*, *AsyncConciseBody*, *scope*, *strict*).
6. Return *closure*.

14.9 Tail Position Calls

14.9.1 Static Semantics: IsInTailPosition (*call*)

The abstract operation *IsInTailPosition* with argument *call* performs the following steps:

1. **Assert:** *call* is a *Parse Node*.
2. If the source code matching *call* is *non-strict code*, return **false**.
3. If *call* is not contained within a *FunctionBody*, *ConciseBody*, or *AsyncConciseBody*, return **false**.
4. Let *body* be the *FunctionBody*, *ConciseBody*, or *AsyncConciseBody* that most closely contains *call*.
5. If *body* is the *FunctionBody* of a *GeneratorBody*, return **false**.
6. If *body* is the *FunctionBody* of an *AsyncFunctionBody*, return **false**.
7. If *body* is the *FunctionBody* of an *AsyncGeneratorBody*, return **false**.
8. If *body* is an *AsyncConciseBody*, return **false**.
9. Return the result of *HasCallInTailPosition* of *body* with argument *call*.

NOTE

Tail Position calls are only defined in *strict mode code* because of a common non-standard language extension (see 9.2.9) that enables observation of the chain of caller contexts.

14.9.2 Static Semantics: HasCallInTailPosition

With parameter *call*.

NOTE

call is a *Parse Node* that represents a specific range of source text. When the following algorithms compare *call* to another *Parse Node*, it is a test of whether they represent the same source text.

14.9.2.1 Statement Rules

ConciseBody : *AssignmentExpression*

1. Return *HasCallInTailPosition* of *AssignmentExpression* with argument *call*.

StatementList : *StatementList* *StatementListItem*

1. Let *has* be *HasCallInTailPosition* of *StatementList* with argument *call*.
2. If *has* is **true**, return **true**.
3. Return *HasCallInTailPosition* of *StatementListItem* with argument *call*.

FunctionStatementList : [empty]

StatementListItem : *Declaration*

Statement :

VariableStatement

EmptyStatement

ExpressionStatement

ContinueStatement

BreakStatement

ThrowStatement

DebuggerStatement

Block : { }

ReturnStatement : **return** ;

LabelledItem : *FunctionDeclaration*

IterationStatement :

for (*LeftHandSideExpression* **of** *AssignmentExpression*) *Statement*
for (**var** *ForBinding* **of** *AssignmentExpression*) *Statement*
for (*ForDeclaration* **of** *AssignmentExpression*) *Statement*

CaseBlock : { }

1. Return **false**.

IfStatement : **if** (*Expression*) *Statement* **else** *Statement*

1. Let *has* be *HasCallInTailPosition* of the first *Statement* with argument *call*.
2. If *has* is **true**, return **true**.
3. Return *HasCallInTailPosition* of the second *Statement* with argument *call*.

IfStatement : **if** (*Expression*) *Statement*

IterationStatement :

do *Statement* **while** (*Expression*) ;
while (*Expression*) *Statement*
for (*Expression*_{opt} ; *Expression*_{opt} ; *Expression*_{opt}) *Statement*
for (**var** *VariableDeclarationList* ; *Expression*_{opt} ; *Expression*_{opt}) *Statement*
for (*LexicalDeclaration* *Expression*_{opt} ; *Expression*_{opt}) *Statement*
for (*LeftHandSideExpression* **in** *Expression*) *Statement*
for (**var** *ForBinding* **in** *Expression*) *Statement*
for (*ForDeclaration* **in** *Expression*) *Statement*

WithStatement : **with** (*Expression*) *Statement*

1. Return *HasCallInTailPosition* of *Statement* with argument *call*.

LabelledStatement :

LabelIdentifier : *LabelledItem*

1. Return *HasCallInTailPosition* of *LabelledItem* with argument *call*.

ReturnStatement : **return** *Expression* ;

1. Return *HasCallInTailPosition* of *Expression* with argument *call*.

SwitchStatement : **switch** (*Expression*) *CaseBlock*

1. Return *HasCallInTailPosition* of *CaseBlock* with argument *call*.

CaseBlock : { *CaseClauses* *DefaultClause* *CaseClauses* }

1. Let *has* be **false**.
2. If the first *CaseClauses* is present, let *has* be *HasCallInTailPosition* of the first *CaseClauses* with argument *call*.
3. If *has* is **true**, return **true**.
4. Let *has* be *HasCallInTailPosition* of the *DefaultClause* with argument *call*.
5. If *has* is **true**, return **true**.
6. If the second *CaseClauses* is present, let *has* be *HasCallInTailPosition* of the second *CaseClauses* with argument *call*.

7. Return *has*.

CaseClauses : *CaseClauses* *CaseClause*

1. Let *has* be *HasCallInTailPosition* of *CaseClauses* with argument *call*.
2. If *has* is **true**, return **true**.
3. Return *HasCallInTailPosition* of *CaseClause* with argument *call*.

CaseClause : **case** *Expression* : *StatementList*

DefaultClause : **default** : *StatementList*

1. If *StatementList* is present, return *HasCallInTailPosition* of *StatementList* with argument *call*.
2. Return **false**.

TryStatement : **try** *Block* *Catch*

1. Return *HasCallInTailPosition* of *Catch* with argument *call*.

TryStatement : **try** *Block* *Finally*

TryStatement : **try** *Block* *Catch* *Finally*

1. Return *HasCallInTailPosition* of *Finally* with argument *call*.

Catch : **catch** (*CatchParameter*) *Block*

1. Return *HasCallInTailPosition* of *Block* with argument *call*.

14.9.2.2 Expression Rules

NOTE

A potential tail position call that is immediately followed by return *GetValue* of the call result is also a possible tail position call. Function calls cannot return reference values, so such a *GetValue* operation will always return the same value as the actual function call result.

AssignmentExpression :

YieldExpression

ArrowFunction

AsyncArrowFunction

LeftHandSideExpression = *AssignmentExpression*

LeftHandSideExpression *AssignmentOperator* *AssignmentExpression*

BitwiseANDExpression : *BitwiseANDExpression* & *EqualityExpression*

BitwiseXORExpression : *BitwiseXORExpression* ^ *BitwiseANDExpression*

BitwiseORExpression : *BitwiseORExpression* | *BitwiseXORExpression*

EqualityExpression :

EqualityExpression == *RelationalExpression*

EqualityExpression != *RelationalExpression*

EqualityExpression === *RelationalExpression*

EqualityExpression !== *RelationalExpression*

RelationalExpression :

RelationalExpression < *ShiftExpression*
RelationalExpression > *ShiftExpression*
RelationalExpression <= *ShiftExpression*
RelationalExpression >= *ShiftExpression*
RelationalExpression **instanceof** *ShiftExpression*
RelationalExpression **in** *ShiftExpression*

ShiftExpression :

ShiftExpression << *AdditiveExpression*
ShiftExpression >> *AdditiveExpression*
ShiftExpression >>> *AdditiveExpression*

AdditiveExpression :

AdditiveExpression + *MultiplicativeExpression*
AdditiveExpression - *MultiplicativeExpression*

MultiplicativeExpression :

MultiplicativeExpression *MultiplicativeOperator* *ExponentiationExpression*

ExponentiationExpression :

UpdateExpression ** *ExponentiationExpression*

UpdateExpression :

LeftHandSideExpression ++
LeftHandSideExpression --
++ *UnaryExpression*
-- *UnaryExpression*

UnaryExpression :

delete *UnaryExpression*
void *UnaryExpression*
typeof *UnaryExpression*
+ *UnaryExpression*
- *UnaryExpression*
~ *UnaryExpression*
! *UnaryExpression*
AwaitExpression

CallExpression :

SuperCall
CallExpression [*Expression*]
CallExpression . *IdentifierName*

NewExpression : **new** *NewExpression*

MemberExpression :

MemberExpression [*Expression*]
MemberExpression . *IdentifierName*
SuperProperty
MetaProperty

new *MemberExpression* *Arguments*

PrimaryExpression :

this

IdentifierReference

Literal

ArrayLiteral

ObjectLiteral

FunctionExpression

ClassExpression

GeneratorExpression

AsyncFunctionExpression

AsyncGeneratorExpression

RegularExpressionLiteral

TemplateLiteral

1. Return **false**.

Expression :

AssignmentExpression

Expression , *AssignmentExpression*

1. Return *HasCallInTailPosition* of *AssignmentExpression* with argument *call*.

ConditionalExpression : *LogicalORExpression* ? *AssignmentExpression* : *AssignmentExpression*

1. Let *has* be *HasCallInTailPosition* of the first *AssignmentExpression* with argument *call*.
2. If *has* is **true**, return **true**.
3. Return *HasCallInTailPosition* of the second *AssignmentExpression* with argument *call*.

LogicalANDExpression : *LogicalANDExpression* && *BitwiseORExpression*

1. Return *HasCallInTailPosition* of *BitwiseORExpression* with argument *call*.

LogicalORExpression : *LogicalORExpression* || *LogicalANDExpression*

1. Return *HasCallInTailPosition* of *LogicalANDExpression* with argument *call*.

CallExpression :

CoverCallExpressionAndAsyncArrowHead

CallExpression *Arguments*

CallExpression *TemplateLiteral*

1. If this *CallExpression* is *call*, return **true**.
2. Return **false**.

MemberExpression :

MemberExpression *TemplateLiteral*

1. If this *MemberExpression* is *call*, return **true**.
2. Return **false**.

PrimaryExpression : *CoverParenthesizedExpressionAndArrowParameterList*

1. Let *expr* be CoveredParenthesizedExpression of CoverParenthesizedExpressionAndArrowParameterList.
2. Return HasCallInTailPosition of *expr* with argument *call*.

ParenthesizedExpression :
 (*Expression*)

1. Return HasCallInTailPosition of *Expression* with argument *call*.

14.9.3 Runtime Semantics: PrepareForTailCall ()

The abstract operation PrepareForTailCall performs the following steps:

1. Let *leafContext* be the running execution context.
2. Suspend *leafContext*.
3. Pop *leafContext* from the execution context stack. The execution context now on the top of the stack becomes the running execution context.
4. Assert: *leafContext* has no further use. It will never be activated as the running execution context.

A tail position call must either release any transient internal resources associated with the currently executing function execution context before invoking the target function or reuse those resources in support of the target function.

NOTE

For example, a tail position call should only grow an implementation's activation record stack by the amount that the size of the target function's activation record exceeds the size of the calling function's activation record. If the target function's activation record is smaller, then the total size of the stack should decrease.

15 ECMAScript Language: Scripts and Modules

15.1 Scripts

Syntax

Script :
 *ScriptBody*_{opt}

ScriptBody :
 *StatementList*_[~Yield, ~Await, ~Return]

15.1.1 Static Semantics: Early Errors

Script : *ScriptBody*

It is a Syntax Error if the LexicallyDeclaredNames of *ScriptBody* contains any duplicate entries.

It is a Syntax Error if any element of the LexicallyDeclaredNames of *ScriptBody* also occurs in the VarDeclaredNames of *ScriptBody*.

ScriptBody : *StatementList*

It is a Syntax Error if *StatementList* Contains **super** unless the source code containing **super** is eval code that is being processed by a [direct eval](#). Additional [early error](#) rules for **super** within [direct eval](#) are defined in [18.2.1.1](#).
It is a Syntax Error if *StatementList* Contains *NewTarget* unless the source code containing *NewTarget* is eval code that is being processed by a [direct eval](#). Additional [early error](#) rules for *NewTarget* in [direct eval](#) are defined in [18.2.1.1](#).

It is a Syntax Error if ContainsDuplicateLabels of *StatementList* with argument « » is **true**.

It is a Syntax Error if ContainsUndefinedBreakTarget of *StatementList* with argument « » is **true**.

It is a Syntax Error if ContainsUndefinedContinueTarget of *StatementList* with arguments « » and « » is **true**.

15.1.2 Static Semantics: IsStrict

ScriptBody : *StatementList*

1. If the [Directive Prologue](#) of *StatementList* contains a [Use Strict Directive](#), return **true**; otherwise, return **false**.

15.1.3 Static Semantics: LexicallyDeclaredNames

ScriptBody : *StatementList*

1. Return TopLevelLexicallyDeclaredNames of *StatementList*.

NOTE

At the top level of a *Script*, function declarations are treated like var declarations rather than like lexical declarations.

15.1.4 Static Semantics: LexicallyScopedDeclarations

ScriptBody : *StatementList*

1. Return TopLevelLexicallyScopedDeclarations of *StatementList*.

15.1.5 Static Semantics: VarDeclaredNames

ScriptBody : *StatementList*

1. Return TopLevelVarDeclaredNames of *StatementList*.

15.1.6 Static Semantics: VarScopedDeclarations

ScriptBody : *StatementList*

1. Return TopLevelVarScopedDeclarations of *StatementList*.

15.1.7 Runtime Semantics: Evaluation

Script : [empty]

1. Return [NormalCompletion\(undefined\)](#).

15.1.8 Script Records

A Script Record encapsulates information about a script being evaluated. Each script record contains the fields listed in [Table 36](#).

Table 36: **Script Record** Fields

Field Name	Value Type	Meaning
[[Realm]]	Realm Record undefined	The realm within which this script was created. undefined if not yet assigned.
[[Environment]]	Lexical Environment undefined	The Lexical Environment containing the top level bindings for this script. This field is set when the script is instantiated.
[[ECMAScriptCode]]	a Parse Node	The result of parsing the source text of this module using <i>Script</i> as the goal symbol .
[[HostDefined]]	Any, default value is undefined .	Field reserved for use by host environments that need to associate additional information with a script.

15.1.9 ParseScript (*sourceText*, *realm*, *hostDefined*)

The abstract operation ParseScript with arguments *sourceText*, *realm*, and *hostDefined* creates a [Script Record](#) based upon the result of parsing *sourceText* as a *Script*. ParseScript performs the following steps:

1. **Assert**: *sourceText* is an ECMAScript source text (see clause 10).
2. Parse *sourceText* using *Script* as the [goal symbol](#) and analyse the parse result for any Early Error conditions. If the parse was successful and no early errors were found, let *body* be the resulting parse tree. Otherwise, let *body* be a [List](#) of one or more **SyntaxError** or **ReferenceError** objects representing the parsing errors and/or early errors. Parsing and [early error](#) detection may be interweaved in an implementation-dependent manner. If more than one parsing error or [early error](#) is present, the number and ordering of error objects in the list is implementation-dependent, but at least one must be present.
3. If *body* is a [List](#) of errors, return *body*.
4. Return [Script Record](#) { [[Realm]]: *realm*, [[Environment]]: **undefined**, [[ECMAScriptCode]]: *body*, [[HostDefined]]: *hostDefined* }.

NOTE

An implementation may parse script source text and analyse it for Early Error conditions prior to evaluation of ParseScript for that script source text. However, the reporting of any errors must be deferred until the point where this specification actually performs ParseScript upon that source text.

15.1.10 ScriptEvaluation (*scriptRecord*)

1. Let *globalEnv* be *scriptRecord*.[[Realm]].[[GlobalEnv]].
2. Let *scriptCxt* be a new ECMAScript code [execution context](#).
3. Set the Function of *scriptCxt* to **null**.
4. Set the [Realm](#) of *scriptCxt* to *scriptRecord*.[[Realm]].
5. Set the ScriptOrModule of *scriptCxt* to *scriptRecord*.
6. Set the VariableEnvironment of *scriptCxt* to *globalEnv*.
7. Set the LexicalEnvironment of *scriptCxt* to *globalEnv*.
8. **Suspend** the currently [running execution context](#).
9. Push *scriptCxt* on to the [execution context stack](#); *scriptCxt* is now the [running execution context](#).

10. Let *scriptBody* be *scriptRecord*.[[ECMAScriptCode]].
11. Let *result* be *GlobalDeclarationInstantiation*(*scriptBody*, *globalEnv*).
12. If *result*.[[Type]] is normal, then
 - a. Set *result* to the result of evaluating *scriptBody*.
13. If *result*.[[Type]] is normal and *result*.[[Value]] is empty, then
 - a. Set *result* to *NormalCompletion*(**undefined**).
14. *Suspend scriptCxt* and remove it from the *execution context stack*.
15. **Assert**: The *execution context stack* is not empty.
16. Resume the context that is now on the top of the *execution context stack* as the *running execution context*.
17. Return *Completion*(*result*).

15.1.11 Runtime Semantics: GlobalDeclarationInstantiation (*script*, *env*)

NOTE 1

When an *execution context* is established for evaluating scripts, declarations are instantiated in the current *global environment*. Each global binding declared in the code is instantiated.

GlobalDeclarationInstantiation is performed as follows using arguments *script* and *env*. *script* is the *ScriptBody* for which the *execution context* is being established. *env* is the global lexical environment in which bindings are to be created.

1. Let *envRec* be *env*'s *EnvironmentRecord*.
2. **Assert**: *envRec* is a global *Environment Record*.
3. Let *lexNames* be the *LexicallyDeclaredNames* of *script*.
4. Let *varNames* be the *VarDeclaredNames* of *script*.
5. For each *name* in *lexNames*, do
 - a. If *envRec*.*HasVarDeclaration*(*name*) is **true**, throw a **SyntaxError** exception.
 - b. If *envRec*.*HasLexicalDeclaration*(*name*) is **true**, throw a **SyntaxError** exception.
 - c. Let *hasRestrictedGlobal* be ? *envRec*.*HasRestrictedGlobalProperty*(*name*).
 - d. If *hasRestrictedGlobal* is **true**, throw a **SyntaxError** exception.
6. For each *name* in *varNames*, do
 - a. If *envRec*.*HasLexicalDeclaration*(*name*) is **true**, throw a **SyntaxError** exception.
7. Let *varDeclarations* be the *VarScopedDeclarations* of *script*.
8. Let *functionsToInitialize* be a new empty *List*.
9. Let *declaredFunctionNames* be a new empty *List*.
10. For each *d* in *varDeclarations*, in reverse list order, do
 - a. If *d* is neither a *VariableDeclaration* nor a *ForBinding* nor a *BindingIdentifier*, then
 - i. **Assert**: *d* is either a *FunctionDeclaration*, a *GeneratorDeclaration*, an *AsyncFunctionDeclaration*, or an *AsyncGeneratorDeclaration*.
 - ii. NOTE: If there are multiple function declarations for the same name, the last declaration is used.
 - iii. Let *fn* be the sole element of the *BoundNames* of *d*.
 - iv. If *fn* is not an element of *declaredFunctionNames*, then
 1. Let *fnDefinable* be ? *envRec*.*CanDeclareGlobalFunction*(*fn*).
 2. If *fnDefinable* is **false**, throw a **TypeError** exception.
 3. Append *fn* to *declaredFunctionNames*.
 4. Insert *d* as the first element of *functionsToInitialize*.
11. Let *declaredVarNames* be a new empty *List*.
12. For each *d* in *varDeclarations*, do
 - a. If *d* is a *VariableDeclaration*, a *ForBinding*, or a *BindingIdentifier*, then

- i. For each String *vn* in the BoundNames of *d*, do
 1. If *vn* is not an element of *declaredFunctionNames*, then
 - a. Let *vnDefinable* be ? *envRec*.CanDeclareGlobalVar(*vn*).
 - b. If *vnDefinable* is **false**, throw a **TypeError** exception.
 - c. If *vn* is not an element of *declaredVarNames*, then
 - i. Append *vn* to *declaredVarNames*.
13. NOTE: No abnormal terminations occur after this algorithm step if the *global object* is an ordinary object. However, if the *global object* is a Proxy *exotic object* it may exhibit behaviours that cause abnormal terminations in some of the following steps.
14. NOTE: Annex B.3.3.2 adds additional steps at this point.
15. Let *lexDeclarations* be the LexicallyScopedDeclarations of *script*.
16. For each element *d* in *lexDeclarations*, do
 - a. NOTE: Lexically declared names are only instantiated here but not initialized.
 - b. For each element *dn* of the BoundNames of *d*, do
 - i. If IsConstantDeclaration of *d* is **true**, then
 1. Perform ? *envRec*.CreateImmutableBinding(*dn*, **true**).
 - ii. Else,
 1. Perform ? *envRec*.CreateMutableBinding(*dn*, **false**).
17. For each *Parse Node f* in *functionsToInitialize*, do
 - a. Let *fn* be the sole element of the BoundNames of *f*.
 - b. Let *fo* be the result of performing InstantiateFunctionObject for *f* with argument *env*.
 - c. Perform ? *envRec*.CreateGlobalFunctionBinding(*fn*, *fo*, **false**).
18. For each String *vn* in *declaredVarNames*, in list order, do
 - a. Perform ? *envRec*.CreateGlobalVarBinding(*vn*, **false**).
19. Return *NormalCompletion*(empty).

NOTE 2

Early errors specified in 15.1.1 prevent name conflicts between function/var declarations and let/const/class declarations as well as redeclaration of let/const/class bindings for declaration contained within a single *Script*. However, such conflicts and redeclarations that span more than one *Script* are detected as runtime errors during GlobalDeclarationInstantiation. If any such errors are detected, no bindings are instantiated for the script. However, if the *global object* is defined using Proxy exotic objects then the runtime tests for conflicting declarations may be unreliable resulting in an *abrupt completion* and some global declarations not being instantiated. If this occurs, the code for the *Script* is not evaluated.

Unlike explicit var or function declarations, properties that are directly created on the *global object* result in global bindings that may be shadowed by let/const/class declarations.

15.1.12 Runtime Semantics: ScriptEvaluationJob (*sourceText*, *hostDefined*)

The job ScriptEvaluationJob with parameters *sourceText* and *hostDefined* parses, validates, and evaluates *sourceText* as a *Script*.

1. **Assert**: *sourceText* is an ECMAScript source text (see clause 10).
2. Let *realm* be the current Realm Record.
3. Let *s* be ParseScript(*sourceText*, *realm*, *hostDefined*).
4. If *s* is a List of errors, then
 - a. Perform HostReportErrors(*s*).

- b. Return `NormalCompletion(undefined)`.
- 5. Return ? `ScriptEvaluation(s)`.

15.2 Modules

Syntax

Module :

*ModuleBody*_{opt}

ModuleBody :

ModuleItemList

ModuleItemList :

ModuleItem

ModuleItemList *ModuleItem*

ModuleItem :

ImportDeclaration

ExportDeclaration

*StatementListItem*_[~Yield, ~Await, ~Return]

15.2.1 Module Semantics

15.2.1.1 Static Semantics: Early Errors

ModuleBody : *ModuleItemList*

It is a Syntax Error if the `LexicallyDeclaredNames` of *ModuleItemList* contains any duplicate entries.

It is a Syntax Error if any element of the `LexicallyDeclaredNames` of *ModuleItemList* also occurs in the `VarDeclaredNames` of *ModuleItemList*.

It is a Syntax Error if the `ExportedNames` of *ModuleItemList* contains any duplicate entries.

It is a Syntax Error if any element of the `ExportedBindings` of *ModuleItemList* does not also occur in either the `VarDeclaredNames` of *ModuleItemList*, or the `LexicallyDeclaredNames` of *ModuleItemList*.

It is a Syntax Error if *ModuleItemList* Contains **super**.

It is a Syntax Error if *ModuleItemList* Contains *NewTarget*.

It is a Syntax Error if `ContainsDuplicateLabels` of *ModuleItemList* with argument « » is **true**.

It is a Syntax Error if `ContainsUndefinedBreakTarget` of *ModuleItemList* with argument « » is **true**.

It is a Syntax Error if `ContainsUndefinedContinueTarget` of *ModuleItemList* with arguments « » and « » is **true**.

NOTE

The duplicate `ExportedNames` rule implies that multiple **export default** *ExportDeclaration* items within a *ModuleBody* is a Syntax Error. Additional error conditions relating to conflicting or duplicate declarations are checked during module linking prior to evaluation of a *Module*. If any such errors are detected the *Module* is not evaluated.

15.2.1.2 Static Semantics: ContainsDuplicateLabels

With parameter *labelSet*.

ModuleItemList : *ModuleItemList* *ModuleItem*

1. Let *hasDuplicates* be ContainsDuplicateLabels of *ModuleItemList* with argument *labelSet*.
2. If *hasDuplicates* is **true**, return **true**.
3. Return ContainsDuplicateLabels of *ModuleItem* with argument *labelSet*.

ModuleItem :

ImportDeclaration

ExportDeclaration

1. Return **false**.

15.2.1.3 Static Semantics: ContainsUndefinedBreakTarget

With parameter *labelSet*.

ModuleItemList : *ModuleItemList* *ModuleItem*

1. Let *hasUndefinedLabels* be ContainsUndefinedBreakTarget of *ModuleItemList* with argument *labelSet*.
2. If *hasUndefinedLabels* is **true**, return **true**.
3. Return ContainsUndefinedBreakTarget of *ModuleItem* with argument *labelSet*.

ModuleItem :

ImportDeclaration

ExportDeclaration

1. Return **false**.

15.2.1.4 Static Semantics: ContainsUndefinedContinueTarget

With parameters *iterationSet* and *labelSet*.

ModuleItemList : *ModuleItemList* *ModuleItem*

1. Let *hasUndefinedLabels* be ContainsUndefinedContinueTarget of *ModuleItemList* with arguments *iterationSet* and « ».
2. If *hasUndefinedLabels* is **true**, return **true**.
3. Return ContainsUndefinedContinueTarget of *ModuleItem* with arguments *iterationSet* and « ».

ModuleItem :

ImportDeclaration

ExportDeclaration

1. Return **false**.

15.2.1.5 Static Semantics: ExportedBindings

NOTE

ExportedBindings are the locally bound names that are explicitly associated with a *Module*'s ExportedNames.

ModuleItemList : *ModuleItemList* *ModuleItem*

1. Let *names* be ExportedBindings of *ModuleItemList*.

2. Append to *names* the elements of the ExportedBindings of *ModuleItem*.
3. Return *names*.

ModuleItem :
ImportDeclaration
StatementListItem

1. Return a new empty *List*.

15.2.1.6 Static Semantics: ExportedNames

NOTE

ExportedNames are the externally visible names that a *Module* explicitly maps to one of its local name bindings.

ModuleItemList : *ModuleItemList* *ModuleItem*

1. Let *names* be ExportedNames of *ModuleItemList*.
2. Append to *names* the elements of the ExportedNames of *ModuleItem*.
3. Return *names*.

ModuleItem : *ExportDeclaration*

1. Return the ExportedNames of *ExportDeclaration*.

ModuleItem :
ImportDeclaration
StatementListItem

1. Return a new empty *List*.

15.2.1.7 Static Semantics: ExportEntries

Module : [empty]

1. Return a new empty *List*.

ModuleItemList : *ModuleItemList* *ModuleItem*

1. Let *entries* be ExportEntries of *ModuleItemList*.
2. Append to *entries* the elements of the ExportEntries of *ModuleItem*.
3. Return *entries*.

ModuleItem :
ImportDeclaration
StatementListItem

1. Return a new empty *List*.

15.2.1.8 Static Semantics: ImportEntries

Module : [empty]

1. Return a new empty *List*.

ModuleItemList : *ModuleItemList* *ModuleItem*

1. Let *entries* be ImportEntries of *ModuleItemList*.
2. Append to *entries* the elements of the ImportEntries of *ModuleItem*.
3. Return *entries*.

ModuleItem :

ExportDeclaration

StatementListItem

1. Return a new empty *List*.

15.2.1.9 Static Semantics: ImportedLocalNames (*importEntries*)

The abstract operation ImportedLocalNames with argument *importEntries* creates a *List* of all of the local name bindings defined by a *List* of ImportEntry Records (see Table 42). ImportedLocalNames performs the following steps:

1. Let *localNames* be a new empty *List*.
2. For each ImportEntry Record *i* in *importEntries*, do
 - a. Append *i*.[[LocalName]] to *localNames*.
3. Return *localNames*.

15.2.1.10 Static Semantics: ModuleRequests

Module : [empty]

1. Return a new empty *List*.

ModuleItemList : *ModuleItem*

1. Return ModuleRequests of *ModuleItem*.

ModuleItemList : *ModuleItemList* *ModuleItem*

1. Let *moduleNames* be ModuleRequests of *ModuleItemList*.
2. Let *additionalNames* be ModuleRequests of *ModuleItem*.
3. Append to *moduleNames* each element of *additionalNames* that is not already an element of *moduleNames*.
4. Return *moduleNames*.

ModuleItem : *StatementListItem*

1. Return a new empty *List*.

15.2.1.11 Static Semantics: LexicallyDeclaredNames

NOTE 1

The LexicallyDeclaredNames of a *Module* includes the names of all of its imported bindings.

ModuleItemList : *ModuleItemList* *ModuleItem*

1. Let *names* be LexicallyDeclaredNames of *ModuleItemList*.
2. Append to *names* the elements of the LexicallyDeclaredNames of *ModuleItem*.
3. Return *names*.

ModuleItem : *ImportDeclaration*

1. Return the BoundNames of *ImportDeclaration*.

ModuleItem : *ExportDeclaration*

1. If *ExportDeclaration* is **export** *VariableStatement*, return a new empty [List](#).
2. Return the BoundNames of *ExportDeclaration*.

ModuleItem : *StatementListItem*

1. Return LexicallyDeclaredNames of *StatementListItem*.

NOTE 2

At the top level of a *Module*, function declarations are treated like lexical declarations rather than like var declarations.

15.2.1.12 Static Semantics: LexicallyScopedDeclarations

Module : [empty]

1. Return a new empty [List](#).

ModuleItemList : *ModuleItemList* *ModuleItem*

1. Let *declarations* be LexicallyScopedDeclarations of *ModuleItemList*.
2. Append to *declarations* the elements of the LexicallyScopedDeclarations of *ModuleItem*.
3. Return *declarations*.

ModuleItem : *ImportDeclaration*

1. Return a new empty [List](#).

15.2.1.13 Static Semantics: VarDeclaredNames

Module : [empty]

1. Return a new empty [List](#).

ModuleItemList : *ModuleItemList* *ModuleItem*

1. Let *names* be VarDeclaredNames of *ModuleItemList*.
2. Append to *names* the elements of the VarDeclaredNames of *ModuleItem*.
3. Return *names*.

ModuleItem : *ImportDeclaration*

1. Return a new empty [List](#).

ModuleItem : *ExportDeclaration*

1. If *ExportDeclaration* is **export** *VariableStatement*, return BoundNames of *ExportDeclaration*.
2. Return a new empty [List](#).

15.2.1.14 Static Semantics: VarScopedDeclarations

Module : [empty]

1. Return a new empty [List](#).

ModuleItemList : *ModuleItemList* *ModuleItem*

1. Let *declarations* be VarScopedDeclarations of *ModuleItemList*.
2. Append to *declarations* the elements of the VarScopedDeclarations of *ModuleItem*.
3. Return *declarations*.

ModuleItem : *ImportDeclaration*

1. Return a new empty *List*.

ModuleItem : *ExportDeclaration*

1. If *ExportDeclaration* is **export** *VariableStatement*, return VarScopedDeclarations of *VariableStatement*.
2. Return a new empty *List*.

15.2.1.15 Abstract Module Records

A Module Record encapsulates structural information about the imports and exports of a single module. This information is used to link the imports and exports of sets of connected modules. A Module Record includes four fields that are only used when evaluating a module.

For specification purposes Module Record values are values of the *Record* specification type and can be thought of as existing in a simple object-oriented hierarchy where Module Record is an abstract class with both abstract and concrete subclasses. This specification defines the abstract subclass named *Cyclic Module Record* and its concrete subclass named *Source Text Module Record*. Other specifications and implementations may define additional Module Record subclasses corresponding to alternative module definition facilities that they defined.

Module Record defines the fields listed in Table 37. All Module Definition subclasses include at least those fields. Module Record also defines the abstract method list in Table 38. All Module definition subclasses must provide concrete implementations of these abstract methods.

Table 37: Module Record Fields

Field Name	Value Type	Meaning
[[Realm]]	<i>Realm Record</i> undefined	The <i>Realm</i> within which this module was created. undefined if not yet assigned.
[[Environment]]	<i>Lexical Environment</i> undefined	The <i>Lexical Environment</i> containing the top level bindings for this module. This field is set when the module is instantiated.
[[Namespace]]	Object undefined	The Module Namespace Object (26.3) if one has been created for this module. Otherwise undefined .
[[HostDefined]]	Any, default value is undefined .	Field reserved for use by host environments that need to associate additional information with a module.

Table 38: Abstract Methods of Module Records

Method	Purpose
--------	---------

GetExportedNames(<i>exportStarSet</i>)	Return a list of all names that are either directly or indirectly exported from this module.
ResolveExport(<i>exportName</i> , <i>resolveSet</i>)	Return the binding of a name exported by this module. Bindings are represented by a ResolvedBinding Record, of the form { [[Module]]: Module Record, [[BindingName]]: String }. Return null if the name cannot be resolved, or "ambiguous" if multiple bindings were found. This operation must be idempotent if it completes normally. Each time it is called with a specific <i>exportName</i>, <i>resolveSet</i> pair as arguments it must return the same result.
Instantiate()	Prepare the module for evaluation by transitively resolving all module dependencies and creating a module Environment Record .
Evaluate()	If this module has already been evaluated successfully, return undefined; if it has already been evaluated unsuccessfully, throw the exception that was produced. Otherwise, transitively evaluate all module dependencies of this module and then evaluate this module. Instantiate must have completed successfully prior to invoking this method.

15.2.1.16 Cyclic Module Records

A Cyclic Module Record is used to represent information about a module that can participate in dependency cycles with other modules that are subclasses of the [Cyclic Module Record](#) type. Module Records that are not subclasses of the [Cyclic Module Record](#) type must not participate in dependency cycles with Source Text Module Records.

In addition to the fields defined in [Table 37](#) Cyclic Module Records have the additional fields listed in [Table 39](#)

Table 39: Additional Fields of Cyclic Module Records

Field Name	Value Type	Meaning
[[Status]]	String	Initially "uninstantiated" . Transitions to "instantiating" , "instantiated" , "evaluating" , "evaluated" (in that order) as the module progresses throughout its lifecycle.
[[EvaluationError]]	An abrupt completion undefined	A completion of type throw representing the exception that occurred during evaluation. undefined if no exception occurred or if [[Status]] is not "evaluated" .

[[DFSIndex]]	Integer undefined	Auxiliary field used during Instantiate and Evaluate only. If [[Status]] is "instantiating" or "evaluating" , this nonnegative number records the point at which the module was first visited during the ongoing depth-first traversal of the dependency graph.
[[DFSAncestorIndex]]	Integer undefined	Auxiliary field used during Instantiate and Evaluate only. If [[Status]] is "instantiating" or "evaluating" , this is either the module's own [[DFSIndex]] or that of an "earlier" module in the same strongly connected component.
[[RequestedModules]]	List of String	A List of all the <i>ModuleSpecifier</i> strings used by the module represented by this record to request the importation of a module. The List is source code occurrence ordered.

In addition to the methods defined in Table 38 Cyclic Module Records have the additional methods listed in Table 40

Table 40: Additional Abstract Methods of Cyclic Module Records

Method	Purpose
InitializeEnvironment()	Initialize the Lexical Environment of the module, including resolving all imported bindings.
ExecuteModule()	Initialize the execution context of the module and evaluate the module's code within it.

15.2.1.16.1 Instantiate () Concrete Method

The Instantiate concrete method of a [Cyclic Module Record](#) implements the corresponding [Module Record](#) abstract method.

On success, Instantiate transitions this module's [[Status]] from **"uninstantiated"** to **"instantiated"**. On failure, an exception is thrown and this module's [[Status]] remains **"uninstantiated"**.

This abstract method performs the following steps (most of the work is done by the auxiliary function [InnerModuleInstantiation](#)):

1. Let *module* be this [Cyclic Module Record](#).
2. **Assert:** *module*.[[Status]] is not **"instantiating"** or **"evaluating"**.
3. Let *stack* be a new empty List.
4. Let *result* be [InnerModuleInstantiation](#)(*module*, *stack*, 0).
5. If *result* is an abrupt completion, then
 - a. For each module *m* in *stack*, do
 - i. **Assert:** *m*.[[Status]] is **"instantiating"**.
 - ii. Set *m*.[[Status]] to **"uninstantiated"**.
 - iii. Set *m*.[[Environment]] to **undefined**.
 - iv. Set *m*.[[DFSIndex]] to **undefined**.
 - v. Set *m*.[[DFSAncestorIndex]] to **undefined**.
 - b. **Assert:** *module*.[[Status]] is **"uninstantiated"**.
 - c. Return *result*.
6. **Assert:** *module*.[[Status]] is **"instantiated"** or **"evaluated"**.

7. **Assert:** *stack* is empty.
8. Return **undefined**.

15.2.1.16.1.1 InnerModuleInstantiation (*module*, *stack*, *index*)

The InnerModuleInstantiation abstract operation is used by Instantiate to perform the actual instantiation process for the **Cyclic Module Record** *module*, as well as recursively on all other modules in the dependency graph. The *stack* and *index* parameters, as well as a module's `[[DFSIndex]]` and `[[DFSAncestorIndex]]` fields, keep track of the depth-first search (DFS) traversal. In particular, `[[DFSAncestorIndex]]` is used to discover strongly connected components (SCCs), such that all modules in an SCC transition to **"instantiated"** together.

This abstract operation performs the following steps:

1. If *module* is not a **Cyclic Module Record**, then
 - a. Perform ? *module*.Instantiate().
 - b. Return *index*.
2. If *module*.`[[Status]]` is **"instantiating"**, **"instantiated"**, or **"evaluated"**, then
 - a. Return *index*.
3. **Assert:** *module*.`[[Status]]` is **"uninstantiated"**.
4. Set *module*.`[[Status]]` to **"instantiating"**.
5. Set *module*.`[[DFSIndex]]` to *index*.
6. Set *module*.`[[DFSAncestorIndex]]` to *index*.
7. Increase *index* by 1.
8. Append *module* to *stack*.
9. For each String *required* that is an element of *module*.`[[RequestedModules]]`, do
 - a. Let *requiredModule* be ? HostResolveImportedModule(*module*, *required*).
 - b. Set *index* to ? InnerModuleInstantiation(*requiredModule*, *stack*, *index*).
 - c. **Assert:** *requiredModule*.`[[Status]]` is either **"instantiating"**, **"instantiated"**, or **"evaluated"**.
 - d. **Assert:** *requiredModule*.`[[Status]]` is **"instantiating"** if and only if *requiredModule* is in *stack*.
 - e. If *requiredModule*.`[[Status]]` is **"instantiating"**, then
 - i. **Assert:** *requiredModule* is a **Cyclic Module Record**.
 - ii. Set *module*.`[[DFSAncestorIndex]]` to `min(module.[[DFSAncestorIndex]], requiredModule.[[DFSAncestorIndex]])`.
10. Perform ? *module*.InitializeEnvironment().
11. **Assert:** *module* occurs exactly once in *stack*.
12. **Assert:** *module*.`[[DFSAncestorIndex]]` is less than or equal to *module*.`[[DFSIndex]]`.
13. If *module*.`[[DFSAncestorIndex]]` equals *module*.`[[DFSIndex]]`, then
 - a. Let *done* be **false**.
 - b. Repeat, while *done* is **false**,
 - i. Let *requiredModule* be the last element in *stack*.
 - ii. Remove the last element of *stack*.
 - iii. Set *requiredModule*.`[[Status]]` to **"instantiated"**.
 - iv. If *requiredModule* and *module* are the same **Module Record**, set *done* to **true**.
14. Return *index*.

15.2.1.16.2 Evaluate () Concrete Method

The Evaluate concrete method of a **Cyclic Module Record** implements the corresponding **Module Record** abstract method.

Evaluate transitions this module's `[[Status]]` from **"instantiated"** to **"evaluated"**.

If execution results in an exception, that exception is recorded in the `[[EvaluationError]]` field and rethrown by future invocations of Evaluate.

This abstract method performs the following steps (most of the work is done by the auxiliary function `InnerModuleEvaluation`):

1. Let *module* be this `Cyclic Module Record`.
2. **Assert:** *module*.`[[Status]]` is **"instantiated"** or **"evaluated"**.
3. Let *stack* be a new empty `List`.
4. Let *result* be `InnerModuleEvaluation(module, stack, 0)`.
5. If *result* is an `abrupt completion`, then
 - a. For each module *m* in *stack*, do
 - i. **Assert:** *m*.`[[Status]]` is **"evaluating"**.
 - ii. Set *m*.`[[Status]]` to **"evaluated"**.
 - iii. Set *m*.`[[EvaluationError]]` to *result*.
 - b. **Assert:** *module*.`[[Status]]` is **"evaluated"** and *module*.`[[EvaluationError]]` is *result*.
 - c. Return *result*.
6. **Assert:** *module*.`[[Status]]` is **"evaluated"** and *module*.`[[EvaluationError]]` is **undefined**.
7. **Assert:** *stack* is empty.
8. Return **undefined**.

15.2.1.16.2.1 InnerModuleEvaluation (*module*, *stack*, *index*)

The `InnerModuleEvaluation` abstract operation is used by `Evaluate` to perform the actual evaluation process for the `Source Text Module Record` *module*, as well as recursively on all other modules in the dependency graph. The *stack* and *index* parameters, as well as *module*'s `[[DFSIndex]]` and `[[DFSAncestorIndex]]` fields, are used the same way as in `InnerModuleInstantiation`.

This abstract operation performs the following steps:

1. If *module* is not a `Cyclic Module Record`, then
 - a. Perform ? *module*.`Evaluate()`.
 - b. Return *index*.
2. If *module*.`[[Status]]` is **"evaluated"**, then
 - a. If *module*.`[[EvaluationError]]` is **undefined**, return *index*.
 - b. Otherwise return *module*.`[[EvaluationError]]`.
3. If *module*.`[[Status]]` is **"evaluating"**, return *index*.
4. **Assert:** *module*.`[[Status]]` is **"instantiated"**.
5. Set *module*.`[[Status]]` to **"evaluating"**.
6. Set *module*.`[[DFSIndex]]` to *index*.
7. Set *module*.`[[DFSAncestorIndex]]` to *index*.
8. Increase *index* by 1.
9. Append *module* to *stack*.
10. For each String *required* that is an element of *module*.`[[RequestedModules]]`, do
 - a. Let *requiredModule* be ! `HostResolveImportedModule(module, required)`.
 - b. NOTE: Instantiate must be completed successfully prior to invoking this method, so every requested module is guaranteed to resolve successfully.
 - c. Set *index* to ? `InnerModuleEvaluation(requiredModule, stack, index)`.

- d. **Assert:** *requiredModule*.[[Status]] is either **"evaluating"** or **"evaluated"**.
- e. **Assert:** *requiredModule*.[[Status]] is **"evaluating"** if and only if *requiredModule* is in *stack*.
- f. If *requiredModule*.[[Status]] is **"evaluating"**, then
 - i. **Assert:** *requiredModule* is a **Cyclic Module Record**.
 - ii. Set *module*.[[DFSAncestorIndex]] to **min**(*module*.[[DFSAncestorIndex]], *requiredModule*.[[DFSAncestorIndex]]).
11. Perform ? *module*.ExecuteModule().
12. **Assert:** *module* occurs exactly once in *stack*.
13. **Assert:** *module*.[[DFSAncestorIndex]] is less than or equal to *module*.[[DFSIndex]].
14. If *module*.[[DFSAncestorIndex]] equals *module*.[[DFSIndex]], then
 - a. Let *done* be **false**.
 - b. Repeat, while *done* is **false**,
 - i. Let *requiredModule* be the last element in *stack*.
 - ii. Remove the last element of *stack*.
 - iii. Set *requiredModule*.[[Status]] to **"evaluated"**.
 - iv. If *requiredModule* and *module* are the same **Module Record**, set *done* to **true**.
15. Return *index*.

15.2.1.16.3 Example Cyclic Module Record Graphs

This non-normative section gives a series of examples of the instantiation and evaluation of a few common module graphs, with a specific focus on how errors can occur.

First consider the following simple module graph:

Figure 2: A simple module graph

Let's first assume that there are no error conditions. When a host first calls *A*.Instantiate(), this will complete successfully by assumption, and recursively instantiate modules *B* and *C* as well, such that *A*.[[Status]] = *B*.[[Status]] = *C*.[[Status]] = **"instantiated"**. This preparatory step can be performed at any time. Later, when the host is ready to incur any possible side effects of the modules, it can call *A*.Evaluate(), which will complete successfully (again by assumption), recursively having evaluated first *C* and then *B*. Each module's [[Status]] at this point will be **"evaluated"**.

Consider then cases involving instantiation errors. If **InnerModuleInstantiation** of *C* succeeds but, thereafter, fails for *B*, for example because it imports something that *C* does not provide, then the original *A*.Instantiate() will fail, and both *A* and *B*'s [[Status]] remain **"uninstantiated"**. *C*'s [[Status]] has become **"instantiated"**, though.

Finally, consider a case involving evaluation errors. If **InnerModuleEvaluation** of *C* succeeds but, thereafter, fails for *B*, for example because *B* contains code that throws an exception, then the original *A*.Evaluate() will fail. The resulting exception will be recorded in both *A* and *B*'s [[EvaluationError]] fields, and their [[Status]] will become **"evaluated"**.

`C` will also become **"evaluated"** but, in contrast to `A` and `B`, will remain without an `[[EvaluationError]]`, as it successfully completed evaluation. Storing the exception ensures that any time a host tries to reuse `A` or `B` by calling their `Evaluate()` method, it will encounter the same exception. (Hosts are not required to reuse Cyclic Module Records; similarly, hosts are not required to expose the exception objects thrown by these methods. However, the specification enables such uses.)

The difference here between instantiation and evaluation errors is due to how evaluation must be only performed once, as it can cause side effects; it is thus important to remember whether evaluation has already been performed, even if unsuccessfully. (In the error case, it makes sense to also remember the exception because otherwise subsequent `Evaluate()` calls would have to synthesize a new one.) Instantiation, on the other hand, is side-effect-free, and thus even if it fails, it can be retried at a later time with no issues.

Now consider a different type of error condition:

Figure 3: A module graph with an unresolvable module



In this scenario, module `A` declares a dependency on some other module, but no `Module Record` exists for that module, i.e. `HostResolveImportedModule` throws an exception when asked for it. This could occur for a variety of reasons, such as the corresponding resource not existing, or the resource existing but `ParseModule` throwing an exception when trying to parse the resulting source text. Hosts can choose to expose the cause of failure via the exception they throw from `HostResolveImportedModule`. In any case, this exception causes an instantiation failure, which as before results in `A`'s `[[Status]]` remaining **"uninstantiated"**.

Lastly, consider a module graph with a cycle:

Figure 4: A cyclic module graph



Here we assume that the entry point is module `A`, so that the host proceeds by calling `A.Instantiate()`, which performs `InnerModuleInstantiation` on `A`. This in turn calls `InnerModuleInstantiation` on `B`. Because of the cycle, this again triggers `InnerModuleInstantiation` on `A`, but at this point it is a no-op since `A. [[Status]]` is already **"instantiating"**. `B. [[Status]]` itself remains **"instantiating"** when control gets back to `A` and `InnerModuleInstantiation` is triggered on `C`. After this returns with `C. [[Status]]` being **"instantiated"**, both `A` and `B` transition from **"instantiating"** to **"instantiated"** together; this is by design, since they form a strongly connected component.

An analogous story occurs for the evaluation phase of a cyclic module graph, in the success case.

Now consider a case where `A` has an instantiation error; for example, it tries to import a binding from `C` that does not exist. In that case, the above steps still occur, including the early return from the second call to `InnerModuleInstantiation` on `A`. However, once we unwind back to the original `InnerModuleInstantiation` on `A`, it fails during

[InitializeEnvironment](#), namely right after [C.ResolveExport\(\)](#). The thrown **SyntaxError** exception propagates up to [A.Instantiate](#), which resets all modules that are currently on its *stack* (these are always exactly the modules that are still **"instantiating"**). Hence both [A](#) and [B](#) become **"uninstantiated"**. Note that [C](#) is left as **"instantiated"**.

Finally, consider a case where [A](#) has an evaluation error; for example, its source code throws an exception. In that case, the evaluation-time analog of the above steps still occurs, including the early return from the second call to [InnerModuleEvaluation](#) on [A](#). However, once we unwind back to the original [InnerModuleEvaluation](#) on [A](#), it fails by assumption. The exception thrown propagates up to [A.Evaluate\(\)](#), which records the error in all modules that are currently on its *stack* (i.e., the modules that are still **"evaluating"**). Hence both [A](#) and [B](#) become **"evaluated"** and the exception is recorded in both [A](#) and [B](#)'s `[[EvaluationError]]` fields, while [C](#) is left as **"evaluated"** with no `[[EvaluationError]]`.

15.2.1.17 Source Text Module Records

A Source Text Module Record is used to represent information about a module that was defined from ECMAScript source text (10) that was parsed using the [goal symbol](#) *Module*. Its fields contain digested information about the names that are imported by the module and its concrete methods use this digest to link, instantiate, and evaluate the module.

A [Source Text Module Record](#) can exist in a module graph with other subclasses of the abstract [Module Record](#) type, and can participate in cycles with other subclasses of the [Cyclic Module Record](#) type.

In addition to the fields defined in [Table 39](#), Source Text Module Records have the additional fields listed in [Table 41](#). Each of these fields is initially set in [ParseModule](#).

Table 41: Additional Fields of Source Text Module Records

Field Name	Value Type	Meaning
<code>[[ECMAScriptCode]]</code>	a Parse Node	The result of parsing the source text of this module using <i>Module</i> as the goal symbol .
<code>[[ImportEntries]]</code>	List of ImportEntry Records	A List of ImportEntry records derived from the code of this module.
<code>[[LocalExportEntries]]</code>	List of ExportEntry Records	A List of ExportEntry records derived from the code of this module that correspond to declarations that occur within the module.
<code>[[IndirectExportEntries]]</code>	List of ExportEntry Records	A List of ExportEntry records derived from the code of this module that correspond to reexported imports that occur within the module.
<code>[[StarExportEntries]]</code>	List of ExportEntry Records	A List of ExportEntry records derived from the code of this module that correspond to export <code>*</code> declarations that occur within the module.

An [ImportEntry](#) Record is a [Record](#) that digests information about a single declarative import. Each [ImportEntry](#) Record has the fields defined in [Table 42](#):

Table 42: [ImportEntry](#) Record Fields

Field Name	Value Type	Meaning
[[ModuleRequest]]	String	String value of the <i>ModuleSpecifier</i> of the <i>ImportDeclaration</i> .
[[ImportName]]	String	The name under which the desired binding is exported by the module identified by [[ModuleRequest]]. The value "*" indicates that the import request is for the target module's namespace object.
[[LocalName]]	String	The name that is used to locally access the imported value from within the importing module.

NOTE 1

Table 43 gives examples of ImportEntry records fields used to represent the syntactic import forms:

Table 43 (Informative): Import Forms Mappings to ImportEntry Records

Import Statement Form	[[ModuleRequest]]	[[ImportName]]	[[LocalName]]
<code>import v from "mod";</code>	"mod"	"default"	"v"
<code>import * as ns from "mod";</code>	"mod"	"*"	"ns"
<code>import {x} from "mod";</code>	"mod"	"x"	"x"
<code>import {x as v} from "mod";</code>	"mod"	"x"	"v"
<code>import "mod";</code>	An ImportEntry Record is not created.		

An ExportEntry Record is a [Record](#) that digests information about a single declarative export. Each [ExportEntry Record](#) has the fields defined in Table 44:

Table 44: ExportEntry Record Fields

Field Name	Value Type	Meaning
[[ExportName]]	String null	The name used to export this binding by this module.
[[ModuleRequest]]	String null	The String value of the <i>ModuleSpecifier</i> of the <i>ExportDeclaration</i> . null if the <i>ExportDeclaration</i> does not have a <i>ModuleSpecifier</i> .
[[ImportName]]	String null	The name under which the desired binding is exported by the module identified by [[ModuleRequest]]. null if the <i>ExportDeclaration</i> does not have a <i>ModuleSpecifier</i> . "*" indicates that the export request is for all exported bindings.
[[LocalName]]	String null	The name that is used to locally access the exported value from within the importing module. null if the exported value is not locally accessible from within the module.

Table 45 gives examples of the ExportEntry record fields used to represent the syntactic export forms:

Table 45 (Informative): Export Forms Mappings to ExportEntry Records

Export Statement Form	[[ExportName]]	[[ModuleRequest]]	[[ImportName]]	[[LocalName]]
<code>export var v;</code>	"v"	null	null	"v"
<code>export default function f() {}</code>	"default"	null	null	"f"
<code>export default function() {}</code>	"default"	null	null	"*default"
<code>export default 42;</code>	"default"	null	null	"*default"
<code>export {x};</code>	"x"	null	null	"x"
<code>export {v as x};</code>	"x"	null	null	"v"
<code>export {x} from "mod";</code>	"x"	"mod"	"x"	null
<code>export {v as x} from "mod";</code>	"x"	"mod"	"v"	null
<code>export * from "mod";</code>	null	"mod"	"*"	null

The following definitions specify the required concrete methods and other [abstract operations](#) for Source Text Module Records

15.2.1.17.1 ParseModule (*sourceText*, *realm*, *hostDefined*)

The abstract operation ParseModule with arguments *sourceText*, *realm*, and *hostDefined* creates a [Source Text Module Record](#) based upon the result of parsing *sourceText* as a *Module*. ParseModule performs the following steps:

1. **Assert:** *sourceText* is an ECMAScript source text (see clause 10).
2. Parse *sourceText* using *Module* as the [goal symbol](#) and analyse the parse result for any Early Error conditions. If the parse was successful and no early errors were found, let *body* be the resulting parse tree. Otherwise, let *body* be a [List](#) of one or more **SyntaxError** or **ReferenceError** objects representing the parsing errors and/or early errors. Parsing and [early error](#) detection may be interweaved in an implementation-dependent manner. If more than one parsing error or [early error](#) is present, the number and ordering of error objects in the list is implementation-dependent, but at least one must be present.
3. If *body* is a [List](#) of errors, return *body*.
4. Let *requestedModules* be the ModuleRequests of *body*.
5. Let *importEntries* be ImportEntries of *body*.
6. Let *importedBoundNames* be ImportedLocalNames(*importEntries*).
7. Let *indirectExportEntries* be a new empty [List](#).
8. Let *localExportEntries* be a new empty [List](#).
9. Let *starExportEntries* be a new empty [List](#).
10. Let *exportEntries* be ExportEntries of *body*.
11. For each [ExportEntry Record](#) *ee* in *exportEntries*, do
 - a. If *ee*.[[ModuleRequest]] is **null**, then

- i. If *ee*.[[LocalName]] is not an element of *importedBoundNames*, then
 1. Append *ee* to *localExportEntries*.
- ii. Else,
 1. Let *ie* be the element of *importEntries* whose [[LocalName]] is the same as *ee*.[[LocalName]].
 2. If *ie*.[[ImportName]] is "*", then
 - a. **Assert**: This is a re-export of an imported module namespace object.
 - b. Append *ee* to *localExportEntries*.
 3. Else this is a re-export of a single name,
 - a. Append the **ExportEntry Record** { [[ModuleRequest]]: *ie*.[[ModuleRequest]], [[ImportName]]: *ie*.[[ImportName]], [[LocalName]]: **null**, [[ExportName]]: *ee*. [[ExportName]] } to *indirectExportEntries*.
- b. Else if *ee*.[[ImportName]] is "*", then
 - i. Append *ee* to *starExportEntries*.
- c. Else,
 - i. Append *ee* to *indirectExportEntries*.
12. Return **Source Text Module Record** { [[Realm]]: *realm*, [[Environment]]: **undefined**, [[Namespace]]: **undefined**, [[Status]]: **"uninstantiated"**, [[EvaluationError]]: **undefined**, [[HostDefined]]: *hostDefined*, [[ECMAScriptCode]]: *body*, [[RequestedModules]]: *requestedModules*, [[ImportEntries]]: *importEntries*, [[LocalExportEntries]]: *localExportEntries*, [[IndirectExportEntries]]: *indirectExportEntries*, [[StarExportEntries]]: *starExportEntries*, [[DFSIndex]]: **undefined**, [[DFSAncestorIndex]]: **undefined** }.

NOTE

An implementation may parse module source text and analyse it for Early Error conditions prior to the evaluation of *ParseModule* for that module source text. However, the reporting of any errors must be deferred until the point where this specification actually performs *ParseModule* upon that source text.

15.2.1.17.2 GetExportedNames (*exportStarSet*) Concrete Method

The *GetExportedNames* concrete method of a **Source Text Module Record** implements the corresponding **Module Record** abstract method.

It performs the following steps:

1. Let *module* be this **Source Text Module Record**.
2. If *exportStarSet* contains *module*, then
 - a. **Assert**: We've reached the starting point of an **import** * circularity.
 - b. Return a new empty **List**.
3. Append *module* to *exportStarSet*.
4. Let *exportedNames* be a new empty **List**.
5. For each **ExportEntry Record** *e* in *module*.[[LocalExportEntries]], do
 - a. **Assert**: *module* provides the direct binding for this export.
 - b. Append *e*.[[ExportName]] to *exportedNames*.
6. For each **ExportEntry Record** *e* in *module*.[[IndirectExportEntries]], do
 - a. **Assert**: *module* imports a specific binding for this export.
 - b. Append *e*.[[ExportName]] to *exportedNames*.
7. For each **ExportEntry Record** *e* in *module*.[[StarExportEntries]], do
 - a. Let *requestedModule* be ? *HostResolveImportedModule*(*module*, *e*.[[ModuleRequest]]).
 - b. Let *starNames* be ? *requestedModule*.*GetExportedNames*(*exportStarSet*).

- c. For each element *n* of *starNames*, do
 - i. If `SameValue(n, "default")` is **false**, then
 1. If *n* is not an element of *exportedNames*, then
 - a. Append *n* to *exportedNames*.
8. Return *exportedNames*.

NOTE

GetExportedNames does not filter out or throw an exception for names that have ambiguous star export bindings.

15.2.1.17.3 ResolveExport (*exportName*, *resolveSet*) Concrete Method

The ResolveExport concrete method of a **Source Text Module Record** implements the corresponding **Module Record** abstract method.

ResolveExport attempts to resolve an imported binding to the actual defining module and local binding name. The defining module may be the module represented by the **Module Record** this method was invoked on or some other module that is imported by that module. The parameter *resolveSet* is used to detect unresolved circular import/export paths. If a pair consisting of specific **Module Record** and *exportName* is reached that is already in *resolveSet*, an import circularity has been encountered. Before recursively calling ResolveExport, a pair consisting of *module* and *exportName* is added to *resolveSet*.

If a defining module is found, a **ResolvedBinding Record** { [[Module]], [[BindingName]] } is returned. This record identifies the resolved binding of the originally requested export. If no definition was found or the request is found to be circular, **null** is returned. If the request is found to be ambiguous, the string **"ambiguous"** is returned.

This abstract method performs the following steps:

1. Let *module* be this **Source Text Module Record**.
2. For each **Record** { [[Module]], [[ExportName]] } *r* in *resolveSet*, do
 - a. If *module* and *r*.[[Module]] are the same **Module Record** and `SameValue(exportName, r.[[ExportName]])` is **true**, then
 - i. **Assert**: This is a circular import request.
 - ii. Return **null**.
3. Append the **Record** { [[Module]]: *module*, [[ExportName]]: *exportName* } to *resolveSet*.
4. For each **ExportEntry Record** *e* in *module*.[[LocalExportEntries]], do
 - a. If `SameValue(exportName, e.[[ExportName]])` is **true**, then
 - i. **Assert**: *module* provides the direct binding for this export.
 - ii. Return **ResolvedBinding Record** { [[Module]]: *module*, [[BindingName]]: *e*.[[LocalName]] }.
5. For each **ExportEntry Record** *e* in *module*.[[IndirectExportEntries]], do
 - a. If `SameValue(exportName, e.[[ExportName]])` is **true**, then
 - i. **Assert**: *module* imports a specific binding for this export.
 - ii. Let *importedModule* be ? `HostResolveImportedModule(module, e.[[ModuleRequest]])`.
 - iii. Return *importedModule*.ResolveExport(*e*.[[ImportName]], *resolveSet*).
6. If `SameValue(exportName, "default")` is **true**, then
 - a. **Assert**: A **default** export was not explicitly defined by this module.
 - b. Return **null**.
 - c. **NOTE**: A **default** export cannot be provided by an **export ***.
7. Let *starResolution* be **null**.
8. For each **ExportEntry Record** *e* in *module*.[[StarExportEntries]], do

- a. Let *importedModule* be ? *HostResolveImportedModule*(*module*, *e*.[[ModuleRequest]]).
- b. Let *resolution* be ? *importedModule*.ResolveExport(*exportName*, *resolveSet*).
- c. If *resolution* is "ambiguous", return "ambiguous".
- d. If *resolution* is not null, then
 - i. **Assert**: *resolution* is a ResolvedBinding Record.
 - ii. If *starResolution* is null, set *starResolution* to *resolution*.
 - iii. Else,
 1. **Assert**: There is more than one * import that includes the requested name.
 2. If *resolution*.[[Module]] and *starResolution*.[[Module]] are not the same Module Record or SameValue(*resolution*.[[BindingName]], *starResolution*.[[BindingName]]) is false, return "ambiguous".
9. Return *starResolution*.

15.2.1.17.4 InitializeEnvironment () Concrete Method

The InitializeEnvironment concrete method of a Source Text Module Record implements the corresponding Cyclic Module Record abstract method.

This abstract method performs the following steps:

1. Let *module* be this Source Text Module Record.
2. For each ExportEntry Record *e* in *module*.[[IndirectExportEntries]], do
 - a. Let *resolution* be ? *module*.ResolveExport(*e*.[[ExportName]], « »).
 - b. If *resolution* is null or "ambiguous", throw a SyntaxError exception.
 - c. **Assert**: *resolution* is a ResolvedBinding Record.
3. **Assert**: All named exports from *module* are resolvable.
4. Let *realm* be *module*.[[Realm]].
5. **Assert**: *realm* is not undefined.
6. Let *env* be NewModuleEnvironment(*realm*.[[GlobalEnv]]).
7. Set *module*.[[Environment]] to *env*.
8. Let *envRec* be *env*'s EnvironmentRecord.
9. For each ImportEntry Record *in* in *module*.[[ImportEntries]], do
 - a. Let *importedModule* be ! *HostResolveImportedModule*(*module*, *in*.[[ModuleRequest]]).
 - b. NOTE: The above call cannot fail because imported module requests are a subset of *module*.[[RequestedModules]], and these have been resolved earlier in this algorithm.
 - c. If *in*.[[ImportName]] is "*", then
 - i. Let *namespace* be ? *GetModuleNamespace*(*importedModule*).
 - ii. Perform ! *envRec*.CreateImmutableBinding(*in*.[[LocalName]], true).
 - iii. Call *envRec*.InitializeBinding(*in*.[[LocalName]], *namespace*).
 - d. Else,
 - i. Let *resolution* be ? *importedModule*.ResolveExport(*in*.[[ImportName]], « »).
 - ii. If *resolution* is null or "ambiguous", throw a SyntaxError exception.
 - iii. Call *envRec*.CreateImportBinding(*in*.[[LocalName]], *resolution*.[[Module]], *resolution*.[[BindingName]]).
10. Let *code* be *module*.[[ECMAScriptCode]].
11. Let *varDeclarations* be the VarScopedDeclarations of *code*.
12. Let *declaredVarNames* be a new empty List.
13. For each element *d* in *varDeclarations*, do
 - a. For each element *dn* of the BoundNames of *d*, do

- i. If *dn* is not an element of *declaredVarNames*, then
 1. Perform ! *envRec*.CreateMutableBinding(*dn*, **false**).
 2. Call *envRec*.InitializeBinding(*dn*, **undefined**).
 3. Append *dn* to *declaredVarNames*.
14. Let *lexDeclarations* be the LexicallyScopedDeclarations of *code*.
15. For each element *d* in *lexDeclarations*, do
 - a. For each element *dn* of the BoundNames of *d*, do
 - i. If IsConstantDeclaration of *d* is **true**, then
 1. Perform ! *envRec*.CreateImmutableBinding(*dn*, **true**).
 - ii. Else,
 1. Perform ! *envRec*.CreateMutableBinding(*dn*, **false**).
 - iii. If *d* is a *FunctionDeclaration*, a *GeneratorDeclaration*, an *AsyncFunctionDeclaration*, or an *AsyncGeneratorDeclaration*, then
 1. Let *fo* be the result of performing InstantiateFunctionObject for *d* with argument *env*.
 2. Call *envRec*.InitializeBinding(*dn*, *fo*).
16. Return **NormalCompletion**(empty).

15.2.1.17.5 ExecuteModule () Concrete Method

The ExecuteModule concrete method of a **Source Text Module Record** implements the corresponding **Cyclic Module Record** abstract method.

This abstract method performs the following steps:

1. Let *module* be this **Source Text Module Record**.
2. Let *moduleCxt* be a new ECMAScript code **execution context**.
3. Set the Function of *moduleCxt* to **null**.
4. **Assert**: *module*.[[Realm]] is not **undefined**.
5. Set the **Realm** of *moduleCxt* to *module*.[[Realm]].
6. Set the ScriptOrModule of *moduleCxt* to *module*.
7. **Assert**: *module* has been linked and declarations in its **module environment** have been instantiated.
8. Set the VariableEnvironment of *moduleCxt* to *module*.[[Environment]].
9. Set the LexicalEnvironment of *moduleCxt* to *module*.[[Environment]].
10. **Suspend** the currently **running execution context**.
11. Push *moduleCxt* on to the **execution context stack**; *moduleCxt* is now the **running execution context**.
12. Let *result* be the result of evaluating *module*.[[ECMAScriptCode]].
13. **Suspend** *moduleCxt* and remove it from the **execution context stack**.
14. Resume the context that is now on the top of the **execution context stack** as the **running execution context**.
15. Return **Completion**(*result*).

15.2.1.18 Runtime Semantics: HostResolveImportedModule (*referencingModule*, *specifier*)

HostResolveImportedModule is an implementation-defined abstract operation that provides the concrete **Module Record** subclass instance that corresponds to the *ModuleSpecifier* String, *specifier*, occurring within the context of the module represented by the **Module Record** *referencingModule*.

The implementation of HostResolveImportedModule must conform to the following requirements:

The normal return value must be an instance of a concrete subclass of **Module Record**.

If a **Module Record** corresponding to the pair *referencingModule*, *specifier* does not exist or cannot be created, an

exception must be thrown.

This operation must be idempotent if it completes normally. Each time it is called with a specific *referencingModule*, *specifier* pair as arguments it must return the same **Module Record** instance.

Multiple different *referencingModule*, *specifier* pairs may map to the same **Module Record** instance. The actual mapping semantic is implementation-defined but typically a normalization process is applied to *specifier* as part of the mapping process. A typical normalization process would include actions such as alphabetic case folding and expansion of relative and abbreviated path specifiers.

15.2.1.19 Runtime Semantics: GetModuleNamespace (*module*)

The GetModuleNamespace abstract operation retrieves the Module Namespace **Exotic object** representing *module*'s exports, lazily creating it the first time it was requested, and storing it in *module*.[[Namespace]] for future retrieval.

This abstract operation performs the following steps:

1. **Assert**: *module* is an instance of a concrete subclass of **Module Record**.
2. **Assert**: *module*.[[Status]] is not "**uninstantiated**".
3. Let *namespace* be *module*.[[Namespace]].
4. If *namespace* is **undefined**, then
 - a. Let *exportedNames* be ? *module*.GetExportedNames(« »).
 - b. Let *unambiguousNames* be a new empty **List**.
 - c. For each *name* that is an element of *exportedNames*, do
 - i. Let *resolution* be ? *module*.ResolveExport(*name*, « »).
 - ii. If *resolution* is a **ResolvedBinding Record**, append *name* to *unambiguousNames*.
 - d. Set *namespace* to **ModuleNamespaceCreate**(*module*, *unambiguousNames*).
5. Return *namespace*.

NOTE

The only way GetModuleNamespace can throw is via one of the triggered **HostResolveImportedModule** calls. Unresolvable names are simply excluded from the namespace at this point. They will lead to a real instantiation error later unless they are all ambiguous star exports that are not explicitly requested anywhere.

15.2.1.20 Runtime Semantics: TopLevelModuleEvaluationJob (*sourceText*, *hostDefined*)

A TopLevelModuleEvaluationJob with parameters *sourceText* and *hostDefined* is a job that parses, validates, and evaluates *sourceText* as a **Module**.

1. **Assert**: *sourceText* is an ECMAScript source text (see clause 10).
2. Let *realm* be the current **Realm Record**.
3. Let *m* be **ParseModule**(*sourceText*, *realm*, *hostDefined*).
4. If *m* is a **List** of errors, then
 - a. Perform **HostReportErrors**(*m*).
 - b. Return **NormalCompletion**(**undefined**).
5. Perform ? *m*.Instantiate().
6. **Assert**: All dependencies of *m* have been transitively resolved and *m* is ready for evaluation.
7. Return ? *m*.Evaluate().

NOTE

An implementation may parse a *sourceText* as a *Module*, analyse it for Early Error conditions, and instantiate it prior to the execution of the *TopLevelModuleEvaluationJob* for that *sourceText*. An implementation may also resolve, pre-parse and pre-analyse, and pre-instantiate module dependencies of *sourceText*. However, the reporting of any errors detected by these actions must be deferred until the *TopLevelModuleEvaluationJob* is actually executed.

15.2.1.21 Runtime Semantics: Evaluation

Module : [empty]

1. Return **NormalCompletion**(**undefined**).

ModuleBody : *ModuleItemList*

1. Let *result* be the result of evaluating *ModuleItemList*.
2. If *result*.[[Type]] is normal and *result*.[[Value]] is empty, then
 - a. Return **NormalCompletion**(**undefined**).
3. Return **Completion**(*result*).

ModuleItemList : *ModuleItemList* *ModuleItem*

1. Let *sl* be the result of evaluating *ModuleItemList*.
2. **ReturnIfAbrupt**(*sl*).
3. Let *s* be the result of evaluating *ModuleItem*.
4. Return **Completion**(**UpdateEmpty**(*s*, *sl*)).

NOTE

The value of a *ModuleItemList* is the value of the last value-producing item in the *ModuleItemList*.

ModuleItem : *ImportDeclaration*

1. Return **NormalCompletion**(**empty**).

15.2.2 Imports

Syntax

ImportDeclaration :

```
import ImportClause FromClause ;  
import ModuleSpecifier ;
```

ImportClause :

```
ImportedDefaultBinding  
NamespaceImport  
NamedImports  
ImportedDefaultBinding , NamespaceImport  
ImportedDefaultBinding , NamedImports
```

ImportedDefaultBinding :

```
ImportedBinding
```

NamespaceImport :

*** as** *ImportedBinding*

NamedImports :

{ }

{ *ImportsList* }

{ *ImportsList* , }

FromClause :

from *ModuleSpecifier*

ImportsList :

ImportSpecifier

ImportsList , *ImportSpecifier*

ImportSpecifier :

ImportedBinding

IdentifierName **as** *ImportedBinding*

ModuleSpecifier :

StringLiteral

ImportedBinding :

BindingIdentifier [*~Yield*, *~Await*]

15.2.2.1 Static Semantics: Early Errors

ModuleItem : *ImportDeclaration*

It is a Syntax Error if the *BoundNames* of *ImportDeclaration* contains any duplicate entries.

15.2.2.2 Static Semantics: BoundNames

ImportDeclaration : **import** *ImportClause* *FromClause* ;

1. Return the *BoundNames* of *ImportClause*.

ImportDeclaration : **import** *ModuleSpecifier* ;

1. Return a new empty *List*.

ImportClause : *ImportedDefaultBinding* , *NamespaceImport*

1. Let *names* be the *BoundNames* of *ImportedDefaultBinding*.
2. Append to *names* the elements of the *BoundNames* of *NamespaceImport*.
3. Return *names*.

ImportClause : *ImportedDefaultBinding* , *NamedImports*

1. Let *names* be the *BoundNames* of *ImportedDefaultBinding*.
2. Append to *names* the elements of the *BoundNames* of *NamedImports*.
3. Return *names*.

NamedImports : { }

1. Return a new empty *List*.

ImportsList : *ImportsList* , *ImportSpecifier*

1. Let *names* be the BoundNames of *ImportsList*.
2. Append to *names* the elements of the BoundNames of *ImportSpecifier*.
3. Return *names*.

ImportSpecifier : *IdentifierName* **as** *ImportedBinding*

1. Return the BoundNames of *ImportedBinding*.

15.2.2.3 Static Semantics: ImportEntries

ImportDeclaration : **import** *ImportClause* *FromClause* ;

1. Let *module* be the sole element of ModuleRequests of *FromClause*.
2. Return ImportEntriesForModule of *ImportClause* with argument *module*.

ImportDeclaration : **import** *ModuleSpecifier* ;

1. Return a new empty List.

15.2.2.4 Static Semantics: ImportEntriesForModule

With parameter *module*.

ImportClause : *ImportedDefaultBinding* , *NamespaceImport*

1. Let *entries* be ImportEntriesForModule of *ImportedDefaultBinding* with argument *module*.
2. Append to *entries* the elements of the ImportEntriesForModule of *NamespaceImport* with argument *module*.
3. Return *entries*.

ImportClause : *ImportedDefaultBinding* , *NamedImports*

1. Let *entries* be ImportEntriesForModule of *ImportedDefaultBinding* with argument *module*.
2. Append to *entries* the elements of the ImportEntriesForModule of *NamedImports* with argument *module*.
3. Return *entries*.

ImportedDefaultBinding : *ImportedBinding*

1. Let *localName* be the sole element of BoundNames of *ImportedBinding*.
2. Let *defaultEntry* be the ImportEntry Record { [[ModuleRequest]]: *module*, [[ImportName]]: "default", [[LocalName]]: *localName* }.
3. Return a new List containing *defaultEntry*.

NamespaceImport : ***** **as** *ImportedBinding*

1. Let *localName* be the StringValue of *ImportedBinding*.
2. Let *entry* be the ImportEntry Record { [[ModuleRequest]]: *module*, [[ImportName]]: "*", [[LocalName]]: *localName* }.
3. Return a new List containing *entry*.

NamedImports : { }

1. Return a new empty [List](#).

ImportsList : *ImportsList* , *ImportSpecifier*

1. Let *specs* be the *ImportEntriesForModule* of *ImportsList* with argument *module*.
2. Append to *specs* the elements of the *ImportEntriesForModule* of *ImportSpecifier* with argument *module*.
3. Return *specs*.

ImportSpecifier : *ImportedBinding*

1. Let *localName* be the sole element of *BoundNames* of *ImportedBinding*.
2. Let *entry* be the *ImportEntry Record* { *[[ModuleRequest]]*: *module*, *[[ImportName]]*: *localName*, *[[LocalName]]*: *localName* }.
3. Return a new [List](#) containing *entry*.

ImportSpecifier : *IdentifierName* **as** *ImportedBinding*

1. Let *importName* be the *StringValue* of *IdentifierName*.
2. Let *localName* be the *StringValue* of *ImportedBinding*.
3. Let *entry* be the *ImportEntry Record* { *[[ModuleRequest]]*: *module*, *[[ImportName]]*: *importName*, *[[LocalName]]*: *localName* }.
4. Return a new [List](#) containing *entry*.

15.2.2.5 Static Semantics: ModuleRequests

ImportDeclaration : **import** *ImportClause* *FromClause* ;

1. Return *ModuleRequests* of *FromClause*.

ModuleSpecifier : *StringLiteral*

1. Return a [List](#) containing the *StringValue* of *StringLiteral*.

15.2.3 Exports

Syntax

ExportDeclaration :

```

export * FromClause ;
export ExportClause FromClause ;
export ExportClause ;
export VariableStatement[~Yield, ~Await]
export Declaration[~Yield, ~Await]
export default HoistableDeclaration[~Yield, ~Await, +Default]
export default ClassDeclaration[~Yield, ~Await, +Default]
export default [lookahead ∉ { function , async [no LineTerminator here] function ,
  class }] AssignmentExpression[+In, ~Yield, ~Await] ;

```

ExportClause :

```

{ }
{ ExportsList }

```

{ *ExportsList* , }

ExportsList :

ExportSpecifier

ExportsList , *ExportSpecifier*

ExportSpecifier :

IdentifierName

IdentifierName **as** *IdentifierName*

15.2.3.1 Static Semantics: Early Errors

ExportDeclaration : **export** *ExportClause* ;

For each *IdentifierName* *n* in ReferencedBindings of *ExportClause*: It is a Syntax Error if StringValue of *n* is a *ReservedWord* or if the StringValue of *n* is one of: "**implements**", "**interface**", "**let**", "**package**", "**private**", "**protected**", "**public**", or "**static**".

NOTE

The above rule means that each ReferencedBindings of *ExportClause* is treated as an *IdentifierReference*.

15.2.3.2 Static Semantics: BoundNames

ExportDeclaration :

export * *FromClause* ;

export *ExportClause* *FromClause* ;

export *ExportClause* ;

1. Return a new empty [List](#).

ExportDeclaration : **export** *VariableStatement*

1. Return the BoundNames of *VariableStatement*.

ExportDeclaration : **export** *Declaration*

1. Return the BoundNames of *Declaration*.

ExportDeclaration : **export default** *HoistableDeclaration*

1. Let *declarationNames* be the BoundNames of *HoistableDeclaration*.
2. If *declarationNames* does not include the element "***default***", append "***default***" to *declarationNames*.
3. Return *declarationNames*.

ExportDeclaration : **export default** *ClassDeclaration*

1. Let *declarationNames* be the BoundNames of *ClassDeclaration*.
2. If *declarationNames* does not include the element "***default***", append "***default***" to *declarationNames*.
3. Return *declarationNames*.

ExportDeclaration : **export default** *AssignmentExpression* ;

1. Return « "***default***" ».

15.2.3.3 Static Semantics: ExportedBindings

ExportDeclaration :

export * *FromClause* ;

export *ExportClause FromClause* ;

1. Return a new empty [List](#).

ExportDeclaration : **export** *ExportClause* ;

1. Return the ExportedBindings of *ExportClause*.

ExportDeclaration : **export** *VariableStatement*

1. Return the BoundNames of *VariableStatement*.

ExportDeclaration : **export** *Declaration*

1. Return the BoundNames of *Declaration*.

ExportDeclaration : **export default** *HoistableDeclaration*

ExportDeclaration : **export default** *ClassDeclaration*

ExportDeclaration : **export default** *AssignmentExpression* ;

1. Return the BoundNames of this *ExportDeclaration*.

ExportClause : { }

1. Return a new empty [List](#).

ExportsList : *ExportsList* , *ExportSpecifier*

1. Let *names* be the ExportedBindings of *ExportsList*.
2. Append to *names* the elements of the ExportedBindings of *ExportSpecifier*.
3. Return *names*.

ExportSpecifier : *IdentifierName*

1. Return a [List](#) containing the StringValue of *IdentifierName*.

ExportSpecifier : *IdentifierName* **as** *IdentifierName*

1. Return a [List](#) containing the StringValue of the first *IdentifierName*.

15.2.3.4 Static Semantics: ExportedNames

ExportDeclaration : **export** * *FromClause* ;

1. Return a new empty [List](#).

ExportDeclaration : **export** *ExportClause FromClause* ;

ExportDeclaration : **export** *ExportClause* ;

1. Return the ExportedNames of *ExportClause*.

ExportDeclaration : **export** *VariableStatement*

1. Return the BoundNames of *VariableStatement*.

ExportDeclaration : **export** *Declaration*

1. Return the BoundNames of *Declaration*.

ExportDeclaration : **export default** *HoistableDeclaration*

ExportDeclaration : **export default** *ClassDeclaration*

ExportDeclaration : **export default** *AssignmentExpression* ;

1. Return « "default" ».

ExportClause : { }

1. Return a new empty [List](#).

ExportsList : *ExportsList* , *ExportSpecifier*

1. Let *names* be the ExportedNames of *ExportsList*.
2. Append to *names* the elements of the ExportedNames of *ExportSpecifier*.
3. Return *names*.

ExportSpecifier : *IdentifierName*

1. Return a [List](#) containing the StringValue of *IdentifierName*.

ExportSpecifier : *IdentifierName* **as** *IdentifierName*

1. Return a [List](#) containing the StringValue of the second *IdentifierName*.

15.2.3.5 Static Semantics: ExportEntries

ExportDeclaration : **export** * *FromClause* ;

1. Let *module* be the sole element of ModuleRequests of *FromClause*.
2. Let *entry* be the [ExportEntry Record](#) { [[ModuleRequest]]: *module*, [[ImportName]]: "*", [[LocalName]]: **null**, [[ExportName]]: **null** }.
3. Return a new [List](#) containing *entry*.

ExportDeclaration : **export** *ExportClause* *FromClause* ;

1. Let *module* be the sole element of ModuleRequests of *FromClause*.
2. Return ExportEntriesForModule of *ExportClause* with argument *module*.

ExportDeclaration : **export** *ExportClause* ;

1. Return ExportEntriesForModule of *ExportClause* with argument **null**.

ExportDeclaration : **export** *VariableStatement*

1. Let *entries* be a new empty [List](#).
2. Let *names* be the BoundNames of *VariableStatement*.
3. For each *name* in *names*, do

- a. Append the **ExportEntry Record** { [[ModuleRequest]]: **null**, [[ImportName]]: **null**, [[LocalName]]: *name*, [[ExportName]]: *name* } to *entries*.
4. Return *entries*.

ExportDeclaration : **export** *Declaration*

1. Let *entries* be a new empty **List**.
2. Let *names* be the BoundNames of *Declaration*.
3. For each *name* in *names*, do
 - a. Append the **ExportEntry Record** { [[ModuleRequest]]: **null**, [[ImportName]]: **null**, [[LocalName]]: *name*, [[ExportName]]: *name* } to *entries*.
4. Return *entries*.

ExportDeclaration : **export default** *HoistableDeclaration*

1. Let *names* be BoundNames of *HoistableDeclaration*.
2. Let *localName* be the sole element of *names*.
3. Return a new **List** containing the **ExportEntry Record** { [[ModuleRequest]]: **null**, [[ImportName]]: **null**, [[LocalName]]: *localName*, [[ExportName]]: **"default"** }.

ExportDeclaration : **export default** *ClassDeclaration*

1. Let *names* be BoundNames of *ClassDeclaration*.
2. Let *localName* be the sole element of *names*.
3. Return a new **List** containing the **ExportEntry Record** { [[ModuleRequest]]: **null**, [[ImportName]]: **null**, [[LocalName]]: *localName*, [[ExportName]]: **"default"** }.

ExportDeclaration : **export default** *AssignmentExpression* ;

1. Let *entry* be the **ExportEntry Record** { [[ModuleRequest]]: **null**, [[ImportName]]: **null**, [[LocalName]]: **"*default*"**, [[ExportName]]: **"default"** }.
2. Return a new **List** containing *entry*.

NOTE

"*default*" is used within this specification as a synthetic name for anonymous default export values.

15.2.3.6 Static Semantics: ExportEntriesForModule

With parameter *module*.

ExportClause : { }

1. Return a new empty **List**.

ExportsList : *ExportsList* , *ExportSpecifier*

1. Let *specs* be the ExportEntriesForModule of *ExportsList* with argument *module*.
2. Append to *specs* the elements of the ExportEntriesForModule of *ExportSpecifier* with argument *module*.
3. Return *specs*.

ExportSpecifier : *IdentifierName*

1. Let *sourceName* be the StringValue of *IdentifierName*.
2. If *module* is **null**, then
 - a. Let *localName* be *sourceName*.
 - b. Let *importName* be **null**.
3. Else,
 - a. Let *localName* be **null**.
 - b. Let *importName* be *sourceName*.
4. Return a new **List** containing the **ExportEntry Record** { [[ModuleRequest]]: *module*, [[ImportName]]: *importName*, [[LocalName]]: *localName*, [[ExportName]]: *sourceName* }.

ExportSpecifier : *IdentifierName* **as** *IdentifierName*

1. Let *sourceName* be the StringValue of the first *IdentifierName*.
2. Let *exportName* be the StringValue of the second *IdentifierName*.
3. If *module* is **null**, then
 - a. Let *localName* be *sourceName*.
 - b. Let *importName* be **null**.
4. Else,
 - a. Let *localName* be **null**.
 - b. Let *importName* be *sourceName*.
5. Return a new **List** containing the **ExportEntry Record** { [[ModuleRequest]]: *module*, [[ImportName]]: *importName*, [[LocalName]]: *localName*, [[ExportName]]: *exportName* }.

15.2.3.7 Static Semantics: IsConstantDeclaration

ExportDeclaration :

```
export * FromClause ;
export ExportClause FromClause ;
export ExportClause ;
export default AssignmentExpression ;
```

1. Return **false**.

NOTE

It is not necessary to treat **export default** *AssignmentExpression* as a constant declaration because there is no syntax that permits assignment to the internal bound name used to reference a module's default object.

15.2.3.8 Static Semantics: LexicallyScopedDeclarations

ExportDeclaration :

```
export * FromClause ;
export ExportClause FromClause ;
export ExportClause ;
export VariableStatement
```

1. Return a new empty **List**.

ExportDeclaration : **export** *Declaration*

1. Return a new [List](#) containing DeclarationPart of *Declaration*.

ExportDeclaration : **export default** *HoistableDeclaration*

1. Return a new [List](#) containing DeclarationPart of *HoistableDeclaration*.

ExportDeclaration : **export default** *ClassDeclaration*

1. Return a new [List](#) containing *ClassDeclaration*.

ExportDeclaration : **export default** *AssignmentExpression* ;

1. Return a new [List](#) containing this *ExportDeclaration*.

15.2.3.9 Static Semantics: ModuleRequests

ExportDeclaration : **export *** *FromClause* ;

ExportDeclaration : **export** *ExportClause* *FromClause* ;

1. Return the ModuleRequests of *FromClause*.

ExportDeclaration :

export *ExportClause* ;

export *VariableStatement*

export *Declaration*

export default *HoistableDeclaration*

export default *ClassDeclaration*

export default *AssignmentExpression* ;

1. Return a new empty [List](#).

15.2.3.10 Static Semantics: ReferencedBindings

ExportClause : { }

1. Return a new empty [List](#).

ExportsList : *ExportsList* , *ExportSpecifier*

1. Let *names* be the ReferencedBindings of *ExportsList*.
2. Append to *names* the elements of the ReferencedBindings of *ExportSpecifier*.
3. Return *names*.

ExportSpecifier : *IdentifierName*

1. Return a [List](#) containing the *IdentifierName*.

ExportSpecifier : *IdentifierName* **as** *IdentifierName*

1. Return a [List](#) containing the first *IdentifierName*.

15.2.3.11 Runtime Semantics: Evaluation

ExportDeclaration :

export * *FromClause* ;

```
export ExportClause FromClause ;  
export ExportClause ;
```

1. Return **NormalCompletion**(empty).

ExportDeclaration : **export** *VariableStatement*

1. Return the result of evaluating *VariableStatement*.

ExportDeclaration : **export** *Declaration*

1. Return the result of evaluating *Declaration*.

ExportDeclaration : **export default** *HoistableDeclaration*

1. Return the result of evaluating *HoistableDeclaration*.

ExportDeclaration : **export default** *ClassDeclaration*

1. Let *value* be the result of **BindingClassDeclarationEvaluation** of *ClassDeclaration*.
2. **ReturnIfAbrupt**(*value*).
3. Let *className* be the sole element of **BoundNames** of *ClassDeclaration*.
4. If *className* is **"*default*"**, then
 - a. Let *env* be the **running execution context**'s **LexicalEnvironment**.
 - b. Perform ? **InitializeBoundName**(**"*default*"**, *value*, *env*).
5. Return **NormalCompletion**(empty).

ExportDeclaration : **export default** *AssignmentExpression* ;

1. If **IsAnonymousFunctionDefinition**(*AssignmentExpression*) is **true**, then
 - a. Let *value* be the result of performing **NamedEvaluation** for *AssignmentExpression* with argument **"default"**.
2. Else,
 - a. Let *rhs* be the result of evaluating *AssignmentExpression*.
 - b. Let *value* be ? **GetValue**(*rhs*).
3. Let *env* be the **running execution context**'s **LexicalEnvironment**.
4. Perform ? **InitializeBoundName**(**"*default*"**, *value*, *env*).
5. Return **NormalCompletion**(empty).

16 Error Handling and Language Extensions

An implementation must report most errors at the time the relevant ECMAScript language construct is evaluated. An early error is an error that can be detected and reported prior to the evaluation of any construct in the *Script* containing the error. The presence of an **early error** prevents the evaluation of the construct. An implementation must report early errors in a *Script* as part of parsing that *Script* in **ParseScript**. Early errors in a *Module* are reported at the point when the *Module* would be evaluated and the *Module* is never initialized. Early errors in **eval** code are reported at the time **eval** is called and prevent evaluation of the **eval** code. All errors that are not early errors are runtime errors.

An implementation must report as an **early error** any occurrence of a condition that is listed in a “Static Semantics: Early Errors” subclause of this specification.

An implementation shall not treat other kinds of errors as early errors even if the compiler can prove that a construct cannot execute without error under any circumstances. An implementation may issue an early warning in such a case, but it should not report the error until the relevant construct is actually executed.

An implementation shall report all errors as specified, except for the following:

Except as restricted in 16.2, an implementation may extend *Script* syntax, *Module* syntax, and regular expression pattern or flag syntax. To permit this, all operations (such as calling **eval**, using a regular expression literal, or using the **Function** or **RegExp constructor**) that are allowed to throw **SyntaxError** are permitted to exhibit implementation-defined behaviour instead of throwing **SyntaxError** when they encounter an implementation-defined extension to the script syntax or regular expression pattern or flag syntax.

Except as restricted in 16.2, an implementation may provide additional types, values, objects, properties, and functions beyond those described in this specification. This may cause constructs (such as looking up a variable in the global scope) to have implementation-defined behaviour instead of throwing an error (such as **ReferenceError**).

16.1 HostReportErrors (*errorList*)

HostReportErrors is an implementation-defined abstract operation that allows host environments to report parsing errors, early errors, and runtime errors.

An implementation of HostReportErrors must complete normally in all cases. The default implementation of HostReportErrors is to unconditionally return an empty normal completion.

NOTE

errorList will be a *List* of ECMAScript language values. If the errors are parsing errors or early errors, these will always be **SyntaxError** or **ReferenceError** objects. Runtime errors, however, can be any ECMAScript value.

16.2 Forbidden Extensions

An implementation must not extend this specification in the following ways:

ECMAScript function objects defined using syntactic constructors in *strict mode code* must not be created with own properties named **"caller"** or **"arguments"**. Such own properties also must not be created for function objects defined using an *ArrowFunction*, *MethodDefinition*, *GeneratorDeclaration*, *GeneratorExpression*, *AsyncGeneratorDeclaration*, *AsyncGeneratorExpression*, *ClassDeclaration*, *ClassExpression*, *AsyncFunctionDeclaration*, *AsyncFunctionExpression*, or *AsyncArrowFunction* regardless of whether the definition is contained in *strict mode code*. Built-in functions, strict functions created using the **Function constructor**, generator functions created using the **Generator constructor**, async functions created using the **AsyncFunction constructor**, and functions created using the **bind** method also must not be created with such own properties.

If an implementation extends any *function object* with an own property named **"caller"** the value of that property, as observed using `[[Get]]` or `[[GetOwnProperty]]`, must not be a *strict function* object. If it is an *accessor property*, the function that is the value of the property's `[[Get]]` attribute must never return a *strict function* when called.

Neither mapped nor unmapped arguments objects may be created with an own property named **"caller"**.

The behaviour of the following methods must not be extended except as specified in ECMA-402:

`Object.prototype.toLocaleString`, `Array.prototype.toLocaleString`,
`Number.prototype.toLocaleString`, `Date.prototype.toLocaleDateString`,
`Date.prototype.toLocaleTimeString`, `Date.prototype.toLocaleTimeString`,
`String.prototype.localeCompare`, `%TypedArray%.prototype.toLocaleString`.

The RegExp pattern grammars in 21.2.1 and B.1.4 must not be extended to recognize any of the source characters A-Z or a-z as *IdentityEscape*_[+U] when the [U] grammar parameter is present.

The Syntactic Grammar must not be extended in any manner that allows the token `:` to immediately follow source text that matches the *BindingIdentifier* nonterminal symbol.

When processing *strict mode code*, the syntax of *NumericLiteral* must not be extended to include *LegacyOctalIntegerLiteral* and the syntax of *DecimalIntegerLiteral* must not be extended to include *NonOctalDecimalIntegerLiteral* as described in B.1.1.

TemplateCharacter must not be extended to include *LegacyOctalEscapeSequence* as defined in B.1.2.

When processing *strict mode code*, the extensions defined in B.3.2, B.3.3, B.3.4, and B.3.6 must not be supported.

When parsing for the *Module goal symbol*, the lexical grammar extensions defined in B.1.3 must not be supported.

17 ECMAScript Standard Built-in Objects

There are certain built-in objects available whenever an ECMAScript *Script* or *Module* begins execution. One, the *global object*, is part of the lexical environment of the executing program. Others are accessible as initial properties of the *global object* or indirectly as properties of accessible built-in objects.

Unless specified otherwise, a built-in object that is callable as a function is a built-in *function object* with the characteristics described in 9.3. Unless specified otherwise, the `[[Extensible]]` internal slot of a built-in object initially has the value **true**. Every built-in *function object* has a `[[Realm]]` internal slot whose value is the *Realm Record* of the *realm* for which the object was initially created.

Many built-in objects are functions: they can be invoked with arguments. Some of them furthermore are constructors: they are functions intended for use with the **new** operator. For each built-in function, this specification describes the arguments required by that function and the properties of that *function object*. For each built-in *constructor*, this specification furthermore describes properties of the prototype object of that *constructor* and properties of specific object instances returned by a **new** expression that invokes that *constructor*.

Unless otherwise specified in the description of a particular function, if a built-in function or *constructor* is given fewer arguments than the function is specified to require, the function or *constructor* shall behave exactly as if it had been given sufficient additional arguments, each such argument being the **undefined** value. Such missing arguments are considered to be “not present” and may be identified in that manner by specification algorithms. In the description of a particular function, the terms “**this** value” and “NewTarget” have the meanings given in 9.3.

Unless otherwise specified in the description of a particular function, if a built-in function or *constructor* described is given more arguments than the function is specified to allow, the extra arguments are evaluated by the call and then ignored by the function. However, an implementation may define implementation specific behaviour relating to such arguments as long as the behaviour is not the throwing of a **TypeError** exception that is predicated simply on the presence of an extra argument.

NOTE 1

Implementations that add additional capabilities to the set of built-in functions are encouraged to do so by adding new functions rather than adding new parameters to existing functions.

Unless otherwise specified every built-in function and every built-in [constructor](#) has the Function prototype object, which is the initial value of the expression **Function.prototype** (19.2.3), as the value of its `[[Prototype]]` internal slot.

Unless otherwise specified every built-in prototype object has the Object prototype object, which is the initial value of the expression **Object.prototype** (19.1.3), as the value of its `[[Prototype]]` internal slot, except the Object prototype object itself.

Built-in function objects that are not identified as constructors do not implement the `[[Construct]]` internal method unless otherwise specified in the description of a particular function.

Each built-in function defined in this specification is created by calling the [CreateBuiltinFunction](#) abstract operation (9.3.3).

Every built-in [function object](#), including constructors, has a **"length"** property whose value is an integer. Unless otherwise specified, this value is equal to the largest number of named arguments shown in the subclause headings for the function description. Optional parameters (which are indicated with brackets: `[ ]`) or rest parameters (which are shown using the form `«...name»`) are not included in the default argument count.

NOTE 2

For example, the [function object](#) that is the initial value of the **map** property of the Array prototype object is described under the subclause heading `«Array.prototype.map (callbackFn [ , thisArg]»` which shows the two named arguments `callbackFn` and `thisArg`, the latter being optional; therefore the value of the **"length"** property of that [function object](#) is 1.

Unless otherwise specified, the **"length"** property of a built-in [function object](#) has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **true** }.

Every built-in [function object](#), including constructors, that is not identified as an anonymous function has a **name** property whose value is a String. Unless otherwise specified, this value is the name that is given to the function in this specification. For functions that are specified as properties of objects, the name value is the [property name](#) string used to access the function. Functions that are specified as get or set accessor functions of built-in properties have **"get "** or **"set "** prepended to the [property name](#) string. The value of the **name** property is explicitly specified for each built-in functions whose property key is a Symbol value.

Unless otherwise specified, the **name** property of a built-in [function object](#), if it exists, has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **true** }.

Every other [data property](#) described in clauses 18 through 26 and in Annex B.2 has the attributes { `[[Writable]]`: **true**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **true** } unless otherwise specified.

Every [accessor property](#) described in clauses 18 through 26 and in Annex B.2 has the attributes { `[[Enumerable]]`: **false**, `[[Configurable]]`: **true** } unless otherwise specified. If only a get accessor function is described, the set accessor function is the default value, **undefined**. If only a set accessor is described the get accessor is the default value, **undefined**.

18 The Global Object

The global object:

is created before control enters any [execution context](#).

does not have a `[[Construct]]` internal method; it cannot be used as a **constructor** with the **new** operator.

does not have a `[[Call]]` internal method; it cannot be invoked as a function.

has a `[[Prototype]]` internal slot whose value is implementation-dependent.

may have host defined properties in addition to the properties defined in this specification. This may include a property whose value is the global object itself.

18.1 Value Properties of the Global Object

18.1.1 Infinity

The value of **Infinity** is $+\infty$ (see 6.1.6). This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **false** }.

18.1.2 NaN

The value of **NaN** is **NaN** (see 6.1.6). This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **false** }.

18.1.3 undefined

The value of **undefined** is **undefined** (see 6.1.1). This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **false** }.

18.2 Function Properties of the Global Object

18.2.1 eval (*x*)

The **eval** function is the `%eval%` intrinsic object. When the **eval** function is called with one argument *x*, the following steps are taken:

1. **Assert**: The `execution context stack` has at least two elements.
2. Let *callerContext* be the second to top element of the `execution context stack`.
3. Let *callerRealm* be *callerContext*'s `Realm`.
4. Let *calleeRealm* be the `current Realm Record`.
5. Perform ? `HostEnsureCanCompileStrings(callerRealm, calleeRealm)`.
6. Return ? `PerformEval(x, calleeRealm, false, false)`.

18.2.1.1 Runtime Semantics: PerformEval (*x*, *evalRealm*, *strictCaller*, *direct*)

The abstract operation `PerformEval` with arguments *x*, *evalRealm*, *strictCaller*, and *direct* performs the following steps:

1. **Assert**: If *direct* is **false**, then *strictCaller* is also **false**.
2. If `Type(x)` is not `String`, return *x*.
3. Let *thisEnvRec* be ! `GetThisEnvironment()`.
4. If *thisEnvRec* is a `function Environment Record`, then
 - a. Let *F* be *thisEnvRec*.`[[FunctionObject]]`.

- b. Let *inFunction* be **true**.
 - c. Let *inMethod* be *thisEnvRec*.HasSuperBinding().
 - d. If *F*.[[ConstructorKind]] is "**derived**", let *inDerivedConstructor* be **true**; otherwise, let *inDerivedConstructor* be **false**.
5. Else,
 - a. Let *inFunction* be **false**.
 - b. Let *inMethod* be **false**.
 - c. Let *inDerivedConstructor* be **false**.
6. Let *script* be the ECMAScript code that is the result of parsing *x*, interpreted as UTF-16 encoded Unicode text as described in 6.1.4, for the goal symbol *Script*. If *inFunction* is **false**, additional early error rules from 18.2.1.1.1 are applied. If *inMethod* is **false**, additional early error rules from 18.2.1.1.2 are applied. If *inDerivedConstructor* is **false**, additional early error rules from 18.2.1.1.3 are applied. If the parse fails, throw a **SyntaxError** exception. If any early errors are detected, throw a **SyntaxError** or a **ReferenceError** exception, depending on the type of the error (but see also clause 16). Parsing and early error detection may be interweaved in an implementation-dependent manner.
7. If *script* Contains *ScriptBody* is **false**, return **undefined**.
8. Let *body* be the *ScriptBody* of *script*.
9. If *strictCaller* is **true**, let *strictEval* be **true**.
10. Else, let *strictEval* be IsStrict of *script*.
11. Let *ctx* be the running execution context.
12. NOTE: If *direct* is **true**, *ctx* will be the execution context that performed the direct eval. If *direct* is **false**, *ctx* will be the execution context for the invocation of the **eval** function.
13. If *direct* is **true**, then
 - a. Let *lexEnv* be NewDeclarativeEnvironment(*ctx*'s LexicalEnvironment).
 - b. Let *varEnv* be *ctx*'s VariableEnvironment.
14. Else,
 - a. Let *lexEnv* be NewDeclarativeEnvironment(*evalRealm*.[[GlobalEnv]]).
 - b. Let *varEnv* be *evalRealm*.[[GlobalEnv]].
15. If *strictEval* is **true**, set *varEnv* to *lexEnv*.
16. If *ctx* is not already suspended, suspend *ctx*.
17. Let *evalCxt* be a new ECMAScript code execution context.
18. Set the *evalCxt*'s Function to **null**.
19. Set the *evalCxt*'s Realm to *evalRealm*.
20. Set the *evalCxt*'s ScriptOrModule to *ctx*'s ScriptOrModule.
21. Set the *evalCxt*'s VariableEnvironment to *varEnv*.
22. Set the *evalCxt*'s LexicalEnvironment to *lexEnv*.
23. Push *evalCxt* on to the execution context stack; *evalCxt* is now the running execution context.
24. Let *result* be EvalDeclarationInstantiation(*body*, *varEnv*, *lexEnv*, *strictEval*).
25. If *result*.[[Type]] is normal, then
 - a. Set *result* to the result of evaluating *body*.
26. If *result*.[[Type]] is normal and *result*.[[Value]] is empty, then
 - a. Set *result* to NormalCompletion(**undefined**).
27. Suspend *evalCxt* and remove it from the execution context stack.
28. Resume the context that is now on the top of the execution context stack as the running execution context.
29. Return Completion(*result*).

NOTE

The eval code cannot instantiate variable or function bindings in the variable environment of the calling context that

invoked the eval if the calling context is evaluating formal parameter initializers or if either the code of the calling context or the eval code is **strict mode code**. Instead such bindings are instantiated in a new `VariableEnvironment` that is only accessible to the eval code. Bindings introduced by **let**, **const**, or **class** declarations are always instantiated in a new `LexicalEnvironment`.

18.2.1.1.1 Additional Early Error Rules for Eval Outside Functions

These **static semantics** are applied by `PerformEval` when a **direct eval** call occurs outside of any function.

ScriptBody : *StatementList*

It is a Syntax Error if *StatementList* Contains *NewTarget*.

18.2.1.1.2 Additional Early Error Rules for Eval Outside Methods

These **static semantics** are applied by `PerformEval` when a **direct eval** call occurs outside of a *MethodDefinition*.

ScriptBody : *StatementList*

It is a Syntax Error if *StatementList* Contains *SuperProperty*.

18.2.1.1.3 Additional Early Error Rules for Eval Outside Constructor Methods

These **static semantics** are applied by `PerformEval` when a **direct eval** call occurs outside of the **constructor method** of a *ClassDeclaration* or *ClassExpression*.

ScriptBody : *StatementList*

It is a Syntax Error if *StatementList* Contains *SuperCall*.

18.2.1.2 HostEnsureCanCompileStrings (*callerRealm*, *calleeRealm*)

`HostEnsureCanCompileStrings` is an implementation-defined abstract operation that allows host environments to block certain ECMAScript functions which allow developers to compile strings into ECMAScript code.

An implementation of `HostEnsureCanCompileStrings` may complete normally or abruptly. Any abrupt completions will be propagated to its callers. The default implementation of `HostEnsureCanCompileStrings` is to unconditionally return an empty normal completion.

18.2.1.3 Runtime Semantics: EvalDeclarationInstantiation (*body*, *varEnv*, *lexEnv*, *strict*)

When the abstract operation `EvalDeclarationInstantiation` is called with arguments *body*, *varEnv*, *lexEnv*, and *strict*, the following steps are taken:

1. Let *varNames* be the `VarDeclaredNames` of *body*.
2. Let *varDeclarations* be the `VarScopedDeclarations` of *body*.
3. Let *lexEnvRec* be *lexEnv*'s `EnvironmentRecord`.
4. Let *varEnvRec* be *varEnv*'s `EnvironmentRecord`.
5. If *strict* is **false**, then
 - a. If *varEnvRec* is a global `Environment Record`, then
 - i. For each *name* in *varNames*, do

1. If `varEnvRec.HasLexicalDeclaration(name)` is **true**, throw a **SyntaxError** exception.
2. NOTE: **eval** will not create a global var declaration that would be shadowed by a global lexical declaration.
- b. Let `thisLex` be `lexEnv`.
- c. **Assert**: The following loop will terminate.
- d. Repeat, while `thisLex` is not the same as `varEnv`,
 - i. Let `thisEnvRec` be `thisLex`'s **EnvironmentRecord**.
 - ii. If `thisEnvRec` is not an object **Environment Record**, then
 1. NOTE: The environment of with statements cannot contain any lexical declaration so it doesn't need to be checked for var/let hoisting conflicts.
 2. For each `name` in `varNames`, do
 - a. If `thisEnvRec.HasBinding(name)` is **true**, then
 - i. Throw a **SyntaxError** exception.
 - ii. NOTE: Annex B.3.5 defines alternate semantics for the above step.
 - b. NOTE: A **direct eval** will not hoist var declaration over a like-named lexical declaration.
 - iii. Set `thisLex` to `thisLex`'s outer environment reference.
6. Let `functionsToInitialize` be a new empty **List**.
7. Let `declaredFunctionNames` be a new empty **List**.
8. For each `d` in `varDeclarations`, in reverse list order, do
 - a. If `d` is neither a *VariableDeclaration* nor a *ForBinding* nor a *BindingIdentifier*, then
 - i. **Assert**: `d` is either a *FunctionDeclaration*, a *GeneratorDeclaration*, an *AsyncFunctionDeclaration*, or an *AsyncGeneratorDeclaration*.
 - ii. NOTE: If there are multiple function declarations for the same name, the last declaration is used.
 - iii. Let `fn` be the sole element of the **BoundNames** of `d`.
 - iv. If `fn` is not an element of `declaredFunctionNames`, then
 1. If `varEnvRec` is a global **Environment Record**, then
 - a. Let `fnDefinable` be ? `varEnvRec.CanDeclareGlobalFunction(fn)`.
 - b. If `fnDefinable` is **false**, throw a **TypeError** exception.
 2. Append `fn` to `declaredFunctionNames`.
 3. Insert `d` as the first element of `functionsToInitialize`.
9. NOTE: Annex B.3.3.3 adds additional steps at this point.
10. Let `declaredVarNames` be a new empty **List**.
11. For each `d` in `varDeclarations`, do
 - a. If `d` is a *VariableDeclaration*, a *ForBinding*, or a *BindingIdentifier*, then
 - i. For each String `vn` in the **BoundNames** of `d`, do
 1. If `vn` is not an element of `declaredFunctionNames`, then
 - a. If `varEnvRec` is a global **Environment Record**, then
 - i. Let `vnDefinable` be ? `varEnvRec.CanDeclareGlobalVar(vn)`.
 - ii. If `vnDefinable` is **false**, throw a **TypeError** exception.
 - b. If `vn` is not an element of `declaredVarNames`, then
 - i. Append `vn` to `declaredVarNames`.
12. NOTE: No abnormal terminations occur after this algorithm step unless `varEnvRec` is a global **Environment Record** and the **global object** is a Proxy **exotic object**.
13. Let `lexDeclarations` be the **LexicallyScopedDeclarations** of `body`.
14. For each element `d` in `lexDeclarations`, do
 - a. NOTE: Lexically declared names are only instantiated here but not initialized.
 - b. For each element `dn` of the **BoundNames** of `d`, do
 - i. If **IsConstantDeclaration** of `d` is **true**, then

1. Perform ? *lexEnvRec*.CreateImmutableBinding(*dn*, **true**).
- ii. Else,
 1. Perform ? *lexEnvRec*.CreateMutableBinding(*dn*, **false**).
15. For each *Parse Node f* in *functionsToInitialize*, do
 - a. Let *fn* be the sole element of the BoundNames of *f*.
 - b. Let *fo* be the result of performing InstantiateFunctionObject for *f* with argument *lexEnv*.
 - c. If *varEnvRec* is a global *Environment Record*, then
 - i. Perform ? *varEnvRec*.CreateGlobalFunctionBinding(*fn*, *fo*, **true**).
 - d. Else,
 - i. Let *bindingExists* be *varEnvRec*.HasBinding(*fn*).
 - ii. If *bindingExists* is **false**, then
 1. Let *status* be ! *varEnvRec*.CreateMutableBinding(*fn*, **true**).
 2. **Assert**: *status* is not an *abrupt completion* because of validation preceding step 12.
 3. Perform ! *varEnvRec*.InitializeBinding(*fn*, *fo*).
 - iii. Else,
 1. Perform ! *varEnvRec*.SetMutableBinding(*fn*, *fo*, **false**).
16. For each String *vn* in *declaredVarNames*, in list order, do
 - a. If *varEnvRec* is a global *Environment Record*, then
 - i. Perform ? *varEnvRec*.CreateGlobalVarBinding(*vn*, **true**).
 - b. Else,
 - i. Let *bindingExists* be *varEnvRec*.HasBinding(*vn*).
 - ii. If *bindingExists* is **false**, then
 1. Let *status* be ! *varEnvRec*.CreateMutableBinding(*vn*, **true**).
 2. **Assert**: *status* is not an *abrupt completion* because of validation preceding step 12.
 3. Perform ! *varEnvRec*.InitializeBinding(*vn*, **undefined**).
17. Return *NormalCompletion*(empty).

NOTE

An alternative version of this algorithm is described in B.3.5.

18.2.2 isFinite (*number*)

The **isFinite** function is the %isFinite% intrinsic object. When the **isFinite** function is called with one argument *number*, the following steps are taken:

1. Let *num* be ? *ToNumber*(*number*).
2. If *num* is NaN, +∞, or -∞, return **false**.
3. Otherwise, return **true**.

18.2.3 isNaN (*number*)

The **isNaN** function is the %isNaN% intrinsic object. When the **isNaN** function is called with one argument *number*, the following steps are taken:

1. Let *num* be ? *ToNumber*(*number*).
2. If *num* is NaN, return **true**.
3. Otherwise, return **false**.

NOTE

A reliable way for ECMAScript code to test if a value **X** is a **NaN** is an expression of the form **X !== X**. The result will be **true** if and only if **X** is a **NaN**.

18.2.4 parseFloat (*string*)

The **parseFloat** function produces a Number value dictated by interpretation of the contents of the *string* argument as a decimal literal.

The **parseFloat** function is the %parseFloat% intrinsic object. When the **parseFloat** function is called with one argument *string*, the following steps are taken:

1. Let *inputString* be ? ToString(*string*).
2. Let *trimmedString* be a substring of *inputString* consisting of the leftmost code unit that is not a *StrWhiteSpaceChar* and all code units to the right of that code unit. (In other words, remove leading white space.) If *inputString* does not contain any such code units, let *trimmedString* be the empty string.
3. If neither *trimmedString* nor any prefix of *trimmedString* satisfies the syntax of a *StrDecimalLiteral* (see 7.1.3.1), return **NaN**.
4. Let *numberString* be the longest prefix of *trimmedString*, which might be *trimmedString* itself, that satisfies the syntax of a *StrDecimalLiteral*.
5. Let *mathFloat* be MV of *numberString*.
6. If *mathFloat* = 0, then
 - a. If the first code unit of *trimmedString* is the code unit 0x002D (HYPHEN-MINUS), return **-0**.
 - b. Return **+0**.
7. Return the Number value for *mathFloat*.

NOTE

parseFloat may interpret only a leading portion of *string* as a Number value; it ignores any code units that cannot be interpreted as part of the notation of a decimal literal, and no indication is given that any such code units were ignored.

18.2.5 parseInt (*string*, *radix*)

The **parseInt** function produces an integer value dictated by interpretation of the contents of the *string* argument according to the specified *radix*. Leading white space in *string* is ignored. If *radix* is **undefined** or 0, it is assumed to be 10 except when the number begins with the code unit pairs **0x** or **0X**, in which case a radix of 16 is assumed. If *radix* is 16, the number may also optionally begin with the code unit pairs **0x** or **0X**.

The **parseInt** function is the %parseInt% intrinsic object. When the **parseInt** function is called, the following steps are taken:

1. Let *inputString* be ? ToString(*string*).
2. Let *S* be a newly created substring of *inputString* consisting of the first code unit that is not a *StrWhiteSpaceChar* and all code units following that code unit. (In other words, remove leading white space.) If *inputString* does not contain any such code unit, let *S* be the empty string.
3. Let *sign* be 1.
4. If *S* is not empty and the first code unit of *S* is the code unit 0x002D (HYPHEN-MINUS), set *sign* to -1.
5. If *S* is not empty and the first code unit of *S* is the code unit 0x002B (PLUS SIGN) or the code unit 0x002D

- (HYPHEN-MINUS), remove the first code unit from *S*.
6. Let *R* be ? *ToInt32(radix)*.
 7. Let *stripPrefix* be **true**.
 8. If *R* ≠ 0, then
 - a. If *R* < 2 or *R* > 36, return **NaN**.
 - b. If *R* ≠ 16, set *stripPrefix* to **false**.
 9. Else *R* = 0,
 - a. Set *R* to 10.
 10. If *stripPrefix* is **true**, then
 - a. If the length of *S* is at least 2 and the first two code units of *S* are either "0x" or "0X", then
 - i. Remove the first two code units from *S*.
 - ii. Set *R* to 16.
 11. If *S* contains a code unit that is not a radix-*R* digit, let *Z* be the substring of *S* consisting of all code units before the first such code unit; otherwise, let *Z* be *S*.
 12. If *Z* is empty, return **NaN**.
 13. Let *mathInt* be the mathematical integer value that is represented by *Z* in radix-*R* notation, using the letters **A-Z** and **a-z** for digits with values 10 through 35. (However, if *R* is 10 and *Z* contains more than 20 significant digits, every significant digit after the 20th may be replaced by a 0 digit, at the option of the implementation; and if *R* is not 2, 4, 8, 10, 16, or 32, then *mathInt* may be an implementation-dependent approximation to the mathematical integer value that is represented by *Z* in radix-*R* notation.)
 14. If *mathInt* = 0, then
 - a. If *sign* = -1, return **-0**.
 - b. Return **+0**.
 15. Let *number* be the Number value for *mathInt*.
 16. Return *sign* × *number*.

NOTE

parseInt may interpret only a leading portion of *string* as an integer value; it ignores any code units that cannot be interpreted as part of the notation of an integer, and no indication is given that any such code units were ignored.

18.2.6 URI Handling Functions

Uniform Resource Identifiers, or URIs, are Strings that identify resources (e.g. web pages or files) and transport protocols by which to access them (e.g. HTTP or FTP) on the Internet. The ECMAScript language itself does not provide any support for using URIs except for functions that encode and decode URIs as described in 18.2.6.2, 18.2.6.3, 18.2.6.4 and 18.2.6.5

NOTE

Many implementations of ECMAScript provide additional functions and methods that manipulate web pages; these functions are beyond the scope of this standard.

18.2.6.1 URI Syntax and Semantics

A URI is composed of a sequence of components separated by component separators. The general form is:

Scheme : *First* / *Second* ; *Third* ? *Fourth*

where the italicized names represent components and “:”, “/”, “;” and “?” are reserved for use as separators. The **encodeURI** and **decodeURI** functions are intended to work with complete URIs; they assume that any reserved code units in the URI are intended to have special meaning and so are not encoded. The **encodeURIComponent** and **decodeURIComponent** functions are intended to work with the individual component parts of a URI; they assume that any reserved code units represent text and so must be encoded so that they are not interpreted as reserved code units when the component is part of a complete URI.

The following lexical grammar specifies the form of encoded URIs.

Syntax

uri :::
*uriCharacters*_{opt}

uriCharacters :::
uriCharacter *uriCharacters*_{opt}

uriCharacter :::
uriReserved
uriUnescaped
uriEscaped

uriReserved ::: **one of**
 ; / ? : @ & = + \$,

uriUnescaped :::
uriAlpha
DecimalDigit
uriMark

uriEscaped :::
 % *HexDigit* *HexDigit*

uriAlpha ::: **one of**
 a b c d e f g h i j k l m n o p q r s t u v w x y z A B C D E F G H I J K L M N O
 P Q R S T U V W X Y Z

uriMark ::: **one of**
 - _ . ! ~ * ' ()

NOTE

The above syntax is based upon RFC 2396 and does not reflect changes introduced by the more recent RFC 3986.

Runtime Semantics

When a code unit to be included in a URI is not listed above or is not intended to have the special meaning sometimes given to the reserved code units, that code unit must be encoded. The code unit is transformed into its UTF-8 encoding, with [surrogate pairs](#) first converted from UTF-16 to the corresponding code point value. (Note that for code units in the range [0, 127] this results in a single octet with the same value.) The resulting sequence of octets is then transformed into a String with each octet represented by an escape sequence of the form "%**xx**".

18.2.6.1.1 Runtime Semantics: Encode (*string*, *unescapedSet*)

The encoding and escaping process is described by the abstract operation Encode taking two String arguments *string* and *unescapedSet*.

1. Let *strLen* be the number of code units in *string*.
2. Let *R* be the empty String.
3. Let *k* be 0.
4. Repeat,
 - a. If *k* equals *strLen*, return *R*.
 - b. Let *C* be the code unit at index *k* within *string*.
 - c. If *C* is in *unescapedSet*, then
 - i. Let *S* be the String value containing only the code unit *C*.
 - ii. Set *R* to the string-concatenation of the previous value of *R* and *S*.
 - d. Else *C* is not in *unescapedSet*,
 - i. If *C* is a trailing surrogate, throw a **URIError** exception.
 - ii. If *C* is not a leading surrogate, then
 1. Let *V* be the code point with the same numeric value as code unit *C*.
 - iii. Else,
 1. Increase *k* by 1.
 2. If *k* equals *strLen*, throw a **URIError** exception.
 3. Let *kChar* be the code unit at index *k* within *string*.
 4. If *kChar* is not a trailing surrogate, throw a **URIError** exception.
 5. Let *V* be UTF16Decode(*C*, *kChar*).
 - iv. Let *Octets* be the List of octets resulting by applying the UTF-8 transformation to *V*.
 - v. For each element *octet* of *Octets* in List order, do
 1. Let *S* be the string-concatenation of:

"%"

the String representation of *octet*, formatted as a two-digit uppercase hexadecimal number,
padded to the left with a zero if necessary

 - 2. Set *R* to the string-concatenation of the previous value of *R* and *S*.
 - e. Increase *k* by 1.

18.2.6.1.2 Runtime Semantics: Decode (*string*, *reservedSet*)

The unescaping and decoding process is described by the abstract operation Decode taking two String arguments *string* and *reservedSet*.

1. Let *strLen* be the number of code units in *string*.
2. Let *R* be the empty String.
3. Let *k* be 0.
4. Repeat,
 - a. If *k* equals *strLen*, return *R*.
 - b. Let *C* be the code unit at index *k* within *string*.
 - c. If *C* is not the code unit 0x0025 (PERCENT SIGN), then
 - i. Let *S* be the String value containing only the code unit *C*.
 - d. Else *C* is the code unit 0x0025 (PERCENT SIGN),
 - i. Let *start* be *k*.
 - ii. If *k* + 2 is greater than or equal to *strLen*, throw a **URIError** exception.
 - iii. If the code units at index (*k* + 1) and (*k* + 2) within *string* do not represent hexadecimal digits, throw a

URIError exception.

- iv. Let *B* be the 8-bit value represented by the two hexadecimal digits at index $(k + 1)$ and $(k + 2)$.
- v. Increment *k* by 2.
- vi. If the most significant bit in *B* is 0, then
 1. Let *C* be the code unit whose value is *B*.
 2. If *C* is not in *reservedSet*, then
 - a. Let *S* be the String value containing only the code unit *C*.
 3. Else *C* is in *reservedSet*,
 - a. Let *S* be the substring of *string* from index *start* to index *k* inclusive.
- vii. Else the most significant bit in *B* is 1,
 1. Let *n* be the smallest nonnegative integer such that $(B \ll n) \& 0x80$ is equal to 0.
 2. If *n* equals 1 or *n* is greater than 4, throw a **URIError** exception.
 3. Let *Octets* be a List of 8-bit integers of size *n*.
 4. Set *Octets*[0] to *B*.
 5. If $k + (3 \times (n - 1))$ is greater than or equal to *strLen*, throw a **URIError** exception.
 6. Let *j* be 1.
 7. Repeat, while $j < n$
 - a. Increment *k* by 1.
 - b. If the code unit at index *k* within *string* is not the code unit 0x0025 (PERCENT SIGN), throw a **URIError** exception.
 - c. If the code units at index $(k + 1)$ and $(k + 2)$ within *string* do not represent hexadecimal digits, throw a **URIError** exception.
 - d. Let *B* be the 8-bit value represented by the two hexadecimal digits at index $(k + 1)$ and $(k + 2)$.
 - e. If the two most significant bits in *B* are not 10, throw a **URIError** exception.
 - f. Increment *k* by 2.
 - g. Set *Octets*[*j*] to *B*.
 - h. Increment *j* by 1.
 8. If *Octets* does not contain a valid UTF-8 encoding of a Unicode code point, throw a **URIError** exception.
 9. Let *V* be the value obtained by applying the UTF-8 transformation to *Octets*, that is, from a List of octets into a 21-bit value.
 10. Let *S* be the String value whose code units are, in order, the elements in *UTF16Encoding*(*V*).
- e. Set *R* to the *string-concatenation* of the previous value of *R* and *S*.
- f. Increase *k* by 1.

NOTE

This syntax of Uniform Resource Identifiers is based upon RFC 2396 and does not reflect the more recent RFC 3986 which replaces RFC 2396. A formal description and implementation of UTF-8 is given in RFC 3629.

In UTF-8, characters are encoded using sequences of 1 to 6 octets. The only octet of a sequence of one has the higher-order bit set to 0, the remaining 7 bits being used to encode the character value. In a sequence of *n* octets, $n > 1$, the initial octet has the *n* higher-order bits set to 1, followed by a bit set to 0. The remaining bits of that octet contain bits from the value of the character to be encoded. The following octets all have the higher-order bit set to 1 and the following bit set to 0, leaving 6 bits in each to contain bits from the character to be encoded. The possible UTF-8 encodings of ECMAScript characters are specified in Table 46.

Table 46 (Informative): UTF-8 Encodings

Code Unit Value	Representation	1 st Octet	2 nd Octet	3 rd Octet	4 th Octet
0x0000 – 0x007F	00000000 0zzzzzzz	0zzzzzzz			
0x0080 – 0x07FF	00000yyy yyzzzzzz	110yyyyy	10zzzzzz		
0x0800 – 0xD7FF	xxxxyyyy yyzzzzzz	1110xxxx	10yyyyyy	10zzzzzz	
0xD800 – 0xDBFF	110110vv vvwwwxx	11110uuu	10uuwww	10xyyyyy	10zzzzzz
followed by	followed by				
0xDC00 – 0xDFFF	110111yy yyzzzzzz				
0xD800 – 0xDBFF	causes URIError				
not followed by					
0xDC00 – 0xDFFF					
0xDC00 – 0xDFFF	causes URIError				
0xE000 – 0xFFFF	xxxxyyyy yyzzzzzz	1110xxxx	10yyyyyy	10zzzzzz	

Where

uuuuu = *vvvv* + 1

to account for the addition of 0x10000 as in section 3.8 of the Unicode Standard (Surrogates).

The above transformation combines each [surrogate pair](#) (for which code unit values in the inclusive range 0xD800 to 0xDFFF are reserved) into a UTF-32 representation and encodes the resulting 21-bit value into UTF-8. Decoding reconstructs the [surrogate pair](#).

RFC 3629 prohibits the decoding of invalid UTF-8 octet sequences. For example, the invalid sequence C0 80 must not decode into the code unit 0x0000. Implementations of the Decode algorithm are required to throw a **URIError** when encountering such invalid sequences.

18.2.6.2 decodeURI (*encodedURI*)

The **decodeURI** function computes a new version of a URI in which each escape sequence and UTF-8 encoding of the sort that might be introduced by the **encodeURI** function is replaced with the UTF-16 encoding of the code points that it represents. Escape sequences that could not have been introduced by **encodeURI** are not replaced.

The **decodeURI** function is the %decodeURI% intrinsic object. When the **decodeURI** function is called with one argument *encodedURI*, the following steps are taken:

1. Let *uriString* be ? ToString(*encodedURI*).
2. Let *reservedURISet* be a String containing one instance of each code unit valid in *uriReserved* plus "#".
3. Return ? Decode(*uriString*, *reservedURISet*).

NOTE

The code point "#" is not decoded from escape sequences even though it is not a reserved URI code point.

18.2.6.3 decodeURIComponent (*encodedURIComponent*)

The **decodeURIComponent** function computes a new version of a URI in which each escape sequence and UTF-8 encoding of the sort that might be introduced by the **encodeURIComponent** function is replaced with the UTF-16 encoding of the code points that it represents.

The **decodeURIComponent** function is the %decodeURIComponent% intrinsic object. When the **decodeURIComponent** function is called with one argument *encodedURIComponent*, the following steps are taken:

1. Let *componentString* be ? ToString(*encodedURIComponent*).
2. Let *reservedURIComponentSet* be the empty String.
3. Return ? Decode(*componentString*, *reservedURIComponentSet*).

18.2.6.4 encodeURI (*uri*)

The **encodeURI** function computes a new version of a UTF-16 encoded (6.1.4) URI in which each instance of certain code points is replaced by one, two, three, or four escape sequences representing the UTF-8 encoding of the code points.

The **encodeURI** function is the %encodeURI% intrinsic object. When the **encodeURI** function is called with one argument *uri*, the following steps are taken:

1. Let *uriString* be ? ToString(*uri*).
2. Let *unescapedURISet* be a String containing one instance of each code unit valid in *uriReserved* and *uriUnescaped* plus "#".
3. Return ? Encode(*uriString*, *unescapedURISet*).

NOTE

The code unit "#" is not encoded to an escape sequence even though it is not a reserved or unescaped URI code point.

18.2.6.5 encodeURIComponent (*uriComponent*)

The **encodeURIComponent** function computes a new version of a UTF-16 encoded (6.1.4) URI in which each instance of certain code points is replaced by one, two, three, or four escape sequences representing the UTF-8 encoding of the code point.

The **encodeURIComponent** function is the %encodeURIComponent% intrinsic object. When the **encodeURIComponent** function is called with one argument *uriComponent*, the following steps are taken:

1. Let *componentString* be ? ToString(*uriComponent*).
2. Let *unescapedURIComponentSet* be a String containing one instance of each code unit valid in *uriUnescaped*.
3. Return ? Encode(*componentString*, *unescapedURIComponentSet*).

18.3 Constructor Properties of the Global Object

18.3.1 Array (. . .)

See 22.1.1.

18.3.2 ArrayBuffer (. . .)

See [24.1.2](#).

18.3.3 Boolean (. . .)

See [19.3.1](#).

18.3.4 DataView (. . .)

See [24.3.2](#).

18.3.5 Date (. . .)

See [20.3.2](#).

18.3.6 Error (. . .)

See [19.5.1](#).

18.3.7 EvalError (. . .)

See [19.5.5.1](#).

18.3.8 Float32Array (. . .)

See [22.2.4](#).

18.3.9 Float64Array (. . .)

See [22.2.4](#).

18.3.10 Function (. . .)

See [19.2.1](#).

18.3.11 Int8Array (. . .)

See [22.2.4](#).

18.3.12 Int16Array (. . .)

See [22.2.4](#).

18.3.13 Int32Array (. . .)

See [22.2.4](#).

18.3.14 Map (. . .)

See [23.1.1](#).

18.3.15 Number (. . .)

See [20.1.1](#).

18.3.16 Object (. . .)

See [19.1.1](#).

18.3.17 Promise (. . .)

See [25.6.3](#).

18.3.18 Proxy (. . .)

See [26.2.1](#).

18.3.19 RangeError (. . .)

See [19.5.5.2](#).

18.3.20 ReferenceError (. . .)

See [19.5.5.3](#).

18.3.21 RegExp (. . .)

See [21.2.3](#).

18.3.22 Set (. . .)

See [23.2.1](#).

18.3.23 SharedArrayBuffer (. . .)

See [24.2.2](#).

18.3.24 String (. . .)

See [21.1.1](#).

18.3.25 Symbol (. . .)

See [19.4.1](#).

18.3.26 SyntaxError (. . .)

See [19.5.5.4](#).

18.3.27 TypeError (. . .)

See [19.5.5.5](#).

18.3.28 Uint8Array (. . .)

See [22.2.4](#).

18.3.29 Uint8ClampedArray (. . .)

See [22.2.4](#).

18.3.30 Uint16Array (. . .)

See [22.2.4](#).

18.3.31 Uint32Array (. . .)

See [22.2.4](#).

18.3.32 URIError (. . .)

See [19.5.5.6](#).

18.3.33 WeakMap (. . .)

See [23.3.1](#).

18.3.34 WeakSet (. . .)

See [23.4](#).

18.4 Other Properties of the Global Object

18.4.1 Atomics

See [24.4](#).

18.4.2 JSON

See [24.5](#).

18.4.3 Math

See 20.2.

18.4.4 Reflect

See 26.1.

19 Fundamental Objects

19.1 Object Objects

19.1.1 The Object Constructor

The Object [constructor](#):

is the intrinsic object %Object%.

is the initial value of the **Object** property of the [global object](#).

creates a new ordinary object when called as a [constructor](#).

performs a type conversion when called as a function rather than as a [constructor](#).

is designed to be subclassable. It may be used as the value of an **extends** clause of a class definition.

19.1.1.1 Object ([[value](#)])

When the **Object** function is called with optional argument [value](#), the following steps are taken:

1. If NewTarget is neither **undefined** nor the active function, then
 - a. Return ? [OrdinaryCreateFromConstructor](#)(NewTarget, "%ObjectPrototype%").
2. If [value](#) is **null**, **undefined** or not supplied, return [ObjectCreate](#)(%ObjectPrototype%).
3. Return ! [ToObject](#)([value](#)).

The "**length**" property of the **Object** [constructor](#) function is 1.

19.1.2 Properties of the Object Constructor

The Object [constructor](#):

has a [[Prototype]] internal slot whose value is the intrinsic object %FunctionPrototype%.

has a "**length**" property.

has the following additional properties:

19.1.2.1 Object.assign ([target](#), ...[sources](#))

The **assign** function is used to copy the values of all of the enumerable own properties from one or more source objects to a [target](#) object. When the **assign** function is called, the following steps are taken:

1. Let [to](#) be ? [ToObject](#)([target](#)).

2. If only one argument was passed, return *to*.
3. Let *sources* be the **List** of argument values starting with the second argument.
4. For each element *nextSource* of *sources*, in ascending index order, do
 - a. If *nextSource* is neither **undefined** nor **null**, then
 - i. Let *from* be ! **ToObject**(*nextSource*).
 - ii. Let *keys* be ? *from*.[[OwnPropertyKeys]]().
 - iii. For each element *nextKey* of *keys* in **List** order, do
 1. Let *desc* be ? *from*.[[GetOwnProperty]](*nextKey*).
 2. If *desc* is not **undefined** and *desc*.[[Enumerable]] is **true**, then
 - a. Let *propValue* be ? **Get**(*from*, *nextKey*).
 - b. Perform ? **Set**(*to*, *nextKey*, *propValue*, **true**).
5. Return *to*.

The "**length**" property of the **assign** function is 2.

19.1.2.2 Object.create (*O*, *Properties*)

The **create** function creates a new object with a specified prototype. When the **create** function is called, the following steps are taken:

1. If **Type**(*O*) is neither Object nor Null, throw a **TypeError** exception.
2. Let *obj* be **ObjectCreate**(*O*).
3. If *Properties* is not **undefined**, then
 - a. Return ? **ObjectDefineProperties**(*obj*, *Properties*).
4. Return *obj*.

19.1.2.3 Object.defineProperties (*O*, *Properties*)

The **defineProperties** function is used to add own properties and/or update the attributes of existing own properties of an object. When the **defineProperties** function is called, the following steps are taken:

1. Return ? **ObjectDefineProperties**(*O*, *Properties*).

19.1.2.3.1 Runtime Semantics: ObjectDefineProperties (*O*, *Properties*)

The abstract operation **ObjectDefineProperties** with arguments *O* and *Properties* performs the following steps:

1. If **Type**(*O*) is not Object, throw a **TypeError** exception.
2. Let *props* be ? **ToObject**(*Properties*).
3. Let *keys* be ? *props*.[[OwnPropertyKeys]]().
4. Let *descriptors* be a new empty **List**.
5. For each element *nextKey* of *keys* in **List** order, do
 - a. Let *propDesc* be ? *props*.[[GetOwnProperty]](*nextKey*).
 - b. If *propDesc* is not **undefined** and *propDesc*.[[Enumerable]] is **true**, then
 - i. Let *descObj* be ? **Get**(*props*, *nextKey*).
 - ii. Let *desc* be ? **ToPropertyDescriptor**(*descObj*).
 - iii. Append the pair (a two element **List**) consisting of *nextKey* and *desc* to the end of *descriptors*.
6. For each *pair* from *descriptors* in list order, do
 - a. Let *P* be the first element of *pair*.
 - b. Let *desc* be the second element of *pair*.

- c. Perform ? [DefinePropertyOrThrow](#)(*O*, *P*, *desc*).
7. Return *O*.

19.1.2.4 **Object.defineProperty** (*O*, *P*, *Attributes*)

The **defineProperty** function is used to add an own property and/or update the attributes of an existing own property of an object. When the **defineProperty** function is called, the following steps are taken:

1. If [Type](#)(*O*) is not Object, throw a **TypeError** exception.
2. Let *key* be ? [ToPropertyKey](#)(*P*).
3. Let *desc* be ? [ToPropertyDescriptor](#)(*Attributes*).
4. Perform ? [DefinePropertyOrThrow](#)(*O*, *key*, *desc*).
5. Return *O*.

19.1.2.5 **Object.entries** (*O*)

When the **entries** function is called with argument *O*, the following steps are taken:

1. Let *obj* be ? [ToObject](#)(*O*).
2. Let *nameList* be ? [EnumerableOwnPropertyNames](#)(*obj*, "key+value").
3. Return [CreateArrayFromList](#)(*nameList*).

19.1.2.6 **Object.freeze** (*O*)

When the **freeze** function is called, the following steps are taken:

1. If [Type](#)(*O*) is not Object, return *O*.
2. Let *status* be ? [SetIntegrityLevel](#)(*O*, "frozen").
3. If *status* is **false**, throw a **TypeError** exception.
4. Return *O*.

19.1.2.7 **Object.fromEntries** (*iterable*)

When the **fromEntries** method is called with argument *iterable*, the following steps are taken:

1. Perform ? [RequireObjectCoercible](#)(*iterable*).
2. Let *obj* be [ObjectCreate](#)(%ObjectPrototype%).
3. **Assert**: *obj* is an extensible ordinary object with no own properties.
4. Let *stepsDefine* be the algorithm steps defined in [CreateDataPropertyOnObject Functions](#).
5. Let *adder* be [CreateBuiltinFunction](#)(*stepsDefine*, « »).
6. Return ? [AddEntriesFromIterable](#)(*obj*, *iterable*, *adder*).

NOTE

The function created for *adder* is never directly accessible to ECMAScript code.

19.1.2.7.1 **CreateDataPropertyOnObject Functions**

A [CreateDataPropertyOnObject](#) function is an anonymous built-in function. When a [CreateDataPropertyOnObject](#) function is called with arguments *key* and *value*, the following steps are taken:

1. Let *O* be the **this** value.
2. **Assert**: *Type*(*O*) is Object.
3. **Assert**: *O* is an extensible ordinary object.
4. Let *propertyKey* be ? *ToPropertyKey*(*key*).
5. Perform ! *CreateDataPropertyOrThrow*(*O*, *propertyKey*, *value*).
6. Return **undefined**.

19.1.2.8 *Object.getOwnPropertyDescriptor* (*O*, *P*)

When the **getOwnPropertyDescriptor** function is called, the following steps are taken:

1. Let *obj* be ? *ToObject*(*O*).
2. Let *key* be ? *ToPropertyKey*(*P*).
3. Let *desc* be ? *obj*.[[*GetOwnProperty*]](*key*).
4. Return *FromPropertyDescriptor*(*desc*).

19.1.2.9 *Object.getOwnPropertyDescriptors* (*O*)

When the **getOwnPropertyDescriptors** function is called, the following steps are taken:

1. Let *obj* be ? *ToObject*(*O*).
2. Let *ownKeys* be ? *obj*.[[*OwnPropertyKeys*]]().
3. Let *descriptors* be ! *ObjectCreate*(%*ObjectPrototype*%).
4. For each element *key* of *ownKeys* in *List* order, do
 - a. Let *desc* be ? *obj*.[[*GetOwnProperty*]](*key*).
 - b. Let *descriptor* be ! *FromPropertyDescriptor*(*desc*).
 - c. If *descriptor* is not **undefined**, perform ! *CreateDataProperty*(*descriptors*, *key*, *descriptor*).
5. Return *descriptors*.

19.1.2.10 *Object.getOwnPropertyNames* (*O*)

When the **getOwnPropertyNames** function is called, the following steps are taken:

1. Return ? *GetOwnPropertyKeys*(*O*, String).

19.1.2.11 *Object.getOwnPropertySymbols* (*O*)

When the **getOwnPropertySymbols** function is called with argument *O*, the following steps are taken:

1. Return ? *GetOwnPropertyKeys*(*O*, Symbol).

19.1.2.11.1 Runtime Semantics: *GetOwnPropertyKeys* (*O*, *type*)

The abstract operation *GetOwnPropertyKeys* is called with arguments *O* and *type* where *O* is an Object and *type* is one of the ECMAScript specification types String or Symbol. The following steps are taken:

1. Let *obj* be ? *ToObject*(*O*).
2. Let *keys* be ? *obj*.[[*OwnPropertyKeys*]]().
3. Let *nameList* be a new empty *List*.
4. For each element *nextKey* of *keys* in *List* order, do
 - a. If *Type*(*nextKey*) is *type*, then

- i. Append *nextKey* as the last element of *nameList*.
5. Return `CreateArrayFromList(nameList)`.

19.1.2.12 `Object.getPrototypeOf ( O )`

When the **getPrototypeOf** function is called with argument *O*, the following steps are taken:

1. Let *obj* be `? ToObject(O)`.
2. Return `? obj.[[GetPrototypeOf]]()`.

19.1.2.13 `Object.is ( value1, value2 )`

When the **is** function is called with arguments *value1* and *value2*, the following steps are taken:

1. Return `SameValue(value1, value2)`.

19.1.2.14 `Object.isExtensible ( O )`

When the **isExtensible** function is called with argument *O*, the following steps are taken:

1. If `Type(O)` is not **Object**, return **false**.
2. Return `? IsExtensible(O)`.

19.1.2.15 `Object.isFrozen ( O )`

When the **isFrozen** function is called with argument *O*, the following steps are taken:

1. If `Type(O)` is not **Object**, return **true**.
2. Return `? TestIntegrityLevel(O, "frozen")`.

19.1.2.16 `Object.isSealed ( O )`

When the **isSealed** function is called with argument *O*, the following steps are taken:

1. If `Type(O)` is not **Object**, return **true**.
2. Return `? TestIntegrityLevel(O, "sealed")`.

19.1.2.17 `Object.keys ( O )`

When the **keys** function is called with argument *O*, the following steps are taken:

1. Let *obj* be `? ToObject(O)`.
2. Let *nameList* be `? EnumerableOwnPropertyNames(obj, "key")`.
3. Return `CreateArrayFromList(nameList)`.

19.1.2.18 `Object.preventExtensions ( O )`

When the **preventExtensions** function is called, the following steps are taken:

1. If `Type(O)` is not **Object**, return *O*.
2. Let *status* be `? O.[[PreventExtensions]]()`.

3. If *status* is **false**, throw a **TypeError** exception.
4. Return *O*.

19.1.2.19 Object.prototype

The initial value of **Object.prototype** is the intrinsic object %ObjectPrototype%.

This property has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **false** }.

19.1.2.20 Object.seal (*O*)

When the **seal** function is called, the following steps are taken:

1. If **Type**(*O*) is not Object, return *O*.
2. Let *status* be ? **SetIntegrityLevel**(*O*, "sealed").
3. If *status* is **false**, throw a **TypeError** exception.
4. Return *O*.

19.1.2.21 Object.setPrototypeOf (*O*, *proto*)

When the **setPrototypeOf** function is called with arguments *O* and *proto*, the following steps are taken:

1. Set *O* to ? **RequireObjectCoercible**(*O*).
2. If **Type**(*proto*) is neither Object nor Null, throw a **TypeError** exception.
3. If **Type**(*O*) is not Object, return *O*.
4. Let *status* be ? *O*.[[SetPrototypeOf]](*proto*).
5. If *status* is **false**, throw a **TypeError** exception.
6. Return *O*.

19.1.2.22 Object.values (*O*)

When the **values** function is called with argument *O*, the following steps are taken:

1. Let *obj* be ? **ToObject**(*O*).
2. Let *nameList* be ? **EnumerableOwnPropertyNames**(*obj*, "value").
3. Return **CreateArrayFromList**(*nameList*).

19.1.3 Properties of the Object Prototype Object

The Object prototype object:

is the intrinsic object %ObjectPrototype%.
is an **immutable prototype exotic object**.
has a [[Prototype]] internal slot whose value is **null**.

19.1.3.1 Object.prototype.constructor

The initial value of **Object.prototype.constructor** is the intrinsic object %Object%.

19.1.3.2 Object.prototype.hasOwnProperty (*V*)

When the **hasOwnProperty** method is called with argument *V*, the following steps are taken:

1. Let *P* be ? **ToPropertyKey**(*V*).
2. Let *O* be ? **ToObject**(**this** value).
3. Return ? **HasOwnProperty**(*O*, *P*).

NOTE

The ordering of steps 1 and 2 is chosen to ensure that any exception that would have been thrown by step 1 in previous editions of this specification will continue to be thrown even if the **this** value is **undefined** or **null**.

19.1.3.3 Object.prototype.isPrototypeOf (*V*)

When the **isPrototypeOf** method is called with argument *V*, the following steps are taken:

1. If **Type**(*V*) is not Object, return **false**.
2. Let *O* be ? **ToObject**(**this** value).
3. Repeat,
 - a. Set *V* to ? *V*.[[GetPrototypeOf]]().
 - b. If *V* is **null**, return **false**.
 - c. If **SameValue**(*O*, *V*) is **true**, return **true**.

NOTE

The ordering of steps 1 and 2 preserves the behaviour specified by previous editions of this specification for the case where *V* is not an object and the **this** value is **undefined** or **null**.

19.1.3.4 Object.prototype.propertyIsEnumerable (*V*)

When the **propertyIsEnumerable** method is called with argument *V*, the following steps are taken:

1. Let *P* be ? **ToPropertyKey**(*V*).
2. Let *O* be ? **ToObject**(**this** value).
3. Let *desc* be ? *O*.[[GetOwnProperty]](*P*).
4. If *desc* is **undefined**, return **false**.
5. Return *desc*.[[Enumerable]].

NOTE 1

This method does not consider objects in the prototype chain.

NOTE 2

The ordering of steps 1 and 2 is chosen to ensure that any exception that would have been thrown by step 1 in previous editions of this specification will continue to be thrown even if the **this** value is **undefined** or **null**.

19.1.3.5 Object.prototype.toLocaleString ([*reserved1* [, *reserved2*]])

When the **toLocaleString** method is called, the following steps are taken:

1. Let *O* be the **this** value.
2. Return ? **Invoke**(*O*, "**toString**").

The optional parameters to this function are not used but are intended to correspond to the parameter pattern used by ECMA-402 **toLocaleString** functions. Implementations that do not include ECMA-402 support must not use those parameter positions for other purposes.

NOTE 1

This function provides a generic **toLocaleString** implementation for objects that have no locale-specific **toString** behaviour. **Array**, **Number**, **Date**, and **Typed Arrays** provide their own locale-sensitive **toLocaleString** methods.

NOTE 2

ECMA-402 intentionally does not provide an alternative to this default implementation.

19.1.3.6 Object.prototype.toString ()

When the **toString** method is called, the following steps are taken:

1. If the **this** value is **undefined**, return "[object Undefined]".
2. If the **this** value is **null**, return "[object Null]".
3. Let *O* be ! **ToObject**(**this** value).
4. Let *isArray* be ? **isArray**(*O*).
5. If *isArray* is **true**, let *builtinTag* be "**Array**".
6. Else if *O* is a String **exotic object**, let *builtinTag* be "**String**".
7. Else if *O* has a [[ParameterMap]] internal slot, let *builtinTag* be "**Arguments**".
8. Else if *O* has a [[Call]] internal method, let *builtinTag* be "**Function**".
9. Else if *O* has an [[ErrorData]] internal slot, let *builtinTag* be "**Error**".
10. Else if *O* has a [[BooleanData]] internal slot, let *builtinTag* be "**Boolean**".
11. Else if *O* has a [[NumberData]] internal slot, let *builtinTag* be "**Number**".
12. Else if *O* has a [[DateValue]] internal slot, let *builtinTag* be "**Date**".
13. Else if *O* has a [[RegExpMatcher]] internal slot, let *builtinTag* be "**RegExp**".
14. Else, let *builtinTag* be "**Object**".
15. Let *tag* be ? **Get**(*O*, @@toStringTag).
16. If **Type**(*tag*) is not String, set *tag* to *builtinTag*.
17. Return the **string-concatenation** of "[object ", *tag*, and "]".

This function is the %ObjProto_toString% intrinsic object.

NOTE

Historically, this function was occasionally used to access the String value of the [[Class]] internal slot that was used in previous editions of this specification as a nominal type tag for various built-in objects. The above definition of **toString** preserves compatibility for legacy code that uses **toString** as a test for those specific kinds of built-in objects. It does not provide a reliable type testing mechanism for other kinds of built-in or program defined objects. In addition, programs can use @@toStringTag in ways that will invalidate the reliability of such legacy type tests.

19.1.3.7 Object.prototype.valueOf ()

When the **valueOf** method is called, the following steps are taken:

1. Return ? **ToObject**(**this** value).

This function is the %ObjProto_valueOf% intrinsic object.

19.1.4 Properties of Object Instances

Object instances have no special properties beyond those inherited from the Object prototype object.

19.2 Function Objects

19.2.1 The Function Constructor

The Function [constructor](#):

is the intrinsic object %Function%.

is the initial value of the **Function** property of the [global object](#).

creates and initializes a new [function object](#) when called as a function rather than as a [constructor](#). Thus the function call **Function(...)** is equivalent to the object creation expression **new Function(...)** with the same arguments.

is designed to be subclassable. It may be used as the value of an **extends** clause of a class definition. Subclass constructors that intend to inherit the specified **Function** behaviour must include a **super** call to the **Function constructor** to create and initialize a subclass instance with the internal slots necessary for built-in function behaviour. All ECMAScript syntactic forms for defining function objects create instances of **Function**. There is no syntactic means to create instances of **Function** subclasses except for the built-in **GeneratorFunction**, **AsyncFunction**, and **AsyncGeneratorFunction** subclasses.

19.2.1.1 Function (*p1*, *p2*, ... , *pn*, *body*)

The last argument specifies the body (executable code) of a function; any preceding arguments specify formal parameters.

When the **Function** function is called with some arguments *p1*, *p2*, ... , *pn*, *body* (where *n* might be 0, that is, there are no “*p*” arguments, and where *body* might also not be provided), the following steps are taken:

1. Let *C* be the [active function object](#).
2. Let *args* be the [argumentsList](#) that was passed to this function by [[Call]] or [[Construct]].
3. Return ? [CreateDynamicFunction](#)(*C*, *NewTarget*, "normal", *args*).

NOTE

It is permissible but not necessary to have one argument for each formal parameter to be specified. For example, all three of the following expressions produce the same result:

```
new Function("a", "b", "c", "return a+b+c")
new Function("a, b, c", "return a+b+c")
new Function("a,b", "c", "return a+b+c")
```

19.2.1.1.1 Runtime Semantics: CreateDynamicFunction (*constructor*, *newTarget*, *kind*, *args*)

The abstract operation CreateDynamicFunction is called with arguments *constructor*, *newTarget*, *kind*, and *args*.

constructor is the *constructor* function that is performing this action, *newTarget* is the *constructor* that **new** was initially applied to, *kind* is either "normal", "generator", "async", or "async generator", and *args* is a *List* containing the actual argument values that were passed to *constructor*. The following steps are taken:

1. **Assert**: The *execution context stack* has at least two elements.
2. Let *callerContext* be the second to top element of the *execution context stack*.
3. Let *callerRealm* be *callerContext*'s *Realm*.
4. Let *calleeRealm* be the current *Realm Record*.
5. Perform ? *HostEnsureCanCompileStrings*(*callerRealm*, *calleeRealm*).
6. If *newTarget* is **undefined**, set *newTarget* to *constructor*.
7. If *kind* is "normal", then
 - a. Let *goal* be the grammar symbol *FunctionBody*_[~Yield, ~Await].
 - b. Let *parameterGoal* be the grammar symbol *FormalParameters*_[~Yield, ~Await].
 - c. Let *fallbackProto* be "%FunctionPrototype%".
8. Else if *kind* is "generator", then
 - a. Let *goal* be the grammar symbol *GeneratorBody*.
 - b. Let *parameterGoal* be the grammar symbol *FormalParameters*_[+Yield, ~Await].
 - c. Let *fallbackProto* be "%Generator%".
9. Else if *kind* is "async", then
 - a. Let *goal* be the grammar symbol *AsyncFunctionBody*.
 - b. Let *parameterGoal* be the grammar symbol *FormalParameters*_[~Yield, +Await].
 - c. Let *fallbackProto* be "%AsyncFunctionPrototype%".
10. Else,
 - a. **Assert**: *kind* is "async generator".
 - b. Let *goal* be the grammar symbol *AsyncGeneratorBody*.
 - c. Let *parameterGoal* be the grammar symbol *FormalParameters*_[+Yield, +Await].
 - d. Let *fallbackProto* be "%AsyncGenerator%".
11. Let *argCount* be the number of elements in *args*.
12. Let *P* be the empty String.
13. If *argCount* = 0, let *bodyText* be the empty String.
14. Else if *argCount* = 1, let *bodyText* be *args*[0].
15. Else *argCount* > 1,
 - a. Let *firstArg* be *args*[0].
 - b. Set *P* to ? *ToString*(*firstArg*).
 - c. Let *k* be 1.
 - d. Repeat, while *k* < *argCount* - 1
 - i. Let *nextArg* be *args*[*k*].
 - ii. Let *nextArgString* be ? *ToString*(*nextArg*).
 - iii. Set *P* to the *string-concatenation* of the previous value of *P*, ", " (a comma), and *nextArgString*.
 - iv. Increase *k* by 1.
 - e. Let *bodyText* be *args*[*k*].
16. Set *bodyText* to ? *ToString*(*bodyText*).
17. Let *parameters* be the result of parsing *P*, interpreted as UTF-16 encoded Unicode text as described in 6.1.4, using *parameterGoal* as the *goal symbol*. Throw a **SyntaxError** exception if the parse fails.
18. Let *body* be the result of parsing *bodyText*, interpreted as UTF-16 encoded Unicode text as described in 6.1.4, using *goal* as the *goal symbol*. Throw a **SyntaxError** exception if the parse fails.
19. Let *strict* be *ContainsUseStrict* of *body*.
20. If any *static semantics* errors are detected for *parameters* or *body*, throw a **SyntaxError** or a **ReferenceError**.

- exception, depending on the type of the error. If *strict* is **true**, the Early Error rules for *UniqueFormalParameters* : *FormalParameters* are applied. Parsing and **early error** detection may be interweaved in an implementation-dependent manner.
21. If *strict* is **true** and *IsSimpleParameterList* of *parameters* is **false**, throw a **SyntaxError** exception.
 22. If any element of the *BoundNames* of *parameters* also occurs in the *LexicallyDeclaredNames* of *body*, throw a **SyntaxError** exception.
 23. If *body* *Contains SuperCall* is **true**, throw a **SyntaxError** exception.
 24. If *parameters* *Contains SuperCall* is **true**, throw a **SyntaxError** exception.
 25. If *body* *Contains SuperProperty* is **true**, throw a **SyntaxError** exception.
 26. If *parameters* *Contains SuperProperty* is **true**, throw a **SyntaxError** exception.
 27. If *kind* is **"generator"** or **"async generator"**, then
 - a. If *parameters* *Contains YieldExpression* is **true**, throw a **SyntaxError** exception.
 28. If *kind* is **"async"** or **"async generator"**, then
 - a. If *parameters* *Contains AwaitExpression* is **true**, throw a **SyntaxError** exception.
 29. If *strict* is **true**, then
 - a. If *BoundNames* of *parameters* contains any duplicate elements, throw a **SyntaxError** exception.
 30. Let *proto* be ? *GetPrototypeFromConstructor*(*newTarget*, *fallbackProto*).
 31. Let *F* be *FunctionAllocate*(*proto*, *strict*, *kind*).
 32. Let *realmF* be *F*.[[*Realm*]].
 33. Let *scope* be *realmF*.[[*GlobalEnv*]].
 34. Perform *FunctionInitialize*(*F*, **Normal**, *parameters*, *body*, *scope*).
 35. If *kind* is **"generator"**, then
 - a. Let *prototype* be *ObjectCreate*(%*GeneratorPrototype*%).
 - b. Perform *DefinePropertyOrThrow*(*F*, **"prototype"**, *PropertyDescriptor* { [[*Value*]]: *prototype*, [[*Writable*]]: **true**, [[*Enumerable*]]: **false**, [[*Configurable*]]: **false** }).
 36. Else if *kind* is **"async generator"**, then
 - a. Let *prototype* be *ObjectCreate*(%*AsyncGeneratorPrototype*%).
 - b. Perform *DefinePropertyOrThrow*(*F*, **"prototype"**, *PropertyDescriptor* { [[*Value*]]: *prototype*, [[*Writable*]]: **true**, [[*Enumerable*]]: **false**, [[*Configurable*]]: **false** }).
 37. Else if *kind* is **"normal"**, perform *MakeConstructor*(*F*).
 38. NOTE: Async functions are not constructable and do not have a [[*Construct*]] internal method or a **"prototype"** property.
 39. Perform *SetFunctionName*(*F*, **"anonymous"**).
 40. Let *prefix* be the prefix associated with *kind* in Table 47.
 41. Let *sourceText* be the *string-concatenation* of *prefix*, **" anonymous (" , P, 0x000A (LINE FEED), ") { "**, **0x000A (LINE FEED), bodyText, 0x000A (LINE FEED), and " } "**.
 42. Set *F*.[[*SourceText*]] to *sourceText*.
 43. Return *F*.

NOTE

A **prototype** property is created for every non-async function created using *CreateDynamicFunction* to provide for the possibility that the function will be used as a **constructor**.

Table 47: Dynamic Function *SourceText* Prefixes

Kind	Prefix
"normal"	"function"

"generator"	"function*"
"async"	"async function"
"async generator"	"async function*"

19.2.2 Properties of the Function Constructor

The Function [constructor](#):

- is itself a built-in [function object](#).

- has a `[[Prototype]]` internal slot whose value is the intrinsic object `%FunctionPrototype%`.

- has the following properties:

19.2.2.1 Function.length

This is a [data property](#) with a value of 1. This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **true** }.

19.2.2.2 Function.prototype

The value of `Function.prototype` is `%FunctionPrototype%`, the intrinsic Function prototype object.

This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **false** }.

19.2.3 Properties of the Function Prototype Object

The Function prototype object:

- is the intrinsic object `%FunctionPrototype%`.

- is itself a built-in [function object](#).

- accepts any arguments and returns **undefined** when invoked.

- does not have a `[[Construct]]` internal method; it cannot be used as a [constructor](#) with the **new** operator.

- has a `[[Prototype]]` internal slot whose value is the intrinsic object `%ObjectPrototype%`.

- does not have a **prototype** property.

- has a **"length"** property whose value is 0.

- has a **name** property whose value is the empty String.

NOTE

The Function prototype object is specified to be a [function object](#) to ensure compatibility with ECMAScript code that was created prior to the ECMAScript 2015 specification.

19.2.3.1 Function.prototype.apply (*thisArg*, *argArray*)

When the **apply** method is called with arguments *thisArg* and *argArray*, the following steps are taken:

1. Let *func* be the **this** value.
2. If `IsCallable(func)` is **false**, throw a **TypeError** exception.
3. If *argArray* is **undefined** or **null**, then

- a. Perform `PrepareForTailCall()`.
- b. Return ? `Call(func, thisArg)`.
4. Let `argList` be ? `CreateListFromArrayLike(argArray)`.
5. Perform `PrepareForTailCall()`.
6. Return ? `Call(func, thisArg, argList)`.

NOTE 1

The `thisArg` value is passed without modification as the **this** value. This is a change from Edition 3, where an **undefined** or **null** `thisArg` is replaced with the `global object` and `ToObject` is applied to all other values and that result is passed as the **this** value. Even though the `thisArg` is passed without modification, non-strict functions still perform these transformations upon entry to the function.

NOTE 2

If `func` is an arrow function or a `bound function` then the `thisArg` will be ignored by the function `[[Call]]` in step 5.

19.2.3.2 `Function.prototype.bind ( thisArg, ...args )`

When the **bind** method is called with argument `thisArg` and zero or more `args`, it performs the following steps:

1. Let `Target` be the **this** value.
2. If `IsCallable(Target)` is **false**, throw a **TypeError** exception.
3. Let `args` be a new (possibly empty) `List` consisting of all of the argument values provided after `thisArg` in order.
4. Let `F` be ? `BoundFunctionCreate(Target, thisArg, args)`.
5. Let `targetHasLength` be ? `HasOwnProperty(Target, "length")`.
6. If `targetHasLength` is **true**, then
 - a. Let `targetLen` be ? `Get(Target, "length")`.
 - b. If `Type(targetLen)` is not `Number`, let `L` be 0.
 - c. Else,
 - i. Set `targetLen` to ! `ToInteger(targetLen)`.
 - ii. Let `L` be the larger of 0 and the result of `targetLen` minus the number of elements of `args`.
7. Else, let `L` be 0.
8. Perform ! `SetFunctionLength(F, L)`.
9. Let `targetName` be ? `Get(Target, "name")`.
10. If `Type(targetName)` is not `String`, set `targetName` to the empty string.
11. Perform `SetFunctionName(F, targetName, "bound")`.
12. Return `F`.

NOTE 1

Function objects created using `Function.prototype.bind` are exotic objects. They also do not have a **prototype** property.

NOTE 2

If `Target` is an arrow function or a `bound function` then the `thisArg` passed to this method will not be used by subsequent calls to `F`.

19.2.3.3 `Function.prototype.call ( thisArg, ...args )`

When the **call** method is called with argument `thisArg` and zero or more `args`, the following steps are taken:

1. Let `func` be the **this** value.

2. If **IsCallable**(*func*) is **false**, throw a **TypeError** exception.
3. Let *argList* be a new empty **List**.
4. If this method was called with more than one argument, then in left to right order, starting with the second argument, append each argument as the last element of *argList*.
5. Perform **PrepareForTailCall**().
6. Return ? **Call**(*func*, *thisArg*, *argList*).

NOTE 1

The *thisArg* value is passed without modification as the **this** value. This is a change from Edition 3, where an **undefined** or **null** *thisArg* is replaced with the **global object** and **ToObject** is applied to all other values and that result is passed as the **this** value. Even though the *thisArg* is passed without modification, non-strict functions still perform these transformations upon entry to the function.

NOTE 2

If *func* is an arrow function or a **bound function** then the *thisArg* will be ignored by the function **[[Call]]** in step 5.

19.2.3.4 **Function.prototype.constructor**

The initial value of **Function.prototype.constructor** is the intrinsic object **%Function%**.

19.2.3.5 **Function.prototype.toString ()**

When the **toString** method is called, the following steps are taken:

1. Let *func* be the **this** value.
2. If *func* is a **Bound Function exotic object** or a **built-in function object**, then return an implementation-dependent String source code representation of *func*. The representation must have the syntax of a *NativeFunction*. Additionally, if *func* is a **Well-known Intrinsic Object** and is not identified as an anonymous function, the portion of the returned String that would be matched by *PropertyName* must be the initial value of the **name** property of *func*.
3. If **Type**(*func*) is **Object** and *func* has a **[[SourceText]]** internal slot and **Type**(*func*.**[[SourceText]]**) is **String** and **!HostHasSourceTextAvailable**(*func*) is **true**, then return *func*.**[[SourceText]]**.
4. If **Type**(*func*) is **Object** and **IsCallable**(*func*) is **true**, then return an implementation-dependent String source code representation of *func*. The representation must have the syntax of a *NativeFunction*.
5. Throw a **TypeError** exception.

NativeFunction :

```
function PropertyName [~Yield, ~Await] opt ( FormalParameters [~Yield, ~Await] ) { [
  native code ] }
```

19.2.3.6 **Function.prototype [@@hasInstance] (V)**

When the **@@hasInstance** method of an object *F* is called with value *V*, the following steps are taken:

1. Let *F* be the **this** value.
2. Return ? **OrdinaryHasInstance**(*F*, *V*).

The value of the **name** property of this function is "**[Symbol.hasInstance]**".

This property has the attributes { **[[Writable]]: false**, **[[Enumerable]]: false**, **[[Configurable]]: false** }.

NOTE

This is the default implementation of **@@hasInstance** that most functions inherit. **@@hasInstance** is called by the **instanceof** operator to determine whether a value is an instance of a specific **constructor**. An expression such as

```
v instanceof F
```

evaluates as

```
F[@@hasInstance](v)
```

A **constructor** function can control which objects are recognized as its instances by **instanceof** by exposing a different **@@hasInstance** method on the function.

This property is non-writable and non-configurable to prevent tampering that could be used to globally expose the target function of a **bound function**.

19.2.4 Function Instances

Every Function instance is an ECMAScript **function object** and has the internal slots listed in [Table 27](#). Function objects created using the **Function.prototype.bind** method ([19.2.3.2](#)) have the internal slots listed in [Table 28](#).

Function instances have the following properties:

19.2.4.1 length

The value of the **"length"** property is an integer that indicates the typical number of arguments expected by the function. However, the language permits the function to be invoked with some other number of arguments. The behaviour of a function when invoked on a number of arguments other than the number specified by its **"length"** property depends on the function. This property has the attributes { **[[Writable]]**: **false**, **[[Enumerable]]**: **false**, **[[Configurable]]**: **true** }.

19.2.4.2 name

The value of the **name** property is a String that is descriptive of the function. The name has no semantic significance but is typically a variable or **property name** that is used to refer to the function at its point of definition in ECMAScript code. This property has the attributes { **[[Writable]]**: **false**, **[[Enumerable]]**: **false**, **[[Configurable]]**: **true** }.

Anonymous functions objects that do not have a contextual name associated with them by this specification do not have a **name** own property but inherit the **name** property of **%FunctionPrototype%**.

19.2.4.3 prototype

Function instances that can be used as a **constructor** have a **prototype** property. Whenever such a Function instance is created another ordinary object is also created and is the initial value of the function's **prototype** property. Unless otherwise specified, the value of the **prototype** property is used to initialize the **[[Prototype]]** internal slot of the object created when that function is invoked as a **constructor**.

This property has the attributes { **[[Writable]]**: **true**, **[[Enumerable]]**: **false**, **[[Configurable]]**: **false** }.

NOTE

Function objects created using **Function.prototype.bind**, or by evaluating a *MethodDefinition* (that is not a

GeneratorMethod or *AsyncGeneratorMethod*) or an *ArrowFunction* do not have a **prototype** property.

19.2.5 HostHasSourceTextAvailable (*func*)

HostHasSourceTextAvailable is an implementation-defined abstract operation that allows host environments to prevent the source text from being provided for a given function.

An implementation of HostHasSourceTextAvailable must complete normally in all cases. This operation must be deterministic with respect to its parameters. Each time it is called with a specific *func* as its argument, it must return the same completion record. The default implementation of HostHasSourceTextAvailable is to unconditionally return a normal completion with a value of **true**.

19.3 Boolean Objects

19.3.1 The Boolean Constructor

The Boolean *constructor*:

is the intrinsic object **%Boolean%**.

is the initial value of the **Boolean** property of the *global object*.

creates and initializes a new Boolean object when called as a *constructor*.

performs a type conversion when called as a function rather than as a *constructor*.

is designed to be subclassable. It may be used as the value of an **extends** clause of a class definition. Subclass constructors that intend to inherit the specified **Boolean** behaviour must include a **super** call to the **Boolean constructor** to create and initialize the subclass instance with a **[[BooleanData]]** internal slot.

19.3.1.1 Boolean (*value*)

When **Boolean** is called with argument *value*, the following steps are taken:

1. Let *b* be **ToBoolean**(*value*).
2. If **NewTarget** is **undefined**, return *b*.
3. Let *O* be ? **OrdinaryCreateFromConstructor**(**NewTarget**, "**%BooleanPrototype%**", « **[[BooleanData]]** »).
4. Set *O*.**[[BooleanData]]** to *b*.
5. Return *O*.

19.3.2 Properties of the Boolean Constructor

The Boolean *constructor*:

has a **[[Prototype]]** internal slot whose value is the intrinsic object **%FunctionPrototype%**.

has the following properties:

19.3.2.1 Boolean.prototype

The initial value of **Boolean.prototype** is the intrinsic object **%BooleanPrototype%**.

This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **false** }.

19.3.3 Properties of the Boolean Prototype Object

The Boolean prototype object:

is the intrinsic object `%BooleanPrototype%`.

is an ordinary object.

is itself a Boolean object; it has a `[[BooleanData]]` internal slot with the value **false**.

has a `[[Prototype]]` internal slot whose value is the intrinsic object `%ObjectPrototype%`.

The abstract operation `thisBooleanValue(value)` performs the following steps:

1. If `Type(value)` is Boolean, return `value`.
2. If `Type(value)` is Object and `value` has a `[[BooleanData]]` internal slot, then
 - a. Let `b` be `value.[[BooleanData]]`.
 - b. **Assert**: `Type(b)` is Boolean.
 - c. Return `b`.
3. Throw a **TypeError** exception.

19.3.3.1 Boolean.prototype.constructor

The initial value of `Boolean.prototype.constructor` is the intrinsic object `%Boolean%`.

19.3.3.2 Boolean.prototype.toString ()

The following steps are taken:

1. Let `b` be ? `thisBooleanValue(this value)`.
2. If `b` is **true**, return `"true"`; else return `"false"`.

19.3.3.3 Boolean.prototype.valueOf ()

The following steps are taken:

1. Return ? `thisBooleanValue(this value)`.

19.3.4 Properties of Boolean Instances

Boolean instances are ordinary objects that inherit properties from the Boolean prototype object. Boolean instances have a `[[BooleanData]]` internal slot. The `[[BooleanData]]` internal slot is the Boolean value represented by this Boolean object.

19.4 Symbol Objects

19.4.1 The Symbol Constructor

The Symbol `constructor`:

is the intrinsic object `%Symbol%`.
is the initial value of the **Symbol** property of the [global object](#).
returns a new Symbol value when called as a function.
is not intended to be used with the **new** operator.
is not intended to be subclassed.
may be used as the value of an **extends** clause of a class definition but a **super** call to it will cause an exception.

19.4.1.1 Symbol ([*description*])

When **Symbol** is called with optional argument *description*, the following steps are taken:

- 1. If NewTarget is not **undefined**, throw a **TypeError** exception.
- 2. If *description* is **undefined**, let *descString* be **undefined**.
- 3. Else, let *descString* be ? [ToString](#)(*description*).
- 4. Return a new unique Symbol value whose `[[Description]]` value is *descString*.

19.4.2 Properties of the Symbol Constructor

The Symbol [constructor](#):

has a `[[Prototype]]` internal slot whose value is the intrinsic object `%FunctionPrototype%`.
has the following properties:

19.4.2.1 Symbol.asyncIterator

The initial value of **Symbol.asyncIterator** is the well known symbol `@@asyncIterator` ([Table 1](#)).

This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **false** }.

19.4.2.2 Symbol.for (*key*)

When **Symbol.for** is called with argument *key* it performs the following steps:

- 1. Let *stringKey* be ? [ToString](#)(*key*).
- 2. For each element *e* of the GlobalSymbolRegistry [List](#), do
 - a. If [SameValue](#)(*e*.`[[Key]]`, *stringKey*) is **true**, return *e*.`[[Symbol]]`.
- 3. [Assert](#): GlobalSymbolRegistry does not currently contain an entry for *stringKey*.
- 4. Let *newSymbol* be a new unique Symbol value whose `[[Description]]` value is *stringKey*.
- 5. Append the [Record](#) { `[[Key]]`: *stringKey*, `[[Symbol]]`: *newSymbol* } to the GlobalSymbolRegistry [List](#).
- 6. Return *newSymbol*.

The GlobalSymbolRegistry is a [List](#) that is globally available. It is shared by all realms. Prior to the evaluation of any ECMAScript code it is initialized as a new empty [List](#). Elements of the GlobalSymbolRegistry are Records with the structure defined in [Table 48](#).

Table 48: GlobalSymbolRegistry [Record](#) Fields

Field Name	Value	Usage
<code>[[Key]]</code>	A String	A string key used to globally identify a Symbol.

[[Symbol]] A Symbol A symbol that can be retrieved from any [realm](#).

19.4.2.3 Symbol.hasInstance

The initial value of **Symbol.hasInstance** is the well-known symbol @@hasInstance ([Table 1](#)).

This property has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **false** }.

19.4.2.4 Symbol.isConcatSpreadable

The initial value of **Symbol.isConcatSpreadable** is the well-known symbol @@isConcatSpreadable ([Table 1](#)).

This property has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **false** }.

19.4.2.5 Symbol.iterator

The initial value of **Symbol.iterator** is the well-known symbol @@iterator ([Table 1](#)).

This property has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **false** }.

19.4.2.6 Symbol.keyFor (*sym*)

When **Symbol.keyFor** is called with argument *sym* it performs the following steps:

1. If **Type**(*sym*) is not Symbol, throw a **TypeError** exception.
2. For each element *e* of the GlobalSymbolRegistry [List](#) (see [19.4.2.2](#)), do
 - a. If **SameValue**(*e*.[[Symbol]], *sym*) is **true**, return *e*.[[Key]].
3. **Assert**: GlobalSymbolRegistry does not currently contain an entry for *sym*.
4. Return **undefined**.

19.4.2.7 Symbol.match

The initial value of **Symbol.match** is the well-known symbol @@match ([Table 1](#)).

This property has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **false** }.

19.4.2.8 Symbol.prototype

The initial value of **Symbol.prototype** is the intrinsic object %SymbolPrototype%.

This property has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **false** }.

19.4.2.9 Symbol.replace

The initial value of **Symbol.replace** is the well-known symbol @@replace ([Table 1](#)).

This property has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **false** }.

19.4.2.10 Symbol.search

The initial value of **Symbol**.**search** is the well-known symbol @@search (Table 1).

This property has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **false** }.

19.4.2.11 Symbol.species

The initial value of **Symbol**.**species** is the well-known symbol @@species (Table 1).

This property has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **false** }.

19.4.2.12 Symbol.split

The initial value of **Symbol**.**split** is the well-known symbol @@split (Table 1).

This property has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **false** }.

19.4.2.13 Symbol.toPrimitive

The initial value of **Symbol**.**toPrimitive** is the well-known symbol @@toPrimitive (Table 1).

This property has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **false** }.

19.4.2.14 Symbol.toStringTag

The initial value of **Symbol**.**toStringTag** is the well-known symbol @@toStringTag (Table 1).

This property has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **false** }.

19.4.2.15 Symbol.unscopables

The initial value of **Symbol**.**unscopables** is the well-known symbol @@unscopables (Table 1).

This property has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **false** }.

19.4.3 Properties of the Symbol Prototype Object

The Symbol prototype object:

is the intrinsic object %SymbolPrototype%.

is an ordinary object.

is not a Symbol instance and does not have a [[SymbolData]] internal slot.

has a [[Prototype]] internal slot whose value is the intrinsic object %ObjectPrototype%.

The abstract operation thisSymbolValue(*value*) performs the following steps:

1. If Type(*value*) is Symbol, return *value*.
2. If Type(*value*) is Object and *value* has a [[SymbolData]] internal slot, then
 - a. Let *s* be *value*.[[SymbolData]].
 - b. Assert: Type(*s*) is Symbol.
 - c. Return *s*.
3. Throw a **TypeError** exception.

19.4.3.1 Symbol.prototype.constructor

The initial value of **Symbol.prototype.constructor** is the intrinsic object %Symbol%.

19.4.3.2 get Symbol.prototype.description

Symbol.prototype.description is an [accessor property](#) whose set accessor function is **undefined**. Its get accessor function performs the following steps:

1. Let *s* be the **this** value.
2. Let *sym* be ? [thisSymbolValue](#)(*s*).
3. Return *sym*.[[Description]].

19.4.3.3 Symbol.prototype.toString ()

The following steps are taken:

1. Let *sym* be ? [thisSymbolValue](#)(**this** value).
2. Return [SymbolDescriptiveString](#)(*sym*).

19.4.3.3.1 Runtime Semantics: SymbolDescriptiveString (*sym*)

When the abstract operation [SymbolDescriptiveString](#) is called with argument *sym*, the following steps are taken:

1. [Assert: Type](#)(*sym*) is Symbol.
2. Let *desc* be *sym*'s [[Description]] value.
3. If *desc* is **undefined**, set *desc* to the empty string.
4. [Assert: Type](#)(*desc*) is String.
5. Return the [string-concatenation](#) of "Symbol(", *desc*, and ")".

19.4.3.4 Symbol.prototype.valueOf ()

The following steps are taken:

1. Return ? [thisSymbolValue](#)(**this** value).

19.4.3.5 Symbol.prototype [@@toPrimitive] (*hint*)

This function is called by ECMAScript language operators to convert a Symbol object to a primitive value. The allowed values for *hint* are "default", "number", and "string".

When the @@toPrimitive method is called with argument *hint*, the following steps are taken:

1. Return ? [thisSymbolValue](#)(**this** value).

The value of the **name** property of this function is "[Symbol.toPrimitive]".

This property has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **true** }.

19.4.3.6 Symbol.prototype [@@toStringTag]

The initial value of the @@toStringTag property is the String value "Symbol".

This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **true** }.

19.4.4 Properties of Symbol Instances

Symbol instances are ordinary objects that inherit properties from the Symbol prototype object. Symbol instances have a `[[SymbolData]]` internal slot. The `[[SymbolData]]` internal slot is the Symbol value represented by this Symbol object.

19.5 Error Objects

Instances of Error objects are thrown as exceptions when runtime errors occur. The Error objects may also serve as base objects for user-defined exception classes.

19.5.1 The Error Constructor

The Error [constructor](#):

is the intrinsic object `%Error%`.

is the initial value of the **Error** property of the [global object](#).

creates and initializes a new Error object when called as a function rather than as a [constructor](#). Thus the function call **Error**(...) is equivalent to the object creation expression **new Error**(...) with the same arguments.

is designed to be subclassable. It may be used as the value of an **extends** clause of a class definition. Subclass constructors that intend to inherit the specified **Error** behaviour must include a **super** call to the **Error** [constructor](#) to create and initialize subclass instances with an `[[ErrorData]]` internal slot.

19.5.1.1 Error (*message*)

When the **Error** function is called with argument *message*, the following steps are taken:

1. If *NewTarget* is **undefined**, let *newTarget* be the [active function object](#), else let *newTarget* be *NewTarget*.
2. Let *O* be ? [OrdinaryCreateFromConstructor](#)(*newTarget*, `"%ErrorPrototype%"`, « `[[ErrorData]]` »).
3. If *message* is not **undefined**, then
 - a. Let *msg* be ? [ToString](#)(*message*).
 - b. Let *msgDesc* be the PropertyDescriptor { `[[Value]]`: *msg*, `[[Writable]]`: **true**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **true** }.
 - c. Perform ! [DefinePropertyOrThrow](#)(*O*, `"message"`, *msgDesc*).
4. Return *O*.

19.5.2 Properties of the Error Constructor

The Error [constructor](#):

has a `[[Prototype]]` internal slot whose value is the intrinsic object `%FunctionPrototype%`.

has the following properties:

19.5.2.1 Error.prototype

The initial value of **Error.prototype** is the intrinsic object `%ErrorPrototype%`.

This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **false** }.

19.5.3 Properties of the Error Prototype Object

The Error prototype object:

- is the intrinsic object `%ErrorPrototype%`.

- is an ordinary object.

- is not an Error instance and does not have an `[[ErrorData]]` internal slot.

- has a `[[Prototype]]` internal slot whose value is the intrinsic object `%ObjectPrototype%`.

19.5.3.1 `Error.prototype.constructor`

The initial value of `Error.prototype.constructor` is the intrinsic object `%Error%`.

19.5.3.2 `Error.prototype.message`

The initial value of `Error.prototype.message` is the empty String.

19.5.3.3 `Error.prototype.name`

The initial value of `Error.prototype.name` is `"Error"`.

19.5.3.4 `Error.prototype.toString ( )`

The following steps are taken:

1. Let *O* be the **this** value.
2. If `Type(O)` is not Object, throw a **TypeError** exception.
3. Let *name* be ? `Get(O, "name")`.
4. If *name* is **undefined**, set *name* to `"Error"`; otherwise set *name* to ? `ToString(name)`.
5. Let *msg* be ? `Get(O, "message")`.
6. If *msg* is **undefined**, set *msg* to the empty String; otherwise set *msg* to ? `ToString(msg)`.
7. If *name* is the empty String, return *msg*.
8. If *msg* is the empty String, return *name*.
9. Return the **string-concatenation** of *name*, the code unit 0x003A (COLON), the code unit 0x0020 (SPACE), and *msg*.

19.5.4 Properties of Error Instances

Error instances are ordinary objects that inherit properties from the Error prototype object and have an `[[ErrorData]]` internal slot whose value is **undefined**. The only specified uses of `[[ErrorData]]` is to identify Error and *NativeError* instances as Error objects within `Object.prototype.toString`.

19.5.5 Native Error Types Used in This Standard

A new instance of one of the *NativeError* objects below is thrown when a runtime error is detected. All of these objects share the same structure, as described in 19.5.6.

19.5.5.1 EvalError

This exception is not currently used within this specification. This object remains for compatibility with previous editions of this specification.

19.5.5.2 RangeError

Indicates a value that is not in the set or range of allowable values.

19.5.5.3 ReferenceError

Indicate that an invalid reference value has been detected.

19.5.5.4 SyntaxError

Indicates that a parsing error has occurred.

19.5.5.5 TypeError

TypeError is used to indicate an unsuccessful operation when none of the other *NativeError* objects are an appropriate indication of the failure cause.

19.5.5.6 URIError

Indicates that one of the global URI handling functions was used in a way that is incompatible with its definition.

19.5.6 *NativeError* Object Structure

When an ECMAScript implementation detects a runtime error, it throws a new instance of one of the *NativeError* objects defined in 19.5.5. Each of these objects has the structure described below, differing only in the name used as the *constructor* name instead of *NativeError*, in the **name** property of the prototype object, and in the implementation-defined **message** property of the prototype object.

For each error object, references to *NativeError* in the definition should be replaced with the appropriate error object name from 19.5.5.

19.5.6.1 The *NativeError* Constructors

Each *NativeError* constructor:

creates and initializes a new *NativeError* object when called as a function rather than as a *constructor*. A call of the object as a function is equivalent to calling it as a *constructor* with the same arguments. Thus the function call *NativeError*(...) is equivalent to the object creation expression **new** *NativeError*(...) with the same arguments.

is designed to be subclassable. It may be used as the value of an **extends** clause of a class definition. Subclass constructors that intend to inherit the specified *NativeError* behaviour must include a **super** call to the *NativeError* constructor to create and initialize subclass instances with an [[ErrorData]] internal slot.

19.5.6.1.1 *NativeError* (*message*)

When a *NativeError* function is called with argument *message*, the following steps are taken:

1. If *NewTarget* is **undefined**, let *newTarget* be the *active function object*, else let *newTarget* be *NewTarget*.
2. Let *O* be ? *OrdinaryCreateFromConstructor*(*newTarget*, "%*NativeError*Prototype%", « [[*ErrorData*]] »).
3. If *message* is not **undefined**, then
 - a. Let *msg* be ? *ToString*(*message*).
 - b. Let *msgDesc* be the PropertyDescriptor { [[*Value*]]: *msg*, [[*Writable*]]: **true**, [[*Enumerable*]]: **false**, [[*Configurable*]]: **true** }.
 - c. Perform ! *DefinePropertyOrThrow*(*O*, "message", *msgDesc*).
4. Return *O*.

The actual value of the string passed in step 2 is either "%*EvalError*Prototype%", "%*RangeError*Prototype%", "%*ReferenceError*Prototype%", "%*SyntaxError*Prototype%", "%*TypeError*Prototype%", or "%*URIError*Prototype%" corresponding to which *NativeError* constructor is being defined.

19.5.6.2 Properties of the *NativeError* Constructors

Each *NativeError* constructor:

- has a [[*Prototype*]] internal slot whose value is the intrinsic object %*Error*%.
- has a **name** property whose value is the String value "*NativeError*".
- has the following properties:

19.5.6.2.1 *NativeError*.prototype

The initial value of *NativeError*.**prototype** is a *NativeError* prototype object (19.5.6.3). Each *NativeError* constructor has a distinct prototype object.

This property has the attributes { [[*Writable*]]: **false**, [[*Enumerable*]]: **false**, [[*Configurable*]]: **false** }.

19.5.6.3 Properties of the *NativeError* Prototype Objects

Each *NativeError* prototype object:

- is an ordinary object.
- is not an *Error* instance and does not have an [[*ErrorData*]] internal slot.
- has a [[*Prototype*]] internal slot whose value is the intrinsic object %*Error*Prototype%.

19.5.6.3.1 *NativeError*.prototype.constructor

The initial value of the **constructor** property of the prototype for a given *NativeError* constructor is the corresponding intrinsic object %*NativeError*% (19.5.6.1).

19.5.6.3.2 *NativeError*.prototype.message

The initial value of the **message** property of the prototype for a given *NativeError* constructor is the empty String.

19.5.6.3.3 *NativeError*.prototype.name

The initial value of the **name** property of the prototype for a given *NativeError* constructor is the String value consisting

of the name of the [constructor](#) (the name used instead of *NativeError*).

19.5.6.4 Properties of *NativeError* Instances

NativeError instances are ordinary objects that inherit properties from their *NativeError* prototype object and have an `[[ErrorData]]` internal slot whose value is **undefined**. The only specified use of `[[ErrorData]]` is by `Object.prototype.toString` (19.1.3.6) to identify `Error` or *NativeError* instances.

20 Numbers and Dates

20.1 Number Objects

20.1.1 The Number Constructor

The Number [constructor](#):

is the intrinsic object `%Number%`.

is the initial value of the **Number** property of the [global object](#).

creates and initializes a new Number object when called as a [constructor](#).

performs a type conversion when called as a function rather than as a [constructor](#).

is designed to be subclassable. It may be used as the value of an **extends** clause of a class definition. Subclass constructors that intend to inherit the specified **Number** behaviour must include a **super** call to the **Number** [constructor](#) to create and initialize the subclass instance with a `[[NumberData]]` internal slot.

20.1.1.1 Number (*value*)

When **Number** is called with argument *value*, the following steps are taken:

1. If no arguments were passed to this function invocation, let *n* be **+0**.
2. Else, let *n* be ? [ToNumber](#)(*value*).
3. If `NewTarget` is **undefined**, return *n*.
4. Let *O* be ? [OrdinaryCreateFromConstructor](#)(`NewTarget`, "`%NumberPrototype%`", « `[[NumberData]]` »).
5. Set *O*.`[[NumberData]]` to *n*.
6. Return *O*.

20.1.2 Properties of the Number Constructor

The Number [constructor](#):

has a `[[Prototype]]` internal slot whose value is the intrinsic object `%FunctionPrototype%`.

has the following properties:

20.1.2.1 Number.EPSILON

The value of `Number.EPSILON` is the difference between 1 and the smallest value greater than 1 that is representable as a Number value, which is approximately $2.2204460492503130808472633361816 \times 10^{-16}$.

This property has the attributes { `[[Writable]]: false`, `[[Enumerable]]: false`, `[[Configurable]]: false` }.

20.1.2.2 `Number.isFinite ( number )`

When `Number.isFinite` is called with one argument *number*, the following steps are taken:

1. If `Type(number)` is not `Number`, return **false**.
2. If *number* is `NaN`, $+\infty$, or $-\infty$, return **false**.
3. Otherwise, return **true**.

20.1.2.3 `Number.isInteger ( number )`

When `Number.isInteger` is called with one argument *number*, the following steps are taken:

1. If `Type(number)` is not `Number`, return **false**.
2. If *number* is `NaN`, $+\infty$, or $-\infty$, return **false**.
3. Let *integer* be `! ToInteger(number)`.
4. If *integer* is not equal to *number*, return **false**.
5. Otherwise, return **true**.

20.1.2.4 `Number.isNaN ( number )`

When `Number.isNaN` is called with one argument *number*, the following steps are taken:

1. If `Type(number)` is not `Number`, return **false**.
2. If *number* is `NaN`, return **true**.
3. Otherwise, return **false**.

NOTE

This function differs from the global `isNaN` function (18.2.3) in that it does not convert its argument to a `Number` before determining whether it is `NaN`.

20.1.2.5 `Number.isSafeInteger ( number )`

When `Number.isSafeInteger` is called with one argument *number*, the following steps are taken:

1. If `Type(number)` is not `Number`, return **false**.
2. If *number* is `NaN`, $+\infty$, or $-\infty$, return **false**.
3. Let *integer* be `! ToInteger(number)`.
4. If *integer* is not equal to *number*, return **false**.
5. If $\text{abs}(\text{integer}) \leq 2^{53} - 1$, return **true**.
6. Otherwise, return **false**.

20.1.2.6 `Number.MAX_SAFE_INTEGER`

NOTE

The value of `Number.MAX_SAFE_INTEGER` is the largest integer *n* such that *n* and *n* + 1 are both exactly representable as a `Number` value.

The value of `Number.MAX_SAFE_INTEGER` is 9007199254740991 ($2^{53} - 1$).

This property has the attributes { `[[Writable]]: false`, `[[Enumerable]]: false`, `[[Configurable]]: false` }.

20.1.2.7 `Number.MAX_VALUE`

The value of `Number.MAX_VALUE` is the largest positive finite value of the `Number` type, which is approximately $1.7976931348623157 \times 10^{308}$.

This property has the attributes { `[[Writable]]: false`, `[[Enumerable]]: false`, `[[Configurable]]: false` }.

20.1.2.8 `Number.MIN_SAFE_INTEGER`

NOTE

The value of `Number.MIN_SAFE_INTEGER` is the smallest integer n such that n and $n - 1$ are both exactly representable as a `Number` value.

The value of `Number.MIN_SAFE_INTEGER` is -9007199254740991 ($-(2^{53} - 1)$).

This property has the attributes { `[[Writable]]: false`, `[[Enumerable]]: false`, `[[Configurable]]: false` }.

20.1.2.9 `Number.MIN_VALUE`

The value of `Number.MIN_VALUE` is the smallest positive value of the `Number` type, which is approximately 5×10^{-324} .

In the IEEE 754-2008 double precision binary representation, the smallest possible value is a denormalized number. If an implementation does not support denormalized values, the value of `Number.MIN_VALUE` must be the smallest non-zero positive value that can actually be represented by the implementation.

This property has the attributes { `[[Writable]]: false`, `[[Enumerable]]: false`, `[[Configurable]]: false` }.

20.1.2.10 `Number.NaN`

The value of `Number.NaN` is `NaN`.

This property has the attributes { `[[Writable]]: false`, `[[Enumerable]]: false`, `[[Configurable]]: false` }.

20.1.2.11 `Number.NEGATIVE_INFINITY`

The value of `Number.NEGATIVE_INFINITY` is $-\infty$.

This property has the attributes { `[[Writable]]: false`, `[[Enumerable]]: false`, `[[Configurable]]: false` }.

20.1.2.12 `Number.parseFloat ( string )`

The value of the `Number.parseFloat` data property is the same built-in function object that is the value of the `parseFloat` property of the `global` object defined in 18.2.4.

20.1.2.13 `Number.parseInt ( string, radix )`

The value of the **Number.parseInt** data property is the same built-in function object that is the value of the **parseInt** property of the global object defined in 18.2.5.

20.1.2.14 Number.POSITIVE_INFINITY

The value of **Number.POSITIVE_INFINITY** is $+\infty$.

This property has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **false** }.

20.1.2.15 Number.prototype

The initial value of **Number.prototype** is the intrinsic object %NumberPrototype%.

This property has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **false** }.

20.1.3 Properties of the Number Prototype Object

The Number prototype object:

- is the intrinsic object %NumberPrototype%.

- is an ordinary object.

- is itself a Number object; it has a [[NumberData]] internal slot with the value **+0**.

- has a [[Prototype]] internal slot whose value is the intrinsic object %ObjectPrototype%.

Unless explicitly stated otherwise, the methods of the Number prototype object defined below are not generic and the **this** value passed to them must be either a Number value or an object that has a [[NumberData]] internal slot that has been initialized to a Number value.

The abstract operation **thisNumberValue**(*value*) performs the following steps:

1. If **Type**(*value*) is Number, return *value*.
2. If **Type**(*value*) is Object and *value* has a [[NumberData]] internal slot, then
 - a. Let *n* be *value*.[[NumberData]].
 - b. **Assert**: **Type**(*n*) is Number.
 - c. Return *n*.
3. Throw a **TypeError** exception.

The phrase “this Number value” within the specification of a method refers to the result returned by calling the abstract operation **thisNumberValue** with the **this** value of the method invocation passed as the argument.

20.1.3.1 Number.prototype.constructor

The initial value of **Number.prototype.constructor** is the intrinsic object %Number%.

20.1.3.2 Number.prototype.toExponential (*fractionDigits*)

Return a String containing this Number value represented in decimal exponential notation with one digit before the significand's decimal point and *fractionDigits* digits after the significand's decimal point. If *fractionDigits* is **undefined**, include as many significand digits as necessary to uniquely specify the Number (just like in **ToString** except that in this case the Number is always output in exponential notation). Specifically, perform the following steps:

1. Let x be ? `thisNumberValue(this value)`.
2. Let f be ? `ToInteger(fractionDigits)`.
3. **Assert**: If `fractionDigits` is **undefined**, then f is 0.
4. If x is **NaN**, return the String **"NaN"**.
5. Let s be the empty String.
6. If $x < 0$, then
 - a. Set s to **"-"**.
 - b. Set x to $-x$.
7. If $x = +\infty$, then
 - a. Return the **string-concatenation** of s and **"Infinity"**.
8. If $f < 0$ or $f > 100$, throw a **RangeError** exception.
9. If $x = 0$, then
 - a. Let m be the String value consisting of $f + 1$ occurrences of the code unit 0x0030 (DIGIT ZERO).
 - b. Let e be 0.
10. Else $x \neq 0$,
 - a. If `fractionDigits` is not **undefined**, then
 - i. Let e and n be integers such that $10^f \leq n < 10^{f+1}$ and for which the exact mathematical value of $n \times 10^{e-f} - x$ is as close to zero as possible. If there are two such sets of e and n , pick the e and n for which $n \times 10^{e-f}$ is larger.
 - b. Else `fractionDigits` is **undefined**,
 - i. Let e , n , and f be integers such that $f \geq 0$, $10^f \leq n < 10^{f+1}$, the Number value for $n \times 10^{e-f}$ is x , and f is as small as possible. Note that the decimal representation of n has $f + 1$ digits, n is not divisible by 10, and the least significant digit of n is not necessarily uniquely determined by these criteria.
 - c. Let m be the String value consisting of the digits of the decimal representation of n (in order, with no leading zeroes).
11. If $f \neq 0$, then
 - a. Let a be the first code unit of m , and let b be the remaining f code units of m .
 - b. Set m to the **string-concatenation** of a , **"."**, and b .
12. If $e = 0$, then
 - a. Let c be **"+"**.
 - b. Let d be **"0"**.
13. Else,
 - a. If $e > 0$, let c be **"+"**.
 - b. Else $e \leq 0$,
 - i. Let c be **"-"**.
 - ii. Set e to $-e$.
 - c. Let d be the String value consisting of the digits of the decimal representation of e (in order, with no leading zeroes).
14. Set m to the **string-concatenation** of m , **"e"**, c , and d .
15. Return the **string-concatenation** of s and m .

NOTE

For implementations that provide more accurate conversions than required by the rules above, it is recommended that the following alternative version of step 10.b.i be used as a guideline:

1. Let e , n , and f be integers such that $f \geq 0$, $10^f \leq n < 10^{f+1}$, the Number value for $n \times 10^{e-f}$ is x , and f is as small as possible. If there are multiple possibilities for n , choose the value of n for which $n \times 10^{e-f}$ is closest in value to x . If there are two such possible values of n , choose the one that is even.

20.1.3.3 Number.prototype.toFixed (*fractionDigits*)

NOTE 1

toFixed returns a String containing this Number value represented in decimal fixed-point notation with *fractionDigits* digits after the decimal point. If *fractionDigits* is **undefined**, 0 is assumed.

The following steps are performed:

1. Let *x* be ? **thisNumberValue**(**this** value).
2. Let *f* be ? **ToInteger**(*fractionDigits*).
3. **Assert**: If *fractionDigits* is **undefined**, then *f* is 0.
4. If *f* < 0 or *f* > 100, throw a **RangeError** exception.
5. If *x* is **NaN**, return the String "**NaN**".
6. Let *s* be the empty String.
7. If *x* < 0, then
 - a. Set *s* to "-".
 - b. Set *x* to -*x*.
8. If *x* ≥ 10²¹, then
 - a. Let *m* be ! **ToString**(*x*).
9. Else *x* < 10²¹,
 - a. Let *n* be an integer for which the exact mathematical value of $n \div 10^f - x$ is as close to zero as possible. If there are two such *n*, pick the larger *n*.
 - b. If *n* = 0, let *m* be the String "0". Otherwise, let *m* be the String value consisting of the digits of the decimal representation of *n* (in order, with no leading zeroes).
 - c. If *f* ≠ 0, then
 - i. Let *k* be the length of *m*.
 - ii. If *k* ≤ *f*, then
 1. Let *z* be the String value consisting of *f* + 1 - *k* occurrences of the code unit 0x0030 (DIGIT ZERO).
 2. Set *m* to the **string-concatenation** of *z* and *m*.
 3. Set *k* to *f* + 1.
 - iii. Let *a* be the first *k* - *f* code units of *m*, and let *b* be the remaining *f* code units of *m*.
 - iv. Set *m* to the **string-concatenation** of *a*, ".", and *b*.
10. Return the **string-concatenation** of *s* and *m*.

NOTE 2

The output of **toFixed** may be more precise than **toString** for some values because **toString** only prints enough significant digits to distinguish the number from adjacent number values. For example,

`(10000000000000000128).toString()` returns "10000000000000000100", while
`(10000000000000000128).toFixed(0)` returns "10000000000000000128".

20.1.3.4 Number.prototype.toLocaleString ([*reserved1* [, *reserved2*]])

An ECMAScript implementation that includes the ECMA-402 Internationalization API must implement the

Number.prototype.toLocaleString method as specified in the ECMA-402 specification. If an ECMAScript implementation does not include the ECMA-402 API the following specification of the **toLocaleString** method is used.

Produces a String value that represents this Number value formatted according to the conventions of the host environment's current locale. This function is implementation-dependent, and it is permissible, but not encouraged, for it to return the same thing as **toString**.

The meanings of the optional parameters to this method are defined in the ECMA-402 specification; implementations that do not include ECMA-402 support must not use those parameter positions for anything else.

20.1.3.5 Number.prototype.toPrecision (*precision*)

Return a String containing this Number value represented either in decimal exponential notation with one digit before the significand's decimal point and *precision* - 1 digits after the significand's decimal point or in decimal fixed notation with *precision* significant digits. If *precision* is **undefined**, call **ToString** instead. Specifically, perform the following steps:

1. Let *x* be ? **thisNumberValue**(**this** value).
2. If *precision* is **undefined**, return ! **ToString**(*x*).
3. Let *p* be ? **ToInteger**(*precision*).
4. If *x* is **NaN**, return the String "**NaN**".
5. Let *s* be the empty String.
6. If *x* < 0, then
 - a. Set *s* to the code unit 0x002D (HYPHEN-MINUS).
 - b. Set *x* to -*x*.
7. If *x* = +∞, then
 - a. Return the **string-concatenation** of *s* and "**Infinity**".
8. If *p* < 1 or *p* > 100, throw a **RangeError** exception.
9. If *x* = 0, then
 - a. Let *m* be the String value consisting of *p* occurrences of the code unit 0x0030 (DIGIT ZERO).
 - b. Let *e* be 0.
10. Else *x* ≠ 0,
 - a. Let *e* and *n* be integers such that $10^{p-1} \leq n < 10^p$ and for which the exact mathematical value of $n \times 10^{e-p+1} - x$ is as close to zero as possible. If there are two such sets of *e* and *n*, pick the *e* and *n* for which $n \times 10^{e-p+1}$ is larger.
 - b. Let *m* be the String value consisting of the digits of the decimal representation of *n* (in order, with no leading zeroes).
 - c. If *e* < -6 or *e* ≥ *p*, then
 - i. **Assert**: *e* ≠ 0.
 - ii. If *p* ≠ 1, then
 1. Let *a* be the first code unit of *m*, and let *b* be the remaining *p* - 1 code units of *m*.
 2. Set *m* to the **string-concatenation** of *a*, ".", and *b*.
 - iii. If *e* > 0, then
 1. Let *c* be the code unit 0x002B (PLUS SIGN).
 - iv. Else *e* < 0,
 1. Let *c* be the code unit 0x002D (HYPHEN-MINUS).
 2. Set *e* to -*e*.
 - v. Let *d* be the String value consisting of the digits of the decimal representation of *e* (in order, with no leading zeroes).

- vi. Return the **string-concatenation** of *s*, *m*, the code unit 0x0065 (LATIN SMALL LETTER E), *c*, and *d*.
- 11. If $e = p - 1$, return the **string-concatenation** of *s* and *m*.
- 12. If $e \geq 0$, then
 - a. Set *m* to the **string-concatenation** of the first $e + 1$ code units of *m*, the code unit 0x002E (FULL STOP), and the remaining $p - (e + 1)$ code units of *m*.
- 13. Else $e < 0$,
 - a. Set *m* to the **string-concatenation** of the code unit 0x0030 (DIGIT ZERO), the code unit 0x002E (FULL STOP), $-(e + 1)$ occurrences of the code unit 0x0030 (DIGIT ZERO), and the String *m*.
- 14. Return the **string-concatenation** of *s* and *m*.

20.1.3.6 Number.prototype.toString ([*radix*])

NOTE

The optional *radix* should be an integer value in the inclusive range 2 to 36. If *radix* is not present or is **undefined** the Number 10 is used as the value of *radix*.

The following steps are performed:

1. Let *x* be ? **thisNumberValue**(**this** value).
2. If *radix* is not present, let *radixNumber* be 10.
3. Else if *radix* is **undefined**, let *radixNumber* be 10.
4. Else, let *radixNumber* be ? **ToInteger**(*radix*).
5. If $radixNumber < 2$ or $radixNumber > 36$, throw a **RangeError** exception.
6. If $radixNumber = 10$, return ! **ToString**(*x*).
7. Return the String representation of this Number value using the radix specified by *radixNumber*. Letters **a-z** are used for digits with values 10 through 35. The precise algorithm is implementation-dependent, however the algorithm should be a generalization of that specified in 7.1.12.1.

The **toString** function is not generic; it throws a **TypeError** exception if its **this** value is not a Number or a Number object. Therefore, it cannot be transferred to other kinds of objects for use as a method.

The "**length**" property of the **toString** method is 1.

20.1.3.7 Number.prototype.valueOf ()

1. Return ? **thisNumberValue**(**this** value).

20.1.4 Properties of Number Instances

Number instances are ordinary objects that inherit properties from the Number prototype object. Number instances also have a [[NumberData]] internal slot. The [[NumberData]] internal slot is the Number value represented by this Number object.

20.2 The Math Object

The Math object:

is the intrinsic object %Math%.

is the initial value of the **Math** property of the **global object**.

is an ordinary object.

has a `[[Prototype]]` internal slot whose value is the intrinsic object `%ObjectPrototype%`.

is not a `function` object.

does not have a `[[Construct]]` internal method; it cannot be used as a `constructor` with the `new` operator.

does not have a `[[Call]]` internal method; it cannot be invoked as a function.

NOTE

In this specification, the phrase “the Number value for *x*” has a technical meaning defined in 6.1.6.

20.2.1 Value Properties of the Math Object

20.2.1.1 Math.E

The Number value for *e*, the base of the natural logarithms, which is approximately 2.7182818284590452354.

This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **false** }.

20.2.1.2 Math.LN10

The Number value for the natural logarithm of 10, which is approximately 2.302585092994046.

This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **false** }.

20.2.1.3 Math.LN2

The Number value for the natural logarithm of 2, which is approximately 0.6931471805599453.

This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **false** }.

20.2.1.4 Math.LOG10E

The Number value for the base-10 logarithm of *e*, the base of the natural logarithms; this value is approximately 0.4342944819032518.

This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **false** }.

NOTE

The value of `Math.LOG10E` is approximately the reciprocal of the value of `Math.LN10`.

20.2.1.5 Math.LOG2E

The Number value for the base-2 logarithm of *e*, the base of the natural logarithms; this value is approximately 1.4426950408889634.

This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **false** }.

NOTE

The value of `Math.LOG2E` is approximately the reciprocal of the value of `Math.LN2`.

20.2.1.6 Math.PI

The Number value for π , the ratio of the circumference of a circle to its diameter, which is approximately 3.1415926535897932.

This property has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **false** }.

20.2.1.7 Math.SQRT1_2

The Number value for the square root of $\frac{1}{2}$, which is approximately 0.7071067811865476.

This property has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **false** }.

NOTE

The value of **Math.SQRT1_2** is approximately the reciprocal of the value of **Math.SQRT2**.

20.2.1.8 Math.SQRT2

The Number value for the square root of 2, which is approximately 1.4142135623730951.

This property has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **false** }.

20.2.1.9 Math [@@toStringTag]

The initial value of the @@toStringTag property is the String value "**Math**".

This property has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **true** }.

20.2.2 Function Properties of the Math Object

Each of the following **Math** object functions applies the **ToNumber** abstract operation to each of its arguments (in left-to-right order if there is more than one). If **ToNumber** returns an **abrupt completion**, that **Completion Record** is immediately returned. Otherwise, the function performs a computation on the resulting Number value(s). The value returned by each function is a Number.

In the function descriptions below, the symbols **NaN**, **-0**, **+0**, $-\infty$ and $+\infty$ refer to the Number values described in 6.1.6.

NOTE

The behaviour of the functions **acos**, **acosh**, **asin**, **asinh**, **atan**, **atanh**, **atan2**, **cbirt**, **cos**, **cosh**, **exp**, **expm1**, **hypot**, **log**, **log1p**, **log2**, **log10**, **pow**, **random**, **sin**, **sinh**, **sqrt**, **tan**, and **tanh** is not precisely specified here except to require specific results for certain argument values that represent boundary cases of interest. For other argument values, these functions are intended to compute approximations to the results of familiar mathematical functions, but some latitude is allowed in the choice of approximation algorithms. The general intent is that an implementer should be able to use the same mathematical library for ECMAScript on a given hardware platform that is available to C programmers on that platform.

Although the choice of algorithms is left to the implementation, it is recommended (but not specified by this standard) that implementations use the approximation algorithms for IEEE 754-2008 arithmetic contained in **fdlibm**, the freely distributable mathematical library from Sun Microsystems (<http://www.netlib.org/fdlibm>).

20.2.2.1 Math.abs (*x*)

Returns the absolute value of *x*; the result has the same magnitude as *x* but has positive sign.

If *x* is **NaN**, the result is **NaN**.

If *x* is **-0**, the result is **+0**.

If *x* is $-\infty$, the result is $+\infty$.

20.2.2.2 Math.acos (*x*)

Returns an implementation-dependent approximation to the arc cosine of *x*. The result is expressed in radians and ranges from **+0** to $+\pi$.

If *x* is **NaN**, the result is **NaN**.

If *x* is greater than 1, the result is **NaN**.

If *x* is less than -1, the result is **NaN**.

If *x* is exactly 1, the result is **+0**.

20.2.2.3 Math.acosh (*x*)

Returns an implementation-dependent approximation to the inverse hyperbolic cosine of *x*.

If *x* is **NaN**, the result is **NaN**.

If *x* is less than 1, the result is **NaN**.

If *x* is 1, the result is **+0**.

If *x* is $+\infty$, the result is $+\infty$.

20.2.2.4 Math.asin (*x*)

Returns an implementation-dependent approximation to the arc sine of *x*. The result is expressed in radians and ranges from $-\pi / 2$ to $+\pi / 2$.

If *x* is **NaN**, the result is **NaN**.

If *x* is greater than 1, the result is **NaN**.

If *x* is less than -1, the result is **NaN**.

If *x* is **+0**, the result is **+0**.

If *x* is **-0**, the result is **-0**.

20.2.2.5 Math.asinh (*x*)

Returns an implementation-dependent approximation to the inverse hyperbolic sine of *x*.

If *x* is **NaN**, the result is **NaN**.

If *x* is **+0**, the result is **+0**.

If *x* is **-0**, the result is **-0**.

If *x* is $+\infty$, the result is $+\infty$.

If *x* is $-\infty$, the result is $-\infty$.

20.2.2.6 Math.atan (*x*)

Returns an implementation-dependent approximation to the arc tangent of *x*. The result is expressed in radians and ranges

from $-\pi / 2$ to $+\pi / 2$.

If x is **NaN**, the result is **NaN**.

If x is **+0**, the result is **+0**.

If x is **-0**, the result is **-0**.

If x is $+\infty$, the result is an implementation-dependent approximation to $+\pi / 2$.

If x is $-\infty$, the result is an implementation-dependent approximation to $-\pi / 2$.

20.2.2.7 Math.atanh (x)

Returns an implementation-dependent approximation to the inverse hyperbolic tangent of x .

If x is **NaN**, the result is **NaN**.

If x is less than -1, the result is **NaN**.

If x is greater than 1, the result is **NaN**.

If x is -1, the result is $-\infty$.

If x is +1, the result is $+\infty$.

If x is **+0**, the result is **+0**.

If x is **-0**, the result is **-0**.

20.2.2.8 Math.atan2 (y, x)

Returns an implementation-dependent approximation to the arc tangent of the quotient y / x of the arguments y and x , where the signs of y and x are used to determine the quadrant of the result. Note that it is intentional and traditional for the two-argument arc tangent function that the argument named y be first and the argument named x be second. The result is expressed in radians and ranges from $-\pi$ to $+\pi$.

If either x or y is **NaN**, the result is **NaN**.

If $y > 0$ and x is **+0**, the result is an implementation-dependent approximation to $+\pi / 2$.

If $y > 0$ and x is **-0**, the result is an implementation-dependent approximation to $+\pi / 2$.

If y is **+0** and $x > 0$, the result is **+0**.

If y is **+0** and x is **+0**, the result is **+0**.

If y is **+0** and x is **-0**, the result is an implementation-dependent approximation to $+\pi$.

If y is **+0** and $x < 0$, the result is an implementation-dependent approximation to $+\pi$.

If y is **-0** and $x > 0$, the result is **-0**.

If y is **-0** and x is **+0**, the result is **-0**.

If y is **-0** and x is **-0**, the result is an implementation-dependent approximation to $-\pi$.

If y is **-0** and $x < 0$, the result is an implementation-dependent approximation to $-\pi$.

If $y < 0$ and x is **+0**, the result is an implementation-dependent approximation to $-\pi / 2$.

If $y < 0$ and x is **-0**, the result is an implementation-dependent approximation to $-\pi / 2$.

If $y > 0$ and y is finite and x is $+\infty$, the result is **+0**.

If $y > 0$ and y is finite and x is $-\infty$, the result is an implementation-dependent approximation to $+\pi$.

If $y < 0$ and y is finite and x is $+\infty$, the result is **-0**.

If $y < 0$ and y is finite and x is $-\infty$, the result is an implementation-dependent approximation to $-\pi$.

If y is $+\infty$ and x is finite, the result is an implementation-dependent approximation to $+\pi / 2$.

If y is $-\infty$ and x is finite, the result is an implementation-dependent approximation to $-\pi / 2$.

If y is $+\infty$ and x is $+\infty$, the result is an implementation-dependent approximation to $+\pi / 4$.

If y is $+\infty$ and x is $-\infty$, the result is an implementation-dependent approximation to $+3\pi / 4$.

If y is $-\infty$ and x is $+\infty$, the result is an implementation-dependent approximation to $-\pi / 4$.

If y is $-\infty$ and x is $-\infty$, the result is an implementation-dependent approximation to $-3\pi / 4$.

20.2.2.9 Math.cbrt (*x*)

Returns an implementation-dependent approximation to the cube root of *x*.

If *x* is NaN, the result is NaN.

If *x* is +0, the result is +0.

If *x* is -0, the result is -0.

If *x* is $+\infty$, the result is $+\infty$.

If *x* is $-\infty$, the result is $-\infty$.

20.2.2.10 Math.ceil (*x*)

Returns the smallest (closest to $-\infty$) Number value that is not less than *x* and is equal to a mathematical integer. If *x* is already an integer, the result is *x*.

If *x* is NaN, the result is NaN.

If *x* is +0, the result is +0.

If *x* is -0, the result is -0.

If *x* is $+\infty$, the result is $+\infty$.

If *x* is $-\infty$, the result is $-\infty$.

If *x* is less than 0 but greater than -1, the result is -0.

The value of **Math.ceil**(*x*) is the same as the value of **-Math.floor**(-*x*).

20.2.2.11 Math.clz32 (*x*)

When **Math.clz32** is called with one argument *x*, the following steps are taken:

1. Let *n* be ?**ToUint32**(*x*).
2. Let *p* be the number of leading zero bits in the 32-bit binary representation of *n*.
3. Return *p*.

NOTE

If *n* is 0, *p* will be 32. If the most significant bit of the 32-bit binary encoding of *n* is 1, *p* will be 0.

20.2.2.12 Math.cos (*x*)

Returns an implementation-dependent approximation to the cosine of *x*. The argument is expressed in radians.

If *x* is NaN, the result is NaN.

If *x* is +0, the result is 1.

If *x* is -0, the result is 1.

If *x* is $+\infty$, the result is NaN.

If *x* is $-\infty$, the result is NaN.

20.2.2.13 Math.cosh (*x*)

Returns an implementation-dependent approximation to the hyperbolic cosine of *x*.

If x is **NaN**, the result is **NaN**.

If x is **+0**, the result is 1.

If x is **-0**, the result is 1.

If x is $+\infty$, the result is $+\infty$.

If x is $-\infty$, the result is $+\infty$.

NOTE

The value of $\cosh(x)$ is the same as $(\exp(x) + \exp(-x)) / 2$.

20.2.2.14 Math.exp (x)

Returns an implementation-dependent approximation to the exponential function of x (e raised to the power of x , where e is the base of the natural logarithms).

If x is **NaN**, the result is **NaN**.

If x is **+0**, the result is 1.

If x is **-0**, the result is 1.

If x is $+\infty$, the result is $+\infty$.

If x is $-\infty$, the result is **+0**.

20.2.2.15 Math.expm1 (x)

Returns an implementation-dependent approximation to subtracting 1 from the exponential function of x (e raised to the power of x , where e is the base of the natural logarithms). The result is computed in a way that is accurate even when the value of x is close 0.

If x is **NaN**, the result is **NaN**.

If x is **+0**, the result is **+0**.

If x is **-0**, the result is **-0**.

If x is $+\infty$, the result is $+\infty$.

If x is $-\infty$, the result is -1.

20.2.2.16 Math.floor (x)

Returns the greatest (closest to $+\infty$) Number value that is not greater than x and is equal to a mathematical integer. If x is already an integer, the result is x .

If x is **NaN**, the result is **NaN**.

If x is **+0**, the result is **+0**.

If x is **-0**, the result is **-0**.

If x is $+\infty$, the result is $+\infty$.

If x is $-\infty$, the result is $-\infty$.

If x is greater than 0 but less than 1, the result is **+0**.

NOTE

The value of **Math.floor(x)** is the same as the value of **-Math.ceil(- x)**.

20.2.2.17 Math.fround (x)

When **Math.fround** is called with argument *x*, the following steps are taken:

1. If *x* is **NaN**, return **NaN**.
2. If *x* is one of **+0**, **-0**, $+\infty$, $-\infty$, return *x*.
3. Let *x32* be the result of converting *x* to a value in IEEE 754-2008 binary32 format using roundTiesToEven.
4. Let *x64* be the result of converting *x32* to a value in IEEE 754-2008 binary64 format.
5. Return the ECMAScript Number value corresponding to *x64*.

20.2.2.18 Math.hypot (*value1*, *value2*, ...*values*)

Math.hypot returns an implementation-dependent approximation of the square root of the sum of squares of its arguments.

If no arguments are passed, the result is **+0**.

If any argument is $+\infty$, the result is $+\infty$.

If any argument is $-\infty$, the result is $+\infty$.

If no argument is $+\infty$ or $-\infty$, and any argument is **NaN**, the result is **NaN**.

If all arguments are either **+0** or **-0**, the result is **+0**.

NOTE

Implementations should take care to avoid the loss of precision from overflows and underflows that are prone to occur in naive implementations when this function is called with two or more arguments.

20.2.2.19 Math.imul (*x*, *y*)

When **Math.imul** is called with arguments *x* and *y*, the following steps are taken:

1. Let *a* be ? **ToUint32**(*x*).
2. Let *b* be ? **ToUint32**(*y*).
3. Let *product* be (*a* × *b*) modulo 2^{32} .
4. If *product* ≥ 2^{31} , return *product* - 2^{32} ; otherwise return *product*.

20.2.2.20 Math.log (*x*)

Returns an implementation-dependent approximation to the natural logarithm of *x*.

If *x* is **NaN**, the result is **NaN**.

If *x* is less than 0, the result is **NaN**.

If *x* is **+0** or **-0**, the result is $-\infty$.

If *x* is 1, the result is **+0**.

If *x* is $+\infty$, the result is $+\infty$.

20.2.2.21 Math.log1p (*x*)

Returns an implementation-dependent approximation to the natural logarithm of 1 + *x*. The result is computed in a way that is accurate even when the value of *x* is close to zero.

If *x* is **NaN**, the result is **NaN**.

If *x* is less than -1, the result is **NaN**.

If x is -1 , the result is $-\infty$.
If x is $+0$, the result is $+0$.
If x is -0 , the result is -0 .
If x is $+\infty$, the result is $+\infty$.

20.2.2.22 **Math.log10** (x)

Returns an implementation-dependent approximation to the base 10 logarithm of x .

If x is **NaN**, the result is **NaN**.
If x is less than 0, the result is **NaN**.
If x is $+0$, the result is $-\infty$.
If x is -0 , the result is $-\infty$.
If x is 1, the result is $+0$.
If x is $+\infty$, the result is $+\infty$.

20.2.2.23 **Math.log2** (x)

Returns an implementation-dependent approximation to the base 2 logarithm of x .

If x is **NaN**, the result is **NaN**.
If x is less than 0, the result is **NaN**.
If x is $+0$, the result is $-\infty$.
If x is -0 , the result is $-\infty$.
If x is 1, the result is $+0$.
If x is $+\infty$, the result is $+\infty$.

20.2.2.24 **Math.max** ($value1$, $value2$, ... $values$)

Given zero or more arguments, calls [ToNumber](#) on each of the arguments and returns the largest of the resulting values.

If no arguments are given, the result is $-\infty$.
If any value is **NaN**, the result is **NaN**.
The comparison of values to determine the largest value is done using the [Abstract Relational Comparison](#) algorithm except that $+0$ is considered to be larger than -0 .

20.2.2.25 **Math.min** ($value1$, $value2$, ... $values$)

Given zero or more arguments, calls [ToNumber](#) on each of the arguments and returns the smallest of the resulting values.

If no arguments are given, the result is $+\infty$.
If any value is **NaN**, the result is **NaN**.
The comparison of values to determine the smallest value is done using the [Abstract Relational Comparison](#) algorithm except that $+0$ is considered to be larger than -0 .

20.2.2.26 **Math.pow** ($base$, $exponent$)

1. Return the result of [Applying the ****** operator](#) with $base$ and $exponent$ as specified in 12.6.4.

20.2.2.27 **Math.random** ()

Returns a Number value with positive sign, greater than or equal to 0 but less than 1, chosen randomly or pseudo randomly with approximately uniform distribution over that range, using an implementation-dependent algorithm or strategy. This function takes no arguments.

Each **Math.random** function created for distinct realms must produce a distinct sequence of values from successive calls.

20.2.2.28 Math.round (*x*)

Returns the Number value that is closest to *x* and is equal to a mathematical integer. If two integer Number values are equally close to *x*, then the result is the Number value that is closer to $+\infty$. If *x* is already an integer, the result is *x*.

If *x* is NaN, the result is NaN.

If *x* is +0, the result is +0.

If *x* is -0, the result is -0.

If *x* is $+\infty$, the result is $+\infty$.

If *x* is $-\infty$, the result is $-\infty$.

If *x* is greater than 0 but less than 0.5, the result is +0.

If *x* is less than 0 but greater than or equal to -0.5, the result is -0.

NOTE 1

Math.round(3.5) returns 4, but **Math.round(-3.5)** returns -3.

NOTE 2

The value of **Math.round(x)** is not always the same as the value of **Math.floor(x + 0.5)**. When *x* is -0 or is less than 0 but greater than or equal to -0.5, **Math.round(x)** returns -0, but **Math.floor(x + 0.5)** returns +0. **Math.round(x)** may also differ from the value of **Math.floor(x + 0.5)** because of internal rounding when computing *x* + 0.5.

20.2.2.29 Math.sign (*x*)

Returns the sign of *x*, indicating whether *x* is positive, negative, or zero.

If *x* is NaN, the result is NaN.

If *x* is -0, the result is -0.

If *x* is +0, the result is +0.

If *x* is negative and not -0, the result is -1.

If *x* is positive and not +0, the result is +1.

20.2.2.30 Math.sin (*x*)

Returns an implementation-dependent approximation to the sine of *x*. The argument is expressed in radians.

If *x* is NaN, the result is NaN.

If *x* is +0, the result is +0.

If *x* is -0, the result is -0.

If *x* is $+\infty$ or $-\infty$, the result is NaN.

20.2.2.31 Math.sinh (*x*)

Returns an implementation-dependent approximation to the hyperbolic sine of x .

If x is **NaN**, the result is **NaN**.

If x is **+0**, the result is **+0**.

If x is **-0**, the result is **-0**.

If x is $+\infty$, the result is $+\infty$.

If x is $-\infty$, the result is $-\infty$.

NOTE

The value of $\sinh(x)$ is the same as $(\exp(x) - \exp(-x)) / 2$.

20.2.2.32 **Math.sqrt** (x)

Returns an implementation-dependent approximation to the square root of x .

If x is **NaN**, the result is **NaN**.

If x is less than 0, the result is **NaN**.

If x is **+0**, the result is **+0**.

If x is **-0**, the result is **-0**.

If x is $+\infty$, the result is $+\infty$.

20.2.2.33 **Math.tan** (x)

Returns an implementation-dependent approximation to the tangent of x . The argument is expressed in radians.

If x is **NaN**, the result is **NaN**.

If x is **+0**, the result is **+0**.

If x is **-0**, the result is **-0**.

If x is $+\infty$ or $-\infty$, the result is **NaN**.

20.2.2.34 **Math.tanh** (x)

Returns an implementation-dependent approximation to the hyperbolic tangent of x .

If x is **NaN**, the result is **NaN**.

If x is **+0**, the result is **+0**.

If x is **-0**, the result is **-0**.

If x is $+\infty$, the result is $+1$.

If x is $-\infty$, the result is -1 .

NOTE

The value of $\tanh(x)$ is the same as $(\exp(x) - \exp(-x)) / (\exp(x) + \exp(-x))$.

20.2.2.35 **Math.trunc** (x)

Returns the integral part of the number x , removing any fractional digits. If x is already an integer, the result is x .

If x is **NaN**, the result is **NaN**.

If x is **-0**, the result is **-0**.

If x is $+0$, the result is $+0$.
If x is $+\infty$, the result is $+\infty$.
If x is $-\infty$, the result is $-\infty$.
If x is greater than 0 but less than 1, the result is $+0$.
If x is less than 0 but greater than -1, the result is -0 .

20.3 Date Objects

20.3.1 Overview of Date Objects and Definitions of Abstract Operations

The following functions are [abstract operations](#) that operate on time values (defined in [20.3.1.1](#)). Note that, in every case, if any argument to one of these functions is **NaN**, the result will be **NaN**.

20.3.1.1 Time Values and Time Range

A Date object contains a Number representing an instant in time with millisecond precision. Such a Number is called a time value. A time value may also be **NaN**, indicating that the Date object does not represent a specific instant in time.

Time is measured in ECMAScript as milliseconds since midnight at the beginning of 01 January, 1970 UTC. Time in ECMAScript does not observe leap seconds; they are ignored. Time calculations assume each and every day contains exactly $60 \times 60 \times 24 \times 1000 = 86,400,000$ milliseconds, to align with the POSIX specification of each and every day containing exactly 86,400 seconds.

A Number can exactly represent all integers from -9,007,199,254,740,992 to 9,007,199,254,740,992 ([20.1.2.8](#) and [20.1.2.6](#)). A time value supports a slightly smaller range of exactly -100,000,000 days to 100,000,000 days measured relative to midnight at the beginning of 01 January, 1970 UTC. This yields an exact supported time value range of -8,640,000,000,000,000 to 8,640,000,000,000,000 milliseconds relative to midnight at the beginning of 01 January, 1970 UTC.

The exact moment of midnight at the beginning of 01 January, 1970 UTC is represented by the time value **+0**.

NOTE

The 400 year cycle of the Gregorian calendar contains 97 leap years. This yields an average of 365.2425 days per year, or an average of 31,556,952,000 milliseconds per year under the Gregorian calendar. ECMAScript applies a proleptic Gregorian calendar for all time computations.

As specified by this section, the maximum year range a Number can represent exactly with millisecond precision is approximately -285,426 to 285,426 years relative to midnight at the beginning of 01 January, 1970 UTC.

As specified by this section, the maximum year range a time value can represent is approximately -273,790 to 273,790 years relative to midnight at the beginning of 01 January, 1970 UTC.

20.3.1.2 Day Number and Time within Day

A given [time value](#) t belongs to day number

$$\text{Day}(t) = \text{floor}(t / \text{msPerDay})$$

where the number of milliseconds per day is

$$\text{msPerDay} = 86400000$$

The remainder is called the time within the day:

$$\text{TimeWithinDay}(t) = t \bmod \text{msPerDay}$$

20.3.1.3 Year Number

ECMAScript uses a proleptic Gregorian calendar to map a day number to a year number and to determine the month and date within that year. In this calendar, leap years are precisely those which are (divisible by 4) and ((not divisible by 100) or (divisible by 400)). The number of days in year number y is therefore defined by

$$\begin{aligned} \text{DaysInYear}(y) &= 365 \text{ if } (y \bmod 4) \neq 0 \\ &= 366 \text{ if } (y \bmod 4) = 0 \text{ and } (y \bmod 100) \neq 0 \\ &= 365 \text{ if } (y \bmod 100) = 0 \text{ and } (y \bmod 400) \neq 0 \\ &= 366 \text{ if } (y \bmod 400) = 0 \end{aligned}$$

All non-leap years have 365 days with the usual number of days per month and leap years have an extra day in February. The day number of the first day of year y is given by:

$$\text{DayFromYear}(y) = 365 \times (y - 1970) + \text{floor}((y - 1969) / 4) - \text{floor}((y - 1901) / 100) + \text{floor}((y - 1601) / 400)$$

The **time value** of the start of a year is:

$$\text{TimeFromYear}(y) = \text{msPerDay} \times \text{DayFromYear}(y)$$

A **time value** determines a year by:

$$\text{YearFromTime}(t) = \text{the largest integer } y \text{ (closest to positive infinity) such that } \text{TimeFromYear}(y) \leq t$$

The leap-year function is 1 for a time within a leap year and otherwise is zero:

$$\begin{aligned} \text{InLeapYear}(t) &= 0 \text{ if } \text{DaysInYear}(\text{YearFromTime}(t)) = 365 \\ &= 1 \text{ if } \text{DaysInYear}(\text{YearFromTime}(t)) = 366 \end{aligned}$$

20.3.1.4 Month Number

Months are identified by an integer in the range 0 to 11, inclusive. The mapping **MonthFromTime**(t) from a **time value** t to a month number is defined by:

$$\begin{aligned} \text{MonthFromTime}(t) &= 0 \text{ if } 0 \leq \text{DayWithinYear}(t) < 31 \\ &= 1 \text{ if } 31 \leq \text{DayWithinYear}(t) < 59 + \text{InLeapYear}(t) \\ &= 2 \text{ if } 59 + \text{InLeapYear}(t) \leq \text{DayWithinYear}(t) < 90 + \text{InLeapYear}(t) \\ &= 3 \text{ if } 90 + \text{InLeapYear}(t) \leq \text{DayWithinYear}(t) < 120 + \text{InLeapYear}(t) \\ &= 4 \text{ if } 120 + \text{InLeapYear}(t) \leq \text{DayWithinYear}(t) < 151 + \text{InLeapYear}(t) \\ &= 5 \text{ if } 151 + \text{InLeapYear}(t) \leq \text{DayWithinYear}(t) < 181 + \text{InLeapYear}(t) \\ &= 6 \text{ if } 181 + \text{InLeapYear}(t) \leq \text{DayWithinYear}(t) < 212 + \text{InLeapYear}(t) \\ &= 7 \text{ if } 212 + \text{InLeapYear}(t) \leq \text{DayWithinYear}(t) < 243 + \text{InLeapYear}(t) \\ &= 8 \text{ if } 243 + \text{InLeapYear}(t) \leq \text{DayWithinYear}(t) < 273 + \text{InLeapYear}(t) \end{aligned}$$

$= 9$ if $273 + \text{InLeapYear}(t) \leq \text{DayWithinYear}(t) < 304 + \text{InLeapYear}(t)$
 $= 10$ if $304 + \text{InLeapYear}(t) \leq \text{DayWithinYear}(t) < 334 + \text{InLeapYear}(t)$
 $= 11$ if $334 + \text{InLeapYear}(t) \leq \text{DayWithinYear}(t) < 365 + \text{InLeapYear}(t)$

where

$$\text{DayWithinYear}(t) = \text{Day}(t) - \text{DayFromYear}(\text{YearFromTime}(t))$$

A month value of 0 specifies January; 1 specifies February; 2 specifies March; 3 specifies April; 4 specifies May; 5 specifies June; 6 specifies July; 7 specifies August; 8 specifies September; 9 specifies October; 10 specifies November; and 11 specifies December. Note that $\text{MonthFromTime}(0) = 0$, corresponding to Thursday, 01 January, 1970.

20.3.1.5 Date Number

A date number is identified by an integer in the range 1 through 31, inclusive. The mapping $\text{DateFromTime}(t)$ from a time value t to a date number is defined by:

$$\begin{aligned}
 \text{DateFromTime}(t) &= \text{DayWithinYear}(t) + 1 \text{ if } \text{MonthFromTime}(t) = 0 \\
 &= \text{DayWithinYear}(t) - 30 \text{ if } \text{MonthFromTime}(t) = 1 \\
 &= \text{DayWithinYear}(t) - 58 - \text{InLeapYear}(t) \text{ if } \text{MonthFromTime}(t) = 2 \\
 &= \text{DayWithinYear}(t) - 89 - \text{InLeapYear}(t) \text{ if } \text{MonthFromTime}(t) = 3 \\
 &= \text{DayWithinYear}(t) - 119 - \text{InLeapYear}(t) \text{ if } \text{MonthFromTime}(t) = 4 \\
 &= \text{DayWithinYear}(t) - 150 - \text{InLeapYear}(t) \text{ if } \text{MonthFromTime}(t) = 5 \\
 &= \text{DayWithinYear}(t) - 180 - \text{InLeapYear}(t) \text{ if } \text{MonthFromTime}(t) = 6 \\
 &= \text{DayWithinYear}(t) - 211 - \text{InLeapYear}(t) \text{ if } \text{MonthFromTime}(t) = 7 \\
 &= \text{DayWithinYear}(t) - 242 - \text{InLeapYear}(t) \text{ if } \text{MonthFromTime}(t) = 8 \\
 &= \text{DayWithinYear}(t) - 272 - \text{InLeapYear}(t) \text{ if } \text{MonthFromTime}(t) = 9 \\
 &= \text{DayWithinYear}(t) - 303 - \text{InLeapYear}(t) \text{ if } \text{MonthFromTime}(t) = 10 \\
 &= \text{DayWithinYear}(t) - 333 - \text{InLeapYear}(t) \text{ if } \text{MonthFromTime}(t) = 11
 \end{aligned}$$

20.3.1.6 Week Day

The weekday for a particular time value t is defined as

$$\text{WeekDay}(t) = (\text{Day}(t) + 4) \text{ modulo } 7$$

A weekday value of 0 specifies Sunday; 1 specifies Monday; 2 specifies Tuesday; 3 specifies Wednesday; 4 specifies Thursday; 5 specifies Friday; and 6 specifies Saturday. Note that $\text{WeekDay}(0) = 4$, corresponding to Thursday, 01 January, 1970.

20.3.1.7 LocalTZA (t , isUTC)

$\text{LocalTZA}(t, \text{isUTC})$ is an implementation-defined algorithm that must return a number representing milliseconds suitable for adding to a Time Value. The local political rules for standard time and daylight saving time in effect at t should be used to determine the result in the way specified in the following three paragraphs.

When isUTC is true, $\text{LocalTZA}(t, \text{true})$ should return the offset of the local time zone from UTC measured in milliseconds at time represented by time value t (UTC). When the result is added to t (UTC), it should yield the local time.

When *isUTC* is false, LocalTZA(*t*, false) should return the offset of the local time zone from UTC measured in milliseconds at local time represented by time value $t_{\text{local}} = t$. When the result is subtracted from the local time t_{local} , it should yield the corresponding UTC.

When t_{local} represents local time repeating multiple times at a negative time zone transition (e.g. when the daylight saving time ends or the time zone adjustment is decreased due to a time zone rule change) or skipped local time at a positive time zone transitions (e.g. when the daylight saving time starts or the time zone adjustment is increased due to a time zone rule change), t_{local} must be interpreted with the time zone adjustment before the transition.

If an implementation does not support a conversion described above or if political rules for time *t* are not available within the implementation, the result must be 0.

NOTE

It is recommended that implementations use the time zone information of the IANA Time Zone Database <https://www.iana.org/time-zones/>.

1:30 AM on November 5, 2017 in America/New_York is repeated twice (fall backward), but it must be interpreted as 1:30 AM UTC-04 instead of 1:30 AM UTC-05. LocalTZA(TimeClip(MakeDate(MakeDay(2017, 10, 5), MakeTime(1, 30, 0, 0))), false) is $-4 \times \text{msPerHour}$.

2:30 AM on March 12, 2017 in America/New_York does not exist, but it must be interpreted as 2:30 AM UTC-05 (equivalent to 3:30 AM UTC-04). LocalTZA(TimeClip(MakeDate(MakeDay(2017, 2, 12), MakeTime(2, 30, 0, 0))), false) is $-5 \times \text{msPerHour}$.

20.3.1.8 LocalTime (*t*)

The abstract operation LocalTime with argument *t* converts *t* from UTC to local time by performing the following steps:

1. Return $t + \text{LocalTZA}(t, \text{true})$.

NOTE

Two different time values (*t* (UTC)) are converted to the same local time t_{local} at a negative time zone transition when there are repeated times (e.g. the daylight saving time ends or the time zone adjustment is decreased.).

20.3.1.9 UTC (*t*)

The abstract operation UTC with argument *t* converts *t* from local time to UTC. It performs the following steps:

1. Return $t - \text{LocalTZA}(t, \text{false})$.

NOTE

UTC(LocalTime(*t*)) is not necessarily always equal to *t*. LocalTime(UTC(t_{local})) is not necessarily always equal to t_{local} , either.

20.3.1.10 Hours, Minutes, Second, and Milliseconds

The following abstract operations are useful in decomposing time values:

HourFromTime(*t*) = floor($t / \text{msPerHour}$) modulo HoursPerDay

$\text{MinFromTime}(t) = \text{floor}(t / \text{msPerMinute}) \bmod \text{MinutesPerHour}$
 $\text{SecFromTime}(t) = \text{floor}(t / \text{msPerSecond}) \bmod \text{SecondsPerMinute}$
 $\text{msFromTime}(t) = t \bmod \text{msPerSecond}$

where

$\text{HoursPerDay} = 24$
 $\text{MinutesPerHour} = 60$
 $\text{SecondsPerMinute} = 60$
 $\text{msPerSecond} = 1000$
 $\text{msPerMinute} = 60000 = \text{msPerSecond} \times \text{SecondsPerMinute}$
 $\text{msPerHour} = 3600000 = \text{msPerMinute} \times \text{MinutesPerHour}$

20.3.1.11 MakeTime (*hour*, *min*, *sec*, *ms*)

The abstract operation MakeTime calculates a number of milliseconds from its four arguments, which must be ECMAScript Number values. This operator functions as follows:

1. If *hour* is not finite or *min* is not finite or *sec* is not finite or *ms* is not finite, return **NaN**.
2. Let *h* be ! **ToInteger**(*hour*).
3. Let *m* be ! **ToInteger**(*min*).
4. Let *s* be ! **ToInteger**(*sec*).
5. Let *milli* be ! **ToInteger**(*ms*).
6. Let *t* be $h * \text{msPerHour} + m * \text{msPerMinute} + s * \text{msPerSecond} + \text{milli}$, performing the arithmetic according to IEEE 754-2008 rules (that is, as if using the ECMAScript operators ***** and **+**).
7. Return *t*.

20.3.1.12 MakeDay (*year*, *month*, *date*)

The abstract operation MakeDay calculates a number of days from its three arguments, which must be ECMAScript Number values. This operator functions as follows:

1. If *year* is not finite or *month* is not finite or *date* is not finite, return **NaN**.
2. Let *y* be ! **ToInteger**(*year*).
3. Let *m* be ! **ToInteger**(*month*).
4. Let *dt* be ! **ToInteger**(*date*).
5. Let *ym* be $y + \text{floor}(m / 12)$.
6. Let *mn* be $m \bmod 12$.
7. Find a value *t* such that **YearFromTime**(*t*) is *ym* and **MonthFromTime**(*t*) is *mn* and **DateFromTime**(*t*) is 1; but if this is not possible (because some argument is out of range), return **NaN**.
8. Return **Day**(*t*) + *dt* - 1.

20.3.1.13 MakeDate (*day*, *time*)

The abstract operation MakeDate calculates a number of milliseconds from its two arguments, which must be ECMAScript Number values. This operator functions as follows:

1. If *day* is not finite or *time* is not finite, return **NaN**.
2. Return $\text{day} \times \text{msPerDay} + \text{time}$.

20.3.1.14 TimeClip (*time*)

The abstract operation TimeClip calculates a number of milliseconds from its argument, which must be an ECMAScript Number value. This operator functions as follows:

1. If *time* is not finite, return **NaN**.
2. If $\text{abs}(\text{time}) > 8.64 \times 10^{15}$, return **NaN**.
3. Let *clippedTime* be ! **ToInteger**(*time*).
4. If *clippedTime* is **-0**, set *clippedTime* to **+0**.
5. Return *clippedTime*.

NOTE

The point of step 4 is that an implementation is permitted a choice of internal representations of time values, for example as a 64-bit signed integer or as a 64-bit floating-point value. Depending on the implementation, this internal representation may or may not distinguish **-0** and **+0**.

20.3.1.15 Date Time String Format

ECMAScript defines a string interchange format for date-times based upon a simplification of the ISO 8601 calendar date extended format. The format is as follows: **YYYY-MM-DDTHH:mm:ss.sssZ**

Where the fields are as follows:

YYYY	is the decimal digits of the year 0000 to 9999 in the proleptic Gregorian calendar.
-	"-" (hyphen) appears literally twice in the string.
MM	is the month of the year from 01 (January) to 12 (December).
DD	is the day of the month from 01 to 31.
T	"T" appears literally in the string, to indicate the beginning of the time element.
HH	is the number of complete hours that have passed since midnight as two decimal digits from 00 to 24.
:	":" (colon) appears literally twice in the string.
mm	is the number of complete minutes since the start of the hour as two decimal digits from 00 to 59.
ss	is the number of complete seconds since the start of the minute as two decimal digits from 00 to 59.
.	"." (dot) appears literally in the string.
sss	is the number of complete milliseconds since the start of the second as three decimal digits.
Z	is the time zone offset specified as " Z " (for UTC) or either "+" or "-" followed by a time expression
HH:mm	

This format includes date-only forms:

YYYY
YYYY-MM
YYYY-MM-DD

It also includes "date-time" forms that consist of one of the above date-only forms immediately followed by one of the following time forms with an optional time zone offset appended:

THH:mm

THH:mm:ss

THH:mm:ss.sss

All numbers must be base 10. If the **MM** or **DD** fields are absent "01" is used as the value. If the **HH**, **mm**, or **ss** fields are absent "00" is used as the value and the value of an absent **sss** field is "000". When the time zone offset is absent, date-only forms are interpreted as a UTC time and date-time forms are interpreted as a local time.

A string containing out-of-bounds or nonconforming fields is not a valid instance of this format.

NOTE 1

As every day both starts and ends with midnight, the two notations 00:00 and 24:00 are available to distinguish the two midnights that can be associated with one date. This means that the following two notations refer to exactly the same point in time: 1995-02-04T24:00 and 1995-02-05T00:00. This interpretation of the latter form as "end of a calendar day" is consistent with ISO 8601, even though that specification reserves it for describing time intervals and does not permit it within representations of single points in time.

NOTE 2

There exists no international standard that specifies abbreviations for civil time zones like CET, EST, etc. and sometimes the same abbreviation is even used for two very different time zones. For this reason, both ISO 8601 and this format specify numeric representations of time zone offsets.

20.3.1.15.1 Expanded Years

Covering the full [time value](#) range of approximately 273,790 years forward or backward from 01 January, 1970 (20.3.1.1) requires representing years before 0 or after 9999. ISO 8601 permits expansion of the year representation, but only by mutual agreement of the partners in information interchange. In the simplified ECMAScript format, such an expanded year representation shall have 6 digits and is always prefixed with a + or - sign. The year 0 is considered positive and hence prefixed with a + sign. Strings matching the [Date Time String Format](#) with expanded years representing instants in time outside the range of a [time value](#) are treated as unrecognizable by [Date.parse](#) and cause that function to return **NaN** without falling back to implementation-specific behavior or heuristics.

NOTE

Examples of date-time values with expanded years:

-271821-04-20T00:00:00Z	271822 B.C.
-000001-01-01T00:00:00Z	2 B.C.
+000000-01-01T00:00:00Z	1 B.C.
+000001-01-01T00:00:00Z	1 A.D.
+001970-01-01T00:00:00Z	1970 A.D.
+002009-12-15T00:00:00Z	2009 A.D.
+275760-09-13T00:00:00Z	275760 A.D.

20.3.2 The Date Constructor

The Date [constructor](#):

is the intrinsic object %Date%.

is the initial value of the **Date** property of the [global object](#).

creates and initializes a new Date object when called as a [constructor](#).

returns a String representing the current time (UTC) when called as a function rather than as a [constructor](#).

is a single function whose behaviour is overloaded based upon the number and types of its arguments.

is designed to be subclassable. It may be used as the value of an **extends** clause of a class definition. Subclass

constructors that intend to inherit the specified **Date** behaviour must include a **super** call to the **Date** [constructor](#) to create and initialize the subclass instance with a [[DateValue]] internal slot.

has a "**length**" property whose value is 7.

20.3.2.1 Date (*year*, *month* [, *date* [, *hours* [, *minutes* [, *seconds* [, *ms*]]]]])

This description applies only if the Date [constructor](#) is called with at least two arguments.

When the **Date** function is called, the following steps are taken:

1. Let *numberOfArgs* be the number of arguments passed to this function call.
2. **Assert:** *numberOfArgs* ≥ 2.
3. If NewTarget is **undefined**, then
 - a. Let *now* be the Number that is the [time value](#) (UTC) identifying the current time.
 - b. Return [ToString\(now\)](#).
4. Else,
 - a. Let *y* be ? [ToNumber\(year\)](#).
 - b. Let *m* be ? [ToNumber\(month\)](#).
 - c. If *date* is present, let *dt* be ? [ToNumber\(date\)](#); else let *dt* be 1.
 - d. If *hours* is present, let *h* be ? [ToNumber\(hours\)](#); else let *h* be 0.
 - e. If *minutes* is present, let *min* be ? [ToNumber\(minutes\)](#); else let *min* be 0.
 - f. If *seconds* is present, let *s* be ? [ToNumber\(seconds\)](#); else let *s* be 0.
 - g. If *ms* is present, let *milli* be ? [ToNumber\(ms\)](#); else let *milli* be 0.
 - h. If *y* is NaN, let *yr* be NaN.
 - i. Else,
 - i. Let *yi* be ! [ToInteger\(y\)](#).
 - ii. If 0 ≤ *yi* ≤ 99, let *yr* be 1900 + *yi*; otherwise, let *yr* be *y*.
 - j. Let *finalDate* be [MakeDate\(MakeDay\(yr, m, dt\), MakeTime\(h, min, s, milli\)\)](#).
 - k. Let *O* be ? [OrdinaryCreateFromConstructor\(NewTarget, "%DatePrototype%", « \[\[DateValue\]\] »\)](#).
 - l. Set *O*.[[DateValue]] to [TimeClip\(UTC\(finalDate\)\)](#).
 - m. Return *O*.

20.3.2.2 Date (*value*)

This description applies only if the Date [constructor](#) is called with exactly one argument.

When the **Date** function is called, the following steps are taken:

1. Let *numberOfArgs* be the number of arguments passed to this function call.
2. **Assert:** *numberOfArgs* = 1.
3. If NewTarget is **undefined**, then
 - a. Let *now* be the Number that is the [time value](#) (UTC) identifying the current time.
 - b. Return [ToString\(now\)](#).
4. Else,

- a. If **Type**(*value*) is Object and *value* has a **[[DateValue]]** internal slot, then
 - i. Let *tv* be **thisTimeValue**(*value*).
- b. Else,
 - i. Let *v* be ? **ToPrimitive**(*value*).
 - ii. If **Type**(*v*) is String, then
 1. **Assert**: The next step never returns an **abrupt completion** because **Type**(*v*) is String.
 2. Let *tv* be the result of parsing *v* as a date, in exactly the same manner as for the **parse** method (20.3.3.2).
 - iii. Else,
 1. Let *tv* be ? **ToNumber**(*v*).
- c. Let *O* be ? **OrdinaryCreateFromConstructor**(NewTarget, "%DatePrototype%", « **[[DateValue]]** »).
- d. Set *O*.**[[DateValue]]** to **TimeClip**(*tv*).
- e. Return *O*.

20.3.2.3 Date ()

This description applies only if the Date **constructor** is called with no arguments.

When the **Date** function is called, the following steps are taken:

1. Let *numberOfArgs* be the number of arguments passed to this function call.
2. **Assert**: *numberOfArgs* = 0.
3. If NewTarget is **undefined**, then
 - a. Let *now* be the Number that is the **time value** (UTC) identifying the current time.
 - b. Return **ToDateString**(*now*).
4. Else,
 - a. Let *O* be ? **OrdinaryCreateFromConstructor**(NewTarget, "%DatePrototype%", « **[[DateValue]]** »).
 - b. Set *O*.**[[DateValue]]** to the **time value** (UTC) identifying the current time.
 - c. Return *O*.

20.3.3 Properties of the Date Constructor

The Date **constructor**:

has a **[[Prototype]]** internal slot whose value is the intrinsic object **%FunctionPrototype%**.

has the following properties:

20.3.3.1 Date.now ()

The **now** function returns a Number value that is the **time value** designating the UTC date and time of the occurrence of the call to **now**.

20.3.3.2 Date.parse (*string*)

The **parse** function applies the **ToString** operator to its argument. If **ToString** results in an **abrupt completion** the **Completion Record** is immediately returned. Otherwise, **parse** interprets the resulting String as a date and time; it returns a Number, the UTC **time value** corresponding to the date and time. The String may be interpreted as a local time, a UTC time, or a time in some other time zone, depending on the contents of the String. The function first attempts to parse the String according to the format described in Date Time String Format (20.3.1.15), including expanded years. If

the String does not conform to that format the function may fall back to any implementation-specific heuristics or implementation-specific date formats. Strings that are unrecognizable or contain out-of-bounds format field values shall cause **Date.parse** to return **NaN**.

If **x** is any Date object whose milliseconds amount is zero within a particular implementation of ECMAScript, then all of the following expressions should produce the same numeric value in that implementation, if all the properties referenced have their initial values:

```
x.valueOf()  
Date.parse(x.toString())  
Date.parse(x.toUTCString())  
Date.parse(x.toISOString())
```

However, the expression

```
Date.parse(x.toLocaleString())
```

is not required to produce the same Number value as the preceding three expressions and, in general, the value produced by **Date.parse** is implementation-dependent when given any String value that does not conform to the Date Time String Format (20.3.1.15) and that could not be produced in that implementation by the **toString** or **toUTCString** method.

20.3.3.3 Date.prototype

The initial value of **Date.prototype** is the intrinsic object **%DatePrototype%**.

This property has the attributes { **[[Writable]]**: **false**, **[[Enumerable]]**: **false**, **[[Configurable]]**: **false** }.

20.3.3.4 Date.UTC (year [, month [, date [, hours [, minutes [, seconds [, ms]]]]]])

When the **UTC** function is called, the following steps are taken:

1. Let **y** be ? **ToNumber**(**year**).
2. If **month** is present, let **m** be ? **ToNumber**(**month**); else let **m** be 0.
3. If **date** is present, let **dt** be ? **ToNumber**(**date**); else let **dt** be 1.
4. If **hours** is present, let **h** be ? **ToNumber**(**hours**); else let **h** be 0.
5. If **minutes** is present, let **min** be ? **ToNumber**(**minutes**); else let **min** be 0.
6. If **seconds** is present, let **s** be ? **ToNumber**(**seconds**); else let **s** be 0.
7. If **ms** is present, let **milli** be ? **ToNumber**(**ms**); else let **milli** be 0.
8. If **y** is **NaN**, let **yr** be **NaN**.
9. Else,
 - a. Let **yi** be ! **ToInteger**(**y**).
 - b. If $0 \leq yi \leq 99$, let **yr** be 1900 + **yi**; otherwise, let **yr** be **y**.
10. Return **TimeClip**(**MakeDate**(**MakeDay**(**yr**, **m**, **dt**), **MakeTime**(**h**, **min**, **s**, **milli**))).

The "**length**" property of the **UTC** function is 7.

NOTE

The **UTC** function differs from the **Date constructor** in two ways: it returns a **time value** as a Number, rather than creating a Date object, and it interprets the arguments in UTC rather than as local time.

20.3.4 Properties of the Date Prototype Object

The Date prototype object:

- is the intrinsic object `%DatePrototype%`.
- is itself an ordinary object.
- is not a Date instance and does not have a `[[DateValue]]` internal slot.
- has a `[[Prototype]]` internal slot whose value is the intrinsic object `%ObjectPrototype%`.

Unless explicitly defined otherwise, the methods of the Date prototype object defined below are not generic and the **this** value passed to them must be an object that has a `[[DateValue]]` internal slot that has been initialized to a **time value**.

The abstract operation `thisTimeValue(value)` performs the following steps:

1. If `Type(value)` is Object and `value` has a `[[DateValue]]` internal slot, then
 - a. Return `value.[[DateValue]]`.
2. Throw a **TypeError** exception.

In following descriptions of functions that are properties of the Date prototype object, the phrase “this Date object” refers to the object that is the **this** value for the invocation of the function. If the Type of the **this** value is not Object, a **TypeError** exception is thrown. The phrase “this time value” within the specification of a method refers to the result returned by calling the abstract operation `thisTimeValue` with the **this** value of the method invocation passed as the argument.

20.3.4.1 Date.prototype.constructor

The initial value of `Date.prototype.constructor` is the intrinsic object `%Date%`.

20.3.4.2 Date.prototype.getDate ()

The following steps are performed:

1. Let `t` be ? `thisTimeValue(this value)`.
2. If `t` is **NaN**, return **NaN**.
3. Return `DateFromTime(LocalTime(t))`.

20.3.4.3 Date.prototype.getDay ()

The following steps are performed:

1. Let `t` be ? `thisTimeValue(this value)`.
2. If `t` is **NaN**, return **NaN**.
3. Return `WeekDay(LocalTime(t))`.

20.3.4.4 Date.prototype.getFullYear ()

The following steps are performed:

1. Let `t` be ? `thisTimeValue(this value)`.
2. If `t` is **NaN**, return **NaN**.
3. Return `YearFromTime(LocalTime(t))`.

20.3.4.5 Date.prototype.getHours ()

The following steps are performed:

1. Let *t* be ? [thisTimeValue](#)(**this** value).
2. If *t* is NaN, return NaN.
3. Return [HourFromTime](#)([LocalTime](#)(*t*)).

20.3.4.6 Date.prototype.getMilliseconds ()

The following steps are performed:

1. Let *t* be ? [thisTimeValue](#)(**this** value).
2. If *t* is NaN, return NaN.
3. Return [msFromTime](#)([LocalTime](#)(*t*)).

20.3.4.7 Date.prototype.getMinutes ()

The following steps are performed:

1. Let *t* be ? [thisTimeValue](#)(**this** value).
2. If *t* is NaN, return NaN.
3. Return [MinFromTime](#)([LocalTime](#)(*t*)).

20.3.4.8 Date.prototype.getMonth ()

The following steps are performed:

1. Let *t* be ? [thisTimeValue](#)(**this** value).
2. If *t* is NaN, return NaN.
3. Return [MonthFromTime](#)([LocalTime](#)(*t*)).

20.3.4.9 Date.prototype.getSeconds ()

The following steps are performed:

1. Let *t* be ? [thisTimeValue](#)(**this** value).
2. If *t* is NaN, return NaN.
3. Return [SecFromTime](#)([LocalTime](#)(*t*)).

20.3.4.10 Date.prototype.getTime ()

The following steps are performed:

1. Return ? [thisTimeValue](#)(**this** value).

20.3.4.11 Date.prototype.getTimezoneOffset ()

The following steps are performed:

1. Let *t* be ? [thisTimeValue](#)(**this** value).
2. If *t* is NaN, return NaN.

3. Return $(t - \text{LocalTime}(t)) / \text{msPerMinute}$.

20.3.4.12 Date.prototype.getUTCDate ()

The following steps are performed:

1. Let t be ? `thisTimeValue`(**this** value).
2. If t is NaN, return NaN.
3. Return `DateFromTime`(t).

20.3.4.13 Date.prototype.getUTCDay ()

The following steps are performed:

1. Let t be ? `thisTimeValue`(**this** value).
2. If t is NaN, return NaN.
3. Return `WeekDay`(t).

20.3.4.14 Date.prototype.getUTCFullYear ()

The following steps are performed:

1. Let t be ? `thisTimeValue`(**this** value).
2. If t is NaN, return NaN.
3. Return `YearFromTime`(t).

20.3.4.15 Date.prototype.getUTCHours ()

The following steps are performed:

1. Let t be ? `thisTimeValue`(**this** value).
2. If t is NaN, return NaN.
3. Return `HourFromTime`(t).

20.3.4.16 Date.prototype.getUTCMilliseconds ()

The following steps are performed:

1. Let t be ? `thisTimeValue`(**this** value).
2. If t is NaN, return NaN.
3. Return `msFromTime`(t).

20.3.4.17 Date.prototype.getUTCMinutes ()

The following steps are performed:

1. Let t be ? `thisTimeValue`(**this** value).
2. If t is NaN, return NaN.
3. Return `MinFromTime`(t).

20.3.4.18 Date.prototype.getUTCMonth ()

The following steps are performed:

1. Let *t* be ? **thisTimeValue**(**this** value).
2. If *t* is **NaN**, return **NaN**.
3. Return **MonthFromTime**(*t*).

20.3.4.19 Date.prototype.getUTCSeconds ()

The following steps are performed:

1. Let *t* be ? **thisTimeValue**(**this** value).
2. If *t* is **NaN**, return **NaN**.
3. Return **SecFromTime**(*t*).

20.3.4.20 Date.prototype.setDate (*date*)

The following steps are performed:

1. Let *t* be **LocalTime**(? **thisTimeValue**(**this** value)).
2. Let *dt* be ? **ToNumber**(*date*).
3. Let *newDate* be **MakeDate**(**MakeDay**(**YearFromTime**(*t*), **MonthFromTime**(*t*), *dt*), **TimeWithinDay**(*t*)).
4. Let *u* be **TimeClip**(**UTC**(*newDate*)).
5. Set the [[DateValue]] internal slot of **this** **Date** object to *u*.
6. Return *u*.

20.3.4.21 Date.prototype.setFullYear (*year* [, *month* [, *date*]])

The following steps are performed:

1. Let *t* be ? **thisTimeValue**(**this** value).
2. If *t* is **NaN**, set *t* to +0; otherwise, set *t* to **LocalTime**(*t*).
3. Let *y* be ? **ToNumber**(*year*).
4. If *month* is not present, let *m* be **MonthFromTime**(*t*); otherwise, let *m* be ? **ToNumber**(*month*).
5. If *date* is not present, let *dt* be **DateFromTime**(*t*); otherwise, let *dt* be ? **ToNumber**(*date*).
6. Let *newDate* be **MakeDate**(**MakeDay**(*y*, *m*, *dt*), **TimeWithinDay**(*t*)).
7. Let *u* be **TimeClip**(**UTC**(*newDate*)).
8. Set the [[DateValue]] internal slot of **this** **Date** object to *u*.
9. Return *u*.

The "length" property of the **setFullYear** method is 3.

NOTE

If *month* is not present, this method behaves as if *month* was present with the value **getMonth** (). If *date* is not present, it behaves as if *date* was present with the value **getDate** () .

20.3.4.22 Date.prototype.setHours (*hour* [, *min* [, *sec* [, *ms*]]])

The following steps are performed:

1. Let *t* be **LocalTime**(? **thisTimeValue**(**this** value)).

2. Let *h* be ? **ToNumber**(*hour*).
3. If *min* is not present, let *m* be **MinFromTime**(*t*); otherwise, let *m* be ? **ToNumber**(*min*).
4. If *sec* is not present, let *s* be **SecFromTime**(*t*); otherwise, let *s* be ? **ToNumber**(*sec*).
5. If *ms* is not present, let *milli* be **msFromTime**(*t*); otherwise, let *milli* be ? **ToNumber**(*ms*).
6. Let *date* be **MakeDate**(**Day**(*t*), **MakeTime**(*h*, *m*, *s*, *milli*)).
7. Let *u* be **TimeClip**(**UTC**(*date*)).
8. Set the [[DateValue]] internal slot of this Date object to *u*.
9. Return *u*.

The "**length**" property of the **setHours** method is 4.

NOTE

If *min* is not present, this method behaves as if *min* was present with the value **getMinutes** (). If *sec* is not present, it behaves as if *sec* was present with the value **getSeconds** (). If *ms* is not present, it behaves as if *ms* was present with the value **getMilliseconds** ().

20.3.4.23 Date.prototype.setMilliseconds (*ms*)

The following steps are performed:

1. Let *t* be **LocalTime**(? **thisTimeValue**(**this** value)).
2. Set *ms* to ? **ToNumber**(*ms*).
3. Let *time* be **MakeTime**(**HourFromTime**(*t*), **MinFromTime**(*t*), **SecFromTime**(*t*), *ms*).
4. Let *u* be **TimeClip**(**UTC**(**MakeDate**(**Day**(*t*), *time*))).
5. Set the [[DateValue]] internal slot of this Date object to *u*.
6. Return *u*.

20.3.4.24 Date.prototype.setMinutes (*min* [, *sec* [, *ms*]])

The following steps are performed:

1. Let *t* be **LocalTime**(? **thisTimeValue**(**this** value)).
2. Let *m* be ? **ToNumber**(*min*).
3. If *sec* is not present, let *s* be **SecFromTime**(*t*); otherwise, let *s* be ? **ToNumber**(*sec*).
4. If *ms* is not present, let *milli* be **msFromTime**(*t*); otherwise, let *milli* be ? **ToNumber**(*ms*).
5. Let *date* be **MakeDate**(**Day**(*t*), **MakeTime**(**HourFromTime**(*t*), *m*, *s*, *milli*)).
6. Let *u* be **TimeClip**(**UTC**(*date*)).
7. Set the [[DateValue]] internal slot of this Date object to *u*.
8. Return *u*.

The "**length**" property of the **setMinutes** method is 3.

NOTE

If *sec* is not present, this method behaves as if *sec* was present with the value **getSeconds** (). If *ms* is not present, this behaves as if *ms* was present with the value **getMilliseconds** ().

20.3.4.25 Date.prototype.setMonth (*month* [, *date*])

The following steps are performed:

1. Let *t* be `LocalTime(? thisTimeValue(this value))`.
2. Let *m* be ? `ToNumber(month)`.
3. If *date* is not present, let *dt* be `DateFromTime(t)`; otherwise, let *dt* be ? `ToNumber(date)`.
4. Let *newDate* be `MakeDate(MakeDay(YearFromTime(t), m, dt), TimeWithinDay(t))`.
5. Let *u* be `TimeClip(UTC(newDate))`.
6. Set the `[[DateValue]]` internal slot of `this Date object` to *u*.
7. Return *u*.

The **"length"** property of the **setMonth** method is 2.

NOTE

If *date* is not present, this method behaves as if *date* was present with the value `getDate()`.

20.3.4.26 Date.prototype.setSeconds (*sec* [, *ms*])

The following steps are performed:

1. Let *t* be `LocalTime(? thisTimeValue(this value))`.
2. Let *s* be ? `ToNumber(sec)`.
3. If *ms* is not present, let *milli* be `msFromTime(t)`; otherwise, let *milli* be ? `ToNumber(ms)`.
4. Let *date* be `MakeDate(Day(t), MakeTime(HourFromTime(t), MinFromTime(t), s, milli))`.
5. Let *u* be `TimeClip(UTC(date))`.
6. Set the `[[DateValue]]` internal slot of `this Date object` to *u*.
7. Return *u*.

The **"length"** property of the **setSeconds** method is 2.

NOTE

If *ms* is not present, this method behaves as if *ms* was present with the value `getMilliseconds()`.

20.3.4.27 Date.prototype.setTime (*time*)

The following steps are performed:

1. Perform ? `thisTimeValue(this value)`.
2. Let *t* be ? `ToNumber(time)`.
3. Let *v* be `TimeClip(t)`.
4. Set the `[[DateValue]]` internal slot of `this Date object` to *v*.
5. Return *v*.

20.3.4.28 Date.prototype.setUTCDate (*date*)

The following steps are performed:

1. Let *t* be ? `thisTimeValue(this value)`.
2. Let *dt* be ? `ToNumber(date)`.
3. Let *newDate* be `MakeDate(MakeDay(YearFromTime(t), MonthFromTime(t), dt), TimeWithinDay(t))`.

- Let *v* be `TimeClip(newDate)`.
- Set the `[[DateValue]]` internal slot of `this Date object` to *v*.
- Return *v*.

20.3.4.29 `Date.prototype.setUTCFullYear ( year [ , month [ , date ] ] )`

The following steps are performed:

- Let *t* be ? `thisTimeValue(this value)`.
- If *t* is `NaN`, set *t* to `+0`.
- Let *y* be ? `ToNumber(year)`.
- If *month* is not present, let *m* be `MonthFromTime(t)`; otherwise, let *m* be ? `ToNumber(month)`.
- If *date* is not present, let *dt* be `DateFromTime(t)`; otherwise, let *dt* be ? `ToNumber(date)`.
- Let *newDate* be `MakeDate(MakeDay(y, m, dt), TimeWithinDay(t))`.
- Let *v* be `TimeClip(newDate)`.
- Set the `[[DateValue]]` internal slot of `this Date object` to *v*.
- Return *v*.

The `"length"` property of the `setUTCFullYear` method is 3.

NOTE

If *month* is not present, this method behaves as if *month* was present with the value `getUTCMonth()`. If *date* is not present, it behaves as if *date* was present with the value `getUTCDate()`.

20.3.4.30 `Date.prototype.setUTCHours ( hour [ , min [ , sec [ , ms ] ] ] )`

The following steps are performed:

- Let *t* be ? `thisTimeValue(this value)`.
- Let *h* be ? `ToNumber(hour)`.
- If *min* is not present, let *m* be `MinFromTime(t)`; otherwise, let *m* be ? `ToNumber(min)`.
- If *sec* is not present, let *s* be `SecFromTime(t)`; otherwise, let *s* be ? `ToNumber(sec)`.
- If *ms* is not present, let *milli* be `msFromTime(t)`; otherwise, let *milli* be ? `ToNumber(ms)`.
- Let *newDate* be `MakeDate(Day(t), MakeTime(h, m, s, milli))`.
- Let *v* be `TimeClip(newDate)`.
- Set the `[[DateValue]]` internal slot of `this Date object` to *v*.
- Return *v*.

The `"length"` property of the `setUTCHours` method is 4.

NOTE

If *min* is not present, this method behaves as if *min* was present with the value `getUTCMinutes()`. If *sec* is not present, it behaves as if *sec* was present with the value `getUTCSeconds()`. If *ms* is not present, it behaves as if *ms* was present with the value `getUTCMilliseconds()`.

20.3.4.31 `Date.prototype.setUTCMilliseconds ( ms )`

The following steps are performed:

1. Let *t* be ? **thisTimeValue**(**this** value).
2. Let *milli* be ? **ToNumber**(*ms*).
3. Let *time* be **MakeTime**(**HourFromTime**(*t*), **MinFromTime**(*t*), **SecFromTime**(*t*), *milli*).
4. Let *v* be **TimeClip**(**MakeDate**(**Day**(*t*), *time*)).
5. Set the [[DateValue]] internal slot of **this** **Date** object to *v*.
6. Return *v*.

20.3.4.32 **Date.prototype.setUTCMinutes** (*min* [, *sec* [, *ms*]])

The following steps are performed:

1. Let *t* be ? **thisTimeValue**(**this** value).
2. Let *m* be ? **ToNumber**(*min*).
3. If *sec* is not present, let *s* be **SecFromTime**(*t*).
4. Else,
 - a. Let *s* be ? **ToNumber**(*sec*).
5. If *ms* is not present, let *milli* be **msFromTime**(*t*).
6. Else,
 - a. Let *milli* be ? **ToNumber**(*ms*).
7. Let *date* be **MakeDate**(**Day**(*t*), **MakeTime**(**HourFromTime**(*t*), *m*, *s*, *milli*)).
8. Let *v* be **TimeClip**(*date*).
9. Set the [[DateValue]] internal slot of **this** **Date** object to *v*.
10. Return *v*.

The "**length**" property of the **setUTCMinutes** method is 3.

NOTE

If *sec* is not present, this method behaves as if *sec* was present with the value **getUTCSeconds** (). If *ms* is not present, it function behaves as if *ms* was present with the value return by **getUTCMilliseconds** () .

20.3.4.33 **Date.prototype.setUTCMonth** (*month* [, *date*])

The following steps are performed:

1. Let *t* be ? **thisTimeValue**(**this** value).
2. Let *m* be ? **ToNumber**(*month*).
3. If *date* is not present, let *dt* be **DateFromTime**(*t*).
4. Else,
 - a. Let *dt* be ? **ToNumber**(*date*).
5. Let *newDate* be **MakeDate**(**MakeDay**(**YearFromTime**(*t*), *m*, *dt*), **TimeWithinDay**(*t*)).
6. Let *v* be **TimeClip**(*newDate*).
7. Set the [[DateValue]] internal slot of **this** **Date** object to *v*.
8. Return *v*.

The "**length**" property of the **setUTCMonth** method is 2.

NOTE

If *date* is not present, this method behaves as if *date* was present with the value **getUTCDate** () .

20.3.4.34 Date.prototype.setUTCSeconds (*sec* [, *ms*])

The following steps are performed:

1. Let *t* be ? [thisTimeValue](#)(**this** value).
2. Let *s* be ? [ToNumber](#)(*sec*).
3. If *ms* is not present, let *milli* be [msFromTime](#)(*t*).
4. Else,
 - a. Let *milli* be ? [ToNumber](#)(*ms*).
5. Let *date* be [MakeDate](#)([Day](#)(*t*), [MakeTime](#)([HourFromTime](#)(*t*), [MinFromTime](#)(*t*), *s*, *milli*)).
6. Let *v* be [TimeClip](#)(*date*).
7. Set the [[DateValue]] internal slot of **this** Date object to *v*.
8. Return *v*.

The "**length**" property of the **setUTCSeconds** method is 2.

NOTE

If *ms* is not present, this method behaves as if *ms* was present with the value [getUTCMilliseconds](#) ().

20.3.4.35 Date.prototype.toString ()

The following steps are performed:

1. Let *O* be **this** Date object.
2. Let *tv* be ? [thisTimeValue](#)(*O*).
3. If *tv* is NaN, return "Invalid Date".
4. Let *t* be [LocalTime](#)(*tv*).
5. Return [DateString](#)(*t*).

20.3.4.36 Date.prototype.toISOString ()

This function returns a String value representing the instance in time corresponding to [this time value](#). The format of the String is the Date Time string format defined in [20.3.1.15](#). All fields are present in the String. The time zone is always UTC, denoted by the suffix Z. If [this time value](#) is not a finite Number or if the year is not a value that can be represented in that format (if necessary using expanded year format), a **RangeError** exception is thrown.

20.3.4.37 Date.prototype.toJSON (*key*)

This function provides a String representation of a Date object for use by [JSON.stringify](#) ([24.5.2](#)).

When the **toJSON** method is called with argument *key*, the following steps are taken:

1. Let *O* be ? [ToObject](#)(**this** value).
2. Let *tv* be ? [ToPrimitive](#)(*O*, hint Number).
3. If [Type](#)(*tv*) is Number and *tv* is not finite, return **null**.
4. Return ? [Invoke](#)(*O*, "**toISOString**").

NOTE 1

The argument is ignored.

NOTE 2

The **toJSON** function is intentionally generic; it does not require that its **this** value be a Date object. Therefore, it can be transferred to other kinds of objects for use as a method. However, it does require that any such object have a **toISOString** method.

20.3.4.38 **Date.prototype.toLocaleDateString** ([*reserved1* [, *reserved2*]])

An ECMAScript implementation that includes the ECMA-402 Internationalization API must implement the **Date.prototype.toLocaleDateString** method as specified in the ECMA-402 specification. If an ECMAScript implementation does not include the ECMA-402 API the following specification of the **toLocaleDateString** method is used.

This function returns a String value. The contents of the String are implementation-dependent, but are intended to represent the “date” portion of the Date in the current time zone in a convenient, human-readable form that corresponds to the conventions of the host environment's current locale.

The meaning of the optional parameters to this method are defined in the ECMA-402 specification; implementations that do not include ECMA-402 support must not use those parameter positions for anything else.

20.3.4.39 **Date.prototype.toLocaleString** ([*reserved1* [, *reserved2*]])

An ECMAScript implementation that includes the ECMA-402 Internationalization API must implement the **Date.prototype.toLocaleString** method as specified in the ECMA-402 specification. If an ECMAScript implementation does not include the ECMA-402 API the following specification of the **toLocaleString** method is used.

This function returns a String value. The contents of the String are implementation-dependent, but are intended to represent the Date in the current time zone in a convenient, human-readable form that corresponds to the conventions of the host environment's current locale.

The meaning of the optional parameters to this method are defined in the ECMA-402 specification; implementations that do not include ECMA-402 support must not use those parameter positions for anything else.

20.3.4.40 **Date.prototype.toLocaleTimeString** ([*reserved1* [, *reserved2*]])

An ECMAScript implementation that includes the ECMA-402 Internationalization API must implement the **Date.prototype.toLocaleTimeString** method as specified in the ECMA-402 specification. If an ECMAScript implementation does not include the ECMA-402 API the following specification of the **toLocaleTimeString** method is used.

This function returns a String value. The contents of the String are implementation-dependent, but are intended to represent the “time” portion of the Date in the current time zone in a convenient, human-readable form that corresponds to the conventions of the host environment's current locale.

The meaning of the optional parameters to this method are defined in the ECMA-402 specification; implementations that do not include ECMA-402 support must not use those parameter positions for anything else.

20.3.4.41 **Date.prototype.toString** ()

The following steps are performed:

1. Let *tv* be ? `thisTimeValue(this value)`.
2. Return `ToDateString(tv)`.

NOTE 1

For any Date object *d* whose milliseconds amount is zero, the result of `Date.parse(d.toString())` is equal to `d.valueOf()`. See 20.3.3.2.

NOTE 2

The `toString` function is not generic; it throws a **TypeError** exception if its **this** value is not a Date object. Therefore, it cannot be transferred to other kinds of objects for use as a method.

20.3.4.41.1 Runtime Semantics: TimeString (*tv*)

The following steps are performed:

1. **Assert:** `Type(tv)` is Number.
2. **Assert:** *tv* is not NaN.
3. Let *hour* be the String representation of `HourFromTime(tv)`, formatted as a two-digit decimal number, padded to the left with a zero if necessary.
4. Let *minute* be the String representation of `MinFromTime(tv)`, formatted as a two-digit decimal number, padded to the left with a zero if necessary.
5. Let *second* be the String representation of `SecFromTime(tv)`, formatted as a two-digit decimal number, padded to the left with a zero if necessary.
6. Return the **string-concatenation** of *hour*, ":", *minute*, ":", *second*, the code unit 0x0020 (SPACE), and "GMT".

20.3.4.41.2 Runtime Semantics: DateString (*tv*)

The following steps are performed:

1. **Assert:** `Type(tv)` is Number.
2. **Assert:** *tv* is not NaN.
3. Let *weekday* be the Name of the entry in Table 49 with the Number `WeekDay(tv)`.
4. Let *month* be the Name of the entry in Table 50 with the Number `MonthFromTime(tv)`.
5. Let *day* be the String representation of `DateFromTime(tv)`, formatted as a two-digit decimal number, padded to the left with a zero if necessary.
6. Let *year* be the String representation of `YearFromTime(tv)`, formatted as a decimal number of at least four digits, padded to the left with zeroes if necessary.
7. Return the **string-concatenation** of *weekday*, the code unit 0x0020 (SPACE), *month*, the code unit 0x0020 (SPACE), *day*, the code unit 0x0020 (SPACE), and *year*.

Table 49: Names of days of the week

Number	Name
0	"Sun"
1	"Mon"
2	"Tue"
3	"Wed"

4	"Thu"
5	"Fri"
6	"Sat"

Table 50: Names of months of the year

Number	Name
0	"Jan"
1	"Feb"
2	"Mar"
3	"Apr"
4	"May"
5	"Jun"
6	"Jul"
7	"Aug"
8	"Sep"
9	"Oct"
10	"Nov"
11	"Dec"

20.3.4.41.3 Runtime Semantics: TimeZoneString (*tv*)

The following steps are performed:

1. **Assert:** `Type(tv)` is Number.
2. **Assert:** *tv* is not NaN.
3. Let *offset* be `LocalTZA(tv, true)`.
4. If *offset* ≥ 0, let *offsetSign* be "+"; otherwise, let *offsetSign* be "-".
5. Let *offsetMin* be the String representation of `MinFromTime(abs(offset))`, formatted as a two-digit decimal number, padded to the left with a zero if necessary.
6. Let *offsetHour* be the String representation of `HourFromTime(abs(offset))`, formatted as a two-digit decimal number, padded to the left with a zero if necessary.
7. Let *tzName* be an implementation-defined string that is either the empty string or the **string-concatenation** of the code unit 0x0020 (SPACE), the code unit 0x0028 (LEFT PARENTHESIS), an implementation-dependent timezone name, and the code unit 0x0029 (RIGHT PARENTHESIS).
8. Return the **string-concatenation** of *offsetSign*, *offsetHour*, *offsetMin*, and *tzName*.

20.3.4.41.4 Runtime Semantics: ToDateString (*tv*)

The following steps are performed:

1. **Assert:** `Type(tv)` is `Number`.
2. If `tv` is `NaN`, return `"Invalid Date"`.
3. Let `t` be `LocalTime(tv)`.
4. Return the **string-concatenation** of `DateString(t)`, the code unit 0x0020 (SPACE), `TimeString(t)`, and `TimeZoneString(tv)`.

20.3.4.42 Date.prototype.toTimeString ()

The following steps are performed:

1. Let `O` be **this** `Date` object.
2. Let `tv` be ? `thisTimeValue(O)`.
3. If `tv` is `NaN`, return `"Invalid Date"`.
4. Let `t` be `LocalTime(tv)`.
5. Return the **string-concatenation** of `TimeString(t)` and `TimeZoneString(tv)`.

20.3.4.43 Date.prototype.toUTCString ()

The following steps are performed:

1. Let `O` be **this** `Date` object.
2. Let `tv` be ? `thisTimeValue(O)`.
3. If `tv` is `NaN`, return `"Invalid Date"`.
4. Let `weekday` be the Name of the entry in Table 49 with the Number `WeekDay(tv)`.
5. Let `month` be the Name of the entry in Table 50 with the Number `MonthFromTime(tv)`.
6. Let `day` be the String representation of `DateFromTime(tv)`, formatted as a two-digit decimal number, padded to the left with a zero if necessary.
7. Let `year` be the String representation of `YearFromTime(tv)`, formatted as a decimal number of at least four digits, padded to the left with zeroes if necessary.
8. Return the **string-concatenation** of `weekday`, `" , "`, the code unit 0x0020 (SPACE), `day`, the code unit 0x0020 (SPACE), `month`, the code unit 0x0020 (SPACE), `year`, the code unit 0x0020 (SPACE), and `TimeString(tv)`.

20.3.4.44 Date.prototype.valueOf ()

The following steps are performed:

1. Return ? `thisTimeValue(this value)`.

20.3.4.45 Date.prototype [@@toPrimitive] (hint)

This function is called by ECMAScript language operators to convert a `Date` object to a primitive value. The allowed values for `hint` are `"default"`, `"number"`, and `"string"`. `Date` objects, are unique among built-in ECMAScript object in that they treat `"default"` as being equivalent to `"string"`. All other built-in ECMAScript objects treat `"default"` as being equivalent to `"number"`.

When the `@@toPrimitive` method is called with argument `hint`, the following steps are taken:

1. Let `O` be the **this** value.
2. If `Type(O)` is not `Object`, throw a `TypeError` exception.

3. If *hint* is the String value **"string"** or the String value **"default"**, then
 - a. Let *tryFirst* be **"string"**.
4. Else if *hint* is the String value **"number"**, then
 - a. Let *tryFirst* be **"number"**.
5. Else, throw a **TypeError** exception.
6. Return ? **OrdinaryToPrimitive**(*O*, *tryFirst*).

The value of the **name** property of this function is **"[Symbol.toPrimitive]"**.

This property has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **true** }.

20.3.5 Properties of Date Instances

Date instances are ordinary objects that inherit properties from the Date prototype object. Date instances also have a [[DateValue]] internal slot. The [[DateValue]] internal slot is the **time value** represented by [this Date object](#).

21 Text Processing

21.1 String Objects

21.1.1 The String Constructor

The String **constructor**:

is the intrinsic object %String%.

is the initial value of the **String** property of the [global object](#).

creates and initializes a new String object when called as a **constructor**.

performs a type conversion when called as a function rather than as a **constructor**.

is designed to be subclassable. It may be used as the value of an **extends** clause of a class definition. Subclass constructors that intend to inherit the specified **String** behaviour must include a **super** call to the **String constructor** to create and initialize the subclass instance with a [[StringData]] internal slot.

21.1.1.1 String (*value*)

When **String** is called with argument *value*, the following steps are taken:

1. If no arguments were passed to this function invocation, let *s* be "".
2. Else,
 - a. If NewTarget is **undefined** and **Type**(*value*) is Symbol, return **SymbolDescriptiveString**(*value*).
 - b. Let *s* be ? **ToString**(*value*).
3. If NewTarget is **undefined**, return *s*.
4. Return ! **StringCreate**(*s*, ? **GetPrototypeFromConstructor**(NewTarget, "%StringPrototype%")).

21.1.2 Properties of the String Constructor

The String **constructor**:

has a `[[Prototype]]` internal slot whose value is the intrinsic object `%FunctionPrototype%`.
has the following properties:

21.1.2.1 `String.fromCharCode ( ...codeUnits )`

The **`String.fromCharCode`** function may be called with any number of arguments which form the rest parameter *codeUnits*. The following steps are taken:

1. Let *codeUnits* be a `List` containing the arguments passed to this function.
2. Let *length* be the number of elements in *codeUnits*.
3. Let *elements* be a new empty `List`.
4. Let *nextIndex* be 0.
5. Repeat, while *nextIndex* < *length*
 - a. Let *next* be *codeUnits*[*nextIndex*].
 - b. Let *nextCU* be ? `ToUint16`(*next*).
 - c. Append *nextCU* to the end of *elements*.
 - d. Increase *nextIndex* by 1.
6. Return the String value whose code units are, in order, the elements in the `List` *elements*. If *length* is 0, the empty string is returned.

The "**`length`**" property of the **`fromCharCode`** function is 1.

21.1.2.2 `String.fromCodePoint ( ...codePoints )`

The **`String.fromCodePoint`** function may be called with any number of arguments which form the rest parameter *codePoints*. The following steps are taken:

1. Let *codePoints* be a `List` containing the arguments passed to this function.
2. Let *length* be the number of elements in *codePoints*.
3. Let *elements* be a new empty `List`.
4. Let *nextIndex* be 0.
5. Repeat, while *nextIndex* < *length*
 - a. Let *next* be *codePoints*[*nextIndex*].
 - b. Let *nextCP* be ? `ToNumber`(*next*).
 - c. If `SameValue`(*nextCP*, ! `ToInteger`(*nextCP*)) is **false**, throw a **`RangeError`** exception.
 - d. If *nextCP* < 0 or *nextCP* > 0x10FFFF, throw a **`RangeError`** exception.
 - e. Append the elements of the `UTF16Encoding` of *nextCP* to the end of *elements*.
 - f. Increase *nextIndex* by 1.
6. Return the String value whose code units are, in order, the elements in the `List` *elements*. If *length* is 0, the empty string is returned.

The "**`length`**" property of the **`fromCodePoint`** function is 1.

21.1.2.3 `String.prototype`

The initial value of **`String.prototype`** is the intrinsic object `%StringPrototype%`.

This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **false** }.

21.1.2.4 String.raw (*template*, ...*substitutions*)

The **String.raw** function may be called with a variable number of arguments. The first argument is *template* and the remainder of the arguments form the *List substitutions*. The following steps are taken:

1. Let *substitutions* be a *List* consisting of all of the arguments passed to this function, starting with the second argument. If fewer than two arguments were passed, the *List* is empty.
2. Let *numberOfSubstitutions* be the number of elements in *substitutions*.
3. Let *cooked* be ? *ToObject*(*template*).
4. Let *raw* be ? *ToObject*(? *Get*(*cooked*, "raw")).
5. Let *literalSegments* be ? *ToLength*(? *Get*(*raw*, "length")).
6. If *literalSegments* ≤ 0, return the empty string.
7. Let *stringElements* be a new empty *List*.
8. Let *nextIndex* be 0.
9. Repeat,
 - a. Let *nextKey* be ! *ToString*(*nextIndex*).
 - b. Let *nextSeg* be ? *Tostring*(? *Get*(*raw*, *nextKey*)).
 - c. Append in order the code unit elements of *nextSeg* to the end of *stringElements*.
 - d. If *nextIndex* + 1 = *literalSegments*, then
 - i. Return the String value whose code units are, in order, the elements in the *List stringElements*. If *stringElements* has no elements, the empty string is returned.
 - e. If *nextIndex* < *numberOfSubstitutions*, let *next* be *substitutions*[*nextIndex*].
 - f. Else, let *next* be the empty String.
 - g. Let *nextSub* be ? *Tostring*(*next*).
 - h. Append in order the code unit elements of *nextSub* to the end of *stringElements*.
 - i. Increase *nextIndex* by 1.

NOTE

String.raw is intended for use as a tag function of a Tagged Template (12.3.7). When called as such, the first argument will be a well formed template object and the rest parameter will contain the substitution values.

21.1.3 Properties of the String Prototype Object

The String prototype object:

is the intrinsic object %StringPrototype%.

is a String *exotic object* and has the internal methods specified for such objects.

has a [[StringData]] internal slot whose value is the empty String.

has a "length" property whose initial value is 0 and whose attributes are { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **false** }.

has a [[Prototype]] internal slot whose value is the intrinsic object %ObjectPrototype%.

Unless explicitly stated otherwise, the methods of the String prototype object defined below are not generic and the **this** value passed to them must be either a String value or an object that has a [[StringData]] internal slot that has been initialized to a String value.

The abstract operation thisStringValue(*value*) performs the following steps:

1. If *Type*(*value*) is String, return *value*.

2. If **Type**(*value*) is Object and *value* has a `[[StringData]]` internal slot, then
 - a. Let *s* be *value*.`[[StringData]]`.
 - b. **Assert**: **Type**(*s*) is String.
 - c. Return *s*.
3. Throw a **TypeError** exception.

21.1.3.1 String.prototype.charAt (*pos*)

NOTE 1

Returns a single element String containing the code unit at index *pos* within the String value resulting from converting this object to a String. If there is no element at that index, the result is the empty String. The result is a String value, not a String object.

If *pos* is a value of Number type that is an integer, then the result of **x**.**charAt** (*pos*) is equal to the result of **x**.**substring**(*pos*, *pos* + 1).

When the **charAt** method is called with one argument *pos*, the following steps are taken:

1. Let *O* be ? **RequireObjectCoercible**(**this** value).
2. Let *S* be ? **ToString**(*O*).
3. Let *position* be ? **ToInteger**(*pos*).
4. Let *size* be the length of *S*.
5. If *position* < 0 or *position* ≥ *size*, return the empty String.
6. Return the String value of length 1, containing one code unit from *S*, namely the code unit at index *position*.

NOTE 2

The **charAt** function is intentionally generic; it does not require that its **this** value be a String object. Therefore, it can be transferred to other kinds of objects for use as a method.

21.1.3.2 String.prototype.charCodeAtAt (*pos*)

NOTE 1

Returns a Number (a nonnegative integer less than 2¹⁶) that is the numeric value of the code unit at index *pos* within the String resulting from converting this object to a String. If there is no element at that index, the result is **NaN**.

When the **charCodeAtAt** method is called with one argument *pos*, the following steps are taken:

1. Let *O* be ? **RequireObjectCoercible**(**this** value).
2. Let *S* be ? **ToString**(*O*).
3. Let *position* be ? **ToInteger**(*pos*).
4. Let *size* be the length of *S*.
5. If *position* < 0 or *position* ≥ *size*, return **NaN**.
6. Return a value of Number type, whose value is the numeric value of the code unit at index *position* within the String *S*.

NOTE 2

The **charCodeAtAt** function is intentionally generic; it does not require that its **this** value be a String object. Therefore it can be transferred to other kinds of objects for use as a method.

21.1.3.3 String.prototype.codePointAt (*pos*)

NOTE 1

Returns a nonnegative integer Number less than 0x110000 that is the code point value of the UTF-16 encoded code point (6.1.4) starting at the string element at index *pos* within the String resulting from converting this object to a String. If there is no element at that index, the result is **undefined**. If a valid UTF-16 surrogate pair does not begin at *pos*, the result is the code unit at *pos*.

When the **codePointAt** method is called with one argument *pos*, the following steps are taken:

1. Let *O* be ? **RequireObjectCoercible**(**this** value).
2. Let *S* be ? **ToString**(*O*).
3. Let *position* be ? **ToInteger**(*pos*).
4. Let *size* be the length of *S*.
5. If *position* < 0 or *position* ≥ *size*, return **undefined**.
6. Let *first* be the numeric value of the code unit at index *position* within the String *S*.
7. If *first* < 0xD800 or *first* > 0xDBFF or *position* + 1 = *size*, return *first*.
8. Let *second* be the numeric value of the code unit at index *position* + 1 within the String *S*.
9. If *second* < 0xDC00 or *second* > 0xDFFF, return *first*.
10. Return **UTF16Decode**(*first*, *second*).

NOTE 2

The **codePointAt** function is intentionally generic; it does not require that its **this** value be a String object. Therefore it can be transferred to other kinds of objects for use as a method.

21.1.3.4 String.prototype.concat (...*args*)

NOTE 1

When the **concat** method is called it returns the String value consisting of the code units of the **this** object (converted to a String) followed by the code units of each of the arguments converted to a String. The result is a String value, not a String object.

When the **concat** method is called with zero or more arguments, the following steps are taken:

1. Let *O* be ? **RequireObjectCoercible**(**this** value).
2. Let *S* be ? **ToString**(*O*).
3. Let *args* be a List whose elements are the arguments passed to this function.
4. Let *R* be *S*.
5. Repeat, while *args* is not empty
 - a. Remove the first element from *args* and let *next* be the value of that element.
 - b. Let *nextString* be ? **ToString**(*next*).
 - c. Set *R* to the string-concatenation of the previous value of *R* and *nextString*.
6. Return *R*.

The "**length**" property of the **concat** method is 1.

NOTE 2

The **concat** function is intentionally generic; it does not require that its **this** value be a String object. Therefore it can be transferred to other kinds of objects for use as a method.

21.1.3.5 String.prototype.constructor

The initial value of **String.prototype.constructor** is the intrinsic object %String%.

21.1.3.6 String.prototype.endsWith (*searchString* [, *endPosition*])

The following steps are taken:

1. Let *O* be ? **RequireObjectCoercible**(**this** value).
2. Let *S* be ? **ToString**(*O*).
3. Let *isRegExp* be ? **IsRegExp**(*searchString*).
4. If *isRegExp* is **true**, throw a **TypeError** exception.
5. Let *searchStr* be ? **ToString**(*searchString*).
6. Let *len* be the length of *S*.
7. If *endPosition* is **undefined**, let *pos* be *len*, else let *pos* be ? **ToInteger**(*endPosition*).
8. Let *end* be **min**(**max**(*pos*, 0), *len*).
9. Let *searchLength* be the length of *searchStr*.
10. Let *start* be *end* - *searchLength*.
11. If *start* is less than 0, return **false**.
12. If the sequence of code units of *S* starting at *start* of length *searchLength* is the same as the full code unit sequence of *searchStr*, return **true**.
13. Otherwise, return **false**.

NOTE 1

Returns **true** if the sequence of code units of *searchString* converted to a String is the same as the corresponding code units of this object (converted to a String) starting at *endPosition* - **length**(this). Otherwise returns **false**.

NOTE 2

Throwing an exception if the first argument is a RegExp is specified in order to allow future editions to define extensions that allow such argument values.

NOTE 3

The **endsWith** function is intentionally generic; it does not require that its **this** value be a String object. Therefore, it can be transferred to other kinds of objects for use as a method.

21.1.3.7 String.prototype.includes (*searchString* [, *position*])

The **includes** method takes two arguments, *searchString* and *position*, and performs the following steps:

1. Let *O* be ? **RequireObjectCoercible**(**this** value).
2. Let *S* be ? **ToString**(*O*).
3. Let *isRegExp* be ? **IsRegExp**(*searchString*).
4. If *isRegExp* is **true**, throw a **TypeError** exception.
5. Let *searchStr* be ? **ToString**(*searchString*).
6. Let *pos* be ? **ToInteger**(*position*).
7. **Assert**: If *position* is **undefined**, then *pos* is 0.
8. Let *len* be the length of *S*.
9. Let *start* be **min**(**max**(*pos*, 0), *len*).
10. Let *searchLen* be the length of *searchStr*.

11. If there exists any integer *k* not smaller than *start* such that *k* + *searchLen* is not greater than *len*, and for all nonnegative integers *j* less than *searchLen*, the code unit at index *k* + *j* within *S* is the same as the code unit at index *j* within *searchStr*, return **true**; but if there is no such integer *k*, return **false**.

NOTE 1

If *searchString* appears as a substring of the result of converting this object to a String, at one or more indices that are greater than or equal to *position*, return **true**; otherwise, returns **false**. If *position* is **undefined**, 0 is assumed, so as to search all of the String.

NOTE 2

Throwing an exception if the first argument is a RegExp is specified in order to allow future editions to define extensions that allow such argument values.

NOTE 3

The **includes** function is intentionally generic; it does not require that its **this** value be a String object. Therefore, it can be transferred to other kinds of objects for use as a method.

21.1.3.8 String.prototype.indexOf (*searchString* [, *position*])

NOTE 1

If *searchString* appears as a substring of the result of converting this object to a String, at one or more indices that are greater than or equal to *position*, then the smallest such index is returned; otherwise, -1 is returned. If *position* is **undefined**, 0 is assumed, so as to search all of the String.

The **indexOf** method takes two arguments, *searchString* and *position*, and performs the following steps:

1. Let *O* be ? **RequireObjectCoercible**(**this** value).
2. Let *S* be ? **ToString**(*O*).
3. Let *searchStr* be ? **ToString**(*searchString*).
4. Let *pos* be ? **ToInteger**(*position*).
5. **Assert**: If *position* is **undefined**, then *pos* is 0.
6. Let *len* be the length of *S*.
7. Let *start* be **min**(**max**(*pos*, 0), *len*).
8. Let *searchLen* be the length of *searchStr*.
9. Return the smallest possible integer *k* not smaller than *start* such that *k* + *searchLen* is not greater than *len*, and for all nonnegative integers *j* less than *searchLen*, the code unit at index *k* + *j* within *S* is the same as the code unit at index *j* within *searchStr*; but if there is no such integer *k*, return the value -1.

NOTE 2

The **indexOf** function is intentionally generic; it does not require that its **this** value be a String object. Therefore, it can be transferred to other kinds of objects for use as a method.

21.1.3.9 String.prototype.lastIndexOf (*searchString* [, *position*])

NOTE 1

If *searchString* appears as a substring of the result of converting this object to a String at one or more indices that are smaller than or equal to *position*, then the greatest such index is returned; otherwise, -1 is returned. If *position* is **undefined**, the length of the String value is assumed, so as to search all of the String.

The **lastIndexOf** method takes two arguments, *searchString* and *position*, and performs the following steps:

1. Let *O* be ? **RequireObjectCoercible**(**this** value).
2. Let *S* be ? **ToString**(*O*).
3. Let *searchStr* be ? **ToString**(*searchString*).
4. Let *numPos* be ? **ToNumber**(*position*).
5. **Assert**: If *position* is **undefined**, then *numPos* is **NaN**.
6. If *numPos* is **NaN**, let *pos* be $+\infty$; otherwise, let *pos* be ! **ToInteger**(*numPos*).
7. Let *len* be the length of *S*.
8. Let *start* be **min**(**max**(*pos*, 0), *len*).
9. Let *searchLen* be the length of *searchStr*.
10. Return the largest possible nonnegative integer *k* not larger than *start* such that *k* + *searchLen* is not greater than *len*, and for all nonnegative integers *j* less than *searchLen*, the code unit at index *k* + *j* within *S* is the same as the code unit at index *j* within *searchStr*; but if there is no such integer *k*, return the value -1.

NOTE 2

The **lastIndexOf** function is intentionally generic; it does not require that its **this** value be a String object. Therefore, it can be transferred to other kinds of objects for use as a method.

21.1.3.10 String.prototype.localeCompare (*that* [, *reserved1* [, *reserved2*]])

An ECMAScript implementation that includes the ECMA-402 Internationalization API must implement the **localeCompare** method as specified in the ECMA-402 specification. If an ECMAScript implementation does not include the ECMA-402 API the following specification of the **localeCompare** method is used.

When the **localeCompare** method is called with argument *that*, it returns a Number other than **NaN** that represents the result of a locale-sensitive String comparison of the **this** value (converted to a String) with *that* (converted to a String). The two Strings are *S* and *That*. The two Strings are compared in an implementation-defined fashion. The result is intended to order String values in the sort order specified by a host default locale, and will be negative, zero, or positive, depending on whether *S* comes before *That* in the sort order, the Strings are equal, or *S* comes after *That* in the sort order, respectively.

Before performing the comparisons, the following steps are performed to prepare the Strings:

1. Let *O* be ? **RequireObjectCoercible**(**this** value).
2. Let *S* be ? **ToString**(*O*).
3. Let *That* be ? **ToString**(*that*).

The meaning of the optional second and third parameters to this method are defined in the ECMA-402 specification; implementations that do not include ECMA-402 support must not assign any other interpretation to those parameter positions.

The **localeCompare** method, if considered as a function of two arguments **this** and *that*, is a consistent comparison function (as defined in 22.1.3.27) on the set of all Strings.

The actual return values are implementation-defined to permit implementers to encode additional information in the value, but the function is required to define a total ordering on all Strings. This function must treat Strings that are canonically equivalent according to the Unicode standard as identical and must return **0** when comparing Strings that are considered canonically equivalent.

NOTE 1

The **localeCompare** method itself is not directly suitable as an argument to **Array.prototype.sort** because the latter requires a function of two arguments.

NOTE 2

This function is intended to rely on whatever language-sensitive comparison functionality is available to the ECMAScript environment from the host environment, and to compare according to the rules of the host environment's current locale. However, regardless of the host provided comparison capabilities, this function must treat Strings that are canonically equivalent according to the Unicode standard as identical. It is recommended that this function should not honour Unicode compatibility equivalences or decompositions. For a definition and discussion of canonical equivalence see the Unicode Standard, chapters 2 and 3, as well as Unicode Standard Annex #15, Unicode Normalization Forms (<https://unicode.org/reports/tr15/>) and Unicode Technical Note #5, Canonical Equivalence in Applications (<https://www.unicode.org/notes/tn5/>). Also see Unicode Technical Standard #10, Unicode Collation Algorithm (<https://unicode.org/reports/tr10/>).

NOTE 3

The **localeCompare** function is intentionally generic; it does not require that its **this** value be a String object. Therefore, it can be transferred to other kinds of objects for use as a method.

21.1.3.11 String.prototype.match (*regex*)

When the **match** method is called with argument *regex*, the following steps are taken:

1. Let *O* be ? **RequireObjectCoercible**(**this** value).
2. If *regex* is neither **undefined** nor **null**, then
 - a. Let *matcher* be ? **GetMethod**(*regex*, @@match).
 - b. If *matcher* is not **undefined**, then
 - i. Return ? **Call**(*matcher*, *regex*, « *O* »).
3. Let *S* be ? **ToString**(*O*).
4. Let *rx* be ? **RegExpCreate**(*regex*, **undefined**).
5. Return ? **Invoke**(*rx*, @@match, « *S* »).

NOTE

The **match** function is intentionally generic; it does not require that its **this** value be a String object. Therefore, it can be transferred to other kinds of objects for use as a method.

21.1.3.12 String.prototype.normalize ([*form*])

When the **normalize** method is called with one argument *form*, the following steps are taken:

1. Let *O* be ? **RequireObjectCoercible**(**this** value).
2. Let *S* be ? **ToString**(*O*).
3. If *form* is not present or *form* is **undefined**, set *form* to **"NFC"**.
4. Let *f* be ? **ToString**(*form*).
5. If *f* is not one of **"NFC"**, **"NFD"**, **"NFKC"**, or **"NFKD"**, throw a **RangeError** exception.
6. Let *ns* be the String value that is the result of normalizing *S* into the normalization form named by *f* as specified in <https://unicode.org/reports/tr15/>.
7. Return *ns*.

NOTE

The **normalize** function is intentionally generic; it does not require that its **this** value be a String object. Therefore it can be transferred to other kinds of objects for use as a method.

21.1.3.13 String.prototype.padEnd (*maxLength* [, *fillString*])

When the **padEnd** method is called, the following steps are taken:

1. Let *O* be ? **RequireObjectCoercible**(**this** value).
2. Let *S* be ? **ToString**(*O*).
3. Let *intMaxLength* be ? **ToLength**(*maxLength*).
4. Let *stringLength* be the length of *S*.
5. If *intMaxLength* is not greater than *stringLength*, return *S*.
6. If *fillString* is **undefined**, let *filler* be the String value consisting solely of the code unit 0x0020 (SPACE).
7. Else, let *filler* be ? **ToString**(*fillString*).
8. If *filler* is the empty String, return *S*.
9. Let *fillLen* be *intMaxLength* - *stringLength*.
10. Let *truncatedStringFiller* be the String value consisting of repeated concatenations of *filler* truncated to length *fillLen*.
11. Return the string-concatenation of *S* and *truncatedStringFiller*.

NOTE 1

The first argument *maxLength* will be clamped such that it can be no smaller than the length of the **this** value.

NOTE 2

The optional second argument *fillString* defaults to " " (the String value consisting of the code unit 0x0020 SPACE).

21.1.3.14 String.prototype.padStart (*maxLength* [, *fillString*])

When the **padStart** method is called, the following steps are taken:

1. Let *O* be ? **RequireObjectCoercible**(**this** value).
2. Let *S* be ? **ToString**(*O*).
3. Let *intMaxLength* be ? **ToLength**(*maxLength*).
4. Let *stringLength* be the length of *S*.
5. If *intMaxLength* is not greater than *stringLength*, return *S*.
6. If *fillString* is **undefined**, let *filler* be the String value consisting solely of the code unit 0x0020 (SPACE).
7. Else, let *filler* be ? **ToString**(*fillString*).
8. If *filler* is the empty String, return *S*.
9. Let *fillLen* be *intMaxLength* - *stringLength*.
10. Let *truncatedStringFiller* be the String value consisting of repeated concatenations of *filler* truncated to length *fillLen*.
11. Return the string-concatenation of *truncatedStringFiller* and *S*.

NOTE 1

The first argument *maxLength* will be clamped such that it can be no smaller than the length of the **this** value.

NOTE 2

The optional second argument *fillString* defaults to " " (the String value consisting of the code unit 0x0020 SPACE).

21.1.3.15 String.prototype.repeat (*count*)

The following steps are taken:

1. Let *O* be ? [RequireObjectCoercible](#)(**this** value).
2. Let *S* be ? [ToString](#)(*O*).
3. Let *n* be ? [ToInteger](#)(*count*).
4. If *n* < 0, throw a **RangeError** exception.
5. If *n* is +∞, throw a **RangeError** exception.
6. If *n* is 0, return the empty String.
7. Return the String value that is made from *n* copies of *S* appended together.

NOTE 1

This method creates the String value consisting of the code units of the **this** object (converted to String) repeated *count* times.

NOTE 2

The **repeat** function is intentionally generic; it does not require that its **this** value be a String object. Therefore, it can be transferred to other kinds of objects for use as a method.

21.1.3.16 String.prototype.replace (*searchValue*, *replaceValue*)

When the **replace** method is called with arguments *searchValue* and *replaceValue*, the following steps are taken:

1. Let *O* be ? [RequireObjectCoercible](#)(**this** value).
2. If *searchValue* is neither **undefined** nor **null**, then
 - a. Let *replacer* be ? [GetMethod](#)(*searchValue*, @@replace).
 - b. If *replacer* is not **undefined**, then
 - i. Return ? [Call](#)(*replacer*, *searchValue*, « *O*, *replaceValue* »).
3. Let *string* be ? [ToString](#)(*O*).
4. Let *searchString* be ? [ToString](#)(*searchValue*).
5. Let *functionalReplace* be [IsCallable](#)(*replaceValue*).
6. If *functionalReplace* is **false**, then
 - a. Set *replaceValue* to ? [ToString](#)(*replaceValue*).
7. Search *string* for the first occurrence of *searchString* and let *pos* be the index within *string* of the first code unit of the matched substring and let *matched* be *searchString*. If no occurrences of *searchString* were found, return *string*.
8. If *functionalReplace* is **true**, then
 - a. Let *replValue* be ? [Call](#)(*replaceValue*, **undefined**, « *matched*, *pos*, *string* »).
 - b. Let *replStr* be ? [ToString](#)(*replValue*).
9. Else,
 - a. Let *captures* be a new empty List.
 - b. Let *replStr* be [GetSubstitution](#)(*matched*, *string*, *pos*, *captures*, **undefined**, *replaceValue*).
10. Let *tailPos* be *pos* + the number of code units in *matched*.
11. Let *newString* be the [string-concatenation](#) of the first *pos* code units of *string*, *replStr*, and the trailing substring of *string* starting at index *tailPos*. If *pos* is 0, the first element of the concatenation will be the empty String.
12. Return *newString*.

NOTE

The **replace** function is intentionally generic; it does not require that its **this** value be a String object. Therefore, it can be transferred to other kinds of objects for use as a method.

21.1.3.16.1 Runtime Semantics: GetSubstitution (*matched*, *str*, *position*, *captures*, *namedCaptures*, *replacement*)

The abstract operation GetSubstitution performs the following steps:

- 1. Assert: Type(*matched*) is String.
- 2. Let *matchLength* be the number of code units in *matched*.
- 3. Assert: Type(*str*) is String.
- 4. Let *stringLength* be the number of code units in *str*.
- 5. Assert: *position* is a nonnegative integer.
- 6. Assert: *position* ≤ *stringLength*.
- 7. Assert: *captures* is a possibly empty List of Strings.
- 8. Assert: Type(*replacement*) is String.
- 9. Let *tailPos* be *position* + *matchLength*.
- 10. Let *m* be the number of elements in *captures*.
- 11. If *namedCaptures* is not **undefined**, then
 - a. Set *namedCaptures* to ? ToObject(*namedCaptures*).
- 12. Let *result* be the String value derived from *replacement* by copying code unit elements from *replacement* to *result* while performing replacements as specified in Table 51. These \$ replacements are done left-to-right, and, once such a replacement is performed, the new replacement text is not subject to further replacements.
- 13. Return *result*.

Table 51: Replacement Text Symbol Substitutions

Code units	Unicode Characters	Replacement text
0x0024, 0x0024	\$ \$	\$
0x0024, 0x0026	\$ &	<i>matched</i>
0x0024, 0x0060	\$ `	If <i>position</i> is 0, the replacement is the empty String. Otherwise the replacement is the substring of <i>str</i> that starts at index 0 and whose last code unit is at index <i>position</i> - 1.
0x0024, 0x0027	\$ '	If <i>tailPos</i> ≥ <i>stringLength</i> , the replacement is the empty String. Otherwise the replacement is the substring of <i>str</i> that starts at index <i>tailPos</i> and continues to the end of <i>str</i> .
0x0024, N Where 0x0031 ≤ N ≤ 0x0039	\$ <i>n</i> where <i>n</i> is one of 1 2 3 4 5 6 7 8 9 and \$ <i>n</i> is not followed by a decimal digit	The <i>n</i> th element of <i>captures</i> , where <i>n</i> is a single digit in the range 1 to 9. If <i>n</i> ≤ <i>m</i> and the <i>n</i> th element of <i>captures</i> is undefined , use the empty String instead. If <i>n</i> > <i>m</i> , no replacement is done.

0x0024, **\$nn** where
N, N **n** is one of
Where **0 1 2 3 4 5 6 7 8 9**
0x0030
≤ N ≤
0x0039

The *nn*th element of *captures*, where *nn* is a two-digit decimal number in the range 01 to 99. If *nn* ≤ *m* and the *nn*th element of *captures* is **undefined**, use the empty String instead. If *nn* is 00 or *nn* > *m*, no replacement is done.

0x0024, **\$<**
0x003C

1. If *namedCaptures* is **undefined**, the replacement text is the String "**\$<**".
2. Else,
 - a. Scan until the next > U+003E (GREATER-THAN SIGN).
 - b. If none is found, the replacement text is the String "**\$<**".
 - c. Else,
 - i. Let *groupName* be the enclosed substring.
 - ii. Let *capture* be ? *Get*(*namedCaptures*, *groupName*).
 - iii. If *capture* is **undefined**, replace the text through > with the empty string.
 - iv. Otherwise, replace the text through > with ? *ToString*(*capture*).

0x0024 **\$** in any context that does **\$**
not match any of the above.

21.1.3.17 String.prototype.search (*regexp*)

When the **search** method is called with argument *regexp*, the following steps are taken:

1. Let *O* be ? *RequireObjectCoercible*(**this** value).
2. If *regexp* is neither **undefined** nor **null**, then
 - a. Let *searcher* be ? *GetMethod*(*regexp*, @@search).
 - b. If *searcher* is not **undefined**, then
 - i. Return ? *Call*(*searcher*, *regexp*, « *O* »).
3. Let *string* be ? *ToString*(*O*).
4. Let *rx* be ? *RegExpCreate*(*regexp*, **undefined**).
5. Return ? *Invoke*(*rx*, @@search, « *string* »).

NOTE

The **search** function is intentionally generic; it does not require that its **this** value be a String object. Therefore, it can be transferred to other kinds of objects for use as a method.

21.1.3.18 String.prototype.slice (*start*, *end*)

The **slice** method takes two arguments, *start* and *end*, and returns a substring of the result of converting this object to a String, starting from index *start* and running to, but not including, index *end* (or through the end of the String if *end* is **undefined**). If *start* is negative, it is treated as *sourceLength* + *start* where *sourceLength* is the length of the String. If *end* is negative, it is treated as *sourceLength* + *end* where *sourceLength* is the length of the String. The result is a String

value, not a String object. The following steps are taken:

1. Let *O* be ? **RequireObjectCoercible**(**this** value).
2. Let *S* be ? **ToString**(*O*).
3. Let *len* be the length of *S*.
4. Let *intStart* be ? **ToInteger**(*start*).
5. If *end* is **undefined**, let *intEnd* be *len*; else let *intEnd* be ? **ToInteger**(*end*).
6. If *intStart* < 0, let *from* be **max**(*len* + *intStart*, 0); otherwise let *from* be **min**(*intStart*, *len*).
7. If *intEnd* < 0, let *to* be **max**(*len* + *intEnd*, 0); otherwise let *to* be **min**(*intEnd*, *len*).
8. Let *span* be **max**(*to* - *from*, 0).
9. Return the String value containing *span* consecutive code units from *S* beginning with the code unit at index *from*.

NOTE

The **slice** function is intentionally generic; it does not require that its **this** value be a String object. Therefore it can be transferred to other kinds of objects for use as a method.

21.1.3.19 String.prototype.split (*separator*, *limit*)

Returns an Array object into which substrings of the result of converting this object to a String have been stored. The substrings are determined by searching from left to right for occurrences of *separator*; these occurrences are not part of any substring in the returned array, but serve to divide up the String value. The value of *separator* may be a String of any length or it may be an object, such as a RegExp, that has a @@split method.

When the **split** method is called, the following steps are taken:

1. Let *O* be ? **RequireObjectCoercible**(**this** value).
2. If *separator* is neither **undefined** nor **null**, then
 - a. Let *splitter* be ? **GetMethod**(*separator*, @@split).
 - b. If *splitter* is not **undefined**, then
 - i. Return ? **Call**(*splitter*, *separator*, « *O*, *limit* »).
3. Let *S* be ? **ToString**(*O*).
4. Let *A* be ! **ArrayCreate**(0).
5. Let *lengthA* be 0.
6. If *limit* is **undefined**, let *lim* be $2^{32} - 1$; else let *lim* be ? **ToUint32**(*limit*).
7. Let *s* be the length of *S*.
8. Let *p* be 0.
9. Let *R* be ? **ToString**(*separator*).
10. If *lim* = 0, return *A*.
11. If *separator* is **undefined**, then
 - a. Perform ! **CreateDataProperty**(*A*, "0", *S*).
 - b. Return *A*.
12. If *s* = 0, then
 - a. Let *z* be **SplitMatch**(*S*, 0, *R*).
 - b. If *z* is not **false**, return *A*.
 - c. Perform ! **CreateDataProperty**(*A*, "0", *S*).
 - d. Return *A*.
13. Let *q* be *p*.
14. Repeat, while *q* ≠ *s*

- a. Let e be `SplitMatch( $S$ ,  $q$ ,  $R$ )`.
- b. If e is **false**, increase q by 1.
- c. Else e is an integer index $\leq s$,
 - i. If $e = p$, increase q by 1.
 - ii. Else $e \neq p$,
 1. Let T be the String value equal to the substring of S consisting of the code units at indices p (inclusive) through q (exclusive).
 2. Perform `! CreateDataProperty( $A$ , ! ToString( $lengthA$ ),  $T$ )`.
 3. Increment $lengthA$ by 1.
 4. If $lengthA = lim$, return A .
 5. Set p to e .
 6. Set q to p .
15. Let T be the String value equal to the substring of S consisting of the code units at indices p (inclusive) through s (exclusive).
16. Perform `! CreateDataProperty( $A$ , ! ToString( $lengthA$ ),  $T$ )`.
17. Return A .

NOTE 1

The value of `separator` may be an empty String. In this case, `separator` does not match the empty substring at the beginning or end of the input String, nor does it match the empty substring at the end of the previous separator match. If `separator` is the empty String, the String is split up into individual code unit elements; the length of the result array equals the length of the String, and each substring contains one code unit.

If the **this** object is (or converts to) the empty String, the result depends on whether `separator` can match the empty String. If it can, the result array contains no elements. Otherwise, the result array contains one element, which is the empty String.

If `separator` is **undefined**, then the result array contains just one String, which is the **this** value (converted to a String). If `limit` is not **undefined**, then the output array is truncated so that it contains no more than `limit` elements.

NOTE 2

The **split** function is intentionally generic; it does not require that its **this** value be a String object. Therefore, it can be transferred to other kinds of objects for use as a method.

21.1.3.19.1 Runtime Semantics: SplitMatch (S , q , R)

The abstract operation SplitMatch takes three parameters, a String S , an integer q , and a String R , and performs the following steps in order to return either **false** or the end index of a match:

1. **Assert:** `Type( $R$ )` is String.
2. Let r be the number of code units in R .
3. Let s be the number of code units in S .
4. If $q + r > s$, return **false**.
5. If there exists an integer i between 0 (inclusive) and r (exclusive) such that the code unit at index $q + i$ within S is different from the code unit at index i within R , return **false**.
6. Return $q + r$.

21.1.3.20 String.prototype.startsWith (`searchString` [, `position`])

The following steps are taken:

1. Let *O* be ? **RequireObjectCoercible**(**this** value).
2. Let *S* be ? **ToString**(*O*).
3. Let *isRegExp* be ? **IsRegExp**(*searchString*).
4. If *isRegExp* is **true**, throw a **TypeError** exception.
5. Let *searchStr* be ? **ToString**(*searchString*).
6. Let *pos* be ? **ToInteger**(*position*).
7. **Assert**: If *position* is **undefined**, then *pos* is 0.
8. Let *len* be the length of *S*.
9. Let *start* be **min**(**max**(*pos*, 0), *len*).
10. Let *searchLength* be the length of *searchStr*.
11. If *searchLength* + *start* is greater than *len*, return **false**.
12. If the sequence of code units of *S* starting at *start* of length *searchLength* is the same as the full code unit sequence of *searchStr*, return **true**.
13. Otherwise, return **false**.

NOTE 1

This method returns **true** if the sequence of code units of *searchString* converted to a String is the same as the corresponding code units of this object (converted to a String) starting at index *position*. Otherwise returns **false**.

NOTE 2

Throwing an exception if the first argument is a RegExp is specified in order to allow future editions to define extensions that allow such argument values.

NOTE 3

The **startsWith** function is intentionally generic; it does not require that its **this** value be a String object. Therefore, it can be transferred to other kinds of objects for use as a method.

21.1.3.21 String.prototype.substring (*start*, *end*)

The **substring** method takes two arguments, *start* and *end*, and returns a substring of the result of converting this object to a String, starting from index *start* and running to, but not including, index *end* of the String (or through the end of the String if *end* is **undefined**). The result is a String value, not a String object.

If either argument is **NaN** or negative, it is replaced with zero; if either argument is larger than the length of the String, it is replaced with the length of the String.

If *start* is larger than *end*, they are swapped.

The following steps are taken:

1. Let *O* be ? **RequireObjectCoercible**(**this** value).
2. Let *S* be ? **ToString**(*O*).
3. Let *len* be the length of *S*.
4. Let *intStart* be ? **ToInteger**(*start*).
5. If *end* is **undefined**, let *intEnd* be *len*; else let *intEnd* be ? **ToInteger**(*end*).
6. Let *finalStart* be **min**(**max**(*intStart*, 0), *len*).
7. Let *finalEnd* be **min**(**max**(*intEnd*, 0), *len*).
8. Let *from* be **min**(*finalStart*, *finalEnd*).
9. Let *to* be **max**(*finalStart*, *finalEnd*).
10. Return the String value whose length is *to* - *from*, containing code units from *S*, namely the code units with indices *from* through *to* - 1, in ascending order.

NOTE

The **substring** function is intentionally generic; it does not require that its **this** value be a String object. Therefore, it can be transferred to other kinds of objects for use as a method.

21.1.3.22 String.prototype.toLocaleLowerCase ([*reserved1* [, *reserved2*]])

An ECMAScript implementation that includes the ECMA-402 Internationalization API must implement the **toLocaleLowerCase** method as specified in the ECMA-402 specification. If an ECMAScript implementation does not include the ECMA-402 API the following specification of the **toLocaleLowerCase** method is used.

This function interprets a String value as a sequence of UTF-16 encoded code points, as described in 6.1.4.

This function works exactly the same as **toLowerCase** except that its result is intended to yield the correct result for the host environment's current locale, rather than a locale-independent result. There will only be a difference in the few cases (such as Turkish) where the rules for that language conflict with the regular Unicode case mappings.

The meaning of the optional parameters to this method are defined in the ECMA-402 specification; implementations that do not include ECMA-402 support must not use those parameter positions for anything else.

NOTE

The **toLocaleLowerCase** function is intentionally generic; it does not require that its **this** value be a String object. Therefore, it can be transferred to other kinds of objects for use as a method.

21.1.3.23 String.prototype.toLocaleUpperCase ([*reserved1* [, *reserved2*]])

An ECMAScript implementation that includes the ECMA-402 Internationalization API must implement the **toLocaleUpperCase** method as specified in the ECMA-402 specification. If an ECMAScript implementation does not include the ECMA-402 API the following specification of the **toLocaleUpperCase** method is used.

This function interprets a String value as a sequence of UTF-16 encoded code points, as described in 6.1.4.

This function works exactly the same as **toUpperCase** except that its result is intended to yield the correct result for the host environment's current locale, rather than a locale-independent result. There will only be a difference in the few cases (such as Turkish) where the rules for that language conflict with the regular Unicode case mappings.

The meaning of the optional parameters to this method are defined in the ECMA-402 specification; implementations that do not include ECMA-402 support must not use those parameter positions for anything else.

NOTE

The **toLocaleUpperCase** function is intentionally generic; it does not require that its **this** value be a String object. Therefore, it can be transferred to other kinds of objects for use as a method.

21.1.3.24 String.prototype.toLowerCase ()

This function interprets a String value as a sequence of UTF-16 encoded code points, as described in 6.1.4. The following steps are taken:

1. Let **O** be ? **RequireObjectCoercible**(**this** value).

2. Let *S* be ? `ToString(O)`.
3. Let *cpList* be a `List` containing in order the code points as defined in 6.1.4 of *S*, starting at the first element of *S*.
4. Let *cuList* be a `List` where the elements are the result of `toLowerCase(cpList)`, according to the Unicode Default Case Conversion algorithm.
5. Let *L* be the String value whose code units are the `UTF16Encoding` of the code points of *cuList*.
6. Return *L*.

The result must be derived according to the locale-insensitive case mappings in the Unicode Character Database (this explicitly includes not only the UnicodeData.txt file, but also all locale-insensitive mappings in the SpecialCasings.txt file that accompanies it).

NOTE 1

The case mapping of some code points may produce multiple code points. In this case the result String may not be the same length as the source String. Because both `toUpperCase` and `toLowerCase` have context-sensitive behaviour, the functions are not symmetrical. In other words, `s.toUpperCase().toLowerCase()` is not necessarily equal to `s.toLowerCase()`.

NOTE 2

The `toLowerCase` function is intentionally generic; it does not require that its `this` value be a String object. Therefore, it can be transferred to other kinds of objects for use as a method.

21.1.3.25 String.prototype.toString ()

When the `toString` method is called, the following steps are taken:

1. Return ? `thisStringValue(this value)`.

NOTE

For a String object, the `toString` method happens to return the same thing as the `valueOf` method.

21.1.3.26 String.prototype.toUpperCase ()

This function interprets a String value as a sequence of UTF-16 encoded code points, as described in 6.1.4.

This function behaves in exactly the same way as `String.prototype.toLowerCase`, except that the String is mapped using the `toUpperCase` algorithm of the Unicode Default Case Conversion.

NOTE

The `toUpperCase` function is intentionally generic; it does not require that its `this` value be a String object. Therefore, it can be transferred to other kinds of objects for use as a method.

21.1.3.27 String.prototype.trim ()

This function interprets a String value as a sequence of UTF-16 encoded code points, as described in 6.1.4.

The following steps are taken:

1. Let *S* be `this` value.
2. Return ? `TrimString(S, "start+end")`.

NOTE

The **trim** function is intentionally generic; it does not require that its **this** value be a String object. Therefore, it can be transferred to other kinds of objects for use as a method.

21.1.3.27.1 Runtime Semantics: TrimString (*string*, *where*)

The abstract operation TrimString is called with arguments *string* and *where*, and interprets the String value *string* as a sequence of UTF-16 encoded code points, as described in 6.1.4. It performs the following steps:

1. Let *str* be ? **RequireObjectCoercible**(*string*).
2. Let *S* be ? **ToString**(*str*).
3. If *where* is "**start**", let *T* be the String value that is a copy of *S* with leading white space removed.
4. Else if *where* is "**end**", let *T* be the String value that is a copy of *S* with trailing white space removed.
5. Else,
 - a. **Assert**: *where* is "**start+end**".
 - b. Let *T* be the String value that is a copy of *S* with both leading and trailing white space removed.
6. Return *T*.

The definition of white space is the union of *WhiteSpace* and *LineTerminator*. When determining whether a Unicode code point is in Unicode general category “Space_Separator” (“Zs”), code unit sequences are interpreted as UTF-16 encoded code point sequences as specified in 6.1.4.

21.1.3.28 String.prototype.trimEnd ()

This function interprets a String value as a sequence of UTF-16 encoded code points, as described in 6.1.4.

The following steps are taken:

1. Let *S* be **this** value.
2. Return ? **TrimString**(*S*, "**end**").

NOTE

The **trimEnd** function is intentionally generic; it does not require that its **this** value be a String object. Therefore, it can be transferred to other kinds of objects for use as a method.

21.1.3.29 String.prototype.trimStart ()

This function interprets a String value as a sequence of UTF-16 encoded code points, as described in 6.1.4.

The following steps are taken:

1. Let *S* be **this** value.
2. Return ? **TrimString**(*S*, "**start**").

NOTE

The **trimStart** function is intentionally generic; it does not require that its **this** value be a String object. Therefore, it can be transferred to other kinds of objects for use as a method.

21.1.3.30 String.prototype.valueOf ()

When the **valueOf** method is called, the following steps are taken:

1. Return ? **thisStringValue**(**this** value).

21.1.3.31 String.prototype [@@iterator] ()

When the **@@iterator** method is called it returns an Iterator object (25.1.1.2) that iterates over the code points of a String value, returning each code point as a String value. The following steps are taken:

1. Let **O** be ? **RequireObjectCoercible**(**this** value).
2. Let **S** be ? **ToString**(**O**).
3. Return **CreateStringIterator**(**S**).

The value of the **name** property of this function is "**[Symbol.iterator]**".

21.1.4 Properties of String Instances

String instances are String exotic objects and have the internal methods specified for such objects. String instances inherit properties from the String prototype object. String instances also have a **[[StringData]]** internal slot.

String instances have a "**length**" property, and a set of enumerable properties with integer-indexed names.

21.1.4.1 length

The number of elements in the String value represented by this String object.

Once a String object is initialized, this property is unchanging. It has the attributes { **[[Writable]]**: **false**, **[[Enumerable]]**: **false**, **[[Configurable]]**: **false** }.

21.1.5 String Iterator Objects

A String Iterator is an object, that represents a specific iteration over some specific String instance object. There is not a named **constructor** for String Iterator objects. Instead, String iterator objects are created by calling certain methods of String instance objects.

21.1.5.1 CreateStringIterator (*string*)

Several methods of String objects return Iterator objects. The abstract operation **CreateStringIterator** with argument *string* is used to create such iterator objects. It performs the following steps:

1. **Assert**: **Type**(*string*) is String.
2. Let *iterator* be **ObjectCreate**(%StringIteratorPrototype%, « **[[IteratedString]]**, **[[StringIteratorNextIndex]]** »).
3. Set *iterator*.**[[IteratedString]]** to *string*.
4. Set *iterator*.**[[StringIteratorNextIndex]]** to 0.
5. Return *iterator*.

21.1.5.2 The %StringIteratorPrototype% Object

The %StringIteratorPrototype% object:

has properties that are inherited by all String Iterator Objects.
is an ordinary object.
has a `[[Prototype]]` internal slot whose value is the intrinsic object `%IteratorPrototype%`.
has the following properties:

21.1.5.2.1 %StringIteratorPrototype%.next ()

1. Let *O* be the **this** value.
2. If `Type(O)` is not Object, throw a **TypeError** exception.
3. If *O* does not have all of the internal slots of a String Iterator Instance (21.1.5.3), throw a **TypeError** exception.
4. Let *s* be *O*.`[[IteratedString]]`.
5. If *s* is **undefined**, return `CreateIterResultObject(undefined, true)`.
6. Let *position* be *O*.`[[StringIteratorNextIndex]]`.
7. Let *len* be the length of *s*.
8. If *position* ≥ *len*, then
 - a. Set *O*.`[[IteratedString]]` to **undefined**.
 - b. Return `CreateIterResultObject(undefined, true)`.
9. Let *first* be the numeric value of the code unit at index *position* within *s*.
10. If *first* < 0xD800 or *first* > 0xDBFF or *position* + 1 = *len*, let *resultString* be the String value consisting of the single code unit *first*.
11. Else,
 - a. Let *second* be the numeric value of the code unit at index *position* + 1 within the String *s*.
 - b. If *second* < 0xDC00 or *second* > 0xDFFF, let *resultString* be the String value consisting of the single code unit *first*.
 - c. Else, let *resultString* be the **string-concatenation** of the code unit *first* and the code unit *second*.
12. Let *resultSize* be the number of code units in *resultString*.
13. Set *O*.`[[StringIteratorNextIndex]]` to *position* + *resultSize*.
14. Return `CreateIterResultObject(resultString, false)`.

21.1.5.2.2 %StringIteratorPrototype% [@@toStringTag]

The initial value of the `@@toStringTag` property is the String value **"String Iterator"**.
This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **true** }.

21.1.5.3 Properties of String Iterator Instances

String Iterator instances are ordinary objects that inherit properties from the `%StringIteratorPrototype%` intrinsic object. String Iterator instances are initially created with the internal slots listed in Table 52.

Table 52: Internal Slots of String Iterator Instances

Internal Slot	Description
<code>[[IteratedString]]</code>	The String value whose code units are being iterated.
<code>[[StringIteratorNextIndex]]</code>	The integer index of the next string index to be examined by this iteration.

21.2 RegExp (Regular Expression) Objects

A RegExp object contains a regular expression and the associated flags.

NOTE

The form and functionality of regular expressions is modelled after the regular expression facility in the Perl 5 programming language.

21.2.1 Patterns

The **RegExp constructor** applies the following grammar to the input pattern String. An error occurs if the grammar cannot interpret the String as an expansion of *Pattern*.

Syntax

```
Pattern[U, N] ::  
    Disjunction[?U, ?N]  
  
Disjunction[U, N] ::  
    Alternative[?U, ?N]  
    Alternative[?U, ?N] | Disjunction[?U, ?N]  
  
Alternative[U, N] ::  
    [empty]  
    Alternative[?U, ?N] Term[?U, ?N]  
  
Term[U, N] ::  
    Assertion[?U, ?N]  
    Atom[?U, ?N]  
    Atom[?U, ?N] Quantifier  
  
Assertion[U, N] ::  
    ^  
    $  
    \ b  
    \ B  
    ( ? = Disjunction[?U, ?N] )  
    ( ? ! Disjunction[?U, ?N] )  
    ( ? <= Disjunction[?U, ?N] )  
    ( ? <! Disjunction[?U, ?N] )  
  
Quantifier ::  
    QuantifierPrefix  
    QuantifierPrefix ?  
  
QuantifierPrefix ::  
    *  
    +  
    ?  
    { DecimalDigits }
```

{ DecimalDigits , }
{ DecimalDigits , DecimalDigits }

*Atom*_[U, N] ::
PatternCharacter
.
*\ AtomEscape*_[?U, ?N]
*CharacterClass*_[?U]
*(GroupSpecifier*_[?U] *Disjunction*_[?U, ?N] *)*
*(? : Disjunction*_[?U, ?N] *)*

SyntaxCharacter :: **one of**
^ \$ \ . * + ? () [] { } |

PatternCharacter ::
SourceCharacter but not *SyntaxCharacter*

*AtomEscape*_[U, N] ::
DecimalEscape
*CharacterClassEscape*_[?U]
*CharacterEscape*_[?U]
*[+N] k GroupName*_[?U]

*CharacterEscape*_[U] ::
ControlEscape
c *ControlLetter*
0 [lookahead *∉* *DecimalDigit*]
HexEscapeSequence
*RegExpUnicodeEscapeSequence*_[?U]
*IdentityEscape*_[?U]

ControlEscape :: **one of**
f n r t v

ControlLetter :: **one of**
a b c d e f g h i j k l m n o p q r s t u v w x y z A B C D E F G H I J K L M N O
P Q R S T U V W X Y Z

*GroupSpecifier*_[U] ::
[empty]
*? GroupName*_[?U]

*GroupName*_[U] ::
*< RegExpIdentifierName*_[?U] *>*

*RegExpIdentifierName*_[U] ::
*RegExpIdentifierStart*_[?U]
*RegExpIdentifierName*_[?U] *RegExpIdentifierPart*_[?U]

*RegExpIdentifierStart*_[U] ::
UnicodeIDStart
\$
—
**** *RegExpUnicodeEscapeSequence*_[?U]

*RegExpIdentifierPart*_[U] ::
UnicodeIDContinue
\$
**** *RegExpUnicodeEscapeSequence*_[?U]
<ZWJ>
<ZWJ>

*RegExpUnicodeEscapeSequence*_[U] ::
_[+U] **u** *LeadSurrogate* **\u** *TrailSurrogate*
_[+U] **u** *LeadSurrogate*
_[+U] **u** *TrailSurrogate*
_[+U] **u** *NonSurrogate*
_[~U] **u** *Hex4Digits*
_[+U] **u** { *CodePoint* }

Each **\u** *TrailSurrogate* for which the choice of associated **u** *LeadSurrogate* is ambiguous shall be associated with the nearest possible **u** *LeadSurrogate* that would otherwise have no corresponding **\u** *TrailSurrogate*.

LeadSurrogate ::
Hex4Digits but only if the SV of *Hex4Digits* is in the inclusive range 0xD800 to 0xDBFF

TrailSurrogate ::
Hex4Digits but only if the SV of *Hex4Digits* is in the inclusive range 0xDC00 to 0xDFFF

NonSurrogate ::
Hex4Digits but only if the SV of *Hex4Digits* is not in the inclusive range 0xD800 to 0xDFFF

*IdentityEscape*_[U] ::
_[+U] *SyntaxCharacter*
_[+U] **/**
_[~U] *SourceCharacter* but not *UnicodeIDContinue*

DecimalEscape ::
NonZeroDigit *DecimalDigits*_{opt} [lookahead **≠** *DecimalDigit*]

*CharacterClassEscape*_[U] ::
d
D
s
S
w
W
_[+U] **p** { *UnicodePropertyValueExpression* }

[+U] P{ UnicodePropertyValueExpression }

UnicodePropertyValueExpression ::

UnicodePropertyName = UnicodePropertyValue

LoneUnicodePropertyNameOrValue

UnicodePropertyName ::

UnicodePropertyNameCharacters

UnicodePropertyNameCharacters ::

UnicodePropertyNameCharacter UnicodePropertyNameCharacters_{opt}

UnicodePropertyValue ::

UnicodePropertyValueCharacters

LoneUnicodePropertyNameOrValue ::

UnicodePropertyValueCharacters

UnicodePropertyValueCharacters ::

UnicodePropertyValueCharacter UnicodePropertyValueCharacters_{opt}

UnicodePropertyValueCharacter ::

UnicodePropertyNameCharacter

0

1

2

3

4

5

6

7

8

9

UnicodePropertyNameCharacter ::

ControlLetter

—

CharacterClass_[U] ::

[[lookahead ∉ { ^ }] ClassRanges_[?U]]

[^ ClassRanges_[?U]]

ClassRanges_[U] ::

[empty]

NonemptyClassRanges_[?U]

NonemptyClassRanges_[U] ::

ClassAtom_[?U]

ClassAtom_[?U] NonemptyClassRangesNoDash_[?U]

ClassAtom_[?U] – ClassAtom_[?U] ClassRanges_[?U]

*NonemptyClassRangesNoDash*_[U] ::
 *ClassAtom*_[?U]
 *ClassAtomNoDash*_[?U] *NonemptyClassRangesNoDash*_[?U]
 *ClassAtomNoDash*_[?U] – *ClassAtom*_[?U] *ClassRanges*_[?U]

*ClassAtom*_[U] ::
 –
 *ClassAtomNoDash*_[?U]

*ClassAtomNoDash*_[U] ::
 SourceCharacter but not one of \ or] or –
 \ *ClassEscape*_[?U]

*ClassEscape*_[U] ::
 b
 [+U] –
 *CharacterClassEscape*_[?U]
 *CharacterEscape*_[?U]

21.2.1.1 Static Semantics: Early Errors

Pattern :: *Disjunction*

It is a Syntax Error if *NcapturingParens* ≥ 2³² – 1.

It is a Syntax Error if *Pattern* contains multiple *GroupSpecifiers* whose enclosed *RegExpIdentifierNames* have the same *StringValue*.

QuantifierPrefix :: { *DecimalDigits* , *DecimalDigits* }

It is a Syntax Error if the MV of the first *DecimalDigits* is larger than the MV of the second *DecimalDigits*.

AtomEscape :: **k** *GroupName*

It is a Syntax Error if the enclosing *Pattern* does not contain a *GroupSpecifier* with an enclosed *RegExpIdentifierName* whose *StringValue* equals the *StringValue* of the *RegExpIdentifierName* of this production's *GroupName*.

AtomEscape :: *DecimalEscape*

It is a Syntax Error if the *CapturingGroupNumber* of *DecimalEscape* is larger than *NcapturingParens* (21.2.2.1).

NonemptyClassRanges :: *ClassAtom* – *ClassAtom* *ClassRanges*

It is a Syntax Error if *IsCharacterClass* of the first *ClassAtom* is **true** or *IsCharacterClass* of the second *ClassAtom* is **true**.

It is a Syntax Error if *IsCharacterClass* of the first *ClassAtom* is **false** and *IsCharacterClass* of the second *ClassAtom* is **false** and the *CharacterValue* of the first *ClassAtom* is larger than the *CharacterValue* of the second *ClassAtom*.

NonemptyClassRangesNoDash :: *ClassAtomNoDash* – *ClassAtom* *ClassRanges*

It is a Syntax Error if *IsCharacterClass* of *ClassAtomNoDash* is **true** or *IsCharacterClass* of *ClassAtom* is **true**.

It is a Syntax Error if *IsCharacterClass* of *ClassAtomNoDash* is **false** and *IsCharacterClass* of *ClassAtom* is **false**

and the CharacterValue of *ClassAtomNoDash* is larger than the CharacterValue of *ClassAtom*.

RegExpIdentifierStart :: \ *RegExpUnicodeEscapeSequence*

It is a Syntax Error if *SV(RegExpUnicodeEscapeSequence)* is none of "\$", or "_", or the [UTF16Encoding](#) of a code point matched by the *UnicodeIDStart* lexical grammar production.

RegExpIdentifierPart :: \ *RegExpUnicodeEscapeSequence*

It is a Syntax Error if *SV(RegExpUnicodeEscapeSequence)* is none of "\$", or "_", or the [UTF16Encoding](#) of either <ZWJ> or <ZWJ>, or the [UTF16Encoding](#) of a Unicode code point that would be matched by the *UnicodeIDContinue* lexical grammar production.

UnicodePropertyValueExpression :: *UnicodePropertyName* = *UnicodePropertyValue*

It is a Syntax Error if the [List](#) of Unicode code points that is SourceText of *UnicodePropertyName* is not identical to a [List](#) of Unicode code points that is a Unicode [property name](#) or property alias listed in the “[Property name and aliases](#)” column of [Table 54](#).

It is a Syntax Error if the [List](#) of Unicode code points that is SourceText of *UnicodePropertyValue* is not identical to a [List](#) of Unicode code points that is a value or value alias for the Unicode property or property alias given by SourceText of *UnicodePropertyName* listed in the “Property value and aliases” column of the corresponding tables [Table 56](#) or [Table 57](#).

UnicodePropertyValueExpression :: *LoneUnicodePropertyNameOrValue*

It is a Syntax Error if the [List](#) of Unicode code points that is SourceText of *LoneUnicodePropertyNameOrValue* is not identical to a [List](#) of Unicode code points that is a Unicode general category or general category alias listed in the “Property value and aliases” column of [Table 56](#), nor a binary property or binary property alias listed in the “[Property name and aliases](#)” column of [Table 55](#).

21.2.1.2 Static Semantics: CapturingGroupNumber

DecimalEscape :: *NonZeroDigit*

1. Return the MV of *NonZeroDigit*.

DecimalEscape :: *NonZeroDigit* *DecimalDigits*

1. Let *n* be the number of code points in *DecimalDigits*.
2. Return (the MV of *NonZeroDigit* × 10^{*n*}) plus the MV of *DecimalDigits*.

The definitions of “the MV of *NonZeroDigit*” and “the MV of *DecimalDigits*” are in [11.8.3](#).

21.2.1.3 Static Semantics: IsCharacterClass

ClassAtom :: –

ClassAtomNoDash :: *SourceCharacter* but not one of \ or] or –

ClassEscape :: **b**

ClassEscape :: –

ClassEscape :: *CharacterEscape*

1. Return **false**.

ClassEscape :: *CharacterClassEscape*

1. Return **true**.

21.2.1.4 Static Semantics: CharacterValue

ClassAtom :: –

1. Return the code point value of U+002D (HYPHEN-MINUS).

ClassAtomNoDash :: *SourceCharacter* but not one of \ or] or –

1. Let *ch* be the code point matched by *SourceCharacter*.
2. Return the code point value of *ch*.

ClassEscape :: **b**

1. Return the code point value of U+0008 (BACKSPACE).

ClassEscape :: –

1. Return the code point value of U+002D (HYPHEN-MINUS).

CharacterEscape :: *ControlEscape*

1. Return the code point value according to [Table 53](#).

Table 53: ControlEscape Code Point Values

ControlEscape	Code Point Value	Code Point	Unicode Name	Symbol
t	9	U+0009	CHARACTER TABULATION	<HT>
n	10	U+000A	LINE FEED (LF)	<LF>
v	11	U+000B	LINE TABULATION	<VT>
f	12	U+000C	FORM FEED (FF)	<FF>
r	13	U+000D	CARRIAGE RETURN (CR)	<CR>

CharacterEscape :: **c** *ControlLetter*

1. Let *ch* be the code point matched by *ControlLetter*.
2. Let *i* be *ch*'s code point value.
3. Return the remainder of dividing *i* by 32.

CharacterEscape :: **0** [lookahead $\notin$ *DecimalDigit*]

1. Return the code point value of U+0000 (NULL).

NOTE

\0 represents the <NUL> character and cannot be followed by a decimal digit.

CharacterEscape :: *HexEscapeSequence*

1. Return the numeric value of the code unit that is the SV of *HexEscapeSequence*.

RegExpUnicodeEscapeSequence :: **u** *LeadSurrogate* \ **u** *TrailSurrogate*

1. Let *lead* be the CharacterValue of *LeadSurrogate*.
2. Let *trail* be the CharacterValue of *TrailSurrogate*.
3. Let *cp* be **UTF16Decode**(*lead*, *trail*).
4. Return the code point value of *cp*.

RegExpUnicodeEscapeSequence :: **u** *LeadSurrogate*

1. Return the CharacterValue of *LeadSurrogate*.

RegExpUnicodeEscapeSequence :: **u** *TrailSurrogate*

1. Return the CharacterValue of *TrailSurrogate*.

RegExpUnicodeEscapeSequence :: **u** *NonSurrogate*

1. Return the CharacterValue of *NonSurrogate*.

RegExpUnicodeEscapeSequence :: **u** *Hex4Digits*

1. Return the MV of *Hex4Digits*.

RegExpUnicodeEscapeSequence :: **u** { *CodePoint* }

1. Return the MV of *CodePoint*.

LeadSurrogate :: *Hex4Digits*

TrailSurrogate :: *Hex4Digits*

NonSurrogate :: *Hex4Digits*

1. Return the MV of *HexDigits*.

CharacterEscape :: *IdentityEscape*

1. Let *ch* be the code point matched by *IdentityEscape*.
2. Return the code point value of *ch*.

21.2.1.5 Static Semantics: SourceText

UnicodePropertyNameCharacters :: *UnicodePropertyNameCharacter UnicodePropertyNameCharacters*

UnicodePropertyValueCharacters :: *UnicodePropertyValueCharacter UnicodePropertyValueCharacters*

1. Return the **List**, in source text order, of Unicode code points in the source text matched by this production.

21.2.1.6 Static Semantics: StringValue

*RegExpIdentifierName*_[U] ::

*RegExpIdentifierStart*_[?U]

*RegExpIdentifierName*_[?U] *RegExpIdentifierPart*_[?U]

1. Return the String value consisting of the sequence of code units corresponding to *RegExpIdentifierName*. In determining the sequence any occurrences of \ *RegExpUnicodeEscapeSequence* are first replaced with the code point represented by the *RegExpUnicodeEscapeSequence* and then the code points of the entire

RegExpIdentifierName are converted to code units by [UTF16Encoding](#) each code point.

21.2.2 Pattern Semantics

A regular expression pattern is converted into an internal procedure using the process described below. An implementation is encouraged to use more efficient algorithms than the ones listed below, as long as the results are the same. The internal procedure is used as the value of a *RegExp* object's `[[RegExpMatcher]]` internal slot.

A *Pattern* is either a BMP pattern or a Unicode pattern depending upon whether or not its associated flags contain a "u". A BMP pattern matches against a String interpreted as consisting of a sequence of 16-bit values that are Unicode code points in the range of the Basic Multilingual Plane. A Unicode pattern matches against a String interpreted as consisting of Unicode code points encoded using UTF-16. In the context of describing the behaviour of a BMP pattern “character” means a single 16-bit Unicode BMP code point. In the context of describing the behaviour of a Unicode pattern “character” means a UTF-16 encoded code point ([6.1.4](#)). In either context, “character value” means the numeric value of the corresponding non-encoded code point.

The syntax and semantics of *Pattern* is defined as if the source code for the *Pattern* was a [List](#) of *SourceCharacter* values where each *SourceCharacter* corresponds to a Unicode code point. If a BMP pattern contains a non-BMP *SourceCharacter* the entire pattern is encoded using UTF-16 and the individual code units of that encoding are used as the elements of the [List](#).

NOTE

For example, consider a pattern expressed in source text as the single non-BMP character U+1D11E (MUSICAL SYMBOL G CLEF). Interpreted as a Unicode pattern, it would be a single element (character) [List](#) consisting of the single code point 0x1D11E. However, interpreted as a BMP pattern, it is first UTF-16 encoded to produce a two element [List](#) consisting of the code units 0xD834 and 0xDD1E.

Patterns are passed to the *RegExp* [constructor](#) as ECMAScript String values in which non-BMP characters are UTF-16 encoded. For example, the single character MUSICAL SYMBOL G CLEF pattern, expressed as a String value, is a String of length 2 whose elements were the code units 0xD834 and 0xDD1E. So no further translation of the string would be necessary to process it as a BMP pattern consisting of two pattern characters. However, to process it as a Unicode pattern [UTF16Decode](#) must be used in producing a [List](#) consisting of a single pattern character, the code point U+1D11E.

An implementation may not actually perform such translations to or from UTF-16, but the semantics of this specification requires that the result of pattern matching be as if such translations were performed.

21.2.2.1 Notation

The descriptions below use the following variables:

Input is a [List](#) consisting of all of the characters, in order, of the String being matched by the regular expression pattern. Each character is either a code unit or a code point, depending upon the kind of pattern involved. The notation *Input*[*n*] means the *n*th character of *Input*, where *n* can range between 0 (inclusive) and *InputLength* (exclusive).

InputLength is the number of characters in *Input*.

NcapturingParens is the total number of left-capturing parentheses (i.e. the total number of *Atom* :: (*GroupSpecifier Disjunction*) Parse Nodes) in the pattern. A left-capturing parenthesis is any (pattern character that is matched by the (terminal of the *Atom* :: (*GroupSpecifier Disjunction*) production.

DotAll is **true** if the RegExp object's `[[OriginalFlags]]` internal slot contains "s" and otherwise is **false**.

IgnoreCase is **true** if the RegExp object's `[[OriginalFlags]]` internal slot contains "i" and otherwise is **false**.

Multiline is **true** if the RegExp object's `[[OriginalFlags]]` internal slot contains "m" and otherwise is **false**.

Unicode is **true** if the RegExp object's `[[OriginalFlags]]` internal slot contains "u" and otherwise is **false**.

Furthermore, the descriptions below use the following internal data structures:

A *CharSet* is a mathematical set of characters, either code units or code points depending up the state of the *Unicode* flag. "All characters" means either all code unit values or all code point values also depending upon the state of *Unicode*.

A *State* is an ordered pair (*endIndex*, *captures*) where *endIndex* is an integer and *captures* is a *List* of *NcapturingParens* values. States are used to represent partial match states in the regular expression matching algorithms. The *endIndex* is one plus the index of the last input character matched so far by the pattern, while *captures* holds the results of capturing parentheses. The n^{th} element of *captures* is either a *List* that represents the value obtained by the n^{th} set of capturing parentheses or **undefined** if the n^{th} set of capturing parentheses hasn't been reached yet. Due to backtracking, many States may be in use at any time during the matching process.

A *MatchResult* is either a State or the special token *failure* that indicates that the match failed.

A *Continuation* procedure is an internal closure (i.e. an internal procedure with some arguments already bound to values) that takes one State argument and returns a MatchResult result. If an internal closure references variables which are bound in the function that creates the closure, the closure uses the values that these variables had at the time the closure was created. The Continuation attempts to match the remaining portion (specified by the closure's already-bound arguments) of the pattern against *Input*, starting at the intermediate state given by its State argument. If the match succeeds, the Continuation returns the final State that it reached; if the match fails, the Continuation returns *failure*.

A *Matcher* procedure is an internal closure that takes two arguments — a State and a Continuation — and returns a MatchResult result. A Matcher attempts to match a middle subpattern (specified by the closure's already-bound arguments) of the pattern against *Input*, starting at the intermediate state given by its State argument. The Continuation argument should be a closure that matches the rest of the pattern. After matching the subpattern of a pattern to obtain a new State, the Matcher then calls Continuation on that new State to test if the rest of the pattern can match as well. If it can, the Matcher returns the State returned by Continuation; if not, the Matcher may try different choices at its choice points, repeatedly calling Continuation until it either succeeds or all possibilities have been exhausted.

An *AssertionTester* procedure is an internal closure that takes a State argument and returns a Boolean result. The assertion tester tests a specific condition (specified by the closure's already-bound arguments) against the current place in *Input* and returns **true** if the condition matched or **false** if not.

21.2.2.2 Pattern

The production *Pattern* :: *Disjunction* evaluates as follows:

1. Evaluate *Disjunction* with +1 as its *direction* argument to obtain a Matcher *m*.
2. Return an internal closure that takes two arguments, a String *str* and an integer *index*, and performs the following steps:
 - a. **Assert**: *index* ≤ the length of *str*.
 - b. If *Unicode* is **true**, let *Input* be a *List* consisting of the sequence of code points of *str* interpreted as a UTF-16 encoded (6.1.4) Unicode string. Otherwise, let *Input* be a *List* consisting of the sequence of code units that are the elements of *str*. *Input* will be used throughout the algorithms in 21.2.2. Each element of *Input* is considered to be a character.
 - c. Let *InputLength* be the number of characters contained in *Input*. This variable will be used throughout the

algorithms in 21.2.2.

- d. Let *listIndex* be the index into *Input* of the character that was obtained from element *index* of *str*.
- e. Let *c* be a Continuation that always returns its State argument as a successful MatchResult.
- f. Let *cap* be a List of *NcapturingParens* **undefined** values, indexed 1 through *NcapturingParens*.
- g. Let *x* be the State (*listIndex*, *cap*).
- h. Call *m(x, c)* and return its result.

NOTE

A Pattern evaluates (“compiles”) to an internal procedure value. *RegExpBuiltinExec* can then apply this procedure to a String and an offset within the String to determine whether the pattern would match starting at exactly that offset within the String, and, if it does match, what the values of the capturing parentheses would be. The algorithms in 21.2.2 are designed so that compiling a pattern may throw a **SyntaxError** exception; on the other hand, once the pattern is successfully compiled, applying the resulting internal procedure to find a match in a String cannot throw an exception (except for any host-defined exceptions that can occur anywhere such as out-of-memory).

21.2.2.3 Disjunction

With parameter *direction*.

The production *Disjunction* :: *Alternative* evaluates as follows:

1. Evaluate *Alternative* with argument *direction* to obtain a Matcher *m*.
2. Return *m*.

The production *Disjunction* :: *Alternative* | *Disjunction* evaluates as follows:

1. Evaluate *Alternative* with argument *direction* to obtain a Matcher *m1*.
2. Evaluate *Disjunction* with argument *direction* to obtain a Matcher *m2*.
3. Return an internal Matcher closure that takes two arguments, a State *x* and a Continuation *c*, and performs the following steps when evaluated:
 - a. Call *m1(x, c)* and let *r* be its result.
 - b. If *r* is not **failure**, return *r*.
 - c. Call *m2(x, c)* and return its result.

NOTE

The | regular expression operator separates two alternatives. The pattern first tries to match the left *Alternative* (followed by the sequel of the regular expression); if it fails, it tries to match the right *Disjunction* (followed by the sequel of the regular expression). If the left *Alternative*, the right *Disjunction*, and the sequel all have choice points, all choices in the sequel are tried before moving on to the next choice in the left *Alternative*. If choices in the left *Alternative* are exhausted, the right *Disjunction* is tried instead of the left *Alternative*. Any capturing parentheses inside a portion of the pattern skipped by | produce **undefined** values instead of Strings. Thus, for example,

```
/a|ab/.exec("abc")
```

returns the result **"a"** and not **"ab"**. Moreover,

```
/((a)|(ab))((c)|(bc))/.exec("abc")
```

returns the array

```
["abc", "a", "a", undefined, "bc", undefined, "bc"]
```


and not

```
["abc", "ab", undefined, "ab", "c", "c", undefined]
```

The order in which the two alternatives are tried is independent of the value of *direction*.

21.2.2.4 Alternative

With parameter *direction*.

The production *Alternative* :: [empty] evaluates as follows:

1. Return a Matcher that takes two arguments, a State *x* and a Continuation *c*, and returns the result of calling *c*(*x*).

The production *Alternative* :: *Alternative Term* evaluates as follows:

1. Evaluate *Alternative* with argument *direction* to obtain a Matcher *m1*.
2. Evaluate *Term* with argument *direction* to obtain a Matcher *m2*.
3. If *direction* is equal to +1, then
 - a. Return an internal Matcher closure that takes two arguments, a State *x* and a Continuation *c*, and performs the following steps when evaluated:
 - i. Let *d* be a Continuation that takes a State argument *y* and returns the result of calling *m2*(*y*, *c*).
 - ii. Call *m1*(*x*, *d*) and return its result.
4. Else,
 - a. **Assert:** *direction* is equal to -1.
 - b. Return an internal Matcher closure that takes two arguments, a State *x* and a Continuation *c*, and performs the following steps when evaluated:
 - i. Let *d* be a Continuation that takes a State argument *y* and returns the result of calling *m1*(*y*, *c*).
 - ii. Call *m2*(*x*, *d*) and return its result.

NOTE

Consecutive *Terms* try to simultaneously match consecutive portions of *Input*. When *direction* is equal to +1, if the left *Alternative*, the right *Term*, and the sequel of the regular expression all have choice points, all choices in the sequel are tried before moving on to the next choice in the right *Term*, and all choices in the right *Term* are tried before moving on to the next choice in the left *Alternative*. When *direction* is equal to -1, the evaluation order of *Alternative* and *Term* are reversed.

21.2.2.5 Term

With parameter *direction*.

The production *Term* :: *Assertion* evaluates as follows:

1. Return an internal Matcher closure that takes two arguments, a State *x* and a Continuation *c*, and performs the following steps when evaluated:
 - a. Evaluate *Assertion* to obtain an AssertionTester *t*.
 - b. Call *t*(*x*) and let *r* be the resulting Boolean value.
 - c. If *r* is **false**, return failure.
 - d. Call *c*(*x*) and return its result.

NOTE

The AssertionTester is independent of *direction*.

The production *Term* :: *Atom* evaluates as follows:

1. Return the Matcher that is the result of evaluating *Atom* with argument *direction*.

The production *Term* :: *Atom Quantifier* evaluates as follows:

1. Evaluate *Atom* with argument *direction* to obtain a Matcher *m*.
2. Evaluate *Quantifier* to obtain the three results: an integer *min*, an integer (or ∞) *max*, and Boolean *greedy*.
3. **Assert:** If *max* is finite, then *max* is not less than *min*.
4. Let *parenIndex* be the number of left-capturing parentheses in the entire regular expression that occur to the left of this *Term*. This is the total number of *Atom* :: (*GroupSpecifier Disjunction*) Parse Nodes prior to or enclosing this *Term*.
5. Let *parenCount* be the number of left-capturing parentheses in *Atom*. This is the total number of *Atom* :: (*GroupSpecifier Disjunction*) Parse Nodes enclosed by *Atom*.
6. Return an internal Matcher closure that takes two arguments, a State *x* and a Continuation *c*, and performs the following steps when evaluated:
 - a. Call RepeatMatcher(*m*, *min*, *max*, *greedy*, *x*, *c*, *parenIndex*, *parenCount*) and return its result.

21.2.2.5.1 Runtime Semantics: RepeatMatcher (*m*, *min*, *max*, *greedy*, *x*, *c*, *parenIndex*, *parenCount*)

The abstract operation RepeatMatcher takes eight parameters, a Matcher *m*, an integer *min*, an integer (or ∞) *max*, a Boolean *greedy*, a State *x*, a Continuation *c*, an integer *parenIndex*, and an integer *parenCount*, and performs the following steps:

1. If *max* is zero, return *c*(*x*).
2. Let *d* be an internal Continuation closure that takes one State argument *y* and performs the following steps when evaluated:
 - a. If *min* is zero and *y*'s *endIndex* is equal to *x*'s *endIndex*, return failure.
 - b. If *min* is zero, let *min2* be zero; otherwise let *min2* be *min* - 1.
 - c. If *max* is ∞ , let *max2* be ∞ ; otherwise let *max2* be *max* - 1.
 - d. Call RepeatMatcher(*m*, *min2*, *max2*, *greedy*, *y*, *c*, *parenIndex*, *parenCount*) and return its result.
3. Let *cap* be a copy of *x*'s *captures List*.
4. For each integer *k* that satisfies *parenIndex* < *k* and *k* ≤ *parenIndex* + *parenCount*, set *cap*[*k*] to **undefined**.
5. Let *e* be *x*'s *endIndex*.
6. Let *xr* be the State (*e*, *cap*).
7. If *min* is not zero, return *m*(*xr*, *d*).
8. If *greedy* is **false**, then
 - a. Call *c*(*x*) and let *z* be its result.
 - b. If *z* is not failure, return *z*.
 - c. Call *m*(*xr*, *d*) and return its result.
9. Call *m*(*xr*, *d*) and let *z* be its result.
10. If *z* is not failure, return *z*.
11. Call *c*(*x*) and return its result.

NOTE 1

An *Atom* followed by a *Quantifier* is repeated the number of times specified by the *Quantifier*. A *Quantifier* can be non-

greedy, in which case the *Atom* pattern is repeated as few times as possible while still matching the sequel, or it can be greedy, in which case the *Atom* pattern is repeated as many times as possible while still matching the sequel. The *Atom* pattern is repeated rather than the input character sequence that it matches, so different repetitions of the *Atom* can match different input substrings.

NOTE 2

If the *Atom* and the sequel of the regular expression all have choice points, the *Atom* is first matched as many (or as few, if non-greedy) times as possible. All choices in the sequel are tried before moving on to the next choice in the last repetition of *Atom*. All choices in the last (n^{th}) repetition of *Atom* are tried before moving on to the next choice in the next-to-last ($(n - 1)^{\text{st}}$) repetition of *Atom*; at which point it may turn out that more or fewer repetitions of *Atom* are now possible; these are exhausted (again, starting with either as few or as many as possible) before moving on to the next choice in the $(n - 1)^{\text{st}}$ repetition of *Atom* and so on.

Compare

```
/a[a-z]{2,4}/.exec("abcdefghi")
```

which returns **"abcde"** with

```
/a[a-z]{2,4}?/.exec("abcdefghi")
```

which returns **"abc"**.

Consider also

```
/(aa|aabaac|ba|b|c)*/.exec("aabaac")
```

which, by the choice point ordering above, returns the array

```
["aaba", "ba"]
```

and not any of:

```
["aabaac", "aabaac"]
```

```
["aabaac", "c"]
```

The above ordering of choice points can be used to write a regular expression that calculates the greatest common divisor of two numbers (represented in unary notation). The following example calculates the gcd of 10 and 15:

```
"aaaaaaaaaa,aaaaaaaaaaaaaaaa".replace(/^(a+)\1*,\1+$/, "$1")
```

which returns the gcd in unary notation **"aaaaa"**.

NOTE 3

Step 4 of the RepeatMatcher clears *Atom*'s captures each time *Atom* is repeated. We can see its behaviour in the regular expression

```
/(z)((a)?(b)?(c))*/.exec("zaacbbbcac")
```

which returns the array

```
["zaacbbbcac", "z", "ac", "a", undefined, "c"]
```

and not

```
["zaacbbbcac", "z", "ac", "a", "bbb", "c"]
```

because each iteration of the outermost ***** clears all captured Strings contained in the quantified *Atom*, which in this case includes capture Strings numbered 2, 3, 4, and 5.

NOTE 4

Step 1 of the RepeatMatcher's *d* closure states that, once the minimum number of repetitions has been satisfied, any more expansions of *Atom* that match the empty character sequence are not considered for further repetitions. This prevents the regular expression engine from falling into an infinite loop on patterns such as:

```
/(a*)*/.exec("b")
```

or the slightly more complicated:

```
/(a*)b\1+/.exec("baaaac")
```

which returns the array

```
["b", ""]
```

21.2.2.6 Assertion

The production *Assertion* :: **^** evaluates as follows:

1. Return an internal AssertionTester closure that takes a State argument *x* and performs the following steps when evaluated:
 - a. Let *e* be *x*'s *endIndex*.
 - b. If *e* is zero, return **true**.
 - c. If *Multiline* is **false**, return **false**.
 - d. If the character *Input*[*e* - 1] is one of *LineTerminator*, return **true**.
 - e. Return **false**.

NOTE

Even when the **y** flag is used with a pattern, **^** always matches only at the beginning of *Input*, or (if *Multiline* is **true**) at the beginning of a line.

The production *Assertion* :: **\$** evaluates as follows:

1. Return an internal AssertionTester closure that takes a State argument *x* and performs the following steps when evaluated:
 - a. Let *e* be *x*'s *endIndex*.
 - b. If *e* is equal to *InputLength*, return **true**.
 - c. If *Multiline* is **false**, return **false**.
 - d. If the character *Input*[*e*] is one of *LineTerminator*, return **true**.
 - e. Return **false**.

The production *Assertion* :: **\b** evaluates as follows:

1. Return an internal AssertionTester closure that takes a State argument *x* and performs the following steps when evaluated:
 - a. Let *e* be *x*'s *endIndex*.
 - b. Call *IsWordChar*(*e* - 1) and let *a* be the Boolean result.

- c. Call `IsWordChar(e)` and let *b* be the Boolean result.
- d. If *a* is **true** and *b* is **false**, return **true**.
- e. If *a* is **false** and *b* is **true**, return **true**.
- f. Return **false**.

The production `Assertion :: \ B` evaluates as follows:

1. Return an internal `AssertionTester` closure that takes a State argument *x* and performs the following steps when evaluated:
 - a. Let *e* be *x*'s *endIndex*.
 - b. Call `IsWordChar(e - 1)` and let *a* be the Boolean result.
 - c. Call `IsWordChar(e)` and let *b* be the Boolean result.
 - d. If *a* is **true** and *b* is **false**, return **false**.
 - e. If *a* is **false** and *b* is **true**, return **false**.
 - f. Return **true**.

The production `Assertion :: ( ? = Disjunction )` evaluates as follows:

1. Evaluate *Disjunction* with +1 as its *direction* argument to obtain a Matcher *m*.
2. Return an internal Matcher closure that takes two arguments, a State *x* and a Continuation *c*, and performs the following steps:
 - a. Let *d* be a Continuation that always returns its State argument as a successful `MatchResult`.
 - b. Call `m(x, d)` and let *r* be its result.
 - c. If *r* is **failure**, return **failure**.
 - d. Let *y* be *r*'s State.
 - e. Let *cap* be *y*'s *captures List*.
 - f. Let *xe* be *x*'s *endIndex*.
 - g. Let *z* be the State (*xe*, *cap*).
 - h. Call `c(z)` and return its result.

The production `Assertion :: ( ? ! Disjunction )` evaluates as follows:

1. Evaluate *Disjunction* with +1 as its *direction* argument to obtain a Matcher *m*.
2. Return an internal Matcher closure that takes two arguments, a State *x* and a Continuation *c*, and performs the following steps:
 - a. Let *d* be a Continuation that always returns its State argument as a successful `MatchResult`.
 - b. Call `m(x, d)` and let *r* be its result.
 - c. If *r* is not **failure**, return **failure**.
 - d. Call `c(x)` and return its result.

The production `Assertion :: ( ? <= Disjunction )` evaluates as follows:

1. Evaluate *Disjunction* with -1 as its *direction* argument to obtain a Matcher *m*.
2. Return an internal Matcher closure that takes two arguments, a State *x* and a Continuation *c*, and performs the following steps:
 - a. Let *d* be a Continuation that always returns its State argument as a successful `MatchResult`.
 - b. Call `m(x, d)` and let *r* be its result.
 - c. If *r* is **failure**, return **failure**.
 - d. Let *y* be *r*'s State.
 - e. Let *cap* be *y*'s *captures List*.
 - f. Let *xe* be *x*'s *endIndex*.

- g. Let z be the State (xe , cap).
- h. Call $c(z)$ and return its result.

The production $Assertion :: (? <! Disjunction)$ evaluates as follows:

1. Evaluate $Disjunction$ with -1 as its *direction* argument to obtain a Matcher m .
2. Return an internal Matcher closure that takes two arguments, a State x and a Continuation c , and performs the following steps:
 - a. Let d be a Continuation that always returns its State argument as a successful MatchResult.
 - b. Call $m(x, d)$ and let r be its result.
 - c. If r is not **failure**, return **failure**.
 - d. Call $c(x)$ and return its result.

21.2.2.6.1 Runtime Semantics: WordCharacters ()

The abstract operation WordCharacters performs the following steps:

1. Let A be a set of characters containing the sixty-three characters:

a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
0	1	2	3	4	5	6	7	8	9	_															

2. Let U be an empty set.
3. For each character c not in set A where $Canonicalize(c)$ is in A , add c to U .
4. **Assert**: Unless *Unicode* and *IgnoreCase* are both **true**, U is empty.
5. Add the characters in set U to set A .
6. Return A .

21.2.2.6.2 Runtime Semantics: IsWordChar (e)

The abstract operation IsWordChar takes an integer parameter e and performs the following steps:

1. If e is -1 or e is *InputLength*, return **false**.
2. Let c be the character $Input[e]$.
3. Let $wordChars$ be the result of ! $WordCharacters()$.
4. If c is in $wordChars$, return **true**.
5. Return **false**.

21.2.2.7 Quantifier

The production $Quantifier :: QuantifierPrefix$ evaluates as follows:

1. Evaluate $QuantifierPrefix$ to obtain the two results: an integer *min* and an integer (or ∞) *max*.
2. Return the three results *min*, *max*, and **true**.

The production $Quantifier :: QuantifierPrefix ?$ evaluates as follows:

1. Evaluate $QuantifierPrefix$ to obtain the two results: an integer *min* and an integer (or ∞) *max*.
2. Return the three results *min*, *max*, and **false**.

The production *QuantifierPrefix* :: * evaluates as follows:

1. Return the two results 0 and ∞ .

The production *QuantifierPrefix* :: + evaluates as follows:

1. Return the two results 1 and ∞ .

The production *QuantifierPrefix* :: ? evaluates as follows:

1. Return the two results 0 and 1.

The production *QuantifierPrefix* :: { *DecimalDigits* } evaluates as follows:

1. Let *i* be the MV of *DecimalDigits* (see 11.8.3).
2. Return the two results *i* and *i*.

The production *QuantifierPrefix* :: { *DecimalDigits* , } evaluates as follows:

1. Let *i* be the MV of *DecimalDigits*.
2. Return the two results *i* and ∞ .

The production *QuantifierPrefix* :: { *DecimalDigits* , *DecimalDigits* } evaluates as follows:

1. Let *i* be the MV of the first *DecimalDigits*.
2. Let *j* be the MV of the second *DecimalDigits*.
3. Return the two results *i* and *j*.

21.2.2.8 Atom

With parameter *direction*.

The production *Atom* :: *PatternCharacter* evaluates as follows:

1. Let *ch* be the character matched by *PatternCharacter*.
2. Let *A* be a one-element CharSet containing the character *ch*.
3. Call *CharacterSetMatcher*(*A*, **false**, *direction*) and return its Matcher result.

The production *Atom* :: . evaluates as follows:

1. If *DotAll* is **true**, then
 - a. Let *A* be the set of all characters.
2. Otherwise, let *A* be the set of all characters except *LineTerminator*.
3. Call *CharacterSetMatcher*(*A*, **false**, *direction*) and return its Matcher result.

The production *Atom* :: \ *AtomEscape* evaluates as follows:

1. Return the Matcher that is the result of evaluating *AtomEscape* with argument *direction*.

The production *Atom* :: *CharacterClass* evaluates as follows:

1. Evaluate *CharacterClass* to obtain a CharSet *A* and a Boolean *invert*.
2. Call *CharacterSetMatcher*(*A*, *invert*, *direction*) and return its Matcher result.

The production *Atom* :: (*GroupSpecifier Disjunction*) evaluates as follows:

1. Evaluate *Disjunction* with argument *direction* to obtain a Matcher *m*.
2. Let *parenIndex* be the number of left-capturing parentheses in the entire regular expression that occur to the left of this *Atom*. This is the total number of *Atom* :: (*GroupSpecifier Disjunction*) Parse Nodes prior to or enclosing this *Atom*.
3. Return an internal Matcher closure that takes two arguments, a State *x* and a Continuation *c*, and performs the following steps:
 - a. Let *d* be an internal Continuation closure that takes one State argument *y* and performs the following steps:
 - i. Let *cap* be a copy of *y*'s *captures List*.
 - ii. Let *xe* be *x*'s *endIndex*.
 - iii. Let *ye* be *y*'s *endIndex*.
 - iv. If *direction* is equal to +1, then
 1. Assert: $xe \leq ye$.
 2. Let *s* be a new *List* whose elements are the characters of *Input* at indices *xe* (inclusive) through *ye* (exclusive).
 - v. Else,
 1. Assert: *direction* is equal to -1.
 2. Assert: $ye \leq xe$.
 3. Let *s* be a new *List* whose elements are the characters of *Input* at indices *ye* (inclusive) through *xe* (exclusive).
 - vi. Set *cap*[*parenIndex* + 1] to *s*.
 - vii. Let *z* be the State (*ye*, *cap*).
 - viii. Call *c*(*z*) and return its result.
 - b. Call *m*(*x*, *d*) and return its result.

The production *Atom* :: (? : *Disjunction*) evaluates as follows:

1. Return the Matcher that is the result of evaluating *Disjunction* with argument *direction*.

21.2.2.8.1 Runtime Semantics: CharacterSetMatcher (*A*, *invert*, *direction*)

The abstract operation CharacterSetMatcher takes three arguments, a CharSet *A*, a Boolean flag *invert*, and an integer *direction*, and performs the following steps:

1. Return an internal Matcher closure that takes two arguments, a State *x* and a Continuation *c*, and performs the following steps when evaluated:
 - a. Let *e* be *x*'s *endIndex*.
 - b. Let *f* be *e* + *direction*.
 - c. If $f < 0$ or $f > \text{InputLength}$, return failure.
 - d. Let *index* be $\min(e, f)$.
 - e. Let *ch* be the character *Input*[*index*].
 - f. Let *cc* be *Canonicalize*(*ch*).
 - g. If *invert* is **false**, then
 - i. If there does not exist a member *a* of set *A* such that *Canonicalize*(*a*) is *cc*, return failure.
 - h. Else,
 - i. Assert: *invert* is **true**.
 - ii. If there exists a member *a* of set *A* such that *Canonicalize*(*a*) is *cc*, return failure.
 - i. Let *cap* be *x*'s *captures List*.
 - j. Let *y* be the State (*f*, *cap*).

- k. Call `c(y)` and return its result.

21.2.2.8.2 Runtime Semantics: Canonicalize (*ch*)

The abstract operation Canonicalize takes a character parameter *ch* and performs the following steps:

1. If *IgnoreCase* is **false**, return *ch*.
2. If *Unicode* is **true**, then
 - a. If the file CaseFolding.txt of the Unicode Character Database provides a simple or common case folding mapping for *ch*, return the result of applying that mapping to *ch*.
 - b. Return *ch*.
3. Else,
 - a. **Assert**: *ch* is a UTF-16 code unit.
 - b. Let *s* be the String value consisting of the single code unit *ch*.
 - c. Let *u* be the same result produced as if by performing the algorithm for **String.prototype.toUpperCase** using *s* as the **this** value.
 - d. **Assert**: **Type**(*u*) is String.
 - e. If *u* does not consist of a single code unit, return *ch*.
 - f. Let *cu* be *u*'s single code unit element.
 - g. If the numeric value of *ch* $\geq$ 128 and the numeric value of *cu* $<$ 128, return *ch*.
 - h. Return *cu*.

NOTE 1

Parentheses of the form (*Disjunction*) serve both to group the components of the *Disjunction* pattern together and to save the result of the match. The result can be used either in a backreference (\ followed by a nonzero decimal number), referenced in a replace String, or returned as part of an array from the regular expression matching internal procedure. To inhibit the capturing behaviour of parentheses, use the form (?: *Disjunction*) instead.

NOTE 2

The form (?: *Disjunction*) specifies a zero-width positive lookahead. In order for it to succeed, the pattern inside *Disjunction* must match at the current position, but the current position is not advanced before matching the sequel. If *Disjunction* can match at the current position in several ways, only the first one is tried. Unlike other regular expression operators, there is no backtracking into a (?: form (this unusual behaviour is inherited from Perl). This only matters when the *Disjunction* contains capturing parentheses and the sequel of the pattern contains backreferences to those captures.

For example,

```
/(?= (a+) ) /.exec ("baaabac")
```

matches the empty String immediately after the first **b** and therefore returns the array:

```
["", "aaa"]
```

To illustrate the lack of backtracking into the lookahead, consider:

```
/(?= (a+) ) a*b\1 /.exec ("baaabac")
```

This expression returns

```
["aba", "a"]
```

and not:

```
["aaaba", "a"]
```

NOTE 3

The form **(?! Disjunction)** specifies a zero-width negative lookahead. In order for it to succeed, the pattern inside *Disjunction* must fail to match at the current position. The current position is not advanced before matching the sequel. *Disjunction* can contain capturing parentheses, but backreferences to them only make sense from within *Disjunction* itself. Backreferences to these capturing parentheses from elsewhere in the pattern always return **undefined** because the negative lookahead must fail for the pattern to succeed. For example,

```
/(.*?)a(?! (a+)b\2c)\2(.*)/.exec("baaabaac")
```

looks for an **a** not immediately followed by some positive number *n* of **a**'s, a **b**, another *n* **a**'s (specified by the first **\2**) and a **c**. The second **\2** is outside the negative lookahead, so it matches against **undefined** and therefore always succeeds. The whole expression returns the array:

```
["baaabaac", "ba", undefined, "abaac"]
```

NOTE 4

In case-insignificant matches when *Unicode* is **true**, all characters are implicitly case-folded using the simple mapping provided by the Unicode standard immediately before they are compared. The simple mapping always maps to a single code point, so it does not map, for example, "ß" (U+00DF) to "SS". It may however map a code point outside the Basic Latin range to a character within, for example, "ſ" (U+017F) to "s". Such characters are not mapped if *Unicode* is **false**. This prevents Unicode code points such as U+017F and U+212A from matching regular expressions such as **/[a-z]/i**, but they will match **/[a-z]/ui**.

21.2.2.8.3 Runtime Semantics: UnicodeMatchProperty (*p*)

The abstract operation UnicodeMatchProperty takes a parameter *p* that is a **List** of Unicode code points and performs the following steps:

1. **Assert:** *p* is a **List** of Unicode code points that is identical to a **List** of Unicode code points that is a Unicode **property name** or property alias listed in the “**Property name** and aliases” column of **Table 54** or **Table 55**.
2. Let *c* be the canonical **property name** of *p* as given in the “**Canonical property name**” column of the corresponding row.
3. Return the **List** of Unicode code points of *c*.

Implementations must support the Unicode property names and aliases listed in **Table 54** and **Table 55**. To ensure interoperability, implementations must not support any other property names or aliases.

NOTE 1

For example, **Script_Extensions** (**property name**) and **scx** (**property alias**) are valid, but **script_extensions** or **Scx** aren't.

NOTE 2

The listed properties form a superset of what **UTS18 RL1.2** requires.

Table 54: Non-binary Unicode property aliases and their canonical property names

Property name and aliases

Canonical property name

Property name and aliases	Canonical property name
General_Category gc	General_Category
Script sc	Script
Script_Extensions scx	Script_Extensions

Table 55: Binary Unicode property aliases and their canonical property names

Property name and aliases	Canonical property name
ASCII	ASCII
ASCII_Hex_Digit AHex	ASCII_Hex_Digit
Alphabetic Alpha	Alphabetic
Any	Any
Assigned	Assigned
Bidi_Control Bidi_C	Bidi_Control
Bidi_Mirrored Bidi_M	Bidi_Mirrored
Case_Ignorable CI	Case_Ignorable
Cased	Cased
Changes_When_Casefolded CWCF	Changes_When_Casefolded
Changes_When_Casemapped CWCM	Changes_When_Casemapped
Changes_When_Lowercased CWL	Changes_When_Lowercased
Changes_When_NFKC_Casefolded CWKCF	Changes_When_NFKC_Casefolded

Property name and aliases	Canonical property name
Changes_When_Titlecased CWT	Changes_When_Titlecased
Changes_When_Uppercased CWU	Changes_When_Uppercased
Dash	Dash
Default_Ignorable_Code_Point DI	Default_Ignorable_Code_Point
Deprecated Dep	Deprecated
Diacritic Dia	Diacritic
Emoji	Emoji
Emoji_Component	Emoji_Component
Emoji_Modifier	Emoji_Modifier
Emoji_Modifier_Base	Emoji_Modifier_Base
Emoji_Presentation	Emoji_Presentation
Extended_Pictographic	Extended_Pictographic
Extender Ext	Extender
Grapheme_Base Gr_Base	Grapheme_Base
Grapheme_Extend Gr_Ext	Grapheme_Extend
Hex_Digit Hex	Hex_Digit
IDS_Binary_Operator IDSB	IDS_Binary_Operator
IDS_Tertiary_Operator IDST	IDS_Tertiary_Operator
ID_Continue IDC	ID_Continue

Property name and aliases	Canonical property name
ID_Start IDS	ID_Start
Ideographic Ideo	Ideographic
Join_Control Join_C	Join_Control
Logical_Order_Exception LOE	Logical_Order_Exception
Lowercase Lower	Lowercase
Math	Math
Noncharacter_Code_Point NChar	Noncharacter_Code_Point
Pattern_Syntax Pat_Syn	Pattern_Syntax
Pattern_White_Space Pat_WS	Pattern_White_Space
Quotation_Mark QMark	Quotation_Mark
Radical	Radical
Regional_Indicator RI	Regional_Indicator
Sentence_Terminal STerm	Sentence_Terminal
Soft_Dotted SD	Soft_Dotted
Terminal_Punctuation Term	Terminal_Punctuation
Unified_Ideograph UIdeo	Unified_Ideograph

Uppercase
Upper

Uppercase

Variation_Selector
VS

Variation_Selector

White_Space
space

White_Space

XID_Continue
XIDC

XID_Continue

XID_Start
XIDS

XID_Start

21.2.2.8.4 Runtime Semantics: UnicodeMatchPropertyValue (*p*, *v*)

The abstract operation UnicodeMatchPropertyValue takes two parameters *p* and *v*, each of which is a [List](#) of Unicode code points, and performs the following steps:

1. **Assert:** *p* is a [List](#) of Unicode code points that is identical to a [List](#) of Unicode code points that is a canonical, unaliased Unicode [property name](#) listed in the “Canonical [property name](#)” column of [Table 54](#).
2. **Assert:** *v* is a [List](#) of Unicode code points that is identical to a [List](#) of Unicode code points that is a property value or property value alias for Unicode property *p* listed in the “Property value and aliases” column of [Table 56](#) or [Table 57](#).
3. Let *value* be the canonical property value of *v* as given in the “Canonical property value” column of the corresponding row.
4. Return the [List](#) of Unicode code points of *value*.

Implementations must support the Unicode property value names and aliases listed in [Table 56](#) and [Table 57](#). To ensure interoperability, implementations must not support any other property value names or aliases.

NOTE 1

For example, **Xpeo** and **Old_Persian** are valid **Script_Extensions** values, but **xpeo** and **Old Persian** aren't.

NOTE 2

This algorithm differs from [the matching rules for symbolic values listed in UAX44](#): case, [white space](#), U+002D (HYPHEN-MINUS), and U+005F (LOW LINE) are not ignored, and the **Is** prefix is not supported.

Table 56: Value aliases and canonical values for the Unicode property [General_Category](#)

Property value and aliases

Canonical property value

Cased_Letter
LC

Cased_Letter

Property value and aliases	Canonical property value
Close_Punctuation Pe	Close_Punctuation
Connector_Punctuation Pc	Connector_Punctuation
Control Cc cntrl	Control
Currency_Symbol Sc	Currency_Symbol
Dash_Punctuation Pd	Dash_Punctuation
Decimal_Number Nd digit	Decimal_Number
Enclosing_Mark Me	Enclosing_Mark
Final_Punctuation Pf	Final_Punctuation
Format Cf	Format
Initial_Punctuation Pi	Initial_Punctuation
Letter L	Letter
Letter_Number Nl	Letter_Number
Line_Separator Zl	Line_Separator
Lowercase_Letter Ll	Lowercase_Letter

Property value and aliases	Canonical property value
Mark M Combining_Mark	Mark
Math_Symbol Sm	Math_Symbol
Modifier_Letter Lm	Modifier_Letter
Modifier_Symbol Sk	Modifier_Symbol
Nonspacing_Mark Mn	Nonspacing_Mark
Number N	Number
Open_Punctuation Ps	Open_Punctuation
Other C	Other
Other_Letter Lo	Other_Letter
Other_Number No	Other_Number
Other_Punctuation Po	Other_Punctuation
Other_Symbol So	Other_Symbol
Paragraph_Separator Zp	Paragraph_Separator
Private_Use Co	Private_Use

Property value and aliases	Canonical property value
Punctuation P punct	Punctuation
Separator Z	Separator
Space_Separator Zs	Space_Separator
Spacing_Mark Mc	Spacing_Mark
Surrogate Cs	Surrogate
Symbol S	Symbol
Titlecase_Letter Lt	Titlecase_Letter
Unassigned Cn	Unassigned
Uppercase_Letter Lu	Uppercase_Letter

Table 57: Value aliases and canonical values for the Unicode properties [Script](#) and [Script_Extensions](#)

Property value and aliases	Canonical property value
Adlam Adlm	Adlam
Ahom Ahom	Ahom
Anatolian_Hieroglyphs Hluw	Anatolian_Hieroglyphs
Arabic Arab	Arabic

Property value and aliases

Canonical property value

Armenian
Arm

Armenian

Avestan
Avst

Avestan

Balinese
Bali

Balinese

Bamum
Bamu

Bamum

Bassa_Vah
Bass

Bassa_Vah

Batak
Batk

Batak

Bengali
Beng

Bengali

Bhaiksuki
Bhks

Bhaiksuki

Bopomofo
Bopo

Bopomofo

Brahmi
Brah

Brahmi

Braille
Brai

Braille

Buginese
Bugi

Buginese

Buhid
Buhd

Buhid

Canadian_Aboriginal
Cans

Canadian_Aboriginal

Carian
Cari

Carian

Property value and aliases	Canonical property value
Caucasian_Albanian Aghb	Caucasian_Albanian
Chakma Cakm	Chakma
Cham Cham	Cham
Cherokee Cher	Cherokee
Common Zyyy	Common
Coptic Copt Qaac	Coptic
Cuneiform Xsux	Cuneiform
Cypriot Cprt	Cypriot
Cyrillic Cyrl	Cyrillic
Deseret Dsrt	Deseret
Devanagari Deva	Devanagari
Dogra Dogr	Dogra
Duployan Dupl	Duployan
Egyptian_Hieroglyphs Egyp	Egyptian_Hieroglyphs
Elbasan Elba	Elbasan

Property value and aliases

Canonical property value

Ethiopic
Ethi

Ethiopic

Georgian
Geor

Georgian

Glagolitic
Glag

Glagolitic

Gothic
Goth

Gothic

Grantha
Gran

Grantha

Greek
Grek

Greek

Gujarati
Gujr

Gujarati

Gunjala_Gondi
Gong

Gunjala_Gondi

Gurmukhi
Guru

Gurmukhi

Han
Hani

Han

Hangul
Hang

Hangul

Hanifi_Rohingya
Rohg

Hanifi_Rohingya

Hanunoo
Hano

Hanunoo

Hatran
Hatr

Hatran

Hebrew
Hebr

Hebrew

Property value and aliases	Canonical property value
Hiragana Hira	Hiragana
Imperial_Aramaic Armi	Imperial_Aramaic
Inherited Zinh Qaai	Inherited
Inscriptional_Pahlavi Phli	Inscriptional_Pahlavi
Inscriptional_Parthian Prti	Inscriptional_Parthian
Javanese Java	Javanese
Kaithi Kthi	Kaithi
Kannada Knda	Kannada
Katakana Kana	Katakana
Kayah_Li Kali	Kayah_Li
Kharoshthi Khar	Kharoshthi
Khmer Khmr	Khmer
Khojki Khoj	Khojki
Khudawadi Sind	Khudawadi
Lao Laoo	Lao

Property value and aliases

Canonical property value

Latin
Latn

Latin

Lepcha
Lepc

Lepcha

Limbu
Limb

Limbu

Linear_A
Lina

Linear_A

Linear_B
Linb

Linear_B

Lisu
Lisu

Lisu

Lycian
Lyci

Lycian

Lydian
Lydi

Lydian

Mahajani
Mahj

Mahajani

Makasar
Maka

Makasar

Malayalam
Mlym

Malayalam

Mandaic
Mand

Mandaic

Manichaeen
Mani

Manichaeen

Marchen
Marc

Marchen

Medefaidrin
Medf

Medefaidrin

Property value and aliases	Canonical property value
Masaram_Gondi Gonm	Masaram_Gondi
Meetei_Mayek Mtei	Meetei_Mayek
Mende_Kikakui Mend	Mende_Kikakui
Meroitic_Cursive Merc	Meroitic_Cursive
Meroitic_Hieroglyphs Mero	Meroitic_Hieroglyphs
Miao Plrd	Miao
Modi Modi	Modi
Mongolian Mong	Mongolian
Mro Mroo	Mro
Multani Mult	Multani
Myanmar Mymr	Myanmar
Nabataean Nbat	Nabataean
New_Tai_Lue Talu	New_Tai_Lue
Newa Newa	Newa
Nko Nkoo	Nko

Property value and aliases

Canonical property value

Nushu
Nshu

Nushu

Ogham
Ogam

Ogham

Ol_Chiki
Olck

Ol_Chiki

Old_Hungarian
Hung

Old_Hungarian

Old_Italic
Ital

Old_Italic

Old_North_Arabian
Narb

Old_North_Arabian

Old_Permic
Perm

Old_Permic

Old_Persian
Xpeo

Old_Persian

Old_Sogdian
Sogo

Old_Sogdian

Old_South_Arabian
Sarb

Old_South_Arabian

Old_Turkic
Orkh

Old_Turkic

Oriya
Orya

Oriya

Osage
Osge

Osage

Osmanya
Osma

Osmanya

Pahawh_Hmong
Hmng

Pahawh_Hmong

Property value and aliases	Canonical property value
Palmyrene Palm	Palmyrene
Pau_Cin_Hau Pauc	Pau_Cin_Hau
Phags_Pa Phag	Phags_Pa
Phoenician Phnx	Phoenician
Psalter_Pahlavi Phlp	Psalter_Pahlavi
Rejang Rjng	Rejang
Runic Runr	Runic
Samaritan Samr	Samaritan
Saurashtra Saur	Saurashtra
Sharada Shrd	Sharada
Shavian Shaw	Shavian
Siddham Sidd	Siddham
SignWriting Sgnw	SignWriting
Sinhala Sinh	Sinhala
Sogdian Sogd	Sogdian

Property value and aliases

Canonical property value

Sora_Sompeng
Sora

Sora_Sompeng

Soyombo
Soyo

Soyombo

Sundanese
Sund

Sundanese

Syloti_Nagri
Sylo

Syloti_Nagri

Syriac
Syrc

Syriac

Tagalog
Tglg

Tagalog

Tagbanwa
Tagb

Tagbanwa

Tai_Le
Tale

Tai_Le

Tai_Tham
Lana

Tai_Tham

Tai_Viet
Tavt

Tai_Viet

Takri
Takr

Takri

Tamil
Taml

Tamil

Tangut
Tang

Tangut

Telugu
Telu

Telugu

Thaana
Thaa

Thaana

Property value and aliases	Canonical property value
Thai Thai	Thai
Tibetan Tibt	Tibetan
Tifinagh Tfng	Tifinagh
Tirhuta Tirh	Tirhuta
Ugaritic Ugar	Ugaritic
Vai Vaii	Vai
Warang_Citi Wara	Warang_Citi
Yi Yiii	Yi
Zanabazar_Square Zanb	Zanabazar_Square

21.2.2.9 AtomEscape

With parameter *direction*.

The production *AtomEscape* :: *DecimalEscape* evaluates as follows:

1. Evaluate *DecimalEscape* to obtain an integer *n*.
2. **Assert**: $n \leq N_{\text{capturingParens}}$.
3. Call **BackreferenceMatcher**(*n*, *direction*) and return its Matcher result.

The production *AtomEscape* :: *CharacterEscape* evaluates as follows:

1. Evaluate *CharacterEscape* to obtain a character *ch*.
2. Let *A* be a one-element CharSet containing the character *ch*.
3. Call **CharacterSetMatcher**(*A*, **false**, *direction*) and return its Matcher result.

The production *AtomEscape* :: *CharacterClassEscape* evaluates as follows:

1. Evaluate *CharacterClassEscape* to obtain a CharSet *A*.
2. Call **CharacterSetMatcher**(*A*, **false**, *direction*) and return its Matcher result.

NOTE

An escape sequence of the form `\` followed by a nonzero decimal number *n* matches the result of the *n*th set of capturing parentheses (21.2.2.1). It is an error if the regular expression has fewer than *n* capturing parentheses. If the regular expression has *n* or more capturing parentheses but the *n*th one is **undefined** because it has not captured anything, then the backreference always succeeds.

The production *AtomEscape* :: **k** *GroupName* evaluates as follows:

1. Search the enclosing *Pattern* for an instance of a *GroupSpecifier* for a *RegExpIdentifierName* which has a *StringValue* equal to the *StringValue* of the *RegExpIdentifierName* contained in *GroupName*.
2. **Assert**: A unique such *GroupSpecifier* is found.
3. Let *parenIndex* be the number of left-capturing parentheses in the entire regular expression that occur to the left of the located *GroupSpecifier*. This is the total number of *Atom* :: (*GroupSpecifier* *Disjunction*) *Parse Nodes* prior to or enclosing the located *GroupSpecifier*.
4. Call *BackreferenceMatcher*(*parenIndex*, *direction*) and return its *Matcher* result.

21.2.2.9.1 Runtime Semantics: BackreferenceMatcher (*n*, *direction*)

The abstract operation *BackreferenceMatcher* takes two arguments, an integer *n* and an integer *direction*, and performs the following steps:

1. Return an internal *Matcher* closure that takes two arguments, a State *x* and a Continuation *c*, and performs the following steps:
 - a. Let *cap* be *x*'s *captures List*.
 - b. Let *s* be *cap*[*n*].
 - c. If *s* is **undefined**, return *c*(*x*).
 - d. Let *e* be *x*'s *endIndex*.
 - e. Let *len* be the number of elements in *s*.
 - f. Let *f* be *e* + *direction* × *len*.
 - g. If *f* < 0 or *f* > *InputLength*, return **failure**.
 - h. Let *g* be **min**(*e*, *f*).
 - i. If there exists an integer *i* between 0 (inclusive) and *len* (exclusive) such that *Canonicalize*(*s*[*i*]) is not the same character value as *Canonicalize*(*Input*[*g* + *i*]), return **failure**.
 - j. Let *y* be the State (*f*, *cap*).
 - k. Call *c*(*y*) and return its result.

21.2.2.10 CharacterEscape

The *CharacterEscape* productions evaluate as follows:

CharacterEscape ::
 ControlEscape
 c *ControlLetter*
 0 [*lookahead* ∉ *DecimalDigit*]
 HexEscapeSequence
 RegExpUnicodeEscapeSequence
 IdentityEscape

1. Let *cv* be the *CharacterValue* of this *CharacterEscape*.

2. Return the character whose character value is *cv*.

21.2.2.11 DecimalEscape

The *DecimalEscape* productions evaluate as follows:

DecimalEscape :: *NonZeroDigit* *DecimalDigits*

1. Return the CapturingGroupName of this *DecimalEscape*.

NOTE

If ** is followed by a decimal number *n* whose first digit is not 0, then the escape sequence is considered to be a backreference. It is an error if *n* is greater than the total number of left-capturing parentheses in the entire regular expression.

21.2.2.12 CharacterClassEscape

The production *CharacterClassEscape* :: **d** evaluates as follows:

1. Return the ten-element set of characters containing the characters 0 through 9 inclusive.

The production *CharacterClassEscape* :: **D** evaluates as follows:

1. Return the set of all characters not included in the set returned by *CharacterClassEscape* :: **d** .

The production *CharacterClassEscape* :: **s** evaluates as follows:

1. Return the set of characters containing the characters that are on the right-hand side of the *WhiteSpace* or *LineTerminator* productions.

The production *CharacterClassEscape* :: **S** evaluates as follows:

1. Return the set of all characters not included in the set returned by *CharacterClassEscape* :: **s** .

The production *CharacterClassEscape* :: **w** evaluates as follows:

1. Return the set of all characters returned by *WordCharacters*() .

The production *CharacterClassEscape* :: **W** evaluates as follows:

1. Return the set of all characters not included in the set returned by *CharacterClassEscape* :: **w** .

The production *CharacterClassEscape* :: **p**{ *UnicodePropertyValueExpression* } evaluates by returning the CharSet containing all Unicode code points included in the CharSet returned by *UnicodePropertyValueExpression*.

The production *CharacterClassEscape* :: **P**{ *UnicodePropertyValueExpression* } evaluates by returning the CharSet containing all Unicode code points not included in the CharSet returned by *UnicodePropertyValueExpression*.

The production *UnicodePropertyValueExpression* :: *UnicodePropertyName* = *UnicodePropertyValue* evaluates as follows:

1. Let *ps* be SourceText of *UnicodePropertyName*.
2. Let *p* be ! *UnicodeMatchProperty*(*ps*).

3. **Assert:** p is a Unicode [property name](#) or property alias listed in the “Property name and aliases” column of [Table 54](#).
4. Let vs be SourceText of *UnicodePropertyValue*.
5. Let v be ! [UnicodeMatchPropertyValue](#)(p , vs).
6. Return the CharSet containing all Unicode code points whose character database definition includes the property p with value v .

The production *UnicodePropertyValueExpression* :: *LoneUnicodePropertyNameOrValue* evaluates as follows:

1. Let s be SourceText of *LoneUnicodePropertyNameOrValue*.
2. If ! [UnicodeMatchPropertyValue](#)("General_Category", s) is identical to a [List](#) of Unicode code points that is the name of a Unicode general category or general category alias listed in the “Property value and aliases” column of [Table 56](#), then
 - a. Return the CharSet containing all Unicode code points whose character database definition includes the property “General_Category” with value s .
3. Let p be ! [UnicodeMatchProperty](#)(s).
4. **Assert:** p is a binary Unicode property or binary property alias listed in the “Property name and aliases” column of [Table 55](#).
5. Return the CharSet containing all Unicode code points whose character database definition includes the property p with value “True”.

21.2.2.13 CharacterClass

The production *CharacterClass* :: [*ClassRanges*] evaluates as follows:

1. Evaluate *ClassRanges* to obtain a CharSet A .
2. Return the two results A and **false**.

The production *CharacterClass* :: [^ *ClassRanges*] evaluates as follows:

1. Evaluate *ClassRanges* to obtain a CharSet A .
2. Return the two results A and **true**.

21.2.2.14 ClassRanges

The production *ClassRanges* :: [empty] evaluates as follows:

1. Return the empty CharSet.

The production *ClassRanges* :: *NonemptyClassRanges* evaluates as follows:

1. Return the CharSet that is the result of evaluating *NonemptyClassRanges*.

21.2.2.15 NonemptyClassRanges

The production *NonemptyClassRanges* :: *ClassAtom* evaluates as follows:

1. Return the CharSet that is the result of evaluating *ClassAtom*.

The production *NonemptyClassRanges* :: *ClassAtom NonemptyClassRangesNoDash* evaluates as follows:

1. Evaluate *ClassAtom* to obtain a CharSet A .

2. Evaluate *NonemptyClassRangesNoDash* to obtain a CharSet *B*.
3. Return the union of CharSets *A* and *B*.

The production *NonemptyClassRanges* :: *ClassAtom* – *ClassAtom* *ClassRanges* evaluates as follows:

1. Evaluate the first *ClassAtom* to obtain a CharSet *A*.
2. Evaluate the second *ClassAtom* to obtain a CharSet *B*.
3. Evaluate *ClassRanges* to obtain a CharSet *C*.
4. Call *CharacterRange*(*A*, *B*) and let *D* be the resulting CharSet.
5. Return the union of CharSets *D* and *C*.

21.2.2.15.1 Runtime Semantics: *CharacterRange* (*A*, *B*)

The abstract operation *CharacterRange* takes two CharSet parameters *A* and *B* and performs the following steps:

1. **Assert:** *A* and *B* each contain exactly one character.
2. Let *a* be the one character in CharSet *A*.
3. Let *b* be the one character in CharSet *B*.
4. Let *i* be the character value of character *a*.
5. Let *j* be the character value of character *b*.
6. **Assert:** $i \leq j$.
7. Return the set containing all characters numbered *i* through *j*, inclusive.

21.2.2.16 *NonemptyClassRangesNoDash*

The production *NonemptyClassRangesNoDash* :: *ClassAtom* evaluates as follows:

1. Return the CharSet that is the result of evaluating *ClassAtom*.

The production *NonemptyClassRangesNoDash* :: *ClassAtomNoDash* *NonemptyClassRangesNoDash* evaluates as follows:

1. Evaluate *ClassAtomNoDash* to obtain a CharSet *A*.
2. Evaluate *NonemptyClassRangesNoDash* to obtain a CharSet *B*.
3. Return the union of CharSets *A* and *B*.

The production *NonemptyClassRangesNoDash* :: *ClassAtomNoDash* – *ClassAtom* *ClassRanges* evaluates as follows:

1. Evaluate *ClassAtomNoDash* to obtain a CharSet *A*.
2. Evaluate *ClassAtom* to obtain a CharSet *B*.
3. Evaluate *ClassRanges* to obtain a CharSet *C*.
4. Call *CharacterRange*(*A*, *B*) and let *D* be the resulting CharSet.
5. Return the union of CharSets *D* and *C*.

NOTE 1

ClassRanges can expand into a single *ClassAtom* and/or ranges of two *ClassAtom* separated by dashes. In the latter case the *ClassRanges* includes all characters between the first *ClassAtom* and the second *ClassAtom*, inclusive; an error occurs if either *ClassAtom* does not represent a single character (for example, if one is \w) or if the first *ClassAtom*'s character value is greater than the second *ClassAtom*'s character value.

NOTE 2

Even if the pattern ignores case, the case of the two ends of a range is significant in determining which characters belong to the range. Thus, for example, the pattern */ [E-F] /i* matches only the letters **E**, **F**, **e**, and **f**, while the pattern

`/[E-F]/i` matches all upper and lower-case letters in the Unicode Basic Latin block as well as the symbols `[`, `\`, `]`, `^`, `_`, and ```.

NOTE 3

A `-` character can be treated literally or it can denote a range. It is treated literally if it is the first or last character of *ClassRanges*, the beginning or end limit of a range specification, or immediately follows a range specification.

21.2.2.17 ClassAtom

The production *ClassAtom* **::** `-` evaluates as follows:

1. Return the CharSet containing the single character `-` U+002D (HYPHEN-MINUS).

The production *ClassAtom* **::** *ClassAtomNoDash* evaluates as follows:

1. Return the CharSet that is the result of evaluating *ClassAtomNoDash*.

21.2.2.18 ClassAtomNoDash

The production *ClassAtomNoDash* **::** *SourceCharacter* but not one of `\` or `]` or `-` evaluates as follows:

1. Return the CharSet containing the character matched by *SourceCharacter*.

The production *ClassAtomNoDash* **::** `\` *ClassEscape* evaluates as follows:

1. Return the CharSet that is the result of evaluating *ClassEscape*.

21.2.2.19 ClassEscape

The *ClassEscape* productions evaluate as follows:

ClassEscape **::** `\b`

ClassEscape **::** `-`

ClassEscape **::** *CharacterEscape*

1. Let *cv* be the CharacterValue of this *ClassEscape*.
2. Let *c* be the character whose character value is *cv*.
3. Return the CharSet containing the single character *c*.

ClassEscape **::** *CharacterClassEscape*

1. Return the CharSet that is the result of evaluating *CharacterClassEscape*.

NOTE

A *ClassAtom* can use any of the escape sequences that are allowed in the rest of the regular expression except for `\b`, `\B`, and backreferences. Inside a *CharacterClass*, `\b` means the backspace character, while `\B` and backreferences raise errors. Using a backreference inside a *ClassAtom* causes an error.

21.2.3 The RegExp Constructor

The RegExp [constructor](#):

is the intrinsic object %RegExp%.

is the initial value of the **RegExp** property of the [global object](#).

creates and initializes a new RegExp object when called as a function rather than as a [constructor](#). Thus the function call **RegExp (...)** is equivalent to the object creation expression **new RegExp (...)** with the same arguments.

is designed to be subclassable. It may be used as the value of an **extends** clause of a class definition. Subclass constructors that intend to inherit the specified **RegExp** behaviour must include a **super** call to the **RegExp** [constructor](#) to create and initialize subclass instances with the necessary internal slots.

21.2.3.1 RegExp (*pattern*, *flags*)

The following steps are taken:

1. Let *patternIsRegExp* be ? **IsRegExp**(*pattern*).
2. If *NewTarget* is **undefined**, then
 - a. Let *newTarget* be the [active function object](#).
 - b. If *patternIsRegExp* is **true** and *flags* is **undefined**, then
 - i. Let *patternConstructor* be ? **Get**(*pattern*, "constructor").
 - ii. If **SameValue**(*newTarget*, *patternConstructor*) is **true**, return *pattern*.
3. Else, let *newTarget* be *NewTarget*.
4. If **Type**(*pattern*) is Object and *pattern* has a [[RegExpMatcher]] internal slot, then
 - a. Let *P* be *pattern*.[[OriginalSource]].
 - b. If *flags* is **undefined**, let *F* be *pattern*.[[OriginalFlags]].
 - c. Else, let *F* be *flags*.
5. Else if *patternIsRegExp* is **true**, then
 - a. Let *P* be ? **Get**(*pattern*, "source").
 - b. If *flags* is **undefined**, then
 - i. Let *F* be ? **Get**(*pattern*, "flags").
 - c. Else, let *F* be *flags*.
6. Else,
 - a. Let *P* be *pattern*.
 - b. Let *F* be *flags*.
7. Let *O* be ? **RegExpAlloc**(*newTarget*).
8. Return ? **RegExpInitialize**(*O*, *P*, *F*).

NOTE

If *pattern* is supplied using a *StringLiteral*, the usual escape sequence substitutions are performed before the String is processed by RegExp. If *pattern* must contain an escape sequence to be recognized by RegExp, any U+005C (REVERSE SOLIDUS) code points must be escaped within the *StringLiteral* to prevent them being removed when the contents of the *StringLiteral* are formed.

21.2.3.2 Abstract Operations for the RegExp Constructor

21.2.3.2.1 Runtime Semantics: RegExpAlloc (*newTarget*)

When the abstract operation RegExpAlloc with argument *newTarget* is called, the following steps are taken:

1. Let *obj* be ? **OrdinaryCreateFromConstructor**(*newTarget*, "%RegExpPrototype%", « [[RegExpMatcher]],

- [[OriginalSource]], [[OriginalFlags]] »).
2. Perform ! **DefinePropertyOrThrow**(*obj*, "lastIndex", PropertyDescriptor { [[Writable]]: **true**, [[Enumerable]]: **false**, [[Configurable]]: **false** }).
 3. Return *obj*.

21.2.3.2.2 Runtime Semantics: RegExpInitialize (*obj*, *pattern*, *flags*)

When the abstract operation RegExpInitialize with arguments *obj*, *pattern*, and *flags* is called, the following steps are taken:

1. If *pattern* is **undefined**, let *P* be the empty String.
2. Else, let *P* be ? **ToString**(*pattern*).
3. If *flags* is **undefined**, let *F* be the empty String.
4. Else, let *F* be ? **ToString**(*flags*).
5. If *F* contains any code unit other than "g", "i", "m", "s", "u", or "y" or if it contains the same code unit more than once, throw a **SyntaxError** exception.
6. If *F* contains "u", let *BMP* be **false**; else let *BMP* be **true**.
7. If *BMP* is **true**, then
 - a. Parse *P* using the grammars in 21.2.1 and interpreting each of its 16-bit elements as a Unicode BMP code point. UTF-16 decoding is not applied to the elements. The **goal symbol** for the parse is *Pattern*_[~U, ~N] . If the result of parsing contains a *GroupName*, reparse with the **goal symbol** *Pattern*_[~U, +N] and use this result instead. Throw a **SyntaxError** exception if *P* did not conform to the grammar, if any elements of *P* were not matched by the parse, or if any Early Error conditions exist.
 - b. Let *patternCharacters* be a **List** whose elements are the code unit elements of *P*.
8. Else,
 - a. Parse *P* using the grammars in 21.2.1 and interpreting *P* as UTF-16 encoded Unicode code points (6.1.4). The **goal symbol** for the parse is *Pattern*_[+U, +N] . Throw a **SyntaxError** exception if *P* did not conform to the grammar, if any elements of *P* were not matched by the parse, or if any Early Error conditions exist.
 - b. Let *patternCharacters* be a **List** whose elements are the code points resulting from applying UTF-16 decoding to *P*'s sequence of elements.
9. Set *obj*.[[OriginalSource]] to *P*.
10. Set *obj*.[[OriginalFlags]] to *F*.
11. Set *obj*.[[RegExpMatcher]] to the internal procedure that evaluates the above parse of *P* by applying the semantics provided in 21.2.2 using *patternCharacters* as the pattern's **List** of *SourceCharacter* values and *F* as the flag parameters.
12. Perform ? **Set**(*obj*, "lastIndex", 0, **true**).
13. Return *obj*.

21.2.3.2.3 Runtime Semantics: RegExpCreate (*P*, *F*)

When the abstract operation RegExpCreate with arguments *P* and *F* is called, the following steps are taken:

1. Let *obj* be ? **RegExpAlloc**(%RegExp%).
2. Return ? **RegExpInitialize**(*obj*, *P*, *F*).

21.2.3.2.4 Runtime Semantics: EscapeRegExpPattern (*P*, *F*)

When the abstract operation EscapeRegExpPattern with arguments *P* and *F* is called, the following occurs:

1. Let *S* be a String in the form of a *Pattern*_[~U] (*Pattern*_[+U] if *F* contains "u") equivalent to *P* interpreted as UTF-16 encoded Unicode code points (6.1.4), in which certain code points are escaped as described below. *S* may or may not be identical to *P*; however, the internal procedure that would result from evaluating *S* as a *Pattern*_[~U] (*Pattern*_[+U] if *F* contains "u") must behave identically to the internal procedure given by the constructed object's `[[RegExpMatcher]]` internal slot. Multiple calls to this abstract operation using the same values for *P* and *F* must produce identical results.
2. The code points `/` or any *LineTerminator* occurring in the pattern shall be escaped in *S* as necessary to ensure that the string-concatenation of `"/"`, *S*, `"/"`, and *F* can be parsed (in an appropriate lexical context) as a *RegularExpressionLiteral* that behaves identically to the constructed regular expression. For example, if *P* is `"/"`, then *S* could be `"\"` or `"\u002F"`, among other possibilities, but not `"/"`, because `///` followed by *F* would be parsed as a *SingleLineComment* rather than a *RegularExpressionLiteral*. If *P* is the empty String, this specification can be met by letting *S* be `"(?:)"`.
3. Return *S*.

21.2.4 Properties of the RegExp Constructor

The RegExp `constructor`:

has a `[[Prototype]]` internal slot whose value is the intrinsic object `%FunctionPrototype%`.
has the following properties:

21.2.4.1 RegExp.prototype

The initial value of `RegExp.prototype` is the intrinsic object `%RegExpPrototype%`.

This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **false** }.

21.2.4.2 get RegExp [@@species]

`RegExp [ @@species ]` is an `accessor property` whose set accessor function is **undefined**. Its get accessor function performs the following steps:

1. Return the **this** value.

The value of the **name** property of this function is `"get [Symbol.species]"`.

NOTE

RegExp prototype methods normally use their **this** object's `constructor` to create a derived object. However, a subclass `constructor` may over-ride that default behaviour by redefining its `@@species` property.

21.2.5 Properties of the RegExp Prototype Object

The RegExp prototype object:

is the intrinsic object `%RegExpPrototype%`.
is an ordinary object.
is not a RegExp instance and does not have a `[[RegExpMatcher]]` internal slot or any of the other internal slots of RegExp instance objects.
has a `[[Prototype]]` internal slot whose value is the intrinsic object `%ObjectPrototype%`.

NOTE

The `RegExp` prototype object does not have a **valueOf** property of its own; however, it inherits the **valueOf** property from the `Object` prototype object.

21.2.5.1 `RegExp.prototype.constructor`

The initial value of `RegExp.prototype.constructor` is the intrinsic object `%RegExp%`.

21.2.5.2 `RegExp.prototype.exec ( string )`

Performs a regular expression match of *string* against the regular expression and returns an `Array` object containing the results of the match, or **null** if *string* did not match.

The String `ToString(string)` is searched for an occurrence of the regular expression pattern as follows:

1. Let *R* be the **this** value.
2. If `Type(R)` is not `Object`, throw a **TypeError** exception.
3. If *R* does not have a `[[RegExpMatcher]]` internal slot, throw a **TypeError** exception.
4. Let *S* be `? ToString(string)`.
5. Return `? RegExpBuiltinExec(R, S)`.

21.2.5.2.1 Runtime Semantics: `RegExpExec ( R, S )`

The abstract operation `RegExpExec` with arguments *R* and *S* performs the following steps:

1. **Assert:** `Type(R)` is `Object`.
2. **Assert:** `Type(S)` is `String`.
3. Let *exec* be `? Get(R, "exec")`.
4. If `IsCallable(exec)` is **true**, then
 - a. Let *result* be `? Call(exec, R, « S »)`.
 - b. If `Type(result)` is neither `Object` or `Null`, throw a **TypeError** exception.
 - c. Return *result*.
5. If *R* does not have a `[[RegExpMatcher]]` internal slot, throw a **TypeError** exception.
6. Return `? RegExpBuiltinExec(R, S)`.

NOTE

If a callable **exec** property is not found this algorithm falls back to attempting to use the built-in `RegExp` matching algorithm. This provides compatible behaviour for code written for prior editions where most built-in algorithms that use regular expressions did not perform a dynamic property lookup of **exec**.

21.2.5.2.2 Runtime Semantics: `RegExpBuiltinExec ( R, S )`

The abstract operation `RegExpBuiltinExec` with arguments *R* and *S* performs the following steps:

1. **Assert:** *R* is an initialized `RegExp` instance.
2. **Assert:** `Type(S)` is `String`.
3. Let *length* be the number of code units in *S*.
4. Let *lastIndex* be `? ToLength(? Get(R, "lastIndex"))`.

5. Let *flags* be *R*.[[OriginalFlags]].
6. If *flags* contains "g", let *global* be **true**, else let *global* be **false**.
7. If *flags* contains "y", let *sticky* be **true**, else let *sticky* be **false**.
8. If *global* is **false** and *sticky* is **false**, set *lastIndex* to 0.
9. Let *matcher* be *R*.[[RegExpMatcher]].
10. If *flags* contains "u", let *fullUnicode* be **true**, else let *fullUnicode* be **false**.
11. Let *matchSucceeded* be **false**.
12. Repeat, while *matchSucceeded* is **false**
 - a. If *lastIndex* > *length*, then
 - i. If *global* is **true** or *sticky* is **true**, then
 1. Perform ? *Set*(*R*, "lastIndex", 0, **true**).
 - ii. Return **null**.
 - b. Let *r* be *matcher*(*S*, *lastIndex*).
 - c. If *r* is **failure**, then
 - i. If *sticky* is **true**, then
 1. Perform ? *Set*(*R*, "lastIndex", 0, **true**).
 2. Return **null**.
 - ii. Set *lastIndex* to *AdvanceStringIndex*(*S*, *lastIndex*, *fullUnicode*).
 - d. Else,
 - i. **Assert**: *r* is a State.
 - ii. Set *matchSucceeded* to **true**.
13. Let *e* be *r*'s *endIndex* value.
14. If *fullUnicode* is **true**, then
 - a. *e* is an index into the *Input* character list, derived from *S*, matched by *matcher*. Let *eUTF* be the smallest index into *S* that corresponds to the character at element *e* of *Input*. If *e* is greater than or equal to the number of elements in *Input*, then *eUTF* is the number of code units in *S*.
 - b. Set *e* to *eUTF*.
15. If *global* is **true** or *sticky* is **true**, then
 - a. Perform ? *Set*(*R*, "lastIndex", *e*, **true**).
16. Let *n* be the number of elements in *r*'s *captures List*. (This is the same value as 21.2.2.1's *NcapturingParens*.)
17. **Assert**: $n < 2^{32} - 1$.
18. Let *A* be ! *ArrayCreate*(*n* + 1).
19. **Assert**: The value of *A*'s "length" property is *n* + 1.
20. Perform ! *CreateDataProperty*(*A*, "index", *lastIndex*).
21. Perform ! *CreateDataProperty*(*A*, "input", *S*).
22. Let *matchedSubstr* be the matched substring (i.e. the portion of *S* between offset *lastIndex* inclusive and offset *e* exclusive).
23. Perform ! *CreateDataProperty*(*A*, "0", *matchedSubstr*).
24. If *R* contains any *GroupName*, then
 - a. Let *groups* be *ObjectCreate*(**null**).
25. Else,
 - a. Let *groups* be **undefined**.
26. Perform ! *CreateDataProperty*(*A*, "groups", *groups*).
27. For each integer *i* such that $i > 0$ and $i \leq n$, do
 - a. Let *captureI* be *i*th element of *r*'s *captures List*.
 - b. If *captureI* is **undefined**, let *capturedValue* be **undefined**.
 - c. Else if *fullUnicode* is **true**, then
 - i. **Assert**: *captureI* is a *List* of code points.

- ii. Let *capturedValue* be the String value whose code units are the **UTF16Encoding** of the code points of *captureI*.
 - d. Else *fullUnicode* is **false**,
 - i. **Assert**: *captureI* is a **List** of code units.
 - ii. Let *capturedValue* be the String value consisting of the code units of *captureI*.
 - e. Perform ! **CreateDataProperty**(*A*, ! **ToString**(*i*), *capturedValue*).
 - f. If the *i*th capture of *R* was defined with a *GroupName*, then
 - i. Let *s* be the StringValue of the corresponding *RegExpIdentifierName*.
 - ii. Perform ! **CreateDataProperty**(*groups*, *s*, *capturedValue*).
28. Return *A*.

21.2.5.2.3 AdvanceStringIndex (*S*, *index*, *unicode*)

The abstract operation AdvanceStringIndex with arguments *S*, *index*, and *unicode* performs the following steps:

1. **Assert**: **Type**(*S*) is String.
2. **Assert**: *index* is an integer such that $0 \leq \text{index} \leq 2^{53} - 1$.
3. **Assert**: **Type**(*unicode*) is Boolean.
4. If *unicode* is **false**, return *index* + 1.
5. Let *length* be the number of code units in *S*.
6. If *index* + 1 $\geq$ *length*, return *index* + 1.
7. Let *first* be the numeric value of the code unit at index *index* within *S*.
8. If *first* < 0xD800 or *first* > 0xDBFF, return *index* + 1.
9. Let *second* be the numeric value of the code unit at index *index* + 1 within *S*.
10. If *second* < 0xDC00 or *second* > 0xDFFF, return *index* + 1.
11. Return *index* + 2.

21.2.5.3 get RegExp.prototype.dotAll

RegExp.prototype.dotAll is an **accessor property** whose set accessor function is **undefined**. Its get accessor function performs the following steps:

1. Let *R* be the **this** value.
2. If **Type**(*R*) is not Object, throw a **TypeError** exception.
3. If *R* does not have an [[OriginalFlags]] internal slot, then
 - a. If **SameValue**(*R*, %RegExpPrototype%) is **true**, return **undefined**.
 - b. Otherwise, throw a **TypeError** exception.
4. Let *flags* be *R*.[[OriginalFlags]].
5. If *flags* contains the code unit 0x0073 (LATIN SMALL LETTER S), return **true**.
6. Return **false**.

21.2.5.4 get RegExp.prototype.flags

RegExp.prototype.flags is an **accessor property** whose set accessor function is **undefined**. Its get accessor function performs the following steps:

1. Let *R* be the **this** value.
2. If **Type**(*R*) is not Object, throw a **TypeError** exception.
3. Let *result* be the empty String.

4. Let *global* be **ToBoolean**(? **Get**(*R*, "global")).
5. If *global* is **true**, append the code unit 0x0067 (LATIN SMALL LETTER G) as the last code unit of *result*.
6. Let *ignoreCase* be **ToBoolean**(? **Get**(*R*, "ignoreCase")).
7. If *ignoreCase* is **true**, append the code unit 0x0069 (LATIN SMALL LETTER I) as the last code unit of *result*.
8. Let *multiline* be **ToBoolean**(? **Get**(*R*, "multiline")).
9. If *multiline* is **true**, append the code unit 0x006D (LATIN SMALL LETTER M) as the last code unit of *result*.
10. Let *dotAll* be **ToBoolean**(? **Get**(*R*, "dotAll")).
11. If *dotAll* is **true**, append the code unit 0x0073 (LATIN SMALL LETTER S) as the last code unit of *result*.
12. Let *unicode* be **ToBoolean**(? **Get**(*R*, "unicode")).
13. If *unicode* is **true**, append the code unit 0x0075 (LATIN SMALL LETTER U) as the last code unit of *result*.
14. Let *sticky* be **ToBoolean**(? **Get**(*R*, "sticky")).
15. If *sticky* is **true**, append the code unit 0x0079 (LATIN SMALL LETTER Y) as the last code unit of *result*.
16. Return *result*.

21.2.5.5 **get** **RegExp.prototype.global**

RegExp.prototype.global is an **accessor property** whose set accessor function is **undefined**. Its get accessor function performs the following steps:

1. Let *R* be the **this** value.
2. If **Type**(*R*) is not Object, throw a **TypeError** exception.
3. If *R* does not have an **[[OriginalFlags]]** internal slot, then
 - a. If **SameValue**(*R*, %**RegExpPrototype**%) is **true**, return **undefined**.
 - b. Otherwise, throw a **TypeError** exception.
4. Let *flags* be *R*.**[[OriginalFlags]]**.
5. If *flags* contains the code unit 0x0067 (LATIN SMALL LETTER G), return **true**.
6. Return **false**.

21.2.5.6 **get** **RegExp.prototype.ignoreCase**

RegExp.prototype.ignoreCase is an **accessor property** whose set accessor function is **undefined**. Its get accessor function performs the following steps:

1. Let *R* be the **this** value.
2. If **Type**(*R*) is not Object, throw a **TypeError** exception.
3. If *R* does not have an **[[OriginalFlags]]** internal slot, then
 - a. If **SameValue**(*R*, %**RegExpPrototype**%) is **true**, return **undefined**.
 - b. Otherwise, throw a **TypeError** exception.
4. Let *flags* be *R*.**[[OriginalFlags]]**.
5. If *flags* contains the code unit 0x0069 (LATIN SMALL LETTER I), return **true**.
6. Return **false**.

21.2.5.7 **RegExp.prototype** [@@match] (*string*)

When the @@**match** method is called with argument *string*, the following steps are taken:

1. Let *rx* be the **this** value.
2. If **Type**(*rx*) is not Object, throw a **TypeError** exception.
3. Let *S* be ? **ToString**(*string*).

4. Let *global* be **ToBoolean**(? **Get**(*rx*, "global")).
5. If *global* is **false**, then
 - a. Return ? **RegExpExec**(*rx*, *S*).
6. Else *global* is **true**,
 - a. Let *fullUnicode* be **ToBoolean**(? **Get**(*rx*, "unicode")).
 - b. Perform ? **Set**(*rx*, "lastIndex", 0, **true**).
 - c. Let *A* be ! **ArrayCreate**(0).
 - d. Let *n* be 0.
 - e. Repeat,
 - i. Let *result* be ? **RegExpExec**(*rx*, *S*).
 - ii. If *result* is **null**, then
 1. If *n* = 0, return **null**.
 2. Return *A*.
 - iii. Else *result* is not **null**,
 1. Let *matchStr* be ? **ToString**(? **Get**(*result*, "0")).
 2. Let *status* be **CreateDataProperty**(*A*, ! **ToString**(*n*), *matchStr*).
 3. **Assert**: *status* is **true**.
 4. If *matchStr* is the empty String, then
 - a. Let *thisIndex* be ? **ToLength**(? **Get**(*rx*, "lastIndex")).
 - b. Let *nextIndex* be **AdvanceStringIndex**(*S*, *thisIndex*, *fullUnicode*).
 - c. Perform ? **Set**(*rx*, "lastIndex", *nextIndex*, **true**).
 5. Increment *n*.

The value of the **name** property of this function is "[Symbol.match]".

NOTE

The @@match property is used by the **IsRegExp** abstract operation to identify objects that have the basic behaviour of regular expressions. The absence of a @@match property or the existence of such a property whose value does not Boolean coerce to **true** indicates that the object is not intended to be used as a regular expression object.

21.2.5.8 get RegExp.prototype.multiline

RegExp.prototype.multiline is an **accessor property** whose set accessor function is **undefined**. Its get accessor function performs the following steps:

1. Let *R* be the **this** value.
2. If **Type**(*R*) is not Object, throw a **TypeError** exception.
3. If *R* does not have an [[OriginalFlags]] internal slot, then
 - a. If **SameValue**(*R*, %RegExpPrototype%) is **true**, return **undefined**.
 - b. Otherwise, throw a **TypeError** exception.
4. Let *flags* be *R*.[[OriginalFlags]].
5. If *flags* contains the code unit 0x006D (LATIN SMALL LETTER M), return **true**.
6. Return **false**.

21.2.5.9 RegExp.prototype [@@replace] (*string*, *replaceValue*)

When the @@replace method is called with arguments *string* and *replaceValue*, the following steps are taken:

1. Let *rx* be the **this** value.
2. If **Type**(*rx*) is not Object, throw a **TypeError** exception.
3. Let *S* be ? **ToString**(*string*).
4. Let *lengthS* be the number of code unit elements in *S*.
5. Let *functionalReplace* be **IsCallable**(*replaceValue*).
6. If *functionalReplace* is **false**, then
 - a. Set *replaceValue* to ? **ToString**(*replaceValue*).
7. Let *global* be **ToBoolean**(? **Get**(*rx*, "global")).
8. If *global* is **true**, then
 - a. Let *fullUnicode* be **ToBoolean**(? **Get**(*rx*, "unicode")).
 - b. Perform ? **Set**(*rx*, "lastIndex", 0, **true**).
9. Let *results* be a new empty **List**.
10. Let *done* be **false**.
11. Repeat, while *done* is **false**
 - a. Let *result* be ? **RegExpExec**(*rx*, *S*).
 - b. If *result* is **null**, set *done* to **true**.
 - c. Else *result* is not **null**,
 - i. Append *result* to the end of *results*.
 - ii. If *global* is **false**, set *done* to **true**.
 - iii. Else,
 1. Let *matchStr* be ? **ToString**(? **Get**(*result*, "0")).
 2. If *matchStr* is the empty String, then
 - a. Let *thisIndex* be ? **ToLength**(? **Get**(*rx*, "lastIndex")).
 - b. Let *nextIndex* be **AdvanceStringIndex**(*S*, *thisIndex*, *fullUnicode*).
 - c. Perform ? **Set**(*rx*, "lastIndex", *nextIndex*, **true**).
12. Let *accumulatedResult* be the empty String value.
13. Let *nextSourcePosition* be 0.
14. For each *result* in *results*, do
 - a. Let *nCaptures* be ? **ToLength**(? **Get**(*result*, "length")).
 - b. Set *nCaptures* to **max**(*nCaptures* - 1, 0).
 - c. Let *matched* be ? **ToString**(? **Get**(*result*, "0")).
 - d. Let *matchLength* be the number of code units in *matched*.
 - e. Let *position* be ? **ToInteger**(? **Get**(*result*, "index")).
 - f. Set *position* to **max**(**min**(*position*, *lengthS*), 0).
 - g. Let *n* be 1.
 - h. Let *captures* be a new empty **List**.
 - i. Repeat, while *n* ≤ *nCaptures*
 - i. Let *capN* be ? **Get**(*result*, ! **ToString**(*n*)).
 - ii. If *capN* is not **undefined**, then
 1. Set *capN* to ? **ToString**(*capN*).
 - iii. Append *capN* as the last element of *captures*.
 - iv. Increase *n* by 1.
 - j. Let *namedCaptures* be ? **Get**(*result*, "groups").
 - k. If *functionalReplace* is **true**, then
 - i. Let *replacerArgs* be « *matched* ».
 - ii. Append in list order the elements of *captures* to the end of the **List** *replacerArgs*.
 - iii. Append *position* and *S* to *replacerArgs*.
 - iv. If *namedCaptures* is not **undefined**, then

1. Append *namedCaptures* as the last element of *replacerArgs*.
- v. Let *replValue* be ? Call(*replaceValue*, **undefined**, *replacerArgs*).
- vi. Let *replacement* be ? ToString(*replValue*).
- l. Else,
 - i. Let *replacement* be GetSubstitution(*matched*, *S*, *position*, *captures*, *namedCaptures*, *replaceValue*).
- m. If *position* ≥ *nextSourcePosition*, then
 - i. NOTE: *position* should not normally move backwards. If it does, it is an indication of an ill-behaving RegExp subclass or use of an access triggered side-effect to change the global flag or other characteristics of *rx*. In such cases, the corresponding substitution is ignored.
 - ii. Set *accumulatedResult* to the string-concatenation of the current value of *accumulatedResult*, the substring of *S* consisting of the code units from *nextSourcePosition* (inclusive) up to *position* (exclusive), and *replacement*.
 - iii. Set *nextSourcePosition* to *position* + *matchLength*.
15. If *nextSourcePosition* ≥ *lengthS*, return *accumulatedResult*.
16. Return the string-concatenation of *accumulatedResult* and the substring of *S* consisting of the code units from *nextSourcePosition* (inclusive) up through the final code unit of *S* (inclusive).

The value of the **name** property of this function is "[Symbol.replace]".

21.2.5.10 RegExp.prototype [@@search] (*string*)

When the @@search method is called with argument *string*, the following steps are taken:

1. Let *rx* be the **this** value.
2. If Type(*rx*) is not Object, throw a **TypeError** exception.
3. Let *S* be ? ToString(*string*).
4. Let *previousLastIndex* be ? Get(*rx*, "lastIndex").
5. If SameValue(*previousLastIndex*, 0) is **false**, then
 - a. Perform ? Set(*rx*, "lastIndex", 0, **true**).
6. Let *result* be ? RegExpExec(*rx*, *S*).
7. Let *currentLastIndex* be ? Get(*rx*, "lastIndex").
8. If SameValue(*currentLastIndex*, *previousLastIndex*) is **false**, then
 - a. Perform ? Set(*rx*, "lastIndex", *previousLastIndex*, **true**).
9. If *result* is **null**, return -1.
10. Return ? Get(*result*, "index").

The value of the **name** property of this function is "[Symbol.search]".

NOTE

The **lastIndex** and **global** properties of this RegExp object are ignored when performing the search. The **lastIndex** property is left unchanged.

21.2.5.11 get RegExp.prototype.source

RegExp.prototype.source is an **accessor property** whose set accessor function is **undefined**. Its get accessor function performs the following steps:

1. Let *R* be the **this** value.
2. If Type(*R*) is not Object, throw a **TypeError** exception.

3. If *R* does not have an `[[OriginalSource]]` internal slot, then
 - a. If `SameValue(R, %RegExpPrototype%)` is **true**, return "`(?:)`".
 - b. Otherwise, throw a **TypeError** exception.
4. **Assert**: *R* has an `[[OriginalFlags]]` internal slot.
5. Let *src* be *R*.`[[OriginalSource]]`.
6. Let *flags* be *R*.`[[OriginalFlags]]`.
7. Return `EscapeRegExpPattern(src, flags)`.

21.2.5.12 `RegExp.prototype [ @@split ] ( string, limit )`

NOTE 1

Returns an Array object into which substrings of the result of converting *string* to a String have been stored. The substrings are determined by searching from left to right for matches of the **this** value regular expression; these occurrences are not part of any substring in the returned array, but serve to divide up the String value.

The **this** value may be an empty regular expression or a regular expression that can match an empty String. In this case, the regular expression does not match the empty substring at the beginning or end of the input String, nor does it match the empty substring at the end of the previous separator match. (For example, if the regular expression matches the empty String, the String is split up into individual code unit elements; the length of the result array equals the length of the String, and each substring contains one code unit.) Only the first match at a given index of the String is considered, even if backtracking could yield a non-empty-substring match at that index. (For example, `/a*/[Symbol.split] ("ab")` evaluates to the array `["a", "b"]`, while `/a*/[Symbol.split] ("ab")` evaluates to the array `["", "b"]`.)

If the *string* is (or converts to) the empty String, the result depends on whether the regular expression can match the empty String. If it can, the result array contains no elements. Otherwise, the result array contains one element, which is the empty String.

If the regular expression contains capturing parentheses, then each time *separator* is matched the results (including any **undefined** results) of the capturing parentheses are spliced into the output array. For example,

```
/<(\/?)?([^\<>]+)>/[Symbol.split] ("A<B>bold</B>and<CODE>coded</CODE>")
```

evaluates to the array

```
["A", undefined, "B", "bold", "/", "B", "and", undefined, "CODE", "coded", "/", "
```

If *limit* is not **undefined**, then the output array is truncated so that it contains no more than *limit* elements.

When the `@@split` method is called, the following steps are taken:

1. Let *rx* be the **this** value.
2. If `Type(rx)` is not Object, throw a **TypeError** exception.
3. Let *S* be `? ToString(string)`.
4. Let *C* be `? SpeciesConstructor(rx, %RegExp%)`.
5. Let *flags* be `? ToString(? Get(rx, "flags"))`.
6. If *flags* contains "u", let *unicodeMatching* be **true**.
7. Else, let *unicodeMatching* be **false**.
8. If *flags* contains "y", let *newFlags* be *flags*.
9. Else, let *newFlags* be the string-concatenation of *flags* and "y".
10. Let *splitter* be `? Construct(C, « rx, newFlags »)`.

11. Let *A* be ! **ArrayCreate**(0).
12. Let *lengthA* be 0.
13. If *limit* is **undefined**, let *lim* be $2^{32} - 1$; else let *lim* be ? **ToUint32**(*limit*).
14. Let *size* be the length of *S*.
15. Let *p* be 0.
16. If *lim* = 0, return *A*.
17. If *size* = 0, then
 - a. Let *z* be ? **RegExpExec**(*splitter*, *S*).
 - b. If *z* is not **null**, return *A*.
 - c. Perform ! **CreateDataProperty**(*A*, "0", *S*).
 - d. Return *A*.
18. Let *q* be *p*.
19. Repeat, while *q* < *size*
 - a. Perform ? **Set**(*splitter*, "lastIndex", *q*, **true**).
 - b. Let *z* be ? **RegExpExec**(*splitter*, *S*).
 - c. If *z* is **null**, set *q* to **AdvanceStringIndex**(*S*, *q*, *unicodeMatching*).
 - d. Else *z* is not **null**,
 - i. Let *e* be ? **ToLength**(? **Get**(*splitter*, "lastIndex")).
 - ii. Set *e* to **min**(*e*, *size*).
 - iii. If *e* = *p*, set *q* to **AdvanceStringIndex**(*S*, *q*, *unicodeMatching*).
 - iv. Else *e* ≠ *p*,
 1. Let *T* be the String value equal to the substring of *S* consisting of the code units at indices *p* (inclusive) through *q* (exclusive).
 2. Perform ! **CreateDataProperty**(*A*, ! **ToString**(*lengthA*), *T*).
 3. Increase *lengthA* by 1.
 4. If *lengthA* = *lim*, return *A*.
 5. Set *p* to *e*.
 6. Let *numberOfCaptures* be ? **ToLength**(? **Get**(*z*, "length")).
 7. Set *numberOfCaptures* to **max**(*numberOfCaptures* - 1, 0).
 8. Let *i* be 1.
 9. Repeat, while *i* ≤ *numberOfCaptures*,
 - a. Let *nextCapture* be ? **Get**(*z*, ! **ToString**(*i*)).
 - b. Perform ! **CreateDataProperty**(*A*, ! **ToString**(*lengthA*), *nextCapture*).
 - c. Increase *i* by 1.
 - d. Increase *lengthA* by 1.
 - e. If *lengthA* = *lim*, return *A*.
 10. Set *q* to *p*.
20. Let *T* be the String value equal to the substring of *S* consisting of the code units at indices *p* (inclusive) through *size* (exclusive).
21. Perform ! **CreateDataProperty**(*A*, ! **ToString**(*lengthA*), *T*).
22. Return *A*.

The value of the **name** property of this function is "[**Symbol.split**]".

NOTE 2

The @@**split** method ignores the value of the **global** and **sticky** properties of this RegExp object.

21.2.5.13 get RegExp.prototype.sticky

RegExp.prototype.sticky is an [accessor property](#) whose set accessor function is **undefined**. Its get accessor function performs the following steps:

1. Let *R* be the **this** value.
2. If **Type**(*R*) is not Object, throw a **TypeError** exception.
3. If *R* does not have an **[[OriginalFlags]]** internal slot, then
 - a. If **SameValue**(*R*, %RegExpPrototype%) is **true**, return **undefined**.
 - b. Otherwise, throw a **TypeError** exception.
4. Let *flags* be *R*.**[[OriginalFlags]]**.
5. If *flags* contains the code unit 0x0079 (LATIN SMALL LETTER Y), return **true**.
6. Return **false**.

21.2.5.14 RegExp.prototype.test (*S*)

The following steps are taken:

1. Let *R* be the **this** value.
2. If **Type**(*R*) is not Object, throw a **TypeError** exception.
3. Let *string* be ? **Tostring**(*S*).
4. Let *match* be ? **RegExpExec**(*R*, *string*).
5. If *match* is not **null**, return **true**; else return **false**.

21.2.5.15 RegExp.prototype.toString ()

1. Let *R* be the **this** value.
2. If **Type**(*R*) is not Object, throw a **TypeError** exception.
3. Let *pattern* be ? **Tostring**(? **Get**(*R*, "source")).
4. Let *flags* be ? **Tostring**(? **Get**(*R*, "flags")).
5. Let *result* be the **string-concatenation** of "/", *pattern*, "/", and *flags*.
6. Return *result*.

NOTE

The returned String has the form of a *RegularExpressionLiteral* that evaluates to another RegExp object with the same behaviour as this object.

21.2.5.16 get RegExp.prototype.unicode

RegExp.prototype.unicode is an [accessor property](#) whose set accessor function is **undefined**. Its get accessor function performs the following steps:

1. Let *R* be the **this** value.
2. If **Type**(*R*) is not Object, throw a **TypeError** exception.
3. If *R* does not have an **[[OriginalFlags]]** internal slot, then
 - a. If **SameValue**(*R*, %RegExpPrototype%) is **true**, return **undefined**.
 - b. Otherwise, throw a **TypeError** exception.
4. Let *flags* be *R*.**[[OriginalFlags]]**.
5. If *flags* contains the code unit 0x0075 (LATIN SMALL LETTER U), return **true**.

6. Return **false**.

21.2.6 Properties of RegExp Instances

RegExp instances are ordinary objects that inherit properties from the RegExp prototype object. RegExp instances have internal slots `[[RegExpMatcher]]`, `[[OriginalSource]]`, and `[[OriginalFlags]]`. The value of the `[[RegExpMatcher]]` internal slot is an implementation-dependent representation of the *Pattern* of the RegExp object.

NOTE

Prior to ECMAScript 2015, **RegExp** instances were specified as having the own data properties **source**, **global**, **ignoreCase**, and **multiline**. Those properties are now specified as accessor properties of `RegExp.prototype`.

RegExp instances also have the following property:

21.2.6.1 lastIndex

The value of the **lastIndex** property specifies the String index at which to start the next match. It is coerced to an integer when used (see 21.2.5.2.2). This property shall have the attributes { `[[Writable]]`: **true**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **false** }.

22 Indexed Collections

22.1 Array Objects

Array objects are exotic objects that give special treatment to a certain class of property names. See 9.4.2 for a definition of this special treatment.

22.1.1 The Array Constructor

The Array [constructor](#):

is the intrinsic object `%Array%`.

is the initial value of the **Array** property of the [global object](#).

creates and initializes a new Array [exotic object](#) when called as a [constructor](#).

also creates and initializes a new Array object when called as a function rather than as a [constructor](#). Thus the function call **Array(...)** is equivalent to the object creation expression **new Array(...)** with the same arguments.

is a single function whose behaviour is overloaded based upon the number and types of its arguments.

is designed to be subclassable. It may be used as the value of an **extends** clause of a class definition. Subclass constructors that intend to inherit the exotic **Array** behaviour must include a **super** call to the **Array** [constructor](#) to initialize subclass instances that are Array exotic objects. However, most of the **Array.prototype** methods are generic methods that are not dependent upon their **this** value being an Array [exotic object](#).

has a "**length**" property whose value is 1.

22.1.1.1 Array ()

This description applies if and only if the Array [constructor](#) is called with no arguments.

1. Let *numberOfArgs* be the number of arguments passed to this function call.
2. **Assert:** *numberOfArgs* = 0.
3. If NewTarget is **undefined**, let *newTarget* be the [active function object](#), else let *newTarget* be NewTarget.
4. Let *proto* be ? [GetPrototypeFromConstructor](#)(*newTarget*, "%ArrayPrototype%").
5. Return ! [ArrayCreate](#)(0, *proto*).

22.1.1.2 Array (*len*)

This description applies if and only if the Array [constructor](#) is called with exactly one argument.

1. Let *numberOfArgs* be the number of arguments passed to this function call.
2. **Assert:** *numberOfArgs* = 1.
3. If NewTarget is **undefined**, let *newTarget* be the [active function object](#), else let *newTarget* be NewTarget.
4. Let *proto* be ? [GetPrototypeFromConstructor](#)(*newTarget*, "%ArrayPrototype%").
5. Let *array* be ! [ArrayCreate](#)(0, *proto*).
6. If [Type](#)(*len*) is not Number, then
 - a. Let *defineStatus* be [CreateDataProperty](#)(*array*, "0", *len*).
 - b. **Assert:** *defineStatus* is **true**.
 - c. Let *intLen* be 1.
7. Else,
 - a. Let *intLen* be [ToUint32](#)(*len*).
 - b. If *intLen* ≠ *len*, throw a **RangeError** exception.
8. Perform ! [Set](#)(*array*, "length", *intLen*, **true**).
9. Return *array*.

22.1.1.3 Array (...*items*)

This description applies if and only if the Array [constructor](#) is called with at least two arguments.

When the **Array** function is called, the following steps are taken:

1. Let *numberOfArgs* be the number of arguments passed to this function call.
2. **Assert:** *numberOfArgs* ≥ 2.
3. If NewTarget is **undefined**, let *newTarget* be the [active function object](#), else let *newTarget* be NewTarget.
4. Let *proto* be ? [GetPrototypeFromConstructor](#)(*newTarget*, "%ArrayPrototype%").
5. Let *array* be ? [ArrayCreate](#)(*numberOfArgs*, *proto*).
6. Let *k* be 0.
7. Let *items* be a zero-origin [List](#) containing the argument items in order.
8. Repeat, while *k* < *numberOfArgs*
 - a. Let *Pk* be ! [ToString](#)(*k*).
 - b. Let *itemK* be *items*[*k*].
 - c. Let *defineStatus* be [CreateDataProperty](#)(*array*, *Pk*, *itemK*).
 - d. **Assert:** *defineStatus* is **true**.
 - e. Increase *k* by 1.
9. **Assert:** The value of *array*'s "length" property is *numberOfArgs*.
10. Return *array*.

22.1.2 Properties of the Array Constructor

The Array [constructor](#):

has a `[[Prototype]]` internal slot whose value is the intrinsic object `%FunctionPrototype%`.
has the following properties:

22.1.2.1 `Array.from ( items [ , mapfn [ , thisArg ] ] )`

When the **from** method is called with argument *items* and optional arguments *mapfn* and *thisArg*, the following steps are taken:

1. Let *C* be the **this** value.
2. If *mapfn* is **undefined**, let *mapping* be **false**.
3. Else,
 - a. If `IsCallable(mapfn)` is **false**, throw a **TypeError** exception.
 - b. If *thisArg* is present, let *T* be *thisArg*; else let *T* be **undefined**.
 - c. Let *mapping* be **true**.
4. Let *usingIterator* be ? `GetMethod(items, @@iterator)`.
5. If *usingIterator* is not **undefined**, then
 - a. If `IsConstructor(C)` is **true**, then
 - i. Let *A* be ? `Construct(C)`.
 - b. Else,
 - i. Let *A* be ! `ArrayCreate(0)`.
 - c. Let *iteratorRecord* be ? `GetIterator(items, sync, usingIterator)`.
 - d. Let *k* be 0.
 - e. Repeat,
 - i. If $k \geq 2^{53} - 1$, then
 1. Let *error* be `ThrowCompletion`(a newly created **TypeError** object).
 2. Return ? `IteratorClose(iteratorRecord, error)`.
 - ii. Let *Pk* be ! `ToString(k)`.
 - iii. Let *next* be ? `IteratorStep(iteratorRecord)`.
 - iv. If *next* is **false**, then
 1. Perform ? `Set(A, "length", k, true)`.
 2. Return *A*.
 - v. Let *nextValue* be ? `IteratorValue(next)`.
 - vi. If *mapping* is **true**, then
 1. Let *mappedValue* be `Call(mapfn, T, « nextValue, k »)`.
 2. If *mappedValue* is an abrupt completion, return ? `IteratorClose(iteratorRecord, mappedValue)`.
 3. Set *mappedValue* to *mappedValue*.`[[Value]]`.
 - vii. Else, let *mappedValue* be *nextValue*.
 - viii. Let *defineStatus* be `CreateDataPropertyOrThrow(A, Pk, mappedValue)`.
 - ix. If *defineStatus* is an abrupt completion, return ? `IteratorClose(iteratorRecord, defineStatus)`.
 - x. Increase *k* by 1.
6. NOTE: *items* is not an Iterable so assume it is an array-like object.
7. Let *arrayLike* be ! `ToObject(items)`.
8. Let *len* be ? `ToLength(? Get(arrayLike, "length"))`.
9. If `IsConstructor(C)` is **true**, then
 - a. Let *A* be ? `Construct(C, « len »)`.

10. Else,
 - a. Let *A* be ? *ArrayCreate*(*len*).
11. Let *k* be 0.
12. Repeat, while *k* < *len*
 - a. Let *Pk* be ! *ToString*(*k*).
 - b. Let *kValue* be ? *Get*(*arrayLike*, *Pk*).
 - c. If *mapping* is **true**, then
 - i. Let *mappedValue* be ? *Call*(*mapfn*, *T*, « *kValue*, *k* »).
 - d. Else, let *mappedValue* be *kValue*.
 - e. Perform ? *CreateDataPropertyOrThrow*(*A*, *Pk*, *mappedValue*).
 - f. Increase *k* by 1.
13. Perform ? *Set*(*A*, "**length**", *len*, **true**).
14. Return *A*.

NOTE

The **from** function is an intentionally generic factory method; it does not require that its **this** value be the *Array constructor*. Therefore it can be transferred to or inherited by any other constructors that may be called with a single numeric argument.

22.1.2.2 *Array.isArray* (*arg*)

The **isArray** function takes one argument *arg*, and performs the following steps:

1. Return ? *IsArray*(*arg*).

22.1.2.3 *Array.of* (...*items*)

When the **of** method is called with any number of arguments, the following steps are taken:

1. Let *len* be the actual number of arguments passed to this function.
2. Let *items* be the *List* of arguments passed to this function.
3. Let *C* be the **this** value.
4. If *IsConstructor*(*C*) is **true**, then
 - a. Let *A* be ? *Construct*(*C*, « *len* »).
5. Else,
 - a. Let *A* be ? *ArrayCreate*(*len*).
6. Let *k* be 0.
7. Repeat, while *k* < *len*
 - a. Let *kValue* be *items*[*k*].
 - b. Let *Pk* be ! *ToString*(*k*).
 - c. Perform ? *CreateDataPropertyOrThrow*(*A*, *Pk*, *kValue*).
 - d. Increase *k* by 1.
8. Perform ? *Set*(*A*, "**length**", *len*, **true**).
9. Return *A*.

NOTE 1

The *items* argument is assumed to be a well-formed rest argument value.

NOTE 2

The **of** function is an intentionally generic factory method; it does not require that its **this** value be the *Array*

constructor. Therefore it can be transferred to or inherited by other constructors that may be called with a single numeric argument.

22.1.2.4 **Array.prototype**

The value of **Array.prototype** is **%ArrayPrototype%**, the intrinsic Array prototype object.

This property has the attributes { **[[Writable]]: false**, **[[Enumerable]]: false**, **[[Configurable]]: false** }.

22.1.2.5 **get Array [@@species]**

Array[@@species] is an **accessor property** whose set accessor function is **undefined**. Its get accessor function performs the following steps:

1. Return the **this** value.

The value of the **name** property of this function is **"get [Symbol.species]"**.

NOTE

Array prototype methods normally use their **this** object's **constructor** to create a derived object. However, a subclass **constructor** may over-ride that default behaviour by redefining its **@@species** property.

22.1.3 Properties of the Array Prototype Object

The Array prototype object:

is the intrinsic object **%ArrayPrototype%**.

is an Array **exotic object** and has the internal methods specified for such objects.

has a **"length"** property whose initial value is 0 and whose attributes are { **[[Writable]]: true**, **[[Enumerable]]: false**, **[[Configurable]]: false** }.

has a **[[Prototype]]** internal slot whose value is the intrinsic object **%ObjectPrototype%**.

NOTE

The Array prototype object is specified to be an Array **exotic object** to ensure compatibility with ECMAScript code that was created prior to the ECMAScript 2015 specification.

22.1.3.1 **Array.prototype.concat (...arguments)**

When the **concat** method is called with zero or more arguments, it returns an array containing the array elements of the object followed by the array elements of each argument in order.

The following steps are taken:

1. Let **O** be ? **ToObject(this value)**.
2. Let **A** be ? **ArraySpeciesCreate(O, 0)**.
3. Let **n** be 0.
4. Let **items** be a **List** whose first element is **O** and whose subsequent elements are, in left to right order, the arguments that were passed to this function invocation.

5. Repeat, while *items* is not empty
 - a. Remove the first element from *items* and let *E* be the value of the element.
 - b. Let *spreadable* be ? *IsConcatSpreadable*(*E*).
 - c. If *spreadable* is **true**, then
 - i. Let *k* be 0.
 - ii. Let *len* be ? *ToLength*(? *Get*(*E*, "**length**").
 - iii. If $n + len > 2^{53} - 1$, throw a **TypeError** exception.
 - iv. Repeat, while $k < len$
 1. Let *P* be ! *ToString*(*k*).
 2. Let *exists* be ? *HasProperty*(*E*, *P*).
 3. If *exists* is **true**, then
 - a. Let *subElement* be ? *Get*(*E*, *P*).
 - b. Perform ? *CreateDataPropertyOrThrow*(*A*, ! *ToString*(*n*), *subElement*).
 4. Increase *n* by 1.
 5. Increase *k* by 1.
 - d. Else *E* is added as a single item rather than spread,
 - i. If $n \geq 2^{53} - 1$, throw a **TypeError** exception.
 - ii. Perform ? *CreateDataPropertyOrThrow*(*A*, ! *ToString*(*n*), *E*).
 - iii. Increase *n* by 1.
6. Perform ? *Set*(*A*, "**length**", *n*, **true**).
7. Return *A*.

The "**length**" property of the **concat** method is 1.

NOTE 1

The explicit setting of the "**length**" property in step 6 is necessary to ensure that its value is correct in situations where the trailing elements of the result Array are not present.

NOTE 2

The **concat** function is intentionally generic; it does not require that its **this** value be an Array object. Therefore it can be transferred to other kinds of objects for use as a method.

22.1.3.1.1 Runtime Semantics: *IsConcatSpreadable* (*O*)

The abstract operation *IsConcatSpreadable* with argument *O* performs the following steps:

1. If *Type*(*O*) is not Object, return **false**.
2. Let *spreadable* be ? *Get*(*O*, @@*isConcatSpreadable*).
3. If *spreadable* is not **undefined**, return *ToBoolean*(*spreadable*).
4. Return ? *IsArray*(*O*).

22.1.3.2 **Array.prototype.constructor**

The initial value of **Array.prototype.constructor** is the intrinsic object %Array%.

22.1.3.3 **Array.prototype.copyWithin** (*target*, *start* [, *end*])

The **copyWithin** method takes up to three arguments *target*, *start* and *end*.

NOTE 1

The *end* argument is optional with the length of the **this** object as its default value. If *target* is negative, it is treated as *length* + *target* where *length* is the length of the array. If *start* is negative, it is treated as *length* + *start*. If *end* is negative, it is treated as *length* + *end*.

The following steps are taken:

1. Let *O* be ? **ToObject**(**this** value).
2. Let *len* be ? **ToLength**(? **Get**(*O*, "**length**").
3. Let *relativeTarget* be ? **ToInteger**(*target*).
4. If *relativeTarget* < 0, let *to* be **max**((*len* + *relativeTarget*), 0); else let *to* be **min**(*relativeTarget*, *len*).
5. Let *relativeStart* be ? **ToInteger**(*start*).
6. If *relativeStart* < 0, let *from* be **max**((*len* + *relativeStart*), 0); else let *from* be **min**(*relativeStart*, *len*).
7. If *end* is **undefined**, let *relativeEnd* be *len*; else let *relativeEnd* be ? **ToInteger**(*end*).
8. If *relativeEnd* < 0, let *final* be **max**((*len* + *relativeEnd*), 0); else let *final* be **min**(*relativeEnd*, *len*).
9. Let *count* be **min**(*final* - *from*, *len* - *to*).
10. If *from* < *to* and *to* < *from* + *count*, then
 - a. Let *direction* be -1.
 - b. Set *from* to *from* + *count* - 1.
 - c. Set *to* to *to* + *count* - 1.
11. Else,
 - a. Let *direction* be 1.
12. Repeat, while *count* > 0
 - a. Let *fromKey* be ! **ToString**(*from*).
 - b. Let *toKey* be ! **ToString**(*to*).
 - c. Let *fromPresent* be ? **HasProperty**(*O*, *fromKey*).
 - d. If *fromPresent* is **true**, then
 - i. Let *fromVal* be ? **Get**(*O*, *fromKey*).
 - ii. Perform ? **Set**(*O*, *toKey*, *fromVal*, **true**).
 - e. Else *fromPresent* is **false**,
 - i. Perform ? **DeletePropertyOrThrow**(*O*, *toKey*).
 - f. Set *from* to *from* + *direction*.
 - g. Set *to* to *to* + *direction*.
 - h. Decrease *count* by 1.
13. Return *O*.

NOTE 2

The **copyWithin** function is intentionally generic; it does not require that its **this** value be an Array object. Therefore it can be transferred to other kinds of objects for use as a method.

22.1.3.4 Array.prototype.entries ()

The following steps are taken:

1. Let *O* be ? **ToObject**(**this** value).
2. Return **CreateArrayIterator**(*O*, "**key+value**").

This function is the %ArrayProto_entries% intrinsic object.

22.1.3.5 Array.prototype.every (*callbackfn* [, *thisArg*])

NOTE 1

callbackfn should be a function that accepts three arguments and returns a value that is coercible to the Boolean value **true** or **false**. **every** calls *callbackfn* once for each element present in the array, in ascending order, until it finds one where *callbackfn* returns **false**. If such an element is found, **every** immediately returns **false**. Otherwise, if *callbackfn* returned **true** for all elements, **every** will return **true**. *callbackfn* is called only for elements of the array which actually exist; it is not called for missing elements of the array.

If a *thisArg* parameter is provided, it will be used as the **this** value for each invocation of *callbackfn*. If it is not provided, **undefined** is used instead.

callbackfn is called with three arguments: the value of the element, the index of the element, and the object being traversed.

every does not directly mutate the object on which it is called but the object may be mutated by the calls to *callbackfn*.

The range of elements processed by **every** is set before the first call to *callbackfn*. Elements which are appended to the array after the call to **every** begins will not be visited by *callbackfn*. If existing elements of the array are changed, their value as passed to *callbackfn* will be the value at the time **every** visits them; elements that are deleted after the call to **every** begins and before being visited are not visited. **every** acts like the "for all" quantifier in mathematics. In particular, for an empty array, it returns **true**.

When the **every** method is called with one or two arguments, the following steps are taken:

1. Let *O* be ? **ToObject**(**this** value).
2. Let *len* be ? **ToLength**(? **Get**(*O*, "length")).
3. If **IsCallable**(*callbackfn*) is **false**, throw a **TypeError** exception.
4. If *thisArg* is present, let *T* be *thisArg*; else let *T* be **undefined**.
5. Let *k* be 0.
6. Repeat, while *k* < *len*
 - a. Let *Pk* be ! **ToString**(*k*).
 - b. Let *kPresent* be ? **HasProperty**(*O*, *Pk*).
 - c. If *kPresent* is **true**, then
 - i. Let *kValue* be ? **Get**(*O*, *Pk*).
 - ii. Let *testResult* be **ToBoolean**(? **Call**(*callbackfn*, *T*, « *kValue*, *k*, *O* »)).
 - iii. If *testResult* is **false**, return **false**.
 - d. Increase *k* by 1.
7. Return **true**.

NOTE 2

The **every** function is intentionally generic; it does not require that its **this** value be an Array object. Therefore it can be transferred to other kinds of objects for use as a method.

22.1.3.6 Array.prototype.fill (*value* [, *start* [, *end*]])

The **fill** method takes up to three arguments *value*, *start* and *end*.

NOTE 1

The *start* and *end* arguments are optional with default values of 0 and the length of the **this** object. If *start* is negative, it

is treated as *length* + *start* where *length* is the length of the array. If *end* is negative, it is treated as *length* + *end*.

The following steps are taken:

1. Let *O* be ? **ToObject**(**this** value).
2. Let *len* be ? **ToLength**(? **Get**(*O*, "**length**").
3. Let *relativeStart* be ? **ToInteger**(*start*).
4. If *relativeStart* < 0, let *k* be **max**((*len* + *relativeStart*), 0); else let *k* be **min**(*relativeStart*, *len*).
5. If *end* is **undefined**, let *relativeEnd* be *len*; else let *relativeEnd* be ? **ToInteger**(*end*).
6. If *relativeEnd* < 0, let *final* be **max**((*len* + *relativeEnd*), 0); else let *final* be **min**(*relativeEnd*, *len*).
7. Repeat, while *k* < *final*
 - a. Let *Pk* be ! **ToString**(*k*).
 - b. Perform ? **Set**(*O*, *Pk*, *value*, **true**).
 - c. Increase *k* by 1.
8. Return *O*.

NOTE 2

The **fill** function is intentionally generic; it does not require that its **this** value be an Array object. Therefore it can be transferred to other kinds of objects for use as a method.

22.1.3.7 Array.prototype.filter (*callbackfn* [, *thisArg*])

NOTE 1

callbackfn should be a function that accepts three arguments and returns a value that is coercible to the Boolean value **true** or **false**. **filter** calls *callbackfn* once for each element in the array, in ascending order, and constructs a new array of all the values for which *callbackfn* returns **true**. *callbackfn* is called only for elements of the array which actually exist; it is not called for missing elements of the array.

If a *thisArg* parameter is provided, it will be used as the **this** value for each invocation of *callbackfn*. If it is not provided, **undefined** is used instead.

callbackfn is called with three arguments: the value of the element, the index of the element, and the object being traversed.

filter does not directly mutate the object on which it is called but the object may be mutated by the calls to *callbackfn*.

The range of elements processed by **filter** is set before the first call to *callbackfn*. Elements which are appended to the array after the call to **filter** begins will not be visited by *callbackfn*. If existing elements of the array are changed their value as passed to *callbackfn* will be the value at the time **filter** visits them; elements that are deleted after the call to **filter** begins and before being visited are not visited.

When the **filter** method is called with one or two arguments, the following steps are taken:

1. Let *O* be ? **ToObject**(**this** value).
2. Let *len* be ? **ToLength**(? **Get**(*O*, "**length**").
3. If **IsCallable**(*callbackfn*) is **false**, throw a **TypeError** exception.
4. If *thisArg* is present, let *T* be *thisArg*; else let *T* be **undefined**.
5. Let *A* be ? **ArraySpeciesCreate**(*O*, 0).

6. Let *k* be 0.
7. Let *to* be 0.
8. Repeat, while *k* < *len*
 - a. Let *Pk* be ! ToString(*k*).
 - b. Let *kPresent* be ? HasProperty(*O*, *Pk*).
 - c. If *kPresent* is **true**, then
 - i. Let *kValue* be ? Get(*O*, *Pk*).
 - ii. Let *selected* be ToBoolean(? Call(*callbackfn*, *T*, « *kValue*, *k*, *O* »)).
 - iii. If *selected* is **true**, then
 1. Perform ? CreateDataPropertyOrThrow(*A*, ! ToString(*to*), *kValue*).
 2. Increase *to* by 1.
 - d. Increase *k* by 1.
9. Return *A*.

NOTE 2

The **filter** function is intentionally generic; it does not require that its **this** value be an Array object. Therefore it can be transferred to other kinds of objects for use as a method.

22.1.3.8 Array.prototype.find (*predicate* [, *thisArg*])

The **find** method is called with one or two arguments, *predicate* and *thisArg*.

NOTE 1

predicate should be a function that accepts three arguments and returns a value that is coercible to a Boolean value. **find** calls *predicate* once for each element of the array, in ascending order, until it finds one where *predicate* returns **true**. If such an element is found, **find** immediately returns that element value. Otherwise, **find** returns **undefined**.

If a *thisArg* parameter is provided, it will be used as the **this** value for each invocation of *predicate*. If it is not provided, **undefined** is used instead.

predicate is called with three arguments: the value of the element, the index of the element, and the object being traversed.

find does not directly mutate the object on which it is called but the object may be mutated by the calls to *predicate*.

The range of elements processed by **find** is set before the first call to *predicate*. Elements that are appended to the array after the call to **find** begins will not be visited by *predicate*. If existing elements of the array are changed, their value as passed to *predicate* will be the value at the time that **find** visits them.

When the **find** method is called, the following steps are taken:

1. Let *O* be ? ToObject(**this** value).
2. Let *len* be ? ToLength(? Get(*O*, "length")).
3. If IsCallable(*predicate*) is **false**, throw a **TypeError** exception.
4. If *thisArg* is present, let *T* be *thisArg*; else let *T* be **undefined**.
5. Let *k* be 0.
6. Repeat, while *k* < *len*
 - a. Let *Pk* be ! ToString(*k*).
 - b. Let *kValue* be ? Get(*O*, *Pk*).

- c. Let *testResult* be **ToBoolean**(? **Call**(*predicate*, *T*, « *kValue*, *k*, *O* »)).
 - d. If *testResult* is **true**, return *kValue*.
 - e. Increase *k* by 1.
7. Return **undefined**.

NOTE 2

The **find** function is intentionally generic; it does not require that its **this** value be an Array object. Therefore it can be transferred to other kinds of objects for use as a method.

22.1.3.9 Array.prototype.findIndex (*predicate* [, *thisArg*])

NOTE 1

predicate should be a function that accepts three arguments and returns a value that is coercible to the Boolean value **true** or **false**. **findIndex** calls *predicate* once for each element of the array, in ascending order, until it finds one where *predicate* returns **true**. If such an element is found, **findIndex** immediately returns the index of that element value. Otherwise, **findIndex** returns -1.

If a *thisArg* parameter is provided, it will be used as the **this** value for each invocation of *predicate*. If it is not provided, **undefined** is used instead.

predicate is called with three arguments: the value of the element, the index of the element, and the object being traversed.

findIndex does not directly mutate the object on which it is called but the object may be mutated by the calls to *predicate*.

The range of elements processed by **findIndex** is set before the first call to *predicate*. Elements that are appended to the array after the call to **findIndex** begins will not be visited by *predicate*. If existing elements of the array are changed, their value as passed to *predicate* will be the value at the time that **findIndex** visits them.

When the **findIndex** method is called with one or two arguments, the following steps are taken:

1. Let *O* be ? **ToObject**(**this** value).
2. Let *len* be ? **ToLength**(? **Get**(*O*, "length")).
3. If **IsCallable**(*predicate*) is **false**, throw a **TypeError** exception.
4. If *thisArg* is present, let *T* be *thisArg*; else let *T* be **undefined**.
5. Let *k* be 0.
6. Repeat, while *k* < *len*
 - a. Let *Pk* be ! **ToInteger**(*k*).
 - b. Let *kValue* be ? **Get**(*O*, *Pk*).
 - c. Let *testResult* be **ToBoolean**(? **Call**(*predicate*, *T*, « *kValue*, *k*, *O* »)).
 - d. If *testResult* is **true**, return *k*.
 - e. Increase *k* by 1.
7. Return -1.

NOTE 2

The **findIndex** function is intentionally generic; it does not require that its **this** value be an Array object. Therefore it can be transferred to other kinds of objects for use as a method.

22.1.3.10 Array.prototype.flat([*depth*])

When the **flat** method is called with zero or one arguments, the following steps are taken:

1. Let *O* be ? **ToObject**(**this** value).
2. Let *sourceLen* be ? **ToLength**(? **Get**(*O*, "length")).
3. Let *depthNum* be 1.
4. If *depth* is not **undefined**, then
 - a. Set *depthNum* to ? **ToInteger**(*depth*).
5. Let *A* be ? **ArraySpeciesCreate**(*O*, 0).
6. Perform ? **FlattenIntoArray**(*A*, *O*, *sourceLen*, 0, *depthNum*).
7. Return *A*.

22.1.3.10.1 FlattenIntoArray(*target*, *source*, *sourceLen*, *start*, *depth* [, *mapperFunction*, *thisArg*])

1. Let *targetIndex* be *start*.
2. Let *sourceIndex* be 0.
3. Repeat, while *sourceIndex* < *sourceLen*
 - a. Let *P* be ! **ToString**(*sourceIndex*).
 - b. Let *exists* be ? **HasProperty**(*source*, *P*).
 - c. If *exists* is **true**, then
 - i. Let *element* be ? **Get**(*source*, *P*).
 - ii. If *mapperFunction* is present, then
 1. **Assert**: *thisArg* is present.
 2. Set *element* to ? **Call**(*mapperFunction*, *thisArg* , « *element*, *sourceIndex*, *source* »).
 - iii. Let *shouldFlatten* be **false**.
 - iv. If *depth* > 0, then
 1. Set *shouldFlatten* to ? **IsArray**(*element*).
 - v. If *shouldFlatten* is **true**, then
 1. Let *elementLen* be ? **ToLength**(? **Get**(*element*, "length")).
 2. Set *targetIndex* to ? **FlattenIntoArray**(*target*, *element*, *elementLen*, *targetIndex*, *depth* - 1).
 - vi. Else,
 1. If *targetIndex* ≥ 2⁵³-1, throw a **TypeError** exception.
 2. Perform ? **CreateDataPropertyOrThrow**(*target*, ! **ToString**(*targetIndex*), *element*).
 3. Increase *targetIndex* by 1.
 - d. Increase *sourceIndex* by 1.
4. Return *targetIndex*.

22.1.3.11 Array.prototype.flatMap (*mapperFunction* [, *thisArg*])

When the **flatMap** method is called with one or two arguments, the following steps are taken:

1. Let *O* be ? **ToObject**(**this** value).
2. Let *sourceLen* be ? **ToLength**(? **Get**(*O*, "length")).
3. If **IsCallable**(*mapperFunction*) is **false**, throw a **TypeError** exception.
4. If *thisArg* is present, let *T* be *thisArg*; else let *T* be **undefined**.
5. Let *A* be ? **ArraySpeciesCreate**(*O*, 0).
6. Perform ? **FlattenIntoArray**(*A*, *O*, *sourceLen*, 0, 1, *mapperFunction*, *T*).
7. Return *A*.

22.1.3.12 Array.prototype.forEach (*callbackfn* [, *thisArg*])

NOTE 1

callbackfn should be a function that accepts three arguments. **forEach** calls *callbackfn* once for each element present in the array, in ascending order. *callbackfn* is called only for elements of the array which actually exist; it is not called for missing elements of the array.

If a *thisArg* parameter is provided, it will be used as the **this** value for each invocation of *callbackfn*. If it is not provided, **undefined** is used instead.

callbackfn is called with three arguments: the value of the element, the index of the element, and the object being traversed.

forEach does not directly mutate the object on which it is called but the object may be mutated by the calls to *callbackfn*.

When the **forEach** method is called with one or two arguments, the following steps are taken:

1. Let *O* be ? **ToObject**(**this** value).
2. Let *len* be ? **ToLength**(? **Get**(*O*, "length")).
3. If **IsCallable**(*callbackfn*) is **false**, throw a **TypeError** exception.
4. If *thisArg* is present, let *T* be *thisArg*; else let *T* be **undefined**.
5. Let *k* be 0.
6. Repeat, while *k* < *len*
 - a. Let *Pk* be ! **ToString**(*k*).
 - b. Let *kPresent* be ? **HasProperty**(*O*, *Pk*).
 - c. If *kPresent* is **true**, then
 - i. Let *kValue* be ? **Get**(*O*, *Pk*).
 - ii. Perform ? **Call**(*callbackfn*, *T*, « *kValue*, *k*, *O* »).
 - d. Increase *k* by 1.
7. Return **undefined**.

This function is the %ArrayProto_forEach% intrinsic object.

NOTE 2

The **forEach** function is intentionally generic; it does not require that its **this** value be an Array object. Therefore it can be transferred to other kinds of objects for use as a method.

22.1.3.13 Array.prototype.includes (*searchElement* [, *fromIndex*])

NOTE 1

includes compares *searchElement* to the elements of the array, in ascending order, using the **SameValueZero** algorithm, and if found at any position, returns **true**; otherwise, **false** is returned.

The optional second argument *fromIndex* defaults to 0 (i.e. the whole array is searched). If it is greater than or equal to the length of the array, **false** is returned, i.e. the array will not be searched. If it is negative, it is used as the offset from the end of the array to compute *fromIndex*. If the computed index is less than 0, the whole array will be searched.

When the **includes** method is called, the following steps are taken:

1. Let *O* be ? **ToObject**(**this** value).
2. Let *len* be ? **ToLength**(? **Get**(*O*, "**length**").
3. If *len* is 0, return **false**.
4. Let *n* be ? **ToInteger**(*fromIndex*).
5. **Assert**: If *fromIndex* is **undefined**, then *n* is 0.
6. If *n* ≥ 0, then
 - a. Let *k* be *n*.
7. Else *n* < 0,
 - a. Let *k* be *len* + *n*.
 - b. If *k* < 0, set *k* to 0.
8. Repeat, while *k* < *len*
 - a. Let *elementK* be the result of ? **Get**(*O*, ! **ToString**(*k*)).
 - b. If **SameValueZero**(*searchElement*, *elementK*) is **true**, return **true**.
 - c. Increase *k* by 1.
9. Return **false**.

NOTE 2

The **includes** function is intentionally generic; it does not require that its **this** value be an Array object. Therefore it can be transferred to other kinds of objects for use as a method.

NOTE 3

The **includes** method intentionally differs from the similar **indexOf** method in two ways. First, it uses the **SameValueZero** algorithm, instead of **Strict Equality Comparison**, allowing it to detect **NaN** array elements. Second, it does not skip missing array elements, instead treating them as **undefined**.

22.1.3.14 Array.prototype.indexOf (*searchElement* [, *fromIndex*])

NOTE 1

indexOf compares *searchElement* to the elements of the array, in ascending order, using the **Strict Equality Comparison** algorithm, and if found at one or more indices, returns the smallest such index; otherwise, -1 is returned.

The optional second argument *fromIndex* defaults to 0 (i.e. the whole array is searched). If it is greater than or equal to the length of the array, -1 is returned, i.e. the array will not be searched. If it is negative, it is used as the offset from the end of the array to compute *fromIndex*. If the computed index is less than 0, the whole array will be searched.

When the **indexOf** method is called with one or two arguments, the following steps are taken:

1. Let *O* be ? **ToObject**(**this** value).
2. Let *len* be ? **ToLength**(? **Get**(*O*, "**length**").
3. If *len* is 0, return -1.
4. Let *n* be ? **ToInteger**(*fromIndex*).
5. **Assert**: If *fromIndex* is **undefined**, then *n* is 0.
6. If *n* ≥ *len*, return -1.
7. If *n* ≥ 0, then
 - a. If *n* is -0, let *k* be +0; else let *k* be *n*.
8. Else *n* < 0,
 - a. Let *k* be *len* + *n*.
 - b. If *k* < 0, set *k* to 0.
9. Repeat, while *k* < *len*

- a. Let *kPresent* be ? **HasProperty**(*O*, ! **ToString**(*k*)).
 - b. If *kPresent* is **true**, then
 - i. Let *elementK* be ? **Get**(*O*, ! **ToString**(*k*)).
 - ii. Let *same* be the result of performing **Strict Equality Comparison** *searchElement* === *elementK*.
 - iii. If *same* is **true**, return *k*.
 - c. Increase *k* by 1.
10. Return -1.

NOTE 2

The **indexOf** function is intentionally generic; it does not require that its **this** value be an Array object. Therefore it can be transferred to other kinds of objects for use as a method.

22.1.3.15 Array.prototype.join (*separator*)

NOTE 1

The elements of the array are converted to Strings, and these Strings are then concatenated, separated by occurrences of the *separator*. If no separator is provided, a single comma is used as the separator.

The **join** method takes one argument, *separator*, and performs the following steps:

1. Let *O* be ? **ToObject**(**this** value).
2. Let *len* be ? **ToLength**(? **Get**(*O*, "length")).
3. If *separator* is **undefined**, let *sep* be the single-element String " , ".
4. Else, let *sep* be ? **ToString**(*separator*).
5. Let *R* be the empty String.
6. Let *k* be 0.
7. Repeat, while *k* < *len*
 - a. If *k* > 0, set *R* to the **string-concatenation** of *R* and *sep*.
 - b. Let *element* be ? **Get**(*O*, ! **ToString**(*k*)).
 - c. If *element* is **undefined** or **null**, let *next* be the empty String; otherwise, let *next* be ? **ToString**(*element*).
 - d. Set *R* to the **string-concatenation** of *R* and *next*.
 - e. Increase *k* by 1.
8. Return *R*.

NOTE 2

The **join** function is intentionally generic; it does not require that its **this** value be an Array object. Therefore, it can be transferred to other kinds of objects for use as a method.

22.1.3.16 Array.prototype.keys ()

The following steps are taken:

1. Let *O* be ? **ToObject**(**this** value).
2. Return **CreateArrayIterator**(*O*, "key").

This function is the %ArrayProto_keys% intrinsic object.

22.1.3.17 Array.prototype.lastIndexOf (*searchElement* [, *fromIndex*])

NOTE 1

lastIndexOf compares *searchElement* to the elements of the array in descending order using the **Strict Equality Comparison** algorithm, and if found at one or more indices, returns the largest such index; otherwise, -1 is returned.

The optional second argument *fromIndex* defaults to the array's length minus one (i.e. the whole array is searched). If it is greater than or equal to the length of the array, the whole array will be searched. If it is negative, it is used as the offset from the end of the array to compute *fromIndex*. If the computed index is less than 0, -1 is returned.

When the **lastIndexOf** method is called with one or two arguments, the following steps are taken:

1. Let *O* be ? **ToObject**(**this** value).
2. Let *len* be ? **ToLength**(? **Get**(*O*, "length")).
3. If *len* is 0, return -1.
4. If *fromIndex* is present, let *n* be ? **ToInteger**(*fromIndex*); else let *n* be *len* - 1.
5. If *n* ≥ 0, then
 - a. If *n* is -0, let *k* be +0; else let *k* be **min**(*n*, *len* - 1).
6. Else *n* < 0,
 - a. Let *k* be *len* + *n*.
7. Repeat, while *k* ≥ 0
 - a. Let *kPresent* be ? **HasProperty**(*O*, ! **ToString**(*k*)).
 - b. If *kPresent* is **true**, then
 - i. Let *elementK* be ? **Get**(*O*, ! **ToString**(*k*)).
 - ii. Let *same* be the result of performing **Strict Equality Comparison** *searchElement* === *elementK*.
 - iii. If *same* is **true**, return *k*.
 - c. Decrease *k* by 1.
8. Return -1.

NOTE 2

The **lastIndexOf** function is intentionally generic; it does not require that its **this** value be an Array object. Therefore it can be transferred to other kinds of objects for use as a method.

22.1.3.18 Array.prototype.map (*callbackfn* [, *thisArg*])

NOTE 1

callbackfn should be a function that accepts three arguments. **map** calls *callbackfn* once for each element in the array, in ascending order, and constructs a new Array from the results. *callbackfn* is called only for elements of the array which actually exist; it is not called for missing elements of the array.

If a *thisArg* parameter is provided, it will be used as the **this** value for each invocation of *callbackfn*. If it is not provided, **undefined** is used instead.

callbackfn is called with three arguments: the value of the element, the index of the element, and the object being traversed.

map does not directly mutate the object on which it is called but the object may be mutated by the calls to *callbackfn*.

The range of elements processed by **map** is set before the first call to *callbackfn*. Elements which are appended to the array after the call to **map** begins will not be visited by *callbackfn*. If existing elements of the array are changed, their value as passed to *callbackfn* will be the value at the time **map** visits them; elements that are deleted after the call to **map**

begins and before being visited are not visited.

When the **map** method is called with one or two arguments, the following steps are taken:

1. Let *O* be ? **ToObject**(**this** value).
2. Let *len* be ? **ToLength**(? **Get**(*O*, "length")).
3. If **IsCallable**(*callbackfn*) is **false**, throw a **TypeError** exception.
4. If *thisArg* is present, let *T* be *thisArg*; else let *T* be **undefined**.
5. Let *A* be ? **ArraySpeciesCreate**(*O*, *len*).
6. Let *k* be 0.
7. Repeat, while *k* < *len*
 - a. Let *Pk* be ! **ToString**(*k*).
 - b. Let *kPresent* be ? **HasProperty**(*O*, *Pk*).
 - c. If *kPresent* is **true**, then
 - i. Let *kValue* be ? **Get**(*O*, *Pk*).
 - ii. Let *mappedValue* be ? **Call**(*callbackfn*, *T*, « *kValue*, *k*, *O* »).
 - iii. Perform ? **CreateDataPropertyOrThrow**(*A*, *Pk*, *mappedValue*).
 - d. Increase *k* by 1.
8. Return *A*.

NOTE 2

The **map** function is intentionally generic; it does not require that its **this** value be an Array object. Therefore it can be transferred to other kinds of objects for use as a method.

22.1.3.19 Array.prototype.pop ()

NOTE 1

The last element of the array is removed from the array and returned.

When the **pop** method is called, the following steps are taken:

1. Let *O* be ? **ToObject**(**this** value).
2. Let *len* be ? **ToLength**(? **Get**(*O*, "length")).
3. If *len* is zero, then
 - a. Perform ? **Set**(*O*, "length", 0, **true**).
 - b. Return **undefined**.
4. Else *len* > 0,
 - a. Let *newLen* be *len* - 1.
 - b. Let *index* be ! **ToString**(*newLen*).
 - c. Let *element* be ? **Get**(*O*, *index*).
 - d. Perform ? **DeletePropertyOrThrow**(*O*, *index*).
 - e. Perform ? **Set**(*O*, "length", *newLen*, **true**).
 - f. Return *element*.

NOTE 2

The **pop** function is intentionally generic; it does not require that its **this** value be an Array object. Therefore it can be transferred to other kinds of objects for use as a method.

22.1.3.20 Array.prototype.push (...*items*)

NOTE 1

The arguments are appended to the end of the array, in the order in which they appear. The new length of the array is returned as the result of the call.

When the **push** method is called with zero or more arguments, the following steps are taken:

1. Let *O* be ? **ToObject**(**this** value).
2. Let *len* be ? **ToLength**(? **Get**(*O*, "length")).
3. Let *items* be a **List** whose elements are, in left to right order, the arguments that were passed to this function invocation.
4. Let *argCount* be the number of elements in *items*.
5. If *len* + *argCount* > 2⁵³ - 1, throw a **TypeError** exception.
6. Repeat, while *items* is not empty
 - a. Remove the first element from *items* and let *E* be the value of the element.
 - b. Perform ? **Set**(*O*, ! **ToString**(*len*), *E*, **true**).
 - c. Increase *len* by 1.
7. Perform ? **Set**(*O*, "length", *len*, **true**).
8. Return *len*.

The "length" property of the **push** method is 1.

NOTE 2

The **push** function is intentionally generic; it does not require that its **this** value be an Array object. Therefore it can be transferred to other kinds of objects for use as a method.

22.1.3.21 Array.prototype.reduce (*callbackfn* [, *initialValue*])

NOTE 1

callbackfn should be a function that takes four arguments. **reduce** calls the callback, as a function, once for each element after the first element present in the array, in ascending order.

callbackfn is called with four arguments: the *previousValue* (value from the previous call to *callbackfn*), the *currentValue* (value of the current element), the *currentIndex*, and the object being traversed. The first time that callback is called, the *previousValue* and *currentValue* can be one of two values. If an *initialValue* was supplied in the call to **reduce**, then *previousValue* will be equal to *initialValue* and *currentValue* will be equal to the first value in the array. If no *initialValue* was supplied, then *previousValue* will be equal to the first value in the array and *currentValue* will be equal to the second. It is a **TypeError** if the array contains no elements and *initialValue* is not provided.

reduce does not directly mutate the object on which it is called but the object may be mutated by the calls to *callbackfn*.

The range of elements processed by **reduce** is set before the first call to *callbackfn*. Elements that are appended to the array after the call to **reduce** begins will not be visited by *callbackfn*. If existing elements of the array are changed, their value as passed to *callbackfn* will be the value at the time **reduce** visits them; elements that are deleted after the call to **reduce** begins and before being visited are not visited.

When the **reduce** method is called with one or two arguments, the following steps are taken:

1. Let *O* be ? **ToObject**(**this** value).
2. Let *len* be ? **ToLength**(? **Get**(*O*, "length")).
3. If **IsCallable**(*callbackfn*) is **false**, throw a **TypeError** exception.
4. If *len* is 0 and *initialValue* is not present, throw a **TypeError** exception.
5. Let *k* be 0.
6. Let *accumulator* be **undefined**.
7. If *initialValue* is present, then
 - a. Set *accumulator* to *initialValue*.
8. Else *initialValue* is not present,
 - a. Let *kPresent* be **false**.
 - b. Repeat, while *kPresent* is **false** and *k* < *len*
 - i. Let *Pk* be ! **ToString**(*k*).
 - ii. Set *kPresent* to ? **HasProperty**(*O*, *Pk*).
 - iii. If *kPresent* is **true**, then
 1. Set *accumulator* to ? **Get**(*O*, *Pk*).
 - iv. Increase *k* by 1.
 - c. If *kPresent* is **false**, throw a **TypeError** exception.
9. Repeat, while *k* < *len*
 - a. Let *Pk* be ! **ToString**(*k*).
 - b. Let *kPresent* be ? **HasProperty**(*O*, *Pk*).
 - c. If *kPresent* is **true**, then
 - i. Let *kValue* be ? **Get**(*O*, *Pk*).
 - ii. Set *accumulator* to ? **Call**(*callbackfn*, **undefined**, « *accumulator*, *kValue*, *k*, *O* »).
 - d. Increase *k* by 1.
10. Return *accumulator*.

NOTE 2

The **reduce** function is intentionally generic; it does not require that its **this** value be an Array object. Therefore it can be transferred to other kinds of objects for use as a method.

22.1.3.22 Array.prototype.reduceRight (*callbackfn* [, *initialValue*])

NOTE 1

callbackfn should be a function that takes four arguments. **reduceRight** calls the callback, as a function, once for each element after the first element present in the array, in descending order.

callbackfn is called with four arguments: the *previousValue* (value from the previous call to *callbackfn*), the *currentValue* (value of the current element), the *currentIndex*, and the object being traversed. The first time the function is called, the *previousValue* and *currentValue* can be one of two values. If an *initialValue* was supplied in the call to **reduceRight**, then *previousValue* will be equal to *initialValue* and *currentValue* will be equal to the last value in the array. If no *initialValue* was supplied, then *previousValue* will be equal to the last value in the array and *currentValue* will be equal to the second-to-last value. It is a **TypeError** if the array contains no elements and *initialValue* is not provided.

reduceRight does not directly mutate the object on which it is called but the object may be mutated by the calls to *callbackfn*.

The range of elements processed by **reduceRight** is set before the first call to *callbackfn*. Elements that are appended to the array after the call to **reduceRight** begins will not be visited by *callbackfn*. If existing elements of the array are changed by *callbackfn*, their value as passed to *callbackfn* will be the value at the time **reduceRight** visits them;

elements that are deleted after the call to **reduceRight** begins and before being visited are not visited.

When the **reduceRight** method is called with one or two arguments, the following steps are taken:

1. Let *O* be ? **ToObject**(**this** value).
2. Let *len* be ? **ToLength**(? **Get**(*O*, "length")).
3. If **IsCallable**(*callbackfn*) is **false**, throw a **TypeError** exception.
4. If *len* is 0 and *initialValue* is not present, throw a **TypeError** exception.
5. Let *k* be *len* - 1.
6. Let *accumulator* be **undefined**.
7. If *initialValue* is present, then
 - a. Set *accumulator* to *initialValue*.
8. Else *initialValue* is not present,
 - a. Let *kPresent* be **false**.
 - b. Repeat, while *kPresent* is **false** and *k* ≥ 0
 - i. Let *Pk* be ! **ToString**(*k*).
 - ii. Set *kPresent* to ? **HasProperty**(*O*, *Pk*).
 - iii. If *kPresent* is **true**, then
 1. Set *accumulator* to ? **Get**(*O*, *Pk*).
 - iv. Decrease *k* by 1.
 - c. If *kPresent* is **false**, throw a **TypeError** exception.
9. Repeat, while *k* ≥ 0
 - a. Let *Pk* be ! **ToString**(*k*).
 - b. Let *kPresent* be ? **HasProperty**(*O*, *Pk*).
 - c. If *kPresent* is **true**, then
 - i. Let *kValue* be ? **Get**(*O*, *Pk*).
 - ii. Set *accumulator* to ? **Call**(*callbackfn*, **undefined**, « *accumulator*, *kValue*, *k*, *O* »).
 - d. Decrease *k* by 1.
10. Return *accumulator*.

NOTE 2

The **reduceRight** function is intentionally generic; it does not require that its this value be an Array object. Therefore it can be transferred to other kinds of objects for use as a method.

22.1.3.23 Array.prototype.reverse ()

NOTE 1

The elements of the array are rearranged so as to reverse their order. The object is returned as the result of the call.

When the **reverse** method is called, the following steps are taken:

1. Let *O* be ? **ToObject**(**this** value).
2. Let *len* be ? **ToLength**(? **Get**(*O*, "length")).
3. Let *middle* be **floor**(*len* / 2).
4. Let *lower* be 0.
5. Repeat, while *lower* ≠ *middle*
 - a. Let *upper* be *len* - *lower* - 1.
 - b. Let *upperP* be ! **ToString**(*upper*).

- c. Let *lowerP* be ! ToString(*lower*).
 - d. Let *lowerExists* be ? HasProperty(*O*, *lowerP*).
 - e. If *lowerExists* is **true**, then
 - i. Let *lowerValue* be ? Get(*O*, *lowerP*).
 - f. Let *upperExists* be ? HasProperty(*O*, *upperP*).
 - g. If *upperExists* is **true**, then
 - i. Let *upperValue* be ? Get(*O*, *upperP*).
 - h. If *lowerExists* is **true** and *upperExists* is **true**, then
 - i. Perform ? Set(*O*, *lowerP*, *upperValue*, **true**).
 - ii. Perform ? Set(*O*, *upperP*, *lowerValue*, **true**).
 - i. Else if *lowerExists* is **false** and *upperExists* is **true**, then
 - i. Perform ? Set(*O*, *lowerP*, *upperValue*, **true**).
 - ii. Perform ? DeletePropertyOrThrow(*O*, *upperP*).
 - j. Else if *lowerExists* is **true** and *upperExists* is **false**, then
 - i. Perform ? DeletePropertyOrThrow(*O*, *lowerP*).
 - ii. Perform ? Set(*O*, *upperP*, *lowerValue*, **true**).
 - k. Else both *lowerExists* and *upperExists* are **false**,
 - i. No action is required.
 - l. Increase *lower* by 1.
6. Return *O*.

NOTE 2

The **reverse** function is intentionally generic; it does not require that its **this** value be an Array object. Therefore, it can be transferred to other kinds of objects for use as a method.

22.1.3.24 Array.prototype.shift ()

NOTE 1

The first element of the array is removed from the array and returned.

When the **shift** method is called, the following steps are taken:

1. Let *O* be ? ToObject(**this** value).
2. Let *len* be ? ToLength(? Get(*O*, "length")).
3. If *len* is zero, then
 - a. Perform ? Set(*O*, "length", 0, **true**).
 - b. Return **undefined**.
4. Let *first* be ? Get(*O*, "0").
5. Let *k* be 1.
6. Repeat, while *k* < *len*
 - a. Let *from* be ! ToString(*k*).
 - b. Let *to* be ! ToString(*k* - 1).
 - c. Let *fromPresent* be ? HasProperty(*O*, *from*).
 - d. If *fromPresent* is **true**, then
 - i. Let *fromVal* be ? Get(*O*, *from*).
 - ii. Perform ? Set(*O*, *to*, *fromVal*, **true**).
 - e. Else *fromPresent* is **false**,
 - i. Perform ? DeletePropertyOrThrow(*O*, *to*).

- f. Increase *k* by 1.
7. Perform ? **DeletePropertyOrThrow**(*O*, ! **ToString**(*len* - 1)).
8. Perform ? **Set**(*O*, "**length**", *len* - 1, **true**).
9. Return *first*.

NOTE 2

The **shift** function is intentionally generic; it does not require that its **this** value be an Array object. Therefore it can be transferred to other kinds of objects for use as a method.

22.1.3.25 Array.prototype.slice (*start*, *end*)

NOTE 1

The **slice** method takes two arguments, *start* and *end*, and returns an array containing the elements of the array from element *start* up to, but not including, element *end* (or through the end of the array if *end* is **undefined**). If *start* is negative, it is treated as *length* + *start* where *length* is the length of the array. If *end* is negative, it is treated as *length* + *end* where *length* is the length of the array.

The following steps are taken:

1. Let *O* be ? **ToObject**(**this** value).
2. Let *len* be ? **ToLength**(? **Get**(*O*, "**length**").
3. Let *relativeStart* be ? **ToInteger**(*start*).
4. If *relativeStart* < 0, let *k* be **max**((*len* + *relativeStart*), 0); else let *k* be **min**(*relativeStart*, *len*).
5. If *end* is **undefined**, let *relativeEnd* be *len*; else let *relativeEnd* be ? **ToInteger**(*end*).
6. If *relativeEnd* < 0, let *final* be **max**((*len* + *relativeEnd*), 0); else let *final* be **min**(*relativeEnd*, *len*).
7. Let *count* be **max**(*final* - *k*, 0).
8. Let *A* be ? **ArraySpeciesCreate**(*O*, *count*).
9. Let *n* be 0.
10. Repeat, while *k* < *final*
 - a. Let *Pk* be ! **ToString**(*k*).
 - b. Let *kPresent* be ? **HasProperty**(*O*, *Pk*).
 - c. If *kPresent* is **true**, then
 - i. Let *kValue* be ? **Get**(*O*, *Pk*).
 - ii. Perform ? **CreateDataPropertyOrThrow**(*A*, ! **ToString**(*n*), *kValue*).
 - d. Increase *k* by 1.
 - e. Increase *n* by 1.
11. Perform ? **Set**(*A*, "**length**", *n*, **true**).
12. Return *A*.

NOTE 2

The explicit setting of the "**length**" property of the result Array in step 11 was necessary in previous editions of ECMAScript to ensure that its length was correct in situations where the trailing elements of the result Array were not present. Setting "**length**" became unnecessary starting in ES2015 when the result Array was initialized to its proper length rather than an empty Array but is carried forward to preserve backward compatibility.

NOTE 3

The **slice** function is intentionally generic; it does not require that its **this** value be an Array object. Therefore it can be transferred to other kinds of objects for use as a method.

22.1.3.26 Array.prototype.some (*callbackfn* [, *thisArg*])

NOTE 1

callbackfn should be a function that accepts three arguments and returns a value that is coercible to the Boolean value **true** or **false**. **some** calls *callbackfn* once for each element present in the array, in ascending order, until it finds one where *callbackfn* returns **true**. If such an element is found, **some** immediately returns **true**. Otherwise, **some** returns **false**. *callbackfn* is called only for elements of the array which actually exist; it is not called for missing elements of the array.

If a *thisArg* parameter is provided, it will be used as the **this** value for each invocation of *callbackfn*. If it is not provided, **undefined** is used instead.

callbackfn is called with three arguments: the value of the element, the index of the element, and the object being traversed.

some does not directly mutate the object on which it is called but the object may be mutated by the calls to *callbackfn*.

The range of elements processed by **some** is set before the first call to *callbackfn*. Elements that are appended to the array after the call to **some** begins will not be visited by *callbackfn*. If existing elements of the array are changed, their value as passed to *callbackfn* will be the value at the time that **some** visits them; elements that are deleted after the call to **some** begins and before being visited are not visited. **some** acts like the "exists" quantifier in mathematics. In particular, for an empty array, it returns **false**.

When the **some** method is called with one or two arguments, the following steps are taken:

1. Let *O* be ? **ToObject**(**this** value).
2. Let *len* be ? **ToLength**(? **Get**(*O*, "length")).
3. If **IsCallable**(*callbackfn*) is **false**, throw a **TypeError** exception.
4. If *thisArg* is present, let *T* be *thisArg*; else let *T* be **undefined**.
5. Let *k* be 0.
6. Repeat, while *k* < *len*
 - a. Let *Pk* be ! **ToString**(*k*).
 - b. Let *kPresent* be ? **HasProperty**(*O*, *Pk*).
 - c. If *kPresent* is **true**, then
 - i. Let *kValue* be ? **Get**(*O*, *Pk*).
 - ii. Let *testResult* be **ToBoolean**(? **Call**(*callbackfn*, *T*, « *kValue*, *k*, *O* »)).
 - iii. If *testResult* is **true**, return **true**.
 - d. Increase *k* by 1.
7. Return **false**.

NOTE 2

The **some** function is intentionally generic; it does not require that its **this** value be an Array object. Therefore it can be transferred to other kinds of objects for use as a method.

22.1.3.27 Array.prototype.sort (*comparefn*)

The elements of this array are sorted. The sort must be stable (that is, elements that compare equal must remain in their original order). If *comparefn* is not **undefined**, it should be a function that accepts two arguments *x* and *y* and returns a negative value if *x* < *y*, zero if *x* = *y*, or a positive value if *x* > *y*.

Upon entry, the following steps are performed to initialize evaluation of the **sort** function:

1. If *comparefn* is not **undefined** and **IsCallable**(*comparefn*) is **false**, throw a **TypeError** exception.
2. Let *obj* be ? **ToObject**(**this** value).
3. Let *len* be ? **ToLength**(? **Get**(*obj*, "**length**").

Within this specification of the **sort** method, an object, *obj*, is said to be *sparse* if the following algorithm returns **true**:

1. For each integer *i* in the range $0 \leq i < \textit{len}$, do
 - a. Let *elem* be *obj*.**[[GetOwnProperty]]**(! **ToString**(*i*)).
 - b. If *elem* is **undefined**, return **true**.
2. Return **false**.

The *sort order* is the ordering, after completion of this function, of the **integer-indexed** property values of *obj* whose integer indexes are less than *len*. The result of the **sort** function is then determined as follows:

If *comparefn* is not **undefined** and is not a consistent comparison function for the elements of this array (see below), the sort order is implementation-defined. The sort order is also implementation-defined if *comparefn* is **undefined** and **SortCompare** does not act as a consistent comparison function.

Let *proto* be *obj*.**[[GetPrototypeOf]]**(). If *proto* is not **null** and there exists an integer *j* such that all of the conditions below are satisfied then the sort order is implementation-defined:

obj is sparse
 $0 \leq j < \textit{len}$
HasProperty(*proto*, **ToString**(*j*)) is **true**.

The sort order is also implementation-defined if *obj* is sparse and any of the following conditions are true:

IsExtensible(*obj*) is **false**.

Any **integer index** property of *obj* whose name is a nonnegative integer less than *len* is a **data property** whose **[[Configurable]]** attribute is **false**.

The sort order is also implementation-defined if any of the following conditions are true:

If *obj* is an **exotic object** (including Proxy exotic objects) whose behaviour for **[[Get]]**, **[[Set]]**, **[[Delete]]**, and **[[GetOwnProperty]]** is not the ordinary object implementation of these internal methods.

If any index property of *obj* whose name is a nonnegative integer less than *len* is an **accessor property** or is a **data property** whose **[[Writable]]** attribute is **false**.

If *comparefn* is **undefined** and the application of **ToString** to any value passed as an argument to **SortCompare** modifies *obj* or any object on *obj*'s prototype chain.

If *comparefn* is **undefined** and all applications of **ToString**, to any specific value passed as an argument to **SortCompare**, do not produce the same result.

The following steps are taken:

1. Perform an implementation-dependent sequence of calls to the **[[Get]]** and **[[Set]]** internal methods of *obj*, to the **DeletePropertyOrThrow** and **HasOwnProperty** abstract operation with *obj* as the first argument, and to **SortCompare** (described below), such that:
 - The property key argument for each call to **[[Get]]**, **[[Set]]**, **HasOwnProperty**, or **DeletePropertyOrThrow** is the string representation of a nonnegative integer less than *len*.
 - The arguments for calls to **SortCompare** are values returned by a previous call to the **[[Get]]** internal method,

unless the properties accessed by those previous calls did not exist according to [HasOwnProperty](#). If both prospective arguments to [SortCompare](#) correspond to non-existent properties, use **+0** instead of calling [SortCompare](#). If only the first prospective argument is non-existent use **+1**. If only the second prospective argument is non-existent use **-1**.

If *obj* is not sparse then [DeletePropertyOrThrow](#) must not be called.

If any `[[Set]]` call returns **false** a **TypeError** exception is thrown.

If an [abrupt completion](#) is returned from any of these operations, it is immediately returned as the value of this function.

2. Return *obj*.

Unless the sort order is specified above to be implementation-defined, the returned object must have the following two characteristics:

There must be some mathematical permutation π of the nonnegative integers less than *len*, such that for every nonnegative integer *j* less than *len*, if property `old[j]` existed, then `new[ $\pi(j)$ ]` is exactly the same value as `old[j]`. But if property `old[j]` did not exist, then `new[ $\pi(j)$ ]` does not exist.

Then for all nonnegative integers *j* and *k*, each less than *len*, if `SortCompare(old[j], old[k]) < 0` (see [SortCompare](#) below), then `new[ $\pi(j)$ ] < new[ $\pi(k)$ ]`.

Here the notation `old[j]` is used to refer to the hypothetical result of calling `obj.[[Get]](j)` before this function is executed, and the notation `new[j]` to refer to the hypothetical result of calling `obj.[[Get]](j)` after this function has been executed.

A function *comparefn* is a consistent comparison function for a set of values *S* if all of the requirements below are met for all values *a*, *b*, and *c* (possibly the same value) in the set *S*: The notation $a <_{CF} b$ means `comparefn(a, b) < 0`; $a =_{CF} b$ means `comparefn(a, b) = 0` (of either sign); and $a >_{CF} b$ means `comparefn(a, b) > 0`.

Calling `comparefn(a, b)` always returns the same value *v* when given a specific pair of values *a* and *b* as its two arguments. Furthermore, `Type(v)` is `Number`, and *v* is not `NaN`. Note that this implies that exactly one of $a <_{CF} b$, $a =_{CF} b$, and $a >_{CF} b$ will be true for a given pair of *a* and *b*.

Calling `comparefn(a, b)` does not modify *obj* or any object on *obj*'s prototype chain.

$a =_{CF} a$ (reflexivity)

If $a =_{CF} b$, then $b =_{CF} a$ (symmetry)

If $a =_{CF} b$ and $b =_{CF} c$, then $a =_{CF} c$ (transitivity of $=_{CF}$)

If $a <_{CF} b$ and $b <_{CF} c$, then $a <_{CF} c$ (transitivity of $<_{CF}$)

If $a >_{CF} b$ and $b >_{CF} c$, then $a >_{CF} c$ (transitivity of $>_{CF}$)

NOTE 1

The above conditions are necessary and sufficient to ensure that *comparefn* divides the set *S* into equivalence classes and that these equivalence classes are totally ordered.

NOTE 2

The **sort** function is intentionally generic; it does not require that its **this** value be an Array object. Therefore, it can be transferred to other kinds of objects for use as a method.

22.1.3.27.1 Runtime Semantics: SortCompare (*x*, *y*)

The `SortCompare` abstract operation is called with two arguments *x* and *y*. It also has access to the *comparefn* argument passed to the current invocation of the **sort** method. The following steps are taken:

1. If *x* and *y* are both **undefined**, return **+0**.

2. If *x* is **undefined**, return 1.
3. If *y* is **undefined**, return -1.
4. If *comparefn* is not **undefined**, then
 - a. Let *v* be ? **ToNumber**(? **Call**(*comparefn*, **undefined**, « *x*, *y* »)).
 - b. If *v* is NaN, return +0.
 - c. Return *v*.
5. Let *xString* be ? **ToString**(*x*).
6. Let *yString* be ? **ToString**(*y*).
7. Let *xSmaller* be the result of performing **Abstract Relational Comparison** *xString* < *yString*.
8. If *xSmaller* is **true**, return -1.
9. Let *ySmaller* be the result of performing **Abstract Relational Comparison** *yString* < *xString*.
10. If *ySmaller* is **true**, return 1.
11. Return +0.

NOTE 1

Because non-existent property values always compare greater than **undefined** property values, and **undefined** always compares greater than any other value, **undefined** property values always sort to the end of the result, followed by non-existent property values.

NOTE 2

Method calls performed by the **ToString** abstract operations in steps 5 and 7 have the potential to cause **SortCompare** to not behave as a consistent comparison function.

22.1.3.28 Array.prototype.splice (*start*, *deleteCount*, ...*items*)

NOTE 1

When the **splice** method is called with two or more arguments *start*, *deleteCount* and zero or more *items*, the *deleteCount* elements of the array starting at **integer index** *start* are replaced by the arguments *items*. An Array object containing the deleted elements (if any) is returned.

The following steps are taken:

1. Let *O* be ? **ToObject**(**this** value).
2. Let *len* be ? **ToLength**(? **Get**(*O*, "length")).
3. Let *relativeStart* be ? **ToInteger**(*start*).
4. If *relativeStart* < 0, let *actualStart* be **max**((*len* + *relativeStart*), 0); else let *actualStart* be **min**(*relativeStart*, *len*).
5. If the number of actual arguments is 0, then
 - a. Let *insertCount* be 0.
 - b. Let *actualDeleteCount* be 0.
6. Else if the number of actual arguments is 1, then
 - a. Let *insertCount* be 0.
 - b. Let *actualDeleteCount* be *len* - *actualStart*.
7. Else,
 - a. Let *insertCount* be the number of actual arguments minus 2.
 - b. Let *dc* be ? **ToInteger**(*deleteCount*).
 - c. Let *actualDeleteCount* be **min**(**max**(*dc*, 0), *len* - *actualStart*).
8. If *len* + *insertCount* - *actualDeleteCount* > 2⁵³ - 1, throw a **TypeError** exception.
9. Let *A* be ? **ArraySpeciesCreate**(*O*, *actualDeleteCount*).
10. Let *k* be 0.

11. Repeat, while $k < \text{actualDeleteCount}$
 - a. Let from be ! ToString($\text{actualStart} + k$).
 - b. Let fromPresent be ? HasProperty(O, from).
 - c. If fromPresent is **true**, then
 - i. Let fromValue be ? Get(O, from).
 - ii. Perform ? CreateDataPropertyOrThrow($A, \text{! ToString}(k), \text{fromValue}$).
 - d. Increment k by 1.
12. Perform ? Set($A, \text{"length"}, \text{actualDeleteCount}, \text{true}$).
13. Let items be a List whose elements are, in left to right order, the portion of the actual argument list starting with the third argument. The list is empty if fewer than three arguments were passed.
14. Let itemCount be the number of elements in items .
15. If $\text{itemCount} < \text{actualDeleteCount}$, then
 - a. Set k to actualStart .
 - b. Repeat, while $k < (\text{len} - \text{actualDeleteCount})$
 - i. Let from be ! ToString($k + \text{actualDeleteCount}$).
 - ii. Let to be ! ToString($k + \text{itemCount}$).
 - iii. Let fromPresent be ? HasProperty(O, from).
 - iv. If fromPresent is **true**, then
 1. Let fromValue be ? Get(O, from).
 2. Perform ? Set($O, \text{to}, \text{fromValue}, \text{true}$).
 - v. Else fromPresent is **false**,
 1. Perform ? DeletePropertyOrThrow(O, to).
 - vi. Increase k by 1.
 - c. Set k to len .
 - d. Repeat, while $k > (\text{len} - \text{actualDeleteCount} + \text{itemCount})$
 - i. Perform ? DeletePropertyOrThrow($O, \text{! ToString}(k - 1)$).
 - ii. Decrease k by 1.
16. Else if $\text{itemCount} > \text{actualDeleteCount}$, then
 - a. Set k to $(\text{len} - \text{actualDeleteCount})$.
 - b. Repeat, while $k > \text{actualStart}$
 - i. Let from be ! ToString($k + \text{actualDeleteCount} - 1$).
 - ii. Let to be ! ToString($k + \text{itemCount} - 1$).
 - iii. Let fromPresent be ? HasProperty(O, from).
 - iv. If fromPresent is **true**, then
 1. Let fromValue be ? Get(O, from).
 2. Perform ? Set($O, \text{to}, \text{fromValue}, \text{true}$).
 - v. Else fromPresent is **false**,
 1. Perform ? DeletePropertyOrThrow(O, to).
 - vi. Decrease k by 1.
17. Set k to actualStart .
18. Repeat, while items is not empty
 - a. Remove the first element from items and let E be the value of that element.
 - b. Perform ? Set($O, \text{! ToString}(k), E, \text{true}$).
 - c. Increase k by 1.
19. Perform ? Set($O, \text{"length"}, \text{len} - \text{actualDeleteCount} + \text{itemCount}, \text{true}$).
20. Return A .

The explicit setting of the **"length"** property of the result Array in step 19 was necessary in previous editions of ECMAScript to ensure that its length was correct in situations where the trailing elements of the result Array were not present. Setting **"length"** became unnecessary starting in ES2015 when the result Array was initialized to its proper length rather than an empty Array but is carried forward to preserve backward compatibility.

NOTE 3

The **splice** function is intentionally generic; it does not require that its **this** value be an Array object. Therefore it can be transferred to other kinds of objects for use as a method.

22.1.3.29 Array.prototype.toLocaleString ([*reserved1* [, *reserved2*]])

An ECMAScript implementation that includes the ECMA-402 Internationalization API must implement the **Array.prototype.toLocaleString** method as specified in the ECMA-402 specification. If an ECMAScript implementation does not include the ECMA-402 API the following specification of the **toLocaleString** method is used.

NOTE 1

The first edition of ECMA-402 did not include a replacement specification for the **Array.prototype.toLocaleString** method.

The meanings of the optional parameters to this method are defined in the ECMA-402 specification; implementations that do not include ECMA-402 support must not use those parameter positions for anything else.

The following steps are taken:

1. Let *array* be ? **ToObject**(**this** value).
2. Let *len* be ? **ToLength**(? **Get**(*array*, **"length"**)).
3. Let *separator* be the String value for the list-separator String appropriate for the host environment's current locale (this is derived in an implementation-defined way).
4. Let *R* be the empty String.
5. Let *k* be 0.
6. Repeat, while *k* < *len*
 - a. If *k* > 0, then
 - i. Set *R* to the **string-concatenation** of *R* and *separator*.
 - b. Let *nextElement* be ? **Get**(*array*, ! **ToString**(*k*)).
 - c. If *nextElement* is not **undefined** or **null**, then
 - i. Let *S* be ? **ToString**(? **Invoke**(*nextElement*, **"toLocaleString"**)).
 - ii. Set *R* to the **string-concatenation** of *R* and *S*.
 - d. Increase *k* by 1.
7. Return *R*.

NOTE 2

The elements of the array are converted to Strings using their **toLocaleString** methods, and these Strings are then concatenated, separated by occurrences of a separator String that has been derived in an implementation-defined locale-specific way. The result of calling this function is intended to be analogous to the result of **toString**, except that the result of this function is intended to be locale-specific.

NOTE 3

The **toLocaleString** function is intentionally generic; it does not require that its **this** value be an Array object. Therefore it can be transferred to other kinds of objects for use as a method.

22.1.3.30 Array.prototype.toString ()

When the **toString** method is called, the following steps are taken:

1. Let *array* be ? **ToObject**(**this** value).
2. Let *func* be ? **Get**(*array*, "join").
3. If **IsCallable**(*func*) is **false**, set *func* to the intrinsic function **%ObjProto_toString%**.
4. Return ? **Call**(*func*, *array*).

NOTE

The **toString** function is intentionally generic; it does not require that its **this** value be an Array object. Therefore it can be transferred to other kinds of objects for use as a method.

22.1.3.31 Array.prototype.unshift (...*items*)

NOTE 1

The arguments are prepended to the start of the array, such that their order within the array is the same as the order in which they appear in the argument list.

When the **unshift** method is called with zero or more arguments *item1*, *item2*, etc., the following steps are taken:

1. Let *O* be ? **ToObject**(**this** value).
2. Let *len* be ? **ToLength**(? **Get**(*O*, "length")).
3. Let *argCount* be the number of actual arguments.
4. If *argCount* > 0, then
 - a. If *len* + *argCount* > 2⁵³ - 1, throw a **TypeError** exception.
 - b. Let *k* be *len*.
 - c. Repeat, while *k* > 0,
 - i. Let *from* be ! **ToString**(*k* - 1).
 - ii. Let *to* be ! **ToString**(*k* + *argCount* - 1).
 - iii. Let *fromPresent* be ? **HasProperty**(*O*, *from*).
 - iv. If *fromPresent* is **true**, then
 1. Let *fromValue* be ? **Get**(*O*, *from*).
 2. Perform ? **Set**(*O*, *to*, *fromValue*, **true**).
 - v. Else *fromPresent* is **false**,
 1. Perform ? **DeletePropertyOrThrow**(*O*, *to*).
 - vi. Decrease *k* by 1.
 - d. Let *j* be 0.
 - e. Let *items* be a **List** whose elements are, in left to right order, the arguments that were passed to this function invocation.
 - f. Repeat, while *items* is not empty
 - i. Remove the first element from *items* and let *E* be the value of that element.
 - ii. Perform ? **Set**(*O*, ! **ToString**(*j*), *E*, **true**).
 - iii. Increase *j* by 1.
5. Perform ? **Set**(*O*, "length", *len* + *argCount*, **true**).
6. Return *len* + *argCount*.

The **"length"** property of the **unshift** method is 1.

NOTE 2

The **unshift** function is intentionally generic; it does not require that its **this** value be an Array object. Therefore it can be transferred to other kinds of objects for use as a method.

22.1.3.32 **Array.prototype.values** ()

The following steps are taken:

1. Let **O** be ? **ToObject**(**this** value).
2. Return **CreateArrayIterator**(**O**, **"value"**).

This function is the **%ArrayProto_values%** intrinsic object.

22.1.3.33 **Array.prototype** [**@@iterator**] ()

The initial value of the **@@iterator** property is the same **function object** as the initial value of the **Array.prototype.values** property.

22.1.3.34 **Array.prototype** [**@@unscopables**]

The initial value of the **@@unscopables** **data property** is an object created by the following steps:

1. Let **unscopableList** be **ObjectCreate**(**null**).
2. Perform **CreateDataProperty**(**unscopableList**, **"copyWithin"**, **true**).
3. Perform **CreateDataProperty**(**unscopableList**, **"entries"**, **true**).
4. Perform **CreateDataProperty**(**unscopableList**, **"fill"**, **true**).
5. Perform **CreateDataProperty**(**unscopableList**, **"find"**, **true**).
6. Perform **CreateDataProperty**(**unscopableList**, **"findIndex"**, **true**).
7. Perform **CreateDataProperty**(**unscopableList**, **"flat"**, **true**).
8. Perform **CreateDataProperty**(**unscopableList**, **"flatMap"**, **true**).
9. Perform **CreateDataProperty**(**unscopableList**, **"includes"**, **true**).
10. Perform **CreateDataProperty**(**unscopableList**, **"keys"**, **true**).
11. Perform **CreateDataProperty**(**unscopableList**, **"values"**, **true**).
12. **Assert**: Each of the above calls returns **true**.
13. Return **unscopableList**.

This property has the attributes { **[[Writable]]**: **false**, **[[Enumerable]]**: **false**, **[[Configurable]]**: **true** }.

NOTE

The own property names of this object are property names that were not included as standard properties of **Array.prototype** prior to the ECMAScript 2015 specification. These names are ignored for **with** statement binding purposes in order to preserve the behaviour of existing code that might use one of these names as a binding in an outer scope that is shadowed by a **with** statement whose binding object is an Array object.

22.1.4 Properties of Array Instances

Array instances are Array exotic objects and have the internal methods specified for such objects. Array instances inherit properties from the Array prototype object.

Array instances have a "**length**" property, and a set of enumerable properties with [array index](#) names.

22.1.4.1 length

The "**length**" property of an Array instance is a [data property](#) whose value is always numerically greater than the name of every configurable own property whose name is an [array index](#).

The "**length**" property initially has the attributes { [\[\[Writable\]\]](#): **true**, [\[\[Enumerable\]\]](#): **false**, [\[\[Configurable\]\]](#): **false** }.

NOTE

Reducing the value of the "**length**" property has the side-effect of deleting own array elements whose [array index](#) is between the old and new length values. However, non-configurable properties can not be deleted. Attempting to set the "**length**" property of an Array object to a value that is numerically less than or equal to the largest numeric own [property name](#) of an existing non-configurable [array-indexed](#) property of the array will result in the length being set to a numeric value that is one greater than that non-configurable numeric own [property name](#). See [9.4.2.1](#).

22.1.5 Array Iterator Objects

An Array Iterator is an object, that represents a specific iteration over some specific Array instance object. There is not a named [constructor](#) for Array Iterator objects. Instead, Array iterator objects are created by calling certain methods of Array instance objects.

22.1.5.1 CreateArrayIterator (*array*, *kind*)

Several methods of Array objects return Iterator objects. The abstract operation CreateArrayIterator with arguments *array* and *kind* is used to create such iterator objects. It performs the following steps:

1. [Assert](#): [Type](#)(*array*) is Object.
2. Let *iterator* be [ObjectCreate](#)(%ArrayIteratorPrototype%, « [\[\[IteratedObject\]\]](#), [\[\[ArrayIteratorNextIndex\]\]](#), [\[\[ArrayIterationKind\]\]](#) »).
3. Set *iterator*.[\[\[IteratedObject\]\]](#) to *array*.
4. Set *iterator*.[\[\[ArrayIteratorNextIndex\]\]](#) to 0.
5. Set *iterator*.[\[\[ArrayIterationKind\]\]](#) to *kind*.
6. Return *iterator*.

22.1.5.2 The %ArrayIteratorPrototype% Object

The %ArrayIteratorPrototype% object:

has properties that are inherited by all Array Iterator Objects.

is an ordinary object.

has a [\[\[Prototype\]\]](#) internal slot whose value is the intrinsic object %IteratorPrototype%.

has the following properties:

22.1.5.2.1 %ArrayIteratorPrototype%.next ()

1. Let *O* be the **this** value.
2. If **Type**(*O*) is not Object, throw a **TypeError** exception.
3. If *O* does not have all of the internal slots of an Array Iterator Instance (22.1.5.3), throw a **TypeError** exception.
4. Let *a* be *O*.[[IteratedObject]].
5. If *a* is **undefined**, return **CreateIterResultObject**(**undefined**, **true**).
6. Let *index* be *O*.[[ArrayIteratorNextIndex]].
7. Let *itemKind* be *O*.[[ArrayIterationKind]].
8. If *a* has a [[TypedArrayName]] internal slot, then
 - a. If **IsDetachedBuffer**(*a*.[[ViewedArrayBuffer]]) is **true**, throw a **TypeError** exception.
 - b. Let *len* be *a*.[[ArrayLength]].
9. Else,
 - a. Let *len* be ? **ToLength**(? **Get**(*a*, "length")).
10. If *index* ≥ *len*, then
 - a. Set *O*.[[IteratedObject]] to **undefined**.
 - b. Return **CreateIterResultObject**(**undefined**, **true**).
11. Set *O*.[[ArrayIteratorNextIndex]] to *index* + 1.
12. If *itemKind* is "key", return **CreateIterResultObject**(*index*, **false**).
13. Let *elementKey* be ! **ToString**(*index*).
14. Let *elementValue* be ? **Get**(*a*, *elementKey*).
15. If *itemKind* is "value", let *result* be *elementValue*.
16. Else,
 - a. **Assert**: *itemKind* is "key+value".
 - b. Let *result* be **CreateArrayFromList**(« *index*, *elementValue* »).
17. Return **CreateIterResultObject**(*result*, **false**).

22.1.5.2.2 %ArrayIteratorPrototype% [@@toStringTag]

The initial value of the @@toStringTag property is the String value **"Array Iterator"**.

This property has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **true** }.

22.1.5.3 Properties of Array Iterator Instances

Array Iterator instances are ordinary objects that inherit properties from the %ArrayIteratorPrototype% intrinsic object. Array Iterator instances are initially created with the internal slots listed in Table 58.

Table 58: Internal Slots of Array Iterator Instances

Internal Slot	Description
[[IteratedObject]]	The object whose array elements are being iterated.
[[ArrayIteratorNextIndex]]	The integer index of the next integer index to be examined by this iteration.
[[ArrayIterationKind]]	A String value that identifies what is returned for each element of the iteration. The possible values are: "key" , "value" , "key+value" .

22.2 TypedArray Objects

TypedArray objects present an array-like view of an underlying binary data buffer (24.1). Each element of a *TypedArray*

instance has the same underlying binary scalar data type. There is a distinct *TypedArray constructor*, listed in Table 59, for each of the nine supported element types. Each *constructor* in Table 59 has a corresponding distinct prototype object.

Table 59: The TypedArray Constructors

Constructor Name and Intrinsic	Element Type	Element Size	Conversion Operation	Description	Equivalent C Type
Int8Array %Int8Array%	Int8	1	ToInt8	8-bit 2's complement signed integer	signed char
Uint8Array %Uint8Array%	Uint8	1	ToUint8	8-bit unsigned integer	unsigned char
Uint8ClampedArray %Uint8ClampedArray%	Uint8C	1	ToUint8Clamp	8-bit unsigned integer (clamped conversion)	unsigned char
Int16Array %Int16Array%	Int16	2	ToInt16	16-bit 2's complement signed integer	short
Uint16Array %Uint16Array%	Uint16	2	ToUint16	16-bit unsigned integer	unsigned short
Int32Array %Int32Array%	Int32	4	ToInt32	32-bit 2's complement signed integer	int
Uint32Array %Uint32Array%	Uint32	4	ToUint32	32-bit unsigned integer	unsigned int
Float32Array %Float32Array%	Float32	4		32-bit IEEE floating point	float
Float64Array %Float64Array%	Float64	8		64-bit IEEE floating point	double

In the definitions below, references to *TypedArray* should be replaced with the appropriate *constructor* name from the above table. The phrase “the element size in bytes” refers to the value in the Element Size column of the table in the row corresponding to the *constructor*. The phrase “element Type” refers to the value in the Element Type column for that row.

22.2.1 The %TypedArray% Intrinsic Object

The %TypedArray% intrinsic object:

- is a *constructor function object* that all of the *TypedArray constructor* objects inherit from.
- along with its corresponding prototype object, provides common properties that are inherited by all *TypedArray* constructors and their instances.
- does not have a global name or appear as a property of the *global object*.
- acts as the abstract superclass of the various *TypedArray* constructors.
- will throw an error when invoked, because it is an abstract class *constructor*. The *TypedArray* constructors do not perform a **super** call to it.

22.2.1.1 %TypedArray% ()

The %TypedArray% constructor performs the following steps:

1. Throw a **TypeError** exception.

The "length" property of the %TypedArray% constructor function is 0.

22.2.2 Properties of the %TypedArray% Intrinsic Object

The %TypedArray% intrinsic object:

has a [[Prototype]] internal slot whose value is the intrinsic object %FunctionPrototype%.

has a **name** property whose value is "TypedArray".

has the following properties:

22.2.2.1 %TypedArray%.from (source [, mapfn [, thisArg]])

When the **from** method is called with argument *source*, and optional arguments *mapfn* and *thisArg*, the following steps are taken:

1. Let *C* be the **this** value.
2. If **IsConstructor**(*C*) is **false**, throw a **TypeError** exception.
3. If *mapfn* is present and *mapfn* is not **undefined**, then
 - a. If **IsCallable**(*mapfn*) is **false**, throw a **TypeError** exception.
 - b. Let *mapping* be **true**.
4. Else, let *mapping* be **false**.
5. If *thisArg* is present, let *T* be *thisArg*; else let *T* be **undefined**.
6. Let *usingIterator* be ? **GetMethod**(*source*, @@iterator).
7. If *usingIterator* is not **undefined**, then
 - a. Let *values* be ? **IterableToList**(*source*, *usingIterator*).
 - b. Let *len* be the number of elements in *values*.
 - c. Let *targetObj* be ? **TypedArrayCreate**(*C*, « *len* »).
 - d. Let *k* be 0.
 - e. Repeat, while *k* < *len*
 - i. Let *Pk* be ! **ToString**(*k*).
 - ii. Let *kValue* be the first element of *values* and remove that element from *values*.
 - iii. If *mapping* is **true**, then
 1. Let *mappedValue* be ? **Call**(*mapfn*, *T*, « *kValue*, *k* »).
 - iv. Else, let *mappedValue* be *kValue*.
 - v. Perform ? **Set**(*targetObj*, *Pk*, *mappedValue*, **true**).
 - vi. Increase *k* by 1.
 - f. **Assert**: *values* is now an empty **List**.
 - g. Return *targetObj*.
8. NOTE: *source* is not an **Iterable** so assume it is already an array-like object.
9. Let *arrayLike* be ! **ToObject**(*source*).
10. Let *len* be ? **ToLength**(? **Get**(*arrayLike*, "length")).
11. Let *targetObj* be ? **TypedArrayCreate**(*C*, « *len* »).
12. Let *k* be 0.
13. Repeat, while *k* < *len*

- a. Let *Pk* be ! ToString(*k*).
 - b. Let *kValue* be ? Get(arrayLike, *Pk*).
 - c. If *mapping* is **true**, then
 - i. Let *mappedValue* be ? Call(*mapfn*, *T*, « *kValue*, *k* »).
 - d. Else, let *mappedValue* be *kValue*.
 - e. Perform ? Set(*targetObj*, *Pk*, *mappedValue*, **true**).
 - f. Increase *k* by 1.
14. Return *targetObj*.

22.2.2.1.1 Runtime Semantics: IterableToList (*items*, *method*)

The abstract operation IterableToList performs the following steps:

1. Let *iteratorRecord* be ? GetIterator(*items*, **sync**, *method*).
2. Let *values* be a new empty List.
3. Let *next* be **true**.
4. Repeat, while *next* is not **false**
 - a. Set *next* to ? IteratorStep(*iteratorRecord*).
 - b. If *next* is not **false**, then
 - i. Let *nextValue* be ? IteratorValue(*next*).
 - ii. Append *nextValue* to the end of the List *values*.
5. Return *values*.

22.2.2.2 %TypedArray%.of (...*items*)

When the **of** method is called with any number of arguments, the following steps are taken:

1. Let *len* be the actual number of arguments passed to this function.
2. Let *items* be the List of arguments passed to this function.
3. Let *C* be the **this** value.
4. If IsConstructor(*C*) is **false**, throw a **TypeError** exception.
5. Let *newObj* be ? TypedArrayCreate(*C*, « *len* »).
6. Let *k* be 0.
7. Repeat, while *k* < *len*
 - a. Let *kValue* be *items*[*k*].
 - b. Let *Pk* be ! ToString(*k*).
 - c. Perform ? Set(*newObj*, *Pk*, *kValue*, **true**).
 - d. Increase *k* by 1.
8. Return *newObj*.

NOTE

The *items* argument is assumed to be a well-formed rest argument value.

22.2.2.3 %TypedArray%.prototype

The initial value of %TypedArray%.**prototype** is the %TypedArrayPrototype% intrinsic object.

This property has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **false** }.

22.2.2.4 get %TypedArray% [@@species]

`%TypedArray%[@@species]` is an [accessor property](#) whose set accessor function is **undefined**. Its get accessor function performs the following steps:

1. Return the **this** value.

The value of the **name** property of this function is `"get [Symbol.species]"`.

NOTE

`%TypedArrayPrototype%` methods normally use their **this** object's [constructor](#) to create a derived object. However, a subclass [constructor](#) may over-ride that default behaviour by redefining its `@@species` property.

22.2.3 Properties of the %TypedArrayPrototype% Object

The `%TypedArrayPrototype%` object:

has a `[[Prototype]]` internal slot whose value is the intrinsic object `%ObjectPrototype%`.

is an ordinary object.

does not have a `[[ViewedArrayBuffer]]` or any other of the internal slots that are specific to `TypedArray` instance objects.

22.2.3.1 get %TypedArray%.prototype.buffer

`%TypedArray%.prototype.buffer` is an [accessor property](#) whose set accessor function is **undefined**. Its get accessor function performs the following steps:

1. Let *O* be the **this** value.
2. If `Type(O)` is not Object, throw a **TypeError** exception.
3. If *O* does not have a `[[TypedArrayName]]` internal slot, throw a **TypeError** exception.
4. **Assert**: *O* has a `[[ViewedArrayBuffer]]` internal slot.
5. Let *buffer* be *O*.`[[ViewedArrayBuffer]]`.
6. Return *buffer*.

22.2.3.2 get %TypedArray%.prototype.byteLength

`%TypedArray%.prototype.byteLength` is an [accessor property](#) whose set accessor function is **undefined**. Its get accessor function performs the following steps:

1. Let *O* be the **this** value.
2. If `Type(O)` is not Object, throw a **TypeError** exception.
3. If *O* does not have a `[[TypedArrayName]]` internal slot, throw a **TypeError** exception.
4. **Assert**: *O* has a `[[ViewedArrayBuffer]]` internal slot.
5. Let *buffer* be *O*.`[[ViewedArrayBuffer]]`.
6. If `IsDetachedBuffer(buffer)` is **true**, return 0.
7. Let *size* be *O*.`[[ByteLength]]`.
8. Return *size*.

22.2.3.3 get %TypedArray%.prototype.byteOffset

`%TypedArray%.prototype.byteOffset` is an [accessor property](#) whose set accessor function is **undefined**. Its get accessor function performs the following steps:

1. Let *O* be the **this** value.
2. If `Type(O)` is not Object, throw a **TypeError** exception.
3. If *O* does not have a `[[TypedArrayName]]` internal slot, throw a **TypeError** exception.
4. **Assert:** *O* has a `[[ViewedArrayBuffer]]` internal slot.
5. Let *buffer* be *O*.`[[ViewedArrayBuffer]]`.
6. If `IsDetachedBuffer(buffer)` is **true**, return 0.
7. Let *offset* be *O*.`[[ByteOffset]]`.
8. Return *offset*.

22.2.3.4 %TypedArray%.prototype.constructor

The initial value of `%TypedArray%.prototype.constructor` is the `%TypedArray%` intrinsic object.

22.2.3.5 %TypedArray%.prototype.copyWithin (*target*, *start* [, *end*])

The interpretation and use of the arguments of `%TypedArray%.prototype.copyWithin` are the same as for `Array.prototype.copyWithin` as defined in 22.1.3.3.

The following steps are taken:

1. Let *O* be **this** value.
2. Perform ? `ValidateTypedArray(O)`.
3. Let *len* be *O*.`[[ArrayLength]]`.
4. Let *relativeTarget* be ? `ToInteger(target)`.
5. If *relativeTarget* < 0, let *to* be `max((len + relativeTarget), 0)`; else let *to* be `min(relativeTarget, len)`.
6. Let *relativeStart* be ? `ToInteger(start)`.
7. If *relativeStart* < 0, let *from* be `max((len + relativeStart), 0)`; else let *from* be `min(relativeStart, len)`.
8. If *end* is **undefined**, let *relativeEnd* be *len*; else let *relativeEnd* be ? `ToInteger(end)`.
9. If *relativeEnd* < 0, let *final* be `max((len + relativeEnd), 0)`; else let *final* be `min(relativeEnd, len)`.
10. Let *count* be `min(final - from, len - to)`.
11. If *count* > 0, then
 - a. NOTE: The copying must be performed in a manner that preserves the bit-level encoding of the source data.
 - b. Let *buffer* be *O*.`[[ViewedArrayBuffer]]`.
 - c. If `IsDetachedBuffer(buffer)` is **true**, throw a **TypeError** exception.
 - d. Let *typedArrayName* be the String value of *O*.`[[TypedArrayName]]`.
 - e. Let *elementSize* be the Number value of the Element Size value specified in Table 59 for *typedArrayName*.
 - f. Let *byteOffset* be *O*.`[[ByteOffset]]`.
 - g. Let *toByteIndex* be *to* × *elementSize* + *byteOffset*.
 - h. Let *fromByteIndex* be *from* × *elementSize* + *byteOffset*.
 - i. Let *countBytes* be *count* × *elementSize*.
 - j. If *fromByteIndex* < *toByteIndex* and *toByteIndex* < *fromByteIndex* + *countBytes*, then
 - i. Let *direction* be -1.
 - ii. Set *fromByteIndex* to *fromByteIndex* + *countBytes* - 1.
 - iii. Set *toByteIndex* to *toByteIndex* + *countBytes* - 1.
 - k. Else,
 - i. Let *direction* be 1.

1. Repeat, while *countBytes* > 0
 - i. Let *value* be *GetValueFromBuffer*(*buffer*, *fromByteIndex*, "Uint8", true, "Unordered").
 - ii. Perform *SetValueInBuffer*(*buffer*, *toByteIndex*, "Uint8", *value*, true, "Unordered").
 - iii. Set *fromByteIndex* to *fromByteIndex* + *direction*.
 - iv. Set *toByteIndex* to *toByteIndex* + *direction*.
 - v. Decrease *countBytes* by 1.
12. Return *O*.

22.2.3.5.1 Runtime Semantics: *ValidateTypedArray* (*O*)

When called with argument *O*, the following steps are taken:

1. If *Type*(*O*) is not Object, throw a **TypeError** exception.
2. If *O* does not have a *[[TypedArrayName]]* internal slot, throw a **TypeError** exception.
3. **Assert**: *O* has a *[[ViewedArrayBuffer]]* internal slot.
4. Let *buffer* be *O*.*[[ViewedArrayBuffer]]*.
5. If *IsDetachedBuffer*(*buffer*) is **true**, throw a **TypeError** exception.
6. Return *buffer*.

22.2.3.6 *%TypedArray%.prototype.entries* ()

The following steps are taken:

1. Let *O* be the **this** value.
2. Perform ? *ValidateTypedArray*(*O*).
3. Return *CreateArrayIterator*(*O*, "key+value").

22.2.3.7 *%TypedArray%.prototype.every* (*callbackfn* [, *thisArg*])

%TypedArray%.prototype.every is a distinct function that implements the same algorithm as *Array.prototype.every* as defined in 22.1.3.5 except that the **this** object's *[[ArrayLength]]* internal slot is accessed in place of performing a *[[Get]]* of "length". The implementation of the algorithm may be optimized with the knowledge that the **this** value is an object that has a fixed length and whose integer-indexed properties are not sparse. However, such optimization must not introduce any observable changes in the specified behaviour of the algorithm and must take into account the possibility that calls to *callbackfn* may cause the **this** value to become detached.

This function is not generic. *ValidateTypedArray* is applied to the **this** value prior to evaluating the algorithm. If its result is an **abrupt completion** that exception is thrown instead of evaluating the algorithm.

22.2.3.8 *%TypedArray%.prototype.fill* (*value* [, *start* [, *end*]])

The interpretation and use of the arguments of *%TypedArray%.prototype.fill* are the same as for *Array.prototype.fill* as defined in 22.1.3.6.

The following steps are taken:

1. Let *O* be the **this** value.
2. Perform ? *ValidateTypedArray*(*O*).
3. Let *len* be *O*.*[[ArrayLength]]*.
4. Set *value* to ? *ToNumber*(*value*).

5. Let *relativeStart* be ? **ToInteger**(*start*).
6. If *relativeStart* < 0, let *k* be **max**((*len* + *relativeStart*), 0); else let *k* be **min**(*relativeStart*, *len*).
7. If *end* is **undefined**, let *relativeEnd* be *len*; else let *relativeEnd* be ? **ToInteger**(*end*).
8. If *relativeEnd* < 0, let *final* be **max**((*len* + *relativeEnd*), 0); else let *final* be **min**(*relativeEnd*, *len*).
9. If **IsDetachedBuffer**(*O*.[[ViewedArrayBuffer]]) is **true**, throw a **TypeError** exception.
10. Repeat, while *k* < *final*
 - a. Let *Pk* be ! **ToString**(*k*).
 - b. Perform ! **Set**(*O*, *Pk*, *value*, **true**).
 - c. Increase *k* by 1.
11. Return *O*.

22.2.3.9 %TypedArray%.prototype.filter (*callbackfn* [, *thisArg*])

The interpretation and use of the arguments of **%TypedArray%.prototype.filter** are the same as for **Array.prototype.filter** as defined in 22.1.3.7.

When the **filter** method is called with one or two arguments, the following steps are taken:

1. Let *O* be the **this** value.
2. Perform ? **ValidateTypedArray**(*O*).
3. Let *len* be *O*.[[ArrayLength]].
4. If **IsCallable**(*callbackfn*) is **false**, throw a **TypeError** exception.
5. If *thisArg* is present, let *T* be *thisArg*; else let *T* be **undefined**.
6. Let *kept* be a new empty **List**.
7. Let *k* be 0.
8. Let *captured* be 0.
9. Repeat, while *k* < *len*
 - a. Let *Pk* be ! **ToString**(*k*).
 - b. Let *kValue* be ? **Get**(*O*, *Pk*).
 - c. Let *selected* be **ToBoolean**(? **Call**(*callbackfn*, *T*, « *kValue*, *k*, *O* »)).
 - d. If *selected* is **true**, then
 - i. Append *kValue* to the end of *kept*.
 - ii. Increase *captured* by 1.
 - e. Increase *k* by 1.
10. Let *A* be ? **TypedArraySpeciesCreate**(*O*, « *captured* »).
11. Let *n* be 0.
12. For each element *e* of *kept*, do
 - a. Perform ! **Set**(*A*, ! **ToString**(*n*), *e*, **true**).
 - b. Increment *n* by 1.
13. Return *A*.

This function is not generic. The **this** value must be an object with a [[TypedArrayName]] internal slot.

22.2.3.10 %TypedArray%.prototype.find (*predicate* [, *thisArg*])

%TypedArray%.prototype.find is a distinct function that implements the same algorithm as **Array.prototype.find** as defined in 22.1.3.8 except that the **this** object's [[ArrayLength]] internal slot is accessed in place of performing a [[Get]] of "length". The implementation of the algorithm may be optimized with the knowledge that the **this** value is an object that has a fixed length and whose **integer-indexed** properties are not sparse.

However, such optimization must not introduce any observable changes in the specified behaviour of the algorithm and must take into account the possibility that calls to *predicate* may cause the **this** value to become detached.

This function is not generic. *ValidateTypedArray* is applied to the **this** value prior to evaluating the algorithm. If its result is an *abrupt completion* that exception is thrown instead of evaluating the algorithm.

22.2.3.11 %TypedArray%.prototype.findIndex (*predicate* [, *thisArg*])

%TypedArray%.prototype.findIndex is a distinct function that implements the same algorithm as *Array.prototype.findIndex* as defined in 22.1.3.9 except that the **this** object's *[[ArrayLength]]* internal slot is accessed in place of performing a *[[Get]]* of "length". The implementation of the algorithm may be optimized with the knowledge that the **this** value is an object that has a fixed length and whose *integer-indexed* properties are not sparse. However, such optimization must not introduce any observable changes in the specified behaviour of the algorithm and must take into account the possibility that calls to *predicate* may cause the **this** value to become detached.

This function is not generic. *ValidateTypedArray* is applied to the **this** value prior to evaluating the algorithm. If its result is an *abrupt completion* that exception is thrown instead of evaluating the algorithm.

22.2.3.12 %TypedArray%.prototype.forEach (*callbackfn* [, *thisArg*])

%TypedArray%.prototype.forEach is a distinct function that implements the same algorithm as *Array.prototype.forEach* as defined in 22.1.3.12 except that the **this** object's *[[ArrayLength]]* internal slot is accessed in place of performing a *[[Get]]* of "length". The implementation of the algorithm may be optimized with the knowledge that the **this** value is an object that has a fixed length and whose *integer-indexed* properties are not sparse. However, such optimization must not introduce any observable changes in the specified behaviour of the algorithm and must take into account the possibility that calls to *callbackfn* may cause the **this** value to become detached.

This function is not generic. *ValidateTypedArray* is applied to the **this** value prior to evaluating the algorithm. If its result is an *abrupt completion* that exception is thrown instead of evaluating the algorithm.

22.2.3.13 %TypedArray%.prototype.includes (*searchElement* [, *fromIndex*])

%TypedArray%.prototype.includes is a distinct function that implements the same algorithm as *Array.prototype.includes* as defined in 22.1.3.13 except that the **this** object's *[[ArrayLength]]* internal slot is accessed in place of performing a *[[Get]]* of "length". The implementation of the algorithm may be optimized with the knowledge that the **this** value is an object that has a fixed length and whose *integer-indexed* properties are not sparse. However, such optimization must not introduce any observable changes in the specified behaviour of the algorithm.

This function is not generic. *ValidateTypedArray* is applied to the **this** value prior to evaluating the algorithm. If its result is an *abrupt completion* that exception is thrown instead of evaluating the algorithm.

22.2.3.14 %TypedArray%.prototype.indexOf (*searchElement* [, *fromIndex*])

%TypedArray%.prototype.indexOf is a distinct function that implements the same algorithm as *Array.prototype.indexOf* as defined in 22.1.3.14 except that the **this** object's *[[ArrayLength]]* internal slot is accessed in place of performing a *[[Get]]* of "length". The implementation of the algorithm may be optimized with the knowledge that the **this** value is an object that has a fixed length and whose *integer-indexed* properties are not sparse. However, such optimization must not introduce any observable changes in the specified behaviour of the algorithm.

This function is not generic. *ValidateTypedArray* is applied to the **this** value prior to evaluating the algorithm. If its

result is an [abrupt completion](#) that exception is thrown instead of evaluating the algorithm.

22.2.3.15 %TypedArray%.prototype.join (*separator*)

%TypedArray%.prototype.join is a distinct function that implements the same algorithm as **Array.prototype.join** as defined in 22.1.3.15 except that the **this** object's `[[ArrayLength]]` internal slot is accessed in place of performing a `[[Get]]` of **"length"**. The implementation of the algorithm may be optimized with the knowledge that the **this** value is an object that has a fixed length and whose [integer-indexed](#) properties are not sparse. However, such optimization must not introduce any observable changes in the specified behaviour of the algorithm.

This function is not generic. [ValidateTypedArray](#) is applied to the **this** value prior to evaluating the algorithm. If its result is an [abrupt completion](#) that exception is thrown instead of evaluating the algorithm.

22.2.3.16 %TypedArray%.prototype.keys ()

The following steps are taken:

1. Let *O* be the **this** value.
2. Perform ? [ValidateTypedArray](#)(*O*).
3. Return [CreateArrayIterator](#)(*O*, **"key"**).

22.2.3.17 %TypedArray%.prototype.lastIndexOf (*searchElement* [, *fromIndex*])

%TypedArray%.prototype.lastIndexOf is a distinct function that implements the same algorithm as **Array.prototype.lastIndexOf** as defined in 22.1.3.17 except that the **this** object's `[[ArrayLength]]` internal slot is accessed in place of performing a `[[Get]]` of **"length"**. The implementation of the algorithm may be optimized with the knowledge that the **this** value is an object that has a fixed length and whose [integer-indexed](#) properties are not sparse. However, such optimization must not introduce any observable changes in the specified behaviour of the algorithm.

This function is not generic. [ValidateTypedArray](#) is applied to the **this** value prior to evaluating the algorithm. If its result is an [abrupt completion](#) that exception is thrown instead of evaluating the algorithm.

22.2.3.18 get %TypedArray%.prototype.length

%TypedArray%.prototype.length is an [accessor property](#) whose set accessor function is **undefined**. Its get accessor function performs the following steps:

1. Let *O* be the **this** value.
2. If [Type](#)(*O*) is not Object, throw a **TypeError** exception.
3. If *O* does not have a `[[TypedArrayName]]` internal slot, throw a **TypeError** exception.
4. **Assert**: *O* has `[[ViewedArrayBuffer]]` and `[[ArrayLength]]` internal slots.
5. Let *buffer* be *O*.`[[ViewedArrayBuffer]]`.
6. If [IsDetachedBuffer](#)(*buffer*) is **true**, return 0.
7. Let *length* be *O*.`[[ArrayLength]]`.
8. Return *length*.

This function is not generic. The **this** value must be an object with a `[[TypedArrayName]]` internal slot.

22.2.3.19 %TypedArray%.prototype.map (*callbackfn* [, *thisArg*])

The interpretation and use of the arguments of `%TypedArray%.prototype.map` are the same as for `Array.prototype.map` as defined in 22.1.3.18.

When the `map` method is called with one or two arguments, the following steps are taken:

1. Let *O* be the **this** value.
2. Perform ? `ValidateTypedArray(O)`.
3. Let *len* be *O*.[[ArrayLength]].
4. If `IsCallable(callbackfn)` is **false**, throw a **TypeError** exception.
5. If *thisArg* is present, let *T* be *thisArg*; else let *T* be **undefined**.
6. Let *A* be ? `TypedArraySpeciesCreate(O, « len »)`.
7. Let *k* be 0.
8. Repeat, while *k* < *len*
 - a. Let *Pk* be ! `ToString(k)`.
 - b. Let *kValue* be ? `Get(O, Pk)`.
 - c. Let *mappedValue* be ? `Call(callbackfn, T, « kValue, k, O »)`.
 - d. Perform ? `Set(A, Pk, mappedValue, true)`.
 - e. Increase *k* by 1.
9. Return *A*.

This function is not generic. The **this** value must be an object with a [[TypedArrayName]] internal slot.

22.2.3.20 %TypedArray%.prototype.reduce (callbackfn [, initialValue])

`%TypedArray%.prototype.reduce` is a distinct function that implements the same algorithm as `Array.prototype.reduce` as defined in 22.1.3.21 except that the **this** object's [[ArrayLength]] internal slot is accessed in place of performing a [[Get]] of "length". The implementation of the algorithm may be optimized with the knowledge that the **this** value is an object that has a fixed length and whose **integer-indexed** properties are not sparse. However, such optimization must not introduce any observable changes in the specified behaviour of the algorithm and must take into account the possibility that calls to *callbackfn* may cause the **this** value to become detached.

This function is not generic. `ValidateTypedArray` is applied to the **this** value prior to evaluating the algorithm. If its result is an **abrupt completion** that exception is thrown instead of evaluating the algorithm.

22.2.3.21 %TypedArray%.prototype.reduceRight (callbackfn [, initialValue])

`%TypedArray%.prototype.reduceRight` is a distinct function that implements the same algorithm as `Array.prototype.reduceRight` as defined in 22.1.3.22 except that the **this** object's [[ArrayLength]] internal slot is accessed in place of performing a [[Get]] of "length". The implementation of the algorithm may be optimized with the knowledge that the **this** value is an object that has a fixed length and whose **integer-indexed** properties are not sparse. However, such optimization must not introduce any observable changes in the specified behaviour of the algorithm and must take into account the possibility that calls to *callbackfn* may cause the **this** value to become detached.

This function is not generic. `ValidateTypedArray` is applied to the **this** value prior to evaluating the algorithm. If its result is an **abrupt completion** that exception is thrown instead of evaluating the algorithm.

22.2.3.22 %TypedArray%.prototype.reverse ()

`%TypedArray%.prototype.reverse` is a distinct function that implements the same algorithm as `Array.prototype.reverse` as defined in 22.1.3.23 except that the **this** object's [[ArrayLength]] internal slot is

accessed in place of performing a `[[Get]]` of **"length"**. The implementation of the algorithm may be optimized with the knowledge that the **this** value is an object that has a fixed length and whose **integer-indexed** properties are not sparse. However, such optimization must not introduce any observable changes in the specified behaviour of the algorithm.

This function is not generic. `ValidateTypedArray` is applied to the **this** value prior to evaluating the algorithm. If its result is an **abrupt completion** that exception is thrown instead of evaluating the algorithm.

22.2.3.23 %TypedArray%.prototype.set (*overloaded* [, *offset*])

`%TypedArray%.prototype.set` is a single function whose behaviour is overloaded based upon the type of its first argument.

This function is not generic. The **this** value must be an object with a `[[TypedArrayName]]` internal slot.

22.2.3.23.1 %TypedArray%.prototype.set (*array* [, *offset*])

Sets multiple values in this *TypedArray*, reading the values from the object *array*. The optional *offset* value indicates the first element index in this *TypedArray* where values are written. If omitted, it is assumed to be 0.

1. **Assert:** *array* is any ECMAScript language value other than an Object with a `[[TypedArrayName]]` internal slot. If it is such an Object, the definition in 22.2.3.23.2 applies.
2. Let *target* be the **this** value.
3. If `Type(target)` is not Object, throw a **TypeError** exception.
4. If *target* does not have a `[[TypedArrayName]]` internal slot, throw a **TypeError** exception.
5. **Assert:** *target* has a `[[ViewedArrayBuffer]]` internal slot.
6. Let *targetOffset* be `? ToInteger(offset)`.
7. If *targetOffset* < 0, throw a **RangeError** exception.
8. Let *targetBuffer* be *target*.`[[ViewedArrayBuffer]]`.
9. If `IsDetachedBuffer(targetBuffer)` is **true**, throw a **TypeError** exception.
10. Let *targetLength* be *target*.`[[ArrayLength]]`.
11. Let *targetName* be the String value of *target*.`[[TypedArrayName]]`.
12. Let *targetElementSize* be the Number value of the Element Size value specified in Table 59 for *targetName*.
13. Let *targetType* be the String value of the Element Type value in Table 59 for *targetName*.
14. Let *targetByteOffset* be *target*.`[[ByteOffset]]`.
15. Let *src* be `? ToObject(array)`.
16. Let *srcLength* be `? ToLength(? Get(src, "length"))`.
17. If *srcLength* + *targetOffset* > *targetLength*, throw a **RangeError** exception.
18. Let *targetByteIndex* be *targetOffset* × *targetElementSize* + *targetByteOffset*.
19. Let *k* be 0.
20. Let *limit* be *targetByteIndex* + *targetElementSize* × *srcLength*.
21. Repeat, while *targetByteIndex* < *limit*
 - a. Let *Pk* be `! ToString(k)`.
 - b. Let *kNumber* be `? ToNumber(? Get(src, Pk))`.
 - c. If `IsDetachedBuffer(targetBuffer)` is **true**, throw a **TypeError** exception.
 - d. Perform `SetValueInBuffer(targetBuffer, targetByteIndex, targetType, kNumber, true, "Unordered")`.
 - e. Increase *k* by 1.
 - f. Set *targetByteIndex* to *targetByteIndex* + *targetElementSize*.
22. Return **undefined**.

22.2.3.23.2 %TypedArray%.prototype.set (*typedArray* [, *offset*])

Sets multiple values in this *TypedArray*, reading the values from the *typedArray* argument object. The optional *offset* value indicates the first element index in this *TypedArray* where values are written. If omitted, it is assumed to be 0.

1. **Assert:** *typedArray* has a [[TypedArrayName]] internal slot. If it does not, the definition in 22.2.3.23.1 applies.
2. Let *target* be the **this** value.
3. If **Type**(*target*) is not Object, throw a **TypeError** exception.
4. If *target* does not have a [[TypedArrayName]] internal slot, throw a **TypeError** exception.
5. **Assert:** *target* has a [[ViewedArrayBuffer]] internal slot.
6. Let *targetOffset* be ? **ToInteger**(*offset*).
7. If *targetOffset* < 0, throw a **RangeError** exception.
8. Let *targetBuffer* be *target*.[[ViewedArrayBuffer]].
9. If **IsDetachedBuffer**(*targetBuffer*) is **true**, throw a **TypeError** exception.
10. Let *targetLength* be *target*.[[ArrayLength]].
11. Let *srcBuffer* be *typedArray*.[[ViewedArrayBuffer]].
12. If **IsDetachedBuffer**(*srcBuffer*) is **true**, throw a **TypeError** exception.
13. Let *targetName* be the String value of *target*.[[TypedArrayName]].
14. Let *targetType* be the String value of the Element Type value in Table 59 for *targetName*.
15. Let *targetElementSize* be the Number value of the Element Size value specified in Table 59 for *targetName*.
16. Let *targetByteOffset* be *target*.[[ByteOffset]].
17. Let *srcName* be the String value of *typedArray*.[[TypedArrayName]].
18. Let *srcType* be the String value of the Element Type value in Table 59 for *srcName*.
19. Let *srcElementSize* be the Number value of the Element Size value specified in Table 59 for *srcName*.
20. Let *srcLength* be *typedArray*.[[ArrayLength]].
21. Let *srcByteOffset* be *typedArray*.[[ByteOffset]].
22. If *srcLength* + *targetOffset* > *targetLength*, throw a **RangeError** exception.
23. If both **IsSharedArrayBuffer**(*srcBuffer*) and **IsSharedArrayBuffer**(*targetBuffer*) are **true**, then
 - a. If *srcBuffer*.[[ArrayBufferData]] and *targetBuffer*.[[ArrayBufferData]] are the same **Shared Data Block** values, let *same* be **true**; else let *same* be **false**.
24. Else, let *same* be **SameValue**(*srcBuffer*, *targetBuffer*).
25. If *same* is **true**, then
 - a. Let *srcByteLength* be *typedArray*.[[ByteLength]].
 - b. Set *srcBuffer* to ? **CloneArrayBuffer**(*srcBuffer*, *srcByteOffset*, *srcByteLength*, %ArrayBuffer%).
 - c. NOTE: %ArrayBuffer% is used to clone *srcBuffer* because it is known to not have any observable side-effects.
 - d. Let *srcByteIndex* be 0.
26. Else, let *srcByteIndex* be *srcByteOffset*.
27. Let *targetByteIndex* be *targetOffset* × *targetElementSize* + *targetByteOffset*.
28. Let *limit* be *targetByteIndex* + *targetElementSize* × *srcLength*.
29. If **SameValue**(*srcType*, *targetType*) is **true**, then
 - a. NOTE: If *srcType* and *targetType* are the same, the transfer must be performed in a manner that preserves the bit-level encoding of the source data.
 - b. Repeat, while *targetByteIndex* < *limit*
 - i. Let *value* be **GetValueFromBuffer**(*srcBuffer*, *srcByteIndex*, "Uint8", **true**, "Unordered").
 - ii. Perform **SetValueInBuffer**(*targetBuffer*, *targetByteIndex*, "Uint8", *value*, **true**, "Unordered").
 - iii. Increase *srcByteIndex* by 1.
 - iv. Increase *targetByteIndex* by 1.
30. Else,

- a. Repeat, while *targetByteIndex* < *limit*
 - i. Let *value* be *GetValueFromBuffer*(*srcBuffer*, *srcByteIndex*, *srcType*, **true**, "Unordered").
 - ii. Perform *SetValueInBuffer*(*targetBuffer*, *targetByteIndex*, *targetType*, *value*, **true**, "Unordered").
 - iii. Set *srcByteIndex* to *srcByteIndex* + *srcElementSize*.
 - iv. Set *targetByteIndex* to *targetByteIndex* + *targetElementSize*.
31. Return **undefined**.

22.2.3.24 %TypedArray%.prototype.slice (*start*, *end*)

The interpretation and use of the arguments of *%TypedArray%.prototype.slice* are the same as for *Array.prototype.slice* as defined in 22.1.3.25. The following steps are taken:

1. Let *O* be the **this** value.
2. Perform ? *ValidateTypedArray*(*O*).
3. Let *len* be *O*.[[*ArrayLength*]].
4. Let *relativeStart* be ? *ToInteger*(*start*).
5. If *relativeStart* < 0, let *k* be *max*((*len* + *relativeStart*), 0); else let *k* be *min*(*relativeStart*, *len*).
6. If *end* is **undefined**, let *relativeEnd* be *len*; else let *relativeEnd* be ? *ToInteger*(*end*).
7. If *relativeEnd* < 0, let *final* be *max*((*len* + *relativeEnd*), 0); else let *final* be *min*(*relativeEnd*, *len*).
8. Let *count* be *max*(*final* - *k*, 0).
9. Let *A* be ? *TypedArraySpeciesCreate*(*O*, « *count* »).
10. Let *srcName* be the String value of *O*.[[*TypedArrayName*]].
11. Let *srcType* be the String value of the Element Type value in Table 59 for *srcName*.
12. Let *targetName* be the String value of *A*.[[*TypedArrayName*]].
13. Let *targetType* be the String value of the Element Type value in Table 59 for *targetName*.
14. If *SameValue*(*srcType*, *targetType*) is **false**, then
 - a. Let *n* be 0.
 - b. Repeat, while *k* < *final*
 - i. Let *Pk* be ! *ToInteger*(*k*).
 - ii. Let *kValue* be ? *Get*(*O*, *Pk*).
 - iii. Perform ! *Set*(*A*, ! *ToInteger*(*n*), *kValue*, **true**).
 - iv. Increase *k* by 1.
 - v. Increase *n* by 1.
15. Else if *count* > 0, then
 - a. Let *srcBuffer* be *O*.[[*ViewedArrayBuffer*]].
 - b. If *IsDetachedBuffer*(*srcBuffer*) is **true**, throw a **TypeError** exception.
 - c. Let *targetBuffer* be *A*.[[*ViewedArrayBuffer*]].
 - d. Let *elementSize* be the Number value of the Element Size value specified in Table 59 for *srcType*.
 - e. NOTE: If *srcType* and *targetType* are the same, the transfer must be performed in a manner that preserves the bit-level encoding of the source data.
 - f. Let *srcByteOffset* be *O*.[[*ByteOffset*]].
 - g. Let *targetByteIndex* be *A*.[[*ByteOffset*]].
 - h. Let *srcByteIndex* be (*k* × *elementSize*) + *srcByteOffset*.
 - i. Let *limit* be *targetByteIndex* + *count* × *elementSize*.
 - j. Repeat, while *targetByteIndex* < *limit*
 - i. Let *value* be *GetValueFromBuffer*(*srcBuffer*, *srcByteIndex*, "Uint8", **true**, "Unordered").
 - ii. Perform *SetValueInBuffer*(*targetBuffer*, *targetByteIndex*, "Uint8", *value*, **true**, "Unordered").
 - iii. Increase *srcByteIndex* by 1.

iv. Increase *targetByteIndex* by 1.

16. Return *A*.

This function is not generic. The **this** value must be an object with a `[[TypedArrayName]]` internal slot.

22.2.3.25 %TypedArray%.prototype.some (*callbackfn* [, *thisArg*])

`%TypedArray%.prototype.some` is a distinct function that implements the same algorithm as `Array.prototype.some` as defined in 22.1.3.26 except that the **this** object's `[[ArrayLength]]` internal slot is accessed in place of performing a `[[Get]]` of `"length"`. The implementation of the algorithm may be optimized with the knowledge that the **this** value is an object that has a fixed length and whose *integer-indexed* properties are not sparse. However, such optimization must not introduce any observable changes in the specified behaviour of the algorithm and must take into account the possibility that calls to *callbackfn* may cause the **this** value to become detached.

This function is not generic. `ValidateTypedArray` is applied to the **this** value prior to evaluating the algorithm. If its result is an *abrupt completion* that exception is thrown instead of evaluating the algorithm.

22.2.3.26 %TypedArray%.prototype.sort (*comparefn*)

`%TypedArray%.prototype.sort` is a distinct function that, except as described below, implements the same requirements as those of `Array.prototype.sort` as defined in 22.1.3.27. The implementation of the `%TypedArray%.prototype.sort` specification may be optimized with the knowledge that the **this** value is an object that has a fixed length and whose *integer-indexed* properties are not sparse. The only internal methods of the **this** object that the algorithm may call are `[[Get]]` and `[[Set]]`.

This function is not generic. The **this** value must be an object with a `[[TypedArrayName]]` internal slot.

Upon entry, the following steps are performed to initialize evaluation of the **sort** function. These steps are used instead of the entry steps in 22.1.3.27:

1. If *comparefn* is not **undefined** and `IsCallable(comparefn)` is **false**, throw a **TypeError** exception.
2. Let *obj* be the **this** value.
3. Let *buffer* be ? `ValidateTypedArray(obj)`.
4. Let *len* be *obj*.`[[ArrayLength]]`.

The implementation-defined sort order condition for exotic objects is not applied by

`%TypedArray%.prototype.sort`.

The following version of `SortCompare` is used by `%TypedArray%.prototype.sort`. It performs a numeric comparison rather than the string comparison used in 22.1.3.27. `SortCompare` has access to the *comparefn* and *buffer* values of the current invocation of the **sort** method.

When the TypedArray `SortCompare` abstract operation is called with two arguments *x* and *y*, the following steps are taken:

1. **Assert**: Both `Type(x)` and `Type(y)` is **Number**.
2. If *comparefn* is not **undefined**, then
 - a. Let *v* be ? `ToNumber(? Call(comparefn, undefined, « x, y »))`.
 - b. If `IsDetachedBuffer(buffer)` is **true**, throw a **TypeError** exception.
 - c. If *v* is **NaN**, return **+0**.
 - d. Return *v*.

3. If x and y are both NaN, return +0.
4. If x is NaN, return 1.
5. If y is NaN, return -1.
6. If $x < y$, return -1.
7. If $x > y$, return 1.
8. If x is -0 and y is +0, return -1.
9. If x is +0 and y is -0, return 1.
10. Return +0.

NOTE

Because NaN always compares greater than any other value, NaN property values always sort to the end of the result when *comparefn* is not provided.

22.2.3.27 %TypedArray%.prototype.subarray (*begin*, *end*)

Returns a new *TypedArray* object whose element type is the same as this *TypedArray* and whose ArrayBuffer is the same as the ArrayBuffer of this *TypedArray*, referencing the elements at *begin*, inclusive, up to *end*, exclusive. If either *begin* or *end* is negative, it refers to an index from the end of the array, as opposed to from the beginning.

1. Let *O* be the **this** value.
2. If Type(*O*) is not Object, throw a **TypeError** exception.
3. If *O* does not have a [[TypedArrayName]] internal slot, throw a **TypeError** exception.
4. **Assert:** *O* has a [[ViewedArrayBuffer]] internal slot.
5. Let *buffer* be *O*.[[ViewedArrayBuffer]].
6. Let *srcLength* be *O*.[[ArrayLength]].
7. Let *relativeBegin* be ? ToInteger(*begin*).
8. If *relativeBegin* < 0, let *beginIndex* be max((*srcLength* + *relativeBegin*), 0); else let *beginIndex* be min(*relativeBegin*, *srcLength*).
9. If *end* is **undefined**, let *relativeEnd* be *srcLength*; else, let *relativeEnd* be ? ToInteger(*end*).
10. If *relativeEnd* < 0, let *endIndex* be max((*srcLength* + *relativeEnd*), 0); else let *endIndex* be min(*relativeEnd*, *srcLength*).
11. Let *newLength* be max(*endIndex* - *beginIndex*, 0).
12. Let *constructorName* be the String value of *O*.[[TypedArrayName]].
13. Let *elementSize* be the Number value of the Element Size value specified in Table 59 for *constructorName*.
14. Let *srcByteOffset* be *O*.[[ByteOffset]].
15. Let *beginByteOffset* be *srcByteOffset* + *beginIndex* × *elementSize*.
16. Let *argumentsList* be « *buffer*, *beginByteOffset*, *newLength* ».
17. Return ? TypedArraySpeciesCreate(*O*, *argumentsList*).

This function is not generic. The **this** value must be an object with a [[TypedArrayName]] internal slot.

22.2.3.28 %TypedArray%.prototype.toLocaleString ([*reserved1* [, *reserved2*]])

%TypedArray%.prototype.toLocaleString is a distinct function that implements the same algorithm as **Array.prototype.toLocaleString** as defined in 22.1.3.29 except that the **this** object's [[ArrayLength]] internal slot is accessed in place of performing a [[Get]] of "length". The implementation of the algorithm may be optimized with the knowledge that the **this** value is an object that has a fixed length and whose integer-indexed properties are not sparse. However, such optimization must not introduce any observable changes in the specified

behaviour of the algorithm.

This function is not generic. [ValidateTypedArray](#) is applied to the **this** value prior to evaluating the algorithm. If its result is an [abrupt completion](#) that exception is thrown instead of evaluating the algorithm.

NOTE

If the ECMAScript implementation includes the ECMA-402 Internationalization API this function is based upon the algorithm for **Array.prototype.toLocaleString** that is in the ECMA-402 specification.

22.2.3.29 %TypedArray%.prototype.toString ()

The initial value of the **%TypedArray%.prototype.toString** data property is the same built-in [function object](#) as the **Array.prototype.toString** method defined in 22.1.3.30.

22.2.3.30 %TypedArray%.prototype.values ()

The following steps are taken:

1. Let *O* be the **this** value.
2. Perform ? [ValidateTypedArray](#)(*O*).
3. Return [CreateArrayIterator](#)(*O*, "value").

22.2.3.31 %TypedArray%.prototype [@@iterator] ()

The initial value of the @@iterator property is the same [function object](#) as the initial value of the **%TypedArray%.prototype.values** property.

22.2.3.32 get %TypedArray%.prototype [@@toStringTag]

%TypedArray%.prototype[@@toStringTag] is an [accessor property](#) whose set accessor function is **undefined**. Its get accessor function performs the following steps:

1. Let *O* be the **this** value.
2. If [Type](#)(*O*) is not Object, return **undefined**.
3. If *O* does not have a **[[TypedArrayName]]** internal slot, return **undefined**.
4. Let *name* be *O*.**[[TypedArrayName]]**.
5. **Assert**: [Type](#)(*name*) is String.
6. Return *name*.

This property has the attributes { **[[Enumerable]]**: **false**, **[[Configurable]]**: **true** }.

The initial value of the **name** property of this function is **"get [Symbol.toStringTag]"**.

22.2.4 The *TypedArray* Constructors

Each *TypedArray* constructor:

is an intrinsic object that has the structure described below, differing only in the name used as the [constructor](#) name instead of *TypedArray*, in [Table 59](#).

is a single function whose behaviour is overloaded based upon the number and types of its arguments. The actual

behaviour of a call of *TypedArray* depends upon the number and kind of arguments that are passed to it. is not intended to be called as a function and will throw an exception when called in that manner. is designed to be subclassable. It may be used as the value of an **extends** clause of a class definition. Subclass constructors that intend to inherit the specified *TypedArray* behaviour must include a **super** call to the *TypedArray* constructor to create and initialize the subclass instance with the internal state necessary to support the *%TypedArray%.prototype* built-in methods. has a "**length**" property whose value is 3.

22.2.4.1 *TypedArray* ()

This description applies only if the *TypedArray* function is called with no arguments.

1. If *NewTarget* is **undefined**, throw a **TypeError** exception.
2. Let *constructorName* be the String value of the **Constructor** Name value specified in Table 59 for this *TypedArray* constructor.
3. Return ? *AllocateTypedArray*(*constructorName*, *NewTarget*, "%*TypedArray*Prototype%", 0).

22.2.4.2 *TypedArray* (*length*)

This description applies only if the *TypedArray* function is called with at least one argument and the Type of the first argument is not Object.

TypedArray called with argument *length* performs the following steps:

1. **Assert**: *Type*(*length*) is not Object.
2. If *NewTarget* is **undefined**, throw a **TypeError** exception.
3. Let *elementLength* be ? *ToIndex*(*length*).
4. Let *constructorName* be the String value of the **Constructor** Name value specified in Table 59 for this *TypedArray* constructor.
5. Return ? *AllocateTypedArray*(*constructorName*, *NewTarget*, "%*TypedArray*Prototype%", *elementLength*).

22.2.4.2.1 Runtime Semantics: *AllocateTypedArray* (*constructorName*, *newTarget*, *defaultProto* [, *length*])

The abstract operation *AllocateTypedArray* with arguments *constructorName*, *newTarget*, *defaultProto* and optional argument *length* is used to validate and create an instance of a *TypedArray* constructor. *constructorName* is required to be the name of a *TypedArray* constructor in Table 59. If the *length* argument is passed, an *ArrayBuffer* of that length is also allocated and associated with the new *TypedArray* instance. *AllocateTypedArray* provides common semantics that is used by all of the *TypedArray* overloads. *AllocateTypedArray* performs the following steps:

1. Let *proto* be ? *GetPrototypeFromConstructor*(*newTarget*, *defaultProto*).
2. Let *obj* be *IntegerIndexedObjectCreate*(*proto*, « [[*ViewedArrayBuffer*]], [[*TypedArrayName*]], [[*ByteLength*]], [[*ByteOffset*]], [[*ArrayLength*]] »).
3. **Assert**: *obj*.[[*ViewedArrayBuffer*]] is **undefined**.
4. Set *obj*.[[*TypedArrayName*]] to *constructorName*.
5. If *length* is not present, then
 - a. Set *obj*.[[*ByteLength*]] to 0.
 - b. Set *obj*.[[*ByteOffset*]] to 0.
 - c. Set *obj*.[[*ArrayLength*]] to 0.
6. Else,
 - a. Perform ? *AllocateTypedArrayBuffer*(*obj*, *length*).

7. Return *obj*.

22.2.4.2.2 Runtime Semantics: AllocateTypedArrayBuffer (*O*, *length*)

The abstract operation AllocateTypedArrayBuffer with arguments *O* and *length* allocates and associates an ArrayBuffer with the TypedArray instance *O*. It performs the following steps:

1. **Assert**: *O* is an Object that has a `[[ViewedArrayBuffer]]` internal slot.
2. **Assert**: *O*.`[[ViewedArrayBuffer]]` is **undefined**.
3. **Assert**: *length* ≥ 0 .
4. Let *constructorName* be the String value of *O*.`[[TypedArrayName]]`.
5. Let *elementSize* be the Element Size value in Table 59 for *constructorName*.
6. Let *byteLength* be *elementSize* $\times$ *length*.
7. Let *data* be ? `AllocateArrayBuffer(%ArrayBuffer%, byteLength)`.
8. Set *O*.`[[ViewedArrayBuffer]]` to *data*.
9. Set *O*.`[[ByteLength]]` to *byteLength*.
10. Set *O*.`[[ByteOffset]]` to 0.
11. Set *O*.`[[ArrayLength]]` to *length*.
12. Return *O*.

22.2.4.3 TypedArray (*typedArray*)

This description applies only if the *TypedArray* function is called with at least one argument and the Type of the first argument is Object and that object has a `[[TypedArrayName]]` internal slot.

TypedArray called with argument *typedArray* performs the following steps:

1. **Assert**: `Type(typedArray)` is Object and *typedArray* has a `[[TypedArrayName]]` internal slot.
2. If `NewTarget` is **undefined**, throw a **TypeError** exception.
3. Let *constructorName* be the String value of the `Constructor` Name value specified in Table 59 for this *TypedArray* constructor.
4. Let *O* be ? `AllocateTypedArray(constructorName, NewTarget, "%TypedArrayPrototype%")`.
5. Let *srcArray* be *typedArray*.
6. Let *srcData* be *srcArray*.`[[ViewedArrayBuffer]]`.
7. If `IsDetachedBuffer(srcData)` is **true**, throw a **TypeError** exception.
8. Let *elementType* be the String value of the Element Type value in Table 59 for *constructorName*.
9. Let *elementLength* be *srcArray*.`[[ArrayLength]]`.
10. Let *srcName* be the String value of *srcArray*.`[[TypedArrayName]]`.
11. Let *srcType* be the String value of the Element Type value in Table 59 for *srcName*.
12. Let *srcElementSize* be the Element Size value in Table 59 for *srcName*.
13. Let *srcByteOffset* be *srcArray*.`[[ByteOffset]]`.
14. Let *elementSize* be the Element Size value in Table 59 for *constructorName*.
15. Let *byteLength* be *elementSize* $\times$ *elementLength*.
16. If `IsSharedArrayBuffer(srcData)` is **false**, then
 - a. Let *bufferConstructor* be ? `SpeciesConstructor(srcData, %ArrayBuffer%)`.
17. Else,
 - a. Let *bufferConstructor* be `%ArrayBuffer%`.
18. If `SameValue(elementType, srcType)` is **true**, then
 - a. Let *data* be ? `CloneArrayBuffer(srcData, srcByteOffset, byteLength, bufferConstructor)`.
19. Else,

- a. Let *data* be ? *AllocateArrayBuffer*(*bufferConstructor*, *byteLength*).
- b. If *IsDetachedBuffer*(*srcData*) is **true**, throw a **TypeError** exception.
- c. Let *srcByteIndex* be *srcByteOffset*.
- d. Let *targetByteIndex* be 0.
- e. Let *count* be *elementLength*.
- f. Repeat, while *count* > 0
 - i. Let *value* be *GetValueFromBuffer*(*srcData*, *srcByteIndex*, *srcType*, **true**, "Unordered").
 - ii. Perform *SetValueInBuffer*(*data*, *targetByteIndex*, *elementType*, *value*, **true**, "Unordered").
 - iii. Set *srcByteIndex* to *srcByteIndex* + *srcElementSize*.
 - iv. Set *targetByteIndex* to *targetByteIndex* + *elementSize*.
 - v. Decrement *count* by 1.
20. Set *O*.[[ViewedArrayBuffer]] to *data*.
21. Set *O*.[[ByteLength]] to *byteLength*.
22. Set *O*.[[ByteOffset]] to 0.
23. Set *O*.[[ArrayLength]] to *elementLength*.
24. Return *O*.

22.2.4.4 *TypedArray* (*object*)

This description applies only if the *TypedArray* function is called with at least one argument and the Type of the first argument is Object and that object does not have either a [[TypedArrayName]] or an [[ArrayBufferData]] internal slot.

TypedArray called with argument *object* performs the following steps:

1. **Assert:** *Type*(*object*) is Object and *object* does not have either a [[TypedArrayName]] or an [[ArrayBufferData]] internal slot.
2. If *NewTarget* is **undefined**, throw a **TypeError** exception.
3. Let *constructorName* be the String value of the *Constructor* Name value specified in Table 59 for this *TypedArray* constructor.
4. Let *O* be ? *AllocateTypedArray*(*constructorName*, *NewTarget*, "%*TypedArray*Prototype%").
5. Let *usingIterator* be ? *GetMethod*(*object*, @@iterator).
6. If *usingIterator* is not **undefined**, then
 - a. Let *values* be ? *IterableToList*(*object*, *usingIterator*).
 - b. Let *len* be the number of elements in *values*.
 - c. Perform ? *AllocateTypedArrayBuffer*(*O*, *len*).
 - d. Let *k* be 0.
 - e. Repeat, while *k* < *len*
 - i. Let *Pk* be ! *ToString*(*k*).
 - ii. Let *kValue* be the first element of *values* and remove that element from *values*.
 - iii. Perform ? *Set*(*O*, *Pk*, *kValue*, **true**).
 - iv. Increase *k* by 1.
 - f. **Assert:** *values* is now an empty *List*.
 - g. Return *O*.
7. NOTE: *object* is not an Iterable so assume it is already an array-like object.
8. Let *arrayLike* be *object*.
9. Let *len* be ? *ToLength*(? *Get*(*arrayLike*, "length")).
10. Perform ? *AllocateTypedArrayBuffer*(*O*, *len*).
11. Let *k* be 0.
12. Repeat, while *k* < *len*

- a. Let *Pk* be ! ToString(*k*).
 - b. Let *kValue* be ? Get(arrayLike, *Pk*).
 - c. Perform ? Set(*O*, *Pk*, *kValue*, true).
 - d. Increase *k* by 1.
13. Return *O*.

22.2.4.5 TypedArray (*buffer* [, *byteOffset* [, *length*]])

This description applies only if the *TypedArray* function is called with at least one argument and the Type of the first argument is Object and that object has an [[ArrayBufferData]] internal slot.

TypedArray called with at least one argument *buffer* performs the following steps:

1. Assert: Type(*buffer*) is Object and *buffer* has an [[ArrayBufferData]] internal slot.
2. If NewTarget is **undefined**, throw a **TypeError** exception.
3. Let *constructorName* be the String value of the Constructor Name value specified in Table 59 for this *TypedArray* constructor.
4. Let *O* be ? AllocateTypedArray(*constructorName*, NewTarget, "%TypedArrayPrototype%").
5. Let *elementSize* be the Number value of the Element Size value in Table 59 for *constructorName*.
6. Let *offset* be ? ToIndex(*byteOffset*).
7. If *offset* modulo *elementSize* ≠ 0, throw a **RangeError** exception.
8. If *length* is present and *length* is not **undefined**, then
 - a. Let *newLength* be ? ToIndex(*length*).
9. If IsDetachedBuffer(*buffer*) is **true**, throw a **TypeError** exception.
10. Let *bufferByteLength* be *buffer*.[[ArrayBufferByteLength]].
11. If *length* is either not present or **undefined**, then
 - a. If *bufferByteLength* modulo *elementSize* ≠ 0, throw a **RangeError** exception.
 - b. Let *newByteLength* be *bufferByteLength* - *offset*.
 - c. If *newByteLength* < 0, throw a **RangeError** exception.
12. Else,
 - a. Let *newByteLength* be *newLength* × *elementSize*.
 - b. If *offset* + *newByteLength* > *bufferByteLength*, throw a **RangeError** exception.
13. Set *O*.[[ViewedArrayBuffer]] to *buffer*.
14. Set *O*.[[ByteLength]] to *newByteLength*.
15. Set *O*.[[ByteOffset]] to *offset*.
16. Set *O*.[[ArrayLength]] to *newByteLength* / *elementSize*.
17. Return *O*.

22.2.4.6 TypedArrayCreate (*constructor*, *argumentList*)

The abstract operation TypedArrayCreate with arguments *constructor* and *argumentList* is used to specify the creation of a new TypedArray object using a *constructor* function. It performs the following steps:

1. Let *newTypedArray* be ? Construct(*constructor*, *argumentList*).
2. Perform ? ValidateTypedArray(*newTypedArray*).
3. If *argumentList* is a List of a single Number, then
 - a. If *newTypedArray*.[[ArrayLength]] < *argumentList*[0], throw a **TypeError** exception.
4. Return *newTypedArray*.

22.2.4.7 TypedArraySpeciesCreate (*exemplar*, *argumentList*)

The abstract operation TypedArraySpeciesCreate with arguments *exemplar* and *argumentList* is used to specify the creation of a new TypedArray object using a *constructor* function that is derived from *exemplar*. It performs the following steps:

1. **Assert:** *exemplar* is an Object that has a `[[TypedArrayName]]` internal slot.
2. Let *defaultConstructor* be the intrinsic object listed in column one of Table 59 for *exemplar*.`[[TypedArrayName]]`.
3. Let *constructor* be ? *SpeciesConstructor*(*exemplar*, *defaultConstructor*).
4. Return ? *TypedArrayCreate*(*constructor*, *argumentList*).

22.2.5 Properties of the *TypedArray* Constructors

Each *TypedArray* constructor:

- has a `[[Prototype]]` internal slot whose value is the intrinsic object `%TypedArray%`.
- has a **name** property whose value is the String value of the *constructor* name specified for it in Table 59.
- has the following properties:

22.2.5.1 *TypedArray*.BYTES_PER_ELEMENT

The value of *TypedArray*.BYTES_PER_ELEMENT is the Number value of the Element Size value specified in Table 59 for *TypedArray*.

This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **false** }.

22.2.5.2 *TypedArray*.prototype

The initial value of *TypedArray*.**prototype** is the corresponding *TypedArray* prototype intrinsic object (22.2.6).

This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **false** }.

22.2.6 Properties of the *TypedArray* Prototype Objects

Each *TypedArray* prototype object:

- has a `[[Prototype]]` internal slot whose value is the intrinsic object `%TypedArrayPrototype%`.
- is an ordinary object.
- does not have a `[[ViewedArrayBuffer]]` or any other of the internal slots that are specific to *TypedArray* instance objects.

22.2.6.1 *TypedArray*.prototype.BYTES_PER_ELEMENT

The value of *TypedArray*.**prototype**.BYTES_PER_ELEMENT is the Number value of the Element Size value specified in Table 59 for *TypedArray*.

This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **false** }.

22.2.6.2 *TypedArray*.prototype.constructor

The initial value of a *TypedArray*.**prototype**.**constructor** is the corresponding `%TypedArray%` intrinsic

object.

22.2.7 Properties of *TypedArray* Instances

TypedArray instances are [Integer-Indexed exotic objects](#). Each *TypedArray* instance inherits properties from the corresponding *TypedArray* prototype object. Each *TypedArray* instance has the following internal slots: `[[TypedArrayName]]`, `[[ViewedArrayBuffer]]`, `[[ByteLength]]`, `[[ByteOffset]]`, and `[[ArrayLength]]`.

23 Keyed Collections

23.1 Map Objects

Map objects are collections of key/value pairs where both the keys and values may be arbitrary ECMAScript language values. A distinct key value may only occur in one key/value pair within the Map's collection. Distinct key values are discriminated using the [SameValueZero](#) comparison algorithm.

Map object must be implemented using either hash tables or other mechanisms that, on average, provide access times that are sublinear on the number of elements in the collection. The data structures used in this Map objects specification is only intended to describe the required observable semantics of Map objects. It is not intended to be a viable implementation model.

23.1.1 The Map Constructor

The Map [constructor](#):

is the intrinsic object `%Map%`.

is the initial value of the **Map** property of the [global object](#).

creates and initializes a new Map object when called as a [constructor](#).

is not intended to be called as a function and will throw an exception when called in that manner.

is designed to be subclassable. It may be used as the value in an **extends** clause of a class definition. Subclass constructors that intend to inherit the specified **Map** behaviour must include a **super** call to the **Map constructor** to create and initialize the subclass instance with the internal state necessary to support the **Map.prototype** built-in methods.

23.1.1.1 Map ([*iterable*])

When the **Map** function is called with optional argument *iterable*, the following steps are taken:

1. If `NewTarget` is **undefined**, throw a **TypeError** exception.
2. Let *map* be `? OrdinaryCreateFromConstructor(NewTarget, "%MapPrototype%", « [[MapData]] »)`.
3. Set *map*.`[[MapData]]` to a new empty [List](#).
4. If *iterable* is not present, or is either **undefined** or **null**, return *map*.
5. Let *adder* be `? Get(map, "set")`.
6. Return `? AddEntriesFromIterable(map, iterable, adder)`.

NOTE

If the parameter *iterable* is present, it is expected to be an object that implements an @@iterator method that returns an iterator object that produces a two element array-like object whose first element is a value that will be used as a Map key and whose second element is the value to associate with that key.

23.1.1.2 AddEntriesFromIterable (*target*, *iterable*, *adder*)

The abstract operation AddEntriesFromIterable accepts a *target* object, an *iterable* of entries, and an *adder* function to be invoked, with *target* as the receiver.

1. If **IsCallable**(*adder*) is **false**, throw a **TypeError** exception.
2. **Assert**: *iterable* is present, and is neither **undefined** nor **null**.
3. Let *iteratorRecord* be ? **GetIterator**(*iterable*).
4. Repeat,
 - a. Let *next* be ? **IteratorStep**(*iteratorRecord*).
 - b. If *next* is **false**, return *target*.
 - c. Let *nextItem* be ? **IteratorValue**(*next*).
 - d. If **Type**(*nextItem*) is not Object, then
 - i. Let *error* be **ThrowCompletion**(a newly created **TypeError** object).
 - ii. Return ? **IteratorClose**(*iteratorRecord*, *error*).
 - e. Let *k* be **Get**(*nextItem*, "0").
 - f. If *k* is an abrupt completion, return ? **IteratorClose**(*iteratorRecord*, *k*).
 - g. Let *v* be **Get**(*nextItem*, "1").
 - h. If *v* is an abrupt completion, return ? **IteratorClose**(*iteratorRecord*, *v*).
 - i. Let *status* be **Call**(*adder*, *target*, « *k*.[[Value]], *v*.[[Value]] »).
 - j. If *status* is an abrupt completion, return ? **IteratorClose**(*iteratorRecord*, *status*).

NOTE

The parameter *iterable* is expected to be an object that implements an @@iterator method that returns an iterator object that produces a two element array-like object whose first element is a value that will be used as a Map key and whose second element is the value to associate with that key.

23.1.2 Properties of the Map Constructor

The Map **constructor**:

has a [[Prototype]] internal slot whose value is the intrinsic object **%FunctionPrototype%**.
has the following properties:

23.1.2.1 Map.prototype

The initial value of **Map.prototype** is the intrinsic object **%MapPrototype%**.

This property has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **false** }.

23.1.2.2 get Map [@@species]

Map[@@species] is an **accessor property** whose set accessor function is **undefined**. Its get accessor function performs the following steps:

1. Return the **this** value.

The value of the **name** property of this function is **"get [Symbol.species]"**.

NOTE

Methods that create derived collection objects should call @@species to determine the **constructor** to use to create the derived objects. Subclass **constructor** may over-ride @@species to change the default **constructor** assignment.

23.1.3 Properties of the Map Prototype Object

The Map prototype object:

- is the intrinsic object %MapPrototype%.
- has a [[Prototype]] internal slot whose value is the intrinsic object %ObjectPrototype%.
- is an ordinary object.
- does not have a [[MapData]] internal slot.

23.1.3.1 Map.prototype.clear ()

The following steps are taken:

1. Let *M* be the **this** value.
2. If **Type**(*M*) is not Object, throw a **TypeError** exception.
3. If *M* does not have a [[MapData]] internal slot, throw a **TypeError** exception.
4. Let *entries* be the **List** that is *M*.[[MapData]].
5. For each **Record** { [[Key]], [[Value]] } *p* that is an element of *entries*, do
 - a. Set *p*.[[Key]] to empty.
 - b. Set *p*.[[Value]] to empty.
6. Return **undefined**.

NOTE

The existing [[MapData]] **List** is preserved because there may be existing Map Iterator objects that are suspended midway through iterating over that **List**.

23.1.3.2 Map.prototype.constructor

The initial value of **Map.prototype.constructor** is the intrinsic object %Map%.

23.1.3.3 Map.prototype.delete (*key*)

The following steps are taken:

1. Let *M* be the **this** value.
2. If **Type**(*M*) is not Object, throw a **TypeError** exception.
3. If *M* does not have a [[MapData]] internal slot, throw a **TypeError** exception.
4. Let *entries* be the **List** that is *M*.[[MapData]].
5. For each **Record** { [[Key]], [[Value]] } *p* that is an element of *entries*, do
 - a. If *p*.[[Key]] is not empty and **SameValueZero**(*p*.[[Key]], *key*) is **true**, then

- i. Set *p*.[[Key]] to **empty**.
 - ii. Set *p*.[[Value]] to **empty**.
 - iii. Return **true**.
6. Return **false**.

NOTE

The value **empty** is used as a specification device to indicate that an entry has been deleted. Actual implementations may take other actions such as physically removing the entry from internal data structures.

23.1.3.4 Map.prototype.entries ()

The following steps are taken:

1. Let *M* be the **this** value.
2. Return ? **CreateMapIterator**(*M*, "key+value").

23.1.3.5 Map.prototype.forEach (*callbackfn* [, *thisArg*])

When the **forEach** method is called with one or two arguments, the following steps are taken:

1. Let *M* be the **this** value.
2. If **Type**(*M*) is not Object, throw a **TypeError** exception.
3. If *M* does not have a [[MapData]] internal slot, throw a **TypeError** exception.
4. If **IsCallable**(*callbackfn*) is **false**, throw a **TypeError** exception.
5. If *thisArg* is present, let *T* be *thisArg*; else let *T* be **undefined**.
6. Let *entries* be the **List** that is *M*.[[MapData]].
7. For each **Record** { [[Key]], [[Value]] } *e* that is an element of *entries*, in original key insertion order, do
 - a. If *e*.[[Key]] is not **empty**, then
 - i. Perform ? **Call**(*callbackfn*, *T*, « *e*.[[Value]], *e*.[[Key]], *M* »).
8. Return **undefined**.

NOTE

callbackfn should be a function that accepts three arguments. **forEach** calls *callbackfn* once for each key/value pair present in the map object, in key insertion order. *callbackfn* is called only for keys of the map which actually exist; it is not called for keys that have been deleted from the map.

If a *thisArg* parameter is provided, it will be used as the **this** value for each invocation of *callbackfn*. If it is not provided, **undefined** is used instead.

callbackfn is called with three arguments: the value of the item, the key of the item, and the Map object being traversed.

forEach does not directly mutate the object on which it is called but the object may be mutated by the calls to *callbackfn*. Each entry of a map's [[MapData]] is only visited once. New keys added after the call to **forEach** begins are visited. A key will be revisited if it is deleted after it has been visited and then re-added before the **forEach** call completes. Keys that are deleted after the call to **forEach** begins and before being visited are not visited unless the key is added again before the **forEach** call completes.

23.1.3.6 Map.prototype.get (*key*)

The following steps are taken:

1. Let *M* be the **this** value.
2. If `Type(M)` is not Object, throw a **TypeError** exception.
3. If *M* does not have a `[[MapData]]` internal slot, throw a **TypeError** exception.
4. Let *entries* be the List that is *M*.`[[MapData]]`.
5. For each Record { `[[Key]]`, `[[Value]]` } *p* that is an element of *entries*, do
 - a. If *p*.`[[Key]]` is not empty and `SameValueZero(p.[[Key]], key)` is **true**, return *p*.`[[Value]]`.
6. Return **undefined**.

23.1.3.7 Map.prototype.has (*key*)

The following steps are taken:

1. Let *M* be the **this** value.
2. If `Type(M)` is not Object, throw a **TypeError** exception.
3. If *M* does not have a `[[MapData]]` internal slot, throw a **TypeError** exception.
4. Let *entries* be the List that is *M*.`[[MapData]]`.
5. For each Record { `[[Key]]`, `[[Value]]` } *p* that is an element of *entries*, do
 - a. If *p*.`[[Key]]` is not empty and `SameValueZero(p.[[Key]], key)` is **true**, return **true**.
6. Return **false**.

23.1.3.8 Map.prototype.keys ()

The following steps are taken:

1. Let *M* be the **this** value.
2. Return ? `CreateMapIterator(M, "key")`.

23.1.3.9 Map.prototype.set (*key*, *value*)

The following steps are taken:

1. Let *M* be the **this** value.
2. If `Type(M)` is not Object, throw a **TypeError** exception.
3. If *M* does not have a `[[MapData]]` internal slot, throw a **TypeError** exception.
4. Let *entries* be the List that is *M*.`[[MapData]]`.
5. For each Record { `[[Key]]`, `[[Value]]` } *p* that is an element of *entries*, do
 - a. If *p*.`[[Key]]` is not empty and `SameValueZero(p.[[Key]], key)` is **true**, then
 - i. Set *p*.`[[Value]]` to *value*.
 - ii. Return *M*.
6. If *key* is **-0**, set *key* to **+0**.
7. Let *p* be the Record { `[[Key]]`: *key*, `[[Value]]`: *value* }.
8. Append *p* as the last element of *entries*.
9. Return *M*.

23.1.3.10 get Map.prototype.size

Map.prototype.size is an **accessor property** whose set accessor function is **undefined**. Its get accessor function performs the following steps:

1. Let *M* be the **this** value.
2. If **Type**(*M*) is not Object, throw a **TypeError** exception.
3. If *M* does not have a `[[MapData]]` internal slot, throw a **TypeError** exception.
4. Let *entries* be the **List** that is *M*.`[[MapData]]`.
5. Let *count* be 0.
6. For each **Record** { `[[Key]]`, `[[Value]]` } *p* that is an element of *entries*, do
 - a. If *p*.`[[Key]]` is not **empty**, increase *count* by 1.
7. Return *count*.

23.1.3.11 Map.prototype.values ()

The following steps are taken:

1. Let *M* be the **this** value.
2. Return ? **CreateMapIterator**(*M*, "value").

23.1.3.12 Map.prototype [@@iterator] ()

The initial value of the @@iterator property is the same **function object** as the initial value of the **entries** property.

23.1.3.13 Map.prototype [@@toStringTag]

The initial value of the @@toStringTag property is the String value "**Map**".

This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **true** }.

23.1.4 Properties of Map Instances

Map instances are ordinary objects that inherit properties from the Map prototype. Map instances also have a `[[MapData]]` internal slot.

23.1.5 Map Iterator Objects

A Map Iterator is an object, that represents a specific iteration over some specific Map instance object. There is not a named **constructor** for Map Iterator objects. Instead, map iterator objects are created by calling certain methods of Map instance objects.

23.1.5.1 CreateMapIterator (*map*, *kind*)

Several methods of Map objects return Iterator objects. The abstract operation CreateMapIterator with arguments *map* and *kind* is used to create such iterator objects. It performs the following steps:

1. If **Type**(*map*) is not Object, throw a **TypeError** exception.
2. If *map* does not have a `[[MapData]]` internal slot, throw a **TypeError** exception.
3. Let *iterator* be **ObjectCreate**(%MapIteratorPrototype%, « `[[Map]]`, `[[MapNextIndex]]`, `[[MapIterationKind]]` »).
4. Set *iterator*.`[[Map]]` to *map*.
5. Set *iterator*.`[[MapNextIndex]]` to 0.
6. Set *iterator*.`[[MapIterationKind]]` to *kind*.
7. Return *iterator*.

23.1.5.2 The %MapIteratorPrototype% Object

The %MapIteratorPrototype% object:

- has properties that are inherited by all Map Iterator Objects.
- is an ordinary object.
- has a [[Prototype]] internal slot whose value is the intrinsic object %IteratorPrototype%.
- has the following properties:

23.1.5.2.1 %MapIteratorPrototype%.next ()

1. Let *O* be the **this** value.
2. If *Type*(*O*) is not Object, throw a **TypeError** exception.
3. If *O* does not have all of the internal slots of a Map Iterator Instance (23.1.5.3), throw a **TypeError** exception.
4. Let *m* be *O*.[[Map]].
5. Let *index* be *O*.[[MapNextIndex]].
6. Let *itemKind* be *O*.[[MapIterationKind]].
7. If *m* is **undefined**, return *CreateIterResultObject*(**undefined**, **true**).
8. **Assert**: *m* has a [[MapData]] internal slot.
9. Let *entries* be the *List* that is *m*.[[MapData]].
10. Let *numEntries* be the number of elements of *entries*.
11. NOTE: *numEntries* must be redetermined each time this method is evaluated.
12. Repeat, while *index* is less than *numEntries*,
 - a. Let *e* be the *Record* { [[Key]], [[Value]] } that is the value of *entries*[*index*].
 - b. Increase *index* by 1.
 - c. Set *O*.[[MapNextIndex]] to *index*.
 - d. If *e*.[[Key]] is not **empty**, then
 - i. If *itemKind* is **"key"**, let *result* be *e*.[[Key]].
 - ii. Else if *itemKind* is **"value"**, let *result* be *e*.[[Value]].
 - iii. Else,
 1. **Assert**: *itemKind* is **"key+value"**.
 2. Let *result* be *CreateArrayFromList*(« *e*.[[Key]], *e*.[[Value]] »).
 - iv. Return *CreateIterResultObject*(*result*, **false**).
13. Set *O*.[[Map]] to **undefined**.
14. Return *CreateIterResultObject*(**undefined**, **true**).

23.1.5.2.2 %MapIteratorPrototype% [@@toStringTag]

The initial value of the @@toStringTag property is the String value **"Map Iterator"**.

This property has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **true** }.

23.1.5.3 Properties of Map Iterator Instances

Map Iterator instances are ordinary objects that inherit properties from the %MapIteratorPrototype% intrinsic object. Map Iterator instances are initially created with the internal slots described in Table 60.

Table 60: Internal Slots of Map Iterator Instances

Internal Slot	Description
---------------	-------------

[[Map]]	The Map object that is being iterated.
[[MapNextIndex]]	The integer index of the next Map data element to be examined by this iterator.
[[MapIterationKind]]	A String value that identifies what is to be returned for each element of the iteration. The possible values are: "key" , "value" , "key+value" .

23.2 Set Objects

Set objects are collections of ECMAScript language values. A distinct value may only occur once as an element of a Set's collection. Distinct values are discriminated using the [SameValueZero](#) comparison algorithm.

Set objects must be implemented using either hash tables or other mechanisms that, on average, provide access times that are sublinear on the number of elements in the collection. The data structures used in this Set objects specification is only intended to describe the required observable semantics of Set objects. It is not intended to be a viable implementation model.

23.2.1 The Set Constructor

The Set [constructor](#):

is the intrinsic object %Set%.

is the initial value of the **Set** property of the [global object](#).

creates and initializes a new Set object when called as a [constructor](#).

is not intended to be called as a function and will throw an exception when called in that manner.

is designed to be subclassable. It may be used as the value in an **extends** clause of a class definition. Subclass constructors that intend to inherit the specified **Set** behaviour must include a **super** call to the [Set constructor](#) to create and initialize the subclass instance with the internal state necessary to support the **Set.prototype** built-in methods.

23.2.1.1 Set ([[iterable](#)])

When the **Set** function is called with optional argument [iterable](#), the following steps are taken:

1. If NewTarget is **undefined**, throw a **TypeError** exception.
2. Let *set* be ? [OrdinaryCreateFromConstructor](#)(NewTarget, "%SetPrototype%", « [[SetData]] »).
3. Set *set*.[[SetData]] to a new empty [List](#).
4. If [iterable](#) is not present, set [iterable](#) to **undefined**.
5. If [iterable](#) is either **undefined** or **null**, return *set*.
6. Let *adder* be ? [Get](#)(*set*, "add").
7. If [IsCallable](#)(*adder*) is **false**, throw a **TypeError** exception.
8. Let *iteratorRecord* be ? [GetIterator](#)([iterable](#)).
9. Repeat,
 - a. Let *next* be ? [IteratorStep](#)(*iteratorRecord*).
 - b. If *next* is **false**, return *set*.
 - c. Let *nextValue* be ? [IteratorValue](#)(*next*).
 - d. Let *status* be [Call](#)(*adder*, *set*, « *nextValue* »).
 - e. If *status* is an [abrupt completion](#), return ? [IteratorClose](#)(*iteratorRecord*, *status*).

23.2.2 Properties of the Set Constructor

The Set [constructor](#):

has a `[[Prototype]]` internal slot whose value is the intrinsic object `%FunctionPrototype%`.
has the following properties:

23.2.2.1 Set.prototype

The initial value of `Set.prototype` is the intrinsic `%SetPrototype%` object.

This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **false** }.

23.2.2.2 get Set [@@species]

`Set[@@species]` is an [accessor property](#) whose set accessor function is **undefined**. Its get accessor function performs the following steps:

1. Return the **this** value.

The value of the **name** property of this function is `"get [Symbol.species]"`.

NOTE

Methods that create derived collection objects should call `@@species` to determine the [constructor](#) to use to create the derived objects. Subclass [constructor](#) may over-ride `@@species` to change the default [constructor](#) assignment.

23.2.3 Properties of the Set Prototype Object

The Set prototype object:

is the intrinsic object `%SetPrototype%`.
has a `[[Prototype]]` internal slot whose value is the intrinsic object `%ObjectPrototype%`.
is an ordinary object.
does not have a `[[SetData]]` internal slot.

23.2.3.1 Set.prototype.add (*value*)

The following steps are taken:

1. Let *S* be the **this** value.
2. If `Type(S)` is not `Object`, throw a **TypeError** exception.
3. If *S* does not have a `[[SetData]]` internal slot, throw a **TypeError** exception.
4. Let *entries* be the [List](#) that is `S.[[SetData]]`.
5. For each *e* that is an element of *entries*, do
 - a. If *e* is not **empty** and `SameValueZero(e, value)` is **true**, then
 - i. Return *S*.
6. If *value* is **-0**, set *value* to **+0**.
7. Append *value* as the last element of *entries*.
8. Return *S*.

23.2.3.2 Set.prototype.clear ()

The following steps are taken:

1. Let *S* be the **this** value.
2. If **Type**(*S*) is not Object, throw a **TypeError** exception.
3. If *S* does not have a **[[SetData]]** internal slot, throw a **TypeError** exception.
4. Let *entries* be the **List** that is *S*.**[[SetData]]**.
5. For each *e* that is an element of *entries*, do
 - a. Replace the element of *entries* whose value is *e* with an element whose value is **empty**.
6. Return **undefined**.

NOTE

The existing **[[SetData]]** **List** is preserved because there may be existing Set Iterator objects that are suspended midway through iterating over that **List**.

23.2.3.3 Set.prototype.constructor

The initial value of **Set.prototype.constructor** is the intrinsic object **%Set%**.

23.2.3.4 Set.prototype.delete (*value*)

The following steps are taken:

1. Let *S* be the **this** value.
2. If **Type**(*S*) is not Object, throw a **TypeError** exception.
3. If *S* does not have a **[[SetData]]** internal slot, throw a **TypeError** exception.
4. Let *entries* be the **List** that is *S*.**[[SetData]]**.
5. For each *e* that is an element of *entries*, do
 - a. If *e* is not **empty** and **SameValueZero**(*e*, *value*) is **true**, then
 - i. Replace the element of *entries* whose value is *e* with an element whose value is **empty**.
 - ii. Return **true**.
6. Return **false**.

NOTE

The value **empty** is used as a specification device to indicate that an entry has been deleted. Actual implementations may take other actions such as physically removing the entry from internal data structures.

23.2.3.5 Set.prototype.entries ()

The following steps are taken:

1. Let *S* be the **this** value.
2. Return ? **CreateSetIterator**(*S*, "**key+value**").

NOTE

For iteration purposes, a Set appears similar to a Map where each entry has the same value for its key and value.

23.2.3.6 Set.prototype.forEach (*callbackfn* [, *thisArg*])

When the **forEach** method is called with one or two arguments, the following steps are taken:

1. Let *S* be the **this** value.
2. If **Type**(*S*) is not Object, throw a **TypeError** exception.
3. If *S* does not have a **[[SetData]]** internal slot, throw a **TypeError** exception.
4. If **IsCallable**(*callbackfn*) is **false**, throw a **TypeError** exception.
5. If *thisArg* is present, let *T* be *thisArg*; else let *T* be **undefined**.
6. Let *entries* be the **List** that is *S*.**[[SetData]]**.
7. For each *e* that is an element of *entries*, in original insertion order, do
 - a. If *e* is not empty, then
 - i. Perform ? **Call**(*callbackfn*, *T*, « *e*, *e*, *S* »).
8. Return **undefined**.

NOTE

callbackfn should be a function that accepts three arguments. **forEach** calls *callbackfn* once for each value present in the set object, in value insertion order. *callbackfn* is called only for values of the Set which actually exist; it is not called for keys that have been deleted from the set.

If a *thisArg* parameter is provided, it will be used as the **this** value for each invocation of *callbackfn*. If it is not provided, **undefined** is used instead.

callbackfn is called with three arguments: the first two arguments are a value contained in the Set. The same value is passed for both arguments. The Set object being traversed is passed as the third argument.

The *callbackfn* is called with three arguments to be consistent with the call back functions used by **forEach** methods for Map and Array. For Sets, each item value is considered to be both the key and the value.

forEach does not directly mutate the object on which it is called but the object may be mutated by the calls to *callbackfn*.

Each value is normally visited only once. However, a value will be revisited if it is deleted after it has been visited and then re-added before the **forEach** call completes. Values that are deleted after the call to **forEach** begins and before being visited are not visited unless the value is added again before the **forEach** call completes. New values added after the call to **forEach** begins are visited.

23.2.3.7 Set.prototype.has (*value*)

The following steps are taken:

1. Let *S* be the **this** value.
2. If **Type**(*S*) is not Object, throw a **TypeError** exception.
3. If *S* does not have a **[[SetData]]** internal slot, throw a **TypeError** exception.
4. Let *entries* be the **List** that is *S*.**[[SetData]]**.
5. For each *e* that is an element of *entries*, do
 - a. If *e* is not empty and **SameValueZero**(*e*, *value*) is **true**, return **true**.
6. Return **false**.

23.2.3.8 Set.prototype.keys ()

The initial value of the **keys** property is the same [function object](#) as the initial value of the **values** property.

NOTE

For iteration purposes, a Set appears similar to a Map where each entry has the same value for its key and value.

23.2.3.9 get Set.prototype.size

Set.prototype.size is an [accessor property](#) whose set accessor function is **undefined**. Its get accessor function performs the following steps:

1. Let *S* be the **this** value.
2. If [Type\(S\)](#) is not Object, throw a **TypeError** exception.
3. If *S* does not have a [\[\[SetData\]\]](#) internal slot, throw a **TypeError** exception.
4. Let *entries* be the [List](#) that is *S*.[\[\[SetData\]\]](#).
5. Let *count* be 0.
6. For each *e* that is an element of *entries*, do
 - a. If *e* is not empty, increase *count* by 1.
7. Return *count*.

23.2.3.10 Set.prototype.values ()

The following steps are taken:

1. Let *S* be the **this** value.
2. Return ? [CreateSetIterator](#)(*S*, "value").

23.2.3.11 Set.prototype [@@iterator] ()

The initial value of the @@iterator property is the same [function object](#) as the initial value of the **values** property.

23.2.3.12 Set.prototype [@@toStringTag]

The initial value of the @@toStringTag property is the String value "Set".

This property has the attributes { [\[\[Writable\]\]](#): **false**, [\[\[Enumerable\]\]](#): **false**, [\[\[Configurable\]\]](#): **true** }.

23.2.4 Properties of Set Instances

Set instances are ordinary objects that inherit properties from the Set prototype. Set instances also have a [\[\[SetData\]\]](#) internal slot.

23.2.5 Set Iterator Objects

A Set Iterator is an ordinary object, with the structure defined below, that represents a specific iteration over some specific Set instance object. There is not a named [constructor](#) for Set Iterator objects. Instead, set iterator objects are created by calling certain methods of Set instance objects.

23.2.5.1 CreateSetIterator (*set*, *kind*)

Several methods of Set objects return Iterator objects. The abstract operation CreateSetIterator with arguments *set* and *kind* is used to create such iterator objects. It performs the following steps:

1. If **Type**(*set*) is not Object, throw a **TypeError** exception.
2. If *set* does not have a **[[SetData]]** internal slot, throw a **TypeError** exception.
3. Let *iterator* be **ObjectCreate**(%SetIteratorPrototype%, « **[[IteratedSet]]**, **[[SetNextIndex]]**, **[[SetIterationKind]]** »).
4. Set *iterator*.**[[IteratedSet]]** to *set*.
5. Set *iterator*.**[[SetNextIndex]]** to 0.
6. Set *iterator*.**[[SetIterationKind]]** to *kind*.
7. Return *iterator*.

23.2.5.2 The %SetIteratorPrototype% Object

The %SetIteratorPrototype% object:

has properties that are inherited by all Set Iterator Objects.

is an ordinary object.

has a **[[Prototype]]** internal slot whose value is the intrinsic object %IteratorPrototype%.

has the following properties:

23.2.5.2.1 %SetIteratorPrototype%.next ()

1. Let *O* be the **this** value.
2. If **Type**(*O*) is not Object, throw a **TypeError** exception.
3. If *O* does not have all of the internal slots of a Set Iterator Instance (23.2.5.3), throw a **TypeError** exception.
4. Let *s* be *O*.**[[IteratedSet]]**.
5. Let *index* be *O*.**[[SetNextIndex]]**.
6. Let *itemKind* be *O*.**[[SetIterationKind]]**.
7. If *s* is **undefined**, return **CreateIterResultObject**(**undefined**, **true**).
8. **Assert**: *s* has a **[[SetData]]** internal slot.
9. Let *entries* be the **List** that is *s*.**[[SetData]]**.
10. Let *numEntries* be the number of elements of *entries*.
11. NOTE: *numEntries* must be redetermined each time this method is evaluated.
12. Repeat, while *index* is less than *numEntries*,
 - a. Let *e* be *entries*[*index*].
 - b. Increase *index* by 1.
 - c. Set *O*.**[[SetNextIndex]]** to *index*.
 - d. If *e* is not **empty**, then
 - i. If *itemKind* is **"key+value"**, then
 1. Return **CreateIterResultObject**(**CreateArrayFromList**(« *e*, *e* »), **false**).
 - ii. Return **CreateIterResultObject**(*e*, **false**).
13. Set *O*.**[[IteratedSet]]** to **undefined**.
14. Return **CreateIterResultObject**(**undefined**, **true**).

23.2.5.2.2 %SetIteratorPrototype% [@@toStringTag]

The initial value of the @@toStringTag property is the String value **"Set Iterator"**.

This property has the attributes { **[[Writable]]**: **false**, **[[Enumerable]]**: **false**, **[[Configurable]]**: **true** }.

23.2.5.3 Properties of Set Iterator Instances

Set Iterator instances are ordinary objects that inherit properties from the %SetIteratorPrototype% intrinsic object. Set Iterator instances are initially created with the internal slots specified in Table 61.

Table 61: Internal Slots of Set Iterator Instances

Internal Slot	Description
[[IteratedSet]]	The Set object that is being iterated.
[[SetNextIndex]]	The integer index of the next Set data element to be examined by this iterator
[[SetIterationKind]]	A String value that identifies what is to be returned for each element of the iteration. The possible values are: "key", "value", "key+value". "key" and "value" have the same meaning.

23.3 WeakMap Objects

WeakMap objects are collections of key/value pairs where the keys are objects and values may be arbitrary ECMAScript language values. A WeakMap may be queried to see if it contains a key/value pair with a specific key, but no mechanism is provided for enumerating the objects it holds as keys. If an object that is being used as the key of a WeakMap key/value pair is only reachable by following a chain of references that start within that WeakMap, then that key/value pair is inaccessible and is automatically removed from the WeakMap. WeakMap implementations must detect and remove such key/value pairs and any associated resources.

An implementation may impose an arbitrarily determined latency between the time a key/value pair of a WeakMap becomes inaccessible and the time when the key/value pair is removed from the WeakMap. If this latency was observable to ECMAScript program, it would be a source of indeterminacy that could impact program execution. For that reason, an ECMAScript implementation must not provide any means to observe a key of a WeakMap that does not require the observer to present the observed key.

WeakMap objects must be implemented using either hash tables or other mechanisms that, on average, provide access times that are sublinear on the number of key/value pairs in the collection. The data structure used in this WeakMap objects specification are only intended to describe the required observable semantics of WeakMap objects. It is not intended to be a viable implementation model.

NOTE

WeakMap and WeakSets are intended to provide mechanisms for dynamically associating state with an object in a manner that does not “leak” memory resources if, in the absence of the WeakMap or WeakSet, the object otherwise became inaccessible and subject to resource reclamation by the implementation's garbage collection mechanisms. This characteristic can be achieved by using an inverted per-object mapping of weak map instances to keys. Alternatively each weak map may internally store its key to value mappings but this approach requires coordination between the WeakMap or WeakSet implementation and the garbage collector. The following references describe mechanism that may be useful to implementations of WeakMap and WeakSets:

Barry Hayes. 1997. Ephemerons: a new finalization mechanism. In *Proceedings of the 12th ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications (OOPSLA '97)*, A. Michael Berman (Ed.). ACM, New York, NY, USA, 176-183, <http://doi.acm.org/10.1145/263698.263733>.

Alexandra Barros, Roberto Ierusalimsky, Eliminating Cycles in Weak Tables. *Journal of Universal Computer Science - J.UCS*, vol. 14, no. 21, pp. 3481-3497, 2008, http://www.jucs.org/jucs_14_21/eliminating_cycles_in_weak

23.3.1 The WeakMap Constructor

The WeakMap [constructor](#):

is the intrinsic object %WeakMap%.

is the initial value of the **WeakMap** property of the [global object](#).

creates and initializes a new WeakMap object when called as a [constructor](#).

is not intended to be called as a function and will throw an exception when called in that manner.

is designed to be subclassable. It may be used as the value in an **extends** clause of a class definition. Subclass constructors that intend to inherit the specified **WeakMap** behaviour must include a **super** call to the **WeakMap** [constructor](#) to create and initialize the subclass instance with the internal state necessary to support the **WeakMap.prototype** built-in methods.

23.3.1.1 WeakMap ([[iterable](#)])

When the **WeakMap** function is called with optional argument [iterable](#), the following steps are taken:

1. If NewTarget is **undefined**, throw a **TypeError** exception.
2. Let [map](#) be ? **OrdinaryCreateFromConstructor**(NewTarget, "%WeakMapPrototype%", « [[WeakMapData]] »).
3. Set [map](#).[[WeakMapData]] to a new empty [List](#).
4. If [iterable](#) is not present, or is either **undefined** or **null**, return [map](#).
5. Let [adder](#) be ? **Get**([map](#), "set").
6. Return ? **AddEntriesFromIterable**([map](#), [iterable](#), [adder](#)).

NOTE

If the parameter [iterable](#) is present, it is expected to be an object that implements an @@iterator method that returns an iterator object that produces a two element array-like object whose first element is a value that will be used as a WeakMap key and whose second element is the value to associate with that key.

23.3.2 Properties of the WeakMap Constructor

The WeakMap [constructor](#):

has a [[Prototype]] internal slot whose value is the intrinsic object %FunctionPrototype%.

has the following properties:

23.3.2.1 WeakMap.prototype

The initial value of **WeakMap.prototype** is the intrinsic object %WeakMapPrototype%.

This property has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **false** }.

23.3.3 Properties of the WeakMap Prototype Object

The WeakMap prototype object:

is the intrinsic object %WeakMapPrototype%.

has a `[[Prototype]]` internal slot whose value is the intrinsic object `%ObjectPrototype%`.
is an ordinary object.
does not have a `[[WeakMapData]]` internal slot.

23.3.3.1 WeakMap.prototype.constructor

The initial value of `WeakMap.prototype.constructor` is the intrinsic object `%WeakMap%`.

23.3.3.2 WeakMap.prototype.delete (*key*)

The following steps are taken:

1. Let *M* be the **this** value.
2. If `Type(M)` is not Object, throw a **TypeError** exception.
3. If *M* does not have a `[[WeakMapData]]` internal slot, throw a **TypeError** exception.
4. Let *entries* be the [List](#) that is *M*.`[[WeakMapData]]`.
5. If `Type(key)` is not Object, return **false**.
6. For each [Record](#) { `[[Key]]`, `[[Value]]` } *p* that is an element of *entries*, do
 - a. If *p*.`[[Key]]` is not **empty** and `SameValue(p.[[Key]], key)` is **true**, then
 - i. Set *p*.`[[Key]]` to **empty**.
 - ii. Set *p*.`[[Value]]` to **empty**.
 - iii. Return **true**.
7. Return **false**.

NOTE

The value **empty** is used as a specification device to indicate that an entry has been deleted. Actual implementations may take other actions such as physically removing the entry from internal data structures.

23.3.3.3 WeakMap.prototype.get (*key*)

The following steps are taken:

1. Let *M* be the **this** value.
2. If `Type(M)` is not Object, throw a **TypeError** exception.
3. If *M* does not have a `[[WeakMapData]]` internal slot, throw a **TypeError** exception.
4. Let *entries* be the [List](#) that is *M*.`[[WeakMapData]]`.
5. If `Type(key)` is not Object, return **undefined**.
6. For each [Record](#) { `[[Key]]`, `[[Value]]` } *p* that is an element of *entries*, do
 - a. If *p*.`[[Key]]` is not **empty** and `SameValue(p.[[Key]], key)` is **true**, return *p*.`[[Value]]`.
7. Return **undefined**.

23.3.3.4 WeakMap.prototype.has (*key*)

The following steps are taken:

1. Let *M* be the **this** value.
2. If `Type(M)` is not Object, throw a **TypeError** exception.
3. If *M* does not have a `[[WeakMapData]]` internal slot, throw a **TypeError** exception.
4. Let *entries* be the [List](#) that is *M*.`[[WeakMapData]]`.

5. If **Type**(*key*) is not Object, return **false**.
6. For each **Record** { [[Key]], [[Value]] } *p* that is an element of *entries*, do
 - a. If *p*.[[Key]] is not empty and **SameValue**(*p*.[[Key]], *key*) is **true**, return **true**.
7. Return **false**.

23.3.3.5 WeakMap.prototype.set (*key*, *value*)

The following steps are taken:

1. Let *M* be the **this** value.
2. If **Type**(*M*) is not Object, throw a **TypeError** exception.
3. If *M* does not have a [[WeakMapData]] internal slot, throw a **TypeError** exception.
4. Let *entries* be the **List** that is *M*.[[WeakMapData]].
5. If **Type**(*key*) is not Object, throw a **TypeError** exception.
6. For each **Record** { [[Key]], [[Value]] } *p* that is an element of *entries*, do
 - a. If *p*.[[Key]] is not empty and **SameValue**(*p*.[[Key]], *key*) is **true**, then
 - i. Set *p*.[[Value]] to *value*.
 - ii. Return *M*.
7. Let *p* be the **Record** { [[Key]]: *key*, [[Value]]: *value* }.
8. Append *p* as the last element of *entries*.
9. Return *M*.

23.3.3.6 WeakMap.prototype [@@toStringTag]

The initial value of the @@toStringTag property is the String value "**WeakMap**".

This property has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **true** }.

23.3.4 Properties of WeakMap Instances

WeakMap instances are ordinary objects that inherit properties from the WeakMap prototype. WeakMap instances also have a [[WeakMapData]] internal slot.

23.4 WeakSet Objects

WeakSet objects are collections of objects. A distinct object may only occur once as an element of a WeakSet's collection. A WeakSet may be queried to see if it contains a specific object, but no mechanism is provided for enumerating the objects it holds. If an object that is contained by a WeakSet is only reachable by following a chain of references that start within that WeakSet, then that object is inaccessible and is automatically removed from the WeakSet. WeakSet implementations must detect and remove such objects and any associated resources.

An implementation may impose an arbitrarily determined latency between the time an object contained in a WeakSet becomes inaccessible and the time when the object is removed from the WeakSet. If this latency was observable to ECMAScript program, it would be a source of indeterminacy that could impact program execution. For that reason, an ECMAScript implementation must not provide any means to determine if a WeakSet contains a particular object that does not require the observer to present the observed object.

WeakSet objects must be implemented using either hash tables or other mechanisms that, on average, provide access times that are sublinear on the number of elements in the collection. The data structure used in this WeakSet objects

specification is only intended to describe the required observable semantics of WeakSet objects. It is not intended to be a viable implementation model.

NOTE

See the NOTE in 23.3.

23.4.1 The WeakSet Constructor

The WeakSet [constructor](#):

is the intrinsic object %WeakSet%.

is the initial value of the **WeakSet** property of the [global object](#).

creates and initializes a new WeakSet object when called as a [constructor](#).

is not intended to be called as a function and will throw an exception when called in that manner.

is designed to be subclassable. It may be used as the value in an **extends** clause of a class definition. Subclass constructors that intend to inherit the specified **WeakSet** behaviour must include a **super** call to the **WeakSet** [constructor](#) to create and initialize the subclass instance with the internal state necessary to support the **WeakSet.prototype** built-in methods.

23.4.1.1 WeakSet ([*iterable*])

When the **WeakSet** function is called with optional argument *iterable*, the following steps are taken:

1. If NewTarget is **undefined**, throw a **TypeError** exception.
2. Let *set* be ? [OrdinaryCreateFromConstructor](#)(NewTarget, "%WeakSetPrototype%", « [[WeakSetData]] »).
3. Set *set*.[[WeakSetData]] to a new empty [List](#).
4. If *iterable* is not present, set *iterable* to **undefined**.
5. If *iterable* is either **undefined** or **null**, return *set*.
6. Let *adder* be ? [Get](#)(*set*, "add").
7. If [IsCallable](#)(*adder*) is **false**, throw a **TypeError** exception.
8. Let *iteratorRecord* be ? [GetIterator](#)(*iterable*).
9. Repeat,
 - a. Let *next* be ? [IteratorStep](#)(*iteratorRecord*).
 - b. If *next* is **false**, return *set*.
 - c. Let *nextValue* be ? [IteratorValue](#)(*next*).
 - d. Let *status* be [Call](#)(*adder*, *set*, « *nextValue* »).
 - e. If *status* is an [abrupt completion](#), return ? [IteratorClose](#)(*iteratorRecord*, *status*).

23.4.2 Properties of the WeakSet Constructor

The WeakSet [constructor](#):

has a [[Prototype]] internal slot whose value is the intrinsic object %FunctionPrototype%.

has the following properties:

23.4.2.1 WeakSet.prototype

The initial value of **WeakSet.prototype** is the intrinsic %WeakSetPrototype% object.

This property has the attributes { `[[Writable]]: false`, `[[Enumerable]]: false`, `[[Configurable]]: false` }.

23.4.3 Properties of the WeakSet Prototype Object

The WeakSet prototype object:

is the intrinsic object `%WeakSetPrototype%`.

has a `[[Prototype]]` internal slot whose value is the intrinsic object `%ObjectPrototype%`.

is an ordinary object.

does not have a `[[WeakSetData]]` internal slot.

23.4.3.1 WeakSet.prototype.add (*value*)

The following steps are taken:

1. Let *S* be the **this** value.
2. If `Type(S)` is not Object, throw a **TypeError** exception.
3. If *S* does not have a `[[WeakSetData]]` internal slot, throw a **TypeError** exception.
4. If `Type(value)` is not Object, throw a **TypeError** exception.
5. Let *entries* be the List that is *S*.`[[WeakSetData]]`.
6. For each *e* that is an element of *entries*, do
 - a. If *e* is not empty and `SameValue(e, value)` is **true**, then
 - i. Return *S*.
7. Append *value* as the last element of *entries*.
8. Return *S*.

23.4.3.2 WeakSet.prototype.constructor

The initial value of `WeakSet.prototype.constructor` is the `%WeakSet%` intrinsic object.

23.4.3.3 WeakSet.prototype.delete (*value*)

The following steps are taken:

1. Let *S* be the **this** value.
2. If `Type(S)` is not Object, throw a **TypeError** exception.
3. If *S* does not have a `[[WeakSetData]]` internal slot, throw a **TypeError** exception.
4. If `Type(value)` is not Object, return **false**.
5. Let *entries* be the List that is *S*.`[[WeakSetData]]`.
6. For each *e* that is an element of *entries*, do
 - a. If *e* is not empty and `SameValue(e, value)` is **true**, then
 - i. Replace the element of *entries* whose value is *e* with an element whose value is empty.
 - ii. Return **true**.
7. Return **false**.

NOTE

The value `empty` is used as a specification device to indicate that an entry has been deleted. Actual implementations may take other actions such as physically removing the entry from internal data structures.

23.4.3.4 WeakSet.prototype.has (*value*)

The following steps are taken:

1. Let *S* be the **this** value.
2. If **Type**(*S*) is not Object, throw a **TypeError** exception.
3. If *S* does not have a **[[WeakSetData]]** internal slot, throw a **TypeError** exception.
4. Let *entries* be the **List** that is *S*.**[[WeakSetData]]**.
5. If **Type**(*value*) is not Object, return **false**.
6. For each *e* that is an element of *entries*, do
 - a. If *e* is not empty and **SameValue**(*e*, *value*) is **true**, return **true**.
7. Return **false**.

23.4.3.5 WeakSet.prototype [@@toStringTag]

The initial value of the @@toStringTag property is the String value "**WeakSet**".

This property has the attributes { **[[Writable]]**: **false**, **[[Enumerable]]**: **false**, **[[Configurable]]**: **true** }.

23.4.4 Properties of WeakSet Instances

WeakSet instances are ordinary objects that inherit properties from the WeakSet prototype. WeakSet instances also have a **[[WeakSetData]]** internal slot.

24 Structured Data

24.1 ArrayBuffer Objects

24.1.1 Abstract Operations For ArrayBuffer Objects

24.1.1.1 AllocateArrayBuffer (*constructor*, *byteLength*)

The abstract operation AllocateArrayBuffer with arguments *constructor* and *byteLength* is used to create an ArrayBuffer object. It performs the following steps:

1. Let *obj* be ? **OrdinaryCreateFromConstructor**(*constructor*, "%ArrayBufferPrototype%", « **[[ArrayBufferData]]**, **[[ArrayBufferByteLength]]**, **[[ArrayBufferDetachKey]]** »).
2. **Assert**: *byteLength* is an integer value ≥ 0.
3. Let *block* be ? **CreateByteDataBlock**(*byteLength*).
4. Set *obj*.**[[ArrayBufferData]]** to *block*.
5. Set *obj*.**[[ArrayBufferByteLength]]** to *byteLength*.
6. Return *obj*.

24.1.1.2 IsDetachedBuffer (*arrayBuffer*)

The abstract operation IsDetachedBuffer with argument *arrayBuffer* performs the following steps:

1. **Assert:** `Type(arrayBuffer)` is Object and it has an `[[ArrayBufferData]]` internal slot.
2. If `arrayBuffer.[[ArrayBufferData]]` is **null**, return **true**.
3. Return **false**.

24.1.1.3 DetachArrayBuffer (*arrayBuffer* [, *key*])

The abstract operation DetachArrayBuffer with argument *arrayBuffer* and optional argument *key* performs the following steps:

1. **Assert:** `Type(arrayBuffer)` is Object and it has `[[ArrayBufferData]]`, `[[ArrayBufferByteLength]]`, and `[[ArrayBufferDetachKey]]` internal slots.
2. **Assert:** `IsSharedArrayBuffer(arrayBuffer)` is **false**.
3. If *key* is not present, set *key* to **undefined**.
4. If `SameValue(arrayBuffer.[[ArrayBufferDetachKey]], key)` is **false**, throw a **TypeError** exception.
5. Set `arrayBuffer.[[ArrayBufferData]]` to **null**.
6. Set `arrayBuffer.[[ArrayBufferByteLength]]` to 0.
7. Return `NormalCompletion(null)`.

NOTE

Detaching an ArrayBuffer instance disassociates the **Data Block** used as its backing store from the instance and sets the byte length of the buffer to 0. No operations defined by this specification use the DetachArrayBuffer abstract operation. However, an ECMAScript implementation or host environment may define such operations.

24.1.1.4 CloneArrayBuffer (*srcBuffer*, *srcByteOffset*, *srcLength*, *cloneConstructor*)

The abstract operation CloneArrayBuffer takes four parameters, an ArrayBuffer *srcBuffer*, an integer offset *srcByteOffset*, an integer length *srcLength*, and a **constructor** function *cloneConstructor*. It creates a new ArrayBuffer whose data is a copy of *srcBuffer*'s data over the range starting at *srcByteOffset* and continuing for *srcLength* bytes. This operation performs the following steps:

1. **Assert:** `Type(srcBuffer)` is Object and it has an `[[ArrayBufferData]]` internal slot.
2. **Assert:** `IsConstructor(cloneConstructor)` is **true**.
3. Let *targetBuffer* be `? AllocateArrayBuffer(cloneConstructor, srcLength)`.
4. If `IsDetachedBuffer(srcBuffer)` is **true**, throw a **TypeError** exception.
5. Let *srcBlock* be `srcBuffer.[[ArrayBufferData]]`.
6. Let *targetBlock* be `targetBuffer.[[ArrayBufferData]]`.
7. Perform `CopyDataBlockBytes(targetBlock, 0, srcBlock, srcByteOffset, srcLength)`.
8. Return *targetBuffer*.

24.1.1.5 RawBytesToNumber (*type*, *rawBytes*, *isLittleEndian*)

The abstract operation RawBytesToNumber takes three parameters, a String *type*, a **List** *rawBytes*, and a Boolean *isLittleEndian*. This operation performs the following steps:

1. Let *elementSize* be the Number value of the Element Size value specified in Table 59 for Element Type *type*.
2. If *isLittleEndian* is **false**, reverse the order of the elements of *rawBytes*.
3. If *type* is **"Float32"**, then
 - a. Let *value* be the byte elements of *rawBytes* concatenated and interpreted as a little-endian bit string encoding of an IEEE 754-2008 binary32 value.

- b. If *value* is an IEEE 754-2008 binary32 NaN value, return the **NaN** Number value.
- c. Return the Number value that corresponds to *value*.
4. If *type* is **"Float64"**, then
 - a. Let *value* be the byte elements of *rawBytes* concatenated and interpreted as a little-endian bit string encoding of an IEEE 754-2008 binary64 value.
 - b. If *value* is an IEEE 754-2008 binary64 NaN value, return the **NaN** Number value.
 - c. Return the Number value that corresponds to *value*.
5. If the first code unit of *type* is the code unit 0x0055 (LATIN CAPITAL LETTER U), then
 - a. Let *intValue* be the byte elements of *rawBytes* concatenated and interpreted as a bit string encoding of an unsigned little-endian binary number.
6. Else,
 - a. Let *intValue* be the byte elements of *rawBytes* concatenated and interpreted as a bit string encoding of a binary little-endian 2's complement number of bit length *elementSize* × 8.
7. Return the Number value that corresponds to *intValue*.

24.1.1.6 GetValueFromBuffer (*arrayBuffer*, *byteIndex*, *type*, *isTypedArray*, *order* [, *isLittleEndian*])

The abstract operation GetValueFromBuffer takes six parameters, an ArrayBuffer or SharedArrayBuffer *arrayBuffer*, an integer *byteIndex*, a String *type*, a Boolean *isTypedArray*, a String *order*, and optionally a Boolean *isLittleEndian*. This operation performs the following steps:

1. **Assert:** IsDetachedBuffer(*arrayBuffer*) is **false**.
2. **Assert:** There are sufficient bytes in *arrayBuffer* starting at *byteIndex* to represent a value of *type*.
3. **Assert:** *byteIndex* is an integer value ≥ 0.
4. Let *block* be *arrayBuffer*.[[ArrayBufferData]].
5. Let *elementSize* be the Number value of the Element Size value specified in Table 59 for Element Type *type*.
6. If IsSharedArrayBuffer(*arrayBuffer*) is **true**, then
 - a. Let *execution* be the [[CandidateExecution]] field of the *surrounding agent*'s Agent Record.
 - b. Let *eventList* be the [[EventList]] field of the element in *execution*.[[EventsRecords]] whose [[AgentSignifier]] is AgentSignifier().
 - c. If *isTypedArray* is **true** and *type* is **"Int8"**, **"Uint8"**, **"Int16"**, **"Uint16"**, **"Int32"**, or **"Uint32"**, let *noTear* be **true**; otherwise let *noTear* be **false**.
 - d. Let *rawValue* be a List of length *elementSize* of nondeterministically chosen byte values.
 - e. NOTE: In implementations, *rawValue* is the result of a non-atomic or atomic read instruction on the underlying hardware. The nondeterminism is a semantic prescription of the *memory model* to describe observable behaviour of hardware with weak consistency.
 - f. Let *readEvent* be ReadSharedMemory { [[Order]]: *order*, [[NoTear]]: *noTear*, [[Block]]: *block*, [[ByteIndex]]: *byteIndex*, [[ElementSize]]: *elementSize* }.
 - g. Append *readEvent* to *eventList*.
 - h. Append Chosen Value Record { [[Event]]: *readEvent*, [[ChosenValue]]: *rawValue* } to *execution*.[[ChosenValues]].
7. Else, let *rawValue* be a List of *elementSize* containing, in order, the *elementSize* sequence of bytes starting with *block*[*byteIndex*].
8. If *isLittleEndian* is not present, set *isLittleEndian* to the value of the [[LittleEndian]] field of the *surrounding agent*'s Agent Record.
9. Return RawBytesToNumber(*type*, *rawValue*, *isLittleEndian*).

24.1.1.7 NumberToRawBytes (*type*, *value*, *isLittleEndian*)

The abstract operation `NumberToRawBytes` takes three parameters, a String *type*, a Number *value*, and a Boolean *isLittleEndian*. This operation performs the following steps:

1. If *type* is **"Float32"**, then
 - a. Let *rawBytes* be a **List** containing the 4 bytes that are the result of converting *value* to IEEE 754-2008 binary32 format using “Round to nearest, ties to even” rounding mode. If *isLittleEndian* is **false**, the bytes are arranged in big endian order. Otherwise, the bytes are arranged in little endian order. If *value* is **NaN**, *rawBytes* may be set to any implementation chosen IEEE 754-2008 binary32 format Not-a-Number encoding. An implementation must always choose the same encoding for each implementation distinguishable **NaN** value.
2. Else if *type* is **"Float64"**, then
 - a. Let *rawBytes* be a **List** containing the 8 bytes that are the IEEE 754-2008 binary64 format encoding of *value*. If *isLittleEndian* is **false**, the bytes are arranged in big endian order. Otherwise, the bytes are arranged in little endian order. If *value* is **NaN**, *rawBytes* may be set to any implementation chosen IEEE 754-2008 binary64 format Not-a-Number encoding. An implementation must always choose the same encoding for each implementation distinguishable **NaN** value.
3. Else,
 - a. Let *n* be the Number value of the Element Size specified in Table 59 for Element Type *type*.
 - b. Let *convOp* be the abstract operation named in the Conversion Operation column in Table 59 for Element Type *type*.
 - c. Let *intValue* be *convOp*(*value*).
 - d. If *intValue* ≥ 0, then
 - i. Let *rawBytes* be a **List** containing the *n*-byte binary encoding of *intValue*. If *isLittleEndian* is **false**, the bytes are ordered in big endian order. Otherwise, the bytes are ordered in little endian order.
 - e. Else,
 - i. Let *rawBytes* be a **List** containing the *n*-byte binary 2's complement encoding of *intValue*. If *isLittleEndian* is **false**, the bytes are ordered in big endian order. Otherwise, the bytes are ordered in little endian order.
4. Return *rawBytes*.

24.1.1.8 SetValueInBuffer (*arrayBuffer*, *byteIndex*, *type*, *value*, *isTypedArray*, *order* [, *isLittleEndian*])

The abstract operation `SetValueInBuffer` takes seven parameters, an ArrayBuffer or SharedArrayBuffer *arrayBuffer*, an integer *byteIndex*, a String *type*, a Number *value*, a Boolean *isTypedArray*, a String *order*, and optionally a Boolean *isLittleEndian*. This operation performs the following steps:

1. **Assert:** `IsDetachedBuffer(arrayBuffer)` is **false**.
2. **Assert:** There are sufficient bytes in *arrayBuffer* starting at *byteIndex* to represent a value of *type*.
3. **Assert:** *byteIndex* is an integer value ≥ 0.
4. **Assert:** `Type(value)` is Number.
5. Let *block* be *arrayBuffer*.[[ArrayBufferData]].
6. Let *elementSize* be the Number value of the Element Size value specified in Table 59 for Element Type *type*.
7. If *isLittleEndian* is not present, set *isLittleEndian* to the value of the [[LittleEndian]] field of the *surrounding agent's Agent Record*.
8. Let *rawBytes* be `NumberToRawBytes(type, value, isLittleEndian)`.
9. If `IsSharedArrayBuffer(arrayBuffer)` is **true**, then
 - a. Let *execution* be the [[CandidateExecution]] field of the *surrounding agent's Agent Record*.
 - b. Let *eventList* be the [[EventList]] field of the element in *execution*.[[EventsRecords]] whose [[AgentSignifier]] is `AgentSignifier()`.

- c. If *isTypedArray* is **true** and *type* is "Int8", "Uint8", "Int16", "Uint16", "Int32", or "Uint32", let *noTear* be **true**; otherwise let *noTear* be **false**.
- d. Append **WriteSharedMemory** { [[Order]]: *order*, [[NoTear]]: *noTear*, [[Block]]: *block*, [[ByteIndex]]: *byteIndex*, [[ElementSize]]: *elementSize*, [[Payload]]: *rawBytes* } to *eventList*.
10. Else, store the individual bytes of *rawBytes* into *block*, in order, starting at *block*[*byteIndex*].
11. Return **NormalCompletion**(**undefined**).

24.1.1.9 GetModifySetValueInBuffer (*arrayBuffer*, *byteIndex*, *type*, *value*, *op* [, *isLittleEndian*])

The abstract operation **GetModifySetValueInBuffer** takes six parameters, a **SharedArrayBuffer** *arrayBuffer*, a nonnegative integer *byteIndex*, a String *type*, a Number *value*, a semantic function *op*, and optionally a Boolean *isLittleEndian*. This operation performs the following steps:

1. **Assert**: **IsSharedArrayBuffer**(*arrayBuffer*) is **true**.
2. **Assert**: There are sufficient bytes in *arrayBuffer* starting at *byteIndex* to represent a value of *type*.
3. **Assert**: *byteIndex* is an integer value ≥ 0 .
4. **Assert**: **Type**(*value*) is **Number**.
5. Let *block* be *arrayBuffer*.[[**ArrayBufferData**]].
6. Let *elementSize* be the **Number** value of the **Element Size** value specified in Table 59 for Element Type *type*.
7. If *isLittleEndian* is not present, set *isLittleEndian* to the value of the [[**LittleEndian**]] field of the **surrounding agent's Agent Record**.
8. Let *rawBytes* be **NumberToRawBytes**(*type*, *value*, *isLittleEndian*).
9. Let *execution* be the [[**CandidateExecution**]] field of the **surrounding agent's Agent Record**.
10. Let *eventList* be the [[**EventList**]] field of the element in *execution*.[[**EventsRecords**]] whose [[**AgentSignifier**]] is **AgentSignifier**() .
11. Let *rawBytesRead* be a **List** of length *elementSize* of nondeterministically chosen byte values.
12. NOTE: In implementations, *rawBytesRead* is the result of a load-link, of a load-exclusive, or of an operand of a read-modify-write instruction on the underlying hardware. The nondeterminism is a semantic prescription of the **memory model** to describe observable behaviour of hardware with weak consistency.
13. Let *rmwEvent* be **ReadModifyWriteSharedMemory** { [[Order]]: "**SeqCst**", [[NoTear]]: **true**, [[Block]]: *block*, [[ByteIndex]]: *byteIndex*, [[ElementSize]]: *elementSize*, [[Payload]]: *rawBytes*, [[ModifyOp]]: *op* }.
14. Append *rmwEvent* to *eventList*.
15. Append **Chosen Value Record** { [[Event]]: *rmwEvent*, [[ChosenValue]]: *rawBytesRead* } to *execution*. [[**ChosenValues**]].
16. Return **RawBytesToNumber**(*type*, *rawBytesRead*, *isLittleEndian*).

24.1.2 The ArrayBuffer Constructor

The **ArrayBuffer constructor**:

is the intrinsic object %ArrayBuffer%.

is the initial value of the **ArrayBuffer** property of the **global object**.

creates and initializes a new **ArrayBuffer** object when called as a **constructor**.

is not intended to be called as a function and will throw an exception when called in that manner.

is designed to be subclassable. It may be used as the value of an **extends** clause of a class definition. Subclass constructors that intend to inherit the specified **ArrayBuffer** behaviour must include a **super** call to the **ArrayBuffer constructor** to create and initialize subclass instances with the internal state necessary to support the **ArrayBuffer.prototype** built-in methods.

24.1.2.1 ArrayBuffer (*length*)

When the **ArrayBuffer** function is called with argument *length*, the following steps are taken:

1. If NewTarget is **undefined**, throw a **TypeError** exception.
2. Let *byteLength* be ? **ToIndex**(*length*).
3. Return ? **AllocateArrayBuffer**(NewTarget, *byteLength*).

24.1.3 Properties of the ArrayBuffer Constructor

The ArrayBuffer **constructor**:

has a **[[Prototype]]** internal slot whose value is the intrinsic object **%FunctionPrototype%**.
has the following properties:

24.1.3.1 ArrayBuffer.isView (*arg*)

The **isView** function takes one argument *arg*, and performs the following steps:

1. If **Type**(*arg*) is not Object, return **false**.
2. If *arg* has a **[[ViewedArrayBuffer]]** internal slot, return **true**.
3. Return **false**.

24.1.3.2 ArrayBuffer.prototype

The initial value of **ArrayBuffer.prototype** is the intrinsic object **%ArrayBufferPrototype%**.

This property has the attributes { **[[Writable]]**: **false**, **[[Enumerable]]**: **false**, **[[Configurable]]**: **false** }.

24.1.3.3 get ArrayBuffer [@@species]

ArrayBuffer[@@species] is an **accessor property** whose set accessor function is **undefined**. Its get accessor function performs the following steps:

1. Return the **this** value.

The value of the **name** property of this function is **"get [Symbol.species]"**.

NOTE

ArrayBuffer prototype methods normally use their **this** object's **constructor** to create a derived object. However, a subclass **constructor** may over-ride that default behaviour by redefining its **@@species** property.

24.1.4 Properties of the ArrayBuffer Prototype Object

The ArrayBuffer prototype object:

is the intrinsic object **%ArrayBufferPrototype%**.
has a **[[Prototype]]** internal slot whose value is the intrinsic object **%ObjectPrototype%**.
is an ordinary object.
does not have an **[[ArrayBufferData]]** or **[[ArrayBufferByteLength]]** internal slot.

24.1.4.1 get `ArrayBuffer.prototype.byteLength`

`ArrayBuffer.prototype.byteLength` is an [accessor property](#) whose set accessor function is **undefined**. Its get accessor function performs the following steps:

1. Let *O* be the **this** value.
2. If `Type(O)` is not Object, throw a **TypeError** exception.
3. If *O* does not have an `[[ArrayBufferData]]` internal slot, throw a **TypeError** exception.
4. If `IsSharedArrayBuffer(O)` is **true**, throw a **TypeError** exception.
5. If `IsDetachedBuffer(O)` is **true**, throw a **TypeError** exception.
6. Let *length* be *O*.`[[ArrayBufferByteLength]]`.
7. Return *length*.

24.1.4.2 `ArrayBuffer.prototype.constructor`

The initial value of `ArrayBuffer.prototype.constructor` is the intrinsic object `%ArrayBuffer%`.

24.1.4.3 `ArrayBuffer.prototype.slice ( start, end )`

The following steps are taken:

1. Let *O* be the **this** value.
2. If `Type(O)` is not Object, throw a **TypeError** exception.
3. If *O* does not have an `[[ArrayBufferData]]` internal slot, throw a **TypeError** exception.
4. If `IsSharedArrayBuffer(O)` is **true**, throw a **TypeError** exception.
5. If `IsDetachedBuffer(O)` is **true**, throw a **TypeError** exception.
6. Let *len* be *O*.`[[ArrayBufferByteLength]]`.
7. Let *relativeStart* be `? ToInteger(start)`.
8. If *relativeStart* < 0, let *first* be `max((len + relativeStart), 0)`; else let *first* be `min(relativeStart, len)`.
9. If *end* is **undefined**, let *relativeEnd* be *len*; else let *relativeEnd* be `? ToInteger(end)`.
10. If *relativeEnd* < 0, let *final* be `max((len + relativeEnd), 0)`; else let *final* be `min(relativeEnd, len)`.
11. Let *newLen* be `max(final - first, 0)`.
12. Let *ctor* be `? SpeciesConstructor(O, %ArrayBuffer%)`.
13. Let *new* be `? Construct(ctor, « newLen »)`.
14. If *new* does not have an `[[ArrayBufferData]]` internal slot, throw a **TypeError** exception.
15. If `IsSharedArrayBuffer(new)` is **true**, throw a **TypeError** exception.
16. If `IsDetachedBuffer(new)` is **true**, throw a **TypeError** exception.
17. If `SameValue(new, O)` is **true**, throw a **TypeError** exception.
18. If *new*.`[[ArrayBufferByteLength]]` < *newLen*, throw a **TypeError** exception.
19. NOTE: Side-effects of the above steps may have detached *O*.
20. If `IsDetachedBuffer(O)` is **true**, throw a **TypeError** exception.
21. Let *fromBuf* be *O*.`[[ArrayBufferData]].`
22. Let *toBuf* be *new*.`[[ArrayBufferData]].`
23. Perform `CopyDataBlockBytes(toBuf, 0, fromBuf, first, newLen)`.
24. Return *new*.

24.1.4.4 `ArrayBuffer.prototype [ @@toStringTag ]`

The initial value of the `@@toStringTag` property is the String value **"ArrayBuffer"**.

This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **true** }.

24.1.5 Properties of ArrayBuffer Instances

ArrayBuffer instances inherit properties from the ArrayBuffer prototype object. ArrayBuffer instances each have an `[[ArrayBufferData]]` internal slot, an `[[ArrayBufferByteLength]]` internal slot, and an `[[ArrayBufferDetachKey]]` internal slot.

ArrayBuffer instances whose `[[ArrayBufferData]]` is **null** are considered to be detached and all operators to access or modify data contained in the ArrayBuffer instance will fail.

ArrayBuffer instances whose `[[ArrayBufferDetachKey]]` is set to a value other than **undefined** need to have all [DetachArrayBuffer](#) calls passing that same "detach key" as an argument, otherwise a `TypeError` will result. This internal slot is only ever set by certain embedding environments, not by algorithms in this specification.

24.2 SharedArrayBuffer Objects

24.2.1 Abstract Operations for SharedArrayBuffer Objects

24.2.1.1 AllocateSharedArrayBuffer (*constructor*, *byteLength*)

The abstract operation `AllocateSharedArrayBuffer` with arguments *constructor* and *byteLength* is used to create a `SharedArrayBuffer` object. It performs the following steps:

1. Let *obj* be ? `OrdinaryCreateFromConstructor`(*constructor*, "%SharedArrayBufferPrototype%", « `[[ArrayBufferData]]`, `[[ArrayBufferByteLength]]` »).
2. **Assert**: *byteLength* is a nonnegative integer.
3. Let *block* be ? `CreateSharedByteDataBlock`(*byteLength*).
4. Set *obj*.`[[ArrayBufferData]]` to *block*.
5. Set *obj*.`[[ArrayBufferByteLength]]` to *byteLength*.
6. Return *obj*.

24.2.1.2 IsSharedArrayBuffer (*obj*)

`IsSharedArrayBuffer` tests whether an object is an `ArrayBuffer`, a `SharedArrayBuffer`, or a subtype of either. It performs the following steps:

1. **Assert**: `Type`(*obj*) is `Object` and it has an `[[ArrayBufferData]]` internal slot.
2. Let *bufferData* be *obj*.`[[ArrayBufferData]]`.
3. If *bufferData* is **null**, return **false**.
4. If *bufferData* is a `Data Block`, return **false**.
5. **Assert**: *bufferData* is a `Shared Data Block`.
6. Return **true**.

24.2.2 The SharedArrayBuffer Constructor

The `SharedArrayBuffer` [constructor](#):

is the intrinsic object `%SharedArrayBuffer%`.

is the initial value of the **SharedArrayBuffer** property of the [global object](#).

creates and initializes a new `SharedArrayBuffer` object when called as a [constructor](#).

is not intended to be called as a function and will throw an exception when called in that manner.

is designed to be subclassable. It may be used as the value of an **extends** clause of a class definition. Subclass constructors that intend to inherit the specified **SharedArrayBuffer** behaviour must include a **super** call to the **SharedArrayBuffer** [constructor](#) to create and initialize subclass instances with the internal state necessary to support the **SharedArrayBuffer.prototype** built-in methods.

NOTE

Unlike an **ArrayBuffer**, a **SharedArrayBuffer** cannot become detached, and its internal `[[ArrayBufferData]]` slot is never **null**.

24.2.2.1 SharedArrayBuffer ([*length*])

When the **SharedArrayBuffer** function is called with optional argument *length*, the following steps are taken:

1. If `NewTarget` is **undefined**, throw a **TypeError** exception.
2. Let *byteLength* be `? ToIndex(length)`.
3. Return `? AllocateSharedArrayBuffer(NewTarget, byteLength)`.

24.2.3 Properties of the SharedArrayBuffer Constructor

The `SharedArrayBuffer` [constructor](#):

has a `[[Prototype]]` internal slot whose value is the intrinsic object `%FunctionPrototype%`.

has the following properties:

24.2.3.1 SharedArrayBuffer.prototype

The initial value of **SharedArrayBuffer.prototype** is the intrinsic object `%SharedArrayBufferPrototype%`.

This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **false** }.

24.2.3.2 get SharedArrayBuffer [@@species]

SharedArrayBuffer [`@@species`] is an [accessor property](#) whose set accessor function is **undefined**. Its get accessor function performs the following steps:

1. Return the **this** value.

The value of the **name** property of this function is `"get [Symbol.species]"`.

24.2.4 Properties of the SharedArrayBuffer Prototype Object

The `SharedArrayBuffer` prototype object:

is the intrinsic object `%SharedArrayBufferPrototype%`.

has a `[[Prototype]]` internal slot whose value is the intrinsic object `%ObjectPrototype%`.

is an ordinary object.

does not have an `[[ArrayBufferData]]` or `[[ArrayBufferByteLength]]` internal slot.

24.2.4.1 `get SharedArrayBuffer.prototype.byteLength`

`SharedArrayBuffer.prototype.byteLength` is an [accessor property](#) whose set accessor function is **undefined**. Its get accessor function performs the following steps:

1. Let *O* be the **this** value.
2. If `Type(O)` is not Object, throw a **TypeError** exception.
3. If *O* does not have an `[[ArrayBufferData]]` internal slot, throw a **TypeError** exception.
4. If `IsSharedArrayBuffer(O)` is **false**, throw a **TypeError** exception.
5. Let *length* be *O*.`[[ArrayBufferByteLength]]`.
6. Return *length*.

24.2.4.2 `SharedArrayBuffer.prototype.constructor`

The initial value of `SharedArrayBuffer.prototype.constructor` is the intrinsic object `%SharedArrayBuffer%`.

24.2.4.3 `SharedArrayBuffer.prototype.slice ( start, end )`

The following steps are taken:

1. Let *O* be the **this** value.
2. If `Type(O)` is not Object, throw a **TypeError** exception.
3. If *O* does not have an `[[ArrayBufferData]]` internal slot, throw a **TypeError** exception.
4. If `IsSharedArrayBuffer(O)` is **false**, throw a **TypeError** exception.
5. Let *len* be *O*.`[[ArrayBufferByteLength]]`.
6. Let *relativeStart* be `? ToInteger(start)`.
7. If *relativeStart* < 0, let *first* be `max((len + relativeStart), 0)`; else let *first* be `min(relativeStart, len)`.
8. If *end* is **undefined**, let *relativeEnd* be *len*; else let *relativeEnd* be `? ToInteger(end)`.
9. If *relativeEnd* < 0, let *final* be `max((len + relativeEnd), 0)`; else let *final* be `min(relativeEnd, len)`.
10. Let *newLen* be `max(final - first, 0)`.
11. Let *ctor* be `? SpeciesConstructor(O, %SharedArrayBuffer%)`.
12. Let *new* be `? Construct(ctor, « newLen »)`.
13. If *new* does not have an `[[ArrayBufferData]]` internal slot, throw a **TypeError** exception.
14. If `IsSharedArrayBuffer(new)` is **false**, throw a **TypeError** exception.
15. If *new*.`[[ArrayBufferData]]` and *O*.`[[ArrayBufferData]]` are the same [Shared Data Block](#) values, throw a **TypeError** exception.
16. If *new*.`[[ArrayBufferByteLength]]` < *newLen*, throw a **TypeError** exception.
17. Let *fromBuf* be *O*.`[[ArrayBufferData]]`.
18. Let *toBuf* be *new*.`[[ArrayBufferData]]`.
19. Perform `CopyDataBlockBytes(toBuf, 0, fromBuf, first, newLen)`.
20. Return *new*.

24.2.4.4 `SharedArrayBuffer.prototype [ @@toStringTag ]`

The initial value of the `@@toStringTag` property is the String value **"SharedArrayBuffer"**.

This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **true** }.

24.2.5 Properties of SharedArrayBuffer Instances

SharedArrayBuffer instances inherit properties from the SharedArrayBuffer prototype object. SharedArrayBuffer instances each have an `[[ArrayBufferData]]` internal slot and an `[[ArrayBufferByteLength]]` internal slot.

NOTE

SharedArrayBuffer instances, unlike ArrayBuffer instances, are never detached.

24.3 DataView Objects

24.3.1 Abstract Operations For DataView Objects

24.3.1.1 GetViewValue (*view*, *requestIndex*, *isLittleEndian*, *type*)

The abstract operation GetViewValue with arguments *view*, *requestIndex*, *isLittleEndian*, and *type* is used by functions on DataView instances to retrieve values from the view's buffer. It performs the following steps:

1. If `Type(view)` is not Object, throw a **TypeError** exception.
2. If *view* does not have a `[[DataView]]` internal slot, throw a **TypeError** exception.
3. **Assert:** *view* has a `[[ViewedArrayBuffer]]` internal slot.
4. Let *getIndex* be ? `ToIndex(requestIndex)`.
5. Set *isLittleEndian* to `ToBoolean(isLittleEndian)`.
6. Let *buffer* be *view*.`[[ViewedArrayBuffer]]`.
7. If `IsDetachedBuffer(buffer)` is **true**, throw a **TypeError** exception.
8. Let *viewOffset* be *view*.`[[ByteOffset]]`.
9. Let *viewSize* be *view*.`[[ByteLength]]`.
10. Let *elementSize* be the Number value of the Element Size value specified in Table 59 for Element Type *type*.
11. If *getIndex* + *elementSize* > *viewSize*, throw a **RangeError** exception.
12. Let *bufferIndex* be *getIndex* + *viewOffset*.
13. Return `GetValueFromBuffer(buffer, bufferIndex, type, false, "Unordered", isLittleEndian)`.

24.3.1.2 SetViewValue (*view*, *requestIndex*, *isLittleEndian*, *type*, *value*)

The abstract operation SetViewValue with arguments *view*, *requestIndex*, *isLittleEndian*, *type*, and *value* is used by functions on DataView instances to store values into the view's buffer. It performs the following steps:

1. If `Type(view)` is not Object, throw a **TypeError** exception.
2. If *view* does not have a `[[DataView]]` internal slot, throw a **TypeError** exception.
3. **Assert:** *view* has a `[[ViewedArrayBuffer]]` internal slot.
4. Let *getIndex* be ? `ToIndex(requestIndex)`.
5. Let *numberValue* be ? `ToNumber(value)`.
6. Set *isLittleEndian* to `ToBoolean(isLittleEndian)`.
7. Let *buffer* be *view*.`[[ViewedArrayBuffer]]`.
8. If `IsDetachedBuffer(buffer)` is **true**, throw a **TypeError** exception.
9. Let *viewOffset* be *view*.`[[ByteOffset]]`.

10. Let *viewSize* be *view*.[[ByteLength]].
11. Let *elementSize* be the Number value of the Element Size value specified in Table 59 for Element Type *type*.
12. If *getIndex* + *elementSize* > *viewSize*, throw a **RangeError** exception.
13. Let *bufferIndex* be *getIndex* + *viewOffset*.
14. Return *SetValueInBuffer*(*buffer*, *bufferIndex*, *type*, *numberValue*, **false**, "Unordered", *isLittleEndian*).

24.3.2 The DataView Constructor

The DataView [constructor](#):

is the intrinsic object %DataView%.

is the initial value of the **DataView** property of the [global object](#).

creates and initializes a new DataView object when called as a [constructor](#).

is not intended to be called as a function and will throw an exception when called in that manner.

is designed to be subclassable. It may be used as the value of an **extends** clause of a class definition. Subclass constructors that intend to inherit the specified **DataView** behaviour must include a **super** call to the **DataView** [constructor](#) to create and initialize subclass instances with the internal state necessary to support the **DataView.prototype** built-in methods.

24.3.2.1 DataView (*buffer* [, *byteOffset* [, *byteLength*]])

When the **DataView** function is called with at least one argument *buffer*, the following steps are taken:

1. If NewTarget is **undefined**, throw a **TypeError** exception.
2. If *Type*(*buffer*) is not Object, throw a **TypeError** exception.
3. If *buffer* does not have an [[ArrayBufferData]] internal slot, throw a **TypeError** exception.
4. Let *offset* be ? *ToIndex*(*byteOffset*).
5. If *IsDetachedBuffer*(*buffer*) is **true**, throw a **TypeError** exception.
6. Let *bufferByteLength* be *buffer*.[[ArrayBufferByteLength]].
7. If *offset* > *bufferByteLength*, throw a **RangeError** exception.
8. If *byteLength* is either not present or **undefined**, then
 - a. Let *viewByteLength* be *bufferByteLength* - *offset*.
9. Else,
 - a. Let *viewByteLength* be ? *ToIndex*(*byteLength*).
 - b. If *offset* + *viewByteLength* > *bufferByteLength*, throw a **RangeError** exception.
10. Let *O* be ? *OrdinaryCreateFromConstructor*(NewTarget, "%DataViewPrototype%", « [[DataView]], [[ViewedArrayBuffer]], [[ByteLength]], [[ByteOffset]] »).
11. If *IsDetachedBuffer*(*buffer*) is **true**, throw a **TypeError** exception.
12. Set *O*.[[ViewedArrayBuffer]] to *buffer*.
13. Set *O*.[[ByteLength]] to *viewByteLength*.
14. Set *O*.[[ByteOffset]] to *offset*.
15. Return *O*.

24.3.3 Properties of the DataView Constructor

The DataView [constructor](#):

has a [[Prototype]] internal slot whose value is the intrinsic object %FunctionPrototype%.

has the following properties:

24.3.3.1 DataView.prototype

The initial value of **DataView.prototype** is the intrinsic object `%DataViewPrototype%`.

This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **false** }.

24.3.4 Properties of the DataView Prototype Object

The DataView prototype object:

is the intrinsic object `%DataViewPrototype%`.

has a `[[Prototype]]` internal slot whose value is the intrinsic object `%ObjectPrototype%`.

is an ordinary object.

does not have a `[[DataView]]`, `[[ViewedArrayBuffer]]`, `[[ByteLength]]`, or `[[ByteOffset]]` internal slot.

24.3.4.1 get DataView.prototype.buffer

DataView.prototype.buffer is an [accessor property](#) whose set accessor function is **undefined**. Its get accessor function performs the following steps:

1. Let *O* be the **this** value.
2. If `Type(O)` is not Object, throw a **TypeError** exception.
3. If *O* does not have a `[[DataView]]` internal slot, throw a **TypeError** exception.
4. **Assert**: *O* has a `[[ViewedArrayBuffer]]` internal slot.
5. Let *buffer* be *O*.`[[ViewedArrayBuffer]]`.
6. Return *buffer*.

24.3.4.2 get DataView.prototype.byteLength

DataView.prototype.byteLength is an [accessor property](#) whose set accessor function is **undefined**. Its get accessor function performs the following steps:

1. Let *O* be the **this** value.
2. If `Type(O)` is not Object, throw a **TypeError** exception.
3. If *O* does not have a `[[DataView]]` internal slot, throw a **TypeError** exception.
4. **Assert**: *O* has a `[[ViewedArrayBuffer]]` internal slot.
5. Let *buffer* be *O*.`[[ViewedArrayBuffer]]`.
6. If `IsDetachedBuffer(buffer)` is **true**, throw a **TypeError** exception.
7. Let *size* be *O*.`[[ByteLength]]`.
8. Return *size*.

24.3.4.3 get DataView.prototype.byteOffset

DataView.prototype.byteOffset is an [accessor property](#) whose set accessor function is **undefined**. Its get accessor function performs the following steps:

1. Let *O* be the **this** value.
2. If `Type(O)` is not Object, throw a **TypeError** exception.
3. If *O* does not have a `[[DataView]]` internal slot, throw a **TypeError** exception.
4. **Assert**: *O* has a `[[ViewedArrayBuffer]]` internal slot.
5. Let *buffer* be *O*.`[[ViewedArrayBuffer]]`.

6. If `IsDetachedBuffer(buffer)` is **true**, throw a **TypeError** exception.
7. Let *offset* be `O.[[ByteOffset]]`.
8. Return *offset*.

24.3.4.4 `DataView.prototype.constructor`

The initial value of `DataView.prototype.constructor` is the intrinsic object `%DataView%`.

24.3.4.5 `DataView.prototype.getFloat32 ( byteOffset [ , littleEndian ] )`

When the `getFloat32` method is called with argument *byteOffset* and optional argument *littleEndian*, the following steps are taken:

1. Let *v* be the **this** value.
2. If *littleEndian* is not present, set *littleEndian* to **false**.
3. Return ? `GetViewValue(v, byteOffset, littleEndian, "Float32")`.

24.3.4.6 `DataView.prototype.getFloat64 ( byteOffset [ , littleEndian ] )`

When the `getFloat64` method is called with argument *byteOffset* and optional argument *littleEndian*, the following steps are taken:

1. Let *v* be the **this** value.
2. If *littleEndian* is not present, set *littleEndian* to **false**.
3. Return ? `GetViewValue(v, byteOffset, littleEndian, "Float64")`.

24.3.4.7 `DataView.prototype.getInt8 ( byteOffset )`

When the `getInt8` method is called with argument *byteOffset*, the following steps are taken:

1. Let *v* be the **this** value.
2. Return ? `GetViewValue(v, byteOffset, true, "Int8")`.

24.3.4.8 `DataView.prototype.getInt16 ( byteOffset [ , littleEndian ] )`

When the `getInt16` method is called with argument *byteOffset* and optional argument *littleEndian*, the following steps are taken:

1. Let *v* be the **this** value.
2. If *littleEndian* is not present, set *littleEndian* to **false**.
3. Return ? `GetViewValue(v, byteOffset, littleEndian, "Int16")`.

24.3.4.9 `DataView.prototype.getInt32 ( byteOffset [ , littleEndian ] )`

When the `getInt32` method is called with argument *byteOffset* and optional argument *littleEndian*, the following steps are taken:

1. Let *v* be the **this** value.
2. If *littleEndian* is not present, set *littleEndian* to **false**.
3. Return ? `GetViewValue(v, byteOffset, littleEndian, "Int32")`.

24.3.4.10 DataView.prototype.getUint8 (*byteOffset*)

When the **getUint8** method is called with argument *byteOffset*, the following steps are taken:

1. Let *v* be the **this** value.
2. Return ? *GetViewValue*(*v*, *byteOffset*, **true**, "Uint8").

24.3.4.11 DataView.prototype.getUint16 (*byteOffset* [, *littleEndian*])

When the **getUint16** method is called with argument *byteOffset* and optional argument *littleEndian*, the following steps are taken:

1. Let *v* be the **this** value.
2. If *littleEndian* is not present, set *littleEndian* to **false**.
3. Return ? *GetViewValue*(*v*, *byteOffset*, *littleEndian*, "Uint16").

24.3.4.12 DataView.prototype.getUint32 (*byteOffset* [, *littleEndian*])

When the **getUint32** method is called with argument *byteOffset* and optional argument *littleEndian*, the following steps are taken:

1. Let *v* be the **this** value.
2. If *littleEndian* is not present, set *littleEndian* to **false**.
3. Return ? *GetViewValue*(*v*, *byteOffset*, *littleEndian*, "Uint32").

24.3.4.13 DataView.prototype.setFloat32 (*byteOffset*, *value* [, *littleEndian*])

When the **setFloat32** method is called with arguments *byteOffset* and *value* and optional argument *littleEndian*, the following steps are taken:

1. Let *v* be the **this** value.
2. If *littleEndian* is not present, set *littleEndian* to **false**.
3. Return ? *SetViewValue*(*v*, *byteOffset*, *littleEndian*, "Float32", *value*).

24.3.4.14 DataView.prototype.setFloat64 (*byteOffset*, *value* [, *littleEndian*])

When the **setFloat64** method is called with arguments *byteOffset* and *value* and optional argument *littleEndian*, the following steps are taken:

1. Let *v* be the **this** value.
2. If *littleEndian* is not present, set *littleEndian* to **false**.
3. Return ? *SetViewValue*(*v*, *byteOffset*, *littleEndian*, "Float64", *value*).

24.3.4.15 DataView.prototype.setInt8 (*byteOffset*, *value*)

When the **setInt8** method is called with arguments *byteOffset* and *value*, the following steps are taken:

1. Let *v* be the **this** value.
2. Return ? *SetViewValue*(*v*, *byteOffset*, **true**, "Int8", *value*).

24.3.4.16 DataView.prototype.setInt16 (*byteOffset*, *value* [, *littleEndian*])

When the **setInt16** method is called with arguments *byteOffset* and *value* and optional argument *littleEndian*, the following steps are taken:

1. Let *v* be the **this** value.
2. If *littleEndian* is not present, set *littleEndian* to **false**.
3. Return ? **SetViewValue**(*v*, *byteOffset*, *littleEndian*, "**Int16**", *value*).

24.3.4.17 DataView.prototype.setInt32 (*byteOffset*, *value* [, *littleEndian*])

When the **setInt32** method is called with arguments *byteOffset* and *value* and optional argument *littleEndian*, the following steps are taken:

1. Let *v* be the **this** value.
2. If *littleEndian* is not present, set *littleEndian* to **false**.
3. Return ? **SetViewValue**(*v*, *byteOffset*, *littleEndian*, "**Int32**", *value*).

24.3.4.18 DataView.prototype.setUint8 (*byteOffset*, *value*)

When the **setUint8** method is called with arguments *byteOffset* and *value*, the following steps are taken:

1. Let *v* be the **this** value.
2. Return ? **SetViewValue**(*v*, *byteOffset*, **true**, "**Uint8**", *value*).

24.3.4.19 DataView.prototype.setUint16 (*byteOffset*, *value* [, *littleEndian*])

When the **setUint16** method is called with arguments *byteOffset* and *value* and optional argument *littleEndian*, the following steps are taken:

1. Let *v* be the **this** value.
2. If *littleEndian* is not present, set *littleEndian* to **false**.
3. Return ? **SetViewValue**(*v*, *byteOffset*, *littleEndian*, "**Uint16**", *value*).

24.3.4.20 DataView.prototype.setUint32 (*byteOffset*, *value* [, *littleEndian*])

When the **setUint32** method is called with arguments *byteOffset* and *value* and optional argument *littleEndian*, the following steps are taken:

1. Let *v* be the **this** value.
2. If *littleEndian* is not present, set *littleEndian* to **false**.
3. Return ? **SetViewValue**(*v*, *byteOffset*, *littleEndian*, "**Uint32**", *value*).

24.3.4.21 DataView.prototype [@@toStringTag]

The initial value of the @@toStringTag property is the String value "**DataView**".

This property has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **true** }.

24.3.5 Properties of DataView Instances

DataView instances are ordinary objects that inherit properties from the DataView prototype object. DataView instances

each have `[[DataView]]`, `[[ViewedArrayBuffer]]`, `[[ByteLength]]`, and `[[ByteOffset]]` internal slots.

NOTE

The value of the `[[DataView]]` internal slot is not used within this specification. The simple presence of that internal slot is used within the specification to identify objects created using the **DataView** constructor.

24.4 The Atomics Object

The Atomics object:

is the intrinsic object `%Atomics%`.

is the initial value of the **Atomics** property of the [global object](#).

is an ordinary object.

has a `[[Prototype]]` internal slot whose value is the intrinsic object `%ObjectPrototype%`.

does not have a `[[Construct]]` internal method; it cannot be used as a [constructor](#) with the **new** operator.

does not have a `[[Call]]` internal method; it cannot be invoked as a function.

The Atomics object provides functions that operate indivisibly (atomically) on shared memory array cells as well as functions that let agents wait for and dispatch primitive events. When used with discipline, the Atomics functions allow multi-[agent](#) programs that communicate through shared memory to execute in a well-understood order even on parallel CPUs. The rules that govern shared-memory communication are provided by the [memory model](#), defined below.

NOTE

For informative guidelines for programming and implementing shared memory in ECMAScript, please see the notes at the end of the [memory model](#) section.

24.4.1 Abstract Operations for Atomics

24.4.1.1 ValidateSharedIntegerTypedArray (*typedArray* [, *onlyInt32*])

The abstract operation `ValidateSharedIntegerTypedArray` takes one argument *typedArray* and an optional Boolean *onlyInt32*. It performs the following steps:

1. If *onlyInt32* is not present, set *onlyInt32* to **false**.
2. If `Type(typedArray)` is not **Object**, throw a **TypeError** exception.
3. If *typedArray* does not have a `[[TypedArrayName]]` internal slot, throw a **TypeError** exception.
4. Let *typeName* be *typedArray*.`[[TypedArrayName]]`.
5. If *onlyInt32* is **true**, then
 - a. If *typeName* is not **"Int32Array"**, throw a **TypeError** exception.
6. Else,
 - a. If *typeName* is not **"Int8Array"**, **"Uint8Array"**, **"Int16Array"**, **"Uint16Array"**, **"Int32Array"**, or **"Uint32Array"**, throw a **TypeError** exception.
7. **Assert**: *typedArray* has a `[[ViewedArrayBuffer]]` internal slot.
8. Let *buffer* be *typedArray*.`[[ViewedArrayBuffer]]`.
9. If `IsSharedArrayBuffer(buffer)` is **false**, throw a **TypeError** exception.
10. Return *buffer*.

24.4.1.2 ValidateAtomicAccess (*typedArray*, *requestIndex*)

The abstract operation ValidateAtomicAccess takes two arguments, *typedArray* and *requestIndex*. It performs the following steps:

1. **Assert:** *typedArray* is an Object that has a `[[ViewedArrayBuffer]]` internal slot.
2. Let *accessIndex* be `? ToIndex(requestIndex)`.
3. Let *length* be *typedArray*.`[[ArrayLength]]`.
4. **Assert:** *accessIndex* ≥ 0 .
5. If *accessIndex* $\geq$ *length*, throw a **RangeError** exception.
6. Return *accessIndex*.

24.4.1.3 GetWaiterList (*block*, *i*)

A WaiterList is a semantic object that contains an ordered list of those agents that are waiting on a location (*block*, *i*) in shared memory; *block* is a **Shared Data Block** and *i* a byte offset into the memory of *block*.

The **agent cluster** has a store of WaiterList objects; the store is indexed by (*block*, *i*). WaiterLists are **agent**-independent: a lookup in the store of WaiterLists by (*block*, *i*) will result in the same WaiterList object in any **agent** in the **agent cluster**.

Operations on a WaiterList -- adding and removing waiting agents, traversing the list of agents, suspending and notifying agents on the list -- may only be performed by agents that have entered the WaiterList's critical section.

The abstract operation GetWaiterList takes two arguments, a **Shared Data Block** *block* and a nonnegative integer *i*. It performs the following steps:

1. **Assert:** *block* is a **Shared Data Block**.
2. **Assert:** *i* and *i* + 3 are valid byte offsets within the memory of *block*.
3. **Assert:** *i* is divisible by 4.
4. Return the **WaiterList** that is referenced by the pair (*block*, *i*).

24.4.1.4 EnterCriticalSection (*WL*)

The abstract operation EnterCriticalSection takes one argument, a **WaiterList** *WL*. It performs the following steps:

1. **Assert:** The calling **agent** is not in the critical section for any **WaiterList**.
2. Wait until no **agent** is in the critical section for *WL*, then enter the critical section for *WL* (without allowing any other **agent** to enter).

24.4.1.5 LeaveCriticalSection (*WL*)

The abstract operation LeaveCriticalSection takes one argument, a **WaiterList** *WL*. It performs the following steps:

1. **Assert:** The calling **agent** is in the critical section for *WL*.
2. Leave the critical section for *WL*.

24.4.1.6 AddWaiter (*WL*, *W*)

The abstract operation AddWaiter takes two arguments, a **WaiterList** *WL* and an **agent** signifier *W*. It performs the following steps:

1. **Assert:** The calling **agent** is in the critical section for **WL**.
2. **Assert:** **W** is not on the list of waiters in any **WaiterList**.
3. Add **W** to the end of the list of waiters in **WL**.

24.4.1.7 RemoveWaiter (**WL**, **W**)

The abstract operation RemoveWaiter takes two arguments, a **WaiterList WL** and an **agent** signifier **W**. It performs the following steps:

1. **Assert:** The calling **agent** is in the critical section for **WL**.
2. **Assert:** **W** is on the list of waiters in **WL**.
3. Remove **W** from the list of waiters in **WL**.

24.4.1.8 RemoveWaiters (**WL**, **c**)

The abstract operation RemoveWaiters takes two arguments, a **WaiterList WL** and nonnegative integer **c**. It performs the following steps:

1. **Assert:** The calling **agent** is in the critical section for **WL**.
2. Let **L** be a new empty **List**.
3. Let **S** be a reference to the list of waiters in **WL**.
4. Repeat, while **c** > 0 and **S** is not an empty **List**,
 - a. Let **W** be the first waiter in **S**.
 - b. Add **W** to the end of **L**.
 - c. Remove **W** from **S**.
 - d. Subtract 1 from **c**.
5. Return **L**.

24.4.1.9 Suspend (**WL**, **W**, **timeout**)

The abstract operation Suspend takes three arguments, a **WaiterList WL**, an **agent** signifier **W**, and a nonnegative, non-**NaN** Number **timeout**. It performs the following steps:

1. **Assert:** The calling **agent** is in the critical section for **WL**.
2. **Assert:** **W** is equal to **AgentSignifier()**.
3. **Assert:** **W** is on the list of waiters in **WL**.
4. **Assert:** **AgentCanSuspend()** is **true**.
5. Perform **LeaveCriticalSection(WL)** and suspend **W** for up to **timeout** milliseconds, performing the combined operation in such a way that a notification that arrives after the critical section is exited but before the suspension takes effect is not lost. **W** can notify either because the timeout expired or because it was notified explicitly by another **agent** calling **NotifyWaiter(WL, W)**, and not for any other reasons at all.
6. Perform **EnterCriticalSection(WL)**.
7. If **W** was notified explicitly by another **agent** calling **NotifyWaiter(WL, W)**, return **true**.
8. Return **false**.

24.4.1.10 NotifyWaiter (**WL**, **W**)

The abstract operation NotifyWaiter takes two arguments, a **WaiterList WL** and an **agent** signifier **W**. It performs the following steps:

1. **Assert**: The calling **agent** is in the critical section for **WL**.
2. **Assert**: **W** is on the list of waiters in **WL**.
3. Let **execution** be the **[[CandidateExecution]]** field of the **surrounding agent's Agent Record**.
4. Let **eventsRecord** be the **Agent Events Record** in **execution. [[EventsRecords]]** whose **[[AgentSignifier]]** is **AgentSignifier()**.
5. Let **agentSynchronizesWith** be **eventsRecord. [[AgentSynchronizesWith]]**.
6. Let **notifierEventList** be **eventsRecord. [[EventList]]**.
7. Let **waiterEventList** be the **[[EventList]]** field of the element in **execution. [[EventsRecords]]** whose **[[AgentSignifier]]** is **W**.
8. Let **notifyEvent** and **waitEvent** be new **Synchronize** events.
9. Append **notifyEvent** to **notifierEventList**.
10. Append **waitEvent** to **waiterEventList**.
11. Append (**notifyEvent**, **waitEvent**) to **agentSynchronizesWith**.
12. Notify the **agent W**.

NOTE

The embedding may delay notifying **W**, e.g. for resource management reasons, but **W** must eventually be notified in order to guarantee forward progress.

24.4.1.11 AtomicReadModifyWrite (**typedArray**, **index**, **value**, **op**)

The abstract operation AtomicReadModifyWrite takes four arguments, **typedArray**, **index**, **value**, and a pure combining operation **op**. The pure combining operation **op** takes two **List** of byte values arguments and returns a **List** of byte values. The operation atomically loads a value, combines it with another value, and stores the result of the combination. It returns the loaded value. It performs the following steps:

1. Let **buffer** be ? **ValidateSharedIntegerTypedArray(typedArray)**.
2. Let **i** be ? **ValidateAtomicAccess(typedArray, index)**.
3. Let **v** be ? **ToInteger(value)**.
4. Let **arrayTypeName** be **typedArray. [[TypedArrayName]]**.
5. Let **elementSize** be the Number value of the Element Size value specified in Table 59 for **arrayTypeName**.
6. Let **elementType** be the String value of the Element Type value in Table 59 for **arrayTypeName**.
7. Let **offset** be **typedArray. [[ByteOffset]]**.
8. Let **indexedPosition** be $(i \times \text{elementSize}) + \text{offset}$.
9. Return **GetModifySetValueInBuffer(buffer, indexedPosition, elementType, v, op)**.

24.4.1.12 AtomicLoad (**typedArray**, **index**)

The abstract operation AtomicLoad takes two arguments, **typedArray**, **index**. The operation atomically loads a value and returns the loaded value. It performs the following steps:

1. Let **buffer** be ? **ValidateSharedIntegerTypedArray(typedArray)**.
2. Let **i** be ? **ValidateAtomicAccess(typedArray, index)**.
3. Let **arrayTypeName** be **typedArray. [[TypedArrayName]]**.
4. Let **elementSize** be the Number value of the Element Size value specified in Table 59 for **arrayTypeName**.
5. Let **elementType** be the String value of the Element Type value in Table 59 for **arrayTypeName**.
6. Let **offset** be **typedArray. [[ByteOffset]]**.
7. Let **indexedPosition** be $(i \times \text{elementSize}) + \text{offset}$.

8. Return `GetValueFromBuffer(buffer, indexedPosition, elementType, true, "SeqCst")`.

24.4.2 `Atomics.add ( typedArray, index, value )`

Let **add** denote a semantic function of two `List` of byte values arguments that applies the addition operation to the Number values corresponding to the `List` of byte values arguments and returns a `List` of byte values corresponding to the result of that operation.

The following steps are taken:

1. Return ? `AtomicReadModifyWrite(typedArray, index, value, add)`.

24.4.3 `Atomics.and ( typedArray, index, value )`

Let **and** denote a semantic function of two `List` of byte values arguments that applies the bitwise-and operation element-wise to the two arguments and returns a `List` of byte values corresponding to the result of that operation.

The following steps are taken:

1. Return ? `AtomicReadModifyWrite(typedArray, index, value, and)`.

24.4.4 `Atomics.compareExchange ( typedArray, index, expectedValue, replacementValue )`

The following steps are taken:

1. Let *buffer* be ? `ValidateSharedIntegerTypedArray(typedArray)`.
2. Let *i* be ? `ValidateAtomicAccess(typedArray, index)`.
3. Let *expected* be ? `ToInteger(expectedValue)`.
4. Let *replacement* be ? `ToInteger(replacementValue)`.
5. Let *arrayTypeName* be `typedArray.[[TypedArrayName]]`.
6. Let *elementType* be the String value of the Element Type value in Table 59 for *arrayTypeName*.
7. Let *isLittleEndian* be the value of the `[[LittleEndian]]` field of the surrounding agent's Agent Record.
8. Let *expectedBytes* be `NumberToRawBytes(elementType, expected, isLittleEndian)`.
9. Let *elementSize* be the Number value of the Element Size value specified in Table 59 for *arrayTypeName*.
10. Let *offset* be `typedArray.[[ByteOffset]]`.
11. Let *indexedPosition* be $(i \times \text{elementSize}) + \text{offset}$.
12. Let **compareExchange** denote a semantic function of two `List` of byte values arguments that returns the second argument if the first argument is element-wise equal to *expectedBytes*.
13. Return `GetModifySetValueInBuffer(buffer, indexedPosition, elementType, replacement, compareExchange)`.

24.4.5 `Atomics.exchange ( typedArray, index, value )`

Let **second** denote a semantic function of two `List` of byte values arguments that returns its second argument.

The following steps are taken:

1. Return ? `AtomicReadModifyWrite(typedArray, index, value, second)`.

24.4.6 `Atomics.isLockFree ( size )`

The following steps are taken:

1. Let *n* be ? `ToInteger(size)`.
2. Let *AR* be the `Agent Record` of the `surrounding agent`.
3. If *n* equals 1, return *AR*.[`IsLockFree1`].
4. If *n* equals 2, return *AR*.[`IsLockFree2`].
5. If *n* equals 4, return **true**.
6. Return **false**.

NOTE

`Atoms.isLockFree()` is an optimization primitive. The intuition is that if the atomic step of an atomic primitive (`compareExchange`, `load`, `store`, `add`, `sub`, `and`, `or`, `xor`, or `exchange`) on a datum of size *n* bytes will be performed without the calling `agent` acquiring a lock outside the *n* bytes comprising the datum, then `Atoms.isLockFree(n)` will return **true**. High-performance algorithms will use `Atoms.isLockFree` to determine whether to use locks or atomic operations in critical sections. If an atomic primitive is not lock-free then it is often more efficient for an algorithm to provide its own locking.

`Atoms.isLockFree(4)` always returns **true** as that can be supported on all known relevant hardware. Being able to assume this will generally simplify programs.

24.4.7 `Atoms.load ( typedArray, index )`

The following steps are taken:

1. Return ? `AtomicLoad(typedArray, index)`.

24.4.8 `Atoms.or ( typedArray, index, value )`

Let **or** denote a semantic function of two `List` of byte values arguments that applies the bitwise-or operation element-wise to the two arguments and returns a `List` of byte values corresponding to the result of that operation.

The following steps are taken:

1. Return ? `AtomicReadModifyWrite(typedArray, index, value, or)`.

24.4.9 `Atoms.store ( typedArray, index, value )`

The following steps are taken:

1. Let *buffer* be ? `ValidateSharedIntegerTypedArray(typedArray)`.
2. Let *i* be ? `ValidateAtomicAccess(typedArray, index)`.
3. Let *v* be ? `ToInteger(value)`.
4. Let *arrayTypeName* be *typedArray*.[`TypedArrayName`].
5. Let *elementSize* be the Number value of the Element Size value specified in Table 59 for *arrayTypeName*.
6. Let *elementType* be the String value of the Element Type value in Table 59 for *arrayTypeName*.
7. Let *offset* be *typedArray*.[`ByteOffset`].
8. Let *indexedPosition* be $(i \times \text{elementSize}) + \text{offset}$.
9. Perform `SetValueInBuffer(buffer, indexedPosition, elementType, v, true, "SeqCst")`.
10. Return *v*.

24.4.10 **Atoms.sub** (*typedArray*, *index*, *value*)

Let **subtract** denote a semantic function of two **List** of byte values arguments that applies the subtraction operation to the Number values corresponding to the **List** of byte values arguments and returns a **List** of byte values corresponding to the result of that operation.

The following steps are taken:

1. Return ? **AtomicReadModifyWrite**(*typedArray*, *index*, *value*, **subtract**).

24.4.11 **Atoms.wait** (*typedArray*, *index*, *value*, *timeout*)

Atoms.wait puts the calling **agent** in a wait queue and puts it to sleep until it is notified or the sleep times out. The following steps are taken:

1. Let *buffer* be ? **ValidateSharedIntegerTypedArray**(*typedArray*, **true**).
2. Let *i* be ? **ValidateAtomicAccess**(*typedArray*, *index*).
3. Let *v* be ? **ToInt32**(*value*).
4. Let *q* be ? **ToNumber**(*timeout*).
5. If *q* is **NaN**, let *t* be $+\infty$, else let *t* be **max**(*q*, 0).
6. Let *B* be **AgentCanSuspend**(*agent*).
7. If *B* is **false**, throw a **TypeError** exception.
8. Let *block* be *buffer*.[[ArrayBufferData]].
9. Let *offset* be *typedArray*.[[ByteOffset]].
10. Let *indexedPosition* be $(i \times 4) + \text{offset}$.
11. Let *WL* be **GetWaiterList**(*block*, *indexedPosition*).
12. Perform **EnterCriticalSection**(*WL*).
13. Let *w* be ! **AtomicLoad**(*typedArray*, *i*).
14. If *v* is not equal to *w*, then
 - a. Perform **LeaveCriticalSection**(*WL*).
 - b. Return the String "**not-equal**".
15. Let *W* be **AgentSignifier**(*agent*).
16. Perform **AddWaiter**(*WL*, *W*).
17. Let *notified* be **Suspend**(*WL*, *W*, *t*).
18. If *notified* is **true**, then
 - a. **Assert**: *W* is not on the list of waiters in *WL*.
19. Else,
 - a. Perform **RemoveWaiter**(*WL*, *W*).
20. Perform **LeaveCriticalSection**(*WL*).
21. If *notified* is **true**, return the String "**ok**".
22. Return the String "**timed-out**".

24.4.12 **Atoms.notify** (*typedArray*, *index*, *count*)

Atoms.notify notifies some agents that are sleeping in the wait queue. The following steps are taken:

1. Let *buffer* be ? **ValidateSharedIntegerTypedArray**(*typedArray*, **true**).
2. Let *i* be ? **ValidateAtomicAccess**(*typedArray*, *index*).
3. If *count* is **undefined**, let *c* be $+\infty$.

4. Else,
 - a. Let *intCount* be ? **ToInteger**(*count*).
 - b. Let *c* be **max**(*intCount*, 0).
5. Let *block* be *buffer*.[[ArrayBufferData]].
6. Let *offset* be *typedArray*.[[ByteOffset]].
7. Let *indexedPosition* be $(i \times 4) + \textit{offset}$.
8. Let *WL* be **GetWaiterList**(*block*, *indexedPosition*).
9. Let *n* be 0.
10. Perform **EnterCriticalSection**(*WL*).
11. Let *S* be **RemoveWaiters**(*WL*, *c*).
12. Repeat, while *S* is not an empty **List**,
 - a. Let *W* be the first **agent** in *S*.
 - b. Remove *W* from the front of *S*.
 - c. Perform **NotifyWaiter**(*WL*, *W*).
 - d. Add 1 to *n*.
13. Perform **LeaveCriticalSection**(*WL*).
14. Return *n*.

24.4.13 **Atomics.xor** (*typedArray*, *index*, *value*)

Let **xor** denote a semantic function of two **List** of byte values arguments that applies the bitwise-xor operation element-wise to the two arguments and returns a **List** of byte values corresponding to the result of that operation.

The following steps are taken:

1. Return ? **AtomicReadModifyWrite**(*typedArray*, *index*, *value*, **xor**).

24.4.14 **Atomics** [@@toStringTag]

The initial value of the @@toStringTag property is the String value "**Atomics**".

This property has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **true** }.

24.5 The JSON Object

The JSON object:

is the intrinsic object %JSON%.

is the initial value of the **JSON** property of the **global object**.

is an ordinary object.

contains two functions, **parse** and **stringify**, that are used to parse and construct JSON texts.

has a [[Prototype]] internal slot whose value is the intrinsic object %ObjectPrototype%.

does not have a [[Construct]] internal method; it cannot be used as a **constructor** with the **new** operator.

does not have a [[Call]] internal method; it cannot be invoked as a function.

The JSON Data Interchange Format is defined in ECMA-404. The JSON interchange format used in this specification is exactly that described by ECMA-404. Conforming implementations of **JSON.parse** and **JSON.stringify** must support the exact interchange format described in the ECMA-404 specification without any deletions or extensions to the

format.

24.5.1 JSON.parse (*text* [, *reviver*])

The **parse** function parses a JSON text (a JSON-formatted String) and produces an ECMAScript value. The JSON format represents literals, arrays, and objects with a syntax similar to the syntax for ECMAScript literals, Array Initializers, and Object Initializers. After parsing, JSON objects are realized as ECMAScript objects. JSON arrays are realized as ECMAScript Array instances. JSON strings, numbers, booleans, and null are realized as ECMAScript Strings, Numbers, Booleans, and **null**.

The optional *reviver* parameter is a function that takes two parameters, *key* and *value*. It can filter and transform the results. It is called with each of the *key/value* pairs produced by the parse, and its return value is used instead of the original value. If it returns what it received, the structure is not modified. If it returns **undefined** then the property is deleted from the result.

1. Let *JText* be ? ToString(*text*).
2. Parse *JText* interpreted as UTF-16 encoded Unicode points (6.1.4) as a JSON text as specified in ECMA-404. Throw a **SyntaxError** exception if *JText* is not a valid JSON text as defined in that specification.
3. Let *scriptText* be the string-concatenation of " (", *JText*, and ") ;".
4. Let *completion* be the result of parsing and evaluating *scriptText* as if it was the source text of an ECMAScript Script. The extended PropertyDefinitionEvaluation semantics defined in B.3.1 must not be used during the evaluation.
5. Let *unfiltered* be *completion*.[[Value]].
6. **Assert**: *unfiltered* is either a String, Number, Boolean, Null, or an Object that is defined by either an *ArrayLiteral* or an *ObjectLiteral*.
7. If IsCallable(*reviver*) is **true**, then
 - a. Let *root* be ObjectCreate(%ObjectPrototype%).
 - b. Let *rootName* be the empty String.
 - c. Let *status* be CreateDataProperty(*root*, *rootName*, *unfiltered*).
 - d. **Assert**: *status* is **true**.
 - e. Return ? InternalizeJSONProperty(*root*, *rootName*).
8. Else,
 - a. Return *unfiltered*.

This function is the %JSONParse% intrinsic object.

The "length" property of the **parse** function is 2.

NOTE

Valid JSON text is a subset of the ECMAScript *PrimaryExpression* syntax as modified by Step 4 above. Step 2 verifies that *JText* conforms to that subset, and step 6 verifies that that parsing and evaluation returns a value of an appropriate type.

24.5.1.1 Runtime Semantics: InternalizeJSONProperty (*holder*, *name*)

The abstract operation InternalizeJSONProperty is a recursive abstract operation that takes two parameters: a *holder* object and the String *name* of a property in that object. InternalizeJSONProperty uses the value of *reviver* that was originally passed to the above parse function.

1. Let *val* be ? [Get](#)(*holder*, *name*).
2. If [Type](#)(*val*) is Object, then
 - a. Let *isArray* be ? [isArray](#)(*val*).
 - b. If *isArray* is **true**, then
 - i. Let *I* be 0.
 - ii. Let *len* be ? [ToLength](#)(? [Get](#)(*val*, "length")).
 - iii. Repeat, while *I* < *len*,
 1. Let *newElement* be ? [InternalizeJSONProperty](#)(*val*, ! [ToString](#)(*I*)).
 2. If *newElement* is **undefined**, then
 - a. Perform ? *val*.[[Delete]](! [ToString](#)(*I*)).
 3. Else,
 - a. Perform ? [CreateDataProperty](#)(*val*, ! [ToString](#)(*I*), *newElement*).
 - b. NOTE: This algorithm intentionally does not throw an exception if [CreateDataProperty](#) returns **false**.
 4. Add 1 to *I*.
 - c. Else,
 - i. Let *keys* be ? [EnumerableOwnPropertyNames](#)(*val*, "key").
 - ii. For each String *P* in *keys*, do
 1. Let *newElement* be ? [InternalizeJSONProperty](#)(*val*, *P*).
 2. If *newElement* is **undefined**, then
 - a. Perform ? *val*.[[Delete]](*P*).
 3. Else,
 - a. Perform ? [CreateDataProperty](#)(*val*, *P*, *newElement*).
 - b. NOTE: This algorithm intentionally does not throw an exception if [CreateDataProperty](#) returns **false**.
3. Return ? [Call](#)(*reviver*, *holder*, « *name*, *val* »).

It is not permitted for a conforming implementation of **JSON.parse** to extend the JSON grammars. If an implementation wishes to support a modified or extended JSON interchange format it must do so by defining a different parse function.

NOTE

In the case where there are duplicate name Strings within an object, lexically preceding values for the same key shall be overwritten.

24.5.2 JSON.stringify (*value* [, *replacer* [, *space*]])

The **stringify** function returns a String in UTF-16 encoded JSON format representing an ECMAScript value, or **undefined**. It can take three parameters. The *value* parameter is an ECMAScript value, which is usually an object or array, although it can also be a String, Boolean, Number or **null**. The optional *replacer* parameter is either a function that alters the way objects and arrays are stringified, or an array of Strings and Numbers that acts as an inclusion list for selecting the object properties that will be stringified. The optional *space* parameter is a String or Number that allows the result to have white space injected into it to improve human readability.

These are the steps in stringifying an object:

1. Let *stack* be a new empty [List](#).
2. Let *indent* be the empty String.

3. Let *PropertyList* and *ReplacerFunction* be **undefined**.
4. If *Type(replacer)* is Object, then
 - a. If *IsCallable(replacer)* is **true**, then
 - i. Set *ReplacerFunction* to *replacer*.
 - b. Else,
 - i. Let *isArray* be ? *IsArray(replacer)*.
 - ii. If *isArray* is **true**, then
 1. Set *PropertyList* to a new empty List.
 2. Let *len* be ? *ToLength(? Get(replacer, "length"))*.
 3. Let *k* be 0.
 4. Repeat, while *k* < *len*,
 - a. Let *v* be ? *Get(replacer, ! ToString(k))*.
 - b. Let *item* be **undefined**.
 - c. If *Type(v)* is String, set *item* to *v*.
 - d. Else if *Type(v)* is Number, set *item* to ! *ToString(v)*.
 - e. Else if *Type(v)* is Object, then
 - i. If *v* has a [[StringData]] or [[NumberData]] internal slot, set *item* to ? *ToString(v)*.
 - f. If *item* is not **undefined** and *item* is not currently an element of *PropertyList*, then
 - i. Append *item* to the end of *PropertyList*.
 - g. Increase *k* by 1.
5. If *Type(space)* is Object, then
 - a. If *space* has a [[NumberData]] internal slot, then
 - i. Set *space* to ? *ToNumber(space)*.
 - b. Else if *space* has a [[StringData]] internal slot, then
 - i. Set *space* to ? *ToString(space)*.
6. If *Type(space)* is Number, then
 - a. Set *space* to *min(10, ! ToInteger(space))*.
 - b. Let *gap* be the String value containing *space* occurrences of the code unit 0x0020 (SPACE). This will be the empty String if *space* is less than 1.
7. Else if *Type(space)* is String, then
 - a. If the length of *space* is 10 or less, let *gap* be *space*; otherwise let *gap* be the String value consisting of the first 10 code units of *space*.
8. Else,
 - a. Let *gap* be the empty String.
9. Let *wrapper* be *ObjectCreate(%ObjectPrototype%)*.
10. Let *status* be *CreateDataProperty(wrapper, the empty String, value)*.
11. **Assert**: *status* is **true**.
12. Return ? *SerializeJSONProperty*(the empty String, *wrapper*).

The "length" property of the **stringify** function is 3.

NOTE 1

JSON structures are allowed to be nested to any depth, but they must be acyclic. If *value* is or contains a cyclic structure, then the **stringify** function must throw a **TypeError** exception. This is an example of a value that cannot be stringified:

```
a = [];  
a[0] = a;  
my_text = JSON.stringify(a); // This must throw a TypeError.
```


NOTE 2

Symbolic primitive values are rendered as follows:

The **null** value is rendered in JSON text as the String **null**.

The **undefined** value is not rendered.

The **true** value is rendered in JSON text as the String **true**.

The **false** value is rendered in JSON text as the String **false**.

NOTE 3

String values are wrapped in QUOTATION MARK (") code units. The code units " and \ are escaped with \ prefixes. Control characters code units are replaced with escape sequences \uHHHH, or with the shorter forms, \b (BACKSPACE), \f (FORM FEED), \n (LINE FEED), \r (CARRIAGE RETURN), \t (CHARACTER TABULATION).

NOTE 4

Finite numbers are stringified as if by calling `ToString(number)`. NaN and Infinity regardless of sign are represented as the String **null**.

NOTE 5

Values that do not have a JSON representation (such as **undefined** and functions) do not produce a String. Instead they produce the **undefined** value. In arrays these values are represented as the String **null**. In objects an unrepresentable value causes the property to be excluded from stringification.

NOTE 6

An object is rendered as U+007B (LEFT CURLY BRACKET) followed by zero or more properties, separated with a U+002C (COMMA), closed with a U+007D (RIGHT CURLY BRACKET). A property is a quoted String representing the key or [property name](#), a U+003A (COLON), and then the stringified property value. An array is rendered as an opening U+005B (LEFT SQUARE BRACKET) followed by zero or more values, separated with a U+002C (COMMA), closed with a U+005D (RIGHT SQUARE BRACKET).

24.5.2.1 Runtime Semantics: SerializeJSONProperty (*key*, *holder*)

The abstract operation `SerializeJSONProperty` with arguments *key*, and *holder* has access to [ReplacerFunction](#) from the invocation of the **stringify** method. Its algorithm is as follows:

1. Let *value* be ? `Get(holder, key)`.
2. If `Type(value)` is Object, then
 - a. Let *toJSON* be ? `Get(value, "toJSON")`.
 - b. If `IsCallable(toJSON)` is **true**, then
 - i. Set *value* to ? `Call(toJSON, value, « key »)`.
3. If [ReplacerFunction](#) is not **undefined**, then
 - a. Set *value* to ? `Call(ReplacerFunction, holder, « key, value »)`.
4. If `Type(value)` is Object, then
 - a. If *value* has a `[[NumberData]]` internal slot, then
 - i. Set *value* to ? `ToNumber(value)`.
 - b. Else if *value* has a `[[StringData]]` internal slot, then
 - i. Set *value* to ? `ToString(value)`.
 - c. Else if *value* has a `[[BooleanData]]` internal slot, then
 - i. Set *value* to *value*.`[[BooleanData]]`.
5. If *value* is **null**, return **"null"**.
6. If *value* is **true**, return **"true"**.

7. If *value* is **false**, return **"false"**.
8. If **Type**(*value*) is String, return **QuoteJSONString**(*value*).
9. If **Type**(*value*) is Number, then
 - a. If *value* is finite, return **! ToString**(*value*).
 - b. Return **"null"**.
10. If **Type**(*value*) is Object and **IsCallable**(*value*) is **false**, then
 - a. Let *isArray* be **? IsArray**(*value*).
 - b. If *isArray* is **true**, return **? SerializeJSONArray**(*value*).
 - c. Return **? SerializeJSONObject**(*value*).
11. Return **undefined**.

24.5.2.2 Runtime Semantics: QuoteJSONString (*value*)

The abstract operation QuoteJSONString with argument *value* wraps a String value in QUOTATION MARK code units and escapes certain other code units within it.

This operation interprets a String value as a sequence of UTF-16 encoded code points, as described in 6.1.4.

1. Let *product* be the String value consisting solely of the code unit 0x0022 (QUOTATION MARK).
2. Let *cpList* be a List containing in order the code points of *value* when interpreted as a sequence of UTF-16 encoded code points as described in 6.1.4.
3. For each code point *C* in *cpList*, do
 - a. If *C* is listed in the Code Point column of Table 62, then
 - i. Set *product* to the string-concatenation of *product* and the Escape Sequence for *C* as specified in Table 62.
 - b. Else if *C* has a numeric value less than 0x0020 (SPACE), or if *C* has the same numeric value as a leading surrogate or trailing surrogate, then
 - i. Let *unit* be the code unit whose numeric value is that of *C*.
 - ii. Set *product* to the string-concatenation of *product* and **UnicodeEscape**(*unit*).
 - c. Else,
 - i. Set *product* to the string-concatenation of *product* and the **UTF16Encoding** of *C*.
4. Set *product* to the string-concatenation of *product* and the code unit 0x0022 (QUOTATION MARK).
5. Return *product*.

Table 62: JSON Single Character Escape Sequences

Code Point	Unicode Character Name	Escape Sequence
U+0008	BACKSPACE	\b
U+0009	CHARACTER TABULATION	\t
U+000A	LINE FEED (LF)	\n
U+000C	FORM FEED (FF)	\f
U+000D	CARRIAGE RETURN (CR)	\r
U+0022	QUOTATION MARK	\"
U+005C	REVERSE SOLIDUS	\\

24.5.2.3 Runtime Semantics: UnicodeEscape (*C*)

The abstract operation UnicodeEscape takes a code unit argument *C* and represents it as a Unicode escape sequence.

1. Let *n* be the numeric value of *C*.
2. **Assert**: $n \leq 0xFFFF$.
3. Return the **string-concatenation** of:
 - the code unit 0x005C (REVERSE SOLIDUS)
 - "u"
 - the String representation of *n*, formatted as a four-digit lowercase hexadecimal number, padded to the left with zeroes if necessary

24.5.2.4 Runtime Semantics: SerializeJSONObject (*value*)

The abstract operation SerializeJSONObject with argument *value* serializes an object. It has access to the *stack*, *indent*, *gap*, and *PropertyList* values of the current invocation of the **stringify** method.

1. If *stack* contains *value*, throw a **TypeError** exception because the structure is cyclical.
2. Append *value* to *stack*.
3. Let *stepback* be *indent*.
4. Set *indent* to the **string-concatenation** of *indent* and *gap*.
5. If *PropertyList* is not **undefined**, then
 - a. Let *K* be *PropertyList*.
6. Else,
 - a. Let *K* be ? **EnumerableOwnPropertyNames**(*value*, "key").
7. Let *partial* be a new empty **List**.
8. For each element *P* of *K*, do
 - a. Let *strP* be ? **SerializeJSONProperty**(*P*, *value*).
 - b. If *strP* is not **undefined**, then
 - i. Let *member* be **QuoteJSONString**(*P*).
 - ii. Set *member* to the **string-concatenation** of *member* and " : ".
 - iii. If *gap* is not the empty String, then
 1. Set *member* to the **string-concatenation** of *member* and the code unit 0x0020 (SPACE).
 - iv. Set *member* to the **string-concatenation** of *member* and *strP*.
 - v. Append *member* to *partial*.
9. If *partial* is empty, then
 - a. Let *final* be "{ }".
10. Else,
 - a. If *gap* is the empty String, then
 - i. Let *properties* be the String value formed by concatenating all the element Strings of *partial* with each adjacent pair of Strings separated with the code unit 0x002C (COMMA). A comma is not inserted either before the first String or after the last String.
 - ii. Let *final* be the **string-concatenation** of "{ ", *properties*, and " }".
 - b. Else *gap* is not the empty String,
 - i. Let *separator* be the **string-concatenation** of the code unit 0x002C (COMMA), the code unit 0x000A (LINE FEED), and *indent*.
 - ii. Let *properties* be the String value formed by concatenating all the element Strings of *partial* with each adjacent pair of Strings separated with *separator*. The *separator* String is not inserted either before the first String or after the last String.

- iii. Let *final* be the **string-concatenation** of "{", the code unit 0x000A (LINE FEED), *indent*, *properties*, the code unit 0x000A (LINE FEED), *stepback*, and "}".
11. Remove the last element of *stack*.
12. Set *indent* to *stepback*.
13. Return *final*.

24.5.2.5 Runtime Semantics: SerializeJSONArray (*value*)

The abstract operation SerializeJSONArray with argument *value* serializes an array. It has access to the *stack*, *indent*, and *gap* values of the current invocation of the **stringify** method.

1. If *stack* contains *value*, throw a **TypeError** exception because the structure is cyclical.
2. Append *value* to *stack*.
3. Let *stepback* be *indent*.
4. Set *indent* to the **string-concatenation** of *indent* and *gap*.
5. Let *partial* be a new empty List.
6. Let *len* be ? **ToLength**(? **Get**(*value*, "length")).
7. Let *index* be 0.
8. Repeat, while *index* < *len*
 - a. Let *strP* be ? **SerializeJSONProperty**(! **ToString**(*index*), *value*).
 - b. If *strP* is **undefined**, then
 - i. Append "**null**" to *partial*.
 - c. Else,
 - i. Append *strP* to *partial*.
 - d. Increment *index* by 1.
9. If *partial* is empty, then
 - a. Let *final* be "[]".
10. Else,
 - a. If *gap* is the empty String, then
 - i. Let *properties* be the String value formed by concatenating all the element Strings of *partial* with each adjacent pair of Strings separated with the code unit 0x002C (COMMA). A comma is not inserted either before the first String or after the last String.
 - ii. Let *final* be the **string-concatenation** of "[", *properties*, and "]".
 - b. Else,
 - i. Let *separator* be the **string-concatenation** of the code unit 0x002C (COMMA), the code unit 0x000A (LINE FEED), and *indent*.
 - ii. Let *properties* be the String value formed by concatenating all the element Strings of *partial* with each adjacent pair of Strings separated with *separator*. The *separator* String is not inserted either before the first String or after the last String.
 - iii. Let *final* be the **string-concatenation** of "[", the code unit 0x000A (LINE FEED), *indent*, *properties*, the code unit 0x000A (LINE FEED), *stepback*, and "]".
11. Remove the last element of *stack*.
12. Set *indent* to *stepback*.
13. Return *final*.

NOTE

The representation of arrays includes only the elements between zero and **array.length** - 1 inclusive. Properties whose keys are not **array indexes** are excluded from the stringification. An array is stringified as an opening LEFT SQUARE BRACKET, elements separated by COMMA, and a closing RIGHT SQUARE BRACKET.

24.5.3 JSON [@@toStringTag]

The initial value of the @@toStringTag property is the String value "JSON".

This property has the attributes { [[Writable]]: false, [[Enumerable]]: false, [[Configurable]]: true }.

25 Control Abstraction Objects

25.1 Iteration

25.1.1 Common Iteration Interfaces

An interface is a set of property keys whose associated values match a specific specification. Any object that provides all the properties as described by an interface's specification *conforms* to that interface. An interface is not represented by a distinct object. There may be many separately implemented objects that conform to any interface. An individual object may conform to multiple interfaces.

25.1.1.1 The *Iterable* Interface

The *Iterable* interface includes the property described in Table 63:

Table 63: *Iterable* Interface Required Properties

Property	Value	Requirements
@@iterator	A function that returns an <i>Iterator</i> object.	The returned object must conform to the <i>Iterator</i> interface.

25.1.1.2 The *Iterator* Interface

An object that implements the *Iterator* interface must include the property in Table 64. Such objects may also implement the properties in Table 65.

Table 64: *Iterator* Interface Required Properties

Property	Value	Requirements
next	A function that returns an <i>IteratorResult</i> object.	The returned object must conform to the <i>IteratorResult</i> interface. If a previous call to the next method of an <i>Iterator</i> has returned an <i>IteratorResult</i> object whose done property is true , then all subsequent calls to the next method of that object should also return an <i>IteratorResult</i> object whose done property is true . However, this requirement is not enforced.

NOTE 1
Arguments may be passed to the next function but their interpretation and validity is dependent upon the target *Iterator*.

The for-of statement and other common users of *Iterators* do not pass any arguments, so *Iterator* objects that expect to be used in such a manner must be prepared to deal with being called with no arguments.

Table 65: *Iterator* Interface Optional Properties

Property	Value	Requirements
return	A function that returns an <i>IteratorResult</i> object.	The returned object must conform to the <i>IteratorResult</i> interface. Invoking this method notifies the <i>Iterator</i> object that the caller does not intend to make any more next method calls to the <i>Iterator</i> . The returned <i>IteratorResult</i> object will typically have a done property whose value is true , and a value property with the value passed as the argument of the return method. However, this requirement is not enforced.
throw	A function that returns an <i>IteratorResult</i> object.	The returned object must conform to the <i>IteratorResult</i> interface. Invoking this method notifies the <i>Iterator</i> object that the caller has detected an error condition. The argument may be used to identify the error condition and typically will be an exception object. A typical response is to throw the value passed as the argument. If the method does not throw , the returned <i>IteratorResult</i> object will typically have a done property whose value is true .

NOTE 2
Typically callers of these methods should check for their existence before invoking them. Certain ECMAScript language features including **for-of**, **yield***, and array destructuring call these methods after performing an existence check. Most ECMAScript library functions that accept *Iterable* objects as arguments also conditionally call them.

25.1.1.3 The *AsyncIterable* Interface

The *AsyncIterable* interface includes the properties described in Table 66:

Table 66: *AsyncIterable* Interface Required Properties

Property	Value	Requirements
@@asyncIterator	A function that returns an <i>AsyncIterator</i> object.	The returned object must conform to the <i>AsyncIterator</i> interface.

25.1.1.4 The *AsyncIterator* Interface

An object that implements the *AsyncIterator* interface must include the properties in Table 67. Such objects may also implement the properties in Table 68.

Table 67: *AsyncIterator* Interface Required Properties

Property	Value	Requirements
----------	-------	--------------

next	A function that returns a promise for an <i>IteratorResult</i> object.	The returned promise, when fulfilled, must fulfill with an object which conforms to the <i>IteratorResult</i> interface. If a previous call to the next method of an <i>AsyncIterator</i> has returned a promise for an <i>IteratorResult</i> object whose done property is true, then all subsequent calls to the next method of that object should also return a promise for an <i>IteratorResult</i> object whose done property is true. However, this requirement is not enforced. Additionally, the <i>IteratorResult</i> object that serves as a fulfillment value should have a value property whose value is not a promise (or "thenable"). However, this requirement is also not enforced.
-------------	--	--

NOTE 1

Arguments may be passed to the next function but their interpretation and validity is dependent upon the target *AsyncIterator*. The **for-await-of** statement and other common users of *AsyncIterators* do not pass any arguments, so *AsyncIterator* objects that expect to be used in such a manner must be prepared to deal with being called with no arguments.

Table 68: *AsyncIterator* Interface Optional Properties

Property	Value	Requirements
return	A function that returns a promise for an <i>IteratorResult</i> object.	The returned promise, when fulfilled, must fulfill with an object which conforms to the <i>IteratorResult</i> interface. Invoking this method notifies the <i>AsyncIterator</i> object that the caller does not intend to make any more next method calls to the <i>AsyncIterator</i>. The returned promise will fulfill with an <i>IteratorResult</i> object which will typically have a done property whose value is true, and a value property with the value passed as the argument of the return method. However, this requirement is not enforced. Additionally, the <i>IteratorResult</i> object that serves as a fulfillment value should have a value property whose value is not a promise (or "thenable"). If the argument value is used in the typical manner, then if it is a rejected promise, a promise rejected with the same reason should be returned; if it is a fulfilled promise, then its fulfillment value should be used as the value property of the returned promise's <i>IteratorResult</i> object fulfillment value. However, these requirements are also not enforced.

throw	A function that returns a promise for an <i>IteratorResult</i> object.	The returned promise, when fulfilled, must fulfill with an object which conforms to the <i>IteratorResult</i> interface. Invoking this method notifies the <i>AsyncIterator</i> object that the caller has detected an error condition. The argument may be used to identify the error condition and typically will be an exception object. A typical response is to return a rejected promise which rejects with the value passed as the argument. If the returned promise is fulfilled, the <i>IteratorResult</i> fulfillment value will typically have a done property whose value is true. Additionally, it should have a value property whose value is not a promise (or "thenable"), but this requirement is not enforced.
--------------	--	---

NOTE 2

Typically callers of these methods should check for their existence before invoking them. Certain ECMAScript language features including **for-await-of** and **yield*** call these methods after performing an existence check.

25.1.1.5 The *IteratorResult* Interface

The *IteratorResult* interface includes the properties listed in [Table 69](#):

Table 69: *IteratorResult* Interface Properties

Property	Value	Requirements
done	Either true or false .	This is the result status of an <i>iterator</i> next method call. If the end of the iterator was reached done is true . If the end was not reached done is false and a value is available. If a done property (either own or inherited) does not exist, it is consider to have the value false .
value	Any ECMAScript language value.	If done is false , this is the current iteration element value. If done is true , this is the return value of the iterator, if it supplied one. If the iterator does not have a return value, value is undefined . In that case, the value property may be absent from the conforming object if it does not inherit an explicit value property.

25.1.2 The %IteratorPrototype% Object

The %IteratorPrototype% object:

has a [[Prototype]] internal slot whose value is the intrinsic object [%ObjectPrototype%](#).
is an ordinary object.

NOTE

All objects defined in this specification that implement the Iterator interface also inherit from %IteratorPrototype%. ECMAScript code may also define objects that inherit from %IteratorPrototype%.The %IteratorPrototype% object provides a place where additional methods that are applicable to all iterator objects may be added.

The following expression is one way that ECMAScript code can access the %IteratorPrototype% object:

```
Object.getPrototypeOf(Object.getPrototypeOf([Symbol.iterator]))
```


25.1.2.1 %IteratorPrototype% [@@iterator] ()

The following steps are taken:

1. Return the **this** value.

The value of the **name** property of this function is "[Symbol.iterator]".

25.1.3 The %AsyncIteratorPrototype% Object

The %AsyncIteratorPrototype% object:

has a [[Prototype]] internal slot whose value is the intrinsic object %ObjectPrototype%.
is an ordinary object.

NOTE

All objects defined in this specification that implement the AsyncIterator interface also inherit from %AsyncIteratorPrototype%. ECMAScript code may also define objects that inherit from %AsyncIteratorPrototype%. The %AsyncIteratorPrototype% object provides a place where additional methods that are applicable to all async iterator objects may be added.

25.1.3.1 %AsyncIteratorPrototype% [@@asyncIterator] ()

The following steps are taken:

1. Return the **this** value.

The value of the **name** property of this function is "[Symbol.asyncIterator]".

25.1.4 Async-from-Sync Iterator Objects

An Async-from-Sync Iterator object is an async iterator that adapts a specific synchronous iterator. There is not a named [constructor](#) for Async-from-Sync Iterator objects. Instead, Async-from-Sync iterator objects are created by the [CreateAsyncFromSyncIterator](#) abstract operation as needed.

25.1.4.1 CreateAsyncFromSyncIterator (*syncIteratorRecord*)

The abstract operation CreateAsyncFromSyncIterator is used to create an async iterator [Record](#) from a synchronous iterator [Record](#). It performs the following steps:

1. Let *asyncIterator* be ! [ObjectCreate](#)(%AsyncFromSyncIteratorPrototype%, « [[SyncIteratorRecord]] »).
2. Set *asyncIterator*.[[SyncIteratorRecord]] to *syncIteratorRecord*.
3. Return ? [GetIterator](#)(*asyncIterator*, async).

25.1.4.2 The %AsyncFromSyncIteratorPrototype% Object

The %AsyncFromSyncIteratorPrototype% object:

has properties that are inherited by all Async-from-Sync Iterator Objects.

is an ordinary object.

has a `[[Prototype]]` internal slot whose value is the intrinsic object `%AsyncIteratorPrototype%`.

has the following properties:

25.1.4.2.1 `%AsyncFromSyncIteratorPrototype%.next ( value )`

1. Let *O* be the **this** value.
2. Let *promiseCapability* be ! `NewPromiseCapability(%Promise%)`.
3. If `Type(O)` is not Object, or if *O* does not have a `[[SyncIteratorRecord]]` internal slot, then
 - a. Let *invalidIteratorError* be a newly created **TypeError** object.
 - b. Perform ! `Call(promiseCapability.[[Reject]], undefined, « invalidIteratorError »)`.
 - c. Return *promiseCapability*.`[[Promise]]`.
4. Let *syncIteratorRecord* be *O*.`[[SyncIteratorRecord]]`.
5. Let *result* be `IteratorNext(syncIteratorRecord, value)`.
6. `IfAbruptRejectPromise(result, promiseCapability)`.
7. Return ! `AsyncFromSyncIteratorContinuation(result, promiseCapability)`.

25.1.4.2.2 `%AsyncFromSyncIteratorPrototype%.return ( value )`

1. Let *O* be the **this** value.
2. Let *promiseCapability* be ! `NewPromiseCapability(%Promise%)`.
3. If `Type(O)` is not Object, or if *O* does not have a `[[SyncIteratorRecord]]` internal slot, then
 - a. Let *invalidIteratorError* be a newly created **TypeError** object.
 - b. Perform ! `Call(promiseCapability.[[Reject]], undefined, « invalidIteratorError »)`.
 - c. Return *promiseCapability*.`[[Promise]]`.
4. Let *syncIterator* be *O*.`[[SyncIteratorRecord]].[[Iterator]]`.
5. Let *return* be `GetMethod(syncIterator, "return")`.
6. `IfAbruptRejectPromise(return, promiseCapability)`.
7. If *return* is **undefined**, then
 - a. Let *iterResult* be ! `CreateIterResultObject(value, true)`.
 - b. Perform ! `Call(promiseCapability.[[Resolve]], undefined, « iterResult »)`.
 - c. Return *promiseCapability*.`[[Promise]]`.
8. Let *result* be `Call(return, syncIterator, « value »)`.
9. `IfAbruptRejectPromise(result, promiseCapability)`.
10. If `Type(result)` is not Object, then
 - a. Perform ! `Call(promiseCapability.[[Reject]], undefined, « a newly created TypeError object »)`.
 - b. Return *promiseCapability*.`[[Promise]]`.
11. Return ! `AsyncFromSyncIteratorContinuation(result, promiseCapability)`.

25.1.4.2.3 `%AsyncFromSyncIteratorPrototype%.throw ( value )`

1. Let *O* be the **this** value.
2. Let *promiseCapability* be ! `NewPromiseCapability(%Promise%)`.
3. If `Type(O)` is not Object, or if *O* does not have a `[[SyncIteratorRecord]]` internal slot, then
 - a. Let *invalidIteratorError* be a newly created **TypeError** object.
 - b. Perform ! `Call(promiseCapability.[[Reject]], undefined, « invalidIteratorError »)`.
 - c. Return *promiseCapability*.`[[Promise]]`.
4. Let *syncIterator* be *O*.`[[SyncIteratorRecord]].[[Iterator]]`.

5. Let *throw* be `GetMethod(syncIterator, "throw")`.
6. `IfAbruptRejectPromise(throw, promiseCapability)`.
7. If *throw* is **undefined**, then
 - a. Perform ! `Call(promiseCapability.[[Reject]], undefined, « value »)`.
 - b. Return `promiseCapability.[[Promise]]`.
8. Let *result* be `Call(throw, syncIterator, « value »)`.
9. `IfAbruptRejectPromise(result, promiseCapability)`.
10. If `Type(result)` is not Object, then
 - a. Perform ! `Call(promiseCapability.[[Reject]], undefined, « a newly created TypeError object »)`.
 - b. Return `promiseCapability.[[Promise]]`.
11. Return ! `AsyncFromSyncIteratorContinuation(result, promiseCapability)`.

25.1.4.2.4 %AsyncFromSyncIteratorPrototype% [@@toStringTag]

The initial value of the @@toStringTag property is the String value **"Async-from-Sync Iterator"**.

This property has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **true** }.

25.1.4.2.5 Async-from-Sync Iterator Value Unwrap Functions

An async-from-sync iterator value unwrap function is an anonymous built-in function that is used by methods of `%AsyncFromSyncIteratorPrototype%` when processing the **value** field of an *IteratorResult* object, in order to wait for its value if it is a promise and re-package the result in a new "unwrapped" *IteratorResult* object. Each async iterator value unwrap function has a [[Done]] internal slot.

When an async-from-sync iterator value unwrap function is called with argument *value*, the following steps are taken:

1. Let *F* be the **active function object**.
2. Return ! `CreateIterResultObject(value, F.[[Done]])`.

25.1.4.3 Properties of Async-from-Sync Iterator Instances

Async-from-Sync Iterator instances are ordinary objects that inherit properties from the `%AsyncFromSyncIteratorPrototype%` intrinsic object. Async-from-Sync Iterator instances are initially created with the internal slots listed in Table 70.

Table 70: Internal Slots of Async-from-Sync Iterator Instances

Internal Slot	Description
[[SyncIteratorRecord]]	A Record , of the type returned by <code>GetIterator</code> , representing the original synchronous iterator which is being adapted.

25.1.4.4 AsyncFromSyncIteratorContinuation (*result*, *promiseCapability*)

1. Let *done* be `IteratorComplete(result)`.
2. `IfAbruptRejectPromise(done, promiseCapability)`.
3. Let *value* be `IteratorValue(result)`.
4. `IfAbruptRejectPromise(value, promiseCapability)`.
5. Let *valueWrapper* be ? `PromiseResolve(%Promise%, « value »)`.
6. Let *steps* be the algorithm steps defined in **Async-from-Sync Iterator Value Unwrap Functions**.

7. Let *onFulfilled* be `CreateBuiltinFunction(steps, « [[Done]] »)`.
8. Set *onFulfilled*.[[Done]] to *done*.
9. Perform ! `PerformPromiseThen(valueWrapper, onFulfilled, undefined, promiseCapability)`.
10. Return *promiseCapability*.[[Promise]].

25.2 GeneratorFunction Objects

GeneratorFunction objects are functions that are usually created by evaluating *GeneratorDeclarations*, *GeneratorExpressions*, and *GeneratorMethods*. They may also be created by calling the `%GeneratorFunction%` intrinsic.

Figure 5 (Informative): Generator Objects Relationships

25.2.1 The GeneratorFunction Constructor

The GeneratorFunction `constructor`:

is the intrinsic object `%GeneratorFunction%`.

creates and initializes a new GeneratorFunction object when called as a function rather than as a `constructor`. Thus the function call **GeneratorFunction (...)** is equivalent to the object creation expression

new GeneratorFunction (...) with the same arguments.

is designed to be subclassable. It may be used as the value of an **extends** clause of a class definition. Subclass constructors that intend to inherit the specified **GeneratorFunction** behaviour must include a **super** call to the **GeneratorFunction constructor** to create and initialize subclass instances with the internal slots necessary for built-in GeneratorFunction behaviour. All ECMAScript syntactic forms for defining generator function objects create direct instances of **GeneratorFunction**. There is no syntactic means to create instances of **GeneratorFunction** subclasses.

25.2.1.1 GeneratorFunction (*p1*, *p2*, ... , *pn*, *body*)

The last argument specifies the body (executable code) of a generator function; any preceding arguments specify formal parameters.

When the **GeneratorFunction** function is called with some arguments *p1*, *p2*, ... , *pn*, *body* (where *n* might be 0, that is, there are no “*p*” arguments, and where *body* might also not be provided), the following steps are taken:

1. Let *C* be the `active function object`.
2. Let *args* be the `argumentsList` that was passed to this function by [[Call]] or [[Construct]].
3. Return ? `CreateDynamicFunction(C, NewTarget, "generator", args)`.

NOTE

See NOTE for 19.2.1.1.

25.2.2 Properties of the GeneratorFunction Constructor

The GeneratorFunction `constructor`:

is a standard built-in [function object](#) that inherits from the **Function** constructor.
has a `[[Prototype]]` internal slot whose value is the intrinsic object `%Function%`.
has a **name** property whose value is **"GeneratorFunction"**.
has the following properties:

25.2.2.1 **GeneratorFunction.length**

This is a [data property](#) with a value of 1. This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **true** }.

25.2.2.2 **GeneratorFunction.prototype**

The initial value of **GeneratorFunction.prototype** is the intrinsic object `%Generator%`.

This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **false** }.

25.2.3 Properties of the GeneratorFunction Prototype Object

The GeneratorFunction prototype object:

is an ordinary object.
is not a [function object](#) and does not have an `[[ECMAScriptCode]]` internal slot or any other of the internal slots listed in [Table 27](#) or [Table 71](#).
is the value of the **prototype** property of the intrinsic object `%GeneratorFunction%`.
is the intrinsic object `%Generator%` (see [Figure 2](#)).
has a `[[Prototype]]` internal slot whose value is the intrinsic object `%FunctionPrototype%`.

25.2.3.1 **GeneratorFunction.prototype.constructor**

The initial value of **GeneratorFunction.prototype.constructor** is the intrinsic object `%GeneratorFunction%`.

This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **true** }.

25.2.3.2 **GeneratorFunction.prototype.prototype**

The value of **GeneratorFunction.prototype.prototype** is the `%GeneratorPrototype%` intrinsic object.

This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **true** }.

25.2.3.3 **GeneratorFunction.prototype [@@toStringTag]**

The initial value of the `@@toStringTag` property is the String value **"GeneratorFunction"**.

This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **true** }.

25.2.4 GeneratorFunction Instances

Every GeneratorFunction instance is an ECMAScript [function object](#) and has the internal slots listed in [Table 27](#). The value of the `[[FunctionKind]]` internal slot for all such instances is **"generator"**.

Each `GeneratorFunction` instance has the following own properties:

25.2.4.1 `length`

The specification for the "**`length`**" property of Function instances given in 19.2.4.1 also applies to `GeneratorFunction` instances.

25.2.4.2 `name`

The specification for the **`name`** property of Function instances given in 19.2.4.2 also applies to `GeneratorFunction` instances.

25.2.4.3 `prototype`

Whenever a `GeneratorFunction` instance is created another ordinary object is also created and is the initial value of the generator function's **`prototype`** property. The value of the prototype property is used to initialize the `[[Prototype]]` internal slot of a newly created Generator object when the generator [function object](#) is invoked using `[[Call]]`.

This property has the attributes { `[[Writable]]`: **true**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **false** }.

NOTE

Unlike Function instances, the object that is the value of the a `GeneratorFunction`'s **`prototype`** property does not have a **`constructor`** property whose value is the `GeneratorFunction` instance.

25.3 AsyncGeneratorFunction Objects

`AsyncGeneratorFunction` objects are functions that are usually created by evaluating *AsyncGeneratorDeclaration*, *AsyncGeneratorExpression*, and *AsyncGeneratorMethod* syntactic productions. They may also be created by calling the `%AsyncGeneratorFunction%` intrinsic.

25.3.1 The AsyncGeneratorFunction Constructor

The `AsyncGeneratorFunction` [constructor](#):

is the intrinsic object `%AsyncGeneratorFunction%`.

creates and initializes a new `AsyncGeneratorFunction` object when called as a function rather than as a [constructor](#).

Thus the function call **`AsyncGeneratorFunction (...)`** is equivalent to the object creation expression **`new AsyncGeneratorFunction (...)`** with the same arguments.

is designed to be subclassable. It may be used as the value of an **`extends`** clause of a class definition. Subclass constructors that intend to inherit the specified **`AsyncGeneratorFunction`** behaviour must include a **`super`** call to the **`AsyncGeneratorFunction`** [constructor](#) to create and initialize subclass instances with the internal slots necessary for built-in `AsyncGeneratorFunction` behaviour. All ECMAScript syntactic forms for defining async generator function objects create direct instances of **`AsyncGeneratorFunction`**. There is no syntactic means to create instances of **`AsyncGeneratorFunction`** subclasses.

25.3.1.1 `AsyncGeneratorFunction ( p1, p2, ..., pn, body )`

The last argument specifies the body (executable code) of an async generator function; any preceding arguments specify formal parameters.

When the **AsyncGeneratorFunction** function is called with some arguments *p1*, *p2*, ... , *pn*, *body* (where *n* might be 0, that is, there are no "*p*" arguments, and where *body* might also not be provided), the following steps are taken:

1. Let *C* be the [active function object](#).
2. Let *args* be the [argumentsList](#) that was passed to this function by `[[Call]]` or `[[Construct]]`.
3. Return ? [CreateDynamicFunction](#)(*C*, NewTarget, "**async generator**", *args*).

NOTE

See NOTE for [19.2.1.1](#).

25.3.2 Properties of the AsyncGeneratorFunction Constructor

The AsyncGeneratorFunction [constructor](#):

is a standard built-in [function object](#) that inherits from the **Function** [constructor](#).

has a `[[Prototype]]` internal slot whose value is the intrinsic object `%Function%`.

has a **name** property whose value is "**AsyncGeneratorFunction**".

has the following properties:

25.3.2.1 AsyncGeneratorFunction.length

This is a [data property](#) with a value of 1. This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **true** }.

25.3.2.2 AsyncGeneratorFunction.prototype

The initial value of **AsyncGeneratorFunction.prototype** is the intrinsic object `%AsyncGenerator%`.

This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **false** }.

25.3.3 Properties of the AsyncGeneratorFunction Prototype Object

The AsyncGeneratorFunction prototype object:

is an ordinary object.

is not a [function object](#) and does not have an `[[ECMAScriptCode]]` internal slot or any other of the internal slots listed in [Table 27](#) or [Table 72](#).

is the value of the **prototype** property of the intrinsic object `%AsyncGeneratorFunction%`.

is the intrinsic object `%AsyncGenerator%`.

has a `[[Prototype]]` internal slot whose value is the intrinsic object `%FunctionPrototype%`.

25.3.3.1 AsyncGeneratorFunction.prototype.constructor

The initial value of **AsyncGeneratorFunction.prototype.constructor** is the intrinsic object `%AsyncGeneratorFunction%`.

This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **true** }.

25.3.3.2 AsyncGeneratorFunction.prototype.prototype

The value of `AsyncGeneratorFunction.prototype.prototype` is the `%AsyncGeneratorPrototype%` intrinsic object.

This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **true** }.

25.3.3.3 AsyncGeneratorFunction.prototype [@@toStringTag]

The initial value of the `@@toStringTag` property is the String value `"AsyncGeneratorFunction"`.

This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **true** }.

25.3.4 AsyncGeneratorFunction Instances

Every AsyncGeneratorFunction instance is an ECMAScript [function object](#) and has the internal slots listed in [Table 27](#). The value of the `[[FunctionKind]]` internal slot for all such instances is `"generator"`.

Each AsyncGeneratorFunction instance has the following own properties:

25.3.4.1 length

The value of the `"length"` property is an integer that indicates the typical number of arguments expected by the AsyncGeneratorFunction. However, the language permits the function to be invoked with some other number of arguments. The behaviour of an AsyncGeneratorFunction when invoked on a number of arguments other than the number specified by its `"length"` property depends on the function.

This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **true** }.

25.3.4.2 name

The specification for the `name` property of Function instances given in [19.2.4.2](#) also applies to AsyncGeneratorFunction instances.

25.3.4.3 prototype

Whenever an AsyncGeneratorFunction instance is created another ordinary object is also created and is the initial value of the async generator function's `prototype` property. The value of the prototype property is used to initialize the `[[Prototype]]` internal slot of a newly created AsyncGenerator object when the generator [function object](#) is invoked using `[[Call]]`.

This property has the attributes { `[[Writable]]`: **true**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **false** }.

NOTE

Unlike function instances, the object that is the value of the an AsyncGeneratorFunction's `prototype` property does not have a `constructor` property whose value is the AsyncGeneratorFunction instance.

25.4 Generator Objects

A Generator object is an instance of a generator function and conforms to both the *Iterator* and *Iterable* interfaces.

Generator instances directly inherit properties from the object that is the value of the **prototype** property of the Generator function that created the instance. Generator instances indirectly inherit properties from the Generator Prototype intrinsic, [%GeneratorPrototype%](#).

25.4.1 Properties of the Generator Prototype Object

The Generator prototype object:

- is the intrinsic object [%GeneratorPrototype%](#).

- is the initial value of the **prototype** property of the intrinsic object [%Generator%](#) (the `GeneratorFunction.prototype`).

- is an ordinary object.

- is not a Generator instance and does not have a `[[GeneratorState]]` internal slot.

- has a `[[Prototype]]` internal slot whose value is the intrinsic object [%IteratorPrototype%](#).

- has properties that are indirectly inherited by all Generator instances.

25.4.1.1 `Generator.prototype.constructor`

The initial value of **`Generator.prototype.constructor`** is the intrinsic object [%Generator%](#).

This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **true** }.

25.4.1.2 `Generator.prototype.next ( value )`

The **next** method performs the following steps:

1. Let *g* be the **this** value.
2. Return ? [GeneratorResume](#)(*g*, *value*).

25.4.1.3 `Generator.prototype.return ( value )`

The **return** method performs the following steps:

1. Let *g* be the **this** value.
2. Let *C* be [Completion](#) { `[[Type]]`: **return**, `[[Value]]`: *value*, `[[Target]]`: **empty** }.
3. Return ? [GeneratorResumeAbrupt](#)(*g*, *C*).

25.4.1.4 `Generator.prototype.throw ( exception )`

The **throw** method performs the following steps:

1. Let *g* be the **this** value.
2. Let *C* be [ThrowCompletion](#)(*exception*).
3. Return ? [GeneratorResumeAbrupt](#)(*g*, *C*).

25.4.1.5 `Generator.prototype [ @@toStringTag ]`

The initial value of the @@toStringTag property is the String value "**Generator**".

This property has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **true** }.

25.4.2 Properties of Generator Instances

Generator instances are initially created with the internal slots described in Table 71.

Table 71: Internal Slots of Generator Instances

Internal Slot	Description
[[GeneratorState]]	The current execution state of the generator. The possible values are: undefined , " suspendedStart ", " suspendedYield ", " executing ", and " completed ".
[[GeneratorContext]]	The execution context that is used when executing the code of this generator.

25.4.3 Generator Abstract Operations

25.4.3.1 GeneratorStart (*generator*, *generatorBody*)

The abstract operation GeneratorStart with arguments *generator* and *generatorBody* performs the following steps:

1. **Assert**: The value of *generator*.[[GeneratorState]] is **undefined**.
2. Let *genContext* be the [running execution context](#).
3. Set the Generator component of *genContext* to *generator*.
4. Set the code evaluation state of *genContext* such that when evaluation is resumed for that [execution context](#) the following steps will be performed:
 - a. Let *result* be the result of evaluating *generatorBody*.
 - b. **Assert**: If we return here, the generator either threw an exception or performed either an implicit or explicit return.
 - c. Remove *genContext* from the [execution context stack](#) and restore the [execution context](#) that is at the top of the [execution context stack](#) as the [running execution context](#).
 - d. Set *generator*.[[GeneratorState]] to "**completed**".
 - e. Once a generator enters the "**completed**" state it never leaves it and its associated [execution context](#) is never resumed. Any execution state associated with *generator* can be discarded at this point.
 - f. If *result*.[[Type]] is normal, let *resultValue* be **undefined**.
 - g. Else if *result*.[[Type]] is return, let *resultValue* be *result*.[[Value]].
 - h. Else,
 - i. **Assert**: *result*.[[Type]] is throw.
 - ii. Return [Completion](#)(*result*).
 - i. Return [CreateIterResultObject](#)(*resultValue*, **true**).
5. Set *generator*.[[GeneratorContext]] to *genContext*.
6. Set *generator*.[[GeneratorState]] to "**suspendedStart**".
7. Return [NormalCompletion](#)(**undefined**).

25.4.3.2 GeneratorValidate (*generator*)

The abstract operation GeneratorValidate with argument *generator* performs the following steps:

1. If `Type(generator)` is not `Object`, throw a **`TypeError`** exception.
2. If `generator` does not have a `[[GeneratorState]]` internal slot, throw a **`TypeError`** exception.
3. **Assert:** `generator` also has a `[[GeneratorContext]]` internal slot.
4. Let `state` be `generator.[[GeneratorState]]`.
5. If `state` is **"executing"**, throw a **`TypeError`** exception.
6. Return `state`.

25.4.3.3 GeneratorResume (*generator*, *value*)

The abstract operation `GeneratorResume` with arguments *generator* and *value* performs the following steps:

1. Let `state` be ? `GeneratorValidate(generator)`.
2. If `state` is **"completed"**, return `CreateIterResultObject(undefined, true)`.
3. **Assert:** `state` is either **"suspendedStart"** or **"suspendedYield"**.
4. Let `genContext` be `generator.[[GeneratorContext]]`.
5. Let `methodContext` be the `running execution context`.
6. `Suspend methodContext`.
7. Set `generator.[[GeneratorState]]` to **"executing"**.
8. Push `genContext` onto the `execution context stack`; `genContext` is now the `running execution context`.
9. Resume the suspended evaluation of `genContext` using `NormalCompletion(value)` as the result of the operation that suspended it. Let `result` be the value returned by the resumed computation.
10. **Assert:** When we return here, `genContext` has already been removed from the `execution context stack` and `methodContext` is the currently `running execution context`.
11. Return `Completion(result)`.

25.4.3.4 GeneratorResumeAbrupt (*generator*, *abruptCompletion*)

The abstract operation `GeneratorResumeAbrupt` with arguments *generator* and *abruptCompletion* performs the following steps:

1. Let `state` be ? `GeneratorValidate(generator)`.
2. If `state` is **"suspendedStart"**, then
 - a. Set `generator.[[GeneratorState]]` to **"completed"**.
 - b. Once a generator enters the **"completed"** state it never leaves it and its associated `execution context` is never resumed. Any execution state associated with `generator` can be discarded at this point.
 - c. Set `state` to **"completed"**.
3. If `state` is **"completed"**, then
 - a. If `abruptCompletion.[[Type]]` is `return`, then
 - i. Return `CreateIterResultObject(abruptCompletion.[[Value]], true)`.
 - b. Return `Completion(abruptCompletion)`.
4. **Assert:** `state` is **"suspendedYield"**.
5. Let `genContext` be `generator.[[GeneratorContext]]`.
6. Let `methodContext` be the `running execution context`.
7. `Suspend methodContext`.
8. Set `generator.[[GeneratorState]]` to **"executing"**.
9. Push `genContext` onto the `execution context stack`; `genContext` is now the `running execution context`.
10. Resume the suspended evaluation of `genContext` using `abruptCompletion` as the result of the operation that suspended it. Let `result` be the completion record returned by the resumed computation.
11. **Assert:** When we return here, `genContext` has already been removed from the `execution context stack` and

methodContext is the currently [running execution context](#).

12. Return [Completion](#)(*result*).

25.4.3.5 GetGeneratorKind ()

1. Let *genContext* be the [running execution context](#).
2. If *genContext* does not have a Generator component, return non-generator.
3. Let *generator* be the Generator component of *genContext*.
4. If *generator* has an `[[AsyncGeneratorState]]` internal slot, return `async`.
5. Else, return `sync`.

25.4.3.6 GeneratorYield (*iterNextObj*)

The abstract operation GeneratorYield with argument *iterNextObj* performs the following steps:

1. **Assert:** *iterNextObj* is an Object that implements the *IteratorResult* interface.
2. Let *genContext* be the [running execution context](#).
3. **Assert:** *genContext* is the [execution context](#) of a generator.
4. Let *generator* be the value of the Generator component of *genContext*.
5. **Assert:** `GetGeneratorKind()` is `sync`.
6. Set *generator*.`[[GeneratorState]]` to `"suspendedYield"`.
7. Remove *genContext* from the [execution context stack](#) and restore the [execution context](#) that is at the top of the [execution context stack](#) as the [running execution context](#).
8. Set the code evaluation state of *genContext* such that when evaluation is resumed with a [Completion](#) *resumptionValue* the following steps will be performed:
 - a. Return *resumptionValue*.
 - b. **NOTE:** This returns to the evaluation of the *YieldExpression* that originally called this abstract operation.
9. Return [NormalCompletion](#)(*iterNextObj*).
10. **NOTE:** This returns to the evaluation of the operation that had most previously resumed evaluation of *genContext*.

25.5 AsyncGenerator Objects

An AsyncGenerator object is an instance of an async generator function and conforms to both the AsyncIterator and AsyncIterable interfaces.

AsyncGenerator instances directly inherit properties from the object that is the value of the **prototype** property of the AsyncGenerator function that created the instance. AsyncGenerator instances indirectly inherit properties from the AsyncGenerator Prototype intrinsic, `%AsyncGeneratorPrototype%`.

25.5.1 Properties of the AsyncGenerator Prototype Object

The AsyncGenerator prototype object:

is the intrinsic object `%AsyncGeneratorPrototype%`.

is the initial value of the **prototype** property of the intrinsic object `%AsyncGenerator%` (the `AsyncGeneratorFunction.prototype`).

is an ordinary object.

is not an AsyncGenerator instance and does not have an `[[AsyncGeneratorState]]` internal slot.

has a `[[Prototype]]` internal slot whose value is the intrinsic object `%AsyncIteratorPrototype%`.

has properties that are indirectly inherited by all AsyncGenerator instances.

25.5.1.1 AsyncGenerator.prototype.constructor

The initial value of **AsyncGenerator.prototype.constructor** is the intrinsic object **%AsyncGenerator%**.

This property has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **true** }.

25.5.1.2 AsyncGenerator.prototype.next (*value*)

1. Let *generator* be the **this** value.
2. Let *completion* be **NormalCompletion**(*value*).
3. Return ! **AsyncGeneratorEnqueue**(*generator*, *completion*).

25.5.1.3 AsyncGenerator.prototype.return (*value*)

1. Let *generator* be the **this** value.
2. Let *completion* be **Completion** { [[Type]]: **return**, [[Value]]: *value*, [[Target]]: **empty** }.
3. Return ! **AsyncGeneratorEnqueue**(*generator*, *completion*).

25.5.1.4 AsyncGenerator.prototype.throw (*exception*)

1. Let *generator* be the **this** value.
2. Let *completion* be **ThrowCompletion**(*exception*).
3. Return ! **AsyncGeneratorEnqueue**(*generator*, *completion*).

25.5.1.5 AsyncGenerator.prototype [@@toStringTag]

The initial value of the @@toStringTag property is the String value **"AsyncGenerator"**.

This property has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **true** }.

25.5.2 Properties of AsyncGenerator Instances

AsyncGenerator instances are initially created with the internal slots described below:

Table 72: Internal Slots of AsyncGenerator Instances

Internal Slot	Description
[[AsyncGeneratorState]]	The current execution state of the async generator. The possible values are: undefined , "suspendedStart" , "suspendedYield" , "executing" , "awaiting-return" , and "completed" .
[[AsyncGeneratorContext]]	The execution context that is used when executing the code of this async generator.
[[AsyncGeneratorQueue]]	A List of AsyncGeneratorRequest records which represent requests to resume the async generator.

25.5.3 AsyncGenerator Abstract Operations

25.5.3.1 AsyncGeneratorRequest Records

The AsyncGeneratorRequest is a [Record](#) value used to store information about how an async generator should be resumed and contains capabilities for fulfilling or rejecting the corresponding promise.

They have the following fields:

Table 73: AsyncGeneratorRequest [Record](#) Fields

Field Name	Value	Meaning
[[Completion]]	A Completion record	The completion which should be used to resume the async generator.
[[Capability]]	A PromiseCapability record	The promise capabilities associated with this request.

25.5.3.2 AsyncGeneratorStart (*generator*, *generatorBody*)

1. [Assert](#): *generator* is an AsyncGenerator instance.
2. [Assert](#): *generator*.[[AsyncGeneratorState]] is **undefined**.
3. Let *genContext* be the [running execution context](#).
4. Set the Generator component of *genContext* to *generator*.
5. Set the code evaluation state of *genContext* such that when evaluation is resumed for that [execution context](#) the following steps will be performed:
 - a. Let *result* be the result of evaluating *generatorBody*.
 - b. [Assert](#): If we return here, the async generator either threw an exception or performed either an implicit or explicit return.
 - c. Remove *genContext* from the [execution context stack](#) and restore the [execution context](#) that is at the top of the [execution context stack](#) as the [running execution context](#).
 - d. Set *generator*.[[AsyncGeneratorState]] to **"completed"**.
 - e. If *result* is a normal completion, let *resultValue* be **undefined**.
 - f. Else,
 - i. Let *resultValue* be *result*.[[Value]].
 - ii. If *result*.[[Type]] is not **return**, then
 1. Return ! [AsyncGeneratorReject](#)(*generator*, *resultValue*).
 - g. Return ! [AsyncGeneratorResolve](#)(*generator*, *resultValue*, **true**).
6. Set *generator*.[[AsyncGeneratorContext]] to *genContext*.
7. Set *generator*.[[AsyncGeneratorState]] to **"suspendedStart"**.
8. Set *generator*.[[AsyncGeneratorQueue]] to a new empty [List](#).
9. Return **undefined**.

25.5.3.3 AsyncGeneratorResolve (*generator*, *value*, *done*)

1. [Assert](#): *generator* is an AsyncGenerator instance.
2. Let *queue* be *generator*.[[AsyncGeneratorQueue]].
3. [Assert](#): *queue* is not an empty [List](#).
4. Remove the first element from *queue* and let *next* be the value of that element.
5. Let *promiseCapability* be *next*.[[Capability]].
6. Let *iteratorResult* be ! [CreateIterResultObject](#)(*value*, *done*).
7. Perform ! [Call](#)(*promiseCapability*.[[Resolve]], **undefined**, « *iteratorResult* »).
8. Perform ! [AsyncGeneratorResumeNext](#)(*generator*).

9. Return **undefined**.

25.5.3.4 AsyncGeneratorReject (*generator*, *exception*)

1. **Assert**: *generator* is an AsyncGenerator instance.
2. Let *queue* be *generator*.[[AsyncGeneratorQueue]].
3. **Assert**: *queue* is not an empty List.
4. Remove the first element from *queue* and let *next* be the value of that element.
5. Let *promiseCapability* be *next*.[[Capability]].
6. Perform ! Call(*promiseCapability*.[[Reject]], **undefined**, « *exception* »).
7. Perform ! AsyncGeneratorResumeNext(*generator*).
8. Return **undefined**.

25.5.3.5 AsyncGeneratorResumeNext (*generator*)

1. **Assert**: *generator* is an AsyncGenerator instance.
2. Let *state* be *generator*.[[AsyncGeneratorState]].
3. **Assert**: *state* is not "executing".
4. If *state* is "awaiting-return", return **undefined**.
5. Let *queue* be *generator*.[[AsyncGeneratorQueue]].
6. If *queue* is an empty List, return **undefined**.
7. Let *next* be the value of the first element of *queue*.
8. **Assert**: *next* is an AsyncGeneratorRequest record.
9. Let *completion* be *next*.[[Completion]].
10. If *completion* is an abrupt completion, then
 - a. If *state* is "suspendedStart", then
 - i. Set *generator*.[[AsyncGeneratorState]] to "completed".
 - ii. Set *state* to "completed".
 - b. If *state* is "completed", then
 - i. If *completion*.[[Type]] is return, then
 1. Set *generator*.[[AsyncGeneratorState]] to "awaiting-return".
 2. Let *promise* be ? PromiseResolve(%Promise%, « *completion*.[[Value]] »).
 3. Let *stepsFulfilled* be the algorithm steps defined in AsyncGeneratorResumeNext Return Processor Fulfilled Functions.
 4. Let *onFulfilled* be CreateBuiltinFunction(*stepsFulfilled*, « [[Generator]] »).
 5. Set *onFulfilled*.[[Generator]] to *generator*.
 6. Let *stepsRejected* be the algorithm steps defined in AsyncGeneratorResumeNext Return Processor Rejected Functions.
 7. Let *onRejected* be CreateBuiltinFunction(*stepsRejected*, « [[Generator]] »).
 8. Set *onRejected*.[[Generator]] to *generator*.
 9. Perform ! PerformPromiseThen(*promise*, *onFulfilled*, *onRejected*).
 10. Return **undefined**.
 - ii. Else,
 1. **Assert**: *completion*.[[Type]] is throw.
 2. Perform ! AsyncGeneratorReject(*generator*, *completion*.[[Value]]).
 3. Return **undefined**.
11. Else if *state* is "completed", return ! AsyncGeneratorResolve(*generator*, **undefined**, true).
12. **Assert**: *state* is either "suspendedStart" or "suspendedYield".

13. Let *genContext* be *generator*.[[AsyncGeneratorContext]].
14. Let *callerContext* be the running execution context.
15. Suspend *callerContext*.
16. Set *generator*.[[AsyncGeneratorState]] to "executing".
17. Push *genContext* onto the execution context stack; *genContext* is now the running execution context.
18. Resume the suspended evaluation of *genContext* using *completion* as the result of the operation that suspended it.
Let *result* be the completion record returned by the resumed computation.
19. Assert: *result* is never an abrupt completion.
20. Assert: When we return here, *genContext* has already been removed from the execution context stack and *callerContext* is the currently running execution context.
21. Return undefined.

25.5.3.5.1 AsyncGeneratorResumeNext Return Processor Fulfilled Functions

An *AsyncGeneratorResumeNext* return processor fulfilled function is an anonymous built-in function that is used as part of the *AsyncGeneratorResumeNext* specification device to unwrap promises passed in to the *AsyncGenerator.prototype.return (value)* method. Each *AsyncGeneratorResumeNext* return processor fulfilled function has a [[Generator]] internal slot.

When an *AsyncGeneratorResumeNext* return processor fulfilled function is called with argument *value*, the following steps are taken:

1. Let *F* be the active function object.
2. Set *F*.[[Generator]].[[AsyncGeneratorState]] to "completed".
3. Return ! *AsyncGeneratorResolve*(*F*.[[Generator]], *value*, true).

The "length" property of an *AsyncGeneratorResumeNext* return processor fulfilled function is 1.

25.5.3.5.2 AsyncGeneratorResumeNext Return Processor Rejected Functions

An *AsyncGeneratorResumeNext* return processor rejected function is an anonymous built-in function that is used as part of the *AsyncGeneratorResumeNext* specification device to unwrap promises passed in to the *AsyncGenerator.prototype.return (value)* method. Each *AsyncGeneratorResumeNext* return processor rejected function has a [[Generator]] internal slot.

When an *AsyncGeneratorResumeNext* return processor rejected function is called with argument *reason*, the following steps are taken:

1. Let *F* be the active function object.
2. Set *F*.[[Generator]].[[AsyncGeneratorState]] to "completed".
3. Return ! *AsyncGeneratorReject*(*F*.[[Generator]], *reason*).

The "length" property of an *AsyncGeneratorResumeNext* return processor rejected function is 1.

25.5.3.6 AsyncGeneratorEnqueue (generator, completion)

1. Assert: *completion* is a Completion Record.
2. Let *promiseCapability* be ! *NewPromiseCapability*(%Promise%).
3. If *Type*(*generator*) is not Object, or if *generator* does not have an [[AsyncGeneratorState]] internal slot, then
 - a. Let *badGeneratorError* be a newly created **TypeError** object.
 - b. Perform ! *Call*(*promiseCapability*.[[Reject]], undefined, « *badGeneratorError* »).

- c. Return *promiseCapability*.[[Promise]].
4. Let *queue* be *generator*.[[AsyncGeneratorQueue]].
5. Let *request* be AsyncGeneratorRequest { [[Completion]]: *completion*, [[Capability]]: *promiseCapability* }.
6. Append *request* to the end of *queue*.
7. Let *state* be *generator*.[[AsyncGeneratorState]].
8. If *state* is not "executing", then
 - a. Perform ! AsyncGeneratorResumeNext(*generator*).
9. Return *promiseCapability*.[[Promise]].

25.5.3.7 AsyncGeneratorYield (*value*)

The abstract operation AsyncGeneratorYield with argument *value* performs the following steps:

1. Let *genContext* be the running execution context.
2. **Assert:** *genContext* is the execution context of a generator.
3. Let *generator* be the value of the Generator component of *genContext*.
4. **Assert:** GetGeneratorKind() is async.
5. Set *value* to ? Await(*value*).
6. Set *generator*.[[AsyncGeneratorState]] to "suspendedYield".
7. Remove *genContext* from the execution context stack and restore the execution context that is at the top of the execution context stack as the running execution context.
8. Set the code evaluation state of *genContext* such that when evaluation is resumed with a Completion *resumptionValue* the following steps will be performed:
 - a. If *resumptionValue*.[[Type]] is not return, return Completion(*resumptionValue*).
 - b. Let *awaited* be Await(*resumptionValue*.[[Value]]).
 - c. If *awaited*.[[Type]] is throw, return Completion(*awaited*).
 - d. **Assert:** *awaited*.[[Type]] is normal.
 - e. Return Completion { [[Type]]: return, [[Value]]: *awaited*.[[Value]], [[Target]]: empty }.
 - f. NOTE: When one of the above steps returns, it returns to the evaluation of the *YieldExpression* production that originally called this abstract operation.
9. Return ! AsyncGeneratorResolve(*generator*, *value*, false).
10. NOTE: This returns to the evaluation of the operation that had most previously resumed evaluation of *genContext*.

25.6 Promise Objects

A Promise is an object that is used as a placeholder for the eventual results of a deferred (and possibly asynchronous) computation.

Any Promise object is in one of three mutually exclusive states: *fulfilled*, *rejected*, and *pending*:

A promise *p* is fulfilled if *p*.then(*f*, *r*) will immediately enqueue a Job to call the function *f*.

A promise *p* is rejected if *p*.then(*f*, *r*) will immediately enqueue a Job to call the function *r*.

A promise is pending if it is neither fulfilled nor rejected.

A promise is said to be *settled* if it is not pending, i.e. if it is either fulfilled or rejected.

A promise is *resolved* if it is settled or if it has been “locked in” to match the state of another promise. Attempting to resolve or reject a resolved promise has no effect. A promise is *unresolved* if it is not resolved. An unresolved promise is always in the pending state. A resolved promise may be pending, fulfilled or rejected.

25.6.1 Promise Abstract Operations

25.6.1.1 PromiseCapability Records

A PromiseCapability is a [Record](#) value used to encapsulate a promise object along with the functions that are capable of resolving or rejecting that promise object. PromiseCapability Records are produced by the [NewPromiseCapability](#) abstract operation.

PromiseCapability Records have the fields listed in [Table 74](#).

Table 74: PromiseCapability [Record](#) Fields

Field Name	Value	Meaning
[[Promise]]	An object	An object that is usable as a promise.
[[Resolve]]	A function object	The function that is used to resolve the given promise object.
[[Reject]]	A function object	The function that is used to reject the given promise object.

25.6.1.1.1 IfAbruptRejectPromise ([value](#), [capability](#))

IfAbruptRejectPromise is a shorthand for a sequence of algorithm steps that use a PromiseCapability [Record](#). An algorithm step of the form:

1. [IfAbruptRejectPromise](#)([value](#), [capability](#)).

means the same thing as:

1. If [value](#) is an [abrupt completion](#), then
 - a. Perform ? [Call](#)([capability](#).[[Reject]], **undefined**, « [value](#).[[Value]] »).
 - b. Return [capability](#).[[Promise]].
2. Else if [value](#) is a [Completion Record](#), set [value](#) to [value](#).[[Value]].

25.6.1.2 PromiseReaction Records

The PromiseReaction is a [Record](#) value used to store information about how a promise should react when it becomes resolved or rejected with a given value. PromiseReaction records are created by the [PerformPromiseThen](#) abstract operation, and are used by a [PromiseReactionJob](#).

PromiseReaction records have the fields listed in [Table 75](#).

Table 75: PromiseReaction [Record](#) Fields

Field Name	Value	Meaning
[[Capability]]	A PromiseCapability Record , or undefined	The capabilities of the promise for which this record provides a reaction handler.
[[Type]]	Either "Fulfill" or "Reject" .	The [[Type]] is used when [[Handler]] is undefined to allow for behaviour specific to the settlement type.

[[Handler]]	A function object or undefined .	The function that should be applied to the incoming value, and whose return value will govern what happens to the derived promise. If [[Handler]] is undefined , a function that depends on the value of [[Type]] will be used instead.
-------------	--	--

25.6.1.3 CreateResolvingFunctions (*promise*)

When CreateResolvingFunctions is performed with argument *promise*, the following steps are taken:

1. Let *alreadyResolved* be a new **Record** { [[Value]]: **false** }.
2. Let *stepsResolve* be the algorithm steps defined in Promise Resolve Functions (25.6.1.3.2).
3. Let *resolve* be **CreateBuiltinFunction**(*stepsResolve*, « [[Promise]], [[AlreadyResolved]] »).
4. Set *resolve*.[[Promise]] to *promise*.
5. Set *resolve*.[[AlreadyResolved]] to *alreadyResolved*.
6. Let *stepsReject* be the algorithm steps defined in Promise Reject Functions (25.6.1.3.1).
7. Let *reject* be **CreateBuiltinFunction**(*stepsReject*, « [[Promise]], [[AlreadyResolved]] »).
8. Set *reject*.[[Promise]] to *promise*.
9. Set *reject*.[[AlreadyResolved]] to *alreadyResolved*.
10. Return a new **Record** { [[Resolve]]: *resolve*, [[Reject]]: *reject* }.

25.6.1.3.1 Promise Reject Functions

A promise reject function is an anonymous built-in function that has [[Promise]] and [[AlreadyResolved]] internal slots.

When a promise reject function is called with argument *reason*, the following steps are taken:

1. Let *F* be the **active function object**.
2. **Assert**: *F* has a [[Promise]] internal slot whose value is an Object.
3. Let *promise* be *F*.[[Promise]].
4. Let *alreadyResolved* be *F*.[[AlreadyResolved]].
5. If *alreadyResolved*.[[Value]] is **true**, return **undefined**.
6. Set *alreadyResolved*.[[Value]] to **true**.
7. Return **RejectPromise**(*promise*, *reason*).

The "**length**" property of a promise reject function is 1.

25.6.1.3.2 Promise Resolve Functions

A promise resolve function is an anonymous built-in function that has [[Promise]] and [[AlreadyResolved]] internal slots.

When a promise resolve function is called with argument *resolution*, the following steps are taken:

1. Let *F* be the **active function object**.
2. **Assert**: *F* has a [[Promise]] internal slot whose value is an Object.
3. Let *promise* be *F*.[[Promise]].
4. Let *alreadyResolved* be *F*.[[AlreadyResolved]].
5. If *alreadyResolved*.[[Value]] is **true**, return **undefined**.

6. Set *alreadyResolved*.[[Value]] to **true**.
7. If *SameValue*(*resolution*, *promise*) is **true**, then
 - a. Let *selfResolutionError* be a newly created **TypeError** object.
 - b. Return *RejectPromise*(*promise*, *selfResolutionError*).
8. If *Type*(*resolution*) is not Object, then
 - a. Return *FulfillPromise*(*promise*, *resolution*).
9. Let *then* be *Get*(*resolution*, "then").
10. If *then* is an abrupt completion, then
 - a. Return *RejectPromise*(*promise*, *then*.[[Value]]).
11. Let *thenAction* be *then*.[[Value]].
12. If *IsCallable*(*thenAction*) is **false**, then
 - a. Return *FulfillPromise*(*promise*, *resolution*).
13. Perform *EnqueueJob*("PromiseJobs", *PromiseResolveThenableJob*, « *promise*, *resolution*, *thenAction* »).
14. Return **undefined**.

The "length" property of a promise resolve function is 1.

25.6.1.4 FulfillPromise (*promise*, *value*)

When the FulfillPromise abstract operation is called with arguments *promise* and *value*, the following steps are taken:

1. **Assert**: The value of *promise*.[[PromiseState]] is "pending".
2. Let *reactions* be *promise*.[[PromiseFulfillReactions]].
3. Set *promise*.[[PromiseResult]] to *value*.
4. Set *promise*.[[PromiseFulfillReactions]] to **undefined**.
5. Set *promise*.[[PromiseRejectReactions]] to **undefined**.
6. Set *promise*.[[PromiseState]] to "fulfilled".
7. Return *TriggerPromiseReactions*(*reactions*, *value*).

25.6.1.5 NewPromiseCapability (*C*)

The abstract operation NewPromiseCapability takes a *constructor* function, and attempts to use that *constructor* function in the fashion of the built-in **Promise** *constructor* to create a Promise object and extract its resolve and reject functions. The promise plus the resolve and reject functions are used to initialize a new PromiseCapability *Record* which is returned as the value of this abstract operation.

1. If *IsConstructor*(*C*) is **false**, throw a **TypeError** exception.
2. NOTE: *C* is assumed to be a *constructor* function that supports the parameter conventions of the **Promise** *constructor* (see 25.6.3.1).
3. Let *promiseCapability* be a new PromiseCapability { [[Promise]]: **undefined**, [[Resolve]]: **undefined**, [[Reject]]: **undefined** }.
4. Let *steps* be the algorithm steps defined in *GetCapabilitiesExecutor Functions*.
5. Let *executor* be *CreateBuiltinFunction*(*steps*, « [[Capability]] »).
6. Set *executor*.[[Capability]] to *promiseCapability*.
7. Let *promise* be ? *Construct*(*C*, « *executor* »).
8. If *IsCallable*(*promiseCapability*.[[Resolve]]) is **false**, throw a **TypeError** exception.
9. If *IsCallable*(*promiseCapability*.[[Reject]]) is **false**, throw a **TypeError** exception.
10. Set *promiseCapability*.[[Promise]] to *promise*.
11. Return *promiseCapability*.

NOTE

This abstract operation supports Promise subclassing, as it is generic on any [constructor](#) that calls a passed executor function argument in the same way as the Promise [constructor](#). It is used to generalize static methods of the Promise [constructor](#) to any subclass.

25.6.1.5.1 GetCapabilitiesExecutor Functions

A GetCapabilitiesExecutor function is an anonymous built-in function that has a `[[Capability]]` internal slot.

When a GetCapabilitiesExecutor function is called with arguments *resolve* and *reject*, the following steps are taken:

1. Let *F* be the [active function object](#).
2. **Assert**: *F* has a `[[Capability]]` internal slot whose value is a PromiseCapability [Record](#).
3. Let *promiseCapability* be *F*.`[[Capability]]`.
4. If *promiseCapability*.`[[Resolve]]` is not **undefined**, throw a **TypeError** exception.
5. If *promiseCapability*.`[[Reject]]` is not **undefined**, throw a **TypeError** exception.
6. Set *promiseCapability*.`[[Resolve]]` to *resolve*.
7. Set *promiseCapability*.`[[Reject]]` to *reject*.
8. Return **undefined**.

The **"length"** property of a GetCapabilitiesExecutor function is 2.

25.6.1.6 IsPromise (*x*)

The abstract operation IsPromise checks for the promise brand on an object.

1. If [Type](#)(*x*) is not Object, return **false**.
2. If *x* does not have a `[[PromiseState]]` internal slot, return **false**.
3. Return **true**.

25.6.1.7 RejectPromise (*promise*, *reason*)

When the RejectPromise abstract operation is called with arguments *promise* and *reason*, the following steps are taken:

1. **Assert**: The value of *promise*.`[[PromiseState]]` is **"pending"**.
2. Let *reactions* be *promise*.`[[PromiseRejectReactions]]`.
3. Set *promise*.`[[PromiseResult]]` to *reason*.
4. Set *promise*.`[[PromiseFulfillReactions]]` to **undefined**.
5. Set *promise*.`[[PromiseRejectReactions]]` to **undefined**.
6. Set *promise*.`[[PromiseState]]` to **"rejected"**.
7. If *promise*.`[[PromiseIsHandled]]` is **false**, perform [HostPromiseRejectionTracker](#)(*promise*, **"reject"**).
8. Return [TriggerPromiseReactions](#)(*reactions*, *reason*).

25.6.1.8 TriggerPromiseReactions (*reactions*, *argument*)

The abstract operation TriggerPromiseReactions takes a collection of PromiseReactionRecords and enqueues a new Job for each record. Each such Job processes the `[[Type]]` and `[[Handler]]` of the PromiseReactionRecord, and if the `[[Handler]]` is a function, calls it passing the given argument. If the `[[Handler]]` is **undefined**, the behaviour is determined by the `[[Type]]`.

1. For each *reaction* in *reactions*, in original insertion order, do
 - a. Perform `EnqueueJob("PromiseJobs", PromiseReactionJob, « reaction, argument »)`.
2. Return **undefined**.

25.6.1.9 HostPromiseRejectionTracker (*promise*, *operation*)

HostPromiseRejectionTracker is an implementation-defined abstract operation that allows host environments to track promise rejections.

An implementation of HostPromiseRejectionTracker must complete normally in all cases. The default implementation of HostPromiseRejectionTracker is to unconditionally return an empty normal completion.

NOTE 1

HostPromiseRejectionTracker is called in two scenarios:

When a promise is rejected without any handlers, it is called with its *operation* argument set to **"reject"**.

When a handler is added to a rejected promise for the first time, it is called with its *operation* argument set to **"handle"**.

A typical implementation of HostPromiseRejectionTracker might try to notify developers of unhandled rejections, while also being careful to notify them if such previous notifications are later invalidated by new handlers being attached.

NOTE 2

If *operation* is **"handle"**, an implementation should not hold a reference to *promise* in a way that would interfere with garbage collection. An implementation may hold a reference to *promise* if *operation* is **"reject"**, since it is expected that rejections will be rare and not on hot code paths.

25.6.2 Promise Jobs

25.6.2.1 PromiseReactionJob (*reaction*, *argument*)

The job PromiseReactionJob with parameters *reaction* and *argument* applies the appropriate handler to the incoming value, and uses the handler's return value to resolve or reject the derived promise associated with that handler.

1. **Assert:** *reaction* is a PromiseReaction Record.
2. Let *promiseCapability* be *reaction*.[[Capability]].
3. Let *type* be *reaction*.[[Type]].
4. Let *handler* be *reaction*.[[Handler]].
5. If *handler* is **undefined**, then
 - a. If *type* is **"Fulfill"**, let *handlerResult* be `NormalCompletion(argument)`.
 - b. Else,
 - i. **Assert:** *type* is **"Reject"**.
 - ii. Let *handlerResult* be `ThrowCompletion(argument)`.
6. Else, let *handlerResult* be `Call(handler, undefined, « argument »)`.
7. If *promiseCapability* is **undefined**, then
 - a. **Assert:** *handlerResult* is not an abrupt completion.
 - b. Return `NormalCompletion(empty)`.
8. If *handlerResult* is an abrupt completion, then
 - a. Let *status* be `Call(promiseCapability.[[Reject]], undefined, « handlerResult.[[Value]] »)`.

9. Else,
 - a. Let *status* be *Call*(*promiseCapability*.[[Resolve]], **undefined**, « *handlerResult*.[[Value]] »).
10. Return *Completion*(*status*).

25.6.2.2 PromiseResolveThenableJob (*promiseToResolve*, *thenable*, *then*)

The job *PromiseResolveThenableJob* with parameters *promiseToResolve*, *thenable*, and *then* performs the following steps:

1. Let *resolvingFunctions* be *CreateResolvingFunctions*(*promiseToResolve*).
2. Let *thenCallResult* be *Call*(*then*, *thenable*, « *resolvingFunctions*.[[Resolve]], *resolvingFunctions*.[[Reject]] »).
3. If *thenCallResult* is an abrupt completion, then
 - a. Let *status* be *Call*(*resolvingFunctions*.[[Reject]], **undefined**, « *thenCallResult*.[[Value]] »).
 - b. Return *Completion*(*status*).
4. Return *Completion*(*thenCallResult*).

NOTE

This Job uses the supplied thenable and its **then** method to resolve the given promise. This process must take place as a Job to ensure that the evaluation of the **then** method occurs after evaluation of any surrounding code has completed.

25.6.3 The Promise Constructor

The Promise *constructor*:

is the intrinsic object *%Promise%*.

is the initial value of the **Promise** property of the *global object*.

creates and initializes a new Promise object when called as a *constructor*.

is not intended to be called as a function and will throw an exception when called in that manner.

is designed to be subclassable. It may be used as the value in an **extends** clause of a class definition. Subclass constructors that intend to inherit the specified **Promise** behaviour must include a **super** call to the **Promise constructor** to create and initialize the subclass instance with the internal state necessary to support the **Promise** and **Promise.prototype** built-in methods.

25.6.3.1 Promise (*executor*)

When the **Promise** function is called with argument *executor*, the following steps are taken:

1. If *NewTarget* is **undefined**, throw a **TypeError** exception.
2. If *IsCallable*(*executor*) is **false**, throw a **TypeError** exception.
3. Let *promise* be ? *OrdinaryCreateFromConstructor*(*NewTarget*, "**%PromisePrototype%**", « [[*PromiseState*]], [[*PromiseResult*]], [[*PromiseFulfillReactions*]], [[*PromiseRejectReactions*]], [[*PromiseIsHandled*]] »).
4. Set *promise*.[[*PromiseState*]] to **"pending"**.
5. Set *promise*.[[*PromiseFulfillReactions*]] to a new empty *List*.
6. Set *promise*.[[*PromiseRejectReactions*]] to a new empty *List*.
7. Set *promise*.[[*PromiseIsHandled*]] to **false**.
8. Let *resolvingFunctions* be *CreateResolvingFunctions*(*promise*).
9. Let *completion* be *Call*(*executor*, **undefined**, « *resolvingFunctions*.[[Resolve]], *resolvingFunctions*.[[Reject]] »).
10. If *completion* is an abrupt completion, then

- a. Perform ? **Call**(*resolvingFunctions*.[[**Reject**]], **undefined**, « *completion*.[[**Value**]] »).

11. Return *promise*.

NOTE

The *executor* argument must be a **function object**. It is called for initiating and reporting completion of the possibly deferred action represented by this Promise object. The executor is called with two arguments: *resolve* and *reject*. These are functions that may be used by the *executor* function to report eventual completion or failure of the deferred computation. Returning from the executor function does not mean that the deferred action has been completed but only that the request to eventually perform the deferred action has been accepted.

The *resolve* function that is passed to an *executor* function accepts a single argument. The *executor* code may eventually call the *resolve* function to indicate that it wishes to resolve the associated Promise object. The argument passed to the *resolve* function represents the eventual value of the deferred action and can be either the actual fulfillment value or another Promise object which will provide the value if it is fulfilled.

The *reject* function that is passed to an *executor* function accepts a single argument. The *executor* code may eventually call the *reject* function to indicate that the associated Promise is rejected and will never be fulfilled. The argument passed to the *reject* function is used as the rejection value of the promise. Typically it will be an **Error** object.

The resolve and reject functions passed to an *executor* function by the Promise **constructor** have the capability to actually resolve and reject the associated promise. Subclasses may have different **constructor** behaviour that passes in customized values for resolve and reject.

25.6.4 Properties of the Promise Constructor

The Promise **constructor**:

- has a [[**Prototype**]] internal slot whose value is the intrinsic object **%FunctionPrototype%**.
- has the following properties:

25.6.4.1 Promise.all (*iterable*)

The **all** function returns a new promise which is fulfilled with an array of fulfillment values for the passed promises, or rejects with the reason of the first passed promise that rejects. It resolves all elements of the passed iterable to promises as it runs this algorithm.

1. Let *C* be the **this** value.
2. If **Type**(*C*) is not Object, throw a **TypeError** exception.
3. Let *promiseCapability* be ? **NewPromiseCapability**(*C*).
4. Let *iteratorRecord* be **GetIterator**(*iterable*).
5. **IfAbruptRejectPromise**(*iteratorRecord*, *promiseCapability*).
6. Let *result* be **PerformPromiseAll**(*iteratorRecord*, *C*, *promiseCapability*).
7. If *result* is an abrupt completion, then
 - a. If *iteratorRecord*.[[**Done**]] is **false**, set *result* to **IteratorClose**(*iteratorRecord*, *result*).
 - b. **IfAbruptRejectPromise**(*result*, *promiseCapability*).
8. Return **Completion**(*result*).

This function is the **%Promise_all%** intrinsic object.

NOTE

The **all** function requires its **this** value to be a [constructor](#) function that supports the parameter conventions of the **Promise constructor**.

25.6.4.1.1 Runtime Semantics: PerformPromiseAll (*iteratorRecord*, *constructor*, *resultCapability*)

When the PerformPromiseAll abstract operation is called with arguments *iteratorRecord*, *constructor*, and *resultCapability*, the following steps are taken:

1. **Assert:** **IsConstructor**(*constructor*) is **true**.
2. **Assert:** *resultCapability* is a PromiseCapability **Record**.
3. Let *values* be a new empty **List**.
4. Let *remainingElementsCount* be a new **Record** { **[[Value]]**: 1 }.
5. Let *index* be 0.
6. Repeat,
 - a. Let *next* be **IteratorStep**(*iteratorRecord*).
 - b. If *next* is an abrupt completion, set *iteratorRecord*.**[[Done]]** to **true**.
 - c. **ReturnIfAbrupt**(*next*).
 - d. If *next* is **false**, then
 - i. Set *iteratorRecord*.**[[Done]]** to **true**.
 - ii. Set *remainingElementsCount*.**[[Value]]** to *remainingElementsCount*.**[[Value]]** - 1.
 - iii. If *remainingElementsCount*.**[[Value]]** is 0, then
 1. Let *valuesArray* be **CreateArrayFromList**(*values*).
 2. Perform ? **Call**(*resultCapability*.**[[Resolve]]**, **undefined**, « *valuesArray* »).
 - iv. Return *resultCapability*.**[[Promise]]**.
 - e. Let *nextValue* be **IteratorValue**(*next*).
 - f. If *nextValue* is an abrupt completion, set *iteratorRecord*.**[[Done]]** to **true**.
 - g. **ReturnIfAbrupt**(*nextValue*).
 - h. Append **undefined** to *values*.
 - i. Let *nextPromise* be ? **Invoke**(*constructor*, "resolve", « *nextValue* »).
 - j. Let *steps* be the algorithm steps defined in **Promise.all Resolve Element Functions**.
 - k. Let *resolveElement* be **CreateBuiltinFunction**(*steps*, « **[[AlreadyCalled]]**, **[[Index]]**, **[[Values]]**, **[[Capability]]**, **[[RemainingElements]]** »).
 - l. Set *resolveElement*.**[[AlreadyCalled]]** to a new **Record** { **[[Value]]**: **false** }.
 - m. Set *resolveElement*.**[[Index]]** to *index*.
 - n. Set *resolveElement*.**[[Values]]** to *values*.
 - o. Set *resolveElement*.**[[Capability]]** to *resultCapability*.
 - p. Set *resolveElement*.**[[RemainingElements]]** to *remainingElementsCount*.
 - q. Set *remainingElementsCount*.**[[Value]]** to *remainingElementsCount*.**[[Value]]** + 1.
 - r. Perform ? **Invoke**(*nextPromise*, "then", « *resolveElement*, *resultCapability*.**[[Reject]]** »).
 - s. Increase *index* by 1.

25.6.4.1.2 Promise.all Resolve Element Functions

A **Promise.all** resolve element function is an anonymous built-in function that is used to resolve a specific **Promise.all** element. Each **Promise.all** resolve element function has **[[Index]]**, **[[Values]]**, **[[Capability]]**, **[[RemainingElements]]**, and **[[AlreadyCalled]]** internal slots.

When a **Promise.all** resolve element function is called with argument *x*, the following steps are taken:

1. Let *F* be the **active function object**.
2. Let *alreadyCalled* be *F*.[[AlreadyCalled]].
3. If *alreadyCalled*.[[Value]] is **true**, return **undefined**.
4. Set *alreadyCalled*.[[Value]] to **true**.
5. Let *index* be *F*.[[Index]].
6. Let *values* be *F*.[[Values]].
7. Let *promiseCapability* be *F*.[[Capability]].
8. Let *remainingElementsCount* be *F*.[[RemainingElements]].
9. Set *values*[*index*] to *x*.
10. Set *remainingElementsCount*.[[Value]] to *remainingElementsCount*.[[Value]] - 1.
11. If *remainingElementsCount*.[[Value]] is 0, then
 - a. Let *valuesArray* be **CreateArrayFromList**(*values*).
 - b. Return ? **Call**(*promiseCapability*.[[Resolve]], **undefined**, « *valuesArray* »).
12. Return **undefined**.

The "**length**" property of a **Promise.all** resolve element function is 1.

25.6.4.2 Promise.prototype

The initial value of **Promise.prototype** is the intrinsic object **%PromisePrototype%**.

This property has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **false** }.

25.6.4.3 Promise.race (*iterable*)

The **race** function returns a new promise which is settled in the same way as the first passed promise to settle. It resolves all elements of the passed *iterable* to promises as it runs this algorithm.

1. Let *C* be the **this** value.
2. If **Type**(*C*) is not Object, throw a **TypeError** exception.
3. Let *promiseCapability* be ? **NewPromiseCapability**(*C*).
4. Let *iteratorRecord* be **GetIterator**(*iterable*).
5. **IfAbruptRejectPromise**(*iteratorRecord*, *promiseCapability*).
6. Let *result* be **PerformPromiseRace**(*iteratorRecord*, *C*, *promiseCapability*).
7. If *result* is an abrupt completion, then
 - a. If *iteratorRecord*.[[Done]] is **false**, set *result* to **IteratorClose**(*iteratorRecord*, *result*).
 - b. **IfAbruptRejectPromise**(*result*, *promiseCapability*).
8. Return **Completion**(*result*).

NOTE 1

If the *iterable* argument is empty or if none of the promises in *iterable* ever settle then the pending promise returned by this method will never be settled.

NOTE 2

The **race** function expects its **this** value to be a **constructor** function that supports the parameter conventions of the **Promise constructor**. It also expects that its **this** value provides a **resolve** method.

25.6.4.3.1 Runtime Semantics: PerformPromiseRace (*iteratorRecord*, *constructor*, *resultCapability*)

When the **PerformPromiseRace** abstract operation is called with arguments *iteratorRecord*, *constructor*, and

resultCapability, the following steps are taken:

1. **Assert**: *IsConstructor*(*constructor*) is **true**.
2. **Assert**: *resultCapability* is a PromiseCapability *Record*.
3. Repeat,
 - a. Let *next* be *IteratorStep*(*iteratorRecord*).
 - b. If *next* is an abrupt completion, set *iteratorRecord*.[[Done]] to **true**.
 - c. *ReturnIfAbrupt*(*next*).
 - d. If *next* is **false**, then
 - i. Set *iteratorRecord*.[[Done]] to **true**.
 - ii. Return *resultCapability*.[[Promise]].
 - e. Let *nextValue* be *IteratorValue*(*next*).
 - f. If *nextValue* is an abrupt completion, set *iteratorRecord*.[[Done]] to **true**.
 - g. *ReturnIfAbrupt*(*nextValue*).
 - h. Let *nextPromise* be ? *Invoke*(*constructor*, "**resolve**", « *nextValue* »).
 - i. Perform ? *Invoke*(*nextPromise*, "**then**", « *resultCapability*.[[Resolve]], *resultCapability*.[[Reject]] »).

25.6.4.4 **Promise.reject (*r*)**

The **reject** function returns a new promise rejected with the passed argument.

1. Let *C* be the **this** value.
2. If *Type*(*C*) is not Object, throw a **TypeError** exception.
3. Let *promiseCapability* be ? *NewPromiseCapability*(*C*).
4. Perform ? *Call*(*promiseCapability*.[[Reject]], **undefined**, « *r* »).
5. Return *promiseCapability*.[[Promise]].

This function is the %Promise_reject% intrinsic object.

NOTE

The **reject** function expects its **this** value to be a *constructor* function that supports the parameter conventions of the **Promise constructor**.

25.6.4.5 **Promise.resolve (*x*)**

The **resolve** function returns either a new promise resolved with the passed argument, or the argument itself if the argument is a promise produced by this *constructor*.

1. Let *C* be the **this** value.
2. If *Type*(*C*) is not Object, throw a **TypeError** exception.
3. Return ? *PromiseResolve*(*C*, *x*).

This function is the %Promise_resolve% intrinsic object.

NOTE

The **resolve** function expects its **this** value to be a *constructor* function that supports the parameter conventions of the **Promise constructor**.

25.6.4.5.1 PromiseResolve (*C*, *x*)

The abstract operation PromiseResolve, given a [constructor](#) and a value, returns a new promise resolved with that value.

1. **Assert:** `Type(C)` is Object.
2. If `IsPromise(x)` is **true**, then
 - a. Let *xConstructor* be ? `Get(x, "constructor")`.
 - b. If `SameValue(xConstructor, C)` is **true**, return *x*.
3. Let *promiseCapability* be ? `NewPromiseCapability(C)`.
4. Perform ? `Call(promiseCapability.[[Resolve]], undefined, « x »)`.
5. Return *promiseCapability*.[[Promise]].

25.6.4.6 get Promise [@@species]

`Promise[@@species]` is an [accessor property](#) whose set accessor function is **undefined**. Its get accessor function performs the following steps:

1. Return the **this** value.

The value of the **name** property of this function is `"get [Symbol.species]"`.

NOTE

Promise prototype methods normally use their **this** object's [constructor](#) to create a derived object. However, a subclass [constructor](#) may over-ride that default behaviour by redefining its @@species property.

25.6.5 Properties of the Promise Prototype Object

The Promise prototype object:

is the intrinsic object `%PromisePrototype%`.

has a `[[Prototype]]` internal slot whose value is the intrinsic object `%ObjectPrototype%`.

is an ordinary object.

does not have a `[[PromiseState]]` internal slot or any of the other internal slots of Promise instances.

25.6.5.1 Promise.prototype.catch (*onRejected*)

When the **catch** method is called with argument *onRejected*, the following steps are taken:

1. Let *promise* be the **this** value.
2. Return ? `Invoke(promise, "then", « undefined, onRejected »)`.

25.6.5.2 Promise.prototype.constructor

The initial value of `Promise.prototype.constructor` is the intrinsic object `%Promise%`.

25.6.5.3 Promise.prototype.finally (*onFinally*)

When the **finally** method is called with argument *onFinally*, the following steps are taken:

1. Let *promise* be the **this** value.

2. If **Type**(*promise*) is not Object, throw a **TypeError** exception.
3. Let *C* be ? **SpeciesConstructor**(*promise*, %Promise%).
4. **Assert**: **IsConstructor**(*C*) is **true**.
5. If **IsCallable**(*onFinally*) is **false**, then
 - a. Let *thenFinally* be *onFinally*.
 - b. Let *catchFinally* be *onFinally*.
6. Else,
 - a. Let *stepsThenFinally* be the algorithm steps defined in **Then Finally Functions**.
 - b. Let *thenFinally* be **CreateBuiltinFunction**(*stepsThenFinally*, « [[Constructor]], [[OnFinally]] »).
 - c. Set *thenFinally*.[[Constructor]] to *C*.
 - d. Set *thenFinally*.[[OnFinally]] to *onFinally*.
 - e. Let *stepsCatchFinally* be the algorithm steps defined in **Catch Finally Functions**.
 - f. Let *catchFinally* be **CreateBuiltinFunction**(*stepsCatchFinally*, « [[Constructor]], [[OnFinally]] »).
 - g. Set *catchFinally*.[[Constructor]] to *C*.
 - h. Set *catchFinally*.[[OnFinally]] to *onFinally*.
7. Return ? **Invoke**(*promise*, "then", « *thenFinally*, *catchFinally* »).

25.6.5.3.1 Then Finally Functions

A Then Finally function is an anonymous built-in function that has a [[Constructor]] and an [[OnFinally]] internal slot. The value of the [[Constructor]] internal slot is a **Promise**-like **constructor function object**, and the value of the [[OnFinally]] internal slot is a **function object**.

When a Then Finally function is called with argument *value*, the following steps are taken:

1. Let *F* be the **active function object**.
2. Let *onFinally* be *F*.[[OnFinally]].
3. **Assert**: **IsCallable**(*onFinally*) is **true**.
4. Let *result* be ? **Call**(*onFinally*, **undefined**).
5. Let *C* be *F*.[[Constructor]].
6. **Assert**: **IsConstructor**(*C*) is **true**.
7. Let *promise* be ? **PromiseResolve**(*C*, *result*).
8. Let *valueThunk* be equivalent to a function that returns *value*.
9. Return ? **Invoke**(*promise*, "then", « *valueThunk* »).

The "length" property of a Then Finally function is 1.

25.6.5.3.2 Catch Finally Functions

A Catch Finally function is an anonymous built-in function that has a [[Constructor]] and an [[OnFinally]] internal slot. The value of the [[Constructor]] internal slot is a **Promise**-like **constructor function object**, and the value of the [[OnFinally]] internal slot is a **function object**.

When a Catch Finally function is called with argument *reason*, the following steps are taken:

1. Let *F* be the **active function object**.
2. Let *onFinally* be *F*.[[OnFinally]].
3. **Assert**: **IsCallable**(*onFinally*) is **true**.
4. Let *result* be ? **Call**(*onFinally*, **undefined**).
5. Let *C* be *F*.[[Constructor]].

6. **Assert:** `IsConstructor(C)` is **true**.
7. Let *promise* be ? `PromiseResolve(C, result)`.
8. Let *thrower* be equivalent to a function that throws *reason*.
9. Return ? `Invoke(promise, "then", « thrower »)`.

The **"length"** property of a `Catch Finally` function is **1**.

25.6.5.4 `Promise.prototype.then` (*onFulfilled*, *onRejected*)

When the **then** method is called with arguments *onFulfilled* and *onRejected*, the following steps are taken:

1. Let *promise* be the **this** value.
2. If `IsPromise(promise)` is **false**, throw a **TypeError** exception.
3. Let *C* be ? `SpeciesConstructor(promise, %Promise%)`.
4. Let *resultCapability* be ? `NewPromiseCapability(C)`.
5. Return `PerformPromiseThen(promise, onFulfilled, onRejected, resultCapability)`.

This function is the `%PromiseProto_then%` intrinsic object.

25.6.5.4.1 `PerformPromiseThen` (*promise*, *onFulfilled*, *onRejected* [, *resultCapability*])

The abstract operation `PerformPromiseThen` performs the “then” operation on *promise* using *onFulfilled* and *onRejected* as its settlement actions. If *resultCapability* is passed, the result is stored by updating *resultCapability*'s promise. (If it is not passed, then `PerformPromiseThen` is being called by a specification-internal operation where the result does not matter.)

1. **Assert:** `IsPromise(promise)` is **true**.
2. If *resultCapability* is present, then
 - a. **Assert:** *resultCapability* is a `PromiseCapability Record`.
3. Else,
 - a. Set *resultCapability* to **undefined**.
4. If `IsCallable(onFulfilled)` is **false**, then
 - a. Set *onFulfilled* to **undefined**.
5. If `IsCallable(onRejected)` is **false**, then
 - a. Set *onRejected* to **undefined**.
6. Let *fulfillReaction* be the `PromiseReaction` { `[[Capability]]`: *resultCapability*, `[[Type]]`: **"Fulfill"**, `[[Handler]]`: *onFulfilled* }.
7. Let *rejectReaction* be the `PromiseReaction` { `[[Capability]]`: *resultCapability*, `[[Type]]`: **"Reject"**, `[[Handler]]`: *onRejected* }.
8. If *promise*.`[[PromiseState]]` is **"pending"**, then
 - a. Append *fulfillReaction* as the last element of the `List` that is *promise*.`[[PromiseFulfillReactions]]`.
 - b. Append *rejectReaction* as the last element of the `List` that is *promise*.`[[PromiseRejectReactions]]`.
9. Else if *promise*.`[[PromiseState]]` is **"fulfilled"**, then
 - a. Let *value* be *promise*.`[[PromiseResult]]`.
 - b. Perform `EnqueueJob("PromiseJobs", PromiseReactionJob, « fulfillReaction, value »)`.
10. Else,
 - a. **Assert:** The value of *promise*.`[[PromiseState]]` is **"rejected"**.
 - b. Let *reason* be *promise*.`[[PromiseResult]]`.
 - c. If *promise*.`[[PromiseIsHandled]]` is **false**, perform `HostPromiseRejectionTracker(promise, "handle")`.
 - d. Perform `EnqueueJob("PromiseJobs", PromiseReactionJob, « rejectReaction, reason »)`.

11. Set *promise*.[[PromiseIsHandled]] to **true**.
12. If *resultCapability* is **undefined**, then
 - a. Return **undefined**.
13. Else,
 - a. Return *resultCapability*.[[Promise]].

25.6.5.5 Promise.prototype [@@toStringTag]

The initial value of the @@toStringTag property is the String value "**Promise**".

This property has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **true** }.

25.6.6 Properties of Promise Instances

Promise instances are ordinary objects that inherit properties from the Promise prototype object (the intrinsic, *%PromisePrototype%*). Promise instances are initially created with the internal slots described in [Table 76](#).

Table 76: Internal Slots of Promise Instances

Internal Slot	Description
[[PromiseState]]	A String value that governs how a promise will react to incoming calls to its then method. The possible values are: " pending ", " fulfilled ", and " rejected ".
[[PromiseResult]]	The value with which the promise has been fulfilled or rejected, if any. Only meaningful if [[PromiseState]] is not " pending ".
[[PromiseFulfillReactions]]	A List of PromiseReaction records to be processed when/if the promise transitions from the " pending " state to the " fulfilled " state.
[[PromiseRejectReactions]]	A List of PromiseReaction records to be processed when/if the promise transitions from the " pending " state to the " rejected " state.
[[PromiseIsHandled]]	A boolean indicating whether the promise has ever had a fulfillment or rejection handler; used in unhandled rejection tracking.

25.7 AsyncFunction Objects

AsyncFunction objects are functions that are usually created by evaluating *AsyncFunctionDeclarations*, *AsyncFunctionExpressions*, *AsyncMethods*, and *AsyncArrowFunctions*. They may also be created by calling the *%AsyncFunction%* intrinsic.

25.7.1 The AsyncFunction Constructor

The AsyncFunction [constructor](#):

is the intrinsic object *%AsyncFunction%*.
 is a subclass of **Function**.
 creates and initializes a new AsyncFunction object when called as a function rather than as a [constructor](#). Thus the function call **AsyncFunction(...)** is equivalent to the object creation expression **new AsyncFunction(...)**

with the same arguments.

is designed to be subclassable. It may be used as the value of an **extends** clause of a class definition. Subclass constructors that intend to inherit the specified AsyncFunction behaviour must include a **super** call to the **AsyncFunction constructor** to create and initialize a subclass instance with the internal slots necessary for built-in async function behaviour.

25.7.1.1 AsyncFunction (*p1*, *p2*, ..., *pn*, *body*)

The last argument specifies the body (executable code) of an async function. Any preceding arguments specify formal parameters.

When the **AsyncFunction** function is called with some arguments *p1*, *p2*, ..., *pn*, *body* (where *n* might be 0, that is, there are no *p* arguments, and where *body* might also not be provided), the following steps are taken:

1. Let *C* be the **active function object**.
2. Let *args* be the *argumentsList* that was passed to this function by `[[Call]]` or `[[Construct]]`.
3. Return `CreateDynamicFunction(C, NewTarget, "async", args)`.

NOTE

See NOTE for 19.2.1.1.

25.7.2 Properties of the AsyncFunction Constructor

The AsyncFunction **constructor**:

is a standard built-in **function object** that inherits from the **Function constructor**.

has a `[[Prototype]]` internal slot whose value is the intrinsic object `%Function%`.

has a **name** property whose value is `"AsyncFunction"`.

has the following properties:

25.7.2.1 AsyncFunction.length

This is a **data property** with a value of 1. This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **true** }.

25.7.2.2 AsyncFunction.prototype

The initial value of **AsyncFunction.prototype** is the intrinsic object `%AsyncFunctionPrototype%`.

This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **false** }.

25.7.3 Properties of the AsyncFunction Prototype Object

The AsyncFunction prototype object:

is an ordinary object.

is not a **function object** and does not have an `[[ECMAScriptCode]]` internal slot or any other of the internal slots listed in Table 27.

is the value of the **prototype** property of the intrinsic object `%AsyncFunction%`.

is the intrinsic object `%AsyncFunctionPrototype%`.

has a `[[Prototype]]` internal slot whose value is the intrinsic object `%FunctionPrototype%`.

25.7.3.1 `AsyncFunction.prototype.constructor`

The initial value of `AsyncFunction.prototype.constructor` is the intrinsic object `%AsyncFunction%`

This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **true** }.

25.7.3.2 `AsyncFunction.prototype [ @@toStringTag ]`

The initial value of the `@@toStringTag` property is the string value `"AsyncFunction"`.

This property has the attributes { `[[Writable]]`: **false**, `[[Enumerable]]`: **false**, `[[Configurable]]`: **true** }.

25.7.4 `AsyncFunction` Instances

Every `AsyncFunction` instance is an ECMAScript [function object](#) and has the internal slots listed in [Table 27](#). The value of the `[[FunctionKind]]` internal slot for all such instances is `"async"`. `AsyncFunction` instances are not constructors and do not have a `[[Construct]]` internal method. `AsyncFunction` instances do not have a `prototype` property as they are not constructable.

Each `AsyncFunction` instance has the following own properties:

25.7.4.1 `length`

The specification for the `"length"` property of `Function` instances given in [19.2.4.1](#) also applies to `AsyncFunction` instances.

25.7.4.2 `name`

The specification for the `name` property of `Function` instances given in [19.2.4.2](#) also applies to `AsyncFunction` instances.

25.7.5 `Async Functions Abstract Operations`

25.7.5.1 `AsyncFunctionStart ( promiseCapability, asyncFunctionBody )`

1. Let *runningContext* be the [running execution context](#).
2. Let *asyncContext* be a copy of *runningContext*.
3. Set the code evaluation state of *asyncContext* such that when evaluation is resumed for that [execution context](#) the following steps will be performed:
 - a. Let *result* be the result of evaluating *asyncFunctionBody*.
 - b. **Assert**: If we return here, the async function either threw an exception or performed an implicit or explicit return; all awaiting is done.
 - c. Remove *asyncContext* from the [execution context stack](#) and restore the [execution context](#) that is at the top of the [execution context stack](#) as the [running execution context](#).
 - d. If *result*.`[[Type]]` is `normal`, then
 - i. Perform `! Call(promiseCapability.[[Resolve]], undefined, « undefined »)`.
 - e. Else if *result*.`[[Type]]` is `return`, then

- i. Perform ! **Call**(*promiseCapability*.[[Resolve]], **undefined**, « *result*.[[Value]] »).
 - f. Else,
 - i. **Assert**: *result*.[[Type]] is **throw**.
 - ii. Perform ! **Call**(*promiseCapability*.[[Reject]], **undefined**, « *result*.[[Value]] »).
 - g. Return.
4. Push *asyncContext* onto the **execution context stack**; *asyncContext* is now the **running execution context**.
 5. Resume the suspended evaluation of *asyncContext*. Let *result* be the value returned by the resumed computation.
 6. **Assert**: When we return here, *asyncContext* has already been removed from the **execution context stack** and *runningContext* is the currently **running execution context**.
 7. **Assert**: *result* is a normal completion with a value of **undefined**. The possible sources of completion values are **Await** or, if the **async** function doesn't await anything, the step 3.g above.
 8. Return.

26 Reflection

26.1 The Reflect Object

The Reflect object:

is the intrinsic object **%Reflect%**.

is the initial value of the **Reflect** property of the **global object**.

is an ordinary object.

has a [[Prototype]] internal slot whose value is the intrinsic object **%ObjectPrototype%**.

is not a **function object**.

does not have a [[Construct]] internal method; it cannot be used as a **constructor** with the **new** operator.

does not have a [[Call]] internal method; it cannot be invoked as a function.

26.1.1 Reflect.apply (*target*, *thisArgument*, *argumentsList*)

When the **apply** function is called with arguments *target*, *thisArgument*, and *argumentsList*, the following steps are taken:

1. If **IsCallable**(*target*) is **false**, throw a **TypeError** exception.
2. Let *args* be ? **CreateListFromArrayLike**(*argumentsList*).
3. Perform **PrepareForTailCall**()
4. Return ? **Call**(*target*, *thisArgument*, *args*).

26.1.2 Reflect.construct (*target*, *argumentsList* [, *newTarget*])

When the **construct** function is called with arguments *target*, *argumentsList*, and *newTarget*, the following steps are taken:

1. If **IsConstructor**(*target*) is **false**, throw a **TypeError** exception.
2. If *newTarget* is not present, set *newTarget* to *target*.
3. Else if **IsConstructor**(*newTarget*) is **false**, throw a **TypeError** exception.
4. Let *args* be ? **CreateListFromArrayLike**(*argumentsList*).

5. Return ? **Construct**(*target*, *args*, *newTarget*).

26.1.3 Reflect.defineProperty (*target*, *propertyKey*, *attributes*)

When the **defineProperty** function is called with arguments *target*, *propertyKey*, and *attributes*, the following steps are taken:

1. If **Type**(*target*) is not Object, throw a **TypeError** exception.
2. Let *key* be ? **ToPropertyKey**(*propertyKey*).
3. Let *desc* be ? **ToPropertyDescriptor**(*attributes*).
4. Return ? *target*.[[DefineOwnProperty]](*key*, *desc*).

26.1.4 Reflect.deleteProperty (*target*, *propertyKey*)

When the **deleteProperty** function is called with arguments *target* and *propertyKey*, the following steps are taken:

1. If **Type**(*target*) is not Object, throw a **TypeError** exception.
2. Let *key* be ? **ToPropertyKey**(*propertyKey*).
3. Return ? *target*.[[Delete]](*key*).

26.1.5 Reflect.get (*target*, *propertyKey* [, *receiver*])

When the **get** function is called with arguments *target*, *propertyKey*, and *receiver*, the following steps are taken:

1. If **Type**(*target*) is not Object, throw a **TypeError** exception.
2. Let *key* be ? **ToPropertyKey**(*propertyKey*).
3. If *receiver* is not present, then
 - a. Set *receiver* to *target*.
4. Return ? *target*.[[Get]](*key*, *receiver*).

26.1.6 Reflect.getOwnPropertyDescriptor (*target*, *propertyKey*)

When the **getOwnPropertyDescriptor** function is called with arguments *target* and *propertyKey*, the following steps are taken:

1. If **Type**(*target*) is not Object, throw a **TypeError** exception.
2. Let *key* be ? **ToPropertyKey**(*propertyKey*).
3. Let *desc* be ? *target*.[[GetOwnProperty]](*key*).
4. Return **FromPropertyDescriptor**(*desc*).

26.1.7 Reflect.getPrototypeOf (*target*)

When the **getPrototypeOf** function is called with argument *target*, the following steps are taken:

1. If **Type**(*target*) is not Object, throw a **TypeError** exception.
2. Return ? *target*.[[GetPrototypeOf]]().

26.1.8 Reflect.has (*target*, *propertyKey*)

When the **has** function is called with arguments *target* and *propertyKey*, the following steps are taken:

1. If **Type**(*target*) is not Object, throw a **TypeError** exception.
2. Let *key* be ? **ToPropertyKey**(*propertyKey*).
3. Return ? *target*.**[[HasProperty]]**(*key*).

26.1.9 Reflect.isExtensible (*target*)

When the **isExtensible** function is called with argument *target*, the following steps are taken:

1. If **Type**(*target*) is not Object, throw a **TypeError** exception.
2. Return ? *target*.**[[IsExtensible]]**() .

26.1.10 Reflect.ownKeys (*target*)

When the **ownKeys** function is called with argument *target*, the following steps are taken:

1. If **Type**(*target*) is not Object, throw a **TypeError** exception.
2. Let *keys* be ? *target*.**[[OwnPropertyKeys]]**() .
3. Return **CreateArrayFromList**(*keys*).

26.1.11 Reflect.preventExtensions (*target*)

When the **preventExtensions** function is called with argument *target*, the following steps are taken:

1. If **Type**(*target*) is not Object, throw a **TypeError** exception.
2. Return ? *target*.**[[PreventExtensions]]**() .

26.1.12 Reflect.set (*target*, *propertyKey*, *V* [, *receiver*])

When the **set** function is called with arguments *target*, *V*, *propertyKey*, and *receiver*, the following steps are taken:

1. If **Type**(*target*) is not Object, throw a **TypeError** exception.
2. Let *key* be ? **ToPropertyKey**(*propertyKey*).
3. If *receiver* is not present, then
 - a. Set *receiver* to *target*.
4. Return ? *target*.**[[Set]]**(*key*, *V*, *receiver*).

26.1.13 Reflect.setPrototypeOf (*target*, *proto*)

When the **setPrototypeOf** function is called with arguments *target* and *proto*, the following steps are taken:

1. If **Type**(*target*) is not Object, throw a **TypeError** exception.
2. If **Type**(*proto*) is not Object and *proto* is not **null**, throw a **TypeError** exception.
3. Return ? *target*.**[[SetPrototypeOf]]**(*proto*).

26.2 Proxy Objects

26.2.1 The Proxy Constructor

The Proxy [constructor](#):

is the intrinsic object `%Proxy%`.

is the initial value of the **Proxy** property of the [global object](#).

creates and initializes a new proxy [exotic object](#) when called as a [constructor](#).

is not intended to be called as a function and will throw an exception when called in that manner.

26.2.1.1 Proxy (*target*, *handler*)

When **Proxy** is called with arguments *target* and *handler*, it performs the following steps:

1. If `NewTarget` is **undefined**, throw a **TypeError** exception.
2. Return ? [ProxyCreate](#)(*target*, *handler*).

26.2.2 Properties of the Proxy Constructor

The Proxy [constructor](#):

has a `[[Prototype]]` internal slot whose value is the intrinsic object `%FunctionPrototype%`.

does not have a **prototype** property because proxy exotic objects do not have a `[[Prototype]]` internal slot that requires initialization.

has the following properties:

26.2.2.1 Proxy.revocable (*target*, *handler*)

The **Proxy.revocable** function is used to create a revocable Proxy object. When **Proxy.revocable** is called with arguments *target* and *handler*, the following steps are taken:

1. Let *p* be ? [ProxyCreate](#)(*target*, *handler*).
2. Let *steps* be the algorithm steps defined in [Proxy Revocation Functions](#).
3. Let *revoker* be [CreateBuiltinFunction](#)(*steps*, « `[[RevocableProxy]]` »).
4. Set *revoker*.`[[RevocableProxy]]` to *p*.
5. Let *result* be [ObjectCreate](#)(`%ObjectPrototype%`).
6. Perform [CreateDataProperty](#)(*result*, **"proxy"**, *p*).
7. Perform [CreateDataProperty](#)(*result*, **"revoke"**, *revoker*).
8. Return *result*.

26.2.2.1.1 Proxy Revocation Functions

A Proxy revocation function is an anonymous function that has the ability to invalidate a specific Proxy object.

Each Proxy revocation function has a `[[RevocableProxy]]` internal slot.

When a Proxy revocation function is called, the following steps are taken:

1. Let *F* be the [active function object](#).
2. Let *p* be *F*.`[[RevocableProxy]]`.
3. If *p* is **null**, return **undefined**.
4. Set *F*.`[[RevocableProxy]]` to **null**.

5. **Assert:** *p* is a Proxy object.
6. Set *p*.[[ProxyTarget]] to **null**.
7. Set *p*.[[ProxyHandler]] to **null**.
8. Return **undefined**.

The "**length**" property of a Proxy revocation function is 0.

26.3 Module Namespace Objects

A Module Namespace Object is a module namespace [exotic object](#) that provides runtime property-based access to a module's exported bindings. There is no [constructor](#) function for Module Namespace Objects. Instead, such an object is created for each module that is imported by an *ImportDeclaration* that includes a *NamespaceImport*.

In addition to the properties specified in [9.4.6](#) each Module Namespace Object has the following own property:

26.3.1 @@toStringTag

The initial value of the @@toStringTag property is the String value "**Module**".

This property has the attributes { [[Writable]]: **false**, [[Enumerable]]: **false**, [[Configurable]]: **false** }.

27 Memory Model

The memory consistency model, or memory model, specifies the possible orderings of [Shared Data Block](#) events, arising via accessing TypedArray instances backed by a SharedArrayBuffer and via methods on the Atomics object. When the program has no data races (defined below), the ordering of events appears as sequentially consistent, i.e., as an interleaving of actions from each [agent](#). When the program has data races, shared memory operations may appear sequentially inconsistent. For example, programs may exhibit causality-violating behaviour and other astonishments. These astonishments arise from compiler transforms and the design of CPUs (e.g., out-of-order execution and speculation). The memory model defines both the precise conditions under which a program exhibits sequentially consistent behaviour as well as the possible values read from data races. To wit, there is no undefined behaviour.

The memory model is defined as relational constraints on events introduced by [abstract operations](#) on SharedArrayBuffer or by methods on the Atomics object during an evaluation.

NOTE

This section provides an axiomatic model on events introduced by the [abstract operations](#) on SharedArrayBuffers. It bears stressing that the model is not expressible algorithmically, unlike the rest of this specification. The nondeterministic introduction of events by [abstract operations](#) is the interface between the operational semantics of ECMAScript evaluation and the axiomatic semantics of the memory model. The semantics of these events is defined by considering graphs of all events in an evaluation. These are neither Static Semantics nor Runtime Semantics. There is no demonstrated algorithmic implementation, but instead a set of constraints that determine if a particular event graph is allowed or disallowed.

27.1 Memory Model Fundamentals

Shared memory accesses (reads and writes) are divided into two groups, atomic accesses and data accesses, defined below. Atomic accesses are sequentially consistent, i.e., there is a strict total ordering of events agreed upon by all agents in an [agent cluster](#). Non-atomic accesses do not have a strict total ordering agreed upon by all agents, i.e., unordered.

NOTE 1

No orderings weaker than sequentially consistent and stronger than unordered, such as release-acquire, are supported.

A Shared Data Block event is either a ReadSharedMemory, WriteSharedMemory, or ReadModifyWriteSharedMemory [Record](#).

Table 77: [ReadSharedMemory](#) Event Fields

Field Name	Value	Meaning
[[Order]]	" SeqCst " or " Unordered "	The weakest ordering guaranteed by the memory model for the event.
[[NoTear]]	A Boolean	Whether this event is allowed to read from multiple write events on equal range as this event.
[[Block]]	A Shared Data Block	The block the event operates on.
[[ByteIndex]]	A nonnegative integer	The byte address of the read in [[Block]].
[[ElementSize]]	A nonnegative integer	The size of the read.

Table 78: [WriteSharedMemory](#) Event Fields

Field Name	Value	Meaning
[[Order]]	" SeqCst ", " Unordered ", or " Init "	The weakest ordering guaranteed by the memory model for the event.
[[NoTear]]	A Boolean	Whether this event is allowed to be read from multiple read events with equal range as this event.
[[Block]]	A Shared Data Block	The block the event operates on.
[[ByteIndex]]	A nonnegative integer	The byte address of the write in [[Block]].
[[ElementSize]]	A nonnegative integer	The size of the write.
[[Payload]]	A List	The List of byte values to be read by other events.

Table 79: [ReadModifyWriteSharedMemory](#) Event Fields

Field Name	Value	Meaning
[[Order]]	" SeqCst "	Read-modify-write events are always sequentially consistent.
[[NoTear]]	true	Read-modify-write events cannot tear.

[[Block]]	A Shared Data Block	The block the event operates on.
[[ByteIndex]]	A nonnegative integer	The byte address of the read-modify-write in [[Block]].
[[ElementSize]]	A nonnegative integer	The size of the read-modify-write.
[[Payload]]	A List	The List of byte values to be passed to [[ModifyOp]].
[[ModifyOp]]	A semantic function	A pure semantic function that returns a modified List of byte values from a read List of byte values and [[Payload]].

These events are introduced by [abstract operations](#) or by methods on the Atomics object.

Some operations may also introduce Synchronize events. A Synchronize event has no fields, and exists purely to directly constrain the permitted orderings of other events.

In addition to [Shared Data Block](#) and Synchronize events, there are host-specific events.

Let the range of a ReadSharedMemory, WriteSharedMemory, or ReadModifyWriteSharedMemory event be the Set of contiguous integers from its [[ByteIndex]] to [[ByteIndex]] + [[ElementSize]] - 1. Two events' ranges are equal when the events have the same [[Block]], and the ranges are element-wise equal. Two events' ranges are overlapping when the events have the same [[Block]], the ranges are not equal and their intersection is non-empty. Two events' ranges are disjoint when the events do not have the same [[Block]] or their ranges are neither equal nor overlapping.

NOTE 2

Examples of host-specific synchronizing events that should be accounted for are: sending a SharedArrayBuffer from one [agent](#) to another (e.g., by **postMessage** in a browser), starting and stopping agents, and communicating within the [agent cluster](#) via channels other than shared memory. It is assumed those events are appended to [agent-order](#) during evaluation like the other SharedArrayBuffer events.

Events are ordered within candidate executions by the relations defined below.

27.2 Agent Events Records

An Agent Events Record is a [Record](#) with the following fields.

Table 80: [Agent Events Record](#) Fields

Field Name	Value	Meaning
[[AgentSignifier]]	A value that admits equality testing	The agent whose evaluation resulted in this ordering.
[[EventList]]	A List of events	Events are appended to the list during evaluation.
[[AgentSynchronizesWith]]	A List of pairs of Synchronize events	Synchronize relationships introduced by the operational semantics.

27.3 Chosen Value Records

A Chosen Value Record is a [Record](#) with the following fields.

Table 81: **Chosen Value Record** Fields

Field Name	Value	Meaning
[[Event]]	A Shared Data Block event	The ReadSharedMemory or ReadModifyWriteSharedMemory event that was introduced for this chosen value.
[[ChosenValue]]	A List of byte values	The bytes that were nondeterministically chosen during evaluation.

27.4 Candidate Executions

A candidate execution of the evaluation of an [agent cluster](#) is a [Record](#) with the following fields.

Table 82: Candidate Execution **Record** Fields

Field Name	Value	Meaning
[[EventsRecords]]	A List of Agent Events Records .	Maps an agent to Lists of events appended during the evaluation.
[[ChosenValues]]	A List of Chosen Value Records.	Maps ReadSharedMemory or ReadModifyWriteSharedMemory events to the List of byte values chosen during the evaluation.
[[AgentOrder]]	An agent-order Relation .	Defined below.
[[ReadsBytesFrom]]	A reads-bytes-from semantic function.	Defined below.
[[ReadsFrom]]	A reads-from Relation .	Defined below.
[[HostSynchronizesWith]]	A host-synchronizes-with Relation .	Defined below.
[[SynchronizesWith]]	A synchronizes-with Relation .	Defined below.
[[HappensBefore]]	A happens-before Relation .	Defined below.

An empty candidate execution is a candidate execution [Record](#) whose fields are empty Lists and Relations.

27.5 Abstract Operations for the Memory Model

27.5.1 EventSet (*execution*)

The abstract operation EventSet takes one argument, a *candidate execution execution*. It performs the following steps:

1. Let *events* be an empty Set.
2. For each *Agent Events Record aer* in *execution*.[[EventsRecords]], do
 - a. For each event *E* in *aer*.[[EventList]], do
 - i. Add *E* to *events*.
3. Return *events*.

27.5.2 SharedDataBlockEventSet (*execution*)

The abstract operation SharedDataBlockEventSet takes one argument, a *candidate execution execution*. It performs the following steps:

1. Let *events* be an empty Set.
2. For each event *E* in EventSet(*execution*), do
 - a. If *E* is a ReadSharedMemory, WriteSharedMemory, or ReadModifyWriteSharedMemory event, add *E* to *events*.
3. Return *events*.

27.5.3 SynchronizeEventSet (*execution*)

The abstract operation SynchronizeEventSet takes one argument, a *candidate execution execution*. It performs the following steps:

1. Let *events* be an empty Set.
2. For each event *E* in EventSet(*execution*), do
 - a. If *E* is a Synchronize event, add *E* to *events*.
3. Return *events*.

27.5.4 HostEventSet (*execution*)

The abstract operation HostEventSet takes one argument, a *candidate execution execution*. It performs the following steps:

1. Let *events* be an empty Set.
2. For each event *E* in EventSet(*execution*), do
 - a. If *E* is not in SharedDataBlockEventSet(*execution*), add *E* to *events*.
3. Return *events*.

27.5.5 ComposeWriteEventBytes (*execution*, *byteIndex*, *Ws*)

The abstract operation ComposeWriteEventBytes takes four arguments, a *candidate execution execution*, a nonnegative integer *byteIndex*, and a List *Ws* of WriteSharedMemory or ReadModifyWriteSharedMemory events. It performs the following steps:

1. Let *byteLocation* be *byteIndex*.
2. Let *bytesRead* be a new empty List.
3. For each element *W* of *Ws* in List order, do

- a. **Assert:** W has $byteLocation$ in its range.
 - b. Let $payloadIndex$ be $byteLocation - W.[[ByteIndex]]$.
 - c. If W is a **WriteSharedMemory** event, then
 - i. Let $byte$ be $W.[[Payload]][[payloadIndex]]$.
 - d. Else,
 - i. **Assert:** W is a **ReadModifyWriteSharedMemory** event.
 - ii. Let $bytes$ be $ValueOfReadEvent(execution, W)$.
 - iii. Let $bytesModified$ be $W.[[ModifyOp]](bytes, W.[[Payload]])$.
 - iv. Let $byte$ be $bytesModified[payloadIndex]$.
 - e. Append $byte$ to $bytesRead$.
 - f. Increment $byteLocation$ by 1.
4. Return $bytesRead$.

NOTE 1

The semantic function $[[ModifyOp]]$ is given by the function properties on the **Atomics** object that introduce **ReadModifyWriteSharedMemory** events.

NOTE 2

This abstract operation composes a **List** of write events into a **List** of byte values. It is used in the event semantics of **ReadSharedMemory** and **ReadModifyWriteSharedMemory** events.

27.5.6 ValueOfReadEvent ($execution, R$)

The abstract operation **ValueOfReadEvent** takes two arguments, a **candidate execution** $execution$ and a **ReadSharedMemory** or **ReadModifyWriteSharedMemory** event R . It performs the following steps:

1. **Assert:** R is a **ReadSharedMemory** or **ReadModifyWriteSharedMemory** event.
2. Let Ws be $execution.[[ReadsBytesFrom]](R)$.
3. **Assert:** Ws is a **List** of **WriteSharedMemory** or **ReadModifyWriteSharedMemory** events with length equal to $R.[[ElementSize]]$.
4. Return $ComposeWriteEventBytes(execution, R.[[ByteIndex]], Ws)$.

27.6 Relations of Candidate Executions

27.6.1 agent-order

For a **candidate execution** $execution$, $execution.[[AgentOrder]]$ is a **Relation** on events that satisfies the following.

For each pair (E, D) in $EventSet(execution)$, (E, D) is in $execution.[[AgentOrder]]$ if there is some **Agent Events Record** aer in $execution.[[EventsRecords]]$ such that E and D are in $aer.[[EventList]]$ and E is before D in **List** order of $aer.[[EventList]]$.

NOTE

Each **agent** introduces events in a **per-agent strict total order** during the evaluation. This is the union of those strict total orders.

27.6.2 reads-bytes-from

For a candidate execution *execution*, *execution*.[[ReadsBytesFrom]] is a semantic function from events in *SharedDataBlockEventSet*(*execution*) to Lists of events in *SharedDataBlockEventSet*(*execution*) that satisfies the following conditions.

For each *ReadSharedMemory* or *ReadModifyWriteSharedMemory* event *R* in *SharedDataBlockEventSet*(*execution*), *execution*.[[ReadsBytesFrom]](*R*) is a List of length equal to *R*.
[[ElementSize]] of *WriteSharedMemory* or *ReadModifyWriteSharedMemory* events *Ws* such that all of the following are true.

Each event *W* with index *i* in *Ws* has *R*.[[ByteIndex]] + *i* in its range.
R is not in *Ws*.

27.6.3 reads-from

For a candidate execution *execution*, *execution*.[[ReadsFrom]] is the least Relation on events that satisfies the following.

For each pair (*R*, *W*) in *SharedDataBlockEventSet*(*execution*), (*R*, *W*) is in *execution*.[[ReadsFrom]] if *W* is in *execution*.[[ReadsBytesFrom]](*R*).

27.6.4 host-synchronizes-with

For a candidate execution *execution*, *execution*.[[HostSynchronizesWith]] is a host-provided strict partial order on host-specific events that satisfies at least the following.

If (*E*, *D*) is in *execution*.[[HostSynchronizesWith]], *E* and *D* are in *HostEventSet*(*execution*).
There is no cycle in the union of *execution*.[[HostSynchronizesWith]] and *execution*.[[AgentOrder]].

NOTE 1

For two host-specific events *E* and *D*, *E* host-synchronizes-with *D* implies *E* happens-before *D*.

NOTE 2

The host-synchronizes-with relation allows the host to provide additional synchronization mechanisms, such as **postMessage** between HTML workers.

27.6.5 synchronizes-with

For a candidate execution *execution*, *execution*.[[SynchronizesWith]] is the least Relation on events that satisfies the following.

For each pair (*R*, *W*) in *execution*.[[ReadsFrom]], (*W*, *R*) is in *execution*.[[SynchronizesWith]] if all the following are true.

R.[[Order]] is "SeqCst".

W.[[Order]] is "SeqCst" or "Init".

If *W*.[[Order]] is "SeqCst", then *R* and *W* have equal ranges.

If *W*.[[Order]] is "Init", then for each event *V* such that (*R*, *V*) is in *execution*.[[ReadsFrom]], *V*.[[Order]] is "Init".

For each element *eventsRecord* of *execution*.[[EventsRecords]], the following is true.

For each pair (*S*, *Sw*) in *eventsRecord*.[[AgentSynchronizesWith]], (*S*, *Sw*) is in *execution*.

[[SynchronizesWith]].

For each pair (E, D) in *execution*.[[HostSynchronizesWith]], (E, D) is in *execution*.[[SynchronizesWith]].

NOTE 1

Owing to convention, write events synchronizes-with read events, instead of read events synchronizes-with write events.

NOTE 2

Not all "SeqCst" events related by reads-from are related by synchronizes-with. Only events that also have equal ranges are related by synchronizes-with.

NOTE 3

For Shared Data Block events R and W such that W synchronizes-with R , R may reads-from other writes than W .

27.6.6 happens-before

For a candidate execution *execution*, *execution*.[[HappensBefore]] is the least Relation on events that satisfies the following.

For each pair (E, D) in *execution*.[[AgentOrder]], (E, D) is in *execution*.[[HappensBefore]].

For each pair (E, D) in *execution*.[[SynchronizesWith]], (E, D) is in *execution*.[[HappensBefore]].

For each pair (E, D) in SharedDataBlockEventSet(*execution*), (E, D) is in *execution*.[[HappensBefore]] if E .[[Order]] is "Init" and E and D have overlapping ranges.

For each pair (E, D) in EventSet(*execution*), (E, D) is in *execution*.[[HappensBefore]] if there is an event F such that the pairs (E, F) and (F, D) are in *execution*.[[HappensBefore]].

NOTE

Because happens-before is a superset of agent-order, candidate executions are consistent with the single-thread evaluation semantics of ECMAScript.

27.7 Properties of Valid Executions

27.7.1 Valid Chosen Reads

A candidate execution *execution* has valid chosen reads if the following abstract operation returns **true**.

1. For each ReadSharedMemory or ReadModifyWriteSharedMemory event R in SharedDataBlockEventSet(*execution*), do
 - a. Let *chosenValueRecord* be the element of *execution*.[[ChosenValues]] whose [[Event]] field is R .
 - b. Let *chosenValue* be *chosenValueRecord*.[[ChosenValue]].
 - c. Let *readValue* be ValueOfReadEvent(*execution*, R).
 - d. Let *chosenLen* be the number of elements of *chosenValue*.
 - e. Let *readLen* be the number of elements of *readValue*.
 - f. If *chosenLen* is not equal to *readLen*, then
 - i. Return **false**.
 - g. If *chosenValue*[i] is not equal to *readValue*[i] for any integer value i in the range 0 through *chosenLen*, exclusive, then
 - i. Return **false**.
2. Return **true**.

27.7.2 Coherent Reads

A **candidate execution** *execution* has coherent reads if the following abstract operation returns **true**.

1. For each **ReadSharedMemory** or **ReadModifyWriteSharedMemory** event *R* in **SharedDataBlockEventSet**(*execution*), do
 - a. Let *Ws* be *execution*.[[ReadsBytesFrom]](*R*).
 - b. Let *byteLocation* be *R*.[[ByteIndex]].
 - c. For each element *W* of *Ws* in **List** order, do
 - i. If (*R*, *W*) is in *execution*.[[HappensBefore]], then
 1. Return **false**.
 - ii. If there is a **WriteSharedMemory** or **ReadModifyWriteSharedMemory** event *V* that has *byteLocation* in its range such that the pairs (*W*, *V*) and (*V*, *R*) are in *execution*.[[HappensBefore]], then
 1. Return **false**.
 - iii. Increment *byteLocation* by 1.
2. Return **true**.

27.7.3 Tear Free Reads

A **candidate execution** *execution* has tear free reads if the following abstract operation returns **true**.

1. For each **ReadSharedMemory** or **ReadModifyWriteSharedMemory** event *R* in **SharedDataBlockEventSet**(*execution*), do
 - a. If *R*.[[NoTear]] is **true**, then
 - i. **Assert**: The remainder of dividing *R*.[[ByteIndex]] by *R*.[[ElementSize]] is 0.
 - ii. For each event *W* such that (*R*, *W*) is in *execution*.[[ReadsFrom]] and *W*.[[NoTear]] is **true**, do
 1. If *R* and *W* have equal ranges, and there is an event *V* such that *V* and *W* have equal ranges, *V*.[[NoTear]] is **true**, *W* is not *V*, and (*R*, *V*) is in *execution*.[[ReadsFrom]], then
 - a. Return **false**.
2. Return **true**.

NOTE

An event's [[NoTear]] field is **true** when that event was introduced via accessing an integer TypedArray, and **false** when introduced via accessing a floating point TypedArray or DataView.

Intuitively, this requirement says when a memory range is accessed in an aligned fashion via an integer TypedArray, a single write event on that range must "win" when in a data race with other write events with equal ranges. More precisely, this requirement says an aligned read event cannot read a value composed of bytes from multiple, different write events all with equal ranges. It is possible, however, for an aligned read event to read from multiple write events with overlapping ranges.

27.7.4 Sequentially Consistent Atomics

For a **candidate execution** *execution*, memory-order is a **strict total order** of all events in **EventSet**(*execution*) that satisfies the following.

For each pair (*E*, *D*) in *execution*.[[HappensBefore]], (*E*, *D*) is in memory-order.

For each pair (*E*, *D*) in *execution*.[[SynchronizesWith]], (*E*, *D*) is in memory-order if there is no

`WriteSharedMemory` or `ReadModifyWriteSharedMemory` event W in `SharedDataBlockEventSet(execution)` with equal range as D such that W is not E , and the pairs (E, W) and (W, D) are in memory-order.

NOTE 1

This clause additionally constrains "**SeqCst**" events on equal ranges.

For each `WriteSharedMemory` or `ReadModifyWriteSharedMemory` event W in `SharedDataBlockEventSet(execution)`, if $W.[[Order]]$ is "**SeqCst**", then it is not the case that there is an infinite number of `ReadSharedMemory` or `ReadModifyWriteSharedMemory` events in `SharedDataBlockEventSet(execution)` with equal range that is memory-order before W .

NOTE 2

This clause together with the forward progress guarantee on agents ensure the liveness condition that "**SeqCst**" writes become visible to "**SeqCst**" reads with equal range in finite time.

A candidate execution has sequentially consistent atomics if a memory-order exists.

NOTE 3

While memory-order includes all events in `EventSet(execution)`, those that are not constrained by `happens-before` or `synchronizes-with` are allowed to occur anywhere in the order.

27.7.5 Valid Executions

A candidate execution $execution$ is a valid execution (or simply an execution) if all of the following are true.

The host provides a `host-synchronizes-with Relation` for $execution.[[HostSynchronizesWith]]$.

$execution.[[HappensBefore]]$ is a strict partial order.

$execution$ has valid chosen reads.

$execution$ has coherent reads.

$execution$ has tear free reads.

$execution$ has sequentially consistent atomics.

All programs have at least one valid execution.

27.8 Races

For an execution $execution$, two events E and D in `SharedDataBlockEventSet(execution)` are in a race if the following abstract operation returns **true**.

1. If E is not D , then
 - a. If the pairs (E, D) and (D, E) are not in $execution.[[HappensBefore]]$, then
 - i. If E and D are both `WriteSharedMemory` or `ReadModifyWriteSharedMemory` events and E and D do not have disjoint ranges, then
 1. Return **true**.
 - ii. If either (E, D) or (D, E) is in $execution.[[ReadsFrom]]$, then
 1. Return **true**.
2. Return **false**.

27.9 Data Races

For an execution *execution*, two events *E* and *D* in `SharedDataBlockEventSet(execution)` are in a data race if the following abstract operation returns **true**.

1. If *E* and *D* are in a race in *execution*, then
 - a. If *E*.[[Order]] is not "**SeqCst**" or *D*.[[Order]] is not "**SeqCst**", then
 - i. Return **true**.
 - b. If *E* and *D* have overlapping ranges, then
 - i. Return **true**.
2. Return **false**.

27.10 Data Race Freedom

An execution *execution* is data race free if there are no two events in `SharedDataBlockEventSet(execution)` that are in a data race.

A program is data race free if all its executions are data race free.

The [memory model](#) guarantees sequential consistency of all events for data race free programs.

27.11 Shared Memory Guidelines

NOTE 1

The following are guidelines for ECMAScript programmers working with shared memory.

We recommend programs be kept data race free, i.e., make it so that it is impossible for there to be concurrent non-atomic operations on the same memory location. Data race free programs have interleaving semantics where each step in the evaluation semantics of each [agent](#) are interleaved with each other. For data race free programs, it is not necessary to understand the details of the [memory model](#). The details are unlikely to build intuition that will help one to better write ECMAScript.

More generally, even if a program is not data race free it may have predictable behaviour, so long as atomic operations are not involved in any data races and the operations that race all have the same access size. The simplest way to arrange for atomics not to be involved in races is to ensure that different memory cells are used by atomic and non-atomic operations and that atomic accesses of different sizes are not used to access the same cells at the same time. Effectively, the program should treat shared memory as strongly typed as much as possible. One still cannot depend on the ordering and timing of non-atomic accesses that race, but if memory is treated as strongly typed the racing accesses will not "tear" (bits of their values will not be mixed).

NOTE 2

The following are guidelines for ECMAScript implementers writing compiler transformations for programs using shared memory.

It is desirable to allow most program transformations that are valid in a single-[agent](#) setting in a multi-[agent](#) setting, to ensure that the performance of each [agent](#) in a multi-[agent](#) program is as good as it would be in a single-[agent](#) setting. Frequently these transformations are hard to judge. We outline some rules about program transformations that are intended to be taken as normative (in that they are implied by the [memory model](#) or stronger than what the [memory model](#) implies) but which are likely not exhaustive. These rules are intended to apply to program transformations that precede the introductions of the events that make up the [agent-order](#).

Let an agent-order slice be the subset of the [agent-order](#) pertaining to a single [agent](#).

Let possible read values of a read event be the set of all values of [ValueOfReadEvent](#) for that event across all valid executions.

Any transformation of an agent-order slice that is valid in the absence of shared memory is valid in the presence of shared memory, with the following exceptions.

Atomics are carved in stone: Program transformations must not cause the "**SeqCst**" events in an agent-order slice to be reordered with its "**Unordered**" operations, nor its "**SeqCst**" operations to be reordered with each other, nor may a program transformation remove a "**SeqCst**" operation from the [agent-order](#).

(In practice, the prohibition on reorderings forces a compiler to assume that every "**SeqCst**" operation is a synchronization and included in the final [memory-order](#), which it would usually have to assume anyway in the absence of inter-[agent](#) program analysis. It also forces the compiler to assume that every call where the callee's effects on the [memory-order](#) are unknown may contain "**SeqCst**" operations.)

Reads must be stable: Any given shared memory read must only observe a single value in an execution.

(For example, if what is semantically a single read in the program is executed multiple times then the program is subsequently allowed to observe only one of the values read. A transformation known as rematerialization can violate this rule.)

Writes must be stable: All observable writes to shared memory must follow from program semantics in an execution.

(For example, a transformation may not introduce certain observable writes, such as by using read-modify-write operations on a larger location to write a smaller datum, writing a value to memory that the program could not have written, or writing a just-read value back to the location it was read from, if that location could have been overwritten by another [agent](#) after the read.)

Possible read values must be nonempty: Program transformations cannot cause the possible read values of a shared memory read to become empty.

(Counterintuitively, this rule in effect restricts transformations on writes, because writes have force in [memory model](#) insofar as to be read by read events. For example, writes may be moved and coalesced and sometimes reordered between two "**SeqCst**" operations, but the transformation may not remove every write that updates a location; some write must be preserved.)

Examples of transformations that remain valid are: merging multiple non-atomic reads from the same location, reordering non-atomic reads, introducing speculative non-atomic reads, merging multiple non-atomic writes to the same location, reordering non-atomic writes to different locations, and hoisting non-atomic reads out of loops even if that affects termination. Note in general that aliased TypedArrays make it hard to prove that locations are different.

NOTE 3

The following are guidelines for ECMAScript implementers generating machine code for shared memory accesses.

For architectures with memory models no weaker than those of ARM or Power, non-atomic stores and loads may be compiled to bare stores and loads on the target architecture. Atomic stores and loads may be compiled down to instructions that guarantee sequential consistency. If no such instructions exist, memory barriers are to be employed, such as placing barriers on both sides of a bare store or load. Read-modify-write operations may be compiled to read-modify-write instructions on the target architecture, such as **LOCK**-prefixed instructions on x86, load-exclusive/store-exclusive instructions on ARM, and load-link/store-conditional instructions on Power.

Specifically, the [memory model](#) is intended to allow code generation as follows.

Every atomic operation in the program is assumed to be necessary.

Atomic operations are never rearranged with each other or with non-atomic operations.

Functions are always assumed to perform atomic operations.

Atomic operations are never implemented as read-modify-write operations on larger data, but as non-lock-free atomics if the platform does not have atomic operations of the appropriate size. (We already assume that every platform has normal memory access operations of every interesting size.)

Naive code generation uses these patterns:

Regular loads and stores compile to single load and store instructions.

Lock-free atomic loads and stores compile to a full (sequentially consistent) fence, a regular load or store, and a full fence.

Lock-free atomic read-modify-write accesses compile to a full fence, an atomic read-modify-write instruction sequence, and a full fence.

Non-lock-free atomics compile to a spinlock acquire, a full fence, a series of non-atomic load and store instructions, a full fence, and a spinlock release.

That mapping is correct so long as an atomic operation on an address range does not race with a non-atomic write or with an atomic operation of different size. However, that is all we need: the [memory model](#) effectively demotes the atomic operations involved in a race to non-atomic status. On the other hand, the naive mapping is quite strong: it allows atomic operations to be used as sequentially consistent fences, which the [memory model](#) does not actually guarantee.

A number of local improvements to those basic patterns are also intended to be legal:

There are obvious platform-dependent improvements that remove redundant fences. For example, on x86 the fences around lock-free atomic loads and stores can always be omitted except for the fence following a store, and no fence is needed for lock-free read-modify-write instructions, as these all use LOCK-prefixed instructions. On many platforms there are fences of several strengths, and weaker fences can be used in certain contexts without destroying sequential consistency.

Most modern platforms support lock-free atomics for all the data sizes required by ECMAScript atomics. Should non-lock-free atomics be needed, the fences surrounding the body of the atomic operation can usually be folded into the lock and unlock steps. The simplest solution for non-lock-free atomics is to have a single lock word per `SharedArrayBuffer`.

There are also more complicated platform-dependent local improvements, requiring some code analysis. For example, two back-to-back fences often have the same effect as a single fence, so if code is generated for two atomic operations in sequence, only a single fence need separate them. On x86, even a single fence separating atomic stores can be omitted, as the fence following a store is only needed to separate the store from a subsequent load.

A Grammar Summary

A.1 Lexical Grammar

SourceCharacter ::

any Unicode code point

InputElementDiv ::

WhiteSpace

LineTerminator

Comment

CommonToken

DivPunctuator

RightBracePunctuator

InputElementRegExp ::

WhiteSpace

LineTerminator

Comment

CommonToken

RightBracePunctuator

RegularExpressionLiteral

InputElementRegExpOrTemplateTail ::

WhiteSpace

LineTerminator

Comment

CommonToken

RegularExpressionLiteral

TemplateSubstitutionTail

InputElementTemplateTail ::

WhiteSpace

LineTerminator

Comment

CommonToken

DivPunctuator

TemplateSubstitutionTail

WhiteSpace ::

<TAB>

<VT>

<FF>

<SP>

<NBSP>

<ZWNBSP>

<USP>

LineTerminator ::

<LF>

<CR>

<LS>

<PS>

LineTerminatorSequence ::

<LF>
<CR>[lookahead ≠ <LF>]
<LS>
<PS>
<CR><LF>

Comment ::
 MultiLineComment
 SingleLineComment

MultiLineComment ::
 /* *MultiLineCommentChars*_{opt} */

MultiLineCommentChars ::
 MultiLineNotAsteriskChar *MultiLineCommentChars*_{opt}
 * *PostAsteriskCommentChars*_{opt}

PostAsteriskCommentChars ::
 MultiLineNotForwardSlashOrAsteriskChar *MultiLineCommentChars*_{opt}
 * *PostAsteriskCommentChars*_{opt}

MultiLineNotAsteriskChar ::
 SourceCharacter but not *

MultiLineNotForwardSlashOrAsteriskChar ::
 SourceCharacter but not one of / or *

SingleLineComment ::
 // *SingleLineCommentChars*_{opt}

SingleLineCommentChars ::
 SingleLineCommentChar *SingleLineCommentChars*_{opt}

SingleLineCommentChar ::
 SourceCharacter but not *LineTerminator*

CommonToken ::
 IdentifierName
 Punctuator
 NumericLiteral
 StringLiteral
 Template

IdentifierName ::
 IdentifierStart
 IdentifierName *IdentifierPart*

IdentifierStart ::
 UnicodeIDStart
 \$
 —

\ UnicodeEscapeSequence

IdentifierPart ::

UnicodeIDContinue

\$

\ UnicodeEscapeSequence

<ZWJ>

<ZWJ>

UnicodeIDStart ::

any Unicode code point with the Unicode property “ID_Start”

UnicodeIDContinue ::

any Unicode code point with the Unicode property “ID_Continue”

ReservedWord ::

Keyword

FutureReservedWord

NullLiteral

BooleanLiteral

Keyword :: **one of**

**await break case catch class const continue debugger default delete do
else export extends finally for function if import in instanceof new
return super switch this throw try typeof var void while with yield**

FutureReservedWord ::

enum

The following tokens are also considered to be *FutureReservedWords* when parsing [strict mode code](#):

implements package protected

interface private public

Punctuator :: **one of**

**{ () [] . . . ; , < > <= >= == != === !== + - * % ** ++ -- << >> >>> & | ^ ! ~
&& || ? : = += -= *= %= **= <<= >>= >>>= &= |= ^= =>**

DivPunctuator ::

/

/=

RightBracePunctuator ::

}

NullLiteral ::

null

BooleanLiteral ::

true

false

NumericLiteral ::

DecimalLiteral

BinaryIntegerLiteral

OctalIntegerLiteral

HexIntegerLiteral

DecimalLiteral ::

DecimalIntegerLiteral . *DecimalDigits*_{opt} *ExponentPart*_{opt}

. *DecimalDigits* *ExponentPart*_{opt}

DecimalIntegerLiteral *ExponentPart*_{opt}

DecimalIntegerLiteral ::

0

NonZeroDigit *DecimalDigits*_{opt}

DecimalDigits ::

DecimalDigit

DecimalDigits *DecimalDigit*

DecimalDigit :: **one of**

0 1 2 3 4 5 6 7 8 9

NonZeroDigit :: **one of**

1 2 3 4 5 6 7 8 9

ExponentPart ::

ExponentIndicator *SignedInteger*

ExponentIndicator :: **one of**

e E

SignedInteger ::

DecimalDigits

+ *DecimalDigits*

- *DecimalDigits*

BinaryIntegerLiteral ::

0b *BinaryDigits*

0B *BinaryDigits*

BinaryDigits ::

BinaryDigit

BinaryDigits *BinaryDigit*

BinaryDigit :: **one of**

0 1

OctalIntegerLiteral ::

0o *OctalDigits*

00 *OctalDigits*

OctalDigits ::

OctalDigit

OctalDigits OctalDigit

OctalDigit :: **one of**

0 1 2 3 4 5 6 7

HexIntegerLiteral ::

0x *HexDigits*

0X *HexDigits*

HexDigits ::

HexDigit

HexDigits HexDigit

HexDigit :: **one of**

0 1 2 3 4 5 6 7 8 9 a b c d e f A B C D E F

StringLiteral ::

" *DoubleStringCharacters*_{opt} "

' *SingleStringCharacters*_{opt} '

DoubleStringCharacters ::

*DoubleStringCharacter DoubleStringCharacters*_{opt}

SingleStringCharacters ::

*SingleStringCharacter SingleStringCharacters*_{opt}

DoubleStringCharacter ::

SourceCharacter but not one of " or \ or *LineTerminator*

<LS>

<PS>

\ *EscapeSequence*

LineContinuation

SingleStringCharacter ::

SourceCharacter but not one of ' or \ or *LineTerminator*

<LS>

<PS>

\ *EscapeSequence*

LineContinuation

LineContinuation ::

\ *LineTerminatorSequence*

EscapeSequence ::

CharacterEscapeSequence

0 [lookahead $\neq$ *DecimalDigit*]

HexEscapeSequence

UnicodeEscapeSequence

CharacterEscapeSequence ::

SingleEscapeCharacter

NonEscapeCharacter

SingleEscapeCharacter :: **one of**

' " \ b f n r t v

NonEscapeCharacter ::

SourceCharacter but not one of *EscapeCharacter* or *LineTerminator*

EscapeCharacter ::

SingleEscapeCharacter

DecimalDigit

x

u

HexEscapeSequence ::

x *HexDigit* *HexDigit*

UnicodeEscapeSequence ::

u *Hex4Digits*

u{ *CodePoint* }

Hex4Digits ::

HexDigit *HexDigit* *HexDigit* *HexDigit*

RegularExpressionLiteral ::

/ *RegularExpressionBody* */* *RegularExpressionFlags*

RegularExpressionBody ::

RegularExpressionFirstChar *RegularExpressionChars*

RegularExpressionChars ::

[empty]

RegularExpressionChars *RegularExpressionChar*

RegularExpressionFirstChar ::

RegularExpressionNonTerminator but not one of ***** or **** or **/** or **[**

RegularExpressionBackslashSequence

RegularExpressionClass

RegularExpressionChar ::

RegularExpressionNonTerminator but not one of **** or **/** or **[**

RegularExpressionBackslashSequence

RegularExpressionClass

RegularExpressionBackslashSequence ::

**** *RegularExpressionNonTerminator*

RegularExpressionNonTerminator ::

SourceCharacter but not *LineTerminator*

RegularExpressionClass ::

[*RegularExpressionClassChars*]

RegularExpressionClassChars ::

[empty]

RegularExpressionClassChars *RegularExpressionClassChar*

RegularExpressionClassChar ::

RegularExpressionNonTerminator but not one of] or \

RegularExpressionBackslashSequence

RegularExpressionFlags ::

[empty]

RegularExpressionFlags *IdentifierPart*

Template ::

NoSubstitutionTemplate

TemplateHead

NoSubstitutionTemplate ::

` *TemplateCharacters*<sub>opt</sub> `

TemplateHead ::

` *TemplateCharacters*_{opt} \${

TemplateSubstitutionTail ::

TemplateMiddle

TemplateTail

TemplateMiddle ::

} *TemplateCharacters*_{opt} \${

TemplateTail ::

} *TemplateCharacters*_{opt} `

TemplateCharacters ::

TemplateCharacter *TemplateCharacters*_{opt}

TemplateCharacter ::

\$ [lookahead ≠ {]

\ *EscapeSequence*

\ *NotEscapeSequence*

LineContinuation

LineTerminatorSequence

SourceCharacter but not one of ` or \ or \$ or *LineTerminator*

NotEscapeSequence ::

0 *DecimalDigit*

DecimalDigit but not 0

- x** [lookahead $\notin$ *HexDigit*]
- x** *HexDigit* [lookahead $\notin$ *HexDigit*]
- u** [lookahead $\notin$ *HexDigit*] [lookahead $\neq$ {]
- u** *HexDigit* [lookahead $\notin$ *HexDigit*]
- u** *HexDigit* *HexDigit* [lookahead $\notin$ *HexDigit*]
- u** *HexDigit* *HexDigit* *HexDigit* [lookahead $\notin$ *HexDigit*]
- u** { [lookahead $\notin$ *HexDigit*]
- u** { *NotCodePoint* [lookahead $\notin$ *HexDigit*]
- u** { *CodePoint* [lookahead $\notin$ *HexDigit*] [lookahead $\neq$ {]

NotCodePoint ::
HexDigits but only if MV of *HexDigits* > 0x10FFFF

CodePoint ::
HexDigits but only if MV of *HexDigits* ≤ 0x10FFFF

A.2 Expressions

*IdentifierReference*_[Yield, Await] :
Identifier
 [~Yield] **yield**
 [~Await] **await**

*BindingIdentifier*_[Yield, Await] :
Identifier
yield
await

Identifier :
IdentifierName but not *ReservedWord*

*AsyncArrowBindingIdentifier*_[Yield] :
*BindingIdentifier*_[?Yield, +Await]

*LabelIdentifier*_[Yield, Await] :
Identifier
 [~Yield] **yield**
 [~Await] **await**

*PrimaryExpression*_[Yield, Await] :
this
*IdentifierReference*_[?Yield, ?Await]
Literal
*ArrayLiteral*_[?Yield, ?Await]
*ObjectLiteral*_[?Yield, ?Await]
FunctionExpression
*ClassExpression*_[?Yield, ?Await]
GeneratorExpression

AsyncFunctionExpression
AsyncGeneratorExpression
RegularExpressionLiteral
*TemplateLiteral*_[?Yield, ?Await, ~Tagged]
*CoverParenthesizedExpressionAndArrowParameterList*_[?Yield, ?Await]

*CoverParenthesizedExpressionAndArrowParameterList*_[Yield, Await] :
 (*Expression*_[+In, ?Yield, ?Await])
 (*Expression*_[+In, ?Yield, ?Await] ,)
 ()
 (... *BindingIdentifier*_[?Yield, ?Await])
 (... *BindingPattern*_[?Yield, ?Await])
 (*Expression*_[+In, ?Yield, ?Await] , ... *BindingIdentifier*_[?Yield, ?Await])
 (*Expression*_[+In, ?Yield, ?Await] , ... *BindingPattern*_[?Yield, ?Await])

When processing an instance of the production *PrimaryExpression* :

CoverParenthesizedExpressionAndArrowParameterList the interpretation of
CoverParenthesizedExpressionAndArrowParameterList is refined using the following grammar:

*ParenthesizedExpression*_[Yield, Await] :
 (*Expression*_[+In, ?Yield, ?Await])

Literal :
NullLiteral
BooleanLiteral
NumericLiteral
StringLiteral

*ArrayLiteral*_[Yield, Await] :
 [*Elision*_{opt}]
 [*ElementList*_[?Yield, ?Await]]
 [*ElementList*_[?Yield, ?Await] , *Elision*_{opt}]

*ElementList*_[Yield, Await] :
*Elision*_{opt} *AssignmentExpression*_[+In, ?Yield, ?Await]
*Elision*_{opt} *SpreadElement*_[?Yield, ?Await]
*ElementList*_[?Yield, ?Await] , *Elision*_{opt} *AssignmentExpression*_[+In, ?Yield, ?Await]
*ElementList*_[?Yield, ?Await] , *Elision*_{opt} *SpreadElement*_[?Yield, ?Await]

Elision :
 ,
Elision ,

*SpreadElement*_[Yield, Await] :
 ... *AssignmentExpression*_[+In, ?Yield, ?Await]

*ObjectLiteral*_[Yield, Await] :

```

{ }
{ PropertyDefinitionList[?Yield, ?Await] }
{ PropertyDefinitionList[?Yield, ?Await] , }

```

```

PropertyDefinitionList[Yield, Await] :
    PropertyDefinition[?Yield, ?Await]
    PropertyDefinitionList[?Yield, ?Await] , PropertyDefinition[?Yield, ?Await]

```

```

PropertyDefinition[Yield, Await] :
    IdentifierReference[?Yield, ?Await]
    CoverInitializedName[?Yield, ?Await]
   PropertyName[?Yield, ?Await] : AssignmentExpression[+In, ?Yield, ?Await]
    MethodDefinition[?Yield, ?Await]
    ... AssignmentExpression[+In, ?Yield, ?Await]

```

```

PropertyName[Yield, Await] :
    LiteralPropertyName
    ComputedPropertyName[?Yield, ?Await]

```

```

LiteralPropertyName :
    IdentifierName
    StringLiteral
    NumericLiteral

```

```

ComputedPropertyName[Yield, Await] :
    [ AssignmentExpression[+In, ?Yield, ?Await] ]

```

```

CoverInitializedName[Yield, Await] :
    IdentifierReference[?Yield, ?Await] Initializer[+In, ?Yield, ?Await]

```

```

Initializer[In, Yield, Await] :
    = AssignmentExpression[?In, ?Yield, ?Await]

```

```

TemplateLiteral[Yield, Await, Tagged] :
    NoSubstitutionTemplate
    SubstitutionTemplate[?Yield, ?Await, ?Tagged]

```

```

SubstitutionTemplate[Yield, Await, Tagged] :
    TemplateHead Expression[+In, ?Yield, ?Await] TemplateSpans[?Yield, ?Await, ?Tagged]

```

```

TemplateSpans[Yield, Await, Tagged] :
    TemplateTail
    TemplateMiddleList[?Yield, ?Await, ?Tagged] TemplateTail

```

```

TemplateMiddleList[Yield, Await, Tagged] :
    TemplateMiddle Expression[+In, ?Yield, ?Await]
    TemplateMiddleList[?Yield, ?Await, ?Tagged] TemplateMiddle Expression[+In, ?Yield, ?Await]

```

```

MemberExpression[Yield, Await] :

```

```

PrimaryExpression[?Yield, ?Await]
MemberExpression[?Yield, ?Await] [ Expression[+In, ?Yield, ?Await] ]
MemberExpression[?Yield, ?Await] . IdentifierName
MemberExpression[?Yield, ?Await] TemplateLiteral[?Yield, ?Await, +Tagged]
SuperProperty[?Yield, ?Await]
MetaProperty
new MemberExpression[?Yield, ?Await] Arguments[?Yield, ?Await]

```

```

SuperProperty[Yield, Await] :
    super [ Expression[+In, ?Yield, ?Await] ]
    super . IdentifierName

```

```

MetaProperty :
    NewTarget

```

```

NewTarget :
    new . target

```

```

NewExpression[Yield, Await] :
    MemberExpression[?Yield, ?Await]
    new NewExpression[?Yield, ?Await]

```

```

CallExpression[Yield, Await] :
    CoverCallExpressionAndAsyncArrowHead[?Yield, ?Await]
    SuperCall[?Yield, ?Await]
    CallExpression[?Yield, ?Await] Arguments[?Yield, ?Await]
    CallExpression[?Yield, ?Await] [ Expression[+In, ?Yield, ?Await] ]
    CallExpression[?Yield, ?Await] . IdentifierName
    CallExpression[?Yield, ?Await] TemplateLiteral[?Yield, ?Await, +Tagged]

```

```

CoverCallExpressionAndAsyncArrowHead[Yield, Await] :
    MemberExpression[?Yield, ?Await] Arguments[?Yield, ?Await]

```

When processing an instance of the production *CallExpression* : *CoverCallExpressionAndAsyncArrowHead* the interpretation of *CoverCallExpressionAndAsyncArrowHead* is refined using the following grammar:

```

CallMemberExpression[Yield, Await] :
    MemberExpression[?Yield, ?Await] Arguments[?Yield, ?Await]

```

```

SuperCall[Yield, Await] :
    super Arguments[?Yield, ?Await]

```

```

Arguments[Yield, Await] :
    ( )
    ( ArgumentList[?Yield, ?Await] )
    ( ArgumentList[?Yield, ?Await] , )

```

*ArgumentList*_[Yield, Await] :

- AssignmentExpression*_[+In, ?Yield, ?Await]
- ... *AssignmentExpression*_[+In, ?Yield, ?Await]
- ArgumentList*_[?Yield, ?Await] , *AssignmentExpression*_[+In, ?Yield, ?Await]
- ArgumentList*_[?Yield, ?Await] , ... *AssignmentExpression*_[+In, ?Yield, ?Await]

*LeftHandSideExpression*_[Yield, Await] :

- NewExpression*_[?Yield, ?Await]
- CallExpression*_[?Yield, ?Await]

*UpdateExpression*_[Yield, Await] :

- LeftHandSideExpression*_[?Yield, ?Await]
- LeftHandSideExpression*_[?Yield, ?Await] [no *LineTerminator* here] ++
- LeftHandSideExpression*_[?Yield, ?Await] [no *LineTerminator* here] --
- ++ *UnaryExpression*_[?Yield, ?Await]
- *UnaryExpression*_[?Yield, ?Await]

*UnaryExpression*_[Yield, Await] :

- UpdateExpression*_[?Yield, ?Await]
- delete** *UnaryExpression*_[?Yield, ?Await]
- void** *UnaryExpression*_[?Yield, ?Await]
- typeof** *UnaryExpression*_[?Yield, ?Await]
- + *UnaryExpression*_[?Yield, ?Await]
- *UnaryExpression*_[?Yield, ?Await]
- ~ *UnaryExpression*_[?Yield, ?Await]
- ! *UnaryExpression*_[?Yield, ?Await]
- _[+Await] *AwaitExpression*_[?Yield]

*ExponentiationExpression*_[Yield, Await] :

- UnaryExpression*_[?Yield, ?Await]
- UpdateExpression*_[?Yield, ?Await] ** *ExponentiationExpression*_[?Yield, ?Await]

*MultiplicativeExpression*_[Yield, Await] :

- ExponentiationExpression*_[?Yield, ?Await]
- MultiplicativeExpression*_[?Yield, ?Await] *MultiplicativeOperator*
- ExponentiationExpression*_[?Yield, ?Await]

MultiplicativeOperator : **one of**

- * / %

*AdditiveExpression*_[Yield, Await] :

- MultiplicativeExpression*_[?Yield, ?Await]
- AdditiveExpression*_[?Yield, ?Await] + *MultiplicativeExpression*_[?Yield, ?Await]
- AdditiveExpression*_[?Yield, ?Await] - *MultiplicativeExpression*_[?Yield, ?Await]

*ShiftExpression*_[Yield, Await] :

```

AdditiveExpression[?Yield, ?Await]
ShiftExpression[?Yield, ?Await] << AdditiveExpression[?Yield, ?Await]
ShiftExpression[?Yield, ?Await] >> AdditiveExpression[?Yield, ?Await]
ShiftExpression[?Yield, ?Await] >>> AdditiveExpression[?Yield, ?Await]

```

RelationalExpression[In, Yield, Await] :

```

ShiftExpression[?Yield, ?Await]
RelationalExpression[?In, ?Yield, ?Await] < ShiftExpression[?Yield, ?Await]
RelationalExpression[?In, ?Yield, ?Await] > ShiftExpression[?Yield, ?Await]
RelationalExpression[?In, ?Yield, ?Await] <= ShiftExpression[?Yield, ?Await]
RelationalExpression[?In, ?Yield, ?Await] >= ShiftExpression[?Yield, ?Await]
RelationalExpression[?In, ?Yield, ?Await] instanceof ShiftExpression[?Yield, ?Await]
[+In] RelationalExpression[+In, ?Yield, ?Await] in ShiftExpression[?Yield, ?Await]

```

EqualityExpression[In, Yield, Await] :

```

RelationalExpression[?In, ?Yield, ?Await]
EqualityExpression[?In, ?Yield, ?Await] == RelationalExpression[?In, ?Yield, ?Await]
EqualityExpression[?In, ?Yield, ?Await] != RelationalExpression[?In, ?Yield, ?Await]
EqualityExpression[?In, ?Yield, ?Await] === RelationalExpression[?In, ?Yield, ?Await]
EqualityExpression[?In, ?Yield, ?Await] !== RelationalExpression[?In, ?Yield, ?Await]

```

BitwiseANDExpression[In, Yield, Await] :

```

EqualityExpression[?In, ?Yield, ?Await]
BitwiseANDExpression[?In, ?Yield, ?Await] & EqualityExpression[?In, ?Yield, ?Await]

```

BitwiseXORExpression[In, Yield, Await] :

```

BitwiseANDExpression[?In, ?Yield, ?Await]
BitwiseXORExpression[?In, ?Yield, ?Await] ^ BitwiseANDExpression[?In, ?Yield, ?Await]

```

BitwiseORExpression[In, Yield, Await] :

```

BitwiseXORExpression[?In, ?Yield, ?Await]
BitwiseORExpression[?In, ?Yield, ?Await] | BitwiseXORExpression[?In, ?Yield, ?Await]

```

LogicalANDExpression[In, Yield, Await] :

```

BitwiseORExpression[?In, ?Yield, ?Await]
LogicalANDExpression[?In, ?Yield, ?Await] && BitwiseORExpression[?In, ?Yield, ?Await]

```

LogicalORExpression[In, Yield, Await] :

```

LogicalANDExpression[?In, ?Yield, ?Await]
LogicalORExpression[?In, ?Yield, ?Await] || LogicalANDExpression[?In, ?Yield, ?Await]

```

ConditionalExpression[In, Yield, Await] :

```

LogicalORExpression[?In, ?Yield, ?Await]
LogicalORExpression[?In, ?Yield, ?Await] ? AssignmentExpression[+In, ?Yield, ?Await] :
AssignmentExpression[?In, ?Yield, ?Await]

```

AssignmentExpression[In, Yield, Await] :

```

ConditionalExpression[?In, ?Yield, ?Await]
[+Yield] YieldExpression[?In, ?Await]
ArrowFunction[?In, ?Yield, ?Await]
AsyncArrowFunction[?In, ?Yield, ?Await]
LeftHandSideExpression[?Yield, ?Await] = AssignmentExpression[?In, ?Yield, ?Await]
LeftHandSideExpression[?Yield, ?Await] AssignmentOperator
AssignmentExpression[?In, ?Yield, ?Await]

```

In certain circumstances when processing an instance of the production *AssignmentExpression* :

LeftHandSideExpression = *AssignmentExpression* the following grammar is used to refine the interpretation of *LeftHandSideExpression*:

```

AssignmentPattern[Yield, Await] :
    ObjectAssignmentPattern[?Yield, ?Await]
    ArrayAssignmentPattern[?Yield, ?Await]

ObjectAssignmentPattern[Yield, Await] :
    { }
    { AssignmentRestProperty[?Yield, ?Await] }
    { AssignmentPropertyList[?Yield, ?Await] }
    { AssignmentPropertyList[?Yield, ?Await] , AssignmentRestProperty[?Yield, ?Await] opt }

ArrayAssignmentPattern[Yield, Await] :
    [ Elisionopt AssignmentRestElement[?Yield, ?Await] opt ]
    [ AssignmentElementList[?Yield, ?Await] ]
    [ AssignmentElementList[?Yield, ?Await] , Elisionopt
      AssignmentRestElement[?Yield, ?Await] opt ]

AssignmentRestProperty[Yield, Await] :
    ... DestructuringAssignmentTarget[?Yield, ?Await]

AssignmentPropertyList[Yield, Await] :
    AssignmentProperty[?Yield, ?Await]
    AssignmentPropertyList[?Yield, ?Await] , AssignmentProperty[?Yield, ?Await]

AssignmentElementList[Yield, Await] :
    AssignmentElisionElement[?Yield, ?Await]
    AssignmentElementList[?Yield, ?Await] , AssignmentElisionElement[?Yield, ?Await]

AssignmentElisionElement[Yield, Await] :
    Elisionopt AssignmentElement[?Yield, ?Await]

AssignmentProperty[Yield, Await] :
    IdentifierReference[?Yield, ?Await] Initializer[+In, ?Yield, ?Await] opt
   PropertyName[?Yield, ?Await] : AssignmentElement[?Yield, ?Await]

AssignmentElement[Yield, Await] :

```


*DestructuringAssignmentTarget*_[?Yield, ?Await] *Initializer*_[+In, ?Yield, ?Await] **opt**

*AssignmentRestElement*_[Yield, Await] :
... *DestructuringAssignmentTarget*_[?Yield, ?Await]

*DestructuringAssignmentTarget*_[Yield, Await] :
*LeftHandSideExpression*_[?Yield, ?Await]

AssignmentOperator : **one of**
***= /= %= += -= <=> >>= &= ^= |= **=**

*Expression*_[In, Yield, Await] :
*AssignmentExpression*_[?In, ?Yield, ?Await]
*Expression*_[?In, ?Yield, ?Await] , *AssignmentExpression*_[?In, ?Yield, ?Await]

A.3 Statements

*Statement*_[Yield, Await, Return] :
*BlockStatement*_[?Yield, ?Await, ?Return]
*VariableStatement*_[?Yield, ?Await]
EmptyStatement
*ExpressionStatement*_[?Yield, ?Await]
*IfStatement*_[?Yield, ?Await, ?Return]
*BreakableStatement*_[?Yield, ?Await, ?Return]
*ContinueStatement*_[?Yield, ?Await]
*BreakStatement*_[?Yield, ?Await]
[+Return] *ReturnStatement*_[?Yield, ?Await]
*WithStatement*_[?Yield, ?Await, ?Return]
*LabelledStatement*_[?Yield, ?Await, ?Return]
*ThrowStatement*_[?Yield, ?Await]
*TryStatement*_[?Yield, ?Await, ?Return]
DebuggerStatement

*Declaration*_[Yield, Await] :
*HoistableDeclaration*_[?Yield, ?Await, ~Default]
*ClassDeclaration*_[?Yield, ?Await, ~Default]
*LexicalDeclaration*_[+In, ?Yield, ?Await]

*HoistableDeclaration*_[Yield, Await, Default] :
*FunctionDeclaration*_[?Yield, ?Await, ?Default]
*GeneratorDeclaration*_[?Yield, ?Await, ?Default]
*AsyncFunctionDeclaration*_[?Yield, ?Await, ?Default]
*AsyncGeneratorDeclaration*_[?Yield, ?Await, ?Default]

```

BreakableStatement[Yield, Await, Return] :
    IterationStatement[?Yield, ?Await, ?Return]
    SwitchStatement[?Yield, ?Await, ?Return]

BlockStatement[Yield, Await, Return] :
    Block[?Yield, ?Await, ?Return]

Block[Yield, Await, Return] :
    { StatementList[?Yield, ?Await, ?Return] opt }

StatementList[Yield, Await, Return] :
    StatementListItem[?Yield, ?Await, ?Return]
    StatementList[?Yield, ?Await, ?Return] StatementListItem[?Yield, ?Await, ?Return]

StatementListItem[Yield, Await, Return] :
    Statement[?Yield, ?Await, ?Return]
    Declaration[?Yield, ?Await]

LexicalDeclaration[In, Yield, Await] :
    LetOrConst BindingList[?In, ?Yield, ?Await] ;

LetOrConst :
    let
    const

BindingList[In, Yield, Await] :
    LexicalBinding[?In, ?Yield, ?Await]
    BindingList[?In, ?Yield, ?Await] , LexicalBinding[?In, ?Yield, ?Await]

LexicalBinding[In, Yield, Await] :
    BindingIdentifier[?Yield, ?Await] Initializer[?In, ?Yield, ?Await] opt
    BindingPattern[?Yield, ?Await] Initializer[?In, ?Yield, ?Await]

VariableStatement[Yield, Await] :
    var VariableDeclarationList[+In, ?Yield, ?Await] ;

VariableDeclarationList[In, Yield, Await] :
    VariableDeclaration[?In, ?Yield, ?Await]
    VariableDeclarationList[?In, ?Yield, ?Await] , VariableDeclaration[?In, ?Yield, ?Await]

VariableDeclaration[In, Yield, Await] :
    BindingIdentifier[?Yield, ?Await] Initializer[?In, ?Yield, ?Await] opt
    BindingPattern[?Yield, ?Await] Initializer[?In, ?Yield, ?Await]

BindingPattern[Yield, Await] :
    ObjectBindingPattern[?Yield, ?Await]
    ArrayBindingPattern[?Yield, ?Await]

ObjectBindingPattern[Yield, Await] :

```

```

{ }
{ BindingRestProperty[?Yield, ?Await] }
{ BindingPropertyList[?Yield, ?Await] }
{ BindingPropertyList[?Yield, ?Await] , BindingRestProperty[?Yield, ?Await] opt }

```

```

ArrayBindingPattern[Yield, Await] :
  [ Elisionopt BindingRestElement[?Yield, ?Await] opt ]
  [ BindingElementList[?Yield, ?Await] ]
  [ BindingElementList[?Yield, ?Await] , Elisionopt BindingRestElement[?Yield, ?Await] opt ]

```

```

BindingRestProperty[Yield, Await] :
  ... BindingIdentifier[?Yield, ?Await]

```

```

BindingPropertyList[Yield, Await] :
  BindingProperty[?Yield, ?Await]
  BindingPropertyList[?Yield, ?Await] , BindingProperty[?Yield, ?Await]

```

```

BindingElementList[Yield, Await] :
  BindingElisionElement[?Yield, ?Await]
  BindingElementList[?Yield, ?Await] , BindingElisionElement[?Yield, ?Await]

```

```

BindingElisionElement[Yield, Await] :
  Elisionopt BindingElement[?Yield, ?Await]

```

```

BindingProperty[Yield, Await] :
  SingleNameBinding[?Yield, ?Await]
 PropertyName[?Yield, ?Await] : BindingElement[?Yield, ?Await]

```

```

BindingElement[Yield, Await] :
  SingleNameBinding[?Yield, ?Await]
  BindingPattern[?Yield, ?Await] Initializer[+In, ?Yield, ?Await] opt

```

```

SingleNameBinding[Yield, Await] :
  BindingIdentifier[?Yield, ?Await] Initializer[+In, ?Yield, ?Await] opt

```

```

BindingRestElement[Yield, Await] :
  ... BindingIdentifier[?Yield, ?Await]
  ... BindingPattern[?Yield, ?Await]

```

```

EmptyStatement :
  ;

```

```

ExpressionStatement[Yield, Await] :
  [lookahead ∉ { { , function , async [no LineTerminator here] function , class , let [ ] } ]
  Expression[+In, ?Yield, ?Await] ;

```

```

IfStatement[Yield, Await, Return] :
  if ( Expression[+In, ?Yield, ?Await] ) Statement[?Yield, ?Await, ?Return] else

```

```

Statement[?Yield, ?Await, ?Return]
if ( Expression[+In, ?Yield, ?Await] ) Statement[?Yield, ?Await, ?Return]

```

IterationStatement[Yield, Await, Return] :

```

do Statement[?Yield, ?Await, ?Return] while ( Expression[+In, ?Yield, ?Await] ) ;
while ( Expression[+In, ?Yield, ?Await] ) Statement[?Yield, ?Await, ?Return]
for ( [lookahead ∉ { let [ ] } ] Expression[~In, ?Yield, ?Await] opt ;
      Expression[+In, ?Yield, ?Await] opt ; Expression[+In, ?Yield, ?Await] opt )
      Statement[?Yield, ?Await, ?Return]
for ( var VariableDeclarationList[~In, ?Yield, ?Await] ;
      Expression[+In, ?Yield, ?Await] opt ; Expression[+In, ?Yield, ?Await] opt )
      Statement[?Yield, ?Await, ?Return]
for ( LexicalDeclaration[~In, ?Yield, ?Await] Expression[+In, ?Yield, ?Await] opt ;
      Expression[+In, ?Yield, ?Await] opt ) Statement[?Yield, ?Await, ?Return]
for ( [lookahead ∉ { let [ ] } ] LeftHandSideExpression[?Yield, ?Await] in
      Expression[+In, ?Yield, ?Await] ) Statement[?Yield, ?Await, ?Return]
for ( var ForBinding[?Yield, ?Await] in Expression[+In, ?Yield, ?Await] )
      Statement[?Yield, ?Await, ?Return]
for ( ForDeclaration[?Yield, ?Await] in Expression[+In, ?Yield, ?Await] )
      Statement[?Yield, ?Await, ?Return]
for ( [lookahead ≠ let] LeftHandSideExpression[?Yield, ?Await] of
      AssignmentExpression[+In, ?Yield, ?Await] ) Statement[?Yield, ?Await, ?Return]
for ( var ForBinding[?Yield, ?Await] of AssignmentExpression[+In, ?Yield, ?Await] )
      Statement[?Yield, ?Await, ?Return]
for ( ForDeclaration[?Yield, ?Await] of AssignmentExpression[+In, ?Yield, ?Await] )
      Statement[?Yield, ?Await, ?Return]
[+Await] for await ( [lookahead ≠ let] LeftHandSideExpression[?Yield, ?Await] of
      AssignmentExpression[+In, ?Yield, ?Await] ) Statement[?Yield, ?Await, ?Return]
[+Await] for await ( var ForBinding[?Yield, ?Await] of
      AssignmentExpression[+In, ?Yield, ?Await] ) Statement[?Yield, ?Await, ?Return]
[+Await] for await ( ForDeclaration[?Yield, ?Await] of
      AssignmentExpression[+In, ?Yield, ?Await] ) Statement[?Yield, ?Await, ?Return]

```

ForDeclaration[Yield, Await] :

```

LetOrConst ForBinding[?Yield, ?Await]

```

ForBinding[Yield, Await] :

```

BindingIdentifier[?Yield, ?Await]
BindingPattern[?Yield, ?Await]

```

ContinueStatement[Yield, Await] :

```

continue ;
continue [no LineTerminator here] LabelIdentifier[?Yield, ?Await] ;

```

BreakStatement[Yield, Await] :

```

break ;
break [no LineTerminator here] LabelIdentifier[?Yield, ?Await] ;

ReturnStatement[Yield, Await] :
    return ;
    return [no LineTerminator here] Expression[+In, ?Yield, ?Await] ;

WithStatement[Yield, Await, Return] :
    with ( Expression[+In, ?Yield, ?Await] ) Statement[?Yield, ?Await, ?Return]

SwitchStatement[Yield, Await, Return] :
    switch ( Expression[+In, ?Yield, ?Await] ) CaseBlock[?Yield, ?Await, ?Return]

CaseBlock[Yield, Await, Return] :
    { CaseClauses[?Yield, ?Await, ?Return] opt }
    { CaseClauses[?Yield, ?Await, ?Return] opt DefaultClause[?Yield, ?Await, ?Return]
      CaseClauses[?Yield, ?Await, ?Return] opt }

CaseClauses[Yield, Await, Return] :
    CaseClause[?Yield, ?Await, ?Return]
    CaseClauses[?Yield, ?Await, ?Return] CaseClause[?Yield, ?Await, ?Return]

CaseClause[Yield, Await, Return] :
    case Expression[+In, ?Yield, ?Await] : StatementList[?Yield, ?Await, ?Return] opt

DefaultClause[Yield, Await, Return] :
    default : StatementList[?Yield, ?Await, ?Return] opt

LabelledStatement[Yield, Await, Return] :
    LabelIdentifier[?Yield, ?Await] : LabelledItem[?Yield, ?Await, ?Return]

LabelledItem[Yield, Await, Return] :
    Statement[?Yield, ?Await, ?Return]
    FunctionDeclaration[?Yield, ?Await, ~Default]

ThrowStatement[Yield, Await] :
    throw [no LineTerminator here] Expression[+In, ?Yield, ?Await] ;

TryStatement[Yield, Await, Return] :
    try Block[?Yield, ?Await, ?Return] Catch[?Yield, ?Await, ?Return]
    try Block[?Yield, ?Await, ?Return] Finally[?Yield, ?Await, ?Return]
    try Block[?Yield, ?Await, ?Return] Catch[?Yield, ?Await, ?Return]
      Finally[?Yield, ?Await, ?Return]

Catch[Yield, Await, Return] :
    catch ( CatchParameter[?Yield, ?Await] ) Block[?Yield, ?Await, ?Return]
    catch Block[?Yield, ?Await, ?Return]

```

*Finally*_[Yield, Await, Return] :
 finally *Block*_[?Yield, ?Await, ?Return]

*CatchParameter*_[Yield, Await] :
 *BindingIdentifier*_[?Yield, ?Await]
 *BindingPattern*_[?Yield, ?Await]

DebuggerStatement :
 debugger ;

A.4 Functions and Classes

*FunctionDeclaration*_[Yield, Await, Default] :
 function *BindingIdentifier*_[?Yield, ?Await] (*FormalParameters*_[~Yield, ~Await]) {
 *FunctionBody*_[~Yield, ~Await] }
 [+Default] **function** (*FormalParameters*_[~Yield, ~Await]) { *FunctionBody*_[~Yield, ~Await] }

FunctionExpression :
 function *BindingIdentifier*_[~Yield, ~Await] *opt* (*FormalParameters*_[~Yield, ~Await]) {
 *FunctionBody*_[~Yield, ~Await] }

*UniqueFormalParameters*_[Yield, Await] :
 *FormalParameters*_[?Yield, ?Await]

*FormalParameters*_[Yield, Await] :
 [empty]
 *FunctionRestParameter*_[?Yield, ?Await]
 *FormalParameterList*_[?Yield, ?Await]
 *FormalParameterList*_[?Yield, ?Await] ,
 *FormalParameterList*_[?Yield, ?Await] , *FunctionRestParameter*_[?Yield, ?Await]

*FormalParameterList*_[Yield, Await] :
 *FormalParameter*_[?Yield, ?Await]
 *FormalParameterList*_[?Yield, ?Await] , *FormalParameter*_[?Yield, ?Await]

*FunctionRestParameter*_[Yield, Await] :
 *BindingRestElement*_[?Yield, ?Await]

*FormalParameter*_[Yield, Await] :
 *BindingElement*_[?Yield, ?Await]

*FunctionBody*_[Yield, Await] :
 *FunctionStatementList*_[?Yield, ?Await]

*FunctionStatementList*_[Yield, Await] :
 *StatementList*_[?Yield, ?Await, +Return] *opt*

```

ArrowFunction[In, Yield, Await] :
    ArrowParameters[?Yield, ?Await] [no LineTerminator here] => ConciseBody[?In]

```

```

ArrowParameters[Yield, Await] :
    BindingIdentifier[?Yield, ?Await]
    CoverParenthesizedExpressionAndArrowParameterList[?Yield, ?Await]

```

```

ConciseBody[In] :
    [lookahead ≠ {] AssignmentExpression[?In, ~Yield, ~Await]
    { FunctionBody[~Yield, ~Await] }

```

When the production *ArrowParameters* : *CoverParenthesizedExpressionAndArrowParameterList* is recognized the following grammar is used to refine the interpretation of *CoverParenthesizedExpressionAndArrowParameterList*:

```

ArrowFormalParameters[Yield, Await] :
    ( UniqueFormalParameters[?Yield, ?Await] )

```

```

AsyncArrowFunction[In, Yield, Await] :
    async [no LineTerminator here] AsyncArrowBindingIdentifier[?Yield] [no LineTerminator here] =>
        AsyncConciseBody[?In]
    CoverCallExpressionAndAsyncArrowHead[?Yield, ?Await] [no LineTerminator here] =>
        AsyncConciseBody[?In]

```

```

AsyncConciseBody[In] :
    [lookahead ≠ {] AssignmentExpression[?In, ~Yield, +Await]
    { AsyncFunctionBody }

```

When the production *AsyncArrowFunction* : *CoverCallExpressionAndAsyncArrowHead* [no LineTerminator here] => *AsyncConciseBody* is recognized the following grammar is used to refine the interpretation of *CoverParenthesizedExpressionAndArrowParameterList*:

```

AsyncArrowHead :
    async [no LineTerminator here] ArrowFormalParameters[~Yield, +Await]

```

```

MethodDefinition[Yield, Await] :
    PropertyName[?Yield, ?Await] ( UniqueFormalParameters[~Yield, ~Await] ) {
        FunctionBody[~Yield, ~Await] }
    GeneratorMethod[?Yield, ?Await]
    AsyncMethod[?Yield, ?Await]
    AsyncGeneratorMethod[?Yield, ?Await]
    get PropertyName[?Yield, ?Await] ( ) { FunctionBody[~Yield, ~Await] }
    set PropertyName[?Yield, ?Await] ( PropertySetParameterList ) {
        FunctionBody[~Yield, ~Await] }

```

```

PropertySetParameterList :

```

*FormalParameter*_[~Yield, ~Await]

*GeneratorMethod*_[Yield, Await] :

***** *PropertyName*_[?Yield, ?Await] (*UniqueFormalParameters*_[+Yield, ~Await]) {
 GeneratorBody }

*GeneratorDeclaration*_[Yield, Await, Default] :

function ***** *BindingIdentifier*_[?Yield, ?Await] (*FormalParameters*_[+Yield, ~Await]) {
 GeneratorBody }
[+Default] **function** ***** (*FormalParameters*[+Yield, ~Await]) { *GeneratorBody* }

GeneratorExpression :

function ***** *BindingIdentifier*_[+Yield, ~Await] _{opt} (*FormalParameters*_[+Yield, ~Await]) {
 GeneratorBody }

GeneratorBody :

*FunctionBody*_[+Yield, ~Await]

*YieldExpression*_[In, Await] :

yield
yield [no *LineTerminator* here] *AssignmentExpression*_[?In, +Yield, ?Await]
yield [no *LineTerminator* here] ***** *AssignmentExpression*_[?In, +Yield, ?Await]

*AsyncGeneratorMethod*_[Yield, Await] :

async [no *LineTerminator* here] ***** *PropertyName*_[?Yield, ?Await] (
 *UniqueFormalParameters*_[+Yield, +Await]) { *AsyncGeneratorBody* }

*AsyncGeneratorDeclaration*_[Yield, Await, Default] :

async [no *LineTerminator* here] **function** ***** *BindingIdentifier*_[?Yield, ?Await] (
 *FormalParameters*_[+Yield, +Await]) { *AsyncGeneratorBody* }
[+Default] **async** [no *LineTerminator* here] **function** ***** (*FormalParameters*[+Yield, +Await]) {
 AsyncGeneratorBody }

AsyncGeneratorExpression :

async [no *LineTerminator* here] **function** ***** *BindingIdentifier*_[+Yield, +Await] _{opt} (
 *FormalParameters*_[+Yield, +Await]) { *AsyncGeneratorBody* }

AsyncGeneratorBody :

*FunctionBody*_[+Yield, +Await]

*AsyncMethod*_[Yield, Await] :

async [no *LineTerminator* here] *PropertyName*_[?Yield, ?Await] (
 *UniqueFormalParameters*_[~Yield, +Await]) { *AsyncFunctionBody* }

*AsyncFunctionDeclaration*_[Yield, Await, Default] :

async [no *LineTerminator* here] **function** *BindingIdentifier*_[?Yield, ?Await] (
 *FormalParameters*_[~Yield, +Await]) { *AsyncFunctionBody* }
[+Default] **async** [no *LineTerminator* here] **function** (*FormalParameters*[~Yield, +Await]) {

AsyncFunctionBody }

AsyncFunctionExpression :

```
async [no LineTerminator here] function ( FormalParameters[~Yield, +Await] ) {  
  AsyncFunctionBody }  
async [no LineTerminator here] function BindingIdentifier[~Yield, +Await] (  
  FormalParameters[~Yield, +Await] ) { AsyncFunctionBody }
```

AsyncFunctionBody :

*FunctionBody*_[~Yield, +Await]

*AwaitExpression*_[Yield] :

await *UnaryExpression*_[?Yield, +Await]

*ClassDeclaration*_[Yield, Await, Default] :

```
class BindingIdentifier[?Yield, ?Await] ClassTail[?Yield, ?Await]  
[+Default] class ClassTail[?Yield, ?Await]
```

*ClassExpression*_[Yield, Await] :

class *BindingIdentifier*_[?Yield, ?Await] *opt* *ClassTail*_[?Yield, ?Await]

*ClassTail*_[Yield, Await] :

*ClassHeritage*_[?Yield, ?Await] *opt* { *ClassBody*_[?Yield, ?Await] *opt* }

*ClassHeritage*_[Yield, Await] :

extends *LeftHandSideExpression*_[?Yield, ?Await]

*ClassBody*_[Yield, Await] :

*ClassElementList*_[?Yield, ?Await]

*ClassElementList*_[Yield, Await] :

*ClassElement*_[?Yield, ?Await]

*ClassElementList*_[?Yield, ?Await] *ClassElement*_[?Yield, ?Await]

*ClassElement*_[Yield, Await] :

*MethodDefinition*_[?Yield, ?Await]

static *MethodDefinition*_[?Yield, ?Await]

;

A.5 Scripts and Modules

Script :

*ScriptBody*_{*opt*}

ScriptBody :

*StatementList*_[~Yield, ~Await, ~Return]

Module :

*ModuleBody*_{opt}

ModuleBody :

ModuleItemList

ModuleItemList :

ModuleItem

ModuleItemList *ModuleItem*

ModuleItem :

ImportDeclaration

ExportDeclaration

*StatementListItem*_[~Yield, ~Await, ~Return]

ImportDeclaration :

import *ImportClause* *FromClause* ;

import *ModuleSpecifier* ;

ImportClause :

ImportedDefaultBinding

NamespaceImport

NamedImports

ImportedDefaultBinding , *NamespaceImport*

ImportedDefaultBinding , *NamedImports*

ImportedDefaultBinding :

ImportedBinding

NamespaceImport :

*** as** *ImportedBinding*

NamedImports :

{ }

{ *ImportsList* }

{ *ImportsList* , }

FromClause :

from *ModuleSpecifier*

ImportsList :

ImportSpecifier

ImportsList , *ImportSpecifier*

ImportSpecifier :

ImportedBinding

IdentifierName **as** *ImportedBinding*

ModuleSpecifier :

StringLiteral

ImportedBinding :

*BindingIdentifier*_[~Yield, ~Await]

ExportDeclaration :

```
export * FromClause ;  
export ExportClause FromClause ;  
export ExportClause ;  
export VariableStatement[~Yield, ~Await]  
export Declaration[~Yield, ~Await]  
export default HoistableDeclaration[~Yield, ~Await, +Default]  
export default ClassDeclaration[~Yield, ~Await, +Default]  
export default [lookahead  $\notin$  { function , async [no LineTerminator here] function ,  
      class }] AssignmentExpression[+In, ~Yield, ~Await] ;
```

ExportClause :

```
{ }  
{ ExportsList }  
{ ExportsList , }
```

ExportsList :

```
ExportSpecifier  
ExportsList , ExportSpecifier
```

ExportSpecifier :

```
IdentifierName  
IdentifierName as IdentifierName
```

A.6 Number Conversions

StringNumericLiteral :::

```
StrWhiteSpaceopt  
StrWhiteSpaceopt StrNumericLiteral StrWhiteSpaceopt
```

StrWhiteSpace :::

```
StrWhiteSpaceChar StrWhiteSpaceopt
```

StrWhiteSpaceChar :::

```
WhiteSpace  
LineTerminator
```

StrNumericLiteral :::

```
StrDecimalLiteral  
BinaryIntegerLiteral  
OctalIntegerLiteral  
HexIntegerLiteral
```

StrDecimalLiteral :::

```
StrUnsignedDecimalLiteral  
+ StrUnsignedDecimalLiteral  
- StrUnsignedDecimalLiteral
```

StrUnsignedDecimalLiteral :::

Infinity

DecimalDigits . *DecimalDigits*_{opt} *ExponentPart*_{opt}
.
DecimalDigits *ExponentPart*_{opt}
DecimalDigits *ExponentPart*_{opt}

DecimalDigits ::

DecimalDigit
DecimalDigits *DecimalDigit*

DecimalDigit :: **one of**

0 1 2 3 4 5 6 7 8 9

ExponentPart ::

ExponentIndicator *SignedInteger*

ExponentIndicator :: **one of**

e E

SignedInteger ::

DecimalDigits
+ *DecimalDigits*
- *DecimalDigits*

HexIntegerLiteral ::

0x *HexDigits*
0X *HexDigits*

HexDigit :: **one of**

0 1 2 3 4 5 6 7 8 9 a b c d e f A B C D E F

All grammar symbols not explicitly defined by the *StringNumericLiteral* grammar have the definitions used in the [Lexical Grammar for numeric literals](#).

A.7 Universal Resource Identifier Character Classes

uri :::

*uriCharacters*_{opt}

uriCharacters :::

uriCharacter *uriCharacters*_{opt}

uriCharacter :::

uriReserved
uriUnescaped
uriEscaped

uriReserved ::: **one of**

; / ? : @ & = + \$,

uriUnescaped :::

uriAlpha
DecimalDigit
uriMark

uriEscaped :::
 % *HexDigit HexDigit*

uriAlpha ::: one of
 a b c d e f g h i j k l m n o p q r s t u v w x y z A B C D E F G H I J K L M N O
 P Q R S T U V W X Y Z

uriMark ::: one of
 - _ . ! ~ * ' ()

A.8 Regular Expressions

*Pattern*_[U, N] ::
*Disjunction*_[?U, ?N]

*Disjunction*_[U, N] ::
*Alternative*_[?U, ?N]
*Alternative*_[?U, ?N] | *Disjunction*_[?U, ?N]

*Alternative*_[U, N] ::
 [empty]
*Alternative*_[?U, ?N] *Term*_[?U, ?N]

*Term*_[U, N] ::
*Assertion*_[?U, ?N]
*Atom*_[?U, ?N]
*Atom*_[?U, ?N] *Quantifier*

*Assertion*_[U, N] ::
 ^
 \$
 \ b
 \ B
 (? = *Disjunction*_[?U, ?N])
 (? ! *Disjunction*_[?U, ?N])
 (? <= *Disjunction*_[?U, ?N])
 (? <! *Disjunction*_[?U, ?N])

Quantifier ::
QuantifierPrefix
QuantifierPrefix ?

QuantifierPrefix ::
 *

+
 ?
 { *DecimalDigits* }
 { *DecimalDigits* , }
 { *DecimalDigits* , *DecimalDigits* }

*Atom*_[U, N] ::
PatternCharacter
 .
 \ *AtomEscape*_[?U, ?N]
*CharacterClass*_[?U]
 (*GroupSpecifier*_[?U] *Disjunction*_[?U, ?N])
 (? : *Disjunction*_[?U, ?N])

SyntaxCharacter :: **one of**
 ^ \$ \ . * + ? () [] { } |

PatternCharacter ::
SourceCharacter but not *SyntaxCharacter*

*AtomEscape*_[U, N] ::
DecimalEscape
*CharacterClassEscape*_[?U]
*CharacterEscape*_[?U]
 [+N] **k** *GroupName*_[?U]

*CharacterEscape*_[U] ::
ControlEscape
c *ControlLetter*
0 [lookahead $\notin$ *DecimalDigit*]
HexEscapeSequence
*RegExpUnicodeEscapeSequence*_[?U]
*IdentityEscape*_[?U]

ControlEscape :: **one of**
f n r t v

ControlLetter :: **one of**
a b c d e f g h i j k l m n o p q r s t u v w x y z A B C D E F G H I J K L M N O
P Q R S T U V W X Y Z

*GroupSpecifier*_[U] ::
 [empty]
 ? *GroupName*_[?U]

*GroupName*_[U] ::
 < *RegExpIdentifierName*_[?U] >

*RegExpIdentifierName*_[U] ::

*RegExpIdentifierStart*_[?U]
*RegExpIdentifierName*_[?U] *RegExpIdentifierPart*_[?U]

*RegExpIdentifierStart*_[U] ::
UnicodeIDStart
 \$
 _
 \ *RegExpUnicodeEscapeSequence*_[?U]

*RegExpIdentifierPart*_[U] ::
UnicodeIDContinue
 \$
 \ *RegExpUnicodeEscapeSequence*_[?U]
 <ZWJ>
 <ZWJ>

*RegExpUnicodeEscapeSequence*_[U] ::
 [+U] **u** *LeadSurrogate* \ **u** *TrailSurrogate*
 [+U] **u** *LeadSurrogate*
 [+U] **u** *TrailSurrogate*
 [+U] **u** *NonSurrogate*
 [~U] **u** *Hex4Digits*
 [+U] **u** { *CodePoint* }

Each \ **u** *TrailSurrogate* for which the choice of associated **u** *LeadSurrogate* is ambiguous shall be associated with the nearest possible **u** *LeadSurrogate* that would otherwise have no corresponding \ **u** *TrailSurrogate*.

LeadSurrogate ::
Hex4Digits but only if the SV of *Hex4Digits* is in the inclusive range 0xD800 to 0xDBFF

TrailSurrogate ::
Hex4Digits but only if the SV of *Hex4Digits* is in the inclusive range 0xDC00 to 0xDFFF

NonSurrogate ::
Hex4Digits but only if the SV of *Hex4Digits* is not in the inclusive range 0xD800 to 0xDFFF

*IdentityEscape*_[U] ::
 [+U] *SyntaxCharacter*
 [+U] /
 [~U] *SourceCharacter* but not *UnicodeIDContinue*

DecimalEscape ::
NonZeroDigit *DecimalDigits*_{opt} [lookahead $\notin$ *DecimalDigit*]

*CharacterClassEscape*_[U] ::
 d
 D
 s

S

w

W

[+U] **P**{ UnicodePropertyValueExpression }

[+U] **P**{ UnicodePropertyValueExpression }

UnicodePropertyValueExpression ::

UnicodePropertyName = *UnicodePropertyValue*

LoneUnicodePropertyNameOrValue

UnicodePropertyName ::

UnicodePropertyNameCharacters

UnicodePropertyNameCharacters ::

UnicodePropertyNameCharacter *UnicodePropertyNameCharacters*_{opt}

UnicodePropertyValue ::

UnicodePropertyValueCharacters

LoneUnicodePropertyNameOrValue ::

UnicodePropertyValueCharacters

UnicodePropertyValueCharacters ::

UnicodePropertyValueCharacter *UnicodePropertyValueCharacters*_{opt}

UnicodePropertyValueCharacter ::

UnicodePropertyNameCharacter

0

1

2

3

4

5

6

7

8

9

UnicodePropertyNameCharacter ::

ControlLetter

—

*CharacterClass*_[U] ::

[[lookahead ∉ { ^ }] *ClassRanges*_[?U]]

[^ *ClassRanges*_[?U]]

*ClassRanges*_[U] ::

[empty]

*NonemptyClassRanges*_[?U]

*NonemptyClassRanges*_[U] ::
 *ClassAtom*_[?U]
 *ClassAtom*_[?U] *NonemptyClassRangesNoDash*_[?U]
 *ClassAtom*_[?U] – *ClassAtom*_[?U] *ClassRanges*_[?U]

*NonemptyClassRangesNoDash*_[U] ::
 *ClassAtom*_[?U]
 *ClassAtomNoDash*_[?U] *NonemptyClassRangesNoDash*_[?U]
 *ClassAtomNoDash*_[?U] – *ClassAtom*_[?U] *ClassRanges*_[?U]

*ClassAtom*_[U] ::
 –
 *ClassAtomNoDash*_[?U]

*ClassAtomNoDash*_[U] ::
 SourceCharacter but not one of \ or] or –
 \ *ClassEscape*_[?U]

*ClassEscape*_[U] ::
 b
 [+U] –
 *CharacterClassEscape*_[?U]
 *CharacterEscape*_[?U]

B Additional ECMAScript Features for Web Browsers

The ECMAScript language syntax and semantics defined in this annex are required when the ECMAScript host is a web browser. The content of this annex is normative but optional if the ECMAScript host is not a web browser.

NOTE

This annex describes various legacy features and other characteristics of web browser based ECMAScript implementations. All of the language features and behaviours specified in this annex have one or more undesirable characteristics and in the absence of legacy usage would be removed from this specification. However, the usage of these features by large numbers of existing web pages means that web browsers must continue to support them. The specifications in this annex define the requirements for interoperable implementations of these legacy features.

These features are not considered part of the core ECMAScript language. Programmers should not use or assume the existence of these features and behaviours when writing new ECMAScript code. ECMAScript implementations are discouraged from implementing these features unless the implementation is part of a web browser or is required to run the same legacy ECMAScript code that web browsers encounter.

B.1 Additional Syntax

B.1.1 Numeric Literals

The syntax and semantics of 11.8.3 is extended as follows except that this extension is not allowed for [strict mode code](#):

Syntax

NumericLiteral ::

DecimalLiteral

BinaryIntegerLiteral

OctalIntegerLiteral

HexIntegerLiteral

LegacyOctalIntegerLiteral

LegacyOctalIntegerLiteral ::

0 *OctalDigit*

LegacyOctalIntegerLiteral *OctalDigit*

DecimalIntegerLiteral ::

0

NonZeroDigit *DecimalDigits*_{opt}

NonOctalDecimalIntegerLiteral

NonOctalDecimalIntegerLiteral ::

0 *NonOctalDigit*

LegacyOctalLikeDecimalIntegerLiteral *NonOctalDigit*

NonOctalDecimalIntegerLiteral *DecimalDigit*

LegacyOctalLikeDecimalIntegerLiteral ::

0 *OctalDigit*

LegacyOctalLikeDecimalIntegerLiteral *OctalDigit*

NonOctalDigit :: **one of**

8 9

B.1.1.1 Static Semantics

The MV of *LegacyOctalIntegerLiteral* :: **0** *OctalDigit* is the MV of *OctalDigit*.

The MV of *LegacyOctalIntegerLiteral* :: *LegacyOctalIntegerLiteral* *OctalDigit* is (the MV of *LegacyOctalIntegerLiteral* times 8) plus the MV of *OctalDigit*.

The MV of *DecimalIntegerLiteral* :: *NonOctalDecimalIntegerLiteral* is the MV of *NonOctalDecimalIntegerLiteral*.

The MV of *NonOctalDecimalIntegerLiteral* :: **0** *NonOctalDigit* is the MV of *NonOctalDigit*.

The MV of *NonOctalDecimalIntegerLiteral* :: *LegacyOctalLikeDecimalIntegerLiteral* *NonOctalDigit* is (the MV of *LegacyOctalLikeDecimalIntegerLiteral* times 10) plus the MV of *NonOctalDigit*.

The MV of *NonOctalDecimalIntegerLiteral* :: *NonOctalDecimalIntegerLiteral* *DecimalDigit* is (the MV of *NonOctalDecimalIntegerLiteral* times 10) plus the MV of *DecimalDigit*.

The MV of *LegacyOctalLikeDecimalIntegerLiteral* :: **0** *OctalDigit* is the MV of *OctalDigit*.

The MV of *LegacyOctalLikeDecimalIntegerLiteral* :: *LegacyOctalLikeDecimalIntegerLiteral* *OctalDigit* is (the MV of *LegacyOctalLikeDecimalIntegerLiteral* times 10) plus the MV of *OctalDigit*.

The MV of *NonOctalDigit* :: **8** is 8.

The MV of *NonOctalDigit* :: **9** is 9.

B.1.2 String Literals

The syntax and semantics of 11.8.4 is extended as follows except that this extension is not allowed for [strict mode code](#):

Syntax

EscapeSequence ::

CharacterEscapeSequence

LegacyOctalEscapeSequence

HexEscapeSequence

UnicodeEscapeSequence

LegacyOctalEscapeSequence ::

OctalDigit [lookahead $\notin$ *OctalDigit*]

ZeroToThree *OctalDigit* [lookahead $\notin$ *OctalDigit*]

FourToSeven *OctalDigit*

ZeroToThree *OctalDigit* *OctalDigit*

ZeroToThree :: **one of**

0 1 2 3

FourToSeven :: **one of**

4 5 6 7

This definition of *EscapeSequence* is not used in strict mode or when parsing *TemplateCharacter*.

B.1.2.1 Static Semantics

The SV of *EscapeSequence* :: *LegacyOctalEscapeSequence* is the SV of the *LegacyOctalEscapeSequence*.

The SV of *LegacyOctalEscapeSequence* :: *OctalDigit* is the code unit whose value is the MV of the *OctalDigit*.

The SV of *LegacyOctalEscapeSequence* :: *ZeroToThree* *OctalDigit* is the code unit whose value is (8 times the MV of the *ZeroToThree*) plus the MV of the *OctalDigit*.

The SV of *LegacyOctalEscapeSequence* :: *FourToSeven* *OctalDigit* is the code unit whose value is (8 times the MV of the *FourToSeven*) plus the MV of the *OctalDigit*.

The SV of *LegacyOctalEscapeSequence* :: *ZeroToThree* *OctalDigit* *OctalDigit* is the code unit whose value is (64 (that is, 8^2) times the MV of the *ZeroToThree*) plus (8 times the MV of the first *OctalDigit*) plus the MV of the second *OctalDigit*.

The MV of *ZeroToThree* :: **0** is 0.

The MV of *ZeroToThree* :: **1** is 1.

The MV of *ZeroToThree* :: **2** is 2.

The MV of *ZeroToThree* :: **3** is 3.

The MV of *FourToSeven* :: **4** is 4.

The MV of *FourToSeven* :: **5** is 5.

The MV of *FourToSeven* :: **6** is 6.

The MV of *FourToSeven* :: **7** is 7.

B.1.3 HTML-like Comments

The syntax and semantics of 11.4 is extended as follows except that this extension is not allowed when parsing source code using the [goal symbol](#) *Module*:

Syntax

Comment ::

MultiLineComment
SingleLineComment
SingleLineHTMLOpenComment
SingleLineHTMLCloseComment
SingleLineDelimitedComment

MultiLineComment ::

*/** *FirstCommentLine*_{opt} *LineTerminator* *MultiLineCommentChars*_{opt} **/* *HTMLCloseComment*_{opt}

FirstCommentLine ::

SingleLineDelimitedCommentChars

SingleLineHTMLOpenComment ::

<!-- *SingleLineCommentChars*_{opt}

SingleLineHTMLCloseComment ::

LineTerminatorSequence *HTMLCloseComment*

SingleLineDelimitedComment ::

*/** *SingleLineDelimitedCommentChars*_{opt} **/*

HTMLCloseComment ::

*WhiteSpaceSequence*_{opt} *SingleLineDelimitedCommentSequence*_{opt} *-->* *SingleLineCommentChars*_{opt}

SingleLineDelimitedCommentChars ::

SingleLineNotAsteriskChar *SingleLineDelimitedCommentChars*_{opt}
*** *SingleLinePostAsteriskCommentChars*_{opt}

SingleLineNotAsteriskChar ::

SourceCharacter but not one of *** or *LineTerminator*

SingleLinePostAsteriskCommentChars ::

SingleLineNotForwardSlashOrAsteriskChar *SingleLineDelimitedCommentChars*_{opt}
*** *SingleLinePostAsteriskCommentChars*_{opt}

SingleLineNotForwardSlashOrAsteriskChar ::

SourceCharacter but not one of */* or *** or *LineTerminator*

WhiteSpaceSequence ::

WhiteSpace *WhiteSpaceSequence*_{opt}

SingleLineDelimitedCommentSequence ::

SingleLineDelimitedComment *WhiteSpaceSequence*_{opt} *SingleLineDelimitedCommentSequence*_{opt}

Similar to a *MultiLineComment* that contains a line terminator code point, a *SingleLineHTMLCloseComment* is considered to be a *LineTerminator* for purposes of parsing by the syntactic grammar.

B.1.4 Regular Expressions Patterns

The syntax of 21.2.1 is modified and extended as follows. These changes introduce ambiguities that are broken by the ordering of grammar productions and by contextual information. When parsing using the following grammar, each alternative is considered only if previous production alternatives do not match.

This alternative pattern grammar and semantics only changes the syntax and semantics of BMP patterns. The following grammar extensions include productions parameterized with the [U] parameter. However, none of these extensions change the syntax of Unicode patterns recognized when parsing with the [U] parameter present on the [goal symbol](#).

Syntax

```
Term[U, N] ::
    [+U] Assertion[+U, ?N]
    [+U] Atom[+U, ?N]
    [+U] Atom[+U, ?N] Quantifier
    [~U] QuantifiableAssertion[?N] Quantifier
    [~U] Assertion[~U, ?N]
    [~U] ExtendedAtom[?N] Quantifier
    [~U] ExtendedAtom[?N]
```

```
Assertion[U, N] ::
    ^
    $
    \ b
    \ B
    [+U] ( ? = Disjunction[+U, ?N] )
    [+U] ( ? ! Disjunction[+U, ?N] )
    [~U] QuantifiableAssertion[?N]
    ( ? <= Disjunction[?U, ?N] )
    ( ? <! Disjunction[?U, ?N] )
```

```
QuantifiableAssertion[N] ::
    ( ? = Disjunction[~U, ?N] )
    ( ? ! Disjunction[~U, ?N] )
```

```
ExtendedAtom[N] ::
    .
    \ AtomEscape[~U, ?N]
    \ [lookahead = c]
    CharacterClass[~U]
    ( Disjunction[~U, ?N] )
    ( ? : Disjunction[~U, ?N] )
    InvalidBracedQuantifier
    ExtendedPatternCharacter
```

```
InvalidBracedQuantifier ::
    { DecimalDigits }
    { DecimalDigits , }
```

{ *DecimalDigits* , *DecimalDigits* }

ExtendedPatternCharacter ::

SourceCharacter but not one of ^ \$ \ . * + ? () [|

*AtomEscape*_[U, N] ::

_[+U] *DecimalEscape*

_[~U] *DecimalEscape* but only if the *CapturingGroupNumber* of *DecimalEscape* is ≤ *_NcapturingParens_*
*CharacterClassEscape*_[?U]

*CharacterEscape*_[~U, ?N]

[+N] **k** *GroupName*[?U]

*CharacterEscape*_[U, N] ::

ControlEscape

c *ControlLetter*

0 [lookahead ≠ *DecimalDigit*]

HexEscapeSequence

*RegExpUnicodeEscapeSequence*_[?U]

_[~U] *LegacyOctalEscapeSequence*

*IdentityEscape*_[?U, ?N]

*IdentityEscape*_[U, N] ::

_[+U] *SyntaxCharacter*

_[+U] /

[~U] *SourceCharacterIdentityEscape*[?N]

*SourceCharacterIdentityEscape*_[N] ::

_[~N] *SourceCharacter* but not **c**

_[+N] *SourceCharacter* but not one of **c** or **k**

*ClassAtomNoDash*_[U, N] ::

SourceCharacter but not one of \ or] or -

\ *ClassEscape*_[?U, ?N]

\ [lookahead = **c**]

*ClassEscape*_[U, N] ::

b

_[+U] -

_[~U] **c** *ClassControlLetter*

*CharacterClassEscape*_[?U]

*CharacterEscape*_[?U, ?N]

ClassControlLetter ::

DecimalDigit

—

NOTE

When the same left hand sides occurs with both _[+U] and _[~U] guards it is to control the disambiguation priority.

B.1.4.1 Static Semantics: Early Errors

The semantics of 21.2.1.1 is extended as follows:

ExtendedAtom :: *InvalidBracedQuantifier*

It is a Syntax Error if any source text matches this rule.

NonemptyClassRanges :: *ClassAtom* – *ClassAtom* *ClassRanges*

It is a Syntax Error if *IsCharacterClass* of the first *ClassAtom* is **true** or *IsCharacterClass* of the second *ClassAtom* is **true** and this production has a [U] parameter.

NonemptyClassRangesNoDash :: *ClassAtomNoDash* – *ClassAtom* *ClassRanges*

It is a Syntax Error if *IsCharacterClass* of *ClassAtomNoDash* is **true** or *IsCharacterClass* of *ClassAtom* is **true** and this production has a [U] parameter.

B.1.4.2 Static Semantics: IsCharacterClass

The semantics of 21.2.1.3 is extended as follows:

ClassAtomNoDash :: \ [lookahead = **c**]

1. Return **false**.

B.1.4.3 Static Semantics: CharacterValue

The semantics of 21.2.1.4 is extended as follows:

ClassAtomNoDash :: \ [lookahead = **c**]

1. Return the code point value of U+005C (REVERSE SOLIDUS).

ClassEscape :: **c** *ClassControlLetter*

1. Let *ch* be the code point matched by *ClassControlLetter*.
2. Let *i* be *ch*'s code point value.
3. Return the remainder of dividing *i* by 32.

CharacterEscape :: *LegacyOctalEscapeSequence*

1. Evaluate the SV of the *LegacyOctalEscapeSequence* (see B.1.2) to obtain a code unit *cu*.
2. Return the numeric value of *cu*.

B.1.4.4 Pattern Semantics

The semantics of 21.2.2 is extended as follows:

Within 21.2.2.5 reference to “ *Atom* :: (*GroupSpecifier* *Disjunction*) ” are to be interpreted as meaning “ *Atom* :: (*GroupSpecifier* *Disjunction*) ” or “ *ExtendedAtom* :: (*Disjunction*) ”.

Term (21.2.2.5) includes the following additional evaluation rules:

The production *Term* :: *QuantifiableAssertion Quantifier* evaluates the same as the production *Term* :: *Atom Quantifier* but with *QuantifiableAssertion* substituted for *Atom*.

The production *Term* :: *ExtendedAtom Quantifier* evaluates the same as the production *Term* :: *Atom Quantifier* but with *ExtendedAtom* substituted for *Atom*.

The production *Term* :: *ExtendedAtom* evaluates the same as the production *Term* :: *Atom* but with *ExtendedAtom* substituted for *Atom*.

Assertion (21.2.2.6) includes the following additional evaluation rule:

The production *Assertion* :: *QuantifiableAssertion* evaluates as follows:

1. Evaluate *QuantifiableAssertion* to obtain a Matcher *m*.
2. Return *m*.

Assertion (21.2.2.6) evaluation rules for the *Assertion* :: (? = Disjunction) and *Assertion* :: (? ! Disjunction) productions are also used for the *QuantifiableAssertion* productions, but with *QuantifiableAssertion* substituted for *Assertion*.

Atom (21.2.2.8) evaluation rules for the *Atom* productions except for *Atom* :: *PatternCharacter* are also used for the *ExtendedAtom* productions, but with *ExtendedAtom* substituted for *Atom*. The following evaluation rules are also added:

The production *ExtendedAtom* :: \ [lookahead = **c**] evaluates as follows:

1. Let *A* be the CharSet containing the single character \ U+005C (REVERSE SOLIDUS).
2. Call *CharacterSetMatcher*(*A*, **false**) and return its Matcher result.

The production *ExtendedAtom* :: *ExtendedPatternCharacter* evaluates as follows:

1. Let *ch* be the character represented by *ExtendedPatternCharacter*.
2. Let *A* be a one-element CharSet containing the character *ch*.
3. Call *CharacterSetMatcher*(*A*, **false**) and return its Matcher result.

CharacterEscape (21.2.2.10) includes the following additional evaluation rule:

The production *CharacterEscape* :: *LegacyOctalEscapeSequence* evaluates as follows:

1. Let *cv* be the CharacterValue of this *CharacterEscape*.
2. Return the character whose character value is *cv*.

NonemptyClassRanges (21.2.2.15) modifies the following evaluation rule:

The production *NonemptyClassRanges* :: *ClassAtom* – *ClassAtom ClassRanges* evaluates as follows:

1. Evaluate the first *ClassAtom* to obtain a CharSet *A*.
2. Evaluate the second *ClassAtom* to obtain a CharSet *B*.
3. Evaluate *ClassRanges* to obtain a CharSet *C*.
4. Call *CharacterRangeOrUnion*(*A*, *B*) and let *D* be the resulting CharSet.
5. Return the union of CharSets *D* and *C*.

NonemptyClassRangesNoDash (21.2.2.16) modifies the following evaluation rule:

The production *NonemptyClassRangesNoDash* :: *ClassAtomNoDash* – *ClassAtom ClassRanges* evaluates as follows:

1. Evaluate *ClassAtomNoDash* to obtain a CharSet *A*.
2. Evaluate *ClassAtom* to obtain a CharSet *B*.
3. Evaluate *ClassRanges* to obtain a CharSet *C*.
4. Call *CharacterRangeOrUnion*(*A*, *B*) and let *D* be the resulting CharSet.
5. Return the union of CharSets *D* and *C*.

ClassEscape (21.2.2.19) includes the following additional evaluation rule:

The production *ClassEscape* :: **c** *ClassControlLetter* evaluates as follows:

1. Let *cv* be the CharacterValue of this *ClassEscape*.
2. Let *c* be the character whose character value is *cv*.
3. Return the CharSet containing the single character *c*.

ClassAtomNoDash (21.2.2.18) includes the following additional evaluation rule:

The production *ClassAtomNoDash* :: \ [lookahead = **c**] evaluates as follows:

1. Return the CharSet containing the single character \ U+005C (REVERSE SOLIDUS).

NOTE

This production can only be reached from the sequence \ **c** within a character class where it is not followed by an acceptable control character.

B.1.4.4.1 Runtime Semantics: CharacterRangeOrUnion (*A*, *B*)

The abstract operation *CharacterRangeOrUnion* takes two CharSet parameters *A* and *B* and performs the following steps:

1. If *Unicode* is **false**, then
 - a. If *A* does not contain exactly one character or *B* does not contain exactly one character, then
 - i. Let *C* be the CharSet containing the single character – U+002D (HYPHEN-MINUS).
 - ii. Return the union of CharSets *A*, *B* and *C*.
2. Return *CharacterRange*(*A*, *B*).

B.2 Additional Built-in Properties

When the ECMAScript host is a web browser the following additional properties of the standard built-in objects are defined.

B.2.1 Additional Properties of the Global Object

The entries in Table 83 are added to Table 7.

Table 83: Additional Well-known Intrinsic Objects

Intrinsic Name	Global Name	ECMAScript Language Association
%escape%	escape	The escape function (B.2.1.1)
%unescape%	unescape	The unescape function (B.2.1.2)

B.2.1.1 `escape ( string )`

The **escape** function is a property of the [global object](#). It computes a new version of a String value in which certain code units have been replaced by a hexadecimal escape sequence.

For those code units being replaced whose value is **0x00FF** or less, a two-digit escape sequence of the form **%xx** is used. For those characters being replaced whose code unit value is greater than **0x00FF**, a four-digit escape sequence of the form **%uxxxx** is used.

The **escape** function is the `%escape%` intrinsic object. When the **escape** function is called with one argument *string*, the following steps are taken:

1. Set *string* to ? [ToString\(string\)](#).
2. Let *length* be the number of code units in *string*.
3. Let *R* be the empty string.
4. Let *k* be 0.
5. Repeat, while *k* < *length*,
 - a. Let *char* be the code unit (represented as a 16-bit unsigned integer) at index *k* within *string*.
 - b. If *char* is one of the code units in
`"ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789@*_+-. /"`,
then
 - i. Let *S* be the String value containing the single code unit *char*.
 - c. Else if *char* ≥ 256, then
 - i. Let *n* be the numeric value of *char*.
 - ii. Let *S* be the [string-concatenation](#) of:
`"%u"`
the String representation of *n*, formatted as a four-digit uppercase hexadecimal number, padded to the left with zeroes if necessary
 - d. Else *char* < 256,
 - i. Let *n* be the numeric value of *char*.
 - ii. Let *S* be the [string-concatenation](#) of:
`"%"`
the String representation of *n*, formatted as a two-digit uppercase hexadecimal number, padded to the left with a zero if necessary
 - e. Set *R* to the [string-concatenation](#) of the previous value of *R* and *S*.
 - f. Increase *k* by 1.
6. Return *R*.

NOTE

The encoding is partly based on the encoding described in RFC 1738, but the entire encoding specified in this standard is described above without regard to the contents of RFC 1738. This encoding does not reflect changes to RFC 1738 made by RFC 3986.

B.2.1.2 `unescape ( string )`

The **unescape** function is a property of the [global object](#). It computes a new version of a String value in which each escape sequence of the sort that might be introduced by the **escape** function is replaced with the code unit that it represents.

The **unescape** function is the %unescape% intrinsic object. When the **unescape** function is called with one argument *string*, the following steps are taken:

1. Set *string* to ? ToString(*string*).
2. Let *length* be the number of code units in *string*.
3. Let *R* be the empty String.
4. Let *k* be 0.
5. Repeat, while *k* ≠ *length*
 - a. Let *c* be the code unit at index *k* within *string*.
 - b. If *c* is the code unit 0x0025 (PERCENT SIGN), then
 - i. If $k \leq \text{length} - 6$ and the code unit at index $k + 1$ within *string* is the code unit 0x0075 (LATIN SMALL LETTER U) and the four code units at indices $k + 2$, $k + 3$, $k + 4$, and $k + 5$ within *string* are all hexadecimal digits, then
 1. Set *c* to the code unit whose value is the integer represented by the four hexadecimal digits at indices $k + 2$, $k + 3$, $k + 4$, and $k + 5$ within *string*.
 2. Increase *k* by 5.
 - ii. Else if $k \leq \text{length} - 3$ and the two code units at indices $k + 1$ and $k + 2$ within *string* are both hexadecimal digits, then
 1. Set *c* to the code unit whose value is the integer represented by two zeroes plus the two hexadecimal digits at indices $k + 1$ and $k + 2$ within *string*.
 2. Increase *k* by 2.
 - c. Set *R* to the string-concatenation of the previous value of *R* and *c*.
 - d. Increase *k* by 1.
6. Return *R*.

B.2.2 Additional Properties of the Object.prototype Object

B.2.2.1 Object.prototype.__proto__

Object.prototype.__proto__ is an **accessor property** with attributes { [[Enumerable]]: **false**, [[Configurable]]: **true** }. The [[Get]] and [[Set]] attributes are defined as follows:

B.2.2.1.1 get Object.prototype.__proto__

The value of the [[Get]] attribute is a built-in function that requires no arguments. It performs the following steps:

1. Let *O* be ? ToObject(this value).
2. Return ? O.[[GetPrototypeOf]]().

B.2.2.1.2 set Object.prototype.__proto__

The value of the [[Set]] attribute is a built-in function that takes an argument *proto*. It performs the following steps:

1. Let *O* be ? RequireObjectCoercible(this value).
2. If Type(*proto*) is neither Object nor Null, return **undefined**.
3. If Type(*O*) is not Object, return **undefined**.
4. Let *status* be ? O.[[SetPrototypeOf]](*proto*).
5. If *status* is **false**, throw a **TypeError** exception.
6. Return **undefined**.

B.2.2.2 Object.prototype.__defineGetter__ (*P*, *getter*)

When the `__defineGetter__` method is called with arguments *P* and *getter*, the following steps are taken:

1. Let *O* be ? `ToObject(this value)`.
2. If `IsCallable(getter)` is **false**, throw a **TypeError** exception.
3. Let *desc* be PropertyDescriptor { `[[Get]]`: *getter*, `[[Enumerable]]`: **true**, `[[Configurable]]`: **true** }.
4. Let *key* be ? `ToPropertyKey(P)`.
5. Perform ? `DefinePropertyOrThrow(O, key, desc)`.
6. Return **undefined**.

B.2.2.3 Object.prototype.__defineSetter__ (*P*, *setter*)

When the `__defineSetter__` method is called with arguments *P* and *setter*, the following steps are taken:

1. Let *O* be ? `ToObject(this value)`.
2. If `IsCallable(setter)` is **false**, throw a **TypeError** exception.
3. Let *desc* be PropertyDescriptor { `[[Set]]`: *setter*, `[[Enumerable]]`: **true**, `[[Configurable]]`: **true** }.
4. Let *key* be ? `ToPropertyKey(P)`.
5. Perform ? `DefinePropertyOrThrow(O, key, desc)`.
6. Return **undefined**.

B.2.2.4 Object.prototype.__lookupGetter__ (*P*)

When the `__lookupGetter__` method is called with argument *P*, the following steps are taken:

1. Let *O* be ? `ToObject(this value)`.
2. Let *key* be ? `ToPropertyKey(P)`.
3. Repeat,
 - a. Let *desc* be ? *O*.`[[GetOwnProperty]](key)`.
 - b. If *desc* is not **undefined**, then
 - i. If `IsAccessorDescriptor(desc)` is **true**, return *desc*.`[[Get]]`.
 - ii. Return **undefined**.
 - c. Set *O* to ? *O*.`[[GetPrototypeOf]]()`.
 - d. If *O* is **null**, return **undefined**.

B.2.2.5 Object.prototype.__lookupSetter__ (*P*)

When the `__lookupSetter__` method is called with argument *P*, the following steps are taken:

1. Let *O* be ? `ToObject(this value)`.
2. Let *key* be ? `ToPropertyKey(P)`.
3. Repeat,
 - a. Let *desc* be ? *O*.`[[GetOwnProperty]](key)`.
 - b. If *desc* is not **undefined**, then
 - i. If `IsAccessorDescriptor(desc)` is **true**, return *desc*.`[[Set]]`.
 - ii. Return **undefined**.
 - c. Set *O* to ? *O*.`[[GetPrototypeOf]]()`.
 - d. If *O* is **null**, return **undefined**.

B.2.3 Additional Properties of the String.prototype Object

B.2.3.1 String.prototype.substr (*start*, *length*)

The **substr** method takes two arguments, *start* and *length*, and returns a substring of the result of converting the **this** object to a String, starting from index *start* and running for *length* code units (or through the end of the String if *length* is **undefined**). If *start* is negative, it is treated as *sourceLength* + *start* where *sourceLength* is the length of the String. The result is a String value, not a String object. The following steps are taken:

1. Let *O* be ? **RequireObjectCoercible**(**this** value).
2. Let *S* be ? **ToString**(*O*).
3. Let *intStart* be ? **ToInteger**(*start*).
4. If *length* is **undefined**, let *end* be $+\infty$; otherwise let *end* be ? **ToInteger**(*length*).
5. Let *size* be the number of code units in *S*.
6. If *intStart* < 0, set *intStart* to $\max(\text{size} + \text{intStart}, 0)$.
7. Let *resultLength* be $\min(\max(\text{end}, 0), \text{size} - \text{intStart})$.
8. If *resultLength* ≤ 0, return the empty String "".
9. Return the String value containing *resultLength* consecutive code units from *S* beginning with the code unit at index *intStart*.

NOTE

The **substr** function is intentionally generic; it does not require that its **this** value be a String object. Therefore it can be transferred to other kinds of objects for use as a method.

B.2.3.2 String.prototype.anchor (*name*)

When the **anchor** method is called with argument *name*, the following steps are taken:

1. Let *S* be the **this** value.
2. Return ? **CreateHTML**(*S*, "a", "name", *name*).

B.2.3.2.1 Runtime Semantics: CreateHTML (*string*, *tag*, *attribute*, *value*)

The abstract operation CreateHTML is called with arguments *string*, *tag*, *attribute*, and *value*. The arguments *tag* and *attribute* must be String values. The following steps are taken:

1. Let *str* be ? **RequireObjectCoercible**(*string*).
2. Let *S* be ? **ToString**(*str*).
3. Let *p1* be the **string-concatenation** of "<" and *tag*.
4. If *attribute* is not the empty String, then
 - a. Let *V* be ? **ToString**(*value*).
 - b. Let *escapedV* be the String value that is the same as *V* except that each occurrence of the code unit 0x0022 (QUOTATION MARK) in *V* has been replaced with the six code unit sequence """.
 - c. Set *p1* to the **string-concatenation** of:
 - p1*
 - the code unit 0x0020 (SPACE)
 - attribute*
 - the code unit 0x003D (EQUALS SIGN)
 - the code unit 0x0022 (QUOTATION MARK)

escapedV

the code unit 0x0022 (QUOTATION MARK)

5. Let *p2* be the [string-concatenation](#) of *p1* and ">".
6. Let *p3* be the [string-concatenation](#) of *p2* and *S*.
7. Let *p4* be the [string-concatenation](#) of *p3*, "</", *tag*, and ">".
8. Return *p4*.

B.2.3.3 String.prototype.big ()

When the **big** method is called with no arguments, the following steps are taken:

1. Let *S* be the **this** value.
2. Return ? [CreateHTML](#)(*S*, "**big**", "", "").

B.2.3.4 String.prototype.blink ()

When the **blink** method is called with no arguments, the following steps are taken:

1. Let *S* be the **this** value.
2. Return ? [CreateHTML](#)(*S*, "**blink**", "", "").

B.2.3.5 String.prototype.bold ()

When the **bold** method is called with no arguments, the following steps are taken:

1. Let *S* be the **this** value.
2. Return ? [CreateHTML](#)(*S*, "**b**", "", "").

B.2.3.6 String.prototype.fixed ()

When the **fixed** method is called with no arguments, the following steps are taken:

1. Let *S* be the **this** value.
2. Return ? [CreateHTML](#)(*S*, "**tt**", "", "").

B.2.3.7 String.prototype.fontcolor (*color*)

When the **fontcolor** method is called with argument *color*, the following steps are taken:

1. Let *S* be the **this** value.
2. Return ? [CreateHTML](#)(*S*, "**font**", "**color**", *color*).

B.2.3.8 String.prototype.fontsize (*size*)

When the **fontsize** method is called with argument *size*, the following steps are taken:

1. Let *S* be the **this** value.
2. Return ? [CreateHTML](#)(*S*, "**font**", "**size**", *size*).

B.2.3.9 String.prototype.italics ()

When the **italics** method is called with no arguments, the following steps are taken:

1. Let **S** be the **this** value.
2. Return ? **CreateHTML**(**S**, "i", "", "").

B.2.3.10 **String.prototype.link (url)**

When the **link** method is called with argument *url*, the following steps are taken:

1. Let **S** be the **this** value.
2. Return ? **CreateHTML**(**S**, "a", "href", *url*).

B.2.3.11 **String.prototype.small ()**

When the **small** method is called with no arguments, the following steps are taken:

1. Let **S** be the **this** value.
2. Return ? **CreateHTML**(**S**, "small", "", "").

B.2.3.12 **String.prototype.strike ()**

When the **strike** method is called with no arguments, the following steps are taken:

1. Let **S** be the **this** value.
2. Return ? **CreateHTML**(**S**, "strike", "", "").

B.2.3.13 **String.prototype.sub ()**

When the **sub** method is called with no arguments, the following steps are taken:

1. Let **S** be the **this** value.
2. Return ? **CreateHTML**(**S**, "sub", "", "").

B.2.3.14 **String.prototype.sup ()**

When the **sup** method is called with no arguments, the following steps are taken:

1. Let **S** be the **this** value.
2. Return ? **CreateHTML**(**S**, "sup", "", "").

B.2.3.15 **String.prototype.trimLeft ()**

NOTE

The property **trimStart** is preferred. The **trimLeft** property is provided principally for compatibility with old code. It is recommended that the **trimStart** property be used in new ECMAScript code.

The initial value of the **trimLeft** property is the same **function object** as the initial value of the **String.prototype.trimStart** property.

B.2.3.16 **String.prototype.trimRight ()**

NOTE

The property **trimEnd** is preferred. The **trimRight** property is provided principally for compatibility with old code. It is recommended that the **trimEnd** property be used in new ECMAScript code.

The initial value of the **trimRight** property is the same [function object](#) as the initial value of the **String.prototype.trimEnd** property.

B.2.4 Additional Properties of the Date.prototype Object

B.2.4.1 Date.prototype.getYear ()

NOTE

The **getFullYear** method is preferred for nearly all purposes, because it avoids the “year 2000 problem.”

When the **getYear** method is called with no arguments, the following steps are taken:

1. Let *t* be ? [thisTimeValue](#)(**this** value).
2. If *t* is **NaN**, return **NaN**.
3. Return [YearFromTime](#)([LocalTime](#)(*t*)) - 1900.

B.2.4.2 Date.prototype.setYear (*year*)

NOTE

The **setFullYear** method is preferred for nearly all purposes, because it avoids the “year 2000 problem.”

When the **setYear** method is called with one argument *year*, the following steps are taken:

1. Let *t* be ? [thisTimeValue](#)(**this** value).
2. If *t* is **NaN**, set *t* to +0; otherwise, set *t* to [LocalTime](#)(*t*).
3. Let *y* be ? [ToNumber](#)(*year*).
4. If *y* is **NaN**, then
 - a. Set the [[DateValue]] internal slot of [this Date object](#) to **NaN**.
 - b. Return **NaN**.
5. Let *yi* be ! [ToInteger](#)(*y*).
6. If $0 \leq yi \leq 99$, let *yyyy* be *yi* + 1900.
7. Else, let *yyyy* be *y*.
8. Let *d* be [MakeDay](#)(*yyyy*, [MonthFromTime](#)(*t*), [DateFromTime](#)(*t*)).
9. Let *date* be [UTC](#)([MakeDate](#)(*d*, [TimeWithinDay](#)(*t*))).
10. Set the [[DateValue]] internal slot of [this Date object](#) to [TimeClip](#)(*date*).
11. Return the value of the [[DateValue]] internal slot of [this Date object](#).

B.2.4.3 Date.prototype.toGMTString ()

NOTE

The property **toUTCString** is preferred. The **toGMTString** property is provided principally for compatibility with old code. It is recommended that the **toUTCString** property be used in new ECMAScript code.

The [function object](#) that is the initial value of **Date.prototype.toGMTString** is the same [function object](#) that is

the initial value of `Date.prototype.toUTCString`.

B.2.5 Additional Properties of the RegExp.prototype Object

B.2.5.1 RegExp.prototype.compile (*pattern*, *flags*)

When the **compile** method is called with arguments *pattern* and *flags*, the following steps are taken:

1. Let *O* be the **this** value.
2. If `Type(O)` is not Object or `Type(O)` is Object and *O* does not have a `[[RegExpMatcher]]` internal slot, then
 - a. Throw a **TypeError** exception.
3. If `Type(pattern)` is Object and *pattern* has a `[[RegExpMatcher]]` internal slot, then
 - a. If *flags* is not **undefined**, throw a **TypeError** exception.
 - b. Let *P* be *pattern*.`[[OriginalSource]]`.
 - c. Let *F* be *pattern*.`[[OriginalFlags]]`.
4. Else,
 - a. Let *P* be *pattern*.
 - b. Let *F* be *flags*.
5. Return ? `RegExpInitialize(O, P, F)`.

NOTE

The **compile** method completely reinitializes the **this** object `RegExp` with a new pattern and flags. An implementation may interpret use of this method as an assertion that the resulting `RegExp` object will be used multiple times and hence is a candidate for extra optimization.

B.3 Other Additional Features

B.3.1 `__proto__` Property Names in Object Initializers

The following Early Error rule is added to those in 12.2.6.1. When *ObjectLiteral* appears in a context where *ObjectAssignmentPattern* is required the Early Error rule is **not** applied. In addition, it is not applied when initially parsing a *CoverParenthesizedExpressionAndArrowParameterList* or a *CoverCallExpressionAndAsyncArrowHead*.

ObjectLiteral : { *PropertyDefinitionList* }

ObjectLiteral : { *PropertyDefinitionList* , }

It is a Syntax Error if *PropertyNameList* of *PropertyDefinitionList* contains any duplicate entries for `"__proto__"` and at least two of those entries were obtained from productions of the form *PropertyDefinition* : *PropertyName* : *AssignmentExpression* .

NOTE

The *List* returned by *PropertyNameList* does not include string literal property names defined as using a *ComputedPropertyName*.

In 12.2.6.8 the *PropertyDefinitionEvaluation* algorithm for the production *PropertyDefinition* : *PropertyName* : *AssignmentExpression*

is replaced with the following definition:

PropertyDefinition : *PropertyName* : *AssignmentExpression*

1. Let *propKey* be the result of evaluating *PropertyName*.
2. *ReturnIfAbrupt(propKey)*.
3. If *propKey* is the String value "**__proto__**" and if *IsComputedPropertyKey(PropertyName)* is **false**, then
 - a. Let *isProtoSetter* be **true**.
4. Else,
 - a. Let *isProtoSetter* be **false**.
5. If *IsAnonymousFunctionDefinition(AssignmentExpression)* is **true** and *isProtoSetter* is **false**, then
 - a. Let *propValue* be the result of performing *NamedEvaluation* for *AssignmentExpression* with argument *propKey*.
6. Else,
 - a. Let *exprValueRef* be the result of evaluating *AssignmentExpression*.
 - b. Let *propValue* be ? *GetValue(exprValueRef)*.
7. If *isProtoSetter* is **true**, then
 - a. If *Type(propValue)* is either *Object* or *Null*, then
 - i. Return *object*.[[*SetPrototypeOf*]](*propValue*).
 - b. Return *NormalCompletion(empty)*.
8. *Assert: enumerable* is **true**.
9. *Assert: object* is an ordinary, extensible object with no non-configurable properties.
10. Return ! *CreateDataPropertyOrThrow(object, propKey, propValue)*.

B.3.2 Labelled Function Declarations

Prior to ECMAScript 2015, the specification of *LabelledStatement* did not allow for the association of a statement label with a *FunctionDeclaration*. However, a labelled *FunctionDeclaration* was an allowable extension for **non-strict code** and most browser-hosted ECMAScript implementations supported that extension. In ECMAScript 2015, the grammar productions for *LabelledStatement* permits use of *FunctionDeclaration* as a *LabelledItem* but 13.13.1 includes an Early Error rule that produces a Syntax Error if that occurs. For web browser compatibility, that rule is modified with the addition of the highlighted text:

LabelledItem : *FunctionDeclaration*

It is a Syntax Error if any strict mode source code matches this rule.

NOTE

The **early error** rules for *WithStatement*, *IfStatement*, and *IterationStatement* prevent these statements from containing a labelled *FunctionDeclaration* in **non-strict code**.

B.3.3 Block-Level Function Declarations Web Legacy Compatibility Semantics

Prior to ECMAScript 2015, the ECMAScript specification did not define the occurrence of a *FunctionDeclaration* as an element of a *Block* statement's *StatementList*. However, support for that form of *FunctionDeclaration* was an allowable extension and most browser-hosted ECMAScript implementations permitted them. Unfortunately, the semantics of such declarations differ among those implementations. Because of these semantic differences, existing web ECMAScript code that uses *Block* level function declarations is only portable among browser implementation if the usage only depends upon the semantic intersection of all of the browser implementations for such declarations. The following are the use

cases that fall within that intersection semantics:

1. A function is declared and only referenced within a single block

One or more *FunctionDeclarations* whose *BindingIdentifier* is the name *f* occur within the function code of an enclosing function *g* and that declaration is nested within a *Block*.

No other declaration of *f* that is not a **var** declaration occurs within the function code of *g*

All occurrences of *f* as an *IdentifierReference* are within the *StatementList* of the *Block* containing the declaration of *f*.

2. A function is declared and possibly used within a single *Block* but also referenced by an inner function definition that is not contained within that same *Block*.

One or more *FunctionDeclarations* whose *BindingIdentifier* is the name *f* occur within the function code of an enclosing function *g* and that declaration is nested within a *Block*.

No other declaration of *f* that is not a **var** declaration occurs within the function code of *g*

There may be occurrences of *f* as an *IdentifierReference* within the *StatementList* of the *Block* containing the declaration of *f*.

There is at least one occurrence of *f* as an *IdentifierReference* within another function *h* that is nested within *g* and no other declaration of *f* shadows the references to *f* from within *h*.

All invocations of *h* occur after the declaration of *f* has been evaluated.

3. A function is declared and possibly used within a single block but also referenced within subsequent blocks.

One or more *FunctionDeclaration* whose *BindingIdentifier* is the name *f* occur within the function code of an enclosing function *g* and that declaration is nested within a *Block*.

No other declaration of *f* that is not a **var** declaration occurs within the function code of *g*

There may be occurrences of *f* as an *IdentifierReference* within the *StatementList* of the *Block* containing the declaration of *f*.

There is at least one occurrence of *f* as an *IdentifierReference* within the function code of *g* that lexically follows the *Block* containing the declaration of *f*.

The first use case is interoperable with the semantics of *Block* level function declarations provided by ECMAScript 2015. Any pre-existing ECMAScript code that employs that use case will operate using the *Block* level function declarations semantics defined by clauses 9, 13, and 14 of this specification.

ECMAScript 2015 interoperability for the second and third use cases requires the following extensions to the clause 9, clause 14, clause 18.2.1 and clause 15.1.11 semantics.

If an ECMAScript implementation has a mechanism for reporting diagnostic warning messages, a warning should be produced when code contains a *FunctionDeclaration* for which these compatibility semantics are applied and introduce observable differences from non-compatibility semantics. For example, if a **var** binding is not introduced because its introduction would create an [early error](#), a warning message should not be produced.

B.3.3.1 Changes to FunctionDeclarationInstantiation

During [FunctionDeclarationInstantiation](#) the following steps are performed in place of step 28:

1. If *strict* is **false**, then
 - a. For each *FunctionDeclaration* *f* that is directly contained in the *StatementList* of a *Block*, *CaseClause*, or *DefaultClause*, do

- i. Let *F* be StringValue of the *BindingIdentifier* of *FunctionDeclaration* *f*.
- ii. If replacing the *FunctionDeclaration* *f* with a *VariableStatement* that has *F* as a *BindingIdentifier* would not produce any Early Errors for *func* and *F* is not an element of *parameterNames*, then
 1. NOTE: A var binding for *F* is only instantiated here if it is neither a *VarDeclaredName*, the name of a formal parameter, or another *FunctionDeclaration*.
 2. If *initializedBindings* does not contain *F* and *F* is not "**arguments**", then
 - a. Perform ! *varEnvRec*.CreateMutableBinding(*F*, **false**).
 - b. Perform *varEnvRec*.InitializeBinding(*F*, **undefined**).
 - c. Append *F* to *instantiatedVarNames*.
 3. When the *FunctionDeclaration* *f* is evaluated, perform the following steps in place of the *FunctionDeclaration* Evaluation algorithm provided in 14.1.22:
 - a. Let *fenv* be the *running execution context*'s *VariableEnvironment*.
 - b. Let *fenvRec* be *fenv*'s *EnvironmentRecord*.
 - c. Let *benv* be the *running execution context*'s *LexicalEnvironment*.
 - d. Let *benvRec* be *benv*'s *EnvironmentRecord*.
 - e. Let *fobj* be ! *benvRec*.GetBindingValue(*F*, **false**).
 - f. Perform ! *fenvRec*.SetMutableBinding(*F*, *fobj*, **false**).
 - g. Return *NormalCompletion*(empty).

B.3.3.2 Changes to GlobalDeclarationInstantiation

During *GlobalDeclarationInstantiation* the following steps are performed in place of step 14:

1. Let *strict* be IsStrict of *script*.
2. If *strict* is **false**, then
 - a. Let *declaredFunctionOrVarNames* be a new empty *List*.
 - b. Append to *declaredFunctionOrVarNames* the elements of *declaredFunctionNames*.
 - c. Append to *declaredFunctionOrVarNames* the elements of *declaredVarNames*.
 - d. For each *FunctionDeclaration* *f* that is directly contained in the *StatementList* of a *Block*, *CaseClause*, or *DefaultClause* Contained within *script*, do
 - i. Let *F* be StringValue of the *BindingIdentifier* of *FunctionDeclaration* *f*.
 - ii. If replacing the *FunctionDeclaration* *f* with a *VariableStatement* that has *F* as a *BindingIdentifier* would not produce any Early Errors for *script*, then
 1. If *envRec*.HasLexicalDeclaration(*F*) is **false**, then
 - a. Let *fnDefinable* be ? *envRec*.CanDeclareGlobalVar(*F*).
 - b. If *fnDefinable* is **true**, then
 - i. NOTE: A var binding for *F* is only instantiated here if it is neither a *VarDeclaredName* nor the name of another *FunctionDeclaration*.
 - ii. If *declaredFunctionOrVarNames* does not contain *F*, then
 - i. Perform ? *envRec*.CreateGlobalVarBinding(*F*, **false**).
 - ii. Append *F* to *declaredFunctionOrVarNames*.
 - iii. When the *FunctionDeclaration* *f* is evaluated, perform the following steps in place of the *FunctionDeclaration* Evaluation algorithm provided in 14.1.22:
 - i. Let *genv* be the *running execution context*'s *VariableEnvironment*.
 - ii. Let *genvRec* be *genv*'s *EnvironmentRecord*.
 - iii. Let *benv* be the *running execution context*'s *LexicalEnvironment*.
 - iv. Let *benvRec* be *benv*'s *EnvironmentRecord*.
 - v. Let *fobj* be ! *benvRec*.GetBindingValue(*F*, **false**).
 - vi. Perform ? *genvRec*.SetMutableBinding(*F*, *fobj*, **false**).

vii. Return `NormalCompletion(empty)`.

B.3.3.3 Changes to EvalDeclarationInstantiation

During `EvalDeclarationInstantiation` the following steps are performed in place of step 9:

1. If `strict` is **false**, then
 - a. Let `declaredFunctionOrVarNames` be a new empty `List`.
 - b. Append to `declaredFunctionOrVarNames` the elements of `declaredFunctionNames`.
 - c. Append to `declaredFunctionOrVarNames` the elements of `declaredVarNames`.
 - d. For each `FunctionDeclaration` `f` that is directly contained in the `StatementList` of a `Block`, `CaseClause`, or `DefaultClause` Contained within `body`, do
 - i. Let `F` be `StringValue` of the `BindingIdentifier` of `FunctionDeclaration` `f`.
 - ii. If replacing the `FunctionDeclaration` `f` with a `VariableStatement` that has `F` as a `BindingIdentifier` would not produce any Early Errors for `body`, then
 1. Let `bindingExists` be **false**.
 2. Let `thisLex` be `lexEnv`.
 3. **Assert**: The following loop will terminate.
 4. Repeat, while `thisLex` is not the same as `varEnv`,
 - a. Let `thisEnvRec` be `thisLex`'s `EnvironmentRecord`.
 - b. If `thisEnvRec` is not an object `Environment Record`, then
 - i. If `thisEnvRec.HasBinding(F)` is **true**, then
 - i. Let `bindingExists` be **true**.
 - c. Set `thisLex` to `thisLex`'s outer environment reference.
 5. If `bindingExists` is **false** and `varEnvRec` is a global `Environment Record`, then
 - a. If `varEnvRec.HasLexicalDeclaration(F)` is **false**, then
 - i. Let `fnDefinable` be ? `varEnvRec.CanDeclareGlobalVar(F)`.
 - b. Else,
 - i. Let `fnDefinable` be **false**.
 6. Else,
 - a. Let `fnDefinable` be **true**.
 7. If `bindingExists` is **false** and `fnDefinable` is **true**, then
 - a. If `declaredFunctionOrVarNames` does not contain `F`, then
 - i. If `varEnvRec` is a global `Environment Record`, then
 - i. Perform ? `varEnvRec.CreateGlobalVarBinding(F, true)`.
 - ii. Else,
 - i. Let `bindingExists` be `varEnvRec.HasBinding(F)`.
 - ii. If `bindingExists` is **false**, then
 - i. Perform ! `varEnvRec.CreateMutableBinding(F, true)`.
 - ii. Perform ! `varEnvRec.InitializeBinding(F, undefined)`.
 - iii. Append `F` to `declaredFunctionOrVarNames`.
 - b. When the `FunctionDeclaration` `f` is evaluated, perform the following steps in place of the `FunctionDeclaration` Evaluation algorithm provided in 14.1.22:
 - i. Let `genv` be the `running execution context`'s `VariableEnvironment`.
 - ii. Let `genvRec` be `genv`'s `EnvironmentRecord`.
 - iii. Let `benv` be the `running execution context`'s `LexicalEnvironment`.
 - iv. Let `benvRec` be `benv`'s `EnvironmentRecord`.
 - v. Let `fobj` be ! `benvRec.GetBindingValue(F, false)`.
 - vi. Perform ? `genvRec.SetMutableBinding(F, fobj, false)`.

vii. Return `NormalCompletion(empty)`.

B.3.3.4 Changes to Block Static Semantics: Early Errors

For web browser compatibility, that rule is modified with the addition of the highlighted text:

Block : { *StatementList* }

It is a Syntax Error if the `LexicallyDeclaredNames` of *StatementList* contains any duplicate entries, unless the source code matching this production is not `strict mode code` and the duplicate entries are only bound by `FunctionDeclarations`.

B.3.3.5 Changes to `switch` Statement Static Semantics: Early Errors

For web browser compatibility, that rule is modified with the addition of the highlighted text:

SwitchStatement : **switch** (*Expression*) *CaseBlock*

It is a Syntax Error if the `LexicallyDeclaredNames` of *CaseBlock* contains any duplicate entries, unless the source code matching this production is not `strict mode code` and the duplicate entries are only bound by `FunctionDeclarations`.

B.3.3.6 Changes to `BlockDeclarationInstantiation`

During `BlockDeclarationInstantiation` the following steps are performed in place of step 4.a.ii.1:

1. If `envRec.HasBinding(dn)` is **false**, then
 - a. Perform ! `envRec.CreateMutableBinding(dn, false)`.

During `BlockDeclarationInstantiation` the following steps are performed in place of step 4.b.iii:

1. If `envRec.HasBinding(fn)` is **false**, then
 - a. Perform `envRec.InitializeBinding(fn, fo)`.
2. Else,
 - a. **Assert**: *d* is a `FunctionDeclaration`.
 - b. Perform `envRec.SetMutableBinding(fn, fo, false)`.

B.3.4 `FunctionDeclarations` in `IfStatement` Statement Clauses

The following augments the *IfStatement* production in 13.6:

```
IfStatement[Yield, Await, Return] :  
  if ( Expression[+In, ?Yield, ?Await] ) FunctionDeclaration[?Yield, ?Await, ~Default]  
    else Statement[?Yield, ?Await, ?Return]  
  if ( Expression[+In, ?Yield, ?Await] ) Statement[?Yield, ?Await, ?Return] else  
    FunctionDeclaration[?Yield, ?Await, ~Default]  
  if ( Expression[+In, ?Yield, ?Await] ) FunctionDeclaration[?Yield, ?Await, ~Default]  
    else FunctionDeclaration[?Yield, ?Await, ~Default]  
  if ( Expression[+In, ?Yield, ?Await] ) FunctionDeclaration[?Yield, ?Await, ~Default]
```

This production only applies when parsing **non-strict code**. Code matching this production is processed as if each matching occurrence of *FunctionDeclaration*_[?Yield, ?Await, ~Default] was the sole *StatementListItem* of a *BlockStatement* occupying that position in the source code. The semantics of such a synthetic *BlockStatement* includes the web legacy compatibility semantics specified in B.3.3.

B.3.5 VariableStatements in Catch Blocks

The content of subclause 13.15.1 is replaced with the following:

Catch : **catch** (*CatchParameter*) *Block*

It is a Syntax Error if BoundNames of *CatchParameter* contains any duplicate elements.

It is a Syntax Error if any element of the BoundNames of *CatchParameter* also occurs in the LexicallyDeclaredNames of *Block*.

It is a Syntax Error if any element of the BoundNames of *CatchParameter* also occurs in the VarDeclaredNames of *Block* unless *CatchParameter* is *CatchParameter* : *BindingIdentifier* .

NOTE

The *Block* of a *Catch* clause may contain **var** declarations that bind a name that is also bound by the *CatchParameter*. At runtime, such bindings are instantiated in the VariableDeclarationEnvironment. They do not shadow the same-named bindings introduced by the *CatchParameter* and hence the *Initializer* for such **var** declarations will assign to the corresponding catch parameter rather than the **var** binding.

This modified behaviour also applies to **var** and **function** declarations introduced by **direct eval** calls contained within the *Block* of a *Catch* clause. This change is accomplished by modifying the algorithm of 18.2.1.3 as follows:

Step 5.d.ii.2.a.i is replaced by:

1. If *thisEnvRec* is not the **Environment Record** for a *Catch* clause, throw a **SyntaxError** exception.

Step 9.d.ii.4.b.i.i is replaced by:

1. If *thisEnvRec* is not the **Environment Record** for a *Catch* clause, let *bindingExists* be **true**.

B.3.6 Initializers in ForIn Statement Heads

The following augments the *IterationStatement* production in 13.7:

*IterationStatement*_[Yield, Await, Return] :

for (**var** *BindingIdentifier*_[?Yield, ?Await] *Initializer*_[~In, ?Yield, ?Await] **in** *Expression*_[+In, ?Yield, ?Await]) *Statement*_[?Yield, ?Await, ?Return]

This production only applies when parsing **non-strict code**.

The **static semantics** of ContainsDuplicateLabels in 13.7.5.3 are augmented with the following:

IterationStatement : **for** (**var** *BindingIdentifier* *Initializer* **in** *Expression*) *Statement*

1. Return ContainsDuplicateLabels of *Statement* with argument *labelSet*.

The **static semantics** of ContainsUndefinedBreakTarget in 13.7.5.4 are augmented with the following:

IterationStatement : **for** (**var** *BindingIdentifier* *Initializer* **in** *Expression*) *Statement*

1. Return *ContainsUndefinedBreakTarget* of *Statement* with argument *labelSet*.

The *static semantics* of *ContainsUndefinedContinueTarget* in 13.7.5.5 are augmented with the following:

IterationStatement : **for** (**var** *BindingIdentifier* *Initializer* **in** *Expression*) *Statement*

1. Return *ContainsUndefinedContinueTarget* of *Statement* with arguments *iterationSet* and « ».

The *static semantics* of *IsDestructuring* in 13.7.5.6 are augmented with the following:

BindingIdentifier :
Identifier
yield
await

1. Return **false**.

The *static semantics* of *VarDeclaredNames* in 13.7.5.7 are augmented with the following:

IterationStatement : **for** (**var** *BindingIdentifier* *Initializer* **in** *Expression*) *Statement*

1. Let *names* be the *BoundNames* of *BindingIdentifier*.
2. Append to *names* the elements of the *VarDeclaredNames* of *Statement*.
3. Return *names*.

The *static semantics* of *VarScopedDeclarations* in 13.7.5.8 are augmented with the following:

IterationStatement : **for** (**var** *BindingIdentifier* *Initializer* **in** *Expression*) *Statement*

1. Let *declarations* be a *List* containing *BindingIdentifier*.
2. Append to *declarations* the elements of the *VarScopedDeclarations* of *Statement*.
3. Return *declarations*.

The *runtime semantics* of *LabelledEvaluation* in 13.7.5.11 are augmented with the following:

IterationStatement : **for** (**var** *BindingIdentifier* *Initializer* **in** *Expression*) *Statement*

1. Let *bindingId* be *StringValue* of *BindingIdentifier*.
2. Let *lhs* be ? *ResolveBinding*(*bindingId*).
3. If *IsAnonymousFunctionDefinition*(*Initializer*) is **true**, then
 - a. Let *value* be the result of performing *NamedEvaluation* for *Initializer* with argument *bindingId*.
4. Else,
 - a. Let *rhs* be the result of evaluating *Initializer*.
 - b. Let *value* be ? *GetValue*(*rhs*).
5. Perform ? *PutValue*(*lhs*, *value*).
6. Let *keyResult* be ? *ForIn/OfHeadEvaluation*(« », *Expression*, **enumerate**).
7. Return ? *ForIn/OfBodyEvaluation*(*BindingIdentifier*, *Statement*, *keyResult*, **enumerate**, *varBinding*, *labelSet*).

B.3.7 The *[[IsHTMLDDA]]* Internal Slot

An *[[IsHTMLDDA]]* internal slot may exist on implementation-defined objects. Objects with an *[[IsHTMLDDA]]*

internal slot behave like **undefined** in the [ToBoolean](#) and [Abstract Equality Comparison](#) abstract operations and when used as an operand for the [typeof](#) operator.

NOTE

Objects with an `[[IsHTMLDDA]]` internal slot are never created by this specification. However, the `document.all` object in web browsers is a host-created [exotic object](#) with this slot that exists for web compatibility purposes. There are no other known examples of this type of object and implementations should not create any with the exception of `document.all`.

B.3.7.1 Changes to ToBoolean

The result column in [Table 9](#) for an argument type of Object is replaced with the following algorithm:

- 1. If *argument* has an `[[IsHTMLDDA]]` internal slot, return **false**.
- 2. Return **true**.

B.3.7.2 Changes to Abstract Equality Comparison

The following steps are inserted after step 3 of the [Abstract Equality Comparison](#) algorithm:

- 1. If `Type(x)` is Object and *x* has an `[[IsHTMLDDA]]` internal slot and *y* is either **null** or **undefined**, return **true**.
- 2. If *x* is either **null** or **undefined** and `Type(y)` is Object and *y* has an `[[IsHTMLDDA]]` internal slot, return **true**.

B.3.7.3 Changes to the typeof Operator

The following table entry is inserted into [Table 35](#) immediately preceeding the entry for "Object (implements `[[Call]]`)":

Table 84: Additional <code>typeof</code> Operator Results	
Type of <i>val</i>	Result
Object (has an <code>[[IsHTMLDDA]]</code> internal slot)	"undefined"

C The Strict Mode of ECMAScript

The strict mode restriction and exceptions

implements, **interface**, **let**, **package**, **private**, **protected**, **public**, **static**, and **yield** are reserved words within [strict mode code](#). (11.6.2).

A conforming implementation, when processing [strict mode code](#), must not extend, as described in [B.1.1](#), the syntax of *NumericLiteral* to include *LegacyOctalIntegerLiteral*, nor extend the syntax of *DecimalIntegerLiteral* to include *NonOctalDecimalIntegerLiteral*.

A conforming implementation, when processing [strict mode code](#), may not extend the syntax of *EscapeSequence* to include *LegacyOctalEscapeSequence* as described in [B.1.2](#).

Assignment to an undeclared identifier or otherwise unresolvable reference does not create a property in the [global object](#). When a simple assignment occurs within [strict mode code](#), its *LeftHandSideExpression* must not evaluate to an unresolvable [Reference](#). If it does a **ReferenceError** exception is thrown (6.2.4.9). The *LeftHandSideExpression* also may not be a reference to a [data property](#) with the attribute value { `[[Writable]]`: **false** }, to an [accessor](#)

property with the attribute value { `[[Set]]`: **undefined** }, nor to a non-existent property of an object whose `[[Extensible]]` internal slot has the value **false**. In these cases a **TypeError** exception is thrown (12.15). An *IdentifierReference* with the StringValue **"eval"** or **"arguments"** may not appear as the *LeftHandSideExpression* of an Assignment operator (12.15) or of an *UpdateExpression* (12.4) or as the *UnaryExpression* operated upon by a Prefix Increment (12.4.6) or a Prefix Decrement (12.4.7) operator. Arguments objects for strict functions define a non-configurable accessor property **"callee"** which throws a **TypeError** exception on access (9.4.4.6).

Arguments objects for strict functions do not dynamically share their **array-indexed** property values with the corresponding formal parameter bindings of their functions. (9.4.4).

For strict functions, if an arguments object is created the binding of the local identifier **arguments** to the arguments object is immutable and hence may not be the target of an assignment expression. (9.2.15).

It is a **SyntaxError** if the StringValue of a *BindingIdentifier* is **"eval"** or **"arguments"** within **strict mode code** (12.1.1).

Strict mode eval code cannot instantiate variables or functions in the variable environment of the caller to eval. Instead, a new variable environment is created and that environment is used for declaration binding instantiation for the eval code (18.2.1).

If **this** is evaluated within **strict mode code**, then the **this** value is not coerced to an object. A **this** value of **undefined** or **null** is not converted to the **global object** and primitive values are not converted to wrapper objects. The **this** value passed via a function call (including calls made using **Function.prototype.apply** and **Function.prototype.call**) do not coerce the passed this value to an object (9.2.1.2, 19.2.3.1, 19.2.3.3). When a **delete** operator occurs within **strict mode code**, a **SyntaxError** is thrown if its *UnaryExpression* is a direct reference to a variable, function argument, or function name (12.5.3.1).

When a **delete** operator occurs within **strict mode code**, a **TypeError** is thrown if the property to be deleted has the attribute { `[[Configurable]]`: **false** } (12.5.3.2).

Strict mode code may not include a *WithStatement*. The occurrence of a *WithStatement* in such a context is a **SyntaxError** (13.11.1).

It is a **SyntaxError** if a *CatchParameter* occurs within **strict mode code** and BoundNames of *CatchParameter* contains either **eval** or **arguments** (13.15.1).

It is a **SyntaxError** if the same *BindingIdentifier* appears more than once in the *FormalParameters* of a **strict function**. An attempt to create such a function using a **Function**, **Generator**, or **AsyncFunction constructor** is a **SyntaxError** (14.1.2, 19.2.1.1.1).

An implementation may not extend, beyond that defined in this specification, the meanings within strict functions of properties named **caller** or **arguments** of function instances.

D Corrections and Clarifications in ECMAScript 2015 with Possible Compatibility Impact

8.1.1.4.15-8.1.1.4.18 Edition 5 and 5.1 used a property existence test to determine whether a **global object** property corresponding to a new global declaration already existed. ECMAScript 2015 uses an own property existence test. This corresponds to what has been most commonly implemented by web browsers.

9.4.2.1: The 5th Edition moved the capture of the current array length prior to the integer conversion of the **array index** or new length value. However, the captured length value could become invalid if the conversion process has the side-effect of changing the array length. ECMAScript 2015 specifies that the current array length must be captured after the possible occurrence of such side-effects.

[20.3.1.14](#): Previous editions permitted the [TimeClip](#) abstract operation to return either **+0** or **-0** as the representation of a **0 time value**. ECMAScript 2015 specifies that **+0** always returned. This means that for ECMAScript 2015 the **time value** of a Date object is never observably **-0** and methods that return time values never return **-0**.

[20.3.1.15](#): If a time zone offset is not present, the local time zone is used. Edition 5.1 incorrectly stated that a missing time zone should be interpreted as **"z"**.

[20.3.4.36](#): If the year cannot be represented using the Date Time String Format specified in [20.3.1.15](#) a RangeError exception is thrown. Previous editions did not specify the behaviour for that case.

[20.3.4.41](#): Previous editions did not specify the value returned by Date.prototype.toString when **this time value** is NaN. ECMAScript 2015 specifies the result to be the String value is **"Invalid Date"**.

[21.2.3.1](#), [21.2.3.2.4](#): Any LineTerminator code points in the value of the **source** property of a RegExp instance must be expressed using an escape sequence. Edition 5.1 only required the escaping of **"/"**.

[21.2.5.7](#), [21.2.5.9](#): In previous editions, the specifications for **String.prototype.match** and **String.prototype.replace** was incorrect for cases where the pattern argument was a RegExp value whose **global** is flag set. The previous specifications stated that for each attempt to match the pattern, if **lastIndex** did not change it should be incremented by 1. The correct behaviour is that **lastIndex** should be incremented by one only if the pattern matched the empty string.

[22.1.3.27](#), [22.1.3.27.1](#): Previous editions did not specify how a NaN value returned by a [comparefn](#) was interpreted by **Array.prototype.sort**. ECMAScript 2015 specifies that such as value is treated as if **+0** was returned from the [comparefn](#). ECMAScript 2015 also specifies that [ToNumber](#) is applied to the result returned by a [comparefn](#). In previous editions, the effect of a [comparefn](#) result that is not a Number value was implementation-dependent. In practice, implementations call [ToNumber](#).

E Additions and Changes That Introduce Incompatibilities with Prior Editions

[7.1.3.1](#): In ECMAScript 2015, [ToNumber](#) applied to a String value now recognizes and converts *BinaryIntegerLiteral* and *OctalIntegerLiteral* numeric strings. In previous editions such strings were converted to NaN.

[6.2.4](#): In ECMAScript 2015, Function calls are not allowed to return a [Reference](#) value.

[11.6](#): In ECMAScript 2015, the valid code points for an *IdentifierName* are specified in terms of the Unicode properties "ID_Start" and "ID_Continue". In previous editions, the valid *IdentifierName* or *Identifier* code points were specified by enumerating various Unicode code point categories.

[11.9.1](#): In ECMAScript 2015, Automatic Semicolon Insertion adds a semicolon at the end of a do-while statement if the semicolon is missing. This change aligns the specification with the actual behaviour of most existing implementations.

[12.2.6.1](#): In ECMAScript 2015, it is no longer an [early error](#) to have duplicate property names in Object Initializers.

[12.15.1](#): In ECMAScript 2015, [strict mode code](#) containing an assignment to an immutable binding such as the function name of a *FunctionExpression* does not produce an [early error](#). Instead it produces a runtime error.

[13.2](#): In ECMAScript 2015, a *StatementList* beginning with the token let followed by the input elements *LineTerminator* then *Identifier* is the start of a *LexicalDeclaration*. In previous editions, automatic semicolon insertion would always

insert a semicolon before the *Identifier* input element.

13.5: In ECMAScript 2015, a *StatementListItem* beginning with the token **let** followed by the token **[** is the start of a *LexicalDeclaration*. In previous editions such a sequence would be the start of an *ExpressionStatement*.

13.6.7: In ECMAScript 2015, the normal completion value of an *IfStatement* is never the value **empty**. If no *Statement* part is evaluated or if the evaluated *Statement* part produces a normal completion whose value is **empty**, the completion value of the *IfStatement* is **undefined**.

13.7: In ECMAScript 2015, if the **(** token of a for statement is immediately followed by the token sequence **let [** then the **let** is treated as the start of a *LexicalDeclaration*. In previous editions such a token sequence would be the start of an *Expression*.

13.7: In ECMAScript 2015, if the **(** token of a for-in statement is immediately followed by the token sequence **let [** then the **let** is treated as the start of a *ForDeclaration*. In previous editions such a token sequence would be the start of an *LeftHandSideExpression*.

13.7: Prior to ECMAScript 2015, an initialization expression could appear as part of the *VariableDeclaration* that precedes the **in** keyword. In ECMAScript 2015, the *ForBinding* in that same position does not allow the occurrence of such an initializer. In ECMAScript 2017, such an initializer is permitted only in **non-strict code**.

13.7: In ECMAScript 2015, the completion value of an *IterationStatement* is never the value **empty**. If the *Statement* part of an *IterationStatement* is not evaluated or if the final evaluation of the *Statement* part produces a completion whose value is **empty**, the completion value of the *IterationStatement* is **undefined**.

13.11.7: In ECMAScript 2015, the normal completion value of a *WithStatement* is never the value **empty**. If evaluation of the *Statement* part of a *WithStatement* produces a normal completion whose value is **empty**, the completion value of the *WithStatement* is **undefined**.

13.12.11: In ECMAScript 2015, the completion value of a *SwitchStatement* is never the value **empty**. If the *CaseBlock* part of a *SwitchStatement* produces a completion whose value is **empty**, the completion value of the *SwitchStatement* is **undefined**.

13.15: In ECMAScript 2015, it is an **early error** for a *Catch* clause to contain a **var** declaration for the same *Identifier* that appears as the *Catch* clause parameter. In previous editions, such a variable declaration would be instantiated in the enclosing variable environment but the declaration's *Initializer* value would be assigned to the *Catch* parameter.

13.15, 18.2.1.3: In ECMAScript 2015, a runtime **SyntaxError** is thrown if a *Catch* clause evaluates a non-strict direct **eval** whose eval code includes a **var** or **FunctionDeclaration** declaration that binds the same *Identifier* that appears as the *Catch* clause parameter.

13.15.8: In ECMAScript 2015, the completion value of a *TryStatement* is never the value **empty**. If the *Block* part of a *TryStatement* evaluates to a normal completion whose value is **empty**, the completion value of the *TryStatement* is **undefined**. If the *Block* part of a *TryStatement* evaluates to a throw completion and it has a *Catch* part that evaluates to a normal completion whose value is **empty**, the completion value of the *TryStatement* is **undefined** if there is no *Finally* clause or if its *Finally* clause evaluates to an **empty** normal completion.

14.3.8 In ECMAScript 2015, the function objects that are created as the values of the **[[Get]]** or **[[Set]]** attribute of accessor properties in an *ObjectLiteral* are not **constructor** functions and they do not have a **prototype** own property. In the previous edition, they were constructors and had a **prototype** property.

19.1.2.6: In ECMAScript 2015, if the argument to **Object.freeze** is not an object it is treated as if it was a non-

extendable ordinary object with no own properties. In the previous edition, a non-object argument always causes a **TypeError** to be thrown.

19.1.2.8: In ECMAScript 2015, if the argument to **Object.getPrototypeOf** is not an object an attempt is made to coerce the argument using **ToObject**. If the coercion is successful the result is used in place of the original argument value. In the previous edition, a non-object argument always causes a **TypeError** to be thrown.

19.1.2.10: In ECMAScript 2015, if the argument to **Object.getPrototypeOf** is not an object an attempt is made to coerce the argument using **ToObject**. If the coercion is successful the result is used in place of the original argument value. In the previous edition, a non-object argument always causes a **TypeError** to be thrown.

19.1.2.12: In ECMAScript 2015, if the argument to **Object.getPrototypeOf** is not an object an attempt is made to coerce the argument using **ToObject**. If the coercion is successful the result is used in place of the original argument value. In the previous edition, a non-object argument always causes a **TypeError** to be thrown.

19.1.2.14: In ECMAScript 2015, if the argument to **Object.isExtensible** is not an object it is treated as if it was a non-extensible ordinary object with no own properties. In the previous edition, a non-object argument always causes a **TypeError** to be thrown.

19.1.2.15: In ECMAScript 2015, if the argument to **Object.isFrozen** is not an object it is treated as if it was a non-extensible ordinary object with no own properties. In the previous edition, a non-object argument always causes a **TypeError** to be thrown.

19.1.2.16: In ECMAScript 2015, if the argument to **Object.isSealed** is not an object it is treated as if it was a non-extensible ordinary object with no own properties. In the previous edition, a non-object argument always causes a **TypeError** to be thrown.

19.1.2.17: In ECMAScript 2015, if the argument to **Object.keys** is not an object an attempt is made to coerce the argument using **ToObject**. If the coercion is successful the result is used in place of the original argument value. In the previous edition, a non-object argument always causes a **TypeError** to be thrown.

19.1.2.18: In ECMAScript 2015, if the argument to **Object.preventExtensions** is not an object it is treated as if it was a non-extensible ordinary object with no own properties. In the previous edition, a non-object argument always causes a **TypeError** to be thrown.

19.1.2.20: In ECMAScript 2015, if the argument to **Object.seal** is not an object it is treated as if it was a non-extensible ordinary object with no own properties. In the previous edition, a non-object argument always causes a **TypeError** to be thrown.

19.2.3.2: In ECMAScript 2015, the **[[Prototype]]** internal slot of a **bound function** is set to the **[[GetPrototypeOf]]** value of its target function. In the previous edition, **[[Prototype]]** was always set to **%FunctionPrototype%**.

19.2.4.1: In ECMAScript 2015, the **"length"** property of function instances is configurable. In previous editions it was non-configurable.

19.5.6.2: In ECMAScript 2015, the **[[Prototype]]** internal slot of a **NativeError constructor** is the **Error constructor**. In previous editions it was the **Function** prototype object.

20.3.4 In ECMAScript 2015, the **Date** prototype object is not a **Date** instance. In previous editions it was a **Date** instance whose **TimeValue** was **NaN**.

21.1.3.10 In ECMAScript 2015, the **String.prototype.localeCompare** function must treat **Strings** that are

canonically equivalent according to the Unicode standard as being identical. In previous editions implementations were permitted to ignore canonical equivalence and could instead use a bit-wise comparison.

[21.1.3.24](#) and [21.1.3.26](#) In ECMAScript 2015, lowercase/upper conversion processing operates on code points. In previous editions such the conversion processing was only applied to individual code units. The only affected code points are those in the Deseret block of Unicode.

[21.1.3.27](#) In ECMAScript 2015, the **String.prototype.trim** method is defined to recognize white space code points that may exists outside of the Unicode BMP. However, as of Unicode 7 no such code points are defined. In previous editions such code points would not have been recognized as white space.

[21.2.3.1](#) In ECMAScript 2015, If the *pattern* argument is a RegExp instance and the *flags* argument is not **undefined**, a new RegExp instance is created just like *pattern* except that *pattern*'s flags are replaced by the argument *flags*. In previous editions a **TypeError** exception was thrown when *pattern* was a RegExp instance and *flags* was not **undefined**.

[21.2.5](#) In ECMAScript 2015, the RegExp prototype object is not a RegExp instance. In previous editions it was a RegExp instance whose pattern is the empty string.

[21.2.5](#) In ECMAScript 2015, **source**, **global**, **ignoreCase**, and **multiline** are accessor properties defined on the RegExp prototype object. In previous editions they were data properties defined on RegExp instances.

F Colophon

This specification is authored on [GitHub](#) in a plaintext source format called [Ecm Markup](#). Ecm Markup is an HTML and Markdown dialect that provides a framework and toolset for authoring ECMAScript specifications in plaintext and processing the specification into a full-featured HTML rendering that follows the editorial conventions for this document. Ecm Markup builds on and integrates a number of other formats and technologies including [Grammarkdown](#) for defining syntax and [Ecm Markup](#) for authoring algorithm steps. PDF renderings of this specification are produced by printing the HTML rendering to a PDF.

Prior editions of this specification were authored using Word—the Ecm Markup source text that formed the basis of this edition was produced by converting the ECMAScript 2015 Word document to Ecm Markup using an automated conversion tool.

G Bibliography

1. IEEE Std 754-2008: *IEEE Standard for Floating-Point Arithmetic*. Institute of Electrical and Electronic Engineers, New York (2008)
2. *The Unicode Standard*, available at <https://unicode.org/versions/latest>
3. *Unicode Technical Note #5: Canonical Equivalence in Applications*, available at <https://unicode.org/notes/tn5/>
4. *Unicode Technical Standard #10: Unicode Collation Algorithm*, available at <https://unicode.org/reports/tr10/>
5. *Unicode Standard Annex #15, Unicode Normalization Forms*, available at <https://unicode.org/reports/tr15/>
6. *Unicode Standard Annex #18: Unicode Regular Expressions*, available at <https://unicode.org/reports/tr18/>
7. *Unicode Standard Annex #24: Unicode **Script** Property*, available at <https://unicode.org/reports/tr24/>
8. *Unicode Standard Annex #31, Unicode Identifiers and Pattern Syntax*, available at <https://unicode.org/reports/tr31/>
9. *Unicode Standard Annex #44: Unicode Character Database*, available at <https://unicode.org/reports/tr44/>

10. *Unicode Technical Standard #51: Unicode Emoji*, available at <<https://unicode.org/reports/tr51/>>
11. *IANA Time Zone Database*, available at <<https://www.iana.org/time-zones>>
12. *ISO 8601:2004(E) Data elements and interchange formats – Information interchange — Representation of dates and times*
13. *RFC 1738 “Uniform Resource Locators (URL)”*, available at <<https://tools.ietf.org/html/rfc1738>>
14. *RFC 2396 “Uniform Resource Identifiers (URI): Generic Syntax”*, available at <<https://tools.ietf.org/html/rfc2396>>
15. *RFC 3629 “UTF-8, a transformation format of ISO 10646”*, available at <<https://tools.ietf.org/html/rfc3629>>

H Copyright & Software License

Ecma International

Rue du Rhone 114

CH-1204 Geneva

Tel: +41 22 849 6000

Fax: +41 22 849 6001

Web: <https://ecma-international.org/>

Copyright Notice

© 2019 Ecma International

This document may be copied, published and distributed to others, and certain derivative works of it may be prepared, copied, published, and distributed, in whole or in part, provided that the above copyright notice and this Copyright License and Disclaimer are included on all such copies and derivative works. The only derivative works that are permissible under this Copyright License and Disclaimer are:

- (i) works which incorporate all or portion of this document for the purpose of providing commentary or explanation (such as an annotated version of the document),
- (ii) works which incorporate all or portion of this document for the purpose of incorporating features that provide accessibility,
- (iii) translations of this document into languages other than English and into different formats and
- (iv) works by making use of this specification in standard conformant products by implementing (e.g. by copy and paste wholly or partly) the functionality therein.

However, the content of this document itself may not be modified in any way, including by removing the copyright notice or references to Ecma International, except as required to translate it into languages other than English or into a different format.

The official version of an Ecma International document is the English language version on the Ecma International website. In the event of discrepancies between a translated version and the official version, the official version shall govern.

The limited permissions granted above are perpetual and will not be revoked by Ecma International or its successors or

assigns.

This document and the information contained herein is provided on an "AS IS" basis and ECMA INTERNATIONAL DISCLAIMS ALL WARRANTIES, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTY THAT THE USE OF THE INFORMATION HEREIN WILL NOT INFRINGE ANY OWNERSHIP RIGHTS OR ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE.

Software License

All Software contained in this document ("Software") is protected by copyright and is being made available under the "BSD License", included below. This Software may be subject to third party rights (rights from parties other than Ecma International), including patent rights, and no licenses under such third party rights are granted under this license even if the third party concerned is a member of Ecma International. SEE THE ECMA CODE OF CONDUCT IN PATENT MATTERS AVAILABLE AT <https://ecma-international.org/memento/codeofconduct.htm> FOR INFORMATION REGARDING THE LICENSING OF PATENT CLAIMS THAT ARE REQUIRED TO IMPLEMENT ECMA INTERNATIONAL STANDARDS.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
3. Neither the name of the authors nor Ecma International may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE ECMA INTERNATIONAL "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL ECMA INTERNATIONAL BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.